Reti di calcolatori

Quinta edizione

Andrew S. Tanenbaum
David J. Wetherall

MyLab Codice per accedere alla piattaforma

Pearson

© 2011 Pearson Italia, Milano-Torino

Authorized translation from the English language edition, entitled: Computer Networks, 5th edition, by Andrew Tanenbaum; David Wetherall published by Pearson Education, Inc, publishing as Prentice Hall, Copyright © 2011

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

Italian language edition published by Pearson Italia S.p.A., Copyright © 2011.

Le informazioni contenute in questo libro sono state verificate e documentate con la massima cura possibile. Nessuna responsabilità derivante dal loro utilizzo potrà venire imputata agli Autori, a Pearson Italia S.p.A. o a ogni persona e società coinvolta nella creazione, produzione e distribuzione di questo libro.

Per i passi antologici, per le citazioni, per le riproduzioni grafiche, cartografiche e fotografiche appartenenti alla proprietà di terzi, inseriti in quest'opera, l'editore è a disposizione degli aventi diritto non potuti reperire nonché per eventuali non volute omissioni e/o errori di attribuzione nei riferimenti.

Curatori per l'edizione italiana: Sabrina Gaito, Dario Maggiorini

Traduzione: Sabrina Gaito, Dario Maggiorini

Copy-editing: Donatella Pepe

Produzione digitale: Digitaltypes - Milano

Progetto grafico di copertina: Achilli Ghizzardi Associati - Milano

Tutti i marchi citati nel testo sono di proprietà dei loro detentori.

ISBN:

a Suzanne, Barbara, Daniel, Aron, Marvin, Matilde,

e alla memoria di Bram e Sweetie π *(AST)*

a Katrin, Lucy e Pepper (DJW)

Colophon
PREFAZIONE
Prefazione all'edizione italiana
Prefazione
Ringraziamenti
CAPITOLO 1 **Introduzione**
1.1 Applicazioni delle reti di calcolatori
1.2 Hardware di rete
1.3 Software di rete
1.4 Modelli di riferimento
1.5 Esempi di reti
1.6 Standardizzazione delle reti
1.7 Unità metriche
1.8 Organizzazione del resto del libro
1.9 Sommario
Problemi
CAPITOLO 2 **Il livello fisico**
2.1 Basi teoriche della comunicazione dati
2.2 Mezzi di trasmissione vincolati
2.3 Trasmissioni wireless
2.4 Comunicazioni satellitari
2.5 Modulazione digitale e multiplexing
2.6 La rete telefonica pubblica commutata
2.7 Il sistema telefonico mobile
2.8 Televisione via cavo
2.9 Sommario
Problemi
CAPITOLO 3 **Il livello data link**
3.1 Progettazione del livello data link
3.2 Rilevazione e correzione degli errori
3.3 Protocolli data link elementari
3.4 Protocolli a finestra scorrevole
3.5 Esempi di protocolli data link
3.6 Sommario
Problemi
CAPITOLO 4 **Il sottolivello MAC**
4.1 Problema dell'allocazione del canale
4.2 Protocolli ad accesso multiplo
4.3 Ethernet
4.4 Lan wireless
4.5 Wireless a banda larga
4.6 Bluetooth
4.7 RFID
4.8 Commutazione a livello data link

4.9 Sommario
Problemi
CAPITOLO 5 **Il livello di rete**
5.1 Problematiche nella progettazione del livello di rete
5.2 Algoritmi di routing
5.3 Algoritmi per il controllo della congestione
5.4 Qualità del servizio
5.5 Internetworking
5.6 Il livello di rete in Internet
5.7 Sommario
Problemi
CAPITOLO 6 **Il livello di trasporto**
6.1 Il servizio di trasporto
6.2 Elementi dei protocolli di trasporto
6.3 Controllo di congestione
6.4 Protocolli di trasporto di Internet: UDP
6.5 Protocolli di trasporto di Internet: TCP
6.6 Problemi di prestazioni
6.7 Reti con tolleranza al ritardo
6.8 Sommario
Problemi
CAPITOLO 7 **Il livello applicazione**
7.1 DNS – domain name system
7.2 Posta elettronica
7.3 Il World Wide Web
7.4 Streaming audio e video
7.5 Distribuzione di contenuti
7.6 Sommario
Problemi
CAPITOLO 8 **Sicurezza delle reti**
8.1 Crittografia
8.2 Algoritmi a chiave simmetrica
8.3 Algoritmi a chiave pubblica
8.4 Firme digitali
8.5 Gestione delle chiavi pubbliche
8.6 Sicurezza delle comunicazioni
8.7 Protocolli di autenticazione
8.8 Sicurezza della posta elettronica
8.9 Sicurezza del Web
8.10 Aspetti sociali
8.11 Sommario
Problemi
CAPITOLO 9 **Letture consigliate e bibliografia**
9.1 Suggerimenti per ulteriori letture
9.2 Bibliografia

Prefazione all'edizione italiana

Giunto ormai alla quinta edizione, il testo *Reti di calcolatori* di Andrew S. Tanenbaum si conferma uno dei testi di maggior successo sia per l'insegnamento delle reti sia per l'uso professionale, grazie anche al coinvolgimento di David J. Wetherall come coautore.

In questi ultimi anni gli operatori nel settore delle telecomunicazioni sono stati sia promotori che testimoni di profondi cambiamenti, che hanno portato all'introduzione di nuove tecnologie e all'obsolescenza di altre. Queste novità hanno costretto il mondo accademico a rivedere significativamente non solo parte della letteratura ma anche il modo di fare didattica. Gli autori si fanno testimoni di questi cambiamenti attraverso una revisione completa e approfondita del testo, con una particolare attenzione alle reti a larga banda, alle tecnologie wireless, alle architetture web e alle reti peer-to-peer.

Il libro sulle reti di Tanenbaum è stato, per anni, il primo testo per l'avvicinamento all'argomento anche per molti di coloro che oggi le insegnano. Nonostante i radicali cambiamenti apportati durante le varie edizioni si continua a riconoscere uno stile inconfondibile: leggero e godibile quando affronta gli argomenti in senso generale, spumeggiante e divertente nel fornire aneddoti e considerazioni sul mondo reale e, infine, preciso e rigoroso, ma mai noioso, nella trattazione tecnica e scientifica.

Il viaggio nel mondo delle telecomunicazioni proposto dagli autori parte dai fondamenti, seguendo quello che ormai è diventato "l'approccio classico". Prosegue poi, in un crescendo di astrazione, attraverso i protocolli e le architetture che costituiscono Internet. Durante il percorso, la visione storica di Tanenbaum è fondamentale per capire l'evoluzione delle reti sotto un profilo non solo tecnologico ma anche commerciale; in particolare gli autori riflettono sempre su quali siano state le cause scatenanti la progettazione (o l'abbandono) di una tecnologia, favorendo l'acquisizione di una metodologia scientifica da parte del lettore, attraverso un continuo susseguirsi di domande e risposte. Prima di congedarsi, parlando di crittografia e sicurezza, gli autori si spingono oltre gli aspetti tecnici, toccando temi quali la libertà di espressione, la censura e la gestione del copyright; argomenti che per un motivo o per l'altro vivono a stretto contatto con il mondo delle reti, ma che in molti libri non vengono trattati in quanto non strettamente tecnici. Il vero valore aggiunto durante tutto il percorso, rispetto ad altri testi, è quello di fare continue osservazioni di carattere non solo storico ma anche organizzativo e commerciale, contestualizzando il contenuto tecnico in una realtà industriale con la quale il lettore, sia esso studente, professionista o semplicemente consumatore, dovrà andarsi a confrontare.

Questa nuova edizione, aggiornata e ampliata, si propone non solo come un valido ausilio per docenti e studenti, ma anche come punto di riferimento per il professionista in cerca di un manuale aggiornato e approfondito. L'edizione italiana, in particolare, cerca di dare un contributo alla dimensione internazionale assunta dal lavorare nel campo delle telecomunicazioni. Si è infatti cercato il miglior compromesso possibile tra l'uso di una terminologia italiana, a noi familiare, e quella inglese, ormai pervasiva in ambito professionale; ove possibile si sono fornite entrambe le diciture di ogni concetto, cercando però di utilizzare nel testo i termini normalmente adottati in un contesto lavorativo.

Sabrina Gaito, Dario Maggiorini

Università degli Studi di Milano

Prefazione

Questo libro è arrivato alla quinta edizione. Ogni edizione trova corrispondenza con una fase diversa nell'uso che è stato fatto delle reti di calcolatori. Quando apparve la prima edizione, nel 1980, le reti erano una curiosità accademica. Al tempo della seconda edizione, il 1988, le reti erano usate da università e grandi aziende. Quando, nel 1996, fu pubblicata la terza edizione, le reti di calcolatori e in special modo Internet, erano diventate una realtà per milioni di persone. Con la quarta edizione, nel 2003, le reti wireless e i dispositivi mobili erano diventati il modo più comune di accedere al Web e a Internet. Ora, nella quinta edizione, le reti si focalizzano sulla distribuzione di contenuti (in particolar modo video attraverso CDN e reti peer-to-peer) e i telefoni cellulari sono piccoli computer collegati a Internet.

Novità della quinta edizione

Tra i molti cambiamenti in questo libro, il più importante è l'aggiunta del professor David J. Wetherall in qualità di coautore. David porta con sé un ricco bagaglio di conoscenza sulle reti avendo iniziato più di venti anni fa con la progettazione di reti metropolitane; da allora si è occupato di Internet e reti wireless e oggi è professore all'Università di Washington, dove insegna e fa ricerca nell'ambito delle reti di calcolatori, e argomenti correlati, da una decina d'anni.

Naturalmente, il libro ha anche subito molti cambiamenti per tenere il passo con il mondo, sempre in movimento, delle reti di calcolatori. Tra questi troviamo materiale nuovo e aggiornato su:

- Reti wireless (802.12 e 802.16)
- Le reti 3G usate dagli smartphone
- RFID e reti di sensori
- Distribuzione di contenuti tramite CDN
- Reti peer-to-peer
- Multimedia in tempo reale (registrata e in diretta)
- Telefonia su Internet (voice over IP)
- Reti con tolleranza al ritardo

Il Capitolo 1 svolge la stessa funzione introduttiva che aveva nella quarta edizione, ma i contenuti sono stati rivisti e aggiornati. Internet, le reti di telefonia mobile, 802.11, RFID e le reti di sensori sono illustrate come esempi di reti di calcolatori. Il materiale riguardante la prima versione di Ethernet e ATM è stato tolto.

Nel Capitolo 2, dove ci si occupa del livello fisico, è stata ampliata la trattazione della modulazione digitale (includendo OFDM in quanto largamente utilizzata nelle reti wireless). Vengono presentate anche nuove tecnologie di rete, incluse quelle che sfruttano fibre ottiche fino all'abitazione dell'utente (*fiber to the home*) o che prevedono l'uso delle linee elettriche domestiche.

Il Capitolo 3, che parla di collegamenti punto a punto, è stato migliorato sotto due punti di vista. Il materiale riguardante i codici di rilevamento e correzione degli errori è stato aggiornato ed include ora una breve descrizione di codici moderni molto importanti nelle applicazioni pratiche (come,

ad esempio, codici convoluzionali e LDPC). I protocolli portati ad esempio ora fanno riferimento all'invio di pacchetti su SONET e ad ADSL. Il materiale riguardante la verifica dei protocolli è stato rimosso perché scarsamente utilizzato.

Nel Capitolo 4, sul sottolivello MAC, i principi sono sempre gli stessi ma la tecnologia è cambiata. I paragrafi sulle reti portate come esempio sono stati riscritti di conseguenza, includendo gigabit Ethernet, 802.11, 802.16, Bluetooth e RFID. È stata anche aggiornata la parte relativa alle LAN commutate, includendo le VLAN.

Il Capitolo 5, sul livello di rete, si occupa degli stessi argomenti della quarta edizione. Nella revisione sono stati aggiornati i contenuti e si è deciso di approfondire la qualità di servizio (importante per i contenuti multimediali in tempo reale) e la connessione tra reti. I paragrafi su BGP, OSPF e CIDR sono stati estesi, come pure è stato esteso quello sull'instradamento multicast. Infine, è stato aggiunto l'instradamento anycast.

Il Capitolo 6, sul livello di trasporto, ha avuto dei contenuti aggiunti, altri rivisti e altri ancora rimossi. I contenuti aggiuntivi descrivono le reti con tolleranza al ritardo e propongono una trattazione generale sul controllo di congestione. I contenuti aggiornati espandono la trattazione relativa al controllo di congestione di TCP. Infine, sono state rimosse le parti relative ai livelli di rete orientati alla connessione in quanto molto rari al giorno d'oggi.

Anche il Capitolo 7, sulle applicazioni, è stato aggiornato ed esteso. Anche se il materiale riguardante DNS e posta elettronica è simile alla quarta edizione, negli ultimi anni ci sono stati molti sviluppi riguardo l'uso del Web, i contenuti multimediali in streaming e la distribuzione di contenuti. Di conseguenza, i paragrafi su Web e multimedia sono stati aggiornati. Un nuovo paragrafo descrive la distribuzione di contenuti includendo CDN e reti peer-to-peer.

Il Capitolo 8, sulla sicurezza, si occupa ancora di entrambe le cifrature a chiave simmetrica e a chiave pubblica per garantire la confidenzialità e il fatto che i messaggi non siano stati alterati. Il materiale sulle tecniche utilizzate nella pratica, tra cui firewall e VPN è stato aggiornato, mentre sono state aggiunte parti riguardanti la sicurezza di 802.11 e Kerberos V5.

Il Capitolo 9 contiene una lista aggiornata di letture consigliate e una bibliografia completa di oltre 300 citazioni. Più della metà di queste citazioni sono articoli e libri scritti dal 2000 in poi, il resto sono citazioni di articoli classici.

Lista degli acronimi

I libri di informatica sono pieni di acronimi, e questo non fa eccezione. Quando avrete finito di leggere questo libro, le seguenti sigle vi saranno familiari: ADSL, AES, AJAX, AODV, AP, ARP, ARQ, AS, BGP, BOC, CDMA, CDN, CGI, CIDR, CRL, CSMA, CSS, DCT, DES, DHCP, DHT, DIFS, DMCA, DMT, DMZ, DNS, DOCSIS, DOM, DSLAM, DTN, FCFS, FDD, FDDI, FDM, FEC, FIFO, FSK, FTP, GPRS, GSM, HDTV, HFC, HMAC, HTTP, IAB, ICANN, ICMP, IDEA, IETF, IMAP, IMP, IP, IPTV, IRTF, ISO, ISP, ITU, JPEG, JSP, JVM, LAN, LATA, LEC, LEO, LLC, LSR, LTE, MAN, MFJ, MIME, MPEG, MPLS, MSC, MTSO, MTU, NAP, NAT, NRZ, NSAP, OFDM, OSI, OSPF, PAWS, PCM, PGP, PIM, PKI, POP, POTS, PPP, PSTN, QAM, QPSK, RED, RFC, RFID, RPC, RSA, RTSP, SHA, SIP, SMTP, SNR, SOAP, SONET, SPE, SSL, TCP, TDD, TDM, TSAP, UDP, UMTS, URL, VLAN, VSAT, WAN, WDM e XML.

Non preoccupatevi: ognuno di loro apparirà in **grassetto** e verrà accuratamente spiegato prima di essere usato. Per fare una prova, vedete quanti siete in grado di identificarne *prima* di leggere il libro, scrivete il numero a margine e provate di nuovo *dopo* aver letto il libro.

Come usare questo libro

Per aiutare i docenti a usare questo libro come testo di riferimento, abbiamo strutturato i capitoli in concetti fondamentali e opzionali. I paragrafi contrassegnati con un "*" nell'indice trattano dei contenuti opzionali. Se il paragrafo principale (ad esempio il 2.7) è contrassegnato in questo modo tutti i suoi sottoparagrafi sono anch'essi opzionali. I paragrafi opzionali propongono materiale utile, ma che può essere omesso in un corso breve senza incorrere in una perdita di continuità didattica. Naturalmente, gli studenti dovrebbero essere incoraggiati a leggere anche questi paragrafi, nella misura in cui hanno tempo disponibile, perché tutto il materiale proposto è attuale e importante.

Materiali didattici di supporto

Dall'indirizzo *http://hpe.pearson.it/tanenbaum* è possibile accedere a un Companion Website (in inglese) contenente utili supplementi, tra cui link a articoli, tutorial e FAQ organizzati per argomenti, simulatori di protocolli e una demo liberamente scaricabile sulla steganografia.

I docenti che adottano il testo, inoltre, potranno richiedere il Solutions Manual (in inglese) e le slide in PowerPoint con le figure del testo (in italiano).

Ringraziamenti

Molte persone ci hanno aiutato durante la stesura della quinta edizione. Vogliamo ringraziare in maniera particolare Emmanuel Agu (Worcester Polytechnic Institute), Yoris Au (University of Texas at San Antonio), Nikhil Bhargava (Aircom International, Inc.), Michael Buettner (University of Washington), John Day (Boston University), Kevin Fall (Intel Labs), Ronald Fulle (Rochester Institute of Technology), Ben Greenstein (Intel Labs), Daniel Halperin (University of Washington), Bob Kinicki (Worcester Polytechnic Institute), Tadayoshi Kohno (University of Washington), Sarvish Kulkarni (Villanova University), Hank Levy (University of Washington), Ratul Mahajan (Microsoft Research), Craig Partridge (BBN), Michael Piatek (University of Washington), Joshua Smith (Intel Labs), Neil Spring (University of Maryland), David Teneyuca (University of Texas at Antonio), Tammy VanDegrift (University of Portland) e Bo Yuan (Rochester Institute of Technology), per averci dato idee e commenti. Melody Kadenko e Julie Svendsen hanno dato supporto organizzativo a David.

Shivakant Mishra (University of Colorado at Boulder) e Paul Nagin (Chimborazo Publishing, Inc.) hanno pensato a molti problemi nuovi e stimolanti per la fine dei capitoli. Il nostro contatto alla Pearson, Tracy Dunkelberger, è stata di aiuto in molti aspetti, come al solito. Melinda Haggerty e Jeff Holcomb hanno fatto un buon lavoro nel far procedere le cose senza problemi. Steve Armstrong (LeTourneau University) ha preparato i lucidi. Stephen Turner (University of Michigan at Flint) ha controllato con arte le risorse Web e i simulatori che accompagnano il libro. Il nostro copyeditor, Rachel Head, è uno strano ibrido: ha l'occhio di un'aquila e la memoria di un elefante. Dopo aver letto tutte le sue correzioni, entrambi ci siamo chiesti come abbiamo fatto a finire le elementari.

Infine, arriviamo alle persone più importanti. Suzanne è già passata diciannove volte attraverso questo e ancora dimostra infinita pazienza e amore. Barbara e Marvin, che adesso conoscono la differenza tra libri di testo buoni e cattivi, sono sempre un'ispirazione per produrne di buoni. Daniel e Matilde sono delle gradite aggiunte alla nostra famiglia. Aron è difficile che legga questo libro nell'immediato futuro, ma a lui piacciono le belle figure del Capitolo 8 (Tanenbaum).

Katrin e Lucy hanno dato un sostegno infinito e sono sempre riuscite a darmi un sorriso. Grazie (Wetherall).

Andrew S. Tanenbaum

David J. Wetherall

CAPITOLO 1
Introduzione

Ciascuno dei tre secoli passati è stato dominato da una specifica tecnologia. Nel diciottesimo secolo i grandi sistemi meccanici hanno accompagnato la rivoluzione industriale. Il diciannovesimo secolo è stato l'era del motore a vapore. Nel corso del ventesimo secolo la tecnologia chiave è stata la raccolta, elaborazione e distribuzione dell'informazione. Alcuni dei principali sviluppi a cui abbiamo assistito sono la costruzione di una rete telefonica mondiale, l'invenzione della radio e della televisione, la nascita e la crescita senza precedenti dell'industria informatica, il lancio dei satelliti per le telecomunicazioni e, naturalmente, l'avvento di Internet.

In conseguenza del rapido processo tecnologico, nel ventunesimo secolo queste aree stanno rapidamente convergendo e le differenze tra raccolta, trasporto, archiviazione ed elaborazione dell'informazione stanno rapidamente svanendo. Enti pubblici e privati con centinaia di uffici sparpagliati su una vasta area geografica pretendono di poter ispezionare periodicamente lo stato del loro ufficio più remoto premendo un tasto. Man mano che la nostra capacità di raccogliere, elaborare e distribuire informazioni cresce, la richiesta di elaborazioni sempre più sofisticate aumenta ancora più velocemente.

Benché l'industria informatica sia ancora giovane a confronto di altre, come quelle automobilistica e dei trasporti aerei, i computer hanno compiuto progressi spettacolari in breve tempo. Nei primi due decenni della loro esistenza i sistemi informatici erano altamente centralizzati, tanto da essere solitamente ospitati in una sola grande stanza, spesso dotata di pareti di vetro, attraverso le quali i visitatori potevano ammirare la grande meraviglia elettronica all'interno. Un'azienda di medie dimensioni o un'università potevano disporre di uno o due computer, mentre le grandi istituzioni ne possedevano al massimo una dozzina. L'idea che entro quarant'anni sarebbero stati prodotti miliardi di computer enormemente più potenti, ma più piccoli di un francobollo, era vera fantascienza.

La convergenza di computer e comunicazioni ha esercitato un'influenza profonda sul modo in cui vengono organizzati i calcolatori. Il concetto un tempo dominante di "centro di calcolo" come una stanza con un grande computer dove gli utenti portavano il loro lavoro per l'elaborazione oggi è completamente superato (anche se data center ospitanti migliaia di server sono sempre più diffusi). Il vecchio modello di un solo computer che soddisfa l'intera necessità di calcolo di un'azienda è stato sostituito da un altro in cui il lavoro è svolto da un gran numero di computer distinti, ma connessi. Questi sistemi sono chiamati **reti di calcolatori** (*computer network*). La progettazione e l'organizzazione di queste reti sono l'oggetto di questo libro.

Nel testo useremo il termine "rete di calcolatori" per indicare un insieme di computer autonomi connessi da una singola tecnologia. Due computer si dicono connessi quando sono in grado di scambiare informazioni. Il collegamento non è necessariamente realizzato con cavi di rame; si possono usare anche fibre ottiche, microonde, luce infrarossa e satelliti per comunicazioni. Le reti, come vedremo in seguito, hanno dimensioni, topologie e forme differenti e vengono solitamente connesse tra di loro a formare reti più grandi, fino ad arrivare a **Internet** che è il più famoso esempio di rete di reti.

C'è molta confusione nella letteratura tra reti di calcolatori e **sistemi distribuiti**. La differenza fondamentale è che in un sistema distribuito l'insieme dei computer indipendenti appare ai propri utenti come un singolo sistema coerente. Di solito un sistema distribuito ha un solo modello o paradigma che presenta agli utenti per mezzo di un livello di software posto sopra al sistema operativo, chiamato **middleware**. Un esempio ben noto di sistema distribuito è il **World Wide Web** che si appoggia su Internet e implementa un modello per cui tutto ha l'aspetto di un documento (pagina Web).

In una rete di calcolatori mancano la coerenza, il modello e il software tipici dei sistemi distribuiti. Gli utenti vedono le macchine effettive e il sistema non compie alcun tentativo di far apparire e agire le macchine in un modo coerente. Se le macchine hanno hardware e sistemi operativi differenti, ciò resta completamente visibile agli utenti. Se un utente vuole eseguire un programma su una macchina remota, deve collegarsi a quella macchina ed eseguirlo lì.

In effetti un sistema distribuito è un'architettura software costruita a partire da una rete. Il software dà al sistema un elevato grado di coesione e trasparenza all'utente. Di conseguenza, la differenza tra una rete e un sistema distribuito sta nel software (in particolare nel sistema operativo) piuttosto che nell'hardware.

In ogni caso c'è una sovrapposizione considerevole tra i due argomenti; per esempio, sia i sistemi distribuiti sia le reti di calcolatori hanno necessità di spostare file. La differenza sta in chi opera questo movimento: il sistema o l'utente.

Benché questo libro si concentri principalmente sulle reti, molti argomenti trattati sono importanti anche per i sistemi distribuiti. Per maggiori informazioni sui sistemi distribuiti si veda il volume di Tanenbaum e Van Steen *Sistemi distribuiti*, Pearson Education Italia, 2007.

1.1 Applicazioni delle reti di calcolatori

Prima di iniziare a esaminare nel dettaglio gli aspetti tecnici, vale la pena soffermarsi a considerare i motivi per cui le reti di calcolatori suscitano un così vivo interesse e gli scopi a cui possono essere destinate. In fondo, se nessuno si fosse interessato alle reti, ne sarebbero state costruite poche. Cominceremo col prendere in esame i tradizionali utilizzi in ambito aziendale, per passare poi alle reti domestiche e ai recenti sviluppi riguardanti gli utenti mobili; per finire ne considereremo i risvolti sociali.

Applicazioni aziendali

Molte aziende dispongono di un numero considerevole di computer. Per esempio, in un'azienda ogni impiegato ha il proprio computer che utilizza per la progettazione di prodotti, per la scrittura di brochure o per la gestione delle buste paga. Inizialmente alcuni di questi computer potevano funzionare separatamente dagli altri, ma a un certo punto si è deciso di collegarli tra loro per distribuire le informazioni a tutta l'azienda. Generalizzando, il punto focale in questo caso è la **condivisione delle risorse** e l'obiettivo è quello di rendere disponibili in rete tutti i programmi, le periferiche e soprattutto i dati, indipendentemente dalla posizione fisica dell'utente e della risorsa. Un esempio ovvio e largamente diffuso riguarda la necessità di condividere una stampante tra un gruppo di utenti in un ufficio. Nessuno ha la necessità effettiva di una stampante personale e una stampante di rete ad alta produttività è spesso più economica, più veloce e più semplice da gestire di una moltitudine di stampanti personali.

Tuttavia, una delle risorse da condividere anche più importante rispetto a quelle fisiche come stampanti o sistemi di backup su nastro è l'informazione. Qualsiasi azienda di dimensioni medie o grandi dipende in modo vitale dalle informazioni che vengono gestite tramite calcolatori. La maggioranza delle aziende mantiene sempre disponibili informazioni sulla clientela e sui prodotti, inventari della merce, fatture, moduli fiscali e molto altro ancora. Se tutti i suoi computer si fermassero all'improvviso, una banca non potrebbe sopravvivere più di cinque minuti. Una fabbrica moderna, con una linea di montaggio controllata dai computer, non resisterebbe nemmeno cinque secondi. Oggi, per consentire agli impiegati l'accesso immediato a informazioni e documenti necessari alla loro attività, anche una piccola agenzia di viaggi o uno studio legale di tre persone si affida pesantemente alle reti di calcolatori.

Nelle aziende più piccole tutti i computer sono probabilmente concentrati in un solo ufficio o edificio, ma in quelle più grandi i computer e gli impiegati possono essere sparsi in dozzine di uffici e impianti situati in diversi paesi. Nonostante ciò, un addetto alle vendite a New York potrebbe occasionalmente dover accedere al database dell'inventario delle merci a Singapore. Le reti chiamate **VPN** (*virtual private network,* reti private virtuali) sono utilizzate per connettere più reti personali situate in luoghi diversi in un'unica rete. In altre parole, il semplice fatto che un utente si trovi a 15.000 km dai dati non deve impedirgli di usarli come se fossero locali. Semplificando al massimo, si tratta di un tentativo di porre fine alla "tirannia della geografia".

In sintesi ci si può immaginare il sistema informatico di un'azienda come formato da uno o più database e da un certo numero di impiegati che hanno bisogno di accedervi da remoto. In questo modello i dati sono memorizzati in computer ad alte prestazioni chiamati **server**. Frequentemente sono installati in un ufficio centrale e gestiti da un amministratore di sistema. Al contrario, gli impiegati dispongono di macchine più semplici, chiamate **client**, tramite le quali hanno accesso ai dati remoti, per esempio per incorporarli nei fogli di calcolo che stanno realizzando (a volte indicheremo col nome "client" l'utente umano del computer client, ma sarà chiaro dal contesto se ci riferiremo al computer o al rispettivo utente). Le macchine client e server sono collegate da una rete, come illustrato nella Figura 1.1. Si osservi che la rete viene indicata con un semplice ovale, senza dettagli. Si utilizzerà questa forma per indicare una rete in modo astratto. Maggiori dettagli saranno forniti in seguito quando necessari.

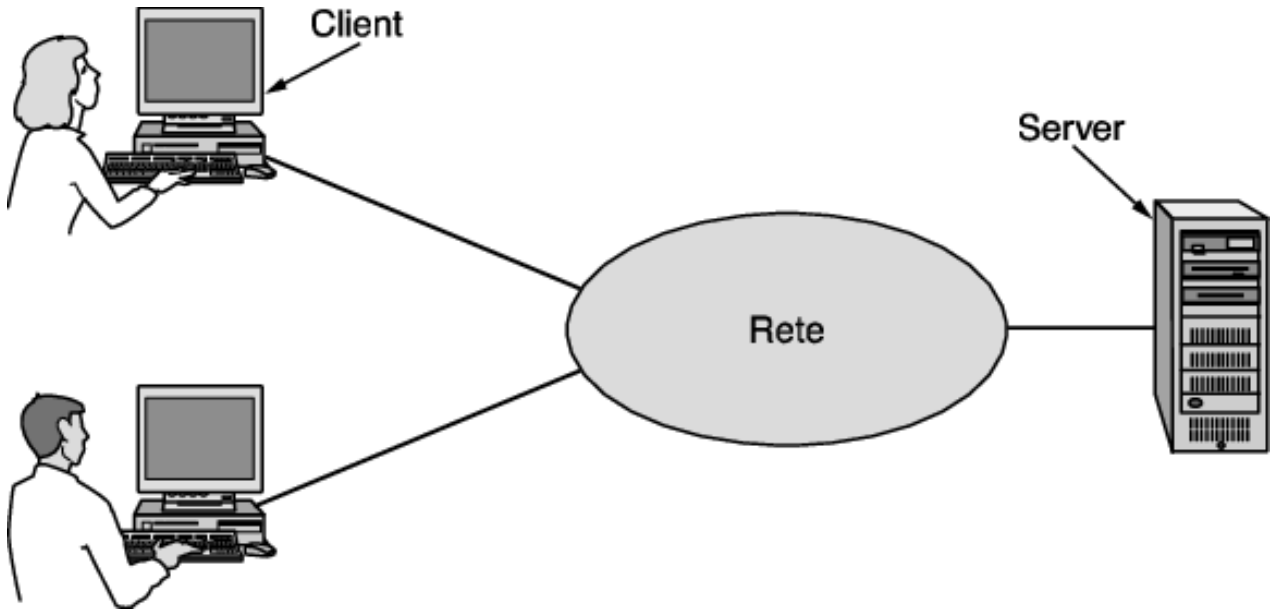


Figura 1.1 Una rete con due client e un server.

Questa configurazione, chiamata **modello client-server**, è ampiamente utilizzata e rappresenta il punto di partenza di gran parte dell'utilizzo delle reti. L'implementazione più nota è quella di un'**applicazione Web** in cui il server genera a partire dal suo database le pagine Web in risposta a richieste da parte del client che potrebbero, a loro volta, aggiornare il database stesso. Il modello client-server è applicabile quando il client e il server sono nello stesso edificio (per esempio appartengono alla stessa azienda), ma anche quando sono molto distanti. Per esempio, quando una persona accede da casa a una pagina sul World Wide Web si utilizza lo stesso modello; in questo caso il server Web remoto svolge il ruolo di server e il personal computer dell'utente è il client. Nella maggior parte delle situazioni un server può soddisfare le richieste di un gran numero (centinaia o migliaia) di client contemporaneamente.

Se guardiamo in dettaglio il modello client-server vediamo che sono coinvolti due processi (programmi in esecuzione), uno sulla macchina client e uno sulla macchina server. La comunicazione è rappresentata da un processo client che manda un messaggio attraverso la rete al processo server e resta in attesa di un messaggio di risposta. Quando il processo server riceve la richiesta, esegue il lavoro o recupera i dati desiderati e restituisce una risposta. Questi messaggi sono mostrati nella Figura 1.2.

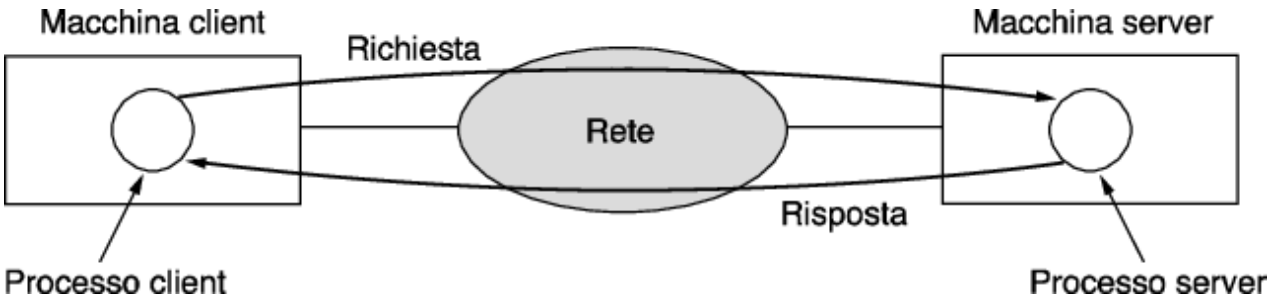


Figura 1.2 Il modello client-server comprende richieste e risposte.

Un secondo motivo per realizzare una rete di calcolatori ha a che fare con le persone piuttosto che con l'informazione o i computer stessi. Una rete di calcolatori può offrire un potente **mezzo di comunicazione** tra gli impiegati. Praticamente, tutte le aziende che hanno due o più computer oggi dispongono di **e-mail** (*posta elettronica*), utilizzata solitamente dagli impiegati per gran parte delle comunicazioni quotidiane. In effetti, una tipica lamentela durante la pausa caffè riguarda la quantità di e-mail da affrontare quotidianamente, per la maggior parte inutili, poiché i dirigenti hanno scoperto che con la semplice pressione di un tasto possono inviare lo stesso messaggio (spesso privo di contenuto) contemporaneamente a tutti i loro collaboratori.

Le telefonate tra impiegati possono essere effettuate tramite la rete dei calcolatori piuttosto che quella di un operatore telefonico. Questa tecnologia, se eseguita tramite Internet, è chiamata **telefonia IP** o **Voice over IP**, **VoIP**. Da entrambi i lati la telefonata può essere fatta usando un telefono abilitato alle chiamate IP o un computer. Un bel risparmio per le aziende!

Le reti di calcolatori permettono anche altre e più sofisticate forme di comunicazione. Il video può essere aggiunto alla voce, cosicché collaboratori e colleghi distanti possano sentirsi e vedersi durante le riunioni, riducendo tempi e costi dovuti agli spostamenti. La condivisione dello schermo (*desktop sharing*) permette a due o più persone lontane tra loro di interagire a distanza leggendo e scrivendo su una lavagna condivisa o redigendo una relazione congiunta. Quando un utente modifica un documento, la modifica apportata è immediatamente visibile a tutti con un conseguente risparmio di tempi e procedure. Questa accelerazione semplifica la cooperazione. Forme più ambiziose di collaborazione in remoto, quali la telemedicina, stanno iniziando a essere utilizzate solo ora (pensiamo al monitoraggio remoto di un paziente), ma in futuro potrebbero diffondersi notevolmente. Spesso si dice che comunicazione e trasporti sono in competizione e chi vincerà renderà l'altro obsoleto.

Per molte aziende un terzo obiettivo consiste nello svolgere i propri affari per via elettronica, in particolare con clienti e fornitori. Questo nuovo modello si chiama **e-commerce** (*electronic commerce*, commercio elettronico) ed è cresciuto rapidamente negli ultimi anni. Compagnie aeree, librerie e altri esercizi commerciali hanno scoperto che molti clienti apprezzano la comodità di fare acquisti da casa. Di conseguenza sono molte le aziende che pubblicano i propri cataloghi di prodotti e servizi e che ricevono ordini on-line. Case automobilistiche, produttori di aerei e computer acquistano componenti da diversi fornitori per poi assemblarli. Utilizzando le reti di calcolatori, i produttori emettono gli ordini di acquisto on-line, secondo le necessità del momento, riducendo le scorte di magazzino in eccesso e aumentando l'efficienza.

Applicazioni domestiche

Nel 1977 Ken Olsen era presidente della Digital Equipment Corporation, allora azienda di produzione di computer seconda solo a IBM nel mondo. Quando gli chiesero perché Digital non stesse entrando nel mercato dei personal computer questi rispose: "Un privato non ha alcun motivo di possedere un computer a casa propria". La storia ha dimostrato il contrario e ora la Digital non esiste più. Inizialmente i personal computer venivano acquistati per l'elaborazione di testi e per i giochi. Recentemente, il motivo principale per l'acquisto di un personal computer è l'accesso a Internet. Attualmente, molti dispositivi elettronici di consumo, come set-top box, console di gioco e radiosveglie, sono venduti con computer incorporati e funzionalità di rete. In particolare, le reti wireless e le reti domestiche sono ampiamente utilizzate per attività di intrattenimento quali ascolto, visione e creazione di musica, video e foto.

L'accesso a Internet fornisce agli utenti domestici la **connessione** a computer remoti. Essi, come le aziende, possono accedere a informazioni, comunicare con altre persone e comprare prodotti e servizi tramite l'e-commerce. Il vantaggio principale è attualmente quello di potersi connettere al mondo da casa propria. Bob Metcalfe, l'inventore di Ethernet, ipotizzò che il valore di una rete fosse proporzionale al quadrato del numero dei suoi utenti, perché questo è all'incirca il numero di potenziali connessioni (Gilder, 1993). Questa ipotesi, nota come "legge di Metcalfe", spiega come l'enorme popolarità di Internet derivi dalle sue dimensioni.

L'accesso alle informazioni a distanza può avvenire in vari modi. Si può navigare il World Wide Web per ottenere informazioni o solo per il puro gusto di farlo. Le informazioni disponibili appartengono a tutti i campi del sapere quali belle arti, affari, cucina, politica, salute, storia, hobby, scienze, sport, viaggi e molto altro. Le possibili forme di intrattenimento reperibili in rete sono talmente numerose da non poter essere elencate e alcune è opportuno tacerle.

Molti quotidiani vengono ora pubblicati on-line e possono essere personalizzati: un utente può decidere di essere informato su politici corrotti, devastanti calamità naturali, scandali di celebrità, epidemie e, magari, anche il calcio.

A volte è possibile scaricare gli articoli selezionati mentre si dorme. Questa tendenza causerà un forte tasso di disoccupazione tra gli addetti alla distribuzione, ma è molto gradita ai giornali che hanno sempre visto nella distribuzione l'anello debole della catena di produzione. Naturalmente, affinché questo modello funzioni, dovranno prima capire come trarre guadagno da questo nuovo scenario, aspetto non secondario, in quanto su Internet gli utenti si aspettano che tutto sia gratis.

Il passo successivo ai quotidiani (oltre ai periodici e alle pubblicazioni scientifiche) è la biblioteca digitale on-line. Molti enti, come ACM (www.acm.org) e IEEE Computer Society (www.computer.org), hanno già disponibili on-line tutte le loro riviste e atti di conferenze. Gli e-book e le librerie on-line rischiano di rendere i libri stampati obsoleti. Gli scettici dovrebbero ricordare l'effetto che la stampa ebbe sui manoscritti miniati del Medioevo.

A molti di questi contenuti si accede tramite il modello client-server, ma esiste un altro popolare modello di accesso alle informazioni che va sotto il nome di comunicazione **peer-to-peer** (Parameswaran et al., 2001). In questa forma di comunicazione, gli individui che costituiscono un gruppo spontaneo sono in grado di comunicare tra di loro, come mostrato nella Figura 1.3. In linea di principio, ognuno può comunicare con chiunque all'interno del gruppo senza che vi sia una divisione predefinita tra server e client.

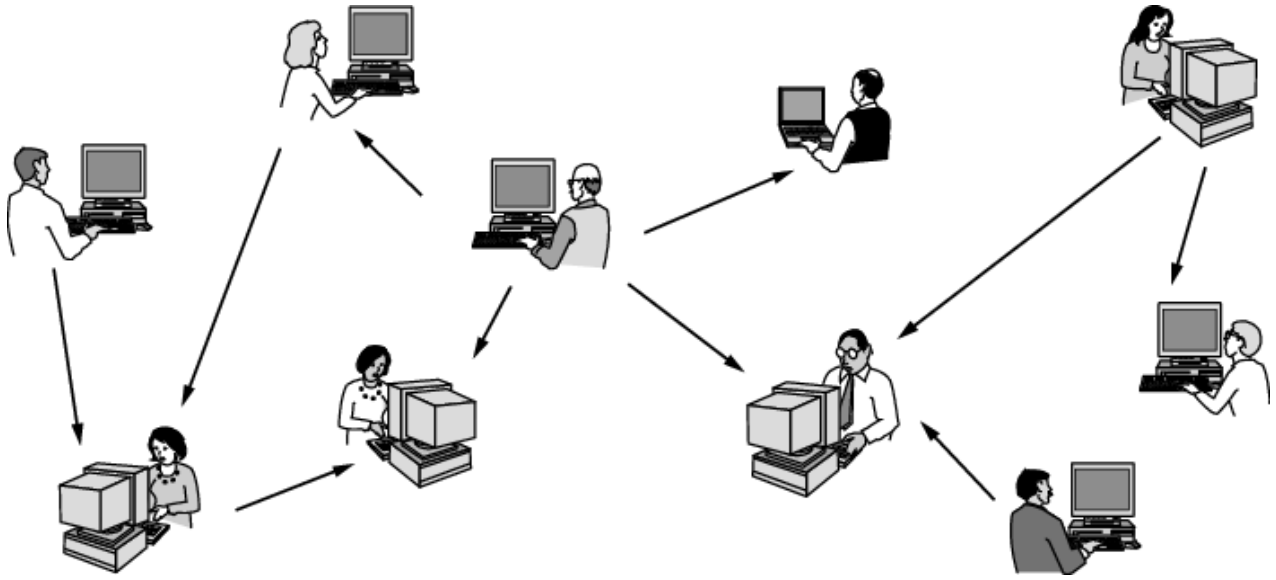


Figura 1.3 In un sistema peer-to-peer non ci sono client e server predefiniti.

Molti sistemi peer-to-peer non hanno alcun database centrale dei contenuti. Invece, ogni utente mantiene il proprio database in locale e fornisce un elenco di altri utenti a lui noti che fanno parte del sistema. Un nuovo utente può quindi visitare qualunque altro membro sia per vedere che cosa abbia sia per ottenere i nominativi di altri membri da ispezionare a loro volta alla ricerca di altri contenuti e nomi. Questo processo di ricerca può essere ripetuto indefinitamente per costruire un ampio database locale. Si tratta di un'attività noiosa per gli esseri umani, ma in cui i computer eccellono.

La comunicazione peer-to-peer viene spesso utilizzata per la condivisione di musica e video. Divenne popolare attorno al 2000 con un servizio di condivisione di file musicali, chiamato Napster, che venne chiuso in quanto fu probabilmente la più grande violazione di copyright nella storia dell'industria discografica (Lam e Tan, 2001; Macedonia, 2000). Esistono anche applicazioni

legali della comunicazione peer-to-peer: appassionati che condividono musica di pubblico dominio o brani musicali che gruppi emergenti rilasciano a scopo promozionale, famiglie che condividono fotografie e filmati e utenti che scaricano software libero.

Tutte le applicazioni citate finora comportano interazioni tra un utente e un database remoto ricco di informazioni. La seconda grande categoria di utilizzo della rete è la comunicazione da persona a persona tramite i sistemi di messaggistica istantanea, fondamentalmente la risposta del ventunesimo secolo al telefono del diciannovesimo secolo. L'e-mail è già utilizzata quotidianamente da milioni di persone in tutto il mondo e il suo uso si va sempre più diffondendo. Essa contiene già normalmente audio e video, oltre che testi e immagini. Per soddisfare anche il senso dell'olfatto bisognerà attendere ancora un po'.

Qualsiasi adolescente sveglio fa un uso continuo di **instant messaging** (**messaggistica istantanea**). Questo servizio, derivato dal programma talk di UNIX, in uso dal 1970 circa, consente a due persone di scambiarsi messaggi in tempo reale. Esistono anche servizi di messaggistica tra più persone, come Twitter che permette agli utenti di inviare brevi messaggi di testo, chiamati "tweet" (cinguettii), al proprio gruppo di amici o ad altra persona interessata.

Internet può essere utilizzato dalle applicazioni per trasportare audio (ad esempio, le stazioni radio Internet) e video (ad esempio, YouTube). Oltre a essere un modo economico per contattare amici lontani, alcune applicazioni possono arricchire le conoscenze dell'utente. Un esempio è come l'**insegnamento a distanza**, grazie al quale si può assistere alla lezione delle 8 del mattino senza l'incomodo di alzarsi all'alba. A lungo termine, l'uso delle reti per migliorare la comunicazione tra persone risulterà prioritario rispetto a qualunque altro utilizzo; potrebbe diventare essenziale per fornire a coloro che vivono in zone disagevoli gli stessi servizi di chi abita nel centro di una grande città.

A metà strada tra le comunicazioni da persona a persona e l'accesso alle informazioni si collocano le applicazioni di **social network**. Qui, il flusso di informazioni è guidato dalle relazioni che le persone dichiarano di avere. Uno dei più popolari siti di social network è **Facebook**. Esso consente agli utenti di aggiornare i propri profili personali e condividere gli aggiornamenti con coloro che essi hanno dichiarato essere amici. Altre applicazioni di social networking permettono di allargare la propria rete di amicizie tramite gli amici degli amici, condividendo notizie, come nel caso di Twitter, e molto altro ancora.

Ancora più liberamente, gruppi di persone possono lavorare insieme per creare contenuti. Un **wiki**, per esempio, è un sito Web che i membri di una comunità aggiornano in maniera collaborativa. Il wiki più celebre è **Wikipedia**, l'enciclopedia che chiunque può aggiornare, ma ne esistono migliaia di altri.

La nostra terza categoria è il commercio elettronico nel senso più ampio del termine. L'acquisto da casa è già popolare e permette agli utenti di esaminare i cataloghi on-line di migliaia di aziende. Alcuni di questi cataloghi sono interattivi, in quanto mostrano i prodotti da diversi punti di vista e in configurazioni personalizzabili. Dopo l'acquisto, il cliente che non sa come usare un prodotto può consultare il supporto tecnico on-line.

Un'altra area in cui l'e-commerce sta già funzionando è l'accesso a servizi finanziari. Sono già molti a pagare le bollette, gestire il conto corrente e seguire i propri investimenti on-line. Tale impiego si diffonderà ulteriormente col miglioramento degli standard di sicurezza delle reti.

Un'area che praticamente nessuno aveva previsto è quella delle vendite e delle aste on-line. Le aste on-line di merce usata sono divenute un'industria significativa. A differenza dell'e-commerce tradizionale, che segue il modello client-server, le aste on-line sono più simili a un sistema peer-to-peer, nel senso che i consumatori agiscono sia come venditori sia come acquirenti.

Ad alcune di esse sono state attribuite simpatiche sigle basate sul fatto che in inglese "to" e "2" si pronunciano allo stesso modo. Le più usate sono elencate nella Figura 1.4.

Sigla	Sigla per esteso	Esempio
B2C	Business-to-consumer	Ordinare libri on-line
B2B	Business-to-business	Il costruttore di automobili ordina pneumatici dal fornitore
G2C	Government-to-consumer	Lo stato raccoglie la dichiarazione dei redditi elettronicamente
C2C	Consumer-to-consumer	Acquistare on-line prodotti usati in un'asta
P2P	Peer-to-peer	Condivisione di musica

Figura 1.4 Alcune forme di e-commerce.

La nostra quarta categoria è l'intrattenimento. Negli ultimi anni l'intrattenimento in rete in ambito domestico ha fatto passi da gigante grazie alla distribuzione di programmi di musica, radio e televisione; i film in Internet competono con quelli erogati dai noleggiatori di DVD. Gli utenti possono trovare, acquistare e scaricare brani MP3 e film in qualità DVD e arricchire la loro collezione personale. I programmi TV ora raggiungono molte case tramite sistemi **IPTV** (**IP TeleVision**), basati sulla tecnologia IP, al posto della TV via cavo o delle radiotrasmissioni. Applicazioni di media streaming consentono agli utenti di sintonizzarsi su stazioni radio in Internet o di assistere agli ultimi episodi dei loro programmi preferiti. Naturalmente, tutti i contenuti possono essere spostati sui diversi dispositivi video e audio, solitamente tramite una rete wireless.

Anche le trasmissioni in diretta TV potrebbero diventare interattive, permettendo al pubblico da casa di partecipare ai quiz, selezionare i concorrenti e così via.

Un'altra forma di intrattenimento è rappresentata dai **videogiochi**. Esistono già giochi di simulazione in tempo reale a più giocatori, quali nascondino in un labirinto virtuale e i simulatori di volo in cui una squadra cerca di abbatterne un'altra. I mondi virtuali offrono un ambiente persistente in cui migliaia di utenti possono sperimentare una realtà condivisa con grafica tridimensionale.

La nostra ultima categoria è l'**ubiquitous computing** in cui la computazione è inglobata nella vita quotidiana, come nella visione di Mark Weiser (1991). Molte case sono già cablate con sistemi di sicurezza che includono sensori per porte e finestre e molti altri sensori possono essere aggiunti per ottenere un monitoraggio intelligente. I consumi di acqua, gas ed elettricità vengono sempre più spesso monitorati in rete, con conseguente riduzione dei costi di lettura dei contatori casa per casa. Un rivelatore di fumo potrebbe allertare i vigili del fuoco al posto di un rumoroso allarme sonoro. In seguito al decremento del prezzo dei sensori e della trasmissione dati, sempre più misurazioni e relativi rilevamenti avverranno tramite la rete.

Un numero sempre crescente di dispositivi elettronici è in grado di collegarsi in rete; ad esempio, alcune fotocamere di alta fascia sono dotate di un collegamento di rete wireless e lo utilizzano per inviare foto a un video nelle vicinanze. I fotografi sportivi professionisti possono inviare le loro foto agli editori in tempo reale, prima connettendosi a un *access point* (punto di accesso) wireless e poi via Internet. I dispositivi che si collegano a una presa a muro, come i televisori, possono utilizzare la **rete elettrica** per inviare informazioni in tutta la casa tramite i fili che trasportano l'energia elettrica. Non c'è da meravigliarsi che oggetti ai quali non pensiamo come computer possano essere dotati di capacità sensoriali e di comunicazione. Ad esempio, la doccia può registrare l'uso dell'acqua, fornire un rapporto visuale mentre vi insaponate e riferire a un sistema di monitoraggio ambientale della casa, per contribuire a risparmiare sulla bolletta dell'acqua.

Una tecnologia chiamata **RFID** (**radio frequency identification**) contribuirà fortemente alla diffusione di queste idee in futuro. Le etichette RFID sono chip passivi (senza alimentazione) delle dimensioni di francobolli e possono già essere apposti su libri, passaporti, animali domestici, carte di credito e altri oggetti. Ciò consente ai lettori RFID di localizzare e comunicare con oggetti distanti fino a diversi metri, a seconda del tipo di RFID. Originariamente, la tecnologia RFID venne commercializzata per sostituire i codici a barre. Non ci è ancora riuscita perché i codici a barre sono gratuiti, mentre gli RFID hanno un costo di qualche centesimo. Naturalmente, le etichette RFID offrono molto di più, il loro prezzo è in rapida diminuzione e potrebbero trasformare il mondo reale in "The Internet of Things" (ITU, Novembre 2005).

Utenti mobili

I computer portatili, come laptop e palmari, sono uno dei segmenti in maggiore crescita nell'industria informatica. Le loro vendite hanno già superato quelle dei computer da tavolo. Perché? Chi è in viaggio vuole spesso usare i propri dispositivi portatili per leggere e inviare e-mail, tweet, guardare film, scaricare musica, giocare o semplicemente navigare sul Web. Vuole poter fare tutto ciò che fa a casa e in ufficio, e naturalmente, lo vuole fare da qualsiasi posizione di terra, mare o cielo.

La **connessione** a Internet rende possibili molte di queste applicazioni mobili. Data l'impossibilità di avere una connessione via cavo in macchina, imbarcazione e aereo le reti wireless suscitano molto interesse. Le reti telefoniche cellulari sono una forma ben nota di rete wireless che fornisce copertura per telefoni cellulari. Gli **hotspot** wireless basati sullo standard 802.11 sono un'altra forma di rete wireless per computer mobili. Queste reti sono spuntate in ogni luogo pubblico fornendo un mosaico di coperture di rete in caffè, alberghi, aeroporti, scuole, treni e aerei. Chiunque abbia un laptop e un modem wireless può accendere il proprio computer ed essere connesso a Internet attraverso l'hotspot, come se il computer fosse collegato a una rete via cavo.

Le reti wireless sono molto utili alle schiere di autisti di camion, taxi, corrieri espresso e addetti alla manutenzione per rimanere in contatto con la base. Per esempio, in molte città i conducenti dei taxi sono imprenditori indipendenti e non impiegati di una società. In alcune di queste città i taxi hanno un display visibile dal conducente. Quando un cliente chiama, un operatore di centrale inserisce i punti di prelievo e destinazione. Questa informazione viene mostrata sul display del conducente che emette anche un segnale sonoro. Il primo conducente che preme un pulsante sul display prende la chiamata.

Le reti wireless sono importanti anche per gli ambienti militari. Se è necessario essere in grado di combattere una guerra in qualsiasi punto della Terra con breve preavviso, probabilmente non è una buona idea fare affidamento su un'infrastruttura di rete fissa locale; è meglio portarsi la propria.

Benché le reti wireless e le applicazioni mobili siano spesso correlate, non sono la stessa cosa, come mostra la Figura 1.5. Vediamo qui una distinzione tra una rete wireless fissa e una portatile. Anche i computer portatili sono a volte collegati in modo fisso, come il caso di un viaggiatore che collega il computer portatile alla presa telefonica di una stanza d'albergo, ottenendo mobilità senza usare una rete wireless.

Wireless	Mobilità	Usi
No	No	Computer da tavolo negli uffici
No	Sì	Computer portatile usato in una stanza di albergo
Sì	No	Reti negli edifici non cablati
Sì	Sì	Palmare per l'inventario di magazzino

Figura 1.5 Combinazioni di reti wireless e applicazioni mobili.

D'altro canto, alcuni computer wireless non sono portatili. In abitazioni, uffici e alberghi privi di cablaggio potrebbe essere più conveniente connettere i propri computer da tavolo e lettori multimediali (media player) in modo wireless, piuttosto che installare dei cavi. Installare una rete wireless può richiedere il semplice acquisto di una scatola contenente qualche dispositivo elettronico. Questa soluzione potrebbe essere molto più economica, rispetto ad avere installatori che posino condotti per cablare l'edificio.

Esistono infine anche vere e proprie applicazioni mobili senza fili quali quelle per i palmari impiegati dagli addetti al controllo di un magazzino per fare l'inventario. In molti aeroporti ad alto traffico, gli addetti al ritiro delle auto a noleggio lavorano nei parcheggi con un computer portatile wireless. Scansionano il codice a barre o il chip RFID dell'auto restituita e il computer portatile, che ha una stampante incorporata, chiama il computer principale, riceve le informazioni sul noleggio e stampa al volo la fattura.

Forse il motore delle applicazioni wireless è il **telefono cellulare**. Lo scambio di messaggi di testo (SMS, *short message service*) è sempre più diffuso. Permette a chi ha un telefono cellulare di scrivere un breve messaggio che viene inviato dalla rete cellulare a un altro utente. Pochi avrebbero previsto fino a qualche anno fa che gli adolescenti che spendono il loro tempo a scrivere messaggini avrebbero costituito una ricca fonte di reddito per le compagnie telefoniche. Il servizio di messaggistica è molto redditizio, perché l'invio del messaggio costa alla compagnia telefonica solo una piccola frazione di centesimo, ma all'utente viene addebitato molto di più.

La tanto attesa convergenza tra telefonia e Internet è stata finalmente raggiunta e accelera la crescita delle applicazioni mobili. Gli **smartphone**, come il popolare iPhone, coniugano aspetti di telefono cellulare e computer portatile. Le reti cellulari (3G e 4G) alle quali si connettono forniscono sia un servizio dati per l'uso di Internet sia la gestione delle telefonate. Molti telefoni evoluti si connettono anche a hotspot wireless e cambiano rete automaticamente per scegliere l'opzione migliore per l'utente.

Anche altri dispositivi elettronici di consumo possono servirsi delle reti cellulari e di hotspot per mantenere la connessione con altri computer. Gli e-book reader possono scaricare un libro appena pubblicato, il nuovo numero di una rivista o un quotidiano, ovunque si trovino. Le cornici digitali possono aggiornare il loro video al momento giusto con nuove immagini.

Poiché i telefoni cellulari conoscono la loro posizione, spesso perché equipaggiati con un **GPS** (*global positioning system*), alcuni servizi sono deliberatamente georeferenziati. Mappe e indicazioni stradali sono ovvi candidati, in quanto il vostro cellulare o automobile dotati di GPS hanno probabilmente le idee più chiare di voi su dove si trovino. Allo stesso modo sono georeferenziati la ricerca di una libreria o di un ristorante cinese nelle vicinanze o le previsioni meteo. Altri servizi memorizzano il luogo, ad esempio annotando su foto e video dove sono stati fatti. Questo sistema di annotazione è noto come "geo-tagging".

Un'area dove i telefoni cellulari si stanno diffondendo è l'**m-commerce** (*mobile commerce*) (Senn, 2000). Invece di ricorrere ai contanti o alle carte di credito, sono usati brevi messaggi di testo dal cellulare per autorizzare il pagamento di un prodotto presso distributori automatici di alimenti, biglietti del cinema e altri piccoli oggetti. L'importo viene addebitato sulla bolletta del telefono cellulare che, se equipaggiato di tecnologia **NFC** (*near field communication*), può agire come una smartcard RFID e interagire con lettori posti nelle vicinanze per effettuare il pagamento. La forza propulsiva alla base di questo fenomeno sono i produttori di dispositivi mobili e gli operatori di rete che tentano disperatamente di capire come aggiudicarsi una fetta della torta dell'e-commerce. Dal punto di vista del rivenditore, questo schema può far risparmiare gran parte della commissione, dovuta alla società che gestisce le carte di credito, che può essere di alcuni punti percentuale. Naturalmente, tale schema può ritorcersi contro i promotori, poiché i clienti del negozio potrebbero servirsi dei loro RFID o dei loro lettori di codice a barre per controllare i prezzi dei concorrenti prima dell'acquisto e usare tali informazioni per avere un rapporto dettagliato su dove viene venduto un articolo nelle vicinanze e a che prezzo.

Un enorme punto a favore dell'm-commerce è che gli utenti mobili sono abituati a pagare per qualsiasi cosa (a differenza degli utenti Internet che si aspettano che tutto sia gratis). Se un sito Web in Internet imponesse un sovrapprezzo per consentire ai propri clienti il pagamento tramite carta di credito, gli utenti protesterebbero vivacemente. Se un operatore di telefonia mobile

permettesse di pagare la merce in un esercizio commerciale usando il telefono e per questo servizio pretendesse un compenso, probabilmente ciò sarebbe percepito come un fatto normale. Chi vivrà vedrà.

Non vi è dubbio che l'uso di cellulari e computer portatili crescerà rapidamente in futuro, perché le dimensioni dei computer diminuiscono in modo imprevedibile. Diamo un'occhiata a qualche possibilità. Le **reti di sensori** (*sensor network*) sono fatte di nodi che raccolgono e consegnano in modo wireless il dato che hanno registrato dal mondo fisico che li circonda. I nodi possono far parte di oggetti di uso comune come l'auto o il cellulare o possono essere piccoli dispositivi a sé stanti. Per esempio, la vostra auto può raccogliere i dati su posizione, velocità, vibrazione e consumo di carburante tramite il sistema diagnostico di bordo e aggiornare tali informazioni su un database (Hull et al., 2006). Tali dati potrebbero aiutare a segnalare strade dissestate, pianificare percorsi per evitare ingorghi di traffico e indicare un eccessivo consumo rispetto agli altri automobilisti nello stesso tratto di strada.

Le reti di sensori stanno rivoluzionando la scienza, in quanto sono in grado di fornire un grande quantitativo di dati su comportamenti che prima non potevano essere osservati. Un esempio è il tracciamento della migrazione di singole zebre ottenuto posizionando sugli animali un piccolo sensore (Juang et al., 2002); per farlo, un gruppo di ricerca ha ridotto un computer portatile a un cubo di 1 mm di lato (Warneke et al., 2001). Con computer portatili così minuscoli possono essere tracciati anche piccoli uccelli, roditori e insetti.

Anche usi ordinari, come quello nei parchimetri, possono essere significativi, perché usano dati un tempo non disponibili. I parchimetri possono accettare carte di credito o di debito con verifica istantanea tramite connessione wireless. Possono anche notificare tramite la rete wireless se siano liberi o meno, permettendo agli automobilisti di scaricare mappe aggiornate dei parcheggi per individuare un posto libero. Naturalmente, alla scadenza del tempo stabilito, tali strumenti potrebbero controllare la presenza dell'auto (tramite un segnale) e segnalarlo ai vigili. È stato stimato che le amministrazioni pubbliche nei soli Stati Uniti d'America potrebbero raccogliere in questo modo 10 miliardi di dollari in più (Harte et al., 2000).

I **computer da indossare** (*wearable computer*) sono un'altra promettente applicazione. Orologi intelligenti con funzioni radio fanno parte dell'immaginario collettivo sin dall'epoca in cui apparvero nei fumetti di Dick Tracy nel 1946; ora potete comprarli. Altri dispositivi di questo tipo possono essere innestati nel nostro organismo, come pacemaker e iniettori di insulina. Alcuni sono controllabili tramite una rete wireless, permettendo ai medici di verificarne il funzionamento e di riconfigurarli più facilmente. Tutto ciò potrebbe causare qualche sgradevole inconveniente se i dispositivi fossero poco sicuri come lo sono in media i PC, facili prede di hacker (Halperin et al., 2008).

Risvolti sociali

Come avvenne per la stampa 500 anni fa, le reti di calcolatori permettono ora a chiunque di distribuire e visualizzare contenuti secondo modalità prima non ipotizzabili. Il risvolto negativo di questa conquista sono i molti problemi sociali, politici ed etici ancora irrisolti che ne conseguono.

Social network, forum, siti di condivisione di contenuti e una miriade di altre applicazioni permettono di condividere le proprie opinioni con persone affini. Argomenti tecnici o hobby come il giardinaggio sono relativamente innocui, ma quando si parla dei temi che veramente stanno a cuore alle persone, quali politica, religione e sesso, sorgono grandi problemi. Opinioni espresse pubblicamente in questi spazi possono essere profondamente offensive per qualcuno. Ancor peggio, potrebbero non essere politicamente corrette. Inoltre, i messaggi non si limitano necessariamente al solo testo; immagini a colori ad alta risoluzione e brevi spezzoni video oggi si possono trasmettere facilmente sulle reti. Alcune persone hanno una mentalità tollerante, ma altre ritengono che la divulgazione di certo materiale (per esempio, attacchi a particolari nazioni o religioni, pornografia, e così via) sia semplicemente inaccettabile e vada censurata. Nazioni diverse hanno su questa materia legislazioni dissimili o in conflitto tra loro; per questo motivo il dibattito è molto acceso.

Nel passato, alcuni hanno citato in giudizio gli operatori di rete, sostenendo la loro responsabilità rispetto al contenuto di ciò che trasportano, così come nel caso di quotidiani e riviste. La risposta inevitabile è che la rete è come una compagnia telefonica o un ufficio postale, e non si può pretendere che eserciti un controllo sulle affermazioni dei propri utenti.

Non sorprende però che alcuni operatori di rete, nel proprio interesse, arrivino talvolta a bloccare dei contenuti. Alcuni utenti di applicazioni peer-to-peer si sono visti bloccare il servizio, perché il gestore di rete non ha ritenuto proficuo sostenere un traffico dati così pesante. Questi stessi operatori adotterebbero volentieri una politica differente a seconda del cliente: una grande azienda in grado di pagare un alto costo ottiene un buon servizio, il piccolo utente ne ottiene uno scadente. Gli oppositori di questa pratica sostengono che servizi quali il peer-to-peer e simili vadano trattati come gli altri, perché per la rete si tratta solo di bit. La visione secondo cui la comunicazione debba essere uguale per tutti indipendentemente dal contenuto, dalla sorgente e dal fornitore del contenuto è nota come **neutralità della rete** (*network neutrality*) (Wu, 2003).

Sono molti i soggetti coinvolti in questo dibattito. Per esempio, musica e film piratati hanno alimentato la crescita massiccia di reti peer-to-peer, sgradita ai detentori di diritti d'autore che hanno minacciato (e talvolta intrapreso) azioni legali. Ci sono ora sistemi automatici di indagine nelle reti peer-to-peer che mandano segnali di allarme agli operatori di rete e agli utenti sospettati di violazione di diritti d'autore. Negli Stati Uniti questi avvisi sono noti come **DMCA takedown notices** a seguito del **Digital Millennium Copyright Act**. Rimane tuttavia molto difficile sviluppare sistemi di allarmistica affidabili contro la violazione dei diritti d'autore; anche la vostra stampante potrebbe esserne accusata (Piatek et al., 2008).

Le reti di calcolatori rendono la comunicazione facilmente fruibile, ma altrettanto sorvegliabile. Ciò crea conflitti su questioni come i diritti dei lavoratori contro i diritti dei datori di lavoro. È usanza comune leggere e scrivere e-mail sul luogo di lavoro. Molti datori di lavoro hanno però rivendicato il diritto di leggere ed eventualmente censurare i messaggi dei dipendenti, inclusi quelli inviati da casa al di fuori dell'orario di lavoro. Non tutti i dipendenti accettano questa politica, specialmente per quanto riguarda quest'ultimo divieto.

Un altro conflitto riguarda il rapporto tra governo e diritti dei cittadini. L'FBI ha installato presso molti Internet Service Provider sistemi di rilevamento di particolari contenuti su tutte le e-mail. In origine uno di questi primi sistemi era denominato Carnivore, ma tale nome gli fece una tale cattiva pubblicità da dover essere ribattezzato con il più innocuo DCS1000 (Blaze e Bellovin, 2000;

Sobel, 2001; e Zacks, 2001). L'obiettivo di tali sistemi è quello di spiare milioni di persone, nella speranza di trovare informazioni riguardanti attività illecite. Purtroppo per chi spia, il quarto emendamento della costituzione americana vieta di effettuare ricerche senza un mandato di perquisizione. Ma il governo spesso non ne tiene conto.

Naturalmente, il governo non ha il monopolio della minaccia alla privacy. Il settore privato fa la sua parte quando profila gli utenti. Per esempio, piccoli file chiamati **cookie** che i browser Web memorizzano nei computer degli utenti consentono alle aziende di tenere traccia delle attività degli utenti in rete e possono consentire anche la diffusione sulla rete di numeri di carte di credito, previdenza sociale e altre informazioni (Berghel, 2001). Le aziende che forniscono servizi basati sul Web hanno la possibilità di memorizzare una grande quantità di informazioni personali sui loro utenti, in modo da studiarne le attività. Ad esempio, quando utilizzate il suo servizio di posta elettronica, **Gmail**, Google può leggere la vostra e-mail e mostrarvi annunci pubblicitari sulla base dei vostri interessi personali.

Una nuova svolta nel campo della privacy è stata impressa dalla diffusione dei dispositivi mobili che forniscono anche la posizione degli utenti (Beresford e Stajano, 2003). Mentre erogano servizi agli utenti, gli operatori apprendono la loro posizione nei vari momenti della giornata e sono in grado di ricostruire i loro movimenti, che tipo di locali frequentano o i centri medici a cui si rivolgono.

Le reti di calcolatori offrono anche la possibilità di migliorare la propria privacy, inviando messaggi anonimi. In alcune situazioni, questa capacità può essere auspicabile. Oltre a impedire alle aziende di apprendere le vostre abitudini, dà modo, ad esempio, a studenti, militari, dipendenti e cittadini di denunciare comportamenti scorretti da parte di professori, ufficiali, superiori e politici senza paura di ritorsioni. D'altra parte, negli Stati Uniti e nella maggior parte delle democrazie la legge permette a un cittadino sotto accusa il diritto di affrontare e sfidare il proprio accusatore in tribunale, in modo che le denunce anonime non siano utilizzabili come prova.

Le informazioni possono essere rapidamente reperite in Internet, ma spesso sono ingiustificate, ingannevoli o addirittura sbagliate. Per esempio, un consiglio medico riguardante dolori al torace può essere stato messo on-line da un vincitore di premio Nobel come da qualcuno in possesso del semplice diploma di terza media. Altre informazioni sono spesso sgradite. La posta elettronica indesiderata (*spam*) è diventata parte della nostra vita, perché gli spammer hanno raccolto milioni di indirizzi e-mail utilizzati da aspiranti operatori di marketing a basso costo per inviare messaggi pubblicitari generati automaticamente. Il flusso di messaggi spam è comparabile a quello che riceviamo da mittenti reali. Fortunatamente, i filtri software sono in grado di controllare ed eliminare lo spam più o meno efficacemente.

Altri contenuti sono pensati a scopo criminale. Pagine Web ed e-mail contengono programmi o macro che vanno in esecuzione sul computer del destinatario, quali virus per rubare password di conti bancari o per usare computer infestati come parte di una **botnet:** un gruppo di macchine compromesse al fine di inviare spam.

I messaggi di **phishing** provengono apparentemente da persone o istituzioni degne di fiducia, per esempio una banca, e istigano l'utente a rivelare informazioni riservate quali il numero di carta di credito. Il furto di identità sta diventando un problema serio, in quanto i malintenzionati collezionano tali e tante informazioni riguardo la vittima da poter ottenere carte di credito e altri documenti a suo nome.

Risulta inoltre difficile in Internet distinguere esseri umani da computer. Questa problematica ha portato allo sviluppo di **CAPTCHA**: test in cui un computer chiede a qualcuno di risolvere un breve test di riconoscimento; ad esempio, scrivere le lettere visualizzate in un'immagine distorta, a dimostrazione di essere un essere umano (von Ahn, 2001). Questo processo è una variante del famoso test di Turing in cui una persona pone delle domande attraverso una rete per giudicare se l'entità che risponde è umana.

Molti di questi problemi si potrebbero risolvere se l'industria informatica prendesse seriamente in considerazione l'argomento della sicurezza. Se tutti i messaggi fossero cifrati e autenticati sarebbe più difficile commettere reati. Questa tecnologia è ben sviluppata e la studieremo in dettaglio nel Capitolo 8. Il problema è che i produttori di hardware e software sanno che inserire le funzionalità di sicurezza costa, e che i loro clienti non le richiedono. Inoltre, una buona parte dei problemi è causata da software difettoso che esiste perché i produttori continuano ad aggiungere sempre nuove caratteristiche ai loro programmi, che inevitabilmente si traducono in codice aggiuntivo e quindi in eventuali difetti. Una tassa sulle nuove funzionalità potrebbe aiutare, ma è probabilmente difficile da accettare. Un rimborso per il software difettoso sarebbe gradito, salvo che manderebbe in bancarotta l'intera industria del software in un solo anno.

Le reti di calcolatori suscitano nuovi problemi legali quando interagiscono con vecchie leggi. Un esempio è rappresentato dalle scommesse elettroniche. I computer hanno simulato di tutto per decenni; perché ora non dovrebbero simulare slot machine, ruote della roulette, mazzieri di blackjack e altre attrezzature per giochi d'azzardo? Bene, perché in molti paesi è illegale. Il problema è che in molti altri paesi le scommesse sono invece legali (per esempio in Inghilterra) e i proprietari di casinò hanno colto il loro potenziale. Che cosa succede se lo scommettitore, il casinò e il server si trovano in nazioni diverse con leggi in conflitto tra di loro? Ottima domanda.

1.2 Hardware di rete

È il momento di spostare la nostra attenzione dalle applicazioni e dagli aspetti sociali delle reti (il piacere) ai dettagli tecnici che riguardano il progetto (il dovere). Non esiste una tassonomia universalmente accettata che risulti applicabile a tutte le reti di calcolatori, ma due caratteristiche spiccano per importanza: la tecnologia di trasmissione e la scala. Esamineremo a turno ciascuna di esse.

Parlando in senso lato, ci sono due tipi di tecnologie di trasmissione diffusamente impiegati: i collegamenti **broadcast** e i collegamenti **punto a punto** (*point to point*).

I collegamenti punto a punto connettono coppie di computer. Brevi messaggi, chiamati in alcuni contesti **pacchetti**, possono dover visitare una o più macchine intermedie per spostarsi dalla sorgente alla destinazione in una rete composta di collegamenti punto a punto. Spesso sono possibili più percorsi di diversa lunghezza; quindi, nelle reti punto a punto è importante trovarne di validi. La trasmissione punto a punto con esattamente un trasmettitore e un ricevitore è a volte chiamata **unicast**.

Al contrario, le reti broadcast hanno un solo canale di comunicazione condiviso da tutte le macchine della rete. I pacchetti inviati da qualunque macchina sono ricevuti da tutte le altre. Un campo indirizzo all'interno del pacchetto individua il destinatario. Alla ricezione del pacchetto ogni macchina controlla il campo indirizzo. Se il pacchetto è indirizzato alla macchina ricevente viene processato; se è indirizzato a un'altra macchina viene semplicemente ignorato.

Una rete wireless è un classico esempio di collegamento broadcast in cui la comunicazione è condivisa in una regione di copertura che dipende dal canale wireless e dalla macchina che trasmette. Come analogia consideriamo qualcuno che si trova alla fine di un corridoio con molte stanze e che urla: "Watson esci, ho bisogno di te". Anche se il pacchetto viene ricevuto (ascoltato) da molte persone, solo Watson risponde; gli altri semplicemente lo ignorano.

I sistemi broadcast danno di solito anche la possibilità d'indirizzare un pacchetto a *tutti* i destinatari usando un codice speciale nel campo destinazione. Quando è trasmesso un pacchetto con questo codice, esso viene ricevuto e processato da tutte le macchine sulla rete. Questa modalità operativa è chiamata **broadcast**. Alcuni sistemi broadcast supportano anche la trasmissione a un sottoinsieme delle macchine, a volte chiamata **multicast**.

Un criterio alternativo per classificare le reti è la loro scala. La distanza è una metrica di classificazione importante, perché reti su scale differenti impiegano tecnologie diverse.

Nella Figura 1.6 classifichiamo sistemi composti da più entità di elaborazione sommariamente in base alla loro dimensione fisica. In cima si trovano le *personal area network,* cioè reti pensate per una sola persona. Successivamente si trovano reti con raggio d'azione maggiore, suddivisibili in reti locali, metropolitane e ad ampio raggio, con scala via via crescente.

Distanza tra i processori	Processori situati nello stesso/a	Esempio
1 m	Metro quadrato	Personal area network
10 m	Stanza	Local area network
100 m	Edificio	
1 km	Campus	
10 km	Città	Metropolitan area network
100 km	Nazione	Wide area network
1.000 km	Continente	
10.000 km	Pianeta	Internet

Figura 1.6 Classificazione di processori interconnessi in base alla scala.

Infine, la connessione di due o più reti è chiamata **internetwork** (o *inter-rete*). Internet è certamente l'esempio più noto (ma non il solo) di internetwork. Presto avremo internetwork ancora più grandi con la **Interplanetary Internet** che connetterà reti attraverso lo spazio (Burleigh et al., 2003).

In questo libro ci occuperemo di reti su tutte le scale. Di seguito forniamo una breve introduzione all'hardware di rete in base alla scala.

Personal area network

Le reti **PAN** (*personal area network,* reti personali) permettono ai dispositivi di comunicare nello spazio fisico alla portata di una persona. Un esempio classico sono le reti wireless che connettono un computer alle sue periferiche. Quasi ogni computer ha attaccati monitor, tastiera, mouse e stampante. Senza una wireless la connessione deve essere implementata tramite cavi. Sono così tanti gli utenti a cui risulta difficile trovare il giusto cavo e inserirlo nel buco corrispondente (anche se solitamente sono codificati tramite colori), che la maggior parte dei rivenditori di computer offre la possibilità di ricevere a casa propria un tecnico che esegua questo lavoro. Per venire loro incontro, alcune aziende si sono unite per costruire una rete wireless a corto raggio, chiamata **Bluetooth**, che connetta queste componenti senza bisogno di cavi. Scopo del progetto è che i dispositivi equipaggiati con Bluetooth possano fare a meno di cavi. Basta sistemare i dispositivi, accenderli ed essi lavoreranno insieme. Per molti questa facilitazione rappresenta un notevole vantaggio.

Nella loro forma più semplice le reti Bluetooth adottano il paradigma master-slave (*padrone-schiavo*) della Figura 1.7. L'unità di sistema (il PC) è normalmente il master che comunica col mouse, la tastiera e gli altri dispositivi che agiscono da slave. Il master comunica agli slave quali indirizzi usare, quando possano effettuare una trasmissione broadcast, per quanto tempo, quali frequenze usare, e così via.

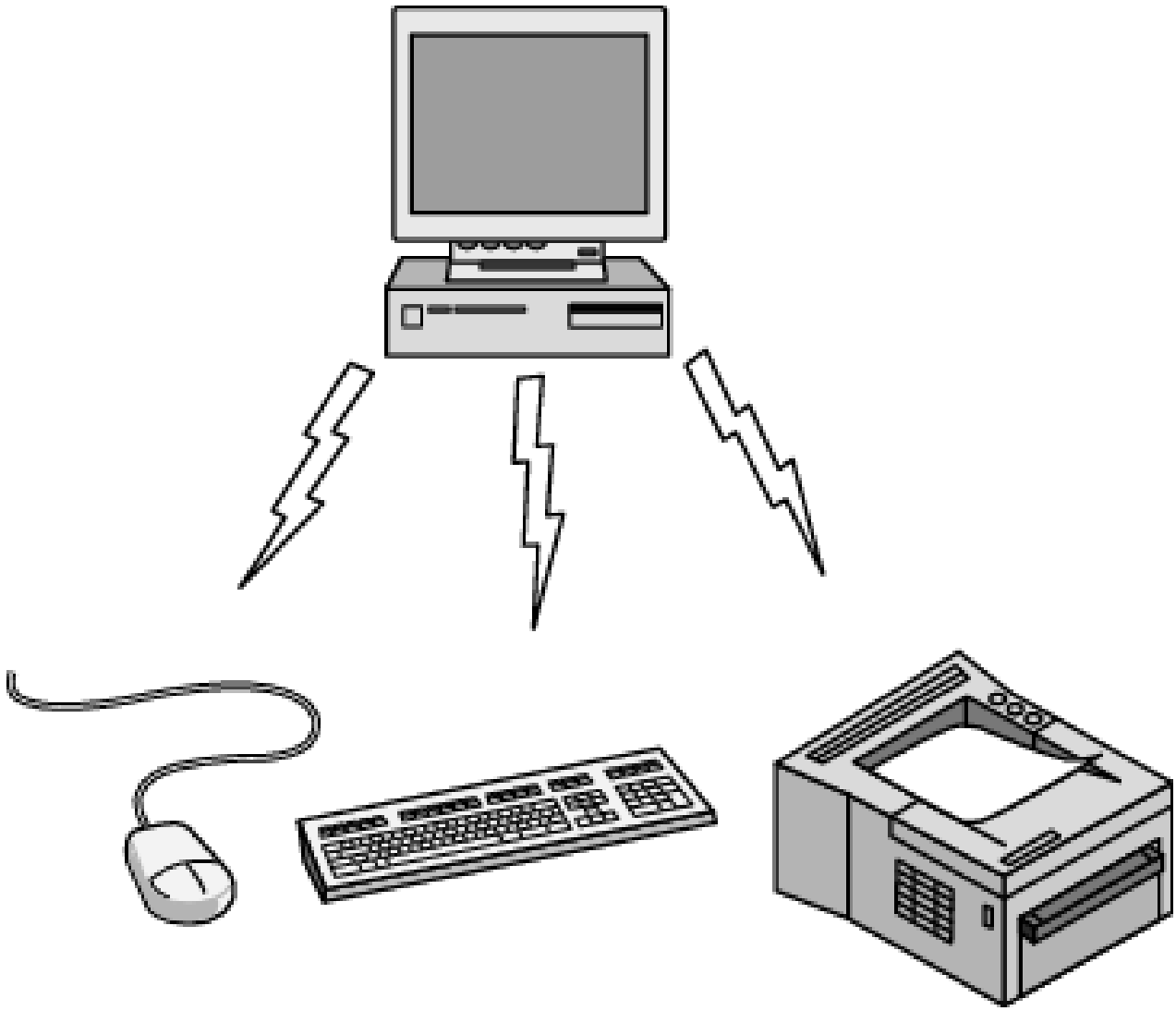


Figura 1.7 Configurazione di una rete PAN con Bluetooth.

Il Bluetooth può essere usato anche in altri contesti. È spesso usato per collegare un auricolare a un telefono cellulare senza bisogno di fili e permette al vostro lettore di musica digitale di connettersi alla vostra automobile semplicemente portandolo abbastanza vicino. Un tipo di rete PAN completamente differente è quella che si instaura quando un dispositivo medico come un pacemaker, un dispensatore di insulina o un apparecchio acustico si collegano a un sistema di controllo. Bluetooth sarà trattato in dettaglio nel Capitolo 4.

Le reti PAN possono anche essere implementate con altre tecnologie in grado di comunicare a corto raggio, come gli RFID nelle smartcard e nei volumi di una biblioteca. Gli RFID saranno oggetto di studio nel Capitolo 4.

Local area network

Il passo successivo sono le **LAN** (*local area network*, reti locali). Una rete LAN è una rete privata che opera all'interno o nelle vicinanze di un singolo edificio come un appartamento, un ufficio o una fabbrica. Le reti LAN sono ampiamente utilizzate per connettere personal computer e altri dispositivi elettronici in modo che questi condividano risorse (per esempio stampanti) e scambino informazioni. Una LAN aziendale viene chiamata **enterprise network** (*rete aziendale*).

Le reti LAN wireless sono attualmente molto diffuse, specialmente nelle case, nei vecchi uffici, nei locali e in tutti quei luoghi dove sarebbe troppo complicato installare cavi. In questi sistemi ogni computer ha un ricevitore radio e un'antenna per comunicare con altri computer. Nella maggior parte dei casi ogni computer comunica con un dispositivo posto sul soffitto come mostrato nella Figura 1.8(a). Questo dispositivo, chiamato **AP** (*access point*), **router wireless**, o **base station** (*stazione base*) distribuisce i pacchetti tra computer collegati in modo wireless e anche tra questi e Internet. Essere un AP è come essere il bambino più popolare della scuola, perché tutti vogliono parlare con lui. Tuttavia, se gli altri computer sono abbastanza vicini tra loro, possono comunicare direttamente in una configurazione peer-to-peer.

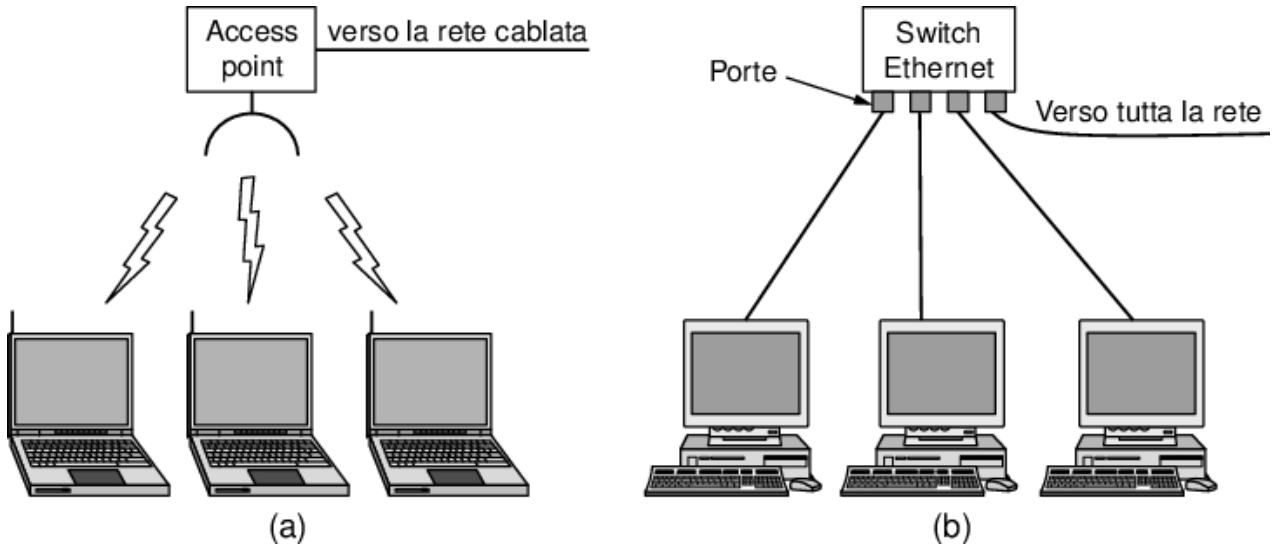


Figura 1.8 Reti LAN wireless e cablate. (a) 802.11. (b) Switched Ethernet.

Lo standard per le reti LAN wireless è **IEEE 802.11**, comunemente noto come **Wi-Fi** e attualmente molto diffuso. Funziona a una velocità che va da undici a centinaia di Mbps. In questo libro ci atterremo alla tradizione e misureremo la velocità di una linea in megabit/s, dove 1 Mbps

corrisponde a 1.000.000 bit/s, e gigabit/s, dove 1 Gbps corrisponde a 1.000.000.000 bit/s. Discuteremo lo standard 802.11 nel Capitolo 4.

Le reti LAN cablate usano diverse tecnologie di trasmissione. La maggior parte utilizza cavi di rame, alcune invece fibra ottica. Le LAN hanno dimensioni contenute per cui il tempo di trasmissione nel caso peggiore è limitato e noto a priori. La progettazione dei protocolli di rete viene semplificata dalla conoscenza di questi limiti. Tipicamente le LAN cablate operano a velocità che vanno dai 100 Mbps a 1 Gbps, hanno bassi tempi di ritardo (microsecondi o nanosecondi) e commettono veramente pochi errori di trasmissione. Una rete LAN non può mai operare a più di 10 Gbps. Le reti LAN cablate superano in ogni indice di prestazione le reti wireless, semplicemente perché è più facile trasmettere un segnale via cavo o fibra piuttosto che via etere.

La topologia di molte reti cablate LAN è formata da connessioni punto a punto. Il protocollo IEEE 802.3, comunemente chiamato **Ethernet**, è il tipo di LAN di gran lunga più diffuso. La Figura 1.8(b) mostra un esempio di topologia **switched Ethernet.** Ogni computer capisce il protocollo Ethernet e si connette a un dispositivo, chamato **switch** (*permutatore*), con una connessione punto a punto. Da qui il nome. Uno switch ha molte **porte**, a ognuna delle quali può connettersi a un computer. Il compito dello switch è di smistare i pacchetti tra i computer a esso connessi, selezionando il computer destinatario in base all'indirizzo scritto nel pacchetto stesso.

Per costruire reti LAN più grandi è possibile collegare tra loro più switch attraverso le porte. Che cosa succede se gli switch vengono connessi insieme a formare un circolo? La rete funzionerà ancora? Per fortuna i progettisti ci hanno pensato. È compito del protocollo decidere il percorso che il pacchetto deve compiere per raggiungere correttamente il computer destinatario. Nel Capitolo 4 ne vedremo il funzionamento.

È anche possibile suddividere una grande rete LAN fisica in due reti LAN logiche più piccole. Ci si potrebbe chiedere l'utilità di questo procedimento. Può accadere che il progetto della rete non combaci con la struttura dell'organizzazione. Per esempio, i reparti di ingegneria e di economia di un'azienda potrebbero disporre di computer connessi sulla stessa LAN fisica perché situati nella stessa ala di un edificio; tuttavia, se ognuno dei due avesse una propria rete logica, chiamata **virtual LAN** o **VLAN**, amministare il sistema potrebbe essere più semplice. In questi progetti ogni porta è contrassegnata da un colore, per esempio verde per il reparto di ingegneria e rosso per quello di economia. Lo switch inoltra i pacchetti in modo che i computer collegati alle porte verdi siano separati dai computer collegati alle porte rosse. I pacchetti broadcast inviati tramite una porta rossa, per esempio, non possono essere ricevuti attraverso una porta verde, esattamente come se ci fossero due reti LAN distinte. Ci occuperemo delle reti VLAN alla fine del Capitolo 4.

Esistono altre tipologie di reti LAN cablate. Infatti, la Ethernet commutata è una variante moderna della versione originale di Ethernet che faceva il broadcast di tutti pacchetti su un singolo cavo. A un dato tempo una sola macchina riusciva a trasmettere il pacchetto e i conflitti venivano risolti da un meccanismo di arbitraggio distribuito. Veniva implementato un semplice algoritmo: un computer poteva trasmettere ogni volta che il cavo era libero. Se uno o più pacchetti collidevano, ogni computer si limitava ad aspettare un tempo casuale prima di riprovare. Chiameremo questa versione **Ethernet classica** per chiarezza e, come ormai vi aspettate, ne parleremo nel Capitolo 4.

Le reti broadcast sia wireless sia cablate si possono ulteriormente dividere in statiche e dinamiche, a seconda del modo in cui è allocato il canale. Una tipica allocazione statica consiste nel suddividere il tempo in intervalli discreti e usare un algoritmo *round-robin,* permettendo a ogni macchina di eseguire il broadcast solo quando è attivo il proprio turno (*time slot*). L'allocazione statica spreca la capacità del canale se una macchina non ha nulla da trasmettere durante il proprio slot temporale; per questo motivo la maggioranza dei sistemi tenta di allocare il canale dinamicamente (cioè su richiesta).

I metodi di allocazione dinamica per un canale condiviso possono essere centralizzati o non centralizzati. Nel metodo di allocazione centralizzato esiste una singola entità, per esempio la stazione base nelle reti cellulari, che stabilisce a chi spetta di volta in volta l'uso del mezzo. Può farlo accettando richieste e prendendo decisioni in base ad algoritmi interni. Nel metodo di allocazione non centralizzato non esiste un'entità centrale; ogni macchina deve decidere in autonomia se trasmettere o meno. Si può pensare che ciò porti sempre al caos, ma non è così, e lo vedremo.

Vale la pena spendere ancora un po' di tempo da dedicare alle LAN domestiche. In futuro è ragionevole pensare che ogni elettrodomestico sarà in grado di comunicare con gli altri apparecchi e tutti saranno accessibili tramite Internet. Questo sviluppo sembra essere una di quelle idee visionare di cui nessuno sente la necessità (come il telecomando per la televisione o i cellulari), ma di cui, una volta provate, non si riesce più a fare a meno.

Molti dispositivi sono già in grado di essere connessi a una rete: computer, dispositivi per l'intrattenimento come la televisione e i DVD, telefoni e altri apparecchi elettronici come le macchine fotografiche, elettrodomestici come le radiosveglie e parti di infrastrutture come contatori e termostati. Questa tendenza può solo crescere. Per esempio, in una casa ci sono mediamente una dozzina di orologi (all'interno di elettrodomestici), ognuno dei quali si potrebbe regolare automaticamente sull'ora legale, qualora fosse connesso a Internet. Il monitoraggio da remoto di una casa sembra essere un'idea vincente, in quanto molti bambini di adesso, una volta cresciuti, desidereranno investire parte dei loro guadagni per aiutare i propri genitori a vivere in modo più sicuro nelle loro case.

Sebbene potremmo pensare a una rete domestica come a un tipo di LAN, essa ha proprietà differenti dalle altre reti. In primo luogo i dispositivi in rete devono essere molto facili da installare. I router wireless sono i dispositivi che più di tutti gli altri vengono resi al negozio dopo l'acquisto. La gente li compra perché vuole a casa la rete wireless, scopre che non funzionano e li restituisce piuttosto che passare il tempo al telefono aspettando di parlare con il supporto tecnico.

In secondo luogo, la rete e i dispositivi devono essere semplici da usare. I condizionatori d'aria una volta avevano una manopola con quattro alternative: "spento", "basso", "medio" e "alto". Ora sono corredati da un manuale di 30 pagine. Una volta messi in rete, il solo capitolo sulla sicurezza sarà lungo 30 pagine. Questo è un problema, perché chi lavora con i computer è abituato ad avere a che fare con prodotti che non funzionano; ma il pubblico abituato a comprare macchine, televisori e condizionatori è molto meno tollerante. Si aspetta di comprare prodotti in grado di funzionare al 100% senza bisogno di ricorrerre a un esperto di informatica.

In terzo luogo, il prezzo. La gente non è disposta a pagare un sovrapprezzo di 50 dollari per un termostato connesso a Internet, in quanto pochi ritengono importante tenere sotto controllo la temperatura di casa dal posto di lavoro, ma se il sovrapprezzo fosse limitato a soli 5 dollari forse si convincerebbero all'acquisto.

In quarto luogo, deve essere possibile acquistare uno o due dispositivi ed espandere la rete gradualmente. Questo implica non avere problemi di formati. Invitare i consumatori all'acquisto di periferiche con interfaccia IEEE 1394 (FireWire) e dopo pochi anni ritrattare quanto detto e comunicare che l'interfaccia del mese è USB 2.0 e quindi cambiare ancora in 802.11, anzi facciamo 802.11n, o volevamo dire 802.16 (reti wireless differenti) li farebbe decisamente adirare. L'interfaccia di rete dovrà rimanere stabile per decenni, come avviene per gli standard delle trasmissioni televisive.

Quinto, la sicurezza e l'affidabilità sono essenziali. Perdere qualche file a causa di un virus nella e-mail è una cosa, avere un topo d'appartamento che disinnesca il vostro sistema di sicurezza da un computer portatile e quindi vi svaligia la casa è qualcosa di completamente diverso.

Un aspetto interessante è se le reti domestiche saranno wireless o cablate. La convenienza e i costi favoriscono le reti wireless, poiché non hanno canaline da aggiungere o, peggio, da adattare a posteriori. La sicurezza favorisce le reti cablate, poiché le onde radio usate dalle reti wireless attraversano facilmente i muri. Non tutti apprezzerebbero che i propri vicini di casa usassero la loro connessione Internet e leggessero le loro e-mail.

Un'ulteriore opzione che potrebbe essere apprezzata è quella di ricorrere alle reti che già esistono negli appartamenti. Il candidato ovvio sono i cavi elettrici già installati in casa. La **rete elettrica** permette ai dispositivi a essa connessi di fare il broadcast delle informazioni attraverso la casa. Dovendo comunque attaccare la televisione alla presa elettrica, tanto vale connetterla anche a Internet. La difficoltà consiste nel trovare un modo per trasportare corrente elettrica e dati nello stesso tempo. Parte della risposta potrebbe essere quella di usare differenti bande di frequenza.

Per concludere, le LAN domestiche offrono molte opportunità e molte sfide. Queste ultime sono legate soprattutto alla necessità che le reti siano facili da maneggiare, affidabili e sicure specialmente nelle mani di utenti non esperti, e contemporaneamente a basso costo.

Metropolitan area network

Una **MAN** (*metropolitan area network*, rete metropolitana) copre un'intera città. L'esempio più noto di MAN è la rete di TV via cavo disponibile in molte città degli Stati Uniti. Questo sistema si è sviluppato partendo dagli originari sistemi di antenna comunitari, installati nelle zone con cattiva ricezione della televisione via etere. In questi sistemi primitivi veniva installata una grande antenna sulla cima di una collina nelle vicinanze e da qui il segnale era trasmesso alle abitazioni.

All'inizio, questi sistemi erano progettati localmente e costruiti in base alle necessità della singola situazione. In seguito, alcune aziende hanno iniziato a entrare nel mercato, ottenendo contratti dalle amministrazioni locali per cablare intere città. Il passo successivo è stata la realizzazione di programmi televisivi e interi canali appositamente pensati per i clienti della TV via cavo. Spesso

questi canali erano altamente specializzati: solo notiziari, solo ricette, solo sport, solo giardinaggio e così via. Dagli inizi fino ai tardi anni '90 furono tuttavia concepiti solo per la ricezione della televisione.

Quando Internet è diventato un fenomeno di massa, gli operatori delle TV via cavo hanno capito che con alcuni cambiamenti al sistema potevano offrire un servizio Internet bidirezionale in porzioni inutilizzate dello spettro. A quel punto il sistema di TV via cavo ha iniziato a trasformarsi da un mezzo per distribuire il segnale televisivo a una MAN. Una MAN assomiglia approssimativamente al sistema mostrato nella Figura 1.9: i segnali televisivi e Internet sono inseriti nella **stazione di testa** (*cable headend*) centralizzata, per la successiva distribuzione nelle abitazioni. Torneremo in dettaglio su questo argomento nel Capitolo 2.

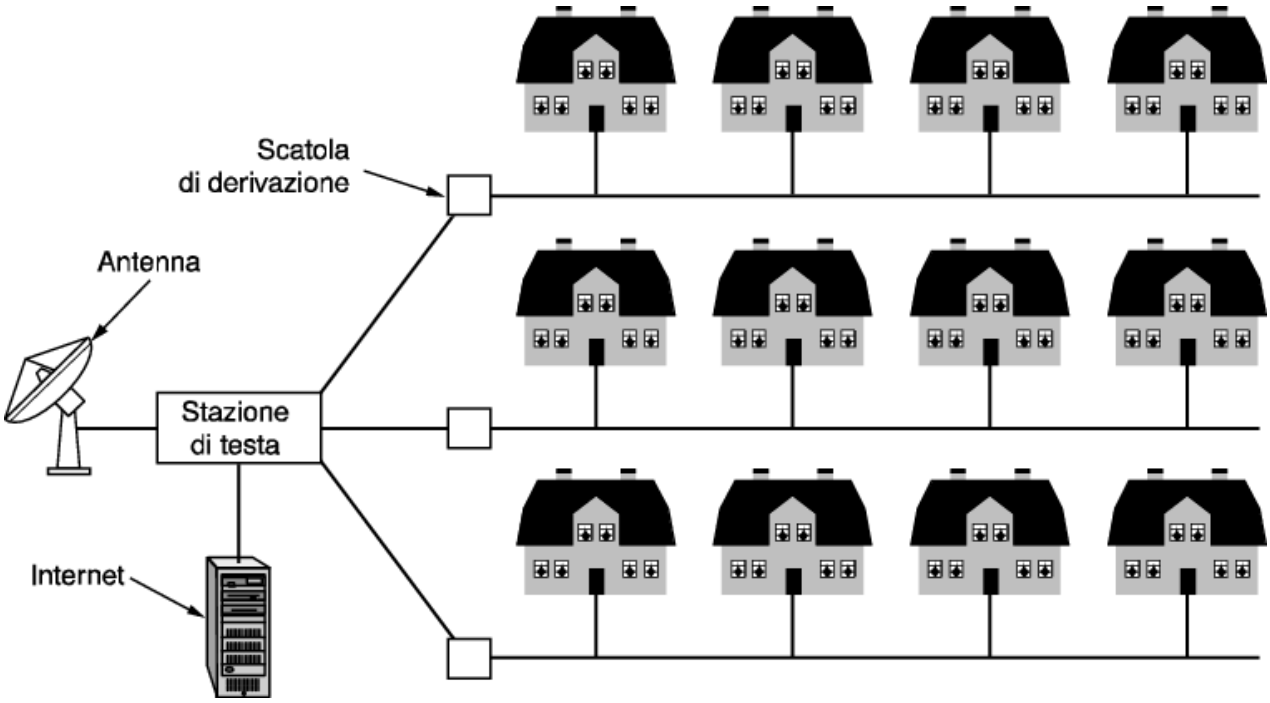


Figura 1.9 Una MAN basata sulla TV via cavo.

La televisione via cavo non è l'unica MAN. Recenti sviluppi nel collegamento wireless a Internet ad alta velocità hanno prodotto un'altra MAN, standardizzata con la sigla IEEE 802.16 e comunemente nota come **WiMAX**. Vedremo anche questo argomento nel Capitolo 4.

Wide area network

Una **WAN** (*wide area network*, rete geografica) copre un'area geograficamente estesa, spesso una nazione o un continente. Cominceremo la nostra analisi da una WAN cablata, usando come esempio un'azienda con uffici sparsi in varie città.

La WAN rappresentata nella Figura 1.10 è una rete che connette uffici nelle città di Perth, Melbourne e Brisbane. Ogni ufficio dispone di computer su cui girano programmi (applicazioni) degli utenti. Seguendo la tradizione chiameremo queste macchine **host** (letteralmente "ospiti", in quanto ospitano software e utenti). La parte restante della rete che connette questi host è quindi chiamata la **sottorete di comunicazione** o in breve **sottorete** (*subnet*). Il compito di una sottorete è quello di trasportare messaggi da un host all'altro, esattamente come il sistema telefonico trasporta le parole (in realtà suoni) da chi parla a chi ascolta.

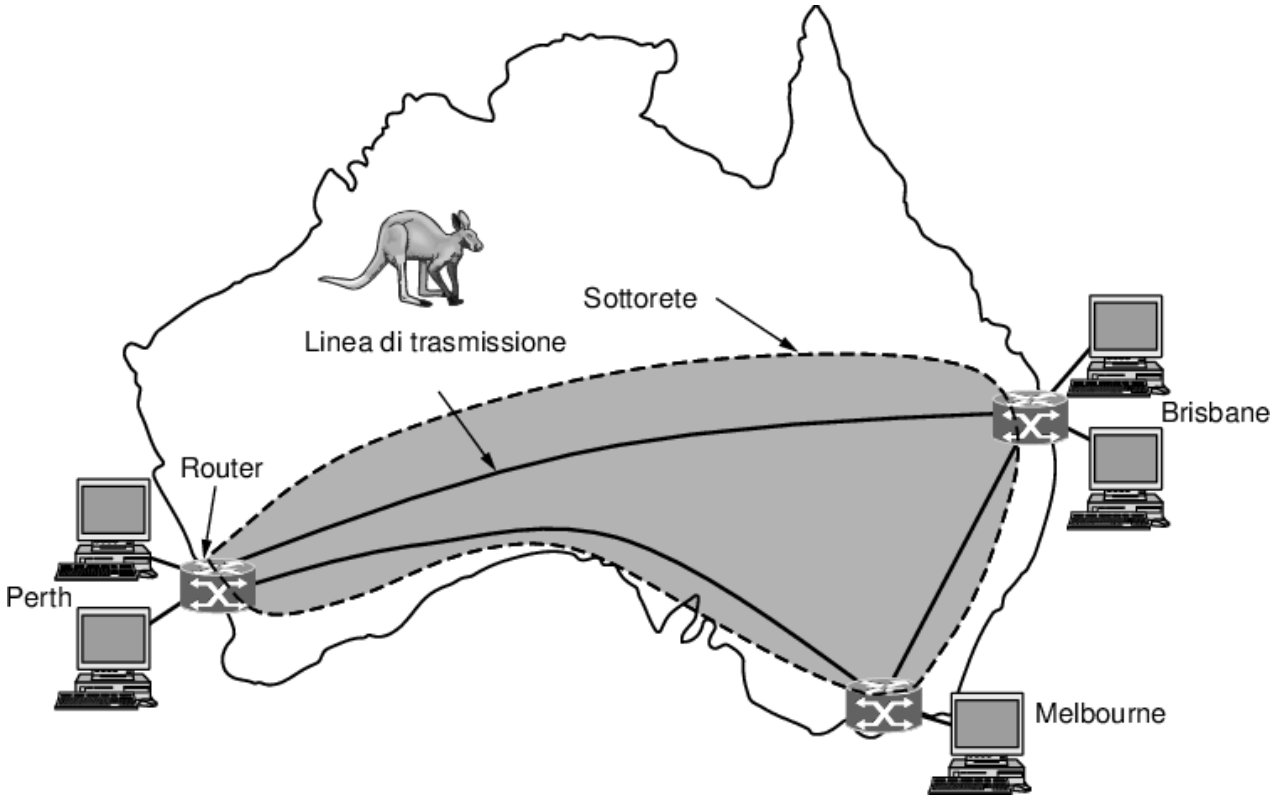


Figura 1.10 Rete WAN che connette tre sedi di una azienda in Australia.

Nella maggior parte delle WAN, la sottorete è formata da due componenti: linee di trasmissione ed elementi di commutazione. Le linee di trasmissione spostano i bit tra le macchine. Possono essere realizzate con cavo in rame, fibra ottica o anche collegamenti radio. Molte aziende non hanno linee di trasmissione proprie, ma le affittano da aziende di telecomunicazione. Gli **elementi di commutazione**, o semplicemente i **commutatori** (*switch*) sono computer specializzati che collegano due o più linee di trasmissione. Quando i dati arrivano da una linea ricevente, l'elemento di commutazione deve scegliere una linea di uscita su cui inoltrarli. In passato, questi computer che commutavano i dati sono stati chiamati in molti modi; oggi il più diffuso è il termine **router**.

È necessario un breve chiarimento sul termine sottorete o subnet. In origine, il solo significato era quello che indicava l'insieme di router e linee di comunicazione che spostano i dati dall'host sorgente a quello destinatario. Il lettore tenga presente che questo termine ha acquisito un secondo

e più moderno significato abbinato all'indirizzo di rete. Discuteremo questo significato nel Capitolo 5 attenendoci per adesso al significato originale (una collezione di linee e router).

Le WAN, come le abbiamo descritte, sembrano simili a una LAN cablata su larga scala, ma vi sono importanti differenze che vanno al di là della lunghezza dei cavi. Usualmente, in una WAN gli host e la sottorete hanno proprietari e operatori diversi. Per esempio, gli impiegati sono responsabili dei loro computer, mentre il reparto IT dell'azienda si incarica di gestire il resto della rete. Nei prossimi esempi i confini saranno marcati in maniera più netta, perché un fornitore di servizi di rete (*network provider*) o una compagnia telefonica saranno i gestori della sottorete. La separazione tra gli aspetti puramente di comunicazione della rete (la sottorete) e le applicazioni (host) semplifica di molto l'intera struttura di rete.

Una seconda differenza consiste nel fatto che i router connettono usualmente reti diverse dal punto di vista tecnologico. Le reti all'interno di uffici usano switch Ethernet, per esempio, mentre le linee di trasmissione su lunga distanza implementano connessioni SONET (ne parleremo nel Capitolo 2). Sono necessari dispositivi che uniscano questi tipi diversi di rete. Il lettore attento avrà notato che quanto appena affermato va oltre la definizione che abbiamo già dato di rete. Questo significa che in realtà molte WAN sono reti composite formate da più di una rete, e quindi delle internetwork.

L'ultima differenza concerne ciò che si connette alla sottorete. Potrebbero essere singoli computer, come quando ci si connette a una rete LAN, o potrebbe essere un'intera LAN. In questo modo si costituiscono reti più grandi a partire da reti più piccole. Dal punto di vista della sottorete il suo funzionamento non cambia.

Siamo ora in grado di esaminare altri due tipi di WAN. In primo luogo un'azienda, piuttosto che affittare linee di trasmissione dedicate, potrebbe connettere i propri uffici a Internet, permettendo così di effettuare le connessioni tra gli uffici come collegamenti virtuali. Questa pianificazione di rete, mostrata nella Figura 1.11, è chiamata **VPN** (*virtual private network*, rete privata virtuale). In confronto a una linea dedicata, una VPN presenta il vantaggio, tipico delle virtualizzazioni, di fornire un riutilizzo flessibile di una risorsa (in questo caso, la connettività verso Internet). Si consideri com'è semplice, in queste condizioni, aggiungere un quarto ufficio. Una VPN presenta anche gli svantaggi tipici di una virtualizzazione quale la mancanza di controllo sulle risorse utilizzate. Con una linea dedicata la capacità è nota; con una VPN le prestazioni possono variare a seconda del servizio Internet utilizzato.

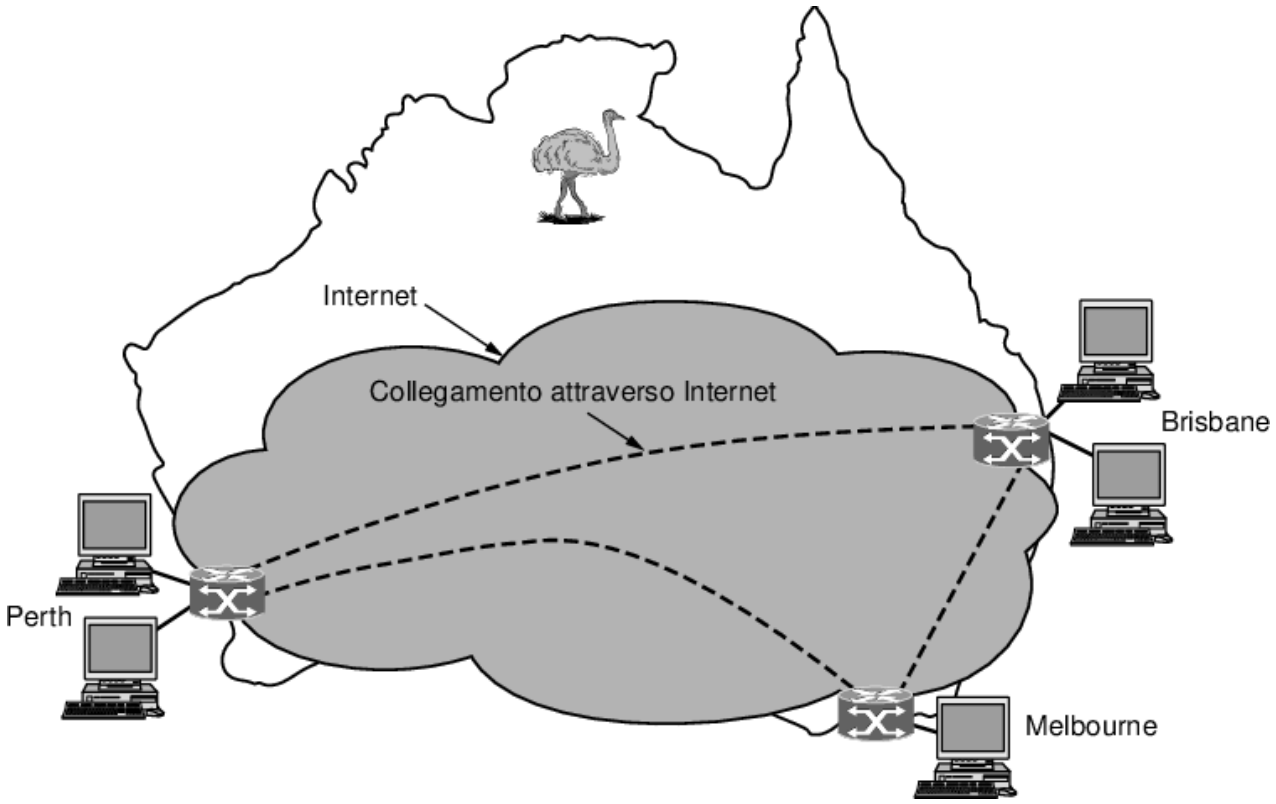


Figura 1.11 WAN facente uso di una rete privata virtuale.

La seconda variante è che la sottorete può essere utilizzata da diverse aziende. L'operatore della sottorete è chiamato **fornitore (o provider) di servizi di rete** e gli uffici sono suoi clienti. Questa struttura è mostrata nella Figura 1.12. L'operatore della sottorete può collegarla anche ad altri clienti, se pagano, e fornire loro servizi. Se i clienti potessero solo trasmettersi pacchetti a vicenda, l'operatore della sottorete offrirebbe un servizio scadente; quindi, deve connettere alla propria sottorete anche altre reti che siano parti di Internet. Un tale operatore di sottorete è chiamato un **ISP** (*internet service provider*) e la sottorete prende il nome di **rete ISP**; i clienti che si connettono a un ISP ricevono un servizio di connessione a Internet.

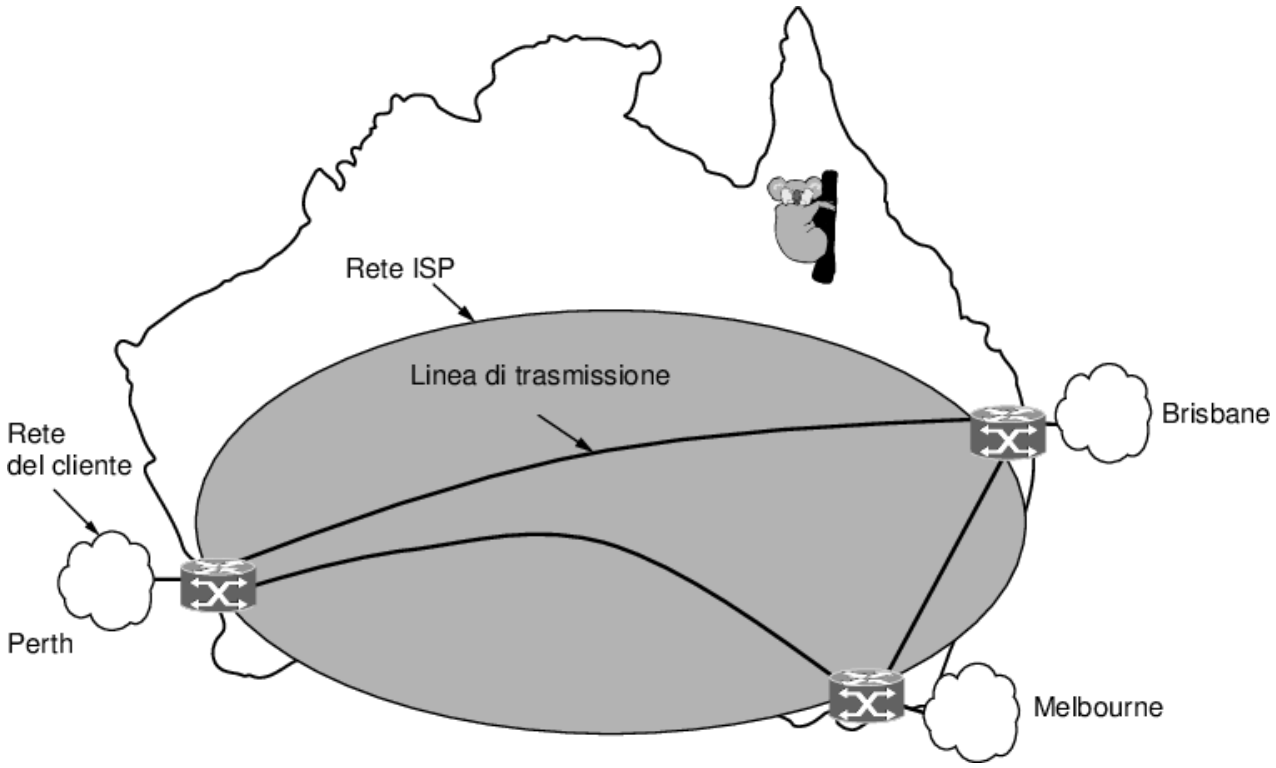


Figura 1.12 WAN facente uso di una rete ISP.

Possiamo usare le reti ISP per anticipare alcuni concetti chiave che verranno studiati nei capitoli seguenti. Nella maggior parte delle WAN la rete contiene molte linee di trasmissione, ognuna delle quali connette una coppia di router. Se due router non appartenenti alla stessa linea di trasmissione vogliono comunicare, devono farlo per via indiretta attraverso altri router. Possono esistere più percorsi nella rete che connettono questi due router. La strategia adottata dalla rete per decidere quale percorso intraprendere prende il nome di **algoritmo di instradamento** (o **algoritmo di routing**); esistono molti di questi algoritmi. La strategia adottata localmente da ogni router per decidere dove inoltrare un pacchetto, per il passo successivo lungo il percorso, è chiamata **algoritmo di inoltro** (o **algoritmo di forwarding**); ne esistono molti anche di questi. Nel Capitolo 5 studieremo alcuni algoritmi di entrambi i tipi in dettaglio.

Altri tipi di reti WAN fanno pesantemente uso di tecnologie wireless. Nei sistemi satellitari ogni computer a terra ha un'antenna attraverso la quale può trasmettere e ricevere dati da un satellite in orbita. Tutti i computer possono ascoltare i messaggi provenienti dal satellite e in alcuni casi possono anche ascoltare le trasmissioni trasmesse da terra da altri computer verso il satellite. Le reti satellitari sono intrinsecamente broadcast e si rivelano particolarmente utili quando questa proprietà è rilevante per la comunicazione.

Le rete telefonica cellulare è un altro esempio di WAN che usa la tecnologia wireless. Questo sistema ha attraversato tre generazioni e una quarta è già all'orizzonte. La prima generazione era analogica e trasmetteva solo la voce. La seconda generazione era digitale e sempre solo per la voce. La terza generazione è digitale e funziona sia per la voce che per i dati. Ogni stazione base cellulare copre una distanza molto più grande di una rete LAN wireless, con un lunghezza misurata in chilometri piuttosto che in decine di metri. Queste stazioni base sono collegate tra di loro da una

rete di backbone (dorsale) usualmente cablata. Il tasso di trasmissione dati delle reti cellulari è spesso dell'ordine di un Mbps, molto inferiore a quello delle LAN wireless che raggiunge i 100 Mbps. Tratterremo a lungo queste reti nel Capitolo 2.

Le internetwork

Nel mondo esistono molte reti, con hardware e software diversi. La gente che si collega a una rete spesso vuole comunicare con persone collegate a una rete differente. La realizzazione di questo desiderio obbliga all'interconnessione di reti diverse e spesso incompatibili tra loro. Un insieme di reti interconnesse si chiama **internetwork** o **internet** (con la i minuscola). Questi termini sono usati nel loro significato generico, in contrasto con la Internet mondiale (che è una specifica internet) che indicheremo sempre con l'iniziale maiuscola. La Internet mondiale usa le reti ISP per connettere reti aziendali, reti domestiche e molte altre reti.

Tra sottoreti, reti e internetwork si fa spesso confusione. Sottorete è un termine particolarmente appropriato al contesto delle *wide area network*, dove si riferisce all'insieme di router e linee di comunicazione possedute dall'operatore di rete. Per analogia, il sistema telefonico è composto da centrali di commutazione connesse una all'altra da linee ad alta velocità e collegate alle abitazioni e uffici da linee a bassa velocità. Linee e apparati, posseduti e gestiti dalla società telefonica, formano le sottoreti della rete telefonica. I telefoni (che in questa analogia sono gli host) non fanno parte delle sottoreti.

Una rete è composta dalla combinazione di una sottorete e dei suoi host. La parola "rete" è tuttavia spesso usata in senso lato. Una sottorete potrebbe essere descritta come una rete, come nel caso della "rete ISP" della Figura 1.12, ma anche una internetwork potrebbe essere descritta come una rete, come nel caso della WAN mostrata nella Figura 1.10. Seguiremo notazioni simili a quelle appena date e, quando dovremo distinguere una rete da un'altra infrastruttura, ci atterremo alla nostra definizione originale di "collezione di computer interconnessi da una singola tecnologia".

Diciamo ora qualcosa in più sui componenti di una internetwork. Sappiamo che una internetwork è formata da reti interconnesse. Nella nostra visione connettere una LAN e una WAN o connettere due LAN è il modo abituale di costituire una internetwork, ma non c'è un accordo definitivo tra le aziende sulla terminologia da adottare in questo campo. Ci sono due regole empiriche che possono risultare utili. Primo, se diversi enti hanno pagato per costruire differenti parti di una rete e ognuno mantiene la propria parte, parliamo di una internetwork piuttosto che di una singola rete. Secondo, se differenti parti della rete implementano differenti tecnologie (per esempio broadcast rispetto a punto a punto e cablata rispetto a wireless) probabilmente abbiamo una internetwork.

Per approfondire la questione dobbiamo parlare di come possono essere connesse due reti diverse. Il nome generale di un dispositivo che effettua la connessione tra due o più reti e fornisce l'adattamento necessario in termini sia di hardware che di software è **gateway**. I gateway si distinguono per il livello a cui operano nella gerarchia dei protocolli. Discuteremo più in dettaglio dei livelli e della gerarchia dei protocolli a partire dal paragrafo successivo, ma per adesso immaginiamo che i livelli più alti siano più legati alle applicazioni, come il Web, mentre i livelli più bassi siano più collegati ai link (collegamenti) di trasmissione, come Ethernet.

Poiché il vantaggio di formare una internet è quello di connettere computer attraverso le reti, non vogliamo utilizzare gateway di basso livello, altrimenti non saremo in grado di connettere tipologie di reti diverse. Non vogliamo però nemmeno usare il gateway a livello troppo alto, altrimenti la connessione funzionerà solo per particolari applicazioni. Il livello intermedio, il giusto livello, è spesso chiamato “livello di rete”, un router è un gateway che lavora a livello di rete. Possiamo quindi individuare una internetwork quando siamo di fronte a una rete che possiede dei router.

1.3 Software di rete

Le prime reti di calcolatori furono progettate con particolare attenzione all'hardware, mentre il software era concepito come un complemento. Questa strategia non è più attuale. Oggi il software di rete è altamente strutturato; nei paragrafi seguenti si esaminerà qualche dettaglio delle tecniche di strutturazione del software. I metodi descritti rappresentano la parte cruciale dell'intero libro e saranno rivisti più volte nel seguito.

Gerarchie dei protocolli

Per diminuirne la complessità, la maggior parte delle reti è organizzata come una pila di **livelli** (*layer*) o **strati**, costruiti uno sull'altro. Numero, nome, contenuto e funzione di ogni livello variano da rete a rete. Lo scopo di ogni livello è quello di offrire determinati servizi ai livelli superiori, schermandoli dai dettagli implementativi. In un certo senso, ogni livello è una specie di macchina virtuale che offre servizi a quello che si trova al di sopra.

Questo concetto è familiare e impiegato in tutta l'informatica, nella quale viene variamente definito come *information hiding*, tipi di dati astratti, incapsulazione dei dati e programmazione orientata agli oggetti. L'idea di base è che una particolare porzione di software (o hardware) offra un servizio ai propri utenti nascondendo però i dettagli dello stato interno e gli algoritmi utilizzati.

Quando il livello *n* all'interno di un computer è in comunicazione con il livello *n* di un altro computer, le regole e le convenzioni usate in questa comunicazione sono globalmente note come protocolli del livello *n*. Fondamentalmente, un **protocollo** è un accordo tra le parti che comunicano sul modo in cui deve procedere la comunicazione. Per analogia, quando una donna viene presentata a un uomo può decidere di porgergli o meno la mano. Egli, a sua volta, può decidere di stringerle la mano o di baciarla, a seconda (per esempio) del fatto che lei sia un avvocato americano in una riunione d'affari o una principessa europea a un ballo ufficiale. Violare il protocollo rende la comunicazione più difficile, se non del tutto impossibile.

Nella Figura 1.13 è illustrata una rete a cinque livelli. Le entità che formano i livelli di pari grado sui diversi computer sono chiamati **peer** (*pari*). I peer possono essere processi software, dispositivi hardware o anche esseri umani. In altri termini, sono i peer a comunicare tra loro usando il protocollo.

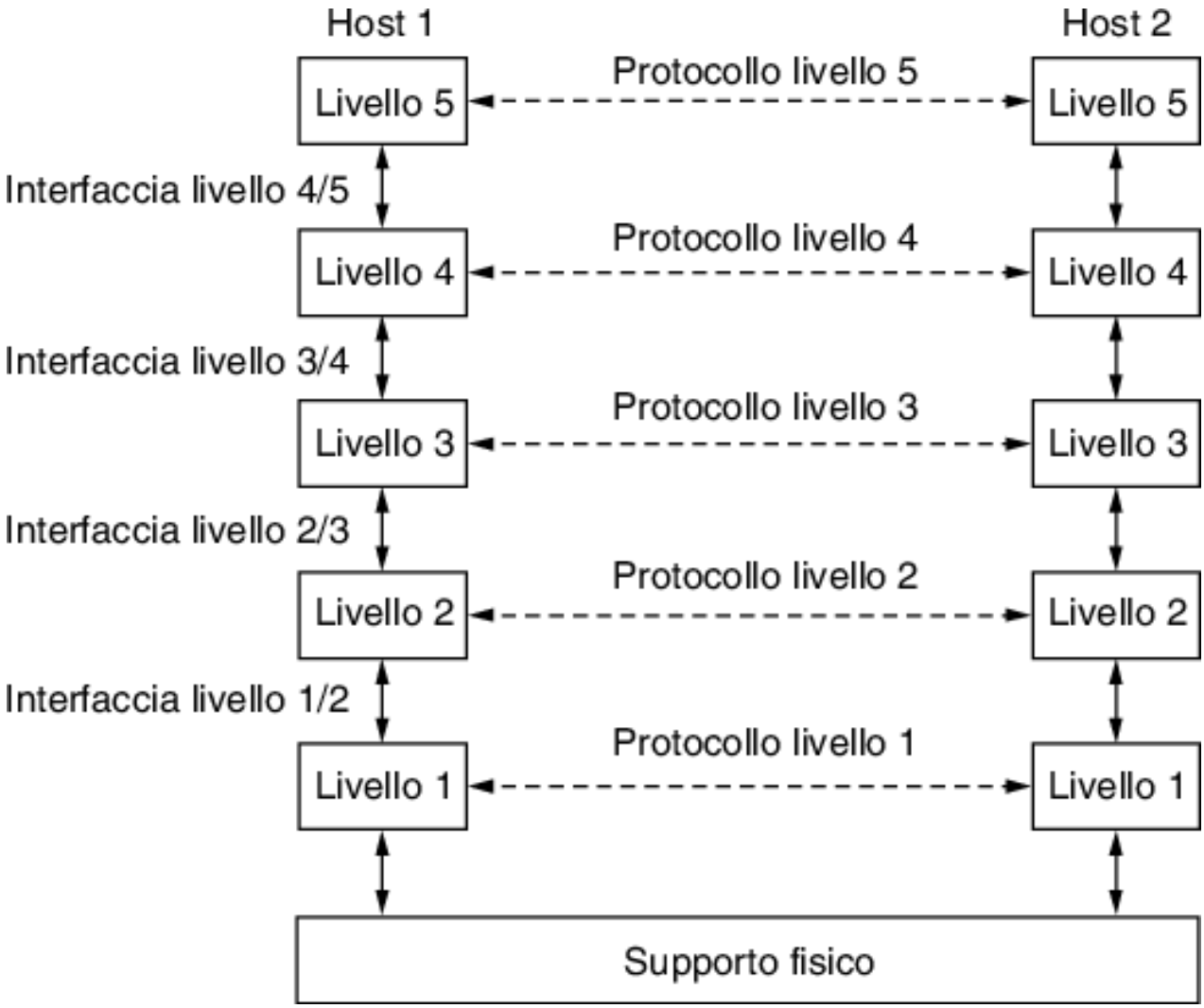


Figura 1.13 Livelli, protocolli e interfacce.

In realtà i dati non sono trasferiti direttamente dal livello *n* di un computer al livello *n* di un altro. Invece, ogni livello passa dati e informazioni di controllo a quello immediatamente sottostante, fino a raggiungere il più basso. Sotto al livello 1 si trova il **supporto fisico** attraverso cui è possibile la comunicazione vera e propria. Nella Figura 1.13 la comunicazione virtuale è indicata da linee tratteggiate e la comunicazione fisica da linee continue.

Tra ciascuna coppia di livelli contigui si trova un'**interfaccia** (*interface*). L'interfaccia definisce le operazioni elementari e i servizi che il livello inferiore rende disponibili a quello soprastante. Quando i progettisti di rete decidono il numero di livelli da includere in una rete e la funzione di ciascuno, uno degli aspetti principali consiste nel definire interfacce pulite tra di essi. Per fare ciò è necessario che ogni livello esegua un insieme specifico di funzioni ben definite. Oltre a minimizzare la quantità di informazioni che devono essere scambiate tra i livelli, interfacce semplici rendono più facile sostituire l'implementazione di un livello con un'altra del tutto diversa (per esempio, tutte le linee telefoniche vengono sostituite con canali satellitari), perché la nuova implementazione dovrà semplicemente offrire al livello immediatamente soprastante esattamente lo

stesso insieme di servizi della vecchia. È normale che host diversi usino differenti implementazioni dello stesso protocollo (spesso scritte da aziende diverse); infatti, il protocollo stesso può cambiare all'interno di un certo livello, senza che i livelli superiori e inferiori se ne accorgano.

L'insieme di livelli e protocolli si chiama **architettura di rete**. Le specifiche di un'architettura devono contenere abbastanza informazioni per consentire all'implementatore di scrivere il programma o costruire l'hardware di ciascun livello in modo che possa seguire correttamente il protocollo. Né i dettagli dell'implementazione né le specifiche delle interfacce fanno parte dell'architettura, perché sono nascoste all'interno dei computer e invisibili dall'esterno. Non è neppure necessario che le interfacce di tutti i computer di una rete siano identiche, purché ogni computer possa usare correttamente tutti i protocolli. L'elenco dei protocolli usati da uno specifico sistema è chiamato **pila di protocolli** (*protocol stack*).[1] Gli argomenti principali di questo libro sono le architetture di rete, le pile di protocolli e i protocolli stessi.
L'idea di una comunicazione multilivello può essere resa efficacemente dalla seguente analogia. Immaginiamo due filosofi (peer a livello 3), uno dei quali parla urdù e inglese mentre l'altro parla cinese e francese. Poiché non hanno linguaggi in comune, ciascuno assume un interprete (peer a livello 2) che a propria volta contatta una segretaria (peer a livello 1). Il filosofo 1 vuole comunicare il proprio apprezzamento verso *oryctolagus cuniculus* al proprio peer. Per farlo, passa un messaggio (in inglese) attraverso l'interfaccia 2/3 al suo interprete che dice "mi piacciono i conigli", come mostrato nella Figura 1.14. Gli interpreti si sono accordati su un linguaggio neutro noto a entrambi, l'olandese, cosicché il messaggio viene convertito in "Ik vind konijnen leuk". La scelta della lingua è il protocollo del livello 2 e dipende dai peer di quel livello.

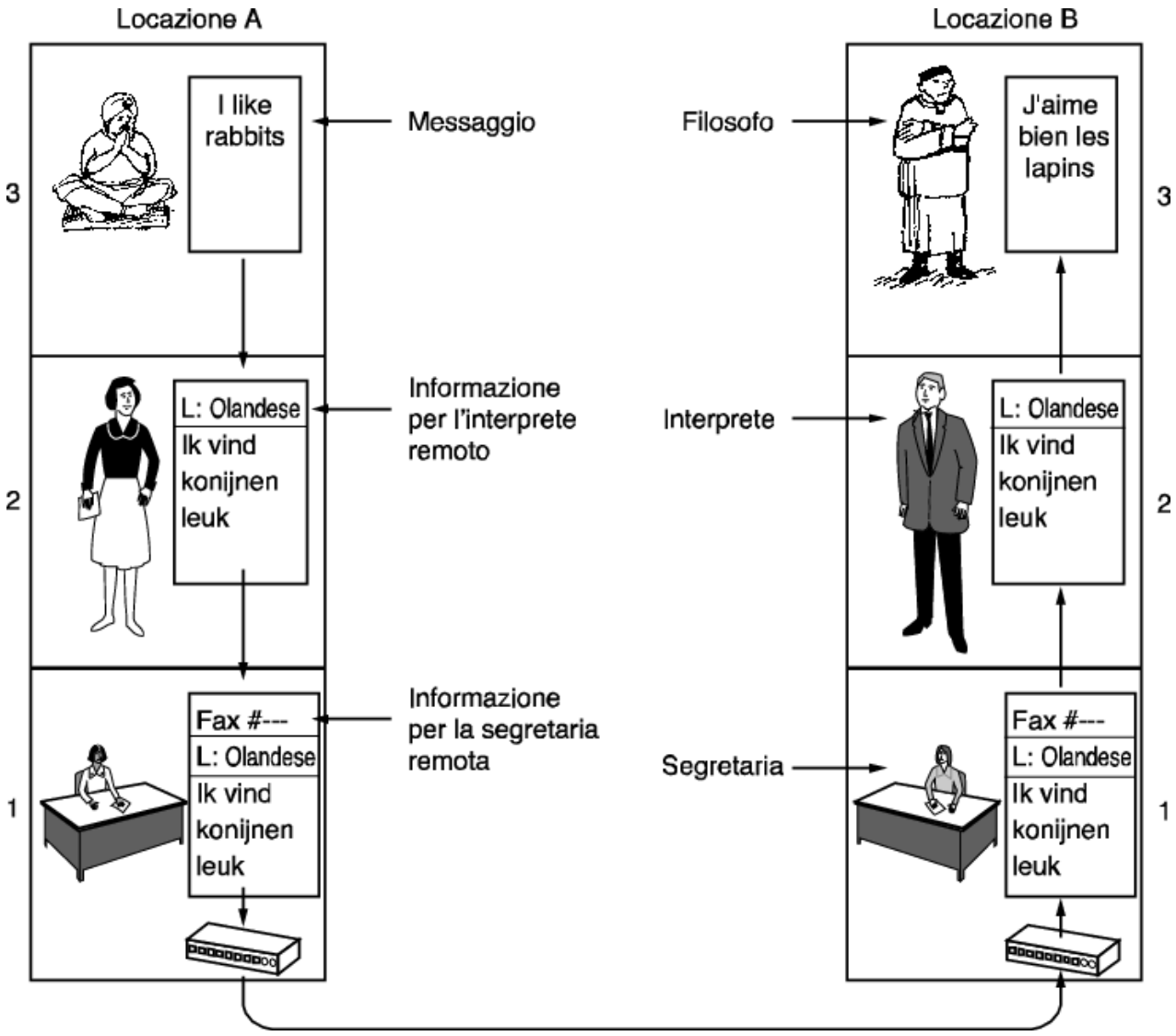


Figura 1.14 L'architettura filosofo-interprete-segretaria.

L'interprete, quindi, dà il messaggio alla segretaria per la trasmissione, per esempio, via e-mail (il protocollo del livello 1). Quando il messaggio arriva all'altra segretaria, è passato all'interprete, tradotto in francese e passato attraverso l'interfaccia 2/3 al filosofo 2. Notiamo che ogni protocollo è completamente indipendente dagli altri, finché non vengono cambiate le interfacce. Gli interpreti potrebbero passare liberamente dall'olandese al finlandese (per esempio), a patto che entrambi acconsentano al cambiamento e nessuno dei due cambi l'interfaccia con il livello 1 o il livello 3. Analogamente le segretarie possono passare dalla e-mail al telefono senza disturbare (o informare) gli altri livelli. Ogni processo può aggiungere informazioni destinate al proprio peer che non vengono passate al livello superiore.

Consideriamo ora un esempio più tecnico: come realizzare la comunicazione tra i livelli più alti della rete della Figura 1.15. Un messaggio *M* è prodotto da un processo applicativo che lavora al livello 5 e passato al livello 4 per la trasmissione. Il livello 4 mette un **header** (*intestazione*) davanti al messaggio per identificarlo e passa il risultato al livello 3. L'header include informazioni di controllo, come l'indirizzo, per consentire al livello 4 del computer destinatario di consegnare il

messaggio. Altri esempi di informazione di controllo usati in alcuni livelli sono i numeri di sequenza (nel caso che il livello sottostante non preservi l'ordine dei messaggi), la grandezza e i tempi.

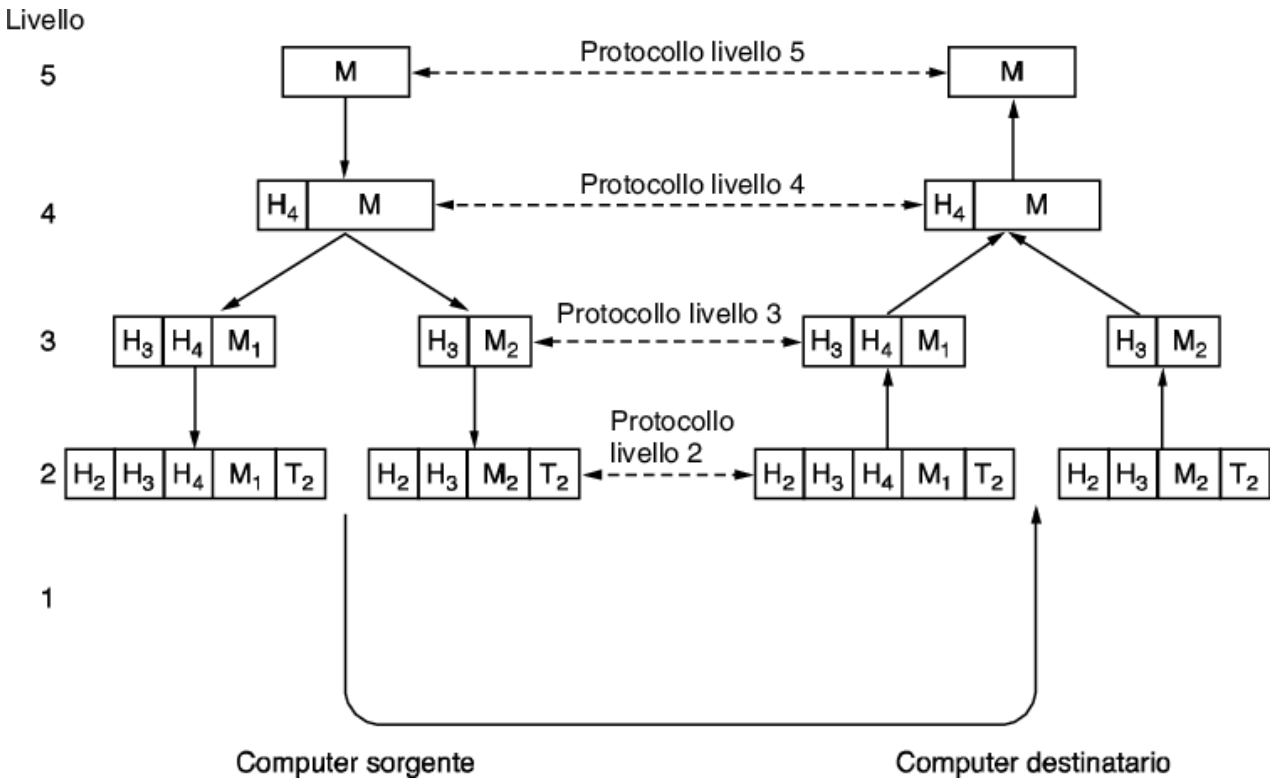


Figura 1.15 Esempio di flusso informativo che supporta la comunicazione virtuale nel livello 5.

In molte reti non c'è limite alla dimensione dei messaggi trasmessi nel protocollo del livello 4, ma quasi sempre esiste un limite imposto dal protocollo del livello 3. Di conseguenza, il livello 3 deve spezzare i messaggi in arrivo in unità più piccole chiamate *pacchetti*, aggiungendo un header di livello 3 davanti a ogni pacchetto. In questo esempio M è diviso in due parti, M_1 e M_2 che verranno trasmessi separatamente.

Il livello 3 decide la linea di uscita da usare e passa i pacchetti al livello 2. Il livello 2 aggiunge a ogni pezzo non solo un header, ma anche un **trailer** (una informazione aggiuntiva in coda) e fornisce l'unità d'informazione risultante al livello 1 per la trasmissione fisica. Nel computer destinatario il messaggio si muove verso l'alto, passando da livello in livello, e gli header vengono rimossi man mano. Nessuno degli header per i livelli inferiori a n viene passato al livello n.

L'aspetto principale da cogliere nella Figura 1.15 è la relazione tra la comunicazione reale e quella virtuale, e la differenza tra protocolli e interfacce. I peer del livello 4, per esempio, modellano concettualmente la loro comunicazione come se fosse "orizzontale", usando il protocollo del livello 4. Ognuno ha verosimilmente una procedura chiamata *SendToOtherSide* e una *GetFromOtherSide*, anche se queste procedure in realtà comunicano mediante livelli inferiori attraverso l'interfaccia 3/4 e non direttamente con la controparte.

L'astrazione dei peer è fondamentale per la progettazione delle reti. Usandola è possibile spezzare in diverse parti più piccole e gestibili il compito impossibile di progettare l'intera rete, poiché basta progettare i singoli livelli. Anche se il Paragrafo 1.3 porta il titolo "Software di rete", vale la pena osservare che i livelli più bassi di una gerarchia di protocolli sono spesso implementati in hardware o firmware. In ogni caso, nei protocolli sono presenti algoritmi complessi, anche quando sono nascosti (del tutto o in parte) all'interno dell'hardware.

Progettazione dei livelli

Alcuni problemi fondamentali nella progettazione delle reti si presentano livello dopo livello. Nel seguito ne menzioneremo brevemente i principali.

Il punto chiave nel progettare una rete che funzioni correttamente, anche se composta da un insieme di componenti di per sé inaffidabili, è l'affidabilità. Si pensi ai bit di un pacchetto che sta viaggiando attraverso la rete. Potrebbe accadere che alcuni di questi bit vengano ricevuti danneggiati (o invertiti, trattandosi di 0 e 1) a causa di rumore elettrico imprevisto, segnali wireless casuali, difetti hardware, errori del software e così via. Come si possono individuare e correggere questi errori?

Un meccanismo per intercettare errori nelle informazioni ricevute si serve di codici per l'**individuazione di errori** (*error detection*). Le informazioni ricevute sbagliate possono essere ritrasmesse finché non siano ricevute correttamente. Codici più complessi permettono la **correzione degli errori** (*error correction*) in cui il messaggio corretto è ricostruito dai possibili bit scorretti originariamente ricevuti. Entrambi questi meccanismi lavorano aggiungendo informazioni ridondanti. Sono usati nei livelli bassi per proteggere i pacchetti spediti su singoli link o, a livelli alti, per controllare che i contenuti vengano ricevuti correttamente.

Un altro problema di affidabilità consiste nel trovare un percorso valido attraverso la rete. Spesso ci sono più percorsi da una sorgente a un destinatario e nelle reti grandi possono esistere link o router non funzionanti. Supponiamo che la rete in Germania non sia funzionante. I pacchetti inviati da Londra a Roma attraverso la Germania non potranno passare, ma potremmo spedirli via Parigi. Le rete dovrebbe prendere questa decisione in modo automatico. Questo argomento è chiamato **routing** (*instradamento*).

Un altro problema di progettazione concerne l'evoluzione della rete. Col passare degli anni le reti crescono sempre di più e nascono nuovi progetti che necessitano di essere connessi alla rete esistente. Abbiamo appena visto il meccanismo chiave di strutturazione usato per sopportare cambiamenti e consistente nel dividere l'intero problema e nascondere i dettagli di implementazione: il **protocol layering** (*stratificazione dei protocolli*). Esistono anche molte altre strategie.

Poiché nella rete ci sono molti computer, ogni livello richiede un meccanismo per identificare chi trasmette e chi riceve un particolare messaggio. Questo meccanismo è chiamato **addressing** o **naming**, rispettivamente nei livelli bassi e alti.

Differenti tecnologie di rete spesso hanno limitazioni diverse. Per esempio, non tutti i canali di comunicazione conservano i messaggi nell'ordine in cui sono stati inviati; per questo vengono numerati. Un altro esempio è la differenza tra le reti nel numero massimo di messaggi che possono

essere trasmessi. Questo porta a meccanismi per frazionare, trasmettere e quindi riassemblare i messaggi.

Tale argomento è chiamato **internetworking**.

Quando le reti diventano grandi sorgono nuovi problemi. Nelle grandi città abbiamo ingorghi dovuti al traffico e alla carenza di numeri telefonici. Una persona difficilmente si perde nel proprio quartiere, ma a livello cittadino orientarsi può diventare un bel problema. Progetti di rete che continuano a funzionare bene quando la rete diventa grande sono detti **scalabili**.

Una terza questione è quella dell'allocazione delle risorse. Le reti forniscono un servizio agli host derivante dalle loro risorse sottostanti, quali la capacità di trasmissione delle linee. Per funzionare bene, le reti necessitano di meccanismi che dividano le loro risorse in modo che un host non interferisca troppo con gli altri.

Molti progetti di rete condividono dinamicamente la banda concedendola alle necessità a breve termine degli host, piuttosto che assegnare a ogni host una frazione prefissata di banda che ciascuno può o meno usare. Questo meccanismo viene chiamato **statistical multiplexing** (*multiplexing statistico*), che significa condivisione sulla base della statistica delle richieste. Tale condivisione viene applicata nei livelli bassi a un singolo link o nei livelli alti a una rete e a applicazioni che usano la rete.

Un problema di allocazione che coinvolge tutti i livelli riguarda come impedire che una sorgente veloce inondi di dati un ricevente lento. Per affrontare tale eventualità viene spesso usato un feedback da parte del ricevente alla sorgente. Questo argomento è chiamato **flow control** (*controllo di flusso*). A volte la rete è intasata perché troppi computer cercano di inviare un traffico eccessivo di dati e non riesce a consegnare i pacchetti. Tale sovraccaricamento della rete è chiamato **congestione**. Una strategia consiste nel fare in modo che ogni computer riduca la propria richiesta quando si accorge di una congestione. Anche questo meccanismo può essere usato a ogni livello.

È interessante osservare che la rete offre più risorse della singola banda. Per applicazioni quali la trasmissione di video in tempo reale la tempestività di consegna è fondamentale. Molte reti devono fornire servizio sia ad applicazioni che vogliono la consegna in **tempo reale** sia ad applicazioni che richiedono una grande capacità di trasferimento. Il meccanismo che concilia queste due richieste in competizione tra di loro è chiamato **qualità del servizio** (QoS, quality of service).

L'ultima importante questione riguardante la progettazione è la sicurezza della rete. che ha come obiettivo la difesa da diversi tipi di minacce, una delle quali, già accennata in precedenza, riguarda l'ascolto abusivo delle comunicazioni. I meccanismi che garantiscono riservatezza sono una difesa da questa minaccia e vengono implementati in molti livelli. I meccanismi per l'autenticazione non permettono che qualcuno impersoni qualcun altro. Potrebbero essere usati per distinguere i siti Web bancari veri da quelli falsi o per permettere alla rete cellulare di controllare che una chiamata venga fatta proprio da un certo telefono a cui addebitarne il costo. Altri meccanismi per l'**integrità** prevengono cambiamenti fraudolenti nei messaggi, come alterare "addebitami 10 dollari sul conto" in "addebitami 1000 dollari sul conto". Tutti questi meccanismi sono basati sulla crittografia che studieremo nel Capitolo 8.

Servizi orientati alla connessione e senza connessione a confronto

Ogni livello può offrire a quelli sovrastanti due tipi diversi di servizio: orientati alla connessione oppure senza connessione. In questo paragrafo si esamineranno le differenze tra i due tipi.

Un **servizio orientato alla connessione** assomiglia al sistema telefonico. Per parlare con qualcuno si deve prendere il telefono, comporre il numero, parlare e poi riagganciare. Allo stesso modo, per usare un servizio di rete orientato alla connessione, l'utente deve innanzitutto stabilire una connessione, usarla e quindi rilasciarla. L'aspetto essenziale di una connessione è che funziona come un tubo: il trasmettitore vi spinge oggetti (bit) a una estremità e il ricevitore li prende dall'altra. Nella maggior parte dei casi l'ordine è conservato, quindi i bit arrivano nella sequenza secondo cui sono stati trasmessi.

In alcuni casi, quando si stabilisce una connessione, il trasmittente, il ricevente e la sottorete eseguono una **negoziazione** dei parametri da usare, come la massima dimensione di un messaggio, la qualità di servizio richiesta e altri elementi. Tipicamente, un lato fa una proposta e il corrispondente la può accettare, rifiutare o fare una controproposta. Un **circuito** è un altro nome per una connessione a cui sono associate delle risorse quali una larghezza di banda prefissata. Questo termine deriva dalla rete telefonica nella quale un circuito era un percorso su un cavo di rame che trasportava la conversazione telefonica.

Al contrario del servizio orientato alla connessione, un servizio senza connessione si comporta come il servizio postale. Ogni messaggio (lettera) porta l'indirizzo completo del destinatario ed è instradato attraverso il sistema postale in modo indipendente dai messaggi successivi. I messaggi assumono nomi diversi a seconda del contesto; a livello rete un messaggio prende il nome di pacchetto. Ogni nodo intermedio lungo il percorso deve ricevere completamente un pacchetto prima di inoltrarlo al nodo successivo; questo approccio viene chiamato **store-and-forward**. Normalmente, quando due messaggi vengono inviati allo stesso destinatario, il primo che viene spedito è anche il primo che arriva. Può tuttavia accadere che il primo che è stato spedito subisca un ritardo e che quindi il secondo arrivi per primo.

Ogni tipo di servizio può ulteriormente essere caratterizzato in base alla sua affidabilità. Alcuni servizi sono affidabili, nel senso che non perdono mai dati. Di solito, un servizio affidabile è implementato in modo tale che il destinatario confermi il ricevimento di ciascun messaggio, cosicché il mittente sia informato del suo arrivo. Anche se conveniente, il processo di conferma causa appesantimenti e ritardi che a volte possono risultare inopportuni.

Una tipica situazione dove un servizio affidabile orientato alla connessione è particolarmente opportuno è il trasferimento dei file. Il proprietario del file vuole essere sicuro che tutti i bit siano arrivati a destinazione senza errori e nello stesso ordine in cui sono stati inviati. Pochissimi utenti preferirebbero un servizio che di tanto in tanto perde o scambia qualche bit, anche se fosse molto più veloce.

Esistono due piccole varianti del servizio affidabile orientato alla connessione: il *message sequence* e il *byte stream*. Nella prima variante i confini dei messaggi vengono preservati. Quando si mandano due messaggi da 1.024 byte, arrivano come due distinti messaggi da 1.024 byte ciascuno e mai come singolo messaggio da 2.048 byte. Nella seconda variante, la connessione è un semplice flusso

di byte, senza divisioni tra i messaggi. Quando 2.048 byte arrivano al ricevente, non è possibile sapere se erano stati spediti come singolo messaggio da 2.048 byte, due messaggi da 1.024 byte o magari 2.048 messaggi da 1 byte. Se le pagine di un libro vengono spedite attraverso la rete a una fotocompositrice come messaggi separati, conservare i confini tra i messaggi può essere importante. Viceversa, per scaricare un film è sufficiente un flusso di byte tra il computer dell'utente e il server. I confini tra i messaggi all'interno del film non hanno importanza.

In alcune applicazioni il ritardo di trasferimento introdotto dalle conferme di ricezione è inaccettabile. Una di queste applicazioni è il traffico vocale digitalizzato per il **voice over IP**. Per un utente telefonico è meglio ascoltare un po' di rumore sulla linea di tanto in tanto che percepire un ritardo nell'attesa delle conferme. Allo stesso modo, quando si trasmette una videoconferenza non è un problema avere qualche pixel sbagliato, ma sarebbe irritante vedere l'immagine che procede a scatti quando il flusso si arresta per correggere gli errori.

Non tutte le applicazioni hanno bisogno di un servizio orientato allo connessione; per esempio, chi invia messaggi pubblicitari a molti destinatari (spam) non vuole prendersi la briga di creare una connessione verso un destinatario e poi distruggerla solo per una comunicazione, come pure non è essenziale un'affidabilità di consegna del 100%, specialmente se è più costosa. Tutto ciò che serve è un modo per spedire un singolo messaggio che abbia un'elevata probabilità di arrivare a destinazione, senza garanzie. Un servizio senza connessione non affidabile (cioè privo di conferma) spesso viene chiamato servizio **datagram**, per analogia con il servizio telegrafico (*telegram service*), che non restituisce una conferma a chi trasmette. Benché sia inaffidabile è il formato dominante nella maggior parte delle reti per ragioni che risulteranno chiare più avanti.

In altre situazioni, la comodità di non dover stabilire una connessione per spedire un breve messaggio è auspicabile, ma l'affidabilità è essenziale. In questi casi è utilizzabile il servizio **datagram con conferma** (**acknowledged datagram service**). È come spedire una raccomandata con ricevuta di ritorno. Quando la ricevuta torna indietro, il mittente è assolutamente certo che la lettera è stata recapitata al destinatario desiderato e non è stata persa.

Vi è ancora un altro servizio chiamato **request-reply** (*domanda-risposta*). In questo servizio la sorgente trasmette un singolo datagramma contenente una richiesta; il messaggio ricevuto dal mittente rappresenta la risposta. Il request-replay è comunemente implementato nel modello client-server: il client inoltra una richiesta e il server risponde. Per esempio, un cellulare client potrebbe fare una richiesta della mappa della sua attuale posizione a un server. La Figura 1.16 riepiloga i tipi di servizio discussi.

	Servizio	Esempio
Orientato alla connessione	Flusso affidabile di messaggi	Sequenza di pagine
	Flusso affidabile di byte	Scaricamento di filmati
	Connessione inaffidabile	Voice over IP
Senza connessione	Datagram inaffidabile	E-mail spazzatura (spam)
	Datagram con conferma	Messaggistica istantanea
	Request-reply	Interrogazione di un database

Figura 1.16 Sei diversi tipi di servizio.

A prima vista il concetto di comunicazione inaffidabile può confondere. Dopotutto, chi mai preferirebbe una comunicazione inaffidabile a una affidabile? Prima di tutto, la comunicazione affidabile (nel senso visto, e cioè con conferma) potrebbe non essere disponibile a un dato livello. Per esempio, Ethernet non fornisce una comunicazione affidabile; i pacchetti si possono corrompere di tanto in tanto mentre sono in transito. È compito dei protocolli dei livelli superiori prendersi carico del problema. In particolare, molti servizi affidabili sono implementati su datagrammi inaffidabili. In secondo luogo, i ritardi necessari per fornire un servizio affidabile potrebbero essere inaccettabili, specialmente nelle applicazioni real-time come quelle multimediali. Per questi motivi comunicazioni affidabili e inaffidabili coesistono.

Primitive di servizio

Un servizio è formalmente specificato da un insieme di **primitive** (operazioni) che i processi utenti hanno a disposizione per accedere al servizio. Queste primitive istruiscono il servizio a eseguire alcune azioni o a riferire quelle prese da entità di pari livello. Se la pila dei protocolli si trova nel sistema operativo, come spesso accade, le primitive sono generalmente chiamate “di sistema”. Queste chiamate causano la commutazione in modalità kernel che a sua volta fa prendere il controllo del computer al sistema operativo per spedire i pacchetti necessari.[2]
L’insieme delle primitive disponibili dipende dalla natura del servizio offerto. Le primitive per un servizio orientato alla connessione sono diverse da quelle di un servizio senza connessione. Come esempio minimo di primitive di servizio che possono essere fornite per implementare un flusso di byte affidabile in un ambiente client-server esaminiamo le primitive elencate nella Figura 1.17. Esse dovrebbero risultare familiari agli appassionati dell’interfaccia socket di Berkeley, in quanto le sue primitive sono una versione semplificata di questa interfaccia.

Primitiva	Significato
LISTEN	Attesa bloccante di una connessione in arrivo
CONNECT	Stabilisce una connessione con un peer in attesa
ACCEPT	Accetta una richiesta di connessione da un peer
RECEIVE	Attesa bloccante per un messaggio in arrivo
SEND	Manda un messaggio al peer
DISCONNECT	Termina una connessione

Figura 1.17 Sei primitive di servizio per implementare un semplice servizio orientato alla connessione.

Queste primitive potrebbero essere usate per un'interazione request-reply in un ambiente client-server. Per mostrare come sia possibile, vediamo un semplice protocollo che implementa questo servizio usando datagrammi di conferma.

Prima di tutto, il server esegue LISTEN per indicare che è pronto ad accettare connessioni in arrivo. Un modo comune per implementare LISTEN consiste nel renderlo una chiamata di sistema bloccante. Dopo aver eseguito la primitiva, il processo server resta bloccato finché non appare una richiesta di connessione.

Successivamente, per stabilire una connessione con il server, il processo client esegue CONNECT. La chiamata a CONNECT deve specificare a chi connettersi, quindi può avere un parametro per indicare l'indirizzo del server. A questo punto il sistema operativo di solito manda un pacchetto al peer chiedendogli di connettersi, come è mostrato da (1) nella Figura 1.18. Il processo client resta sospeso finché non arriva una risposta.

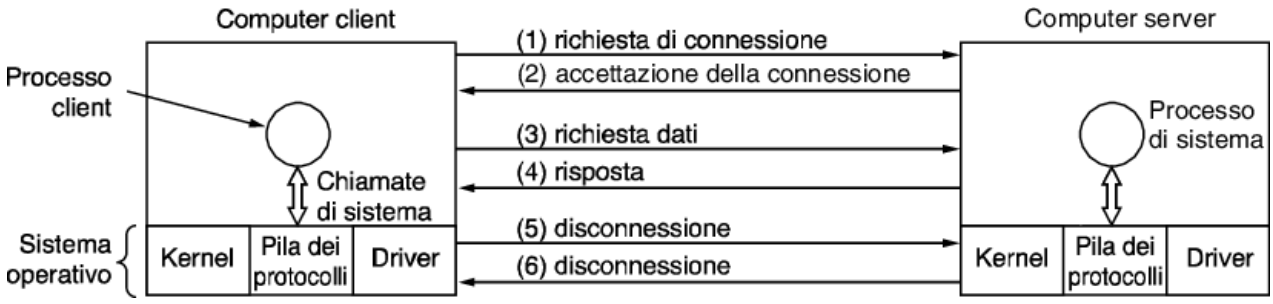


Figura 1.18 Una semplice interazione client-server facente uso di datagrammi di conferma.

Quando il pacchetto raggiunge il server il sistema operativo si accorge che sta richiedendo una connessione, controlla se c'è un ricevente in attesa e in caso affermativo lo sblocca. Il processo server può quindi stabilire una connessione con la chiamata ACCEPT. Questa restituisce una risposta (2) al processo client e accetta la connessione. Il suo arrivo sblocca il client e a questo punto client e server sono entrambi in esecuzione e la connessione è stata stabilita.

L'ovvia analogia tra questo protocollo e la realtà è un utente (client) che chiama un addetto commerciale del proprio fornitore (server). Il commerciale inizia la sua attività stando vicino al telefono per rispondere nel caso in cui squilli. Il cliente fa la chiamata e quando l'addetto commerciale prende il telefono la comunicazione è stabilita.

Il passo successivo per il server è quello di eseguire RECEIVE per prepararsi ad accettare la prima richiesta. Normalmente il server lo fa immediatamente dopo essere stato sbloccato da LISTEN, prima che la conferma (*acknowledgment*) ritorni al client. La chiamata a RECEIVE blocca il server.

Ora il client esegue SEND per trasmettere le sue richieste (3), seguito da RECEIVE per ottenere la risposta. L'arrivo del pacchetto di richiesta al computer server sblocca il processo server che può elaborare la richiesta. Eseguito il lavoro, usa SEND per restituire la risposta al client (4). L'arrivo di questo pacchetto sblocca il client che ora può esaminare la risposta. Se il client ha richieste aggiuntive, può farle ora.

Se ha finito, il client può usare DISCONNECT per terminare la connessione (5). Normalmente, il DISCONNECT iniziale è una chiamata bloccante che sospende il client e spedisce un pacchetto al server indicando che la connessione non è più necessaria. Quando il server riceve il pacchetto, emette a sua volta un DISCONNECT dando conferma al client e rilasciando la connessione (6). Quando il pacchetto del server ritorna alla macchina client, il processo client è rilasciato e la connessione interrotta. In sintesi, questo è il modo in cui funziona la comunicazione orientata alla connessione.

Naturalmente la vita non è mai semplice. Molte cose possono andare storte. Le temporizzazioni potrebbero essere sbagliate (per esempio CONNECT viene fatto prima di LISTEN), i pacchetti possono andare persi e molto altro può accadere. In seguito si esamineranno questi problemi in maggior dettaglio, ma per il momento la Figura 1.18 riassume brevemente il funzionamento della comunicazione client-server su una rete con datagrammi di conferma, in modo da non tenere conto dei pacchetti persi.

Dato che per completare la sequenza di questo protocollo servono sei pacchetti, ci si può chiedere perché non si usi invece un protocollo senza connessione. La risposta è che in un mondo perfetto si potrebbe fare, nel qual caso basterebbero solo due pacchetti: uno per la richiesta e uno per la risposta. Tuttavia, di fronte alla possibilità di messaggi enormi in entrambe le direzioni (per esempio un file di alcuni megabyte), errori di trasmissione e pacchetti persi, la situazione cambia. Se la risposta consiste in centinaia di pacchetti, alcuni dei quali possono andare persi nella trasmissione, come può il client sapere se qualche pezzo manca? Come può sapere se l'ultimo pacchetto che ha ricevuto è veramente l'ultimo inviato? Supponiamo che il client abbia chiesto un secondo file: come si può distinguere il pacchetto 1 del secondo file da un pacchetto 1 del primo file, perso in precedenza, che improvvisamente trova la strada per il client? In breve, nel mondo reale un semplice protocollo request-reply su una rete inaffidabile è spesso inadeguato. Nel

Capitolo 3 si studierà in dettaglio un insieme di protocolli che risolve questo e altri problemi. Per il momento, basta dire che a volte è molto utile avere un flusso di byte tra processi che sia affidabile e rispetti la sequenza.

Relazione tra servizi e protocolli

Servizi e protocolli sono concetti distinti. La distinzione è però così importante che è necessario evidenziarla. Un *servizio* è un insieme di primitive (operazioni) che un livello offre a quello superiore. Il servizio definisce quali operazioni il livello sia in grado di offrire su richiesta dei suoi utenti, ma non dice nulla di come queste operazioni sono implementate. Un servizio è correlato all'interfaccia tra due livelli, dove quello inferiore è il provider del servizio mentre quello superiore è l'utente.

Un protocollo, invece, è un insieme di regole che controllano il formato e il significato dei pacchetti o messaggi scambiati tra le entità pari all'interno di un livello. Le entità usano i protocolli per implementare le loro definizioni dei servizi. Sono libere di cambiare i loro protocolli, a patto di non cambiare il servizio visibile agli utenti. In questo modo, il servizio e il protocollo sono completamente disaccoppiati. Questo è un concetto chiave che qualunque progettista di rete dovrebbe comprendere bene.

In altri termini, i servizi si riferiscono alle interfacce tra i livelli, come mostrato nella Figura 1.19. I protocolli, invece, riguardano i pacchetti scambiati tra entità di pari livello che risiedono su computer diversi. È importante non confondere i due concetti.

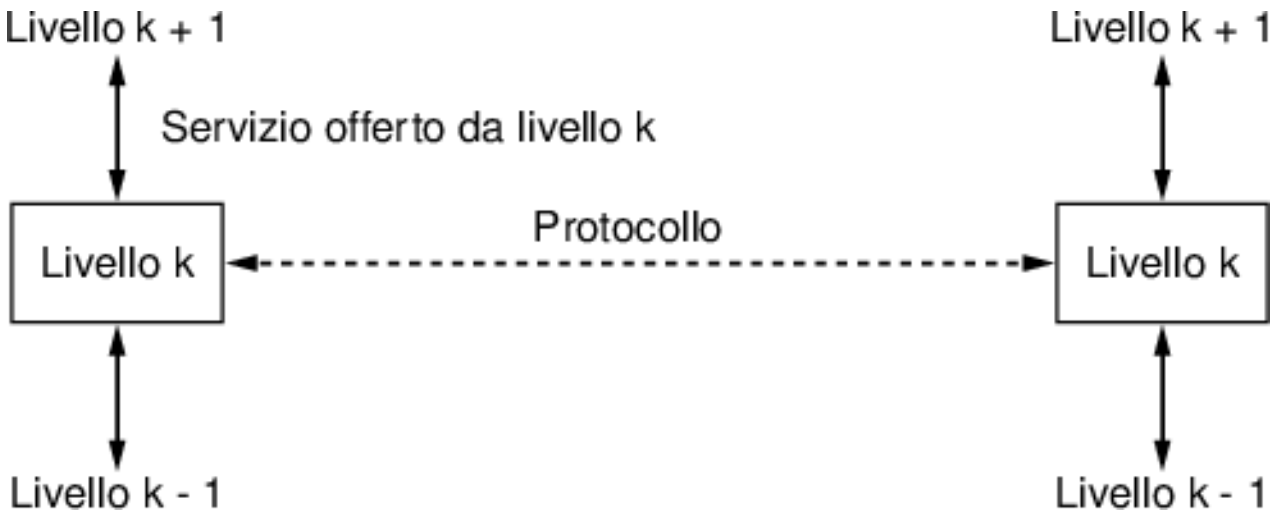


Figura 1.19 Relazione tra un servizio e un protocollo.

Vale la pena ricorrere a un'analogia con i linguaggi di programmazione. Un servizio è come un tipo di dati astratto o un oggetto (in un linguaggio orientato agli oggetti): definisce operazioni che si possono fare sull'oggetto, ma non il modo in cui queste operazioni sono implementate. Un

protocollo, invece, si riferisce all'implementazione del servizio e pertanto non è visibile agli utenti del servizio.

Molti vecchi protocolli non facevano distinzione tra servizio e protocollo. Un tipico livello poteva avere una primitiva di servizio SEND PACKET in cui l'utente doveva fornire il puntatore a un pacchetto completamente assemblato. Questa soluzione implicava che tutte le modifiche a un protocollo diventavano immediatamente visibili agli utenti. La maggior parte dei progettisti di rete oggi considera queste tecniche un errore grossolano.

1.4 Modelli di riferimento

Dopo aver discusso dei livelli di rete in senso astratto, è il momento di guardare alcuni esempi. Nei prossimi due paragrafi esamineremo due importanti architetture di rete, il modello di riferimento OSI e il modello TCP/IP. Anche se i protocolli associati al modello OSI sono ormai in disuso, il modello in sé ha valore generale ed è ancora valido, e le caratteristiche che discuteremo per ogni livello sono ancora molto importanti. Il modello TCP/IP ha caratteristiche opposte: il modello in sé è poco utilizzabile, ma i protocolli sono largamente impiegati; per questo motivo entrambi saranno esaminati in dettaglio. A volte si impara più dai fallimenti che dai successi.

Il modello di riferimento OSI

Il modello OSI (eccetto il mezzo trasmissivo fisico) è mostrato nella Figura 1.20. Questo modello si fonda su una proposta sviluppata dall'**international standards organization** (**ISO**) come primo passo verso la standardizzazione internazionale dei protocolli impiegati nei diversi livelli (Day e Zimmermann, 1983), ed è stato revisionato nel 1995 (Day, 1995). Si chiama modello di riferimento **ISO OSI** (*open system interconnection*) perché riguarda la connessione di sistemi aperti, cioè sistemi che sono "aperti" verso la comunicazione con altri; per brevità verrà chiamato in seguito **modello OSI**.

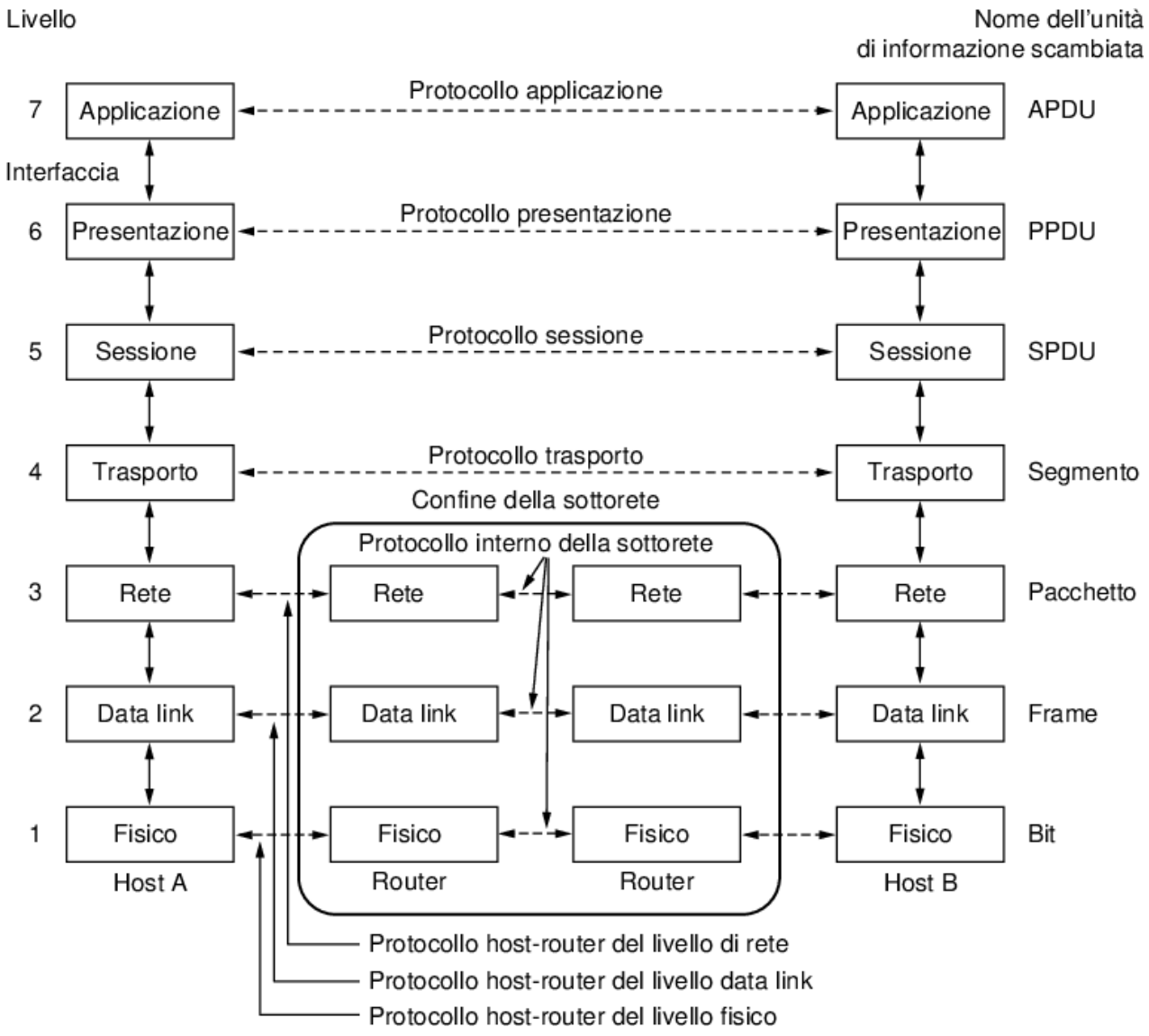


Figura 1.20 Il modello di riferimento OSI.

Il modello OSI ha sette livelli. I principi applicati per arrivare ai sette livelli si possono brevemente riassumere come segue:

1. si deve creare un livello quando è richiesta un'astrazione diversa
2. ogni livello deve svolgere una funzione ben definita
3. la funzione di ogni livello va scelta con uno sguardo rivolto alla definizione di standard per protocolli a livello internazionale
4. i confini dei livelli vanno scelti in modo da minimizzare il flusso di informazioni attraverso le interfacce
5. il numero di livelli deve bastare per evitare la necessità di radunare funzioni distinte nello stesso livello, ma essere abbastanza piccolo da rendere l'architettura gestibile.

In seguito i livelli del modello saranno discussi partendo da quello più basso. Si osservi che il modello OSI in sé non è un'architettura di rete, perché non specifica quali sono esattamente i servizi e i protocolli da usare in ciascun livello; si limita infatti a definire ciò che ogni livello deve fare. Tuttavia, ISO ha prodotto anche standard per ciascun livello, benché non facciano parte del

modello di riferimento: ognuno è stato pubblicato come standard internazionale distinto. Il modello (in parte) è largamente usato anche se i protocolli a esso associati sono stati dimenticati da tempo.

Il livello fisico

Il **livello fisico** si occupa della trasmissione di bit grezzi sul canale di comunicazione. I requisiti di progetto devono assicurare che ogni bit trasmesso con valore 1 sia ricevuto ancora con valore 1 e non con valore 0. Problemi tipici riguardano quali segnali elettrici dovrebbero essere usati per rappresentare un 1 e uno 0, quanti nanosecondi deve durare un bit, se la trasmissione può avvenire simultaneamente in entrambe le direzioni, come si stabilisce la connessione iniziale e come viene abbattuta quando entrambe le parti hanno terminato, quanti contatti deve avere il connettore di rete e quale funzione va assegnata a ciascuno. Le specifiche riguardano per lo più interfacce meccaniche o elettriche e temporizzazioni, oltre al mezzo di trasmissione che si trova sotto al livello fisico.

Il livello data link

Il compito principale del **livello data link** (*collegamento dati*) consiste nel far diventare una trasmissione grezza una linea che appare priva di errori non rilevati. Esegue questo compito mascherando gli errori reali in modo che il livello di rete non li veda. L'obiettivo è raggiunto forzando il trasmittente a suddividere i dati d'ingresso in **frame** (tipicamente qualche centinaio o migliaio di byte) trasmessi sequenzialmente. Se il servizio è affidabile, il ricevente conferma la corretta ricezione di ciascun frame restituendo un **frame di acknowledgment**.

Un altro problema che nasce nel livello data link (e nella maggior parte dei livelli superiori) riguarda il modo per evitare che un trasmittente veloce saturi il buffer di un ricevente lento. Spesso occorre un meccanismo per regolare il traffico, in grado di informare il trasmittente della quantità di spazio libero nel buffer del ricevente.

Le reti broadcast hanno un problema in più nel livello data link: come controllare l'accesso al canale condiviso. Di questo problema si occupa uno speciale sottoinsieme del livello data link, chiamato **medium access control** (*controllo di accesso al mezzo trasmissivo*).

Il livello di rete

Il **livello di rete** controlla il funzionamento della sottorete. Un problema chiave riguarda la modalità con cui i pacchetti sono inoltrati dalla sorgente alla destinazione. I percorsi lungo i quali effettuare l'inoltro possono essere basati su tabelle statiche "cablate" dentro la rete e raramente modificate o, più spesso, vengono aggiornati automaticamente per aggirare componenti di rete non funzionanti. Si possono stabilire all'inizio di ogni conversazione come una sessione di terminale (collegamento a un computer remoto). Infine, potrebbero essere altamente dinamici, determinati per ogni singolo pacchetto, in modo da riflettere lo stato corrente del carico della rete.

Quando nella sottorete sono presenti contemporaneamente troppi pacchetti, vanno a interferire l'un l'altro creando colli di bottiglia. Anche il controllo di queste congestioni spetta al livello di rete in collaborazione con i livelli più alti che si adattano al carico della rete. Più in generale, la qualità del servizio offerto (ritardo, tempo di transito, jitter e così via) è un problema del livello di rete.

Quando un pacchetto deve viaggiare da una rete all'altra per arrivare a destinazione possono nascere molti problemi. L'indirizzamento usato dalla seconda rete potrebbe essere diverso da quello della prima. La seconda rete potrebbe rifiutare il pacchetto perché troppo grosso, i protocolli potrebbero essere diversi, e così via. È compito del livello di rete risolvere questi problemi per consentire la comunicazione tra reti eterogenee.

Nelle reti broadcast il problema dell'instradamento è facile, quindi il livello di rete è solitamente ridotto o addirittura assente.

Il livello di trasporto

La funzione essenziale del **livello di trasporto** è quella di accettare dati dal livello superiore, dividerli in unità più piccole quando necessario, passarle al livello di rete e assicurarsi che tutti i pezzi arrivino correttamente a destinazione. Inoltre, tutto ciò va fatto in un modo efficiente che isoli i livelli superiori dalle inevitabili variazioni delle tecnologie hardware nel corso del tempo.

Il livello di trasporto inoltre stabilisce che tipo di servizio offrire al livello sessione e, in definitiva, agli utenti della rete. Il tipo di connessione trasporto più comune è un canale punto a punto privo di errori che consegna messaggi o byte nello stesso ordine usato per la trasmissione. Altri tipi di servizio a livello di trasporto sono lo spostamento di singoli messaggi senza garanzia sull'ordine di consegna e il broadcast di messaggi a destinazioni multiple. Il tipo di servizio è fissato all'instaurarsi della connessione. Come corollario, è impossibile ottenere un canale privo di errori; ciò che s'intende realmente con questa espressione è un tasso di errore così basso da poter essere trascurato nella pratica.

Il livello di trasporto copre tutto il percorso da sorgente a destinazione; in altri termini, un programma sul computer sorgente instaura una conversazione con un programma corrispondente sul computer destinatario, utilizzando intestazioni dei messaggi e messaggi di controllo. Nei livelli inferiori i protocolli riguardano la comunicazione tra ciascun computer e i vicini immediati, e non tra i computer sorgente e destinatario, che possono essere separati da molti router. La Figura 1.20 mostra le differenze tra i livelli da 1 a 3 (che sono concatenati) e i livelli da 4 a 7 (che agiscono tra punti terminali, end-to-end).

Il livello sessione

Il livello sessione permette a utenti su computer diversi di stabilire tra loro una **sessione**. Le sessioni offrono diversi servizi, tra cui: **controllo del dialogo** (tenere traccia di quando è il turno di trasmettere e quando di ricevere), **gestione dei token** (evitare che le due parti tentino la stessa operazione critica al medesimo istante) e **sincronizzazione** (supervisionare una lunga trasmissione per consentire la sua ripresa dal punto in cui si è interrotta a causa di un crash e del seguente recupero).

Il livello presentazione

A differenza di livelli inferiori, che per lo più si occupano di spostare bit, il **livello presentazione** si occupa della sintassi e della semantica dell'informazione trasmessa. Per consentire la comunicazione tra computer con differenti rappresentazioni dei dati, le strutture dati da scambiare si possono

definire in modo astratto, insieme a una codifica standard usata "sul filo". Il livello presentazione gestisce queste strutture dati astratte e consente lo scambio e la definizione di strutture dati di livello superiore (per esempio transazioni bancarie).

Il livello applicazione

Il **livello applicazione** comprende una varietà di protocolli comunemente richiesti dagli utenti. Un protocollo applicativo largamente usato è **HTTP** (*hypertext transfer protocol*), la base del World Wide Web. Quando un browser richiede una pagina Web, invia al server il nome della pagina desiderata usando HTTP, quindi il server risponde inviandola. Altri protocolli applicativi si usano per trasferimento file, posta elettronica e news.

Il modello di riferimento TCP/IP

Lasciamo da parte il modello di riferimento OSI per analizzare il modello di riferimento del progenitore di tutte le reti di calcolatori geografiche, ARPANET, e il suo successore Internet. Una breve storia di ARPANET sarà fornita più avanti, ma è utile menzionare alcuni dei suoi aspetti chiave. ARPANET fu una rete sperimentale sponsorizzata dal DoD (dipartimento della difesa USA). Connetteva centinaia di università e installazioni governative tramite linee telefoniche in affitto. Quando furono aggiunte reti satellitari e via radio, i protocolli esistenti incontrarono difficoltà nel collegamento, richiedendo una nuova architettura di riferimento. Per questo motivo la capacità di collegare tra loro più reti in modo semplice è stata sin dall'inizio uno dei principali obiettivi di progetto. Questa architettura è poi diventata nota con il nome di **modello di riferimento TCP/IP**, dal nome dei suoi due protocolli principali. È stata definita per la prima volta in (Cerf e Kahn, 1974) e rivista e definita come standard nella comunità Internet in (Braden, 1989). La filosofia di progetto alla base del modello è discussa in (Clark, 1988).

Il dipartimento della difesa temeva che alcuni dei suoi preziosi host, router e gateway tra reti potessero essere improvvisamente ridotti in briciole da un attacco sovietico; perciò un altro obiettivo principale era che la rete potesse sopravvivere alla perdita dell'hardware di sottorete senza interrompere le conversazioni in corso. In altri termini, il dipartimento della difesa voleva che le connessioni rimanessero intatte per tutto il tempo in cui il computer sorgente e quello destinazione fossero in funzione, anche se alcuni dei computer o delle linee di trasmissione tra essi fossero andati improvvisamente in avaria. Poiché erano previste applicazioni con richieste divergenti, che spaziavano dal trasferimento di file alla trasmissione della voce in tempo reale, era inoltre necessaria un'architettura flessibile.

Il livello link

Tutti i requisiti portarono alla scelta di una rete a commutazione di pacchetto basata su un livello privo di connessione, in grado di operare attraverso differenti reti. Il livello più basso nel modello, il **livello link**, descrive che cosa devono fare i collegamenti (linee seriali e la classica Ethernet) per esaudire le necessità di questo livello internet senza connessione. Più che un vero e proprio livello nel senso usuale del termine, è un'interfaccia tra l'host e il mezzo trasmissivo. I primi documenti riguardanti il modello TCP/IP presentavano scarse informazioni su questo livello.

Il livello internet

Il livello internet è il perno che tiene insieme l'intera architettura; è mostrato nella Figura 1.21 più o meno in corrispondenza del livello OSI della rete. Il suo compito è permettere agli host di inviare pacchetti su qualsiasi rete e fare in modo che questi possano viaggiare indipendentemente verso la destinazione (potenzialmente su reti diverse). Potrebbero persino arrivare con un ordine diverso da quello con cui sono stati spediti, e in questo caso (se è richiesta una consegna in sequenza) è compito dei livelli superiori riordinarli. Il termine "internet" viene usato qui in senso generico, anche se questo livello è presente in Internet.

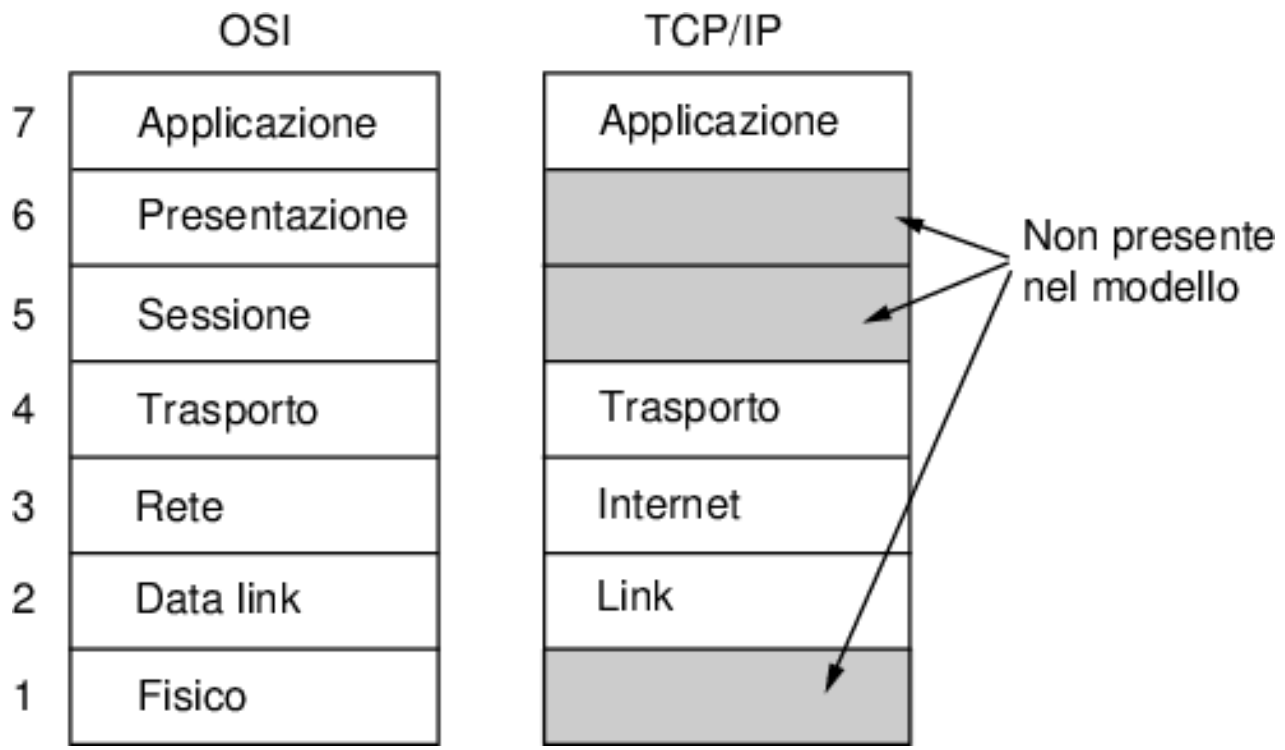


Figura 1.21 Il modello di riferimento TCP/IP.

Una possibile analogia è quella di una persona che imbuca una serie di lettere in sequenza con destinatari internazionali: con un po'di fortuna, la maggior parte delle lettere sarà consegnata nei paesi di destinazione agli indirizzi corrispondenti. Probabilmente le lettere viaggeranno attraverso uno o più centri di smistamento internazionali, ma ciò è invisibile all'utente; resta nascosto anche il fatto che ogni nazione (cioè ogni rete) ha propri francobolli, dimensioni standard per le buste e regole per la consegna della posta.

Il livello internet definisce un formato ufficiale per i pacchetti e un protocollo chiamato **IP** (*internet protocol*) accompagnato da un protocollo di supporto chiamato **IMCP** (*internet control message protocol*). Lo scopo del livello internet è quello di consegnare i pacchetti IP alla destinazione corretta. L'instradamento dei pacchetti è chiaramente il problema principale, insieme alla necessità di garantire l'assenza di congestioni (benché IP non si sia mostrato efficace nell'evitarle).

Il livello di trasporto

Nel modello TCP/IP il livello superiore a quello internet viene attualmente chiamato **livello di trasporto** (transport layer). È progettato per consentire la comunicazione tra peer degli host sorgente e destinazione, come nel livello trasporto OSI. In questo livello sono stati definiti due protocolli di trasporto end-to-end: il primo, **TCP** (*transmission control protocol*), è un protocollo affidabile orientato alla connessione che permette a un flusso di byte emessi da un computer di raggiungere senza errori qualsiasi altro computer sulla internet. Suddivide il flusso di byte entrante in messaggi e passa ciascun frammento al livello internet. A destinazione, il processo TCP ricevente ricompone il messaggio ricevuto per formare il flusso di uscita. TCP gestisce anche il controllo di flusso, per garantire che una sorgente veloce non possa congestionare un ricevente lento con una quantità di messaggi superiore a quelli che sa gestire.

Il secondo protocollo di questo livello, **UDP** (*user datagram protocol*), è un protocollo inaffidabile senza connessione per le applicazioni che non vogliono la garanzia di ordinamento e il controllo di flusso di TCP, ma preferiscono gestire queste funzioni in modo autonomo. È inoltre largamente impiegato per le richieste client-server singole di tipo request-reply, e dalle applicazioni dove la consegna rapida è più importante dell'accuratezza, come la trasmissione di voce e filmati. La relazione tra IP, TCP e UDP è mostrata nella Figura 1.22.

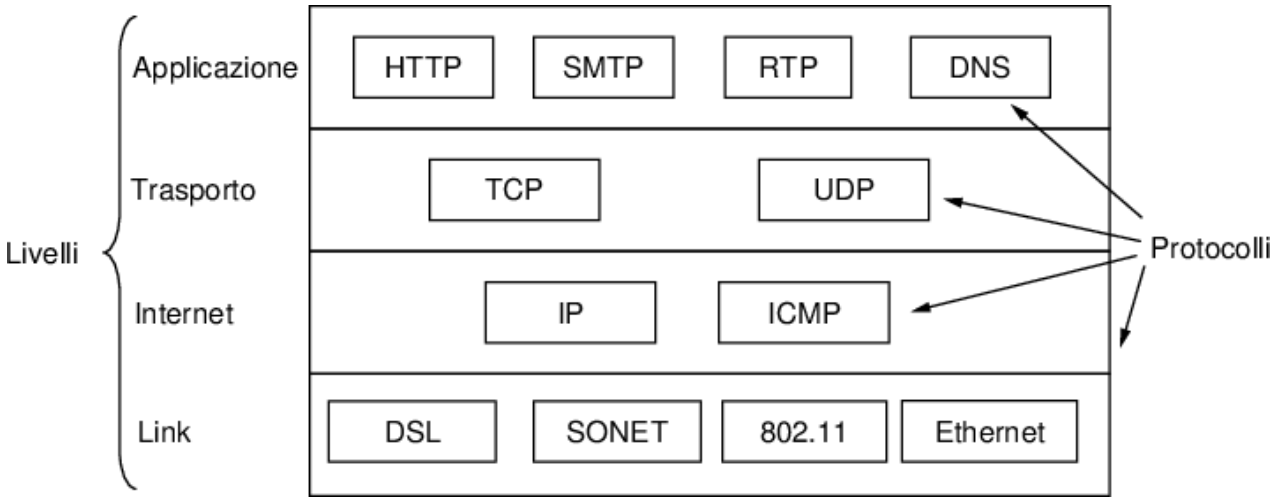


Figura 1.22 Il modello TCP/IP con alcuni protocolli che studieremo.

Il livello applicazione

Il modello TCP/IP non ha livello sessione e presentazione, poiché di essi non fu percepito alcun bisogno. Sono le applicazioni che includono semplicemente le funzioni di sessione e presentazione necessarie. L'esperienza del modello OSI ha dimostrato la fondatezza di questa scelta: nella maggior parte dei casi tali livelli servono a poco.

Sopra il livello di trasporto c'è il **livello applicazione**, che contiene tutti i protocolli di livello superiore. I primi gestivano un terminale virtuale (TELNET), lo scambio dei file (FTP) e la posta elettronica (SMTP). Con gli anni sono stati aggiunti molti altri protocolli. Alcuni di questi,

mostrati nella Figura 1.22, sono il Domain Name System (DNS) che fa corrispondere i nomi degli host ai loro indirizzi di rete; HTTP, il protocollo per prelevare pagine sul World Wide Web e RTP, il protocollo per trasmettere contenuti in tempo reale come la voce e i film.

Il modello usato in questo libro

Come accennato prima, la forza del modello di riferimento OSI risiede nel modello stesso (ad esclusione dei livelli di presentazione e sessione), che si è dimostrato eccezionalmente utile nelle discussioni riguardanti le reti di calcolatori. Invece la forza del modello di riferimento TCP/IP sono i protocolli ampiamente utilizzati per molti anni. Siccome gli informatici amano farsi le cose da soli noi useremo come ambiente per il nostro libro il modello ibrido della Figura 1.23.

5	Applicazione
4	Trasporto
3	Rete
2	Link
1	Fisico

Figura 1.23 Il modello di riferimento usato in questo libro.

Questo modello ha cinque livelli che vanno dal livello fisico attraverso quelli di link, rete, trasporto fino al livello applicazione. Il livello fisico specifica come trasmettere bit su differenti tipi di mezzi di trasporto in forma di segnali elettrici (o altri analogici). Il livello link è coinvolto quando bisogna spedire messaggi di lunghezza finita direttamente tra computer connessi con un livello prefissato di affidabilità. Ethernet e 802.11 sono esempi di protocollo a livello link.

Il livello di rete tratta come combinare link multipli nelle reti e nelle internetwork in modo da poter spedire pacchetti tra computer distanti. Questo include il compito di trovare un percorso attraverso cui mandare i pacchetti. IP rimane l'esempio principale di protocollo che studieremo per questo livello. Il livello di trasporto si occupa di dare garanzie di consegna al livello di rete aumentandone l'affidabilità e fornisce astrazioni sulle modalità di consegna dei byte, che possono anche essere visti come una sequenza ordinata e affidabile (byte stream). TCP è un importante esempio di protocollo di livello di trasporto.

Infine, il livello applicazione contiene programmi che usano la rete. Molte, ma non tutte, le applicazioni di rete hanno un'interfaccia utente come quella dei browser Web. Noi ci occupiamo comunque di quella parte del programma che usa la rete. Nel caso dei browser Web questo è il protocollo HTTP. Esistono anche importanti programmi di supporto nel livello applicazione, quali il DNS, usato frequentemente.

La sequenza dei capitoli è basata su questo modello, così conserviamo il valore del modello OSI per capire le architetture di reti, concentrandoci soprattutto sui protocolli che sono importanti nella pratica, dal TCP/IP e i protocolli a esso collegati fino ai più recenti come 802.11, SONET e Bluetooth.

Confronto tra i modelli di riferimento OSI e TCP/IP

I modelli di riferimento OSI e TCP/IP hanno molto in comune. Sono entrambi basati sul concetto di pila (stack) di protocolli indipendenti, e la funzione dei livelli è grosso modo simile. Per esempio, in entrambi i modelli i livelli di trasporto e superiori forniscono ai processi che vogliono comunicare un servizio di trasmissione end-to-end e indipendente dalla rete; formano cioè il mezzo di trasporto. Di nuovo, in entrambi i modelli i livelli che stanno sopra a quello di trasporto ne rappresentano gli utenti e sono orientati alle applicazioni.

Nonostante queste somiglianze fondamentali, i due modelli di riferimento hanno molte differenze che esamineremo in questo paragrafo. È importante notare che il confronto è fatto tra modelli di riferimento e non tra le corrispondenti pile di protocolli. I protocolli veri e propri saranno infatti discussi in seguito. Un intero libro dedicato al confronto tra TCP/IP e OSI è (Piscitello e Chapin, 1993).

Nel modello OSI sono presenti tre concetti essenziali:

1. servizi
2. interfacce
3. protocolli.

Probabilmente il contributo maggiore del modello OSI è la sua capacità di esplicitare la distinzione tra questi concetti. Ogni livello offre un servizio a quello che lo sovrasta; la definizione del servizio descrive ciò che fa il livello e non le modalità d'accesso da parte delle entità sovrastanti o quelle di funzionamento; definisce cioè la semantica del livello.

L'interfaccia di un livello spiega le modalità di accesso ai processi sovrastanti. Specifica quali sono i parametri e i risultati, ma non dice nulla delle modalità di funzionamento interno.

Infine, l'essenza fondamentale di un livello sono i protocolli paritetici (*peer protocol*) che vengono impiegati al suo interno. Il livello può usare i protocolli che preferisce, se portano ai risultati desiderati (cioè svolgono i servizi offerti) e li può cambiare senza disturbare il software dei livelli superiori.

Queste idee si adeguano alla perfezione ai moderni concetti sulla programmazione orientata agli oggetti. Un oggetto, come un livello, ha un insieme di metodi (operazioni) invocati dai processi esterni all'oggetto. La semantica di questi metodi definisce l'insieme di servizi offerti dall'oggetto, mentre i parametri dei metodi e i risultati formano la sua interfaccia. Il codice interno all'oggetto è il suo protocollo, invisibile e ininfluente per ciò che si trova all'esterno dell'oggetto.

Il modello TCP/IP in origine non faceva una netta distinzione tra servizio, interfaccia e protocollo, anche se molti hanno tentato di aggiungere in un secondo tempo questi concetti per renderlo più simile al modello OSI. Per esempio, gli unici veri servizi offerti dal livello internet sono SEND IP PACKET e RECEIVE IP PACKET. La conseguenza è che nel modello OSI i protocolli sono

nascosti meglio che nel modello TCP/IP e si possono sostituire con relativa facilità all'evolvere della tecnologia. La capacità di operare cambiamenti di questa portata è uno degli scopi principali dell'esistenza dei protocolli stratificati.

Il modello di riferimento OSI è stato concepito *prima* d'inventare i protocolli corrispondenti, quindi non è orientato verso un insieme specifico di protocolli, cosa che lo rende decisamente generico. L'altra faccia della medaglia è che i progettisti non avevano molta esperienza e neppure precise idee sulle funzionalità da inserire in ciascuno livello.

Per esempio, il livello data link in origine copriva solo il caso di reti punto a punto, e quando nacquero le reti broadcast fu necessario incastrare nel modello un nuovo sottolivello. Quando s'iniziarono a costruire vere reti usando il modello OSI e i suoi protocolli, si scoprì che non soddisfacevano le specifiche di servizio richieste; fu quindi necessario integrare il modello con sottolivelli di convergenza per appianare le differenze. Per colmare la misura, il comitato originariamente aveva supposto che ogni paese avrebbe avuto una sola rete, gestita dal governo e basata sui protocolli OSI, quindi non venne dato peso all'internetworking. In sintesi, le cose hanno preso una piega diversa.

Nel caso del TCP/IP si è verificato tutto l'opposto: prima arrivarono i protocolli, e il modello realizzato rappresentava proprio una semplice rappresentazione dei protocolli esistenti. Non ci fu alcun problema per far aderire i protocolli al modello al quale corrispondevano perfettamente. L'unico ostacolo era che il modello non corrispondeva ad alcun altra pila di protocolli e quindi era poco utile per descrivere altre reti non basate su TCP/IP. Passando da questioni filosofiche ad argomenti più specifici, una differenza ovvia tra i due modelli consiste nel numero di livelli: il modello OSI ne ha sette, TCP/IP quattro. Entrambi hanno livelli di rete, trasporto e applicazione, ma gli altri sono diversi.

Un'altra differenza consiste nella modalità di comunicazione, orientata o meno alla connessione. Il modello OSI supporta nel livello di rete entrambi i tipi di comunicazione, ma nel livello di trasporto (che è quello che conta, perché il servizio di trasporto è visibile agli utenti) supporta solo la comunicazione orientata alla connessione. Il modello TCP/IP ha solo una modalità nel livello di rete (senza connessione), ma supporta entrambe in quello di trasporto offrendo così agli utenti una vera scelta, di grande importanza per i semplici protocolli richiesta-risposta.

Critica del modello e dei protocolli OSI

Imperfezioni sono presenti sia nei modelli e protocolli OSI, sia in quelli TCP/IP e per questo motivo entrambi sono stati bersaglio di critiche accese che esamineremo in questo paragrafo e nel successivo, iniziando da OSI.

Quando fu pubblicata la seconda edizione di questo libro (1989) molti esperti del settore erano dell'opinione che il modello OSI e i corrispondenti protocolli avrebbero dominato la scena mondiale, spazzando via tutto il resto. Perché non è successo? È utile gettare uno sguardo indietro per capire ciò che ha decretato la sconfitta di OSI e cioè:

- 1. scarso tempismo
- 2. tecnologia scadente
- 3. implementazioni scadenti
- 4. incapacità politica.

Scarso tempismo

Iniziamo dal primo motivo: la pessima scelta dei tempi. Per il successo di uno standard è d'importanza vitale l'epoca in cui è messo a punto. David Clark del M.I.T. (Massachusetts Institute of Technology) ha una teoria sugli standard che chiama *apocalisse dei due elefanti*, illustrata nella Figura 1.24. Questa figura mostra l'entità dell'attività che circonda un nuovo argomento. Quando viene scoperto c'è un brulicare di attività di ricerca che si manifesta in dibattiti, articoli e incontri. Dopo un po' questa attività scema, le grandi aziende scoprono l'argomento e partono gli investimenti da miliardi di dollari.

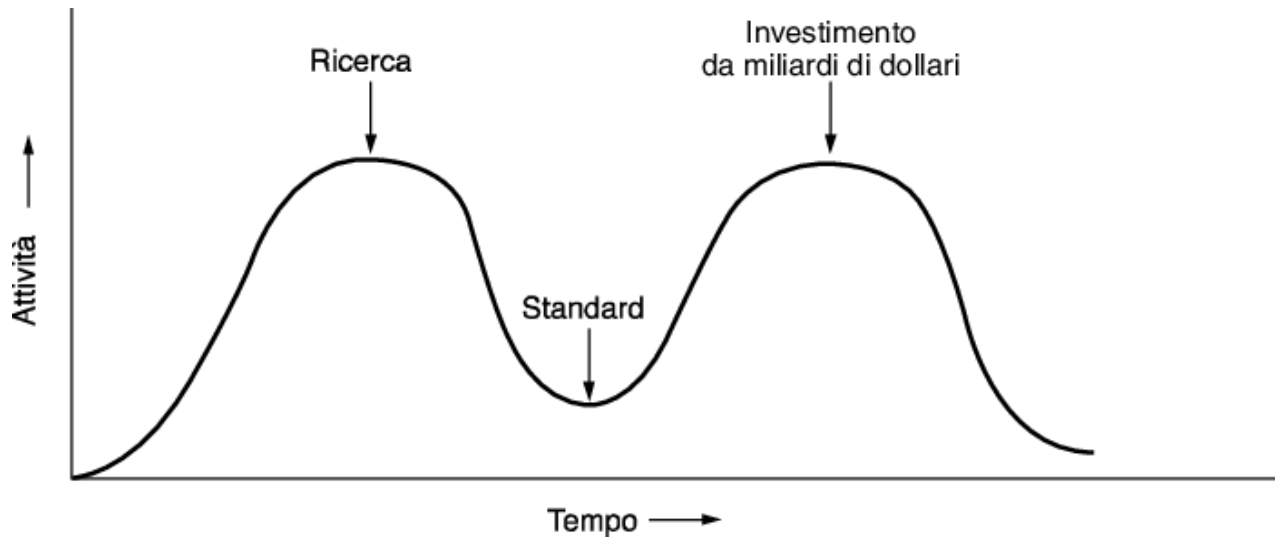


Figura 1.24 L'apocalisse dei due elefanti.

È essenziale che gli standard siano scritti nell'intervallo tra i due "elefanti". Se gli standard sono scritti troppo presto, prima del termine delle ricerche, l'argomento potrebbe essere ancora poco conosciuto e si otterrebbero standard scadenti. Se vengono scritti troppo tardi, un numero eccessivo di aziende potrebbe aver già fatto investimenti sostanziali nelle diverse modalità di realizzare le cose, quindi gli standard verrebbero ignorati. Se l'intervallo tra i due elefanti è molto breve (perché tutti sono ansiosi di partire), chi sviluppa gli standard potrebbe essere sopraffatto.

Ormai è evidente che i protocolli standard OSI sono stati polverizzati. Quando sono apparsi, le controparti TCP/IP erano già largamente impiegate dalle università che facevano ricerca. Anche se l'onda degli investimenti milionari non era ancora arrivata, il mercato accademico era già abbastanza grande da spingere molti costruttori a offrire cautamente prodotti TCP/IP. Quando OSI è arrivato, non c'è stata disponibilità per supportare una seconda pila di protocolli (salvo quando il costruttore è stato costretto), quindi è mancata l'offerta iniziale. Mentre ogni produttore aspettava la prima mossa dell'altro, nessuno è partito e OSI è svanito.

Tecnologia scadente

Il secondo motivo che ha impedito il successo di OSI è che il modello e i protocolli presentano dei difetti. La scelta di sette livelli fu più politica che tecnica: due livelli (sessione e presentazione) sono quasi vuoti, mentre altri due (data link e rete) sono sovraccarichi.

Il modello OSI con le corrispondenti definizioni di servizi e protocolli ha una complessità straordinaria: se vengono messe una sull'altra, le copie stampate degli standard formano una pila di carta alta quasi un metro; per giunta sono difficili da implementare e inefficienti. Viene in mente una freddura di Paul Mockapetris citata in (Rose, 1993):

Domanda: "Che cosa si ottiene incrociando un mafioso con uno standard internazionale"?

Risposta: "Qualcuno che ti fa un'offerta che non puoi capire".

OSI non ha solo il problema di essere incomprensibile: alcune funzioni come l'indirizzamento, il controllo di flusso e quello di errore riappaiono più e più volte in ogni livello. Saltzer et al. (1984), per esempio, hanno segnalato che per essere efficace, il controllo di errore va fatto nel livello più alto; quindi ripeterlo in ogni livello è inutile e inefficiente.

Implementazioni scadenti

Vista l'enorme complessità del modello e dei protocolli, non fu una sorpresa che le implementazioni iniziali fossero enormi, scomode e lente. Tutti quelli che ci tentarono rimasero scottati. Non ci volle molto per creare l'associazione mentale tra "OSI" e "scarsa qualità". I prodotti migliorarono col tempo, ma la cattiva fama rimase.

Al contrario, una delle prime implementazioni di TCP/IP faceva parte dello UNIX di Berkeley ed era decisamente buona (e gratuita). Venne rapidamente adottata, sollecitando la formazione di una grande comunità di utenti, che stimolò miglioramenti destinati ad ampliare ancora di più la comunità. La spirale era in ascesa invece che in discesa.

Incapacità politica

In seguito all'implementazione iniziale, molti (soprattutto in ambiente accademico) ritenevano TCP/IP parte integrante di UNIX, e nell'ambiente accademico degli anni '80 UNIX era amato come la mamma e la torta di mele.

OSI, d'altro canto, era largamente ritenuto la creatura dei ministeri delle telecomunicazioni europei, della Comunità Europea, e più tardi del governo USA. L'impressione era in parte vera, ma evocò l'immagine di un gruppo di burocrati che cercavano di cacciare in gola ai poveri ricercatori e programmatori uno standard mediocre, ostacolando gli sforzi per ottenere qualcosa che funzionasse davvero. C'è chi vide in questo sviluppo la stessa immagine degli annunci di metà degli anni '60 dove IBM proclamava che PL/I era il linguaggio del futuro o quando il dipartimento della difesa USA più tardi correggeva il tiro affermando che invece era Ada.

Critica del modello di riferimento TCP/IP

Anche il modello e i protocolli TCP/IP hanno i loro problemi. Innanzitutto, il modello non distingue in modo chiaro i concetti di servizio, interfaccia e protocollo. Un buon approccio all'ingegneria del software impone di tenere distinte le specifiche dall'implementazione, cosa che

OSI segue alla lettera al contrario di TCP/IP. La conseguenza è che il modello TCP/IP serve a poco se bisogna progettare reti basate su tecnologie nuove.

In secondo luogo, il modello TCP/IP è poco generale e inadatto per descrivere pile di protocolli diverse dal TCP/IP: per esempio, è assolutamente impossibile usare il modello TCP/IP per descrivere Bluetooth.

Terzo, il livello link non è un vero e proprio livello, inteso con il significato che ha nel contesto dei protocolli stratificati. È piuttosto un'interfaccia tra i livelli di rete e link. La distinzione tra interfaccia e livello è cruciale e non dovrebbe essere trascurata.

Quarto, il modello TCP/IP non fa distinzione (né menziona) i livelli fisico e link, totalmente diversi tra loro. Il livello fisico ha a che fare con le caratteristiche trasmissive di cavi in rame, fibre ottiche e comunicazioni wireless. Il compito del livello link è quello di delimitare inizio e fine dei frame e spostarli da una parte all'altra con il livello di affidabilità richiesto. Un buon modello dovrebbe includerli come livelli separati, al contrario di TCP/IP.

Infine, anche se i protocolli IP e TCP furono concepiti con attenzione e implementati bene, sebbene non realizzati con la stessa dedizione, molti altri protocolli risolvono problemi ad hoc (e di solito si tratta di prodotti di dottorandi che facevano tentativi finché non si stancavano). Le implementazioni del protocollo erano quindi distribuite gratuitamente, diventando largamente utilizzate e molto radicate, quindi difficili da sostituire. Oggi alcune sono un po' imbarazzanti. Un esempio è il protocollo per l'emulazione terminale virtuale, TELNET, progettato per un terminale Teletype meccanico[3] da dieci caratteri al secondo. Nonostante non sia dotato di interfaccia grafica e di mouse, 30 anni dopo è ancora ampiamente usato.

1.5 Esempi di reti

Esistono molti tipi di rete, piccole e grandi, note e meno note, con diversi obiettivi, scale e tecnologie. Nei paragrafi seguenti esamineremo qualche esempio per avere un'idea della varietà di situazioni coperte dall'argomento delle reti di calcolatori.

Inizieremo con Internet, probabilmente la rete più nota, esaminando la sua storia, evoluzione e tecnologia. Successivamente esamineremo la rete cellulare, tecnicamente molto diversa da Internet. Passeremo poi a IEEE 802.11, lo standard per LAN wireless. Infine ci occuperemo degli RFID e delle reti di sensore, tecnologie che accrescono la ricchezza della rete includendo il mondo reale e gli oggetti di uso quotidiano.

Internet

Internet non è affatto una rete, ma una vasta raccolta di reti diverse che usano determinati protocolli e offrono certi servizi comuni. È un sistema inconsueto, che non ha un progettista e non è controllato da nessuno. Per comprenderlo meglio, partiamo dall'inizio e vediamo come e perché si è sviluppato. Per una splendida storia di Internet raccomando il libro di John Naughton (2000), uno dei pochi testi che non è solo divertente da leggere, ma che ha anche 20 pagine di citazioni bibliografiche per stimolare i lettori più curiosi e appassionati; una parte del materiale che segue è tratta da questo libro.

Ovviamente, è stata scritta un'infinità di libri su Internet e i suoi protocolli. Per ulteriori informazioni si veda per esempio (Maufer, 1999).

ARPANET

La storia inizia alla fine degli anni '50. Nel pieno della guerra fredda, il dipartimento della difesa USA commissiona una rete di controllo che possa sopravvivere a una guerra nucleare. All'epoca, tutte le comunicazioni militari usavano la rete telefonica pubblica, considerata vulnerabile: il motivo di questo timore si può capire osservando la Figura 1.25(a). I punti neri rappresentano le centrali di commutazione telefonica, ognuna delle quali era connessa a migliaia di telefoni. A loro volta queste centrali erano collegate da centrali di commutazione di livello superiore per formare una gerarchia nazionale con poca ridondanza. La vulnerabilità del sistema stava nel fatto che la distruzione di poche centrali di alto livello avrebbe frammentato la rete in molte isole separate.

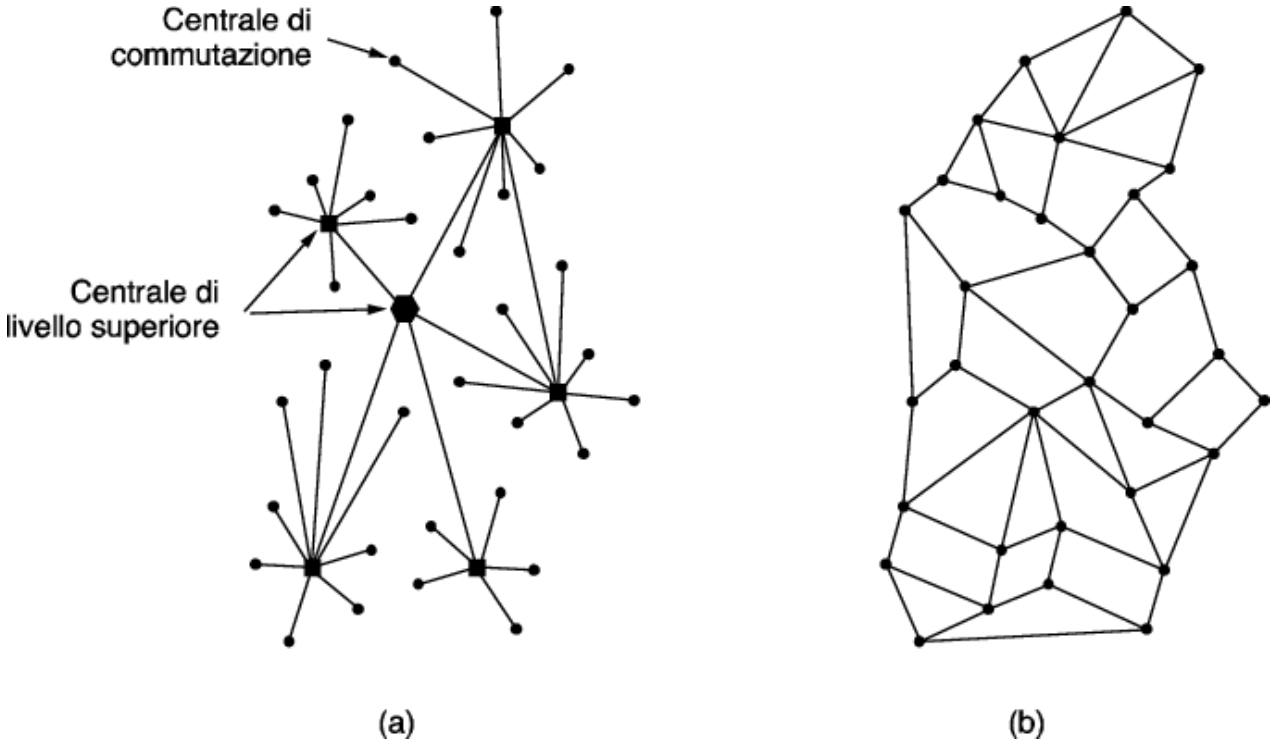


Figura 1.25 (a) Struttura della rete telefonica. (b) Il sistema distribuito a commutazione proposto da Baran.

Attorno al 1960, il dipartimento della difesa assegnò il compito di trovare la soluzione alla RAND Corporation. Uno dei suoi impiegati, Paul Baran, sviluppò il progetto altamente distribuito e resistente ai guasti rappresentato dalla Figura 1.25(b). Poiché il percorso tra due centrali di commutazione qualunque era maggiore rispetto a quello che i segnali analogici potevano percorrere senza distorsioni, Baran propose di usare una tecnologia digitale a commutazione di pacchetto. Baran scrisse diverse relazioni per il dipartimento della difesa descrivendo nei dettagli le sue idee, apprezzate dagli ufficiali del Pentagono spingendoli a chiedere alla AT&T (che allora gestiva in monopolio la rete telefonica USA) di costruire un prototipo. AT&T respinse in pieno le idee di Baran. La più grande e potente corporation del mondo non avrebbe consentito a qualche giovane esaltato di darle lezioni su come costruire una rete telefonica. Dissero che la rete di Baran non si poteva costruire e il caso fu chiuso.

Passò qualche anno e il dipartimento della difesa continuava a non avere una rete di comando e controllo migliore. Per capire ciò che successe in seguito, dobbiamo tornare all'ottobre 1957 quando l'Unione Sovietica batté gli USA nella corsa alla conquista dello spazio con il lancio del primo satellite artificiale, Sputnik. Quando il presidente Eisenhower cercò di capire chi aveva "poltrito", rimase sconvolto scoprendo come esercito, marina e aviazione si contendessero il budget per la ricerca del Pentagono. La sua risposta immediata fu la creazione di una singola organizzazione per la ricerca, **ARPA** (*advanced research projects agency*). ARPA non disponeva di scienziati o laboratori; in effetti non aveva altro che un ufficio e un piccolo (per gli standard del Pentagono) budget. Il suo compito era erogare fondi e stipulare contratti con università e aziende che avevano idee promettenti. ARPA spese i primi anni cercando di capire quale doveva essere la

sua missione, ma nel 1967 l'attenzione dell'allora direttore di ARPA (Larry Roberts) si diresse verso le reti, e contattò molti esperti per decidere che cosa fare. Uno di essi, Wesley Clark, suggerì di costruire una sottorete a commutazione di pacchetto, connettendo ogni host a un proprio router.

Dopo lo scetticismo iniziale, Roberts sposò l'idea e presentò un articolo generico sull'argomento all'ACM SIGOPS (*symposium on operating system principles*) che si tenne a Gatlinburg in Tennessee nel tardo 1967 (Roberts, 1967). Con sorpresa di Roberts, un altro articolo presentato durante la conferenza descriveva un sistema analogo che non solo era stato progettato, ma persino costruito sotto la direzione di Donald Davies al National Physical Laboratory in Gran Bretagna. Il sistema NPL non era a livello nazionale (connetteva solo alcuni computer nel campus di NPL), ma dimostrava che la tecnologia a commutazione di pacchetto poteva funzionare e inoltre citava il lavoro precedente di Baran. Roberts tornò da Gatlinburg intenzionato a costruire ciò che successivamente divenne noto con il nome **ARPANET**.

La sottorete sarebbe stata composta da minicomputer chiamati **IMP** (*interface message processors*) collegati da linee di trasmissione a 56 kbps. Per raggiungere un'alta affidabilità ogni IMP doveva essere collegato ad almeno altri due IMP; la sottorete era basata su datagrammi, così in caso di distruzione di alcune linee o IMP i messaggi sarebbero stati automaticamente instradati su percorsi alternativi.

Ogni nodo della rete doveva consistere in un IMP e un host, posti nella stessa stanza e collegati da un cavo corto. Un host avrebbe potuto mandare messaggi con lunghezza massima di 8.063 bit al suo IMP che li avrebbe suddivisi in pacchetti di massimo 1.008 bit per inoltrarli indipendentemente verso la destinazione. Ogni pacchetto sarebbe stato integralmente ricevuto prima dell'inoltro; quindi la sottorete fu la prima rete a commutazione di pacchetto del tipo store-and-forward.

ARPA bandì una gara d'appalto per costruire la sottorete, a cui parteciparono 12 società. Dopo aver valutato tutte le proposte, ARPA scelse BBN, una società di consulenza di Cambridge, Massachusetts, e nel dicembre 1968 le assegnò l'appalto per costruire la sottorete e scrivere il relativo software. BBN scelse come IMP dei minicomputer Honeywell DDP-316 appositamente modificati, con memoria a nuclei magnetici di 12 K organizzata in word di 16 bit, senza dischi, poiché le parti mobili erano considerate inaffidabili. Gli IMP erano collegati da linee affittate a 56 kbps che all'epoca erano quanto di meglio la tecnologia poteva proporre. Il software fu diviso in due parti: sottorete e host. Il software della sottorete era composto dalla parte lato IMP della connessione tra IMP e host, dal protocollo IMP-IMP e da un protocollo da IMP sorgente a IMP destinazione progettato per migliorare l'affidabilità. Il progetto originale di ARPANET è mostrato nella Figura 1.26.

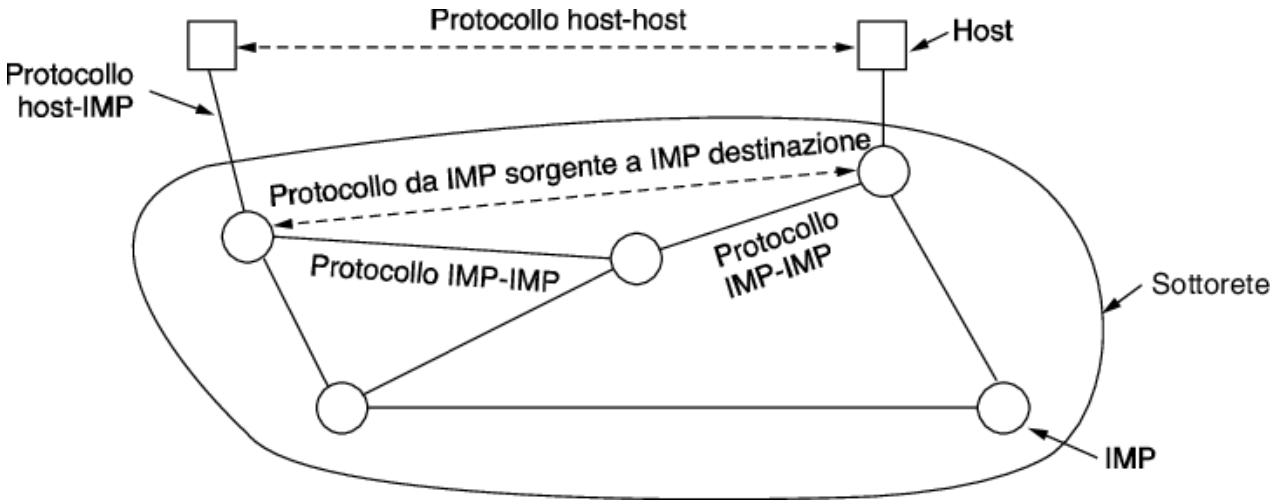


Figura 1.26 Il progetto originale di ARPANET.

Fuori dalla sottorete era necessario altro software, in particolare il lato host della connessione host-IMP, il protocollo host-host e il software applicativo. Divenne presto evidente che BBN riteneva concluso il suo compito con l'accettazione del messaggio sul cavo host-IMP sorgente e la sua consegna sul cavo host-IMP destinazione.

Roberts aveva un problema: anche all'host serviva del software. Per affrontarlo, organizzò a Snowbird, Utah, un incontro di ricercatori specializzati in reti (soprattutto dottorandi). Questi ultimi si aspettavano che qualche esperto di reti spiegasse loro i principi generali della rete e del suo software, assegnando a ciascuno il compito di scriverne una parte, e rimasero sbalorditi quando seppero che non c'era alcun esperto né progetto di massima: dovevano cavarsela da soli.

In qualche modo una rete sperimentale entrò in servizio nel dicembre 1969 con quattro nodi: UCLA, UCSB, SRI[4] e Università dello Utah, scelti perché ciascuno aveva un gran numero di contratti con ARPA e utilizzava computer host diversi e completamente incompatibili (giusto per divertirsi di più). La rete crebbe rapidamente man mano che altri IMP vennero consegnati e installati; presto coprì tutti gli Stati Uniti. La Figura 1.27 mostra la velocità della crescita di ARPANET nei primi tre anni.

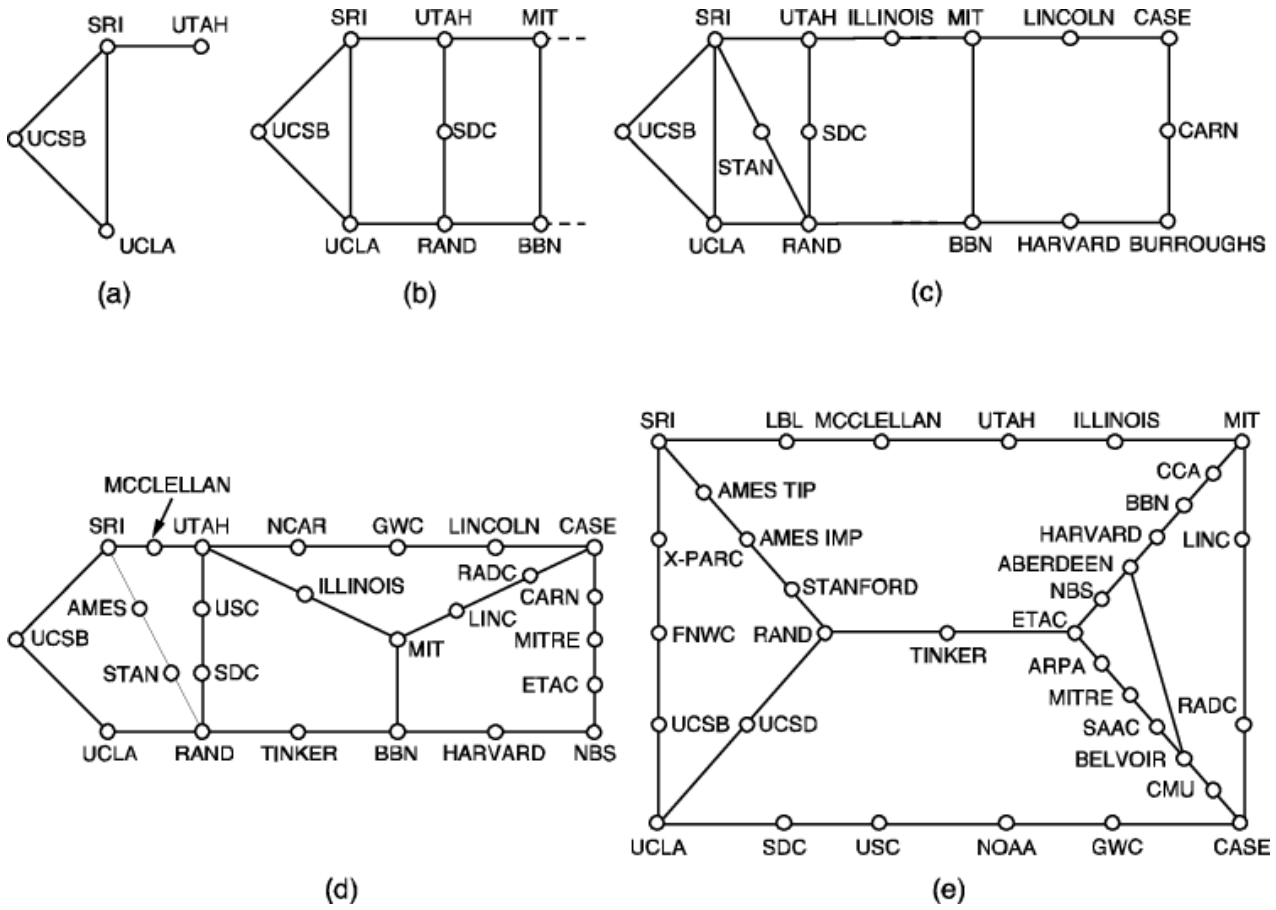


Figura 1.27 Crescita di ARPANET. (a) Dicembre 1969. (b) Luglio 1970. (c) Marzo 1971. (d) Aprile 1972. (e) Settembre 1972.

Oltre ad aiutare la crescita di ARPANET, ARPA finanziò ricerche sull'uso di reti satellitari e reti radiomobili a pacchetto. Durante una celebre dimostrazione, un camion in movimento situato in California usò la rete radiomobile a pacchetto per mandare messaggi a SRI, inoltrati su ARPANET fino alla costa Est, e quindi inviati alla University College di Londra tramite una rete satellitare. I ricercatori nel camion riuscirono a usare un computer a Londra mentre si muovevano per la California.

Questo esperimento dimostrò anche che i protocolli ARPANET esistenti non si potevano usare per reti multiple. Tale osservazione stimolò le ricerche sui protocolli, culminate con l'invenzione dei protocolli e del modello TCP/IP (Cerf e Kahn, 1974). Il modello TCP/IP fu specificamente progettato per gestire le comunicazioni sulle internetwork, che stavano crescendo d'importanza man mano che nuove reti venivano collegate ad ARPANET. Per incoraggiare l'adozione di questi nuovi protocolli, ARPA concesse molti contratti a BBN e all'Università della California a Berkeley per integrarli nello UNIX di Berkeley. I ricercatori di Berkeley svilupparono una pratica interfaccia di programmazione verso la rete (socket) e scrissero molte applicazioni, utility e programmi di gestione per semplificare la connessione.

La tempistica era perfetta. Molte università avevano appena acquistato un secondo o terzo computer VAX e una LAN per connetterli, ma non avevano software di rete. Quando uscì BSD 4.2 con TCP/IP, le socket e una montagna di utility per la rete, l'adozione dell'intero pacchetto fu

immediata. Inoltre, con TCP/IP diventava semplice collegare una LAN ad ARPANET, e molti lo fecero.

Nel corso degli anni '80 altre reti furono connesse ad ARPANET, soprattutto LAN. Al crescere delle dimensioni della rete rintracciare un host diventò sempre più difficoltoso, quindi fu creato il DNS (*domain name system*) per organizzare i computer in domini e abbinare i nomi degli host agli indirizzi IP. Da allora DNS si è evoluto in un database distribuito, a uso generale e capace di memorizzare una quantità d'informazioni relative ai nomi; lo studieremo nel Capitolo 7.

NSFNET

Negli ultimi anni '70 l'organismo statunitense NSF (*national science foundation*) si accorse dell'enorme impatto che ARPANET aveva sulla ricerca universitaria permettendo a scienziati di tutto il paese di condividere i dati e collaborare a progetti di ricerca. Tuttavia, per collegarsi ad ARPANET un'università doveva avere un contratto di ricerca aperto con il ministero della difesa, e molte non lo avevano. NSF inizialmente reagì fondando nel 1981 la **CSNET** (*computer science network*). Connetteva i dipartimenti di informatica e i laboratori industriali ad ARPANET via dial-up e linee in affitto. Negli ultimi anni '80 NSF progettò un successore di ARPANET che sarebbe stato aperto a tutti i gruppi di ricerca universitari.

Per partire da qualcosa di concreto, NSF decise di costruire una rete di dorsale (*backbone*) per collegare i suoi sei centri (che ospitavano supercomputer) di San Diego, Boulder, Champaign, Pittsburg, Ithaca e Princeton. A ogni supercomputer fu affiancato un fratellino, rappresentato da un microcomputer LSI-11 chiamato **fuzzball**. I fuzzball erano collegati tra loro con linee affittate a 56 kbps e formavano una sottorete, con la stessa tecnologia hardware usata da ARPANET. La tecnologia software era invece differente: i fuzzball usarono sin dall'inizio TCP/IP, costituendo così la prima WAN TCP/IP.

NSF finanziò anche circa 20 reti regionali che si collegavano al backbone per consentire agli utenti di migliaia di università, centri di ricerca, biblioteche e musei l'accesso ai supercomputer, oltre che permettere la comunicazione reciproca. L'intera rete, composta da backbone e reti regionali, fu chiamata **NSFNET**. Il collegamento con ARPANET era realizzato da una connessione tra un IMP e un fuzzball situati nella sala macchine della Carnegie Mellon University. La prima dorsale NSFNET è mostrata nella Figura 1.28 sulla mappa degli Stati Uniti.

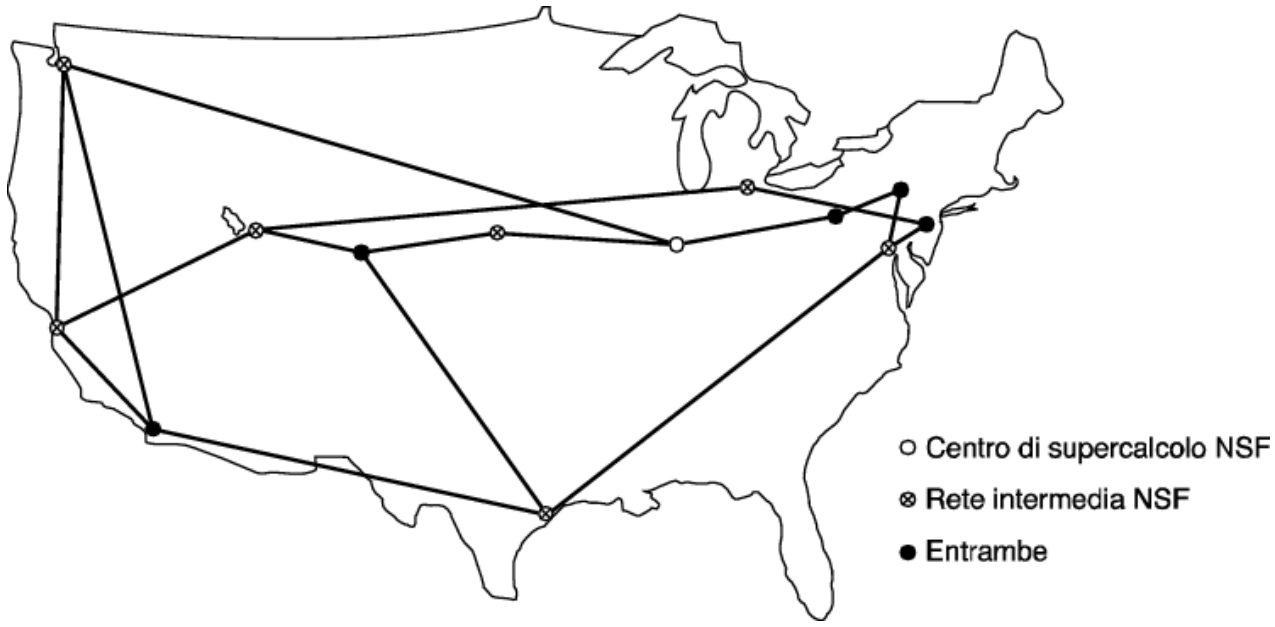


Figura 1.28 La dorsale di NSFNET nel 1988.

NSFNET ebbe un successo istantaneo e risultò sovraccarica sin dal primo giorno. NSF iniziò immediatamente a progettare il suo successore e assegnò l'appalto per realizzarlo al consorzio MERIT situato nel Michigan. La dorsale di seconda generazione fu realizzata affittando da MCI (poi inglobata in WorldCom) canali in fibra ottica da 448 kbps, mentre come router vennero impiegati dei PC-RT IBM. Anche questa rete fu presto sovraccaricata e nel 1990 il secondo backbone fu aggiornato a 1,5 Mbps.

Con il proseguire della crescita NSF si rese conto che il governo non poteva finanziare le reti per sempre; inoltre avrebbero voluto collegarsi anche organizzazioni commerciali, ma non potevano farlo perché le regole di NSF vietavano l'uso della rete per attività estranee a quelle di NSF. Per questi motivi, come primo passo verso la commercializzazione, NSF incoraggiò MERIT, MCI e IBM a formare la società non-profit **ANS** (*advanced networks and services*). Nel 1990 ANS prese in carico NSFNET e aggiornò i collegamenti da 1,5 Mbps portandoli a 45 Mbps per formare **ANSNET**; questa rete restò operativa per cinque anni e venne poi venduta ad America Online. A quel punto molte società offrivano servizi IP commerciali, ed era chiaro che per il governo era arrivato il momento di uscire dal mercato delle reti.

Per facilitare la transizione e garantire che ogni rete regionale potesse comunicare con tutte le altre, NSF stipulò contratti con quattro operatori di rete perché ognuno realizzasse un **NAP** (*network access point*). Questi operatori furono PacBell (San Francisco), Ameritech (Cicago), MSF (Washington, D.C.) e Sprint (New York City, che a questo scopo era rappresentata da Pennsauken, New Jersey). Ogni operatore di rete che voleva offrire un servizio di backbone alle reti regionali NSF doveva collegarsi a tutti i NAP.

Questa soluzione garantì che un pacchetto originato da una qualsiasi rete regionale potesse scegliere l'operatore di backbone per transitare dal proprio NAP a quello di destinazione; gli operatori di rete quindi furono costretti a competere tra loro per conquistare i clienti rappresentati

dalle reti regionali, confrontandosi per qualità del servizio e prezzo. Di conseguenza, il concetto di un singolo backbone predefinito fu sostituito da una struttura competitiva guidata da logiche commerciali. Molti accusano il governo federale di essere poco innovativo, ma nel caso del networking furono proprio il dipartimento della difesa e NSF a creare l'infrastruttura che formò la base di Internet, per poi cederla all'industria.

Durante gli anni '90 molti altri paesi costruirono reti nazionali per la ricerca, spesso seguendo il modello di ARPANET e NSFNET. In Europa erano operative EuropaNET ed EBONE, che iniziarono con linee da 2 Mbps poi aggiornate a 34 Mbps. Anche l'infrastruttura di rete europea fu ceduta all'industria.

Internet è cambiata molto da quei primi tempi. La sua dimensione è esplosa nei primi anni '90 con l'avvento del World Wide Web. Dati recenti dell'Internet Systems Consortium parlano di più di 600 milioni di host Internet visibili. Questa è solo una stima per difetto, ma è di gran lunga superiore ai pochi milioni di host della prima conferenza sul WWW tenuta al CERN nel 1994.

Anche il modo in cui usiamo Internet è cambiato. All'inizio le applicazioni dominanti erano le e-mail nel mondo accademico, i newsgroup, le sessioni di lavoro remote e il trasferimento di file. Più tardi Internet venne usata dal grande pubblico per l'e-mail, la distribuzione di contenuti via Web e peer-to-peer, come Napster, che ora è stato chiuso. Attualmente le applicazioni in crescita vertiginosa sono la distribuzione di multimedia in tempo reale, i social network (Facebook) e i microblogging (Twitter). Questi cambiamenti hanno portato in Internet contenuti più ricchi e quindi più traffico. In effetti, il traffico dominante in Internet sembra cambiare con cadenza regolare; per esempio, nuovi e migliori modi per lavorare con la musica o i film potrebbero diffondersi molto presto portando applicativi con profili di traffico mai visti prima.

Architettura di Internet

Anche l'architettura di Internet è cambiata molto a causa della sua crescita dirompente. In questo paragrafo faremo un breve riepilogo dell'aspetto attuale di Internet. In seguito alle molte fusioni tra società telefoniche (telco), compagnie di televisione via cavo (cable company) e ISP, le acque si sono intorpidite e diventa sempre più difficile capire chi fa che cosa.

Uno dei motori di questo subbuglio è la convergenza dei servizi di telecomunicazioni, che porta a utilizzare una rete in modo sempre nuovo rispetto ai precedenti. Per esempio, un'azienda vi vende telefono, TV e servizio Internet sulla stessa connessione di rete, dando per scontato che questo vi farà risparmiare. Di conseguenza, la descrizione che daremo sarà necessariamente più semplice della realtà e ciò che è vero oggi potrebbe non esserlo più domani.

Lo schema di massima è mostrato nella Figura 1.29. Esaminiamola pezzo a pezzo, partendo da un computer domestico (ai lati della figura). Per collegarsi a Internet, il computer si connette a un **internet service provider**, o più semplicemente **ISP**, che fornisce all'utente l'**accesso a Internet** o **connettività**. Questo permette al computer di scambiare pacchetti con tutti gli altri host accessibili in Internet. L'utente potrebbe inviare pacchetti per navigare sul Web o per migliaia di altri motivi, non importa quali. Esistono molti tipi di accesso a Internet e di solito sono classificati in base alla banda che forniscono e al loro costo, anche se la caratteristica più importante è la connettività.

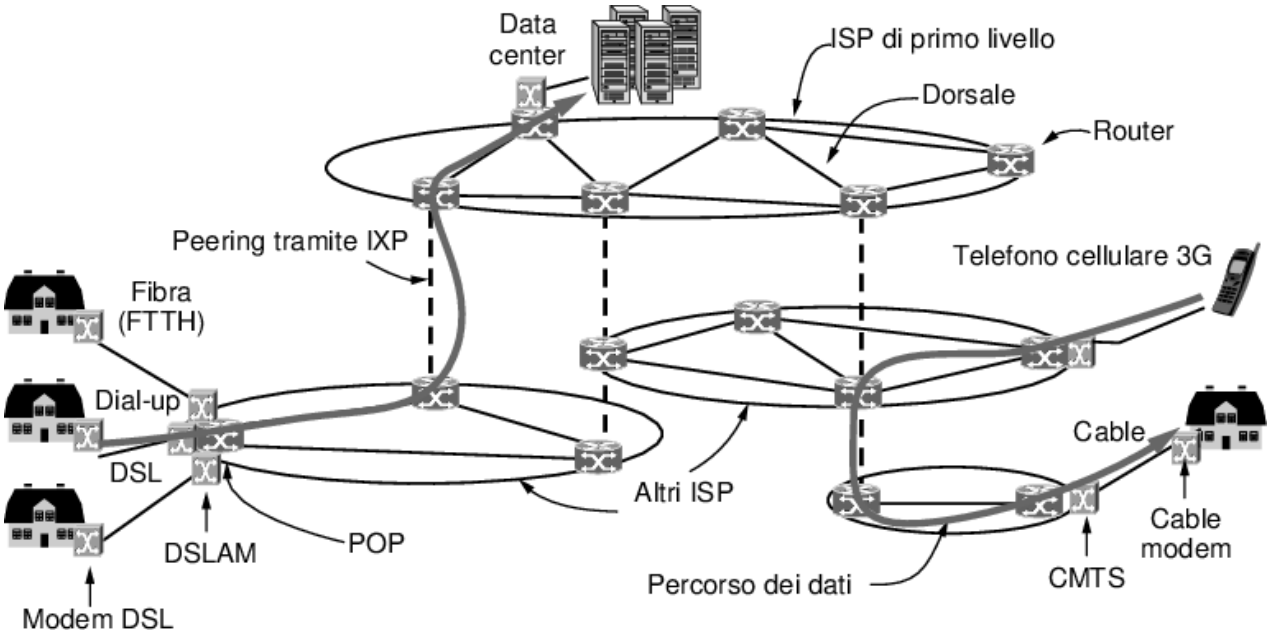


Figura 1.29 Schema dell'architettura di Internet.

Il modo comune di connettersi a un ISP è utilizzare la linea telefonica di casa, nel qual caso la compagnia telefonica è l'ISP stesso. La **DSL**, acronimo di **digital subscriber line**, riutilizza la linea telefonica di casa per la trasmissione di dati digitali. Il computer è connesso a un dispositivo chiamato **modem DSL** che effettua la conversione da pacchetti digitali a segnali analogici, che in questo modo passano sulla linea telefonica. Dall'altro capo, un dispositivo chiamato **DSLAM** (*digital subscriber line access multiplexer*) converte i segnali in pacchetti.

Nella Figura 1.29 sono presenti alcuni dei modi più diffusi per connettersi a un ISP. La DSL è un modo di usare la linea telefonica locale che fornisce una banda più larga del **dial-up**, che consiste nell'inviare bit su una chiamata telefonica tradizionale, al posto di una comunicazione vocale. Il **dial-up** viene effettuato con molti tipi di modem ai due capi. La parola **modem** è l'abbreviazione di *modulator demodulator* e si riferisce a qualunque dispositivo che converta bit digitali in segnali analogici.

Un altro metodo è quello di inviare segnali sul sistema della TV via cavo (*cable*). Anche questo, come la DSL, è un modo per riutilizzare un'infrastruttura esistente, in questo caso i canali della TV via cavo che altrimenti rimarrebbero inutilizzati. Il dispositivo collegato al capo situato in casa è chiamato **cable modem**, mentre il dispositivo all'altro capo (**cable headend**) è chiamato **CMTS** (*cable modem termination system*).

La DSL e il cable forniscono accesso Internet a tassi di trasmissione che vanno dalla piccola frazione di megabit/s a multipli di megabit/s, a seconda del sistema. Questi tassi (*rate*) sono molto più elevati di quelli del dial-up, limitati a 56 kbps a causa della banda stretta usata per le chiamate vocali. Un accesso Internet molto più veloce del dial-up è chiamato **broadband** (*a banda larga*). Il nome si riferisce alla banda più larga usata per le reti veloci piuttosto che a una particolare velocità.

I metodi di accesso menzionati sopra sono limitati dalla banda dell'"ultimo miglio" della trasmissione. Posando fibre ottiche sino ad arrivare nelle case, si otterrebbero accessi Internet più veloci, negli ordini di grandezza dai 10 ai 100 Mbps. Questa configurazione è chiamata **FTTH** (*fiber to the home*). Per un'azienda in un'area commerciale è ragionevole affittare linee di trasmissione ad alta velocità che colleghino gli uffici all'ISP più vicino. Per esempio, nel Nord America, una linea T3 viaggia a circa 45 Mbps.

Anche la modalità wireless viene utilizzata per l'accesso a Internet. Esamineremo l'esempio delle reti cellulari 3G che possono distribuire dati ai telefoni cellulari e agli utenti fissi nelle aree dotate di copertura a un tasso di 1 Mbps o più.

Siamo ora in grado di mandare pacchetti da casa all'ISP. Chiamiamo il punto in cui i pacchetti entrano nella rete ISP con il nome di **POP** (*point of presence*) dell'ISP. Spiegheremo in seguito come i pacchetti si muovono tra POP di diversi ISP. Da questo punto in poi il sistema è completamente digitale e a commutazione di pacchetto.

Le reti ISP possono essere regionali, nazionali o internazionali. Abbiamo già visto che la loro architettura è costituita da linee di trasmissione a lunga distanza che interconnettono POP in diverse città servite dall'ISP. Questo equipaggiamento è chiamato **backbone** (*dorsale*) dell'ISP. Se il pacchetto è destinato a un host servito direttamente dall'ISP, il pacchetto viene instradato sulla dorsale e consegnato all'host, altrimenti deve essere passato a un altro ISP.

Gli ISP interconnettono le loro reti per scambiarsi traffico tramite degli **IXP** (*Internet eXchange Point*). Si dice che gli ISP connessi fanno **peering** uno con l'altro. In molte città del mondo esistono degli IXP. Nella Figura 1.29 sono disegnati verticalmente perché le reti ISP si sovrappongono geograficamente. Semplicemente, un IXP è una stanza piena di router, almeno uno per ogni ISP. Una rete LAN, all'interno della stanza, connette tutti i router in modo che i pacchetti possano essere inoltrati dalla dorsale di un ISP a quella di un altro. Gli IXP possono essere grandi centri gestiti da terze parti. Uno dei più grandi IXP è l'Amsterdam Internet Exchange, a cui sono connessi centinaia di ISP che si scambiano centinaia di gigabit di traffico ogni secondo.

Il peering che avviene tramite gli IXP dipende dalle relazioni commerciali tra gli ISP. Per esempio, un piccolo ISP potrebbe pagare a un ISP più grande la connettività Internet per raggiungere host distanti esattamente come un consumatore acquisterebbe il servizio da un provider Internet. In questo caso si dice che il piccolo ISP paga per il **transito**. Oppure due ISP grandi potrebbero decidere di scambiarsi il traffico cosicché ognuno dei due possa consegnare un po' del suo traffico dati all'altro ISP senza dover pagare per il transito. Uno dei tanti paradossi di Internet è che gli ISP si fanno concorrenza pubblicamente per conquistare clienti ma spesso cooperano, in privato, facendo peering (Metz, 2001).

Il percorso di un pacchetto dipende dalle scelte di peering effettuate dagli ISP. Se l'ISP che deve consegnare il pacchetto ha un rapporto di peering con l'ISP destinatario, può consegnare il pacchetto direttamente; altrimenti, instrada il pacchetto verso il più vicino punto di connessione a un provider di transito a pagamento, che consegnerà il pacchetto. Nella Figura 1.29 sono mostrati due esempi di percorsi che attraversano degli ISP; spesso il percorso intrapreso da un pacchetto in Internet non è il più corto possibile.

In cima alla "catena alimentare" c'è un pugno di aziende, quali AT&T e Sprint, che dispongono di grandi dorsali internazionali con migliaia di router connessi da fibre ottiche a banda larga. Questi ISP, chiamati **tier 1** (di primo livello), non pagano il transito e formano una dorsale di Internet dato che chiunque altro deve collegarsi a loro per poter raggiungere il resto di Internet.

Le aziende che forniscono molti contenuti come Google e Yahoo! concentrano i loro calcolatori in **data center** ben connessi al resto di Internet. I data center, essendo progettati per ospitare computer, non persone, possono essere riempiti di armadi di calcolatori chiamati **server farm**. Data center per **colocation** o **hosting** permettono ai clienti di posizionare le loro attrezzature, quali i server, molto vicine ai POP degli ISP, in tal modo le connessioni tra server e dorsali sono corte e veloci. L'industria dell'hosting su Internet diventa sempre più virtualizzata, tanto che è ormai prassi comune affittare una macchina virtuale all'interno di una server farm, piuttosto che installare fisicamente un computer. Questi data center sono così grandi (decine o centinaia o migliaia di macchine) che il costo maggiore è quello dell'elettricità; per questo motivo vengono a volte costruiti in aree dove l'elettricità ha un costo basso.

Con questo concludiamo la nostra breve rassegna su Internet. Nei capitoli seguenti ne analizzeremo ampiamente ogni componente e la relativa progettazione, gli algoritmi e i protocolli. Un altro punto che val la pena di menzionare qui è che sta cambiando il senso di che cosa voglia dire essere in Internet. Si era soliti dire che una macchina era in Internet se: (1) implementava lo stack di protocolli TCP/IP; (2) possedeva un indirizzo IP; (3) poteva inviare pacchetti IP a qualunque altra macchina in Internet. Tuttavia, gli ISP spesso riutilizzano gli indirizzi IP sulla base dei computer attivi al momento e spesso le reti domestiche fanno condividere un indirizzo IP a molti computer. Questa procedura indebolisce la seconda condizione. Misure di sicurezza come i firewall possono bloccare parzialmente i pacchetti che un computer riceve, indebolendo la terza condizione. Nonostante queste difficoltà è sensato considerare queste macchine come appartenenti a Internet quando sono connesse al loro ISP.

Accenniamo inoltre brevemente al fatto che alcune aziende hanno interconnesso le loro reti private già esistenti ricorrendo spesso alla stessa tecnologia usata in Internet. Queste **intranet** sono tipicamente accessibili solo all'interno della sede aziendale o da computer portatili dell'azienda, ma per tutto il resto funzionano nello stesso modo di Internet.

La terza generazione di reti cellulari

La passione della gente di parlare al cellulare ancor più che navigare in Internet ha reso la rete cellulare la più popolare delle reti. Vanta più di quattro miliardi di iscritti nel mondo, vale a dire circa il 60% della popolazione mondiale, più della somma del numero di host in Internet e dei telefoni fissi (ITU, 2009).

A seguito alla sua enorme crescita, l'architettura della rete cellulare è parecchio cambiata negli ultimi 40 anni. I sistemi di telefonia cellulare della prima generazione o 1G trasmettevano le chiamate vocali come segnali analogici continui piuttosto che sequenze di bit (digitali). Il sistema **AMPS** (*advanced mobile phone system*), sviluppato negli Stati Uniti nel 1982, è stato il primo sistema ampiamente utilizzato. I sistemi cellulari di seconda generazione, o 2G, trasmettevano le chiamate vocali in forma digitale per accrescere la capacità e la sicurezza della rete, oltre che offrire un

servizio di messaggistica. Il **GSM** (*global system for mobile communications*), che venne installato a partire dal 1991 ed è diventato il sistema di telefonia cellulare più usato nel mondo, è un sistema 2G.

I sistemi di terza generazione, o 3G, vennero per la prima volta installati nel 2001 e offrono servizi sia di chiamate vocali sia di trasmissione dati digitali a banda larga. Sono portatori di un ampio gergo tecnico e molti standard diversi tra cui scegliere. Il sistema 3G è definito in maniera poco formale dall'ITU (un organismo internazionale per gli standard di cui discuteremo nel prossimo paragrafo) come un sistema che fornisce tassi di trasmissione di almeno 2 Mbps per utenti fermi o che camminano e 384 kbps per veicoli in movimento. L'**UMTS** (*universal mobile telecommunications system*), chiamato anche **WCDMA** (*wideband code division multiple access*), è il principale sistema 3G che si sta velocemente installando a livello mondiale. Può fornire fino a 14 Mbps in downlink (in download, dalla rete verso il cellulare) e circa 6 Mbps in uplink (in upload, dal cellulare verso la rete). Le versioni future utilizzeranno più antenne per offrire agli utenti velocità ancora maggiori.

La risorsa più scarsa nei sistemi 3G, così come in quelli 2G e 1G precedenti, è lo spettro radio. I governi concedono le licenze di parte dello spettro radio agli operatori di rete cellulare tramite gare d'appalto. Il possesso della licenza per una parte dello spettro facilita l'operatore nella progettazione dei sistemi, poiché nessun altro può trasmettere in quello spettro, ma risulta estremamente costoso. Per esempio, in Inghilterra nel 2000 in una gara d'appalto vennero vinte cinque licenze 3G per 40 miliardi di dollari.

Fu proprio questa scarsità di spettro che portò al progetto di **rete cellulare**, ora in uso, mostrato nella Figura 1.30. Per gestire l'interferenza radio tra gli utenti l'area di copertura viene divisa in celle; all'interno di una cella agli utenti vengono assegnati canali che non interferiscono tra di loro e che non sono causa di interferenze eccessive per le celle adiacenti. Questo permette un buon riutilizzo dello spettro in celle adiacenti (*frequency reuse*) e accresce la capacità della rete. Nei sistemi 1G che trasmettevano ogni chiamata vocale su una specifica frequenza, le frequenze erano accuratamente scelte in modo che non interferissero con le celle adiacenti. In questo modo una certa frequenza potrebbe essere riutilizzata solo una volta in un certo numero di celle. I moderni sistemi 3G permettono a ogni cella di usare tutte le frequenze, ma in modo tale che il livello di interferenza tra celle adiacenti risulti tollerabile. Benché ci siano alcune varianti possibili nel progettare la rete cellulare, quali l'uso di antenne direzionali o antenne in grado di dividere la cella in settori per ridurre ulteriormente le interferenze, l'idea di base è sempre la stessa.

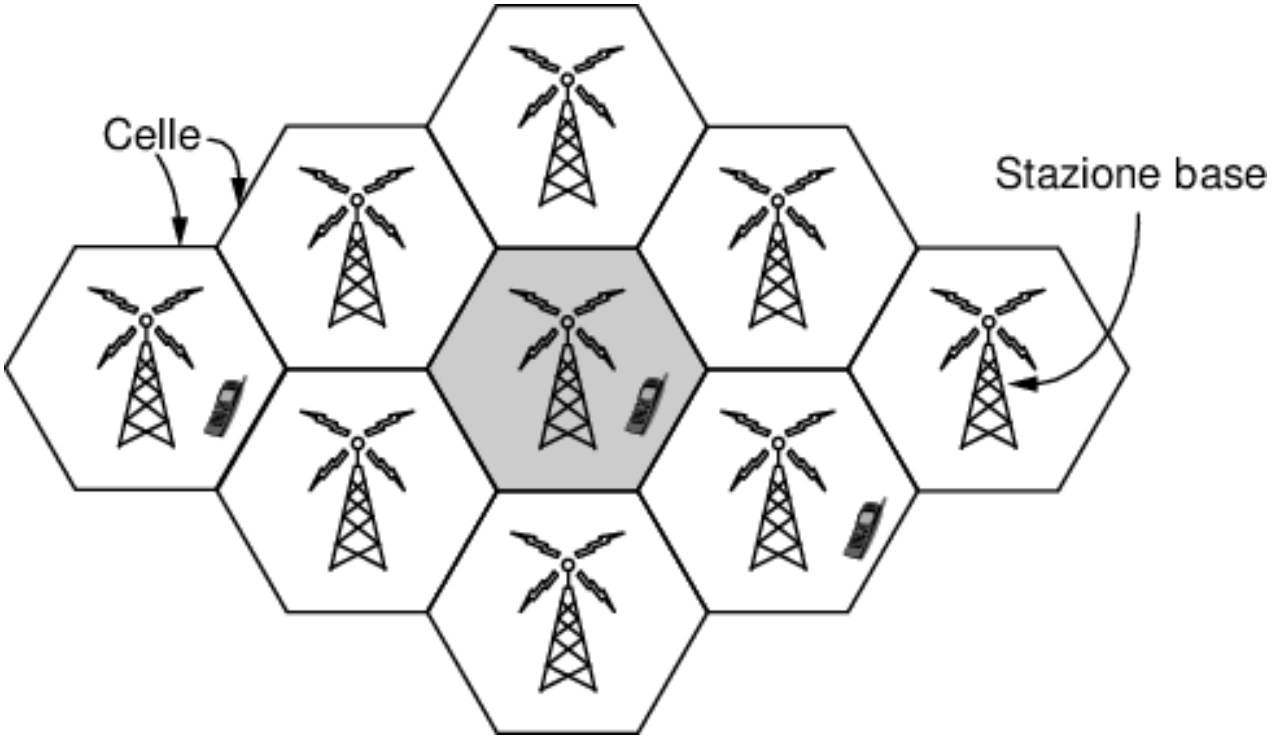


Figura 1.30 Schema a celle di una rete per telefonia mobile.

L'architettura di una rete cellulare è molto diversa da quella di Internet. È composta di più parti; una versione semplificata dell'architettura UMTS è mostrata nella Figura 1.31. In primo luogo l'**air interface**: un nome fantasioso per indicare il protocollo di comunicazione radio usato nell'etere tra il dispositivo mobile (telefono cellulare) e la **stazione base cellulare**. I progressi nel campo delle air interface degli ultimi decenni hanno enormemente aumentato il tasso di trasferimenti dati wireless. L'UMTS air interface è basata sul **CDMA** (*code division multiple access*), una tecnica che studieremo nel Capitolo 2.

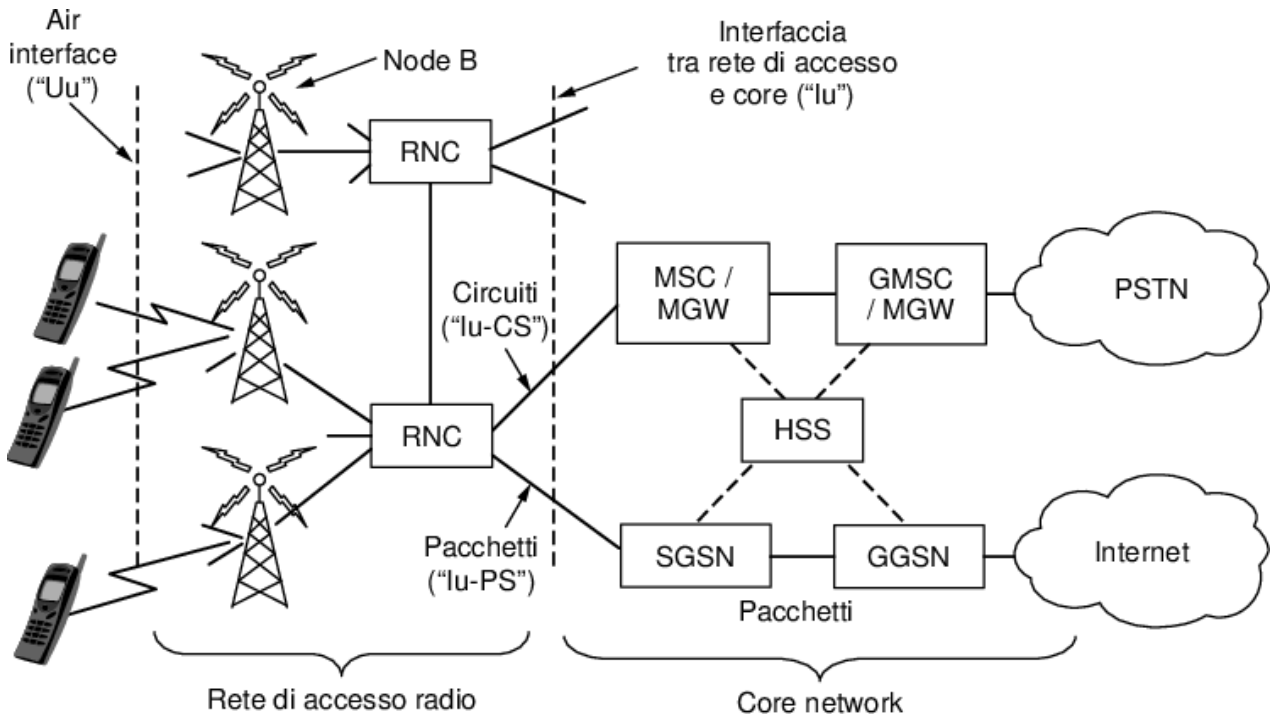


Figura 1.31 Architettura della rete UMTS.

La stazione base cellulare insieme al suo controller formano la **rete di accesso radio**. Questo è il lato wireless della rete cellulare. Il nodo controller o **RNC** (*radio network controller*) gestisce l'uso dello spettro. La stazione base implementa la air interface. Viene chiamata **node B**, un'etichetta temporanea che non è più stata abbandonata.

Il resto della rete cellulare trasporta il traffico per la rete di accesso radio ed è chiamato **core network**. Il core UMTS è l'evoluzione di quello usato per il sistema GSM 2G precedente. Tuttavia nel core network UMTS c'è qualcosa di sorprendente.

Sin dall'inizio delle reti è sorta una disputa tra chi preferiva le reti a commutazione di pacchetto (sottorete senza connessione) e chi quelle a commutazione di circuito (sottorete orientata alla connessione). I fautori della prima alternativa provengono perlopiù dalla comunità di Internet. In uno schema senza connessioni ogni pacchetto è instradato in modo indipendente da tutti gli altri. Di conseguenza, se alcuni router cessassero di funzionare durante una sessione, non ci sarebbero danni fintantoché il sistema fosse in grado di auto riconfigurare in modo tale che i pacchetti successivi possano trovare un percorso verso la destinazione, anche se diverso da quello intrapreso dai pacchetti precedentemente inviati.

L'idea dei sistemi a commutazione di circuito proviene invece dal mondo delle compagnie telefoniche. In un sistema telefonico chi chiama deve comporre il numero del ricevente e aspettare che si attivi una connessione prima di parlare o spedire dati. La creazione della connessione stabilisce un percorso attraverso il sistema telefonico che viene mantenuto finché la chiamata non è terminata. Tutte le parole o i pacchetti seguono lo stesso percorso. Se una linea o uno switch del percorso cessano di funzionare la chiamata viene interrotta. Ciò rende il sistema meno tollerante alle cadute rispetto ai sistemi senza connessione.

Il vantaggio della commutazione di circuito è che supporta più facilmente la qualità del servizio. Stabilendo in anticipo la connessione, la sottorete può riservare risorse quali la banda della connessione, il buffer dello switch e la CPU. Se, nel momento in cui si tenta di effettuare una chiamata, le risorse disponibili sono insufficienti, la chiamata è rifiutata e chi trasmette riceve un segnale di occupato. In questo modo, una volta stabilita la connessione, il servizio è di buona qualità.

In una rete senza connessione, un router a cui arrivano troppi pacchetti nello stesso momento si congestiona e può perdere pacchetti. Chi trasmette potrebbe accorgersene e rispedire i pacchetti, ma la qualità del servizio risulterebbe discontinua e non adatta alla trasmissione audio o video, a meno che la rete non sia poco carica. Inutile dire che le compagnie telefoniche ritengono fondamentale fornire una qualità audio e video adeguata e quindi preferiscono le reti a commutazione di circuito.

Si noti nella Figura 1.31 che nel core network sono presenti dispositivi a commutazione sia di pacchetto sia di circuito. Questo mostra come le reti cellulari siano in un momento di transizione, in quanto le compagnie di telefonia mobile sono in grado di implementare una o a volte entrambe le alternative. Come le reti telefoniche tradizionali le reti cellulari più vecchie usavano un core a commutazione di circuito per trasportare le chiamate vocali. Questo retaggio si mostra nelle reti UMTS attraverso gli **MSC** (*mobile switching center*), i **GMSC** (*gateway mobile switching center*) e i **MGW** (*media gateway*), che stabiliscono connessioni con reti il cui core è a commutazione di circuito, come la rete **PSTN** (*public switched telephone network*).

I servizi di trasmissione dati sono diventati una parte sempre più importante delle reti cellulari, a partire dai servizi di messaggistica e dai primi servizi di trasferimento dati a pacchetto come il **GPRS** (*general packet radio service*) dei sistemi GSM. Questi primi servizi di trasmissione dati mettevano a disposizione decine di kbps, ma gli utenti richiedevano velocità maggiori. Le reti cellulari più moderne trasportano dati alla velocità di molti Mbps. Per fare un paragone si consideri che una chiamata vocale usa 64 kbps e, se compressa, anche 3 o 4 volte meno.

Per trasportare simili quantità di dati, i nodi del core UMTS si connettono direttamente a una rete a commutazione di pacchetto; **SGSN** (*serving GPRS support node*) e **GGSN** (*gateway GPRS support node*) consegnano i pacchetti di dati ai e dai cellulari e si interfacciano con reti esterne, come Internet.

Questa transizione prosegue anche nelle reti cellulari attualmente in fase di progettazione o installazione. I protocolli Internet sono usati anche sui cellulari per stabilire le connessioni durante le chiamate vocali effettuate su reti a commutazione di pacchetto, come nel caso di voice over IP. IP e i pacchetti sono usati ovunque, dall'accesso radio fino al core della rete. Naturalmente, anche per garantire una qualità di servizio migliore, il modo in cui le reti IP sono progettate sta cambiando; anche perché, se così non fosse, i problemi di audio e video discontinui non attirerebbero molti clienti paganti. Torneremo su questo argomento nel Capitolo 5.

Un'altra differenza tra le reti cellulari e quelle tradizionali quali Internet è la mobilità. Quando un utente esce dal raggio di una stazione base cellulare ed entra in quello di un'altra, il flusso di dati deve essere reinstradato dalla vecchia alla nuova cella. Questa tecnica è nota come **handover** o **handoff** ed è mostrata nella Figura 1.32.

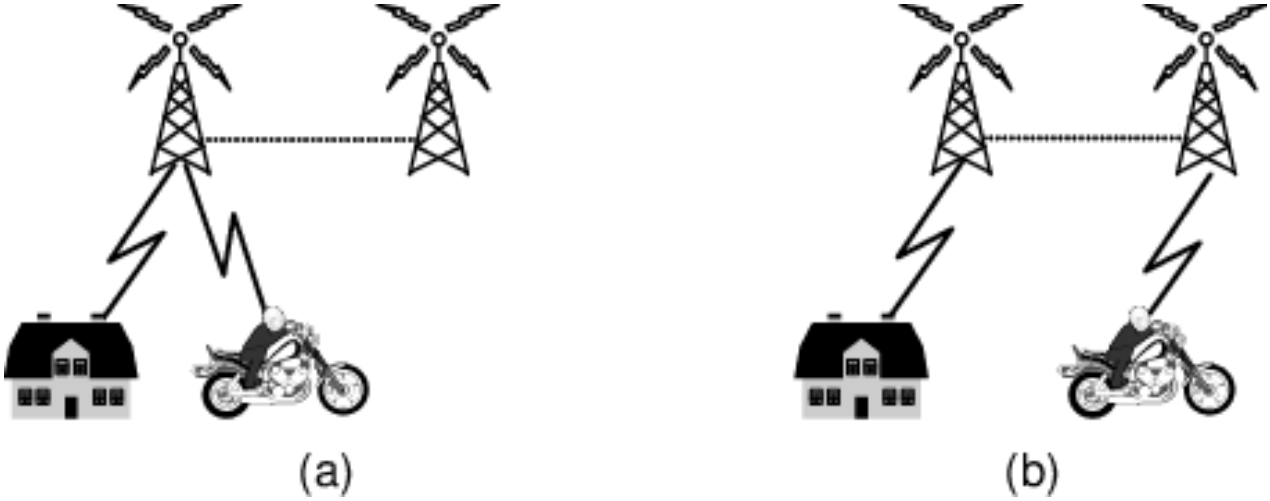


Figura 1.32 Handover di un telefono cellulare: (a) prima e (b) dopo.

Quando la qualità del segnale decade, sia il dispositivo mobile sia la stazione base possono richiedere un handover. In alcune reti cellulari, tipicamente quelle basate sulla tecnologia CDMA, è possibile connettersi alla nuova stazione base prima di disconnettersi da quella vecchia. Questa procedura migliora la connessione del dispositivo mobile, in quanto non vede alcuna interruzione di servizio; in effetti, il dispositivo è connesso per un breve lasso di tempo a due stazioni base cellulari. Questo modo di effettuare un handover è chiamato **soft handover** per distinguerlo dall'**hard handover**, nel quale il dispositivo si disconnette dalla vecchia stazione base prima di connettersi a quella nuova.

Un problema correlato a queste procedure riguarda in prima istanza il modo in cui trovare un dispositivo al momento in cui deve ricevere una chiamata. Ogni rete cellulare ha un **HSS** (*home subscriber server*) nel core che conosce la posizione di ogni utente insieme ad altre informazioni sul suo profilo usate per le procedure di autenticazione e autorizzazione. In questo modo tramite l'HSS è possibile rintracciare qualsiasi dispositivo.

L'ultimo argomento da trattare è la sicurezza. Storicamente le compagnie telefoniche, dovendo emettere le bollette ed evitare frodi, hanno affrontato il problema della sicurezza a lungo e con maggiore serietà di quelle operanti in Internet. Anche se questo può sembrare poco rilevante, passando dalla tecnologia 1G a quella 3G le compagnie di telefoni cellulari sono state in grado di produrre dei meccanismi di base per la sicurezza dei dispositivi mobili.

A partire dai sistemi GSM il cellulare è stato suddiviso in un dispositivo hardware e in un chip removibile contenente informazioni riguardanti l'identità dell'utente. Il chip è informalmente chiamato **carta SIM** (SIM card), abbreviazione di **subscriber identity module**. Le SIM possono essere spostate tra diversi dispositivi per attivarli e forniscono le funzionalità di base per la sicurezza. Quando un cliente GSM è in viaggio in un paese straniero per vacanza o per lavoro, spesso porta con sé il proprio cellulare, ma compra una nuova carta SIM a poco prezzo nel paese di destinazione per fare le chiamate locali senza pagare il roaming.

Per ridurre le frodi, le informazioni contenute nella carta SIM sono usate anche dalla rete cellulare per effettuare l'autenticazione degli utenti e controllare che siano autorizzati a usare la rete. Con la tecnologia UMTS anche il cellulare usa le informazioni della carta SIM per controllare se la rete a cui è connesso è davvero quella che si aspetta.

Un altro aspetto della sicurezza è la privacy. Poiché i segnali wireless sono ricevuti in broadcast da tutti i dispositivi nelle vicinanze, per rendere difficile intercettare illegalmente le conversazioni vengono usate chiavi crittografiche presenti sulla carta SIM per cifrare le trasmissioni. Questo approccio fornisce una privacy molto maggiore dei sistemi di prima generazione, che erano facilmente intercettabili, ma non risolve del tutto il problema, date le debolezze dei sistemi crittografici.

Le reti cellulari sono destinate a giocare un ruolo centrale nelle reti del futuro. Attualmente, si occupano più di applicazioni mobili a banda larga che di chiamate vocali e questo ha grandi ripercussioni sulle interfacce radio, sull'architettura del core della rete e sulla sicurezza delle reti future. Le tecnologie 4G, migliori, più veloci e prossime all'installazione negli Stati Uniti, sono sul tavolo dei progettisti con il nome di **LTE** (*long term evolution*), anche se la progettazione e l'istallazione di reti 3G continua. Altre tecnologie wireless, quali le reti 802.16 comunemente note come **WiMAX**, offrono accesso Internet a banda larga a clienti sia fissi che mobili. È possibile che LTE e WiMAX siano in rotta di collisione, ma è arduo prevedere che cosa succederà.

LAN wireless: 802.11

Con l'avvento dei computer portatili è sorto il desiderio, arrivando in un ufficio, di accendere il proprio notebook e trovarlo magicamente connesso a Internet. Svariati gruppi iniziarono a lavorare sui metodi per ottenere questo risultato, verificando che l'approccio più pratico consisteva nell'equipaggiare l'ufficio e il computer portatile con ricetrasmettitori a bassa potenza per consentire la comunicazione reciproca.

Questo lavoro indusse molte aziende alla commercializzazione di reti LAN senza fili (wireless). La difficoltà stava nel fatto che queste reti risultavano incompatibili tra loro. La proliferazione degli standard comportava che un computer equipaggiato con una scheda radio marca *X* non poteva funzionare in una stanza attrezzata con la stazione base marca *Y*. A metà degli anni '90 l'industria stabilì che sarebbe stata una buona idea creare uno standard per reti LAN senza fili; venne quindi dato il compito al comitato IEEE che aveva standardizzato le LAN cablate di progettare uno standard LAN wireless. Innanzitutto si dovette stabilire come chiamarlo. Tutti gli altri standard LAN avevano una numerazione del tipo 802.1, 802.2, 802.3, fino a 802.10; quindi le LAN wireless vennero denominate 802.11. Sono note anche con il termine colloquiale **WiFi**, ma poiché si tratta di uno standard importante (che è giusto rispettare) lo chiameremo con il suo vero nome: 802.11.

Il resto fu più difficile. Il problema era trovare una banda di frequenza opportuna, preferibilmente disponibile in tutto il mondo. Si usò un approccio opposto a quello usato per le reti cellulari. Invece di impiegare uno spettro costoso e sotto licenza, i sistemi 802.11 operarono su bande libere quali le **ISM** (*industrial, scientific and medical*) definite dall'ITU-R (come 902-928 MHz, 2.4-2.5 Ghz o 5.725-5.825 GHz). Tutti i dispositivi possono usare questo spettro, purché limitino la loro

potenza di trasmissione in modo da permettere la coesistenza con altri dispositivi. Naturalmente, ciò implica che le radio 802.11 possono trovarsi in competizione con telefoni cordless, telecomandi per garage e forni a microonde.

Le reti 802.11 sono formate da client, come computer portatili e cellulari, e da infrastrutture chiamate **AP** (*access point,* punto di accesso) installate negli edifici. Tutti gli access point sono connessi a una rete cablata e tutte le comunicazioni tra i client passano attraverso almeno uno di loro. È anche possibile che due client che siano nel raggio radio uno dell'altro si parlino direttamente, come due computer in un ufficio, senza un access point. Questa configurazione è chiamata **rete ad hoc** (*ad hoc network*), ma è usata molto meno di quella che prevede l'uso di access point. Entrambe le modalità sono mostrate nella Figura 1.33.

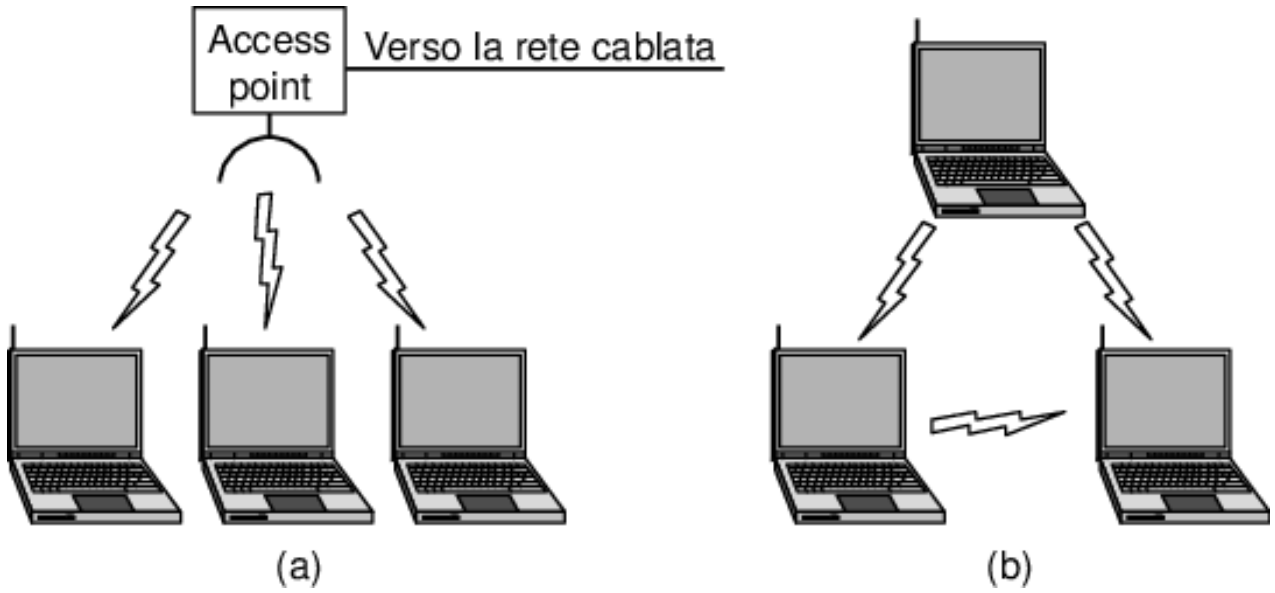


Figura 1.33 (a) Una rete wireless con un access point. (b) Una rete ad hoc.

Le trasmissioni 802.11 sono complicate dalle condizioni wireless che variano molto anche con piccoli cambiamenti dell'ambiente. Alle frequenze usate dall'802.11 i segnali radio possono essere riflessi dagli oggetti solidi così che il ricevente può venire raggiunto da molti echi di trasmissione attraverso percorsi diversi. Gli echi possono cancellarsi o rinforzarsi a vicenda causando grandi fluttuazioni nel segnale ricevuto. Questo fenomeno è chiamato **multipath fading** ed è mostrato nella Figura 1.34.

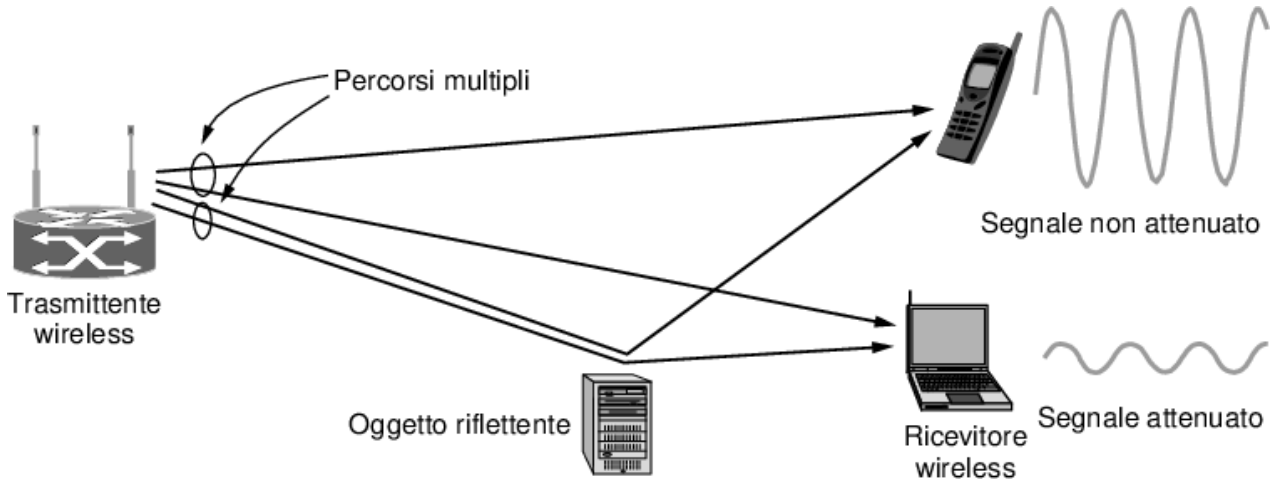


Figura1.34 Multipath fading.

L'idea chiave per superare i problemi legati alle condizioni wireless è la diversificazione dei percorsi (**path diversity**), vale a dire trasmettere le informazioni su percorsi multipli e indipendenti. In questo modo l'informazione viene verosimilmente ricevuta anche se uno dei percorsi attenua molto il segnale. Questi percorsi indipendenti sono tipicamente implementati nello schema digitale di modulazione a livello fisico. Vi sono varie opzioni quali, ad esempio, l'uso di frequenze diverse pur rimanendo nella banda permessa, seguire percorsi spaziali diversi tra coppie di antenne distinte o la ripetizione di bit a intervalli di tempo indipendenti.

Differenti versioni dell'802.11 si sono servite di tutte queste tecniche. Lo standard iniziale (1997) definì una LAN wireless che operava a 1 o 2 Mbps saltando tra le frequenze o diffondendo il segnale su tutto lo spettro permesso. Immediatamente però gli utenti si lamentarono che fosse troppo lento e quindi si cominciò subito a lavorare su standard più veloci. Lo spettro di quel progetto venne esteso e divenne nel 1999 lo standard 802.11b con una capacità trasmissiva fino a 11 Mbps. L'802.11a (1999) e l'802.11g (2003) adottarono un differente schema di modulazione chiamato **OFDM** (*orthogonal frequency division multiplexing*). Questa modulazione funziona dividendo una larga banda spettrale in molte piccole fette sulle quali vengono trasmessi diversi bit in parallelo. Questo schema innovativo, che studieremo nel Capitolo 2, portò l'802.11a/g fino a 54 Mbps. Benché fosse un incremento significativo, gli utenti richiesero un capacità di trasferimento ancora superiore, per supportare applicazioni sempre più pesanti. La versione più recente è l'802.11n (2009) che usa una banda di frequenza ancora più larga e fino a quattro antenne per raggiungere 450 Mbps.

Poiché il wireless è un mezzo intrinsecamente broadcast, gli apparati radio 802.11 devono anche affrontare il problema delle trasmissioni multiple inviate nello stesso momento che, collidendo, possono interferire nella ricezione. Per risolvere questo problema, 802.11 adotta uno schema chiamato **CSMA** (*carrier sense multiple access*), che sfrutta molte idee provenienti dallo schema classico dell'Ethernet cablata la quale, ironicamente, deriva da una prima rete wireless sviluppata nelle Hawaii chiamata **ALOHA**. I computer aspettano un intervallo di tempo breve e casuale prima di iniziare la trasmissione e differiscono le loro trasmissioni se sentono che qualcun altro sta già trasmettendo. Questo schema rende meno probabile che due computer effettuino una

trasmissione in contemporanea. In ogni caso, questo sistema non funziona bene come nel caso delle reti cablate. Per capirne il perché esaminiamo la Figura 1.35. Si supponga che il computer A stia transmettendo al computer B, ma che la portata radio del trasmittente A sia troppo corta per raggiungere il computer C. Se C vuole, a sua volta, trasmettere a B, può mettersi in ascolto prima di iniziare la trasmissione, ma il fatto che non senta niente non significa che la sua trasmissione avverrà con successo. L'impossibilità che C senta A prima di iniziare la trasmissione provoca collisioni. Dopo una collisione il mittente aspetta un altro, più lungo, tempo di ritardo casuale per ritrasmettere il pacchetto. Nonostante questo e altri problemi, tale tipo di approccio funziona sufficientemente bene nella pratica.

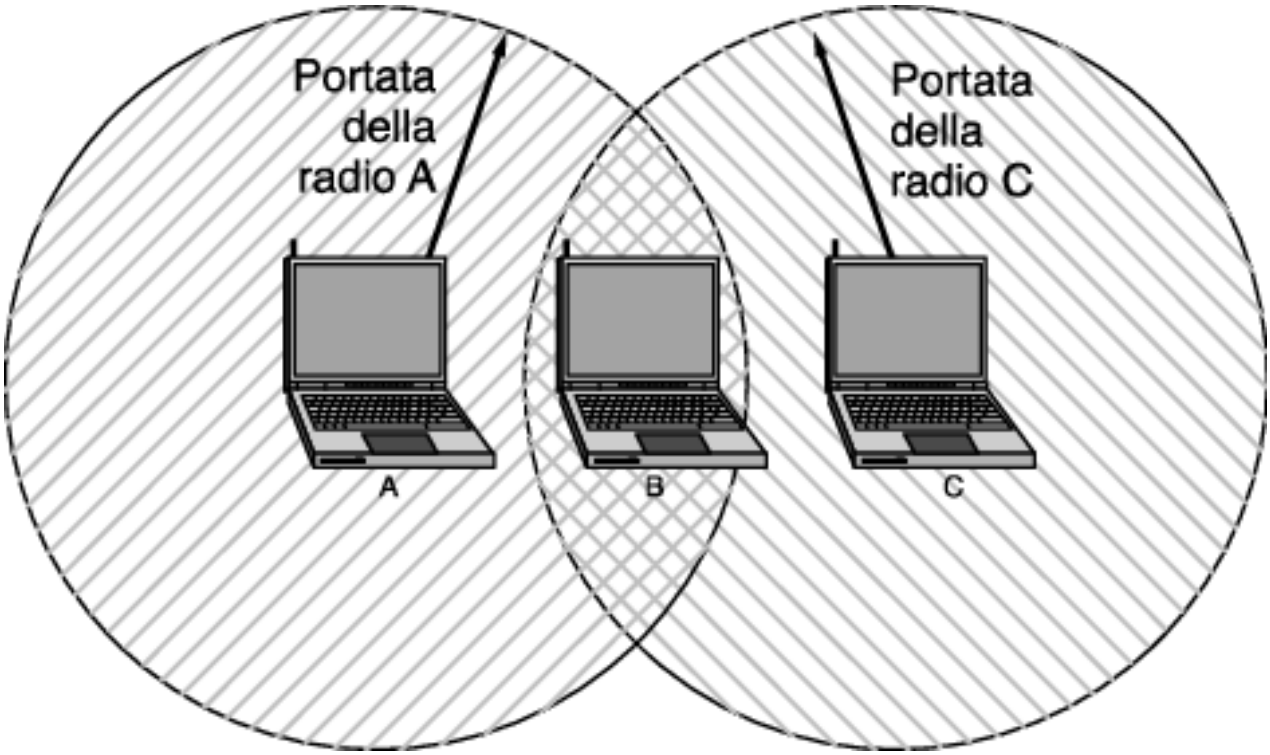


Figura 1.35 La portata di una singola sezione radio potrebbe non coprire l'intero sistema.

Un altro problema è rappresentato dalla mobilità. Quando un utente mobile si allontana dall'access point che sta usando ed entra nella portata di un altro, sorge la necessità di un sistema che effettui il trasferimento. La soluzione delle reti 802.11 consiste nel costruire la rete come tante celle, ognuna dotata di un access point, connesse tra di loro da un sistema di distribuzione. Il sistema di distribuzione può essere implementato con qualunque tecnologia, anche se la più utilizzata è Ethernet. L'utente, quando si muove, può trovare un altro access point il cui segnale sia migliore di quello che sta usando in quel momento e cambiare associazione. Visto dall'esterno, l'intero sistema appare come un'unica rete LAN cablata.

Tuttavia la mobilità nell'802.11 ha un ruolo limitato se paragonata a quella delle reti cellulari. Tipicamente il protocollo 802.11 è usato da client che si muovono da una locazione fissa a un'altra, piuttosto che essere sempre in movimento. La mobilità in realtà non è necessaria per client di

questo tipo e, anche quando viene implementata, si estende su una sola rete 802.11 che può coprire al massimo un grosso edificio. Gli sviluppi futuri dovranno invece fornire mobilità attraverso reti e tecnologie differenti (come 802.21).

Infine, sussiste il problema della sicurezza. Poiché le trasmissioni wireless sono broadcast, è facile per computer vicini ricevere dei pacchetti non destinati a loro. Perché questo non accada, lo standard 802.11 è dotato di uno schema crittografico noto come **WEP** (*wired equivalent privacy*). L'idea era quella di rendere il livello di sicurezza su wireless pari a quello su reti cablate; nonostante la sua bontà, lo schema era imperfetto e venne presto compromesso (Borisov et al., 2001). È stato poi sostituito da schemi crittografici più moderni che sono variamente dettagliati nello standard 802.11i, inizialmente chiamati **WPA**, poi rimpiazzati da **WPA2** e noti anche come **WiFi protected access**.

Il protocollo 802.11 ha portato una rivoluzione nelle reti wireless che è ancora in atto. Oltre agli edifici, esso inizia a essere installato su treni, aerei, navi e automobili in modo che gli utenti possano navigare in Internet in qualsiasi momento. I telefoni cellulari e tutti gli altri dispositivi elettronici di consumo, dalle consolle di gioco alle videocamere, possono effettuare comunicazioni usando questo protocollo. Torneremo a descriverlo in dettaglio nel Capitolo 4.

RFID e reti di sensori

Le reti studiate finora sono composte da dispositivi facilmente riconoscibili, quali computer e telefoni cellulari. Con la tecnologia **RFID** *(radio frequency identification)* anche gli oggetti di uso comune possono essere parte di una rete di calcolatori.

Un'etichetta RFID appare come un adesivo delle dimensioni di un francobollo che può essere posizionato o inserito in un oggetto, in modo tale che esso sia tracciabile. L'oggetto può essere una mucca, un passaporto o un libro. L'etichetta è formata da un piccolo microchip con un identificatore univoco e un'antenna che riceva le trasmissioni radio. I lettori RFID, installati nei punti di tracciamento, rilevano le etichette quando entrano nella loro portata e ne estraggono le informazioni come mostrato nella Figura 1.36. Tra le tante applicazioni possibili ricordiamo il controllo dell'identità, la gestione dei processi produttivi, la rivelazione dei tempi nelle corse e la sostituzione dei codici a barre.

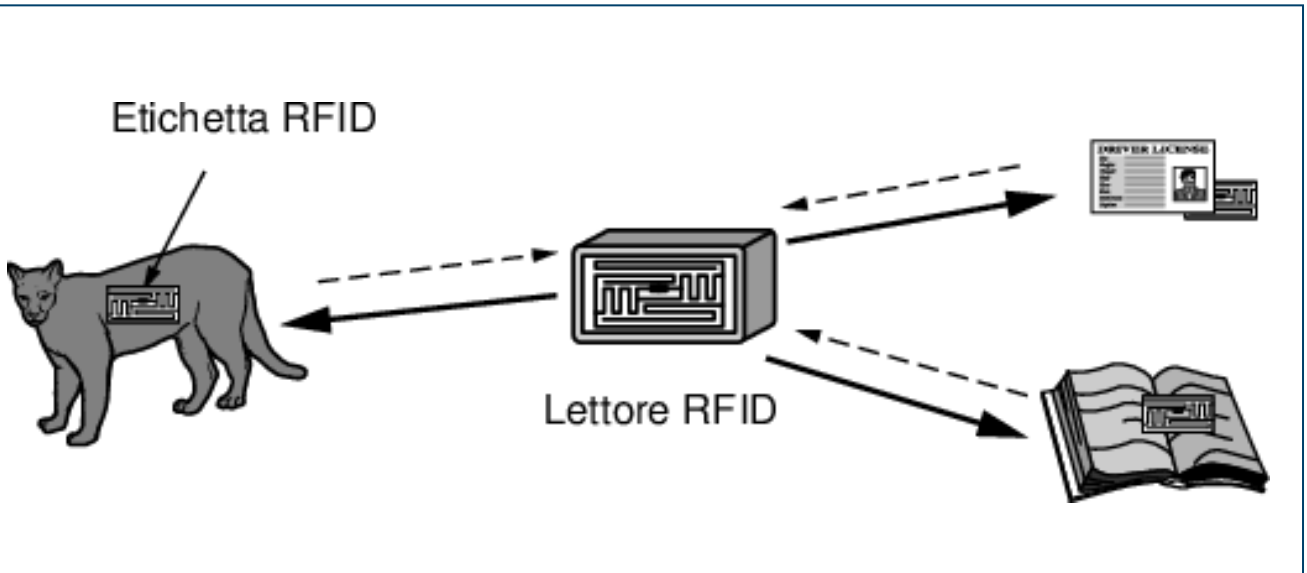


Figura 1.36 Uso di RFID per costruire una rete con oggetti di uso quotidiano.

Vi sono molti tipi diversi di RFID, ognuno con proprietà differenti, ma forse l'aspetto più interessante della tecnologia RFID è che la maggior parte delle etichette RFID non ha né una presa elettrica né una batteria. Tutta l'energia di cui hanno bisogno per funzionare è fornita in forma di onde radio dai lettori RFID. Questa tecnologia è chiamata **RFID passiva** per distinguerla da quella (meno comune) in cui esiste una sorgente di energia nell'etichetta, chiamata **RFID attiva**.

Un tipo comune di etichetta RFID, usata nei pallet per le spedizioni e in alcune patenti di guida, è la **UHF RFID** (*ultra-high frequency RFID*). I lettori RFID inviano (negli Stati Uniti) segnali sulla banda di frequenza 902-928 Mhz. Le etichette comunicano anche a distanza di alcuni metri usando la metodologia del **backscatter**: l'etichetta cambia il modo in cui i segnali vengono riflessi e il lettore è in grado di raccogliere queste riflessioni.

Un altro tipo molto comune di RFID è l'**HF RFID** (*high frequency* RFID) che opera alla frequenza di 13.56 MHz e che potreste ritrovare in passaporti, carte di credito, libri e sistemi di pagamento senza contatto. Le etichette HF RFID sono a corto raggio, tipicamente di 1 metro o meno, perché il fenomeno fisico su cui sono basate è quello dell'induzione piuttosto che del backscatter. Vi sono altri tipi di RFID che usano altre frequenze, quali le **LF RFID** (*low frequency RFID*), che vennero sviluppate prima delle HF RFID e vengono usate per il tracciamento degli animali. È il tipo di RFID che verosimilmente potrebbe avere il vostro gatto.

I lettori RFID devono in qualche modo risolvere il problema di avere più etichette all'interno del loro raggio di lettura. Questo significa che un'etichetta non può semplicemente rispondere quando sente un lettore, altrimenti i segnali provenienti dalle diverse etichette potrebbero collidere. La soluzione adottata è simile a quella dell'802.11: le etichette aspettano per un breve intervallo di tempo casuale prima di inviare il loro identificativo; questo permette al lettore di individuare le singole etichette e di interrogarle ulteriormente.

La sicurezza rappresenta un altro problema. Il fatto che un lettore RFID possa facilmente tracciare un oggetto, e quindi la persona che lo usa, può essere un'invasione della privacy. Sfortunatamente è difficile rendere sicure le etichette RFID, poiché non hanno grandi capacità di calcolo né di comunicazione e quindi non possono utilizzare sofisticati algoritmi di crittografia. Invece, vengono usate solo deboli misure di sicurezza quali le password (che possono essere facilmente compromesse). Se la vostra carta d'identità può essere letta in remoto da un controllore della dogana, che cosa impedisce che la stessa carta venga tracciata da altre persone senza che ve ne accorgiate? Non molto.

Le etichette RFID sono nate come chip di identificazione, ma stanno ora rapidamente diventando computer in piena regola. Per esempio, alcune etichette sono dotate di una memoria che può essere aggiornata e più tardi interrogata, in modo da poter registrare che cosa sia accaduto all'oggetto etichettato. Rieback et al. (2006) dimostrarono che questo implica che tutti i problemi relativi a software maligno (malware) possono riguardare anche le RFID, per cui il vostro gatto o il vostro passaporto potrebbero diventare i vettori di un virus RFID.

Un ulteriore passo avanti rispetto alle RFID sono le **reti di sensori**, installate per registrare informazioni sul mondo fisico. Finora sono state usate principalmente per sperimentazioni scientifiche, quali il controllo delle abitudini degli uccelli, dell'attività vulcanica, della migrazione delle zebre, ma anche per applicazioni commerciali come l'assistenza sanitaria e il rilevamento delle vibrazioni; il controllo di cibi congelati, surgelati, o comunque degradabili arriverà molto presto.

I nodi sensori sono piccoli computer, spesso della grandezza di un portachiavi, che rilevano temperatura, vibrazione e altre informazioni. Nell'ambiente che deve essere controllato vengono piazzati molti nodi, in genere dotati di batterie, sebbene possano ricavare energia dalle vibrazioni o dal sole. Come per le RFID, la sfida principale è avere abbastanza l'energia; per questo motivo i nodi devono comunicare tra di loro con attenzione, in modo da poter consegnare le loro informazioni a un punto di raccolta dati. Una strategia molto praticata è quella di organizzare i nodi in modo che si passino i messaggi da uno all'altro, come mostrato nella Figura 1.37; questo tipo di rete è chiamato **multihop network**.

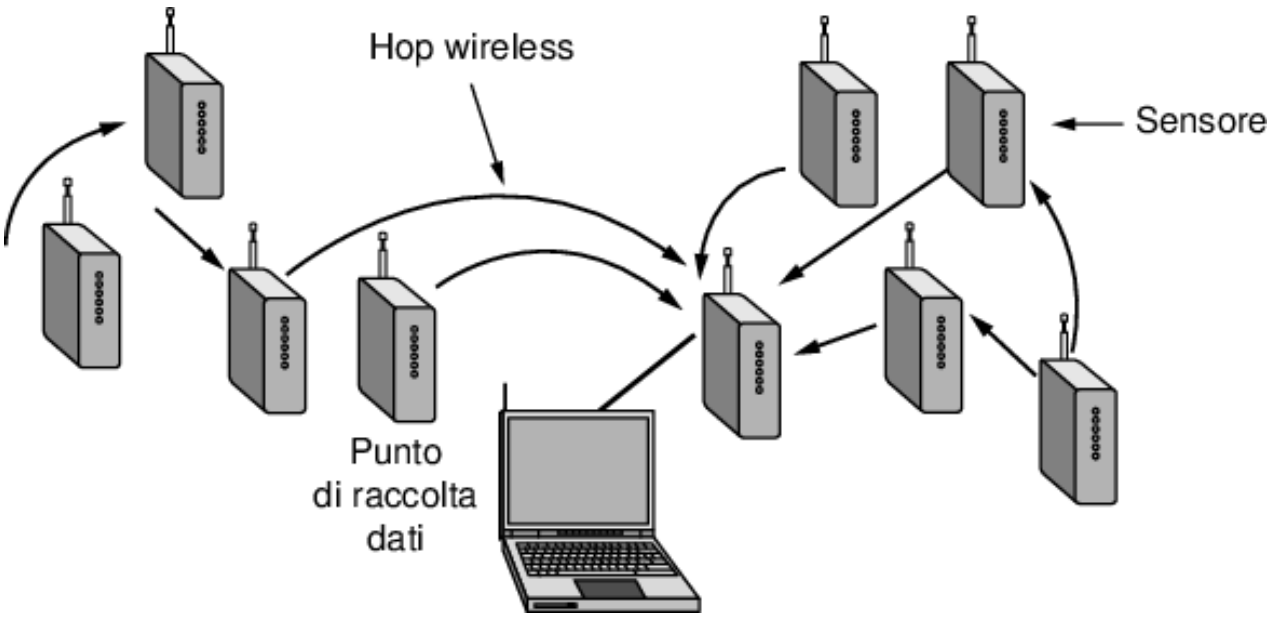


Figura 1.37 Topologia multihop di una rete di sensori.

Le reti RFID e le reti di sensori diventeranno verosimilmente molto potenti e pervasive in futuro. Gruppi di ricerca hanno già combinato il meglio di entrambe le tecnologie per costruire etichette RFID programmabili e dotate di sensori di luce, movimento e altre grandezze (Sample et al. 2008).

1.6 Standardizzazione delle reti

Esistono molti costruttori e fornitori di reti, ognuno con le proprie impostazioni. In assenza di coordinamento il caos sarebbe totale, e gli utenti non avrebbero nulla in mano. L'unica strada è raggiungere un accordo su alcuni standard di rete. Gli standard non solo permettono la comunicazione tra computer diversi, ma aumentano la quota di mercato dei prodotti che vi aderiscono. Un mercato più ampio sostiene la produzione in serie, assicura economie di scala nella costruzione, migliora l'implementazione e garantisce altri benefici che diminuiscono i prezzi dei prodotti.

In questo paragrafo daremo un rapido sguardo al mondo importante, ma poco conosciuto, della standardizzazione internazionale. Chiediamoci però prima che cosa sia uno standard. Una persona ragionevole potrebbe assumere che uno standard regoli il funzionamento del protocollo affinché lo si possa implementare bene, ma si sbaglierebbe.

Gli standard definiscono né più né meno che cosa sia necessario per l'interoperabilità. Ciò permette che emergano mercati più grandi e che le aziende competano sulla base della bontà dei loro prodotti. Per esempio, lo standard 802.11 definisce vari tassi di trasmissione, ma non dice quale di questi dovrebbe essere usato in una particolare trasmissione: un fattore chiave per assicurare buone prestazioni che viene lasciato a chi costruisce il prodotto. Spesso risulta difficile raggiungere l'interoperabilità in questo modo, in quanto esistono molte scelte implementative diverse e gli standard definiscono di solito molte opzioni possibili. Per quanto riguarda l'802.11, i problemi sono talmente tanti che un gruppo commerciale, chiamato **WiFi Alliance**, ha iniziato a lavorare sull'interoperabilità all'interno dello standard 802.11; una strategia questa che è diventata una pratica comune.

Analogamente un protocollo standard definisce il protocollo su cavo, ma non l'interfaccia di servizio all'interno, fatta eccezione per ciò che serve a spiegare il protocollo. Spesso le vere interfacce di servizio sono proprietare. Per esempio, il modo in cui il protocollo TCP si interfaccia con IP in un computer non si occupa di come comunicare con un host remoto. L'unica cosa che importa è che l'host remoto comprenda il protocollo TCP/IP. In effetti, TCP e IP sono comunemente implementati insieme, senza alcuna interfaccia distinta. Detto ciò, buone interfacce di servizio, come buone API, sono fondamentali affinché un protocollo venga utilizzato e quelle migliori (come le socket di Berkeley) possono diventare molto popolari.

Gli standard ricadono in due categorie: de facto e de jure. Gli standard **de facto** (cioè "di fatto") sono quelli affermatisi sul campo, senza una pianificazione formale. HTTP, il protocollo su cui è basato il Web, partì come standard de facto. Era parte dei primi browser WWW sviluppati da Tim Berners-Lee al CERN e la crescita del suo utilizzo andò di pari passo con la crescita del Web. Il Bluetooth è un altro esempio: originariamente sviluppato da Ericsson, è ora usato da tutti.

Gli standard **de jure** (in latino significa "per legge") sono al contrario formali, adottati da qualche organismo di standardizzazione autorizzato. Le autorità per la standardizzazione internazionale vengono solitamente divise in due classi: quelle stabilite da trattati tra i governi delle nazioni e

quelle che comprendono organizzazioni volontarie. Nel campo degli standard per le reti di calcolatori esistono diverse organizzazioni di entrambi i tipi, discusse nel seguito.

Nel concreto le relazioni tra standard, aziende e organi di standardizzazione sono complicate. Gli standard de facto evolvono spesso in standard de jure, specialmente se hanno avuto successo. Questo accadde nel caso di HTTP, che venne velocemente formalizzato da IETF. Gli organi di standardizzazione spesso si ratificano a vicenda gli standard, un'operazione che ricorda il darsi una pacca sulla spalla a vicenda, per incrementare la quota di mercato di una certa tecnologia. Di questi tempi, molte alleanze commerciali, formatesi intorno a particolari tecnologie, giocano anche un ruolo significativo nello sviluppare e definire standard di rete. Per esempio, **3GPP** (*third generation partnership project*) è una collaborazione tra associazioni di telecomunicazione che guidano lo standard su cellulari UMTS 3G.

Il Who's Who del mondo delle telecomunicazioni

Lo status giuridico delle aziende telefoniche nel mondo varia notevolmente da nazione a nazione. A un estremo si trovano gli Stati Uniti, che hanno più di 2000 aziende telefoniche private (per la maggior parte molto piccole) e indipendenti. Alcune di queste si aggiunsero nel 1984 con lo smembramento di AT&T, la principale corporation del mondo che offriva il servizio telefonico all'80% dei telefoni americani, e con il Telecommunications Act del 1996 che revisionò il regolamento per promuovere la concorrenza.

All'estremo opposto si trovano quelle nazioni in cui il governo ha il monopolio completo di tutte le comunicazioni, incluse posta, telegrafi, telefono e spesso anche radio e televisione. La maggior parte del mondo ricade in questa categoria. In alcuni casi l'authority per le telecomunicazioni è una società a partecipazione statale, mentre in altri è semplicemente un ramo del governo generalmente noto con il nome **PTT** (*amministrazione postale, telegrafica e telefonica*). Nel mondo, la tendenza è verso la liberalizzazione e la concorrenza, prendendo le distanze dal monopolio statale. La maggioranza dei paesi europei ha (parzialmente) liberalizzato le proprie PTT, ma altrove il processo è ancora agli inizi. Con tutti questi diversi fornitori di servizio, è chiaramente necessario offrire compatibilità su scala mondiale per garantire che gli abitanti (e i computer) di una data nazione possano chiamare le controparti in un'altra. Questa necessità esiste da molto tempo: nel 1865 i rappresentanti di molti governi europei si riunirono per formare il predecessore dell'attuale **ITU** (*international telecommunication union*). Il suo compito era quello di standardizzare le telecomunicazioni internazionali, che all'epoca significavano la telegrafia. Anche allora fu chiaro che sarebbero sorti problemi se metà delle nazioni avesse usato il codice Morse e l'altra metà un altro. Quando il telefono divenne un servizio internazionale, ITU si prese l'incarico di standardizzare la telefonia, e nel 1947 divenne un'agenzia delle Nazioni Unite.

L'ITU ha circa 200 membri governativi, che includono praticamente ogni membro delle Nazioni Unite. Poiché gli Stati Uniti non hanno una PTT, all'ITU li deve rappresentare qualche altro organismo. Il compito spetta al Dipartimento di Stato, probabilmente perché ITU ha a che fare con le nazioni estere, specialità del Dipartimento di Stato. Ci sono circa 700 settori e membri associati, che comprendono società telefoniche (quali AT&T, Vodafone e Sprint), produttori di apparati per telecomunicazioni (come Cisco, Nokia e Nortel), produttori di computer (ad esempio Microsoft, Agilent e Toshiba), produttori di chip (Intel, Motorola, Texas Instruments) e altre aziende interessate (per esempio Boeing, CBS, VeriSign).

ITU possiede tre settori principali. Ci concentreremo primariamente su **ITU-T**, il settore di standardizzazione delle telecomunicazioni, che si occupa di sistemi di telefonia e di scambio dati. Prima del 1993 questo settore era chiamato **CCITT**, un acronimo per il suo nome francese: Comité Consultatif International Telegraphique et Téléphonique. **ITU-R**, il settore delle radiocomunicazioni, si occupa di coordinare l'uso delle frequenze radio da parte di gruppi di interesse in competizione tra di loro ovunque nel mondo. L'altro settore è quello dello sviluppo, l'**ITU-D**, che promuove lo sviluppo di tecnologie di informazione e comunicazione terrestre per ridurre il "digital divide" tra le nazioni che hanno effettivamente accesso alle tecnologie dell'informazione e i paesi con accesso limitato.

Il compito di ITU-T è emanare raccomandazioni tecniche su interfacce per telefonia, telegrafia e trasmissione dati, che spesso diventano standard riconosciuti a livello internazionale anche se, tecnicamente, si tratta di semplici suggerimenti che i governi possono o meno adottare (i politici assomigliano a volte a quegli adolescenti tredicenni che non accettano volentieri che gli si dia loro degli ordini). In pratica, un paese che desideri adottare uno standard di telefonia, diverso da quello usato dal resto del mondo, è libero di farlo, pagando il prezzo di rimanere isolato da tutti gli altri. Questo potrebbe forse funzionare per la Corea del Nord, ma in ogni altro paese rappresenterebbe un vero problema.

Il lavoro di ITU-T è di fatto portato avanti dai suoi 10 gruppi di studio, che possono comprendere anche più di 400 persone e che affrontano argomenti che spaziano dalla tariffazione delle telefonate, ai servizi multimediali, alla sicurezza. Per raggiungere gli obiettivi, i gruppi di studio sono divisi in gruppi di lavoro, che a loro volta sono divisi in squadre di esperti, divise infine in gruppi ad hoc.

Nonostante tutta questa burocrazia, ITU-T funziona perfettamente. Dalla sua formazione ha prodotto più di 3.000 raccomandazioni, molte delle quali sono ampiamente utilizzate. Per ulteriori informazioni su ITU ci si riferisca a (Irmer, 1994).

Il Who's Who del mondo degli standard internazionali

Gli standard internazionali sono definiti e pubblicati da **ISO** (*international standards organization*),[5] una libera organizzazione di volontari fondata nel 1946. I suoi membri sono gli organismi di standardizzazione nazionali dei 157 paesi membri. I membri includono ANSI (USA), BSI (Gran Bretagna), AFNOR (Francia), DIN (Germania) e altri 153.
ISO definisce standard su una vastissima gamma di argomenti che spazia da dadi e bulloni (letteralmente) al rivestimento dei pali telefonici, senza trascurare i semi di cacao (ISO 2451), le reti da pesca (ISO 1530), la biancheria intima femminile (ISO 4416) e tanti altri argomenti che a prima vista non si pensa abbiano bisogno di essere standardizzati. Per quanto riguarda le telecomunicazioni ISO e ITU-T spesso collaborano (ISO è membro di ITU-T), per evitare di emettere ufficialmente due standard internazionali incompatibili tra loro.

Sono stati emanati più di 17.000 standard, inclusi gli standard OSI. ISO ha più di 200 comitati tecnici (*technical committee* o TC), numerati in ordine di creazione, ognuno dei quali affronta un argomento specifico. TC1 si occupa di viti e bulloni (standardizzando il passo dei filetti delle viti). JTC1 si occupa di tecnologia dell'informazione definendo, tra gli altri, standard per reti, computer

e software. È il primo, e finora unico, TC congiunto, creato nel 1987 dall'unione di TC97 con parte di un altro organismo di standardizzazione chiamato IEC. Ogni TC ha dei sottocomitati (SC) divisi in working group (WG).

Il lavoro è svolto soprattutto nei WG, da più di 100.000 volontari sparsi in tutto il mondo. Molti di loro ricevono l'incarico di lavorare per ISO dai loro datori di lavoro, i cui prodotti devono essere standardizzati. Altri volontari lavorano per il loro governo, desideroso che venga accettata a livello internazionale una propria metodologia. In molti WG sono attivi anche esperti accademici.

ISO adotta gli standard seguendo una procedura pensata per ottenere il maggior consenso possibile. Il processo inizia quando uno degli enti normativi nazionali sente la necessità di uno standard internazionale in qualche area. Viene quindi formato un gruppo di lavoro per produrre un **CD** (*committee draft*). Il CD viene fatto circolare tra tutti i membri, che hanno sei mesi per esporre le loro critiche. Se viene approvato a larga maggioranza, viene prodotto un documento chiamato **DIS** (*draft international standard*), fatto circolare per commenti e votazione. Sulla base dei risultati di questa fase, viene preparato, approvato e pubblicato il testo finale dell'**IS** (*international standard*). Nei settori che suscitano controversie un CD o DIS può richiedere parecchie versioni prima di raggiungere un numero sufficiente di voti, e l'intero processo può impiegare anni.

Il **NIST** (*national institute of standards and technology*) fa parte del dipartimento del commercio U.S.A. e, in precedenza, era chiamato *national bureau of standards*. Emette standard obbligatori per gli acquisti di materiale fatto dal governo U.S.A., con l'eccezione di quelli del dipartimento della difesa, che segue standard interni.

Un altro attore importante nel mondo degli standard è l'**IEEE** (*institute of electrical and electronics engineers*), il principale ente professionale del mondo. Oltre alla pubblicazione di montagne di periodici e all'organizzazione di centinaia di conferenze ogni anno, IEEE ha un gruppo di standardizzazione che sviluppa standard nel campo dell'ingegneria elettrica e dei computer. Il comitato IEEE 802 ha standardizzato molti tipi di LAN, alcuni dei quali saranno studiati nel seguito di questo libro. L'attività è svolta da un insieme di gruppi di lavoro, elencati nella Figura 1.38. Il livello di successo dei diversi gruppi di lavoro 802 è stato basso; avere un numero 802.x non è garanzia di successo, anche se l'impatto dei casi di successo sull'industria e sul mondo (specialmente 802.3 e 802.11) è stato enorme.

Numero	Argomento
802.1	Aspetti generali e architettura delle LAN
802.2 ↓	Logicai link control
802.3 *	Ethernet
802.4 *	Token bus (usato per poco tempo negli impianti produttivi)
802.5	Token ring (la proposta IBM per le LAN)
802.6 ↓	Dual queue dual bus (standard originario per le MAN)

Numero	Argomento
802.7 ↓	Technical advisory group per le tecnologie a larga banda
802.8 †	Technical advisory group per le tecnologie a fibra ottica
802.9 ↓	LAN isocrone (per applicazioni real-time)
802.10 ↓	Virtual LAN e sicurezza
802.11 *	Wireless LAN (WiFi)
802.12 ↓	Demand Priority (lo standard AnyLAN di HP)
802.13	Numero ritenuto sfortunato, nessuno lo ha voluto
802.14 ↓	Cable modem (modem per TV via cavo, standard defunto perché anticipato da un consorzio industriale)
802.15 *	Personal area network (Bluetooth, Zigbee)
802.16 *	Wireless a banda larga (WiMAX)
802.17	Resilient packet ring
802.18	Technical advisory group per problemi di regolamentazione radio
802.19	Technical advisory group per la coesistenza di tutti questi standard
802.20	Wireless mobile a banda larga (simile a 802.16e)
802.21	Handoff indipendente dal mezzo trasmissivo (per spostamenti che coinvolgono tecnologie diverse)
802.22	Wireless regional area network

Figura 1.38 I gruppi di lavoro 802.1 più importanti sono contrassegnati con *; quelli marcati da ↓ sono in quiescenza. Quelli con † hanno rinunciato all'incarico e si sono sciolti.

Il Who's Who del mondo degli standard di Internet

La Internet mondiale ha un proprio meccanismo di standardizzazione, molto diverso da ITU-T e ISO. Per analogia la differenza può essere sbrigativamente riassunta con l'esempio seguente: la gente che partecipa agli incontri ITU e ISO veste in giacca e cravatta, mentre quella che partecipa agli incontri di standardizzazione Internet indossa i jeans (eccetto quando si riunisce a San Diego, dove porta pantaloncini e T-shirt).

Gli incontri ITU-T e ISO sono frequentati da manager e da pubblici ufficiali per i quali la standardizzazione è il lavoro principale. Vedono la standardizzazione come il Bene e gli dedicano la loro esistenza. Al contrario, il popolo di Internet fondamentalmente preferisce l'anarchia, ma con centinaia di milioni di utenti che svolgono le loro attività sarebbe difficile comunicare. Per questo motivo gli standard, sebbene deprecabili, sono a volte necessari. In questo contesto David Clark dell'M.I.T. fece un celebre commento sulla standardizzazione Internet definendola come un "vago consenso su codice che funziona".

Quando fu creata ARPANET, il dipartimento della difesa creò un comitato informale per supervisionarla. Nel 1983 il comitato fu rinominato **IAB** (*internet activities board*) e gli fu data una missione leggermente più ampia, cioè mantenere orientati nella stessa direzione i ricercatori interessati ad ARPANET/Internet: un'attività che si può paragonare a fare il bagno al gatto. Il significato dell'acronimo **IAB** venne poi cambiato in **Internet architecture board**.

Ciascuno dei circa 10 membri di IAB guidava un gruppo di lavoro dedito a qualche argomento importante. IAB si riuniva diverse volte all'anno per discutere dei risultati e renderne conto al dipartimento della difesa e a NSF, che all'epoca fornivano la maggior parte dei finanziamenti. Quando era necessario uno standard (per esempio un nuovo algoritmo di routing), i membri di IAB criticavano aspramente la vecchia versione e poi annunciavano il cambiamento, così che i dottorandi, che formavano il cuore della produzione software, potevano implementarlo. La comunicazione era affidata a una serie di rapporti tecnici chiamati **RFC** (*request for comments*). Gli RFC, numerati cronologicamente, sono disponibili on-line e qualsiasi interessato li può scaricare da *http://www.ietf.org/rfc*. Ne esistono più di 5.000 e in questo libro faremo riferimento a molti di essi.

Nel 1989 le dimensioni di Internet erano tali che questo stile altamente informale non funzionava più. All'epoca molti costruttori offrivano prodotti TCP/IP e non li volevano modificare solo perché dieci ricercatori avevano avuto un'idea migliore. Nell'estate del 1989 IAB fu nuovamente riorganizzato e i ricercatori si spostarono nell'**IRTF** (*Internet research task force*), che venne resa subordinata a IAB assieme a **IETF** (*Internet engineering task force*). IAB fu ripopolata con persone che non rappresentavano solo la comunità dei ricercatori. All'inizio era un gruppo a "trasmissione ereditaria", dove i membri restavano in carica due anni e designavano il loro successore. Più tardi fu creata la **Internet society**, frequentata da persone interessate a Internet: per questo motivo Internet society è in un certo senso paragonabile ad ACM o IEEE. È guidata da rappresentanti eletti, che a loro volta eleggono i membri IAB.

Lo scopo di questa divisione fu quello di concentrare IRTF sulla ricerca a lungo termine, mentre IETF avrebbe affrontato i problemi tecnici a breve termine. IETF fu divisa in gruppi di lavoro, ciascuno con un problema specifico da risolvere. Inizialmente i capi di ciascun gruppo di lavoro si riunivano a formare il comitato direttivo che guidava gli sforzi tecnici. Gli argomenti dei gruppi di lavoro riguardano nuove applicazioni, informazioni degli utenti, integrazione OSI, instradamento e indirizzamento, sicurezza, gestione della rete e standard. Si crearono così tanti gruppi di lavoro (più di 70) che è stato necessario raggruppare per aree; il capo di ogni area partecipa al comitato direttivo.

Fu inoltre adottato un processo di standardizzazione più formale, modellato secondo ISO. L'idea originale, per diventare un **proposed standard**, deve essere dettagliatamente illustrata in un RFC e deve destare abbastanza interesse nella comunità da essere presa in considerazione. Per passare allo stadio **draft standard**, un'implementazione funzionante deve essere stata provata rigorosamente da almeno due siti indipendenti per un minimo di quattro mesi. Se IAB è convinto che l'idea regge e il software funziona, può dichiarare l'RFC **Internet standard**. Alcuni standard Internet sono diventati standard del dipartimento della difesa (MIL-STD), obbligatori per i fornitori del dipartimento.

Per quanto riguarda gli standard Web, il **World Wide Web consortium** (W3C) sviluppa protocolli e linee guida che facilitino la crescita a lungo termine del Web. È un consorzio industriale guidato da Tim Berners-Lee e fondato nel 1994 quando il Web iniziò realmente la sua diffusione. W3C ha ora più di 300 membri sparsi in tutto il mondo e ha prodotto più di 100 raccomandazioni, come vengono chiamati i suoi standard, riguardanti argomenti quali HTML e la privacy.

1.7 Unità metriche

Questo testo, come tutta l'informatica, adotta le unità di misura metriche invece di quelle inglesi tradizionali. I prefissi metrici principali sono elencati nella Figura 1.39, abbreviati con le loro prime lettere, dove le unità maggiori di 1 hanno iniziali maiuscole (KB, MB, ecc.); fa eccezione per motivi storici la sigla kbps che sta per kilobit/s. Per quanto detto, una linea di comunicazione da 1 Mbps trasmette 10^6 bit/s e un clock di 100 ps commuta ogni 10^{10} secondi. Poiché milli e micro iniziano entrambi con la lettera "m" si è resa indispensabile una scelta, e normalmente "m" sta per milli, mentre la lettera greca μ (mu) indica micro.

Esp	Esplicito	Prefisso	Esp	Esplicito	Prefisso
10^{-3}	0,001	milli	10^{3}	1.000	Kilo
10^{-6}	0,000001	micro	10^{6}	1.000.000	Mega
10^{-9}	0,000000001	nano	10^{9}	1.000.000.000	Giga
10^{-12}	0,000000000001	pico	10^{12}	1.000.000.000.000	Tera
10^{-15}	0,000000000000001	femto	10^{15}	1.000.000.000.000.000	Peta
10^{-18}	0,000000000000000001	atto	10^{18}	1.000.000.000.000.000.000	Exa
10^{-21}	0,000000000000000000001	zepto	10^{21}	1.000.000.000.000.000.000.000	Zetta
10^{-24}	0,000000000000000000000001	yocto	10^{24}	1.000.000.000.000.000.000.000.000	Yotta

Figura 1.39 Prefissi metrici principali.

È importante osservare che la misura delle dimensioni di memorie, dischi, file e database segue una convenzione un po' diversa nella pratica industriale. Kilo significa 2^{10} (1.024) invece di 10^3 (1.000), poiché le memorie sono sempre potenze di due. Pertanto una memoria da 1 KB contiene 1.024 byte, e non 1.000. Si noti inoltre l'uso della maiuscola "B" per denotare "byte" (pari a 8 bit), invece della minuscola "b" che indica bit. Analogamente una memoria da 1 MB contiene 2^{20} (1.048.576) byte, una da 1 GB ne contiene 2^{30} (1.073.741.824) byte e un database da 1 TB contiene 2^{40} (1.099.511.627.776) byte. Tuttavia una linea di comunicazione da 1 kbps trasmette 1.000 bit al secondo e una LAN da 10 Mbps lavora a 10.000.000 bit/s poiché queste velocità non sono potenze di due. Sfortunatamente molti mescolano i due metodi, soprattutto per misurare le dimensioni dei dischi. Per evitare ambiguità in questo libro useremo i simboli KB, MB, GB e TB rispettivamente per 2^{10}, 2^{20} e 2^{30} e 2^{40} byte, e i simboli kbps, Mbps, Gbps e Tbps rispettivamente per 10^3, 10^6, 10^9 e 10^{12} bit/s.

1.8 Organizzazione del resto del libro

Il libro tratta teoria e pratica delle reti di calcolatori. La maggior parte dei capitoli inizia con una presentazione dei principi rilevanti, seguita da esempi che li illustrano. Gli esempi sono generalmente tratti dalle reti Internet e wireless come le reti cellulari, poiché sono entrambe importanti e molto diverse tra loro. Dove necessario saranno mostrati altri esempi.

La struttura del libro segue il modello ibrido della Figura 1.23. Con il Capitolo 2 inizia l'esame della gerarchia dei protocolli, cominciando dal basso. Il secondo capitolo offre le informazioni di base nel campo della comunicazione dati e copre i sistemi cablati e wireless. Il materiale si occupa del livello fisico, ma saranno coperti gli aspetti architetturali invece di quelli hardware. Sono discussi molti esempi del livello fisico, come la rete telefonica pubblica, i telefoni cellulari e la rete televisiva via cavo.

I Capitoli 3 e 4 affrontano l'argomento del livello data link dividendolo in due parti. Il Capitolo 3 tratta come trasmettere i pacchetti su di un link, comprese l'individuazione e la correzione degli errori. Vedremo la DSL (usata per l'accesso a banda larga su linee telefoniche) come esempio reale di protocollo data link.

Il Capitolo 4 si occupa del sottolivello di accesso al mezzo di trasmissione, che fa parte del livello data link. Il problema che affronta riguarda come condividere un canale tra molti computer. Gli esempi proposti includono wireless come 802.11 e RFID oltre che LAN cablate come la classica Ethernet. Vengono discussi anche switch a livello data link, che servono per collegare tra loro le LAN, come nel caso di switched Ethernet.

Il Capitolo 5 descrive il livello di rete e in particolare l'instradamento, analizzando molti algoritmi di routing statici e dinamici. Anche se gli algoritmi di routing sono validi, quando la rete è sottoposta a più traffico di quello che è in grado di gestire, alcuni pacchetti vengono consegnati in ritardo o persi. Discuteremo questo argomento partendo da come prevenire una congestione fino a come garantire una certa qualità di servizio. I problemi che nascono quando si collegano reti eterogenee per formare una internetwork sono numerosi, per cui è presente una completa analisi del livello di rete di Internet.

Il Capitolo 6 riguarda il livello di trasporto. L'enfasi è sui protocolli orientati alla connessione, poiché sono richiesti da molte applicazioni. Vengono studiati in dettaglio i due protocolli di trasporto Internet (UDP e TCP) e i loro problemi di prestazioni.

Il Capitolo 7 è dedicato al livello applicazione, ai suoi protocolli e alle sue applicazioni. Il primo argomento è DNS, la rubrica telefonica di Internet, poi viene la e-mail e la discussione dei suoi protocolli. Ci muoveremo quindi sul Web, con una discussione dettagliata dei contenuti statici e dinamici, di ciò che accade sul lato client e su quello server. Infine esamineremo le applicazioni multimediali di rete tra cui lo streaming audio e video. Infine discuteremo delle reti per la diffusione di contenuti, inclusa la tecnologia punto a punto.

Il Capitolo 8 riguarda la sicurezza delle reti. Questo argomento ha aspetti che coinvolgono tutti i livelli, quindi è più semplice trattarlo dopo che essi sono stati dettagliatamente spiegati. Il capitolo inizia con un'introduzione alla crittografia, poi mostra come può essere usata per rendere sicura la comunicazione, la posta elettronica e il Web. Il capitolo termina con la discussione di alcune aree dove la sicurezza collide con privacy, libertà di parola, censura e altri problemi di carattere sociale.

Il Capitolo 9 contiene un elenco di letture consigliate, organizzate per capitolo. È offerto per aiutare i lettori che vogliano proseguire il loro studio delle reti. Il capitolo contiene anche una bibliografia alfabetica di tutte citazioni in questo testo. Il sito Web dell'Autore presso l'editore Pearson Education:

http://www.pearsonhighered.com/tanenbaum

ha una pagina con collegamenti a tutorial, FAQ, aziende, consorzi industriali, organizzazioni professionali, enti normativi, tecnologie, saggi e altro.

1.9 Sommario

Le reti di calcolatori hanno molte applicazioni, per le aziende come per i privati, a casa o in movimento. Per le aziende, le reti di personal computer connessi a server condivisi sono spesso lo strumento per accedere alle informazioni aziendali. Generalmente seguono il modello client-server, dove le postazioni client sulla scrivania degli utenti accedono a potenti server collocati nella sala macchine. Per i privati, le reti danno accesso a una vasta quantità d'informazioni e intrattenimento, come anche alla compravendita di prodotti e servizi. Di solito si accede a Internet tramite telefono o con un provider via cavo, anche se un numero crescente di persone ha una connessione wireless dal proprio portatile o cellulare. Lo sviluppo tecnologico permette nuovi tipi di applicazioni mobili e reti con computer inglobati in elettrodomestici e altri dispositivi di consumo. Gli stessi sviluppi causano problemi di carattere sociale quale la privacy.

A grandi linee le reti si possono dividere in LAN, MAN, WAN e internetwork. Le reti LAN coprono un edificio e lavorano ad alta velocità. Le reti MAN coprono una città, come il sistema televisivo via cavo usato da molti per accedere a Internet. Le WAN coprono una nazione o continente. Alcune di queste tecnologie venivano usate per costruire reti punto a punto (un cavo, per esempio) mentre altre sono broadcast (come il wireless). Le reti si possono collegare per formare internetwork via router; di queste Internet è l'esempio più grande e conosciuto. Anche le reti wireless, come le LAN 802.11 e la telefonia mobile 3G, stanno diventando estremamente popolari.

Il software di rete è costruito partendo dai protocolli, che sono le regole seguite dai processi per comunicare. La maggior parte delle reti supporta le gerarchie di protocolli, dove ogni livello offre un servizio al livello superiore e lo isola dai dettagli dei protocolli usati nei livelli inferiori. Le pile di protocolli sono solitamente basate sul modello OSI oppure su quello TCP/IP. Entrambi hanno livelli di rete, trasporto e applicazione, ma differiscono negli altri livelli. I problemi di progettazione includono affidabilità, allocazione delle risorse, crescita, sicurezza e altro. La maggior parte di questo libro affronta i protocolli e la loro progettazione.

Le reti offrono ai loro utenti servizi, che possono andare dalla consegna di pacchetti senza connessione né garanzie fino alla consegna orientata alla connessione e garantita. In alcune reti è offerto un servizio senza connessione in un livello e un servizio orientato alla connessione in quello sovrastante.

Internet, 3G e IEEE 802.11 sono reti molto note. Internet si è evoluta da ARPANET, alla quale furono aggiunte altre reti per formare una internetwork. L'attuale Internet non è una rete unica, ma una raccolta di migliaia di reti, caratterizzate dall'utilizzo della pila di protocolli TCP/IP. La rete di telefonia cellulare 3G fornisce accesso wireless e mobile a Internet a velocità di multipli di Mbps e, ovviamente, trasporta anche le chiamate vocali. Le LAN wireless basate sullo standard 802.11 sono installate in molte case e luoghi di ritrovo e possono fornire connessione a velocità che eccedono i 100 Mbps. Stanno inoltre emergendo nuovi tipi di reti come quelle di sensori e di RFID.

Per garantire la comunicazione tra più computer è necessaria una grande operazione di standardizzazione, nell'hardware e nel software. Organizzazioni come ITU-T, ISO, IEEE e IAB gestiscono parti differenti del processo di standardizzazione.

Problemi

1. Immaginiamo di aver addestrato Bernie, il nostro San Bernardo, a trasportare una scatola con tre nastri da 8 mm invece di una fiaschetta di brandy (consideriamo un'emergenza il momento in cui si riempie il disco fisso). Ciascuno di questi nastri contiene 7 gigabyte. Il cane può viaggiare al nostro fianco ovunque, alla velocità di 18 km all'ora. Per quale distanza Bernie ha una capacità di trasferimento dati maggiore di una linea di trasmissione con tasso (overhead escluso) di 150 Mbps? Come cambia la risposta se (i) si raddoppia la velocità di Bernie; (ii) la capacità di ogni nastro è raddoppiata; (iii) il tasso della linea di trasmissione è raddoppiato?

2. Un'alternativa alla LAN è semplicemente un grande sistema a divisione di tempo con un terminale per ciascuno degli utenti. Descrivete due vantaggi di un sistema client-server basato su una LAN.

3. Le prestazioni di un sistema client-server sono influenzate da due caratteristiche della rete: ampiezza di banda (quanti bit/s può trasportare) e latenza (quanti secondi impiega il primo bit per passare dal client al server). Fate un esempio di una rete con banda e latenza elevate e di una rete con banda e latenza basse.

4. Oltre a banda e latenza, quali parametri servono per dare una buona descrizione della qualità di servizio di una rete usata per (i) traffico vocale digitalizzato; (ii) traffico video; (iii) traffico per transazioni finanziarie?

5. Una parte del ritardo di un sistema store-and-forward a commutazione di pacchetto è dovuta al tempo che occorre per memorizzare e inoltrare un pacchetto attraverso uno switch. Se il tempo di commutazione fosse 10 microsecondi, sarebbe una frazione significativa del tempo di risposta di un sistema client-server dove il client è a New York e il server in California? Si assuma che la velocità di propagazione nel rame e nella fibra ottica sia 2/3 della velocità della luce nel vuoto.

6. Un sistema client-server usa una rete satellitare, dove il satellite si trova a un'altezza di 40.000 km. Qual è il più breve ritardo possibile in risposta a una richiesta?

7. In futuro, quando tutti avranno un terminale domestico collegato a una rete di calcolatori, sarà possibile condurre referendum pubblici immediati su importanti leggi in corso di approvazione. In prospettiva sarebbe possibile eliminare le attuali legislature e lasciare che la volontà del popolo si esprima in modo diretto. Gli aspetti positivi di questa democrazia diretta sono evidenti; discutete alcuni degli aspetti negativi.

8. Un insieme di cinque router è collegato a formare una sottorete punto a punto. Tra ogni coppia di router i progettisti possono collocare una linea ad alta, media o bassa velocità, oppure nessuna linea. Se il computer impiega 100 ms per generare e ispezionare ciascuna topologia, quanto tempo occorre per ispezionarle tutte?

9. Uno svantaggio delle sottoreti funzionanti in modalità broadcast è lo spreco di capacità quando più host tentano di accedere al canale simultaneamente. Come esempio semplicistico, si supponga che il tempo sia diviso in slot discreti, e durante ciascuno di essi ognuno degli n host

tenti di usare il canale con probabilità *p*. Quale frazione degli slot va sprecata a causa delle collisioni?

10. Si diano due motivi per usare protocolli organizzati in livelli e un possibile svantaggio nell'usarli.

11. Il presidente della Vernici Speciali s.p.a. ha l'idea di collaborare con la birreria locale per realizzare una lattina di birra invisibile (come metodo per ridurre lo smaltimento di alluminio). Il presidente dà l'incarico di esaminare la questione all'ufficio legale, che a sua volta chiede aiuto agli ingegneri. Di conseguenza, il capo degli ingegneri chiama la sua controparte nell'altra azienda per discutere degli aspetti tecnici del progetto. Gli ingegneri riferiscono i risultati ai loro uffici legali, che si telefonano per discutere degli aspetti legali. Infine i presidenti delle due società discutono gli aspetti finanziari dell'accordo. Quale principio del protocollo organizzato in livelli nel senso del modello OSI viola questo meccanismo di comunicazione?

12. Due reti offrono un servizio affidabile orientato alla connessione. Una delle due offre un byte-stream affidabile e l'altra un message stream affidabile. Sono identiche? Se sì, si indichi perché viene fatta una distinzione tra le due; se no, si dia un esempio del perché differiscono.

13. Che cosa significa "negoziazione" nel contesto dei protocolli di rete? Si dia un esempio.

14. Nella Figura 1.19 è mostrato un servizio. In questa figura sono impliciti altri servizi? In caso affermativo si indichi dove, in caso negativo si spieghi il perché.

15. In alcune reti il livello data link tratta gli errori di trasmissione richiedendo una nuova trasmissione dei frame danneggiati. Se la probabilità che un frame venga danneggiato è *p,* qual è il numero medio di trasmissioni necessarie per mandare un frame? Si ipotizzi che le conferme non vadano mai perse.

16. Un sistema ha una gerarchia di protocolli a *n* livelli. Le applicazioni generano messaggi di lunghezza pari a *M* byte. A ogni livello è aggiunta un'intestazione di *h* byte. Che frazione della banda della rete è impegnata da intestazioni?

17. Qual è la principale differenza tra TCP e UDP?

18. La sottorete della Figura 1.25(b) fu progettata per resistere a una guerra nucleare. Quante bombe sono necessarie per partizionare i nodi in due insiemi separati? Si supponga che ogni bomba distrugga un nodo e tutti i collegamenti che vi fanno capo.

19. A grandi linee Internet raddoppia le sue dimensioni ogni 18 mesi. Anche se nessuno ne è certo, una stima degli host presenti nel 2009 è di 600 milioni. Si usino questi dati per prevedere il numero di host Internet nell'anno 2018. È credibile? Si spieghi perché sì o perché no.

20. Quando si trasferisce un file tra due computer sono possibili due politiche di conferma. Nella prima il file è diviso in pacchetti, a ciascuno dei quali il ricevente risponde con una conferma. Nella seconda i pacchetti non ricevono una conferma individuale, ma si conferma l'arrivo dell'intero file. Si discutano i due approcci.

21. Gli operatori di reti cellulari hanno bisogno di conoscere la posizione dei cellulari dei loro utenti (e quindi dei loro clienti). Si spieghino i vantaggi e gli svantaggi che questa conoscenza porta agli utenti.

22. Quanto era lungo, in metri, un bit nello standard 802.3 originale? Si utilizzi un tasso di trasmissione di 10 Mbps e si supponga che la velocità di propagazione nel cavo coassiale impiegato sia 2/3 della velocità della luce nel vuoto.

23. Un'immagine ha 1600 × 1200 pixel con 3 byte per pixel. Si supponga che l'immagine non sia compressa. Quanto tempo impiega per essere trasmessa da un modem su un canale a 56 kbps? E da un cable modem da 1 Mbps? Da una Ethernet a 10 Mbps? Da una Ethernet a 100 Mbps? Da una Ethernet a un gigabit?

24. Ethernet e le reti wireless hanno elementi simili e altri diversi. Una proprietà di Ethernet è che sul mezzo può essere trasmesso solo un frame per volta. Il protocollo 802.11 ha la stessa caratteristica? Si discuta la risposta.

25. Si indichino due vantaggi e due svantaggi dell'esistenza di standard per i protocolli di rete.

26. Quando un sistema ha una parte fissa e una rimovibile (per esempio il lettore CD-ROM e il disco CD-ROM) è importante che sia standardizzato, in modo tale che più aziende possano costruire la parte fissa e quella rimovibile garantendone il funzionamento. Si diano tre esempi fuori dal campo dell'industria informatica dove esiste uno standard internazionale simile e altrettanti dove non esiste.

27. Si supponga che gli algoritmi usati per implementare le operazioni al livello k vengano cambiate. Che impatto si avrebbe sulle operazioni ai livelli $k - 1$ e $k + 1$?

28. Si supponga che venga apportato un cambiamento nel servizio (insieme di operazioni) fornito al livello k. Che impatto si avrebbe sul servizio dei livelli $k - 1$ e $k + 1$?

29. Si fornisca una lista di motivi per cui il tempo di risposta di un client potrebbe essere maggiore del ritardo nel caso migliore.

30. Quali sono gli svantaggi nell'utilizzo di celle piccole e di lunghezza fissata in ATM?

31. Si elenchino le proprie attività quotidiane che implicano l'uso di una rete di calcolatori. Come cambierebbe la propria vita se queste reti fossero improvvisamente spente?

32. Si scopra quali reti siano usate nella propria università o posto di lavoro. Si descriva il tipo di rete, la topologia e i metodi di commutazione.

33. Il programma *ping* permette l'invio a un luogo specifico di un pacchetto di test e la misura del tempo che occorre per arrivare a destinazione e ritornare. Si provi a usare *ping* per vedere quanto tempo serve a raggiungere diversi luoghi noti dalla propria postazione. Da questi dati, si tracci il tempo di transito unidirezionale attraverso Internet in funzione della distanza. È meglio usare delle università, poiché la posizione dei loro server è nota con grande precisione. Per esempio *berkeley.edu* è a Berkeley, California, *mit.edu* è a Cambridge, Massachusetts, *vu.nl* è ad Amsterdam, Paesi Bassi, *www.usyd.edu.au* è a Sydney, Australia e *www.uct.ac.za* è a Città del Capo, Sud Africa.

34. Si vada sul sito Web di IETF, *www.ietf.org,* per vedere le attività in corso. Si scelga un progetto di proprio gradimento e si scriva una relazione di mezza pagina sul problema e sulla soluzione proposta.

35. Internet è composta da un gran numero di reti, la cui struttura ne determina la topologia. On-line è disponibile un gran numero d'informazioni sulla topologia di Internet: si usi un motore di ricerca per rintracciarle e si scriva una breve relazione che riassuma quanto scoperto.

36. Si cerchino in Internet alcuni importanti punti di smistamento usati attualmente per instradare pacchetti.

37. Si scriva un programma che implementi il passaggio di messaggi dal livello in cima a quello più basso del modello di protocollo a 7 livelli. Il programma includa una funzione di protocollo separata per ogni livello. Gli header di protocollo sono sequenze di non più di 64 caratteri. Ogni funzione di protocollo ha due parametri: un messaggio passato dal livello più alto del protocollo (un buffer di caratteri) e la dimensione del messaggio. Questa funzione attacca il suo header davanti al messaggio, stampa il nuovo messaggio sull'output standard e quindi invoca la funzione di protocollo del livello più basso. L'input del programma è un messaggio a livello applicazione (una sequenza di 80 caratteri o meno).

CAPITOLO 2
Il livello fisico

In questo capitolo ci occuperemo del livello più basso del nostro modello di organizzazione dei protocolli: il livello fisico. Esso definisce gli aspetti elettrici, di temporizzazione e le altre modalità con cui i bit, sotto forma di segnali elettromagnetici, vengono spediti sui canali di comunicazione. Il livello fisico è la colonna portante sulla quale viene costruita una rete. Le caratteristiche dei vari tipi di canali determinano le prestazioni (ad esempio, il throughput, cioè la capacità trasmissiva effettivamente utilizzata, il ritardo di trasmissione e il tasso di errore) e sono quindi un buon punto di partenza per iniziare il nostro viaggio nel mondo delle reti.

Cominceremo con un'analisi teorica della trasmissione dati, solo per scoprire che esistono dei limiti fisici su che cosa sia possibile inviare su un canale. In seguito, ci occuperemo di tre tipi di canali di comunicazione: a trasmissione vincolata da un mezzo fisico (cavo in rame e fibra ottica), wireless (trasmissioni radio) e satellitare. Ognuna di queste tecnologie ha delle specifiche caratteristiche che influenzano la progettazione e le prestazioni delle reti che le adottano. Questa trattazione fornirà informazioni di base sulle principali tecnologie di trasmissione dati utilizzate nelle reti odierne.

L'argomento successivo sarà il trattamento dei dati digitali e cioè come i segnali analogici vengono convertiti in bit e viceversa. Dopo di che ci si occuperà di strategie di multiplexing (indicato anche con il termine italiano *multiplazione*), per capire come più trasmissioni possano condividere un canale senza interferire tra loro.

Infine ci interesseremo di tre esempi di sistemi di comunicazione adottati nel mondo reale per reti su larga scala: i sistemi telefonici mobile e fisso e la televisione via cavo.

2.1 Basi teoriche della comunicazione dati

Le informazioni possono essere trasmesse via cavo variando alcune proprietà fisiche, per esempio la tensione o la corrente. Rappresentando il valore di questa tensione o corrente attraverso una funzione a un valore nel tempo, $f(t)$, è possibile modellare il comportamento del segnale e analizzarlo matematicamente. Questa analisi sarà l'argomento dei prossimi paragrafi.

Analisi di Fourier

All'inizio del diciannovesimo secolo, il matematico francese Jean-Baptiste Fourier dimostrò che qualunque funzione periodica sufficientemente regolare $g(t)$ con periodo T può essere ottenuta sommando un numero (anche infinito) di funzioni seno e coseno:

$$g(t) = \frac{1}{2}c + \sum_{n=1}^{\infty} a_n \sin(2\pi nft) + \sum_{n=1}^{\infty} b_n \cos(2\pi nft) \quad \text{(2-1)}$$

dove $f = 1/T$ rappresenta la frequenza fondamentale, a_n e b_n sono rispettivamente le ampiezze seno e coseno delle **armoniche** (termini) n-esime, e c rappresenta una costante. Questa scomposizione è chiamata anche **serie di Fourier**. La funzione può essere ricostruita a partire dalla sua serie di Fourier: se il periodo T è noto e le ampiezze sono definite, la funzione originale del tempo si ricava eseguendo le somme della Equazione (2-1).

Un segnale che ha una durata finita (e questa è una caratteristica comune a tutti segnali) può essere gestito immaginando semplicemente che esso ripeta infinite volte l'intero schema; quindi l'intervallo compreso tra T e $2T$ è identico all'intervallo che va da 0 a T, e così via.

Le ampiezze a_n si possono calcolare per ogni $g(t)$ data, moltiplicando entrambe le parti dell'Equazione (2-1) per $sen(2pkft)$ e poi integrando da 0 a T. Poiché vale

$$\int_0^T \sin(2\pi kft)\sin(2\pi nft)\,dt = \begin{cases} 0 \text{ se } k \neq n \\ T/2 \text{ se } k = n \end{cases}$$

alla fine rimane solo un termine della sommatoria: a_n. La sommatoria b_n sparisce completamente.

In modo analogo è possibile derivare b_n moltiplicando l'Equazione (2-1) per $cos(2pkft)$ e integrando tra 0 e T. Per trovare c si integrano entrambe le parti dell'equazione così com'è. I risultati ottenuti eseguendo queste operazioni sono elencati di seguito:

$$a_n = \frac{2}{T}\int_0^T g(t)\sin(2\pi nft)\,dt \qquad b_n = \frac{2}{T}\int_0^T g(t)\cos(2\pi nft)\,dt \qquad c = \frac{2}{T}\int_0^T g(t)\,dt$$

Segnali a banda limitata

La rilevanza di tutto ciò nella comunicazione dei dati è che i canali reali influiscono in modo non omogeneo sui segnali a diverse frequenze. Consideriamo un esempio specifico: la trasmissione del carattere ASCII "b" codificato in un byte (composto da 8 bit). La sequenza dei bit da trasmettere è 01100010. La parte sinistra della Figura 2.1(a) mostra la tensione di uscita del computer trasmittente. L'analisi di Fourier di questo segnale produce i coefficienti:

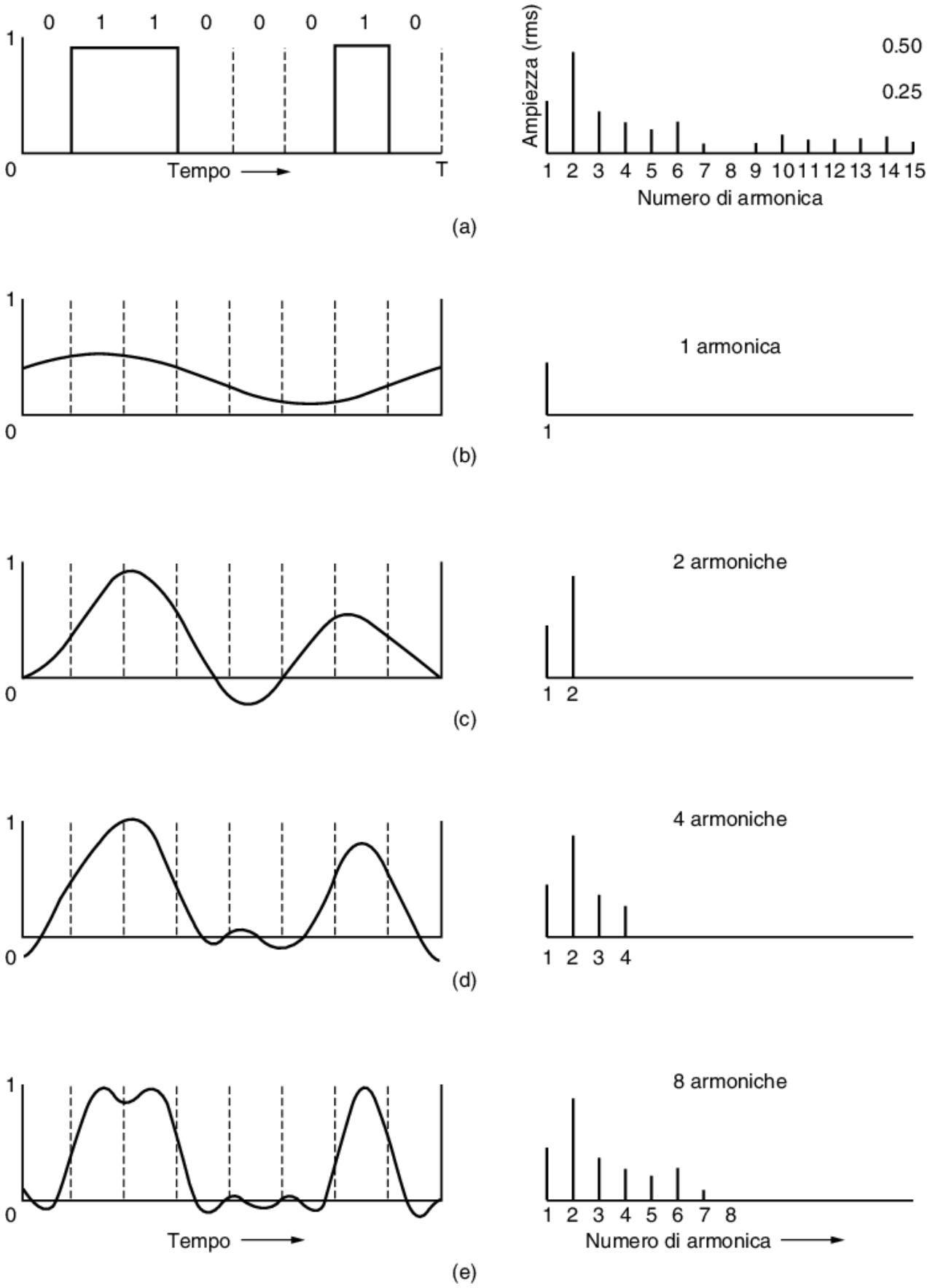


Figura 2.1 (a) Un segnale binario e la media quadratica delle ampiezze rms dei coefficienti di Fourier. (b)-(e) Approssimazioni successive del segnale originale.

$$a_n = \frac{1}{\pi n}[\cos(\pi n/4) - \cos(3\pi n/4) + \cos(6\pi n/4) - \cos(7\pi n/4)]$$

$$b_n = \frac{1}{\pi n}[\sin(3\pi n/4) - \sin(\pi n/4) + \sin(7\pi n/4) - \sin(6\pi n/4)]$$

$$c = 3/4$$

La media quadratica (rms) $\sqrt{(a_n^2 + b_n^2)}$ delle ampiezze relative ai primi termini è rappresentata nel lato destro della Figura 2.1(a). Tali grandezze sono importanti, in quanto il loro quadrato è proporzionale all'energia trasmessa alla frequenza corrispondente.

Nessun mezzo di trasmissione è in grado di trasmettere segnali senza perdere parte dell'energia durante il processo. Se tutte le componenti di Fourier fossero attenuate in modo uniforme, il segnale risultante verrebbe ridotto in ampiezza, ma non risulterebbe distorto; quindi avrebbe la stessa forma squadrata rappresentata nella Figura 2.1(a). Sfortunatamente i mezzi di trasmissione non attenuano le componenti della serie di Fourier in modo uniforme, e ciò genera una distorsione. Di solito, su un cavo, le ampiezze sono trasmesse senza modifiche da 0 fino a una certa frequenza f_c (misurata in cicli al secondo o Hertz) e attenuate per tutte le frequenze superiori a questo limite. L'intervallo di frequenze trasmesse senza una forte attenuazione è chiamato **banda** (*bandwidth*). Nella realtà il limite non è molto preciso, perciò spesso viene indicata la banda passante compresa tra 0 e la frequenza dove la potenza è attenuata del 50%.

La banda passante è una proprietà fisica del mezzo di trasmissione e di solito dipende dalla costruzione, dallo spessore e dalla lunghezza del mezzo. In alcuni casi nel circuito s'introduce un filtro per limitare l'ampiezza della banda. I canali wireless 802.11 possono usare grosso modo fino a 20 MHz, per cui le interfacce radio filtrano la banda del segnale a questa dimensione.

Come ulteriore esempio, i canali della televisione tradizionale (analogica) occupano ognuno 6 MHz sia su cavo sia nell'etere. Effettuare questa operazione di filtro permette a più segnali di condividere una data regione dello spettro e migliora l'efficienza globale del sistema. Questo vuol dire che la gamma di frequenze dei segnali non parte da zero, ma questo ha poca importanza. La larghezza di banda è comunque la larghezza della banda delle frequenze e l'informazione che può essere trasportata dipende solo da questa larghezza e non dalle frequenze di inizio e fine. I segnali che partono da 0 fino a una frequenza massima prendono il nome di *banda base* (**baseband**), mentre i segnali che vengono traslati per occupare una gamma di frequenze più alta, come nel caso delle trasmissioni wireless, vengono chiamati segnali in *banda passante* (**passband**).

Consideriamo ora che aspetto avrebbe il segnale rappresentato nella Figura 2.1(a) se la banda disponibile fosse così scarsa da consentire solo la trasmissione delle frequenze più basse, ossia se la funzione fosse approssimata utilizzando soltanto i primi termini dell'Equazione (2-1). La Figura 2.1(b) mostra il segnale generato su un canale che fa passare solo la prima armonica (quella

fondamentale, *f*). In modo analogo, le Figure 2.1(c), 2.1(d) e 2.1(e) mostrano lo spettro e le funzioni ricostruite per i canali di ampiezza di banda più elevata. Nelle trasmissioni digitali l'obiettivo è ricevere un segnale in modo sufficientemente fedele da poter ricostruire la sequenza di bit inviata. È già possibile farlo facilmente nella Figura 2.1(e) per cui è inutile usare più armoniche per ricevere una replica più accurata.

Dato un tasso di invio dei bit pari a *b* bit/s, il tempo richiesto per inviare 8 bit, un bit alla volta, è pari a 8/*b* al secondo, perciò la frequenza della prima armonica è *b*/8 Hz. Una linea telefonica ordinaria, detta anche **linea voice-grade** (*con qualità vocale*) ha una frequenza limite introdotta artificialmente appena sopra i 3.000 Hz. Questa restrizione indica che il numero della più alta armonica che attraversa il canale di trasmissione è approssimativamente 3.000/(*b*/8) o 24.000/*b* (il limite non è preciso).

Per alcune velocità di trasmissione i numeri si comportano come mostrato nella Figura 2.2. Da questi valori appare evidente che quando si tenta di trasmettere dati a 9.600 bps attraverso una linea telefonica con qualità vocale, il segnale mostrato nella Figura 2.1 (a) si trasforma in qualcosa di simile al segnale della Figura 2.1(e), situazione che rende particolarmente difficile ricevere il flusso di bit originario. Dovrebbe apparire ovvio che i segnali digitali non hanno alcuna speranza di transitare a velocità superiori ai 38,4 kbps, anche quando il sistema di trasmissione è privo di rumore. In altre parole, il limite imposto all'ampiezza di banda vincola la velocità dei dati anche su canali perfetti, ma esistono schemi di codifica che fanno uso di diversi livelli di tensione per ottenere velocità di trasmissione dati superiori. Questo argomento sarà trattato più avanti.

Bps	T (ms)	Prima armonica (Hz)	Numero di armoniche trasmesse
300	26,67	37,5	80
600	13,33	75	40
1.200	6,67	150	20
2.400	3,33	300	10
4.800	1,67	600	5
9.600	0,83	1.200	2
19.200	0,42	2.400	1
38.400	0,21	4.800	0

Figura 2.2 Relazione tra tasso di trasmissione dati e armoniche per il caso considerato.

C'è molta confusione sulla definizione di banda, in quanto essa assume significati differenti nei campi dell'ingegneria e dell'informatica. Per un ingegnere, la banda (analogica) è, come descritto precedentemente, una quantità da misurare in Hz. Per un informatico, la banda (digitale) è il massimo tasso con cui un canale è in grado di trasportare dati e si misura in bit al secondo. Questo tasso è il risultato che si ottiene utilizzando una banda analogica per il trasferimento di dati digitali all'interno di un canale; le due grandezze sono correlate, come vedremo in seguito. All'interno di questo libro la distinzione tra banda analogica (Hz) e banda digitale (bit/s) sarà sempre chiarita dal contesto.

Velocità massima di trasmissione di un canale

Nel 1924 Henry Nyquist, ingegnere di AT&T, si rese conto che anche un canale perfetto ha una capacità di trasmissione limitata e ottenne un'equazione che esprimeva la velocità massima di trasmissione dati di un canale senza rumore ad ampiezza di banda limitata. Nel 1948 Claude Shannon portò avanti il lavoro di Nyquist, estendendolo al caso di un canale soggetto a rumore casuale (termodinamico) (Shannon, 1948). Questo paragrafo riepiloga brevemente i risultati, ormai classici, dei loro studi.

Nyquist dimostrò che se si trasmette un segnale arbitrario attraverso un filtro passa basso la cui ampiezza di banda è pari a B, il segnale filtrato può essere ricostruito completamente prendendo solo $2B$ (esatti) campioni al secondo. Campionare la linea più velocemente di $2B$ volte al secondo non ha senso, perché le componenti delle frequenze superiori sono già state filtrate. Se il segnale è composto da V livelli discreti, il teorema di Nyquist afferma che:

$$B_2 \; V \quad (2\text{-}2)$$

Per esempio, un canale a 3 kHz senza rumore non è in grado di trasmettere segnali binari (cioè a due livelli) a velocità maggiore di 6.000 bps.

Finora sono stati considerati solo i canali senza rumore; se sul canale è presente un rumore casuale, la situazione peggiora rapidamente, ma purtroppo il rumore casuale (termico) provocato dal movimento delle molecole del sistema è sempre presente. Il livello di rumore termico si misura facendo il rapporto tra la potenza del segnale e la potenza del rumore chiamato **SNR** (*signal-to-noise ratio*, rapporto segnale-rumore). Chiamando S la potenza del segnale e N la potenza del rumore, il rapporto segnale-rumore è pari a S/N. Di solito, invece di indicare direttamente il rapporto si cita la quantità $10 \log_{10} S/N$ che è misurata in **decibel** (**dB**), dove "bel" è stato scelto in onore di Alexander Graham Bell, l'inventore del telefono. Poiché un rapporto S/N di 10 è pari a 10 dB, un rapporto uguale a 100 è pari a 20 dB, un rapporto uguale a 1.000 equivale a 30 dB e così via. I produttori di amplificatori stereo spesso caratterizzano la banda passante (intervallo di frequenze) in cui l'apparecchio è lineare indicando i due estremi di frequenza dove l'attenuazione è pari a 3 dB: sono i punti dove il fattore di amplificazione è approssimativamente dimezzato (perché $10 \log_{10} 0{,}5 \approx -3$). Il risultato principale ottenuto da Shannon si può riassumere così: il massimo tasso di invio dei dati (o capacità) in bit/s su un canale rumoroso la cui ampiezza di banda è di B Hz e il cui rapporto segnale-rumore è pari a S/N, è dato dal numero

$$B_2 \; S \quad /N \quad (2\text{-}3)$$

che definisce la massima capacità di un canale fisico. Ad esempio, una linea **ADSL** (*asymmetric sigital subscriber line*), che fornisce connettività Internet facendo uso di canali telefonici standard, utilizza una banda di circa 1 MHz. Il rapporto segnale-rumore è fortemente dipendente dalla distanza percorsa, e un SNR di circa 40 dB per una linea ADSL di 1 o 2 km è un valore più che accettabile. Con queste caratteristiche, il canale non è in grado di trasmettere molto più di 13 Mbps, indipendentemente dalla quantità dei segnali e dalla frequenza di campionamento. Nella pratica, la capacità di una linea ADSL viene indicata essere di 12 Mbps, anche se gli utenti vedono spesso valori minori. Questa capacità di trasferimento è molto buona se pensiamo ai 60 e più anni di evoluzione tecnologica che hanno ridotto drasticamente il divario tra il valore calcolato da Shannon e le prestazioni dei sistemi reali.

Il risultato di Shannon deriva dalla teoria dell'informazione e si applica a qualunque canale soggetto a rumore termico. Esempi contrari devono essere considerati alla stregua di quelli utilizzati per descrivere le macchine a moto perpetuo. Per far sì che l'ADSL superi i 13 Mbps occorre migliorare il valore di SNR (aggiungendo, ad esempio, dei ripetitori di segnale) o usare più banda, come nel caso dell'evoluzione verso ASDL2+.

2.2 Mezzi di trasmissione vincolati

Lo scopo del livello fisico è trasportare un flusso grezzo di bit da una macchina a un'altra. È possibile utilizzare diversi tipi di mezzi fisici per realizzare una trasmissione; ognuno è caratterizzato da specifica banda passante, ritardo, costo e facilità di installazione e manutenzione. I mezzi di trasmissione sono generalmente divisi in mezzi vincolati (cavi in rame, fibre ottiche e così via) e mezzi non vincolati (onde radio, laser e così via). I paragrafi successivi esaminano in dettaglio le loro caratteristiche.

Supporti magnetici

Uno dei sistemi più comuni adottati per trasferire i dati da un computer a un altro funziona nel modo seguente: si scrivono le informazioni su un nastro magnetico o su un supporto rimovibile (per esempio un DVD), si trasporta fisicamente il nastro o il disco e infine si utilizza l'apposita unità installata nel computer di destinazione per leggere i dati. Anche se non è sofisticato al pari della comunicazione satellitare geosincrona, questo metodo è spesso più economico, soprattutto nel caso di applicazioni in cui un'elevata ampiezza di banda o il costo per bit trasmesso rappresentano i fattori chiave.

Un semplice calcolo renderà tutto più chiaro. Un nastro in standard Ultrium può immagazzinare 800 GB di dati; un alloggiamento grande 60 × 60 × 60 cm può contenere 1.000 di questi nastri, per una capacità totale di 800 TB (terabyte) o 6400 terabit (pari a 6,4 peta-bit). Una scatola di nastri come questa può essere spedita ovunque negli Stati Uniti in 24 ore con un corriere. L'ampiezza di banda effettiva di questa trasmissione è pari a 6400 terabit in 86.400 secondi, ossia 70 Gbps. Se la destinazione si trova a un'ora di strada, l'ampiezza di banda supera i 1700 Gbps. Nessuna rete di calcolatori è in grado di raggiungere questa velocità. Naturalmente, le reti stanno diventando più veloci, ma anche la densità dei nastri cresce.

Un quadro analogo si ottiene analizzando il costo dell'operazione: un singolo nastro costa circa 40 dollari e può essere utilizzato almeno dieci volte, perciò il costo di un alloggiamento (200 TB di dati) è di circa 4.000 dollari per utilizzo; se a questa cifra si aggiungono i circa 1000 dollari (e magari molto meno) per il trasporto si ottiene un costo totale di 5000 dollari per trasportare 800 TB, il che vuol dire un gigabyte per meno di mezzo centesimo. Nessuna rete può battere questa cifra. La morale di tutto questo è:

> *mai sottovalutare l'ampiezza di banda di una station wagon piena di nastri lanciata a tutta velocità lungo l'autostrada.*

Doppino

L'ampiezza di banda del nastro magnetico è eccellente, ma il suo ritardo è eccessivo; la durata della trasmissione è espressa in minuti e in ore, non in millisecondi. Molte applicazioni richiedono una connessione on-line: uno dei mezzi di trasmissione più vecchi, ma ancora molto in voga, è **il doppino** (*twisted pair*). Un doppino è composto da due conduttori di rame isolati, spessi circa 1

mm, avvolti uno intorno all'altro in una forma elicoidale che ricorda un po' quella della molecola del DNA. L'intreccio è utilizzato perché due cavi paralleli formano un'eccellente antenna; quando invece i cavi sono intrecciati, i campi elettromagnetici generati dai due conduttori si annullano a vicenda, perciò il cavo irradia meno. Un segnale è generalmente costituito da una differenza di potenziale tra i due cavi della coppia; questo fornisce una maggiore protezione dal rumore esterno che, influenzando entrambi i cavi, mantiene inalterato il valore di questa differenza.

L'applicazione più comune del doppino è il sistema telefonico. Quasi tutti i telefoni sono collegati alla centrale telefonica attraverso un doppino. Sia le chiamate telefoniche sia l'accesso a Internet tramite ADSL fanno uso di queste linee. I doppini possono raggiungere una lunghezza di alcuni chilometri senza bisogno di amplificazione, ma per distanze maggiori il segnale si attenua troppo ed è necessario fare uso di ripetitori. Quando molti doppini procedono in parallelo per lunghe distanze, come accade per i cavi che collegano uno stabile alla centrale telefonica, è necessario fasciare insieme i cavi, rivestendoli con una guaina protettiva; se non ci fosse l'intreccio, i doppini in questi fasci interferirebbero l'uno con l'altro. Nelle zone in cui le linee telefoniche sono aeree, capita di vedere sui pali telefonici fasci di cavi con diametro di alcuni centimetri.

I doppini si possono usare per trasmettere segnali analogici e digitali. L'ampiezza di banda dipende dal diametro del cavo e dalla distanza percorsa, ma il più delle volte per tratti lunghi pochi chilometri è possibile raggiungere velocità di alcuni megabit al secondo. Per il loro basso costo e il discreto livello di prestazioni, i doppini sono largamente utilizzati e probabilmente lo saranno anche per gli anni a venire.

Esistono diverse varietà di doppini, quella più comune e posata nella maggior parte degli edifici a uso ufficio prende il nome di **categoria 5** o "Cat. 5". Un doppino di categoria 5 consiste di due cavi elettricamente isolati e intrecciati tra loro. Quattro di queste coppie sono raccolte all'interno di una guaina di plastica che le protegge e le mantiene unite. Questa disposizione è visibile nella Figura 2.3.

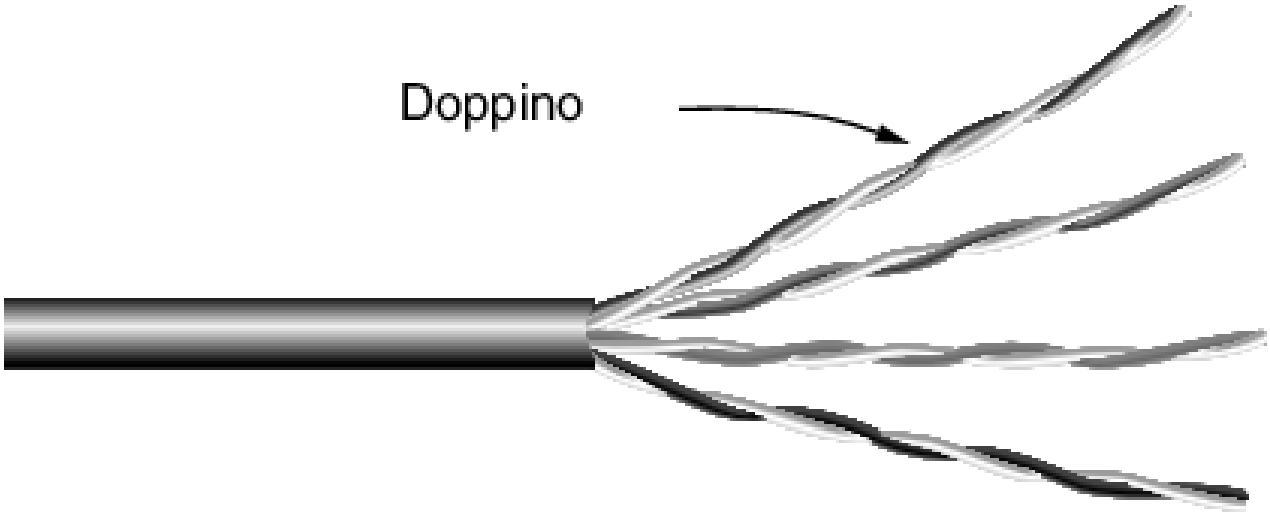


Figura 2.3 Cavo UTP in categoria 5 con quattro doppini.

Standard differenti di LAN possono usare i doppini in maniera diversa. Ad esempio, Ethernet a 100 Mbps usa solo due coppie: una per ogni direzione. Per raggiungere velocità maggiori, Ethernet a 1 Gbps fa uso di tutte e quattro le coppie in entrambe le direzioni; questo richiede a chi trasmette di tenere conto, ed eliminare, il segnale trasmesso.

Per chiarire la terminologia, collegamenti utilizzabili in entrambe le direzioni contemporaneamente, come strade a due corsie, vengono definiti **full-duplex**. Di contro, collegamenti utilizzabili in entrambe le direzioni ma che ne sfruttano solo una alla volta, come una strada a senso unico alternato, prendono il nome di collegamenti **half-duplex**. Una terza categoria consiste nei collegamenti che trasportano traffico in una sola direzione, come una strada a senso unico; questi vengono definiti **simplex**.Tornando al doppino, Cat 5 ha rimpiazzato la precedente **categoria 3** con un cavo quasi identico facente uso dello stesso connettore, ma con un numero di intrecci più elevato per ogni metro. Il maggiore numero di intrecci permette di diminuire il crosstalk[1] e ottenere una migliore qualità del segnale sulle lunghe distanze; in questo modo i cavi sono più adatti a sostenere comunicazioni ad alta velocità, come nel caso di LAN a 100 Mbps e 1 Gbps.
È probabile che cablaggi recenti vengano fatti con cavi di **categoria 6** se non addirittura **categoria 7**. Queste categorie hanno delle specifiche molto stringenti riguardo alla gestione di segnali con banda larga. Alcuni cavi in categoria 6 e oltre sono standardizzati per segnali di 500 MHz e possono supportare i collegamenti a 10 Gbps che saranno disponibili a breve.

Fino alla categoria 6 questi cablaggi sono identificati come **UTP** (*unshielded twisted pair*, doppino non schermato), in quanto consistono solamente di cavi e isolanti. Diversamente, i cavi in categoria 7 possiedono una schermatura su ogni singolo doppino come pure attorno all'intero cavo. La schermatura riduce la sensibilità alle interferenze esterne e al crosstalk con cavi nelle vicinanze al fine di poter raggiungere prestazioni molto elevate. Questi cavi ricordano i doppini schermati prodotti da IBM all'inizio degli anni '80, che però erano ingombranti e molto costosi e non sono stati molto utilizzati al di fuori delle stesse installazioni IBM.

Cavo coassiale

Un altro mezzo di trasmissione molto comune è il **cavo coassiale** (detto anche *coax* e pronunciato "co-ax"). Essendo più schermato del doppino, il cavo coassiale può estendersi per distanze più lunghe e consente velocità più elevate. Esistono due tipi di cavi coassiali: il primo, a 50 ohm, è stato utilizzato per le trasmissioni digitali fin dall'inizio. L'altro tipo, il cavo a 75 ohm, è tipicamente usato per le trasmissioni analogiche e la TV via cavo. Questa distinzione si deve a motivi storici più che tecnologici (le prime antenne dipolari avevano un'impedenza di 300 ohm ed era semplice utilizzare trasformatori con rapporto di impedenza 4:1). A partire da metà degli anni '90 gli operatori di TV via cavo cominciarono a offrire accesso a Internet, aumentando di conseguenza l'importanza dei cavi a 75 ohm nel campo della trasmissione dati.

Un cavo coassiale è composto da un nucleo conduttore coperto da un rivestimento isolante, a sua volta circondato da un conduttore cilindrico solitamente realizzato con una calza di conduttori sottili, che infine è avvolto da una guaina protettiva di plastica. La Figura 2.4 mostra lo spaccato di un cavo coassiale.

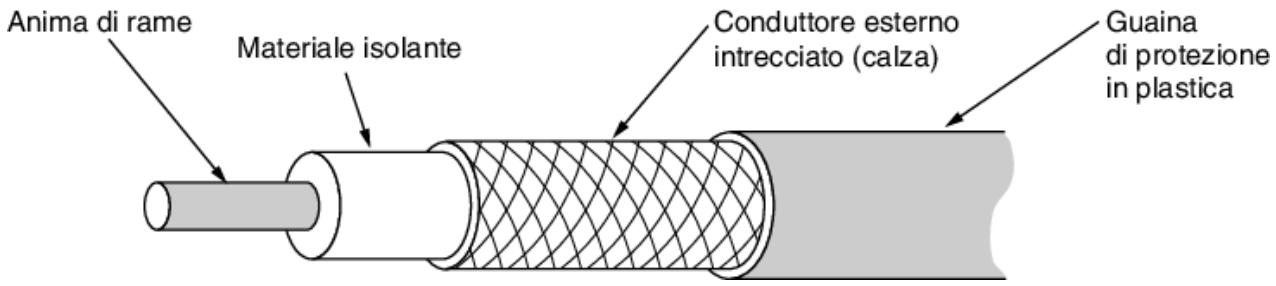


Figura 2.4 Un cavo coassiale.

La costruzione e la schermatura del cavo coassiale forniscono ampiezza di banda ed eccellente immunità al rumore. La banda disponibile dipende dalla qualità e dalla lunghezza del cavo. I cavi moderni hanno un'ampiezza di banda fino a qualche GHz. I cavi coassiali furono ampiamente usati dalle aziende telefoniche per le connessioni sulle lunghe distanze, ma in seguito sono stati sostituiti dalla fibra ottica nelle tratte più lunghe. Il cavo coassiale è ancora comunque molto utilizzato per le reti metropolitane e per la televisione via cavo.

Linee elettriche

Le reti telefoniche e per la TV via cavo non sono gli unici cablaggi che possono essere riutilizzati per la trasmissione dati. Esiste, infatti, un tipo di cablaggio ancora più comune: le linee elettriche. Le linee elettriche portano energia alle abitazioni e il circuito interno alle stesse distribuisce questa energia a tutte le prese.

L'idea di usare le linee elettriche per la trasmissione dati non è recente; sono state già usate per molti anni dalle compagnie elettriche per **comunicazioni a basso tasso di invio** o **bit rate** (ad esempio, il monitoraggio remoto) e nelle case per pilotare dispositivi domestici (come nel caso dello standard X10). Recentemente, si è assistito a un rinnovato interesse per comunicazioni con alti tassi di invio su questi tipi di canali, sia dentro le mura domestiche, come le reti LAN, sia fuori da esse come l'accesso a Internet a banda larga. In questo contesto ci concentreremo sullo scenario più comune: l'uso di cavi elettrici all'interno di un'abitazione.

La semplicità d'impiego delle linee elettriche per una rete dovrebbe essere chiara. Semplicemente inserendo le spine di un televisore e un decodificatore video nelle prese elettriche (cosa che dovreste fare comunque, perché questi hanno bisogno di alimentazione) i due dispositivi sono in grado di inviare e ricevere filmati attraverso l'impianto elettrico. Tale situazione è illustrata nella Figura 2.5. Non ci sono altre prese o segnali radio: il segnale dati è sovrapposto al segnale elettrico a bassa frequenza.

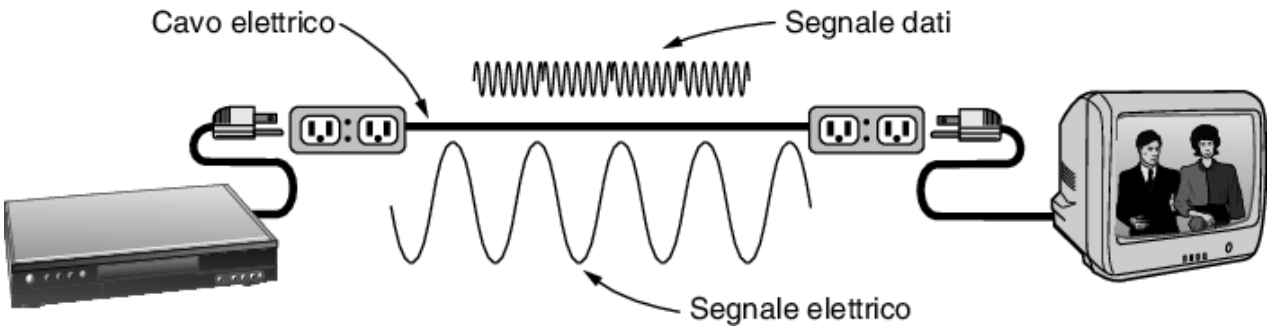


Figura 2.5 Una rete che fa uso di un cablaggio elettrico domestico.

La difficoltà nell'usare un circuito elettrico casalingo per la trasmissione dati sta nel fatto che questo è stato progettato per distribuire segnale elettrico, cosa molto diversa rispetto ai dati, per cui questo tipo di cablaggio ha delle pessime prestazioni. Il segnale elettrico viene inviato a 50-60 Hz e il mezzo trasmissivo attenua le frequenze più alte (dell'ordine dei MHz) richieste dalle trasmissioni dati. Le caratteristiche elettriche dei cavi, inoltre, variano da un'abitazione all'altra e risentono del fatto che gli elettrodomestici siano accesi o meno. Il transitorio di alimentazione di un elettrodomestico che viene acceso genera del rumore su un ampio spettro di frequenze e, se il doppino non è intrecciato a dovere, l'impianto si trasforma in una sottile antenna che riceve e irradia segnali; per questo motivo il segnale dati, per sottostare alle norme in vigore, deve escludere frequenze allocate ad altri scopi come quelle delle radio amatoriali.

Nonostante queste difficoltà, è possibile inviare almeno 100 Mbps facendo uso del circuito elettrico di un'abitazione se si adottano tecniche che possano resistere a frequenze disturbate e sequenze di errori. Molti prodotti commerciali adottano standard proprietari, per cui standard a livello internazionale sono attualmente oggetto di studio.

Fibre ottiche

Nell'industria informatica sono molti a essere orgogliosi del rapido sviluppo della tecnologia, in quanto segue la legge di Moore che prospetta un raddoppiamento del numero di transistor in ogni processore grosso modo ogni biennio (Schaller, 1997). Il primo PC IBM (prodotto nel 1981) aveva una velocità di clock pari a 4,77 MHz. Ventotto anni dopo, i PC utilizzano un processore quad-core a 3 GHz, con un fattore di miglioramento di 2500, 16 per decennio.

Nello stesso arco temporale, le comunicazioni geografiche sono passate da 45 Mbps (una linea T3 nel sistema telefonico) a 100 Gbps (una moderna linea a lunga distanza) con un fattore di miglioramento superiore a 2000 (e di quasi 16 per decennio), mentre la frequenza degli errori è passata da 10^{-5} per bit a circa 0. Inoltre, mentre le singole CPU stanno raggiungendo limiti fisici costruttivi e si preferisce ora aumentare il loro numero all'interno del chip, con l'attuale tecnologia della fibra ottica l'ampiezza di banda raggiungibile va oltre i 50.000 Gbps (50 Tbps) e siamo ancora molto lontani dal raggiungere quella soglia. Oggi, il limite pratico per la trasmissione dei segnali,

pari a circa 100 Gbps, deriva dalla nostra incapacità di convertire più velocemente i segnali da elettrici a ottici. Per poter ottenere collegamenti con capacità più elevata, più canali trasmissivi vengono trasportati per mezzo di una singola fibra.

In questo paragrafo ci occuperemo di fibre ottiche per capire il funzionamento della tecnologia trasmissiva. Nella continua contesa tra calcolo e comunicazione quest'ultima potrebbe avere il sopravvento grazie alle reti in fibra ottica. Le conseguenti implicazioni potrebbero essere la disponibilità di banda infinita e l'accettazione del fatto che i calcolatori saranno sempre più lenti; quindi la rete dovrebbe evitare in ogni modo possibile di fare elaborazioni, non importa quanta banda venga sprecata per questo. Tale cambiamento necessiterà di tempo per essere accettato da una generazione di informatici e ingegneri abituati a pensare tenendo conto dei vincoli imposti dal rame.

Naturalmente lo scenario descritto non è completo, in quanto non tiene conto degli aspetti economici. Il costo dell'installazione della fibra sull'ultimo miglio per raggiungere il consumatore, eliminando le limitazioni di banda e spettro, è altissimo; inoltre, è energeticamente più oneroso spostare bit piuttosto che fare elaborazioni. Potremmo comunque essere in presenza di zone con trattamenti iniqui dove il calcolo o il trasporto dei dati potrebbero essere essenzialmente gratuiti. Ad esempio, alla periferia di Internet usiamo capacità computazionale e risorse di memorizzazione per comprimere e fare caching di contenuti al fine di sfruttare meglio le linee di accesso, mentre all'interno possiamo fare il contrario: compagnie quali Google spostano grandi quantità di dati attraverso la rete dove è più economico elaborarli o memorizzarli.

Le fibre ottiche sono utilizzate per le trasmissioni a lunga distanza nelle dorsali di rete, le reti locali ad alta velocità (anche se in questo caso il rame è riuscito a proporsi come valida alternativa) e l'accesso a Internet ad alta velocità come **FttH** (*fiber to the home*, fibra fino all'abitazione). Un sistema di trasmissione ottico è formato da tre componenti fondamentali: la sorgente luminosa, il mezzo trasmissivo e il rilevatore. Per convenzione, un impulso di luce indica il valore 1 e l'assenza di luce indica il valore 0. Il mezzo trasmissivo è una fibra di vetro sottilissima. Quando viene colpito dalla luce, il rilevatore genera un impulso elettrico. Collegando a un estremo della fibra una sorgente di luce e un rivelatore all'altro, si crea un sistema di trasmissione unidirezionale che, preso un segnale elettrico, lo converte e lo trasmette sotto forma di un impulso luminoso; all'altra estremità della fibra l'impulso viene convertito nuovamente in segnale elettrico.

Questo sistema di trasmissione è utile perché un interessante principio della fisica gli consente di non disperdere la luce: infatti, quando un raggio luminoso passa da un materiale a un altro, per esempio dal silicio fuso all'aria, si rifrange (si curva) sul confine tra i due materiali come mostrato nella Figura 2.6(a). La figura mostra un raggio incidente che colpisce il confine con un'inclinazione di a_1 gradi ed emerge con un'inclinazione di b_1 gradi. L'entità della rifrazione dipende dalle proprietà dei due materiali e in particolare dal loro indice di rifrazione; per angoli d'incidenza che superano un particolare valore critico, la luce è rifratta indietro nel silicio senza disperdersi nell'aria. Per questo motivo un raggio di luce con angolo d'incidenza pari o superiore all'angolo critico rimane intrappolato nella fibra, come mostrato nella Figura 2.6(b) e può propagarsi per molti chilometri senza di fatto alcuna perdita.

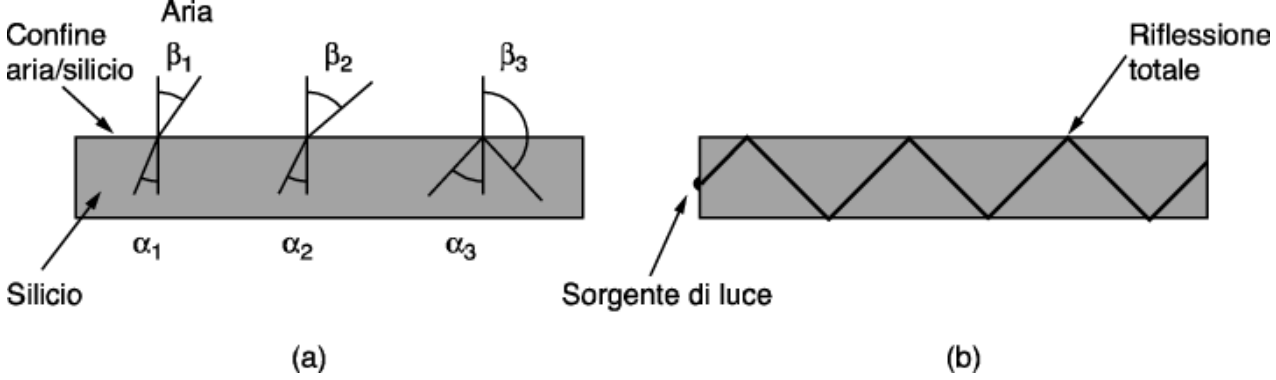


Figura 2.6 (a) Tre esempi di un raggio di luce che dall'interno di una fibra di silicio colpisce il confine aria/silicio con diversi angoli d'incidenza. (b) La luce rimane intrappolata a causa della riflessione totale.

L'immagine rappresentata nella Figura 2.6(b) mostra un solo raggio intrappolato nel mezzo di trasmissione; ma poiché tutti i raggi di luce che colpiscono l'interfaccia con angolo maggiore di quello critico sono riflessi internamente, la fibra può contenere molti raggi che rimbalzano ad angoli diversi. In questo caso si dice che ogni raggio ha una modalità diversa, perciò una fibra dotata di questa proprietà è detta **fibra multimodale**.

Se invece il diametro della fibra viene ridotto fino a poche lunghezze d'onda della luce, la fibra si comporta come una guida d'onda e la luce può propagarsi solo in linea retta, senza rimbalzare; questa variante viene detta **fibra monomodale**; è più costosa ed è utilizzata soprattutto sulle lunghe distanze. Le fibre monomodali disponibili possono trasmettere dati a 100 Gbps per 100 Km senza amplificazione. Velocità più elevate sono state raggiunte in laboratorio per brevi distanze.

Trasmissione della luce attraverso la fibra

La fibra ottica è fatta di vetro che, a sua volta, è composto da sabbia, un materiale grezzo poco costoso disponibile in quantità illimitate. La fabbricazione del vetro era nota già agli antichi Egizi, ma i loro vetri dovevano essere spessi meno di 1 mm per permettere alla luce di passare. Il vetro trasparente adatto alla costruzione di finestre è stato inventato nel Rinascimento. Il vetro utilizzato dalle moderne fibre ottiche è talmente trasparente che, se l'oceano fosse pieno di fibra invece che di acqua, dalla superficie sarebbe possibile osservare il fondale marino così come dal finestrino di un aeroplano si può osservare il terreno in una giornata di sole.

L'attenuazione della luce attraverso il vetro dipende dalla sua lunghezza d'onda (come pure dalle caratteristiche fisiche del vetro) ed è definita come il rapporto tra la potenza del segnale di ingresso e quello di uscita. Per il tipo di vetro usato nelle fibre ottiche, l'attenuazione è mostrata nella Figura 2.7 in decibel per chilometro lineare. Ad esempio, un fattore 2 di perdita di segnale comporta un'attenuazione di 10 *$\log_{10}$ 2 = 3 dB. La parte di spettro usata nella pratica è quella vicina agli infrarossi e riportata nella figura, mentre la luce visibile ha delle lunghezze d'onda più corte: tra 0,4 e 0,7 micron (1 micron corrisponde a 10^{-6} metri). I puristi si riferirebbero a queste lunghezze d'onda come da 400 nm a 700 nm, ma noi ci atterremo all'uso tradizionale.

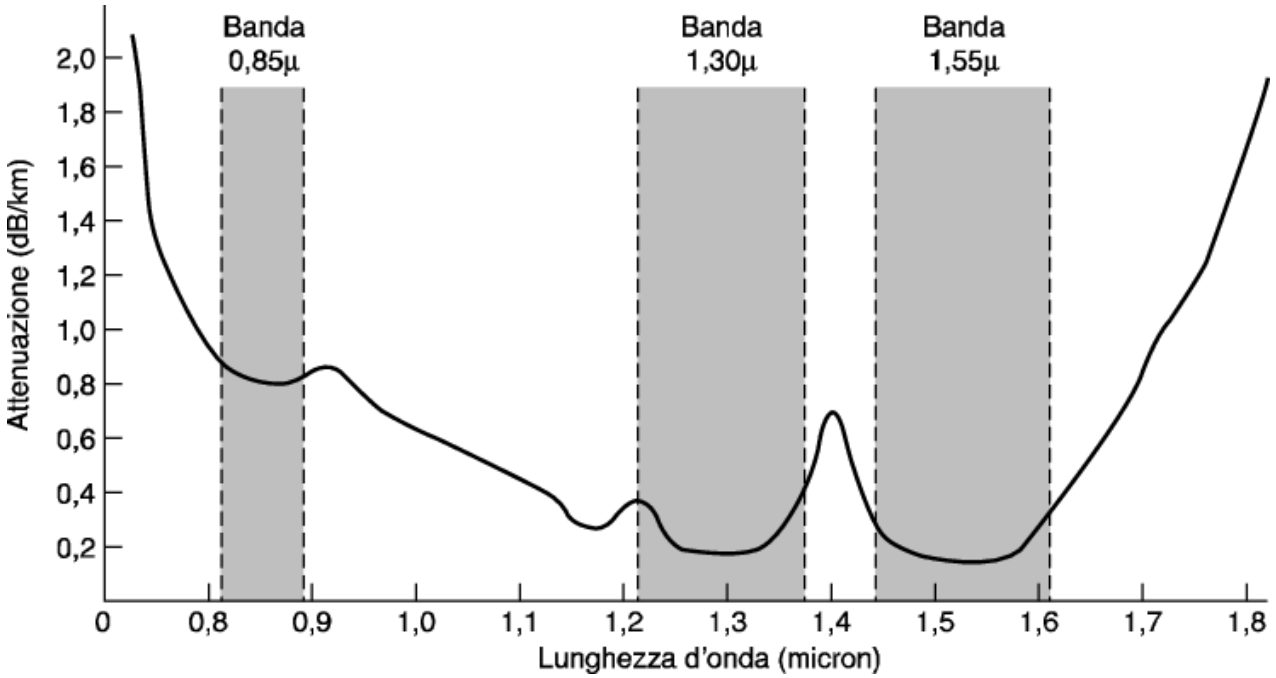


Figura 2.7 Attenuazione della luce attraverso una fibra nella regione dell'infrarosso.

Le lunghezze d'onda più comuni per le comunicazioni ottiche sono tre e sono centrate rispettivamente a 0,85, 1,30 e 1,55 micron con una largheza di banda tra i 25.000 e 30.000 GHz. La banda a 0,85 micron è stata la prima a essere usata; ha il fattore di attenuazione più forte e quindi viene usata solo per brevi distanze, tuttavia a quella lunghezza d'onda laser e parti elettroniche possono essere fatte con lo stesso materiale: arseniuro di gallio. Le altre due bande hanno buone proprietà di attenuazione (meno del 5% per chilometro) e la banda a 1,55 micron è molto usata con amplificatori drogati con erbio, che risultano in grado di operare direttamente nel dominio ottico.

Gli impulsi luminosi trasmessi attraverso la fibra si espandono nella lunghezza d'onda durante la propagazione; il fenomeno si chiama **dispersione cromatica**. Il livello di dispersione cromatica dipende dalla lunghezza d'onda. Per evitare la sovrapposizione degli impulsi diffusi in questo modo è necessario aumentare la distanza tra gli stessi, ma ciò può essere fatto solo riducendo la velocità del segnale. Per fortuna si è scoperto che creando impulsi di una particolare forma (collegata al reciproco del coseno iperbolico) è possibile annullare quasi tutti gli effetti della dispersione e inviare impulsi per migliaia di chilometri senza che la forma subisca modifiche sensibili. Questi impulsi sono chiamati **solitoni**. I laboratori di ricerca si stanno impegnando molto per creare applicazioni pratiche basate sui solitoni.

Cavi in fibra ottica

I cavi in fibra ottica sono simili ai cavi coassiali, ma non sono avvolti dalla calza conduttrice. La Figura 2.8(a) mostra una singola fibra osservata dal lato. Al centro si trova il nucleo (core) di vetro attraverso il quale si propaga la luce; nelle fibre multimodali ha un diametro di 50 micron, circa lo spessore di un capello umano. Il nucleo delle fibre monomodali ha un diametro che varia dagli 8 ai 10 micron.

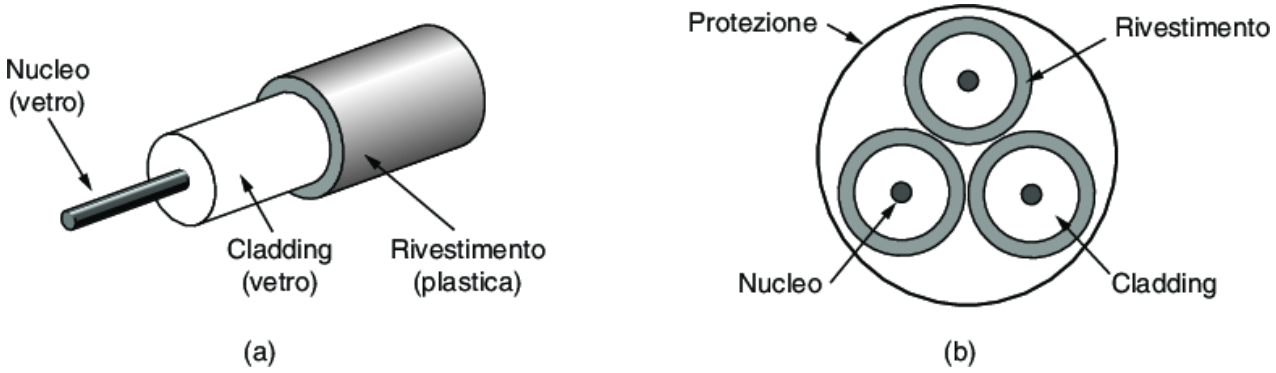


Figura 2.8 (a) Vista laterale di una singola fibra. (b) Vista frontale di una guaina con tre fibre.

Il nucleo è circondato da un rivestimento di vetro (cladding) che ha un indice di rifrazione più basso; ciò costringe la luce a rimanere nel nucleo. Lo strato successivo è una sottile fodera di plastica che protegge il rivestimento. Le fibre di solito sono raggruppate in fasci, protetti da una guaina più esterna. La Figura 2.8(b) mostra una guaina contenente tre fibre.

Le guaine delle fibre terrestri normalmente sono collocate sottoterra a circa un metro dalla superficie; occasionalmente vengono danneggiate da escavatori o talpe. In prossimità della costa, le guaine delle fibre transoceaniche sono sotterrate in fossati da una specie di aratro marino. Nell'acqua profonda queste guaine sono stese sul fondale; possono quindi essere strappate dai sistemi di pesca a strascico e dai calamari giganti.

Le fibre si possono collegare in tre diversi modi. Primo, possono terminare in connettori inseriti in apposite prese; i connettori perdono circa il 10-20% della luce, ma semplificano la riconfigurazione dei sistemi. In secondo luogo, possono essere attaccate meccanicamente: l'estremità di un cavo viene appoggiata all'estremità dell'altro e il segmento, dopo essere stato avvolto in una manichetta speciale, viene pinzato. L'allineamento può essere migliorato passando la luce attraverso la giunzione e apportando piccole correzioni che massimizzano il segnale. Personale addestrato può fare una giunzione meccanica in circa 5 minuti; causa una perdita del 10% della luce.

Infine, due pezzi di fibra possono essere fusi per formare una connessione solida. Una giuntura di questo tipo è buona quasi quanto una fibra uniforme, ma, anche in questo caso, si genera una piccola attenuazione del segnale.

Per tutti e tre i tipi di giunture, le riflessioni avvengono nel punto di giuntura e l'energia riflessa può interferire col segnale.

Per generare il segnale normalmente si impiegano due tipi di sorgenti luminose: i **LED** (*light emitting diode*) e i laser a semiconduttore. I due dispositivi hanno proprietà differenti (Figura 2.9). La lunghezza d'onda può essere regolata inserendo interferometri Fabry-Perot o Mach-Zehnder tra la sorgente e la fibra. Gli interferometri Fabry-Perot sono semplici cavità risonanti composte da due specchi paralleli; la luce incide perpendicolarmente agli specchi. La lunghezza della cavità

esclude le lunghezze d'onda che si propagano un numero intero di volte. Gli interferometri Mach-Zehnder separano la luce in due raggi che viaggiano a distanze leggermente diverse; i raggi sono ricombinati alla fine e sono in fase solo per certe lunghezze d'onda.

Elemento	LED	Laser a semiconduttore
Tasso di trasmissione dei dati	Bassa	Alta
Tipo di fibra	Multimodale	Multimodale o monomodale
Distanza	Breve	Lunga
Durata	Lunga	Breve
Sensibilità alla temperatura	Scarsa	Elevata
Costo	Basso	Alto

Figura 2.9 Confronto tra le sorgenti luminose realizzate con diodi a semiconduttore e LED.

Il lato ricevente di una fibra ottica si compone di un fotodiodo che produce un impulso elettrico quando colpito dalla luce. Il tempo di risposta dei fotodiodi usati per la conversione del segnale da ottico a elettrico limita la velocità di trasmissione dei dati a 100 Gbps. Il rumore termico rappresenta un ulteriore problema: un impulso luminoso deve trasportare abbastanza energia da poter essere rilevato; rendendo gli impulsi abbastanza potenti, il tasso di errore può essere ridotto a piacere.

Confronto tra fibre ottiche e cavi in rame

È interessante confrontare la fibra e il cavo in rame. La fibra offre molti vantaggi, primo fra tutti la maggiore ampiezza di banda; già questa caratteristica la rende necessaria per le reti di fascia alta. A causa del basso livello di attenuazione, i ripetitori possono essere installati ogni 50 Km di linea, mentre nel caso dei cavi in rame i ripetitori devono essere installati ogni 5 Km. Il risparmio è evidente. La fibra ha anche il vantaggio di non essere influenzata dalle sorgenti elettriche, dai campi elettromagnetici e dalle interruzioni della linea elettrica. Poiché non è influenzata dai corrosivi chimici presenti nell'aria, la fibra è adatta anche agli ambienti più inospitali.

Stranamente, le aziende telefoniche utilizzano la fibra per un altro motivo: perché è sottile e leggera. Molti condotti sono completamente pieni e non c'è spazio per aggiungere altri cavi; però, sostituendo tutti i cavi in rame con cavi in fibra, è possibile svuotare i condotti. Inoltre, poiché il rame ha ancora un elevato valore sul mercato, il materiale recuperato può essere venduto ai raffinatori di rame. La fibra, poi, è molto più leggera del cavo in rame: 1 Km di cavo contenente mille coppie di doppini pesa circa 8.000 Kg, due fibre hanno una capacità superiore e pesano solo 100 Kg; ciò riduce enormemente i costi di cablaggio e manutenzione. Infine, le fibre non hanno perdite di luce ed è molto difficile intercettare i dati trasportati. Queste proprietà conferiscono alle

fibre ottiche un buon livello di sicurezza contro i malintenzionati. D'altro canto, la fibra è una tecnologia meno nota, che richiede conoscenze che non tutti gli ingegneri possiedono e si può danneggiare se la si piega troppo. Poiché la trasmissione ottica è intrinsecamente unidirezionale, la comunicazione bidirezionale richiede due fibre o due bande di frequenza su una singola fibra. Infine, le interfacce per la fibra ottica costano più di quelle elettriche. Ciò nonostante, la fibra rappresenta sicuramente il futuro di tutte le comunicazioni dati su reti fisse per distanze che superano i pochi metri. Per una discussione di tutti gli aspetti delle fibre ottiche e delle reti che le usano si veda Hecht (2005).

2.3 Trasmissioni wireless

Questa è l'era dei drogati dell'informazione, persone che hanno la necessità di essere sempre on-line. Per questi utenti mobili, doppini, cavo coassiale e fibre ottiche non hanno alcuna utilità: devono consultare i loro dati attraverso computer portatili, palmari e da polso senza dipendere dai sistemi di comunicazione terrestri. Tale esigenza è soddisfatta solo dalle comunicazioni wireless.

Nei paragrafi che seguono ci occuperemo di generiche comunicazioni wireless che, al di là di fornire connettività a utenti che vogliano accedere al Web anche in spiaggia, possono avere molte importanti applicazioni. Le tecnologie wireless sono in grado di offrire vantaggi anche a dispositivi fissi; ad esempio, se posare una fibra fino a un palazzo è difficoltoso per ragioni ambientali (si pensi a montagne, giungle, paludi o altro), il wireless può rappresentare un approccio migliore.[2] È importante far notare che le moderne comunicazioni wireless hanno avuto origine nelle isole Hawaii dove ampi tratti di oceano separavano gli utenti dai centri di elaborazione e il sistema telefonico era inadeguato a risolvere il problema.

Lo spettro elettromagnetico

Quando gli elettroni si spostano creano onde elettromagnetiche che si propagano attraverso lo spazio, anche nel vuoto. L'esistenza di queste onde è stata intuita dal fisico inglese James Clerk Maxwell nel 1865; la loro esistenza, tuttavia, è stata osservata per la prima volta nel 1887 dal fisico tedesco Heinrich Hertz. Il numero di oscillazioni al secondo di un'onda è chiamato **frequenza**, f, ed è misurato in **Hz** (in onore di Heinrich Hertz). La distanza tra due massimi (o minimi) consecutivi è chiamata **lunghezza d'onda** ed è universalmente indicata dalla lettera greca l (lambda).

Quando un'antenna di dimensioni appropriate è collegata a un circuito elettrico, le onde elettromagnetiche sono trasmesse in modo efficiente e un ricevitore posto a una certa distanza le può captare. Tutte le comunicazioni wireless si basano su questo principio.

Nel vuoto tutte le onde elettromagnetiche viaggiano alla stessa velocità, qualunque sia la loro frequenza. Questa velocità, chiamata **velocità della luce**, c, è pari a circa 3×10^8 m/s, ossia circa 30 cm per nanosecondo. Nei cavi in rame e nelle fibre ottiche la velocità di trasmissione scende a 2/3 di questo valore e diventa leggermente dipendente dalla frequenza. La velocità della luce è il limite ultimo della velocità: nessun oggetto o segnale può muoversi più velocemente.

La relazione fondamentale tra f, l e c (nel vuoto) è:

$$lf = c \quad (2\text{-}4)$$

Essendo c una costante, se si conosce f è possibile ricavare l e viceversa. Come regola generale, quando l è espressa in metri e f è espressa in MHz, lf vale circa 300. Per esempio, onde di 100 MHz sono lunghe circa 3 metri, onde di 1.000 MHz sono lunghe circa 0,3 metri e onde di 0,1 metri hanno una frequenza di 3.000 MHz.

La Figura 2.10 mostra lo spettro elettromagnetico. Le porzioni dello spettro indicate come radio, microonde, infrarosso e luce visibile si possono utilizzare per trasmettere informazioni modulando ampiezza, frequenza o fase delle onde. La luce ultravioletta, i raggi X e i raggi gamma funzionerebbero anche meglio, per le loro elevate frequenze, ma sono difficili da generare e da modulare, non si propagano bene attraverso i muri e sono dannose per gli esseri viventi. Le bande elencate nella Figura 2.10 sono i nomi ITU (*International Telecommunication Union*) ufficiali e si distinguono per le lunghezze d'onda, perciò la banda LF va da 1 a 10 Km (circa da 30 kHz a 300 kHz). I termini LF, MF e HF indicano rispettivamente le frequenze basse (low), medie (medium) e alte (high). Logicamente, quando vennero assegnati i nomi, nessuno si aspettava di superare i 10 MHz, perciò le bande più alte sono state chiamate successivamente bande di frequenza Very, Ultra, Super, Extremely e Tremendously High.

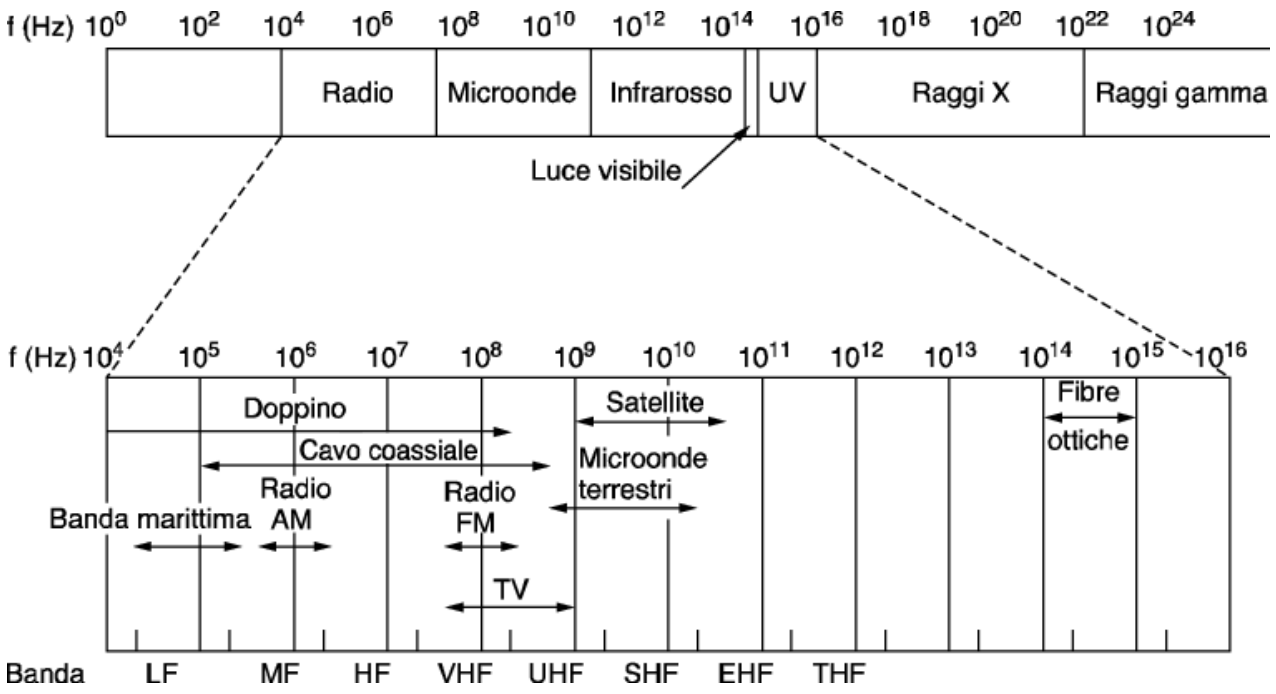


Figura 2.10 Lo spettro elettromagnetico e il suo utilizzo nella comunicazione.

Come dimostrato da Shannon (Equazione 2-3) la quantità d'informazione che un'onda elettromagnetica può trasportare dipende dall'energia ricevuta ed è proporzionale alla sua banda. Dalla Figura 2.10 dovrebbe dedursi come mai le fibre ottiche piacciano così tanto a chi si occupa di reti. Molti GHz di banda sono disponibili per essere sfruttati nella zona delle microonde e ancora di più nelle fibre, in quanto si collocano ancora più a destra nella nostra scala logaritmica. Ad esempio, si consideri un segnale nella banda a 1,30 micron nella Figura 2.7 con un'ampiezza di 0,17 micron; si usa l'Equazione 2-4 per calcolare le frequenze iniziali e finali a partire dalle lunghezze d'onda, si trova una gamma di frequenze di circa 30.000 GHz. Con un rapporto segnale-rumore, realistico, di 10 dB, questo equivale a 300 Tbps.

La maggior parte delle trasmissioni utilizza una banda di frequenze ristretta ($Df/f << 1$) per ottenere la migliore ricezione (molti watt/Hz), ma in certi casi si usa una banda larga con due varianti. Nella tecnica a **spettro distribuito a frequenza variabile** (*frequency hopping spread spectrum*), il trasmettitore salta da una frequenza a un'altra centinaia di volte al secondo. Questa variante è utilizzata nelle comunicazioni militari, perché rende difficile rilevare le trasmissioni ed è quasi impossibile da disturbare. Offre inoltre una buona protezione dalla dispersione multipath e dalle interferenze a banda stretta, in quanto il ricevente non sarà bloccato su una frequenza disturbata abbastanza a lungo da dover interrompere la comunicazione. Data la sua stabilità, questa tecnica è particolarmente utile in zone dello spettro molto affollate, come le bande IMS che tratteremo tra breve, e viene sfruttata commercialmente per Bluetooth e le versioni meno recenti di 802.11.

Una nota curiosa: questa tecnica è stata inventata da Hedy Lamarr (sex symbol di origine austriaca), la prima donna ad apparire nuda in un film (*Estasi*, film Ceco del 1933); il suo primo marito, un fabbricante di armi, le spiegò quanto fosse facile bloccare i segnali radio utilizzati per controllare i siluri. Quando scoprì che il marito vendeva armi a Hitler rimase inorridita; fuggì travestita da cameriera e volò a Hollywood per continuare la sua carriera di attrice. Nel tempo libero inventò il sistema a frequenza variabile con lo scopo di aiutare lo sforzo bellico degli Alleati. Lo schema utilizzava 88 frequenze (che corrisponde al numero di tasti e frequenze del pianoforte). Per questa invenzione lei e il suo amico, il compositore musicale George Antheil, ricevettero il brevetto U.S. 2.292.387. Purtroppo, i due non riuscirono a convincere la Marina degli Stati Uniti che il sistema poteva avere applicazioni pratiche e non ricevettero diritti d'autore. La tecnica divenne popolare solo alcuni anni dopo la scadenza della licenza.

Una seconda tecnica, chiamata **spettro distribuito a sequenza diretta** (*direct sequence spread spectrum*) fa uso di una sequenza codificata per distribuire il segnale su una banda di frequenza molto più ampia ed è molto efficiente, dal punto di vista dello spettro, nel permettere a più segnali di condividere le stesse bande di frequenza. A ogni segnale può essere associato un diverso codice secondo un approccio che prende il nome di **CDMA** (*code division multiple access*, accesso multiplo a divisione di codice), sul quale torneremo più avanti in questo stesso capitolo. Questo metodo, che fornisce le basi per le reti di telefonia mobile 3G e viene usato nel sistema GPS (*global positioning system*), è confrontato con il sistema a frequenza variabile nella Figura 2.11. Anche senza differenziare le codifiche il metodo a sequenza diretta, come quello a frequenza variabile, è in grado di tollerare interferenze su parti limitate di banda e la dispersione multipath, in quanto solo una frazione del segnale totale viene persa. Per un resoconto dettagliato sull'evoluzione delle comunicazioni a spettro distribuito si veda Scholtz (1982).

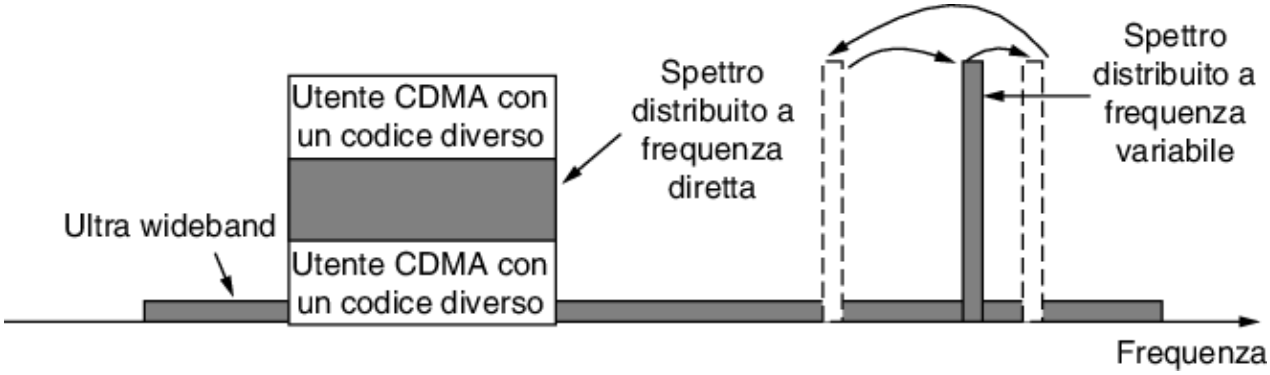


Figura 2.11 Spettro distribuito e comunicazione UWB.

Una terza metodologia per comunicazioni facente uso di una banda ancora più larga prende il nome di **UWB** (*ultra wideband*, banda ultra larga); UWB (anch'esso nella Figua 2.11) trasmette dati tramite una serie di impulsi molto rapidi in posizioni diverse; in questo modo il segnale viene disperso su una banda di frequenza estremamente ampia e viene infatti definito come segnale avente una banda di almeno 500 MHz o di almeno il 20% della frequenza centrale della banda dove opera. Con tutta questa banda a disposizione, UWB ha la potenzialità di trasmettere a velocità elevatissime e siccome è disperso su una banda di frequenza molto ampia può tollerare interferenze relativamente forti da parte di altri segnali a banda più stretta. Inoltre, siccome UWB fa uso di una bassa quantità di energia per ogni frequenza, non causa interferenze quando usato per comunicazioni a corto raggio; in gergo, si dice che si "infila sotto" (*underlay*) agli altri segnali. Questa convivenza pacifica ha permesso l'uso di UWB in PAN con una capacità fino a 1Gbps (anche se queste non sono sempre state un successo commerciale), sistemi di rilevazione d'immagine attraverso corpi solidi (terreno, pareti, muri, persone) e in sistemi di localizzazione ad alta precisione.

Ci occuperemo ora di descrivere come vengono utilizzate le varie parti dello spettro elettromagnetico nella Figura 2.11. Dove non diversamente specificato assumeremo trasmissioni facenti uso di una frequenza di banda stretta.

Trasmissioni radio

Le onde in radio frequenza (RF) sono semplici da generare, possono viaggiare per lunghe distanze e attraversano facilmente gli edifici; per questi motivi sono largamente utilizzate per la comunicazione, sia all'interno delle costruzioni sia all'esterno. Le onde radio sono omnidirezionali, ossia si espandono dalla sorgente in tutte le direzioni, perciò il trasmettitore e il ricevitore non devono essere fisicamente allineati.

Qualche volta la proprietà omnidirezionale è utile, ma altre invece non lo è. Nel 1970 General Motors decise di dotare tutte le sue nuove Cadillac con un sistema ABS controllato da computer. Quando il guidatore premeva il pedale del freno, il computer modulava l'attivazione dei freni invece di attivarli in un colpo solo. Un bel giorno un poliziotto della stradale dell'Ohio, mentre

utilizzava la sua nuova autoradio per chiamare la centrale, si accorse che la Cadillac che viaggiava accanto alla sua auto si comportava come un cavallo imbizzarrito. Quando l'agente fermò l'auto, il conducente affermò di non aver fatto nulla e che la sua auto era improvvisamente impazzita.

Si presentò quindi un quadro bizzarro: le Cadillac qualche volta sembravano impazzire, ma solo sulle autostrade dell'Ohio e solo in prossimità delle auto di pattuglia della stradale. Per molto tempo General Motors non riuscì a capire come mai le Cadillac funzionavano bene in tutti gli altri stati e nelle strade secondarie dell'Ohio; solo dopo molte ricerche gli ingegneri capirono che i cavi elettrici della Cadillac si comportavano come ottime antenne per le frequenze del nuovo sistema di comunicazione radio adottato dalla polizia stradale dell'Ohio.

Le proprietà delle onde radio dipendono dalla frequenza. Alle frequenze più basse, le onde radio attraversano bene gli ostacoli, ma la potenza diminuisce bruscamente allontanandosi dalla sorgente (nell'aria, di almeno $1/r^2$), in quanto l'energia del segnale si distribuisce su una superficie maggiore. Questo tipo di attenuazione prende il nome di **path loss**. Alle frequenze più alte, le onde radio tendono a viaggiare in linea retta e rimbalzano contro gli ostacoli; anche in questo caso il path loss riduce l'energia del segnale, anche se la parte preponderante dell'attenuazione è dovuta alla riflessione. Inoltre, le onde radio ad alta frequenza vengono assorbite più facilmente da pioggia o altri ostacoli rispetto a quelle a bassa frequenza. A tutte le frequenze le onde radio sono soggette a interferenze da parte di motori e altri dispositivi elettrici.

È interessante fare un confronto tra l'attenuazione delle onde radio e quella dei segnali in mezzi trasmissivi cablati; con fibra, coassiale e doppino il segnale decade della stessa frazione per ogni unità di distanza percorsa; ad esempio, 20 dB ogni 100 metri per il doppino. Nelle trasmissioni radio il segnale decresce della stessa frazione al raddoppiare della distanza; ad esempio, 6 dB a ogni raddoppiamento di distanza nel vuoto. Questo comportamento implica che le onde radio possono percorrere lunghe distanze e l'interferenza diventa un problema; per questa ragione l'uso dei trasmettitori radio viene rigidamente regolamentato dai governi, a parte poche eccezioni che verranno analizzate in seguito.

Nelle bande VLF, LF e MF le onde radio seguono il terreno come mostrato nella Figura 2.12(a). Queste onde si possono ricevere per circa 1000 Km alle frequenze più basse e a distanze più brevi alle frequenze maggiori. Le stazioni radio AM utilizzano la banda MF, ecco perché le stazioni radiofoniche AM di Boston non possono essere ascoltate facilmente a New York. Le onde radio in queste bande attraversano facilmente gli edifici, motivo per cui le radio portatili funzionano al chiuso. Nel caso della comunicazione dati, il problema principale di queste bande è la loro ridotta ampiezza di banda (come si può notare osservando l'Equazione 2-4).

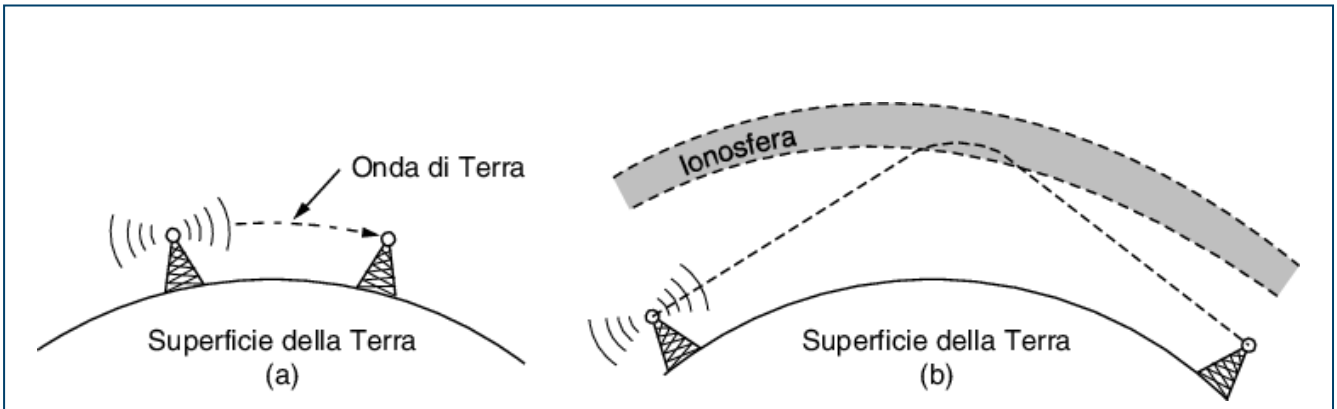


Figura 2.12 (a) Nelle bande VLF, LF ed MF le onde radio seguono la curvatura del pianeta. (b) Nella banda HF rimbalzano contro la ionosfera.

Nelle bande HF e VHF le onde terrestri tendono a essere assorbite dal pianeta, ma le onde che raggiungono la ionosfera (uno strato composto da particelle cariche che circonda la Terra a un'altezza compresa tra i 100 e i 500 Km) sono riflesse e tornano verso il pianeta come mostrato nella Figura 2.12(b). In particolari condizioni atmosferiche, i segnali possono rimbalzare diverse volte. I radioamatori utilizzano queste bande per comunicare su lunghe distanze; anche i militari comunicano attraverso le bande HF e VHF.

Trasmissione a microonde

Sopra i 100 MHz le onde viaggiano quasi in linea retta e pertanto si mettono a fuoco facilmente. Concentrando tutta l'energia in un piccolo raggio per mezzo di un'antenna parabolica (come quella delle TV satellitari) si ottiene un rapporto segnale/rumore molto più alto, ma le antenne trasmittenti e riceventi devono essere accuratamente allineate le une con le altre. Inoltre, questa direzionalità permette a più trasmettitori allineati di comunicare senza interferenze con più ricevitori allineati, purché si osservino alcune regole di distanza minima. Per decenni, prima dell'avvento delle fibre ottiche, le microonde hanno costituito il cuore dei sistemi di trasmissione telefonica su lunghe distanze. MCI, uno dei primi concorrenti di AT&T, costruì il suo intero sistema con comunicazioni a microonde basate su antenne distanti anche decine di chilometri. Già il nome dell'azienda rifletteva questo sistema di comunicazione: MCI è infatti l'acronimo di Microwave Communications, Inc. MCI ha da tempo rinunciato alle fibre ottiche e attraverso una lunga serie di acquisizioni e fallimenti di aziende nel campo delle telecomunicazioni è diventata parte di Verizon.

Poiché le microonde viaggiano in linea retta, se le antenne sono troppo lontane (si pensi per esempio a un collegamento Seattle-Amsterdam) entra in gioco la curvatura della Terra; di conseguenza sono necessari dei ripetitori. Più sono alte le antenne dei ripetitori, maggiore può essere la loro distanza. La distanza tra ripetitori varia grossolanamente con la radice quadrata dell'altezza dell'antenna. Nel caso di antenne alte 100 metri, i ripetitori possono trovarsi a 80 Km di distanza.

A differenza delle onde radio a bassa frequenza, le microonde non attraversano molto bene gli edifici; inoltre, anche se il raggio può essere messo correttamente a fuoco, c'è ancora un po' di divergenza nello spazio. Alcune onde si possono rifrangere sugli strati più bassi dell'atmosfera e arrivano un po' dopo le onde dirette. Le onde in ritardo possono arrivare fuori fase con le onde dirette e questo può annullare il segnale. L'effetto è noto come **multipath fading** (*dissolvenza multipercorso*); dipende dalle condizioni climatiche e dalla frequenza e spesso diventa un problema serio. Alcuni operatori tengono il 10% dei loro canali inattivi come riserva, da utilizzare quando il multipath fading elimina temporaneamente parte della banda di frequenza.

La richiesta sempre maggiore di spettro ha portato gli operatori a utilizzare anche le frequenze più alte. Le bande sopra i 10 GHz sono ormai di uso comune, ma a circa 4 GHz è sorto un nuovo problema: l'assorbimento da parte dell'acqua. Queste onde, lunghe solo pochi centimetri, vengono

assorbite dalla pioggia. L'effetto sarebbe utile se si volesse costruire un enorme forno a microonde esterno per la cottura degli uccelli di passaggio, ma per la comunicazione è un vero problema. Come nel caso del multipath fading, l'unica soluzione è interrompere i collegamenti sui quali sta piovendo e deviare le comunicazioni.

Riepilogando, la comunicazione a microonde è talmente utilizzata nelle comunicazioni telefoniche su lunghe distanze, nella telefonia cellulare, nella televisione e in altre applicazioni, che si sta manifestando una grave scarsità di spettro. Questo sistema di trasmissione offre diversi vantaggi rispetto alla fibra; il principale è che non richiede alcun diritto di passaggio: acquistando un piccolo appezzamento di terreno ogni 50 Km e costruendo su di esso un'antenna a microonde è possibile scavalcare completamente il sistema telefonico e comunicare direttamente. In questo modo MCI è riuscita a diventare in poco tempo un'azienda telefonica per lunghe distanze. Sprint ha seguito una direzione completamente diversa: facendo parte di Southern Pacific Railroad, azienda ferroviaria che già possedeva una gran quantità di diritti di passaggio, ha semplicemente steso le fibre lungo i binari.

Le microonde sono anche abbastanza economiche. L'installazione di due semplici antenne, magari due banali pali controventati, costa meno della posa di 50 Km di fibra in un'area urbana molto congestionata o attraverso una montagna. Magari è anche più economica del noleggio della fibra ottica posseduta da un'azienda telefonica, specialmente se questa azienda non è ancora stata pagata di tutto il rame che ha sradicato durante l'installazione della fibra.

Politiche dello spettro elettromagnetico

Per evitare il caos totale sono stati stipulati accordi nazionali e internazionali sui soggetti e le frequenze utilizzabili. Da una parte ognuno vuole una maggiore velocità di trasmissione dati, dall'altra tutti vogliono più spettro. I governi nazionali assegnano lo spettro per le radio AM e FM, le televisioni e i telefoni cellulari, così come per le aziende telefoniche, la polizia, la marina, l'aeronautica, l'esercito, il governo e gli altri utenti concorrenti. A livello mondiale, un'agenzia di ITU-R (WRC) tenta di coordinare queste assegnazioni in modo che sia possibile produrre dispositivi in grado di funzionare in paesi diversi. Le nazioni però non sono vincolate alle raccomandazioni di ITU-R; la **FCC** (*federal communication commission*), che si occupa dell'assegnazione negli Stati Uniti, ha saltuariamente rifiutato tali indicazioni (in genere perché richiedevano che qualche lobby politicamente forte rinunciasse a un certa porzione dello spettro).

Anche quando una parte dello spettro viene assegnato a un particolare utilizzo, per esempio ai telefoni cellulari, sorge il problema della suddivisione delle frequenze tra i fornitori di servizi. In passato sono stati utilizzati tre algoritmi. Il più vecchio, chiamato anche concorso di bellezza (*beauty contest*), richiedeva che ogni fornitore motivasse il valore della sua proposta per il pubblico interesse; in seguito gli agenti del governo sceglievano la motivazione che sembrava più interessante. Questo sistema era spesso causa di corruzione e scelte arbitrarie. Inoltre, un agente scrupoloso e onesto che pensava che un'azienda straniera potesse essere più conveniente di quelle nazionali sarebbe stato costretto a motivare la sua scelta.Tali osservazioni hanno condotto al secondo algoritmo, basato su una **lotteria** tra le aziende interessate. Il problema qui era che anche aziende non interessate all'utilizzo dello spettro potevano partecipare alla lotteria. Una catena di ristoranti o di negozi di scarpe, dopo aver vinto la gara, poteva rivendere lo spettro a un fornitore di servizi e ottenere un grosso profitto senza alcun rischio. Da queste considerazioni è nato il terzo algoritmo: la **vendita all'asta** di porzioni di spettro elettromagnetico. Quando nel 2000 il governo

britannico mise all'asta le frequenze necessarie per i sistemi di telefonia mobile di terza generazione, calcolava di guadagnare circa 4 miliardi. Al termine dell'asta, il governo riuscì ad accaparrarsi qualcosa come 40 miliardi, dato che le aziende interessate, prese dal timore di perdere il "treno", si fecero travolgere da una frenesia crescente. Questo evento accese molto interesse nei governi vicini e ispirò nuove aste. La cosa ha funzionato, ma ha anche lasciato alcune aziende così piene di debiti da costringerle alla chiusura. Anche nei casi migliori, ci vorranno molti anni per recuperare il costo delle licenze.

Un approccio completamente differente consiste nel non assegnare affatto le frequenze, lasciando che ognuno trasmetta a piacere, ma regolando la potenza utilizzata in modo da limitare la portata delle stazioni per evitare l'interferenza reciproca. Seguendo questo principio la maggior parte dei governi ha riservato alcune bande di frequenza, chiamate **ISM** (*industrial, scientific, medical*), per l'utilizzo senza licenze. Telecomandi che aprono i garage, telefoni senza fili, giocattoli telecomandati, mouse senza fili e numerosi altri apparecchi domestici utilizzano le bande ISM. Per limitare l'interferenza tra questi dispositivi non coordinati, FCC ordina che tutti gli apparecchi nelle bande ISM limitino la potenza trasmissiva (ad esempio, a un watt) e utilizzino tecniche di trasmissione a spettro distribuito. Inoltre, i dispositivi devono anche occuparsi di evitare interferenza con attrezzature radar.

La posizione delle bande ISM varia leggermente da paese a paese. Negli Stati Uniti, per esempio, le bande che i dispositivi possono usare senza chiedere una licenza a FCC sono illustrate nella Figura 2.13. La banda dei 900 MHz è stata usata dalle prime versioni di 802.1, ma è molto affollata. La banda dei 2,4 GHz è disponibile nella maggior parte dei paesi ed è ampiamente utilizzata da 802.11 b/g e Bluetooth, anche se è soggetta a interferenza da forni a microonde e installazioni radar. La parte dello spettro verso i 5 GHz include le bande **U-NII** (*unlicensed national information infrastructure*). Le bande a 5 GHz sono relativamente poco sfruttate ma, siccome sono le più capienti e vengono sfruttate da 802.11a, stanno rapidamente acquistando importanza.

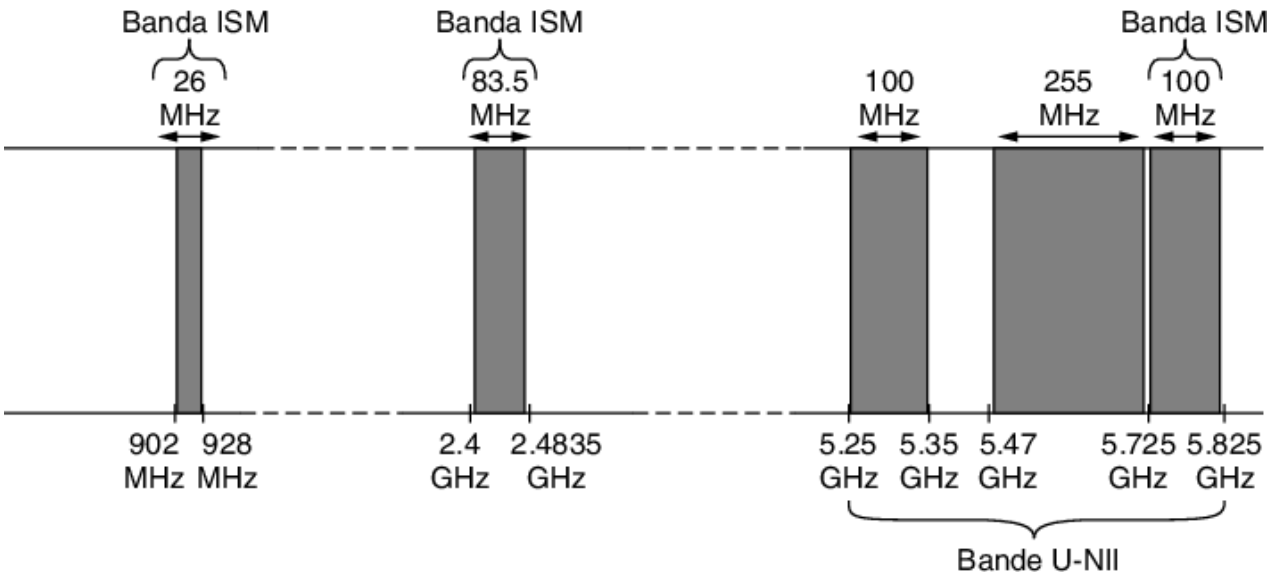


Figura 2.13 Bande ISM e U-NII utilizzate dai dispositivi wireless negli Stati Uniti.

Nell'ultimo decennio le bande non regolamentate hanno riscosso grande successo. La possibilità di utilizzare liberamente lo spettro ha alimentato notevolmente l'innovazione nei campi di LAN e PAN wireless con conseguente crescita di disponibilità di tecnologie come 802.11 e Bluetooth. Per proseguire con questa evoluzione tecnologica è necessario però avere a disposizione più spettro; un interessante sviluppo in questa direzione è stata la decisione presa negli Stati Uniti nel 2009 da FCC di lasciare libero l'uso delle frequenze non utilizzate (*white spaces*, o *spazi bianchi*) attorno ai 700 MHz. Gli spazi bianchi sono bande di frequenza formalmente allocate, ma che non vengono utilizzate in una certa zona geografica; nel 2010, la transizione della televisione da analogica a digitale ha creato negli Stati Uniti degli spazi bianchi attorno ai 700 MHz. L'unica difficoltà sta nel fatto che, per usare gli spazi bianchi, i dispositivi devono essere in grado di rilevare qualunque sistema trasmissivo nei paraggi, inclusi i microfoni wireless, facente uso delle stesse frequenze in maniera regolamentata, avente quindi diritto alla precedenza.

Un'altra zona dove l'attività è frenetica è la banda dai 57 ai 64 GHz, lasciata non regolamentata da FCC nel 2001. Queste frequenze occupano una parte enorme dello spettro, più di tutte le bande ISM sommate, e sono quindi in grado di supportare reti ad alta velocità, ad esempio quelle per fare streaming di video ad alta definizione in ambito domestico. A 60 GHz, le onde radio vengono assorbite dall'ossigeno; per cui questo tipo di segnali non può propagarsi molto distante ed è più appropriato per reti a corto raggio. Le alte frequenze (60 GHz) si collocano nella banda delle **EHF** (*extremely high frequency*) o "millimetrica", appena al di sotto degli infrarossi; all'inizio sono state un problema per i costruttori di apparecchiature, ma oggi sono reperibili prodotti sul mercato.

Trasmissione a infrarossi

Per la comunicazione a corto raggio si usano spesso raggi infrarossi non vincolati. I telecomandi dei televisori, dei videoregistratori e degli stereo adottano tutti la comunicazione a infrarossi. Questo sistema è relativamente direzionale, economico e facile da costruire, ma ha un grande difetto: non riesce ad attraversare ostacoli solidi (provate a mettervi tra il vostro telecomando e la televisione e controllate se funziona ancora). In generale, più ci si allontana dalle onde radio lunghe e ci si avvicina allo spettro della luce visibile, più le onde si comportano come luce e meno come onde radio.

D'altra parte, il fatto che le onde infrarosse non attraversino i muri rappresenta anche un vantaggio; significa che un sistema a infrarossi che si trova in una stanza di un edificio non interferirà con sistemi simili collocati nelle stanze o negli edifici adiacenti: così il nostro telecomando non interferirà con il televisore del vicino. Inoltre, i sistemi a infrarossi sono più sicuri di quelli basati sulle onde radio, in quanto più difficili da intercettare e non richiedono alcuna licenza governativa (come invece accade per i sistemi a onde radio che non utilizzano le bande ISM). I sistemi a infrarossi sono utilizzati anche per collegare i computer portatili alle stampanti tramite lo standard **IrDA** (*infrared data association*), ma hanno un ruolo secondario nella gara delle telecomunicazioni.

Trasmissione a onde luminose

I **segnali ottici non vincolati** (*free-space optic*) sono utilizzati da secoli. Paul Revere trasmise un segnale ottico binario dalla Old North Church prima di iniziare la sua famosa corsa. In un'applicazione un po' più moderna, laser montati sui tetti permettono di realizzare una LAN tra due edifici. La segnalazione ottica coerente basata sui laser è intrinsecamente unidirezionale, perciò ogni edificio ha bisogno di un laser e di un rilevatore fotoelettrico. Questo schema offre un'ampiezza di banda elevata a un costo ridotto; inoltre è relativamente facile da installare e non richiede licenze FCC, a differenza delle microonde.

La potenza del laser, un raggio molto sottile, rappresenta anche la sua debolezza: puntare un raggio laser di 1 mm su un bersaglio della dimensione di una capocchia di spillo distante 500 metri richiede un'abilità incredibile. Di solito nel sistema vengono introdotte lenti per rendere il raggio meno focalizzato. Per complicare la situazione, vento e sbalzi di temperatura possono distorcere il raggio e la luce laser non è in grado di penetrare pioggia e nebbia fitta. A ogni modo nessuno di questi fattori entra in gioco nella comunicazione tra stazioni spaziali.

Uno degli autori partecipò in passato a una conferenza europea ospitata all'interno di un albergo moderno, nel quale gli organizzatori avevano messo a disposizione una sala piena di terminali per permettere ai partecipanti di leggere la posta elettronica durante le presentazioni più noiose. Siccome la compagnia telefonica locale non era disponibile a installare tutte le linee richieste per solo tre giorni di utilizzo, gli organizzatori installarono un laser sul tetto e lo puntarono verso il dipartimento di informatica dell'università, a pochi chilometri di distanza. I test effettuati la notte prima furono positivi ma alle 9 del mattino dopo, con una splendida giornata, il sistema non funzionò e il collegamento rimase inattivo per tutto il giorno; così fu anche per i due giorni successivi. Fu solo alla fine della conferenza che gli organizzatori scoprirono il problema: il calore del sole durante il giorno faceva salire correnti di convezione sul tetto, come descritto nella Figura 2.14. Questa turbolenza deviava il raggio facendolo spostare rispetto al ricevitore. In questo caso la morale è che per poter funzionare correttamente in qualunque tipo di condizione i sistemi ottici senza cablatura devono essere progettati con un margine di sicurezza adeguato.

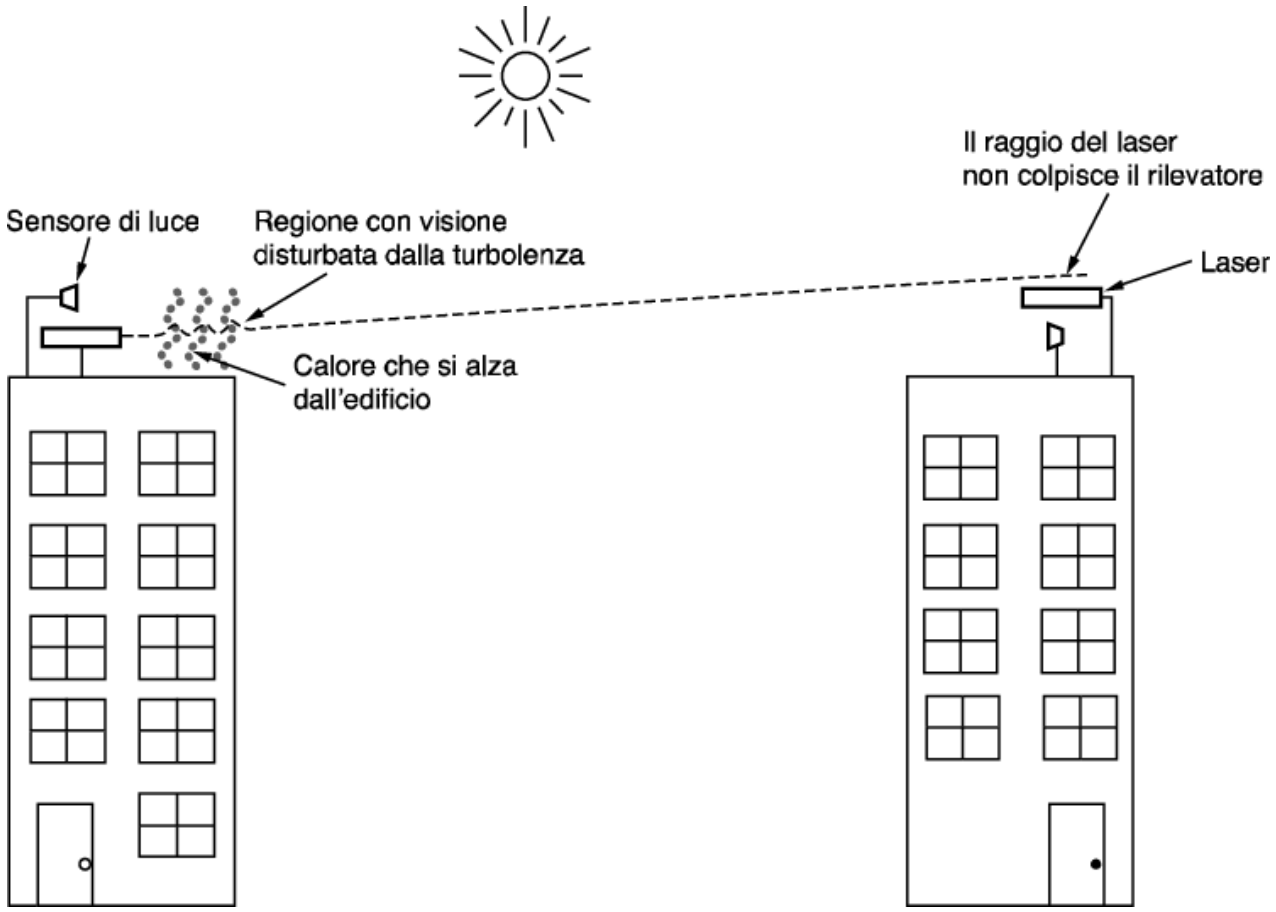


Figura 2.14 Le correnti di convezione possono interferire con i sistemi di comunicazione laser. L'immagine rappresenta un sistema bidirezionale con due laser.

Oggi i sistemi ottici non cablati possono sembrare una tecnologia stravagante per una rete, ma potrebbero diffondersi ampiamente in breve tempo; siamo infatti circondati da telecamere (che rilevano luce) e display (che la emetteno attraverso LED o altre tecnologie). Una trasmissione dati può avvenire codificando le informazioni come successioni di accensione e spegnimento dei LED a una velocità non percepibile da un essere umano. Questo tipo di comunicazione con luce visibile risulta intrinsecamente sicura e permette di costruire una rete a bassa velocità nelle immediate vicinanze del display, rendendo possibili una serie di scenari di calcolo pervasivo finora impensabile: la luce lampeggiante di un'ambulanza che allerta i semafori per favorire lo smaltimento del traffico, i cartelloni che visualizzano mappe digitali o le luci natalizie che diffondono musica sincronizzata.

2.4 Comunicazioni satellitari

A cavallo degli anni ’50 e ’60 gli scienziati tentarono di creare sistemi di comunicazione cercando di far rimbalzare segnali radio su palloni aerostatici meteorologici metallizzati. Sfortunatamente, essendo troppo deboli, i segnali ricevuti non erano di alcuna utilità pratica. Successivamente la Marina degli Stati Uniti constatò che nel cielo esisteva una specie di pallone aerostatico permanente, ossia la Luna, e costruì un sistema funzionante che permetteva alle navi di comunicare facendo rimbalzare i segnali su di essa.

I progressi successivi nel settore delle comunicazioni astronomiche hanno dovuto attendere il lancio del primo satellite di comunicazioni. La differenza principale tra un satellite artificiale e un satellite reale è che il primo è in grado di amplificare i segnali prima di ritrasmetterli, caratteristica che trasforma una bizzarria in un potente sistema di comunicazione.

I satelliti di comunicazione sono dotati di alcune interessanti proprietà che li rendono allettanti per molte applicazioni. Nella sua forma più semplice, un satellite di comunicazione può essere immaginato come un grande ripetitore di microonde collocato nel cielo. Questo dispositivo contiene diversi **transponder**, ossia ricetrasmettitori satellitari. Ognuno ascolta una parte dello spettro, amplifica il segnale in ingresso e lo ritrasmette su un’altra frequenza per evitare interferenze con il segnale in arrivo. I raggi puntati verso il basso possono essere larghi e coprire una considerevole frazione della superficie terrestre, oppure stretti e coprire un’area dal diametro di poche centinaia di chilometri. Questa modalità operativa è chiamata **bent pipe**.

Attraverso l’elaborazione del segnale digitale è possibile manipolare o ridirigere i flussi di dati all’interno della banda; l’informazione digitale può anche essere ricevuta da un satellite e ridistribuita. Ricostruire il segnale in questo modo migliora le prestazioni rispetto al metodo precedente, in quanto il satellite non amplifica il rumore del segnale in salita. Il raggio discendente può essere molto ampio e coprire una parte consistente della crosta terrestre, o sottile e coprire solo un’area del diametro di qualche centinaio di chilometri.

Secondo la legge di Keplero, il periodo orbitale di un satellite varia in base al raggio della sua orbita elevato alla potenza 3/2. Più è alto il satellite, più è lungo il periodo. In prossimità della superficie terrestre il periodo è di circa 90 minuti. Di conseguenza, i satelliti su orbite basse scompaiono dalla vista rapidamente e ne servono molti per fornire una copertura continua. A un’altitudine di circa 35.800 Km il periodo è di 24 ore; a 384.000 Km è di circa un mese; basta osservare regolarmente la Luna per verificarlo. II periodo di un satellite è importante, ma non è l’unica caratteristica che ne determina la collocazione. Un altro problema è rappresentato dalla presenza delle fasce di Van Allen, strati di particelle molto cariche intrappolate dal campo magnetico terrestre; un satellite che le attraversasse verrebbe velocemente distrutto. Questi fattori conducono alle tre zone in cui i satelliti possono essere collocati senza pericolo, che sono illustrate nella Figura 2.15 insieme ad alcune delle loro proprietà. I paragrafi successivi descrivono brevemente i satelliti che popolano ciascuna zona.

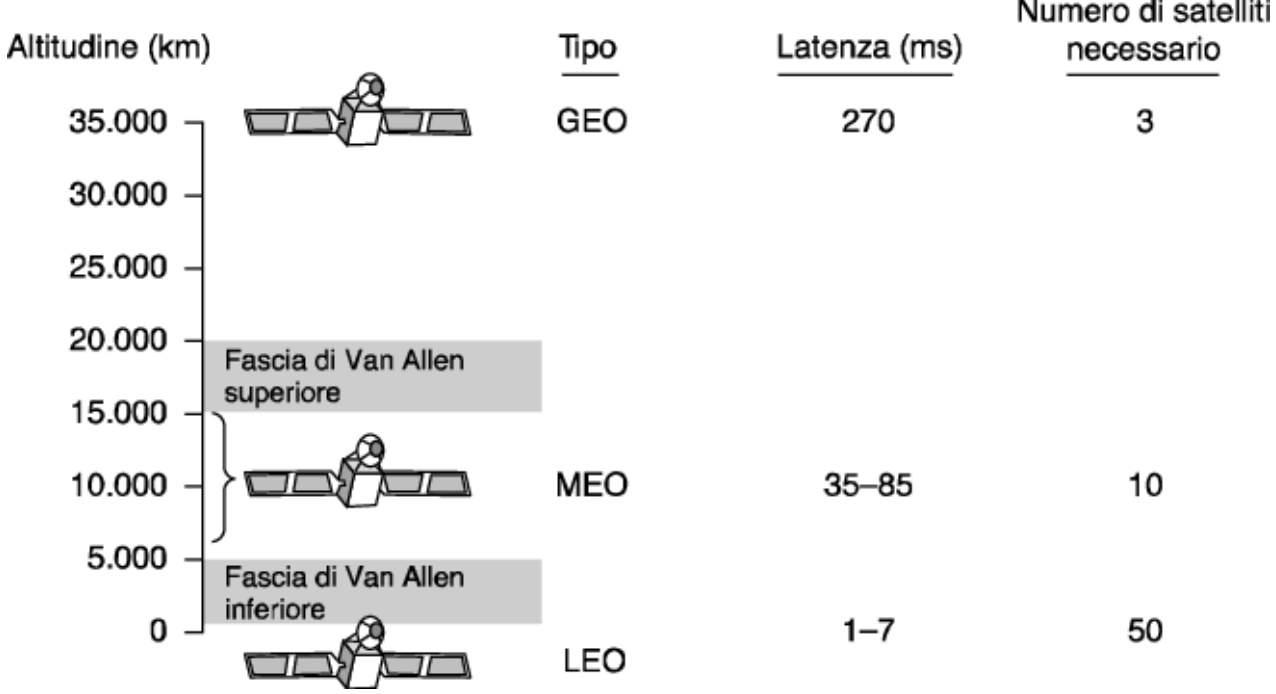


Figura 2.15 Satelliti di comunicazione e alcune delle loro proprietà, incluse le altitudini, il ritardo di andata e ritorno del segnale e il numero di satelliti necessari per fornire una copertura globale.

Satelliti geostazionari

Nel 1945 lo scrittore di fantascienza Arthur C. Clarke calcolò che un satellite posto a un'altitudine di 35.800 Km in un'orbita equatoriale circolare sarebbe apparso fermo nel cielo agli osservatori terrestri, perciò non avrebbe avuto bisogno di essere tracciato (Clarke, 1945). L'autore continuò descrivendo un sistema di comunicazione completo basato su questi **satelliti geostazionari**, specificando orbite, pannelli solari, frequenze radio e procedure di lancio. Purtroppo concluse che i satelliti non si sarebbero potuti realizzare a causa dell'impossibilità di collocare in orbita amplificatori dall'alto consumo energetico basati su fragili valvole a vuoto, perciò non portò avanti la sua idea, anche se scrisse diverse storie di fantascienza dedicate all'argomento.

L'invenzione del transistor cambiò tutto quanto, e nel luglio 1962 permise di lanciare il primo satellite artificiale di comunicazione, chiamato Telstar. Da quel momento, i satelliti di comunicazione sono diventati affari da molti miliardi di dollari (è l'unico settore dell'industria spaziale che ha prodotto alti profitti). Questi satelliti collocati su orbite alte sono anche chiamati satelliti **GEO** (*geostationary earth orbit*, orbita terrestre geostazionaria).

Con la tecnologia attuale, per evitare interferenze, è meglio non disporre i satelliti geostazionari nel piano equatoriale a intervalli minori di 2 gradi. Con un intervallo di 2 gradi nel cielo ci possono essere solo 360/2 = 180 satelliti di questo tipo, ma ogni transponder può utilizzare frequenze e polarizzazioni multiple per aumentare l'ampiezza di banda disponibile.

Per evitare il caos totale nel cielo, l'allocazione degli slot orbitali è gestita dall'ITU. Questo processo è molto politico e ci sono paesi, appena usciti dall'età della pietra, che chiedono di avere i propri slot orbitali per poterli poi affittare al miglior offerente. Altri paesi sostengono che i diritti di proprietà nazionale non si estendono alla Luna e che nessun paese ha un diritto legale sugli slot

orbitali collocati sul proprio territorio. Ad alimentare la discordia, le telecomunicazioni commerciali non rappresentano l'unica applicazione pratica; anche le stazioni televisive, i governi e i militari pretendono di avere voce in capitolo.

I moderni satelliti possono raggiungere dimensioni enormi: alcuni pesano 5000 Kg e consumano diversi kilowatt di energia elettrica prodotta da pannelli solari. Gravità solare, lunare e planetaria tendono ad allontanarli dagli slot e dagli orientamenti assegnati, ma l'effetto è contrastato dai motori a razzo installati a bordo. La continua attività di messa a punto è chiamata **station keeping** e quando, dopo una decina di anni circa, il propellente dei motori si esaurisce, il satellite va alla deriva e precipita senza che si possa far nulla, quindi deve essere disattivato. Alla fine, l'orbita decade e il satellite rientra nell'atmosfera e brucia o qualche volta precipita al suolo.

Gli slot orbitali non sono l'unico pomo della discordia; anche le frequenze creano problemi, poiché le trasmissioni dirette verso il basso interferiscono con i sistemi a microonde esistenti. Di conseguenza, l'ITU ha assegnato alle applicazioni satellitari alcune bande di frequenza, e la Figura 2.16 ne elenca le principali. La banda C è stata la prima assegnata al traffico commerciale e comprende due intervalli di frequenze assegnate: il più basso per il traffico diretto verso la Terra e il più alto per quello diretto verso il satellite. Per consentire al traffico di viaggiare contemporaneamente in entrambe le direzioni servono due canali, uno per ciascuna direzione. Queste bande sono sovraffollate, perché utilizzate anche dalle aziende telefoniche per i collegamenti a microonde terrestri. Le bande L ed S sono state aggiunte nel 2000 in base ad accordi internazionali, ma sono strette e affollate.

Banda	Downlink	Uplink	Larghezza della banda	Problemi
L	1,5 GHz	1,6 GHz	15 MHz	Banda stretta; affollamento
S	1,9 GHz	2,2 GHz	70 MHz	Banda stretta; affollamento
C	4,0 GHz	6,0 GHz	500 MHz	Interferenza terrestre
Ku	11 GHz	14 GHz	500 MHz	Pioggia
Ka	20 GHz	30 GHz	3.500 MHz	Pioggia; costo degli apparati

Figura 2.16 Le principali bande usate dai satelliti.

La prossima banda a disposizione delle società di telecomunicazioni commerciali è la banda **Ku** (*K under*). Tale banda non è ancora congestionata e a queste frequenze i satelliti possono essere collocati a solo un grado angolare di distanza l'uno dall'altro; purtroppo esiste un altro problema: la pioggia. L'acqua assorbe in modo eccellente queste microonde corte. Per fortuna, le tempeste più violente sono di solito localizzate, perciò è possibile eludere il problema utilizzando diverse stazioni terrestri molto distanti tra loro invece di usarne solo una, ma al prezzo di antenne aggiuntive, cavi extra e dispositivi elettronici in grado di attivare rapidamente un cambio di stazione. Varie

ampiezze di banda sono state assegnate al traffico commerciale anche nella banda **Ka** (*K above*), ma l'attrezzatura necessaria è costosa. Oltre a queste bande commerciali, ne esistono molte governative e militari.

Un satellite moderno dispone di circa 40 transponder, la maggior parte dei quali con un'ampiezza di banda di 36 MHz. Di solito ogni transponder opera come un bent pipe, ma satelliti recenti hanno capacità di elaborazione interna che permette operazioni più sofisticate. Nei primi satelliti la divisione dei transponder in canali era statica: l'ampiezza di banda era semplicemente divisa in bande di frequenza prefissate. Oggi ogni raggio di transponder è diviso in slot temporali, passati a diversi utenti. Entrambe le tecniche (multiplexing a divisione di frequenza e multiplexing a divisione di tempo) saranno esaminate meglio più avanti in questo capitolo.

I primi satelliti geostazionari avevano una singola emissione, chiamata **impronta**, che illuminava circa 1/3 della superficie terrestre. L'enorme diminuzione dei costi, delle dimensioni e dei requisiti di alimentazione della microelettronica ha reso possibile una strategia di trasmissione molto più sofisticata. Ogni satellite è dotato di più antenne e transponder; ogni raggio diretto verso il basso può essere concentrato su una piccola area geografica, perciò possono avvenire contemporaneamente più trasmissioni nei due sensi. In genere questi raggi puntiformi ellittici (**spot**) possono avere un diametro di poche centinaia di chilometri. Un satellite per telecomunicazioni statunitense normalmente prevede un'emissione ampia per i 48 stati contigui, più alcune emissioni puntiformi per l'Alaska e le Hawaii.

Un recente sviluppo nel settore delle comunicazioni satellitari è rappresentato dalle microstazioni a basso costo chiamate anche **VSAT** (*very small aperture terminal*) (Abramson, 2000). Questi piccoli terminali hanno antenne grandi un metro o anche meno (le antenne GEO sono grandi dieci metri) e possono generare circa 1 watt di potenza. Il collegamento in salita (*uplink*) va bene per trasportare fino a 1 Mbps, mentre quello in discesa (*downlink*) è generalmente di diversi Mbps. Le televisioni satellitari utilizzano questa tecnologia per le loro trasmissioni unidirezionali.

In molti sistemi VSAT le microstazioni non hanno abbastanza energia per comunicare direttamente le une con le altre (via satellite, logicamente), per questo motivo è necessario installare speciali stazioni terrestri, chiamate **hub** e dotate di grosse antenne ad alto guadagno che trasmettono il traffico attraverso le stazioni VSAT (Figura 2.17). In questa modalità operativa la stazione trasmittente oppure quella ricevente fanno uso di una grande antenna e di un potente amplificatore. Il compromesso per avere stazioni terminali utente più economiche è quello di un maggior ritardo di propagazione.VSAT ha un grande potenziale nelle aree rurali; non è apprezzato da tutti, ma più della metà della popolazione mondiale vive a un'ora di distanza dal telefono più vicino. Stendere cavi telefonici per raggiungere migliaia di piccoli villaggi non rientra nel bilancio della maggior parte dei governi del Terzo Mondo, ma installare parabole VSAT di un metro di diametro alimentate da celle a energia solare è spesso fattibile. VSAT fornisce la tecnologia che permetterà di cablare il mondo.

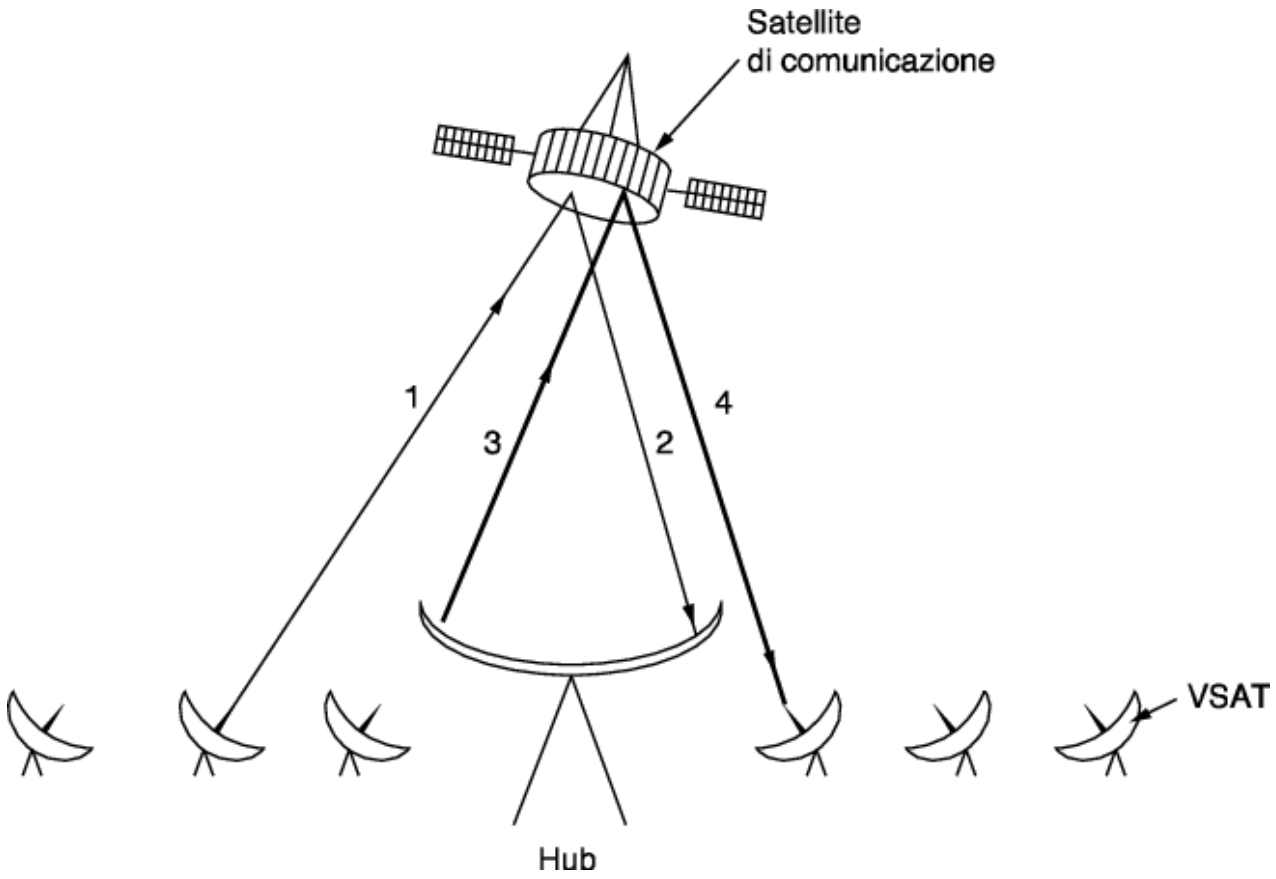


Figura 2.17 VSAT che usano un hub.

I satelliti di comunicazione hanno molte proprietà radicalmente diverse da quelle dei collegamenti terrestri punto a punto. Prima di tutto, anche se i segnali trasmessi e ricevuti da un satellite viaggiassero a velocità della luce (circa 300.000 Km/s), il lungo viaggio di andata e ritorno introdurrebbe un sostanziale ritardo nei satelliti GEO. A seconda della distanza tra l'utente e la stazione terrestre e dell'elevazione del satellite sopra l'orizzonte, il tempo di trasmissione è tra i 250 e i 300 ms; di solito si aggira intorno ai 270 ms (540 ms per un sistema VSAT con hub).

In confronto, i collegamenti terrestri a microonde hanno un ritardo di propagazione pari a circa 3 ms/km, mentre quelli basati su cavi coassiali e fibre ottiche hanno un ritardo che si aggira intorno ai 5 ms/km. I cavi sono più lenti, poiché i segnali elettromagnetici viaggiano più velocemente in aria che attraverso materiali solidi.

Un'altra proprietà importante dei satelliti è che sono mezzi di trasmissione broadcast per natura: inviare un messaggio a migliaia di stazioni poste nella stessa area del transponder costa quanto inviare un messaggio a una singola stazione. In alcune applicazioni questa proprietà è molto utile. Per esempio, si potrebbe immaginare un satellite che trasmette le pagine Web più popolari alle cache di numerosi computer sparsi in una vasta area. Anche quando la diffusione broadcast può essere simulata con linee punto a punto, la trasmissione satellitare può essere molto più economica. D'altro canto, dal punto di vista della sicurezza e della privacy, i satelliti sono un vero disastro: tutto può essere ascoltato da tutti. Per proteggere le comunicazioni è necessario adottare sistemi di crittografia.

I satelliti godono anche della proprietà per cui il costo della trasmissione di un messaggio è indipendente dalla distanza attraversata. Una chiamata oltre oceano costa quanto una chiamata urbana. I satelliti hanno anche un eccellente tasso di errore e possono essere messi in esercizio quasi istantaneamente, caratteristica cruciale per le comunicazioni militari.

Satelliti su orbite medie

Ad altitudini molto più basse, comprese tra le due fasce di Van Alien, si trovano i satelliti **MEO** (*medium earth orbit*). Visti dalla Terra questi satelliti si spostano lentamente lungo la longitudine, impiegando circa 6 ore per compiere un giro intorno al pianeta; di conseguenza devono essere seguiti mentre si spostano nel cielo. Poiché sono più bassi dei satelliti GEO, coprono un'area più piccola e sono raggiungibili per mezzo di trasmettitori meno potenti. Al momento sono utilizzati per i sistemi di navigazione più che per le telecomunicazioni. I circa 30 satelliti **GPS** (*global positioning system*) che orbitano a circa 20.200 Km di altezza sono di tipo MEO.

Satelliti su orbite basse

Scendendo di altezza si incontrano i satelliti **LEO** (*low earth orbit*). Poiché si spostano rapidamente, la realizzazione di un sistema completo richiede l'utilizzo di numerosi satelliti di questo tipo. D'altra parte, poiché i satelliti sono molto vicini alla superficie del pianeta, le stazioni terrestri non hanno bisogno di molta energia, il ritardo nelle comunicazioni è di pochi millisecondi e anche il costo di lancio è nettamente inferiore. Questo paragrafo esamina due esempi di costellazioni satellitari per servizi vocali: Iridium e Globalstar.

Nei primi 30 anni dell'era satellitare, a causa della loro eccessiva velocità, i LEO sono stati utilizzati raramente. Nel 1990 Motorola aprì un nuovo orizzonte presentando una domanda all'FCC per poter lanciare su orbite basse i 77 satelliti del progetto **Iridium** (l'elemento numero 77 nella tavola degli elementi è appunto l'iridio). Il piano venne successivamente corretto, in modo da utilizzare solo 66 satelliti e rinominato Dysprosium (elemento 66), ma probabilmente il nome ricordava troppo quello di una malattia. L'idea era che non appena un satellite spariva dalla vista, ne sarebbe apparso un altro all'orizzonte. Questa proposta esaltò le altre società di telecomunicazioni; in men che non si dica tutte volevano lanciare la propria catena di satelliti su orbite basse.

Dopo sette anni trascorsi alla ricerca di partner e finanziatori, i satelliti Iridium vennero lanciati nel 1997, e il servizio di comunicazione iniziò nel novembre del 1998. Sfortunatamente la richiesta commerciale dei pesanti e scomodi telefoni satellitari era inesistente, perché la rete di telefonia mobile era cresciuta in modo spettacolare dal 1990. Di conseguenza, non essendo redditizio, Iridium fallì (agosto 1999) rappresentando uno dei più spettacolari fiaschi aziendali della storia. I satelliti e altri beni (per un valore di 5 miliardi di euro) vennero in seguito acquistati da un investitore per 25 milioni di euro in una sorta di liquidazione extraterrestre.

L'attività di Iridium ripartì nel marzo del 2001 e da allora ha continuato a crescere. Esso fornisce ovunque per terra, aria e mare servizi per voce, dati, messaggi, fax e navigazione Web attraverso dispositivi palmari che comunicano direttamente con i satelliti Iridium. Alcuni clienti sono industrie marittime, aeronautiche e di esplorazione petrolifera, oltre a coloro che viaggiano in aree del mondo con infrastrutture di telecomunicazioni carenti (deserti, montagne, Polo Sud e paesi del Terzo Mondo).

I satelliti Iridium si trovano a un'altitudine di 750 Km, in orbite polari circolari; sono sistemati in catene che vanno da Nord a Sud, con un satellite ogni 32 gradi di latitudine, come mostra la Figura 2.18. Ogni satellite ha un massimo di 48 celle (spot puntiformi) e una capacità di 3840 canali, alcuni dei quali sono usati per messaggistica e navigazione, mentre altri trasportano dati e voce.

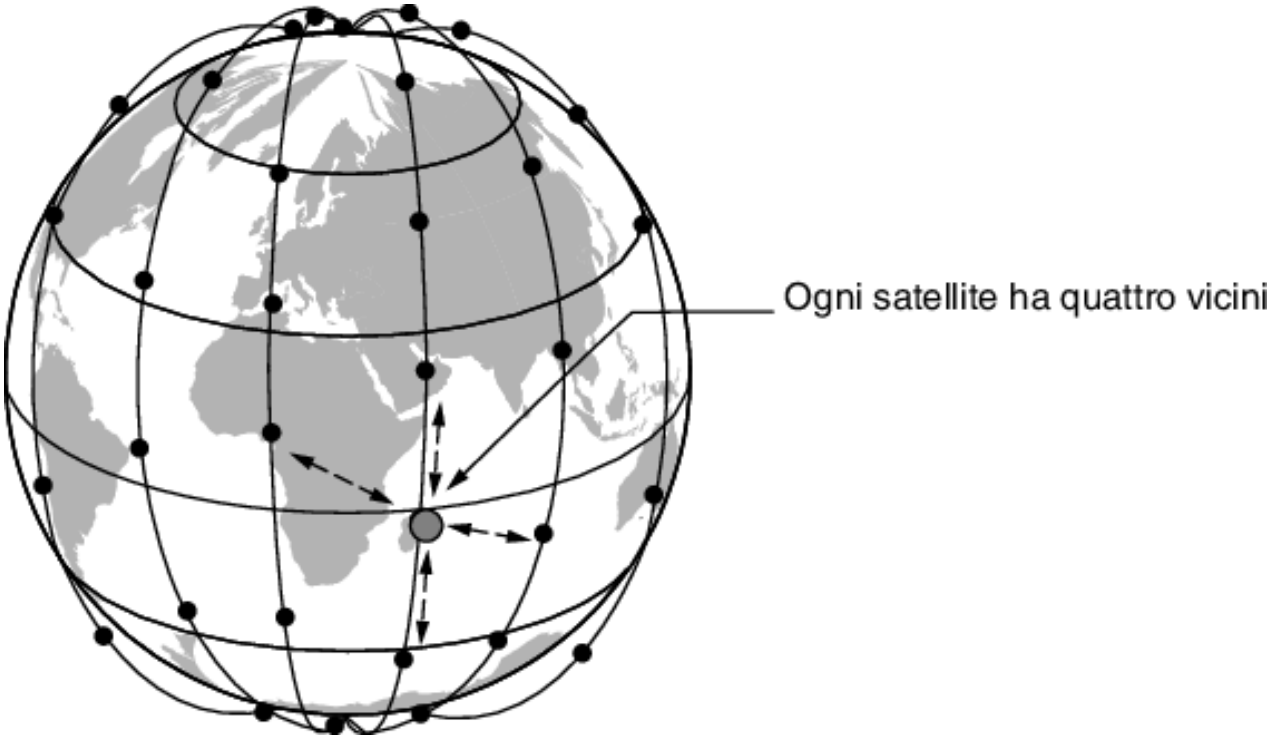


Figura 2.18 I satelliti Iridium formano sei collane attorno alla Terra.

Come mostrato nella Figura 2.18, sei collane di satelliti ricoprono tutta la Terra. Una proprietà interessante di Iridium è che la comunicazione tra clienti distanti avviene nello spazio, dove ogni satellite trasmette i dati al successivo come mostrato nella Figura 2.19(a). Nel disegno, la persona che chiama si trova al Polo Nord e contatta un satellite posto proprio sopra la sua testa. Ogni satellite ha quattro vicini con cui è in grado di comunicare; di questi, due nella stessa collana (mostrati) e due in collane adiacenti (non mostrati). I satelliti reindirizzano la chiamata attraverso l'infrastruttura finché non viene mandata a terra all'utente chiamato, che potrebbe trovarsi al Polo Sud.

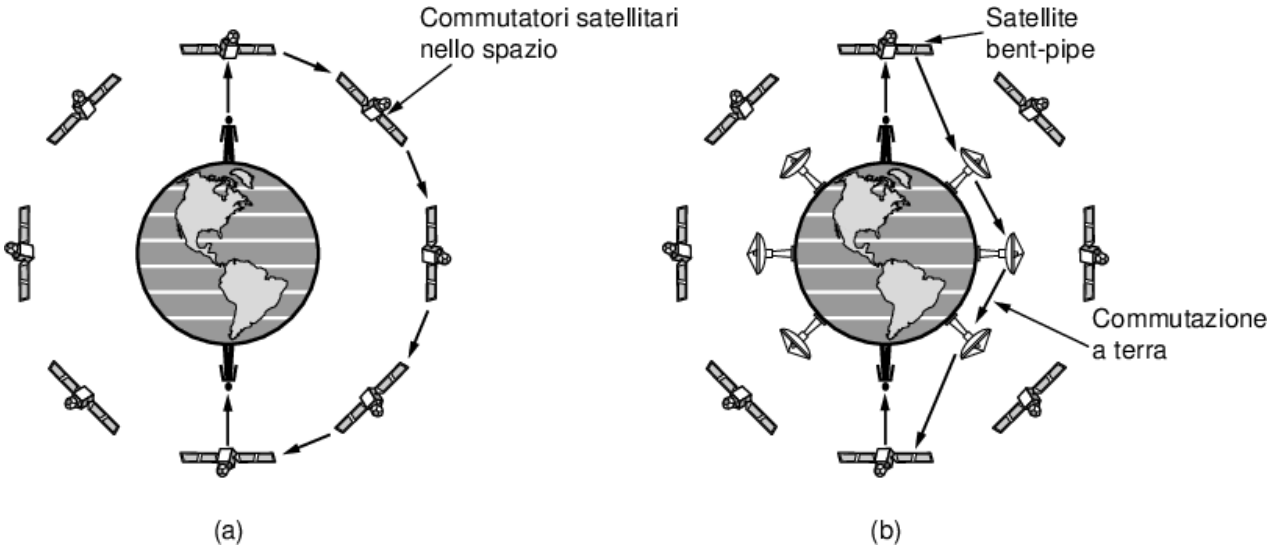


Figura 2.19 (a) Transito della comunicazione nello spazio. (b) Transito della comunicazione a terra.

Un progetto alternativo a Iridium è **Globalstar**; si basa su 48 satelliti LEO e utilizza un diverso schema di commutazione. Mentre Iridium ritrasmette le chiamate da satellite a satellite, una tecnica che richiede la presenza di sofisticate attrezzature di commutazione in ogni satellite, Globalstar, invece, utilizza un modello tradizionale bent-pipe. La chiamata che ha origine al Polo Nord nella Figura 2.19(b) viene ritrasmessa sulla Terra, raccolta da una grande stazione terrestre vicina al laboratorio di Babbo Natale e dirottata attraverso una rete terrestre alla stazione più vicina al destinatario, che la riceve tramite un'altra connessione bent-pipe. Questo schema ha il grande vantaggio di tenere la complessità a terra, dove è più facile da gestire. Inoltre le grandi antenne delle basi terrestri, capaci di trasmettere segnali potenti e di ricevere segnali deboli, consentono di adoperare telefoni di bassa potenza. Dopo tutto, il telefono trasmette solo pochi milliwatt di energia, perciò il segnale che arriva alla stazione terrestre è abbastanza debole, anche se è stato amplificato dal satellite.

Nuovi satelliti continuano a essere lanciati a un ritmo di circa 20 all'anno; ci sono satelliti enormi dal peso superiore a 5000 chilogrammi, ma anche satelliti molto piccoli per organizzazioni con budget più ristretti. Per rendere più accessibile la ricerca spaziale alcuni ricercatori universitari del California Polytechnic e di Stanford si sono uniti nel 1999 per definire uno standard per satelliti in miniatura e le relative piattaforme di lancio. Tale standard mira a diminuire sensibilmente i costi di messa in orbita (Nugent et al., 2008). I **CubeSat** sono satelliti di forma cubica da 10 cm di lato, pesanti non più di un chilogrammo, che possono essere messi in orbita con soli 40.000 dollari ognuno. La piattaforma di lancio vola come carico supplementare su missioni spaziali commerciali e si compone fondamentalmente di un tubo che contiene fino a tre CubeSat, spinti in orbita tramite delle molle. Finora sono stati messi in orbita circa 20 CubeSat, ma molti sono in programma. La maggior parte di essi comunica con stazioni terrestri usando bande UHF e VHF.

Satelliti o fibra ottica?

Un confronto tra la comunicazione satellitare e quella terrestre è istruttivo. Solo 25 anni fa si pensava che il futuro della comunicazione sarebbe stato dominato dai satelliti. Dopo tutto il sistema telefonico era cambiato pochissimo nei precedenti 100 anni e non sembrava voler cambiare nei successivi 100. Questa sorta di immobilità dipendeva in buona parte dall'insieme di norme imposte alle aziende telefoniche di fornire un buon servizio di comunicazione vocale a un prezzo ragionevole, garantendo agli investitori un profitto sicuro. Le persone che avevano dati da trasmettere si dovevano accontentare di modem a 1.200 bps.

L'esordio della concorrenza nel 1984 negli Stati Uniti e in alcuni paesi dell'Europa cambiò tutto radicalmente. Le aziende telefoniche iniziarono a sostituire le loro reti a lunga tratta con fibre ottiche, introdussero servizi a banda larga come **ADSL** (*asymmetric digital subscriber line*) e smisero di far pagare prezzi artificiosamente alti per le chiamate a lunga distanza per sovvenzionare il servizio locale. Tutto d'un tratto le connessioni basate su fibre terrestri avevano preso il sopravvento.

Ciò nonostante i satelliti di comunicazione potevano contare su alcune grandi nicchie di mercato che la fibra non era in grado di raggiungere. Quando l'installazione rapida è un fattore critico i satelliti sono la scelta vincente; una risposta veloce è utile nelle comunicazioni militari in tempo di guerra e per la protezione civile in tempo di pace. Per esempio, dopo il grande terremoto di Sumatra nel dicembre del 2004, seguito da uno tsunami, i satelliti sono stati in grado di ristabilire le comunicazioni per i primi soccorsi entro 24 ore. Questa risposta rapida è stata possibile grazie a dei grandi fornitori di servizi satellitari, come Intelsat, in grado di rivendere connettività ovunque sia necessario grazie ai loro 50 e più satelliti. Per chi è già coperto dal servizio di un operatore satellitare è facile farsi installare un VSAT e avere un link con una capacità dell'ordine dei megabit al secondo in qualunque altro punto del globo.

Una seconda nicchia è invece quella delle comunicazioni dove le infrastrutture terrestri non sono ancora sviluppate. Al giorno d'oggi sono molti a voler comunicare in qualunque punto si trovino; le reti di telefonia mobile coprono i luoghi dove la densità di popolazione è alta, ma non sono presenti in altre zone (ad esempio, in mare o nel deserto). Viceversa, Iridium fornisce servizio di telefonia in qualunque posto sulla Terra, Polo Sud compreso. Le infrastrutture terrestri possono essere costose da installare a seconda del terreno e delle concessioni necessarie. L'Indonesia, per esempio, ha una sua rete satellitare per le chiamate telefoniche locali: lanciare un satellite è più economico che posare migliaia di cavi sottomarini tra le 13.677 isole dell'arcipelago.

Una terza nicchia è quando il broadcast è una caratteristica irrinunciabile del sistema di comunicazione. Un messaggio spedito da un satellite può essere ricevuto contemporaneamente da migliaia di stazioni terrestri; per questa ragione sono usati i satelliti per distribuire gran parte dei programmi televisivi. In questo momento vi è un florido mercato della distribuzione via satellite di TV digitale e trasmissioni radiofoniche.

In breve sembra che il sistema di comunicazione principale del futuro sarà quello terrestre basato su fibre ottiche combinato con la rete radio cellulare, ma per alcune applicazioni specializzate i satelliti sono migliori. A tutto ciò va contrapposto l'aspetto economico: sebbene le fibre offrano una maggiore ampiezza di banda, è certamente possibile che la comunicazione terrestre e quella satellitare si facciano concorrenza in modo aggressivo anche sui prezzi. Se gli sviluppi tecnologici

ridurranno drasticamente il costo del dispiegamento dei satelliti (per esempio con veicoli spaziali in grado di portare in orbita dozzine di satelliti alla volta) oppure se i satelliti su orbite basse avranno successo, la fibra potrebbe rischiare di perdere il dominio dei mercati.

2.5 Modulazione digitale e multiplexing

Ora che abbiamo visto le proprietà dei collegamenti cablati e wireless spostiamo l'attenzione al problema dell'invio di informazione digitale. I canali sia cablati che wireless trasportano segnali analogici che possono essere voltaggi, intensità di luce o suoni che variano in maniera continua nel tempo. Per inviare informazioni digitali dobbiamo usare dei segnali analogici per rappresentare bit. Il processo di conversione tra bit e i segnali che li rappresentano prende il nome di **modulazione digitale** (*digital modulation*).

Partiremo analizzando strategie che convertono direttamente bit in segnali per ottenere una **trasmissione in banda base** (*baseband transmission*), nella quale i segnali occupano frequenze che vanno da zero fino a un massimo che dipende dal tasso di trasmissione. Successivamente ci occuperemo di strategie che agiscono su ampiezza, fase o frequenza di un segnale di base (o portante) per il trasferimento dei dati; si ottiene così una **trasmissione in banda passante** (*passband transmission*), nella quale il segnale occupa una banda di frequenze attorno a quella del segnale portante. Quest'ultimo tipo di trasmissione è tipico del wireless e dei canali a trasmissione ottica per cui i segnali devono trovarsi all'interno di una data banda di frequenze.

I canali di trasmissione vengono normalmente condivisi da più segnali; dopo tutto è molto più conveniente usare un singolo cavo per trasportare molti segnali piuttosto che installare un cavo per ogni segnale. Questo tipo di condivisione viene chiamato **multiplexing**. Il multiplexing si può ottenere in vari modi; discuteremo i metodi basati su divisione di tempo, frequenza e codice.

Le tecniche di modulazione e multiplexing che descriveremo in questo capitolo sono tutte largamente utilizzate per comunicazioni via cavo, fibra e wireless sia terrestre che satellitare.

Trasmissione in banda base

La forma più naturale di modulazione digitale prevede l'uso di una tensione positiva per rappresentare un 1 e negativa per rappresentare uno 0. Per una fibra ottica la presenza di luce potrebbe rappresentare un 1 e l'assenza uno 0. Questo tipo di approccio prende il nome di **NRZ** (*non-return-to-zero*, non-ritorno-a-zero). Questo nome un po' strano ha motivazioni storiche e significa semplicemente che il segnale segue l'andamento dei dati; un esempio è presentato nella Figura 2.20(b).

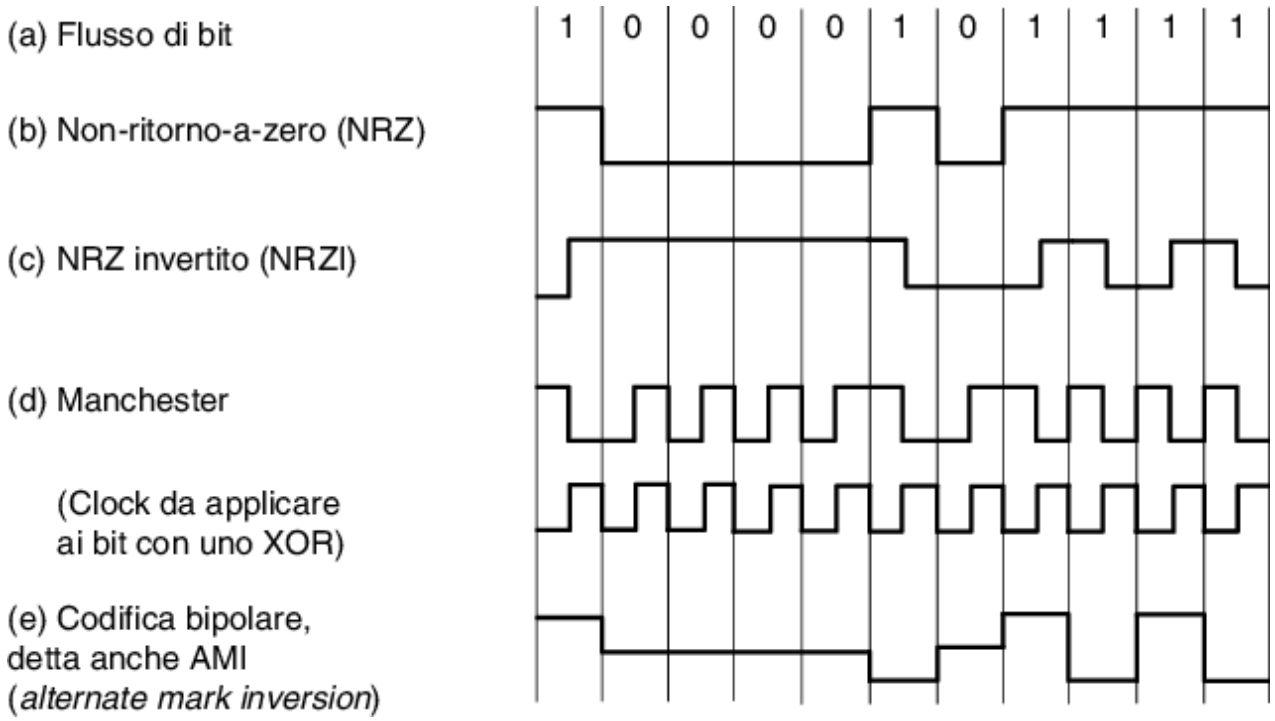


Figura 2.20 Codifiche di linea: (a) Bit, (b) NRZ, (c) NRZI, (d) Manchester, (e) Bipolare o AMI.

Una volta spedito, il segnale NRZ si propaga lungo il cavo e all'altro capo il ricevente lo converte in bit campionando il segnale a intervalli di tempo regolari. Questo segnale, però, non sarà esattamente come quello spedito: sarà attenuato e distorto dal canale e dal rumore alla stazione ricevente. Per decodificare i bit il ricevente assegna i campioni ai simboli più vicini. Per NRZ, una tensione positiva sarà interpretata come l'invio di un 1, mentre una negativa come l'invio di uno 0.

NRZ è un buon punto di inizio per il nostro studio, in quanto è semplice, ma viene raramente utilizzato così com'è. Strategie più complesse possono convertire bit in segnali con conseguenti problematiche tecniche; questi bit vengono chiamati **line code** (*codifiche di linea*). Descriveremo ora alcuni line code che possono risultare utili per efficienza di banda, clock recovery e bilanciamento in corrente continua.

Efficienza di banda

Con NRZ il segnale potrebbe oscillare tra i livelli positivi e negativi ogni 2 bit (nel caso di un'alternanza di 1 e 0). Questo vuol dire che abbiamo bisogno di una banda di almeno $B/2$ Hz se il tasso di invio dei bit (*bit rate*) è B bit/s. Questa relazione arriva fino al tasso di Nyquist [Eq. (2-2)] ed è un limite fondamentale, per cui non possiamo far andare NRZ più veloce a meno di non usare più banda. La banda è spesso una risorsa limitata anche nei mezzi trasmissivi cablati; i segnali ad alte frequenze vengono attenuati sempre di più e sono quindi meno sfruttabili, richiedendo delle apparecchiature molto più veloci (e costose).

Una possibile strategia per sfruttare meglio la banda è quella di usare più di due livelli di segnale; usando quattro livelli di tensione, per esempio, siamo in grado di spedire due bit per volta come un unico simbolo; tale approccio, però, funzionerà solo se il segnale al lato ricevente sarà abbastanza

forte da permettere di distinguere i quattro livelli. Il tasso a cui cambia il segnale è quindi la metà del bit rate e la banda richiesta è di conseguenza ridotta.

Chiamiamo **symbol rate** il tasso con cui il segnale cambia, per distinguerlo da **bit rate**. Il bit rate equivale al symbol rate moltiplicato per il numero di bit in ogni simbolo. **Baud rate** è il vecchio nome utilizzato per symbol rate, quando si parla di dispositivi chiamati modem telefonici, per la trasmissione di dati digitali su linee telefoniche. In letteratura i termini "bit rate" e "baud rate" sono spesso usati in maniera sbagliata.

Si noti che il numero dei livelli di segnale non è necessariamente una potenza di due, e normalmente non lo è. Inoltre alcuni dei livelli sono usati come protezione contro gli errori e semplificano la progettazione dell'apparato ricevente.

Clock recovery

Per quanto riguarda tutte le strategie che codificano bit in simboli, per poter effettuare correttamente la decodifica il ricevente è tenuto a conoscere quando un simbolo termina e il successivo inizia. Con NRZ, in cui i simboli sono semplici livelli di tensione, una sequenza molto lunga di 0 o 1 lascia il segnale invariato; dopo un po' diventa difficile distinguere i bit a meno di non disporre di una temporizzazione (*clock*) molto precisa.

Clock precisi potrebbero essere utili in questo caso, ma sono una risposta molto costosa per apparati a uso domestico; ricordatevi che stiamo temporizzando bit su collegamenti da molti megabit/s, quindi il clock dovrebbe disallinearsi di non più di una frazione di microsecondo anche nel caso della sequenza di simboli uguali più lunga possibile. Questo potrebbe adattarsi a collegamenti lenti o messaggi brevi, ma non è una soluzione sempre valida.

Una possibile alternativa è di spedire al destinatario un segnale di clock separato; aggiungere una connessione non è un grosso problema per il bus di un calcolatore o un cavo corto dove ci sono più conduttori in parallelo, ma è uno spreco di risorse per molte connessioni di rete, visto che, se avessimo un'altra linea disponibile, la potremmo usare per mandare dati. Una scappatoia possibile è quella di mandare il segnale di clock miscelato con i dati usando una operazione logica di XOR (*exclusive or*, or esclusivo), così da non dover usare una seconda connessione; il risultato è raffigurato nella Figura 2.20(d). Il clock effettua una transizione a ogni bit, in questo modo procede al doppio del bit rate; quando viene applicato con uno XOR a un livello zero effettua una transizione dal basso verso l'alto che è semplicemente il clock, e questo rappresenta uno 0; quando, invece, viene combinato con un 1, si inverte ed effettua una transizione dall'alto verso il basso, rappresentando un 1. Questo tipo di approccio prende il nome di **Manchester encoding** (*codifica Manchester*), usato per la prima versione di Ethernet. L'aspetto negativo della codifica Manchester è che, a causa del clock, richiede il doppio di banda rispetto a NRZ e ormai si sa che la banda è spesso importante. Un diverso approccio si basa sull'intuizione secondo la quale dovremmo codificare i dati in maniera da assicurare che ci siano sufficienti transizioni. Una strategia differente prevede di codificare i dati per avere abbastanza transizioni nel segnale. NRZ avrà problemi di sincronia solo a fronte di lunghe sequenze di 0 o 1; se ci sono frequenti transizioni di stato sarà più facile per il destinatario rimanere sincronizzato con la sequenza di simboli in arrivo.

Per fare un passo nella direzione giusta possiamo semplificare la situazione codificando un 1 come una transizione e uno 0 come una situazione stazionaria (o viceversa); questo tipo di codifica prende il nome di **NRZI** (*non-return-to-zero inverted*, non ritorno a zero invertito) di cui è mostrato un esempio nella Figura 2.20(c). Lo standard **USB** (*universal serial bus*), usato per connettere periferiche ai calcolatori, usa NRZI; con esso, lunghe sequenze di 1 non causano problemi.

Naturalmente lunghe sequenze di 0 sono ancora fonte di problemi e non possiamo trascurarle; se fossimo una compagnia telefonica potremmo semplicemente richiedere che il mittente non trasmetta troppi 0. Vecchie linee telefoniche digitali degli Stati Uniti, chiamate **linee T1**, richiedevano, per poter funzionare correttamente, che non fossero inviati più di quindici 0 consecutivi. Per risolvere realmente questo problema possiamo spezzare sequenze di 0 troppo lunghe associando loro piccoli gruppi di bit da trasmettere; in quel modo gruppi con sequenze di 0 sono associati a sequenze leggermente più lunghe che però non hanno troppi 0 consecutivi.

Un codice ben conosciuto che effettua questa operazione ha il nome di **4B/5B**; ogni gruppo di 4 bit è associato a una sequenza di 5 bit secondo una tabella di conversione nota. Le sequenze di 5 bit vengono scelte in modo tale da non avere mai una sequenza di più di tre 0 consecutivi; la tabella è presentata nella Figura 2.21. Questo tipo di codifica aggiunge un overhead del 25%, che è meglio del 100% della codifica Mancheser. Siccome ci sono 16 combinazioni di input e 32 combinazioni di output, alcune delle sequenze di output non sono utilizzate; anche escludendo le combinazioni con troppi 0 ci sono ancora dei codici disponibili che possono essere usati come segnali di controllo per il livello fisico. Ad esempio, in alcuni casi "11111" rappresenta il canale non occupato e "11000" rappresenta l'inizio di un frame.

Dato (4B)	Codice (5B)	Dato (4B)	Codice (5B)
0000	11110	1000	10010
0001	01001	1001	10011
0010	10100	1010	10110
0011	10101	1011	10111
0100	01010	1100	11010
0101	01011	1101	11011
0110	01110	1110	11100
0111	01111	1111	11101

Figura 2.21 Tabella di associazione 4B/5B.

Un approccio alternativo, noto anche come **scrambling**, può essere quello di far sembrare i dati come generati casualmente. In questo caso è altamente probabile che ci siano frequenti transizioni. Uno **scrambler** opera applicando, prima della trasmissione, un'operazione di XOR tra i dati e una sequenza di numeri pseudocasuali; in questo modo i dati sembreranno casuali tanto quanto la sequenza estratta (assumendo che questi siano indipendenti dalla sequenza pseudocasuale). Il ricevente applicherà uno XOR ai bit in arrivo con la stessa sequenza pseudocasuale per recuperare i dati originali. Per poter implementare questa soluzione la sequenza pseudocasuale deve essere facile da generare e viene di solito definita come il seme di un semplice generatore di numeri casuali.

L'operazione di scrambling è interessante perché non aggiunge overhead di banda o tempo; in effetti aiuta a vincolare il segnale in maniera tale che non abbia la sua energia concentrata in componenti di frequenza specifiche (effetto causato da sequenze di dati ripetitive) perché in tal caso potrebbe irradiare delle interferenze elettromagnetiche. Effettuare scrambling è utile perché segnali casuali tendono a essere "bianchi", cioè ad avere la loro energia distribuita su tutte le componenti di frequenza.

Tuttavia, lo scrambling non garantisce che non ci saranno lunghe sequenze di simboli ripetuti; può sempre capitare di essere sfortunati. Se i dati sono gli stessi della sequenza pseudocasuale, lo XOR produrrà solo 0, anche se questo generalmente non accade con una lunga sequenza pseudocasuale che sia difficile da predire. Con una sequenza corta o predicibile può tuttavia essere possibile per utenti malintenzionati spedire sequenze di bit che risultino in lunghe successioni di 0 bloccando così il funzionamento del canale. Le prime versioni degli standard usati per inviare pacchetti IP su linee SONET avevano questo difetto (Malis e Simpson, 1999): era possibile che gli utenti mandassero "pacchetti killer" che avrebbero causato sicuramente problemi.

Segnali bilanciati

Segnali che presentano una tensione tanto positiva quanto negativa anche se su brevi periodi temporali prendono il nome di **segnali bilanciati** (*balanced signals*); la loro media è zero, il che significa che non hanno componenti in corrente continua. La mancanza di questa componente è un vantaggio, in quanto alcuni canali trasmissivi, come il cavo coassiale o le linee con trasformatori, attenuano fortemente la componente in corrente continua per via delle loro proprietà fisiche. Inoltre, uno dei metodi per collegare il ricevente al canale, chiamato ad **accoppiamento capacitivo** (*capacitive coupling*) lascia transitare solo la porzione a corrente alternata di un segnale. In entrambi i casi, se inviamo un segnale la cui media non è zero stiamo sprecando dell'energia, in quanto la componente continua verrà filtrata.

Il bilanciamento aiuta ad aggiungere transizioni per il clock recovery, in quanto siamo in presenza di una commistione di valori positivi e negativi. Esso fornisce anche un modo semplice per calibrare i riceventi, in quanto è possibile misurare la media del segnale e usarla come soglia decisionale per la decodifica dei simboli in arrivo. Usando segnali non bilanciati la media potrebbe disallinearsi dalla vera soglia, ad esempio a causa della densità degli 1, il che causerebbe un incremento degli errori di decodifica.

Un modo diretto di formulare un codice bilanciato è di usare due livelli di tensione per rappresentare un 1 (ad esempio +1 V e –1 V) e 0 V per rappresentare uno 0. Per spedire un 1 il trasmittente alterna tra i livelli +1 V e –1 V in maniera tale che questi si annullino in media. Tale schema prende il nome di **codifica bipolare** (*bipolar encoding*) e nelle reti telefoniche viene

chiamato **AMI** (*alternate mark inversion*); quest'ultimo nome deriva dalle vecchie tecnologie telefoniche in cui un 1 veniva chiamato "*mark*" (letteralmente segno o simbolo) e uno 0 "*space*" (spazio). Un esempio è quello della Figura 2.20(e).

La codifica bipolare aggiunge un livello di tensione per ottenere il bilanciamento; in alternativa possiamo usare un'associazione come 4B/5B per ottenere lo stesso risultato (e contemporaneamente le transizioni per il clock recovery). Un esempio di codice bilanciato di questo tipo è la codifica di linea **8B/10B** che mappa 8 bit di input su 10 bit di output con una efficienza dell'80%. Gli 8 bit sono divisi in un gruppo di 5 bit, associato a una sequenza di 6 bit e un gruppo di 3 bit associato a una sequenza di 4. I simboli di 6 e 4 bit vengono poi concatenati. In ciascun gruppo alcune sequenze di input possono essere associate a sequenze di output che hanno le stesso numero di 0 e 1. Ad esempio, "001" è associato a "1001", che è bilanciato. Purtroppo non ci sono abbastanza combinazioni per bilanciare tutte le sequenze di output; per porre rimedio ogni sequenza di input è associata a due sequenze di output, una con un 1 in più e l'altra con uno 0 in più. Ad esempio, "000" è associato sia a "1011" sia al suo complemento "0100". Quando i bit in input vengono associati a quelli di output il codificatore ricorda la **disparità** del simbolo precedente (la disparità è in numero totale di 0 o 1 per cui il segnale è sbilanciato) e seleziona la sequenza di output che riduce la disparità. Con 8B/10B la disparità si riduce a essere al massimo di 2 bit; quindi, il segnale non sarà mai troppo sbilanciato, e non ci saranno neppure mai più di cinque 1 o 0 consecutivi per permettere il clock recovery.

Trasmissione in banda passante

Per spedire informazioni su un canale si usa spesso una gamma di frequenze che non inizia dallo zero. Per canali wireless non è pratico usare segnali a frequenze molto basse, in quanto la dimensione dell'antenna deve essere una frazione della lunghezza d'onda del segnale e in questo caso sarebbe molto grande. In ogni modo vincoli legislativi e la necessità di evitare interferenze impongono la scelta delle frequenze. Anche nel caso di reti cablate collocare un segnale in una certa banda di frequenza è utile per permettere a diversi tipi di segnali di coesistere sullo stesso canale. Questo tipo di trasmissione prende il nome di **trasmissione in banda passante** (*passband transmission*), in quanto una banda di frequenza arbitraria è usata per far passare il segnale.

Fortunatamente i contributi illustrati finora in questo capitolo sono tutti in termini di banda o ampiezza della banda di frequenza; il valore assoluto della frequenza non influenza la capacità di trasferimento dati. Questo vuol dire che possiamo prendere un segnale in **banda base** che occupa da 0 a B Hz e traslarlo fino a occupare una banda passante da S a $S + B$ Hz, senza cambiare il quantitativo di informazione che può trasportare. Per elaborare un segnale al ricevente possiamo traslarlo indietro in banda base dove è più semplice riconoscere i simboli.

La modulazione digitale è ottenuta sulla trasmissione in banda passante modulando un segnale portante che risiede in banda passante. Possiamo modulare l'ampiezza, la frequenza o la fase del segnale portante; ognuno di questi metodi ha un suo nome. In **ASK** (*amplitude shift keying*) due diverse ampiezze sono usate per rappresentare 0 e 1. Un esempio con un livello non a zero e uno a zero è rappresentato nella Figura 2.22(b). È possibile usare più di due livelli per rappresentare più simboli. Analogamente, con **FSK** (*frequency shift keying*), si possono usare due o più frequenze; l'esempio nella Figura 2.22(c) riporta l'uso di due frequenze. Nella versione più semplice di **PSK** (*phase shift keying*) l'onda portante è sistematicamente traslata di 0 o 180 gradi all'inizio della

trasmissione di ogni simbolo; in questo caso, essendoci due fasi, il sistema viene chiamato **BPSK** (*binary phase shift keying*). Qui "binario" si riferisce ai due simboli 0 e 1, non al fatto che i simboli rappresentano 2 bit. Un esempio è illustrato nella Figura 2.22(c). Uno schema migliore e più efficiente dal punto di vista della banda fa uso di quattro traslazioni: 45, 135, 225, e 315 gradi per trasmettere 2 bit di informazioni per simbolo. Questa versione è nota come **QPSK** (*quadrature phase shift keying*).

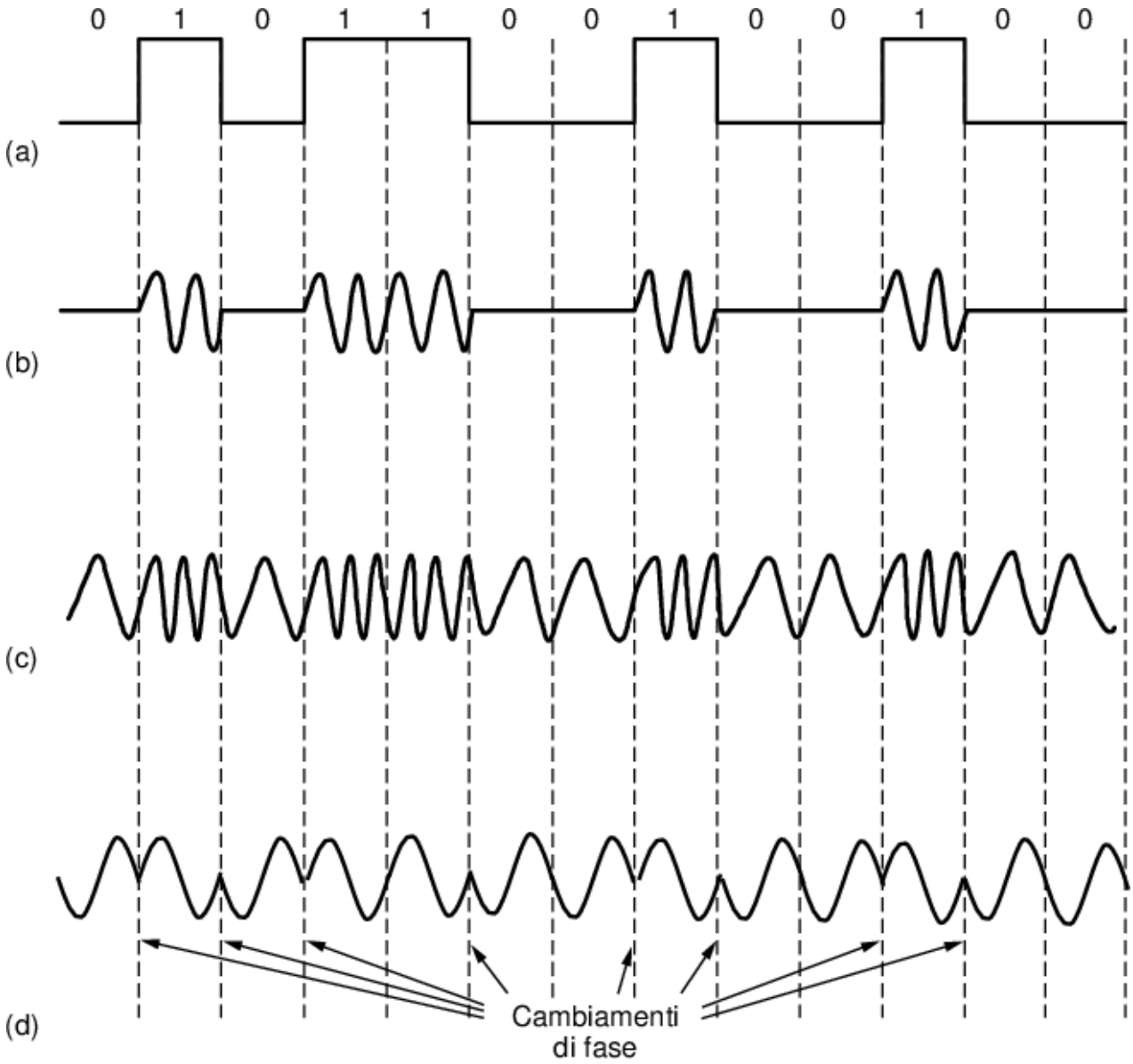


Figura 2.22 (a) Un segnale binario. (b) Modulazione di ampiezza. (c) Modulazione di frequenza. (d) Modulazione di fase.

Per trasmettere più bit per simbolo possiamo combinare questi approcci e usare più livelli. Solo una tra frequenza e fase può essere modulata ogni volta, perché esse sono correlate: la frequenza è infatti il tasso di variazione della fase nel tempo. Tipicamente, ampiezza e fase vengono modulate in maniera combinata. Tre esempi sono rappresentati nella Figura 2.23. In ogni esempio i punti rappresentano le combinazioni di ampiezza e fase ammissibili per ogni simbolo. Nella Figura

2.23(a), che è una rappresentazione del QPSK, possiamo osservare dei punti equidistanti a 45, 135, 225 e 315 gradi. Per ogni punto la fase è l'angolo tra la retta che lo congiunge con l'origine e l'asse delle ordinate, mentre l'ampiezza è la sua distanza dall'origine.

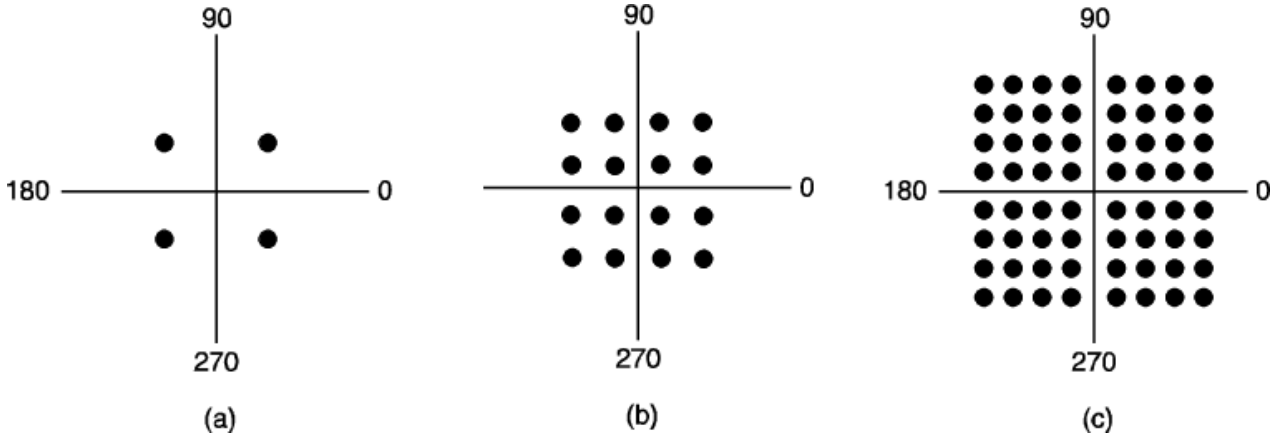
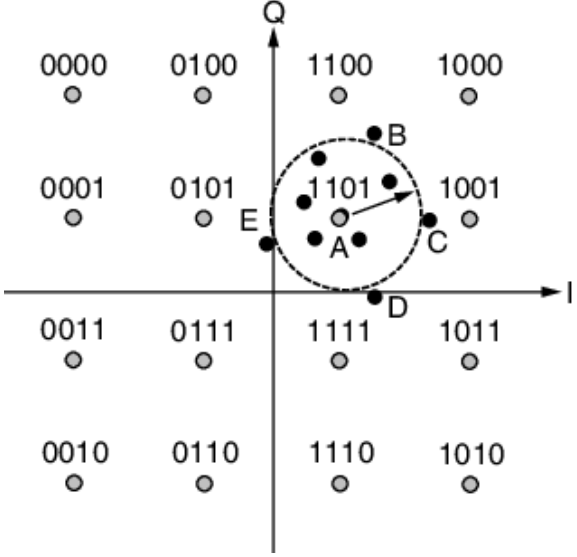


Figura 2.23 (a) QPSK. (b) QAM-16. (c) QAM-64.

Questo tipo di diagramma prende il nome di **constellation diagram** (*diagramma a costellazione*). Nella Figura 2.23(b) vediamo uno schema di modulazione con una costellazione più densa: vengono usate sedici combinazioni di ampiezza e fase, così si possono trasmettere 4 bit per simbolo. Questo schema viene chiamato **QAM-16** (*quadrature amplitude modulation*). Nella Figura 2.23(c) possiamo osservare una costellazione ancora più densa, con 64 combinazioni diverse in grado di codificare 6 bit per simbolo; il sistema viene chiamato **QAM-64**. È possibile anche usare QAM di ordine superiore. Come potreste sospettare dalla forma delle costellazioni è più facile costruire dei circuiti per la produzione di simboli come combinazione di valori sui due assi, anziché come combinazione di ampiezza e fase; questo è il motivo per cui la disposizione è a quadrati e non a cerchi concentrici.

Le costellazioni viste finora non mostrano come i bit siano assegnati ai simboli; al momento dell'assegnazione è necessario fare attenzione che bassi livelli di rumore in ricezione non portino a troppi errori sui bit. Questo potrebbe capitare se avessimo assegnato valori di bit consecutivi a simboli adiacenti. Con il QAM-16, ad esempio, se un simbolo rappresenta "0111" e il suo vicino "1000", un errore da parte del ricevente, per cui viene preso il simbolo adiacente, comporterebbe un errore di tutti i bit. Una soluzione migliore è quella di associare bit ai simboli in maniera tale che simboli adiacenti differiscano di solo un bit. Tale associazione viene chiamata **Gray code** (*codifica di Gray*); la Figura 2.24 mostra una costellazione QAM-16 codificata secondo Gray. Se il ricevente commette un errore di decodifica su di un simbolo, verrà fatto un errore su un solo bit nel caso in cui sarà preso un simbolo adiacente.



Quando viene spedito 1101

Punto	Decodificato come	Bit di errore
A	1101	0
B	1100	1
C	1001	1
D	1111	1
E	0101	1

Figura 2.24 Codifica di Gray per QAM-16.

Multiplexing a divisione di frequenza

Le tecniche di modulazione viste finora ci permettono di inviare un segnale per trasportare bit su un collegamento cablato o wireless. Tuttavia, le economie di scala giocano un ruolo importante nel modo di usare le reti: installare e mantenere una linea con molta banda e una con poca tra due uffici costa essenzialmente lo stesso quantitativo di denaro (questo perché il costo è dato dallo scavo e non dal tipo di cavo o fibra che viene interrato). Per permettere a molti segnali di condividere uno stesso canale trasmissivo sono nate le tecniche di multiplexing.

FDM (*frequency division multiplexing*, multiplexing a divisione di frequenza) sfrutta la trasmissione in banda passante per condividere un canale: divide lo spettro in bande di frequenza di cui ogni utente ha uso esclusivo per inviare i propri segnali. Un esempio tipico sono le trasmissioni radiofoniche AM, per cui lo spettro allocato è di circa un 1 MHZ (dai 500 a 1500 kHz) e frequenze distinte sono allocate a canali logici distinti (stazioni radio), ciascuno operante in una data porzione dello spettro, con una separazione tra canali abbastanza grande da evitare interferenze.

Per un esempio più dettagliato si faccia riferimento alla Figura 2.25 dove tre canali telefonici sono soggetti a multiplexing con FDM. L'uso di filtri limita la banda utilizzabile a circa 3100 Hz per i canali vocali; quando più canali sono soggetti a multiplexing, 4000 Hz vengono allocati a ogni canale. L'eccesso di allocazione prende il nome di **banda di guardia** (*guard band*) e permette di tenere i canali ben separati tra loro. Per prima cosa i canali vocali sono traslati verso l'alto in frequenza ognuno di un valore differente; successivamente possono essere combinati, in quanto nessuno di loro occupa la stessa porzione dello spettro. Si noti che, anche se grazie alla bande di guardia ci sono degli spazi tra i canali, siamo comunque in presenza di sovrapposizioni tra canali adiacenti; questo perché i filtri nel mondo reale non hanno spigoli netti come nella teoria. Quindi un forte picco ai bordi di un canale sarà trattato come rumore (non termico) nel canale adiacente.

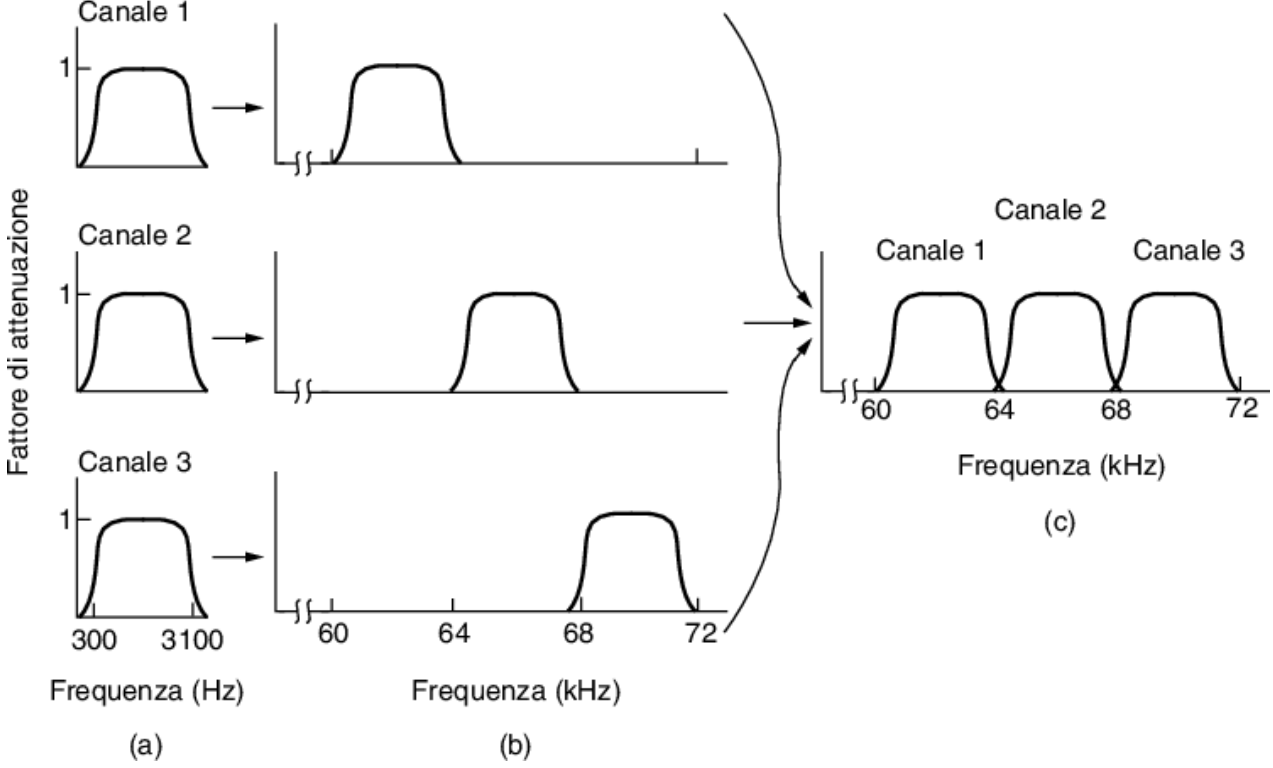


Figura 2.25 FDM. (a) Bande di frequenza originali. (b) Bande traslate in frequenza. (c) Canale con multiplexing.

Tale approccio è stato utilizzato per molti anni per fare multiplexing di chiamate nei sistemi telefonici; ora però si preferisce fare multiplexing sul tempo; tuttavia, FDM continua a essere usato ad alto livello nelle reti telefoniche come pure in quelle cellulari, wireless e satellitari.

Quando vengono inviati dati digitali è possibile dividere lo spettro in maniera efficiente anche senza usare le bande di guardia. **OFDM** (*orthogonal frequency division multiplexing*, multiplexing a divisione di frequenze ortogonali) divide la banda del canale in molte sottoportanti che inviano dati in maniera indipendente (ad esempio con QAM); queste sottoportanti sono stipate una a ridosso all'altra nel dominio delle frequenze, per cui ogni sottoportante si estende in quelle adiacenti. Tuttavia, come si può vedere nella Figura 2.26, la risposta in frequenza di ogni sottoportante è progettata in maniera tale da essere zero in corrispondenza del centro delle sottoportanti a essa adiacenti e può quindi essere campionata nella frequenza centrale senza interferenza da parte dei vicini. Per far funzionare tutto questo è richiesto un tempo di guardia per ripetere un sottoinsieme dei simboli trasmessi, in maniera tale da ottenere la risposta in frequenza richiesta. Tuttavia, questa aggiunta spreca meno risorse di quelle richieste da molte bande di guardia.

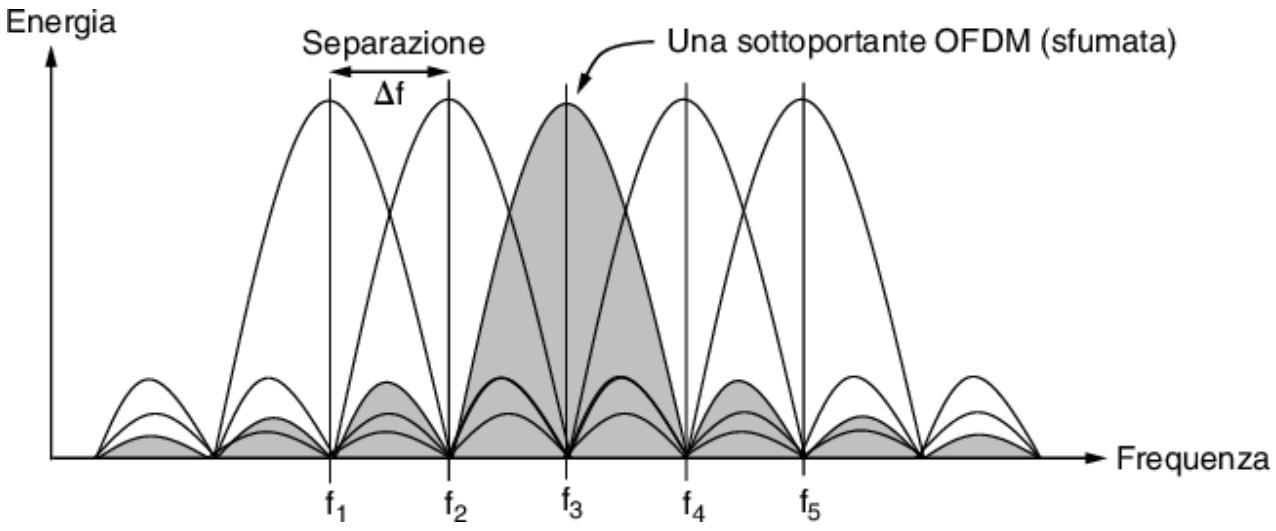


Figura 2.26 Multiplexing a divisione di frequenze ortogonali (OFDM).

L'idea alla base di OFDM è nota da parecchio tempo, ma è solo nell'ultimo decennio che è stata adottata su larga scala, dopo aver capito che è possibile implementarla in maniera efficiente operando una trasformata di Fourier dei dati digitali su tutte le sottoportanti (anziché modulare separatamente ogni sottoportante). OFDM è usata per 802.11, la trasmissione dati con TV via cavo e linee elettriche e si ipotizza di utilizzarla nella prossima generazione di reti cellulari. Di solito un flusso di informazioni digitali ad alta velocità è diviso in molti altri più lenti che sono trasmessi in parallelo sulle sottoportanti. Questa divisione è utile, perché semplifica la gestione delle perdite di segnale di un cavo a livello di sottoportante; se una di queste è troppo disturbata la si esclude in favore di altre che vengono ricevute meglio.

Multiplexing a divisione di tempo

Un'alternativa a FTM è **TDM** (*time division multiplexing*, multiplexing a divisione di tempo); in questo caso gli utenti fanno i turni secondo una politica round-robin e ognuno di loro, periodicamente, prende possesso della banda completa per un tempo limitato. Un esempio di tre flussi soggetti a TDM è rappresentato nella Figura 2.27, dove i bit di ogni flusso in ingresso sono presi in un certo intervallo temporale (**time slot**) e mandati in uscita come un flusso aggregato che procede alla somma delle velocità dei singoli flussi in ingresso. Per poter funzionare, i flussi devono essere sincronizzati nel tempo; piccoli intervalli (*tempi di guardia*), analoghi alle bande di guardia in frequenza, possono essere aggiunti per permettere minimi aggiustamenti.

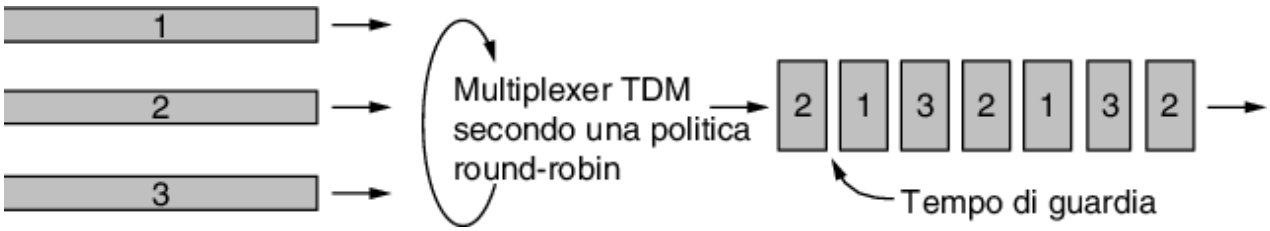


Figura 2.27 Time Division Multiplexing (TDM).

TDM viene spesso usato come parte delle reti telefoniche e cellulari. Per evitare confusione sia chiaro che è molto diverso dall'alternativa **STDM** (*statistical time division multiplexing*, multiplexing statistico a divisione di tempo). Il termine "statistico" è aggiunto a indicare che i flussi individuali contribuiscono *senza* seguire un piano preciso, ma in accordo con le statistiche delle loro richieste. STDM è anche conosciuto come *commutazione di pacchetto* (packet switching).

Multiplexing a divisione di codice

Esiste un terzo tipo di multiplexing che funziona in maniera completamente diversa da FDM e TDM: **CDM** (*code division multiplexing*, multiplexing a divisione di codice). Si tratta di una forma di comunicazione a spettro distribuito in cui un segnale a banda stretta viene sparso su una banda di frequenza più ampia. Ciò rende il segnale più tollerante alle interferenze e permette a più segnali di utenti diversi di condividere la stessa banda di frequenza. Siccome CDM viene spesso usato per il secondo scopo viene anche comunemente chiamato **CDMA** (*code division multiple access*, accesso con multiplexing a divisione di codice).

CDMA permette a ogni stazione di trasmettere su tutto lo spettro di frequenze in ogni momento. Le trasmissioni simultanee vengono separate usando la teoria dei codici. Prima di occuparci degli algoritmi consideriamo un'analogia: la sala d'aspetto di un aeroporto con molte coppie di persone che parlano. TDM equivale a coppie di persone che fanno a turno per parlare (globalmente, non tra loro); FDM, invece, equivale ad avere coppie che parlano ognuna con toni di voce diversi, in maniera tale che ognuna possa avere la propria conversazione contemporaneamente, ma indipendentemente dalle altre. CDMA equivale invece ad avere ogni coppia che parla contemporaneamente, ma in un linguaggio diverso; la coppia francese si sintonizza sulla lingua francese e rifiuta tutte le altre come rumore. Quindi, il punto centrale di CDMA è essere in grado di estrarre il segnale e rifiutare tutto il resto come rumore casuale. Di seguito proponiamo una descrizione abbastanza semplificata di CDMA.

In CDMA il tempo di trasmissione di ogni bit è suddiviso in m intervalli più brevi chiamati **chip**. Di solito ci sono da 64 a 128 chip per bit anche se nell'esempio qui discusso useremo 8 chip/bit per semplicità. A ogni stazione viene assegnato un codice univoco di m bit chiamato **chip sequence** (*sequenza di chip*). Per motivi didattici è conveniente usare una notazione bipolare e scrivere questi codici come sequenze di "–1" e "+1". Indicheremo le sequenze di chip tra parentesi.

Per trasmettere un bit con valore 1 una stazione invia la sua sequenza di chip, mentre per trasmettere uno 0 ne invia la negazione; nessun'altra sequenza è ammessa. Quindi, per $m = 8$ se alla stazione A è stata assegnata la sequenza (–1 –1 –1 +1 +1 –1 +1 +1) può spedire un 1 trasmettendo la sequenza di chip e uno 0 trasmettendo (+1 +1 +1 –1 –1 +1 –1 –1). In realtà vengono mandate delle tensioni, ma è sufficiente per noi pensare in termini di queste sequenze.

Aumentare l'informazione che deve essere spedita da b bit/s a mb chip/s per ogni stazione comporta una necessità di banda per il CDMA più grande di un fattore m della banda necessaria a una stazione che non ne fa uso (assumendo che non ci siano modifiche nelle tecniche di modulazione e codifica). Se è disponibile una banda di 1 MHz disponibile per 100 stazioni, con FDM ognuna di esse potrebbe avere 10 kHz e potrebbe inviare dati a 10 kbps (assumendo un bit a Hz). Con CDMA ogni stazione usa completamente 1 MHz, per cui il tasso di invio è di 100 chip per bit per distribuire i 10 kbps su tutto il canale.

Nelle Figure 2.28(a) e (b) facciamo vedere la sequenza di chip assegnata a quattro stazioni insieme ai segnali rappresentati dalle sequenze. Ogni stazione ha una sua sequenza di chip univoca e useremo il simbolo **S** per indicare il vettore di m chip della stazione S, e **S–** per la sua negazione. Tutte le sequenze di chip sono ortogonali a coppie; ciò significa che il prodotto interno normalizzato di ogni coppia distinta di sequenze di chip **S** e **T** (scritto come **S•T**) è zero. Tale sequenza di chip ortogonali viene generata tramite un metodo noto come **codice di Walsh**. In termini matematici, l'ortogonalità delle sequenze di chip può essere espressa come segue:

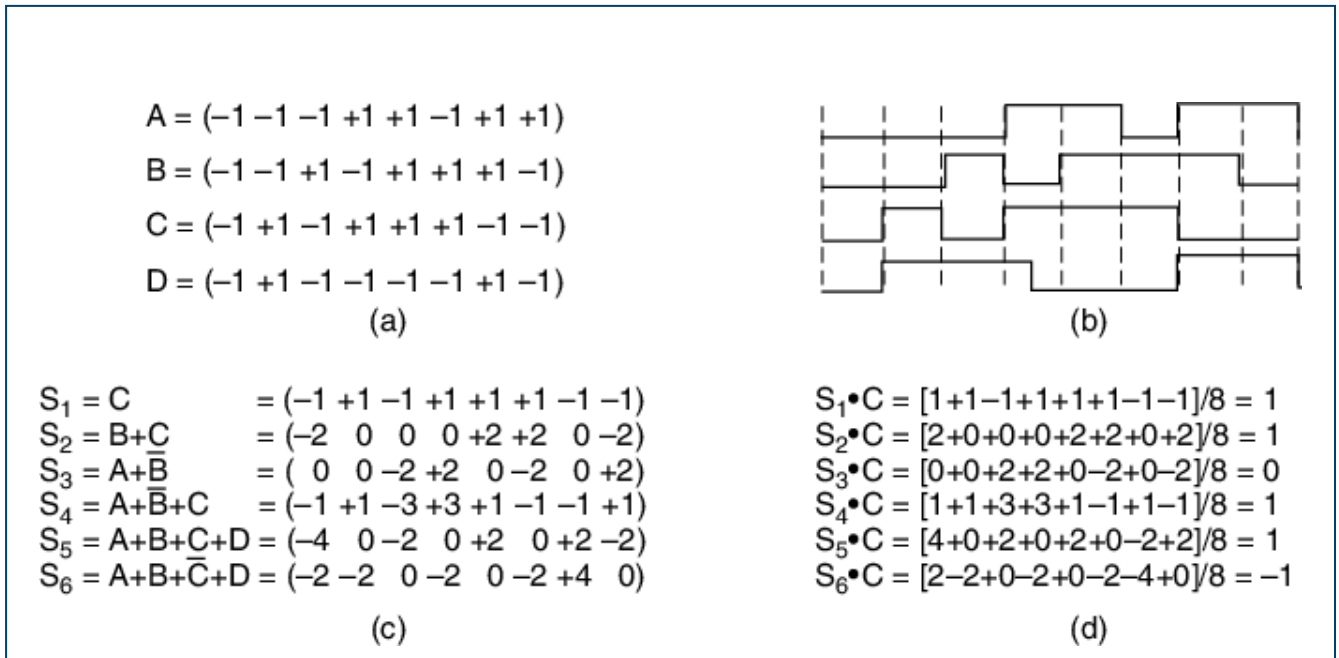


Figura 2.28 (a) Sequenze di chip per quattro stazioni. (b) Segnali rappresentati dalle sequenze. (c) Sei esempi di trasmissione. (d) Recupero del segnale della stazione C.

$$\mathbf{S \bullet T} \equiv \frac{1}{m} \sum_{i=1}^{m} S_i T_i = 0 \quad (2\text{-}5)$$

In altri termini, ci sono tante coppie di bit diverse quante coppie uguali. Questa proprietà di ortogonalità si dimostrerà strategica più tardi. Si noti che se **S•T = 0**, allora **S•T–** è anch'esso 0. Il valore del prodotto interno normalizzato di ogni sequenza di chip con se stessa è 1:

$$\mathbf{S \bullet S} = \frac{1}{m}\sum_{i=1}^{m} S_i S_i = \frac{1}{m}\sum_{i=1}^{m} S_i^2 = \frac{1}{m}\sum_{i=1}^{m} (\pm 1)^2 = 1$$

Questo perché ognuno degli *m* termini del prodotto interno è uguale a 1, per cui la somma è *m*. Si noti inoltre che **S•S–** = –1.

Durante il tempo di trasmissione di ogni bit una stazione può trasmettere un 1 (inviando la sua sequenza di chip), uno 0 (inviando il negativo della sua sequenza di chip) o rimanere in silenzio e non trasmettere nulla. Assumiamo per ora che tutte le stazioni siano sincronizzate e che quindi tutte le sequenze di chip inizino nello stesso istante. Quando due stazioni trasmettono contemporaneamente le loro sequenze bipolari si sommano linearmente. Ad esempio, se in un dato istante tre stazioni trasmettono +1 e una trasmette –1, sarà ricevuto un segnale di +2. È possibile pensare a livelli di tensione che si sovrappongono sul canale: tre stazioni emettono +1V e una stazione –1V; in tal modo vengono ricevuti 2V. Nella Figura 2.28(c) vediamo sei esempi di una o più stazioni che trasmettono un bit con valore 1 nello stesso istante. Nel primo esempio solo *C* trasmette, per cui otteniamo la sequenza di chip di *C*; nel secondo esempio sia *B* che *C* trasmettono, quindi riceveranno la somma delle loro sequenze bipolari, cioè:

```
(–1 –1 +1 –1 +1 +1 +1 –1) + (–1 +1 –1 +1 +1 +1 –1 –1) = (–2 0 0 0 +2 +2 0 –2)
```

Per recuperare la sequenza di bit generata da una certa stazione il ricevente deve conoscere in anticipo la sequenza di chip di quella stazione. Il recupero si effettua calcolando il prodotto interno normalizzato tra la sequenza di chip ricevuta e la sequenza della stazione i cui bit vogliamo recuperare. Se la sequenza di chip ricevuta è **S** e il ricevente sta cercando di ascoltare la stazione la cui sequenza di chip è **C**, basta calcolare il prodotto interno normalizzato **S•C**.

Per vedere come questo funzioni basta immaginare due stazioni *A* e *C* che trasmettono un bit con valore 1 contemporaneamente a *B* che trasmette un bit con valore 0, come nel caso del terzo esempio. Il ricevente vede la somma **S = A + B– + C**, e calcola

```
S•C = (A + B– + C)•C = A•C + B–•C + C•C = 0 + 0 + 1 = 1
```

I primi due termini scompaiono, in quanto tutte le coppie della sequenza di chip sono state scelte in modo da essere ortogonali, come indicato nell'Equazione (2-5). Quindi dovrebbe ora essere chiaro perché questa proprietà deve essere imposta alla sequenza di chip.

Per rendere il processo di decodifica più concreto sono mostrati sei esempi nella Figura 2.28(d). Si supponga che il ricevente sia interessato a estrarre il bit spedito dalla stazione *C* per ognuno dei sei segnali da S_1 a S_6. Il ricevente calcola il bit sommando il prodotto coppia a coppia di quanto ricevuto con in vettore *C* nella Figura 2.28(a) e poi prendendo 1/8 del risultato (dato che *m* = 8). Gli esempi includono casi in cui *C* è silenzioso, spedisce bit di valore 1 o 0 individualmente e in combinazione con altre trasmissioni. Come si può vedere il bit corretto viene decodificato ogni volta; esattamente come parlare francese.

In linea di principio, avendo sufficiente capacità computazionale, il ricevente può ascoltare tutti i trasmittenti contemporaneamente eseguendo l'algoritmo di decodifica per ognuno di loro in parallelo. Nella vita reale basti dire che è più facile a dirsi che a farsi ed è utile sapere quali trasmittenti potrebbero trasmettere.

Nel sistema CDMA ideale e senza rumore studiato qui il numero di stazioni che inviano dati può essere esteso in maniera arbitraria usando sequenze di chip più lunghe. Per 2^n stazioni i codici di Walsh possono fornire 2^n sequenze di chip ortogonali di lunghezza 2^n. Tuttavia, un'importante limitazione è che abbiamo assunto che tutte le chip delle sequenze siano sincronizzate nel tempo; questa sincronizzazione non è vera neppure come approssimazione in alcune applicazioni quali reti cellulari (in cui CDMA è stata largamente adottata a partire dagli anni '90) portando a delle progettazioni diverse. Torneremo su questo argomento più tardi all'interno di questo stesso capitolo descrivendo come CDMA asincrono differisca da CDMA sincrono.

Oltre che nelle reti cellulari, CDMA è usato da satelliti e reti per TV via cavo. Durante questa breve introduzione abbiamo trascurato molte altre complicazioni; chi volesse approfondire la conoscenza di CDMA può leggere Viterbi (1995) e Lee e Miller (1998), che richiedono però qualche conoscenza di ingegneria delle comunicazioni.

2.6 La rete telefonica pubblica commutata

Quando due computer appartenenti alla stessa azienda e situati in prossimità devono comunicare tra loro, spesso la cosa più facile da fare è stendere un cavo dall'uno all'altro; le LAN funzionano proprio in questo modo. Se invece le distanze sono grandi, i computer sono molti oppure i cavi devono attraversare strade e altri luoghi pubblici; stendere cavi privati di solito ha un costo proibitivo.

Inoltre, in quasi ogni nazione del mondo, stendere linee di trasmissione private attraverso (o al di sotto di) proprietà pubbliche è illegale. Di conseguenza i progettisti di reti devono affidarsi ai sistemi di telecomunicazioni esistenti.

Questi servizi, in particolare la **PSTN** (*public switched telephone network*, rete telefonica pubblica commutata), sono stati progettati molti anni fa, avendo in mente un obiettivo completamente diverso: trasmettere la voce umana in una forma più o meno comprensibile. L'adattabilità di questi mezzi alla comunicazione tra singoli computer è spesso marginale. Per apprezzare la dimensione del problema, si consideri che un semplice cavo tra due computer può trasferire dati a 1 Gbps o più. Al contrario, l'ADSL, la velocissima alternativa a un modem telefonico, viaggia a circa 1 Mbps. La differenza tra i due casi è la stessa che intercorre tra viaggiare in aeroplano e farsi una passeggiata tranquilla.

Nonostante tutto, il sistema telefonico è strettamente correlato con le reti di calcolatori (su larga scala), per cui vale la pena destinare del tempo a studiarlo nel dettaglio. Il fattore limitante ai fini della rete risulta essere ciò che commercialmente viene definito "ultimo miglio", cioè il tratto di cavo/doppino tramite cui il cliente si connette, e non le linee principali (*trunk*) o gli apparati all'interno della rete telefonica. Questa situazione sta cambiando con la graduale adozione di fibre e tecnologie digitali alla periferia della rete, ma ciò comporterà molto tempo e denaro. Nell'attesa i progettisti di computer, abituati a lavorare con sistemi che hanno prestazioni migliori di tre ordini di grandezza, hanno dedicato molto tempo e sforzi a capire come usare la rete telefonica in maniera efficiente.

Nei successivi paragrafi descriveremo il sistema telefonico e mostreremo come funziona. Per informazioni aggiuntive riguardo il suo funzionamento interno si veda Bellamy (2000).

Struttura del sistema telefonico

Non appena Alexander Graham Bell brevettò il telefono nel 1876 (solo poche ore prima del suo rivale Elisha Gray), la nuova invenzione suscitò un'enorme richiesta. Il mercato iniziale fu quello della vendita dei telefoni; gli apparecchi erano venduti a coppie e spettava al cliente tirare un cavo tra i due telefoni. Se voleva comunicare con altri *n* proprietari di telefoni, un utente doveva tirare cavi separati tra la sua e le altre *n* abitazioni. In un anno le città furono coperte da cavi che passavano alla rinfusa sopra le abitazioni e gli alberi. Divenne immediatamente chiaro che il modello usato per collegare ogni telefono a ogni altro telefono, rappresentato nella Figura 2.29(a), non poteva funzionare.

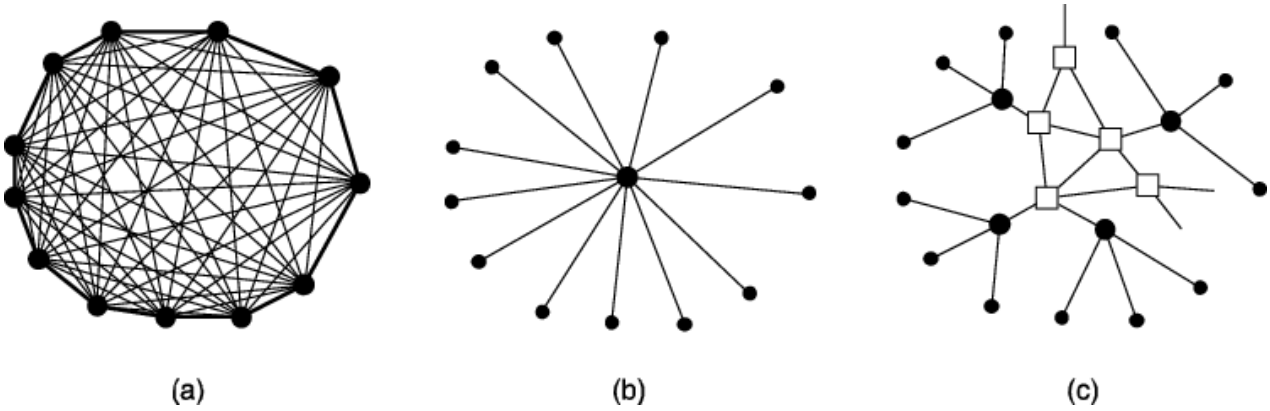


Figura 2.29 (a) Una rete completamente connessa. (b) Commutatore centralizzato. (c) Gerarchia a due livelli.

Bell se ne accorse e costituì la Bell Telephone Company, azienda che aprì il suo primo ufficio di commutazione (in New Haven, nel Connecticut) nel 1878. La società stese cavi fino alle case e agli uffici dei clienti. Per fare una chiamata, un cliente doveva girare la manovella del telefono in modo da far suonare un campanello nell'ufficio dell'azienda telefonica, dove richiamava l'attenzione di un operatore; questi manualmente collegava il chiamante al chiamato mediante un cavo a ponte. La Figura 2.29(b) mostra un modello di ufficio a commutazione singola.

Ben presto gli uffici di commutazione di Bell System apparvero un po' ovunque e le persone cominciarono a richiedere telefonate interurbane; allora la società iniziò a collegare tra loro i suoi uffici di commutazione. A questo punto si ripresentò il problema originale: poiché era inimmaginabile collegare ogni ufficio di commutazione a ogni altra centrale attraverso un cavo, venne inventata una centrale di commutazione di secondo livello. In breve tempo furono necessari diverse centrali di secondo livello, come mostrato nella Figura 2.29(c). Alla fine la gerarchia crebbe fino a raggiungere cinque livelli.

Nel 1890 le tre parti principali del sistema telefonico furono pronte: gli uffici di commutazione, i cavi tra clienti e centrale (doppini bilanciati e isolati, non più cavi aperti con un ritorno a terra) e le connessioni su lunghe distanze tra le centrali di commutazione. Per una breve storia del sistema telefonico si veda (Hawley, 1991).

Anche se da allora ci sono stati dei miglioramenti in tutte e tre queste aree, il modello della Bell System è rimasto essenzialmente invariato per oltre 100 anni. La descrizione che segue è estremamente semplificata, ma riesce comunque a spiegare i concetti di base. Da ogni telefono partono due cavi di rame che si collegano direttamente alla centrale più vicina dell'azienda telefonica (chiamata anche **centrale locale**, *end office* e *local central office*); di solito la centrale si trova a una distanza compresa tra 1 e 10 km, è più vicina nelle città, mentre è più lontana nelle aree rurali. Negli Stati Uniti ci sono circa 22.000 centrali locali. Le connessioni tramite doppino tra ogni telefono e le centrali sono chiamate commercialmente **local loop**, *ultimo miglio* e *collegamento locale*. Se i collegamenti locali di tutto il mondo fossero stesi uno dopo l'altro, coprirebbero mille volte la distanza che separa la Terra dalla Luna.

Un tempo l'80% del valore di AT&T era rappresentato dal rame usato per costruire i collegamenti locali. AT&T era, in effetti, la più grande miniera di rame del mondo. Per fortuna questo fatto non era molto noto nella comunità degli investitori. Se lo fosse stato, alcuni razziatori di società avrebbero potuto acquistare AT&T, interrompere il servizio telefonico in tutti gli Stati Uniti, estrarre tutti i cavi e vendere il rame ai raffinatori per ottenere un rapido guadagno.

Se un abbonato collegato a una certa centrale locale chiama un altro abbonato collegato alla stessa centrale, il meccanismo di commutazione integrato nella centrale crea una connessione elettrica diretta tra i due collegamenti locali; questa connessione rimane attiva per tutta la durata della chiamata.

Se il telefono chiamato è collegato a un'altra centrale locale si attiva una procedura differente: ogni centrale locale ha diverse linee in uscita che conducono a uno o più centri di commutazione attigui chiamati **centrali interurbane** (*tool office* o *tandem office*, a seconda che siano o meno in aree diverse). Queste linee sono chiamate **linee di connessione interurbana** (*tool connecting trunk*). La varietà dei centri di commutazione e la loro topologia varia da paese a paese a seconda della densità dei telefoni.

Se la centrale locale del chiamante e quella del chiamato hanno una linea di connessione interurbana diretta verso la stessa centrale interurbana (situazione molto comune quando le centrali sono relativamente vicine), è in quest'ultima che si può stabilire la connessione. La Figura 2.29(c) mostra una rete telefonica composta da telefoni (punti più piccoli), centrali locali (punti più grandi) e centrali interurbane (quadrati).

Se il chiamante e il chiamato non hanno una centrale interurbana in comune, il percorso sarà stabilito più in alto nella gerarchia. Le centrali interurbane comunicano tra loro attraverso speciali linee a larga banda, **intertoll trunk** o **interoffice trunk**. Prima della scissione di AT&T nel 1984 il sistema telefonico americano per trovare un percorso usava un sistema di instradamento gerarchico: si andava verso l'alto nella gerarchia fino a che non si trovava una centrale di commutazione in comune per poi ridiscendere. Questo è stato poi sostituito con un instradamento più flessibile e non gerarchico. La Figura 2.30 illustra come viene fatto l'instradamento di una chiamata a lunga distanza.

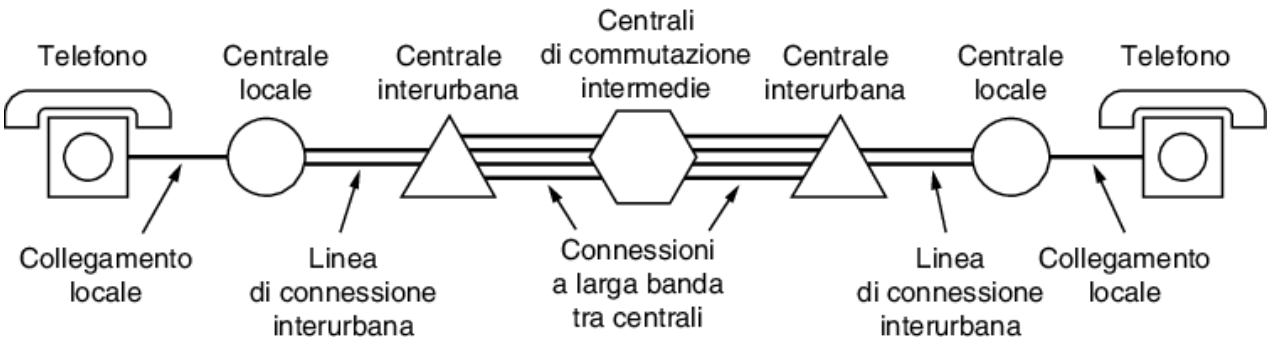


Figura 2.30 Il percorso tipico di una chiamata a media distanza.

Per le telecomunicazioni viene usata una moltitudine di mezzi trasmissivi. Diversamente dagli edifici moderni, dove il cablaggio è normalmente in categoria 5, l'ultimo miglio a uso domestico consiste tipicamente di un doppino intrecciato di categoria 3 e le fibre stanno cominciando solo ora ad apparire. Tra centrali di commutazione si fa largo uso di cavi coassiali, microonde e, in special modo, fibre ottiche.

In passato la trasmissione attraverso il sistema telefonico era analogica: il segnale vocale era trasmesso dalla sorgente alla destinazione sotto forma di tensione elettrica. Con l'avvento delle fibre ottiche, dell'elettronica digitale e dei computer, tutte le linee e i commutatori sono diventati digitali e l'ultimo miglio rappresenta l'ultimo tassello di tecnologia analogica del sistema.

La trasmissione digitale è preferita perché rende superfluo riprodurre in modo accurato una forma d'onda analogica dopo il passaggio attraverso molti amplificatori in una chiamata a lunga distanza. È sufficiente poter distinguere correttamente uno 0 da un 1. Questa proprietà rende la trasmissione digitale più affidabile di quella analogica; inoltre è più economica e facile da gestire.

Riepilogando, il sistema telefonico è costituito da tre componenti principali:

1. ultimo miglio/collegamenti locali (doppini analogici che arrivano nelle abitazioni e negli uffici)
2. trunk/connessioni interurbane (fibre ottiche digitali che collegano le centrali di commutazione)
3. centrali di commutazione (che spostano le chiamate da una linea all'altra).

Dopo un breve riepilogo delle politiche che riguardano la telefonia, il testo esaminerà in dettaglio ognuno di questi componenti. I collegamenti locali permettono l'accesso a tutti, perciò sono cruciali; sfortunatamente sono anche il punto debole del sistema. Per le linee a lunga distanza il problema principale riguarda il modo di raccogliere e trasmettere più chiamate attraverso la stessa fibra con multiplexing e per farlo useremo FDT e TDM. Infine, l'ultima parte è dedicata ai due metodi fondamentalmente diversi che vengono utilizzati per realizzare la commutazione.

Politiche telefoniche

Nei decenni che hanno preceduto il 1984, Bell System ha gestito il servizio locale e quello sulle lunghe distanze attraverso gran parte degli Stati Uniti. Negli anni '70 il governo federale arrivò a credere che questo monopolio fosse illegale e fece di tutto per smantellarlo. Il governo vinse e il primo gennaio 1984 AT&T venne divisa in AT&T Long Lines, 23 **BOC** (*bell operating companies*) e pochi altri pezzi. Le 23 BOC vennero raggruppate in sette BOC regionali (*RBOC*) per renderle economicamente vitali. Nello spazio di una notte l'intera natura delle telecomunicazioni negli Stati Uniti venne stravolta dall'ordine di un giudice (non da un atto del Congresso).

Tutti i dettagli di questo disinvestimento sono descritti nel cosiddetto **MFJ** (*modified final judgment*), evidente ossimoro, poiché se il giudizio può essere modificato non è certo finale. Questo evento incrementò la concorrenza, migliorò il servizio e fece diminuire i prezzi delle telefonate interurbane. Tuttavia i prezzi dei servizi locali aumentarono: l'eliminazione dei sussidi incrociati derivanti dalle chiamate interurbane, infatti, costrinse il servizio locale ad autosostenersi. Molti altri paesi hanno poi introdotto la concorrenza seguendo la stessa linea di condotta.

Questo nuovo e competitivo ambiente riveste grande rilevanza per i nostri studi, in quanto è stato la causa dell'aggiunta di una caratteristica chiave all'infrastruttura della rete telefonica. Per far capire chi poteva fare che cosa, gli Stati Uniti sono stati divisi in 164 **LATA** (*local access and transport area*); grosso modo, una LATA è grande quanto l'area coperta da un prefisso telefonico. In ogni LATA c'è un **LEC** (*local exchange carrier*) che ha il monopolio del servizio telefonico tradizionale in quell'area. I LEC più importanti erano i BOC, anche se alcuni LATA contenevano una o più delle 1500 compagnie telefoniche indipendenti che operavano come LEC.

La nuova caratteristica fu che tutto il traffico tra le LATA veniva gestito da una diversa azienda, definita **IXC** (*interexchange carrier*). Inizialmente AT&T Long Lines era l'unico vero IXC, ma ora Verizon e Sprint sono diventati concorrenti stabili nel mercato IXC. Durante la scissione, una delle preoccupazioni principali fu garantire che tutti gli IXC fossero trattati equamente in termini di qualità della linea, tariffe e numero di cifre assegnate agli abbonati; la Figura 2.31 mostra com'è stata soddisfatta questa necessità. L'immagine rappresenta tre esempi di LATA, ognuna con diverse centrali locali. Le LATA 2 e 3 hanno anche una piccola gerarchia con centrali interurbane. Ogni IXC che desidera gestire le chiamate originate in una LATA può costruire all'interno dell'area una centrale di commutazione chiamata POP (*point of presence*). Il LEC deve collegare ogni IXC a ogni centrale locale, direttamente (come nel caso delle LATA 1 e 3) o indirettamente (come nel caso della LATA 2). Inoltre, le caratteristiche tecniche ed economiche della connessione devono essere identiche per tutti gli IXC; in tal modo, per esempio, un utente in LATA 1 può scegliere quale IXC utilizzare per chiamare gli abbonati in LATA 3.

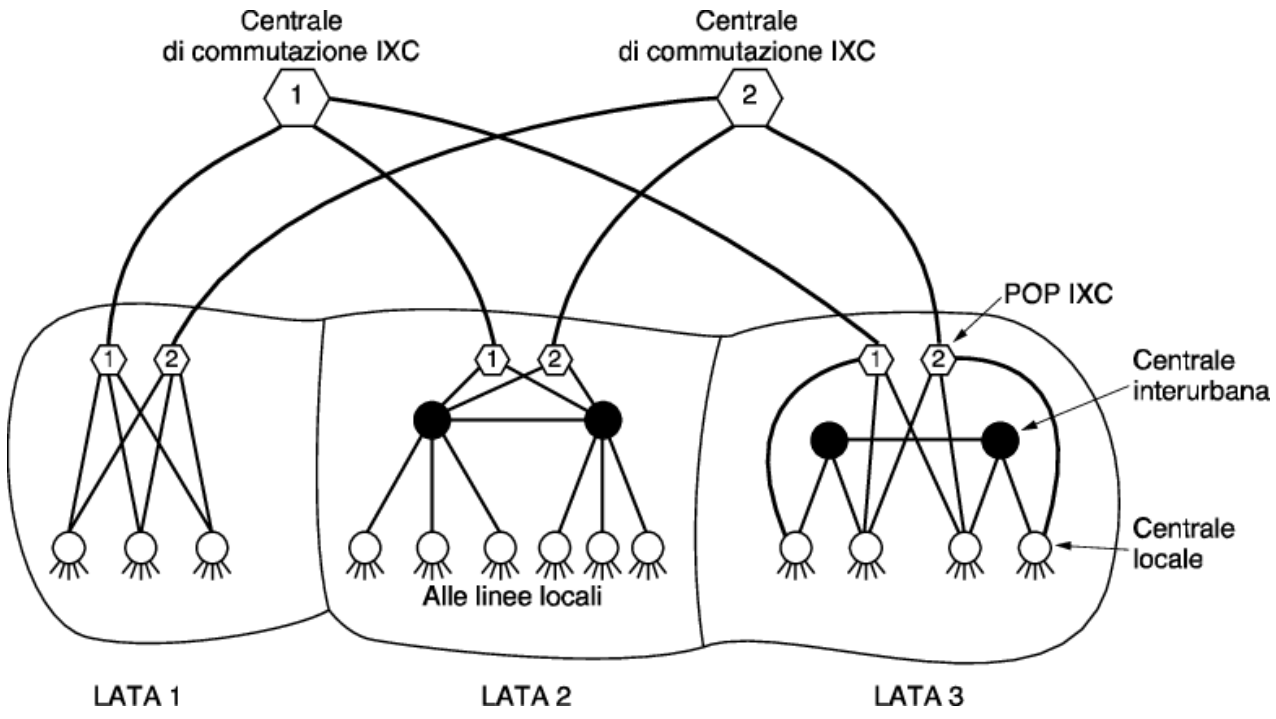


Figura 2.31 Le relazioni tra LATA, LEC e IXC. I cerchi sono le centrali di commutazione LEC. Ogni esagono appartiene all'IXC indicato dal numero al suo interno.

L'MFJ vieta agli IXC di offrire servizi di telefonia locale e ai LEC di offrire servizi di telefonia tra LATA, ma entrambi possono entrare in altri settori commerciali, per esempio in quello della ristorazione. Quella del 1984 fu una decisione abbastanza ambigua. Sfortunatamente la tecnologia ha un modo tutto suo di rendere obsoleta la legge. Né la televisione via cavo né i telefoni cellulari furono considerati nell'accordo. Con l'avvento della televisione via cavo e dei telefoni cellulari, LEC e IXC iniziarono ad acquistare o fondersi con operatori di telefonia cellulare e trasmissione via cavo.

Nel 1995 il Congresso capì che non era più possibile mantenere una distinzione tra i diversi tipi di aziende e preparò un documento che, mantenendo la concorrenza, consentiva a televisioni via cavo, aziende telefoniche locali, operatori su lunghe distanze e operatori mobili l'ingresso in qualunque settore. L'idea era che ogni società poteva offrire ai propri clienti un singolo pacchetto integrato comprendente TV via cavo, telefonia e servizi d'informazione, e che società diverse potevano competere sui servizi e sui prezzi. Il documento divenne legge nel febbraio 1996, e come conseguenza alcuni BOC divennero IXC e alcune altre aziende, tra cui i gestori di televisioni via cavo, iniziarono a offrire servizi di telefonia locale concorrendo con i LEC.

Una proprietà interessante della legge del 1996 è il requisito che impone ai LEC di implementare la **portabilità del numero locale**; questo significa che un cliente può cambiare il fornitore del servizio telefonico senza cambiare il numero di telefono. Tale provvedimento rimuove quello che per molti rappresentava un enorme ostacolo, rendendo gli utenti più inclini al cambio di LEC e incrementando la concorrenza. Oggi le telecomunicazioni statunitensi stanno vivendo una radicale ristrutturazione, e molti altri paesi stanno iniziando a comportarsi allo stesso modo. Gli Stati Uniti fanno infatti da modello per molte nazioni: se l'esperimento riuscisse altri seguirebbero la stessa strada; in caso contrario si tenterebbe qualcos'altro.

Collegamenti locali: modem, ADSL e fibre

È giunto il momento di affrontare l'esame dettagliato del funzionamento del sistema telefonico. Cominciamo con quella parte del sistema con cui gli utenti hanno più familiarità, ossia il collegamento locale bifilare che collega la centrale della società telefonica alle abitazioni e a piccoli uffici. Il collegamento locale è chiamato anche "ultimo miglio", sebbene la lunghezza superi spesso questa estensione; ha utilizzato la trasmissione analogica per più di 100 anni e probabilmente continuerà a farlo per gli anni a venire, anche a causa degli elevati costi richiesti dalla conversione al digitale.

Molti sforzi sono stati fatti per permettere la trasmissione dati sull'ultimo miglio in rame, di cui era già stata fatta l'installazione. I modem telefonici spediscono segnali digitali tra computer usando lo stesso stretto canale messo a disposizione per le chiamate vocali. Una volta erano molto utilizzati, ma sono stati per lo più rimpiazzati da tecnologie a larga banda come ADSL che usa l'ultimo miglio per inviare dati digitali tra utente e centrale locale dove vengono convogliati verso Internet. Entrambi, modem analogici e ADSL, hanno a che fare con le limitazioni imposte da un ultimo miglio a volte datato: banda relativamente stretta, attenuazione e distorsione dei segnali e sensibilità al rumore elettrico come il crosstalk.

In alcuni casi l'ultimo miglio è stato modernizzato installando fibre ottiche fino (o quasi) all'abitazione. Queste installazioni forniscono la base per le reti di calcolatori grazie a un ultimo miglio a larga banda per servizi dati. Il fattore limitante è quanto gli utenti devono pagare, non la fisica dell'ultimo miglio.

In questo paragrafo studieremo l'ultimo miglio, sia vecchio che nuovo, occupandoci di modem telefonici, ADSL e fibre.

Modem telefonici

Per spedire dei bit sull'ultimo miglio, come pure su ogni altro canale fisico, questi devono essere convertiti in segnali analogici che possano essere trasmessi sul canale. Tale conversione è ottenuta usando i metodi per la modulazione digitale che abbiamo studiato nel paragrafo precedente. All'altro capo del canale il segnale analogico viene convertito di nuovo in bit.

Un dispositivo che converte un flusso di bit in un segnale analogico che li rappresenta prende il nome di **modem**, abbreviazione di "mo*dulator* dem*odulator*" (modulatore demodulatore). Esistono molte varietà di modem: telefonici, DSL, cable, wireless e così via. Un modem può essere interno a un computer (pratica piuttosto comune per modem telefonici) o risiedere in un contenitore separato (come nel caso di modem DSL e cable). Dal punto di vista logico un modem si colloca tra il computer (digitale) e il sistema telefonico (analogico), come rappresentato nella Figura 2.32.

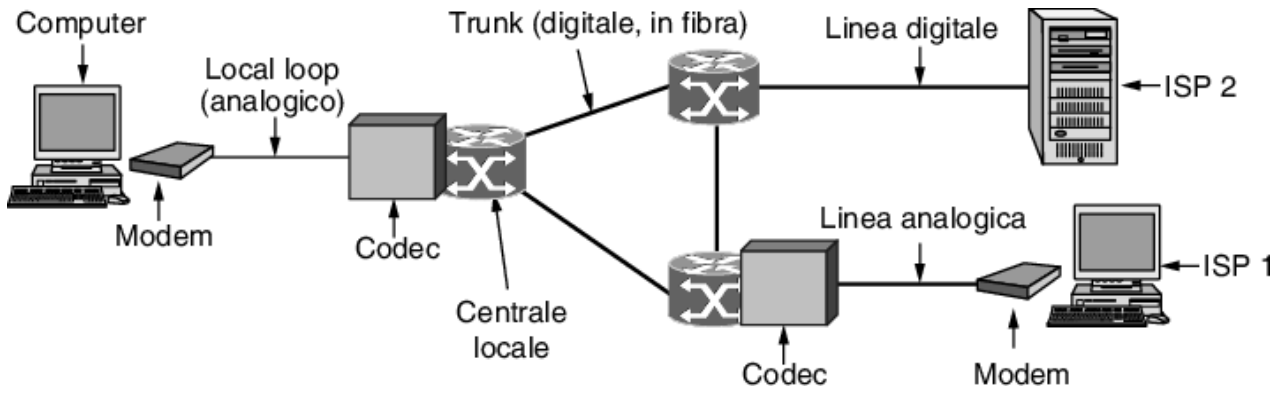


Figura 2.32 Uso di trasmissioni sia analogiche che digitali per chiamata da computer a computer. La conversione viene effettuata da modem e codec.

I modem telefonici sono usati per inviare bit tra due computer su una linea telefonica al posto della comunicazione vocale che normalmente ha luogo. La difficoltà principale nel fare questo è che una linea telefonica per la voce è limitata a 3100 Hz, circa quello che è sufficiente a trasferire una conversazione. Questa banda è inferiore di almeno quattro ordini di grandezza di quella utilizzata per Ethernet o 802.11 (WiFi). Non sorprende che le velocità di trasmissione dati dei modem telefonici siano anch'esse 4 ordini di grandezza inferiori a Ethernet e 802.11.

Facciamo ora due conti per capire perché questo accade. Il teorema di Nyquist ci dice che anche con una linea perfetta a 3000 Hz (cosa che una linea telefonica non è assolutamente) non è possibile spedire simboli a un tasso più veloce di 6000 baud. Nella pratica molti modem inviano dati a velocità di 2400 baud concentrandosi ad aumentare il numero di bit per simbolo e permettere il traffico in entrambe le direzioni contemporaneamente (usando frequenze diverse per le due direzioni).

Il modesto modem a 2400 bps usa 0 volt per lo 0 logico e 1 volt per un 1 logico con un bit per simbolo. Facendo un passo avanti è possibile usare 4 simboli diversi come nelle quattro fasi di QPSK per cui con 2 bit per simbolo può arrivare a una velocità di 4800 bps.

Una lunga sequenza di velocità sempre maggiori è stata ottenuta via via che la tecnologia avanzava. Velocità maggiori richiedono un insieme di simboli più grande o costellazioni; con molti simboli anche una bassa quantità di rumore nella rilevazione dell'ampiezza o della fase può risultare un errore. Per ridurre la probabilità di errore gli standard per modem ad alta velocità impongono alcuni simboli per la correzione di errore; questi schemi sono noti come **TCM** (*trellis coded modulation*) (Ungerboeck, 1987).

Lo standard **V.32** per i modem usa una costellazione a 32 punti per trasmettere 4 bit di dati e un bit di controllo per ogni simbolo a 2400 baud per ottenere un trasferimento a 9600 bps con correzione di errore. Il passo successivo a 9600 bps sono 14.400 bps, chiamato **V.32 bis** e trasmette 6 bit di dati e un bit di controllo per simbolo a 2400 baud. Poi viene il **V.34** che è in grado di ottenere 28.800 bps trasmettendo 12 bit di dati per simbolo a 2400 baud; la costellazione ha ora migliaia di punti. L'ultimo modem di questo elenco è il **V.34 bis** che usa 14 bit di dati per simbolo a 2400 baud per ottenere 33.600 bps.

Perché fermarsi qui? La ragione per cui gli standard per i modem si fermano a 33.600 deriva dal fatto che il limite di Shannon per il sistema telefonico, basandosi sulla lunghezza media dell'ultimo miglio e la qualità delle linee, è circa 35 kbps. Andare più veloce equivarrebbe a violare le leggi della fisica (in particolare, del dipartimento di termodinamica).

Tuttavia, esiste solo un modo in cui possiamo cambiare la situazione: nella centrale locale della compagnia telefonica i dati sono convertiti in digitale per essere trasmessi sulla rete telefonica (la parte centrale della rete telefonica è stata convertita da analogica a digitale molto tempo fa). Il limite di 35 kbps è per situazioni in cui si deve tenere conto dell'ultimo miglio due volte: uno per ogni capo del cavo. Ognuno dei due aggiunge rumore al segnale e, se possiamo toglierne di mezzo uno, aumenterebbe il valore di SNR e la massima velocità di invio risulterebbe raddoppiata.

Questo spiega come funzionano i modem a 56 kbps. Un lato, tipicamente un ISP, dispone di una connessione digitale ad alta qualità con la centrale locale per cui il massimo tasso di invio dati può arrivare fino a 70 kbps. Tra due utenti domestici con modem e linee analogiche il massimo è ancora 33,6 kbps.

La ragione per cui si usano modem a 56 kbps (piuttosto che a 70 kbps) ha a che fare con il teorema di Nyquist: un canale telefonico è trasportato all'interno del sistema sotto forma di campioni digitali. Ogni canale telefonico ha un'ampiezza di 4000 Hz quando la banda di guardia viene inclusa. Il numero di campioni al secondo necessari per ricostruirlo è quindi 8000. Il numero di bit per campione negli Stati Uniti è di 8, uno dei quali può essere usato per controllare la linea,

permettendo quindi ai dati una velocità di 56.000 bit/s. In Europa, tutti gli 8 bit sono disponibili per gli utenti, quindi sarebbe possibile usare modem a 64.000 bit/s, ma per ragioni di standard internazionale è stato scelto di usare 56.000.

Il risultato finale sono gli standard **V.90** e **V.92**; questi sono in grado di fornire un canale dall'ISP verso l'utente (*downstream*) di 33.6 kbps e 48 kbps nell'altra direzione (*upstream*). L'asimmetria è dovuta al fatto che generalmente ci sono più dati trasportati dall'ISP verso l'utente piuttosto che nell'altra direzione. Significa inoltre che una parte maggiore della banda in downstream può essere allocata per aumentare le possibilità di farlo funzionare effettivamente a 56 kbps (aumentando la banda disponibile ai bit per la correzione di errori).

Linee DSL

Quando raggiunse finalmente i 56 kbps, l'industria telefonica si congratulò con se stessa. Nel frattempo, però, l'industria televisiva aveva iniziato a offrire velocità che raggiungevano i 10 Mbps sui cavi condivisi. Quando l'accesso a Internet divenne una parte importante dell'attività commerciale, le aziende telefoniche (LEC) si resero conto di aver bisogno di un prodotto più competitivo. Per questo iniziarono a offrire nuovi servizi digitali sui collegamenti locali.

Inizialmente ci furono molte offerte sovrapposte, tutte raccolte sotto un unico nome: **xDSL** (*digital subscriber line*), con vari possibili valori di *x*. Servizi con più banda di quelli telefonici standard vengono tipicamente definiti **broadband** (*a banda larga*) anche se la scelta del termine è più a fini di marketing che tecnologici. Più tardi vedremo quello che è diventato il più popolare di questi servizi: **ADSL** (*asymmetric DSL*). Useremo inoltre il termine DSL (o xDSL) come abbreviazione per tutte le varietà.

I modem sono così lenti perché i telefoni sono stati inventati per trasmettere la voce umana e l'intero sistema è stato attentamente ottimizzato per questo scopo; i dati sono sempre stati considerati come degli intrusi. Nel punto in cui ogni collegamento locale termina nella centrale locale, il cavo attraversa un filtro che attenua tutte le frequenze sotto i 300 Hz e sopra i 3.400 Hz. Il limite non è netto (300 Hz e 3.400 Hz sono i punti a 3 dB) perciò l'ampiezza di banda si considera solitamente 4.000 Hz anche se la distanza tra i punti a 3 dB è di 3.100 Hz. I dati sono perciò limitati a questa banda.

Il trucco che fa funzionare xDLS è che, quando un cliente si abbona a questo servizio, la linea in ingresso viene collegata a un diverso tipo di commutatore che, non usando questo filtro, rende disponibile l'intera capacità del collegamento locale. In questo modo il fattore limitante è diventato l'insieme di proprietà fisiche del collegamento locale che supporta grosso modo 1 MHz, non l'ampiezza di banda artificiale 3.100 Hz creata dal filtro.

Purtroppo, la capacità del collegamento locale dipende anche da altri fattori, inclusi quello della sua lunghezza, il diametro dei cavi e la qualità generale. La Figura 2.33 fornisce un quadro d'insieme dell'ampiezza di banda potenziale come funzione della distanza; la figura presuppone che tutti gli altri fattori siano ottimali (cavi nuovi, fasci non troppo grandi e così via).

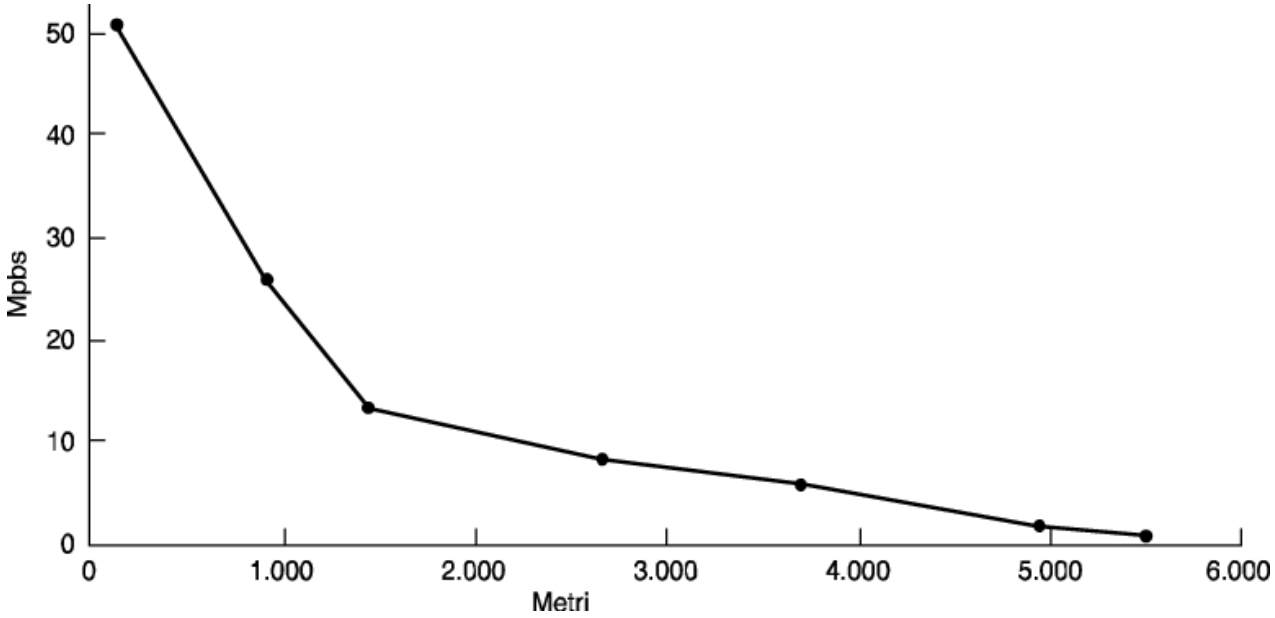


Figura 2.33 Ampiezza di banda e distanza per UTP di categoria 3 usati da DSL.

L'implicazione di questa figura crea un problema per l'azienda telefonica: la portata massima (distanza dalle centrali locali) del servizio dipende dalla velocità che si sceglie di offrire. Questo significa che gli utenti più distanti che tentano di abbonarsi al servizio potrebbero sentirsi dire: "La ringraziamo per l'interesse dimostrato, ma purtroppo la sua abitazione dista 100 metri di troppo dalla più vicina centrale locale". Più è bassa la velocità della connessione, più è ampio il raggio del servizio e maggiore è il numero di clienti raggiunti; ma al diminuire della velocità, diventa meno allettante il servizio e diminuisce il numero di utenti disposti a pagare per ottenerlo. È qui che il mondo degli affari incontra la tecnologia.

I servizi xDSL sono stati progettati tenendo conto di alcuni obiettivi. Primo, i servizi devono funzionare sui doppini di categoria 3 esistenti adoperati per i collegamenti locali; secondo, non devono influire sugli apparecchi telefonici e sui fax in dotazione degli utenti; terzo, devono essere molto più veloci della connessione a 56 kbps; quarto, dovrebbero sempre essere attivi e avere un costo mensile non legato al tempo di utilizzo.

Per venire incontro ai requisiti tecnologici, lo spetto disponibile di 1,1 MHz sull'ultimo miglio è diviso in 256 canali indipendenti di 4312,5 Hz ciascuno. Questa disposizione è rappresentata nella Figura 2.34. La tecnica OFDM, vista nel paragrafo precedente, è usata per inviare dati su questi canali, nonostante in tale contesto venga spesso chiamata **DMT** (*discrete multitone*). Il canale 0 è usato per il **POTS** (*plain old telephone service*, semplice e vecchio servizio telefonico); i canali da 1 a 5 non sono utilizzati per evitare che i segnali di voce e dati interferiscano tra loro e dei rimanenti 250 canali uno è usato per controllare il canale in upstream e uno per controllare quello in downstream. I canali rimanenti sono disponibili per trasportate i dati degli utenti.

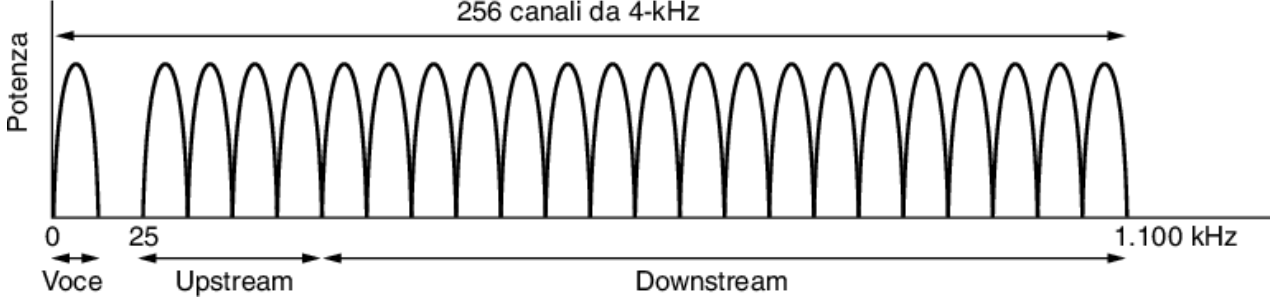


Figura 2.34 ADSL che sfrutta DMT.

In linea di principio, ognuno dei canali rimanenti può essere utilizzato per un flusso di dati full duplex, ma le armoniche, l'interferenza e altri effetti tengono i sistemi reali molto al di sotto del limite teorico. Il fornitore del servizio decide quanti canali utilizzare per la trasmissione e quanti per la ricezione. Una combinazione 50 e 50 è tecnicamente possibile, ma la maggior parte dei fornitori assegna al canale utilizzato per scaricare i dati l'80-90% dell'ampiezza di banda, poiché la maggior parte degli utenti preleva più di quanto trasmette. Questa soluzione fa nascere la "A" (Asimmetrica) di ADSL. Di solito 32 canali sono dedicati alla trasmissione e il resto alla ricezione. È possibile aumentare l'ampiezza di banda rendendo bidirezionali alcuni dei canali più alti in trasmissione, tuttavia questa ottimizzazione richiede l'aggiunta di uno speciale circuito per annullare l'effetto eco.

Lo standard internazione per ADSL, noto come **G.dtm**, è stato approvato nel 1999 ed è in grado di raggiungere velocità fino a 8 Mbps in downstream e 1 Mbps in upstream. È stato sostituito nel 2002 da una seconda generazione chiamata ADSL2 con molti miglioramenti in grado di portare la velocità fino a 12 Mbps in downstream e 1 Mbps in upstream. Ora disponiamo di ADSL2+, in grado di raddoppiare la velocità in downstream fino a 24 Mbps tramite il raddoppiamento a 2,2 Mhz della banda utilizzabile sul doppino.

Tuttavia, i numeri riportati qui corrispondono alla velocità ottimale per linee nelle migliori condizioni e vicine (da 1 a 2 Km) al centralino. Poche linee supportano queste velocità e pochi provider le offrono. Tipicamente, i provider forniscono come servizio standard qualcosa come 1 Mbps downstream e 256 kbps upstream, 4 Mbps downstream e 1 Mbps upstream come servizio migliorato e 8 Mbps downstream e 2 Mbps upstream come servizio di eccellenza.[3]
All'interno di ogni canale la modulazione QAM è usata con un tasso di 4000 simboli al secondo e la qualità della linea viene costantemente monitorata in modo da aggiustare la velocità di trasmissione usando costellazioni con più o meno punti, come quelle nella Figura 2.23. Canali distinti potrebbero avere velocità diverse per l'invio dei dati fino a 15 bit per simbolo inviato su un canale con un alto SNR, e giù fino a due, uno o nessun bit per simbolo su un canale con basso SNR a seconda dallo standard.

La Figura 2.35 mostra una tipica configurazione ADSL. In questo schema, un tecnico della società telefonica deve installare nell'edificio del cliente un **NID** (*network interface device*), una piccola scatola di plastica che segna la fine della proprietà dell'azienda telefonica e l'inizio della proprietà del cliente. Accanto al NID (e qualche volta al suo interno) si trova uno **splitter**, un filtro analogico che divide i dati dalla banda 0-4.000 Hz utilizzata da POTS. Il segnale POTS è inviato all'apparecchio telefonico o al fax esistente, mentre i dati sono instradati verso un modem ADSL,

che elabora i segnali e implementa OFDM. Siccome molti modem ADSL sono esterni, il computer deve essere connesso a questi ultimi ad alta velocità; in genere ciò avviene facendo uso di Ethernet, un cavo USB o 802.11.

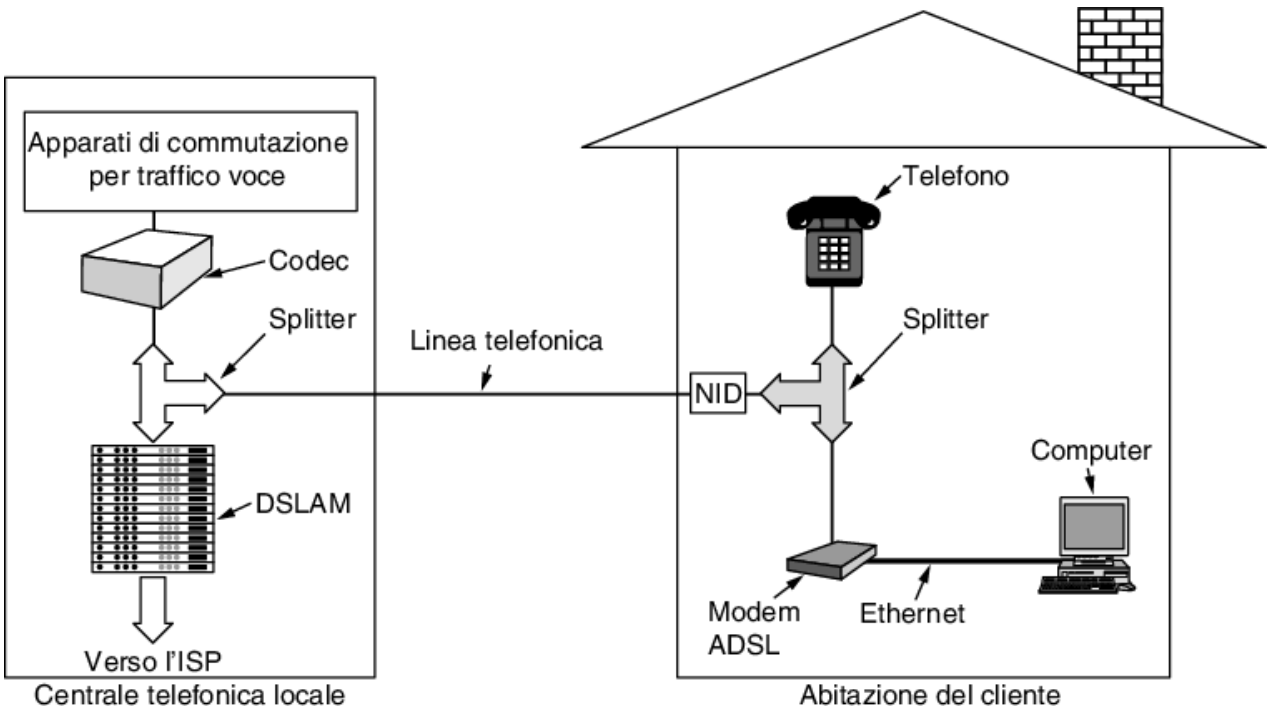


Figura 2.35 Una configurazione ADSL tipica.

All'altra estremità del cavo, nella centrale locale, deve essere installato un analogo splitter. Qui il segnale vocale è filtrato e inviato al normale commutatore per il traffico vocale. Il segnale sopra i 26 kHz, invece, è passato a un nuovo tipo di dispositivo chiamato **DSLAM** (*digital subscriber line access multiplexer*) che contiene lo stesso tipo di processore di segnale digitale integrato nel modem ADSL. Dopo essere stato riorganizzato in un flusso di bit, il segnale digitale è prima organizzato in pacchetti e poi inviato all'ISP.

Questa completa separazione tra sistema vocale e ADSL facilita all'azienda telefonica l'allestimento del servizio ADSL: è sufficiente acquistare i dispositivi DSLAM, gli splitter e collegare gli abbonati ADSL a quest'ultimi. Altri servizi con grande ampiezza di banda (per esempio ISDN) richiedono un maggior numero di modifiche agli apparati esistenti.

Uno svantaggio della configurazione mostrata nella Figura 2.35 è la presenza del NID e dello splitter nell'edificio del cliente. Poiché questi componenti sono installabili solo da tecnici dell'azienda telefonica, è necessario organizzare e pagare un intervento in loco. Per questo è stato creato uno standard alternativo basato su una configurazione che non usa splitter e chiamato **G.lite**. La configurazione è identica a quella mostrata nella Figura 2.35, ma non ha lo splitter. La linea telefonica esistente è utilizzata così com'è; l'unica differenza è che tra ogni presa telefonica e il telefono o modem collegato si deve interporre un microfiltro.

Il filtro per il telefono è un filtro passa-basso che elimina le frequenze sopra i 3.400 Hz; il filtro per il modem ADSL è un passa-alto che elimina le frequenze sotto i 26 kHz. Tuttavia questo sistema non è affidabile come quello basato sullo splitter, perciò G.lite può essere utilizzato solo fino a 1,5 Mbps (mentre la configurazione ADSL con splitter arriva a 8 Mbps). Per ulteriori informazioni su ADSL si veda Starr (2003).

FTTH – Fiber to the home

L'ultimo miglio in rame attualmente in uso limita le prestazioni di ADSL e modem telefonici. Per poter fornire servizi di rete migliori e più veloci le compagnie telefoniche stanno sostituendo alla prima opportunità il rame con fibre ottiche che arrivino fino all'abitazione o nell'ufficio. Il risultato viene chiamato **FttH** (*fiber to the home*, "fibra fino all'abitazione").[4] Sebbene la tecnologia FttH sia disponibile da lungo tempo, l'effettiva stesura dei cavi è cominciata in maniera significativa solo nel 2005 con la crescita della domanda di accesso a Internet ad alta velocità da parte di utenti DSL che volevano scaricare film. Circa il 4% delle abitazioni americane sono ora connesse a Internet con FttH e usufruiscono di una velocità di accesso fino a 100 Mbps.
Esistono molte varianti nella forma "FttX" (dove *X* sta per cantina, marciapiede o quartiere), usate per denotare quanto vicino all'abitazione può giungere la fibra ottica. In questo caso il rame (doppino o cavo coassiale) fornisce velocità sufficiente su breve distanza; la scelta di quanto tenere distante la fibra è puramente economica per bilanciare i costi rispetto ai guadagni prospettati. In ogni caso, il punto è che le fibre ottiche hanno superato la barriera dell'ultimo miglio.

Così come accade per il cavo in rame, l'ultimo miglio in fibra è passivo: questo vuol dire che non è necessario disporre di apparati attivi (che consumano energia) per amplificare o elaborare il segnale. La fibra semplicemente porta i segnali dall'abitazione alla centrale locale con conseguente riduzione di costi e aumento di affidabilità.

Tipicamente, le fibre provenienti dalle case sono così unite tra loro da ottenere che una sola fibra proveniente da un gruppo anche di 100 case raggiunga la centrale locale. Andando verso gli utilizzatori, alcuni separatori ottici (*splitter*) dividono il segnale proveniente dalla centrale locale per permettere a esso di raggiungere tutte le abitazioni; se anche solo un'abitazione dovesse essere in grado di decodificare il segnale, la crittografia sarebbe necessaria per fini di sicurezza. Nella direzione opposta alcuni combinatori ottici uniscono i segnali provenienti dalle case in maniera tale che solo un singolo segnale sia poi ricevuto dalla centrale locale.

Questa architettura prende il nome di **PON** (*passive optical network*, rete ottica passiva) ed è rappresenta nella Figura 2.36; è prassi comune usare una sola lunghezza d'onda condivisa tra tutte le abitazioni per la trasmissione in downstream e un'altra lunghezza d'onda per le trasmissioni in senso inverso.

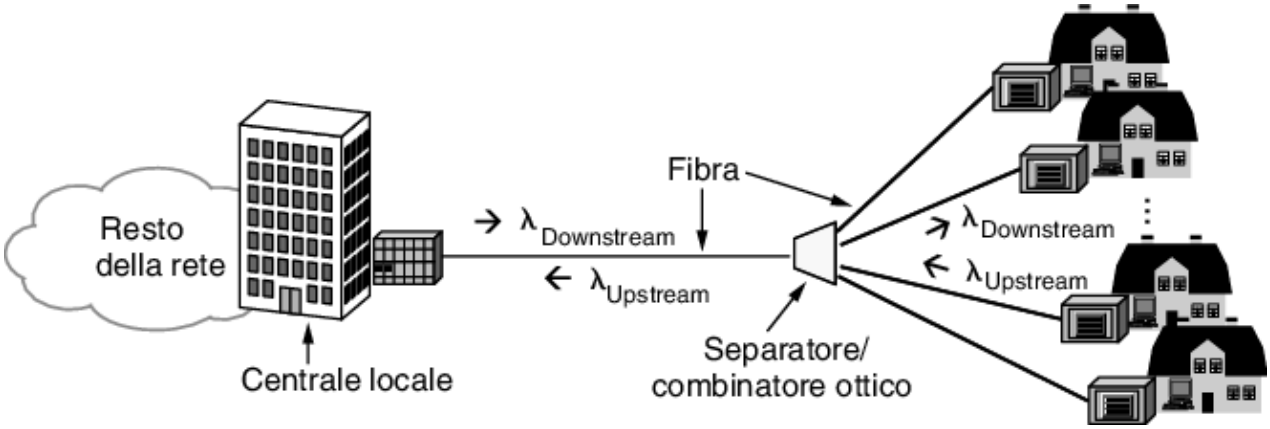


Figura 2.36 FttH con fibra ottica passiva.

Anche con la suddivisione, la larga banda e la bassa attenuazione delle fibre ottiche permettono alle PON di fornire agli utenti grandi velocità su distanze fino a 20 Km. La reale velocità di trasmissione dei dati e altri dettagli dipendono dal tipo di PON; i due tipi più noti sono **GPON** (*gigabit-capable PON*), proveniente dal mondo delle telecomunicazioni e standardizzata dall'ITU ed **EPON** (*Ethernet PON*), più in sintonia con il mondo delle reti e regolamentate da standard IEEE. Entrambe vanno circa alla velocità di un gigabit e sono in grado di trasportare traffico per molteplici servizi compresi Internet, video e voce. Per fare un esempio, le GPON possono fornire 2,4 Gbps in downstream e 1,2 o 2,4 Gbps in upstream.

Per far condividere alle applicazioni la capacità della singola fibra che arriva alla centrale locale è necessario un protocollo. Andando verso la periferia non ci sono problemi: la centrale locale può mandare messaggi singolarmente a ogni casa nell'ordine che preferisce. Nella direzione opposta, invece, messaggi provenienti dalle abitazioni non possono essere mandati contemporaneamente, altrimenti potrebbero collidere; le abitazioni, inoltre, non possono ascoltare a vicenda le proprie trasmissioni, per cui non possono controllare prima di effettuare la trasmissione. La soluzione è che l'apparato in casa concordi con quello della centrale locale gli intervalli di tempo da utilizzare per trasmettere; a questo scopo si procede per approssimazioni successive tarando il tempo di trasmissione in modo tale che tutti i segnali ricevuti dalla centrale locale siano sincronizzati. Il funzionamento è simile a quello dei cable model di cui ci si occuperà più avanti in questo capitolo. Per maggiori informazioni sul futuro delle POS si veda Grobe ed Elbers (2008).

Trunk e multiplexing

Con il termine **trunk** si indicano i segmenti principali di una rete telefonica: quelli destinati a trasportare la maggior parte del traffico e a collegare tra loro i principali centri di commutazione. Essi non solo offrono una velocità molto maggiore dell'ultimo miglio, ma differiscono da quest'ultimo per due altri aspetti. Primo, la parte interna della rete trasporta informazioni digitali e non analogiche (cioè bit e non voce) e quindi è necessaria una conversione alla centrale locale per trasmettere sui trunk a lunga distanza. Secondo, i trunk trasportano migliaia se non milioni di chiamate simultaneamente; questa condivisione di risorse, generalmente ottenuta con TDM e

FDM, è importante per ottenere economie di scala, in quanto il costo di installazione e manutenzione di un collegamento ad alta capacità tra due centrali di commutazione è lo stesso di uno a bassa capacità. Ciò si ottiene tramite multiplexing TDM e FDM.

Di seguito esamineremo velocemente come i segnali vocali vengano digitalizzati in modo tale da poter essere distribuiti da una rete telefonica. Quindi vedremo come TDM venga utilizzata sui trunk, in particolare nel caso delle fibre ottiche (SONET). Infine, ci occuperemo di come FDM venga usata sulle fibre ottiche, tecnologia chiamata **WDM** (*wavelength division multiplexing*, multiplexing a divisione di lunghezza d'onda).

Digitalizzazione di segnali vocali

All'inizio dello sviluppo delle reti telefoniche la rete gestiva le chiamate vocali come se fossero composte da informazioni analogiche. Le tecniche FDM sono state usate per molti anni per fare multiplexing di canali voce da 4000 Hz (3100 Hz più bande di guardia) in flussi via via più grandi. Ad esempio, 12 chiamate nella banda che va da 60 kHz a 108 kHz sono note come un **gruppo**, e cinque gruppi (per un totale di 60 chiamate) prendono il nome di **supergruppo**. FDM viene ancora utilizzata per i collegamenti in rame e le microonde, tuttavia FDM necessitando di apparecchiature analogiche non può essere gestita con un computer. Diversamente, TDM è gestibile interamente da apparecchiature elettroniche ed è diventata molto più diffusa in questi ultimi anni. Poiché TDM può essere usata solo per dati digitali e l'ultimo miglio produce segnali analogici, è necessario fare una conversione nella centrale locale, dove i segnali vengono raggruppati per essere spediti lungo i trunk.

I segnali analogici sono digitalizzati da un dispositivo chiamato **codec** (abbreviazione di co*der*-dec*oder*, codificatore-decodificatore). Il codec estrae 8000 campioni al secondo (uno ogni 125 ms) perché secondo il teorema di Nyquist questi sono sufficienti a catturare tutta l'informazione da un canale telefonico a 4 kHz. A un tasso di campionamento più basso l'informazione andrebbe persa, a uno più alto non sarebbe ottenuta alcuna informazione aggiuntiva. Ogni campione dell'ampiezza del segnale è quantizzato in un numero di 8 bit.

Questa tecnica prende il nome di **PCM** (*pulse code modulation*, modulazione codificata di impulsi) ed è il cuore dei sistemi telefonici moderni. Di conseguenza, quasi tutti gli intervalli di tempo all'interno di un sistema telefonico sono multipli di 125 ms. La velocità di trasmissione per una trasmissione vocale standard non compressa è quindi di 8 bit ogni 125 ms, ovvero 60 kbps.

All'altro capo della linea viene ricreato un segnale analogico dai campioni quantizzati riproducendoli e smussandoli nel tempo; a causa della quantizzazione dei segnali il risultato non sarà esattamente identico al segnale analogico originale, anche se si è campionato secondo il teorema Nyquist. Per ridurre l'errore, i livelli di quantizzazione sono distanziati tra loro in maniera non uniforme: viene infatti usata una scala logaritmica che destina più bit ai segnali con bassa ampiezza e meno a quelli con alta; in questo modo l'errore è proporzionale all'ampiezza del segnale.

I principali tipi di quantizzazione utilizzati sono due: m**-law**, usato in Nord America e in Giappone, e **A-law**, usato in Europa e nel resto del mondo; entrambe le versioni sono descritte nello standard ITU G.711. Un modo equivalente di pensare a questo processo è immaginarsi che l'ampiezza variabile di un segnale (o il rapporto tra i valori massimo e minimo) venga compressa

prima di essere (uniformemente) quantizzata e poi espansa quando il segnale analogico viene ricreato. Per questa ragione il processo viene chiamato **companding** (*compression and expanding*). È anche possibile comprimere i campioni dopo che essi sono stati digitalizzati in maniera tale che richiedano molto meno di 60 kbps. A ogni modo riprenderemo questo argomento quando esploreremo le applicazioni audio, come il **VoIP**.

Time division multiplexing

TDM basata su PCM è usata per trasportare chiamate vocali su trunk spedendo un campione ogni 125 ms. Quando le trasmissioni digitali cominciarono a dimostrarsi una tecnologia affidabile, ITU (allora chiamata CCITT) non fu in grado di trovare un accordo per uno standard internazionale per PCM; di conseguenza nei vari paesi del mondo vengono ora utilizzate una serie di modalità incompatibili tra loro.

Il metodo impiegato in Nord America e in Giappone è la portante **T1** descritta nella Figura 2.37 (tecnicamente parlando, il formato è chiamato DS1 e la portante è chiamata T1, ma seguendo una tradizione industriale qui non faremo questa sottile distinzione). La portante T1 è composta da 24 canali vocali uniti in multiplexing. Ognuno dei 24 canali, a turno, inserisce 8 bit nel flusso in uscita.

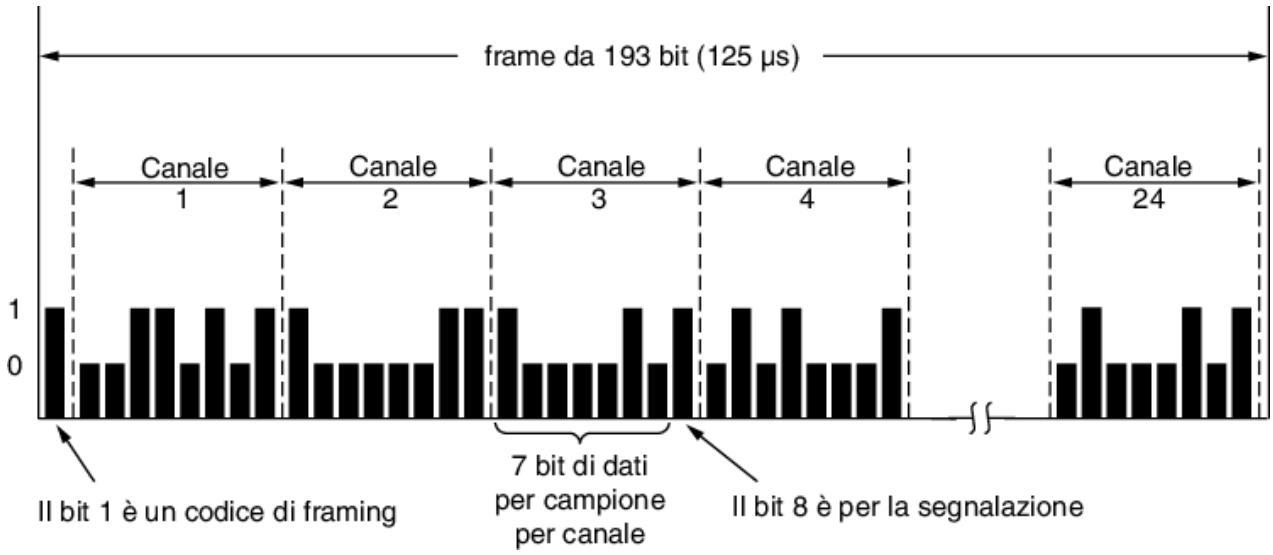


Figura 2.37 La portante T1 (1,544 Mbps).

Un frame è composto da 24 × 8 = 192 bit più un bit aggiuntivo utilizzato per il controllo, per un totale di 193 bit ogni 125 ms. Questo significa che la velocità dati finale è di circa 1,544 Mbps, di cui 8 kbps per le segnalazioni di controllo della comunicazione. Il 193-esimo bit è usato per la sincronizzazione dei frame e anche per le segnalazioni. In una delle varianti il 193-esimo bit è usato trasversalmente su un gruppo di 24 frame che prendono il nome di **extended superframe**. Sei dei 139-esimi bit nei frame in posizione numero 4, 8, 12, 16, 20 e 24 assumono la sequenza

"001011"; il ricevente tiene traccia di questa sequenza per assicurarsi di non aver perso la sincronizzazione. Altri sei bit vengono invece utilizzati per spedire un codice di controllo d'errore in modo da aiutare il ricevente a confermare che è sincronizzato. Se il ricevente finisce fuori sincronia, per acquisirla nuovamente può scandire i frame fino a che non riconosce la sequenza di bit e quindi può validare il codice di controllo. I rimanenti 12 bit sparsi nei 24 frame sono usati come bit di controllo per la gestione della rete, ad esempio per un rendiconto delle prestazioni.

Il formato T1 ha molte varianti. Le prime versioni spedivano le informazioni di controllo **in-band**, cioè con lo stesso canale usato per i dati. Questo approccio è una forma di **channel-associated signaling** (*segnalazione associata al canale*) in quanto ogni canale ha il suo sotto-canale di segnalazione privato. In un caso il bit meno significativo su un campione di 8 bit su ogni canale viene utilizzato ogni sei frame; questo prende il curioso nome di **robbed-bit signaling** (*segnalazione con bit rubato*); l'idea alla base è che rubare pochi bit non è significativo per la voce: nessuno si accorgerà della differenza.

Per i dati, invece, il discorso cambia: consegnare bit sbagliati non è utile (per usare un eufemismo). Se vecchie versioni di T1 vengono usate per trasportare dati, solo 7 bit su 8 (quindi 56 kbps) possono essere usati su ognuno del 24 canali. Invece, versioni più recenti di T1 forniscono canali in cui tutti i bit possono essere utilizzati per inviare dati; tali canali sono definiti **clear channel** (*canali in chiaro*). Un canale in chiaro è quello che richiedono le aziende che affittano una linea T1 nel momento in cui spediscono dati e non campioni vocali attraverso la rete telefonica. La segnalazione per le chiamate vocali viene quindi fatta **out-of-band**, ovvero in un canale separato dai dati. Tipicamente la segnalazione avviene usando **common-channel signaling** (*segnalazione a canale comune*), in cui c'è un singolo canale condiviso per tutto il traffico di segnalazione; questo canale può essere uno dei 24 disponibili.

Al di fuori del Nord America e del Giappone al posto della T1 viene usata la portante **E1** da 2,048 Mbps. Questa portante incapsula all'interno del frame da 125 ms 32 campioni da 8 bit. Trenta dei canali sono usati per trasportare informazioni e fino a due sono destinati per le segnalazioni. Ogni gruppo di 4 frame mette a disposizione 64 bit di segnalazione di cui metà usati per il controllo del canale (sia esso channel-associated o common-channel) e l'altra metà usati per la sincronizzazione dei frame o riservati a uso locale secondo la regolamentazione del singolo paese.

Il multiplexing a divisione di tempo permette di unire diverse portanti T1 in portanti di ordine più elevato. La Figura 2.38 mostra come avviene tutto ciò: a sinistra appaiono quattro canali T1 uniti in un canale T2; l'unione dei flussi è eseguita bit per bit, e non byte per byte tra i 24 canali vocali che compongono un frame T1. Quattro flussi T1 a 1,544 Mbps dovrebbero generare 6,176 Mbps, ma T2 in realtà funziona a 6,312 Mbps. I bit aggiuntivi sono utilizzati per le operazioni di gestione della struttura e quelle di ripristino in caso che la portante vada fuori sincronia. T1 e T3 sono molto utilizzate dagli utenti, mentre T2 e T4 sono utilizzate solo all'interno del sistema telefonico e pertanto sono poco note.

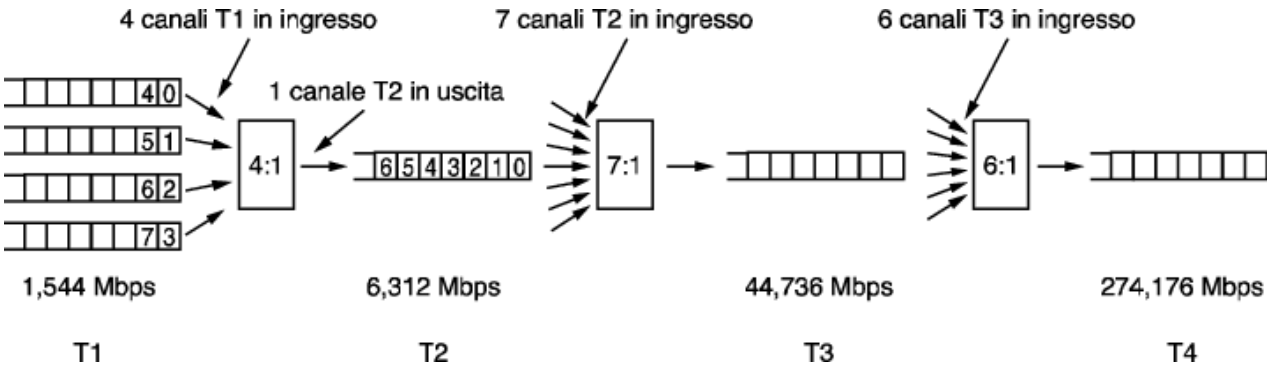


Figura 2.38 Unione di flussi T1 in portanti di ordine superiore.

Al livello successivo, sette flussi T2 sono uniti bit per bit in modo da formare un flusso T3. Poi sei flussi T3 sono uniti per formare un flusso T4. In caso d'interruzione della sincronizzazione tra il trasmittente e il ricevente, a ogni passaggio aumenta leggermente il carico di lavoro richiesto per gestire le operazioni di framing e ripristino.

Così come sulla portante di base c'è poca intesa tra gli Stati Uniti e il resto del mondo, ce n'è altrettanto poca anche rispetto al modo in cui la portante deve essere unita in multiplexing per formare portanti di ampiezza di banda maggiore. Lo schema statunitense basato su variazioni di 4, 7 e 6 non ha colpito favorevolmente tutti gli altri paesi, così lo standard ITU impone un multiplexing di quattro canali per flusso per ogni livello. Anche i dati per la gestione dei frame e le operazioni di ripristino sono diversi. La gerarchia ITU definisce 32, 128, 512, 2.048 e 8.192 canali che operano a velocità rispettivamente di 2,048, 8,848, 34,304, 139,264 e 565,148 Mbps.

SONET/SDH

Ai primi esordi della fibra ottica ogni azienda telefonica adottava un sistema TDM ottico proprietario. Dopo lo scioglimento di AT&T avvenuto nel 1984, le aziende telefoniche locali dovevano collegarsi con differenti portanti a lunga distanza, ognuna delle quali adoperava il proprio sistema TDM ottico; divenne ovvia la necessità di definire uno standard. Nel 1985, Bellcore, il centro di ricerca RBOC, iniziò a lavorare su uno standard chiamato **SONET** (*synchronous optical network*).

Successivamente si unì anche ITU. Alla fine, nel 1989, si ottenne uno standard SONET affiancato da una serie di raccomandazioni CCITT (G.707, G.708 e G.709). Le raccomandazioni ITU sono chiamate **SDH** (*synchronous digital hierarchy*), ma differiscono da SONET solo in minima parte. Oggi virtualmente tutto il traffico telefonico su lunghe distanze negli Stati Uniti e in quasi tutto il resto del mondo utilizza linee che adoperano SONET a livello fisico. Per maggiori informazioni su SONET, si veda Bellamy (2000), Goralski, (2002) e Shepard, (2001).

L'architettura SONET doveva raggiungere i seguenti quattro obiettivi principali.

1. L'obiettivo più importante era il dover consentire la comunicazione tra portanti diverse. Per raggiungere tale scopo è stato necessario definire uno standard della segnalazione comune su lunghezza d'onda, delle temporizzazioni, della struttura dei frame e così via.

2. Servivano strumenti per l'unificazione dei sistemi digitali adottati da Stati Uniti, Giappone ed Europa; tutti i sistemi si basavano su canali PCM a 64 kbps, ma ognuno combinava quei canali in modo diverso (e incompatibile).
3. SONET doveva fornire un mezzo per unire in multiplexing diversi canali digitali. Quando fu ideata SONET la portante digitale più veloce utilizzata ampiamente negli Stati Uniti era T3, a 44,736 Mbps; T4 era stata definita, ma non molto utilizzata e nulla era stato stabilito in relazione alla velocità T4. Parte della missione di SONET fu portare quella gerarchia a velocità misurabili in Gbps. Era anche necessario trovare un modo standard di unire in multiplexing tanti canali lenti in un unico canale SONET.
4. SONET doveva fornire supporto OAM (*operation, administration and manteinance,* operatività, amministrazione e manutenzione). I precedenti sistemi non erano infatti soddisfacenti.

Una delle prime decisioni fu di rendere SONET un sistema TDM tradizionale, con l'intera ampiezza di banda della fibra dedicata a un canale contenente slot temporali per vari sottocanali. Di per sé, SONET è un sistema sincrono; è controllato da un orologio principale che ha una precisione di circa 1 su 10^9. I bit sulla linea SONET sono inviati a intervalli precisi, controllati dall'orologio principale (*master clock*).

Il frame SONET di base è un blocco di 810 byte emesso ogni 125 ms. Poiché SONET è sincrona, i frame sono emessi anche in assenza di dati utili da trasmettere. Gli 8000 frame/s disponibili coincidono perfettamente con la velocità di campionamento dei canali PCM utilizzati in tutti i sistemi telefonici digitali.

I frame SONET a 810 byte si possono immaginare come casellari di byte larghi 90 colonne e alti 9 righe. Perciò 8 × 810 = 6.480 bit sono trasmessi 8.000 volte al secondo, per una velocità totale di circa 51,84 Mbps. Questo è il canale SONET di base, chiamato anche **STS-1** (*synchronous transport signal-1*). Tutte le linee SONET sono multipli di STS-1.

Le prime tre colonne di ogni frame sono riservate alle informazioni di gestione del sistema (Figura 2.39); all'interno di questo blocco le prime tre righe contengono l'overhead di sezione; le successive sei contengono l'overhead di linea. L'overhead di sezione è generato e controllato all'inizio e alla fine di ogni sezione, mentre l'overhead di linea è generato e controllato all'inizio e alla fine di ogni linea.

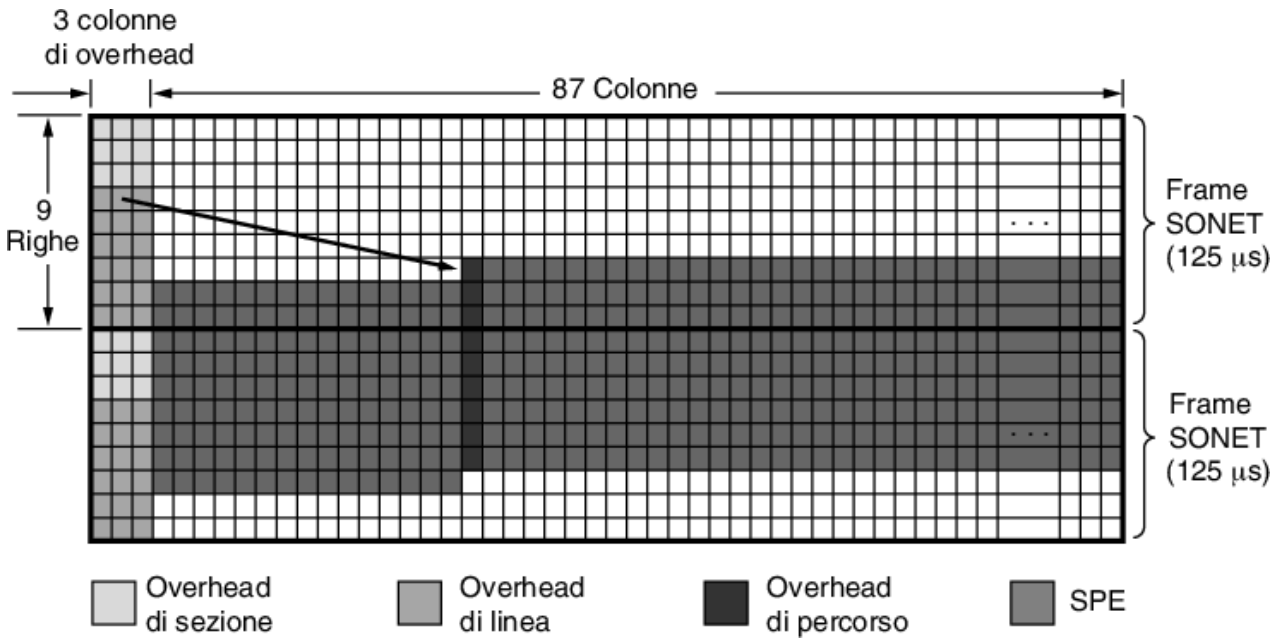


Figura 2.39 Due frame SONET consecutivi.

Un trasmettitore SONET invia frame a 810 byte addossati, senza creare alcuna interruzione tra frame consecutivi anche in assenza di dati utili (in questo caso il dispositivo invia dati fittizi). Poiché i bit formano un flusso continuo, come riesce il ricevitore a individuare il punto iniziale e finale di ogni frame? La risposta è semplice: i primi due byte di ogni frame contengono uno schema fisso e il ricevitore cerca questi elementi per determinare il punto iniziale della struttura. Se trova questo schema nello stesso posto in un gran numero di frame consecutivi, il ricevitore presume di essere in sincronia con il trasmettitore. In teoria un utente potrebbe inserire questo schema in modo regolare nel payload, ma nella pratica non è possibile eseguire questa operazione a causa dell'unione in multiplexing di più utenti nello stesso frame (e anche per altri motivi).

Le rimanenti 87 colonne contengono 87 × 9 × 8 × 8000 = 50,112 Mbps di dati degli utenti. I dati utente potrebbero essere campioni vocali, T1 o altre portanti inglobate completamente o pacchetti; SONET è semplicemente un comodo contenitore per trasferire bit. Questi dati, chiamati anche **SPE** (*synchronous payload envelope*), non sempre iniziano alle coordinate riga 1, colonna 4; gli SPE possono iniziare in qualunque punto all'interno del frame. Un puntatore associato al primo byte si trova nella prima riga del codice di controllo della linea. La prima colonna degli SPE rappresenta l'overhead di percorso.

Due proprietà dei frame SONET rendono il sistema ancora più flessibile: la capacità di iniziare gli SPE in qualunque punto all'interno del frame e la possibilità di estendere i dati su due frame (Figura 2.39). Per esempio, se arrivano dei dati da spedire mentre sta costruendo un frame fittizio, la sorgente può inserire i dati nel frame corrente invece di attendere il frame successivo.

La Figura 2.40 mostra la gerarchia di multiplexing adottata da SONET/SDH. Sono state definite velocità da STS-1 a STS-768, che variano grosso modo da una linea T3 a 40 Gbps. Velocità di trasmissione ancora più alte saranno sicuramente definite in futuri standard quali ad esempio l'OC-3072 a 160 Gbps, che sarà la prossima generazione non appena diverrà tecnologicamente fattibile.

La portante ottica corrispondente a STS-*n* è chiamata OC-*n* ed è del tutto identica, a eccezione di un particolare riordinamento di bit utilizzato per la sincronizzazione. I nomi SDH sono diversi e iniziano da OC-3 perché i sistemi basati su ITU non hanno velocità prossime a 51,84 Mbps. Abbiamo mostrato le velocità più comuni, che partendo da OC-3 progrediscono a multipli di quattro. La velocità di trasmissione dati globale include anche tutto l'overhead, mentre SPE esclude l'overhead di linea e sezione e la velocità di trasferimento a livello utente esclude tutto l'overhead e conta solo le 87 colonne per il trasferimento dati.

SONET		SDH	Capacità (Mbps)		
Elettrico	**Ottico**	**Ottico**	**Grezza**	**SPE**	**Utente**
STS-1	OC-1		51.84	50.112	49.536
STS-3	OC-3	STM-1	155.52	150.336	148.608
STS-12	OC-12	STM-4	622.08	601.344	594.432
STS-48	OC-48	STM-16	2488.32	2405.376	2377.728
STS-192	OC-192	STM-64	9953.28	9621.504	9510.912
STS-768	OC-768	STM-256	39.813,12	38.486,016	38.043,648

Figura 2.40 Tasso di multiplazione per SONET e SDH.

Tanto per curiosità, quando una portante, per esempio OC-3, non è unita in multiplexing ma trasporta i dati da una singola sorgente, la lettera *c* (concatenateci) viene aggiunta alla designazione; così OC-3 indica una portante a 155,52 Mbps composta da tre portanti OC-1 separate, ma OC-3c indica un flusso di 155,52 Mbps di dati generati da una singola sorgente. I tre flussi OC-1 dentro il flusso OC-3c si alternano per colonna: prima colonna 1 di flusso 1, poi colonna 1 di flusso 2, quindi colonna 1 di flusso 3 seguito da colonna 2 di flusso 1 e così via, fino a riempire 270 colonne per 9 righe.

WDM – Wavelength Division Multiplexing

Una forma di multiplexing a divisione di frequenza chiamata **WDM** (*wavelength division multiplexing*, multiplexing a divisione di lunghezza d'onda) viene usata, come pure TDM, per gestire la larghissima banda offerta da canali in fibra ottica. Il principio alla base di WDM sulle fibre ottiche è illustrato nella Figura 2.41, dove quattro fibre giungono a un combinatore ottico ognuna con la sua energia presente a una diversa lunghezza d'onda. I quattro raggi luminosi vengono combinati in un'unica fibra condivisa per essere trasmessi a destinazione. All'altro capo il raggio viene scomposto su tante fibre quante erano quelle che trasportavano il segnale in ingresso.

Ogni fibra in uscita contiene un nucleo costruito appositamente che filtra tutte le lunghezze d'onda a parte quella desiderata. I segnali ottenuti possono essere instradati a destinazione o ricombinati in vari modi per essere inviati di nuovo tramite multiplexing.

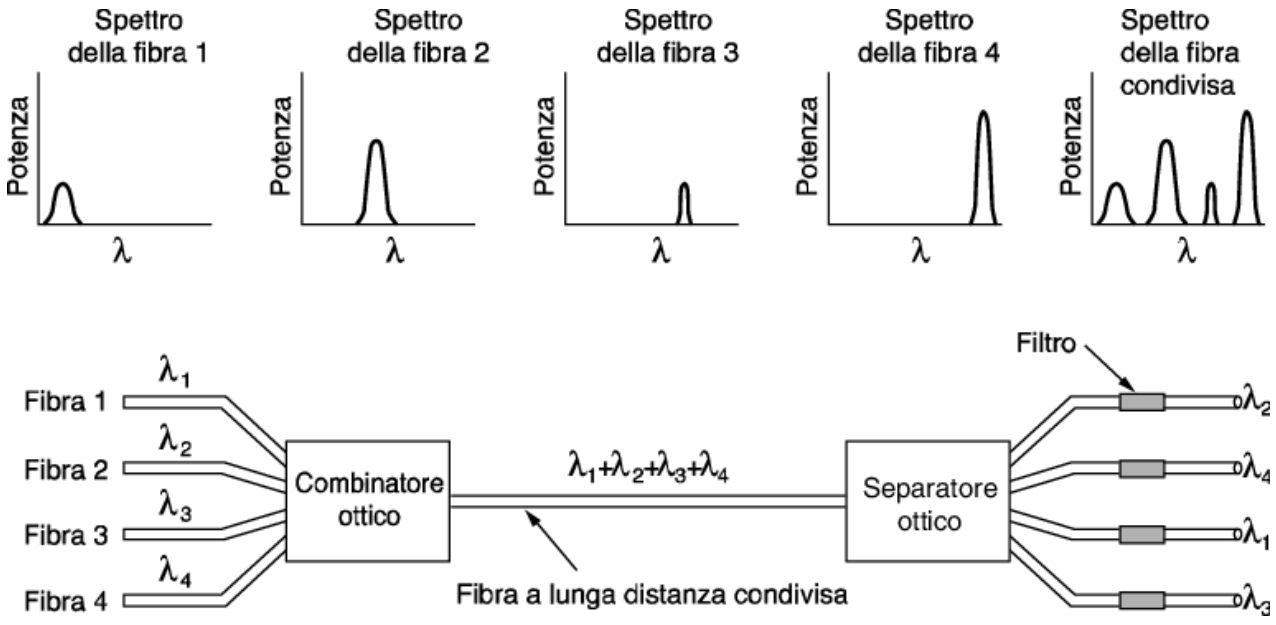


Figura 2.41 Multiplexing a divisione di lunghezza d'onda.

In realtà non si tratta di niente di nuovo; questo modo di procedere è solo FDM applicato a frequenze molto alte; il termine WDM si deve alla descrizione dei canali su fibra ottica in base alla loro lunghezza d'onda (o colore) piuttosto che alla frequenza. Fintanto che ogni canale ha un suo insieme di frequenze (o lunghezze d'onda) e tutti questi insiemi sono disgiunti, i canali possono essere mandati congiuntamente sulla fibra tramite multiplexing. La sola differenza con FDM classico è che un sistema ottico che usa una griglia di difrazione è completamente passivo e quindi altamente affidabile.

La ragione della popolarità di WDM è che l'energia su un singolo canale è tipicamente larga pochi GHz, perché quello è il limite a cui riusciamo a convertire segnali elettrici in segnali ottici. Facendo andare molti canali in parallelo su diverse lunghezze d'onda la banda aggregata aumenta linearmente con il numero di canali. Siccome la banda di una singola fibra è di circa 25.000 GHz (si veda la Figura 2.7) c'è teoricamente spazio per 2500 canali da 10 Gbps anche usando 1 bit/Hz (e velocità superiori sono comunque possibili).

La tecnologia WDM sta progredendo a un ritmo tale da far vergognare la tecnologie hardware: inventata nel 1990 circa, il primo sistema commerciale aveva 8 canali da 2,5 Gbps; nel 1998 sul mercato erano disponibili sistemi da 40 canali da 2,5 Gbps. Nel 2006 c'erano prodotti con 192 canali da 10 Gbps e con 64 canali da 40 Gbps con una capacità di trasferimento totale di 2,56 Tbps. Questa banda è sufficiente per trasmettere 80 film in DVD al secondo. I canali sono anche molto fitti all'interno della fibra con bande di separazione di 200, 100 o anche solo 50 GHz. Prove in

laboratorio hanno dimostrato la possibilità di ottenere capacità 10 volte più elevate, ma la strada da percorrere dal laboratorio al mondo reale è lunga qualche anno. Quando il numero di canali è molto alto e lo spazio tra questi ridotto si usa il termine **DWDM** (*dense WDM*).

Uno degli impulsi della tecnologia WDM è la produzione di componenti puramente ottici. Inizialmente, ogni 100 km era necessario separare tutti i canali e convertire ognuno di essi in un segnale elettrico per amplificarlo separatamente prima di riconvertirlo in segnali ottici e riaggregare il tutto. Oggi, amplificatori completamente ottici sono in grado di rigenerare l'intero segnale ogni 1000 km senza la necessità di effettuare conversioni opto-elettroniche.

Nell'esempio della Figura 2.41 siamo in presenza di un sistema a lunghezza d'onda costante. I bit della fibra 1 in ingresso vanno nella fibra 3 in uscita, i bit della fibra 2 in ingresso vanno nella fibra 1 e così via. Tuttavia, è anche possibile costruire sistemi WDM permutati direttamente in modalità ottica. In questi dispositivi i filtri sulla fibra in uscita sono sintonizzabili tramite interferometri Fabry-Perot o Mach-Zehnder, per cui le frequenze selezionate possono essere cambiate dinamicamente da un computer. Questa possibilità fornisce una grande flessibilità, per cui con un insieme fissato di fibre è possibile gestire molti percorsi all'interno della rete telefonica. Per approfondire il discorso delle reti in fibra ottica e WDM si veda Ramaswami et al. (2009).

Commutazione

Dal punto di vista dell'ingegnere delle telecomunicazioni medio, il sistema telefonico è diviso in due parti principali: l'impianto esterno (ossia i collegamenti locali e i trunk, entrambi collocati fisicamente all'esterno delle centrali di commutazione) e l'impianto interno (composto dai commutatori che si trovano dentro le centrali di commutazione). Finora abbiamo descritto l'impianto esterno, ora è giunto il momento di esaminare in dettaglio quello interno.

Attualmente nelle reti si utilizzano due diverse tecniche di commutazione: la commutazione di circuito e quella di pacchetto. Il sistema telefonico tradizionale è basato su commutazione di circuito, anche se la commutazione di pacchetto sta cominciando a farsi strada con l'avanzare di tecnologie come il voice over IP. Ci occuperemo della commutazione di circuito in dettaglio e la confronteremo con la commutazione di pacchetto. Entrambi i tipi di commutazione sono importanti e torneremo a parlarne quando ci occuperemo del livello di rete.

Commutazione di circuito

Concettualmente, quando una persona o un computer avviano una telefonata, l'apparecchiatura di commutazione posta dentro il sistema telefonico cerca di creare un percorso fisico completo tra il telefono del chiamante e quello dell'utente chiamato. Questa tecnica, detta **commutazione di circuito**, è rappresentata nella Figura 2.42(a). Ognuno dei sei rettangoli rappresenta una centrale di commutazione dell'operatore. In questo esempio ogni centrale ha tre linee in ingresso e tre in uscita. Quando una chiamata passa attraverso una centrale di commutazione, viene stabilita (concettualmente) una connessione fisica tra la linea di provenienza della chiamata e una delle linee di uscita, come evidenziato dalle linee tratteggiate.

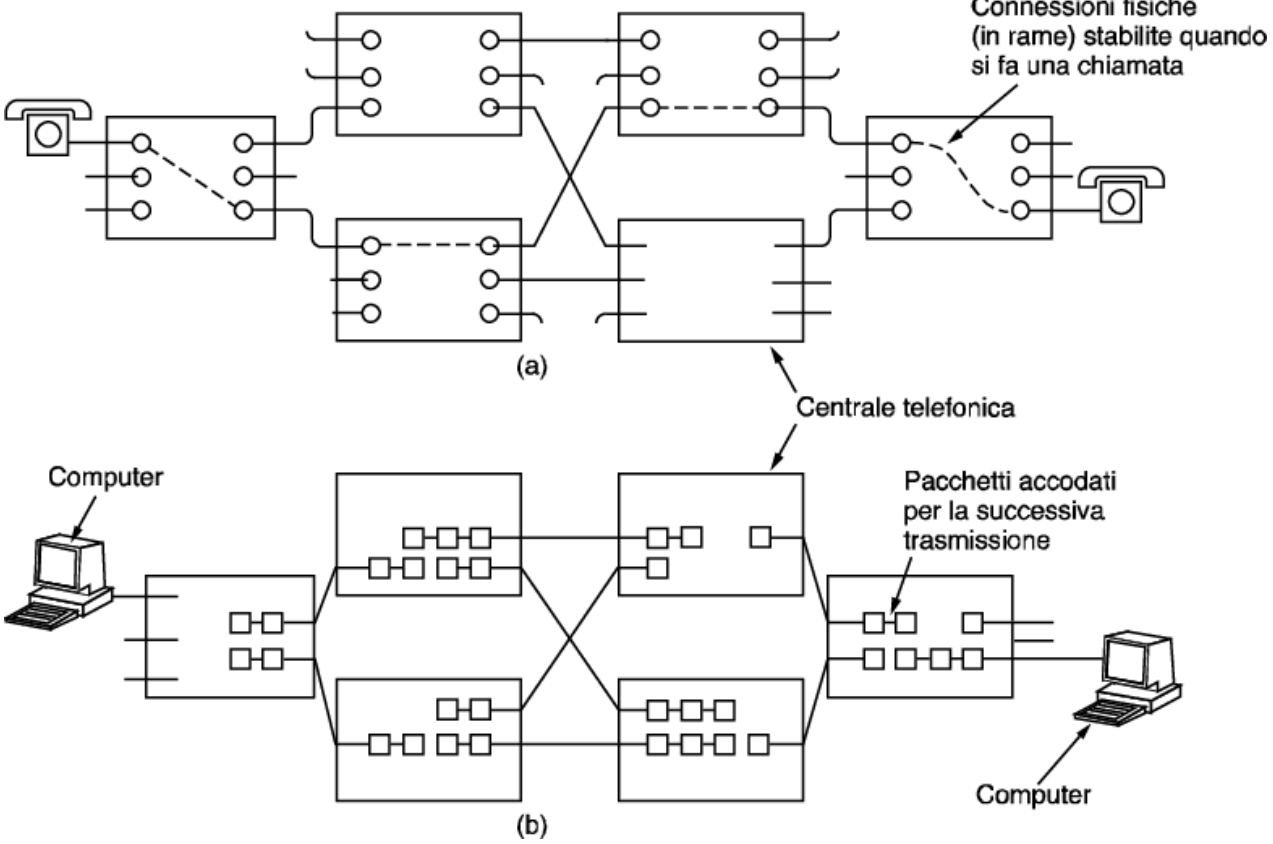


Figura 2.42 (a) Commutazione di circuito. (b) Commutazione di pacchetto.

Ai primordi della telefonia, era l'operatore a creare la connessione inserendo un cavo a ponte nelle prese di input e output. All'invenzione dei dispositivi di commutazione automatica è associata una storia sorprendente. Questi apparecchi furono inventati nel diciannovesimo secolo da un impresario di pompe funebri del Missouri che si chiamava Almon B. Strowger. Poco dopo l'invenzione del telefono, quando moriva qualcuno, i parenti del defunto chiamavano l'operatore cittadino e chiedevano di essere collegati con l'impresario delle pompe funebri. Sfortunatamente per il signor Strowger, in città esistevano due imprese di pompe funebri e la moglie dell'altro impresario era l'operatrice telefonica di quella città. Le possibilità per il povero signor Strowger a questo punto erano solo due: inventare un dispositivo di commutazione automatica per telefoni o finire sul lastrico. Egli scelse la prima opzione. Per quasi 100 anni, l'apparecchiatura a commutazione di circuito utilizzata in tutto il mondo è stata chiamata **dispositivo Strowger**. (La storia non riporta se la centralinista disoccupata abbia poi trovato lavoro come addetta alle informazioni per rispondere a domande quali "qual è il numero di telefono dell'impresario delle pompe funebri?").

Il modello mostrato nella Figura 2.42(a) è molto semplificato, perché parte del percorso fisico tra i due telefoni può essere costituito da collegamenti su microonde o fibra attraverso i quali viaggiano migliaia di chiamate unite in multiplexing. Ciò nonostante, l'idea di base è corretta: l'impostazione della chiamata crea tra le due estremità un percorso dedicato che perdura fino al termine della chiamata.

Una proprietà importante della commutazione di circuito è legata alla necessità di configurare un percorso da un punto all'altro *prima* di iniziare a trasmettere i dati. Tra la fine della composizione del numero e l'inizio dello squillo possono tranquillamente trascorrere 10 secondi, di più se la

chiamata è a lunga distanza o internazionale. Durante questo intervallo, il sistema telefonico cerca di definire il percorso della connessione come mostrato nella Figura 2.43(a). È bene notare che, prima che la trasmissione dei dati possa avere inizio, il segnale della richiesta di chiamata deve propagarsi fino alla destinazione ed essere riconosciuto. Per molte applicazioni informatiche (come le verifiche delle carte di credito) lunghi tempi di configurazione non sono accettabili.

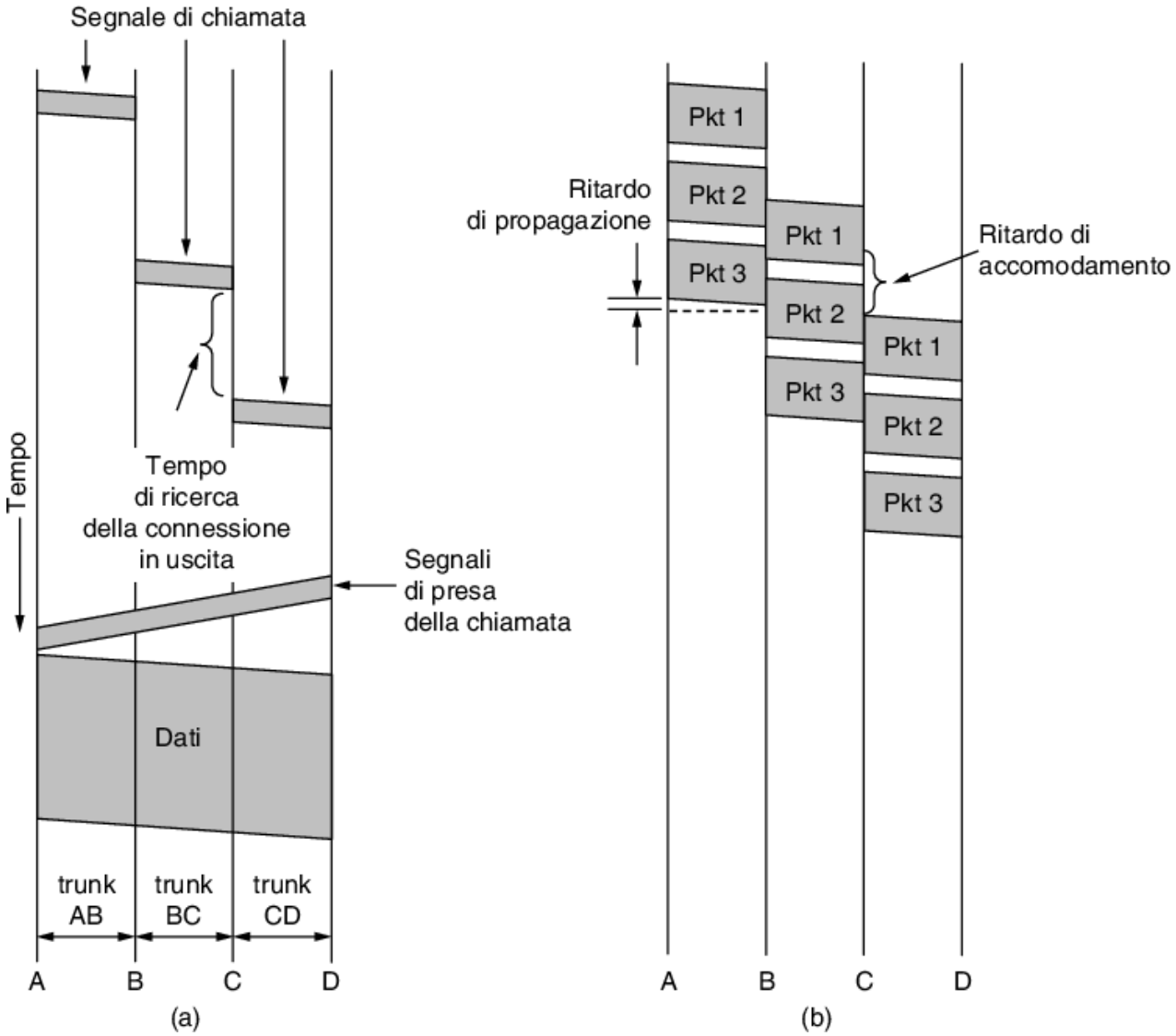


Figura 2.43 Temporizzazione degli eventi in (a) commutazione di circuito. (b) Commutazione di pacchetto.

Poiché viene creato un percorso dedicato tra chiamante e chiamato, una volta completata l'impostazione l'unico ritardo per i dati è quello dovuto al tempo di propagazione del segnale elettromagnetico, valore pari a circa 5 ms per 1.000 Km. Per lo stesso motivo non si corre il rischio di creare una congestione: una volta che la chiamata è stata messa in linea non è più possibile ricevere il segnale di occupato. Logicamente, il segnale di occupato può essere ricevuto prima che la connessione sia stabilita, se la commutazione o la capacità della linea non sono sufficienti.

Commutazione di pacchetto

L'alternativa alla commutazione di circuito è la **commutazione di pacchetto** (*packet switching*) descritta nella Figura 2.42(b) e trattata nel Capitolo 1. Con questa tecnologia i pacchetti vengono inviati non appena sono disponibili. Diversamente che con la commutazione di circuito non c'è alcuna necessità di creare un percorso dedicato in anticipo.

È compito dei router usare una trasmissione di tipo *store-and-forward* per spedire ogni pacchetto verso la destinazione in maniera indipendente. Questa procedura è molto diversa dalla commutazione di circuito, nella quale la creazione della connessione garantisce una prenotazione della banda dal mittente al ricevente. Inoltre, se tutti i pacchetti seguono lo stesso percorso non è possibile che arrivino fuori ordine. Con la commutazione di pacchetto non c'è un percorso prestabilito, per cui pacchetti distinti possono fare strade diverse a seconda delle condizioni della rete all'istante di invio; quindi è possibile che arrivino fuori ordine.

Le reti a commutazione di pacchetto impongono uno stretto limite superiore alla dimensione dei pacchetti, in maniera tale che nessun utente possa monopolizzare una linea di trasmissione per molto tempo (per molti millisecondi); in questo modo le reti a commutazione di pacchetto possono gestire traffico interattivo. Inoltre, il ritardo risulta ridotto, perché il primo pacchetto di un messaggio molto lungo verrà inoltrato verso la destinazione senza il bisogno che il secondo sia arrivato completamente. Tuttavia, il ritardo introdotto dallo store-and-forward per l'accumularsi di pacchetti nella memoria del router prima dell'inoltro al router successivo è maggiore di quello della commutazione di circuito. Con la commutazione di circuito i bit corrono attraverso i collegamenti in maniera continua.

Le commutazioni di pacchetto e circuito differiscono anche per altri motivi. Dato che non c'è prenotazione di banda, un pacchetto potrebbe dover aspettare prima di essere inoltrato; questo produce un **ritardo di accodamento** (*queueing delay*) e congestione della rete se molti pacchetti vengono inviati contemporaneamente. D'altra parte, però, non c'è più pericolo di avere un segnale di occupato e non essere in grado di usare la rete. Per cui, la congestione avviene in momenti diversi con la commutazione di circuito (alla creazione del circuito) e con la commutazione di pacchetto (quando il pacchetto viene spedito).

Se un circuito è stato allocato a un particolare utente e non c'è traffico, la banda viene sprecata perché non può essere usata per altro traffico. La commutazione di pacchetto non spreca banda ed è quindi più efficiente dal punto di vista del sistema. Questo compromesso è cruciale per capire la differenza tra commutazione di circuito e di pacchetto: dobbiamo scegliere tra un servizio garantito che spreca delle risorse e uno non garantito che non ne spreca.

La commutazione di pacchetto è più resistente agli errori della commutazione di circuito, anzi è stata inventata proprio per questo motivo. Se un commutatore si blocca, tutti i circuiti che lo utilizzano vengono chiusi e nessun dato può essere inviato; nel caso della commutazione di pacchetto, i pacchetti possono essere instradati aggirando i commutatori bloccati.

L'ultima differenza riguarda l'algoritmo di addebito. Con la commutazione di circuito, storicamente l'addebito dipende da tempo e distanza; per i telefoni cellulari la distanza di solito non gioca un ruolo importante (tranne che per le chiamate internazionali), mentre il tempo gioca solo un ruolo. Con la commutazione di pacchetto, il tempo di connessione non è un problema, ma il

volume del traffico qualche volta lo è. Di solito gli ISP addebitano alla maggior parte degli utenti domestici una tariffa netta mensile, che richiede meno lavoro e viene capita meglio dai clienti, ma le portanti dorsali fanno pagare le reti regionali in base al volume del loro traffico.

Un riepilogo delle differenze è riportato nella Figura 2.44. Storicamente, le reti telefoniche hanno usato la commutazione di circuito perché devono erogare servizi voce ad alta qualità e le reti di calcolatori hanno usato la commutazione di pacchetto perché semplice ed efficiente. Tuttavia, ci sono delle importanti eccezioni: alcune vecchie reti di calcolatori usano commutazione di circuito (ad esempio X.25)[5] e alcune reti telefoniche di ultima generazione usano commutazione di pacchetto con VoIP. In particolare, l'ultimo caso ha permesso la nascita di un mercato per chiamate internazionali economiche, anche se con una qualità inferiore, facendo uso di carte prepagate.

Caratteristica	**Commutazione di circuito**	**Commutazione di pacchetto**
Impostazione della chiamata	Richiesta	Non richiesta
Percorso fisico dedicato	Sì	No
Ogni pacchetto segue la stessa strada	Sì	No
I pacchetti arrivano in ordine	Sì	No
Il guasto dello switch è fatale	Sì	No
Banda disponibile	Fissa	Dinamica
Istante di possibile congestione	All'impostazione	A ogni pacchetto
Potenziale banda sprecata	Sì	No
Trasmissione store-and-forward	No	Sì
Tariffazione	Al minuto	Al pacchetto

Figura 2.44 Confronto tra reti a commutazione di circuito e a commutazione di pacchetto.

2.7 Il sistema telefonico mobile

Il sistema telefonico tradizionale, anche aggiornato con fibre multi-gigabit, tuttora non riesce a soddisfare un gruppo di utenti sempre crescente: quelli mobili. Le persone oggi vogliono telefonare e usare il proprio cellulare per controllare la e-mail e accedere al Web in aeroplano, in auto, in piscina o mentre corrono nel parco. Di conseguenza, la telefonia wireless suscita grandi aspettative. I paragrafi seguenti esaminano l'argomento in dettaglio.

Il sistema telefonico mobile è usato per comunicazioni a grande distanza sia vocali che dati. I **telefoni mobili** (comunemente chiamati **cellulari**) sono passati attraverso tre diverse generazioni, chiamate anche **1G**, **2G** e **3G**, che sono:

1. voce analogica
2. voce digitale
3. voce e dati digitali (Internet, posta elettronica e altro).

(I telefoni cellulari non devono essere confusi con i **telefoni cordless**, i quali consistono di una base e un dispositivo portatile che operano congiuntamente e venduti per uso domestico. Questi non vengono mai usati per attività di rete per cui non li considereremo).

La maggior parte della discussione verte della tecnologia adottata da questi sistemi, tuttavia è interessante notare l'impatto che possono avere le decisioni politiche e commerciali. Il primo sistema mobile è stato creato negli Stati Uniti da AT&T su mandato della FCC. Di conseguenza tutti gli Stati Uniti avevano un singolo sistema (analogico) e un telefono mobile acquistato in California funzionava anche a New York. Al contrario, quando il sistema mobile arrivò in Europa, ogni paese ideò un suo sistema e il risultato fu un fiasco.

L'Europa fece tesoro di questo errore: quando apparve la tecnologia digitale, le PTT concordarono un singolo sistema standardizzato (GSM), così qualunque telefono mobile europeo avrebbe funzionato in qualunque paese europeo. Nello stesso periodo il governo degli Stati Uniti, dopo aver deciso che si sarebbe occupato degli standard, lasciò il controllo del digitale al mercato. Questa decisione fece sì che i produttori di apparecchiature producessero diversi tipi di telefoni cellulari.

Di conseguenza, ora gli Stati Uniti utilizzano due (e del tutto incompatibili) sistemi digitali mobili principali e altri sistemi minori.

Nonostante un ritardo iniziale rispetto agli Stati Uniti, oggi l'Europa può vantare un numero di cellulari e un utilizzo maggiore di quello statunitense. L'adozione di un unico sistema non è l'unico motivo di questo successo. Europa e Stati Uniti differiscono anche per un altro aspetto: i numeri di telefono. Negli Stati Uniti, i telefoni cellulari sono mischiati ai telefoni fissi, perciò chi chiama non è in grado di sapere se per esempio il numero (212) 234-5678 appartiene a un telefono fisso (telefonata economica o addirittura gratuita) oppure a un cellulare (telefonata costosa). Per non far innervosire gli utenti, le aziende telefoniche hanno deciso di far pagare ai possessori di cellulari le telefonate in arrivo. Di conseguenza, molte persone esitano ad acquistare un cellulare per paura di dover pagare bollette salate dovute alle telefonate ricevute. In Europa, invece, i telefoni cellulari usano speciali prefissi facilmente riconoscibili, di conseguenza è applicata la regola classica che addebita il costo della chiamata alla persona che chiama.

Un terzo elemento che ha avuto un grande impatto sull'adozione dei telefoni mobili in Europa è l'utilizzo molto esteso delle schede prepagate (in alcune aree il fenomeno ha raggiunto il 75%). Questi telefoni possono essere acquistati in molti negozi senza complicazioni e dispongono di una scheda da 20 o 50 euro che può essere ricaricata non appena si esaurisce. Di conseguenza, praticamente tutti gli adolescenti e molti bambini europei posseggono un telefono cellulare (di solito con scheda prepagata) che consente ai genitori di localizzare i figli senza correre il rischio di dover pagare bollette salate. Se il telefono mobile è utilizzato solo occasionalmente, il suo impiego è essenzialmente gratuito, perché non esiste alcun canone fisso e le telefonate ricevute non hanno alcun costo.

Prima generazione (1G): voce analogica

Ora esaminiamo la tecnologia, incominciando dal sistema più vecchio. I radiotelefoni mobili furono utilizzati sporadicamente per la comunicazione marittima e militare durante i primi decenni del ventesimo secolo. Nel 1946 a St. Louis venne creato il primo sistema telefonico per auto. Questo sistema utilizzava un singolo trasmettitore di grandi dimensioni collocato in cima a un alto edificio e aveva un unico canale utilizzato sia per trasmettere sia per ricevere. Per parlare l'utente doveva premere un pulsante che attivava il trasmettitore e disattivava il ricevitore.

Questi sistemi, conosciuti come **push-to-talk** ("premere per parlare"), furono installati in diverse città a partire dalla fine degli anni '50. Radio CB, taxi e macchine della polizia spesso utilizzano questa tecnologia.

Negli anni '60 venne installato **IMTS** (*improved mobile telephone system*); anche questo sistema utilizzava un trasmettitore ad alta potenza (200 Watt) collocato in cima a una collina, ma le frequenze questa volta erano due: una per trasmettere e una per ricevere, perciò il pulsante che attivava la comunicazione scomparve. Poiché la comunicazione che dal telefono mobile era diretta verso l'esterno passava attraverso un canale diverso da quello utilizzato per le chiamate in arrivo, gli utenti non potevano ascoltarsi (a differenza del push-to-talk utilizzato nei taxi).

IMTS supportava 23 canali distribuiti tra 150 MHz e 450 MHz. A causa del limitato numero di canali, gli utenti spesso dovevano attendere per molto tempo prima di ottenere il segnale di libero. Inoltre, a causa della grande potenza dei trasmettitori, sistemi adiacenti dovevano trovarsi a centinaia di chilometri di distanza per evitare interferenze. Alla fine la capacità limitata rese il sistema poco pratico.

AMPS – advanced mobile phone system

Tutto questo cambiò con **AMPS** (*advanced mobile phone system*), sistema inventato da Bell Labs e installato per la prima volta negli Stati Uniti nel 1982. AMPS fu usato anche in Inghilterra (e in Italia), dove venne chiamato TACS, e in Giappone, dove fu chiamato MCS-L1. AMPS è stato formalmente ritirato dal mercato nel 2008, ma lo esamineremo ugualmente per capire da quale contesto i sistemi 2G e 3G si sono evoluti.

In tutti i sistemi telefonici mobili un'area geografica è divisa in celle; ecco perché i dispositivi sono chiamati anche telefoni cellulari. In AMPS, le celle sono grandi 10-20 Km; nei sistemi digitali le celle sono più piccole. Ogni cella utilizza un insieme di frequenze non impiegate dalle celle vicine. L'idea chiave che dà al sistema cellulare una capacità superiore a quella dei sistemi precedenti è

l'utilizzo di celle relativamente piccole e il riutilizzo delle frequenze di trasmissione delle celle vicine (ma non adiacenti). Mentre un sistema IMTS che si estende per 100 Km può avere una chiamata per ogni frequenza, un sistema AMPS può avere nella stessa area 10 celle di 10 Km e gestire da 10 a 15 chiamate per ogni frequenza, in celle distanti. L'architettura cellulare aumenta pertanto la capacità del sistema di almeno un ordine di grandezza e anche di più se le celle rimpiccioliscono. Inoltre, tanto più le celle sono piccole, tanto minore è la potenza richiesta e tanto più piccoli ed economici sono i trasmettitori e gli apparecchi portatili.

Il concetto di riutilizzo della frequenza è rappresentato nella Figura 2.45 (a). Le celle di solito hanno una forma approssimativamente circolare, ma sono più semplici da modellare come esagoni. Nella Figura 2.45 (a) le celle hanno tutte la stessa dimensione e sono organizzate in gruppi di sette; ogni lettera indica un gruppo di frequenze. Si noti come per ogni insieme di frequenze ci sia un'area cuscinetto attorno a ogni cella grande circa altre 2 celle in cui quella frequenza non viene riutilizzata; questo sistema consente di separare le frequenze e ridurre l'interferenza.

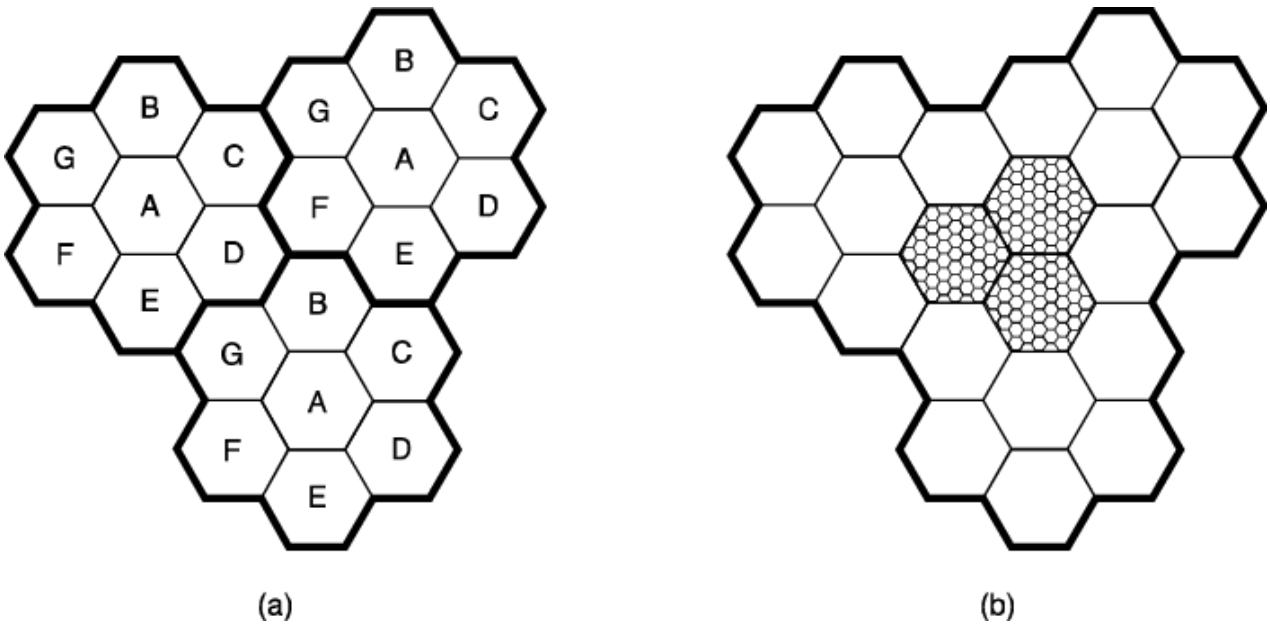


Figura 2.45 (a) Le frequenze non sono riutilizzate nelle celle adiacenti. (b) Per aumentare il numero di utenti si possono utilizzare celle più piccole.

Un grosso problema è quello di individuare posizioni elevate dove collocare le antenne delle stazioni base. Questo aspetto critico ha costretto alcune società di telecomunicazioni a stringere alleanze con la Chiesa Cattolica Romana, in quanto proprietaria di un altissimo numero di antenne potenziali sparse in tutto il mondo, gestite da un unico ente. In un'area dove il numero di utenti è cresciuto al punto che il sistema si è sovraccaricato, la potenza viene ridotta e le celle sovraccaricate sono divise in celle più piccole chiamate **microcelle** per aumentare il riutilizzo delle frequenze, come nella Figura 2.45(b). A volte le aziende telefoniche, in occasione di importanti eventi sportivi, concerti e così via dove si radunano numerose persone, creano microcelle temporanee utilizzando torri portatili collegate ai satelliti.

Al centro di ogni cella si trova una stazione base che comunica con tutti i telefoni che si trovano nella cella. La stazione base è composta da un computer e un trasmettitore/ricevitore collegato a un'antenna. In un sistema di dimensioni ridotte tutte le stazioni base sono collegate a un singolo dispositivo chiamato **MSC** (*mobile switching center*) o **MTSO** (*mobile telephone switching office*). Un sistema più esteso può richiedere diversi MSC, tutti collegati a un MSC di secondo livello e così via. I MSC sono come le centrali locali del sistema telefonico, e sono collegati ad almeno una centrale del sistema telefonico. I MSC comunicano tra loro, con le stazioni base e con la rete telefonica fissa mediante una rete a commutazione di pacchetto.

In ogni istante, ogni telefono cellulare è collocato dal punto di vista logico in una specifica cella e sotto il controllo della stazione base di quella cella. Quando un telefono mobile abbandona fisicamente una cella, poiché si accorge che il segnale dell'apparecchio si sta affievolendo, la stazione base di quella cella verifica il livello di potenza del segnale ricevuto dalle stazioni che si trovano nelle celle adiacenti. A questo punto la stazione trasferisce la gestione dell'apparecchio alla cella che riceve il segnale più forte, di solito alla cella in cui ora si trova il telefono. Il telefono viene informato dell'identità della nuova centrale di controllo e, se durante lo spostamento era in corso una chiamata, l'apparecchio viene forzato a passare su un nuovo canale (perché quello vecchio non è riutilizzato nelle celle adiacenti). Questo processo, chiamato **handoff**, richiede circa 300 ms. L'assegnazione del canale è eseguita dal MSC, il centro nevralgico del sistema, mentre le stazioni base sono semplici ripetitori radio.

Canali

AMPS usa FDM per separare i canali. L'architettura usa 832 canali full duplex, ognuno costituito da un paio di canali simplex. Questa organizzazione è nota come **FDD** (*frequency division duplex*, duplex a divisione di frequenza). Gli 832 canali simplex che vanno da 824 a 849 MHz sono usati per le trasmissioni dal telefono alla stazione base e gli 832 canali che vanno da 869 a 894 MHz sono usati per le comunicazioni nella direzione opposta. Ognuno di questi canali simplex è ampio 30 kHz.

Gli 832 canali sono divisi in quattro categorie:

1. controllo (dalla base al mobile) per gestire il sistema
2. paging (dalla base al mobile) per avvisare gli utenti mobili di chiamate in arrivo
3. accesso (bidirezionale) per impostare la chiamata e l'assegnazione del canale
4. dati (bidirezionale) per voce, fax o dati.

Poiché ventuno canali in ogni cella sono riservati al controllo e le stesse frequenze non possono essere riutilizzate nelle celle adiacenti, il numero reale di canali a disposizione della voce è molto più piccolo di 832; in genere è circa 45.

Gestione delle chiamate

Ogni telefono mobile AMPS ha un numero seriale lungo 32 bit e un numero di telefono di 10 cifre registrato nella sua memoria in sola lettura. Il numero di telefono è rappresentato da un prefisso di 3 cifre (10 bit) e dal numero dell'abbonato composto da 7 cifre (24 bit). Quando viene acceso, il telefono esplora l'elenco programmato dei 21 canali di controllo per trovare il segnale più potente. L'apparecchio quindi trasmette in broadcast il numero seriale di 32 bit e il numero di

telefono di 34 bit. Come tutte le informazioni di controllo trasmesse in AMPS, questo pacchetto è inviato in forma digitale, più volte e con un codice di correzione degli errori anche se i canali vocali veri e propri sono analogici.

Quando riceve la comunicazione, la stazione base aggiorna l'MSC che registra la presenza del nuovo cliente e informa l'MSC principale del cliente, tenendola aggiornata sulla posizione corrente. Durante il normale utilizzo la registrazione del telefono mobile viene eseguita circa una volta ogni 15 minuti.

Per fare una telefonata, l'utente mobile deve accendere l'apparecchio, inserire il numero che desidera chiamare e premere il pulsante che trasmette la richiesta. A questo punto il telefono trasmette il numero da chiamare e la propria identità attraverso il canale di accesso; in caso di collisione, l'operazione viene ripetuta più tardi. Quando riceve la richiesta, la stazione base informa l'MSC. Se il chiamante è un cliente dell'azienda dell'MSC (o di un suo partner commerciale), l'MSC cerca un canale libero da assegnare alla chiamata. Se ne trova uno, trasmette il numero del canale attraverso il canale di controllo. Il telefono mobile allora passa automaticamente al canale vocale selezionato e attende che la persona chiamata risponda.

Le chiamate in arrivo funzionano in modo diverso. Intanto, tutti i telefoni inattivi rimangono in ascolto sul canale di trasferimento per rilevare eventuali messaggi inviati loro. Quando una chiamata viene trasmessa a un cellulare (da un telefono fisso oppure da un altro apparecchio mobile), l'MSC principale dell'utente chiamato riceve un pacchetto che ha la funzione di individuare il destinatario. Ora viene trasmesso un altro pacchetto verso la stazione base della cella dove si trova l'utente, che successivamente trasmette in broadcast attraverso il canale di trasferimento un messaggio la cui forma è: "Unità 14, ci sei?". Il telefono chiamato risponde con un "Sì" sul canale di accesso. La base invia quindi un messaggio del tipo: "Unità 14, c'è una chiamata per te sul canale 3". A questo punto, il telefono chiamato passa al canale 3 e inizia a squillare (o a suonare qualche melodia che il proprietario ha ricevuto come regalo di compleanno).

Seconda generazione (2G): voce digitale

La prima generazione di telefonia mobile era analogica, la seconda è digitale. Questo cambiamento ha portato una serie di vantaggi, tra cui un guadagno di capacità trasmissiva grazie alla digitalizzazione e compressione della voce. Questo risulta anche un utile deterrente per frodi e intercettazioni sia intenzionali che casuali. Infine, rende possibili nuovi servizi come i messaggi di testo.

Tanto quanto non esisteva una standardizzazione a livello mondiale per la prima generazione, non ce n'era una neppure per la seconda. Molti sistemi diversi sono stati sviluppati e tre di questi installati su larga scala. **D-AMPS** (*digital advanced mobile phone system*) è una versione digitale di AMPS che coesiste con AMPS e usa TDM per instaurare più chiamate sullo stesso canale; viene descritto negli standard internazionali IS-54 e I-136. **GSM** (*global system for mobile communications*) è emerso come il sistema dominante e anche se ci ha messo del tempo ad affermarsi negli Stati Uniti è ora usato virtualmente in tutto il mondo. Così come D-AMPS, GSM utilizza una combinazione di FDM e TDM.

A volte viene usato il termine **PCS** (*personal communications services*) nei documenti commerciali per indicare un sistema di seconda generazione (in quanto digitale). Originariamente stava a significare un telefono mobile facente uso della banda a 1900 MHz, ma ora quella distinzione non è più molto usata.

Descriveremo ora il GSM in quanto è il sistema 2G più importante. Nel paragrafo successivo torneremo a parlare di CDMA trattando dei sistemi 3G.

GSM – global system for mobile communication

GSM ha preso il via negli anni '80 dalla volontà di produrre uno standard europeo per il 2G. L'attività fu assegnata a un gruppo di esperti di telecomunicazioni chiamato (in francese) Groupe Specialé Mobile, da cui l'acronimo originale. Le prime infrastrutture GSM furono installate a partire dal 1991 e riscossero un successo immediato. Fu presto chiaro che il GSM stava per diventare molto di più di un successo europeo, visto che veniva utilizzato anche in paesi lontani come l'Australia, per cui a GSM fu attribuito un nome più adatto a livello internazionale.

GSM e gli altri sistemi di telefonia mobile che studieremo hanno mantenuto dalla prima generazione un'organizzazione a celle, il riutilizzo delle frequenze tra celle, e la gestione della mobilità degli utenti tramite handoff. La differenza sta però nei dettagli. Qui discuteremo alcune delle principali caratteristiche del GSM. Tuttavia, lo standard GSM in formato cartaceo occupa più di 5000 (!) pagine. Una parte significativa di questo materiale tratta di aspetti tecnologici, in particolare della progettazione dei ricevitori per la gestione della propagazione multipath e la sincronizzazione tra trasmittenti e ricevitori. Niente di ciò sarà menzionato qui.

La Figura 2.46 mostra che l'architettura GSM è simile a quella di AMPS anche se i componenti hanno nomi diversi. Il terminale mobile è ora diviso in un dispositivo e un chip rimovibile chiamato **SIM card**, (o SIM), forma abbreviata per *subscriber identity module* ("modulo di identificazione dell'abbonato"*)*. È la SIM card che permette al telefono di funzionare e contiene dati segreti che consentono all'apparecchio e alla rete di identificarsi vicendevolmente e di cifrare le telefonate. La SIM può essere rimossa e inserita in un dispositivo diverso per farlo diventare, dal punto di vista della rete, il vostro telefono mobile.

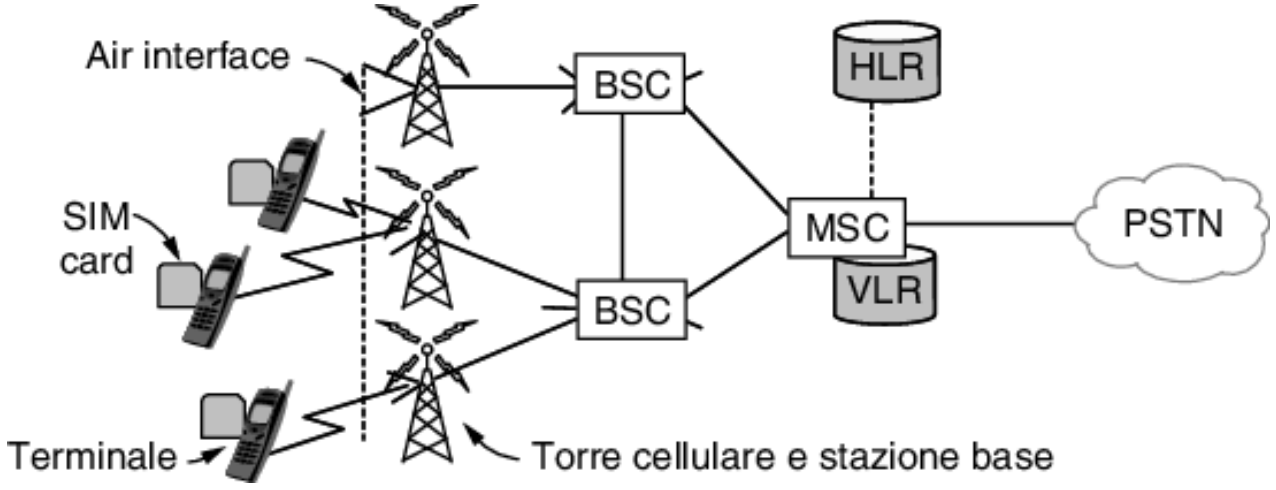


Figura 2.46 Architettura di rete GSM.

Il telefono parla con la stazione base cellulare usando un collegamento wireless che prende il nome di **air interface** (*interfaccia aerea*). Le stazioni base cellulare sono collegate a un **BSC** (*base station controller*, controllore di stazione base) che gestisce le risorse radio delle celle e handoff dei dispositivi. Il BSC è a sua volta collegato a un MSC (come nel caso di AMPS) che instrada le chiamate e si connette alla rete telefonica fissa, **PSTN** (*public switched telephone network*).

Per essere in grado di instradare le telefonate, l'MSC ha bisogno di sapere dove trovare i dispositivi mobili, per cui mantiene un database dei telefoni associati con le celle che gestisce. Questo database prende il nome di **VLR** (*visitor location register*). Esiste inoltre un database nella rete che fornisce l'ultima posizione nota di ogni terminale ed è chiamato **HLR** (*home location register*); tale database viene usato per instradare le chiamate in arrivo verso le giuste posizioni. Entrambi i database devono essere tenuti aggiornati mano a mano che i terminali mobili si spostano da una cella all'altra.

Descriveremo ora in dettaglio l'interfaccia aerea. GSM usa una gamma di frequenze allocate a livello globale, incluse 900, 1800 e 1900 MHz. Per supportare un numero molto più alto di utenti viene allocato più spettro di AMPS. GSM è un sistema cellulare duplex a divisione di frequenza, come AMPS. Quindi ogni terminale trasmette su una frequenza e riceve su un'altra frequenza più alta (55 MHz più alta per il GSM contro 80 MHz per l'AMPS). Tuttavia, diversamente da AMPS, nel GSM ogni coppia di frequenze viene divisa con TDM in intervalli di tempo per cui può essere condivisa da più terminali.

Per gestire più terminali i canali GSM sono molto più ampi di quelli AMPS (200 kHz contro 30 kHz); un canale da 200 kHz è mostrato nella Figura 2.47. Un sistema GSM che opera nella regione dei 900 MHz possiede 124 paia di canali simplex. Ogni canale simplex è ampio 200 kHz e supporta otto connessioni separate mediante multiplexing a divisione di tempo. A ogni stazione attiva è assegnato uno slot temporale su una coppia di canali. In teoria supporta 992 canali, ma molti non sono disponibili per evitare conflitti di frequenza con le celle adiacenti. Nella Figura 2.47 gli otto intervalli temporali condivisi (slot) appartengono tutti alla stessa connessione, quattro per ogni direzione. Trasmissione e ricezione non avvengono nello stesso intervallo temporale perché le radio GSM non sono in grado di trasmettere e ricevere contemporaneamente e passare da un'operazione all'altra richiede tempo. Se la stazione mobile assegnata nell'esempio al secondo slot temporale della coppia 890,4/935,4 MHz desidera trasmettere alla stazione base, deve utilizzare i quattro slot inferiori (in grigio) e quelli successivi, inserendo i dati in ogni slot fino al loro esaurimento.

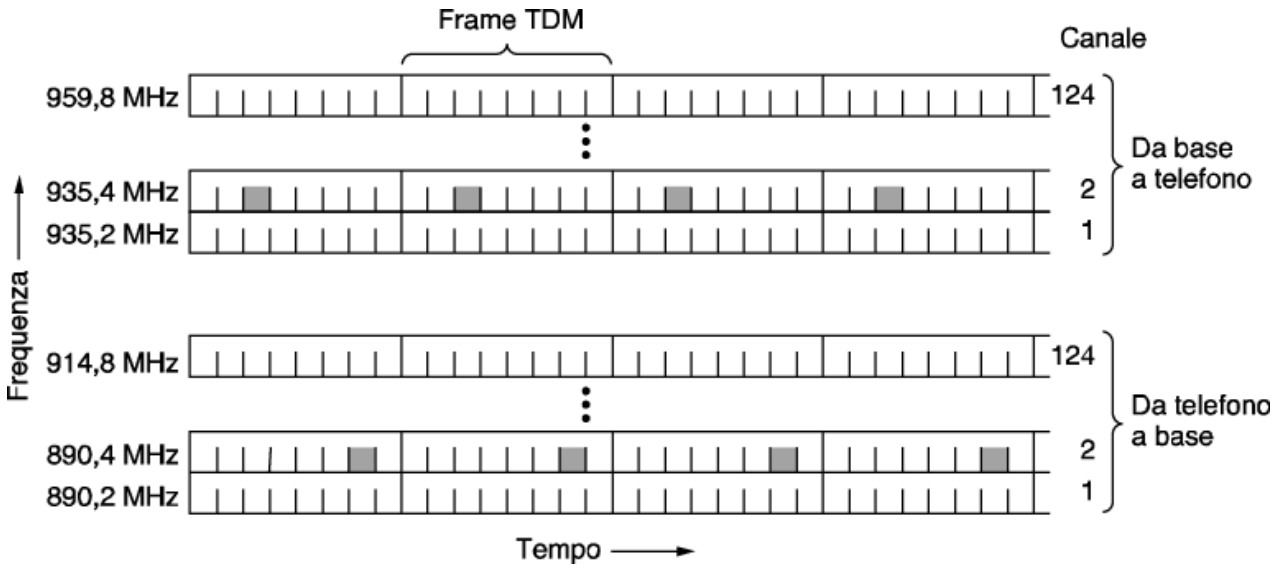


Figura 2.47 GSM usa 124 frequenze di canale; ogni canale utilizza un sistema TDM a 8 slot.

Gli slot TDM mostrati nella Figura 2.47 fanno parte di una complessa gerarchia di frame. Ogni slot TDM ha una struttura specifica e gruppi di slot TDM formano multiframe, anche questi con una struttura specifica. Una versione semplificata di questa gerarchia è mostrata nella Figura 2.48. Come si può notare dall'esempio, ogni slot TDM è composto da un frame con 148 bit di dati che occupa il canale per 577 ms (incluso un intervallo di guardia di 30 ms dopo ogni slot). Ogni frame dati inizia e finisce con tre bit a 0 che servono a delimitarlo; la struttura contiene anche due campi *Information* a 57 bit, ognuno dotato di un bit di controllo che indica se il successivo campo *Information* è per la voce oppure per i dati. Tra i campi Information si trova un campo *Sync* (sincronizzazione) a 26 bit, utilizzato dal ricevitore per sincronizzarsi con i limiti del frame del trasmettitore.

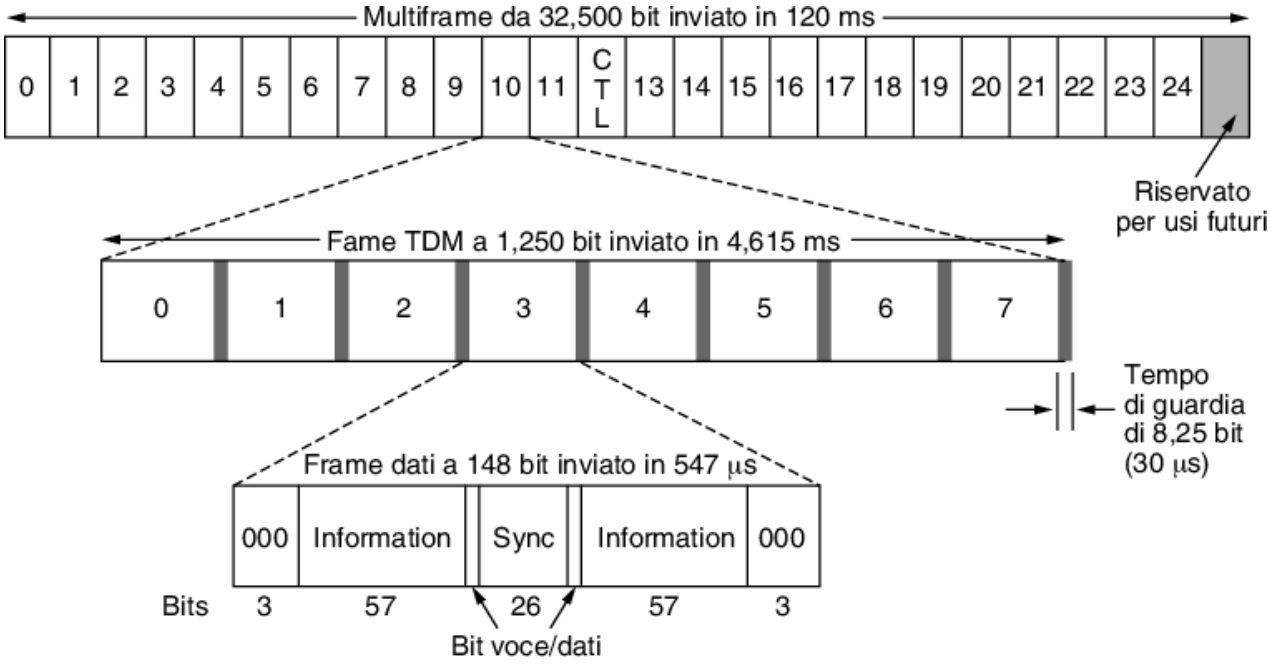


Figura 2.48 Una parte della struttura di frame GSM.

Un frame di dati è trasmesso in 547 ms, ma il trasmettitore può spedire un solo frame dati ogni 4,615 ms poiché sta condividendo il canale con altre sette stazioni. La velocità complessiva di ogni canale è di 270.833 bps, condivisa da otto utenti. Tuttavia, come nel caso di AMPS, il tempo di elaborazione dati consuma gran parte dell'ampiezza di banda e lascia in definitiva soltanto 24,7 kbps per utente a disposizione al lordo della correzione degli errori. Dopo la correzione degli errori lo spazio a disposizione della voce è di soli 13 kbps. Anche se si tratta di molto meno dei 64 kbps PCM per la voce non compressa delle reti telefoniche fisse, la compressione sul dispositivo mobile più raggiungere questi livelli con poca perdita di qualità.

Come si può notare osservando la Figura 2.48, otto frame di dati compongono un frame TDM e 26 frame TDM compongono un multiframe di 120 ms. Esaminando i 26 frame TDM di un multiframe, si nota che lo slot 12 è utilizzato per operazioni di controllo e il 25 è riservato ad applicazioni future, perciò il traffico dell'utente ha a disposizione solo 24 slot.

Oltre al multiframe con 26 slot mostrato nella Figura 2.48 si utilizza anche un multiframe con 51 slot; alcuni sono utilizzati per contenere i canali di controllo necessari a gestire il sistema. Il **canale di controllo broadcast** (*broadcast control channel*) è un flusso continuo di dati trasmessi dalla stazione base che annuncia identità e stato del canale della stazione base. Tutte le stazioni mobili tengono sotto controllo la potenza del loro segnale per rilevare i cambiamenti di cella.

Il **canale di controllo dedicato** (*dedicated control channel*) è utilizzato per aggiornare la posizione, registrare il terminale nella rete e impostare la chiamata. Ogni BSC mantiene in un archivio gli identificativi delle stazioni mobili presenti nella cella: il VLR; le informazioni richieste per gestire questo database sono trasmesse attraverso il canale di controllo dedicato.

Infine, c'è un **canale di controllo comune** (*common control channel*) diviso in tre sottocanali logici. Il primo è il **canale di paging** (*paging channel*), utilizzato dalla stazione base per annunciare le chiamate in arrivo; ogni stazione mobile tiene continuamente sotto controllo questo canale per rilevare eventuali chiamate. Il secondo sottocanale si chiama **canale ad accesso casuale** (*random access channel*) e permette agli utenti di richiedere uno slot sul canale di controllo dedicato. Due richieste che collidono risultano confuse e per questo devono essere ripetute in un secondo momento. La stazione può configurare una chiamata usando uno slot del canale di controllo dedicato; lo slot assegnato è annunciato attraverso il terzo sottocanale, chiamato **canale di concessione dell'accesso** (*access grant channel*).

In conclusione, GSM differisce da AMPS nella gestione dell'handoff. In AMPS l'MSC gestisce tutto in autonomia senza aiuto da parte del terminale mobile. Poiché un terminale mobile è spesso inattivo, molti degli intervalli di tempo risultanti dal TDM di GSM rimangono inutilizzati e possono essere sfruttati dal terminale per misurare la qualità del segnale con altre stazioni base vicine. I risultati delle misurazioni vengono inviati al BSC che li usa per determinare quando un terminale sta per lasciare una cella per entrare in un'altra in maniera tale da poter gestire l'handoff. Questo approccio è chiamato **MAHO** (*mobile-assisted handoff*).

Terza generazione (3G): voce digitale e dati

La prima generazione di telefonia mobile era per voce analogica e la seconda per voce digitale. La terza generazione, o 3G, è tutta centrata su voce digitale *e* dati.

Una quantità di fattori sta condizionando l'industria. Primo, il traffico dati è già più alto di quello vocale sulle reti fisse e continua a crescere in maniera esponenziale, mentre la voce è essenzialmente stabile; molti esperti prospettano lo stesso fenomeno a breve anche su dispositivi mobili. Secondo, le industrie che si occupano di telefonia, intrattenimento e computer si sono tutte convertite al digitale e stanno rapidamente convergendo; molte persone non desiderano altro che avere un dispositivo leggero e portatile che funzioni da telefono, riproduttore di suoni e video, lettore per e-mail, browser Web, console per videogiochi e molto altro; tutto con una connessione wireless a banda larga verso Internet.

L'iPhone della Apple è un buon esempio di questo tipo di dispositivo 3G; attraverso di esso la gente viene attirata dai servizi wireless per trasmissione dati e il volume di traffico di AT&T aumenta molto rapidamente, insieme alla popolarità dell'iPhone. Il problema è che l'iPhone usa una rete 2.5G (una rete 2G potenziata, ma non una vera rete 3G)[6] e non c'è sufficiente capacità di trasporto dati per soddisfare gli utenti. La telefonia 3G ha un unico obiettivo: fornire abbastanza banda wireless per appagare le attese di questi futuri utenti.
Già nel 1992 l'ITU tentò di essere più specifica a proposito di questa visione, pubblicando un piano di sviluppo chiamato **IMT-2000**, dove IMT stava per *international mobile telecommunications*. I servizi di base che la rete IMT-2000 avrebbe dovuto fornire ai propri utenti sarebbero stati:

1. 1trasmissione voce ad alta qualità
2. trasmissione messaggi (al posto di e-mail, fax, SMS, chat e altro)
3. applicazioni multimediali (riproduzione di musica, video, film, televisione e altro)
4. accesso Internet (Web, incluse le pagine con audio e video).

Servizi aggiuntivi potrebbero essere videoconferenze, telepresenze, giochi di gruppo ed m-commerce (impiego del telefono cellulare per pagare i propri acquisti). Inoltre, tutti questi servizi dovrebbero essere disponibili ovunque mondo (con connessione automatica via satellite quando non è disponibile una rete terrestre), istantaneamente (quindi sempre attivi) e con garanzie di qualità.

ITU concepì per IMT-2000 un'unica tecnologia a livello globale per consentire ai produttori di costruire un singolo apparecchio che potesse essere venduto e utilizzato ovunque nel mondo (come i riproduttori di CD e i computer, non come i cellulari e i televisori) L'utilizzo di una singola tecnologia avrebbe anche semplificato il lavoro degli operatori di rete e incoraggiato un numero maggiore di persone a utilizzare i servizi. Le guerre dei formati, come quella tra Betamax e VHS all'inizio dell'era della videoregistrazione, non aiutano gli affari.

Questa previsione risultò però un po' troppo ottimistica. Il numero 2000 aveva tre significati: (1) l'anno in cui avrebbe dovuto andare in esercizio, (2) la frequenza alla quale avrebbe dovuto operare (in MHz) e (3) la banda che avrebbero dovuto avere (in kbps) il servizio. Non ce l'ha fatta su nessuno di questi fronti: niente è stato implementato entro il 2000. L'ITU ha raccomandato a tutti i governi di riservare lo spettro a 2 GHz in maniera che i dispositivi potessero spostarsi senza problemi da un paese all'altro; solo la Cina lo ha fatto. Infine, è stato appurato che 2 Mbps non è praticabile per utenti che risultano essere *troppo* mobili per via della difficoltà di gestire l'handoff abbastanza velocemente. 2 Mbps è più realistico per utenti stazionari e per uso domestico (cosa che va a collidere con ADSL), 384 kbps per persone che camminano e 144 kbps per connessione in auto.

Nonostante questi problemi iniziali, molto è stato fatto da allora. Sono state formulate varie proposte per IMT e dopo alcune discussioni sono state identificate le due principali. Una, **WCDMA** (*wideband CDMA*), è stata proposta da Ericsson e poi pubblicata dall'Unione Europea che l'ha chiamata **UMTS** (*universal mobile telecommunications system*); l'altra, **CDMA2000**, è stata proposta da Qualcomm.

Questi sistemi sono molto simili tra loro, anche tenuto conto del fatto che entrambi sono basati su CDMA a larga banda; WCDMA usa canali a 5 MHz e CDMA2000 usa canali a 1,25 MHz. Se i progettisti di Ericsson e Qualcomm fossero messi in una stanza e gli fosse detto di concordare uno schema comune probabilmente lo avrebbero trovato molto presto. Il reale problema però non è tanto ingegneristico, quanto (come al solito) politico. L'Europa voleva un sistema compatibile con GSM, mente gli Stati Uniti lo volevano compatibile con un altro sistema già largamente installato sul loro territorio (IS-95). Ogni contendente, inoltre, portava avanti la compagnia locale (Ericsson è svedese e Qualcomm californiana). Infine, Ericsson e Qualcomm furono coinvolte in numerose denunce riguardo i rispettivi brevetti CDMA.

A livello mondiale il 10-15% degli abbonati a servizi mobili usa già tecnologie 3G. In Nord America e in Europa circa un terzo degli utenti sono 3G. Il Giappone ha abbracciato per primo la tecnologia e ora quasi tutti i telefoni giapponesi sono 3G. Questi numeri includono entrambe le installazioni di UMTS e CDMA2000 e il 3G continua a essere un gran calderone di attività, mentre il mercato si agita nervosamente. Ad aumentare la confusione, UMTS è diventato un singolo standard 3G con tante opzioni incompatibili tra loro, compresa CDMA2000. Questo

cambiamento è stato un tentativo di unificare i vari partiti, ma ha solo aggiunto pile di carta sulle differenze tecnologiche e confuso l'obiettivo delle attività in corso. Useremo UMTS per indicare WCDMA, distinto da CDMA2000.

Concentreremo la nostra attenzione sull'impiego di CDMA nelle reti cellulari, dato che è il tratto distintivo di entrambi i sistemi appena descritti. CDMA non è né FDM né TDM, ma una specie di via di mezzo tra i due, in cui ogni utente invia dati sulla stessa banda di frequenza di tutti gli altri nello stesso istante. Quando è stato proposto per la prima volta l'industria ebbe più o meno la stessa reazione della regina Isabella nei confronti di Colombo che proponeva di raggiungere l'India navigando nella direzione sbagliata. Tuttavia, grazie alla persistenza di una singola azienda, Qualcomm, il CDMA ha avuto successo come sistema 2G (IS-95) ed è maturato al punto da diventare la base tecnologica per il 3G.

L'analisi di CDMA nello scenario della telefonia mobile richiede un corredo tecnico maggiore rispetto a quello fornito finora. Nello specifico, abbiamo descritto il CDMA sincrono in cui le sequenze di chip sono esattamente ortogonali. Questo approccio funziona quando tutti gli utenti sono sincronizzati sull'inizio della propria sequenza; ottenere questa sincronizzazione in caso di telefoni mobili indipendenti è molto difficile. Senza i giusti accorgimenti le trasmissioni arriverebbero alla stazione base senza garanzie di ortogonalità. Per poter trasmettere senza sincronizzazione abbiamo bisogno di sequenze di codice che siano ortogonali tra loro indipendentemente dallo sfasamento e non solo quando sono allineate.

Anche se non è possibile trovare delle sequenze esattamente ortogonali per questo caso generale, usando una lunga sequenza pseudocasuale ci si avvicina abbastanza. Queste sequenze hanno con alta probabilità una bassa **correlazione incrociata** (*cross-correlation*) tra loro, indipendentemente dallo sfasamento. Ciò vuol dire che quando una sequenza viene moltiplicata per un'altra sequenza e poi sommata per ottenere il prodotto interno il risultato sarà molto basso (sarebbe zero se fossero davvero ortogonali). Intuitivamente, sequenze casuali dovrebbero essere sempre diverse l'una dall'altra; moltiplicarle tra loro dovrebbe quindi produrre un segnale casuale, che andrebbe a contribuire marginalmente alla somma totale. Questo permette a un ricevitore di filtrare dal segnale ricevuto tutte le trasmissioni indesiderate; inoltre, anche l'**autocorrelazione** di sequenze casuali ha un'alta probabilità di essere piccola, a parte quando le sequenze sono sincronizzate. Ciò vuol dire che se una sequenza viene moltiplicata per una copia ritardata nel tempo di se stessa e poi sommata il risultato sarà piccolo eccetto nel caso in cui il ritardo è nullo. Intuitivamente, una sequenza casuale con ritardo equivale a una sequenza diversa e si torna al caso della correlazione incrociata. Ciò permette a un ricevente di sincronizzarsi all'inizio della trasmissione desiderata all'interno del segnale ricevuto.

L'uso di sequenze pseudocasuali permette alla stazione base CDMA di ricevere messaggi da apparati mobili non sincronizzati; tuttavia, è stato assunto implicitamente che i livelli di potenza di tutti i dispositivi mobili siano gli stessi del ricevente; se non lo sono, una piccola correlazione incrociata con un segnale potente potrebbe avere la meglio su una grande autocorrelazione con un segnale debole. Per questo motivo, la potenza trasmissiva sui dispositivi mobili deve essere gestita per minimizzare le interferenze tra segnali in competizione tra loro, perché è questa interferenza che limita la capacità dei sistema CDMA.

La potenza del segnale che raggiunge la stazione base dipende non solo dall'energia usata dai trasmittenti, ma anche dalla loro distanza. Poiché potrebbero esserci molte stazioni mobili a distanze diverse dalla stazione base, una buona euristica per uniformare l'energia ricevuta può essere che ogni stazione mobile trasmetta con un'energia che è l'inverso di quella con cui riceve il segnale dalla stazione base. Per aumentare la precisione la stazione base restituisce anche delle indicazioni su aumento, diminuzione o mantenimento dell'energia di trasmissione. Queste segnalazioni sono frequenti (1500 volte al secondo) in quanto un buon controllo energetico è fondamentale per minimizzare le interferenze.

Un altro miglioramento rispetto al CDMA descritto prima è quello di permettere a utenti diversi di mandare dati a velocità distinte. Questo accorgimento si adotta facilmente con CDMA fissando il tasso a cui vengono trasmesse le chip e assegnando agli utenti sequenze di chip di lunghezze differenti. Ad esempio, in WCDMA il tasso di invio è di 3,84 Mchip/s e i codici usano da 4 a 256 chip. Un codice a 256 chip può trasportare, dopo aver effettuato la correzione d'errore, 12 kbps (sufficiente per una chiamata vocale), mentre in un codice a 4 chip il tasso percepito dall'utente è quasi 1 Mbps. Codici con lunghezze intermedie portano a tassi di trasmissione intermedi; per ottenere una banda multipla di Mbps il terminale mobile deve usare più di un canale da 5 MHz contemporaneamente.

Dopo esserci occupati di come far funzionare CDMA passiamo ora a descrivere i suoi tre vantaggi principali. Per prima cosa è in grado di aumentare la capacità trasmissiva sfruttando i brevi periodi in cui i trasmettitori sono silenziosi. Nelle normali telefonate si parla a turno e la linea è occupata solo per il 40% del tempo. Tuttavia, le pause potrebbero essere piccole e difficili da prevedere, con TDM o FDM non è possibile riassegnare gli intervalli di tempo o canali di frequenza abbastanza velocemente da sfruttare queste piccole pause. Con CDMA il semplice fatto che un utente non trasmetta abbassa l'interferenza a cui sono soggetti gli altri utenti.

Secondo, con CDMA ogni cella usa le stesse frequenze; diversamente da GSM e AMPS, non è necessario usare FDM per separare le trasmissioni di utenti diversi. Questo elimina una complicata pianificazione delle frequenze e aumenta la capacità trasmissiva. È quindi anche più facile per una stazione base usare delle **antenne direzionali** (o *antenne settoriali*) invece di una sola omnidirezionale. Le antenne direzionali concentrano il segnale solo in una direzione e permettono di ridurre le interferenze nelle altre direzioni; questo porta anche a un aumento della capacità trasmissiva; normalmente queste antenne vengono progettate facendo uso di tre settori. La stazione base deve tenere traccia di un dispositivo mobile che cambia settore; questo è facile con CDMA, perché tutte le frequenze sono usate in tutti i settori.

Terzo, CDMA facilita il **soft handoff**, in cui il dispositivo mobile viene acquisito dalla nuova stazione base prima che quella precedente lo sganci; in questo modo non c'è perdita di continuità. Il meccanismo di soft handoff è mostrato nella Figura 2.49. Anche in questo caso il fatto di usare tutte le frequenze in ogni cella facilita il lavoro. L'alternativa è l'**hard handoff**, in cui la vecchia stazione base sgancia la chiamata prima che quella nuova la acquisisca; se la nuova stazione base non è in grado di prendersi cura della chiamata (perché, ad esempio, non ci sono frequenze disponibili) la telefonata cessa senza preavviso. L'hard handoff è la norma con FDM, perché evita al dispositivo mobile di trasmettere o ricevere su due frequenze simultaneamente.

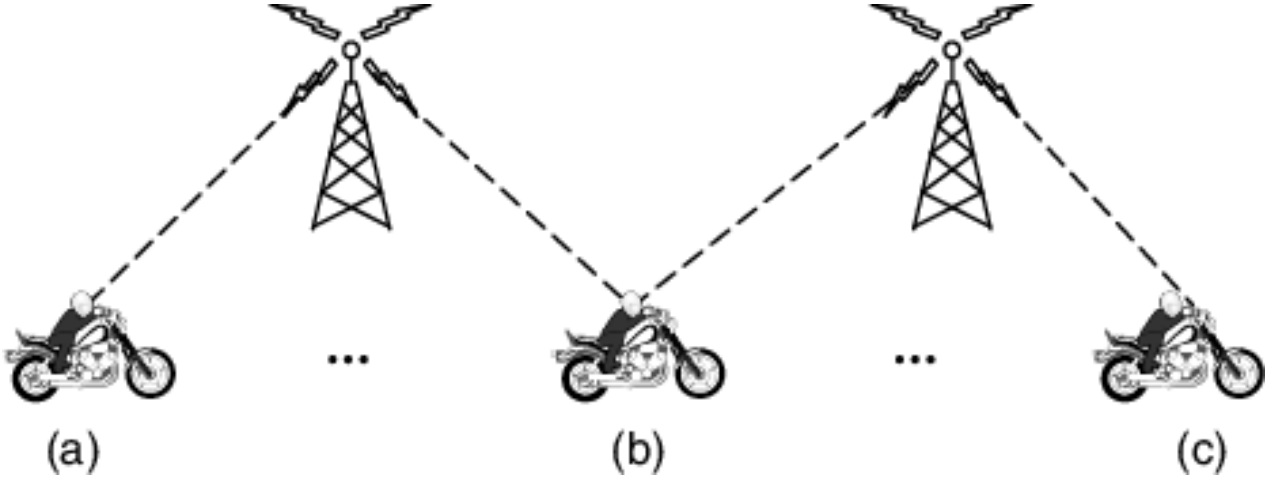


Figura 2.49 Soft handoff (a) prima, (b) durante, e (c) dopo.

Molto è stato scritto a proposito di 3G, dichiarandolo una delle migliori cose al mondo; contemporaneamente molti operatori si sono cautamente avviati verso il 3G attraverso quello che viene qualche volta definito **2.5G** (anche se il termine 2.1G potrebbe essere più preciso). Tra i sistemi 2.5G troviamo **EDGE** (*enhanced data rates for GSM*), che è solo un GSM con più bit per simbolo. Il problema è che più bit per simbolo implicano più errori per simbolo, per cui EDGE possiede nove diversi sistemi di modulazione e correzione degli errori che differiscono tra loro per la frazione di banda destinata alla correzione degli errori dovuti all'aumento di velocità. EDGE è un passo lungo un percorso evolutivo da GSM verso WCDMA; allo stesso modo c'è un percorso evolutivo per gli operatori per passare da reti IS-95 a CDMA2000.

Anche se le reti 3G non sono state ancora completamente installate, i ricercatori considerano il loro compito completato, infatti, stanno già lavorando a sistemi 4G sotto il nome di **LTE** (*long term evolution*).[7] Alcune delle caratteristiche proposte per 4G includono larga banda, ubiquità (connettività ovunque), integrazione senza soluzione di continuità con reti basate su IP (sia wireless che non), gestione dinamica delle risorse e dello spettro e un'alta qualità del servizio per i contenuti multimediali. Per maggiori dettagli si veda Astely et al. (2009) e Larmo et al. (2009). Nel frattempo, reti wireless con prestazioni a livello di 4G sono già disponibili. L'esempio principale è **802.16**, conosciuto anche come **WiMAX**. Per una panoramica su WiMAX mobile si veda Ahmadi (2009). Dire che l'industria sia in uno stato di continuo cambiamento sarebbe minimizzare. Tornate a controllare tra qualche anno per vedere che cosa è successo.

2.8 Televisione via cavo

I sistemi telefonici fissi e cellulari esaminati in dettaglio nei precedenti paragrafi giocano chiaramente un ruolo importante nella comunicazione via rete del futuro. Esiste però una rete fissa alternativa, che ultimamente sta acquisendo maggiore importanza: la televisione via cavo. Molte persone già telefonano e accedono ai servizi Internet adoperando il cavo, e gli operatori di questo settore stanno lavorando attivamente per aumentare la loro fetta di mercato. I paragrafi seguenti descrivono dettagliatamente il sistema di comunicazione di rete basato sulla televisione via cavo, confrontandolo con i sistemi telefonici appena studiati. Alcune fonti di informazione rilevanti sull'argomento sono Donaldson e Jones (2001), Dutta-Roy (2001) e Fellows e Jones (2001).

Televisione ad antenna collettiva

La televisione via cavo fu inventata alla fine degli anni '40 per offrire una migliore ricezione agli utenti che vivevano in aree rurali e montane. Il sistema inizialmente consisteva in una grande antenna collocata sulla cima di una collina che captava il segnale televisivo trasmesso attraverso l'etere, un amplificatore chiamato **headend** (amplificatore) utilizzato per rinforzare il segnale e un cavo coassiale che distribuiva il segnale nelle case degli utenti (Figura 2.50).

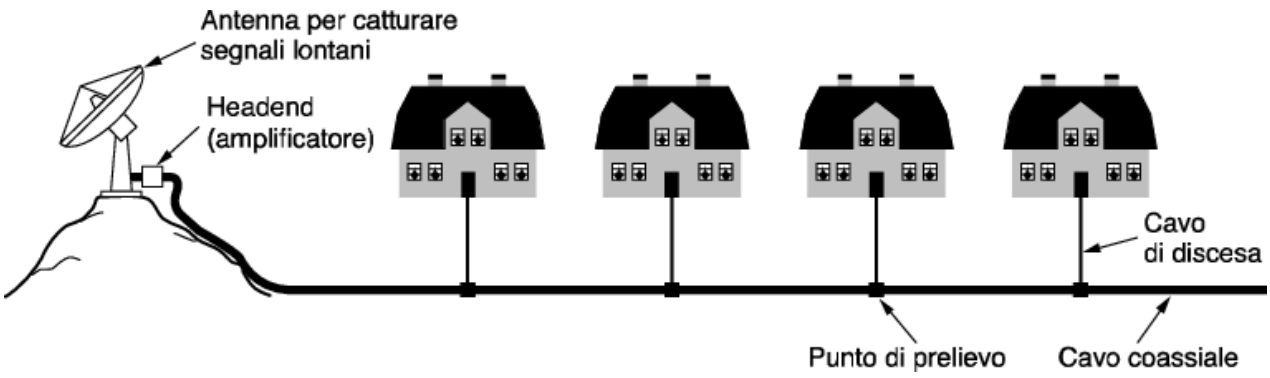


Figura 2.50 Uno dei primi sistemi di televisione via cavo.

Nei primi anni, la televisione via cavo era chiamata televisione ad antenna collettiva. Si trattava essenzialmente di un'operazione "fatta in casa": chiunque, con un po' di esperienza di elettronica, poteva attivare il servizio nella sua città; gli utenti finali avrebbero contribuito a pagare i costi di attivazione. Quando il numero di abbonati aumentò, al cavo originale furono collegati nuovi cavi e altri amplificatori vennero aggiunti al sistema. La trasmissione era unidirezionale, dall'headend agli utenti. All'inizio degli anni '70 esistevano migliaia di sistemi indipendenti.

Nel 1974 Time Inc. creò un nuovo canale, chiamato Home Box Office, che distribuiva film recenti solo via cavo. Seguirono altri canali trasmessi solo via cavo dedicati a notizie, sport, cucina e molti altri argomenti. Questo sviluppo diede il via a due cambiamenti nell'industria. Primo, grandi società iniziarono ad acquistare i sistemi via cavo esistenti e a posare nuovi cavi per acquisire nuovi abbonati. Secondo, per poter distribuire i nuovi canali via cavo sorse la necessità di collegare sistemi diversi, spesso situati in città distanti. Le società iniziarono a stendere cavi tra le loro città per collegarle a un unico sistema, secondo uno schema analogo a quello adottato dall'industria dei telefoni 80 anni prima, quando il collegamento degli uffici locali prima isolati aveva reso possibili le chiamate a lunga distanza.

Internet via cavo

Nel corso degli anni, il sistema via cavo crebbe e i cavi tra le varie città furono sostituiti da fibre a elevata ampiezza di banda, proprio come era successo nel sistema telefonico. Un sistema che usa la fibra per tratte più lunghe e i cavi coassiali per le abitazioni degli utenti è chiamato **HFC** (*hybrid fiber coax*). I convertitori elettro-ottici che fungono da interfacce tra le parti elettriche e quelle ottiche del sistema sono chiamati **fiber node**. Poiché l'ampiezza di banda della fibra è molto più elevata di quella del cavo coassiale, un fiber node può alimentare diversi cavi coassiali. La Figura 2.51(a) mostra una parte di un moderno sistema HFC.

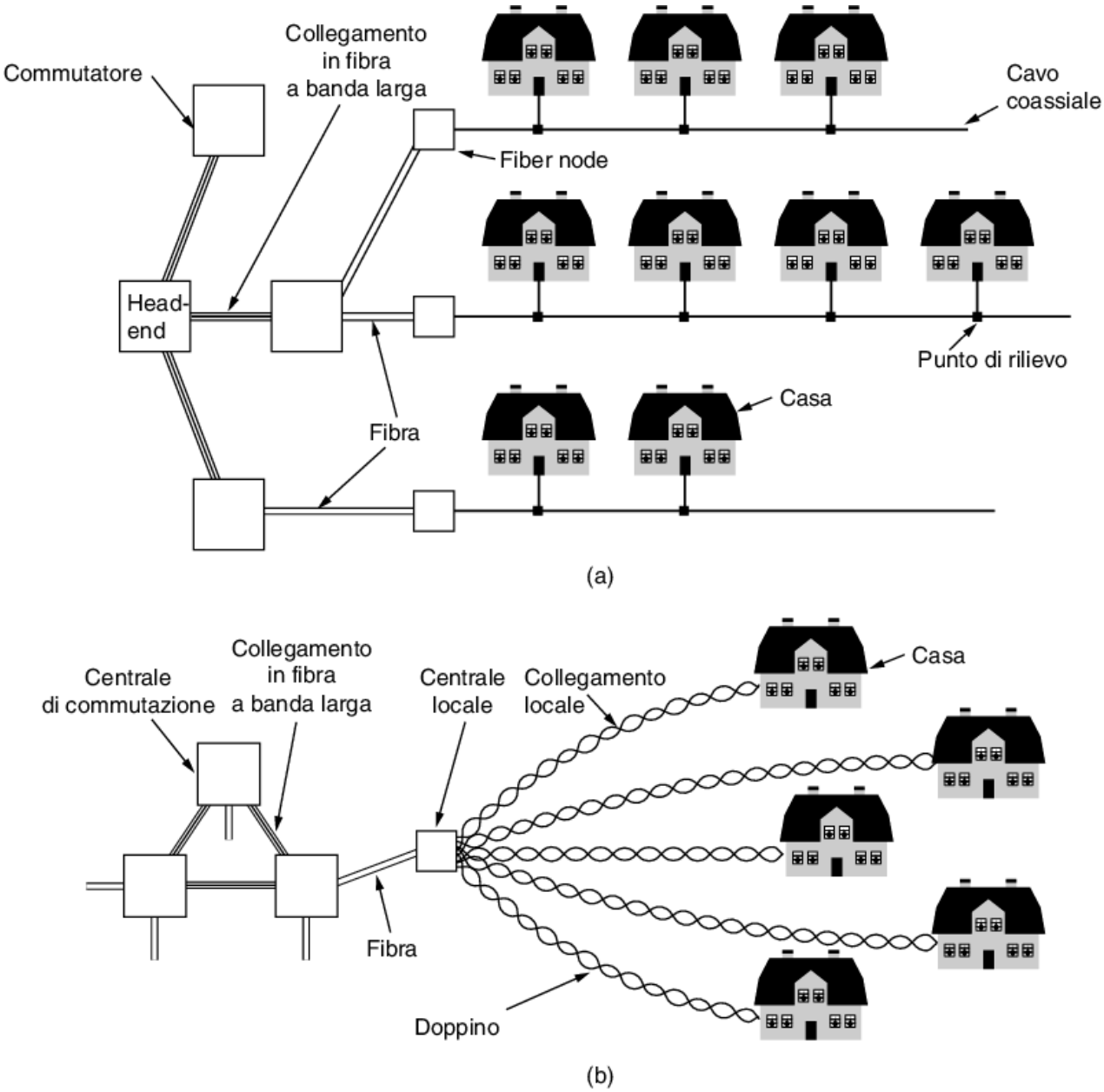


Figura 2.51 (a) Televisione via cavo. (b) Sistema telefonico su rete fissa.

Nell'ultimo decennio molti operatori via cavo hanno deciso di entrare nel settore dell'accesso Internet e spesso anche in quello telefonico, ma le differenze tecniche tra l'impianto via cavo e quello telefonico influenzano ciò che deve essere fatto per raggiungere questi obiettivi. Tanto per cominciare, tutti gli amplificatori unidirezionali usati nel sistema dovevano essere sostituiti con amplificatori bidirezionali per supportare trasmissioni sia upstream sia downstream. Parallelamente, i primi sistemi per la connessione a Internet su cavo usavano la rete televisiva per la trasmissione downstream e la rete telefonica per la trasmissione upstream tramite una connessione via modem. Era una soluzione intelligente, ma non una gran rete rispetto a quella che avrebbe potuto essere.

Tra il sistema HFC mostrato nella Figura 2.51(a) e il sistema telefonico mostrato nella Figura 2.51(b) esiste tuttavia un'altra grande differenza, molto più difficile da superare. Presso le abitazioni degli utenti un singolo cavo è condiviso tra più case, mentre nel sistema telefonico ogni casa ha il suo collegamento locale privato. Quando è utilizzata per la trasmissione televisiva, questa

condivisione non ha importanza: tutti i programmi sono trasmessi via cavo e non fa alcuna differenza se gli spettatori siano 10 oppure 10.000. Se però lo stesso cavo è utilizzato per accedere a Internet, il numero di utenti fa la differenza; se un utente decide di scaricare un file di grandi dimensioni, quella banda viene potenzialmente sottratta a tutti gli altri utenti. Al crescere del numero degli utenti aumenta la contesa per l'ampiezza di banda. Il sistema telefonico non ha questa proprietà: scaricare un file molto grande attraverso una linea ADSL non riduce l'ampiezza di banda del vicino; d'altra parte, l'ampiezza di banda di un cavo coassiale è molto più grande di quella del doppino telefonico; quindi è una fortuna avere dei vicini che non usano molto Internet.

L'industria della televisione via cavo ha risolto questo problema dividendo i cavi lunghi e collegandoli direttamente ai fiber node. L'ampiezza di banda dalla terminazione principale al fiber node si può considerare infinita, perciò la quantità di traffico è gestibile se non ci sono troppi abbonati su ogni segmento di cavo. Attualmente ogni cavo collega 500–2.000 abitazioni, ma se il numero degli abbonati al servizio Internet via cavo aumenterà sarà necessario incrementare il numero di ripartizioni e di fiber node.

Allocazione dello spettro

Sopprimere tutti i canali televisivi e utilizzare l'infrastruttura basata sui cavi per accedere esclusivamente a Internet avrebbe potuto generare un discreto numero di clienti adirati, perciò le aziende hanno indugiato a farlo. Inoltre, poiché la maggior parte delle città disciplina pesantemente il contenuto trasmesso via cavo, gli operatori non avrebbero potuto farlo neanche se avessero voluto: le società del settore hanno dovuto trovare il modo di far coesistere sullo stesso cavo televisione e Internet.

La soluzione è adottare FDM. I canali televisivi via cavo in Nord America normalmente occupano la banda 54–550 MHz (con l'eccezione dell'intervallo 88–108 MHz dedicato alla radio FM). Questi canali sono ampi 6 MHz, includendo anche le bande di guardia. In Europa il limite inferiore di solito è 65 MHz, e i canali sono ampi 6–8 MHz a causa della più alta risoluzione richiesta dal sistema PAL e SECAM; a parte queste differenze lo schema di allocazione è simile a quello statunitense. La parte bassa della banda non è utilizzata. I cavi moderni possono anche operare bene sopra i 550 MHz, spesso a 750 MHz o più; la soluzione adottata, perciò, fu di introdurre canali upstream nella banda 5–42 MHz (un po' più alta in Europa) e utilizzare le frequenze più alte per la ricezione dei dati. La Figura 2.52 mostra lo spettro finale.

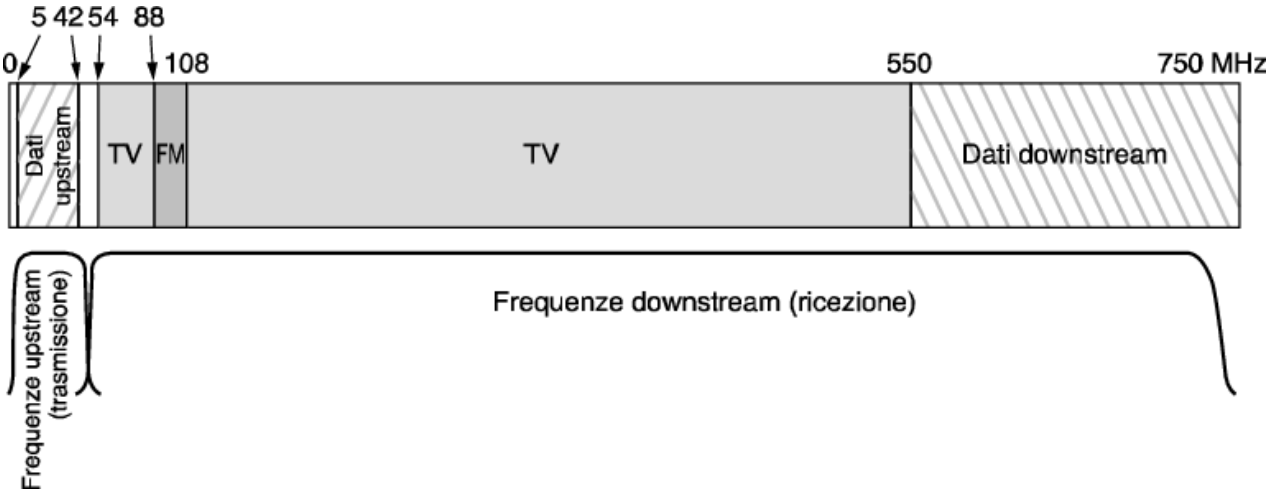


Figura 2.52 Assegnazione delle frequenze in un sistema televisivo via cavo usato per accedere a Internet.

Poiché i segnali televisivi sono tutti in ricezione, è possibile utilizzare amplificatori di trasmissione che funzionano solo nella banda 5-42 MHz e amplificatori di ricezione che funzionano solo sopra i 54 MHz, come mostrato nella figura. Di conseguenza si ottiene una asimmetria tra le ampiezze di banda in trasmissione e ricezione perché sopra il segnale televisivo è disponibile più spettro che non sotto di esso. D'altra parte, poiché la maggior parte del traffico è diretta verso gli utenti, gli operatori apprezzano questa situazione. Com'è stato spiegato precedentemente, le aziende telefoniche di solito offrono un servizio DSL asimmetrico, anche se non c'è alcun motivo tecnico per farlo.

Oltre ad aggiornare gli amplificatori, l'operatore deve aggiornare anche l'headend, sostituendo l'amplificatore passivo con un sistema di elaborazione digitale intelligente, dotato di un'interfaccia per fibra ad alta ampiezza di banda collegata a un ISP. Spesso anche il nome viene aggiornato, da headend si passa così a **CMTS** (*cable modem termination system*). Nel testo che segue ci tratterremo dal fare un aggiornamento del nome e ci atterremo al termine tradizionale "headend".

Cable modem

Per accedere a Internet è necessario utilizzare un **cable modem** (*modem via cavo*), dispositivo che ha due interfacce: una verso il computer e l'altra verso la rete via cavo. Nei primi anni dell'era dell'accesso a Internet via cavo ogni operatore adoperava un modem via cavo proprietario che veniva installato da un tecnico dell'azienda, ma presto fu evidente che uno standard aperto avrebbe creato un mercato competitivo e avrebbe fatto diminuire i prezzi dei dispositivi, incoraggiando l'uso del servizio. Inoltre, permettendo ai clienti di acquistare il modem via cavo nei negozi e d'installarlo da soli (come avviene per gli access point wireless) sarebbe stato possibile eliminare la spesa dell'intervento del tecnico.

I più importanti operatori del settore cooperarono con una società chiamata CableLabs per produrre un modem via cavo standard, per testare i prodotti e renderli conformi. Questo standard, chiamato **DOCSIS** (*data over cable service interface specification*), ha sostituito la maggior parte dei modem proprietari. La versione 1.0 di DOCSIS si è resa disponibile nel 1997 ed è stata presto seguita da DOCSIS 2.0 nel 2001; la seconda versione ha aumentato la velocità di upstream per supportare meglio servizi con trasferimento dati simmetrico come la telefonia su IP. La versione più recente è DOCSIS 3.0, disponibile dal 2006, che usa più banda per aumentare la velocità in entrambe le direzioni. La versione europea si chiama **EuroDOCSIS**. Non a tutti gli operatori è piaciuta l'idea dello standard, anche perché il noleggio dei modem proprietari rendeva parecchi soldi. Uno standard aperto con decine di produttori che vendono modem via cavo in negozio pone fine a questa pratica redditizia. L'interfaccia modem/computer è semplice: di solito si usa Ethernet a 10 Mbps e qualche volta USB. L'altro capo è più complicato, in quanto usa sia FDM che TDM che CDMA per dividere la banda disponibile tra gli abbonati.

Quando un cable modem è collegato e acceso, esso scansiona i canali in downstream per cercare un particolare pacchetto inviato periodicamente dall'headend e contenente i parametri di configurazione. Quando trova questo pacchetto il nuovo modem annuncia la sua presenza su uno dei canali in upstream. L'headend risponde assegnando al modem i suoi canali in upstream e downstream. Questa assegnazione può anche essere cambiata in seguito per motivi di bilanciamento del carico di rete.

L'uso dei canali a 6 MHz o 8 MHz è la parte relativa a FDM. Ogni cable modem utilizza un canale in downstream e uno in upstream (o più canali nel caso di DOCSIS 3.0). L'approccio classico è di prendere ognuno dei canali in downstream da 6 (o 8) MHz e modularli con QAM-64 o, se la qualità del cavo è particolarmente buona, QAM-256. Con un canale a 6 MHz e QAM-64 otteniamo circa 36 Mbps; escludendo l'overhead rimangono circa 27 Mbps. Con QAM-256 la capacità al netto dell'overhead sarebbe di 39 Mbps. I valori europei sono maggiori di un terzo.

Il traffico in upstream è soggetto a più rumore, in quanto il sistema non è stato originariamente progettato per i dati e i disturbi provenienti dagli abbonati sono incanalati verso l'headend; quindi viene adottato un approccio più prudente, che varia da QPSK a QAM-128, in cui alcuni dei simboli sono usati per correggere gli errori con TCM. Con meno bit per simbolo in upstream l'asimmetria dei due canali è molti più marcata di come sia visibile nella Figura 2.52.

TDM viene poi usato per fare condividere la banda in downstream tra più abbonati, altrimenti le loro trasmissioni potrebbero collidere all'headend. Il tempo viene diviso in intervalli più piccoli chiamati **minislot** e abbonati distinti inviano dati in minislot diversi. Per poter fare questo, il modem determina la sua distanza dall'headend spedendo un pacchetto di segnalazione e misurando il tempo di risposta. Questo processo è chiamato **ranging** (*allineamento*). Determinare la distanza consente al modem di adattare il modo in cui operano i canali in trasmissione e regolare la sincronizzazione. Ogni pacchetto trasmesso sui canali upstream deve occupare uno o più minislot consecutivi. L'headend annuncia periodicamente l'inizio di un nuovo ciclo di minislot, ma il via libera non viene ricevuto simultaneamente da tutti i modem, a causa del tempo di propagazione attraverso il cavo. Una volta a conoscenza della distanza dall'headend, ogni modem può calcolare l'istante in cui inizia il primo minislot. La lunghezza del minislot dipende dalla rete e solitamente contiene 8 byte.

Durante l'inizializzazione, l'headend assegna a ogni modem un minislot da utilizzare per richiedere ampiezza di banda in trasmissione. Quando vuole inviare un pacchetto, il computer trasferisce il pacchetto al modem che poi richiede il numero di minislot necessario. Se la richiesta viene accettata, l'headend emette un acknowledgement sul canale di ricezione per indicare al modem quali minislot sono stati riservati al suo pacchetto. Quindi il pacchetto viene trasmesso, iniziando con il minislot assegnato; utilizzando un campo dell'intestazione si possono richiedere altri pacchetti.

Di norma a modem distinti verrà assegnato lo stesso minislot, con conseguente contesa sulle risorse. Ci sono due modi per risolvere questo problema. Il primo è che CDMA venga usato per far condividere i minislot fra gli abbonati; questo risolve il problema della contesa perché tutti gli abbonati possono inviare contemporaneamente, anche se a una velocità più bassa. La seconda soluzione prevede che CDMA non venga utilizzato per nulla; in tal caso potrebbe non esserci conferma alla richiesta a causa di una collisione. A fronte di una collisione il modem aspetta per un tempo casuale e riprova; dopo ogni fallimento consecutivo il tempo massimo casuale viene raddoppiato. (Per i lettori già pratici con le tecnologie di rete questo algoritmo è slotted ALOHA con back off esponenziale binario. Ethernet non può essere usata su questo tipo di collegamento perché le stazioni non possono rilevare l'utilizzo del mezzo trasmissivo. Torneremo su questo argomento nel Capitolo 4).

I canali in ricezione sono gestiti in modo diverso. Tanto per cominciare c'è un unico trasmettitore (l'headend), perciò non c'è alcuna contesa e i minislot sono inutili, c'è solo un multiplexing statistico a divisione di tempo. Inoltre, il traffico in arrivo di solito è molto più grande di quello in uscita, perciò si utilizza un pacchetto di dimensione fissa grande 204 byte. Parte di questo pacchetto contiene il codice per la correzione degli errori Reed-Solomon e altri codici di controllo; alla fine, il payload ha a disposizione 184 byte. Questi numeri sono stati scelti per compatibilità con la televisione digitale che utilizza MPEG-2, perciò i canali televisivi e quelli downstream dedicati ai dati adottano lo stesso formato. La Figura 2.53 mostra le connessioni dal punto di vista logico.

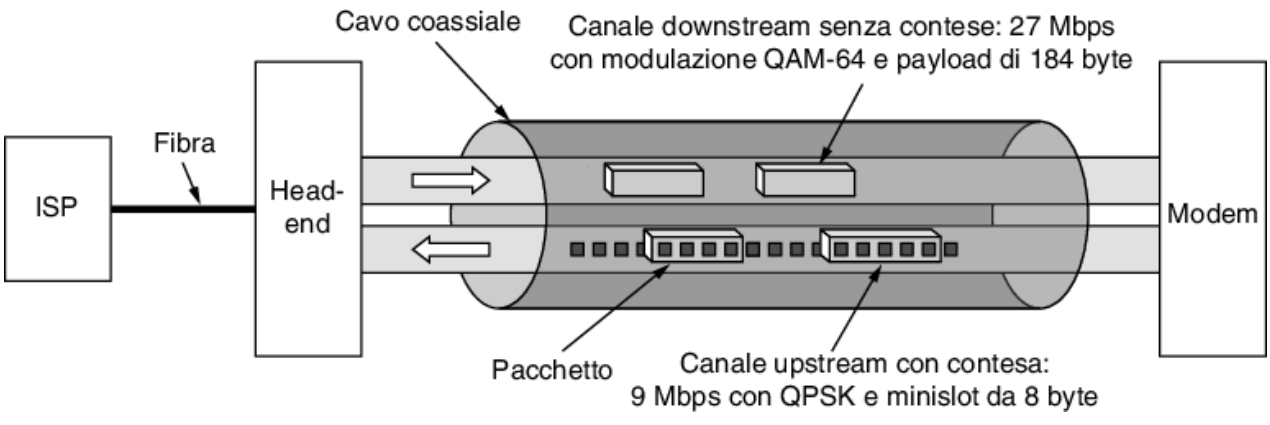


Figura 2.53 Dettagli tipici dei canali di trasmissione e ricezione in Nord America.

ADSL o connessione via cavo?

Qual è la soluzione migliore, ADSL o la connessione via cavo? È un po' come chiedere di scegliere il sistema operativo, il linguaggio o la religione migliore. Tutto dipende da a chi lo chiediamo. Sia ADSL sia la connessione via cavo utilizzano la fibra ottica nelle dorsali, la differenza è nel tratto finale: l'accesso via cavo utilizza il cavo coassiale, ADSL usa il doppino telefonico. La capacità di trasporto teorica del cavo coassiale è centinaia di volte maggiore di quella del doppino, ma l'intera capacità del cavo non è a disposizione dei dati degli utenti, perché una parte dell'ampiezza di banda è dedicata ad applicazioni inutili, come i programmi televisivi.

In pratica è difficile generalizzare sulla capacità effettiva. I fornitori di servizi ADSL danno indicazioni precise sull'ampiezza di banda (per esempio, 1 Mbps in ricezione e 256 kbps in trasmissione) e in genere riescono a raggiungere l'80% delle velocità indicate. I fornitori di servizi via cavo possono limitare la banda del singolo utente per poter fare delle previsioni, in realtà non possono dare alcuna garanzia, perché la capacità effettiva dipende dal numero di persone attive sul segmento di cavo dell'utente. Qualche volta si ottengono velocità migliori di ADSL, altre volte invece no. Quello che dà veramente fastidio, comunque, è l'imprevedibilità del servizio: un istante di ottimo servizio non garantisce un servizio altrettanto eccellente in quello successivo, soprattutto se il più grande "intasatore" di banda della città ha appena acceso il computer.

Man mano che una rete ADSL attira utenti, il numero crescente di nuovi abbonati influenza molto poco gli utenti esistenti, poiché ognuno ha una connessione dedicata. Nel caso del modem via cavo, invece, le prestazioni sono inversamente proporzionali al numero di abbonati. Per limitare questo problema è necessario che l'operatore divida i cavi con maggior traffico e colleghi ogni segmento direttamente al fiber node. Purtroppo ciò richiede tempo e denaro e spesso non viene fatto.

Come digressione, nei paragrafi precedenti è stato studiato un altro sistema basato su un canale condiviso simile al cavo: il sistema telefonico mobile. Anche in questo caso un gruppo di utenti, che potrebbero essere chiamati "compagni di cella", condividono una quantità fissa di ampiezza di banda. Normalmente essa è divisa rigidamente in blocchi fissi da FDM e TDM tra gli utenti attivi, perché il traffico vocale è abbastanza uniforme. Ma nel caso del traffico dati, questa divisione rigida è molto inefficiente, perché gli utenti spesso rimangono inattivi, facendo così sprecare l'ampiezza di banda loro riservata; da questo punto di vista l'accesso via cavo assomiglia più al sistema telefonico mobile che a quello fisso.

Essendo un mezzo di trasmissione punto a punto, ADSL è intrinsecamente più sicuro del modem via cavo. Qualunque utente collegato al cavo televisivo può facilmente leggere tutti i pacchetti trasmessi attraverso il cavo; per questo motivo la maggior parte dei fornitori di servizi via cavo codifica tutto il traffico dati in entrambe le direzioni. Tuttavia, la possibilità che il vicino di casa possa intercettare i vostri messaggi codificati è sempre meno sicura che se non intercettasse niente di niente.

Il sistema telefonico è generalmente più affidabile di quello via cavo. Per esempio, dispone di un alimentazione di riserva e continua a funzionare normalmente anche durante le interruzioni di corrente. Con il cavo, se uno degli amplificatori lungo la catena si spegne, tutti gli utenti in ricezione sono istantaneamente tagliati fuori.

Infine, la maggior parte dei fornitori di servizi ADSL permette di scegliere tra diversi ISP (qualche volta è proprio la legge che impone questo comportamento); lo stesso non sempre accade con gli operatori via cavo.

La conclusione è che ADSL e il cavo sono molto più simili che diversi: offrono servizi comparabili e, al crescere della concorrenza tra i due, probabilmente anche i prezzi convergeranno.

2.9 Sommario

Il livello fisico è la base di tutte le reti. La natura impone due limiti fondamentali su tutti i canali, che determinano la loro ampiezza di banda: il limite di Nyquist, che si occupa dei canali senza rumore, e il limite di Shannon, che si occupa di canali che ne sono soggetti.

I mezzi trasmissivi possono essere vincolati o meno. I principali mezzi di trasmissione vincolati sono il doppino telefonico, il cavo coassiale e la fibra ottica. I mezzi non vincolati includono le onde radio, le microonde, i raggi infrarossi e i laser trasmessi attraverso l'aria e i satelliti.

Le tecniche di modulazione digitale inviano dati sotto forma di segnali analogici. I codici di linea operano in banda base e i segnali possono essere trasmessi in banda passante modulando ampiezza, frequenza e fase di una portante. I canali possono essere condivisi tra gli utenti usando multiplexing a divisione di tempo, frequenza e codice.

Un elemento chiave nella maggior parte delle reti geografiche è il sistema telefonico. I suoi componenti principali sono l'ultimo miglio, i trunk e i commutatori. L'ADSL offre velocità fino a 40 Mbps sull'ultimo miglio facendo uso di molte sottoportanti in parallelo e superando di gran lunga la capacità dei modem telefonici. PON porta le fibre ottiche fino in casa dell'utente per un accesso ancora più veloce di ADSL.

I trunk trasportano informazioni digitali sfruttando il multiplexing con WDM per fornire servizio a molti collegamenti ad alta capacità su fibre ottiche. È possibile usare anche TDM per far condividere agli utenti i collegamenti ad alta capacità. Entrambe le commutazioni di circuito e pacchetto sono importanti.

Il sistema telefonico fisso non è adatto alle applicazioni mobili; oggi si usano ovunque i telefoni cellulari principalmente per trasmettere la voce, ma sempre di più per scambiare dati. Questi, in particolare, hanno attraversato tre generazioni. La prima generazione, 1G, era analogica e principalmente rappresentata da AMPS. 2G era digitale e GSM è attualmente il sistema di telefonia mobile più diffuso al mondo. 3G è digitale e fa uso di CDMA con WCDMA e CDMA2000 attualmente in fase di installazione.

Un sistema alternativo di accesso alle reti è il sistema televisivo via cavo, che si è evoluto gradualmente da antenna collettiva a ibrido fibra/coassiale, e dalla televisione a televisione e Internet. Potenzialmente offre un'ampiezza di banda molto grande, ma in realtà questa dipende pesantemente dal numero degli utenti in quanto condivisa.

Problemi

1. Calcolate i coefficienti di Fourier per la funzione $f(t) = t$ dove $0 \leq t \leq 1$.

2. Un canale a 4 kHz senza rumore è campionato ogni 1 ms. Qual è il massimo tasso di invio dei dati? Come cambia questo tasso se il canale è soggetto a rumore con un rapporto segnale-rumore di 30 dB?

3. I canali televisivi sono larghi 6 MHz. Quanti bit al secondo è possibile inviare se si utilizzano segnali digitali a quattro livelli? Si assuma che il canale sia senza rumore.

4. Un segnale binario è trasmesso attraverso un canale a 3 kHz il cui rapporto segnale rumore è di 20 dB; qual è la velocità dati massima che si può raggiungere?

5. Quale rapporto segnale rumore consente di inserire una portante T1 su una linea a 50 kHz?

6. Quali sono i vantaggi delle fibre ottiche rispetto al rame come mezzi trasmissivi? Ci sono degli svantaggi nell'uso delle fibre ottiche rispetto al rame?

7. Quanta ampiezza di banda c'è in una porzione dello spettro ampia 0,1 micron alla lunghezza d'onda di 1 micron?

8. Si desidera inviare attraverso una fibra ottica una sequenza di immagini catturate dallo schermo di un computer. La risoluzione delle immagini è di 2560 × 1600 pixel, ogni pixel occupa 24 bit, e si vogliono inviare 60 immagini al secondo. Quale ampiezza di banda serve? Quanti micron di spettro sono necessari per ottenere questa banda, alla lunghezza d'onda di 1,30 micron?

9. Il teorema di Nyquist vale anche per la fibra ottica o vale solo per il cavo in rame?

10. Le antenne radio spesso funzionano meglio quando il loro diametro è uguale alla lunghezza d'onda. Antenne ragionevoli hanno diametri compresi tra 1 cm e 5 m. Quale intervallo di frequenza è coperto da queste antenne?

11. Un raggio laser largo 1 mm viene puntato su un ricevitore largo 1 mm a 100 metri di distanza sul tetto di un palazzo. Quale deviazione angolare (in gradi) deve avere il laser per mancare il ricevitore ?

12. I 66 satelliti a orbita bassa del progetto Iridium sono divisi in 6 collane attorno alla Terra. Data l'altitudine a cui sono collocati il loro periodo è di 90 minuti. Qual è il tempo medio tra un handoff e l'altro per un trasmittente immobile a terra?

13. Si calcoli il tempo di trasmissione end-to-end per un pacchetto per un sistema di trasmissione satellitare GEO (altitudine: 35.800 km), MEO (altitudine: 18.000 km) e LEO (altitudine: 750 km).

14. Qual è la latenza di una chiamata con origine al Polo Nord e diretta al Polo Sud se questa viene gestita con satelliti Iridium? Si assuma che il tempo di commutazione sui satelliti sia 10 microsecondi e il raggio terrestre di 6371 km.

15. Qual è la minima banda richiesta per ottenere una velocità di trasferimento dati di B bit/s se il segnale è trasmesso usando NRZ, MLT-3 e la codifica Manchester? Motivate la risposta.

16. Si dimostri che nella codifica 4B/5B una transizione di segnale avviene al massimo nel tempo di trasmissione di 4 bit.

17. Quanti prefissi di centrale locale esistevano prima del 1984, quando ogni centrale era identificata dal suo codice di tre cifre unito alle prime tre cifre del numero locale? I prefissi iniziavano con un numero compreso tra 2 e 9, avevano 0 o 1 come seconda cifra e potevano terminare con qualunque cifra. Le prime due cifre di un numero locale erano sempre comprese tra 2 e 9 e la terza cifra poteva essere qualunque numero.

18. Un semplice sistema telefonico è composto da due centrali locali e una singola centrale interurbana, alla quale si collega ogni centrale locale mediante una linea full duplex a 1 MHz. Il telefono è utilizzato mediamente per fare quattro chiamate durante le 8 ore del giorno lavorativo. La durata media di una chiamata è 6 minuti. Dieci chiamate su cento sono interurbane (e quindi passano attraverso la centrale interurbana). Qual è il numero massimo di telefoni che una centrale locale può supportare? (Si supponga di disporre di 4 kHz per circuito). Spiegate perché una compagnia telefonica può decidere di gestire un numero di telefoni minore di questo massimo numero sulla centrale locale.

19. Un'azienda telefonica regionale ha 10 milioni di abbonati. Ogni telefono si collega a una centrale attraverso un doppino. La lunghezza media di questi doppini è di 10 Km. Qual è il valore del rame utilizzato nei collegamenti locali? Si supponga che la sezione di ogni cavo abbia un diametro di 1 mm, che la densità del rame sia di 9 grammi/cm^3 e che il prezzo del rame sia di 6,00 dollari al Kg.

20. L'oleodotto (oil pipeline) è un sistema simplex, half duplex, full duplex o di un altro tipo? E un fiume o una comunicazione via walkie-talkie?

21. Il prezzo di un microprocessore veloce è diminuito a tal punto che oggi è possibile inserirne uno in ogni modem. In che modo questo influenza la gestione degli errori di linea del telefono? Vuol dire che non è più necessario fare rilevamento/correzione di errori a livello 2?

22. Un diagramma a costellazione di un modem simile a quello mostrato nella Figura 2.23 ha punti collocati alle seguenti coordinate: (1, 1), (1, –1), (–1, 1) e (–1, –1). Quanti bps può raggiungere un modem con questi parametri a 1.200 baud?

23. Qual è il massimo tasso di invio dati ottenibile a 1200 baud con un model V.32 standard se non si usa correzione di errore?

24. Quante frequenze utilizza un modem full duplex QAM-64?

25. Dieci segnali, ognuno dei quali richiede 4.000 Hz, sono uniti in multiplexing su un singolo canale utilizzando la tecnica FDM. Qual è l'ampiezza di banda minima richiesta per il canale in multiplexing? Si supponga che le bande di guardia siano ampie 400 Hz.

26. Perché il tempo di campionamento PCM è stato impostato a 125 ms?

27. Qual è il valore percentuale utilizzato per il codice di controllo su una portante T1? Ossia, quale percentuale di 1,544 Mbps non è utilizzata per i dati degli utenti? Come si relaziona con la percentuale di overhead in linee OC-1 o OC-768?

28. Confrontate la velocità dati massima di un canale a 4 kHz senza rumore che utilizza:
(a) la codifica analogica (per esempio QPSK) con 2 bit per campione
(b) il sistema PCM T1.

29. Se cade e perde traccia della sua posizione, un sistema portante T1 tenta di risincronizzarsi utilizzando il primo bit di ogni frame. Quanti frame devono essere esaminati, in media, per effettuare la sincronizzazione con una probabilità di errore dello 0,001?

30. Qual è la differenza, se c'è, tra la parte del modem che si occupa della demodulazione e la parte di un codec che gestisce la codifica? Dopotutto, entrambi i dispositivi convertono segnali analogici in segnali digitali.

31. I clock di SONET hanno una velocità di deriva pari a 1 su 10^9. Quanto tempo ci vuole perché questa deriva diventi uguale alla larghezza di un bit? Vedete qualche implicazione pratica in questo calcolo? Se sì quale?

32. Quanto tempo occorre per trasmettere un file di 1 GB da un VSAT a un altro usando un hub come descritto nella Figura 2.17? Si assuma che l'uplink sia 1 Mbps, il downlink sia 7 Mbps e che si faccia uso di commutazione di circuito con un tempo di creazione del circuito di 1,2 secondi.

33. Si calcoli il tempo di trasmissione nel problema precedente se viene invece usata la commutazione di pacchetto. Si assuma che la dimensione del pacchetto sia 64 KB, il ritardo di commutazione su satellite e hub di 10 microsecondi e l'intestazione del pacchetto sia di 32 byte.

34. Nella Figura 2.40 il tasso di invio dei dati a livello utente di OC-3 è di 148,608 Mbps. Si dimostri come questo numero possa essere ricavato dai parametri di SONET OC-3. Quali saranno i tassi totali, SPE e a livello utente di una linea OC-3072?

35. Per fornire velocità dati più basse di STS-1, SONET usa un sistema di tributari virtuali (VT). Un VT è un payload che può essere inserito in un frame STS-1 ed essere unito con payload utili parziali in modo da riempire il frame. VT1.5 utilizza 3 colonne, VT2 usa 4 colonne, VT3 utilizza 6 colonne e VT6 utilizza 12 colonne di un frame STS-1. Quale VT può soddisfare:
(a) un servizio DS-1 (1,544 Mbps)
(b) un servizio europeo CEPT-1 (2,048 Mbps)
(c) un servizio DS-2 (6,312 Mbps).

36. Qual è l'ampiezza di banda a disposizione di ogni utente in una connessione OC-12c?

37. Si considerino tre reti a commutazione di pacchetto, ognuna contenente *n* nodi. La prima rete ha una topologia a stella con uno switch centrale, la seconda è un anello (bidirezionale) e la terza è totalmente interconnessa, con un cavo che unisce ogni nodo a ogni altro nodo. Qual è il percorso (espresso in salti) di trasmissione medio, qual è il migliore e qual è il peggiore?

38. Si confrontino il ritardo della trasmissione di un messaggio a x bit attraverso un percorso di k salti in una rete a commutazione di circuito e in una rete a commutazione di pacchetto (in una situazione di traffico leggero). Il tempo di setup del circuito è di s secondi, il ritardo di propagazione è di d secondi per salto, la dimensione del pacchetto è di p bit e la velocità dati è di b bps. Sotto quali condizioni la rete a commutazione di pacchetto ha un ritardo più basso? Si spieghi inoltre sotto quali condizioni una rete a commutazione di pacchetto è preferibile a una rete a commutazione di circuito.

39. Si supponga di dover trasmettere x bit di dati utente attraverso un percorso a k salti in una rete a commutazione di pacchetto sotto forma di una serie di pacchetti ognuno contenente p dati e h bit di intestazione, con $x >> p + h$. La velocità della linea è di b bps e il ritardo di propagazione è trascurabile. Quale valore di p minimizza il ritardo totale?

40. In un sistema telefonico mobile tipico con celle esagonali, è vietato riutilizzare una banda di frequenza nelle celle adiacenti. Se sono disponibili 840 frequenze, quante di queste possono essere utilizzate per ogni cella?

41. Il tracciato reale di celle raramente è regolare come quello mostrato nella Figura 2.45; anche la forma delle celle di solito non è regolare. Spiegate uno dei possibili motivi di questa situazione. In che modo la forma irregolare di queste celle influenza l'assegnazione delle frequenze?

42. Fate una stima approssimativa del numero di microcelle PCS di 100 m di diametro necessarie per coprire l'area della città di San Francisco (120 Km^2).

43. Qualche volta, quando un utente mobile attraversa i confini di una cella ed entra in un'altra cella, la chiamata viene interrotta bruscamente anche se tutti i trasmettitori e i ricevitori funzionano perfettamente. Come mai?

44. Si supponga che A, B e C stiano trasmettendo contemporaneamente i bit 0 usando un sistema CDMA con le sequenze di chip rappresentate nella Figura 2.28(a). Qual è la sequenza di chip risultante?

45. Si consideri un modo diverso di osservare la proprietà ortogonale delle sequenze di chip CDMA. Ogni bit in una coppia di sequenze può coincidere o non coincidere. Si esprima l'ortogonalità in termini di coincidenza e non coincidenza.

46. Un ricevitore CDMA riceve i seguenti chip: (–1 +1 –3 +1 –1 –3 +1 +1). Supponendo che le sequenze di chip siano quelle definite nella Figura 2.28(a), quale stazione ha trasmesso e quali bit sono stati trasmessi da ogni stazione?

47. Nella Figura 2.28 ci sono 4 stazioni che possono trasmettere. Supponendo di aggiungere altre 4 stazioni, si ricavi la loro sequenza di chip.

48. La parte terminale del sistema telefonico ha una topologia a stella, con i collegamenti locali che uniscono le abitazioni degli utenti alla centrale locale; invece la televisione via cavo è composta da un singolo cavo che si snoda attraverso le abitazioni degli utenti. Si supponga di utilizzare per la TV una fibra a 10 Gbps al posto del cavo in rame; sarebbe possibile simulare il modello telefonico con ogni utente che dispone di una linea privata diretta verso la centrale? In caso affermativo, quante abitazioni dotate di un singolo telefono potrebbero essere agganciate a una singola fibra?

49. Un'azienda che fornisce servizi via cavo decide di fornire l'accesso a Internet attraverso il cavo in una zona dove abitano 5.000 famiglie. La società utilizza il cavo coassiale e alloca lo spettro per raggiungere un'ampiezza di banda in ricezione di 100 Mbps per cavo. Per attirare clienti, l'azienda decide di garantire a ogni abitazione un'ampiezza di banda minima di 2 Mbps in ricezione in qualunque momento. Si descriva quello di cui l'azienda ha bisogno per soddisfare questa garanzia.

50. In base all'allocazione dello spettro mostrata nella Figura 2.52 e alle informazioni riportate nel testo, quanti Mbps alloca un sistema via cavo al canale di trasmissione e quanti al canale di ricezione?

51. A quale velocità un utente di una rete via cavo riceve i dati se la rete non è occupata? Si assuma che l'interfaccia utente sia:

(a) 10-Mbps Ethernet
(b) 100-Mbps Ethernet
(c) 54-Mbps Wireless.

52. La tecnica di unire in multiplexing diversi flussi di dati STS-1, chiamati tributari, gioca un ruolo importante in SONET. Con un multiplexing di 3:1, tre tributari STS-1 in input sono uniti in un flusso di output STS-3. Questo multiplexing è eseguito byte per byte, ossia i primi tre byte in uscita sono rispettivamente il primo byte del tributario 1, il primo byte del tributario 2 e il primo byte del tributario 3; i successivi tre byte sono i secondi byte dei tre tributari e così via. Scrivete un programma che simuli questo multiplexing 3:1. Il programma deve essere composto da cinque processi. Il processo principale crea quattro processi, tre per i tre tributari STS-1 e uno per il dispositivo che esegue il multiplexing. Ogni processo tributario legge un frame STS-1 prelevando una sequenza di 810 byte da un file di input. I processi inviano i loro frame (byte per byte) al processo che esegue il multiplexing. Quest'ultimo riceve i byte e genera in uscita un frame STS-3 (byte per byte) scritto in output standard. Si utilizzino le pipe per gestire la comunicazione tra processi.

53. Si scriva un programma per implementare CDMA. Si assuma che la lunghezza di una sequenza di chip sia 8 e il numero di stazioni che trasmettono sia 4. Il programma consiste di tre gruppi di processi: 4 processi che trasmettono (t0, t1, t2 e t3), un processo aggregatore e 4 processi che ricevono (r0, r1, r2 e r3). Il programma principale, che svolge anche la funzione di aggregatore, come prima cosa legge dal suo standard input 4 sequenze di chip (in notazione bipolare) e 4 bit (1 bit per ogni processo che trasmette) e poi crea le 4 coppie di processi. A ogni coppia trasmittente/ricevente t0,r0; t1,r1; t2,r2; t3,r3) è assegnata una sequenza di chip e a ogni processo trasmittente è assegnato un bit. Successivamente ogni processo trasmittente calcola il segnale che deve trasmettere (una sequenza di 8 bit) e lo spedisce all'aggregatore. Dopo aver ricevuto i segnali da tutti e quattro i processi trasmittenti l'aggregatore combina i segnali e spedisce il risultato ai 4 processi riceventi. Ogni processo ricevente calcola il bit che ha ricevuto e lo stampa sullo standard output. Si usino delle pipe per la comunicazione tra processi.

CAPITOLO 3
Il livello data link

In questo capitolo studieremo i principi dell'architettura del livello numero 2: il livello *data link.* Discuteremo gli algoritmi per ottenere una comunicazione affidabile ed efficiente tra unità di informazione chiamate *frame* (piuttosto che singoli bit, come nel livello fisico) fra due macchine adiacenti. Con il termine "adiacenti" indichiamo il fatto che le due macchine siano connesse da un canale di comunicazione che agisce concettualmente come un cavo (per esempio un cavo coassiale, una linea telefonica o un canale wireless). La proprietà essenziale che un canale deve possedere per essere simile a un cavo è che i bit vengano consegnati nell'esatto ordine in cui sono stati inviati.

In prima battuta si può pensare che il problema sia così semplice da non richiedere alcuno studio: la macchina *A* mette i bit sul cavo e la macchina *B* li preleva. Sfortunatamente i circuiti di comunicazione a volte commettono degli errori. Inoltre la velocità di trasmissione è finita, quindi esiste un ritardo di propagazione non nullo dal momento in cui i bit vengono inviati a quando vengono ricevuti. Queste limitazioni hanno ripercussioni importanti sull'efficienza del trasferimento dati. I protocolli usati per le comunicazioni, che sono l'argomento del presente capitolo, devono tenere in considerazione tutti questi fattori.

Dopo un'introduzione alle principali caratteristiche strutturali del livello data link, inizieremo il nostro studio dei protocolli esaminando la natura degli errori, le loro cause e come possono essere rilevati e corretti. Studieremo poi una serie di protocolli di complessità crescente e vedremo come ognuno di questi risolva un numero sempre maggiore di problemi presenti nel livello data link. Infine, concluderemo parlando di modelli per lo studio della correttezza dei protocolli e daremo alcuni esempi di protocollo.

3.1 Progettazione del livello data link

Il livello data link fa uso del servizio messo a disposizione dal livello fisico per inviare e ricevere bit su canali di comunicazione.

Ha molte funzioni, tra cui:

1. fornire un'interfaccia di servizio ben definita per il livello di rete
2. gestire gli errori di trasmissione
3. regolare il flusso dati in modo che i buffer di dispositivi riceventi lenti non vengano inondati dai trasmittenti veloci.

Per raggiungere questi obiettivi, il livello data link prende i pacchetti provenienti dal livello di rete e li incapsula in **frame** prima di trasmetterli. Ogni frame contiene una intestazione (*header* o, nel caso specifico, *frame header*), una sezione per contenere il pacchetto proveniente dal livello di rete (*payload field*) e una sequenza di chiusura (*frame trailer*) come illustrato nella Figura 3.1. La gestione dei frame costituisce il cuore dell'attività del livello data link, che studieremo in dettaglio nel paragrafo successivo.

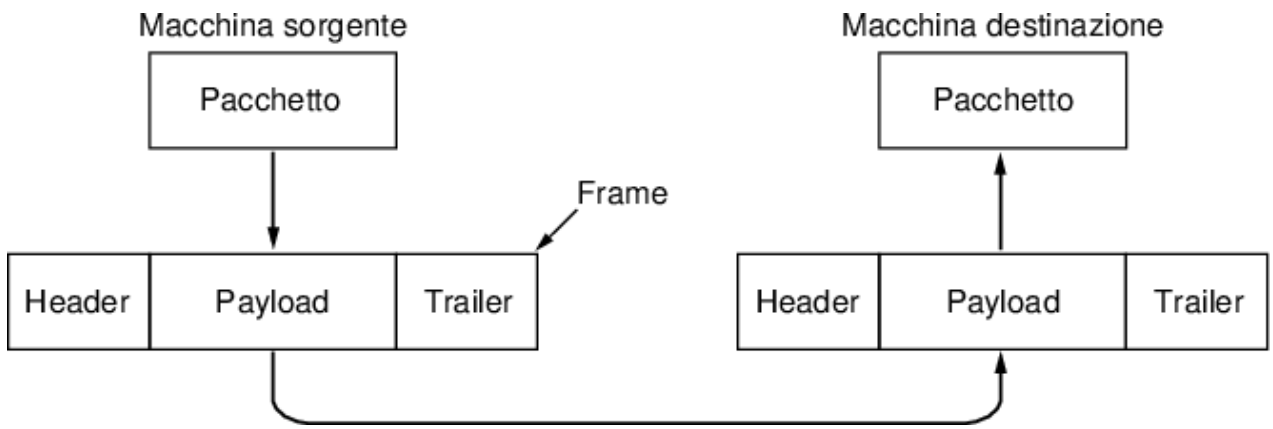


Figura 3.1 Relazione fra pacchetti e frame.

Questo capitolo tratta esplicitamente del livello data link e dei protocolli a esso associati; nonostante ciò, molti dei principi che studieremo (per esempio la correzione degli errori e il controllo di flusso) si ritrovano anche nei protocolli di trasporto così come in altri. L'affidabilità, infatti, viene raggiunta solo quando tutti i livelli lavorano insieme. In effetti, in molte reti questo tipo di funzioni si trova soprattutto nei livelli superiori al data link che, in questo modo, si limita a fare il minimo sforzo necessario. I principi base sono sempre gli stessi, indipendentemente da dove si trovano implementati. Nel livello data link queste funzioni sono spesso implementate nella loro forma più pura e semplice, il che ci motiva a studiarle in questo capitolo.

Servizi forniti al livello di rete

La funzione del livello data link consiste nel fornire servizi al livello di rete. Il servizio principale è quello di trasferire dati dal livello di rete della macchina sorgente al livello di rete della macchina destinazione. Sulla macchina sorgente consideriamo un'entità, che possiamo chiamare processo, localizzata nel livello di rete e che passa alcuni bit al livello data link perché vengano trasmessi alla destinazione. Il lavoro del livello data link è quello di trasmettere i bit alla macchina destinazione, in modo che possano essere passati al livello di rete della destinazione stessa, come mostrato nella Figura 3.2(a). Nella Figura 3.2(b) è rappresentato il percorso reale della trasmissione dei dati. Risulta tuttavia più semplice rappresentare la situazione tramite un modello dove i due processi a livello data link comunicano fra di loro direttamente, tramite un protocollo di tipo data link come mostrato nella Figura 3.2(a). Questo tipo di modello verrà usato implicitamente nel resto di questo capitolo.

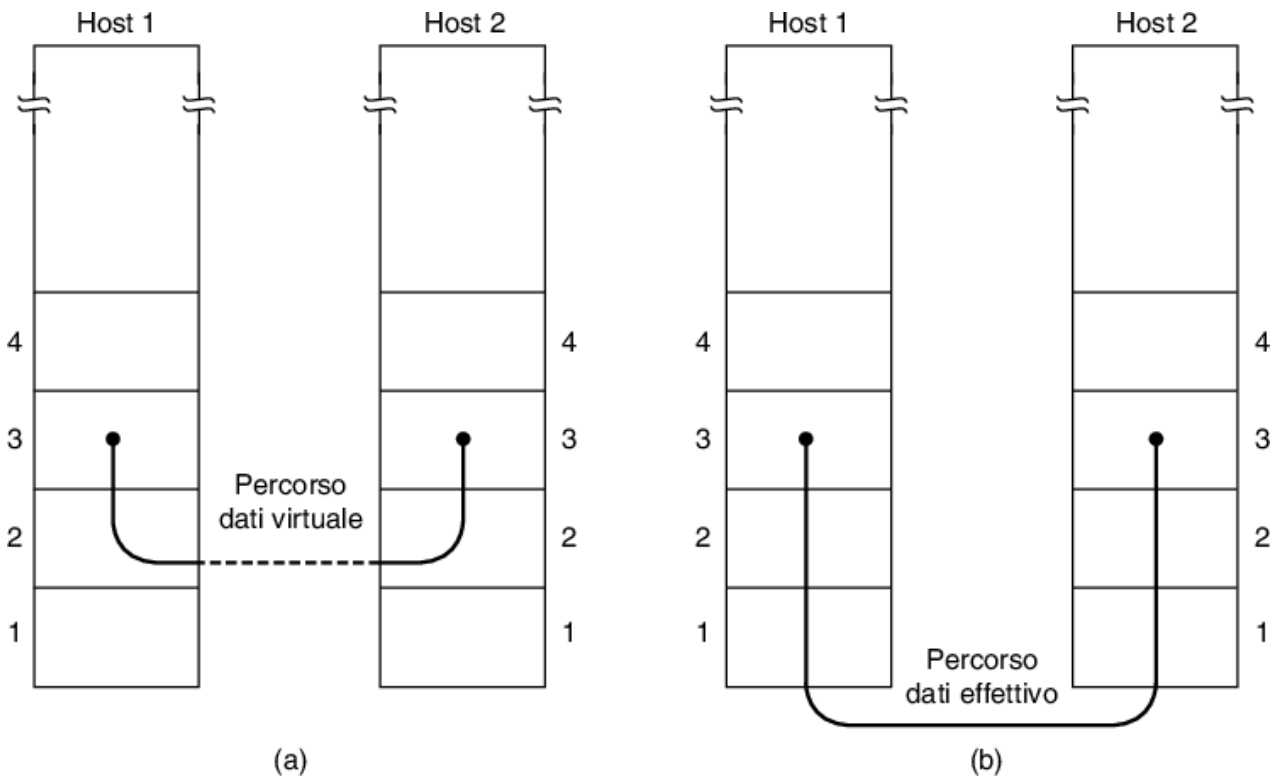


Figura 3.2 (a) Comunicazione virtuale. (b) Comunicazione reale.

Il livello data link può essere progettato in modo da offrire una varietà di servizi, che cambiano da protocollo a protocollo. Tre dei servizi che vengono comunemente forniti a questo livello sono:

1. servizio senza conferma (*unacknowledged*) senza connessione
2. servizio con conferma (*acknowledged*) senza connessione
3. servizio con conferma orientato alla connessione.

Il servizio di tipo *unacknowledged* senza connessione consiste nell'avere una macchina sorgente che invia dei frame indipendenti alla macchina destinazione, senza che quest'ultima debba dare una conferma dell'avvenuta ricezione. Ethernet è un buon esempio di livello data link che fornisce questa classe di servizi. Nessun tipo di connessione logica viene stabilita prima della trasmissione né viene rilasciata dopo. Se un frame viene perso a causa del rumore sulla linea, non viene fatto alcun tentativo di rilevare la perdita o di correggerla nel livello data link. L'uso di questa classe di servizio è appropriato quando la frequenza degli errori di trasmissione è molto bassa, così che la correzione può essere fatta dai livelli superiori. È altresì appropriato per il traffico di tipo real-time, come le trasmissioni di voce, per le quali gli effetti del ritardo sono peggiori di quelli derivanti dalla trasmissione di dati sbagliati.

Il passo successivo in termini di affidabilità è dato dai servizi di tipo *acknowledged* senza connessione. Questo genere di servizio continua a non usare alcun tipo di connessione logica, però ciascun frame è inviato individualmente e ne viene data conferma della ricezione tramite un *acknowledgement*. In questo modo, il mittente riesce a sapere se un frame è arrivato a destinazione in modo corretto oppure no. Se non è arrivato entro uno specifico intervallo di tempo, il frame può essere rispedito. Si tratta di un servizio utile per i canali di trasmissione non affidabili, come le reti wireless. Il protocollo 802.11 (WiFi) è un classico esempio di questa classe di servizio.

Vale la pena enfatizzare il fatto che l'acknowlegdement per il livello data link è da considerarsi semplicemente come un'ottimizzazione, mai come un requisito obbligatorio. Il livello di rete può sempre inviare un pacchetto e aspettare l'acknowlegdement dal suo peer sulla macchina remota. Se l'acknowledgement non arriva entro una data scadenza temporale, il mittente può semplicemente rispedire l'intero messaggio. Il problema che rende inefficiente questa strategia è che i link hanno frame di lunghezza massima limitata e imposta dall'hardware e un ritardo di propagazione noto. Il livello di rete, che non conosce questi parametri, potrebbe inviare un pacchetto molto grande da suddividere, per esempio, in 10 frame di cui mediamente 2 vengono persi. Potrebbe quindi essere necessario un tempo considerevole per far passare l'intero pacchetto. Se invece i frame hanno un acknowledgement individuale con eventuale ritrasmissione, i pacchetti nella loro interezza vengono trasmessi molto più velocemente. Su canali affidabili come le fibre ottiche, la ridondanza nel protocollo data link può rivelarsi inutile; al contrario, nei canali di tipo wireless, per via della loro intrinseca inaffidabilità, il costo è ampiamente ripagato.

Tornando alla descrizione dei servizi per il livello data link, quello più sofisticato ha la possibilità di erogare servizio orientato alla connessione (*connection oriented service*). Ovvero, con questo tipo di servizio le macchine sorgente e destinazione stabiliscono una connessione prima d'iniziare a trasferire i dati. Ogni frame trasferito attraverso la connessione è numerato e il livello data link garantisce che venga effettivamente ricevuto una sola volta e nell'ordine corretto. Quindi, il servizio orientato alla connessione fornisce ai processi del livello di rete un flusso di bit affidabile. È adeguato per collegamenti lunghi e inaffidabili come i canali satellitari e linee telefoniche a lunga distanza. Se venisse usato un servizio *acknowledged* senza connessione è possibile che gli acknowledgement persi facciano sì che un frame venga trasmesso e ricevuto più volte con spreco di banda.

Quando si usa un servizio orientato alla connessione, il trasferimento dei dati avviene in tre fasi distinte. Nella prima fase viene stabilita la connessione; il mittente e il destinatario inizializzano variabili e contatori necessari per tenere traccia di quali frame siano stati ricevuti e quali no. Nella

seconda fase uno o più frame vengono realmente trasmessi. Nella terza fase la connessione viene rilasciata, liberando le variabili, i buffer e le altre risorse usate per mantenerla.

Suddivisione in frame

Per erogare un servizio al livello di rete, il livello data link deve usare a sua volta il servizio che gli è fornito dal livello fisico, che ha come compito quello di prendere un flusso di bit e cercare di portarlo a destinazione. Se il canale è soggetto a rumore, come avviene nella maggior parte dei collegamenti cablati e wireless, il livello fisico introduce ridondanza ai segnali per portare a livello accettabile il tasso di errore. Non esiste tuttavia la garanzia che il flusso di bit ricevuto dal livello data link sia privo di errori. Alcuni bit potrebbero aver cambiato valore e il numero di bit ricevuti potrebbe essere uguale, maggiore o minore del numero dei bit trasmessi. Quello di rilevare, ed eventualmente correggere, gli errori è uno dei compiti del livello data link.

Il tipico approccio del livello data link è quello di suddividere il flusso di bit in una serie discreta di frame, calcolare un valore a lunghezza fissa, chiamato checksum, per ogni frame e includere il checksum nel frame quando viene trasmesso. Quando un frame arriva a destinazione, il checksum viene ricalcolato. Se il checksum calcolato dal destinatario è differente da quello contenuto nel frame, il livello data link sa che c'è stato un errore e prende i provvedimenti necessari, per esempio scartando il frame corrotto ed eventualmente restituendo un messaggio di errore.

Suddividere il flusso di bit in frame, operazione che prende il nome di **framing**, è più difficile di quanto appaia a prima vista. Una buona progettazione deve facilitare al destinatario l'identificazione dell'inizio di nuovi frame usando poca banda. In questo capitolo vedremo quattro metodi:

1. conteggio dei byte
2. flag byte con byte stuffing
3. flag bit con bit stuffing
4. violazioni della codifica del livello fisico.

Il primo metodo di framing usa un campo nell'intestazione per specificare il numero di byte nel frame. Quando il livello data link della destinazione legge tale numero, sa quanti byte seguiranno e quindi sa dove si troverà la fine del frame. Questa tecnica è mostrata nella Figura 3.3(a) per quattro piccoli frame di dimensioni di 5, 5, 8 e 8 byte rispettivamente.

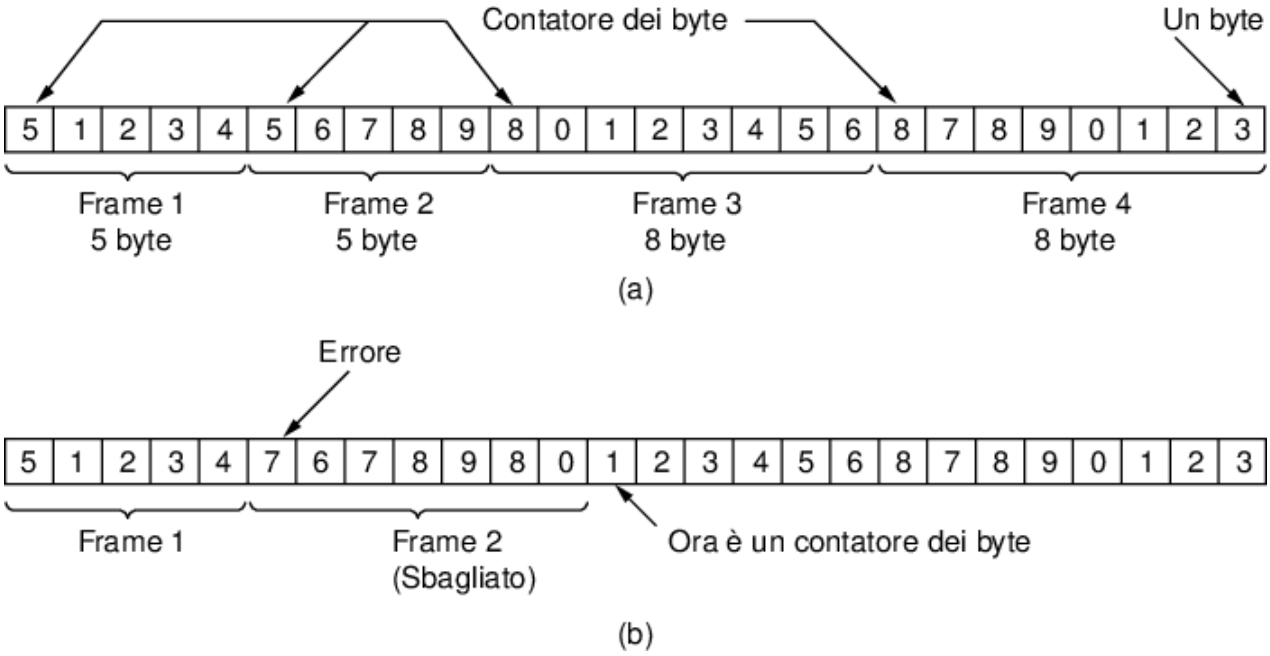


Figura 3.3 Un flusso di byte: (a) senza errori, (b) con un errore.

Il problema di questo algoritmo è che il conteggio può venire alterato da un errore di trasmissione. Per esempio, se il conteggio dei 5 byte nel secondo frame della Figura 3.3(a) diventa un 7 a causa del cambiamento di un singolo bit, il destinatario andrà fuori sincronia e non sarà in grado di trovare l'inizio del frame successivo. Anche se il calcolo del checksum è scorretto e perciò il destinatario sa che il frame è effettivamente da scartare, non gli sarà possibile trovare la posizione di partenza del frame successivo. L'invio sulla rete un frame alla sorgente per chiederne la ritrasmissione non è una soluzione, in quanto il destinatario non riesce a sapere quanti byte debba saltare per arrivare all'inizio del nuovo frame. Per questo motivo il metodo del conteggio dei byte non è quasi più utilizzato.

Il secondo metodo di framing aggira il problema di una nuova sincronizzazione, necessaria a seguito di un errore, inserendo byte speciali all'inizio e al termine di ogni frame. La maggior parte dei protocolli usa lo stesso byte, chiamato **flag byte**, per delimitare sia l'inizio sia la fine dei frame come indicato nella Figura 3.4(a) con FLAG. Due flag byte consecutivi indicano la fine di un frame e l'inizio del successivo. In questo modo, quando il destinatario perde la sincronizzazione, può semplicemente cercare due flag byte per trovare la fine del frame corrente e l'inizio di quello successivo.

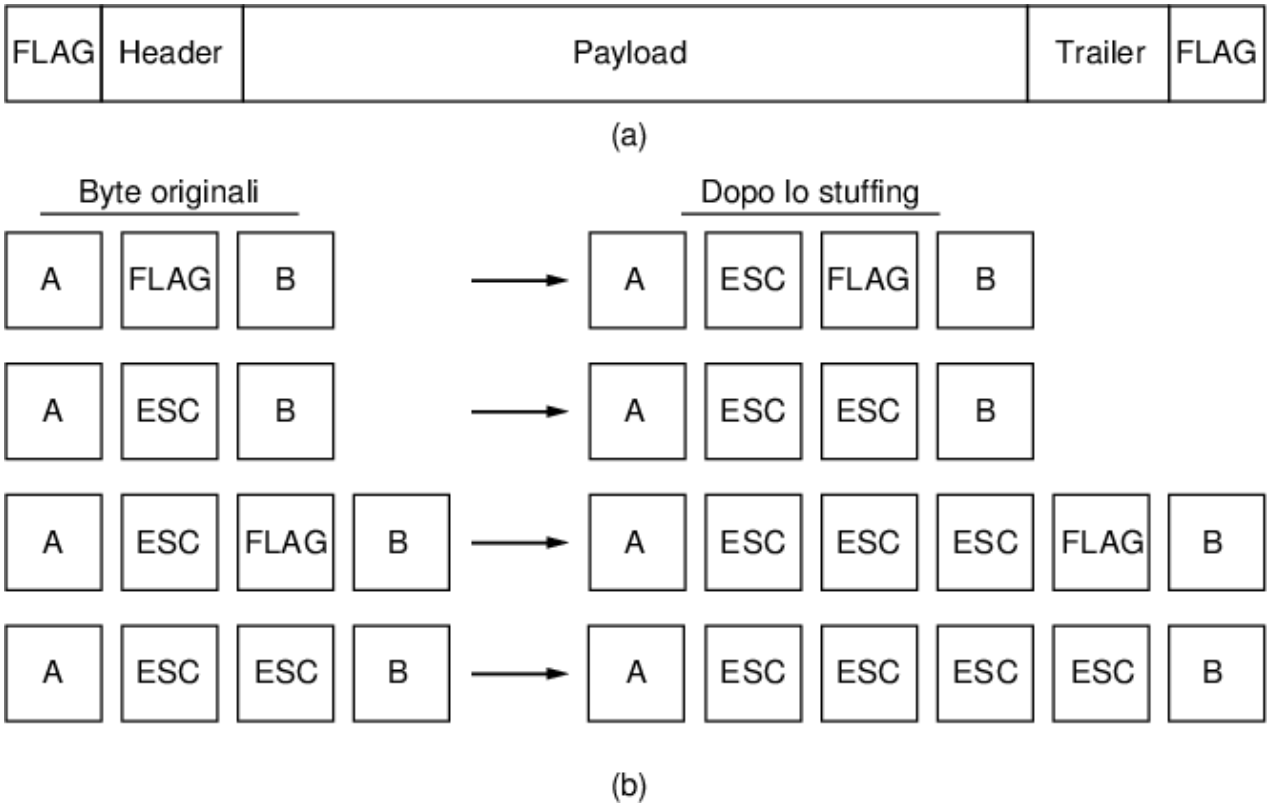


Figura 3.4 (a) Un frame delimitato dai flag byte. (b) Quattro esempi di sequenze di byte prima e dopo il byte stuffing.

Questo metodo presenta tuttavia un problema, in quanto può accadere che il valore corrispondente al flag byte compaia naturalmente dentro ai dati, specialmente in quelli binari, quali fotografie o audio, interferendo così con le operazioni di framing. Un modo per risolvere questo problema consiste nel far sì che la sorgente inserisca un byte di escape (ESC) subito prima di ogni occorrenza "accidentale" del flag byte nei dati. In questo modo un flag byte per il framing viene distinto da uno proprio dei dati in base all'assenza o presenza di un byte di escape che lo preceda. Il livello data link della destinazione provvederà a rimuovere i byte di escape prima di passare i dati al livello di rete. Questa tecnica è chiamata **byte stuffing**.

Naturalmente la domanda successiva sarà: che cosa succede se un byte di escape si trova dentro ai dati? La risposta, anche in questo caso, è che deve essere preceduto da un byte di escape. Al momento della ricezione il primo byte di escape viene rimosso, mentre il byte di dati successivo, che potrebbe essere un byte di escape o di flag, viene lasciato. Alcuni esempi sono riportati nella Figura 3.4(b). In tutti i casi la sequenza di byte ricavata dopo la conversione dei byte di escape (*destuffing*) è esattamente uguale alla sequenza originale. È ancora possibile identificare i bordi dei frame individuando due byte di flag in una riga, senza curarsi di eliminare i byte di escape.

Lo schema byte stuffing rappresentato nella Figura 3.4 è una leggera semplificazione di quello usato nel protocollo **PPP** (*point-to-point protocol*), usato per trasportare i pacchetti su link di comunicazione punto a punto. Discuteremo il protocollo PPP più avanti in questo capitolo.

Il terzo metodo per delimitare il flusso di dati aggira il limite imposto dal byte stuffing di usare byte a 8 bit. L'operazione di framing può essere effettuata anche a livello di bit in modo che i frame possano contenere un numero arbitrario di bit derivanti da unità di qualsiasi dimensione. È stato sviluppato per il **protocollo HDLC** (*high-level data link control*) che una volta era molto popolare. Ogni frame inizia e finisce con una speciale sequenza di bit, 01111110 o 0x7E in esadecimale, corrispondente a un flag byte. Il livello data link di chi trasmette, ogni volta che incontra cinque bit a 1 consecutivi nei dati, automaticamente inserisce un bit 0 nel flusso di bit in uscita. Questa operazione è chiamata **bit stuffing** analogamente al caso del byte stuffing, dove nel flusso dei dati in uscita s'inserisce un byte di escape subito prima di un valore uguale al flag byte. Questo meccanisno assicura inoltre un numero minimo di transizioni in un'unità di tempo che aiuta il livello fisico a mantenere la sincronizzazione. Ecco il motivo per cui **USB** (*universal serial bus*) usa il bit stuffing.

Quando la destinazione riceve cinque bit consecutivi con valore 1 seguiti da uno 0, automaticamente elimina lo 0. Così come il byte stuffing risultava completamente trasparente ai livelli di rete della sorgente e della destinazione, così accade anche per il bit stuffing. Se i dati da trasferire contengono la sequenza 01111110 al loro interno, tale sequenza verrà trasmessa come 011111010, ma verrà scritta nella memoria della destinazione come 01111110 dopo l'operazione di destuffing. La Figura 3.5 rappresenta un esempio di bit stuffing.

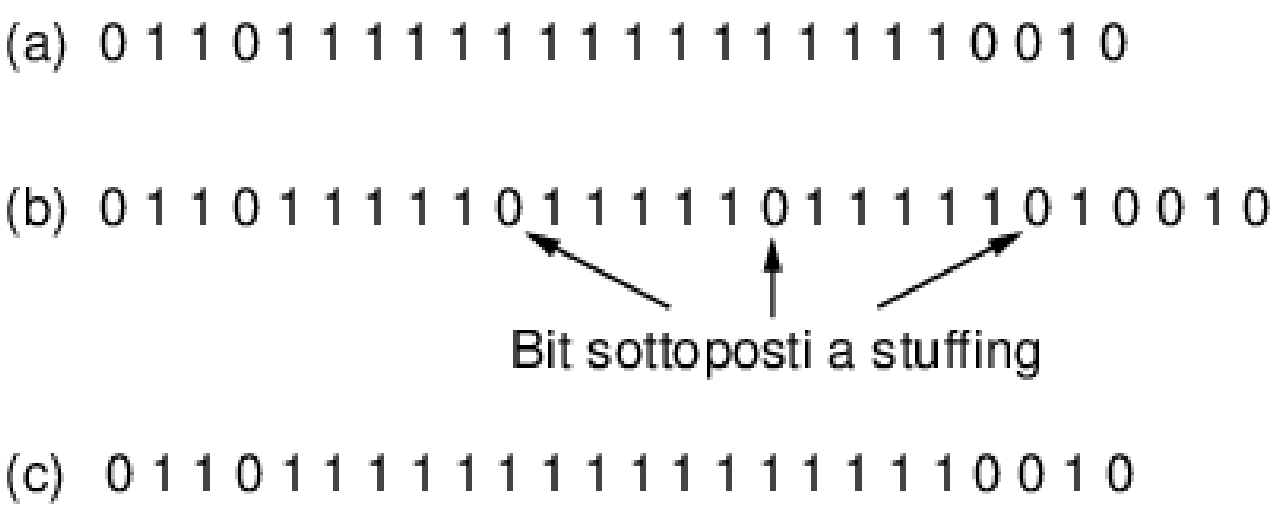


Figura 3.5 Bit stuffing. (a) I dati originali. (b) I dati che appaiono sulla linea. (c) I dati che vengono memorizzati nella memoria della destinazione dopo il destuffing.

Con il bit stuffing il confine fra due frame viene riconosciuto in modo inequivocabile tramite l'uso della sequenza nel flag byte, visto che questa può essere presente solo al confine tra i frame e mai al loro interno.

Un effetto secondario sia del bit stuffing che del byte stuffing è che la lunghezza del frame dipende dal contenuto dei dati che porta. Per esempio, 100 byte di dati che non contengano flag byte potrebbero essere portati da un frame di circa 100 byte; se, al contrario, fossero tutti flag byte, il

frame sarebbe di circa 200 byte, in quanto prima di ogni flag byte verrebbe inserito un byte di escape. Con il metodo del bit stuffing si avrebbe un incremento della lunghezza del frame di circa il 12,5% dovuto all'inserimento di un bit a ogni byte.

L'ultimo metodo di framimg consiste nell'usare una scorciatoia a livello fisico. Nel Capitolo 2 si è visto che la codifica di bit come segnali spesso inserisce ridondanza nei dati per aiutare chi riceve. Questa ridondanza implica che alcuni segnali non compaiono nei dati regolari. Per esempio, nella linea di codice 4B/5B 4 bit di dati vengono tradotti in 5 bit di segnali per garantire un numero sufficiente di transizioni di bit. Questo significa che 16 delle 32 possibilità di segnale non vengono utilizzate e quindi si possono usare alcuni segnali riservati per indicare l'inizio e la fine dei frame. In effetti si effettua una **coding violation** (*violazione della codifica*) per delimitare i frame. La bellezza di questo metodo è che, usando segnali riservati, è facile individuare l'inizio e la fine dei frame e non è necessario aggiungere dati.

Molti protocolli data link usano per sicurezza una combinazione di questi metodi. I protocolli Ethernet e 802.11 iniziano ogni frame con una sequenza ben definita chiamata **preambolo** (*preamble*). Questa sequenza può essere abbastanza lunga (tipicamente 72 bit nel protocollo 802.11) per lasciare il tempo a chi riceve di prepararsi per il pacchetto in arrivo. Il preambolo è seguito da un header con un campo indicante la lunghezza del frame usato per localizzarne la fine.

Controllo degli errori

Dopo aver risolto il problema di come delimitare l'inizio e la fine di ogni frame, procediamo con il successivo: come assicurarsi che al livello di rete della destinazione arrivino tutti i frame, e che arrivino nell'ordine corretto. Supponiamo per il momento che la destinazione possa sapere se riceve informazioni corrette o errate (nel Paragrafo 3.2 vedremo il codice usato per individuare e correggere gli errori di trasmissione). Per un servizio *unacknowledged* senza connessione sarebbe sufficiente che la sorgente continuasse semplicemente a inviare frame, senza controllare se stiano arrivando in modo corretto. Certamente questo non sarebbe accettabile per un servizio con connessione.

Il modo usuale per assicurare l'affidabilità dell'arrivo dei dati consiste nel fornire alla sorgente una qualche forma di reazione su quello che sta succedendo dall'altro capo della linea. Tipicamente il protocollo richiede che la destinazione invii sulla rete degli speciali frame di controllo, che contengano un acknowledgement positivo o negativo relativamente ai frame ricevuti. Se la sorgente riceve un acknowledgement positivo riguardo a un certo frame sa che il frame è stato ricevuto correttamente. Al contrario, un feedback negativo significa che qualcosa è andato storto e il frame va ritrasmesso.

Un'altra complicazione nasce quando problemi hardware possono far scomparire totalmente un frame (per esempio a causa di un picco di rumore). In questo caso la destinazione non reagirà in alcun modo, non avendone motivo. Analogamente, se il frame di acknowledgement viene perso, la sorgente non saprà come procedere. Dovrebbe essere chiaro che un protocollo in cui la sorgente trasmetta un frame e poi aspetti un acknowledgement, positivo o negativo, rimarrà bloccato per sempre nel caso in cui un frame venga perso, per esempio per un malfunzionamento hardware o un'interruzione sul canale di comunicazione. Questa possibilità è gestita introducendo timer (allarmi a tempo) nel livello data link. Quando la sorgente trasmette un frame, normalmente fa

anche partire un timer. Il timer viene impostato in modo da scattare dopo un intervallo abbastanza lungo da permettere che il frame giunga alla destinazione, vi venga elaborato e che l'acknowledgement possa ritornare alla sorgente. Normalmente, il frame raggiungerà la destinazione e l'acknowledgement ritornerà prima che il timer scatti, nel qual caso il timer verrà ignorato.

Se invece il frame o l'acknowledgement vengono persi, il timer scatterà così da segnalare alla sorgente un potenziale problema. L'ovvia soluzione consiste nel rinviare il frame. Se il frame è nuovamente trasmesso si rischia però che la destinazione lo accetti due o più volte e lo passi quindi ripetutamente al livello di rete. Per ovviare al problema è necessario assegnare un numero di sequenza ai frame in uscita: in questo modo la destinazione riesce a distinguere le ritrasmissioni dagli originali.

Per assicurarsi che ogni frame venga (in ultima analisi) passato al livello di rete esattamente una e una sola volta, il problema della gestione di timer e numeri di sequenza rappresenta una parte importante dei compiti del livello data link e di quelli superiori. Per studiare com'è implementata questa funzione, più avanti nel capitolo vedremo una serie di esempi con un livello crescente di complessità.

Controllo di flusso

Un altro importante punto da considerare nella progettazione del livello data link (e anche nei livelli superiori della gerarchia) riguarda che cosa fare quando la sorgente vuole sistematicamente trasmettere i frame più velocemente di quanto la destinazione sia in grado di accettarli. Questa situazione si verifica facilmente se la sorgente è implementata su un computer veloce, mentre la destinazione è implementata su un computer lento. Un esempio è quando uno smartphone richiede una pagina Web a un server molto più potente di lui, che continua a pompare i frame ad alta velocità verso quel povero telefono finché non ne risulta completamente sommerso. Anche se la trasmissione non presenta errori, la destinazione sarà semplicemente incapace di gestire i frame in arrivo e comincerà a perderne alcuni.

Chiaramente, per prevenire questo tipo di situazioni bisogna fare qualcosa. Due sono gli approcci comunemente usati. Nel primo, il **controllo di flusso tramite feedback** (*feedback-based flow control*), la destinazione rimanda alla sorgente delle informazioni per darle il permesso di mandare altri dati o comunque per informarla del suo stato. Nel secondo, il **controllo di flusso tramite limitazione del tasso di invio** (*rate-based flow control*), il protocollo contiene al proprio interno un meccanismo che limita il tasso al quale la sorgente può trasmettere i dati, senza alcun feedback da parte della destinazione.

In questo capitolo studieremo i metodi di controllo del flusso tramite feedback, poiché quelli a limitazione del tasso verranno visti solo come parte del livello di trasporto nel Capitolo 5, mentre i primi sono visti a livello data link e a quelli superiori. Questi ultimi sono inoltre più usati attualmente, perché l'hardware del livello data link è ormai progettato in modo da essere abbastanza veloce da non causare perdite. Per esempio, implementazioni in hardware tramite **NIC** (*network interface card,* scheda di interfaccia di rete)[1] del livello data link vanno spesso alla velocità detta *wire speed*, per indicare che possono gestire i frame alla stessa velocità del link. A questo punto, qualsiasi problema di eccessiva velocità è trattato dai livelli superiori.

Esistono diversi metodi di controllo di flusso tramite feedback, ma la maggior parte usa gli stessi principi base. Il protocollo contiene delle regole ben definite su quando una sorgente può trasmettere il frame successivo. Queste regole spesso proibiscono l'invio di frame finché la destinazione non abbia dato il permesso implicito o esplicito. Per esempio, quando viene aperta una connessione, la destinazione può dire: "Puoi mandarmi fino a *n* frame adesso e poi non mandare altro finché non ti dico di continuare". Esamineremo i dettagli di questi meccanismi nella restante parte del capitolo.

3.2 Rilevazione e correzione degli errori

Come visto nel Capitolo 2 i canali di comunicazione hanno un'ampia gamma di caratteristiche che li distinguono. Alcuni canali, come le fibre ottiche nelle reti di telecomunicazioni, hanno bassissimi tassi di errore e quindi gli errori di trasmissione sono eventi rari. Ma altri canali, specialmente le connessioni wireless e i doppini telefonici datati, hanno tassi di errore superiori di vari ordini di grandezza. Per questi collegamenti gli errori di trasmissione sono la norma e non possono essere evitati con una spesa o un costo in termini di prestazioni ragionevoli. In conclusione, gli errori di trasmissione esistono e dobbiamo imparare a fronteggiarli.

Per trattare questi errori i progettisti di rete hanno sviluppato due strategie di base: entrambe aggiungono informazioni ridondanti ai dati che devono essere spediti. Una strategia consiste nell'includere informazioni ridondanti in numero sufficiente da permettere al destinatario di dedurre i dati realmente trasmessi. L'altra consiste nell'introdurre una ridondanza di informazioni sufficiente a permettere al destinatario di dedurre che c'è stato un errore (ma non quale errore) e richiedere una nuova trasmissione. La prima strategia usa **codici a correzione d'errore** (*error-correction code*), mentre la seconda usa **codici a rilevazione d'errore** (*error-detecting code*). L'uso di codici a correzione d'errore viene spesso indicato come **FEC** (*forward error correction*, correzione d'errore in anticipo).

Ognuna di queste tecniche occupa una diversa nicchia. Su canali altamente affidabili, come la fibra ottica, è più vantaggioso usare un codice a rilevazione d'errore e ritrasmettere il blocco dei dati che si è riscontrato essere errato. Su canali con facilità di errore, come le connessioni wireless, è meglio aggiungere ridondanza a ogni blocco, in modo che la destinazione sia in grado di capire quale blocco sia stato originariamente trasmesso. La tecnica FEC è usata su canali soggetti a molto rumore, in quanto le successive trasmissioni di dati sono probabilmente errate come la prima.

Un punto chiave per questi codici è il tipo di errori che verosimilmente si verifica. Né i codici a correzione d'errore né i codici a rilevazione d'errore possono trattare tutti i possibili errori, in quanto i bit ridondanti, che dovrebbero offrire protezione, verosimilmente possono essere ricevuti con errori, esattamente come i bit dei dati (e questo potrebbe compromettere la protezione che offrono). Sarebbe bello se il canale trattasse i bit ridondanti in modo diverso dai bit di dati, ma questo non accade; sul canale tutto è solo bit. Ciò significa che, per evitare di non rilevare errori, la codifica deve essere sufficientemente robusta da trattare gli errori previsti.

Un possibile modello considera gli errori come causati da valori estremi di rumore termico che sovrasta brevemente e occasionalmente il segnale, generando errori isolati sul singolo bit. Un altro modello prevede che gli errori tendano ad avvenire a grappolo (*errori a burst*), piuttosto che isolatamente. Questo modello si basa sui processi fisici che generano gli errori, quali una grave dissolvenza del canale wireless o un'interferenza elettrica transitoria su un canale cablato.

Entrambi i modelli, pur giungendo a compromessi diversi, vengono implementati nella pratica. Gli errori a burst presentano sia vantaggi che svantaggi rispetto agli errori isolati su singolo bit. Uno dei vantaggi è che i dati digitali sono sempre trasmessi in blocchi di bit. Si supponga che la dimensione di un blocco sia di 1000 bit e che il tasso di errore sia 0,001 per bit. Se gli errori fossero

indipendenti, la maggior parte dei blocchi conterrebbe un errore. Se invece gli errori avvengono a grappoli di 100, in media solo un blocco su 100 ne sarebbe affetto; gli errori a burst sono però più difficilmente correggibili degli errori isolati.

Esistono anche altri tipi di errore. A volte è noto dove avviene un errore, magari perché il livello fisico ha ricevuto un segnale analogico molto diverso dal valore aspettato di 0 o 1 e in questo caso il bit viene dichiarato perso. In tale situazione il modello prende il nome di **canale a cancellazione** (*erasure channel*). È più facile correggere errori nei canali a cancellazione che nei canali in cui un bit viene invertito perché, anche se il valore del bit viene perso, almeno si conosce quale bit ha subìto un errore. Spesso però non possiamo usufruire dei benefici dei canali a cancellazione.

Esamineremo i codici a correzione d'errore e quelli a rilevazione d'errore in seguito, tenendo tuttavia in mente i seguenti due punti. In primo luogo tratteremo queste codifiche nel livello data link dove abbiamo incontrato per la prima volta il problema di trasmettere gruppi di bit in modo affidabile. Tuttavia, tali codifiche sono ampiamente utilizzate perché l'affidabilità è un problema che riguarda qualunque livello. I codici a correzione d'errore saranno esaminati anche a livello fisico, in particolare per i canali soggetti a molto rumore e nei livelli più alti, in particolare per la distribuzione di contenuti in tempo reale e multimediali. I codici a rilevazione d'errore sono comunemente usati nei livelli link, di rete e di trasporto.

Un altro punto da tenere a mente è che i codici per la gestione degli errori sono trattati nell'ambito della matematica applicata. Almeno che non siate particolarmente esperti dei campi di Galois o delle proprietà delle matrici sparse, fareste meglio a usare codici con buone proprietà tratte da una sorgente affidabile, piuttosto che tentare di farne uno nuovo. In effetti, i protocolli standard usano sempre gli stessi codici. Di seguito studieremo nel dettaglio un codice semplice, mentre descriveremo solo brevemente i codici più avanzati. In questo modo potremo capire i compromessi accettati nell'implementare i codici semplici rispetto a quelli complessi e, trattando questi ultimi, parlare di quelli usati nella pratica.

Codici a correzione di errore

Esamineremo quattro diversi codici a correzione di errore:

1. codice di Hamming
2. codice convoluzionale binaria
3. codice di Reed-Solomon
4. codice a controllo di parità a bassa densità.

Tutti questi codici aggiungono ridondanza alle informazioni trasmesse. Un frame consiste di m bit di dati (cioè il messaggio) e r bit di ridondanza (cioè controllo). In un **codice a blocco** gli r bit di controllo sono calcolati unicamente in funzione degli m bit di dati a cui sono associati, come se gli m bit fossero usati come indice in una tabella per trovare i corrispondenti r bit di controllo. In un **codice sistematico** gli m bit di dati sono trasmessi insieme a quelli di controllo piuttosto che essere prima codificati. In un **codice lineare** gli r bit di controllo sono una funzione lineare degli m bit di dati quali l'OR esclusivo (XOR) o la somma in modulo 2, in modo che la codifica possa avvenire tramite operazioni come il prodotto di matrici o semplici circuiti logici. Tutti i codici trattati in questo paragrafo sono lineari, sistematici e a blocco, se non esplicitamente detto diversamente.

Denotiamo la lunghezza totale di un blocco con n, dove $n = m + r$, e il relativo codice come un codice (n, m). Un'unità di n bit che contiene dati e bit di controllo viene chiamata **parola del codice** (*codeword*) di n bit. Il *code rate*, o semplicemente *rate*, è la frazione di parole che trasportano bit non ridondanti pari a m/n. Nella pratica si usano rate molto diversi tra loro: da 1/2 per i canali rumorosi, nel qual caso metà delle informazioni ricevute è ridondante, a quasi 1 per i canali ad alta qualità, nei quali solo un piccolo numero di bit di controllo viene aggiunto a messaggi anche di grandi dimensioni.

Per capire come trattare gli errori è necessario innanzitutto approfondire il concetto di errore stesso. Date due parole, per esempio 10001001 e 10110001 è possibile calcolare quanti bit corrispondenti sono differenti. In questo caso i bit differenti sono 3; in generale si esegue l'OR esclusivo delle due parole del codice e si conta il numero di bit pari a 1 nel risultato come nel seguente esempio:

```
10001001

10110001

00111000
```

Il numero di bit corrispondenti diversi nelle due sequenze è detto **distanza di Hamming** (Hamming, 1950). Il suo significato è che, se due parole di codice sono a distanza di Hamming d una dall'altra, saranno necessari d errori su singoli bit per convertire una sequenza nell'altra.

Dato l'algoritmo per calcolare i bit di controllo, è possibile costruire una lista completa delle parole di codice valide per trovare due parole di codice con la distanza di Hamming minima. Questa distanza è definita distanza di Hamming dell'intera codifica.

Nella maggior parte delle applicazioni di trasmissione dati, tutti i 2^m possibili messaggi sono validi ma, per come sono calcolati i bit di controllo, non tutte le possibili 2^m parole di codice vengono usate. Infatti, se vengono inseriti r bit di controllo, solo una piccola frazione del numero di messaggi possibili, pari a $2^m/2^n$ o $1/2^r$, saranno parole di codice valide. Proprio la bassa densità con cui il messaggio è rappresentato nello spazio delle parole di codice permette al destinatario di rilevare e correggere gli errori.

Le proprietà di rilevazione e correzione degli errori di una codifica dipendono dalla sua distanza di Hamming. Per trovare in modo affidabile d errori è necessaria una codifica con distanza $d + 1$, in quanto garantisce che d errori su singoli bit non possano cambiare una parola di codice valida con un'altra anch'essa valida. Quando la destinazione vede una parola di codice non valida capisce che c'è stato un errore. Analogamente, per correggere d errori è necessaria una codifica con distanza $2d + 1$; in tal modo le parole di codice valide sono così distanziate fra di loro che anche con d cambiamenti la parola originale è sempre la più vicina secondo la distanza di Hamming e può quindi essere determinata univocamente partendo dall'assunto che sia meno probabile che si verifichi un numero maggiore di errori.

Come esempio semplice di codifica a correzione d'errore consideriamo una codifica con solo quattro parole di codice valide:

0000000000, 0000011111, 1111100000, 1111111111

Questa codifica ha distanza 5, il che significa che può correggere errori doppi o rilevare errori quadrupli. Se arriva la parola di codice 0000000111 e si attendono errori su singolo o doppio bit, la destinazione sa che l'originale doveva essere 0000011111. Se invece un triplo errore cambia 0000000000 in 0000000111, allora l'errore non verrà trattato correttamente. Se, invece, ci aspettassimo tutti questi errori, allora potremo rilevarli; nessuna delle parole ricevute sarebbe valida e si dedurrebbe che c'è stato un errore. Appare evidente dall'esempio che non si può contemporaneamente correggere un errore doppio e rilevare un errore quadruplo, perché dovremmo interpretare la parola ricevuta in due modi diversi.

Nell'esempio precedente si trova la parola valida più vicina a quella ricevuta ispezionando l'intero dizionario. Poiché, nel caso generale in cui tutte le parole devono essere valutate, tale ricerca ha un costo di esecuzione molto alto, nella pratica i codici sono progettati in modo da ammettere delle scorciatoie che trovino qual era verosimilmente la parola di codice originaria.

Supponiamo di voler progettare una codifica con m bit di messaggio ed r bit di controllo che permetta di correggere tutti gli errori singoli. Per ognuno dei 2^m messaggi validi esistono n parole non valide a distanza 1, formate invertendo sistematicamente ognuno degli n bit della parola valida. Per questo motivo ognuno dei 2^m messaggi validi ha la necessità di avere $n + 1$ combinazioni di bit dedicate. Visto che il numero totale di combinazioni di bit è 2^n, dobbiamo avere $(n + 1)\ 2^m \leq 2^n$. Sostituendo $n = m + r$, la disuguaglianza diventa:

mr ^{mr} (3-1)

Dato *m,* questa disuguaglianza impone un limite inferiore al numero di bit di controllo necessari per correggere gli errori singoli. Questo limite teorico può essere effettivamente raggiunto usando un metodo dovuto a Hamming (1950). I bit della parola di codice vengono numerati consecutivamente a partire da 1 per il primo bit sulla sinistra, 2 per quello immediatamente alla sua destra e così via. I bit che sono una potenza di 2 (1, 2, 4, 8, 16, ecc.) sono bit di controllo, i restanti bit (3, 5, 6, 7, 9, ecc.) sono riempiti con *m* bit di dati, come mostrato nella Figura 3.6 per un codice di Hamming (11,7) con 7 bit di dati e 4 bit di controllo. Ogni bit di controllo forza la somma modulo 2, o parità, di alcuni gruppi di bit incluso se stesso, a essere pari (o dispari). Un bit può essere incluso in più calcoli di bit di controllo. Per vedere a quali bit di controllo il bit di dati in posizione k contribuisce, riscriviamo k come somma di potenze di 2. Per esempio 11 = l + 2 + 8 e 29 = l + 4 + 8 + 16. Un bit di dati è controllato solo dai bit di controllo presenti nella sua espansione in somma di potenze di 2 (per esempio 11 è controllato solo dai bit 1, 2 e 8). Nell'esempio i bit di controllo sono calcolati per somme di parità pari in un messaggio che rappresenta in ASCII la lettera "A".

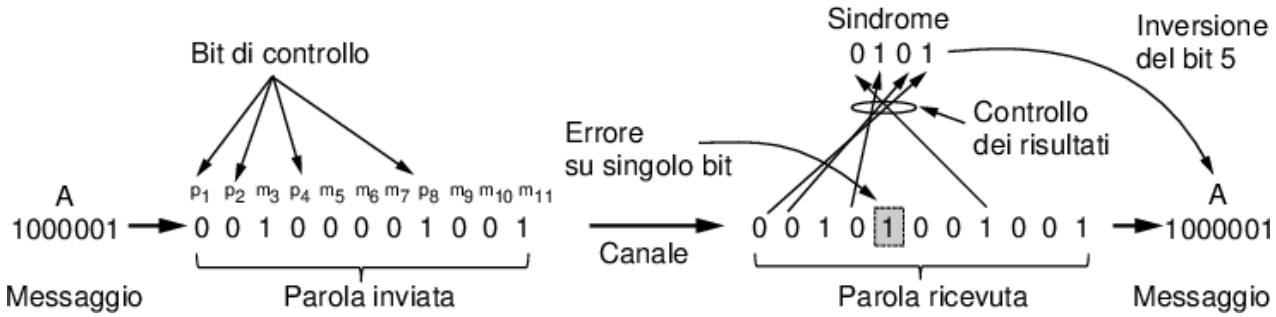


Figura 3.6 Esempio di un codice di Hamming (11,7) che corregge un errore su singolo bit.

Questa costruzione fornisce un codice con distanza di Hamming pari a 3, in grado di correggere singoli errori (o rilevare errori doppi). La ragione per l'accurata numerazione dei messaggi e dei bit di controllo diventa evidente nel processo di decodifica. Il destinatario, quando riceve una parola, calcola nuovamente i bit di controllo includendo i valori di quelli appena ricevuti; i bit ottenuti in questo modo prendono il nome di *check result*. Se i bit di controllo sono corretti per ogni somma di parità pari, ogni bit del *check result* deve essere pari a zero. In questo caso la parola di codice viene accettata come valida.

Se invece non tutti i risultati del controllo sono nulli significa che si è rilevato un errore. L'insieme dei risultati di controllo forma la **sindrome d'errore** (*error syndrome*) utilizzata per individuare e correggere l'errore. Nella Figura 3.6 è rappresentato il caso in cui si verifica un errore di un singolo bit sul canale; i risultati del controllo sono 0, 1, 0, 1 per k = 8, 4, 2, 1 rispettivamente. La sindrome risultante è 0101 o 4 + 1 = 5 che, per com'è progettato il sistema, significa che si è verificato un errore sul quinto bit. Invertendo il bit errato, che potrebbe essere sia un bit di controllo che di dati, ed eliminando i bit di controllo, si ottiene il messaggio corretto corrispondente alla lettera "A" in ASCII.

Le distanze di Hamming sono preziose per capire i codici a blocco e i codici di Hamming sono usati in dispositivi di memorizzazione in grado di correggere gli errori. Molte reti tuttavia usano codici più robusti. Il secondo codice che analizziamo è il **codice convoluzionale** (*convolutional code*), l'unico non a blocco che tratteremo. In un codice convoluzionale un sistema di codifica elabora una sequenza di bit in input e genera una sequenza di bit di output. A differenza di come invece avveniva nei codici a blocchi, non vengono fissati a priori né lunghezza né delimitatori del messaggio. L'output dipende sia dai bit di input correnti sia da quelli precedenti; il sistema di codifica è quindi un processo con memoria. Il numero di bit precedenti su cui la codifica è basata è chiamato **lunghezza dei vincoli** (*constraint length*) del codice. I codici convoluzionali sono specificati in funzione del loro rate e della loro lunghezza dei vincoli.

I codici convoluzionali sono largamente utilizzati nelle reti oggi esistenti, quali la rete di telefonia cellulare GSM, in parte, le comunicazioni satellitari e l'802.11. Un esempio di codice molto diffuso è mostrato nella Figura 3.7. Questo codice è noto come il codice convoluzionale NASA con r = 1/2 e k = 7 perché venne usato per la prima volta nella missione spaziale del Voyager nel 1977. Venne poi impiegato liberamente anche come parte dell'802.11.

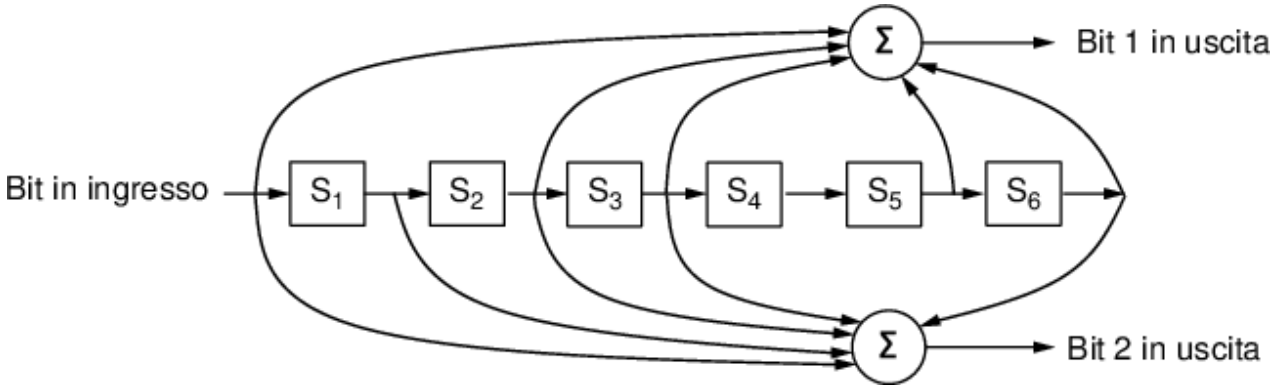


Figura 3.7 Il codice convoluzionale binaria della NASA usato in 802.11.

Nella Figura 3.7 ogni bit di input sul lato sinistro fornisce due bit di output sul lato destro, dati dall'OR esclusivo dell'input e dello stato interno. Questo codice, effettuando operazioni lineari su bit, è un codice binario, lineare e a convoluzione e poiché 1 bit di input produce 2 bit di output il *code rate* è di 1/2. Il codice è anche non sistematico, nessuno dei bit in output è semplicemente il bit in input.

Lo stato interno è mantenuto in sei registri di memoria. Quando un bit viene dato in input, i valori nei registri vengono fatti scorrere verso destra. Per esempio, se viene dato in input 111 e lo stato iniziale era 000000, lo stato interno, scritto da sinistra a destra, diviene 100000, 110000 e 111000 rispettivamente dopo il primo, il secondo e il terzo bit di input. I bit di output sono 11, quindi 10 e infine 01. Sono necessari 7 spostamenti per far passare completamente un input in modo che non alteri l'output. La lunghezza dei vincoli di questo codice è quindi $k = 7$.

Un codice convoluzionale è decodificato trovando la sequenza di bit di input che più verosimilmente ha prodotto la sequenza di bit osservata che, ricordiamo, include anche i bit di controllo. Per piccoli valori di k si utilizza spesso l'algoritmo sviluppato da Viterbi (Forney, 1973). L'algoritmo scandisce la sequenza osservata, memorizzando a ogni passo e per ogni stato interno, la sequenza di input in grado di produrre la sequenza osservata col minor numero di errori. La sequenza di input che fornisce il minor numero di errori alla fine del processo è il messaggio con massima verosimiglianza.

I codici convoluzionali sono molto usati nella pratica perché risulta facile esprimere l'incertezza che un bit sia 1 o 0 nel sistema di decodifica. Si supponga per esempio, che –1V corrisponda allo 0 logico e +1V all'1 logico e che si ricevano un bit di 0,9V e uno di –0,1V. Invece di convertirli direttamente in 1 e 0, si vorrebbe trattare 0,9V come "verosimilmente 1" e –0,1V come "verosimilmente 0" e correggere la sequenza nella sua interezza. Le estensioni dell'algoritmo di Viterbi lavorano con queste incertezze per fornire correzioni d'errore più potenti. Questo tipo di approccio che lavora con l'incertezza sui bit è chiamato **soft-decision decoding**, mentre i metodi che decidono se un bit è 0 o 1 prima delle successive correzioni d'errore sono chiamati **hard-decision decoding**.

Il terzo tipo di codice a correzione d'errore che esaminiamo è quello di **Reed-Solomon**. Anche il codice di Reed-Solomon, come quello di Hamming, è lineare, a blocchi e spesso anche sistematico. Mentre il codice di Hamming lavora sui singoli bit, quello di Reed-Solomon opera su gruppi di m simboli di bit. Poiché la matematica coinvolta è più sofisticata ci limiteremo a descriverli per analogia.

I codici di Reed-Solomon si basano sul fatto che un polinomio di grado n è univocamente determinato da $n + 1$ punti. Per esempio, una retta della forma $ax + b$ è univocamente determinata da due punti, mentre gli altri punti della retta sono ridondanti e quindi utili per la correzione degli errori. Si supponga di avere due punti di dati che rappresentano una retta e di trasmetterli insieme a due di controllo che giacciono sulla stessa retta. Se uno dei punti pervenisse errato, si potrebbero comunque recuperare i dati effettuando la regressione della retta sulla base dei punti ricevuti. Tre punti giacerebbero sulla retta mentre il punto errato giacerebbe fuori. Trovando la retta si corregge l'errore.

I codici di Reed-Solomon sono effettivamente definiti come polinomi che operano su campi finiti, ma il loro modo di operare è simile a quello descritto sopra. Con simboli di m bit le parole sono lunghe $2^m - 1$. Una scelta comune è porre $m = 8$ in modo che i simboli siano byte e una parola sia lunga 255 byte. Il codice (255,233) che aggiunge 32 simboli ridondanti ai 233 di dati è molto diffuso. La decodifica a correzione d'errore viene effettuata con un algoritmo sviluppato da Berlekamp e Massey che effettua efficientemente la regressione per codici di lunghezza moderata (Massey, 1969).

I codici di Reed-Solomon sono molto usati nella pratica per la loro grande capacità di correzione degli errori, specialmente se burst. Sono usati nelle DSL, nella trasmissione dati via cavo, nelle comunicazioni satellitari e ancora di più nei CD, DVD e Blu-ray disc. Essendo basati su simboli a m bit, un errore su singolo bit o un errore a burst su m bit sono trattati ugualmente come errori su un simbolo. Se vengono aggiunti $2t$ simboli ridondanti, un codice di Reed-Solomon è in grado di correggere fino a t errori per ogni simbolo trasmesso. Per esempio, il codice (255,233) che ha 32 simboli ridondanti può correggere fino a 16 simboli errati. Poiché i simboli sono consecutivi e ognuno è formato da 8 bit, è possibile correggere un errore a burst al massimo di 128 bit. La situazione è ancora più favorevole se il modello di errore è a cancellazione, come un graffio su un CD che distrugga alcuni simboli. In questo caso si può correggere fino a $2t$ errori.

I codici di Reed-Solomon sono frequentemente usati in combinazione con altri codici quali quelli a convoluzione. Infatti, i codici convoluzionali sono efficienti nel trattare errori su bit isolati, ma falliscono se vi sono troppi errori nello stream di dati ricevuto. Aggiungendo un codice di Reed-Solomon all'interno del codice convoluzionale, la sua decodifica spazza via efficientemente gli errori a burst. Il codice nella sua interezza fornisce una buona protezione sia contro gli errori isolati sia contro quelli a burst.

L'ultimo codice a correzione d'errore che trattiamo è quello **LDPC** (*low-density parity check*, controllo di parità a bassa densità) che è un codice lineare e a blocchi inventato da Robert Gallager nella sua tesi di dottorato (Gallager, 1962). Come molti altri lavori di tesi, anche questo venne prontamente dimenticato per poi resuscitare nel 1995 quando l'aumento della capacità computazionale lo rese implementabile nella pratica.

Poiché in un codice LDPC ogni bit di output è formato solo da una frazione dei bit di input, si ottengono rappresentazioni matriciali con una bassa densità di 1, proprietà da cui deriva il nome. Le parole di codice ricevute vengono decodificate con un algoritmo di approssimazione che iterativamente migliora, sulla base dei dati ricevuti, l'approssimazione della parola di codice valida, correggendo in questo modo gli errori.

I codici LDPC sono molto comodi da utilizzare con blocchi di grandi dimensioni e hanno una capacià di correggere gli errori che supera molti altri codici, compresi quelli di cui abbiamo già parlato. Per questo motivo vengono inclusi sempre di più nei nuovi protocolli. Sono parte dello standard del broadcast di video digitali, di Ethernet a 10 Gbps, delle reti facenti usi di linee elettriche e dell'ultima versione del protocollo 802.11 e ci aspettiamo che saranno in futuro inclusi in molte altre reti.

Codici a rilevazione di errore

Le codifiche a correzione d'errore sono ampiamente usate per le trasmissioni wireless, notoriamente soggette a rumore e piene di errori se confrontate con le fibre ottiche. Senza i codici a correzione di errore sarebbe davvero difficile riuscire a trasmettere su canali wireless. Tuttavia, nelle trasmissioni su fibra ottica o cavi di rame di elevata qualità, il tasso di errore è molto più basso e quindi la rilevazione degli errori e la ritrasmissione sono di solito più efficienti rispetto alle trasmisssioni con errori occasionali.

Esamineremo 3 diversi codici a rilevazione d'errore, tutti lineari, sistematici e a blocchi:

- 1. parità

1. checksum
2. controllo di ridondanza ciclica (CRC).

Per rendersi conto di quanto siano più efficienti dei codici a correzione di errore, si consideri un primo codice nel quale un singolo **bit di parità** (*parity bit*) viene aggiunto in coda ai dati. Il bit di parità viene scelto in modo che il numero di 1 nella parola di codice sia dispari (o pari). Questa operazione è equivalente a calcolare il bit di parità (dispari) come somma modulo 2 o lo XOR dei bit di dati. Per esempio, se viene inviata la sequenza 1011010 in parità dispari, viene aggiunto un bit alla fine per ottenere 10110101. Un codice con un singolo bit di parità ha distanza pari a 2, poiché un errore su singolo bit produce una parola di codice con parità errata; quindi è in grado di rilevare errori su singoli bit.

Si consideri un canale con errori isolati e un tasso di errore di 10^{-6} per bit. Benché possa sembrare un tasso di errore molto piccolo, diventa difficilmente trattabile se la trasmissione avviene su un lungo cavo cablato. I collegamenti LAN hanno tipicamente un tasso di errore di 10^{-10}. Se si considerano blocchi di 1000 bit si calcola dall'Equazione 3-1 che sono necessari 10 bit di controllo. Quindi un megabit di dati richiede 10000 bit di controllo. Se invece si vuole solamente riconoscere i blocchi con un singolo bit in errore, è sufficiente avere un bit di parità per blocco. Un blocco ogni 1000 ha un errore e un blocco extra (di 1001 bit) deve essere trasmesso per correggere l'errore. Il carico aggiunto per rilevare l'errore e ritrasmettere il messaggio è di soli 2001 bit per megabit di dati contro i 10000 bit necessari a un codice di Hamming.

La difficoltà di questo codice è che un singolo bit di parità può rilevare in modo affidabile solo un errore a singolo bit sul blocco. In un blocco pesantemente alterato da un lungo errore a burst, la probabilità, del tutto inaccettabile, che l'errore venga rilevato è pari a 0,5. La probabilità può essere incrementata in modo considerevole se si considera ogni blocco da inviare come una matrice rettangolare di n colonne e k righe. Se si calcola e si trasmette un bit di parità per ogni riga si potranno rilevare fino a k errori, ipotizzando al più un errore per riga.

Tuttavia, si può migliorare la protezione offerta contro gli errori a burst: si possono calcolare i bit di parità sulla base dei bit di dati considerati in un ordine diverso rispetto a quelli con cui vengono trasmessi. Questa tecnica è chiamata **interleaving**. Si calcola il bit di parità per ognuna delle n colonne e si trasmettono tutti i bit di dati come k righe inviate dall'alto al basso e, all'interno di ogni riga, da sinistra a destra. All'ultima riga si inviano gli n bit di parità. L'ordine di trasmissione è mostrato nella Figura 3.8 per un caso con $n = 7$ e $k = 7$.

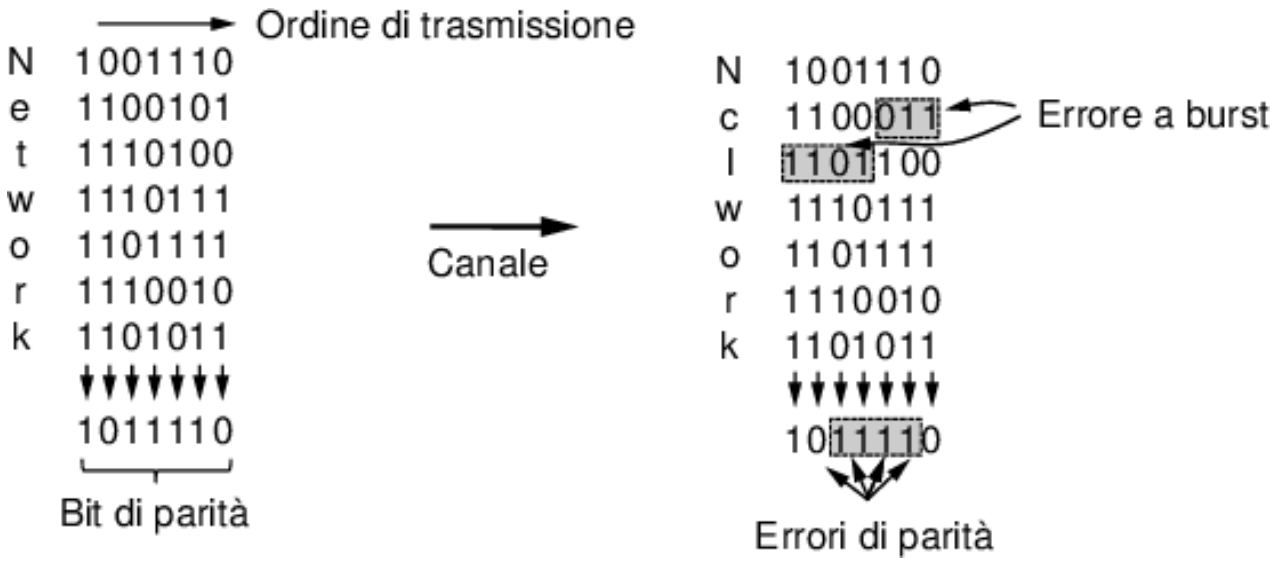


Figura 3.8 Interleaving dei bit di parità per la rilevazione di un errore a burst.

L'interleaving è una tecnica generale per convertire un codice a rilevazione o correzione di errori isolati in uno in grado di trattare errori a burst. Nella Figura 3.8 si vede il caso in cui avviene un errore a burst di lunghezza $n = 7$ e i bit errati sono sparsi su varie colonne. Si ricordi che un errore a burst non implica che tutti i bit siano errati; implica solo che siano sbagliati almeno il primo e l'ultimo bit. Nella Figura 3.8 i bit sono invertiti su 7 bit. Poiché verrebbe alterato al massimo un bit per ogni colonna, i relativi bit di parità rileverebbero l'errore. Questo metodo utilizza n bit di parità su blocchi di kn bit di parità per rilevare un singolo errore a burst di lunghezza n o meno.

Tuttavia, un errore a burst di lunghezza $n + 1$ non verrebbe rilevato se fossero invertiti solo il primo e l'ultimo bit. Se il blocco fosse pesantemente alterato da un unico lungo errore o da molti brevi errori a burst, la probabilità che una delle n colonne abbia per caso parità corretta è di 0,5; quindi la probabilità che un blocco sbagliato venga accettato è di 2^{-n}.

Il secondo tipo di codice a rilevazione d'errore che trattiamo, il **checksum**, è strettamente correlato a quello di parità. Il termine checksum è spesso usato per indicare un gruppo di bit di controllo associati al messaggio, qualunque sia il modo di calcolarli. Un gruppo di bit di parità è un esempio di checksum, tuttavia esistono altri, più potenti checksum basati sulla somma progressiva dei bit di dati del messaggio. Il checksum è di solito posizionato alla fine del messaggio come complemento a uno del risultato della somma dei dati, in modo che gli errori possano essere rilevati sommando l'intera parola contenente sia i bit di dati sia il checksum. Se il risultato è zero nessun errore è stato rilevato.

Un esempio di checksum è quello di 16-bit di Internet usato in tutti i pacchetti Internet come parte del protocollo IP (Braden et al., 1988). Questo checksum è la somma dei bit del messaggio suddivisi in parole di 16 bit. Poiché questo metodo lavora su parole invece che su bit come la parità, gli errori che non alterano la parità possono cambiare la somma ed essere rilevati. Se, per esempio, il bit di valore più basso in due differenti parole viene invertito da 0 a 1, un controllo di parità fallirebbe, mentre il checksum lo rileverebbe perché sarebbero aggiunti due 1.

Il checksum usato comunemente in Internet viene calcolato usando aritmetica complemento a uno anziché una somma modulo 2^{16}. Nell'aritmetica complemento a uno un numero negativo è l'inversione bit a bit della sua controparte positiva. I calcolatori moderni fanno uso internamente di un'aritmetica complemento a due per la quale un numero negativo viene calcolato come il complemento a uno più uno. In un calcolatore che usa il complemento a due, la somma con complemento a uno è equivalente a prendere la somma modulo 2^{16} aggiungendo tutti i bit di overflow ai bit di ordine più basso.

Per decenni è stato dato per scontato che i frame su cui calcolare i checksum contenessero dei bit con distribuzioni casuali. Tutte le analisi degli algoritmi di checksum sono state fatte sotto queste ipotesi. L'analisi di dati reali (Partridge et al., 1995), ha però dimostrato che tale ipotesi è decisamente sbagliata. Di conseguenza gli errori non rilevati sono, in alcuni casi, molto più diffusi di quanto si pensasse.

Il checksum di Internet, in particolare, è semplice ed efficiente, ma in alcuni casi essendo solamente una somma, fornisce una protezione debole. Non rileva la cancellazione e l'introduzione di dati a valore zero, lo scambio di posizione di parti del messaggio e l'unione di sottoinsiemi di messaggi diversi. Benché sia inverosimile che questi tipi di errore accadano in processi casuali, essi possono però avvenire se si utilizzano hardware difettosi.

Una scelta di codice migliore è rappresentata dal checksum di Fletcher (Fletcher, 1982) che, includendo una componente posizionale ottenuta aggiungendo il prodotto dei dati e la sua posizione durante la somma progressiva, fornisce una protezione più forte contro i cambiamenti di posizione.

Sebbene i due metodi descritti sopra possano risultare a volte adeguati per i livelli più alti, nella pratica a livello data link si usa diffusamente un terzo, più potente codice a rilevazione di errore: il **CRC** (*cyclic redundancy check*, controllo ciclico di ridondanza), noto anche come *codifica polinomiale*. Le codifiche polinomiali si basano sul fatto di trattare le sequenze di bit come dei polinomi a coefficienti che possono assumere solo i valori 0 e 1. Un frame di k bit è visto come una lista di coefficienti per un polinomio con k termini che variano da x^{K-1} a x^0. Tale polinomio è

detto di grado $k - 1$. Il coefficiente di termine più alto (quello più a sinistra) è il coefficiente di x^{K-1}; il successivo è quello di x^{k-2} e così via. Per esempio 110001 ha 6 bit e quindi rappresenta un polinomio a sei termini con coefficienti 1, 1, 0, 0, 0 e 1: $1x^5 + 1x^4 + 0x^3 + 0x^2 + 0x^1 + 1x^0$.

L'aritmetica dei polinomi si gestisce in modulo 2 secondo le regole della teoria dei campi algebrici. Non ci sono riporti per le addizioni o prestiti per le sottrazioni. Sia l'addizione sia la sottrazione sono identiche all'OR esclusivo. Per esempio:

10011011	00110011	11110000	01010101
+ 11001010	+ 11001101	− 10100110	− 10101111
————	————	————	————
01010001	11111110	01010110	11111010

Le divisioni lunghe vengono eseguite come nel calcolo binario, salvo che le sottrazioni sono in modulo 2 come spiegato precedentemente. Si dice che un divisore "sta" nel dividendo se il dividendo ha tanti bit quanti il divisore.

Quando si utilizza una codifica polinomiale, sorgente e destinazione devono mettersi d'accordo in anticipo su un **polinomio generatore**, $G(x)$. Il generatore deve avere i bit di ordine più alto e più basso uguali a 1. Per poter calcolare il **checksum** di un frame di m bit corrispondente al polinomio $M(x)$, il frame deve essere più lungo del polinomio generatore. L'idea è quella di aggiungere un checksum alla fine del frame, in modo che il polinomio rappresentato dal frame con checksum sia divisibile per $G(x)$. Quando la destinazione riceve il frame con checksum prova a dividerlo per $G(x)$. Se c'è un resto vuol dire che c'è stato un errore di trasmissione. L'algoritmo per calcolare il checksum è il seguente:

1. sia r il grado di $G(x)$, si aggiungano r bit con valore zero dopo la parte di ordine più basso del frame, così che venga a contenere $m + r$ bit e corrisponda al polinomio $x^rM(x)$;
2. si divida la sequenza di bit corrispondente a $x^rM(x)$ per la sequenza corrispondente a $G(x)$ usando la divisione modulo 2;
3. si sottragga il resto (che contiene sempre al massimo r bit) dalla sequenza corrispondente a $x^rM(x)$ usando la sottrazione in modulo 2. Il risultato è il frame con checksum pronto per la trasmissione. Chiamiamolo polinomio $T(x)$.

La Figura 3.9 illustra il calcolo per il frame 1101011011 usando il generatore:

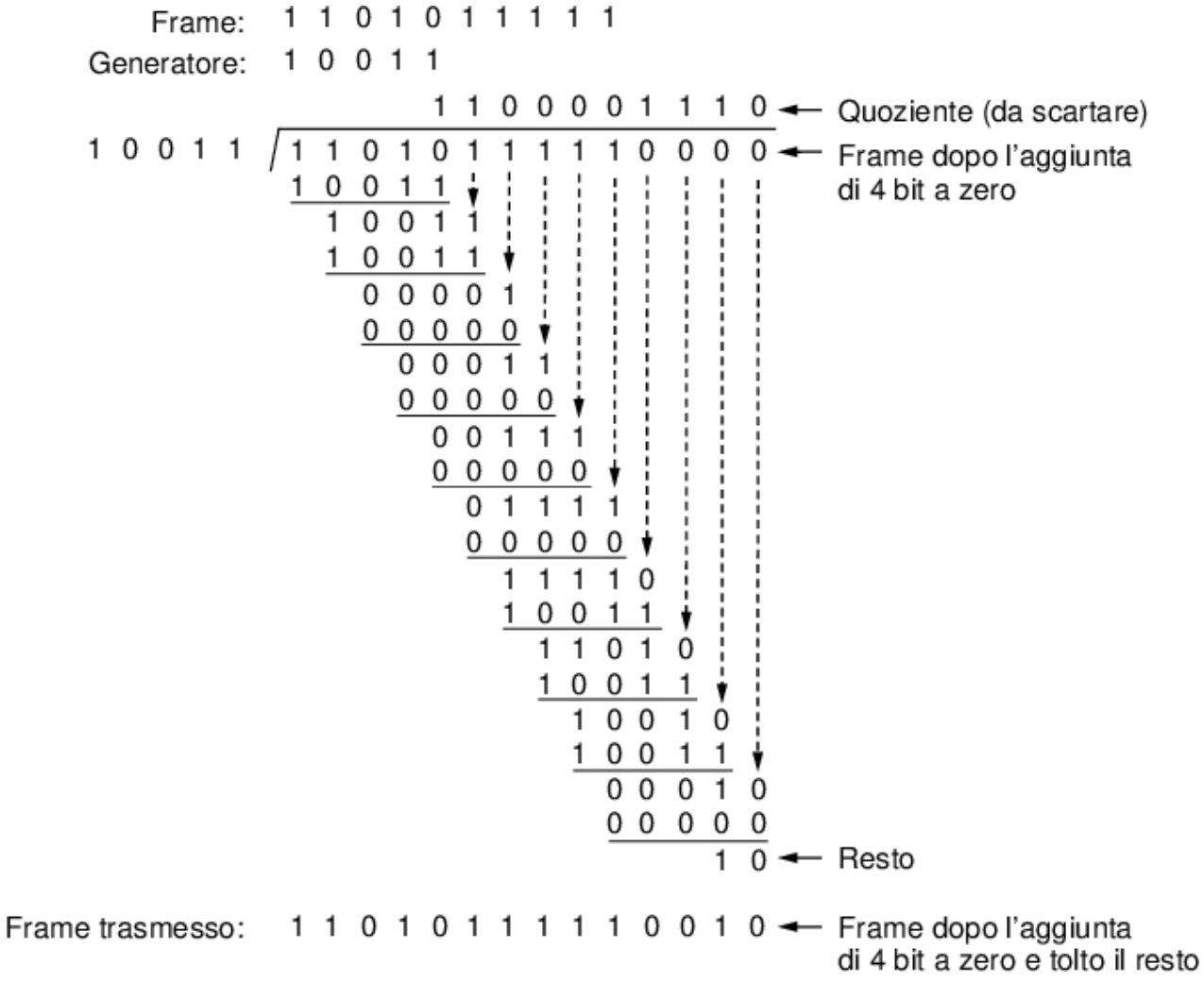


Figura 3.9 Esempio di calcolo del CRC.

$G(x) = x^4 + x + 1.$

Dovrebbe essere chiaro che $T(x)$ è divisibile (modulo 2) per $G(x)$. In ogni divisione, se si sottrae il resto dal dividendo, quello che resta è divisibile per il divisore. Vediamo un esempio: in base 10, se dividiamo 210.278 per 10.941 il resto è 2.399. Sottraendo 2.399 da 210.278, quello che rimane (207.879) è divisibile per 10.941.

Analizziamo la potenza di questo metodo. Quali tipologie di errori potrà rilevare? Immaginiamo che si verifichi un errore tale che al posto della sequenza di bit $T(x)$ arrivi una sequenza $T(x) + E(x)$. Ogni bit a 1 in $E(x)$ corrisponde a un bit che è stato invertito. Se ci sono k bit a 1 in $E(x)$, vuol dire che ci sono stati k errori su singoli bit. Un singolo errore a burst è caratterizzato da un 1 iniziale, una miscela di 0 e 1 e un 1 finale, mentre tutti gli altri bit sono a 0.

Dopo aver ricevuto il frame con checksum, la destinazione lo divide per $G(x)$, cioè calcola $[T(x)+E(x)]/G(x)$. Siccome $T(x)/G(x)$ ha resto 0, il risultato è pari al resto di $E(x)/G(x)$. Gli errori corrispondenti a polinomi che contengono $G(x)$ passeranno inosservati, mentre tutti gli altri verranno rilevati.

Se c'è stato un errore su un singolo bit, allora $E(x) = x^i$, dove i determina quale bit è in errore. Se $G(x)$ contiene due o più termini, non potrà mai dividere $E(x)$, così che tutti gli errori di singoli bit saranno rilevati.

Se si hanno 2 errori isolati, ognuno su un singolo bit, allora risulta che $E(x) = x^i+x^j$, con $i > j$, che possiamo anche scrivere come $E(x) = x^j (x^{i-j} + 1)$. Assumendo che $G(x)$ non sia divisibile per x, una condizione sufficiente per poter rilevare tutti gli errori doppi è che $G(x)$ non divida $x^k + 1$ per ogni k fino a un massimo valore di $i - j$ (cioè fino alla massima lunghezza del frame). Polinomi semplici e di basso grado che possono proteggere frame di considerevole lunghezza sono noti in letteratura. Per esempio, $x^{15}+x^{14}+1$ non divide alcun $x^k + 1$ per tutti i valori di k inferiori a 32.768.

Se c'è un numero dispari di bit in errore, $E(x)$ contiene un numero dispari di termini (per esempio $x^5 + x^2 + 1$, ma non $x^2 + 1$). Nessun polinomio con un numero dispari di termini ha $x + 1$ come fattore nel sistema a modulo 2. Perciò, basta usare un $G(x)$ che contenga $x + 1$ come fattore per riuscire a rilevare tutti gli errori che consistano in un numero dispari di bit invertiti.

In conclusione, è importante notare come una codifica polinomiale con r bit di controllo sia in grado di rilevare delle sequenze di errori di lunghezza $\leq r$. Un errore a burst di lunghezza k può essere rappresentato da $x^i(x^{k-1}+ ... + 1)$, dove i determina quanto lontano si trovi l'errore a burst dal termine più a destra della sequenza ricevuta. Se $G(x)$ contiene un termine x^0, non avrà x^1 come fattore; quindi, se il grado dell'espressione tra parentesi è inferiore al grado di $G(x)$, il resto non potrà mai essere zero.

Se la lunghezza di errori a burst è $r + 1$, il resto della divisione sarà zero se e solo se l'intero burst sarà identico a $G(x)$. Per definizione di burst, il primo e ultimo bit devono essere uguali a 1, quindi se c'è l'uguaglianza dipende solo dagli $r - 1$ bit intermedi. Se consideriamo tutte le combinazioni come ugualmente possibili, la probabilità che un frame non corretto venga accettato come valido è $1/2^{r-1}$.

Si può anche dimostrare che per un errore a burst di lunghezza superiore a $r + 1$ bit oppure per una serie di diversi errori più corti, la probabilità che un frame incorretto passi senza essere notato è pari a $1/2^r$, assumendo che tutte le combinazioni di bit siano ugualmente probabili. Certi polinomi sono diventati degli standard internazionali. Seguendo l'esempio di Ethernet per l'IEEE 802 è:

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16}+ x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5+ x^4+ x^2+ x^1 + 1$$

Questo polinomio ha molte proprietà utili, fra cui quella di riuscire a rilevare tutti gli errori a burst di lunghezza uguale o minore a 32 e tutti quelli che toccano un numero dispari di bit. Benché sia stato diffusamente impiegato negli anni '80, non rappresenta la scelta migliore. Effettuando un'esaustiva ricerca computazionale Castagnoli et al. (1993) e Koopman (2002) hanno trovato che il miglior CRC ha distanza di Hamming pari a 6 per dimensioni tipiche dei messaggi, mentre quella dello standard IEEE CRC-32 è solo 4.

Sebbene le operazioni necessarie per calcolare i checksum possano sembrare complicate Peterson e Brown (1961) hanno invece dimostrato che basta costruire un semplice circuito di registri a scorrimento per calcolare e controllare tutti i checksum via hardware. In pratica viene usata quasi

sempre una implementazione hardware. Dozzine di standard di rete includono vari CRC: quasi tutte le LAN (come Ethernet e 802.11) e i collegamenti punto a punto (come i pacchetti sulle SONET).

3.3 Protocolli data link elementari

Per introdurre l'argomento dei protocolli cominceremo analizzandone tre di complessità crescente. Per il lettore interessato, un simulatore di questi protocolli e degli altri che seguiranno è disponibile via Web (si veda la Prefazione). Prima di studiare i protocolli è utile esplicitare alcuni presupposti fondamentali del modello di comunicazione.

Innanzitutto assumiamo che nei livelli fisico, data link e di rete siano presenti dei processi indipendenti che comunicano passandosi dei messaggi l'uno con l'altro. Un'implementazione comune è mostrata nella Figura 3.10. I processi del livello fisico e alcuni del livello data link vengono eseguiti su un hardware specifico: una **NIC** (*network interface card*, scheda di rete). Gli altri processi dei livelli data link e di rete vengono eseguiti sulla CPU principale come parti del sistema operativo, dove i processi di livello data link prendono spesso la forma di *device driver*. Tuttavia, sono possibili anche altre implementazioni quali, per esempio, tre processi eseguiti su hardware dedicato, chiamato **network accelerator**, o tre processi eseguiti dalla CPU principale nel caso di un dispositivo radio implementatio via software (*software-defined radio*). In effetti, l'implemetazione più popolare si evolve di decennio in decennio di pari passo ai compromessi tecnologici. In ogni caso considerare i tre livelli come processi separati rende la trattazione concettualmente più chiara ed enfatizza l'indipendenza dei livelli.

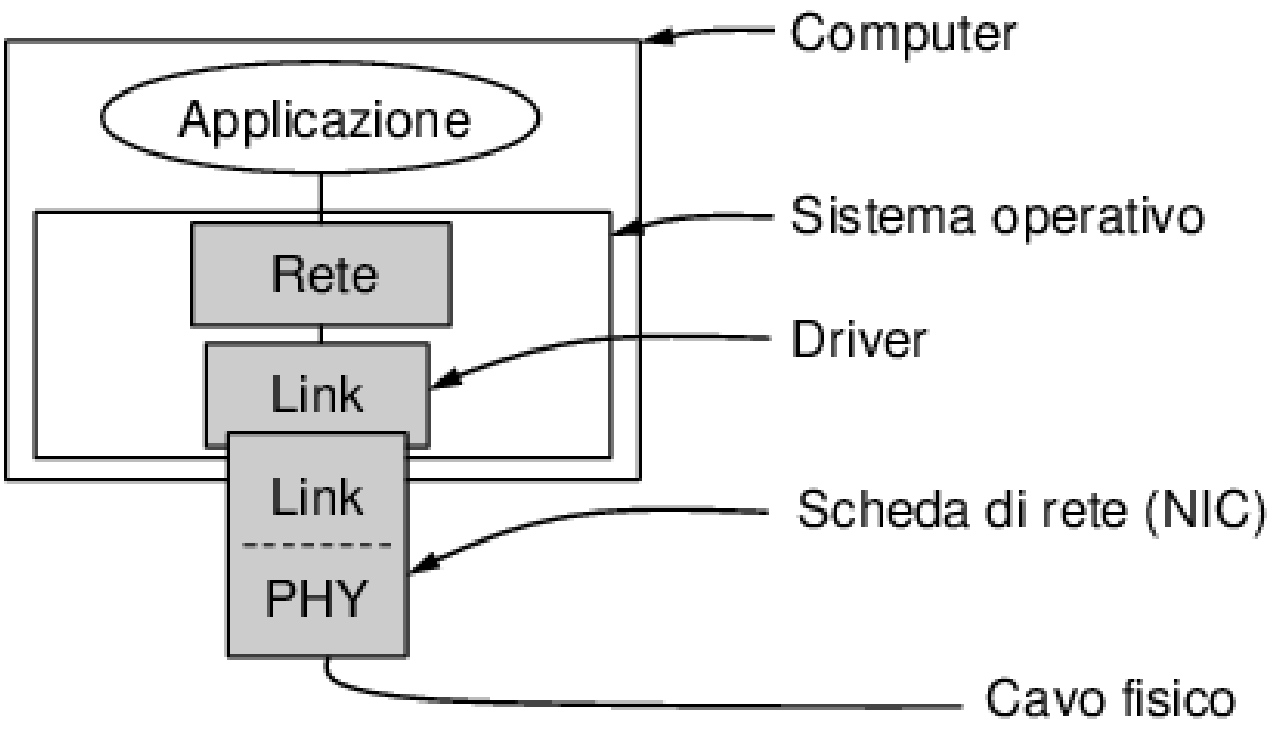


Figura 3.10 Implementazione dei livelli fisico, data link e di rete.

Un altro presupposto fondamentale è che la macchina *A* voglia mandare un consistente flusso di dati alla macchina *B*, usando un servizio affidabile e orientato alla connessione. Successivamente considereremo il caso in cui anche *B* voglia simultaneamente mandare dei dati ad *A*. Assumiamo inoltre che la sorgente *A* abbia a disposizione una quantità illimitata di dati pronti da trasmettere e che quindi non debba mai aspettare che i dati vengano prodotti. Quindi il livello di rete sarà sempre pronto a fornire i dati quando il livello data link della macchina *A* li richiederà (più avanti considereremo anche dei casi in cui questa ipotesi non vale).

Nel nostro schema le macchine non sono soggette a errori che bloccano il sistema operativo (*crash*), cioè i protocolli che trattiamo rilevano gli errori di comunicazione, ma non dei problemi causati dai crash dei computer e dai conseguenti reboot.

Per quanto concerne il livello data link, i bit contenuti nei pacchetti che gli arrivano dal livello di rete sono tutti da considerare come dati e quindi da trasferire al livello di rete della destinazione. Il fatto che il livello di rete possa considerare alcuni di questi bit come appartenenti all'intestazione dei pacchetti o altro non riguarda assolutamente il livello data link.

Quando il livello data link accetta un pacchetto, lo incapsula in un frame aggiungendogli un'intestazione (*header*) e una coda (*trailer*) (si veda anche la Figura 3.1). Il frame è quindi trasmesso al livello data link dell'altra macchina. Assumiamo che esista una libreria di procedure che contenga: *to_physical_layer,* per inviare un frame e *from_physical_layer* per ricevere un frame. Queste procedure calcolano e aggiungono in coda il checksum, usualmente in hardware, così che il software al livello data link non debba preoccuparsi di questa operazione. Il checksum può essere calcolato, per esempio, con l'algoritmo CRC discusso in precedenza.

All'inizio della trasmissione la destinazione non ha niente da fare, deve solamente aspettare che succeda qualcosa. Negli esempi di questo capitolo indicheremo che il livello data link aspetta che accada qualcosa con la chiamata a *wait_for_event*(*&event*). Questa procedura termina solo quando è successo qualcosa (per esempio è arrivato un frame). Al ritorno la variabile *event* è istanziata in modo da far capire che cosa sia successo. L'insieme dei possibili eventi differisce a seconda del protocollo e viene definito per ogni implementazione. In una situazione più realistica il livello data link non entra in un loop infinito in attesa di un evento, come indicato nel precedente esempio, ma invece aspetta un interrupt. Al suo arrivo interromperà qualunque cosa stia facendo e si dedicherà alla gestione del frame in arrivo. Nel seguito ignoreremo quest'ultimo dettaglio, che ha a che vedere con il parallelismo interno delle attività del livello data link. Per semplicità, supporremo di avere un processo dedicato a tempo pieno all'elaborazione di un solo canale.

Quando un frame arriva alla destinazione, l'hardware calcola il checksum. Se il checksum non è corretto (se cioè si è verificato un errore di trasmissione), il livello data link viene informato (*event* = *cksum_err*). Se il frame è arrivato senza problemi il livello data link è altresì informato (*event* = *frame_arrival*) e lo può acquisire per esaminarlo usando *from_physical_layer.* Quando il livello data link della destinazione ha prelevato un frame integro, come prima cosa ispeziona le informazioni di controllo nell'intestazione e, se tutto va bene, passa la parte relativa al pacchetto dei dati al livello di rete. In nessuna circostanza l'intestazione di un frame viene passata al livello di rete.

C'è un buon motivo per cui il livello di rete non deve mai ricevere alcuna parte dell'intestazione dei frame: tenere completamente separati i protocolli data link e di rete. Se il livello di rete non conosce alcun dettaglio del protocollo data link o del formato dei frame, questi ultimi potranno

essere cambiati senza richiedere un cambiamento nel software del livello di rete. Ciò accade ogni qualvolta una nuova scheda di rete viene installata in un computer. Avere un'interfaccia rigida fra i livelli di rete e data link semplifica notevolmente la progettazione del software, in quanto rende possibile lo sviluppo e l'evoluzione indipendente dei diversi protocolli coinvolti.

La Figura 3.11 mostra alcune dichiarazioni (in linguaggio C) comuni a molti protocolli discussi nel seguito. Vengono definite cinque strutture dati: *boolean, seq_nr, packet, frame_kind* e *frame.* Un *boolean* può assumere solo due valori: vero e falso. Un *seq_nr* è un piccolo intero usato per numerare i frame per distinguerli l'uno dall'altro. Questi numeri vanno da 0 fino a *MAX_SEQ* che è definito in ciascun protocollo che lo usa. Un *packet* è l'unità d'informazione che viene scambiata fra il livello di rete e il data link della stessa macchina, oppure fra i livelli di rete di macchine comunicanti. Nel nostro modello contiene *MAX_PKT* byte, ma in situazioni più realistiche può essere di lunghezza variabile.

```
#define MAX_PKT 1024    /* determina la dimensione in byte dei pacchetti */

typedef enum {false, true} boolean;    /* tipo booleano */

typedef unsigned int seq_nr;    /* numeri di sequenza o ack */

typedef struct {unsigned char data[MAX_PKT];} packet;    /* definizione del pacchetto */

typedef enum {data, ack, nak} frame_kind;    /* definizione del frame_kind */

typedef struct {    /* I frame sono trasportati in questo livello */

   frame_kind kind;    /* che tipo di frame è? */

   seq_nr seq;    /* numero di sequenza */
```

```
   seq_nr ack;    /* numero di acknowledgement */

   packet info;    /* pacchetto del livello di rete */

} frame;

/* Aspetta che accada un evento e restituisce il tipo di evento. */

void wait_for_event(event_type *event);

/* Prende un pacchetto dal livello di rete per trasmetterlo sul canale. */

void from_network_layer(packet *p);

/* Porta l'informazione dal frame appena arrivato verso il livello di rete. */

void to_network_layer(packet *p);

/* Prende un frame dal livello fisico e lo copia in r. */

void from_physical_layer(frame *r);

/* Passa il frame al livello fisico di trasmissione. */

void to_physical_layer(frame *s);

/* Fa partire l'orologio e abilita l'evento di timeout. */

void start_timer(seq_nr k);

/* Ferma l'orologio e disabilita l'evento di timeout. */

void stop_timer(seq_nr k);

/* Fa partire il timer ausiliario e abilita l'evento ack_timeout. */

void start_ack_timer(void);

/* Ferma il timer ausiliario e disabilita l'evento di ack_timeout. */
```

```
void stop_ack_timer(void);

/* Abilita il livello di rete a scatenare eventi network_layer_ready. */

void enable_network_layer(void);

/* Proibisce al livello di rete a scatenare eventi network_layer_ready. */

void disable_network_layer(void);

/* La macro inc viene espansa in linea: incrementa k in modo circolare. */

#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0
```

Figura 3.11 Alcune definizioni necessarie per i protocolli che seguono. Queste definizioni si trovano nel file protocol.h.

Un *frame* è composto da 4 campi: *kind, seq, ack* e *info.* I primi tre contengono le informazioni di controllo, l'ultimo contiene i dati da trasferire. I campi con le informazioni di controllo sono definiti come **frame header** (*intestazione del frame*).

Il campo *kind* indica se ci siano dati nel frame, poiché alcuni protocolli distinguono i frame che contengono solo informazioni di controllo da quelli che contengono anche i dati. I campi *seq* e *ack* sono usati rispettivamente per i numeri di sequenza e per l'acknowledgement. Il loro uso verrà descritto con maggior dettaglio più avanti. Il campo *info* di un frame dati contiene un singolo pacchetto e non è usato per i frame di controllo. Un'implementazione più realistica potrebbe usare un campo *info* a lunghezza variabile, omettendolo completamente per i frame di controllo.

A questo punto conviene enfatizzare la relazione che intercorre fra un pacchetto e un frame. Il livello di rete costruisce i pacchetti prendendo i messaggi dal livello di trasporto e aggiungendo una sua intestazione. Questo pacchetto viene passato al livello data link, che lo include nel campo *info* del frame in uscita. Quando il frame arriva a destinazione, il livello data link estrae il pacchetto dal frame e lo passa al livello di rete. In questo modo il livello di rete agisce come se le due macchine si stessero scambiando direttamente i pacchetti.

La Figura 3.11 contiene anche un elenco di procedure. Queste sono funzioni di libreria, i cui dettagli dipendono dall'implementazione. Non ci occuperemo qui dei dettagli interni di tali funzioni. La procedura *wait_for_event* contiene un loop infinito e aspetta che accada qualcosa, come abbiamo detto in precedenza. Le procedure *from_network_layer* e *to_network_layer* sono usate dal livello data link per trasferire i pacchetti, rispettivamente da e verso il livello di rete. Analogamente *from_physical_layer* e *to_physical_layer* trasferiscono i dati da e verso il livello fisico. In altre parole *from_network_layer* e *to_network_layer* si occupano dell'interfaccia fra i livelli 2 e 3, mentre *from_physical_layer* e *to_physical_layer* hanno a che fare con l'interfaccia fra i livelli 1 e 2.

Nella maggior parte dei protocolli si assume che il canale non sia affidabile e quindi in alcune occasioni perda degli interi frame. Per poter ripristinare tali anomalie il livello data link della sorgente deve far partire un timer interno ogni volta che invia un frame. Se non viene ricevuta alcuna risposta entro un tempo determinato, il timer scatta e il livello data link riceve un segnale di interrupt. Nei nostri protocolli questa funzione è gestita facendo in modo che la procedura *wait_for_event* possa terminare con *event* = *timeout.* Le procedure *start_timer* e *stop_timer* fanno rispettivamente partire e fermare il timer. L'evento di timeout si può verificare solo se il timer è stato fatto partire. È esplicitamente permesso chiamare *start_timer* mentre un timer è in attesa. Tale chiamata semplicemente esegue un reset del timer, così che il successivo timeout avverrà dopo un intero intervallo del timer (a patto che nel frattempo non avvenga un altro reset o il timer sia fermato).

Le procedure *start_ack_timer* e *stop_ack_timer* controllano un timer ausiliario per generare gli acknowledgement sotto certe condizioni.

Le procedure *enable_network_layer* e *disable_network_layer* sono usate in protocolli più sofisticati, dove non si assume che il livello di rete abbia sempre pacchetti da inviare. Quando il livello data link abilita il livello di rete vuol dire che questo è autorizzato a mandare interrupt se ha pacchetti da inviare. Indichiamo questo con *event* = *network_layer_ready.* Quando il livello di rete è disabilitato, invece, non può causare eventi di questo tipo. Usando con attenzione le funzioni per abilitare e disabilitare il livello di rete, il livello data link può evitare di essere sommerso di pacchetti di dati che potrebbero riempire il suo buffer.

I numeri di sequenza dei frame sono sempre compresi fra 0 e *MAX_SEQ* (estremi inclusi), dove *MAX_SEQ* dipende dal protocollo. Spesso è necessario far avanzare il numero di sequenza con incrementi di 1 e in modo circolare (cioè *MAX_SEQ* è seguito da 0). La macro *inc* esegue tale operazione. È stata definita come una macro per motivi di prestazioni, in quanto viene utilizzata all'interno del percorso critico; così facendo il codice viene sostituito risparmiando il costo della chiamata a una funzione. Infatti, come vedremo più avanti, il fattore limitante delle prestazioni della rete è spesso dato dal protocollo, quindi definire operazioni semplici (come l'incremento) tramite macro, non cambia la leggibilità del codice, ma aumenta le prestazioni.

Le dichiarazioni della Figura 3.11 fanno parte di tutti i protocolli che descriveremo in seguito. Sono state selezionate in modo da risparmiare spazio e fornire un utile riferimento, tuttavia dobbiamo pensarle come appartenenti a tutte le implementazioni di protocolli. In C ciò è possibile inserendo le definizioni in uno speciale file di intestazione, in questo caso *protocol.h,* e utilizzando la direttiva *#include* del preprocessore C, che permette di includere il file di intestazione nel file sorgente.

Un protocollo simplex utopistico

Come primo esempio considereremo il protocollo più semplice possibile, in quanto non si preoccupa che qualcosa possa andare male: i dati sono trasmessi in una sola direzione,[2] i livelli di rete della sorgente e della destinazione sono sempre pronti, il tempo per elaborare i dati può essere ignorato e i buffer sono infiniti. Infine, meraviglia: il canale di comunicazione tra i livelli data link

non danneggia né perde mai alcun frame. Questo è chiaramente un protocollo non realistico, per cui lo soprannomineremo "utopia" e lo useremo solo per mostrarne la struttura di base. La sua implementazione è rappresentata nella Figura 3.12.

```
/* Il protocollo 1 (utopia) permette la trasmissione dei dati in una sola direzione, dalla sorgente
alla destinazione. Si assume che il canale di comunicazione sia privo di errori e che
la destinazione sia in grado di elaborare tutto l'input a velocità infinita. Di conseguenza, la sorgente
semplicemente invia i dati alla massima velocità possibile da dentro un loop infinito. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender1(void)
{
  frame s;                                  /* buffer per i frame in uscita */
  packet buffer;                            /* buffer per i pacchetti in uscita */

  while (true) {
          from_network_layer(&buffer);      /* va a prendere qualcosa da trasmettere */
          s.info = buffer;                  /* copia il buffer in s per trasmetterlo */
          to_physical_layer(&s);            /* trasmette il buffer */
  }                                         /* Tomorrow, and tomorrow, and tomorrow,
                                               Creeps in this petty pace from day to day
                                               To the last syllable of recorded time.
                                                     - Macbeth, V, v */
}
void receiver1(void)
{
  frame r;
  event_type event;                         /* riempito da wait, ma non è implementato */

  while (true) {
          wait_for_event(&event);           /* la sola possibilità è frame_arrival */
          from_physical_layer(&r);          /* va a prendere il frame in arrivo */
          to_network_layer(&r.info);        /* passa i dati al livello di rete */
  }
}
```

Figura 3.12 Un protocollo simplex utopistico.

Il protocollo consiste di due distinte procedure, un mittente e un destinatario. Il mittente viene eseguito nel livello data link della macchina sorgente, mentre il destinatario nel livello data link della destinazione. Non usiamo, in questo caso, né numeri di sequenza né acknowledgement, così *MAX_SEQ* non serve. L'unico tipo di evento possibile è *frame_arrival* (cioè l'arrivo di un frame intatto).

Il mittente invia dati dall'interno di un ciclo infinito implementato con l'istruzione `while`, semplicemente mandando fuori i dati alla massima velocità possibile. Nel corpo del ciclo troviamo tre azioni: andare a prendere il pacchetto dal livello di rete (che sarà sempre pronto a fornirlo), costruire un frame in uscita usando la variabile *s*, spedire il frame. Questo protocollo usa solamente il campo *info* del frame, in quanto gli altri campi hanno a che vedere con il controllo degli errori e del flusso, ma non ci sono errori o restrizioni di flusso in questo caso.

Il destinatario ha una struttura ugualmente semplice. Inizialmente aspetta che succeda qualcosa, e l'unica possibilità è l'arrivo di un frame integro. In seguito il frame arriva e la procedura *wait_for_event* termina, con *event* impostato a *frame_arrival* (che comunque è ignorato). La chiamata a *from_physical_layer* rimuove dal buffer il frame appena arrivato e lo mette nella variabile *r*, dove il destinatario lo può prendere. Infine, la porzione di frame che contiene i dati viene passata al livello di rete, mentre il livello data link torna ad aspettare un nuovo frame; in pratica sospende la sua attività finché tale frame non arriva.

Il protocollo "utopia" non è realistico, perché non si occupa né del flusso di controllo né della correzione degli errori. Il processo che esegue è simile a quello di un servizio senza connessione e senza acknowledgement che confida nei livelli superiori per risolvere questi problemi, anche se persino un servizio senza acknowledgement dovrebbe preoccuparsi di rilevare gli errori.

Un protocollo simplex stop-and-wait per un canale privo di errori

Il problema che affrontiamo ora è come prevenire che il mittente sommerga il destinatario con dati trasmessi a velocità maggiore di quanto quest'ultimo sia in grado di elaborare, una situazione che ricorre facilmente nella pratica. Tuttavia si assume anche qui che il canale di comunicazione sia privo di errori e il traffico dati sia monodirezionale.

Una soluzione è che il destinatario sia abbastanza potente da elaborare un flusso continuo di frame uno in coda all'altro o, equivalentemente, che il livello data link sia abbastanza lento da permettere al destinatario di stare al passo. Deve avere un buffer e una capacità di elaborazione sufficienti a permettergli di funzionare alla velocità della linea trasmissiva e deve passare al livello di rete i frame che riceve con sufficiente velocità. Tuttavia, questo rappresenta il caso peggiore che richiede hardware dedicato e che spreca risorse se l'utilizzazione del collegamento è per la maggior parte lenta. Inoltre sposta semplicemente il problema al livello di rete.

Una soluzione più generale a questo problema consiste nel far sì che il destinatario mandi un feedback al mittente. Dopo aver passato un pacchetto al livello di rete, il destinatario restituisce al mittente un piccolo frame senza dati (*dummy*), che dà al mittente il permesso di inviare il frame successivo. Dopo aver mandato un frame, il mittente è obbligato dal protocollo ad aspettare finché non riceve il pacchetto dummy (in questo caso l'acknowledgement). Questo ritardo è un semplice esempio di un protocollo per il controllo del flusso.

Protocolli in cui il mittente manda un frame e poi aspetta l'acknowledgement prima di procedere sono chiamati **stop-and-wait** *(fermati e aspetta)*. La Figura 3.13 mostra un esempio di protocollo simplex di tipo stop-and-wait.

```
/* Il protocollo 2 (stop-and-wait) fornisce anch'esso un flusso unidirezionale di dati dalla sorgente
alla destinazione. Si assume ancora che il canale di comunicazione sia privo di errori, come per il
protocollo 1. Questa volta, però, la destinazione ha buffer di capacità finita e velocità di elaborazio-
ne finita. Perciò il protocollo deve esplicitamente prevenire la possibilità che la destinazione venga
sommersa di dati da parte della sorgente, nel caso in cui questa trasmetta a una velocità superiore
a quanto la destinazione riesce a elaborare */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender2(void)
{
  frame s;                          /* buffer per i frame in uscita */
  packet buffer;                    /* buffer per i pacchetti in uscita */
  event_type event;                 /* frame_arrival è la sola possibilità */

  while (true) {
      from_network_layer(&buffer);  /* va a prendere qualcosa da inviare */
      s.info = buffer;              /* copialo in s per la trasmissione */
      to_physical_layer(&s);        /* invia al livello fisico */
      wait_for_event(&event);       /* non procedere prima di aver ricevuto l'ok */
  }
}

void receiver2(void)
{
  frame r, s;                       /* buffer per i frame */
  event_type event;                 /* frame_arrival è l'unica possibilità */
  while (true) {
      wait_for_event(&event);       /* frame_arrival è l'unica possibilità */
      from_physical_layer(&r);      /* va a prendere il frame in arrivo */
      to_network_layer(&r.info);    /* passa i dati al livello di rete */
      to_physical_layer(&s);        /* manda un frame qualsiasi per svegliare la sorgente */
  }
}
```

Figura 3.13 Un protocollo simplex di tipo stop-and-wait.

Anche se il traffico di questo esempio è simplex (cioè va solamente dalla sorgente alla destinazione) i frame viaggiano in entrambe le direzioni. Conseguentemente, il canale di comunicazione fra i due livelli data link deve essere capace di trasmettere informazioni in modo bidirezionale, ma questo protocollo impone una rigida alternanza del flusso: prima il mittente manda un frame, poi il destinatario manda un frame, poi è di nuovo il turno del mittente, e così via. Un canale fisico half-duplex può bastare per questo tipo di comunicazione.

Come nel protocollo "utopia", il mittente comincia prendendo un pacchetto dal livello di rete, lo usa per costruire un frame e lo spedisce. Adesso, a differenza del protocollo "utopia", il mittente deve aspettare che arrivi un frame di acknowledgement prima di continuare il ciclo e andare a

prendere il pacchetto successivo dal livello di rete. Il livello data link della sorgente non deve neanche esaminare il frame in arrivo, in quanto c'è una sola possibilità: il frame è un acknowledgement.

L'unica differenza fra *receiver1* e *receiver2* è che dopo aver mandato il pacchetto al livello di rete, *receiver2* restituirà un frame di acknowledgement al mittente prima di entrare di nuovo nel loop di attesa. Solo l'arrivo del frame di acknowledgement è importante per il mittente, quindi la destinazione non dovrà preoccuparsi di aggiungervi informazioni particolari.

Un protocollo simplex stop-and-wait per un canale soggetto a rumore

Consideriamo adesso una situazione normale, in cui il canale di comunicazione commette degli errori. I frame possono essere danneggiati o addirittura persi completamente. Supponiamo che se un frame è danneggiato durante la trasmissione, l'hardware della destinazione riesca ad accorgersene calcolando il checksum. Nell'eventualità remota che il frame sia danneggiato in modo tale che il checksum risulti tuttavia corretto, circostanza piuttosto improbabile, questo protocollo (come tutti gli altri) potrebbe fallire, cioè potrebbe trasferire al livello di rete un pacchetto errato.

A prima vista può sembrare che una variazione del protocollo 2 possa aiutarci: l'aggiunta di un timer. L'idea è che quando la sorgente invia un frame, la destinazione risponda con un acknowledgement solo quando il frame sia stato ricevuto correttamente. Se arriva un frame danneggiato, allora la destinazione semplicemente lo scarta. Dopo un po' di tempo il timer della sorgente scatta e il frame viene inviato di nuovo. L'operazione si può ripetere finché il frame non arriva intatto a destinazione.

Lo schema appena descritto contiene però un errore fatale. Prima di continuare a leggere il lettore è invitato a pensarci e a scoprire che cosa potrebbe andare storto con questo modo di operare.

Per vedere che cosa possa andare storto, ricordiamo che il compito dei processi al livello data link è quello di fornire ai processi a livello di rete la possibilità di comunicare in modo trasparente e senza errori. Il livello di rete della macchina *A* passa una serie di pacchetti al suo livello data link, che deve preoccuparsi che una serie identica di pacchetti arrivi al livello di rete della macchina *B*, tramite il rispettivo livello data link. In particolare, il livello di rete di *B* non ha alcun modo per sapere se un pacchetto sia stato perso o duplicato, perciò il livello data link deve poter garantire che nessuna combinazione di errori, per quanto improbabile, possa causare la duplicazione di pacchetti.

Si consideri il seguente scenario.

1. Il livello di rete di *A* manda il pacchetto 1 al suo livello data link. Il pacchetto viene ricevuto correttamente da *B* e passato al livello di rete di *B*. *B* manda un frame di acknowledgement ad *A*.
2. Il frame di acknowledgement viene perso completamente. Non arriva proprio. La vita sarebbe più facile se il canale perdesse solo frame dati, ma purtroppo non è così: il canale non discrimina i frame fra di loro.

3. Dopo un po' il timer al livello data link di *A* scatta. Non avendo ricevuto un acknowledgement, *A* assume erroneamente che il frame dati non sia arrivato integro a destinazione e quindi lo invia di nuovo.
4. Il frame duplicato arriva integro al livello data link di *B* e lo passa, inavvertitamente, al livello di rete. Se *A* sta trasmettendo un file a *B*, quella parte del file sarà duplicata, cioè la copia del file ottenuta da *B* sarà sbagliata e l'errore passerà senza essere notato. In altre parole il protocollo avrà fallito.

Quello che serve, quindi, è un metodo che la destinazione possa usare per distinguere i frame che vede per la prima volta dalle ritrasmissioni. Il metodo ovvio è quello d'inserire un numero di sequenza nell'intestazione del frame quando viene inviato. La destinazione può quindi controllare i numeri di sequenza dei pacchetti man mano che arrivano e quindi riuscire a sapere se si tratta di un frame nuovo o di un duplicato da scartare.

Visto che è auspicabile avere un'intestazione per i frame, la domanda successiva sarà: qual è il minimo numero di bit necessari per il numero di sequenza? L'intestazione può fornire un bit, alcuni bit, un byte, un po' di byte a seconda del protocollo. L'importante è che siano in numero sufficiente ad avere sequenze di numeri abbastanza grandi da far in modo che il protocollo funzioni.

La sola possibile ambiguità in questo protocollo è fra il frame m e il suo successore diretto $m + 1$. Se il frame m viene perso o danneggiato, la destinazione non manderà l'acknowledgement e quindi la sorgente continuerà a trasmetterlo. Una volta che il frame sia stato ricevuto correttamente, la destinazione invierà l'acknowledgement alla sorgente. È qui che emerge il potenziale problema: a seconda che il frame di acknowledgement arrivi alla sorgente oppure no, questa continuerà inviando il frame $m + 1$ oppure ritrasmettendo il frame m.

L'evento che spinge la sorgente a cominciare a inviare il frame $m + 1$ è l'arrivo dell'acknowledgement del frame m. Questo implica che anche $m - 1$ sia già stato ricevuto, così come in seguito il suo acknowledgement (altrimenti la sorgente non avrebbe neanche cominciato a inviare m, figuriamoci $m + 1$). La conseguenza di tale ragionamento è che la possibile ambiguità da risolvere per ogni frame riguarda solamente il frame stesso e il suo immediato predecessore o successore, e non è mai fra il suo predecessore e il suo successore.

Un numero di sequenza con un solo bit (0 o 1) è quindi sufficiente. Quando arriva un frame con il numero di sequenza corretto, viene accettato, passato al livello di rete e quindi viene inviato l'acknowledgement. A questo punto il numero di sequenza atteso è incrementato in aritmetica modulo 2 (cioè lo 0 diventa 1 e viceversa). Ogni frame in arrivo con il numero di sequenza sbagliato viene scartato come duplicato, ma l'ultimo acknowledgement è ripetuto in modo che la sorgente possa scoprire se il frame sia stato ricevuto.

Un esempio di questo tipo di protocollo è mostrato nella Figura 3.14. Protocolli in cui la sorgente aspetta un acknowledgement positivo prima di avanzare nella trasmissione sono spesso chiamati **PAR** (*positive acknowledgement with retransmission*, risposta positiva con ritrasmissione) oppure **ARQ** (*automatic repeat request*, richiesta automatica di ripetizione). Come nel caso del protocollo 2, anche in questo i dati sono trasmessi in una sola direzione.

```
/* Il protocollo 3 (PAR) permette di gestire il flusso di dati unidirezionale su un canale non affidabile. */

#define MAX_SEQ 1                       /* deve essere a 1 per il protocollo 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
  seq_nr next_frame_to_send;            /* numero di sequenza del prossimo frame in uscita */
  frame s;                              /* variabile di servizio  */
  packet buffer;                        /* buffer per i pacchetti in uscita */
  event_type event;

  next_frame_to_send = 0;               /* inizializza i numeri di sequenza in uscita */
  from_network_layer(&buffer);          /* prende il primo pacchetto */
  while (true) {
     s.info = buffer;                   /* costruisce un frame per la trasmissione */
     s.seq = next_frame_to_send;        /* inserisce il numero di sequenza nel frame */
     to_physical_layer(&s);             /* lo trasmette */
     start_timer(s.seq);                /* se la risposta impiega troppo tempo, va in timeout */
     wait_for_event(&event);            /* frame_arrival, cksum_err, timeout */
     if (event == frame_arrival) {
        from_physical_layer(&s);        /* prende l'acknowledgement */
        if (s.ack == next_frame_to_send) {
            stop_timer(s.ack);          /* ferma il timer */
            from_network_layer(&buffer);/* prende il prossimo pacchetto da inviare */
            inc(next_frame_to_send);    /* inverte next_frame_to_send */
        }
     }
  }
}

void receiver3(void)
{
  seq_nr frame_expected;
  frame r, s;
  event_type event;

  frame_expected = 0;
  while (true) {
     wait_for_event(&event);            /* possibilità: frame_arrival, cksum_err */
     if (event == frame_arrival) {      /* è arrivato un frame valido */
        from_physical_layer(&r);        /* va a prendere il frame che è arrivato */
        if (r.seq == frame_expected) {  /* questo è quello che aspettavamo */
            to_network_layer(&r.info);  /* passa i dati al livello di rete */
            inc(frame_expected);        /* la prossima volta aspetta in successivo num. di sequenza */
        }
        s.ack = 1 - frame_expected;     /* dice quale frame ha ricevuto l'acknowledgement */
        to_physical_layer(&s);          /* manda l'acknowledgement */
     }
  }
}
```

Figura 3.14 Un protocollo con acknowledgement positivo con ritrasmissione.

Il protocollo 3 differisce dai suoi predecessori per il fatto che sorgente e destinazione hanno una variabile che deve essere ricordata mentre il livello data link è in stato di attesa. La sorgente memorizza il numero di sequenza del frame successivo da inviare dentro a *next_frame_to_send*, la

destinazione invece memorizza il numero di sequenza che si aspetta di ricevere dentro *frame_expected.* Ogni protocollo ha una piccola fase di inizializzazione prima di entrare nel loop infinito.

Dopo la trasmissione del frame, la sorgente fa partire il timer. Se il timer era già stato avviato, esegue un reset e lo fa ripartire. L'intervallo di tempo dovrebbe essere scelto in modo da permettere ai frame di arrivare a destinazione ed esservi elaborati (consideriamo il caso peggiore come tempo di elaborazione), e al frame di acknowledgement di tornare indietro. Solo dopo un simile intervallo di tempo è ragionevole assumere che o il frame o il suo acknowledgement siano stati persi e che sia quindi necessario inviare un duplicato. Se l'intervallo di timeout è impostato a un valore troppo breve, la sorgente invierà dei frame inutili. Questi frame non incideranno sulla correttezza del protocollo, ma avranno delle ricadute sulle prestazioni.

Dopo aver trasmesso un frame e fatto partire il timer, la sorgente aspetta che accada qualcosa. Le possibilità sono solamente tre: l'arrivo di un frame di acknowledgement intatto, l'arrivo di un frame di acknowledgement con problemi, lo scattare di un timeout. Nel primo caso la sorgente prende il pacchetto successivo dal livello di rete e lo mette nel buffer, scrivendolo al posto del pacchetto precedente. Se invece arriva un frame di acknowledgement danneggiato oppure non arriva alcun frame, allora, né il buffer, né il numero di sequenza vengono cambiati e viene inviato un duplicato.

Quando un frame valido arriva alla destinazione, il suo numero di sequenza viene controllato per vedere se si tratta di un duplicato. Se non è un duplicato, il frame viene accettato e passato al livello di rete, quindi è generato un acknowledgement. I frame duplicati o danneggiati non vengono passati al livello di rete, ma fanno in modo che venga generato l'acknowledgement dell'ultimo frame corretto per segnalare alla sorgente di passare al frame successivo o ritrasmettere il frame danneggiato.

3.4 Protocolli a finestra scorrevole

Nei protocolli precedenti i frame di dati venivano trasmessi solamente in una direzione (simplex). Nella maggior parte delle situazioni reali risulta invece necessario poter trasmettere in entrambe le direzioni (duplex). Un modo per ottenere una trasmissione di dati full-duplex è quello di avere due canali di trasmissione e usare ciascuno di essi per una trasmissione simplex (in direzioni diverse). Si ottengono così due canali fisici separati, ognuno con un canale "di andata" (per i dati) e uno "di ritorno" (per gli acknowledgement). In entrambi i casi la banda del canale di ritorno risulta quasi interamente sprecata.

Un'idea migliore consiste nell'usare lo stesso collegamento per entrambe le direzioni di comunicazione. Dopotutto, già nei protocolli 2 e 3 si usa lo stesso circuito per trasmettere i frame in entrambe le direzioni, inoltre il canale di andata e quello di ritorno hanno la stessa capacità. In questo modello i frame dati che passano da *A* a *B* sono mescolati a quelli di acknowledgement da *A* a *B*. Guardando al campo *kind* nell'intestazione del frame in arrivo, il ricevente può determinare se si tratta di dati oppure acknowledgement.

Dopo aver inframezzato dati e frame di controllo sullo stesso canale possiamo aggiungere un'ulteriore miglioria. Quando arriva un frame di dati, la destinazione non invia subito un frame di controllo separato, ma aspetta che il livello di rete gli passi il pacchetto successivo. L'acknowledgement viene aggiunto al frame di dati in uscita usando il campo *ack* nell'intestazione del frame. In questo modo l'acknowledgement si procura un passaggio gratis insieme al successivo frame dati trasmesso. La tecnica che consiste nel ritardare gli acknowledgement in uscita in modo da agganciarli al successivo frame di dati è chiamata **piggybacking** (letteralmente *trasportare in groppa*).

Il vantaggio principale del piggybacking rispetto agli acknowledgement separati sta nel miglior uso della banda disponibile. Il campo *ack* nell'intestazione del frame costa solamente alcuni bit, mentre costruire un frame separato richiederebbe un'intestazione, un acknowledgement e un checksum. Inoltre un minor numero di frame inviati significa anche un minor numero di interrupt di tipo "frame in arrivo", e quindi magari anche un minor numero di buffer nella destinazione, a seconda di com'è strutturato il software. Nel prossimo protocollo che esamineremo il campo piggyback costerà solo 1 bit nell'intestazione del frame, ed è difficile che possa costare più di qualche bit.

A ogni modo il campo piggyback solleva una complicazione non presente nei protocolli con acknowledgement separato. Quanto deve aspettare il livello data link per disporre di un pacchetto su cui effettuare il piggybacking dell'acknowledgement? Se il livello data link aspetta più a lungo del periodo di timeout della sorgente, il pacchetto verrà ritrasmesso, rendendo vano l'acknowledgement. Se il livello data link fosse un oracolo e potesse predire il futuro, saprebbe quanto tempo intercorrerà per avere il prossimo pacchetto dati dal livello di rete e quindi potrebbe decidere se è meglio aspettarlo o inviare subito un acknowledgement separato. Ovviamente il livello data link non può predire il futuro, così è necessario prevedere uno schema specifico per questa situazione, per esempio aspettare alcuni millisecondi. Se il nuovo pacchetto arriva

rapidamente, viene fatto il piggybacking dell'acknowledgement. Altrimenti, se alla fine del periodo di attesa il pacchetto non è arrivato, il livello data link invia un frame di acknowledgement separato.

I successivi tre protocolli sono di tipo bidirezionale e appartengono alla classe chiamata a **finestra scorrevole** (*sliding window*). Le differenze fra i tre protocolli riguardano l'efficienza, la complessità e i requisiti in termini di buffer, come discuteremo nel seguito. In questi, come in tutti i protocolli a finestra scorrevole, ogni frame in uscita contiene un numero di sequenza che va da 0 fino a un valore massimo. Il massimo è di solito esprimibile come $2^n - 1$, così che il numero di sequenza entra esattamente in un campo di n bit. I protocolli stop-and-wait a finestra scorrevole a 1 bit hanno $n = 1$, restringendo i numeri di sequenza a 0 e 1. Esistono delle versioni più sofisticate che usano valori arbitrari di n.

L'essenza dei protocolli a finestra scorrevole è che, a ogni istante, la sorgente tiene traccia di un insieme di numeri di sequenza corrispondenti ai frame che è autorizzata a inviare. Si dice che questi frame si trovano nella **finestra di invio**. In modo analogo la destinazione tiene traccia della **finestra di ricezione**, corrispondente all'insieme dei frame che può accettare. Le finestre di invio e di ricezione non devono necessariamente avere gli stessi limiti inferiori o superiori e neppure la stessa dimensione. In alcuni protocolli tali finestre hanno delle dimensioni fisse, ma in altri possono avere dimensioni variabili nel tempo.

Anche se questi protocolli danno più libertà al livello data link per quanto riguarda l'ordine in cui mandare o ricevere dei frame, non per questo dobbiamo trasgredire alla regola che impone al protocollo di passare i pacchetti al livello di rete della destinazione nello stesso ordine in cui erano stati passati al livello data link della sorgente. Non abbiamo neanche abbandonato l'ipotesi che il canale fisico di comunicazione sia assimilabile a un cavo, cioè che debba trasferire i frame nell'ordine in cui sono stati inviati.

Il numero di sequenza nella finestra di invio rappresenta i frame che sono già stati trasmessi correttamente oppure che sono in transito, ma non hanno ancora ricevuto l'acknowledgement corrispondente. Quando un nuovo pacchetto arriva dal livello di rete, gli viene assegnato il numero di sequenza successivo in ordine crescente e il limite superiore della finestra è incrementato di uno. Quando invece arriva un acknowledgement, s'incrementa di uno il limite inferiore della finestra. In questo modo la finestra continua a mantenere la lista dei frame che ancora necessitano di acknowledgement. La Figura 3.15 ne mostra un esempio.

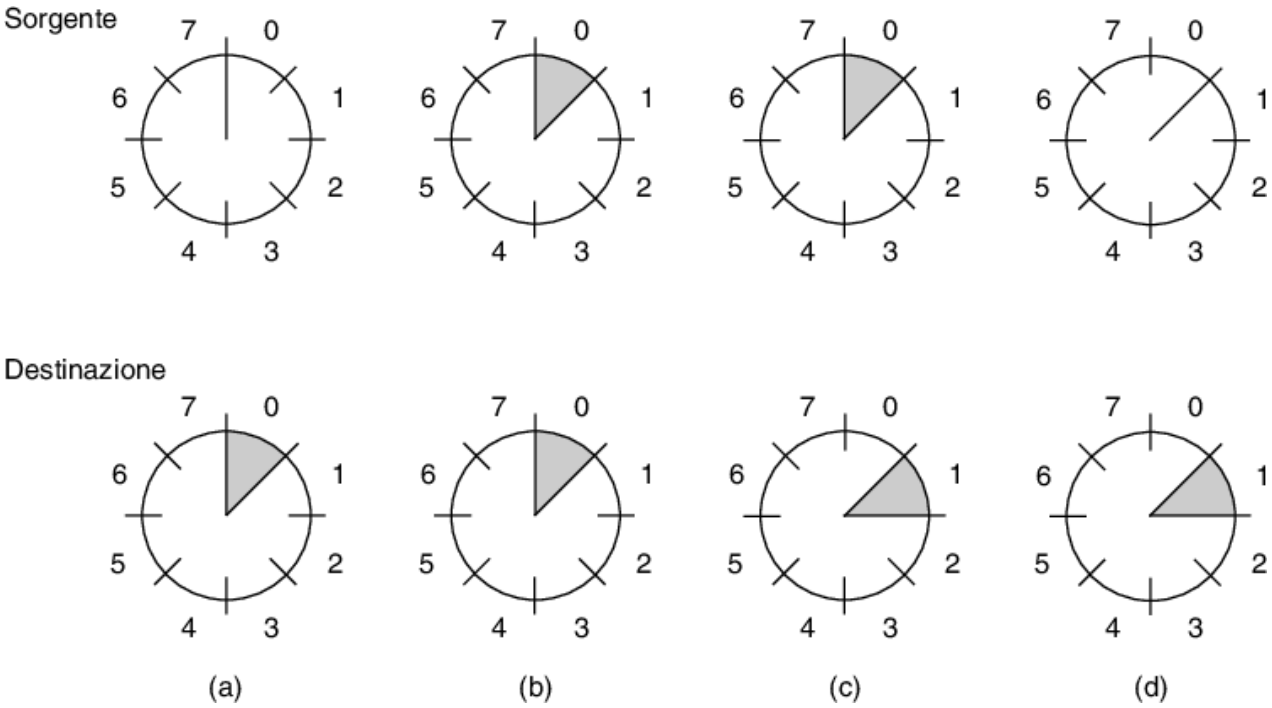


Figura 3.15 Una finestra scorrevole con dimensione 1 e numero di sequenza a 3 bit. (a) Inizialmente. (b) Dopo l'invio del primo frame. (c) Dopo la ricezione del primo frame. (d) Dopo la ricezione del primo acknowledgement.

Visto che i frame all'interno della finestra di invio potrebbero andare persi o danneggiati durante la trasmissione, la sorgente li deve mantenere tutti nella sua memoria per l'eventuale ritrasmissione. Quindi, se la massima dimensione della finestra è *n*, la sorgente avrà bisogno di disporre di *n* buffer per contenere i frame in attesa di acknowledgement. Se la finestra cresce fino alla sua massima dimensione, il livello data link è costretto a chiudere forzatamente il livello di rete finché non si libera almeno un buffer.

La finestra di ricezione del livello data link tiene traccia dei frame che possono essere accettati dalla destinazione. Ogni frame al di fuori della finestra viene scartato senza commenti. Quando viene ricevuto un frame con numero di sequenza uguale al limite inferiore della finestra il frame è passato al livello di rete e la finestra è ruotata di una posizione. In tutti questi casi viene generato un acknowledgement in modo che la sorgente sappia come procedere. Si noti che fissare una dimensione della finestra pari a 1 significa stabilire che il livello data link accetta solo i frame nella giusta sequenza, mentre con finestre più larghe questo non è più vero. Il livello di rete riceve sempre i dati nell'ordine corretto, indipendentemente dalla dimensione della finestra nel livello data link.

La Figura 3.15 mostra un esempio di finestra con dimensione massima uguale a 1. Inizialmente non ci sono frame in uscita, quindi i limiti inferiore e superiore della finestra di invio sono uguali, ma con il passare del tempo la situazione si evolve come mostrato. Diversamente dalla finestra di invio, la finestra di ricezione rimane sempre alla sua dimensione iniziale, ruotando quando il frame successivo viene accettato e consegnato al livello di rete.

Un protocollo a finestra scorrevole a 1 bit

Prima di affrontare il caso generale, esaminiamo un protocollo a finestra scorrevole con una finestra di dimensione massima uguale a 1. Un protocollo simile usa il metodo stop-and-wait, il che significa che prima la sorgente trasmette un frame e poi aspetta di ricevere il corrispondente acknowledgement per poter continuare a trasmettere il successivo frame.

La Figura 3.16 mostra un protocollo di questo tipo. Come per gli altri protocolli, nella prima parte vengono definite alcune variabili. *Next_frame_to_send* indica quale frame la sorgente vuole trasmettere. Analogamente, *frame_expected* indica il frame che la destinazione si aspetta di ricevere. In entrambi i casi le sole possibilità sono 0 e 1.

```
/* Il protocollo 4 (a finestra scorrevole) è bidirezionale. */

#define MAX_SEQ 1                                   /* deve essere 1 per il protocollo 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void protocol4 (void)
{
  seq_nr next_frame_to_send;                        /* 0 o 1 */
  seq_nr frame_expected;                            /* 0 o 1 */
  frame r, s;                                       /* variabili di servizio */
  packet buffer;                                    /* pacchetto corrente in invio */
  event_type event;

  next_frame_to_send = 0;                           /* prossimo frame nel flusso in uscita */
  frame_expected = 0;                               /* il frame atteso come prossimo */
  from_network_layer(&buffer);                      /* prende un pacchetto dal livello di rete */)
  s.info = buffer;                                  /* prepara per mandare il frame iniziale */
  s.seq = next_frame_to_send;                       /* inserisce il numero di sequenza nel frame */
  s.ack = 1 - frame_expected;                       /* acknowledgement in piggyback */
  to_physical_layer(&s);                            /* trasmette il frame */
  start_timer(s.seq);                               /* fa partire il timer */

  while (true) {
      wait_for_event(&event);                       /* frame_arrival, cksum_err, o timeout */
      if (event == frame_arrival) {                 /* un frame è arrivato senza danni */
          from_physical_layer(&r);                  /* va a prenderlo */
          if (r.seq == frame_expected) {            /* gestisce il flusso in ingresso */)
              to_network_layer(&r.info);            /* passa il pacchetto al livello di rete */)
              inc(frame_expected);                  /* inverte num. seq. atteso come prossimo */
          }
          if (r.ack == next_frame_to_send) {        /* gestisce il flusso di frame in uscita */
              stop_timer(r.ack);                    /* ferma il timer */
              from_network_layer(&buffer);          /* successivo pacchetto al livello di rete */)
              inc(next_frame_to_send);              /* inverte il numero di sequenza della sorgente */
          }
      }
      s.info = buffer;                              /* costruisce il frame in uscita */
      s.seq = next_frame_to_send;                   /* inserisce il numero di sequenza */
      s.ack = 1 - frame_expected;                   /* il num. seq. dell'ultimo frame ricevuto */
      to_physical_layer(&s);                        /* trasmette il frame */
      start_timer(s.seq);                           /* fa partire il timer */
  }
}
```

Figura 3.16 Un protocollo a finestra scorrevole a 1 bit.

In circostanze normali, uno dei due livelli data link parte e trasmette il primo frame. In altre parole, solo uno dei programmi del livello data link dovrebbe contenere anche all'esterno del ciclo principale le chiamate alle procedure *to_physical_layer* e *start_timer*. Se, invece, entrambi i livelli data link partono simultaneamente, allora abbiamo una situazione particolare, come discusso nel seguito. La macchina che parte prende il primo pacchetto dal suo livello di rete, lo usa per costruire

un frame e lo invia. Quando questo (o un altro) frame arriva a destinazione, il livello data link della destinazione controlla per vedere se è un duplicato (come nel protocollo 3). Se il frame è quello che la destinazione si aspettava, lo passa al livello di rete e la finestra della destinazione viene fatta scorrere in avanti.

Il campo di acknowledgement contiene il numero dell'ultimo frame ricevuto senza errori. Se questo numero concorda con il numero di sequenza che la sorgente vuole trasmettere, allora la sorgente sa di aver finito il lavoro con il frame memorizzato in *buffer* e può prendere il successivo pacchetto dal suo livello di rete. Se i numeri di sequenza non concordano, allora deve provare a inviare di nuovo lo stesso frame. Quando un frame viene ricevuto, un altro frame è restituito.

Esaminiamo il protocollo 4 per vedere quanto sia resistente e capace di affrontare anche scenari patologici. Prendiamo la situazione in cui il computer *A* tenta di inviare il frame 0 al computer *B,* mentre *B* vuole mandare il frame 0 ad *A*. Supponiamo che *A* mandi il frame a *B,* ma l'intervallo di timeout di *A* sia leggermente troppo breve. Di conseguenza, *A* può generare timeout a raffica mandando una serie di frame identici, tutti con *seq* = 0 e *ack* = 1.

Quando il computer *B* riceve il primo frame valido lo accetta e imposta la variabile *frame_expected* a 1. Tutti i frame successivi saranno rifiutati, perché *B* ora si aspetta dei frame con numero di sequenza 1 e non 0. Inoltre, visto che i duplicati hanno *ack* = 1 e *B* continua ad aspettare l'acknowledgement di 0, *B* non prenderà altri pacchetti dal suo livello di rete.

Dopo aver rifiutato ogni frame duplicato in arrivo, *B* manda ad *A* un frame contenente *seq* = 0 e *ack* = 0. Infine, uno di questi arriva correttamente ad *A* e fa sì che *A* mandi il pacchetto successivo. Nessuna combinazione di pacchetti persi o di timeout prematuri può causare la trasmissione di pacchetti duplicati al livello di rete, la perdita di pacchetti oppure situazioni di deadlock. Il protocollo è quindi corretto.

Una situazione particolare si ha quando entrambi i soggetti mandano simultaneamente un pacchetto iniziale. La difficoltà della sincronizzazione è illustrata dalla Figura 3.17: nella parte (a) viene mostrata la normale operatività del protocollo, in (b) si vede invece la situazione particolare. Se *B* aspetta il primo frame da *A* prima di inviarne uno dei suoi, la sequenza è come mostrato in (a), e ogni frame viene accettato. Se invece *A*e *B* iniziano simultaneamente la comunicazione, i primi frame si incrociano e il livello data link raggiunge la situazione (b). In (a) l'arrivo di ogni frame porta un nuovo pacchetto per il livello di rete. In (b) metà dei frame contengono dei duplicati, anche quando non ci sono errori di trasmissione. Situazioni simili possono accadere in conseguenza a timeout prematuri, anche se un soggetto inizia a trasmettere nettamente prima dell'altro. In effetti, se accadono molti timeout prematuri i frame possono essere trasmessi anche tre o più volte, sprecando banda preziosa.

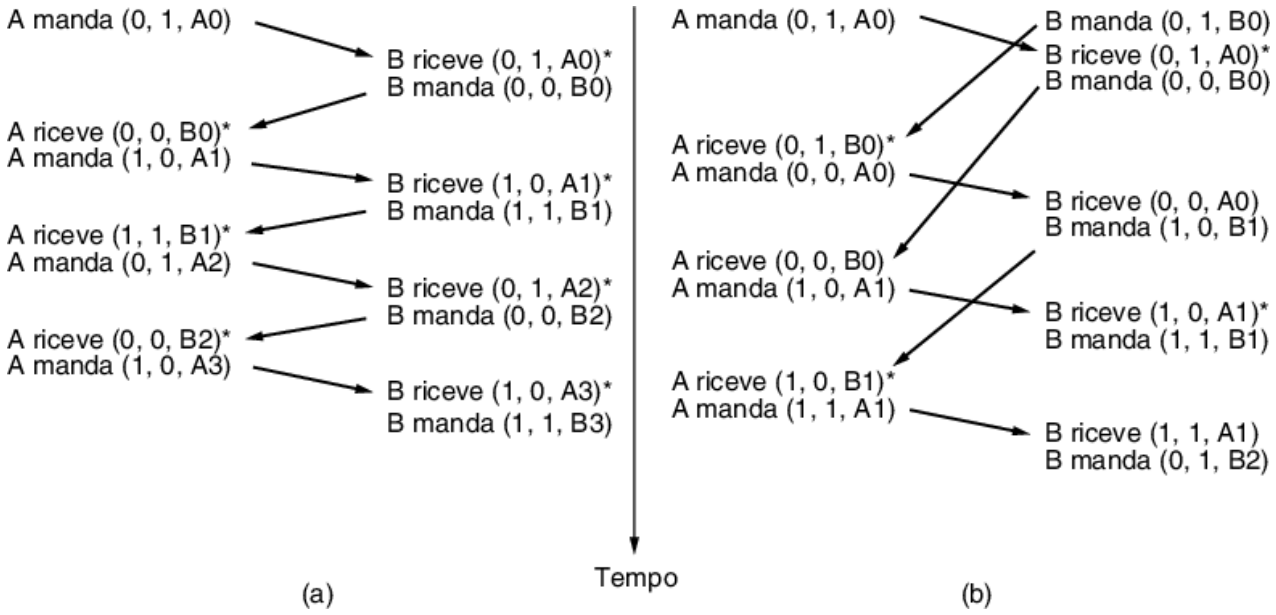


Figura 3.17 Due scenari per il protocollo 4. (a) Caso normale. (b) Caso con problema. La notazione è (seq. ack, numero di pacchetto). Un asterisco indica il momento in cui il livello di rete accetta un pacchetto.

Un protocollo che usa go-back-n

Finora abbiamo tacitamente supposto che il tempo necessario a trasmettere un frame alla destinazione sommato al tempo per l'acknowledgement sia trascurabile. In alcuni casi questa ipotesi è chiaramente falsa. In tali situazioni, i tempi lunghi di andata e ritorno dei frame possono avere pesanti implicazioni in termini di efficienza nell'uso della banda. Come esempio consideriamo un canale satellitare a 50 kbps con un ritardo di propagazione totale di 500 millisecondi. Immaginiamo di usare il protocollo 4 per inviare dei frame da 1000 bit via satellite. La sorgente comincia a trasmettere a $t = 0$. Al tempo $t = 20$ millisecondi il frame è stato completamente trasmesso. Il frame viene ricevuto in modo completo dalla destinazione non prima di $t = 270$ millisecondi e l'acknowledgement torna indietro non prima di $t = 520$ millisecondi. Questo nelle condizioni migliori, in cui non ci sono ritardi da parte della destinazione e il frame di acknowledgement è corto; ciò significa che la sorgente è rimasta bloccata per 500/520 ms o il 96% del tempo. In altre parole, è stato usato solo il 4% della banda disponibile. Chiaramente la combinazione di tempi di transito lunghi, larga banda e frame corti è disastrosa in termini di efficienza.

Il problema descritto sopra può essere visto come una conseguenza della regola che richiede alla sorgente di aspettare l'acknowledgement della destinazione prima di mandare il frame successivo. Se alleggeriamo questo requisito possiamo migliorare di molto l'efficienza. In pratica, la soluzione sta nel permettere alla sorgente di mandare un numero w di frame maggiore di uno, prima di bloccarsi. Con una scelta abbastanza ampia di w la sorgente è in grado di continuare a trasmettere frame, poiché gli acknowledgement arriveranno per i frame precedenti prima che la finestra si riempia, facendo in modo che la sorgente non si blocchi.

Per calcolare un valore opportuno di w è necessario conoscere quanti frame porta il canale tra la sorgente e la destinazione. Questa capacità è determinata dalla banda misurata in bit/s moltiplicata per il tempo di transito in una direzione. Questa grandezza è chiamata **bandwith-delay product** *(prodotto banda-ritardo)* del link. Se viene divisa per il numero di bit per frame si ottiene il numero di frame: BD. Quindi w dovrebbe essere pari a $2BD + 1$. Il numero di frame che possono essere gestiti con una sorgente che trasmetta continuamente, considerando il tempo di andata e ritorno per ricevere l'acknowledgement, è pari al doppio del prodotto banda-ritardo. Il "+1" è dovuto al fatto che un frame di acknowledgement non viene inviato fino a quando non venga ricevuto un frame completo.

Per esempio, un collegamento con una larghezza di banda di 50 kbps e un tempo di transito in una direzione di 250 ms ha un prodotto banda-ritardo pari a 12,5 kbit o 12,5 frame di 1000 bit ognuno. Quando è arrivato ad aver spedito 26 frame siamo a t = 520 ms e arriva l'acknowledgement del frame 0. Da lì in avanti gli acknowledgement arrivano ogni 20 ms; in questo modo la sorgente ha il permesso di proseguire quando ne ha bisogno. In ogni momento ci sono 25 o 26 frame in attesa di acknowledgement. Detto in altri termini, la dimensione massima della finestra della sorgente è 26.

Per finestre di dimensione minore l'utilizzo del collegamento sarà inferiore al 100%, poiché la sorgente rimarrà a volte bloccata. Possiamo definire l'utilizzazione come la frazione di tempo in cui la sorgente non è bloccata:

$$\text{utilizzazione del collegamento} \leq w/(1 + 2BD)$$

Questo valore rappresenta un limite superiore, in quanto non tiene conto del tempo di elaborazione dei frame e tratta gli acknowledgement come aventi dimensione zero. Questa equazione mostra la necessità di avere una finestra w larga quando il prodotto banda-ritardo è grande. Se il ritardo è grande, la sorgente satura rapidamente la sua finestra, anche con una banda moderata, come avviene per esempio nei canali satellitari. Se la banda è larga, anche in presenza di un piccolo ritardo, la sorgente satura la finestra velocemente, a meno che questa non sia di grandi dimensioni. Per esempio, un collegamento di 1 Gbps con un ritardo di 1 ms richiede 1 megabit. Con il protocollo stop-and-wait per il quale w = 1, se anche un solo frame ha un ritardo di propagazione, l'efficienza crolla sotto il 50%.

Questa tecnica che consiste nel tenere più frame in viaggio è un esempio di **pipelining**. Il pipelining dei frame su un canale di comunicazione non affidabile comporta alcuni seri problemi. Che cosa succede quando un frame nel mezzo di una lunga sequenza viene danneggiato o perso? Una gran quantità di frame arriverà alla destinazione prima che la sorgente riesca a sapere che qualcosa è andato storto. Quando un frame danneggiato arriva a destinazione deve ovviamente essere scartato. Che cosa deve fare, però, la destinazione con i frame corretti che seguono quello danneggiato? Si ricordi che il livello data link della destinazione è obbligato a mandare i pacchetti al livello di rete nella corretta sequenza.

Per il ripristino degli errori in presenza di pipelining sono disponibili due approcci base, entrambi mostrati nella Figura 3.18.

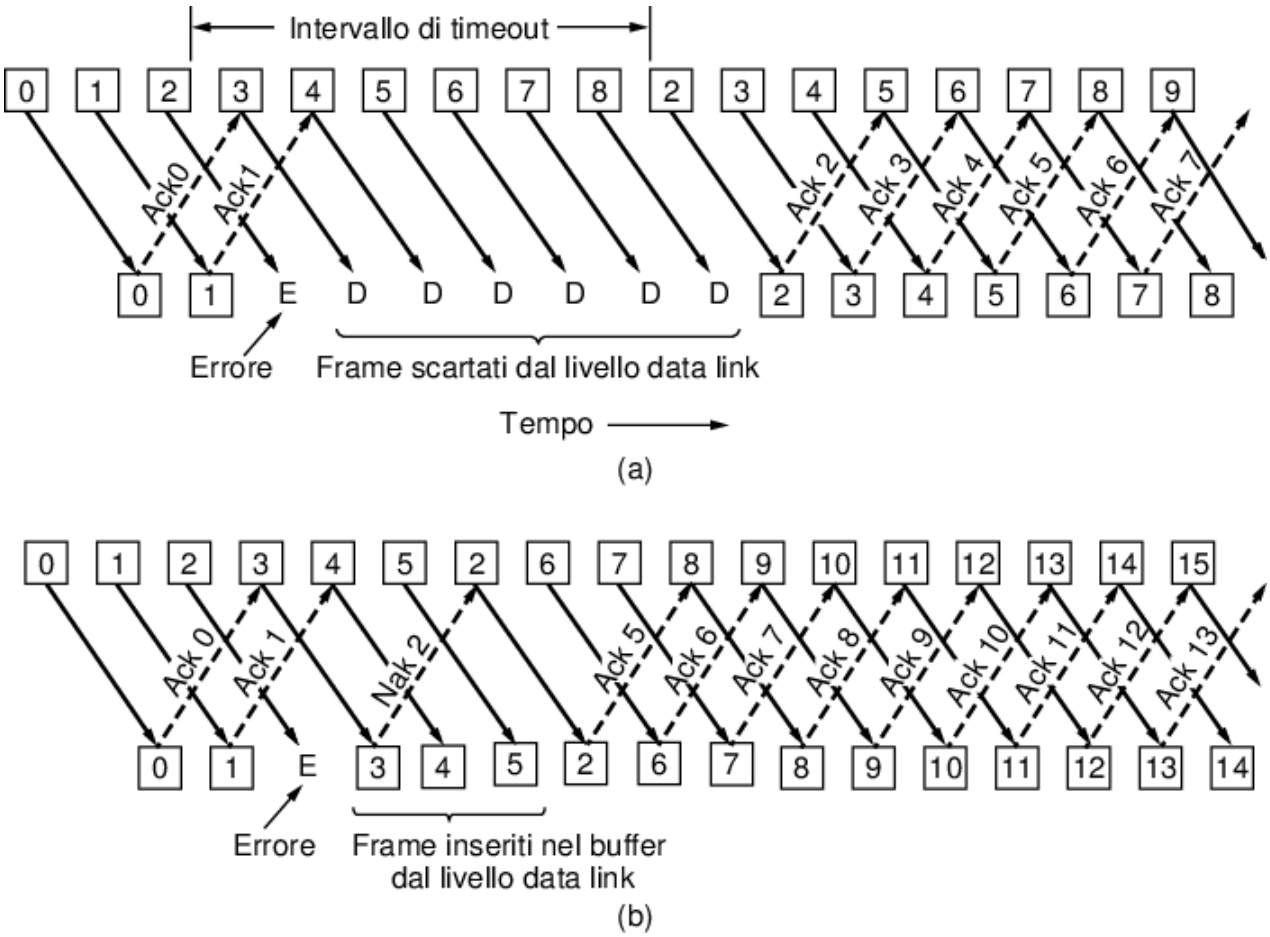


Figura 3.18 Pipelining e ripristino degli errori. Effetto di un errore quando (a) la finestra della destinazione ha dimensione 1 e (b) la finestra della destinazione è grande.

Uno di essi è chiamato **go-back-n** (*indietro a n*) e vuole che la destinazione semplicemente scarti tutti i frame successivi all'errore senza mandare l'acknowledgement per questi frame scartati.[3] La strategia corrisponde a una finestra di ricezione di dimensione 1. In altre parole, il livello data link, rilevato l'errore, si rifiuta di accettare qualunque frame eccetto il successivo che deve inviare al livello di rete. Se la finestra della sorgente si riempie prima che il timer scatti, la pipeline comincia a svuotarsi. Dopo un po' di tempo la sorgente va in timeout e ritrasmette in ordine tutti i frame che non hanno ricevuto l'acknowledgement, cominciando da quello danneggiato o perso. Questo approccio può far perdere molta banda se la frequenza degli errori è alta.
Nella Figura 3.18(b) vediamo go-back-n nel caso che la finestra della destinazione sia larga. I frame 0 e 1 sono ricevuti correttamente con acknowledgement. Il frame 2, invece, è danneggiato o perso. La sorgente, ignara del problema, continua a inviare frame fino allo scatto del timer 2. A quel punto ritorna al frame 2 e ricomincia da lì, inviando di nuovo i frame 2, 3, 4 e così via.

L'altra strategia generale per trattare gli errori in presenza di pipelining è chiamata **selective repeat** (*ripetizione selettiva*). Con tale metodo, un frame in errore viene scartato, mentre i frame corretti, ricevuti successivamente, vengono messi in un buffer. Quando la sorgente va in timeout, solo il frame più vecchio senza acknowledgement viene ritrasmesso. Se quel frame arriva correttamente, la destinazione può passare in sequenza al livello di rete tutti i frame che ha nel buffer. La ripetizione selettiva corrisponde a una finestra della destinazione più grande di 1. Questo approccio può richiedere molta memoria a livello data link quando la finestra è grande.

La ripetizione selettiva è spesso associata al fatto che la destinazione invia un acknowledgement negativo (*NAK*) quando trova un errore, per esempio quando riceve un errore di checksum o un frame fuori sequenza. I NAK stimolano la ritrasmissione prima che il timer corrispondente scatti, e quindi aumentano la performance.

Nella Figura 3.18(b) i frame 0 e 1 sono di nuovo ricevuti correttamente con acknowledgement, mentre il frame 2 è perso. Quando il frame 3 arriva a destinazione, il livello data link nota che si è perso un frame, quindi invia un NAK per il frame 2 e mette il 3 nel buffer. Quando anche i frame 4 e 5 arrivano, anche questi vengono messi nel buffer dal livello data link, ma non vengono ancora passati al livello di rete. Dopo un po' il NAK 2 raggiunge la sorgente, che manda immediatamente una nuova copia del frame 2. Quando la copia arriva a destinazione il livello data link ha tutti i frame da 2 a 5 e li può quindi passare nell'ordine corretto al livello di rete. A questo punto può anche mandare l'acknowledgement di tutti i frame fino a 5 incluso, come mostrato nella figura. Se si dovesse perdere il NAK, la sorgente andrebbe comunque in timeout per il frame 2 e autonomamente prenderebbe la decisione di ritrasmettere il frame 2 (e solamente quello), ma ciò avviene con un certo ritardo.

Questi due approcci alternativi giungono a compromessi diversi sull'uso della banda e dello spazio nel buffer del livello data link. Si può usare un approccio oppure l'altro a seconda di quale risorsa è più scarsa. La Figura 3.19 mostra un protocollo con pipelining nel quale il livello data link della destinazione accetta i frame solamente in ordine: i frame successivi a un frame errato vengono scartati.

```
/* Il protocollo 5 (go-back-n) gestisce multipli frame in uscita. La sorgente può trasmettere fino a
MAX_SEQ frame senza dover aspettare un ack. A differenza dei precedenti protocolli, non si assu-
me più che il livello di rete abbia un pacchetto disponibile in ogni momento. Invece, il livello di rete
scatena un evento network_layer_ready quando c'è un pacchetto da inviare.*/

#define MAX_SEQ 7
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Ritorna vero se a <= b < c in modo circolare; falso altrimenti */
  if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
        return(true);
  else
        return(false);
  }

static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{
/* Costruisce e invia un frame di dati */
  frame s;                                  /* variabile di servizio */

  s.info = buffer[frame_nr];                /* inserisce il pacchetto nel frame */
  s.seq = frame_nr;                         /* inserisce il numero di sequenza nel frame */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1); /* piggyback ack */
  to_physical_layer(&s);                    /* trasmette il frame */
  start_timer(frame_nr);                    /* fa partire il timer */
}

void protocol5(void)
{
  seq_nr next_frame_to_send;                /* MAX_SEQ > 1; usato per il flusso in uscita */
  seq_nr ack_expected;                      /* frame più vecchio ancora senza acknowledgement */
  seq_nr frame_expected;                    /* prossimo frame atteso nel flusso in ingresso */
  frame r;   /* variabile di servizio */
  packet buffer[MAX_SEQ + 1];               /* buffer per il flusso in uscita */
  seq_nr nbuffered;                         /* numero buffer di output attualmente in uso */
  seq_nr i;                                 /* usato come indice per l'array dei buffer */
  event_type event;

  enable_network_layer();                   /* abilita gli eventi network_layer_ready */
  ack_expected = 0;                         /* il successivo ack atteso in ingresso */
  next_frame_to_send = 0;                   /* il successivo frame in uscita */
  frame_expected = 0;                       /* numero del frame atteso in ingresso */
  nbuffered = 0;                            /* inizialmente non ci sono pacchetti nei buffer */
```

Figura 3.19 Un protocollo a finestra scorrevole che usa go-back-n (*cont.*).

In questo protocollo, per la prima volta, abbiamo eliminato l'ipotesi che il livello di rete abbia sempre una riserva infinita di pacchetti da mandare. Quando il livello di rete vuole mandare un pacchetto, può farlo scatenando un evento *network_layer_ready*. Tuttavia, per rinforzare il limite di

controllo del flusso sulla finestra della sorgente, il livello data link deve essere in grado di fermare il livello di rete, se necessario. Le procedure *enable_network_layer* e *disable_network_layer* servono a tale scopo.

Il numero massimo di frame che possono essere in attesa di acknowledgement in un dato istante non è uguale alla dimensione dello spazio dei numeri di sequenza.

Per go-back-n a ogni istante possono essere in attesa *MAX_SEQ* frame, anche se ci sono *MAX_SEQ* + 1 numeri di sequenza distinti: 0, 1, 2, ... , *MAX_SEQ*. Per vedere il perché di questa restrizione consideriamo il seguente scenario con *MAX_SEQ* = 7:

1. la sorgente invia i frame da 0 a 7
2. un acknowledgement per il frame 7 ritorna alla sorgente tramite piggybacking
3. la sorgente manda altri otto frame con numeri di sequenza da 0 a 7
4. un altro acknowledgement per il frame 7 ritorna tramite piggybacking.

A questo punto la domanda è: gli otto frame del secondo gruppo sono arrivati tutti con successo, oppure sono stati persi tutti (contando quelli scartati che seguono un errore come fossero persi)? In entrambi i casi la destinazione manderebbe il frame 7 come acknowledgement senza che la sorgente abbia modo di discriminare fra le due situazioni. Per questo motivo il massimo numero di frame che possono essere in attesa di acknowledgement deve essere limitato a *MAX_SEQ*.

Anche se il protocollo 5 non mette nel buffer i frame che arrivano dopo un errore, questo non elimina del tutto il problema dell'uso dei buffer. Dato che a un certo punto una sorgente si può trovare nella situazione di dover ritrasmettere tutti i frame che non hanno ricevuto acknowledgement, deve per forza tenere traccia di tutti i frame inviati finché non è sicura che siano stati accettati dalla destinazione. Quando arriva l'acknowledgement per il frame *n,* anche i frame $n - 1$, $n - 2$, e così via hanno automaticamente l'acknowledgement, chiamato in questo caso **cumulative acknowledgement**. Questa proprietà è particolarmente importante nel caso in cui alcuni dei frame contenenti i precedenti acknowledgement siano andati persi o si siano corrotti. Quando arriva un qualunque acknowledgement, il livello data link controlla se può rilasciare un buffer. In quest'ultimo caso viene liberato dello spazio nella finestra e quindi il livello di rete, se era bloccato, adesso può lanciare dei nuovi eventi di tipo *network_layer_ready*.

Per questo protocollo assumiamo che ci sia sempre del traffico all'indietro sul quale effettuare il piggybacking degli acknowledgement. Il protocollo 4 non ha bisogno di questa ipotesi, poiché restituisce un frame per ognuno di quelli che riceve, anche nel caso in cui abbia già mandato proprio quel frame. Nel protocollo successivo risolveremo in modo elegante il problema del traffico che va in una sola direzione.

Il protocollo 5 ha logicamente bisogno di più timer, in quanto deve gestire il transito di più frame alla volta e ognuno di questi necessita di un timer. Ogni frame può andare in timeout in modo indipendente dagli altri. Tutti questi timer possono essere facilmente simulati via software usando un singolo orologio hardware che lancia degli interrupt periodici. I timeout registrati formano una lista concatenata (*linked list*), dove ogni nodo della lista contiene il numero di intervalli (*tick*) dell'orologio che mancano per arrivare al timeout, il frame a cui corrisponde il timer e un puntatore al nodo successivo.

Per illustrare una possibile implementazione dei timer, consideriamo l'esempio della Figura 3.20(a). Assumiamo che l'orologio scandisca intervalli di 1 ms. Inizialmente l'ora segnata dall'orologio è 10:00:00.000 e ci sono tre timeout in coda: alle 10:00:00.005, alle 10:00:00.013 e alle 10:00:00.019. Ogni volta che l'orologio scandisce un intervallo, l'ora segnata viene incrementata, mentre il contatore dei tick in testa alla lista viene decrementato. Quando il contatore dei tick diventa zero, viene generato un timeout e il nodo è rimosso dalla lista, come si vede nella Figura 3.20(b). Questo modo di implementare il timer non richiede molto lavoro per ogni intervallo dell'orologio, anche se è necessario scandire la lista tutte le volte che vengono chiamate *start_timer* oppure *stop_timer.* Nel protocollo 5 entrambe queste routine sono state implementate con un parametro che indica il frame al quale il timer fa riferimento.

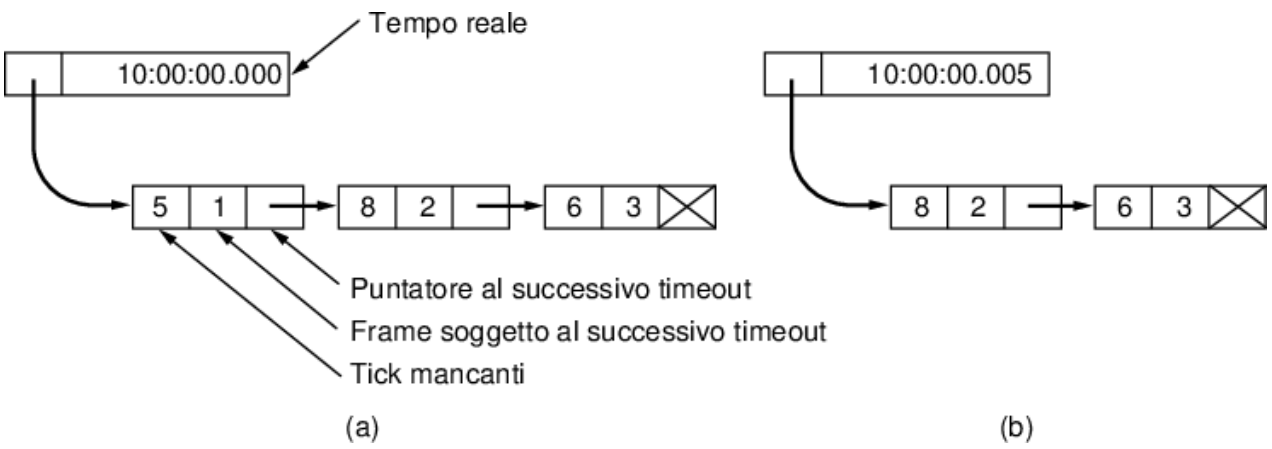


Figura 3.20 Simulazione di timer multipli via software. (a) I timeout in coda. (b) La situazione dopo che il primo timeout è scattato.

Un protocollo che usa la ripetizione selettiva

Il protocollo go-back-n funziona bene se gli errori sono rari. Se invece la qualità della linea è scarsa, perde una gran quantità di banda con la ritrasmissione dei frame. Una strategia alternativa per la gestione degli errori consiste nel permettere alla destinazione di accettare, dentro ad appositi buffer, anche i frame che seguono un frame perso o danneggiato.

In questo protocollo, sorgente e destinazione mantengono una finestra di numeri di sequenza accettabili. La finestra della sorgente comincia da 0 e cresce fino a un valore massimo predefinito, MAX_SEQ. La destinazione, invece, ha un buffer di dimensione fissa pari a MAX_SEQ. La destinazione ha un buffer riservato per ogni numero di sequenza all'interno della sua finestra. Associato a ogni buffer c'è un bit (detto *arrived*) che indica quando il buffer è pieno o vuoto. Quando un frame arriva, il suo numero di sequenza è controllato dalla funzione *between* per vedere se si trova all'interno della finestra. In caso affermativo, se non è già stato ricevuto, il frame viene accettato e memorizzato. Quest'azione è eseguita indipendentemente dal fatto che il frame contenga il successivo pacchetto per il livello di rete. Ovviamente, il frame non viene passato

subito al livello di rete, ma è trattenuto dal livello data link fino al momento in cui tutti i frame con numeri di sequenza inferiori siano stati ricevuti e passati in ordine al livello di rete. Un protocollo che usa questo algoritmo è mostrato nella Figura 3.21.

```
/* Il protocollo 6 (ripetizione selettiva) accetta dei frame fuori ordine, ma passa i pacchetti al livello
di rete in ordine. A ogni frame in uscita è associato un timer. Quando il timer scatta solamente
il frame coinvolto viene ritrasmesso, non tutti i frame in transito come nel protocollo 5. */

#define MAX_SEQ 7                                   /* deve essere 2^n - 1 */
#define NR_BUFS ((MAX3SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;
#include "protocol.h"
boolean no_nak = true;                              /* nessun nak è stato inviato finora */
seq_nr oldest_frame = MAX_SEQ + 1;                  /* il valore iniziale è solo per il simulatore */

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Come per il protocollo 5, ma più corto e oscuro */
  return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}

static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{
/* Costruisce e invia un frame di dati, ack, o nak */
  frame s;                                          /* variabile di servizio */

  s.kind = fk;/* kind == data, ack, or nak */
  if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
  s.seq = frame_nr;                                 /* ha senso solo per i frame di dati */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
  if (fk == nak) no_nak = false;                    /* un solo nak per frame */
  to_physical_layer(&s);                            /* trasmette il frame */
  if (fk == data) start_timer(frame_nr % NR_BUFS);
  stop_ack_timer();                                 /* non c'è bisogno di un frame di ack separato */
}

void protocol6(void)
{
  seq_nr ack_expected;                              /* limite inferiore della finestra della sorgente */
  seq_nr next_frame_to_send;                        /* limite superiore della finestra della sorgente + 1 */
  seq_nr frame_expected;                            /* limite inferiore della finestra della destinazione */
  seq_nr too_far;                                   /* limite sup. della finestra della destinazione + 1*/
  int i;                                            /* indice del buffer */
  frame r; /* variabile di servizio */
  packet out_buf[NR_BUFS];                          /* buffer per il flusso in uscita */
  packet in_buf[NR_BUFS];                           /* buffer per il flusso in entrata */
  boolean arrived[NR_BUFS];                         /* bit map di ingresso */
  seq_nr nbuffered;                                 /* quanti buffer di output sono usati correntemente */
  event_type event;

  enable_network_layer();                           /* inizializza */
  ack_expected = 0;                                 /* il successivo ack atteso nel flusso di ingresso */
  next_frame_to_send = 0;                           /* numero del successivo frame in uscita */
  frame_expected = 0;
  too_far = NR_BUFS;
  nbuffered = 0;                                    /* inizialmente non ci sono pacchetti nei buffer */
  for (i = 0; i < NR_BUFS; i++) arrived[i] = false;
while (true) {
```

Figura 3.21 Un protocollo a finestra scorrevole che usa la ripetizione selettiva (*cont.*).

La ricezione non sequenziale solleva problemi che non sono presenti in protocolli dove i frame sono accettati solamente in ordine. Per illustrare la questione ci serviamo di un esempio. Supponiamo di avere un numero di sequenza a tre bit, così che la sorgente può trasmettere fino a sette frame prima di dover aspettare un acknowledgement. Gli stati iniziali delle finestre della sorgente e della destinazione sono come nella Figura 3.22(a). A questo punto la sorgente trasmette i frame da 0 a 6. La finestra della destinazione permette di ricevere tutti i frame con numero di sequenza fra 0 e 6 incluso. Tutti e sette i frame arrivano in modo corretto, così che la destinazione invia un acknowledgement e porta avanti la sua finestra per permettere la ricezione di 7, 0, 1, 2, 3, 4 o 5, come mostrato nella Figura 3.22(b). Tutti i sette buffer sono segnati come vuoti.

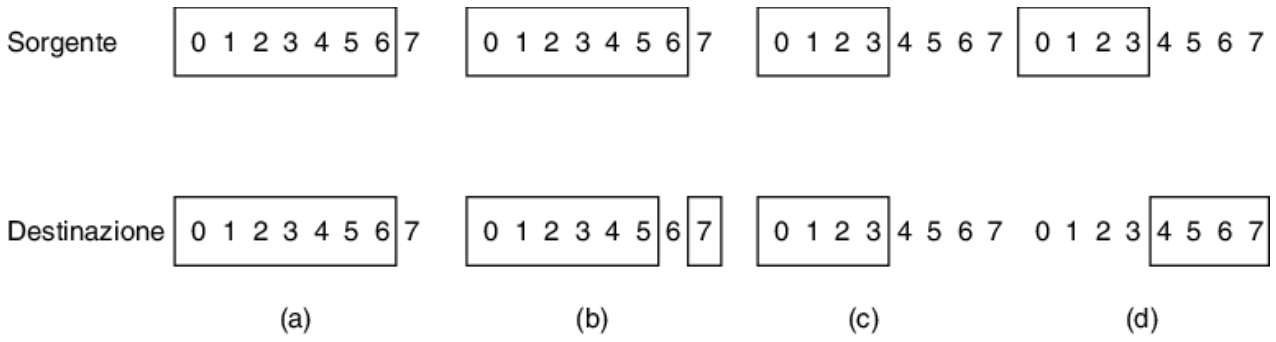


Figura 3.22 (a) Situazione iniziale con una finestra di dimensione 7. (b) Dopo che 7 frame sono stati inviati e ricevuti, ma ancora in attesa di acknowledgement. (c) Situazione iniziale con una finestra di dimensione 4. (d) Dopo che 4 frame sono stati inviati e ricevuti, ma ancora in attesa di acknowledgement.

A questo punto accade un disastro, come fosse un fulmine che colpisce un palo della linea telefonica e spazza via tutti gli acknowledgement. Il protocollo dovrebbe funzionare correttamente nonostante il disastro accaduto. La sorgente dopo un po' di tempo va in timeout e ritrasmette il frame 0. Quando esso arriva alla destinazione, viene fatto un controllo per vedere se ricade all'interno della finestra della destinazione. Sfortunatamente, nella Figura 3.22(b) il frame 0 è all'interno della nuova finestra, quindi verrà accettato come frame nuovo. La destinazione manda (usando il piggybacking) un acknowledgement per il frame 6, visto che i frame da 0 a 6 sono stati ricevuti.

La sorgente è contenta di sapere che tutti i frame trasmessi sono arrivati correttamente, quindi fa scorrere in avanti la sua finestra e manda immediatamente i frame 7, 0, 1, 2, 3, 4, 5. Il frame 7 sarà accettato dalla destinazione e il pacchetto corrispondente verrà passato direttamente al livello di rete. Subito dopo, il livello data link della destinazione controlla per vedere se ha già un frame 0 valido, scopre che ce l'ha e passa il pacchetto al livello di rete. Pertanto il livello di rete riceve un pacchetto sbagliato, quindi il protocollo ha fallito.

L'essenza del problema è che, dopo che la destinazione ha portato avanti la propria finestra, il nuovo intervallo di numeri di sequenza validi si è sovrapposto a quello vecchio. Di conseguenza la serie di frame trasmessa successivamente può essere o un duplicato della precedente (se tutti gli acknowledgement sono stati persi) oppure una nuova serie (se tutti gli acknowledgement sono stati ricevuti). La destinazione non è in grado di distinguere i due casi.

Il metodo per risolvere questo problema consiste nell'essere sicuri che, quando la destinazione ha portato avanti la sua finestra, non ci siano sovrapposizioni con la finestra originale. A questo scopo, la massima dimensione della finestra dovrebbe essere al massimo pari alla metà della sequenza di numeri, com'è mostrato nella Figura 3.22(c) e nella Figura 3.22(d). Se, per esempio, usiamo 3 bit per i numeri di sequenza, l'intervallo sarà da 0 a 7. Solo quattro frame senza acknowledgement dovrebbero essere in transito a ogni istante. In quel modo, se la destinazione ha appena accettato i frame da 0 a 3 e portato avanti la finestra in modo da ricevere i frame da 4 a 7, può sicuramente distinguere se i frame successivi sono ritrasmissioni (numeri da 0 a 3) o nuove sequenze (numeri da 4 a 7). In generale, la dimensione della finestra per il protocollo 6 sarà (MAX_SEQ + l)/2.

Una domanda interessante è: quanti buffer deve avere la destinazione? In nessuna circostanza la destinazione accetterà dei frame con numero di sequenza inferiore al valore minimo presente nella finestra o, analogamente, valori maggiori del valore massimo nella finestra. Di conseguenza, il numero di buffer necessari è uguale alla dimensione della finestra, non all'intervallo dei numeri di sequenza. Nell'esempio precedente, con un numero di sequenza di 3 bit sono necessari 4 buffer identificati da 0 a 3. Quando arriva il frame *i*, viene messo nel buffer *i* mod 4. Notiamo che, anche se *i* e (*i* + 4) mod 4 "competono" per lo stesso buffer, non verranno mai a trovarsi nella finestra allo stesso tempo, perché questo significherebbe avere una finestra di dimensione 5 o maggiore.

Per lo stesso motivo, il numero di timer necessari è uguale al numero dei buffer, non alla dimensione della sequenza. A tutti gli effetti un timer è associato a ogni buffer. Quando il timer scatta, i contenuti del buffer sono ritrasmessi.

Nel protocollo 6 si elimina l'assunzione implicita che il canale sia pesantemente utilizzato. Abbiamo fatto tale assunzione nel protocollo 5, dove si usavano i frame inviati nella direzione opposta per mandare acknowledgement in piggyback. Se il traffico di ritorno è ridotto, l'acknowledgement può essere trattenuto per un tempo lungo, causando dei problemi. Nel caso estremo in cui ci sia molto traffico in una data direzione e nessun traffico in quella opposta, il protocollo si bloccherà quando la finestra della sorgente raggiungerà il massimo.

Per eliminare questa assunzione, dopo l'arrivo di una sequenza di frame di dati, viene fatto partire un timer ausiliario usando *start_ack_timer*. Un frame di acknowledgement separato è inviato se non si è verificato alcun traffico di ritorno prima del timeout. Un interrupt causato dal timer ausiliario è chiamato un evento di *ack_timeout*. Con questa modifica è possibile gestire il flusso di traffico unidirezionale, in quanto l'assenza di frame di dati all'indietro su cui effettuare il piggyback degli acknowledgement adesso non è più un ostacolo. Esiste un solo timer ausiliario e quando *start_ack_timer* è invocato mentre il timer sta girando, non ha alcun effetto. Il timer non viene né fatto ripartire né esteso, perché il suo scopo è garantire un minimo tasso di acknowledgement.

È essenziale che l'intervallo di tempo associato al timer ausiliario sia decisamente più corto di quello usato per il timer dei frame di dati. Questa condizione è necessaria per assicurarsi che i frame ricevuti correttamente riescano anche a trasmettere l'acknowledgement in tempo e non

siano ritrasmessi per via di un timeout del timer ausiliario.

Il protocollo 6 usa una strategia più efficiente del protocollo 5 per gestire gli errori. Quando la destinazione sospetta che sia accaduto un errore, rimanda alla sorgente un frame di acknowledgement negativo (NAK), necessario per la ritrasmissione del frame specificato nel NAK. Ci sono due casi in cui la destinazione dovrebbe sospettare un errore: l'arrivo di un frame danneggiato oppure l'arrivo di un frame diverso da quello atteso (è possibile che un frame sia stato perso). Per evitare domande di ritrasmissione multipla per lo stesso frame, la destinazione dovrebbe tenere traccia dei frame di cui ha chiesto la ritrasmissione con un NAK. La variabile *no_nak* nel protocollo 6 è impostata a vero se finora nessun NAK è stato inviato per il frame *frame_expected*. Se il NAK viene perso o corrotto non accade nulla di grave, in quanto la sorgente andrà comunque in timeout e ritrasmetterà il frame. Se arriva un frame sbagliato dopo che un NAK è stato inviato e poi perso, allora *no_nak* verrà impostato a vero e il timer ausiliario verrà fatto partire. Al raggiungimento del timeout sarà inviato un ACK per sincronizzare nuovamente lo stato corrente della sorgente e della destinazione.

In alcune situazioni il tempo necessario perché un frame si propaghi a destinazione, sia elaborato e l'acknowledgement ritorni è (praticamente) constante. In tali casi la sorgente può impostare il suo timer in modo che abbia un timeout leggermente più lungo dell'intervallo normalmente misurato fra l'invio di un frame e la ricezione del suo acknowledgement. In questo caso usare un NAK è inutile.

Tuttavia, in alcune situazioni il tempo è molto variabile. Se, per esempio, il traffico di ritorno è sporadico, il tempo necessario all'acknowledgement sarà più corto quando c'è traffico di ritorno e più lungo quando non ce n'è. La sorgente deve scegliere fra un timeout corto (quindi rischiare delle ritrasmissioni inutili) e uno lungo (che comporta un lungo periodo di tempo di attesa dopo un errore). Entrambe le scelte sprecano banda. In generale, quando la deviazione standard dell'intervallo di acknowledgement è piccola rispetto all'intervallo stesso, il timer è impostato in modo "flessibile" per prudenza. I NAK possono velocizzare considerevolmente la ritrasmissione dei frame danneggiati o persi.

Legato strettamente al discorso dei timeout e dei NAK vi è quello di identificare i frame sui quali è scattato un timeout. Nel protocollo 5, la risposta è *ack_expected*, cioè è sempre il frame più vecchio. Nel protocollo 6 non c'è un modo semplice per determinare quale frame ha ricevuto un timeout. Supponiamo che i frame da 0 a 4 siano stati trasmessi, ovvero che la lista dei frame in transito sia, in ordine cronologico: 0, 1, 2, 3, 4. Immaginiamo la situazione in cui: 0 vada in timeout, 5 (un nuovo frame) venga trasmesso, 1 vada in timeout, 2 vada in timeout e 6 (un altro nuovo frame) venga trasmesso. A questo punto la lista dei frame in transito sarà: 3, 4, 0, 5, 1, 2, 6 (in ordine cronologico). Se tutto il traffico in ingresso (cioè i frame che portano acknowledgement) viene perso per un po' di tempo, i sette frame in transito andranno in timeout in ordine cronologico.

Per non complicare troppo l'esempio non abbiamo mostrato l'amministrazione del timer; abbiamo invece supposto che la variabile *oldest_frame* venga impostata al momento del timeout, in modo da indicare quale frame ha ricevuto il timeout stesso.

3.5 Esempi di protocolli data link

Le reti LAN sono ampiamente utilizzate per le connesioni all'interno di un singolo edificio, ma la maggior parte delle infrastrutture di rete su grandi aree è costituita da collegamenti punto a punto. Vedremo le reti LAN nel Capitolo 4, mentre in questo paragrafo esamineremo i protocolli data link che si trovano sui collegamenti punto a punto di Internet in due situazioni piuttosto comuni. La prima si ha quando i pacchetti vengono inviati facendo uso di SONET su un collegamento in fibra ottica in reti geografiche. Questi collegamenti sono, per esempio, molto usati per connettere router lontani tra loro nella rete di un ISP.

La seconda si ha con le linee ADSL che agiscono sull'ultimo miglio (*local loop*) della rete telefonica ai bordi di Internet, connettendo milioni di privati e aziende.

Internet necessita per questi scopi di collegamenti punto a punto, così come di modem dial-up, linee in affitto, modem via cavo e molti altri dispositivi. Il protocollo standard usato su questi collegamenti è chiamato **PPP** (*point-to-point protocol,* protocollo punto a punto) ed è stato definito nell'RFC 1661, revisionato nell'RFC 1662 e in altri ancora (Simpson, 1994a, 1994b). Sia le linee SONET sia quelle ADSL implementano PPP, ma in modo diverso.

Pacchetti su SONET

SONET, di cui abbiamo discusso nel Sottoparagrafo 2.6.4, è il protocollo di livello fisico più usato nei collegamenti in fibra ottica delle WAN, che formano la dorsale delle reti di comunicazione, incluso il sistema telefonico. Fornisce un flusso (*stream*) di bit (o *bitstream*) con un tasso di produzione fisso quale 2,4 Gbps per una linea OC-48. Questo bitstream è suddiviso in payload di dimensione fissata che vengono inviati ogni 125 μs, indipendentemente dal fatto che vi siano o meno dati da inviare.

Per trasmettere pacchetti su queste linee bisogna disporre di meccanismi di framing che distinguano i pacchetti occasionali dal flusso di bit continuo in cui vengono trasportati. Tale meccanismo è fornito dal protocollo PPP che opera sui router IP come mostrato nella Figura 3.23.

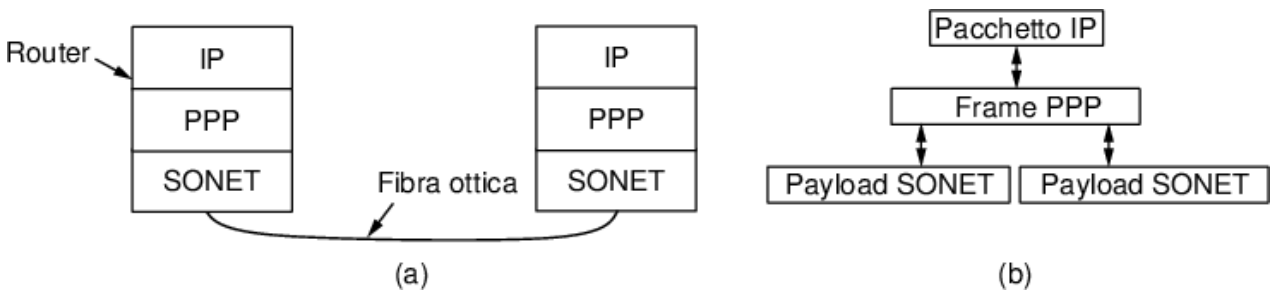


Figura 3.23 Pacchetto su SONET. (a) Lo stack di protocolli. (b) Le relazioni tra i frame.

Il protocollo PPP è nato come miglioramento di un protocollo precedente, più semplice, chiamato **SLIP** *(serial line internet protocol)* e viene utilizzato per gestire la configurazione della rilevazione d'errore di una linea, supportare molteplici protocolli, permettere l'autenticazione e molto altro. Provvisto di numerose opzioni, il protocollo PPP ha le seguenti tre caratteristiche principali.

1. Un metodo di framing che permette di delimitare in modo non ambiguo la fine di un frame e l'inizio del successivo. Il formato del frame permette anche di gestire la rilevazione degli errori.
2. Un protocollo per gestire la connessione, il test della linea, negoziare le opzioni di collegamento e gestire la disconnessione in modo pulito quando la linea non serve più. Questo protocollo è chiamato **LCP** *(link control protocol*, protocollo di controllo del collegamento).
3. Una modalità per negoziare le opzioni relative al livello di rete, in modo indipendente dall'implementazione di tale livello che verrà usata per la comunicazione. Il metodo scelto avrà un diverso **NCP** *(network control protocol*, protocollo di controllo della rete) per ogni livello di rete supportato.

Il formato del frame PPP è stato scelto in modo da assomigliare a quello di **HDLC** (*high level data link control*), un protocollo largamente utilizzato che fa parte di una famiglia di protocolli più vecchi, visto che non c'era ragione di reinventare la ruota.

La principale differenza fra PPP e HDLC è che PPP è orientato ai byte, mentre HDLC è orientato ai bit. In particolare PPP usa il byte stuffing, così che tutti i frame sono costituiti da un numero intero di byte. HDLC usa il bit stuffing e permette frame di ad esempio 30,25 byte.

Nella pratica, però, esiste anche una seconda differenza importante. HDLC fornisce trasmissioni affidabili con finestra scorrevole, acknowledgement e timeout come visto prima. PPP è in grado di fornire trasmissioni affidabili anche nel caso di canali soggetti a rumore come nelle reti wireless: i dettagli sono definiti nell'RFC 1663. Tuttavia, questo viene fatto raramente in Internet dove invece si preferisce usare una modalità senza numerazione (*unnumbered mode*) per fornire servizi senza connessione e senza acknowledgement.

Il formato del frame PPP è mostrato nella Figura 3.24. Tutti i frame PPP cominciano con il flag byte standard HDLC 0x7E (01111110) a cui, quando occorre, può essere applicato il byte stuffing usando il byte di escape 0x7D all'interno del campo *Payload*. Il byte successivo è il byte di escape a cui è applicato lo XOR con 0x20 che inverte il quinto bit. Per esempio 0x7D 0x5E è la sequenza di escape per il flag byte 0x7E. Quindi l'inizio e la fine dei frame vengono individuati ricercando il byte 0x7E che non può comparire in altre posizioni. La regola di destuffing quando si riceve un frame è di cercare 0x7D, rimuoverlo ed effettuare lo XOR del byte successivo con 0x20. Inoltre è necessario solo un flag byte tra i frame. Per riempire la linea quando non vi sono frame da spedire possono essere usati più flag byte.

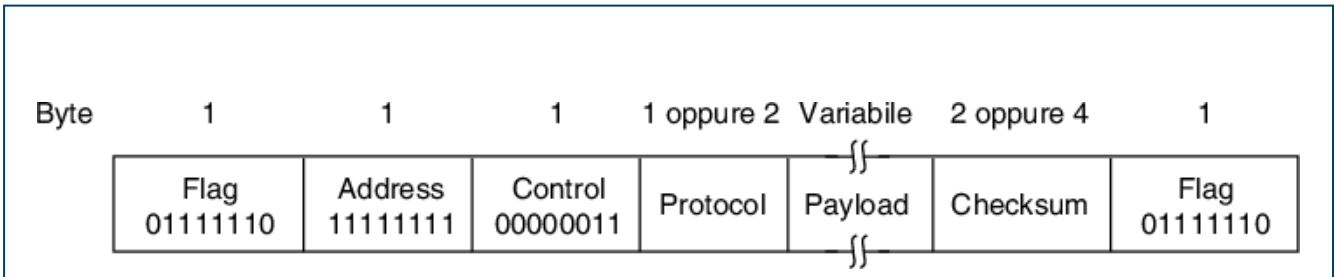


Figura 3.24 Il formato completo del frame PPP per il funzionamento senza numero di sequenza.

Al flag byte di inizio frame segue poi il campo *Address* che viene sempre impostato al valore binario 11111111 per indicare che tutte le stazioni devono accettare il frame. In questo modo si evita il problema di dover assegnare indirizzi data link.

Il campo *Address* è seguito dal campo *Control*, che ha come valore di default 00000011. Questo valore indica un frame senza numero.

Visto che *Address* e *Control* sono sempre costanti nella configurazione di default, LCP contiene un meccanismo per far sì che la sorgente e la destinazione possano negoziare l'opzione di omettere completamente questi campi e salvare quindi due byte per frame.

Il quarto campo PPP è *Protocol* e serve per comunicare quale tipo di pacchetto è contenuto nel campo *Payload*. Alcuni codici che cominciano con un bit a 0 sono definiti per IP versione 4, IP versione 6 e altri protocolli a livello di rete come IPX e AppleTalk. I codici che cominciano invece con un bit a 1 sono usati per protocolli di configurazione quali LCP e un diverso NCP per ogni livello di rete supportato. La dimensione di default del campo Protocol è 2 byte, ma può essere negoziata fino a 1 byte tramite LCP. I progettisti sono forse stati un po' troppo cauti ipotizzando che un giorno ci sarebbero stati più di 256 protocolli da poter utilizzare.

Il campo *Payload* ha lunghezza variabile fino a un valore massimo negoziabile usando LCP durante l'inizializzazione della connessione. In caso tale negoziazione non avvenga, il valore di default è di 1500 byte. Eventualmente è possibile aggiustare la dimensione del payload aggiungendo dei byte in coda (padding).

Il campo successivo è *Checksum*, normalmente di 2 byte, ma può essere anche negoziato un checksum di 4 byte che di fatto è lo stesso CRC a 32 bit il cui generatore polinomiale è stato dato alla fine del Sottoparagrafo 3.2.2. Il checksum di 2 byte è anch'esso un CRC standard industriale.

PPP è una strategia di framing in grado di trasportare i pacchetti di molteplici protocolli su molti tipi diversi di livelli fisici. Per usare PPP su una SONET ci si riferisce all'RFC 2615 (Malis e Simpson, 1999). Viene utilizzato un checksum di 4 byte, in quanto si tratta dello strumento principale con cui effettuare la rilevazione di errori di trasmissione sui livelli fisico, data link e di rete. Viene raccomandato che i campi *Address*, *Control* e *Protocol* non siano compressi, poiché le linee SONET hanno già una velocità sostenuta.

C'è anche una caratteristica insolita. Prima di venire inserito nel payload SONET il payload PPP viene rimescolato (*scrambled*) nel modo descritto nel Sottoparagrafo 2.5.1. Lo *scrambling* effettua lo XOR del payload con una lunga sequenza pseudocasuale prima della trasmissione. Il problema è che il bitstream SONET ha bisogno di frequenti transizioni di bit per la sincronizzazione. Queste transizioni avvengono naturalmente con la variazione dei segnali vocali, ma nelle comunicazioni di dati sono gli utenti a scegliere che cosa inviare e potrebbe essere una lunga sequenza di zeri. Con lo scrambling la possibilità che un utente crei problemi inviando una lunga sequenza di zeri è molto remota.

Prima che i frame PPP possano essere trasportati su una SONET, il collegamento PPP deve essere stabilito e configurato. Nella Figura 3.25 vengono mostrate le fasi di attivazione, utilizzo e chiusura del collegamento.

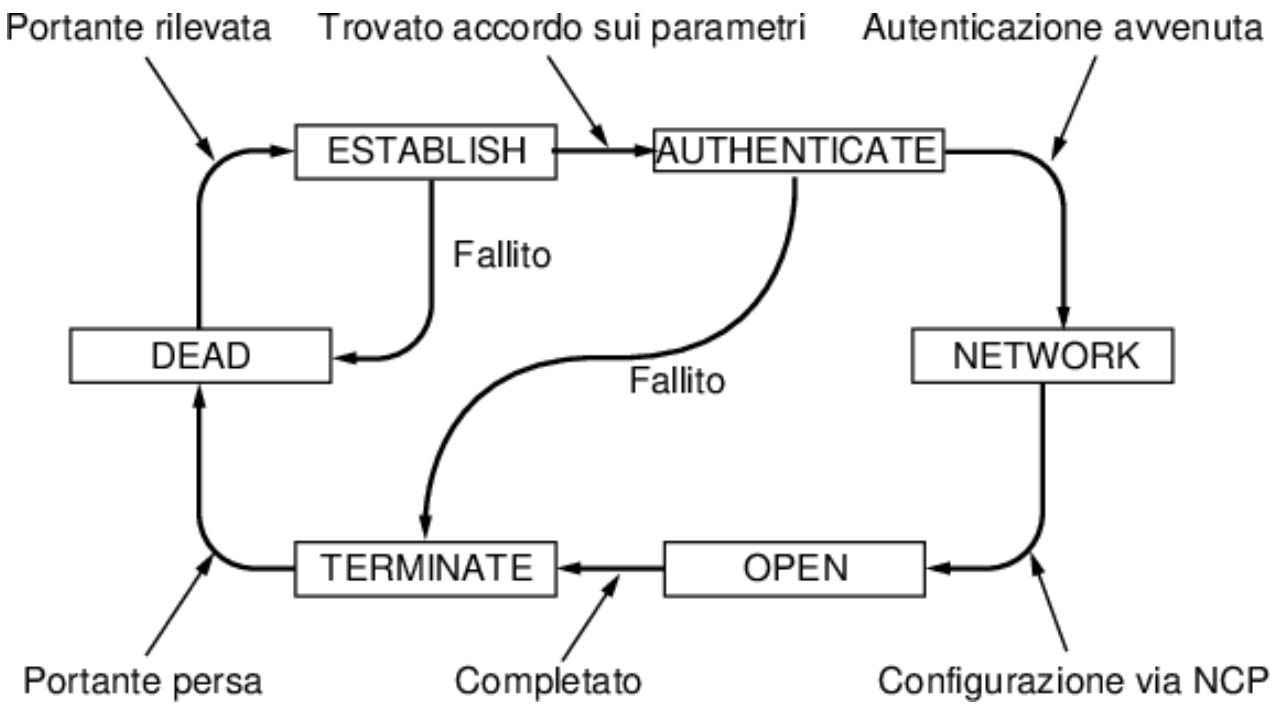


Figura 3.25 Diagramma degli stati per la gestione di una connessione PPP.

Il protocollo comincia con la linea nello stato *DEAD*, che significa che a livello fisico non esiste connessione. Dopo aver stabilito una connessione fisica, la linea si porta nello stato *ESTABLISH.* A questo punto i peer PPP scambiano una serie di pacchetti LCP trasportati nel campo *Payload* di un frame PPP, per selezionare le opzioni PPP per il collegamento. Il peer che inizializza la linea propone le opzioni che possono essere accettate o rifiutate, anche in parte, dal peer all'altro capo che può fare proposte alternative.

Se la negoziazione LCP ha successo, il collegamento entra nello stato *AUTHENTICATE*. Ora sorgente e destinazione possono controllare la loro reciproca identità, se lo desiderano. Se l'autenticazione ha successo, si entra nella fase *NETWORK* e viene inviata una serie di pacchetti NCP per configurare il livello di rete. È difficile generalizzare la trattazione sui protocolli NCP, poiché ognuno di essi è specifico di un protocollo di livello rete e permette richieste di configurazione a esso specifiche. Per esempio, per IP la scelta più importante è l'asssegnamento dell'indirizzo a entrambi i capi della linea.

Se si raggiunge lo stato *OPEN*, allora può cominciare la trasmissione di pacchetti IP in frame PPP attraverso la linea SONET. Quando il trasporto è finito, la linea si sposta nello stato di *TERMINATE* e da lì torna a *DEAD* quando la connessione a livello fisico viene rilasciata.

ADSL – asymmetric digital subscriber loop

L'ADSL connette milioni di abitazioni a Internet con velocità di megabit/s sfruttando lo stesso doppino telefonico usato per il vecchio servizio telefonico. Nel Sottoparagrafo 2.5.3 abbiamo visto come un dispositivo che prende il nome di modem DSL viene aggiunto lato abitazione; questo manda dei bit sfruttando l'**ultimo miglio** a un dispositivo chiamato **DSLAM** (DSL access multiplexer) posto nei locali di una compagnia telefonica. Ora vedremo in dettaglio come i pacchetti vengono trasportati su una linea ADSL.

La visione d'insieme per protocolli e dispositivi usati con ADSL è mostrata nella Figura 3.26. Svariati protocolli vengono utilizzati in varie reti, per cui abbiamo scelto di mostrare lo scenario più comune. All'interno di un'abitazione un computer, ad esempio un PC, manda pacchetti IP al modem DSL usando un protocollo a livello data link, come Ethernet. Il modem DSL, a sua volta, manda i pacchetti IP al DSLAM tramite il doppino telefonico utilizzando il protocollo che stiamo per studiare. All'interno del DSLAM (o di un router a lui connesso, a seconda dell'implementazione) i pacchetti IP sono estratti ed entrano nella rete dell'ISP in modo da poter raggiungere la giusta destinazione su Internet.

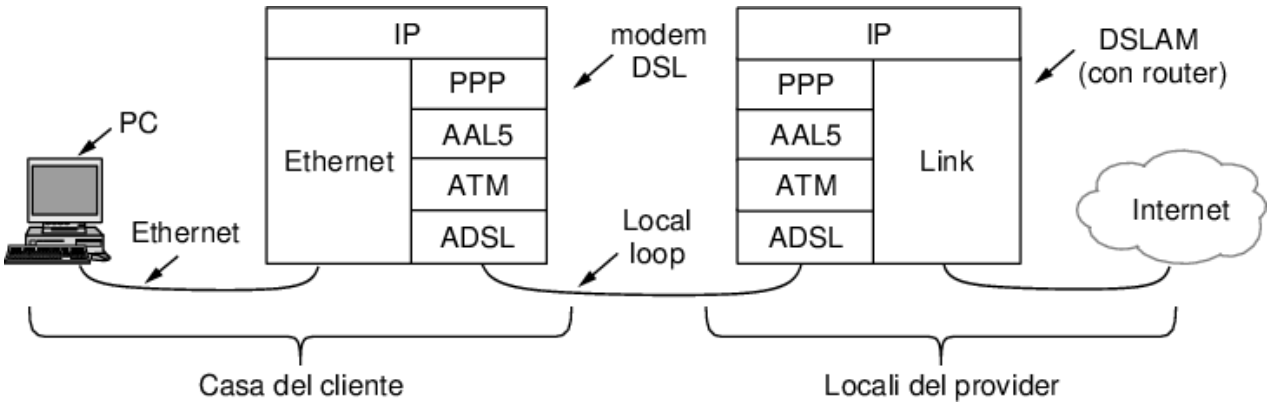


Figura 3.26 Stack dei protocolli di ADSL.

I protocolli rappresentati sopra il collegamento ADSL nella Figura 3.26 partono dal basso con il livello fisico ADSL. Essi si basano su un sistema di modulazione digitale chiamato **OFDM (multiplazione a divisione di frequenze ortogonali**, *orthogonal frequency division multiplexing*, *discrete multitone*), come già visto nel Sottoparagrafo 2.5.3. In cima allo stack, appena sotto il livello di rete IP, c'è PPP. Questo protocollo è lo stesso PPP che abbiamo appena descritto per il trasporto di pacchetti su SONET; funziona allo stesso modo per quanto riguarda la gestione del collegamento e il trasporto dei pacchetti IP.

A metà tra ADSL e PPP troviamo ATM e AAL5. Questi sono protocolli che non abbiamo mai incontrato prima. **ATM** (*asynchronous transfer mode*) è stato progettato nei primi anni '90 e poi lanciato con grande clamore. Era stata promessa una tecnologia di rete in grado di risolvere tutti i problemi di telecomunicazione del mondo facendo convergere voce, dati, TV via cavo, e poi

telegrafo, piccioni viaggiatori, lattine collegate da spago, navigatori GPS e tutto il resto in un sistema integrato che avrebbe fatto ogni cosa al posto nostro. Questo non è accaduto. La maggior parte dei problemi di ATM somigliava a quelli discussi per i protocolli OSI, e cioè: tempestività, tecnologia, implementazione e politica. Ciononostante, ATM ha avuto un successo molto maggiore di OSI; sebbene non abbia conquistato il mondo è tutt'oggi largamente utilizzata in settori come le linee di accesso a banda larga come DSL e i collegamenti WAN all'interno di reti telefoniche.

ATM è un livello data link che basa il suo funzionamento sulla trasmissione di **celle** di informazione a lunghezza fissa. Il termine "asincrono" nel nome sta a significare che non è necessario che le celle siano inviate secondo un flusso continuo (e sincrono) di bit, come in SONET. Piuttosto, le celle vengono inviate solo quando c'è effettivamente dell'informazione da trasportare. ATM è una tecnologia orientata alla connessione. Ogni cella porta nel suo header un identificatore di **circuito virtuale** (*virtual circuit*) e gli apparati di rete usano questo identificatore per indirizzare la cella lungo il percorso di una connessione già stabilita.

Le celle di ATM sono di 53 byte, di cui 48 di payload e 5 di intestazione. Usando celle così piccole ATM è in grado di suddividere in maniera efficiente e precisa la banda messa a disposizione dal mezzo trasmissivo tra diversi utenti. Questa caratteristica torna utile nel caso in cui si vogliano spedire sia voce che dati attraverso il medesimo link; la presenza di grandi pacchetti contenenti dati potrebbe infatti causare forti variazioni di ritardo a quelli più piccoli contenenti la voce campionata. La scelta poco usuale della grandezza delle celle (se confrontata con una più naturale come una potenza di due) è un indicatore del fatto che la progettazione di ATM è stata molto politicizzata. Il payload di 48 byte è stato un compromesso per risolvere una disputa tra Europa, che voleva celle da 32 byte, e Stati Uniti, che le volevano da 64. Una veloce panoramica di ATM è proposta da Siu e Jain (1995).

Per spedire dei dati attraverso una rete ATM è necessario associarli a una sequenza di celle. Di questa associazione si occupa un livello detto di adattamento di ATM in un processo che prende il nome di "segmentazione e riassemblamento". Diversi livelli di adattamento sono stati definiti per i vari servizi, spaziando dai campioni vocali periodici fino a pacchetti di dati. Quello principalmente usato per i pacchetti contenenti dati è **AAL5** (*ATM adaptation layer 5*).

Un frame AAL5 è rappresentato nella Figura 3.27. Anziché di un'intestazione, è dotato di un trailer che contiene la lunghezza e un CRC di 4 byte per il rilevamento degli errori. Naturalmente il CRC è lo stesso usato per PPP e reti IEEE 802, come Ethernet. Wang e Crowcroft (1992) hanno dimostrato che questo CRC è abbastanza robusto da individuare errori non tradizionali, come il riordino delle celle. In quanto payload, un frame AAL5 può contenere dei byte di padding per rendere la sua lunghezza totale un multiplo di 48 byte e poter essere diviso in un numero intero di celle. Nessun indirizzo è necessario per il frame, in quanto l'identificatore di circuito virtuale in ogni cella lo porterà alla giusta destinazione.

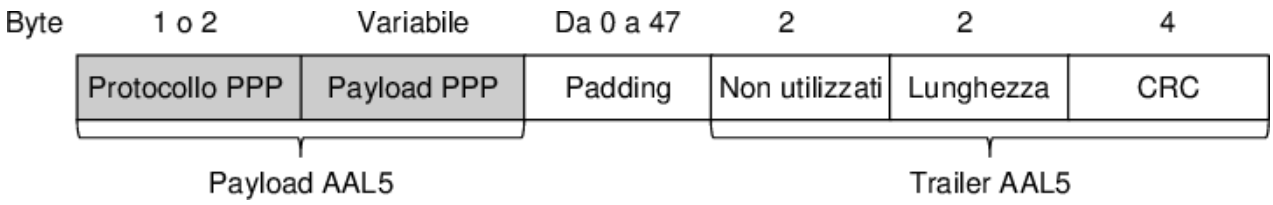


Figura 3.27 Un frame AAL5 che trasporta dati PPP.

Dopo ATM ci rimane solo da discutere come PPP faccia uso di ATM nel caso di ADSL. Ciò avviene attraverso un ennesimo standard chiamato **PPPoA** (*PPP over ATM*). Questo standard non è tanto un protocollo (per questo non compare nella Figura 3.26) quanto una specifica di come operare con i frame PPP e AAL5 e viene descritto nell'RFC 2364 (Gross et al., 1998).

Come visibile nella Figura 3.27, nel campo di payload di AAL5 vengono posti solo campi e payload di PPP. Il campo protocollo indica al DSLAM all'altro capo della connessione se il contenuto del frame sia un pacchetto IP o un pacchetto pertinente a un altro protocollo, come LCP. L'apparato remoto sa che le celle contengono informazioni PPP, dato che un circuito virtuale ATM è stato istituito a questo scopo.

All'interno di un frame AAL5 non è necessario suddividere tra loro i frame PPP; sopperiscono già a questo ATM e AAL5. Il CRC di PPP è anch'esso inutile, in quanto AAL5 include esattamente lo stesso tipo di CRC. Questo sistema di rilevamento degli errori aggiunge alla codifica del livello fisico di ADSL una codifica Reed-Solomon per la correzione degli errori e un CRC di un byte per l'individuazione di tutti gli errori non corretti dalla precedente. Tale schema ha un sistema di correzione degli errori molto più sofisticato rispetto a quando i pacchetti sono spediti tramite SONET, perché ADSL è un canale soggetto a molto più rumore.

3.6 Sommario

Il compito del livello data link è quello di convertire una sequenza di bit proveniente dal livello fisico in una sequenza di frame che possa essere usata dal livello di rete. Il livello data link può gestire queste sequenze con diversi livelli di affidabilità che variano da un servizio senza connessione e senza conferma di ricevimento a uno affidabile e orientato alla connessione.

Possono essere usate diverse politiche per la suddivisione dei frame, come il conteggio dei byte, il byte stuffing e il bit stuffing. Protocolli a livello data link possono fornire un sistema di gestione degli errori per individuare o correggere frame danneggiati e per ritrasmettere quelli persi. Per evitare che una sorgente molto veloce possa avere il sopravvento su una destinazione più lenta, il protocollo data link può anche fornire un servizio di controllo di flusso. L'approccio a finestra scorrevole è ampiamente adottato per integrare in maniera semplice il controllo di errori con il controllo di flusso. Quando la grandezza della finestra è di un solo pacchetto, il protocollo si comporta secondo la modalità stop-and-wait.

Codici per la correzione e il rilevamento degli errori aggiungono informazione ridondante ai messaggi attraverso una varietà di tecniche matematiche. Codici convoluzionali e di Reed-Salomon sono largamente implementati per la correzione di errori. Nel frattempo, anche i codici LDPC sono sempre più diffusi. I codici per rilevazione di errori più utilizzati nella pratica sono CRC e checksum. Tutti i codici menzionati sono applicabili a livello data link come pure a livello fisico e a livelli più alti.

Abbiamo esaminato una serie di protocolli in grado di fornire un livello data link affidabile usando acknowledgement e ritrasmissioni oppure ARQ, con assunzioni più realistiche. Partendo da un ambiente privo di errori, in cui il destinatario è in grado di gestire qualunque frame a lui spedito abbiamo introdotto il controllo di flusso seguito poi dal controllo di errore attraverso numeri di sequenza e l'algoritmo stop-and-wait; successivamente abbiamo usato l'algoritmo della finestra scorrevole per permettere la comunicazione bidirezionale e presentare il concetto di piggyback. Gli ultimi due protocolli gestiscono frame multipli lungo il percorso implementando una pipeline in maniera tale da prevenire che la sorgente si blocchi in presenza di un collegamento con un tempo di propagazione molto lungo. Il destinatario può decidere di scartare tutti i frame che non siano il successivo in sequenza o di memorizzare i frame fuori sequenza e spedire acknowledgement negativi per aumentare l'efficienza di uso della banda. La prima strategia prende il nome di go-back-n, mentre la seconda è detta ripetizione selettiva.

Internet fa uso principalmente di PPP come protocollo a livello data link su connessioni punto a punto. PPP fornisce un servizio non orientato alla connessione e unacknowledged usando flag byte per delimitare i frame e CRC per la correzione di errori. Viene normalmente usato per spedire pacchetti attraverso una varietà di collegamenti: ad esempio, SONET su WAN e ADSL in ambito residenziale.

Problemi

1. Un pacchetto di un livello superiore viene suddiviso in 10 frame, ognuno dei quali ha l'80% di probabilità di arrivare integro a destinazione. Se non viene eseguito alcun controllo degli errori da parte del livello data link, quante volte, in media, deve essere ritrasmesso il messaggio perché possa essere infine trasmesso interamente senza errori?

2. Un protocollo data link usa la seguente codifica a caratteri:
   A: 01000111; B: 11100011; FLAG: 01111110; ESC: 11100000
   Trovate la sequenza di bit trasmessi (in binario) per il frame di quattro caratteri A B ESC FLAG, per ognuno dei seguenti metodi di framing:
   a) conteggio dei caratteri
   b) flag byte con byte stuffing
   c) flag byte di inizio e fine con bit stuffing.

3. Il seguente frammento si trova nel mezzo di un flusso di dati trasmesso usando l'algoritmo di byte stuffing descritto nel testo: A B ESC C ESC FLAG FLAG D. Qual è l'output dopo il byte stuffing?

4. Qual è il massimo overhead in algoritmi di byte-stuffing?

5. Uno dei tuoi compagni di corso pensa che avere un flag byte alla fine di un frame e poi subito un altro all'inizio del successivo frame sia uno spreco, si potrebbe infatti risparmiare un byte usando un solo flag byte. Sei d'accordo?

6. Una stringa di bit, 0111101111101111110, deve essere trasmessa al livello data link. Qual è la stringa che viene effettivamente trasmessa dopo il bit stuffing?

7. Esistono delle circostanze in cui un protocollo open-loop (cioè una codifica di Hamming) risulti preferibile ai protocolli di tipo feedback discussi in questo capitolo?

8. Per ottenere un'affidabilità maggiore di quella data dal singolo bit di parità viene usata una codifica per la rilevazione degli errori che consiste in un bit di parità per tutti i bit dispari e uno per tutti i bit pari. Qual è la distanza di Hamming di questa codifica?

9. Una codifica di Hamming viene usata per trasmettere messaggi da 16 bit. Quanti bit di controllo sono necessari per assicurarsi che la destinazione possa rilevare e correggere gli errori di singoli bit? Qual è la sequenza di bit usata per trasmettere il messaggio 1101001100110101, usando una codifica di Hamming a parità pari?

10. Un codice di Hamming di 12 bit con valore (in esadecimale) 0xE4F arriva a destinazione. Qual è il valore originale, in esadecimale? Supponiamo che non più di 1 bit sia in errore.

11. Un modo per rilevare gli errori consiste nel trasmettere i dati come blocchi di n righe di k bit per riga e aggiungere i bit di parità a ogni riga e ogni colonna. Il bit in basso a destra controlla la parità della sua riga e della sua colonna. Questo schema riesce a rilevare tutti gli errori di singoli

bit? E di due bit? E di tre? Si dimostri che questo schema non individua tutti i possibili errori su 4 bit.

12. Si supponga di trasferire dati in blocchi della dimensione di 1000 bit. Considerando un bit di parità per blocco, qual è il massimo tasso di errore sotto il quale è meglio usare un sistema di rilevamento degli errori e ritrasmissione rispetto a un codice di Hamming?

13. Un blocco di bit con n righe e k colonne usa la parità orizzontale e verticale per rilevare gli errori. Supponiamo che gli errori di trasmissione invertano esattamente 4 bit. Si determini la proba-bilità che l'errore passi senza essere identificato.

14. Usando il codice convoluzionale della Figura 3.7, qual è la sequenza di output a fronte di un input, di 10101010 (da sinistra a destra) sapendo che lo stato interno iniziale è di soli zero?

15. Si supponga che il messaggio 1001 1100 1010 0011 sia trasmesso facendo uso del checksum di Internet (4 bit). Qual è il valore del checksum?

16. Qual è il resto della divisione di $x^1 + x^5 + 1$ per il polinomio generatore $x^3 + 1$?

17. La sequenza di bit 10011101 viene trasmessa usando il metodo standard del CRC descritto nel testo. Il polinomio generatore è $x^3 + 1$. Si determini la sequenza di bit realmente trasmessa dal livello data link. Supponiamo che il terzo bit da sinistra sia invertito durante la trasmissione. Si mostri il dettaglio di come questo errore viene rilevato dalla destinazione. Si dia un esempio di errore nella stringa trasmessa che non sarà rilevato dal destinatario.

18. Si consideri la spedizione di un messaggio di 1024 bit contenente 992 bit di dati e 32 bit di CRC. Il CRC è calcolato usando lo standard IEEE 802 polinomiale di 32-esimo grado. Per ognuna dalle affermazioni seguenti si dica se l'errore durante la trasmissione è rilevato dal destinatario:

a) c'è stato un errore su un singolo bit
b) ci sono stati due errori su due bit isolati
c) ci sono stati 18 errori su 18 bit isolati
d) ci sono stati 47 errori su 47 bit isolati
e) c'è stato un errore a burst lungo 24 bit
f) c'è stato un errore a burst lungo 35 bit.

19. Nella discussione sul protocollo ARQ nel Sottoparagrafo 3.3.3 è stato ipotizzato uno scenario in cui il destinatario accetta due copie dello stesso frame a causa della perdita di un acknowledgement. È possibile che un destinatario possa accettare più copie delle stesso frame quando nessuno dei frame (messaggi e acknowledgement) è andato perso?

20. Un canale trasmette alla velocità di 4 kbps (kilobit per secondo) e ha un ritardo di propagazione di 20 ms. Se si vuole usare un algoritmo stop-and-wait con efficienza di almeno 50%, qual è l'intervallo ammissibile per la dimensione dei frame?

21. Nel protocollo 3 è possibile che la sorgente faccia partire il timer mentre sta già girando? Se sì, quando può accadere? Se no, perché è impossibile?

22. Una dorsale T1 lunga 3.000 km viene usata per trasmettere frame di 64 byte usando il protocollo 5. Se la velocità di propagazione è 6 μs*km (micro secondi al kilometro), quanti bit si devono usare per il numero di sequenza?

23. Immaginiamo un protocollo a finestra scorrevole che utilizzi così tanti bit per i numeri di sequenza da non dover mai riciclare i numeri. Quale relazione intercorre fra i quattro bordi della finestra e la sua dimensione, che è un valore costante ed è valido tanto per la sorgente che per la destinazione?

24. Se la procedura *between* nel protocollo 5 controllasse la condizione $a \leq b \leq c$ al posto di $a \leq b < c$, ci sarebbero degli effetti sulla correttezza del protocollo o sulla sua efficienza? Spiegate la vostra risposta.

25. Nel protocollo 6, quando arriva un frame di dati si effettua il controllo per vedere se il numero di sequenza sia differente dal valore atteso e se *no_nak* sia vero. Se valgono entrambe le condizioni, viene inviato un NAK. Altrimenti viene fatto partire un timer ausiliario. Supponiamo che quest'ultima operazione venga omessa. Ci sarebbero degli effetti sulla correttezza del protocollo?

26. Supponiamo che il ciclo while con tre istruzioni vicino alla fine del protocollo 6 sia rimosso dal programma. Questo avrebbe degli effetti sulla correttezza del protocollo o solamente sulle performance? Spiegate la vostra risposta.

27. La distanza della Terra da un pianeta molto lontano è approssimativamente di 9×10^{10} m. Qual è l'uso del canale se un protocollo stop-and-wait viene usato per trasferire un frame su un collegamento punto a punto di 64 Mbps? Si assuma che la dimensione del frame sia 32 KB (kilobyte) e la velocità della luce sia 3×10^8 m/s.

28. Nel problema precedente si supponga di usare invece un protocollo a finestra scorrevole. Per quale finestra della sorgente l'uso del collegamento sarebbe del 100% ? Si ignorino i tempi di elaborazione del protocollo a entrambi i lati del collegamento.

29. Nel protocollo 6 il codice della procedura *frame_arrival* ha una sezione usata per i NAK. Questa sezione è invocata se il frame in arrivo è un NAK e vale anche un'altra condizione. Si trovi uno scenario in cui la presenza di quest'altra condizione risulti essenziale.

30. Consideriamo l'uso del protocollo 6 su una linea senza errori a 1 Mbps. La massima dimensione dei frame è 1.000 bit. I pacchetti vengono generati a 1 secondo l'uno dall'altro. L'intervallo di timeout è 10 millisecondi. Se lo speciale timer per gli acknowledgement fosse eliminato, avremmo dei timeout non necessari. Quante volte sarebbe trasmesso in media un messaggio?

31. Nel protocollo 6, $MAX_SEQ = 2^n - 1$. Questa condizione è ovviamente auspicabile per un uso efficiente dei bit dell'header, ma non abbiamo dimostrato che è anche essenziale. Per esempio, il protocollo può funzionare correttamente con $MAX_SEQ = 4$?

32. Frame da 1.000 bit vengono inviati su un canale a 1 Mbps usando un satellite geostazionario con un tempo di propagazione dalla Terra di 270 millisecondi. Gli acknowledgement sono sempre aggiunti con piggybacking ai frame di dati. Le intestazioni sono molto corte e vengono usati numeri di sequenza di 3 bit. Qual è la massima utilizzazione del canale raggiungibile per i seguenti protocolli:

a) stop-and-wait
b) protocollo 5

c) protocollo 6.

33. Si calcoli la frazione di banda che è persa in ridondanza (intestazioni e ritrasmissioni) per il protocollo 6 su un canale satellitare a 50 kbps con molto traffico. I data frame sono composti da 40 bit di header e 3.960 bit di dati. Assumiamo che il tempo di propagazione dalla Terra al satellite sia 270 millisecondi. Non ci sono frame di tipo ACK, mentre i NAK sono lunghi 40 bit. La frequenza degli errori per i frame di dati è 1%, mentre per i NAK è trascurabile. I numeri di sequenza sono da 8 bit.

34. Consideriamo un canale satellitare da 64 kbps e privo di errori, usato per trasmettere dei frame dati da 512 byte in una direzione, con degli acknowledgement molto corti nella direzione opposta. Qual è la massima velocità di trasmissione per finestre di dimensioni 1, 7, 15 e 127? Il tempo di propagazione dalla Terra al satellite è 270 millisecondi.

35. Un cavo di lunghezza 100 km trasmette i dati come una linea T1. La velocità di propagazione nel cavo è 2/3 la velocità della luce nel vuoto. Quanti bit stanno dentro al cavo?

36. Si dia almeno una spiegazione del perché il PPP faccia uso di byte stuffing anziché di bit stuffing per prevenire che la presenza di flag byte all'interno del payload causi confusione.

37. Qual è il minimo overhead spedendo un pacchetto IP tramite PPP? Si consideri solo l'overhead introdotto dallo stesso PPP e non quello dell'header IP. Qual è invece il massimo overhead?

38. Un pacchetto IP di 100 byte viene trasmesso su un local loop usando lo stack di protocolli di ADSL. Quante celle ATM verranno trasmesse? Si descriva sommariamente il loro contenuto.

39. Lo scopo di questo esercizio di laboratorio è quello di implementare un meccanismo per identificare gli errori usando l'algoritmo standard di CRC descritto nel testo. Vogliamo scrivere due programmi: un generatore e un verificatore. Il generatore legge dallo standard input un messaggio di n bit rappresentato da una stringa di 0 e 1 in una linea di testo ASCII. La seconda linea è il polinomio di k bit, anch'esso in ASCII. Il generatore scrive sullo standard output una linea di testo ASCII con $n + k$ 0s e 1s, che rappresenta il messaggio da trasmettere, e poi stampa anche il polinomio, così come l'aveva letto in input. Il programma verificatore legge l'output del generatore e stampa in output un messaggio che ne indica la correttezza. Infine, scriviamo un programma, alteratore, che inverte un bit nella prima linea a seconda degli argomenti con cui è invocato (il numero di bit si conta a partire da sinistra e partendo da 1), mentre lascia invariato il resto delle due linee. Scrivendo:

i) generatore < file | verificatore
si dovrà vedere un messaggio che indica che tutto è corretto, mentre

ii) generatore < file | alteratore arg | verificatore
si dovrà ricevere un messaggio di errore.

CAPITOLO 4
Il sottolivello MAC

I collegamenti di rete sono suddivisibili in due categorie: quelli che usano connessioni punto a punto e quelli che usano canali di trasmissione broadcast, in cui ogni messaggio viene recepito da tutti gli utilizzatori. Nel Capitolo 2 sono stati analizzati i collegamenti punto a punto; questo capitolo tratterà i collegamenti broadcast e i loro protocolli.

In qualsiasi rete broadcast il problema chiave è la scelta dell'entità che dovrà acquisire il diritto di utilizzo del canale in caso di competizione. Per chiarire questo punto consideriamo una conference call in cui sei persone sono munite di altrettanti telefoni, ma cablati in modo che ciascuno ascolti e parli con tutti. Non appena una persona smette di parlare è molto probabile che

altri due (o più) utenti inizino a parlare contemporaneamente, generando il caos. Nell'incontro faccia a faccia il caos è evitato grazie a mezzi esterni, per esempio alzando la mano a turno per chiedere la parola, ma quando è disponibile un solo canale è molto più difficile determinare chi deve essere il prossimo. Per risolvere il problema sono stati sviluppati molti protocolli che verranno esaminati in questo capitolo. In letteratura i canali broadcast qualche volta sono chiamati anche **canali ad accesso multiplo** (*multiaccess channel*) o **canali ad accesso casuale** (*random access channel*).

I protocolli per allocare l'uso di un canale ad accesso multiplo appartengono a un sottolivello del livello data link, chiamato **MAC** (*medium access control*). Il sottolivello MAC è importante soprattutto nelle LAN, specialmente se wireless, poiché molte si servono per la comunicazione proprio di un canale ad accesso multiplo; al contrario, le WAN (con l'eccezione delle reti satellitari) preferiscono le connessioni punto a punto. Poiché i canali ad accesso multiplo sono strettamente collegati alla tecnologia delle LAN, questo capitolo esamina le reti locali in generale, includendo alcuni problemi che non riguardano direttamente il sottolivello MAC, anche se l'argomento principale rimane il controllo del canale.

Tecnicamente parlando, il sottolivello MAC è la parte inferiore del livello data link; perciò, dal punto di vista logico, sarebbe stato più corretto studiarlo prima di esaminare i protocolli punto a punto descritti nel Capitolo 3. Tuttavia, molti trovano più semplice comprendere i protocolli che coinvolgono molteplici utilizzatori dopo aver studiato quelli che ne coinvolgono solo due e per questo motivo si è preferito non seguire una presentazione strettamente lineare dall'alto verso il basso (*bottom-up*).

4.1 Problema dell'allocazione del canale

Il tema centrale di questo capitolo sarà come allocare un singolo canale di trasmissione tra utenti in competizione tra loro. Il canale potrà essere una porzione dello spettro in una data regione geografica o un singolo cavo o una fibra ottica, a cui sono collegati più nodi. In ogni caso, il canale connetterà ogni utente a tutti gli altri e ogni utente che utilizza a pieno il canale interferirà con gli altri utenti che a loro volta desiderano utilizzare quel canale.

Esamineremo inizialmente le carenze relative alle strategie di allocazione statica per traffico irregolare. Quindi esporremo le ipotesi di base usate per modellare gli schemi dinamici che esamineremo nei prossimi paragrafi.

Allocazione statica del canale

Il modo tradizionale per allocare un canale singolo come una linea in una rete telefonica tra molteplici utenti in competizione è dividere la sua capacità usando una delle strategie di multiplexing descritte nel Paragrafo 2.5, quale FDM. Se ci sono N utenti, la larghezza di banda è divisa equamente in N parti e ogni utente è assegnato a una parte. Dato che ogni utente ha una frequenza di banda privata non ci saranno interferenze tra gli utenti. Quando c'è un basso e costante numero di utenti ognuno dei quali produce un flusso costante o un pesante carico di traffico, questa divisione risulta essere un meccanismo di allocazione semplice ed efficiente. Un esempio nel campo del wireless sono le stazioni radio FM: ogni stazione prende una porzione della banda FM e la usa, per la maggior parte del tempo, per trasmettere il proprio segnale.

Quando però il numero di utenti che trasmettono informazioni è molto elevato e variabile oppure quando il traffico è irregolare, FDM presenta alcuni problemi. Se lo spettro è diviso in N regioni, ma il numero di utenti impegnato a comunicare è inferiore a N, si spreca parte dello spettro; se invece più di N utenti cercano di comunicare alcuni non potranno farlo (perché non è disponibile la banda necessaria), anche se alcuni utenti che in precedenza hanno ricevuto una banda di frequenza non stanno ricevendo o trasmettendo alcunché.

Anche assumendo che il numero di utenti possa in qualche modo essere mantenuto costante e pari a N, dividere il singolo canale disponibile in un numero di sottocanali statici è intrinsecamente inefficiente. Il problema di base è che, quando qualche utente è inattivo, la relativa larghezza di banda va semplicemente persa: l'utente non la sta usando e nessun altro è abilitato ad accedervi. Un'allocazione statica non si combina bene con la maggior parte dei sistemi informatici, nei quali il traffico di dati è estremamente irregolare, spesso con un rapporto tra picchi di traffico e media di traffico pari a 1000:1. Di conseguenza, molti dei canali resteranno inattivi per la maggior parte del tempo.

Le scarse prestazioni del sistema FDM statico si possono facilmente evidenziare considerando un semplice calcolo di teoria delle code. Cominciamo calcolando il ritardo medio T relativo alla spedizione di un frame su un canale con una capacità di C bps; si assuma che il tasso medio degli

arrivi casuali sia l frame/s e che ogni frame abbia una lunghezza variabile con una media di 1/m bit. Con questi parametri il tasso di servizio del canale è m*C*. Un risultato standard della teoria delle code è:

$$T = \frac{1}{\mu C - \lambda}$$

(Per i curiosi, questo risultato vale per le code di tipo "M/M/1". Ciò richiede che i valori casuali relativi al tempo di interarrivo tra i frame e alla loro lunghezza seguano una distribuzione esponenziale o, equivalentemente, siano il risultato di un processo Poissoniano).

Nel nostro esempio, posto *C* pari a 100 Mbps, la lunghezza media del frame (1/m) è 10.000 bit e il tasso di arrivo dei frame (l), è pari a 5000, risulta *T* = 200 ms. Si noti che, ignorando il ritardo di accodamento e chiedendo semplicemente di calcolare il tempo impiegato per inviare un frame di 10.000 bit attraverso una rete a 100 Mbps, si otterrebbe la risposta (errata) di 100 ms, risultato che vale solo in caso di assenza di competizione per il canale.

Dividiamo ora il canale in *N* sottocanali indipendenti, ognuno di capacità *C*/*N* bps. Il tasso medio di ingresso su ogni sottocanale ora è pari a l/*N*. Ricalcolando *T* si ottiene:

$$T_N = \frac{1}{\mu(C/N) - (\lambda/N)} = \frac{N}{\mu C - \lambda} = NT \quad (4\text{-}1)$$

Il ritardo medio per un canale suddiviso è *N* volte peggiore che se tutti i frame fossero in qualche modo magicamente ordinati in una grande coda centrale. Rapportando questo assunto all'ingresso di una banca pieno di sportelli bancomat, sarebbe meglio avere una singola coda per tutte le macchine, piuttosto che code distinte per ogni singola macchina.

Lo stesso ragionamento applicabile a FDM vale anche per altri modi di dividere staticamente il canale. Se dovessimo usare TDM e assegnare a ogni utente l'*N*-esimo intervallo temporale, un intervallo non utilizzato dall'utente andrebbe sprecato. Lo stesso accadrebbe se le reti venissero divise fisicamente: nell'esempio precedente, sostituendo la rete a 100 Mbps con dieci reti a 10 Mbps assegnate staticamente ai singoli utenti, il ritardo medio passerebbe da 200 ms a 2 ms.

Poiché nessuno dei metodi tradizionali di allocazione statica del canale funziona bene con il traffico irregolare, passeremo ora a esaminare soluzioni dinamiche.

Ipotesi per l'allocazione di canali dinamici

Prima di studiare il primo dei tanti metodi di allocazione di un canale descritti in questo capitolo conviene formulare attentamente il problema dell'allocazione. Le premesse fondamentali alla base di tutto il lavoro svolto in questo settore sono cinque.

1. **Traffico indipendente**. Il modello consiste di *N***stazioni** indipendenti (come computer, telefoni e così via), ognuna con un programma o un utente che genera frame per la trasmissione. Il numero atteso di frame generati in un intervallo di lunghezza D*t* è lD*t*, dove l è una costante (il tasso di arrivo dei frame). Quando un frame è stato generato la stazione è bloccata e non fa nulla finché il frame non sia stato trasmesso con successo.

2. **Canale singolo**. Un unico canale è disponibile per tutte le comunicazioni. Tutte le stazioni possono trasmettere dati su di esso e possono riceverne da esso. Pur supponendo che le stazioni siano tutte equamente capaci, i protocolli potranno assegnare loro ruoli differenti (ad esempio delle priorità).
3. **Collisioni osservabili**. Due frame trasmessi simultaneamente si sovrapporranno e il segnale risultante sarà incomprensibile. Questo evento è chiamato **collisione**. Tutte le stazioni possono percepire che ha avuto luogo una collisione. Un frame che ha subìto una collisione dovrà essere ritrasmesso in seguito. Non ci sono altri errori a parte quelli generati dalle collisioni.
4. **Tempo continuo o diviso in intervalli**. Si potrebbe supporre che il tempo sia continuo, nel cui caso la trasmissione di un frame può iniziare in qualsiasi momento. In alternativa, il tempo potrebbe essere diviso in intervalli discreti (*slot*). La trasmissione dei frame dovrà quindi incominciare con l'inizio dello slot. Uno slot può contenere 0, 1 o più frame, corrispondenti rispettivamente a uno slot inutilizzato, a una trasmissione riuscita e a una collisione.
5. **Rilevamento di portante o non rilevamento di portante**. Nel caso di un rilevamento della portante le stazioni possono indicare se il canale è in uso prima di provare a utilizzarlo. Nessuna stazione proverà a utilizzare un canale mentre è percepito come occupato. In assenza di rilevamento della portante, le stazioni non possono rilevare lo stato del canale prima di provare a utilizzarlo e quindi procedono semplicemente a trasmettere. Solo in seguito potranno determinare se la trasmissione ha avuto successo.

È doveroso fare un po'di analisi su queste ipotesi. La prima afferma che l'arrivo di ogni frame è indipendente dagli altri arrivi e che i frame vengono prodotti in maniera predicibile, a un tasso costante. In realtà, questo assunto non è un modello particolarmente valido per il traffico di rete, in quanto si sa bene che i pacchetti arrivano a raffiche (*burst*) in intervalli di tempo variabili (Paxson e Floyd, 1995; Leland et al., 1994). Nonostante tutto, questi **modelli Poissoniani**, come spesso sono chiamati, sono utili perché trattabili matematicamente. Essi ci aiutano ad analizzare i protocolli per capire approssimativamente come le prestazioni della rete cambino al variare dei parametri di utilizzo e come questa si confronti con altre architetture.

L'assunzione riguardo il singolo canale è il punto centrale del modello: non esistono altri modi di comunicare. Le stazioni non possono alzare la mano per richiedere che il maestro gli dia la parola, per cui si dovrà trovare una soluzione migliore.

Le tre ipotesi rimanenti dipendono dall'architettura del sistema e diremo quali di esse saranno valide ogni volta che si esaminerà un particolare protocollo.

L'assunto relativo alle collisioni è basilare: se le stazioni devono ritrasmettere frame anziché perderli, avranno bisogno di un qualche modo di percepire se ci siano o meno collisioni. Per quanto riguarda i canali trasmissivi cablati è possibile progettare l'hardware di un nodo per rilevare quando accadono le collisioni; le stazioni possono quindi terminare le loro trasmissioni prematuramente in modo da evitare sprechi. Perché questo rilevamento è molto più difficile per i canali wireless, le collisioni in genere sono desunte a seguito della mancanza di un frame di riscontro. È anche possibile che alcuni frame coinvolti in una collisione vengano ricevuti con successo, a seconda delle caratteristiche del segnale e dell'apparato di ricezione. Tuttavia, questa situazione non è quella più comune, quindi assumeremo che tutti i frame coinvolti in una collisione vadano perduti. Esamineremo anche i protocolli progettati per prevenire le collisioni.

La ragione delle due ipotesi alternative sul tempo (continuo o diviso in intervalli) è che quello diviso in intervalli può essere usato per migliorare le prestazioni. Tuttavia, ciò comporta che le stazioni dovranno attenersi a un orologio principale (*master clock*) o sincronizzare le loro azioni per dividere il tempo in intervalli discreti, cosa che non è sempre fattibile. Discuteremo e analizzeremo sistemi di entrambi i tipi; per un dato sistema solo uno di essi sarà valido.

Analogamente una rete potrà fare rilevamento di portante oppure no; le reti cablate in generale lo faranno. Le reti wireless non sempre potranno farlo in modo efficiente, perché non tutte le stazioni saranno entro la portata radio di tutte le altre. Allo stesso modo, il rilevamento della portante non sarà disponibile nei casi in cui una stazione non potrà comunicare direttamente con le altre, come nel caso di un cable modem in cui le stazioni devono comunicare tramite la stazione di testa.

Per evitare ogni equivoco vale la pena notare che nessun protocollo per canali ad accesso multiplo garantisce l'affidabiltà della consegna. Anche in assenza di collisioni il destinatario potrebbe aver ricevuto parte del frame in modo scorretto per diversi motivi. Altre parti del livello di collegamento o dei livelli superiori sono addette a fornire l'affidabilità.

4.2 Protocolli ad accesso multiplo

Per allocare un canale ad accesso multiplo esistono molti algoritmi; nei paragrafi successivi esamineremo alcuni dei più interessanti e daremo alcuni esempi su come generalmente vengono utilizzati nella pratica.

ALOHA

La storia del primo MAC comincia nelle incontaminate Hawaii nei primi anni '70. In questo caso il termine "incontaminato" va inteso come "privo di un sistema telefonico funzionante". Questo non facilitò la vita al ricercatore Norman Abramson e ai suoi colleghi dell'Università delle Hawaii che stavano cercando di connettere gli utenti delle varie isole al computer principale a Honolulu. Stendere dei cavi sul fondo dell'Oceano Pacifico non era possibile, quindi cercarono una soluzione diversa.

Quella che trovarono usava radio a corto raggio, in cui ogni terminale utilizzava la stessa frequenza di upstream per spedire i frame al computer centrale: un metodo semplice ed elegante per risolvere il problema dell'allocazione del canale. Da allora le ricerche furono sviluppate da molti altri ricercatori (Schwartz e Abramson, 2009). Anche se il lavoro di Abramson, chiamato ALOHA, utilizzava la radiodiffusione terrestre, l'idea di base è applicabile a ogni altro sistema in cui utenti senza coordinazione sono in competizione per utilizzare un solo canale condiviso.

Vedremo di seguito due versioni di ALOHA: puro e slotted. La differenza tra loro sta nel tempo: continuo nella versione pura o diviso in intervalli discreti ai quali tutti i frame devono attenersi nella versione slotted.

ALOHA puro

L'idea alla base del sistema ALOHA è semplice: consentire agli utenti di trasmettere ogni volta che hanno dati da inviare; naturalmente ci potranno essere collisioni che danneggeranno i frame e chi trasmette avrà bisogno di sapere se è successo. In ALOHA, dopo che ogni stazione gli ha spedito il suo frame, il computer centrale rispedisce il frame, in broadcast, a tutte le stazioni. Una stazione trasmittente può ascoltare la trasmissione del coordinatore (*hub*) per vedere se il frame è arrivato a destinazione. In altri sistemi, come le LAN cablate, la trasmittente deve essere in grado di rilevare le collisioni mentre sta trasmettendo.

Se il frame è andato perso, il trasmettitore rimane in attesa per un intervallo casuale prima di ripetere la trasmissione. Il tempo di attesa deve essere casuale, altrimenti gli stessi frame continueranno a collidere in un ciclo senza fine. I sistemi dove più utenti condividono un canale in un modo che può causare conflitto si dicono **sistemi a contesa**.

La Figura 4.1 mostra lo schema di generazione dei frame in un sistema ALOHA. Tutti i frame hanno la stessa lunghezza perché la capacità di trasporto dei sistemi ALOHA è massima quando si utilizzano frame con dimensione fissa piuttosto che permettere una dimensione variabile.

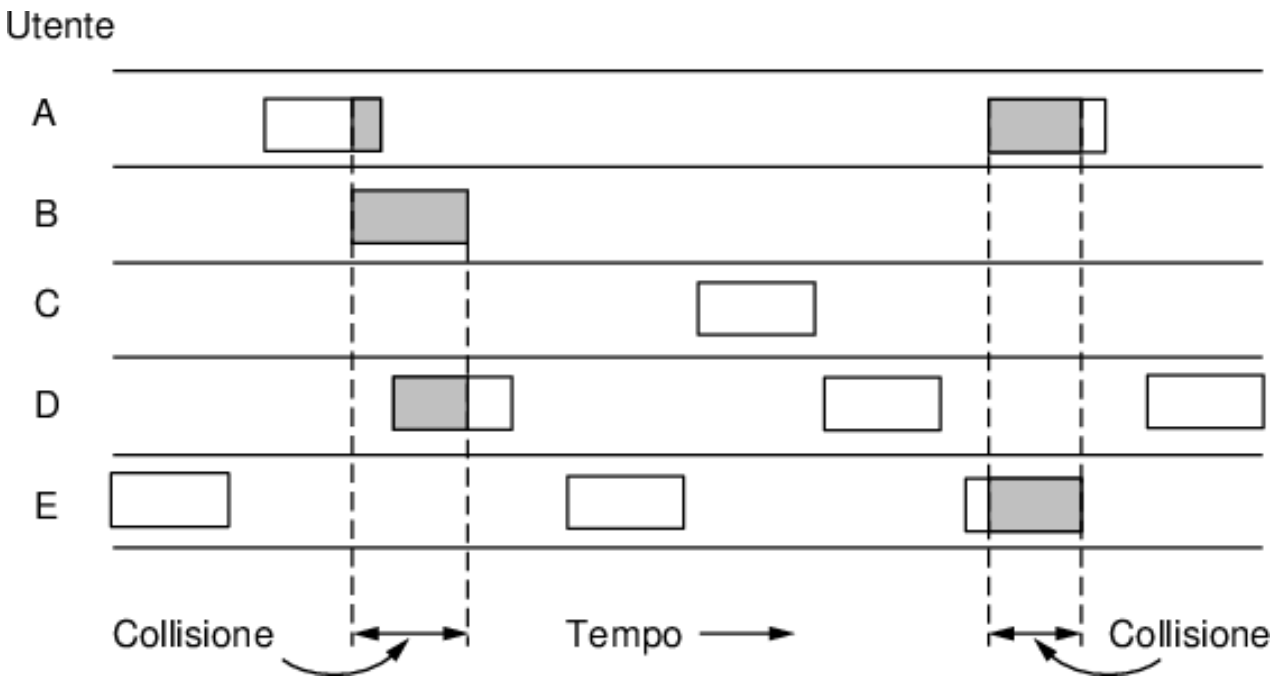


Figura 4.1 In ALOHA puro i frame sono trasmessi in momenti totalmente arbitrari.

Ogni volta che due frame tentano di occupare contemporaneamente il canale si verifica una collisione (come appare nella Figura 4.1) che li danneggia entrambi. Basta che il primo bit di un nuovo frame si sovrapponga all'ultimo bit di un frame quasi completato per considerarli entrambi totalmente distrutti (il checksum è sbagliato) e quindi da ritrasmettere. Il checksum non sa (né deve) distinguere tra perdita totale e successo parziale della trasmissione.

Ma qual è l'efficienza di un canale ALOHA, ossia che frazione dei frame trasmessi sfugge alla collisione in queste circostanze caotiche? Si consideri prima di tutto un gruppo infinito di utenti interattivi seduti davanti ai loro computer (stazioni). Un utente si trova sempre in uno di due stati: sta scrivendo oppure è in attesa. Inizialmente, tutti gli utenti stanno scrivendo. Quando termina una riga, l'utente smette di scrivere e attende una risposta; a questo punto la stazione trasmette al computer centrale un frame che contiene la riga sul canale condiviso e controlla il canale per scoprire se la trasmissione abbia avuto successo. In caso positivo, l'utente vede apparire la risposta e ricomincia a scrivere; in caso negativo, l'utente continua ad aspettare e il frame è ritrasmesso finché la trasmissione non ha successo.

Si supponga che il tempo di frame (*frame time*) indichi l'intervallo di tempo richiesto per trasmettere un frame di lunghezza standard e fissa (pertanto pari alla lunghezza del frame divisa per la capacità del collegamento). A questo punto si può assumere che i frame generati dalle stazioni siano ben modellati da una distribuzione Poissoniana con media pari a N frame per tempo di frame. Il presupposto della popolazione infinita garantisce che N non diminuisce quando gli utenti cominciano a essere bloccati. Se $N > 1$ la comunità di utenti sta generando frame a un tasso più elevato di quello che il canale è in grado di gestire,[1] quasi ogni frame subirà una collisione. Per garantire una ragionevole capacità di trasporto si dovrà assumere $0 \leq N \leq 1$.

Oltre ai nuovi frame, le stazioni ritrasmettono anche i frame entrati precedentemente in collisione. Supporremo inoltre che i frame vecchi e nuovi combinati insieme siano ben modellati da una distribuzione Poissoniana con una media di G frame per tempo di frame e che, chiaramente, sia $G \geq N$. A basso carico (ad esempio $N \approx 0$) ci saranno poche collisioni e di conseguenza poche ritrasmissioni, per cui $G \approx N$. A carico elevato ci saranno molte collisioni, perciò $G > N$. Se G è il carico che si presenta, qualunque sia il carico la capacità di trasporto S sarà G volte la probabilità P_0 di una trasmissione che ha successo, ossia $S = GP_0$, dove P_0 è la probabilità che un frame non entri in collisione con un altro frame.

Un frame non entrerà in collisione se nessun altro sarà trasmesso nello stesso intervallo temporale, come mostrato nella Figura 4.2. In quali condizioni il frame evidenziato arriverà a destinazione senza riportare alcun danno? Si indichi con t il tempo richiesto per inviare il frame: la parte iniziale del frame evidenziato entra in collisione con la fine di un qualsiasi altro frame generato da un utente nel tempo compreso tra t_0 e $t_0 + t$. Il destino del frame evidenziato era segnato già prima che iniziasse la trasmissione del primo bit, ma poiché in ALOHA puro una stazione non ascolta il canale prima di iniziare a trasmettere, non c'è modo di sapere se un altro frame sia in fase di trasmissione. Analogamente, i frame che inizieranno a essere trasmessi nel periodo compreso tra $t_0 + t$ e $t_0 + 2t$ si scontreranno sicuramente nella parte finale del frame evidenziato.

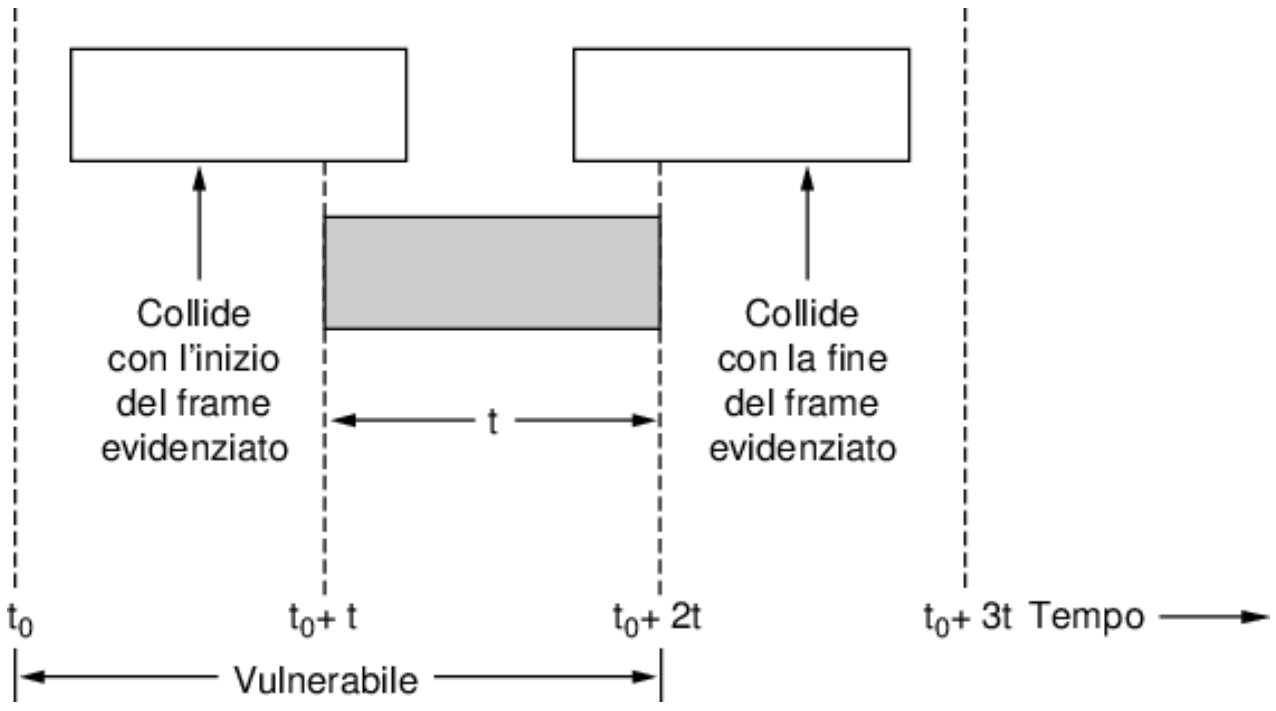


Figura 4.2 Il periodo vulnerabile del frame evidenziato.

La probabilità che k frame siano generati durante un dato tempo di frame è data dalla distribuzione di Poisson:

$$\Pr[k] = \frac{G^k e^{-G}}{k!} \quad (4\text{-}2)$$

perciò la probabilità di generare zero frame è e^{-G}. In un intervallo pari a due frame time, il numero medio di frame generati è pari a $2G$. La probabilità che nessun altro frame sia generato durante l'intero periodo vulnerabile è perciò data da $P_0 = e^{-2G}$.

Ponendo $S = GP_0$ si ottiene:

$$S = Ge^{-2G}$$

La relazione tra il traffico che si presenta e la capacità di trasporto è mostrata nella Figura 4.3. La capacità di trasporto massima si ha per $G = 0{,}5$ con $S = 1/2e$, che è circa 0,184. In altri termini, si può sperare di utilizzare al massimo il 18% del canale. Questo risultato non è molto incoraggiante, ma se tutti trasmettono a piacimento è difficile aspettarsi una percentuale di successo del cento per cento.

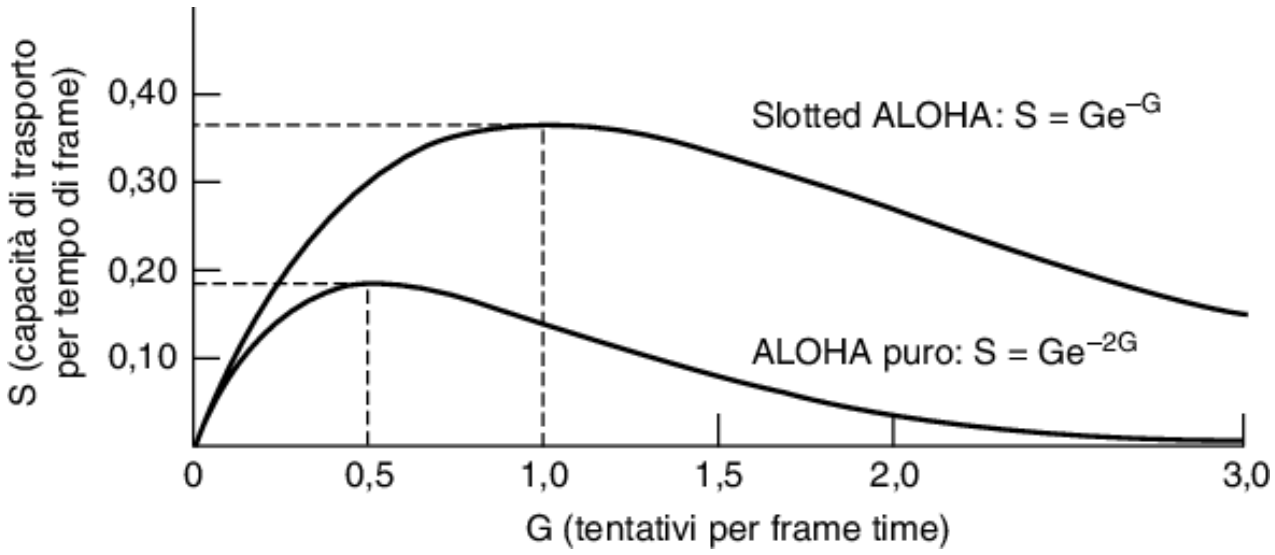


Figura 4.3 La capacità di trasporto in funzione del traffico per un sistema ALOHA.

Slotted ALOHA

Subito dopo l'avvento di ALOHA, Roberts (1972) pubblicò un sistema per duplicarne la capacità. La sua proposta era di dividere il tempo in intervalli discreti chiamati **slot**, ognuno corrispondente a un frame. Questo approccio richiede che gli utenti concordino sui limiti degli intervalli; la necessaria sincronizzazione si può soddisfare con una speciale stazione che emetta un segnale all'inizio di ogni intervallo, proprio come un metronomo.

Rispetto al metodo di Abramson, **ALOHA puro**, da quello di Roberts, conosciuto come **slotted ALOHA**, emerge una differenza importante: un computer non può inviare dati ogni volta che viene premuto il tasto Invio. Quando l'utente completa una riga il computer deve attendere l'inizio dello slot successivo e ciò trasforma ALOHA da un sistema a tempo continuo a uno a tempo discreto. Poiché ora il periodo vulnerabile è dimezzato, la probabilità che non ci sia altro traffico durante lo stesso intervallo occupato dal frame di prova è pari a e^{-G}, quindi:

$$S = Ge^{-G} \quad (4\text{-}3)$$

Come si può notare osservando la Figura 4.3, slotted ALOHA ha il massimo in $G = 1$, con una capacità di trasporto di $S = 1/e$, che equivale a circa 0,368, il doppio del valore di ALOHA puro. Se il sistema sta operando a $G = 1$, la probabilità che si presenti uno slot vuoto è di 0,368 (dall'Equazione 4-2). La migliore situazione che si può ottenere con slotted ALOHA prevede il 37% di slot vuoti, il 37% di trasmissioni senza errori e il 26% di collisioni. Operando con valori di G più alti si riduce il numero di slot vuoti, ma si aumenta esponenzialmente il numero di collisioni. Per osservare con quanta rapidità può crescere il numero di collisioni si consideri la trasmissione di un frame di prova: la probabilità che esso eviti la collisione è e^{-G}, valore uguale alla probabilità che tutti gli altri utenti non trasmettano nello stesso slot; la probabilità di una collisione è allora $1 - e^{-G}$; la probabilità che una trasmissione richieda esattamente k tentativi *(*$k - 1$ collisioni seguite da un successo) sarà:

$$P_k = e^{-G}(1 - e^{-G})^{k-1}$$

Il numero atteso di trasmissioni, E, necessario ogni volta che si preme il tasto Invio su un terminale è quindi pari a:

$$E = \sum_{k=1}^{\infty} kP_k = \sum_{k=1}^{\infty} ke^{-G}(1 - e^{-G})^{k-1} = e^{G}$$

A causa della dipendenza esponenziale di E da G, piccole variazioni nel carico del canale possono ridurre drasticamente le sue prestazioni.

L'importanza di slotted ALOHA non è così ovvia. Concepito negli anni '70, venne utilizzato da alcuni sistemi sperimentali per poi essere quasi dimenticato. Quando fu inventato il metodo di accesso a Internet via cavo, improvvisamente si presentò il problema di come allocare un canale condiviso tra più utenti in competizione tra loro e slotted ALOHA venne tirato fuori dal cestino dei rifiuti per salvare la situazione. In seguito, avere diverse etichette RFID che parlano allo stesso lettore RFID presentò un'altra variante dello stesso problema; slotted ALOHA, con un pizzico di altre idee mescolate insieme, venne in aiuto ancora una volta. È successo spesso che protocolli perfettamente validi cadessero in disuso per motivi politici (ad esempio, qualche grande azienda che imponeva di fare le cose a modo proprio) o per seguire mode tecnologiche in continuo mutamento. Anni dopo, improvvisamente qualcuno si rende conto che un protocollo caduto nell'oblio può risolvere i suoi problemi contingenti. Per questo motivo qui studieremo diversi eleganti protocolli che oggi non sono molto diffusi, ma che potrebbero facilmente tornare utili in applicazioni future, se saranno portati a conoscenza di un numero sufficiente di progettisti delle reti. Naturalmente, studieremo anche molti protocolli di uso comune.

Protocolli ad accesso multiplo con rilevamento della portante

Con slotted ALOHA il massimo utilizzo possibile del canale è di 1/*e*. Questo risultato non deve sorprendere, poiché con stazioni che trasmettono a piacimento senza sapere che cosa stanno facendo le altre è naturale avere un numero elevato di collisioni. Fortunatamente nelle LAN le stazioni sono in grado di rilevare le azioni intraprese dalle altre e di adattare di conseguenza il proprio comportamento. Queste reti possono raggiungere un livello di utilizzo molto maggiore di 1/*e*. In questo paragrafo descriveremo alcuni protocolli che consentono di migliorare le prestazioni.

I protocolli in cui le stazioni si mettono in ascolto di una portante (ossia di una trasmissione) e si comportano di conseguenza sono chiamati **protocolli con rilevamento della portante** (*carrier sense protocol*): negli anni ne sono stati proposti molti. Kleinrock e Tobagi (1975) hanno analizzato in dettaglio diversi protocolli di questo tipo.

CSMA persistente e non persistente

II primo protocollo con rilevamento della portante che analizzeremo si chiama **CSMA 1-persistente** (1-*persistent carrier sense multiple access*). Si tratta di un'anticipazione del più semplice schema CSMA. Quando una stazione ha dei dati da trasmettere, prima di tutto ascolta il canale per scoprire se qualcun altro in quel momento sta trasmettendo. Se il canale è libero la stazione trasmette i propri dati; altrimenti, se il canale è occupato, la stazione aspetta finché non si libera per poi trasmettere un frame. Se avviene una collisione la stazione attende un tempo casuale e ricomincia da capo. Il protocollo è chiamato "1-persistente" perché la stazione trasmette con probabilità 1 quando trova il canale libero.

Vi potreste aspettare che questo schema eviti le collisioni eccetto che nel raro caso di invii simultanei, ma in realtà non lo fa. Se due stazioni diventassero pronte durante la trasmissione di una terza, entrambe attenderebbero pazientemente fino al termine della trasmissione e quindi entrambe trasmetterebbero esattamente in simultanea, dando luogo a una collisione. Se esse non fossero così impazienti il numero di collisioni sarebbe inferiore.

Più precisamente, il ritardo di propagazione ha un effetto importante sulle collisioni. C'è la possibilità che appena dopo l'inizio dell'invio da parte di una stazione, un'altra diventi pronta a spedire e ascolti il canale. Se il segnale della prima stazione non è ancora arrivato alla seconda, quest'ultima sentirà il canale libero e comincerà a inviare, risultando così in collisione. Questa casualità dipende dal numero di frame che possono stare sul canale o dal **prodotto banda-ritardo del canale** (*bandwidth-delay product*). Se solo una piccola parte di un frame occupa già tutto il canale, che è il caso della maggior parte delle LAN quando il ritardo di propagazione è ridotto, la possibilità che avvenga una collisione è minima. Più grande è il prodotto banda-ritardo, più significativo sarà questo effetto e peggiori le prestazioni del protocollo.

Tuttavia, questo protocollo ha prestazioni migliori di ALOHA puro, poiché entrambe le stazioni hanno la decenza di desistere dall'interferire con i frame della terza. Esattamente la stessa cosa vale per slotted ALOHA.

Un secondo protocollo con rilevamento della portante si chiama **CSMA non persistente**. In questo protocollo si tenta consapevolmente di essere meno ingordi. Come nel caso precedente, prima di trasmettere, ogni stazione controlla il canale: se nessun altro sta trasmettendo inizia a inviare i dati, ma se il canale è occupato la stazione non esegue un controllo continuo per impossessarsene subito alla fine della trasmissione, attende invece per un intervallo di tempo casuale prima di ripetere l'algoritmo. Di conseguenza, questo meccanismo permette di utilizzare meglio il canale, ma allunga i ritardi rispetto a CSMA 1-persistente.

L'ultimo protocollo è detto **CSMA p-persistente**; si applica ai canali divisi in intervalli temporali e funziona così: quando è pronta a trasmettere, ogni stazione controlla il canale. Se lo trova libero, trasmette subito con una probabilità p e rimanda la trasmissione all'intervallo successivo con probabilità $q = 1 - p$. Se anche quell'intervallo risultasse libero, la stazione trasmetterebbe oppure rimanderebbe un'altra volta (anche in questo caso con le probabità p e q rispettivamente). Il processo si ripete fino a quando il frame non è stato trasmesso o un'altra stazione ha iniziato a trasmettere (lo slot non era libero); in questo ultimo caso, la stazione sfortunata si comporterebbe come se ci fosse stata una collisione (ossia attenderebbe un intervallo di tempo casuale per poi ricominciare). Se la stazione inizialmente avesse trovato il canale occupato, avrebbe atteso fino all'intervallo successivo per poi applicare l'algoritmo descritto in precedenza. IEEE 802.11 usa un raffinamento di CSMA p-persistente che discuteremo nel Paragrafo 4.4.

La Figura 4.4 mostra la capacità di trasporto calcolata rispetto al traffico atteso per tutti e tre i protocolli e per ALOHA sia puro che slotted.

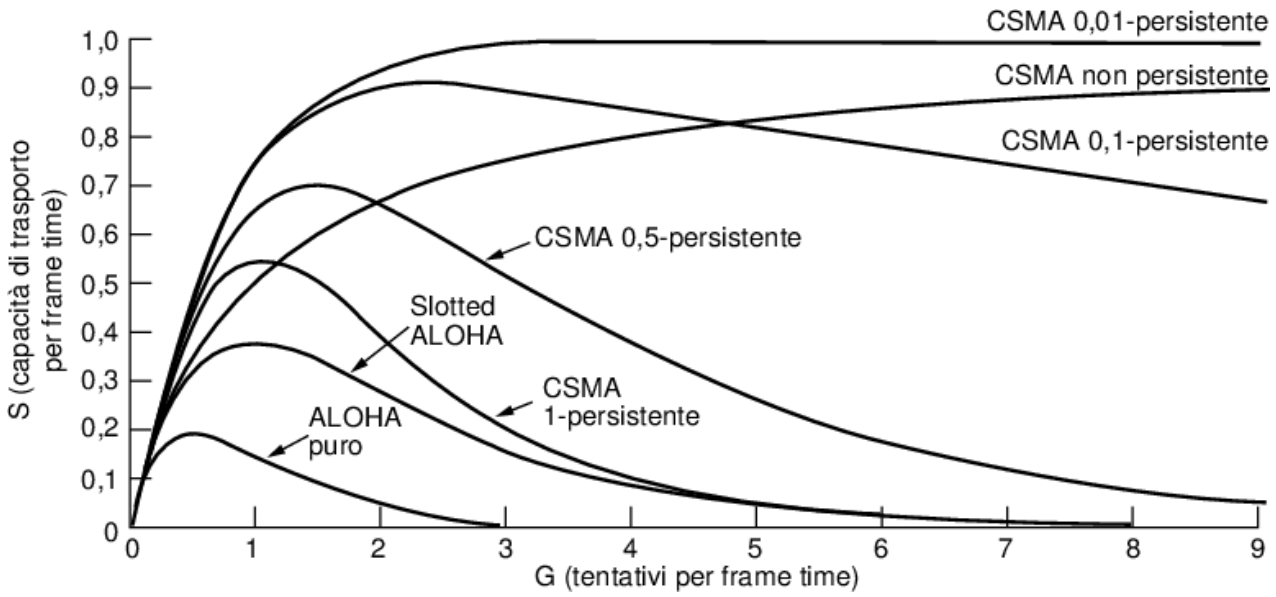


Figura 4.4 Confronto tra gli utilizzi del canale in funzione del carico per protocolli ad accesso casuale.

CSMA con rilevamento delle collisioni

I protocolli CSMA persistente e CSMA non persistente rappresentano chiaramente un miglioramento rispetto ad ALOHA, in quanto garantiscono che nessuna stazione inizi a trasmettere se il canale è occupato. Tuttavia, se due stazioni percepiscono un canale libero e cominciano a trasmettere simultaneamente, i loro segnali collideranno. Un altro miglioramento consiste nel rilevare velocemente la collisione e interrompere bruscamente le trasmissioni (anziché portarle a termine) dal momento che sono comunque irrimediabilmente rovinate. Questa strategia permette di risparmiare tempo e banda.

Questo protocollo, chiamato **CSMA/CD** (*CSMA with collision detection*), è alla base delle classiche LAN Ethernet, perciò vale la pena esaminarlo in dettaglio. È importante capire che l'individuazione delle collisioni è un processo analogico. L'hardware della stazione deve ascoltare il canale durante la trasmissione. Se il segnale letto è diverso da quello inviato, sa che sta avvenendo una collisione. Questo implica che il segnale letto non debba essere molto piccolo rispetto a quello inviato (ciò è difficile per il wireless, poiché i segnali ricevuti possono essere un milione di volte più deboli dei segnali trasmessi) e la modulazione deve essere scelta in maniera tale da permettere la determinazione delle collisioni (ad esempio, una collisione tra due segnali a 0 volt sarà impossibile da rilevare).

CSMA/CD, come molti altri protocolli per LAN, utilizza il modello concettuale rappresentato nella Figura 4.5. Nel punto contrassegnato da t_0 una stazione ha appena finito di trasmettere il suo frame; ora qualunque altra stazione che abbia un frame da inviare può tentare di trasmettere. Se due o più stazioni decidono contemporaneamente di trasmettere si verifica una collisione. Se una stazione rileva una collisione, interrompe la sua trasmissione, aspetta per una quantità di tempo casuale e quindi prova di nuovo (presumendo che nel frattempo nessun'altra stazione abbia iniziato a trasmettere). Quindi il nostro modello per CSMA/CD consisterà nell'alternanza di periodi di conflitto e trasmissione, con periodi di inattività che si verificano quando tutte le stazioni sono silenziose (per mancanza di lavoro).

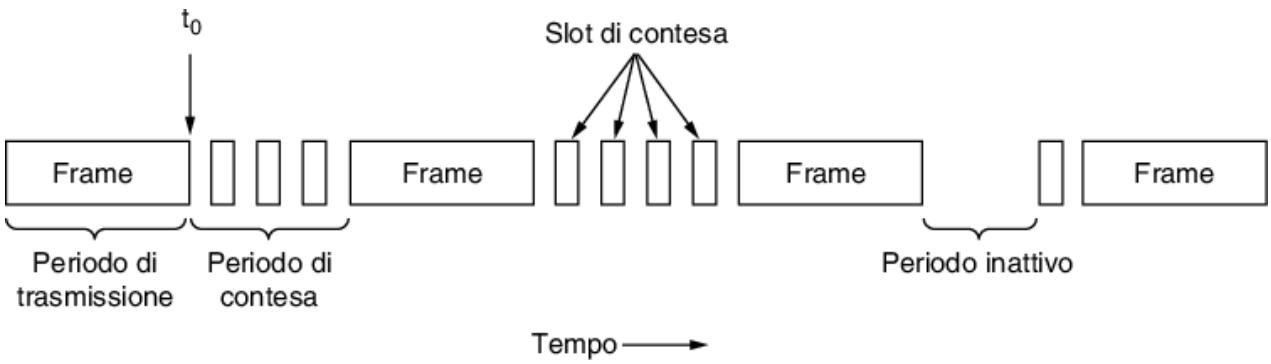


Figura 4.5 CSMA/CD può essere in uno di questi stati: contesa, trasmissione o inattivo.

Osserviamo ora più in dettaglio l'algoritmo di contesa. Si supponga che due stazioni inizino entrambe a trasmettere esattamente al tempo t_0. Quanto tempo passerà prima che le stazioni realizzino che è avvenuta una collisione? La risposta è vitale, perché permette di determinare la lunghezza del periodo di contesa e di conseguenza i valori del ritardo e della capacità di trasporto.

Il tempo minimo per rilevare una collisione è pari al tempo impiegato dal segnale per propagarsi da una stazione all'altra. In base a questo ragionamento si potrebbe pensare che se una stazione, dopo aver avviato la propria trasmissione, non rileva una collisione per un tempo corrispondente al tempo massimo di propagazione sul cavo, può essere certa di averne assunto il controllo. Con "assunzione di controllo" si intende che tutte le altre stazioni siano a conoscenza della trasmissione e non interferiranno, ma purtroppo non è così.

Consideriamo lo scenario che descrive la situazione peggiore possibile: si supponga che t sia il tempo che il segnale impiega per propagarsi tra le due stazioni più lontane; nell'istante t_0 una stazione inizia a trasmettere. A t_0 + t – e, un istante prima che il segnale arrivi alla stazione più distante, anche questa inizia a trasmettere. La stazione ovviamente rileva la collisione e interrompe l'operazione quasi istantaneamente, ma il piccolo picco di rumore causato dalla collisione riesce a tornare indietro alla stazione di origine solo al tempo 2t – e. In altri termini, nel caso peggiore una stazione può essere certa di aver assunto il controllo del canale solo se non rileva alcuna collisione per un tempo pari a 2t.

Alla luce di tutto ciò, possiamo pensare che il conflitto CSMA/CD sia come un sistema slotted ALOHA con uno slot di 2t. Su di un cavo coassiale di 1 km, t ≈ 5ms. La differenza tra CSMA/CD e slotted ALOHA è che gli slot nei quali solo una stazione trasmette (quelli in cui il canale è stato occupato) sono seguiti dal resto di un frame. Questa differenza tanto più migliorerà nettamente le prestazioni, quanto più il tempo di frame sarà maggiore del tempo di propagazione.

Protocolli senza collisione

Una volta che la stazione ha acquisito il canale con certezza, con CSMA/CD non ci possono essere collisioni, ma questo problema può ancora presentarsi durante il periodo di contesa. Le collisioni influenzano sfavorevolmente le prestazioni del sistema, specialmente quando il prodotto banda-ritardo è molto grande, ad esempio quando il cavo è lungo (cioè in cui t è grande) e i frame sono corti. Le collisioni non solo riducono l'ampiezza di banda, ma anche rendono il tempo di trasmissione dei frame variabile, il che non è una buona cosa per il traffico in tempo reale come VoIP.

In questo paragrafo esamineremo alcuni protocolli in grado di risolvere la contesa per il canale senza alcuna collisione, neppure durante il periodo stesso di contesa. Molti di questi protocolli non sono oggi utilizzati dalle principali architetture ma, in un settore in continuo cambiamento, avere protocolli con eccellenti proprietà disponibili per il futuro è spesso una buona cosa.

Nei protocolli descritti di seguito si presuppone che ci siano esattamente N stazioni, ognuna associata a un indirizzo univoco che varia da 0 a N – 1. Non importa se alcune stazioni possono essere inattive per parte del tempo. Si presuppone anche che il ritardo di propagazione sia trascurabile. La domanda di base è sempre la stessa: a quale stazione deve essere assegnato il canale dopo una trasmissione che ha avuto successo? Proseguiamo utilizzando il modello, rappresentato nella Figura 4.5, basato su intervalli di contesa discreti.

Protocollo a mappa di bit

Nel primo protocollo senza collisione, che usa un **metodo a mappa di bit elementare** (*basic bit-map method*), ogni periodo di contesa è composto da esattamente N slot. Se la stazione 0 ha un frame da inviare, trasmette un bit 1 durante l'intervallo 0. A nessun'altra stazione è concesso di trasmettere durante questo slot. Indipendentemente da quello che sta facendo la stazione 0, la stazione 1 ha la possibilità di trasmettere un 1 durante lo slot 1, ma solo se ha un frame in coda. In generale, la stazione j può annunciare il possesso di un frame da inviare inserendo un bit 1 nello slot j. Una volta trascorsi tutti gli N intervalli, ogni stazione sa quali sono le stazioni che intendono trasmettere; a questo punto, le stazioni iniziano a trasmettere in ordine numerico (Figura 4.6).

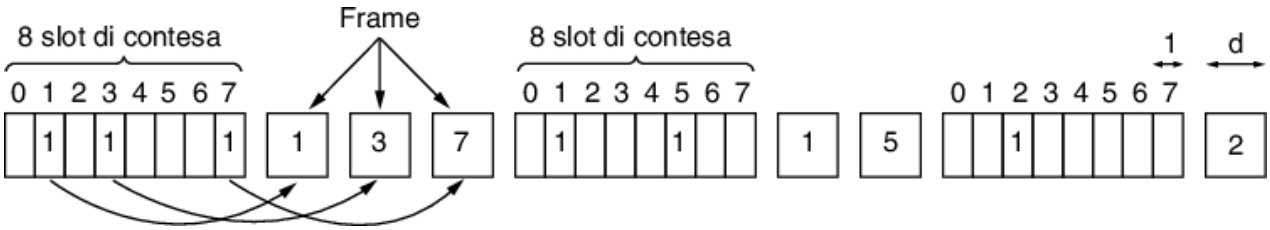


Figura 4.6 Il protocollo a mappa di bit elementare.

Poiché tutti sono d'accordo su chi sarà il successivo, non ci sarà mai alcuna collisione. Dopo che l'ultima stazione pronta ha trasmesso il proprio frame (evento che può essere facilmente monitorato da tutte le stazioni) ha inizio un altro periodo di contesa di N bit. Se diventa pronta quando il suo intervallo è ormai scaduto, la stazione deve rimanere in silenzio finché tutte le altre non abbiano avuto una possibilità e la mappa di bit non ricominci daccapo.

I protocolli di questo tipo, in cui l'intenzione di trasmettere è comunicata a tutti prima di iniziare la trasmissione vera e propria, si chiamano **protocolli a prenotazione** (*reservation protocol*) poiché riservano la proprietà del canale in anticipo e prevengono le collisioni. Analizziamo brevemente le prestazioni di questo tipo di protocollo. Per comodità, si assume come unità di misura del tempo la durata della trasmissione di un bit di contesa, con frame di dati composti da d unità temporali.

Ipotizzando un basso livello di carico, la mappa di bit si ripeterà in continuazione per via della mancanza di frame di dati. Si consideri la situazione dal punto di vista di una stazione identificata da un numero basso, per esempio 0 o 1. Generalmente, quando questa stazione sarà pronta a trasmettere lo slot corrente si troverà più o meno al centro della mappa di bit e prima di poter trasmettere la stazione dovrà attendere in media $N/2$ slot (tempo richiesto per completare la maschera di bit corrente) e poi altri N intervalli (tempo richiesto per completare quella successiva).

Le aspettative delle stazioni identificate da numeri alti sono migliori; in generale queste potranno trasmettere dopo solo mezza maschera di bit ($N/2$ slot). Le stazioni con numeri molto alti raramente devono aspettare il turno successivo. Poiché le stazioni con numeri bassi devono

attendere in media 1,5N slot e le stazioni con numeri più alti devono attendere in media 0,5N slot, la media per tutte le stazioni è pari a N slot.

È facile calcolare l'efficienza del canale a basso carico: i bit usati per il controllo sono N per frame, la quantità di dati è pari a d bit e l'efficienza è pari a $d/(d + N)$.

Se il carico è elevato, ossia se tutte le stazioni hanno sempre qualcosa da trasmettere, il periodo di contesa di N bit è distribuito proporzionalmente su N frame, quindi un solo bit per frame è usato per il controllo e l'efficienza è $d/(d + 1)$. Il ritardo medio per un frame è uguale al tempo di accodamento dentro la stazione più un tempo aggiuntivo pari a $(N – 1)d + N$ una volta che il frame ha raggiunto la prima posizione nella coda interna. Questo intervallo è il tempo che tutte le altre stazioni devono attendere perché arrivi il loro turno per spedire un frame e un'altra mappa di bit.

Protocolli token passing

L'essenza del protocollo a mappa di bit è che permette a ogni stazione di trasmettere un frame a turno in un ordine predefinito. Un altro modo di ottenere la stessa cosa è passare un breve messaggio chiamato **token** (*gettone*) da una stazione alla successiva nello stesso ordine predefinito. Il token rappresenta il permesso di inviare dati; se una stazione ha un frame in coda per la trasmissione, quando riceve il token essa può spedire quel frame prima di passare il token alla stazione successiva. Se non ha frame in coda, semplicemente passerà il token.

In un protocollo **token ring** la topologia della rete è utilizzata per definire l'ordine nel quale le stazioni inviano dati: le stazioni sono collegate l'una all'altra in un anello, passare il token alla stazione successiva consiste semplicemente nel riceverlo da una direzione e passarlo nell'altra, come si evince dalla Figura 4.7. Anche i frame sono trasmessi nella direzione del token; in questo modo circoleranno nell'anello fino a raggiungere la destinazione qualunque stazione sia. Tuttavia, per impedire che un frame circoli all'infinito (come il token), alcune stazioni devono rimuoverlo dall'anello; questa stazione può essere o la stazione che ha spedito il frame, dopo che ha fatto un giro completo, o la stazione che era la destinataria del frame.

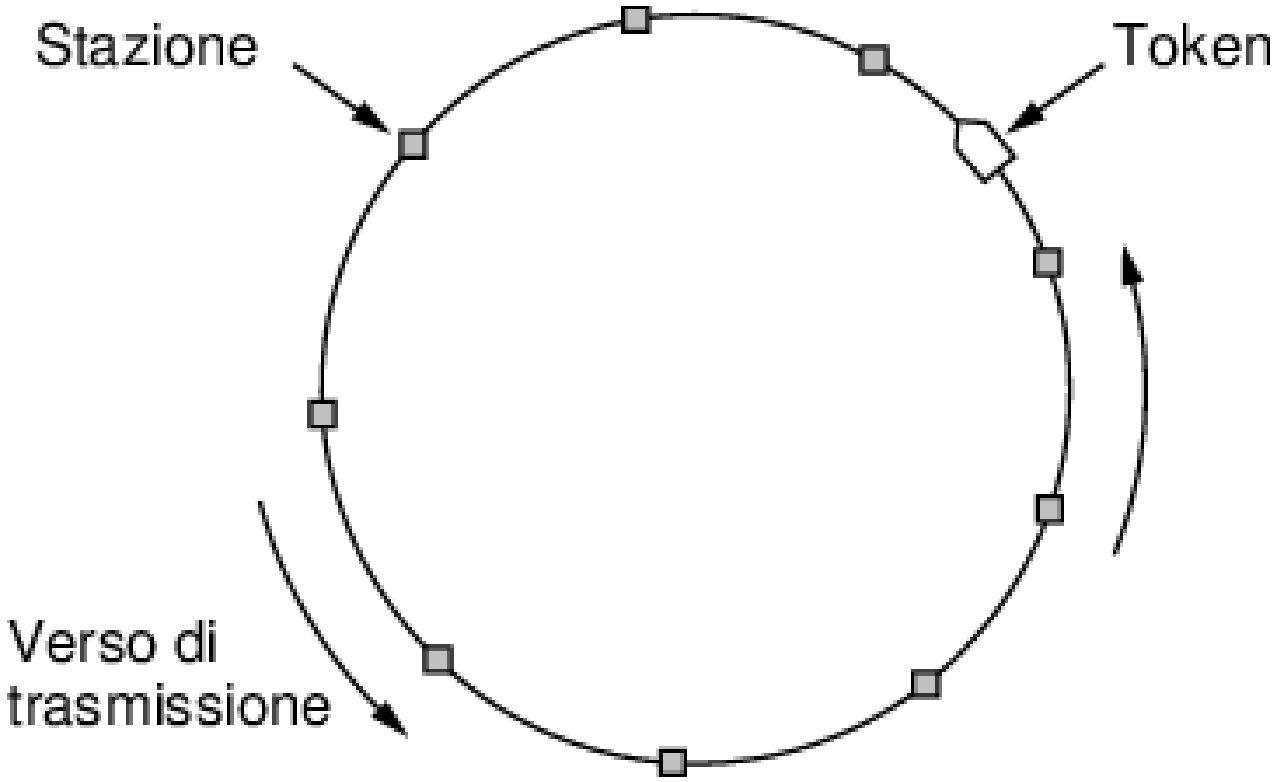


Figura 4.7 Token ring.

Si noti che per implementare il token passing non si ha bisogno di un anello fisico: il canale che collega le stazioni può essere inteso come un unico lungo bus dati, ogni stazione lo utilizza per spedire il token alla stazione successiva in una sequenza predefinita. Come nel caso precedente, il possesso del token permette a una stazione di spedire un frame. Questo protocollo è chiamato **token bus**.

Le prestazioni del token passing sono simili a quelle del protocollo a mappa di bit, anche se gli slot di contesa e i frame di un ciclo sono ora mescolati. Dopo che ha spedito il frame, ogni stazione dovrà attendere che tutte le N stazioni (inclusa se stessa) abbiano spedito il token al proprio vicino e che le altre $N - 1$ stazioni abbiano spedito un frame, se ne hanno uno da spedire. Una sottile differenza è che, poiché tutte le posizioni dell'anello sono equivalenti, non c'è differenza tra le stazioni a numerazione alta e bassa. Per quanto riguarda il token ring, ogni stazione spedisce il token solo fino alla sua stazione vicina prima che il protocollo possa fare il passo successivo. Perché il protocollo avanzi al passaggio successivo ogni token non ha bisogno di propagarsi a tutte le stazioni.

I protocolli token ring sono cresciuti come protocolli MAC di una certa consistenza. Uno dei primi protocolli token ring (chiamato "Token Ring" e standardizzato come IEEE 802.5) era popolare negli anni '80 come alternativa a Ethernet classica. Negli anni '90 un protocollo token ring molto più veloce, chiamato **FDDI** (*fiber distributed data interface*) fu surclassato da Ethernet commutata (*switched Ethernet*). Negli anni 2000 un token ring chiamato **RPR** (*resilient packet ring*) è stato definto come IEEE 802.17 per standardizzare la varietà di anelli metropolitani utilizzati dagli ISP. Ci chiediamo che cosa avrà da offrire la decade del 2010.

Protocolli binary countdown

Un problema del protocollo a mappa di bit elementare e, per estensione, dei protocolli a token passing, è dato dalla necessità di 1 bit di controllo per stazione; perciò questo lo rende poco adattabile alle reti composte da migliaia di stazioni. Un risultato migliore è ottenibile usando indirizzi binari per le stazioni e un canale che combina tra loro le trasmissioni. Una stazione intenzionata a utilizzare il canale comunica in broadcast il proprio indirizzo sotto forma di stringa binaria, partendo dal bit più significativo. Si suppone che tutti gli indirizzi abbiano la stessa lunghezza. I bit che occupano la stessa posizione negli indirizzi di stazioni diverse vengono combinati dal canale mediante l'operatore booleano OR quando vengono spediti contemporaneamente. Questo tipo di protocollo, chiamato **binary countdown** (*conto alla rovescia binario*), è stato utilizzato in Datakit (Fraser, 1987). Si presuppone implicitamente che i ritardi di trasmissione siano trascurabili, in modo che tutte le stazioni possano vedere istantaneamente i bit dichiarati.

Per evitare conflitti si deve applicare una regola di arbitraggio: la stazione rinuncia non appena si accorge che una posizione di bit di ordine elevato che nel proprio indirizzo vale 0 è stata sovrascritta da un 1. Per esempio, se le stazioni 0010, 0100, 1001 e 1010 vogliono usare il canale, in un primo istante trasmettono rispettivamente 0, 0, 1 e 1; il risultato ottenuto dal canale unendo questi valori binari mediante l'operatore OR è 1. Le stazioni 0010 e 0100 notano l'1 e capiscono che una stazione con un numero maggiore sta concorrendo per il canale, perciò si ritirano dal turno corrente. Le stazioni 1001 e 1010 invece vanno avanti.

Il bit successivo è 0 ed entrambe continuano. Il terzo bit è 1, perciò la stazione 1001 si arrende. Il vincitore alla fine sarà 1010, perché è questa la stazione che ha l'indirizzo più alto. Una volta vinta la gara la stazione può trasmettere un frame e al termine della trasmissione ricomincia un nuovo turno. Il protocollo, descritto nella Figura 4.8, ha la proprietà per cui le stazioni con il numero più alto hanno una priorità maggiore rispetto alle stazioni con il numero più basso; questa caratteristica può essere utile o svantaggiosa a seconda del contesto applicativo.

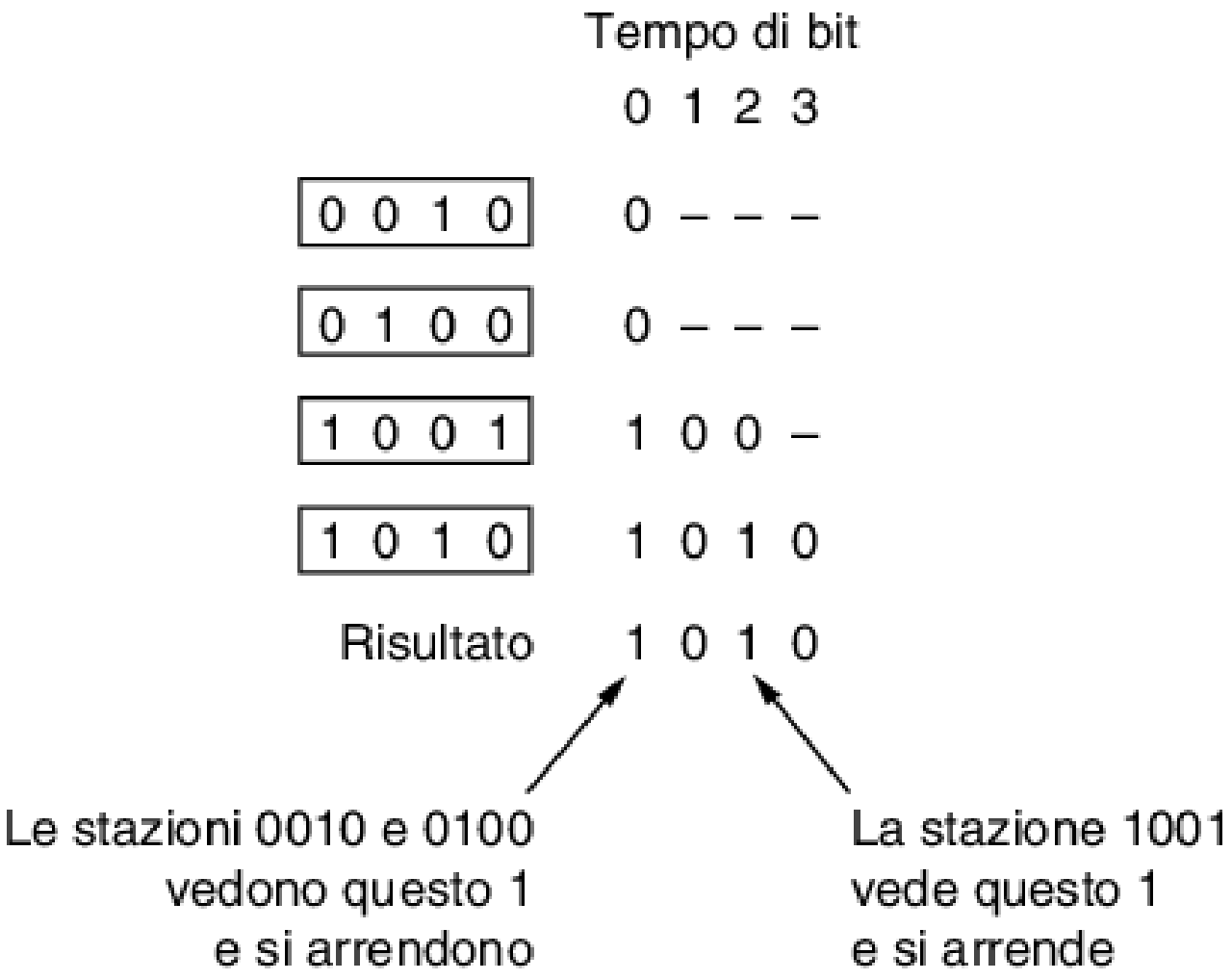


Figura 4.8 Il protocollo binary countdown. I trattini indicano i periodi di silenzio.

Con questo metodo l'efficienza del canale è pari a $d/(d + \log_2 N)$, ma se il formato del frame viene scelto con cura in modo che l'indirizzo del mittente costituisca il primo campo del frame, si recuperano anche questi $\log_2 N$ bit e l'efficienza raggiunge il 100%.

Il conteggio binario è un esempio di protocollo semplice, elegante ed efficiente in attesa di essere riscoperto. Forse un giorno troverà un nuovo campo di utilizzo.

Protocolli a contesa limitata

Fin qui sono state considerate due strategie di base adottate per acquisire il controllo del canale in una rete broadcast: il metodo della contesa, come in CSMA, e il metodo senza collisioni. Ogni strategia può essere giudicata calcolando due importanti misure di prestazioni: il ritardo in situazioni di basso carico e l'efficienza in situazioni di alto carico. Nelle situazioni di carico leggero la contesa (cioè ALOHA puro o slotted) è preferibile per il suo basso ritardo (poiché le collisioni sono rare). Al crescere del carico, però, la contesa diventa sempre più inefficiente, perché la quantità di dati necessari all'arbitraggio del canale aumenta. Con i protocolli senza collisioni si verifica esattamente il contrario: a basso carico hanno un ritardo relativamente elevato, ma al crescere del carico l'efficienza del canale migliora (poiché i bit di controllo sono fissati).

Naturalmente sarebbe bello se si potessero combinare le proprietà migliori dei due tipi di protocolli, in modo da crearne uno capace di usare il metodo della contesa per ottenere un ritardo limitato a basso carico e il metodo senza collisioni per raggiungere una buona efficienza di canale nelle situazioni a carico più elevato. Protocolli di questo tipo esistono realmente, si chiamano **protocolli a contesa limitata** (*limited contention protocol*) e concludono il nostro studio delle reti con rilevazione di portante.

I protocolli a contesa finora esaminati sono simmetrici, cioè ogni stazione tenta di acquisire il canale con una probabilità p che risulta identica per tutte. Curiosamente, le prestazioni generali del sistema qualche volta migliorano usando un protocollo che assegna a ogni stazione una probabilità diversa.

Prima di esaminare i protocolli asimmetrici è utile rivedere rapidamente le prestazioni del caso simmetrico. Si supponga che k stazioni cerchino di accedere contemporaneamente al canale. Ognuna ha una probabilità p di trasmettere durante ciascuno slot. La probabilità che una stazione si appropri del canale durante uno slot è la probabilità che una stazione trasmetta con probabilità p e che tutte le altre $k - 1$ stazioni non lo facciano, ognuna con probabilità $1 - p$; questo valore è $kp(1 - p)^{k-1}$. Si scopre così che il valore ottimo di p è $1/k$. Sostituendo $p = 1/k$, si ottiene:

$$\Pr[\text{successo con } p \text{ ottimo}] = \left[\frac{k-1}{k}\right]^{k-1} \qquad (4\text{-}4)$$

Questa probabilità è rappresentata graficamente nella Figura 4.9. Se il numero di stazioni è basso le possibilità di successo sono buone, ma non appena le stazioni diventano anche solo 5 la probabilità diminuisce fino a sfiorare il valore asintotico di $1/e$.

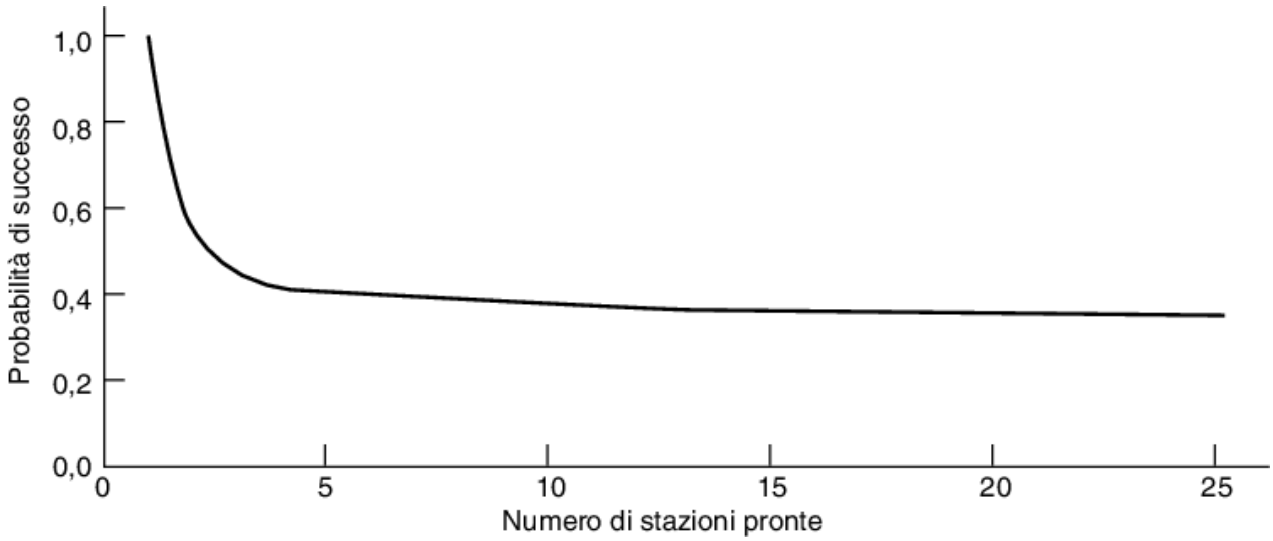


Figura 4.9 Probabilità di acquisizione di un canale simmetrico a contesa.

Osservando la Figura 4.9 diventa evidente che la probabilità che alcune stazioni acquisiscano il controllo del canale può aumentare solo diminuendo il livello di competizione. I protocolli a contesa limitata fanno precisamente questo. Prima di tutto dividono le stazioni in gruppi (non necessariamente disgiunti); solo ai membri del gruppo 0 è permesso competere per lo slot 0. Se uno di loro vince, acquisisce il controllo del canale e trasmette il frame. Se l'intervallo rimane inutilizzato o se c'è una collisione, i membri del gruppo 1 si contendono l'intervallo 1 e così via. Dividendo le stazioni in gruppi in modo appropriato è possibile ridurre il livello di contesa per ogni intervallo, in modo da operare sempre nella parte più a sinistra del grafico mostrato nella Figura 4.9.

Il trucco è ora capire come assegnare le stazioni agli slot. Prima di esaminare il caso generale è utile considerare alcuni casi particolari. A un estremo abbiamo il caso in cui ogni gruppo è composto da un unico membro. Questa ripartizione garantisce l'assenza di collisioni, poiché solo una stazione potrà contendersi un dato slot. Un protocollo di questo tipo è già stato descritto, e si tratta del protocollo binary countdown. Il successivo caso particolare è quello in cui si assegnano due stazioni per gruppo; la probabilità che entrambe tentino di trasmettere durante un intervallo è p^2; per piccoli valori di p questa quantità è trascurabile. Il caso limite è un singolo gruppo contenente tutte le stazioni (come nel caso di slotted ALOHA). Serve pertanto un meccanismo che consenta di assegnare dinamicamente le stazioni agli slot, in modo da avere molte stazioni quando il carico è limitato e poche stazioni (o addirittura una sola) quando il carico è elevato.

Protocollo adaptive tree walk

Un modo particolarmente semplice per eseguire l'allocazione si basa sullo stesso algoritmo che l'esercito americano utilizzava durante la Seconda Guerra Mondiale per verificare se i soldati erano affetti da sifilide (Dorfman, 1943). In breve, l'esercito prelevava un campione di sangue da N soldati; una parte di ogni campione veniva versata in un'unica provetta. Questa miscela di campioni era quindi analizzata per verificare la presenza di anticorpi. Se non se ne trovavano, tutti i soldati di quel gruppo erano dichiarati sani; se invece venivano trovati anticorpi, i medici preparavano due nuovi test, uno contenente un estratto dei campioni dei soldati da 1 a $N/2$ e l'altro contenente i campioni rimanenti. Il procedimento veniva ripetuto in modo ricorsivo fino a quando non venivano individuati i soldati infetti. Per la versione informatica di questo algoritmo (Capetanakis, 1979) è utile immaginare le stazioni come foglie di una struttura ad albero binario (Figura 4.10). Nel primo slot che segue una trasmissione senza collisioni, lo slot 0, tutte le stazioni possono tentare di acquisire il controllo del canale. Se una ci riesce, bene; altrimenti, in caso di collisione, durante l'intervallo 1 possono competere solo quelle stazioni che si trovano sotto il nodo 2 della struttura ad albero. Se una di queste acquisisce il controllo del canale, l'intervallo successivo è riservato alle stazioni che si trovano sotto il nodo 3. Se, d'altra parte, due o più stazioni sotto il nodo 2 vogliono trasmettere, allora durante lo slot 1 ci sarà una collisione e di conseguenza durante lo slot 2 il turno passerà al nodo 4.

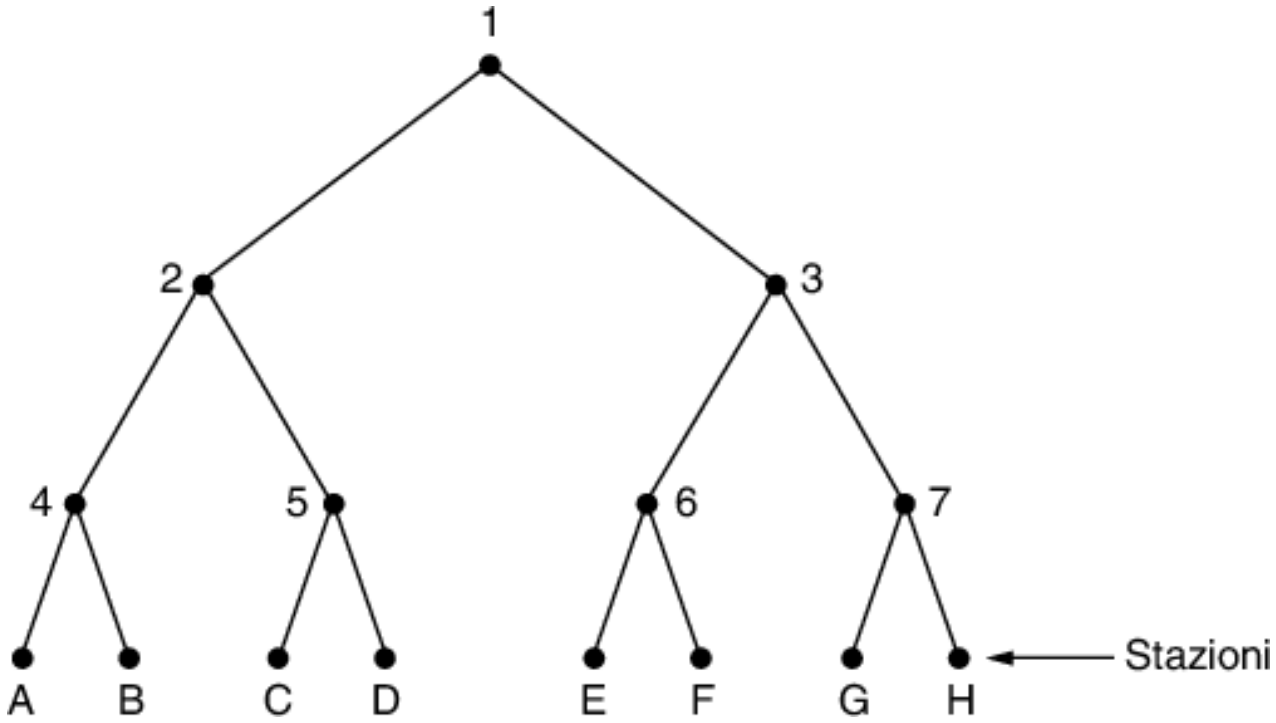


Figura 4.10 La struttura ad albero nel caso di otto stazioni.

In definitiva, in caso di collisione durante l'intervallo 0 l'algoritmo analizza l'intera struttura ad albero partendo dal basso per individuare tutte le stazioni pronte. Ogni intervallo di bit è associato a qualche particolare nodo dell'albero. In caso di collisioni, la ricerca continua in modo ricorsivo con gli elementi figli posti a sinistra e a destra del nodo. Se un intervallo di bit è libero oppure se una sola stazione trasmette durante quel periodo, la ricerca del nodo relativo a tale stazione può interrompersi perché tutte le stazioni pronte sono state individuate (se fossero più di una ci sarebbe stata una collisione).

Quando il carico sul sistema è pesante non vale affatto la pena di dedicare l'intervallo 0 al nodo 1; questo approccio, infatti, avrebbe senso solo nell'evento improbabile che una sola stazione abbia un frame da inviare. In modo analogo si potrebbe discutere del fatto che anche i nodi 2 e 3 dovrebbero essere saltati per lo stesso motivo. In termini più generali, a quale livello della struttura è giusto iniziare la ricerca? Chiaramente, più è pesante il carico, più in basso deve iniziare la ricerca. Si supponga che ogni stazione abbia una buona stima del numero di stazioni pronte, q, informazione ottenuta per esempio attraverso un meccanismo di analisi del traffico recente.

Per procedere, si supponga di numerare i livelli della struttura ad albero partendo dalla cima: con il nodo 1 nella Figura 4.10 al livello 0, il nodo 2 e il nodo 3 al livello 1 e così via. Si noti che ogni nodo al livello i ha una frazione 2^{-i} di stazioni poste sotto di esso. Se le q stazioni pronte sono distribuite uniformemente, il numero atteso di stazioni poste sotto un nodo specifico a livello i è uguale a $2^{-i}q$. Intuitivamente, ci aspettiamo che il livello ottimale da dove iniziare la ricerca sia quello che ha un numero medio di stazioni contendenti per slot pari a 1, ossia il livello dove $2^{-i}q = 1$. Risolvendo l'equazione si ottiene $i = \log_2 q$.

Sono stati scoperti numerosi miglioramenti all'algoritmo di base, discussi in dettaglio da Bertsekas e Gallager (1992). Per esempio, si consideri il caso in cui solo le stazioni *G* e *H* cercano di trasmettere. Al nodo 1 avviene una collisione, perciò l'algoritmo tenta con il 2 e scopre che è libero. Non ha senso esaminare il nodo 3, perché si può star certi che si genererà una collisione (poiché due o più stazioni sotto 1 sono pronte e nessuna di queste si trova sotto 2, entrambe devono essere sotto 3). La scansione di 3 può essere saltata per passare a 6. Quando anche questo controllo non dà alcun risultato utile, l'algoritmo salta il nodo 7 e prova *G*.

Protocolli per LAN wireless

Come si è detto nel Paragrafo 1.5.3, un sistema di computer portatili che comunicano attraverso onde radio può essere considerato come una LAN wireless. Tale LAN è un esempio di canale broadcast, ma ha anche qualcosa di diverso dalle LAN cablate, il che porta a protocolli MAC differenti. Qui esamineremo alcuni di questi protocolli, mentre nel Paragrafo 4.4 analizzeremo in dettaglio l'802.11 (*WiFi*).

Una configurazione comune per le LAN wireless è quella di un palazzo di uffici con punti di accesso (*access point*, AP) posizionati strategicamente all'interno dell'edificio. Gli AP sono collegati tra loro con rame o fibra e forniscono connettività alle stazioni collegate a essi. Se la potenza di trasmissione degli AP e dei computer è configurata in modo da avere un raggio di copertura di una decina di metri, stanze adiacenti diventeranno come una singola cella e l'intero edificio sarà come il sistema di telefonia cellulare che abbiamo studiato nel Capitolo 2, tranne per il fatto che ogni cella avrà un solo canale. Questo canale sarà condiviso da tutte le stazioni nella cella, AP incluso. Tipicamente fornisce bande dell'ordine dei Mbps, fino a 600 Mbps.

Abbiamo già fatto notare che le reti wireless non sono normalmente in grado di individuare una collisione mentre sta avvenendo. Il segnale ricevuto da una stazione sarà basso, forse un milione di volte più tenue del segnale trasmesso. Individuarlo sarebbe come cercare un'onda in mezzo al mare. Invece, per scoprire collisioni e altri errori una volta accaduti si usano degli acknowledgement.

Tra LAN wireless e cablate c'è un'altra differenza ancora più importante: una stazione su una LAN wireless potrebbe non essere in grado di trasmettere o ricevere frame dalle altre stazioni a causa del limitato raggio di copertura radio. Nelle LAN cablate, quando una stazione spedisce un frame, tutte le altre stazioni lo ricevono. La mancanza di questa proprietà nelle LAN wireless causa diverse complicazioni.

Ricorriamo all'ipotesi semplificativa che ogni trasmettitore radio abbia un certo raggio fisso, rappresentato da una regione di copertura circolare all'interno della quale un'altra stazione può sentire e ricevere le trasmissioni. È importante capire che nella realtà le regioni coperte non sono così regolari, poiché la propagazione dei segnali radio dipende dall'ambiente: muri e altri ostacoli che attenuano e riflettono il segnale possono far differire considerevolmente il raggio di copertura nelle diverse posizioni, ma un semplice modello circolare andrà bene per i nostri scopi.

Un approccio banale alle LAN wireless potrebbe essere quello di provare a usare CSMA: si ascoltano le altre trasmissioni e si trasmette solo se nessun altro sta inviando dati. Peccato che questo protocollo non sia il più adatto, perché ciò che importa è l'interferenza presso il ricevitore, non quella presso il trasmettitore. Per comprendere la natura del problema si consideri la Figura

4.11 che rappresenta quattro stazioni wireless. Per i nostri scopi non importa sapere quali sono gli AP e quali sono i computer portatili. La portata del segnale radio è tale che *A* e *B* sono uno dentro il campo dell'altro e possono potenzialmente interferire tra loro; inoltre *C* può interferire sia con *B* sia con *D,* ma non con *A*.

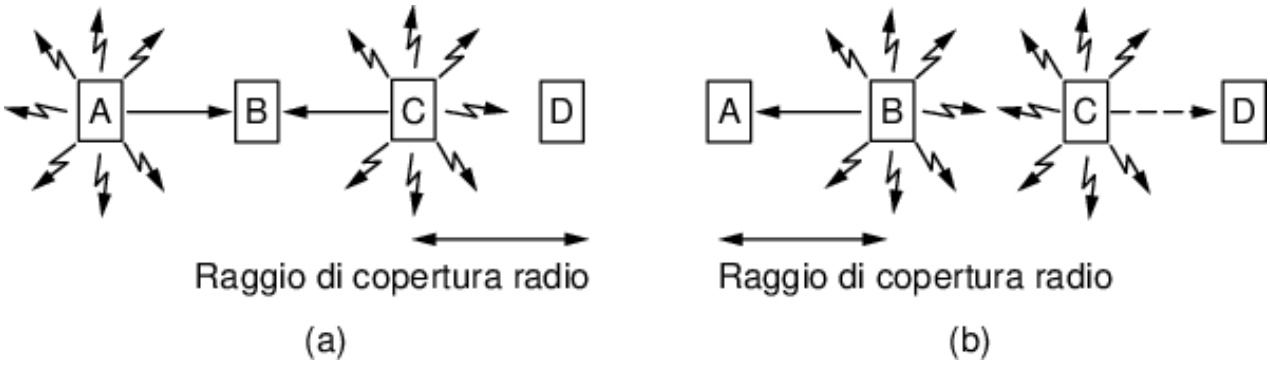


Figura 4.11 Una LAN wireless. (a) A e C sono terminali nascosti quando trasmettono a B. (b) B e C sono terminali esposti quando trasmettono ad A e D.

Innanzitutto consideriamo che cosa succede quando *A* e *C* trasmettono a *B*, come illustrato nella Figura 4.11(a). Se *A* invia un frame e *C* controlla immediatamente il canale, non sentirà *A* perché *A* è fuori portata. Quindi *C* concluderà erroneamente di poter trasmettere a *B*. Se *C* inizia a trasmettere, interferirà su *B*, spazzando via il frame in arrivo da *A*. Supponendo che non sia usato alcuno schema di tipo CDMA per fornire canali multipli, le collisioni alterano il segnale e distruggono entrambi i frame. Vogliamo un protocollo MAC che prevenga questo tipo di collisioni, perché si spreca banda. Il problema di una stazione che non è in grado di rilevare i potenziali concorrenti per il mezzo di trasmissivo a causa della distanza eccessiva è chiamato **problema del terminale nascosto** (*hidden terminal problem*).

Occupiamoci ora di una diversa situazione: *B* trasmette ad *A* nello stesso momento in cui *C* vuole trasmettere a *D*, come si vede nella Figura 4.11(b). Se *C* controlla il canale, sentirà una trasmissione ed erroneamente concluderà di non poter trasmettere a *D* (illustrato con una linea tratteggiata). Infatti, tale trasmissione potrebbe causare una cattiva ricezione solo nella zona tra *B* e *C*, dove nessuno dei destinatari è posizionato. Vogliamo un protocollo MAC che prevenga questo tipo di ritardo, perché si spreca banda. Questo problema è chiamato **problema del terminale esposto** (*exposed terminal problem*).

Il problema è che, prima di avviare una trasmissione, una stazione vorrebbe davvero sapere se c'è attività intorno al ricevitore; CSMA permette solo di scoprire se c'è attività intorno alla stazione trasmittente analizzando la portante. Nel caso di un cavo, tutti i segnali si propagano a tutte le stazioni, perciò questa distinzione non esiste. Al contrario, in un sistema basato su onde radio a portata limitata possono aver luogo contemporaneamente più trasmissioni, se tutte hanno destinazioni diverse e se queste destinazioni sono fuori portata l'una dell'altra. Vogliamo che questa

concorrenza tra comunicazioni sia possibile quando la cella aumenta di dimensione, allo stesso modo in cui i partecipanti a una festa per parlare non devono aspettare che tutti i presenti nella stanza siano in silenzio; conversazioni multiple possono avere luogo in contemporanea in una grande stanza fintanto che non sono dirette alla stessa persona.

Uno dei primi e più influenti protocolli che affrontava questi problemi per le LAN wireless è stato **MACA** (*multiple access with collision avoidance*) (Karn, 1990). L'idea di base è semplice: il trasmittente incita il ricevente a trasmettere un piccolo frame, in modo che le stazioni che si trovano nelle vicinanze, rilevando questa trasmissione, evitino di inviare dati durante la trasmissione imminente del frame di dati più grande. Questa tecnica è utilizzata al posto della rilevazione della portante.

MACA è rappresentato nella Figura 4.12. Vediamo ora come *A* invia un frame a *B*. *A* invia prima di tutto un frame **RTS** (*request to send*) a *B,* come mostrato nella Figura 4.12(a). Questo piccolo frame (30 byte) contiene la lunghezza del frame di dati che verrà inviato successivamente. *B* risponde con un frame **CTS** (*clear to send*), come mostrato nella Figura 4.12(b). Il frame CTS contiene la lunghezza dei dati copiata dal frame RTS. Non appena riceve il frame CTS, *A* inizia la trasmissione.

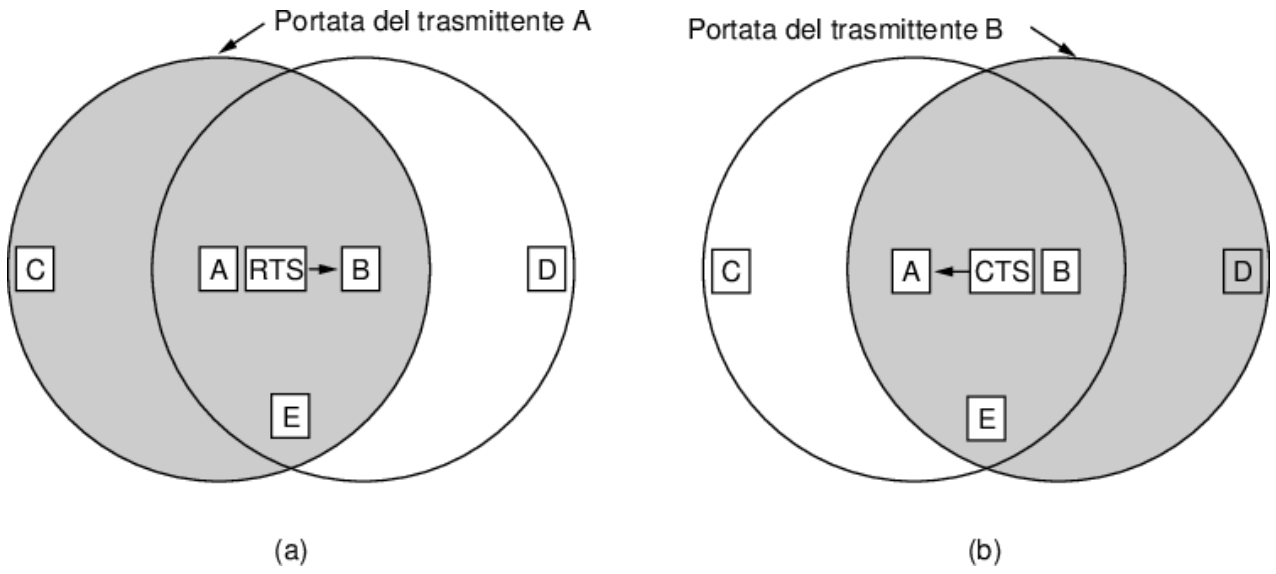


Figura 4.12 Il protocollo MACA. (a) A invia un frame RTS a B. (b) B risponde ad A con un frame CTS.

In che modo reagiscono le altre stazioni? Tutte le stazioni che ricevono il frame RTS si trovano chiaramente vicine ad *A* e devono rimanere in silenzio per un tempo che consenta al frame CTS di raggiungere *A* senza causare conflitti. Le stazioni che ricevono il frame CTS sono chiaramente vicine a *B* e devono rimanere in silenzio durante la trasmissione dei dati la cui lunghezza può essere determinata esaminando il frame CTS.

Nella Figura 4.12*C* si trova nel raggio di copertura di *A* ma non in quello di *B*, perciò riceve il frame RTS inviato da *A*, ma non il frame CTS trasmesso da *B.* Fintanto che non interferisce con il frame CTS, *C* è libero di trasmettere durante l'invio del frame di dati. Al contrario, *D* si trova nel raggio d'azione di *B*, ma non in quello di *A*; *D* non riceve il frame RTS, ma rileva quello CTS. Poiché rileva il frame CTS, *D* sa che si trova vicino a una stazione che sta per ricevere un frame, perciò rimanda qualunque invio fintanto che la trasmissione di quel frame non sia stata terminata. La stazione *E* rileva entrambi i messaggi di controllo; come *D,* deve rimanere in silenzio fino al termine della trasmissione del frame di dati.

Nonostante queste precauzioni possono ancora verificarsi collisioni. Per esempio, *B* e *C* potrebbero inviare contemporaneamente due frame RTS ad *A*. Questi frame entrerebbero in collisione e andrebbero perduti; in caso di collisione, un trasmittente che non ha avuto successo (ossia un trasmittente che non riceve un frame CTS entro un intervallo di tempo previsto) aspetta per un periodo di tempo casuale prima di ripetere l'operazione.

4.3 Ethernet

Conclusa la discussione generale sui protocolli utilizzati per allocare il canale è giunto il momento di scoprire in che modo questi principi astratti si applicano ai sistemi reali. Molte delle architetture per aree di rete personali, locali e metropolitane sono state standardizzate sotto il nome di IEEE 802. Alcune sono sopravvissute, ma molte no, come abbiamo visto nella Figura 1.38. Qualcuno che crede nella reincarnazione pensa che Charles Darwin sia tornato come membro della *IEEE standards association* per estirpare i non adatti. I più importanti tra i sopravvissuti sono 802.3 (Ethernet) e 802.11 (wireless LAN). Bluetooth (wireless PAN) è ampiamente diffuso, ma ora è stato standardizzato all'esterno di 802.15. Di 802.16 (wireless MAN) è troppo presto per dire qualcosa; consultate la successiva edizione di questo libro per scoprirlo.

Cominceremo il nostro studio dei sistemi reali con Ethernet, probabilmente il tipo di rete di calcolatori più onnipresente al mondo. Esistono due tipi di Ethernet:

- **Ethernet classica** che risolve il problema dell'accesso multiplo usando le tecniche studiate in questo capitolo;
- **Ethernet commutata** (*switched Ethernet*), in cui dispositivi chiamati commutatori (*switch*) sono usati per connettere diversi computer.

È importante notare che, sebbene siano entrambe identificate come Ethernet, sono abbastanza diverse tra loro. Ethernet classica è la forma originale e permette dei tassi di trasmissione dati tra i 3 e i 10 Mbps. Ethernet commutata è l'evoluzione di Ethernet e raggiunge velocità di 100, 1000 e 10.000 Mbps, in varianti chiamate rispettivamente fast Ethernet, gigabit Ethernet e 10-gigabit Ethernet. In pratica oggi viene utilizzata principalmente switched Ethernet.

Discuteremo queste varianti storiche di Ethernet in ordine cronologico illustrandone lo sviluppo. Poiché Ethernet e IEEE 802.3 sono identiche a eccezione di qualche differenza secondaria (di cui parleremo bevemente), molti usano i termini "Ethernet" e "IEEE 802.3" come sinonimi. Lo faremo anche noi. Per maggiori informazioni su Ethernet si veda Spurgeon (2000).

Livello fisico di Ethernet classica

La storia di Ethernet inizia all'incirca nello stesso periodo di ALOHA, quando uno studente di nome Bob Metcalfe si laureò al M.I.T. di Boston e poi si spostò per ottenere il dottorato di ricerca a Harvard. Durante i suoi studi venne a conoscenza del lavoro di Abramson. Ne fu così interessato che dopo il dottorato a Harvard, decise di passare l'estate alle Hawaii per lavorare con Abramson prima di iniziare a lavorare allo Xerox PARC (*Palo Alto research center*). Quando arrivò a PARC, vide che i ricercatori avevano progettato e costruito quelli che più tardi sarebbero stati chiamati personal computer. Ma le macchine erano isolate. Usando le sue conoscenze sul lavoro di Abramson, Bob Metcalfe e il collega David Boggs progettarono e implementarono la prima LAN (Metcalfe e Boggs, 1976), nella quale veniva usato un singolo cavo coassiale lungo e spesso e andava a 3 Mbps.

Chiamarono il sistema **Ethernet** ispirandosi a *luminiferous ether*, attraverso il quale un tempo si pensava si propagassero le radiazioni elettromagnetiche. Quando, nel XIX secolo, il fisico inglese James Clerk Maxwell scoprì che la radiazione elettromagnetica può essere descritta tramite l'equazione di un'onda, alcuni scienziati ipotizzarono che lo spazio dovesse essere pieno di un qualche mezzo etereo nel quale le radiazioni si propagavano. Solo dopo il famoso esperimento Michelson-Morley del 1887 i fisici scoprirono che le radiazioni potevano propagarsi anche nel vuoto.

L'Ethernet di Xerox ebbe un successo tale che DEC, Intel e Xerox proposero uno standard nel 1978 per una Ethernet da 10 Mbps, chiamata **standard DIX**. Con un minimo cambiamento, nel 1983 lo standard DIX divenne lo standard IEEE 802.3. Sfortunatamente, Xerox aveva già esperienza nel creare invenzioni che avrebbero influenzato il futuro (come il personal computer) fallendo nella loro commercializzazione, una storia raccontata in *Fumbling the Future* (Smith e Alexander, 1988). Quando la Xerox mostrò poco interesse per Ethernet limitandosi a standardizzarla, Metcalfe costituì la propria compagnia, 3Com, per vendere adattatori Ethernet per PC; ne vendette svariati milioni.

Ethernet classica si insinuava attraverso un edificio sotto forma di un singolo lungo cavo al quale tutti i computer erano collegati. Questa architettura è mostrata nella Figura 4.13. Il primo tipo, comunemente chiamata **thick Ethernet** (*Ethernet spessa*), ricordava un tubo giallo da giardinaggio, con segnali ogni 2,5 metri che mostravano dove collegare i computer. (Lo standard 802.3 non richiede che il tubo sia davvero giallo, ma lo raccomanda.) Fu seguita da **thin Ethernet** (*Ethernet sottile*), più flessibile e che utilizzava collettori standard BNC. Thin Ethernet era più economica e più facile da installare, ma poteva estendersi per soli 185 metri per segmento (invece dei 500 metri di thick Ethernet), ognuno dei quali poteva gestire solo 30 macchine (invece di 100).

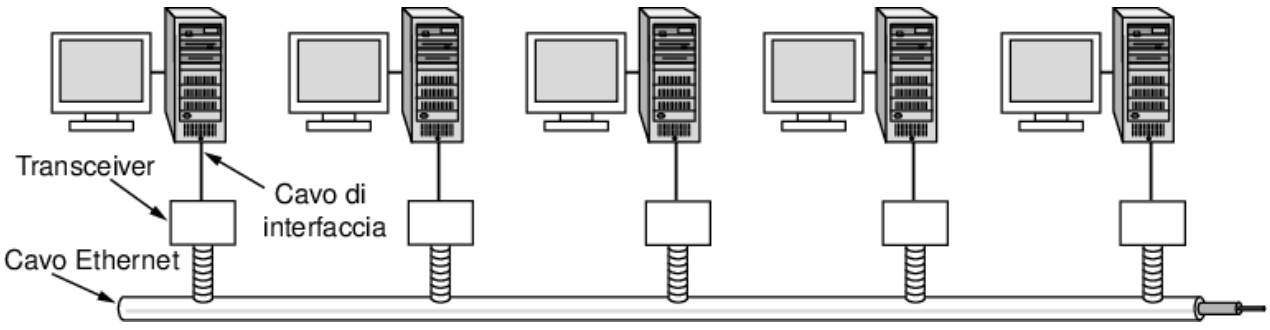


Figura 4.13 Architettura di Ethernet classica.

Ogni variante di Ethernet ha una lunghezza massima del cavo per ogni segmento (cioè la lunghezza su cui non si effettua una amplificazione) su cui il segnale si propaga. Per permettere di costruire reti più grandi, più cavi possono essere connessi attraverso **repeater** (*ripetitori*). Un

repeater è un dispositivo di livello fisico che riceve, amplifica (cioè rigenera) e ritrasmette segnali in entrambe le direzioni. Per quanto riguarda il software, una serie di segmenti connessi da ripetitori non sono diversi da un singolo cavo (eccezion fatta per i piccoli ritardi causati dai ripetitori).

Su ognuno di quei cavi le informazioni sono spedite usando la codifica Manchester illustrata nel Paragrafo 2.5. Una Ethernet può contenere diversi segmenti di cavo e diversi repeater, ma non ci possono essere due transceiver[2] a più di 2,5 km di distanza e nessun percorso tra due transceiver può attraversare più di quattro ripetitori. Il motivo di queste restrizioni è che il protocollo MAC, che vedremo in seguito, in questo modo lavora correttamente.

Protocollo del sottolivello MAC di Ethernet classica

Il formato usato per spedire i frame è mostrato nella Figura 4.14. Per prima cosa c'è un campo *Preamble* (*preambolo*) di 8 byte, ognuno dei quali contiene la sequenza di bit 0101010, con l'eccezione dell'ultimo byte, in cui gli ultimi 2 bit sono impostati a 11; questo ultimo byte è chiamato *delimitatore di inizio frame* (*start of frame delimiter, SoF*) per 802.3. La codifica Manchester di questa sequenza produce un'onda quadra di 10 MHz per 6,4 ms che permette al clock del ricevente di sincronizzarsi con quello del trasmittente. Gli ultimi due bit a 1 comunicano al ricevente che il resto del frame sta per cominciare.

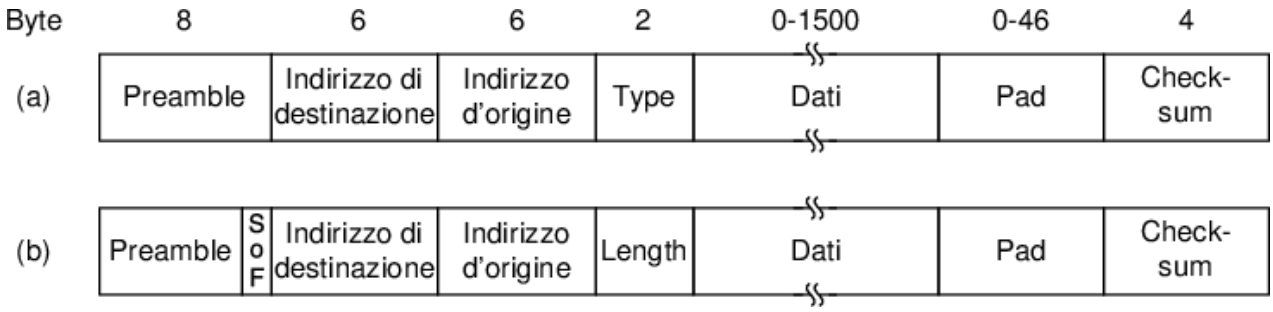


Figura 4.14 Formati di frame. (a) Ethernet DIX. (b) IEEE 802.3.

Di seguito vengono due indirizzi: uno per la destinazione e uno per la sorgente. Ognuno è lungo 6 byte; il primo bit trasmesso dell'indirizzo di destinzione è uno 0 per indirizzi odinari e 1 per un gruppo di indirizzi. Gli indirizzi di gruppo permettono a molte stazioni di rimanere in ascolto usando un singolo indirizzo. Quando un frame viene inviato a un indirizzo di gruppo, tutte le stazioni di quel gruppo lo ricevono. La trasmissione diretta a un gruppo di stazioni è definita **multicast**. L'indirizzo composto da tutti bit 1 è riservato per le trasmissioni **broadcast**. Un frame che contiene tutti 1 nel campo "Indirizzo di destinazione" è ricevuto da tutte le stazioni della rete. La trasmissione multicast è più selettiva, ma richiede una gestione del gruppo per definire quali stazioni ne fanno parte. Viceversa, la trasmissione broadcast non fa alcuna distinzione tra le stazioni, quindi non richiede alcuna gestione del gruppo.

Una caratteristica interessante degli indirizzi della stazione sorgente è che sono tutti unici al mondo, assegnati centralmente da IEEE per assicurare che non esistano due stazioni al mondo con lo stesso indirizzo. L'idea è che ogni stazione si possa rivolgere a un'altra solo dando il giusto numero di 48 bit. Per fare ciò, i primi 3 byte del campo d'indirizzo sono usati per un **OUI** (*organizationally unique identifier*). I valori per questo campo sono assegnati da IEEE e indicano un produttore; ai produttori sono assegnati blocchi di 2^{24} indirizzi. Il produttore assegna gli ultimi 3 byte dell'indirizzo e programma l'intero indirizzo all'interno di NIC prima che sia venduto.

Il campo successivo è *Type* (*tipo*) o *Length* (*lunghezza*), a seconda che il frame sia Ethernet o IEEE 802.3. Ethernet usa un campo *Type* per indicare al ricevente che cosa fare col frame. Più protocolli a livello di rete potrebbero essere utilizzati contemporaneamente sulla stessa macchina, quindi quando un frame Ethernet arriva, il sistema operativo deve capire a quale di questi passare il frame. Il campo *Type* specifica a quale processo bisogna associare il frame; per esempio, il codice 0x0800 significa che i dati contengono un pacchetto IPv4.

IEEE 802.3, nella sua saggezza, decise che questo campo avrebbe contenuto la lunghezza del frame, poiché la dimensione di Ethernet era determinata guardando dentro il dato (una violazione delle regole, se mai ve ne fu una). Certo, ciò voleva dire che non c'era modo per il ricevente di capire che cosa fare con un frame in arrivo. Quel problema fu gestito con l'aggiunta di un'altra intestazione per il protocollo **LLC** (*logical link control*) all'interno dei dati; questo utilizza 8 byte per esprimere i 2 byte dell'informazione del tipo di protocollo.

Sfortunatamente, nel momento in cui 802.3 fu reso pubblico, era già in uso parecchio hardware e software per Ethernet DIX, per cui pochi produttori e utenti furono entusiasti di ricodificare i campi *Type* e *Length*. Nel 1997 IEEE gettò la spugna dichiarando che entrambi i modi andavano bene. Fortunatamente, tutti i campi *Type* in uso prima del 1997 avevano valori più grandi di 1500, già allora largamente adottato come dimensione massima dei dati. Ora la regola è che ogni numero minore o uguale a 0x600 (1536) può essere interpretato come *Length* e ogni numero maggiore di 0x600 può essere considerato come *Type*. Ora IEEE può continuare ad affermare che tutti stanno usando il suo standard e chiunque può continuare a fare quello che stava già facendo (senza preoccuparsi di LLC) senza sentirsi in colpa per questo.

Dopo questo campo arriva il campo *Data* (dati), lungo fino a 1.500 byte. Questo limite, scelto abbastanza arbitrariamente al tempo della creazione dello standard Ethernet, fu scolpito nella pietra; si basava più che altro sul fatto che un transceiver aveva bisogno di una RAM che gli consentisse di conservare l'intero frame: nel 1978 la RAM era costosa. Un limite più alto avrebbe significato più RAM e quindi apparecchi più cari.

Il frame non ha solo una lunghezza massima, ma anche una lunghezza minima; anche se qualche volta può tornare utile, un campo *Data* lungo 0 byte causa un problema. Quando rileva una collisione, un transceiver tronca il frame corrente: ciò significa che sul cavo compaiono continuamente bit sparsi e pezzi di frame. Per aiutare a distinguere i frame validi dalla spazzatura, Ethernet richiede che i frame validi siano lunghi almeno 64 byte dall'indirizzo di destinazione al checksum inclusi. Se la parte occupata dai dati è lunga meno di 46 byte, il campo *Pad* (riempimento) viene utilizzato per riempire il frame fino alla dimensione minima.

Imporre una lunghezza minima serve anche per un altro (e più importante) motivo: impedire a una stazione di completare la trasmissione di un frame breve prima che il primo bit abbia raggiunto la fine del cavo, dove potrebbe collidere con un altro frame. Questo problema è rappresentato graficamente nella Figura 4.15. Al tempo 0, la stazione *A* che si trova a un'estremità della rete invia un frame. Sia t il tempo di propagazione impiegato dal frame per raggiungere l'altra estremità della rete. Appena prima che il frame raggiunga l'altra estremità (cioè al tempo t – e), la stazione più distante *B* inizia a trasmettere. Quando si accorge di ricevere più potenza di quella emessa, *B* capisce che è avvenuta una collisione, perciò interrompe la sua trasmissione e genera un picco di rumore di 48 bit per avvisare tutte le altre stazioni. In altre parole, ostruisce il mezzo di trasmissione per essere sicura che il trasmettitore rilevi la collisione. All'istante 2t anche il trasmettitore vede il picco di rumore e interrompe la sua trasmissione, aspettando quindi per un periodo di tempo casuale prima di ritentare.

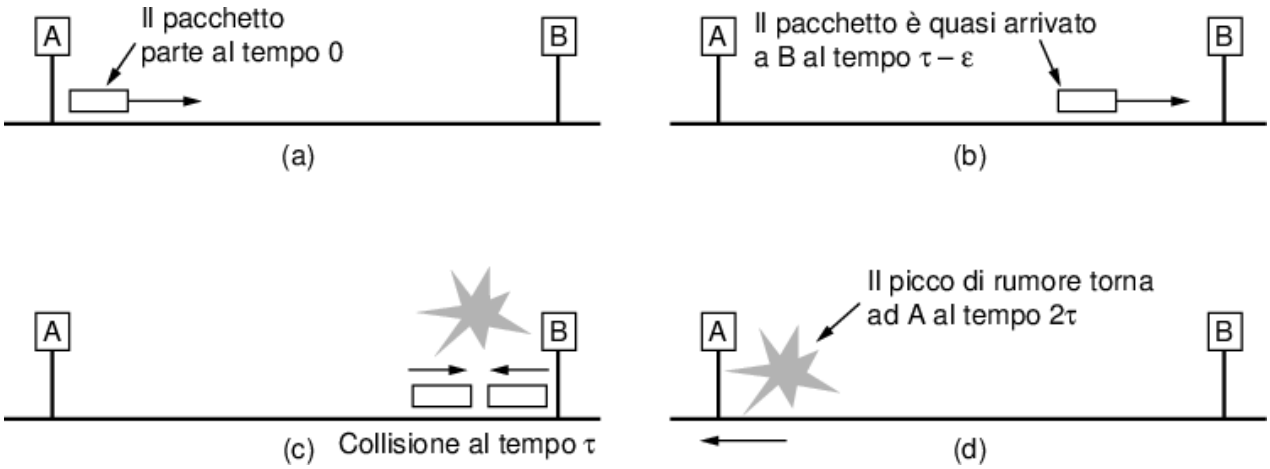


Figura 4.15 Il rilevamento della collisione può richiedere fino a 2t.

Se una stazione tenta di trasmettere un frame molto corto, potrebbe succedere che avvenga una collisione, ma la trasmissione si concluderebbe prima del ritorno dal picco di rumore (ossia, prima dell'istante 2t). In questo caso il trasmettitore concluderà erroneamente che il frame è stato inviato con successo. Per evitare che si presenti una situazione di questo tipo, tutti i frame devono impiegare più di 2t per essere trasmessi; in tal modo la trasmissione è ancora in corso quando il picco di rumore torna al trasmettitore. Per una LAN a 10 Mbps che può essere lunga al massimo 2500 metri e può contenere non più di quattro ripetitori (secondo le specifiche 802.3), il viaggio di andata e ritorno (considerando anche il tempo di propagazione attraverso i quattro ripetitori) dura circa 50 ms nel peggiore dei casi. Quindi, per essere trasmesso, il più breve frame permesso deve utilizzare almeno questo tempo. A 10 Mbps, un bit impiega 100 ns a essere trasmesso, quindi 500 bit è il frame più piccolo di cui si garantisce il funzionamento; per aggiungere un margine di sicurezza, questo numero è arrotondato a 512 bit o 64 byte.

Il campo finale è il *Checksum*; si tratta di un CRC a 32 bit del tipo analizzato nel Paragrafo 3.2. Infatti è definito esattamente dal generatore polinominale che abbiamo visto usare per PPP, ADSL e anche per altri collegamenti. Questo CRC è un codice di rilevazione di errore che si usa per determinare se i bit del frame siano stati ricevuti correttamente. Esso rivela unicamente gli errori, facendo scartare il frame in caso positivo.

CSMA/CD con backoff esponenziale binario

Ethernet classica usa l'algoritmo CSMA/CD 1-persistente che abbiamo studiato nel Paragrafo 4.2. Con ciò si intende che le stazioni controllano il canale quando hanno un frame da spedire e lo spediscono non appena il canale risulta libero; monitorano il canale per trovare collisioni mentre spediscono. Se c'è una collisione, interrompono la trasmissione con un breve messaggio caotico per poi ritrasmettere dopo un periodo di tempo casuale.

Vediamo ora come si determina l'intervallo di tempo casuale quando si verifica una collisione. Il modello è sempre quello rappresentato nella Figura 4.5. Dopo una collisione, il tempo viene diviso in intervalli discreti la cui lunghezza è uguale al tempo di propagazione di andata e ritorno sul mezzo di trasmissione nel caso peggiore (2t). Per contenere il percorso più lungo supportato da Ethernet, l'intervallo temporale è stato impostato a intervalli di 512 bit o di 51,2 ms.

Dopo la prima collisione, ogni stazione aspetta casualmente 0 o 1 slot prima di ritentare. Se due stazioni collidono e ognuna sceglie lo stesso numero casuale, la collisione si ripeterà. Dopo la seconda collisione, ogni stazione sceglie 0, 1, 2 o 3 a caso e rimane in attesa di quel numero di slot. Se avviene una terza collisione (la probabilità che questo accada è pari a 0,25) la volta successiva il numero di intervalli di attesa è scelto a caso tra 0 e $2^3 - 1$.

In generale, dopo i collisioni, viene scelto un numero casuale compreso tra 0 e $2^i - 1$ e si salta quel numero di slot; dopo dieci collisioni il tetto massimo di scelta rimane bloccato a 1023 slot. Dopo 16 collisioni il controller della scheda di rete getta la spugna e comunica al computer un errore; ulteriori operazioni di ripristino sono demandate ai livelli più alti.

Questo algoritmo, chiamato **backoff esponenziale binario** (*binary exponential backoff*), è stato scelto per adattarsi dinamicamente al numero di stazioni che tentano di trasmettere. Se l'intervallo casuale per tutte le collisioni fosse 1023, la possibilità che due stazioni collidano una seconda volta sarebbe trascurabile, ma l'attesa media dopo una collisione durerebbe centinaia di slot e il ritardo introdotto sarebbe rilevante. D'altro canto, si supponga che ogni stazione aspetti sempre per 0 o 1 slot; allora, se 100 stazioni tentassero di trasmettere contemporaneamente, le collisioni si ripeterebbero fino a quando 99 di esse non scegliessero l'intervallo 1 e la rimanente non scegliesse lo 0. Questo potrebbe richiedere anni. Poiché l'intervallo di scelta casuale cresce esponenzialmente con il numero di collisioni consecutive, l'algoritmo assicura un basso ritardo quando poche stazioni collidono e garantisce la risoluzione della collisione in un intervallo di tempo ragionevole quando la collisione coinvolge molte stazioni. Troncando il backoff a 1023 si impedisce al confine di crescere troppo.

Se non ci sono collisioni, il mittente può supporre che la spedizione del frame sia andata a buon fine, in quanto né CSMA/CD né Ethernet forniscono una conferma della consegna. Questa scelta è appropriata per canali in rame o fibra ottica che hanno una piccola percentuale d'errore. Ogni

errore che avviene deve essere individuato dal CRC e recuperato dai livelli superiori. Per i canali wireless che presentano più errori vedremo che vengono utilizzati degli acknowledgement.

Prestazioni di Ethernet

Esaminiamo ora brevemente le prestazioni di Ethernet classica in condizioni di carico pesante e costante, cioè con k stazioni sempre pronte a trasmettere. Un'analisi rigorosa dell'algoritmo di backoff esponenziale binario risulterebbe molto complessa; seguiremo invece il ragionamento di Metcalfe e Boggs (1976) e supporremo una probabilità di ritrasmissione costante in ogni slot. Se ogni stazione trasmette durante un'intervallo di contesa con probabilità p, allora la probabilità A che qualche stazione acquisisca il controllo del canale in quell'intervallo è:

$$A = kp(1 - p)^{k-1} \quad (4\text{-}5)$$

Il valore di A è massimo quando $p = 1/k$, con $A \rightarrow 1/e$ e $k \rightarrow {}^{+}\infty$. La probabilità che l'intervallo di contesa abbia esattamente j slot al suo interno è $A(1 - A)^{j-1}$, per cui il numero medio di slot per contesa è dato da:

$$\sum_{j=0}^{+\infty} jA(1 - A)^{j-1} = \frac{1}{A}$$

Poiché ogni slot ha una durata di 2t, l'intervallo di contesa medio w dura $2t/A$. Supponendo p ottimale, il numero medio di slot di contesa non è mai maggiore di e, perciò w è al più $2te \approx 5{,}4t$.

Se il frame medio impiega P secondi per essere trasmesso, quando molte stazioni hanno frame da inviare, si ha:

$$\text{Efficienza del canale} = \frac{P}{P + 2\tau/A} \quad (4\text{-}6)$$

L'equazione mostra il rapporto che lega la distanza massima tra coppie di stazioni lungo il cavo alle prestazioni della rete: più il cavo è lungo, più tempo dura l'intervallo di contesa. Questa osservazione spiega come mai lo standard Ethernet specifica una lunghezza massima per i cavi.

È istruttivo formulare l'Equazione (4-6) in termini di lunghezza di frame, F, banda di rete, B, lunghezza del cavo, L, e velocità di propagazione del segnale c (c = velocità della luce) per il caso ottimale di e slot di contesa per frame. Con $P = F/B$, l'Equazione (4-6) diventa:

$$\text{Efficienza del canale} = \frac{1}{1 + 2BLe/cF} \quad (4\text{-}7)$$

Quando il secondo termine nel denominatore è grande, l'efficienza della rete è bassa. Per la precisione, aumentando la banda della rete o la distanza (ossia il prodotto BL) si riduce l'efficienza per una data dimensione di frame. Sfortunatamente, gran parte della ricerca sull'hardware di rete punta proprio ad aumentare questo prodotto. Gli utenti vogliono ampiezze di banda più grandi su lunghe distanze (MAN su fibra ottica, per esempio). Ethernet classica implementata in questo modo non è ancora il sistema migliore per simili applicazioni; nel paragrafo successivo ne vedremo altre implementazioni.

La Figura 4.16 mostra l'efficienza del canale calcolata mediante l'Equazione (4-7) in base al numero di stazioni pronte, per 2t = 51,2 ms e una velocità di 10 Mbps. Con slot ampi 64 byte, non sorprende che i frame grandi 64 byte non siano efficienti. D'altro canto, con frame da 1024 byte e un valore asintotico di *e* intervalli di 64 byte per intervallo di contesa, il periodo di contesa è lungo 174 byte e l'efficienza è dell'85%. Questo risultato è molto migliore rispetto al 37% di efficienza di slotted ALOHA.

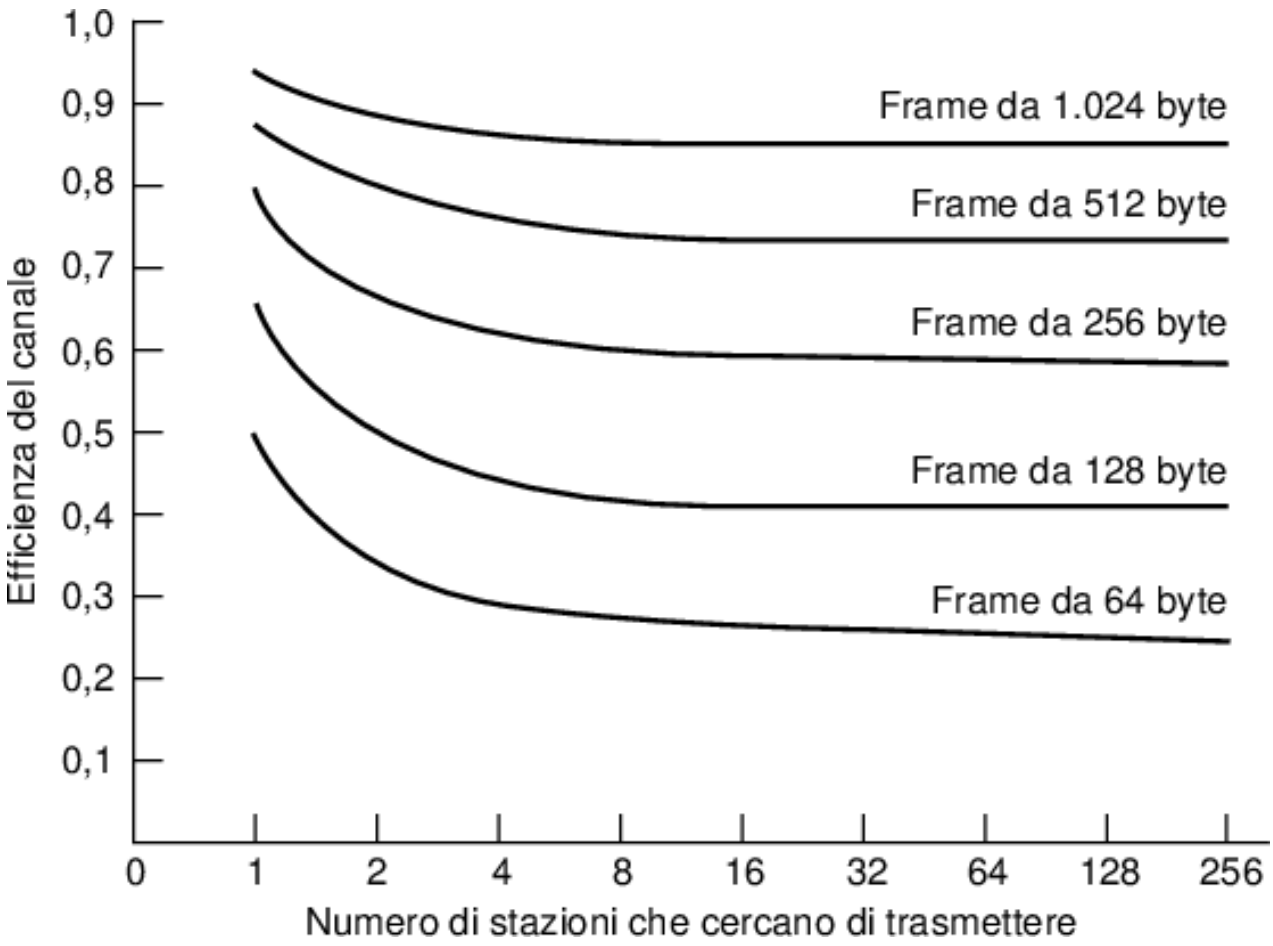


Figura 4.16 Efficienza di Ethernet a 10 Mbps con slot da 64 byte.

Vale probabilmente la pena di ricordare la grande attività teorica intorno alle analisi delle prestazioni di Ethernet (e di altre reti). Molti dei risultati devono essere presi con un pizzico (o meglio ancora, una tonnellata) di sale in zucca per due ragioni. Primo, virtualmente tutti i lavori teorici ipotizzano un traffico di distribuzione Poissoniana; non appena i ricercatori iniziarono ad analizzare dati reali, fu subito chiaro che raramente il traffico seguiva una Poissoniana; tendeva invece a essere autosimilare o ad avere dei picchi su una gamma di scale temporali (Paxson e Floyd, 1995; Leland et al., 1994). Ciò significa che fare la media su un lungo periodo di tempo non renderà il traffico più uniforme. Oltre a utilizzare modelli discutibili, molte delle analisi si concentrarono su "interessanti" casi di carico anormalmente elevato. Boggs et al. (1988) dimostrarono che Ethernet lavorava bene nella pratica, anche con carichi di traffico moderatamente elevati.

Ethernet commutata

Ethernet si emancipò presto dall'architettura del lungo cavo di Ethernet classica. I problemi associati al reperimento di guasti o connettori lenti la guidarono verso un diverso tipo di cablaggio, nel quale ogni stazione aveva un cavo dedicato che andava verso un **hub** centrale. Un hub connette elettricamente tutti i cavi collegati come se fossero saldati assieme. Questa configurazione è mostrata nella Figura 4.17(a).

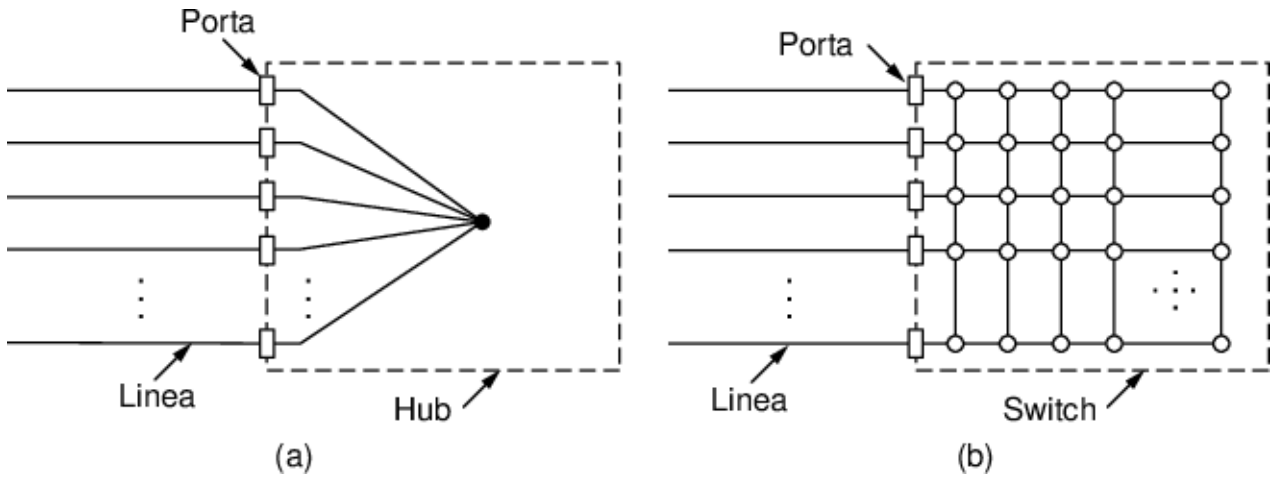


Figura 4.17 (a) Hub. (b) Switch.

I fili erano doppini telefonici, dato che molti uffici erano già cablati in questo modo e le scorte erano abbondanti. Questo riutilizzo fu una scelta vincente, ma ridusse la lunghezza massima di cavo che partiva dall'hub a 100 metri (200 metri se venivano utilizzati doppini di Categoria 5 ad alta qualità). Aggiungere o rimuovere una stazione è più semplice con questa configurazione e i cavi guasti possono essere individuati facilmente. Con i vantaggi della possibilità di riutilizzo di cablaggi esistenti e della facile manutenzione, gli hub con doppini telefonici divennero la forma dominante di Ethernet.

Tuttavia, poiché sono logicamente equivalenti a un singolo lungo cavo di Ethernet classica, gli hub non aumentano la capacità. Con la continua aggiunta di stazioni, ognuna di esse ottiene una porzione via via più piccola della capacità totale, saturando col tempo la LAN. Una via d'uscita è orientarsi verso velocità più elevate, cioè da 10 Mbps a 100 Mbps, 1 Gbps o anche oltre. Con l'aumento del traffico generato dalle applicazioni multimediali e di server sempre più potenti, tuttavia, anche una Ethernet da 1 Gbps può saturarsi.

Fortunatamente esiste un modo alternativo di gestire il carico crescente: si tratta di Ethernet commutata (switched Ethernet). Il cuore di questo sistema è uno **switch** (*commutatore*) che contiene una scheda hardware di collegamento tra le interfacce di rete o **backplane** ad alta velocità che connette tutte le porte, come illustrato nella Figura 4.17(b). Dall'esterno uno switch assomiglia a un hub. Sono entrambi scatole, tipicamente dotate di un numero di porte da 4 a 48, ognuna

fornita di un connettore standard RJ-45 per doppino. Ogni cavo connette lo switch o hub a un solo computer, come si vede nella Figura 4.18. Uno switch presenta anche gli stessi vantaggi di un hub: è facile aggiungere o rimuovere una nuova stazione collegando o scollegando un cavo e lo è altrettanto individuare molti malfunzionamenti, poiché il cavo o la porta difettosi avranno di solito effetto su una sola stazione. C'è ancora un componente che può fallire, lo switch stesso, ma se tutte le stazioni perdono la connettività i tecnici sanno che cosa si deve fare per risolvere il problema: rimpiazzare l'intero switch.

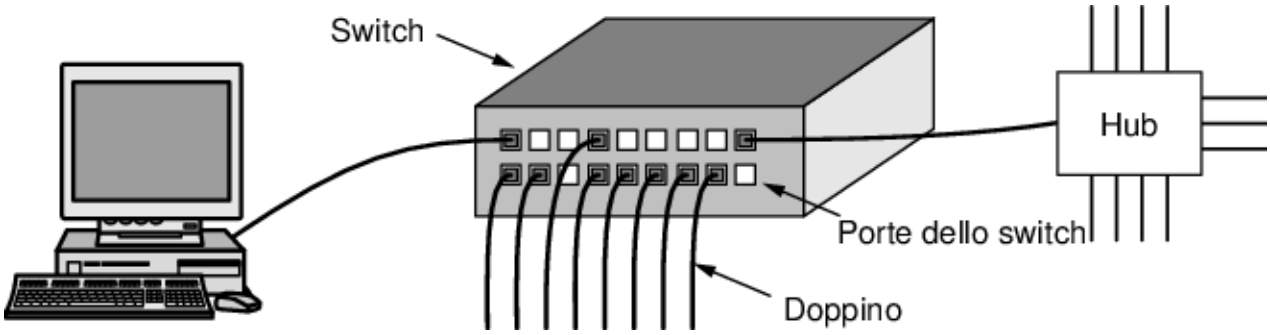


Figura 4.18 Uno switch Ethernet.

Dentro lo switch, tuttavia, accade qualcosa di molto diverso. Gli switch devono solo far uscire i frame verso le giuste porte di destinazione. Quando una porta dello switch riceve un frame Ethernet da una stazione, lo switch verifica l'indirizzo Ethernet per capire a quale porta sia destinato. Questo passaggio richiede che lo switch sia in grado di capire quali porte corrispondono a quali indirizzi, un processo che descriveremo nel Paragrafo 4.8 quando tratteremo il caso generale di switch connessi ad altri switch. Per ora ipotizziamo solo che lo switch sappia quale sia la porta di destinazione, per cui inoltra il frame sul backplane ad alta velocità verso la porta di destinazione. Il backplane funziona in genere a molti Gbps, utilizzando un protocollo proprietario che non ha bisogno di essere standardizzato, poiché è interamente nascosto all'interno dello switch. La porta di destinazione trasmette poi il frame sul cavo in maniera tale da raggiungere la stazione desiderata. Nessuna delle altre porte sa che il frame esiste.

Che cosa succede se più di una stazione o porta vuole spedire un frame nello stesso istante? Anche in questo caso gli switch si differenziano dagli hub. In un hub tutte le stazioni sono nello stesso **dominio di collisione** (*collision domain*). Devono utilizzare l'algoritmo CSMA/CD per pianificare le loro trasmissioni. In uno switch ogni porta ha il proprio dominio di collisione indipendente. Nel caso comune in cui il cavo sia full duplex, sia la stazione sia la porta possono spedire il frame lungo il cavo contemporaneamente, senza preoccuparsi delle altre stazioni e porte. Le collisioni sono ora impossibili e CSMA/CD non è più necessario. Tuttavia, se il cavo è half duplex, la stazione e la porta si contenderanno la trasmissione nel solito modo con CSMA/CD.

Rispetto a un hub uno switch migliora la prestazioni in due modi. Innanzitutto, poiché non ci sono collisioni, la capacità è utilizzata più efficientemente. Secondo, e più importante, con uno switch frame diversi possono essere spediti simultaneamente (da stazioni differenti). Questi frame raggiungeranno le porte dello switch e viaggeranno attaverso il backplane per essere spediti poi alle porte appropriate. Tuttavia, poiché due frame potrebbero essere spediti alla stessa porta di uscita nello stesso istante, lo switch deve avere un buffer, cosicché possa temporaneamente accodare il frame in ingresso finché non potrà essere trasmesso alla porta di uscita. Complessivamente, questi miglioramenti danno luogo a un notevole miglioramento prestazionale che sarebbe impossibile con un hub. Il rendimento totale del sistema può essere migliorato di un ordine di grandezza, a seconda del numero di porte e del tipo di traffico.

La diversa gestione dei frame migliora anche la sicurezza. La maggior parte delle interfacce di rete per LAN supporta una **modalità promisqua** (*promiscuous mode*) in cui un computer raccoglie *tutti* i frame e non solo quelli a esso indirizzati. Con un hub, ogni computer collegato può vedere il traffico spedito tra tutti gli altri computer; spie e ficcanaso amano questa catteristica. Con uno switch, il traffico è inoltrato solo alle porte a cui è destinato. Questa restrizione fornisce un migliore isolamento grazie al quale il traffico ha meno possibilità di cadere nelle mani sbagliate. In ogni caso è meglio cifrare il traffico se la sicurezza è davvero necessaria.

Poiché lo switch si aspetta solo frame Ethernet standard in ogni porta d'ingresso è possibile utilizzare alcune porte come concentratori. Nella Figura 4.18 la porta nell'angolo in alto a destra non è connessa a una singola stazione, ma a un hub a 12 porte. Quando i frame arrivano all'hub si contendono il mezzo trasmissivo nel solito modo, incluse collisioni e backoff binari. I frame che avranno successo attraverseranno l'hub ed entreranno nello switch, dove saranno gestiti come ogni altro frame in ingresso. Lo switch non sa che si sono dovuti conquistare il loro accesso. Una volta nello switch, sono inviati alla giusta linea d'uscita attraverso il backplane ad alta velocità. È anche possibile che la giusta destinazione sia una delle linee collegate all'hub; in quel caso il frame è già stato consegnato e quindi lo switch semplicemente lo scarta. Gli hub sono più semplici ed economici degli switch ma, a causa del crollo dei prezzi di questi ultimi, sono diventati una specie in via di estinzione. Le reti moderne utilizzano soprattutto Ethernet commutata, tuttavia gli hub di un tempo esistono ancora per motivi di compatibilità.

Fast Ethernet

Mentre gli switch diventavano sempre più popolari, la velocità di 10 Mbps di Ethernet venne messa sotto pressione. All'inizio, 10 Mbps sembravano un paradiso, esattamente come lo sembravano i cable modem per gli utenti di modem telefonici. Ma la novità è svanita in fretta e, come una sorta di corollario alla legge di Parkinson ("Il lavoro si espande riempiendo il tempo disponibile al suo completamento"), sembra che i dati si siano espansi riempiendo la banda disponibile per la loro trasmissione.

Molte installazioni hanno bisogno di una banda più grande e perciò hanno numerose LAN a 10 Mbps connese attraverso un dedalo di ripetitori, hub e switch, tanto che ai gestori di rete a volte sembra che siano tenute insieme da gomma da masticare e spago. Ma anche per gli switch Ethernet la banda massima di un singolo computer era limitata dal cavo che lo connetteva alla porta dello switch.

È con queste premesse che nel 1992 IEEE riunì il comitato 802.3 dando il mandato di ideare una LAN più veloce. Una proposta fu di mantenere lo standard 802.3 esattamente com'era, aumentando solo la velocità; un'altra proposta fu di rifare tutto da capo, per definire nuove funzionalità come il traffico in tempo reale e la voce digitale, mantenendo però il vecchio nome (per motivi di marketing). Dopo la solita baruffa, il comitato decise di mantenere 802.3 così com'era, migliorando però la sua velocità. Questa strategia avrebbe potuto funzionare prima che la tecnologia fosse cambiata evitando problemi imprevisti derivanti da una nuova progettazione. Il nuovo sistema sarebbe anche stato retrocompatibile con le LAN Ethernet esistenti. Coloro che caldeggiavano la proposta scartata fecero quello che ogni rispettabile industriale avrebbe fatto nelle stesse circostanze: se ne andarono e formarono un loro comitato standardizzando comunque la loro LAN (come 802.12), che fallì miseramente. Il lavoro venne svolto velocemente (secondo i tempi dei comitati per gli standard) e il risultato, **802.3u**, fu approvato ufficialmente da IEEE nel giugno del 1995. Tecnicamente 802.3u non è un nuovo standard, ma un'aggiunta allo standard 802.3 esistente (per enfatizzare la compatibilità all'indietro). Poiché praticamente tutti lo chiamano **fast Ethernet** e non 802.3u, lo faremo anche noi.

L'idea alla base di fast Ethernet era semplice: mantenere tutti i vecchi formati di frame per calcoli, interfacce e regole procedurali, riducendo semplicemente il tempo bit da 100 ns a 10 ns. Tecnicamente sarebbe stato possibile copiare Ethernet classica a 10 Mbps e rilevare in tempo le collisioni semplicemente riducendo la lunghezza massima del cavo di un fattore 10. Tuttavia, i vantaggi della cablatura con doppino erano talmente schiaccianti che fast Ethernet si basò interamente su questa architettura. Quindi, tutti i sistemi fast Ethernet utilizzano hub e commutatori; cavi multidrop con connettori a vampiro e connettori BNC non sono più consentiti.

Nonostante tutto era necessario fare anche altre scelte: la più importante riguardava i tipi di cavi da supportare. Un contendente era il doppino di Categoria 3; a suo favore c'era il fatto che praticamente ogni ufficio del mondo occidentale disponeva di almeno quattro doppini di Categoria 3 (o migliori) collegati a una centralina telefonica distante non più di 100 metri. Alcuni uffici avevano due di questi cavi per posto di lavoro. Utilizzando i doppini di Categoria 3 sarebbe stato possibile collegare i computer desktop con fast Ethernet senza dover ricablare l'edificio, un enorme vantaggio per molte aziende.

Lo svantaggio principale del doppino di Categoria 3 è la sua incapacità di trasportare 100 Mbps per 100 metri, perché questa era la distanza massima da computer a hub specificata per hub a 10 Mbps. Diversamente, i doppini di Categoria 5 potevano arrivare facilmente a 100 metri e le fibre ottiche potevano estendersi per una distanza anche superiore. Il compromesso scelto fu di permettere tutte e tre le possibilità, come mostrato nella Figura 4.19, conferendo però alla soluzione basata sui cavi di Categoria 3 la capacità di trasporto aggiuntiva necessaria.

Nome	Cavo	Lunghezza max. segmento	Vantaggi
100Base-T4	Doppino	100 m	UTP di categoria 3

Nome	Cavo	Lunghezza max. segmento	Vantaggi
100Base-TX	Doppino	100 m	Full duplex a 100 Mbps (UTP di categoria 5)
100Base-FX	Fibra ottica	2.000 m	Full duplex a 100 Mbps; distanze elevate

Figura 4.19 Le cablature fast Ethernet originali.

La struttura di UTP in Categoria 3, chiamata anche **100Base-T4**, usa una velocità di segnalazione di 20 MHz, solo il 25% più veloce dei 20 MHz dello standard Ethernet (ricordate che la codifica Manchester, discussa nel Paragrafo 2.5, richiede due periodi di clock per ognuno dei 10 milioni di bit spediti ogni secondo). Tuttavia, per raggiungere la velocità necessaria, 100Base-T4 necessita di 4 doppini. Di questi quattro, uno va sempre all'hub, uno esce sempre dall'hub, e gli altri due possono cambiare a seconda della direzione della trasmissione in corso. Per ottenere 100 Mbps dai tre doppini nella direzione di trasmissione si utilizza uno schema abbastanza complicato su ogni doppino. Ciò comporta inviare cifre ternarie con tre diversi livelli di voltaggio. Questo schema difficilmente otterrà un premio in merito all'eleganza e ne salteremo i dettagli. Tuttavia, poiché il cablaggio telefonico standard per decenni ha avuto quattro doppini per cavo, la maggior parte degli uffici è in grado di usare la cablatura esistente. Naturalmente, ciò significa non poter più disporre del telefono in ufficio, ma questo è un modesto prezzo da pagare in cambio di una posta elettronica più veloce.

100Base-T4 fu accantonato non appena numerosi uffici furono ricablati con UTP di Categoria 5 per Ethernet **100Base-TX**, che iniziò a dominare il mercato. La sua progettazione è più semplice, in quanto i cavi possono gestire una velocità di clock di 125 MHz. Sono utilizzati solo due doppini per stazione: uno da e uno verso l'hub. Non sono utilizzate né una codifica binaria diretta (come NRZ) né la codifica Manchester: viene invece utilizzata la codifica 4B/5B descritta nel Paragrafo 2.5: 4 bit di dati sono codificati con 5 bit di segnale e spediti a 125 MHz per ottenere 100 Mbps. Questo schema è semplice, ma presenta sufficienti transizioni per la sincronizzazione e utilizza la banda del cavo relativamente bene. Il sistema 100Base-TX è full duplex; le stazioni possono trasmettere 100 Mbps su di un doppino e ricevere a 100 Mbps sull'altro nello stesso momento.

L'ultima variante, **100Base-FX**, usa due fasci di fibre multimodali, uno per ogni direzione, per cui anch'essa può fornire un full duplex con 100 Mbps in ciascuna direzione. Con questa soluzione, la distanza tra una stazione e lo switch può arrivare fino a 2 km.

Fast Ethernet permette interconnessioni attraverso sia hub che switch. Per assicurare che l'algoritmo CSMA/CD continui a funzionare, la relazione tra la dimensione minima del frame e la lunghezza massima del cavo deve essere mantenuta in proporzione alla velocità della rete che va da 10 Mbps a 100 Mbps. Quindi o va aumentata la dimensione minima di frame di 64 byte o dovrà diminuire proporzionalmente la lunghezza massima di cavo di 2500 m. La scelta più facile è stata di abbassare la distanza tra ogni coppia di stazioni di un fattore 10, poiché un hub con un cavo da 100 metri rientra già in questo nuovo limite massimo. Tuttavia, cavi 100Base-FX da 2 km sono

troppo lunghi per permettere a un hub da 100 Mbps di funzionare con il normale algoritmo delle collisioni di Ethernet. Questi cavi devono invece essere collegati a uno switch e funzionare in modalità full duplex per non avere collisioni.

Gli utenti iniziarono presto ad adottare fast Ethernet, ma non gettarono via le schede Ethernet da 10 Mbps dei computer più vecchi. Di conseguenza, praticamente tutti gli switch fast Ethernet possono gestire un misto di stazioni a 10 e 100 Mbps. Per facilitare l'aggiornamento, lo stesso standard fornisce un meccanismo chiamato **autonegoziazione** (*autonegotiation*) che permette a due stazioni di negoziare automaticamente la velocità ottimale (10 o 100 Mbps) e il tipo di duplex (half o full). Funziona bene per la maggior parte dei casi, ma causa problemi di disallineamento del duplex: un capo del collegamento negozia e l'altro no ed è impostato in modalità full-duplex (Shalunov e Carlson, 2005). Molti dei prodotti Ethernet utilizzano questa caratteristica per l'autoconfigurazione.

Gigabit Ethernet

L'inchiostro impiegato per scrivere lo standard fast Ethernet si era appena asciugato quando il comitato 802 iniziò a lavorare su una Ethernet ancora più veloce, subito soprannominata **gigabit Ethernet**. Lo standard fu ratificato da IEEE nel 1999 con il nome di 802.3ab. Questo paragrafo descrive alcune delle funzionalità chiave di gigabit Ethernet. Maggiori informazioni sono reperibili in Spurgeon (2000).

Gli obiettivi del comitato per gigabit Ethernet erano essenzialmente gli stessi del comitato per fast Ethernet: aumentare le prestazioni di dieci volte mantenendo la compatibilità con tutti gli standard Ethernet esistenti. In particolare, gigabit Ethernet doveva offrire servizi datagram senza acknowledgement di tipo sia unicast che broadcast, adottare lo stesso schema di indirizzamento a 48 bit già in uso e mantenere lo stesso formato di frame, incluse le dimensioni massima e minima. Lo standard finale risponde a tutti questi requisiti.

Come fast Ethernet, tutte le configurazioni gigabit Ethernet sono punto a punto. Nella sua configurazione più semplice gigabit Ethernet, mostrata nella Figura 4.20(a), collega due computer direttamente tra loro; il caso più comune, tuttavia, è quello dove uno switch o un hub si collegano a più computer e potenzialmente ad altri switch o hub, come mostrato nella Figura 4.20(b). In entrambe le configurazioni ogni singolo cavo Ethernet è collegato a due e solo due dispositivi.

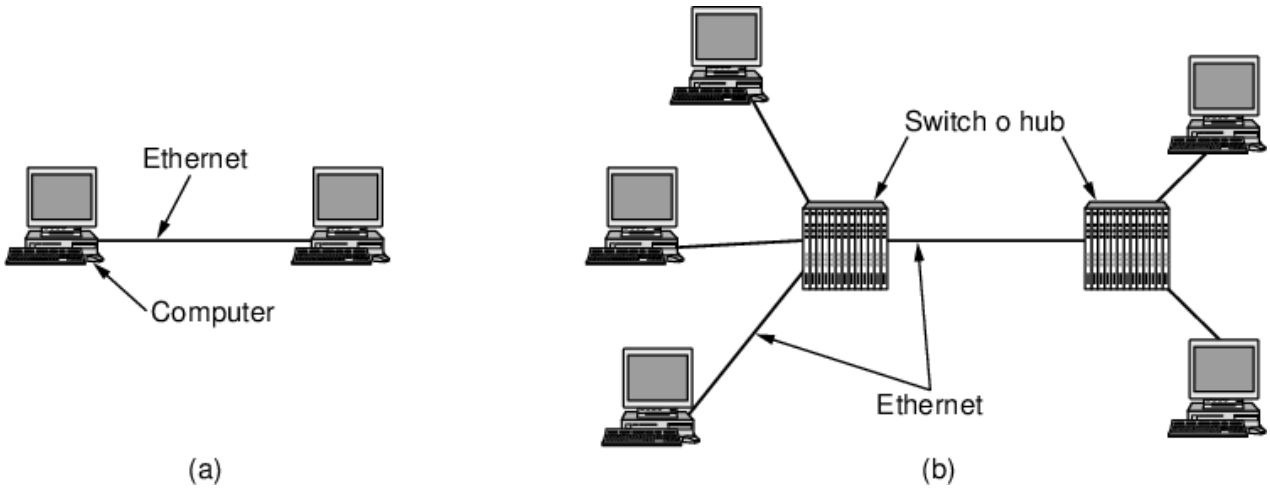


Figura 4.20 (a) Una Ethernet a due stazioni. (b) Una Ethernet a più stazioni.

Come fast Ethernet, gigabit Ethernet supporta due modalità operative: full duplex e half duplex. La modalità normale è quella full duplex, che permette al traffico di viaggiare contemporaneamente in entrambe le direzioni. Questa modalità è utilizzata quando c'è uno switch centrale collegato ai computer (o ad altri switch). In questa configurazione tutte le linee hanno dei buffer, per cui ogni computer o switch è libero di inviare frame quando lo desidera. Il trasmittente non deve esaminare il canale per vedere se qualcun altro lo sta già utilizzando, perché la contesa è impossibile. Sulla linea che collega un computer allo switch, il computer è l'unico che può trasmettere allo switch e la trasmissione ha successo anche se lo switch sta contemporaneamente trasmettendo un frame al computer, perché la linea è full duplex. Poiché non si verifica contesa, non si utilizza il protocollo CSMA/CD, perciò la lunghezza massima del cavo dipende dall'energia del segnale e non dal tempo che un picco di rumore impiega nel peggiore dei casi per propagarsi all'indietro fino al trasmittente. Gli switch sono liberi di gestire più velocità e l'autoconfigurazione è supportata proprio come in fast Ethernet, solo che ora la scelta è tra 10, 100 e 1000 Mbps.

L'altra modalità operativa, half duplex, è utilizzata quando i computer sono collegati a un hub e non a uno switch. L'hub non ha un buffer dove memorizzare i frame in arrivo; il dispositivo al suo interno collega elettricamente tutte le linee per simulare il cavo multidrop utilizzato in Ethernet classica. In questa modalità le collisioni sono ancora possibili, perciò è richiesto il protocollo CSMA/CD standard. Poiché un frame di 64 byte (il più piccolo ammissibile) può essere trasmesso 100 volte più velocemente che nella Ethernet classica, la lunghezza massima del cavo risulta 100 volte più corta: 25 metri. Ciò consente di preservare la proprietà essenziale per cui il trasmittente sta ancora trasmettendo quando il picco di rumore torna indietro, anche nel peggiore dei casi. Con un cavo lungo 2.500 metri la trasmissione di un frame da 64 byte a 1 Gbps di velocità si sarebbe da tempo già conclusa prima che il frame avesse raggiunto anche solo un decimo del percorso verso la destinazione, figuriamoci arrivarci e tornare indietro.

Questa restrizione di lunghezza fu così dolorosa da richiedere l'aggiunta di due funzionalità allo standard, al fine di aumentare la lunghezza massima del cavo a 200 metri, il che è probabilmente sufficiente per molti uffici. La prima funzionalità, chiamata **carrier extension** (*estensione della*

portante), indica essenzialmente all'hardware di aggiungere dei dati di riempimento dopo il frame normale, in modo da estendere la dimensione del pacchetto fino a 512 byte. Poiché questo riempimento è aggiunto dall'hardware che trasmette ed è rimosso dall'hardware che riceve i dati, il software è ignaro della sua esistenza e non è necessario apportare alcuna modifica ai programmi esistenti. Il problema è che utilizzare 512 byte per trasmettere 46 byte di dati utente (la parte utile di un frame da 64 byte) comporta un'efficienza del canale di solo il 9%.

La seconda funzionalità, chiamata **frame bursting**, permette a un trasmittente di inviare una sequenza concatenata di più frame in una singola trasmissione. Se il burst totale è minore di 512 byte, l'hardware aggiunge i dati riempitivi. Se un numero sufficiente di frame è in attesa di trasmissione, questo schema è assai efficiente e preferibile alla carrier extension.

In tutta onestà è difficile immaginare che un'azienda che acquista computer moderni dotati di schede di rete gigabit Ethernet poi li colleghi a un vecchio hub per simulare una Ethernet classica con tutte le sue collisioni. Le interfacce e gli switch di gigabit Ethernet erano costosi, ma il loro prezzo diminuì rapidamente non appena crebbe il volume degli acquisti. Ancora una volta la compatibilità a ritroso è sacra nel settore informatico, quindi il comitato fu obbligato a fornirla. Oggi la maggior parte dei computer viene venduto con un'interfaccia Ethernet capace di operare a 10, 100 e 1000 Mbps.

Gigabit Ethernet supporta cavi sia in rame sia in fibra ottica, come illustrato nella Figura 4.21.

Nome	Cavo	Lunghezza max. segmento	Vantaggi
1000Base-SX	Fibra ottica	550 m	Fibra multimodale (50, 62,5 mm)
1000Base-LX	Fibra ottica	5.000 m	Fibra mono (10 mm) o multimediale (50, 62,5 mm)
1000Base-CX	2 coppie di STP	25 m	Doppino schermato
1000Base-T	4 coppie di UTP	100 m	UTP standard in Categoria 5

Figura 4.21 Cablatura gigabit Ethernet.

Mandare messaggi di segnalazione a 1 Gbps richiede di codificare e spedire un bit per nanosecondo. Questo fu inizialmente ottenuto usando cavi in rame brevi e schermati (la versione 1000Base-CX) e fibre ottiche. Per quanto riguarda quest'ultime, sono permesse due diverse lunghezze d'onda e quindi due versioni diverse: 0,85 micron (corta, per 1000Base-SX) e 1,3 m (lunga, per 1000Base-LX).

La segnalazione con lunghezza d'onda corta può essere ottenuta con LED più economici. È utilizzata con fibra multimodale ed è utile per connessioni all'interno di un edificio, poiché riesce a percorrere 500 metri con una fibra ottica da 50 micron. Il trasferimento di segnali con lunghezza d'onda lunga richiede laser più costosi. D'altra parte, quando è combinata con fibre monomodali (10 micron), la lunghezza del cavo può arrivare a 5 km. Questo limite permette collegamenti a lunga distanza tra edifici, come la dorsale di un campus o un collegamento dedicato punto a punto. Le successive varianti dello standard permisero collegamenti ancora più lunghi su fibra monomodale.

Per spedire bit su queste versioni di gigabit Ethernet, la codifica **8B/10B**, descritta nel Paragrafo 2.5, fu presa in prestito da un'altra tecnologia di rete chiamata *fibre channel*. Lo schema codifica 8 bit di dati in 10 bit di parole di codice spedite lungo il cavo o la fibra, da qui il nome 8B/10B. Le parole di codice furono scelte perché potessero essere bilanciate (cioè avere la stessa quantità di 0 e di 1) con transizioni sufficienti per il ripristino del clock. Spedire i bit codificati con NRZ richiede un'ampiezza della banda di segnalazione del 25% più ampia di quella richiesta dai bit non codificati, un grande miglioramento rispetto all'espansione del 100% della codifica Manchester.

Tuttavia, tutte queste opzioni richiedono nuovi cavi in rame o fibra ottica per sostenere la trasmissione più veloce. Nessuna di esse fece uso della gran quantità di UTP in Categoria 5 installata con fast Ethernet. Nel giro di un anno, 1000Base-T è riuscita a colmare questa lacuna e da allora è diventata la forma più popolare di gigabit Ethernet. Pare che alla gente non piaccia ricablare i propri edifici. È necessario un sistema di segnalazione più complicato per far andare Ethernet a 1000 Mbps su cavi di Categoria 5. Innanzitutto tutti e quattro i doppini del cavo vengono utilizzati e ogni paio è impiegato in entrambe le direzioni contemporaneamente, grazie a un'elaborazione digitale per separare i segnali. Su ogni cavo sono utilizzati cinque livelli di tensione che trasportano 2 bit per ottenere 125 milioni di simboli al secondo. La mappa per produrre i simboli dai bit non è semplice; si tratta di mescolare attraverso transizioni e poi applicare un codice di correzione d'errore per associare quattro valori a cinque livelli di segnale.

La velocità di 1 Gbps è decisamente alta. Per esempio, se un ricevitore è occupato in qualche altra operazione anche solo per 1 ms e non svuota il buffer di input su qualche linea, in quell'intervallo di 1 ms possono accumularsi fino a 1953 frame. Inoltre, quando un computer su una gigabit Ethernet manda dati a un computer collegato a una Ethernet classica è molto probabile che il buffer si esaurisca. Come conseguenza di queste due osservazioni, gigabit Ethernet implementa il controllo di flusso. Il meccanismo consiste in uno dei due terminali che spedisce all'altro uno speciale frame di controllo indicandogli di mettersi in pausa per un certo periodo di tempo. Questi frame di PAUSA sono normali frame Ethernet in cui il campo *Type* vale 0x8808. Le pause sono date in unità del minimo tempo di frame. Per gigabit Ethernet l'unità di tempo è di 512 ns, permettendo pause fino a 33,6 ms.

C'è un'altra estensione introdotta insieme a gigabit Ethernet. I **jumbo frame** consentono ai frame di essere più lunghi di 1500 byte, di solito fino a 9 KB. Questa estensione è proprietaria e non viene inclusa nello standard, perché se fosse usata Ethernet non sarebbe più compatibile con le versioni precedenti, anche se molte case produttrici la supportano ugualmente. Il fondamento logico è che 1500 byte sono un'unità piccola alla velocità di gigabit. Manipolando blocchi di informazione più grandi è possibile diminuire il tasso dei frame, e con esso l'elaborazione associata, come se si interrompesse il processore a indicare l'arrivo di un frame o si dividessero e ricombinassero messaggi troppo lunghi per stare in un singolo frame Ethernet.

10-gigabit Ethernet

Subito dopo la standardizzazione di gigabit Ethernet la commissione 802 iniziò ad annoiarsi e a desiderare la ripresa del lavoro. IEEE la esortò a lavorare su 10-gigabit Ethernet. La relativa procedura seguì la stessa trafila dei precedenti standard Ethernet, con standard per cavi in fibra ottica e in rame schermato che apparvero per la prima volta nel 2002 e 2004, seguiti nel 2006 da standard per doppini di rame.

10 Gbps è una velocità davvero prodigiosa, 1000 volte più veloce della Ethernet originale: a chi potrebbe servire? La risposta è all'interno dei data center e nelle centrali di commutazione per collegare router di fascia alta, switch e server, così come su collegamenti a lunga distanza e larga banda tra centrali che gestiscono intere MAN basate su Ethernet e fibra. I collegamenti a lunga distanza usano fibra ottica, mentre quelli brevi possono utilizzare rame o fibra. Tutte le versioni di 10-gigabit Ethernet supportano solo la modalità full-duplex.

CSMA/CD non fa più parte dell'architettura e gli standard si concentrano sui dettagli dei livelli fisici che possono andare a velocità davvero elevata. La compatibilità è ancora importante, quindi le interfacce di rete 10-gigabit Ethernet fanno uso di auto negoziazione e, se necessario, ripiegano verso la velocità più elevata supportata da entrambe le estremità della linea.

I principali tipi di 10-gigabit Ethernet sono elencate nella Figura 4.22. La fibra multimodale con un'ampiezza d'onda di 0,85m (corta) è utilizzata per distanze medie e le fibre monomodali da 1,3m (lunga) e 1,5m (estesa) sono utilizzate per lunghe distanze. 10GBase-ER può raggiungere distanze fino a 40 km, rendendola adatta per applicazioni su vaste aree. Tutte queste versioni inviano un flusso sequenziale di dati prodotto mischiando i bit e poi applicando una codifica **64B/66B**, che ha un overhead minore della codifica 8B/10B.

Nome	Cavo	Lunghezza max. segmento	Vantaggi
10GBase-SR	**Fibra ottica**	**Fino a 300 m**	**Fibra multimodale (0,85 mm)**
10GBase-LR	**Fibra ottica**	**10 Km**	**Fibra monomodale (1,3 mm)**
10GBase-ER	**Fibra ottica**	**40 Km**	**Fibra monomodale (1,5 mm)**
10GBase-CX4	**4 coppie di biassiali**	**15 m**	**Rame biassiale**
10GBase-T	**4 coppie di UTP**	**100 m**	**UTP di Categoria 6a**

Figura 4.22 Cablatura di 10-gigabit Ethernet.

La prima versione in rame, definita 10GBase-CX4, usa un cavo con quattro paia di doppini in rame biassiali. Ogni doppino utilizza la codifica 8B/10B e veicola 3,125 miliardi di simboli al secondo per raggiungere 10 Gbps. Questa versione è più economica della fibra ottica e fu messa presto sul mercato, ma resta da vedere se verrà battuta sul lungo termine da un 10-gigabit Ethernet facente uso di un doppino più ordinario.

10GBase-T è la versione che utilizza cavi UTP. Nonostante richieda un cablaggio in Categoria 6a, per le brevi distanze può utilizzare categorie inferiori (inclusa la Categoria 5) per permettere il parziale riutilizzo delle cablature esistenti. Non stupisce che il livello fisico sia particolarmente coinvolto per raggiungere 10 Gbps su doppino. Ci limiteremo ad abbozzare alcuni dei dettagli di alto livello. Ognuno dei quattro doppini è utilizzato per spedire 2500 Mbps in entrambe le direzioni. Questa velocità viene raggiunta utilizzando un tasso di trasmissione di 800 milioni di simboli al secondo con simboli che usano 16 livelli di tensione. I simboli sono prodotti mischiando i dati, proteggendoli con un codice **LDPC** (*low density parity check*) e codificandoli ancora per la correzione degli errori.

10-gigabit Ethernet è tuttora vivacemente presente sul mercato, ma il comitato 802.3 è già andato oltre. Alla fine del 2007 IEEE ha creato un gruppo per standardizzare Ethernet a 40 e 100 Gbps. Questo aggiornamento permetterà a Ethernet di competere in ambienti dove le prestazioni sono molto elevate, incluse connessioni a lunga distanza sulle dorsali e connessioni a breve distanza su backplane delle attrezzature. Lo standard non è ancora completo, ma prodotti proprietari sono già disponibili.

Retrospettiva su Ethernet

Ethernet è stata utilizzata per più di 30 anni e non ha alcun serio concorrente in vista, perciò è probabile che continuerà a dominare il mercato per molti anni a venire. Poche architetture di CPU, sistemi operativi o linguaggi di programmazione sono rimasti i re della foresta per tre decenni. Chiaramente, Ethernet è stata una scelta vincente. Quale?

Probabilmente questa longevità dipende dalla semplicità e dalla flessibilità. Nella pratica, semplice si traduce in affidabile, economico e facile da manutenere. Una volta adottata l'architettura di hub e switch i guasti sono diventati estremamente rari. La gente è restia a sostituire ciò che funziona perfettamente, specialmente perché si sa che nel settore informatico è facile incappare in cose che funzionano davvero male, tanto che alcuni cosiddetti "aggiornamenti" sono peggiori dei loro predecessori.

Il concetto di semplicità si traduce anche in convenienza. Il cablaggio con doppini è relativamente economico come lo sono i componenti hardware. All'inizio sono costosi durante la transizione, ma in realtà sono solo delle integrazioni a una rete ben avviata e non una sostituzione di essa; ne sono un esempio le nuove interfacce di rete o gli switch per gigabit Ethernet. Inoltre il prezzo cala velocemente non appena il volume di vendite aumenta.

Ethernet è facile da manutenere: non richiede l'installazione di alcun software (a parte i driver) e non dipende da tabelle di configurazione difficili da gestire. Inoltre, aggiungere nuovi terminali è semplice come collegare un cavo.

Un altro dettaglio importante è che Ethernet è in grado di funzionare facilmente con TCP/IP, un protocollo ormai dominante. IP è un protocollo senza connessione, perciò si adatta perfettamente a Ethernet, a sua volta senza connessione. IP si adatta molto meno a soluzioni orientate alla connessione, come ATM; questo accoppiamento sbagliato ha notevolmente danneggiato ATM.

Infine, ed è forse la cosa più importante, Ethernet è stata in grado di evolversi in modo strategico: le velocità sono aumentate di diversi ordini di grandezza e sono stati introdotti hub e switch, ma queste modifiche non hanno richiesto alcun cambiamento nel software e hanno spesso permesso di riutilizzare le cablature esistenti per un certo periodo. Quando un commerciale si presenta in una grande azienda e dice "ho questa rete fantastica che è perfetta per voi, tutto quello che dovete fare è buttare tutto il vostro vecchio hardware e riscrivere tutto il software" va incontro a seri problemi.

Molte tecnologie alternative che probabilmente non avete mai sentito nominare erano più veloci di Ethernet quando furono introdotte. Oltre che ATM, questo elenco include **FDDI** (*fiber distributed data interface*) e **fibre channel**, due LAN ottiche ad anello. Entrambe incompatibili con Ethernet, nessuna delle due ce l'ha fatta. Erano troppo complicate, il che significa processori complessi e prezzi elevati. La conclusione da tirare è KISS (*keep it simple, stupid*). Alla fine Ethernet le raggiunse quanto a velocità, spesso prendendo in prestito un po' della loro tecnologia; per esempio, la codifica 4B/5B da FDDI e la codifica 8B/10B da fibre channel. Quindi, non avendo alcun vantaggio, lentamente scomparvero o vennero limitate a ruoli specializzati.

Sembra che Ethernet continuerà a espandersi ancora per qualche tempo. Grazie a 10-gigabit Ethernet si è liberata dal vincolo di distanza di CSMA/CD. Un grande sforzo si sta facendo per **carrier-grade Ethernet** per consentire ai provider di offrire ai propri clienti MAN e WAN servizi basati su Ethernet (Fouli e Maler, 2009). Quest'ultima applicazione porta i frame di Ethernet a lunga distanza tramite fibra ottica e richiede delle migliori funzionalità di gestione per aiutare gli operatori a offrire servizi affidabili e di alta qualità. Le reti molto veloci stanno anche trovando utilizzi nella connessione di componenti su backplane in grandi router o server. Entrambi questi impieghi sono aggiuntivi a quello di occuparsi della spedizione di frame tra computer negli uffici.

4.4 Lan wireless

Le LAN wireless sono sempre più diffuse e case, uffici, bar, librerie, areoporti, zoo e altri luoghi pubblici ne sono forniti per collegare a Internet computer, PDA e smart phone. Le LAN wireless possono anche essere usate per permettere a due o più computer vicini di comunicare tra loro senza usare Internet.

Il principale standard per LAN wireless è 802.11. Ne abbiamo dato alcune informazioni di base nel Paragrafo 1.5.3. Ora è tempo di dedicare uno sguardo più preciso alla tecnologia. Nei paragrafi successivi vedremo lo stack dei protocolli, tecniche di trasmissione radio per il livello fisico, il protocollo di sottolivello MAC, la struttura dei frame e i servizi forniti. Per saperne di più su 802.11, si veda Gast (2005). Per vederci davvero chiaro, si consultino direttamente gli standard: IEEE 802.11-2007.

Architettura e stack di protocolli di 802.11

Le reti 802.11 possono essere utilizzate in due modalità; quella più comune è connettere dei client, come portatili e smart phone, a un'altra rete, come una intranet aziendale o Internet, come illustrato nella Figura 4.23(a). Nella modalità con infrastruttura, ogni client è associato a un **AP** (*access point*) connesso a sua volta all'altra rete. Il client spedisce e riceve i propri pacchetti attraverso l'AP. Alcuni access point possono essere connessi insieme, in genere attraverso una rete cablata che prende il nome di **sistema di distribuzione** (*distribution system*), per formare una rete 802.11 estesa. In questo caso i client possono spedire frame agli altri client attraverso i loro AP.

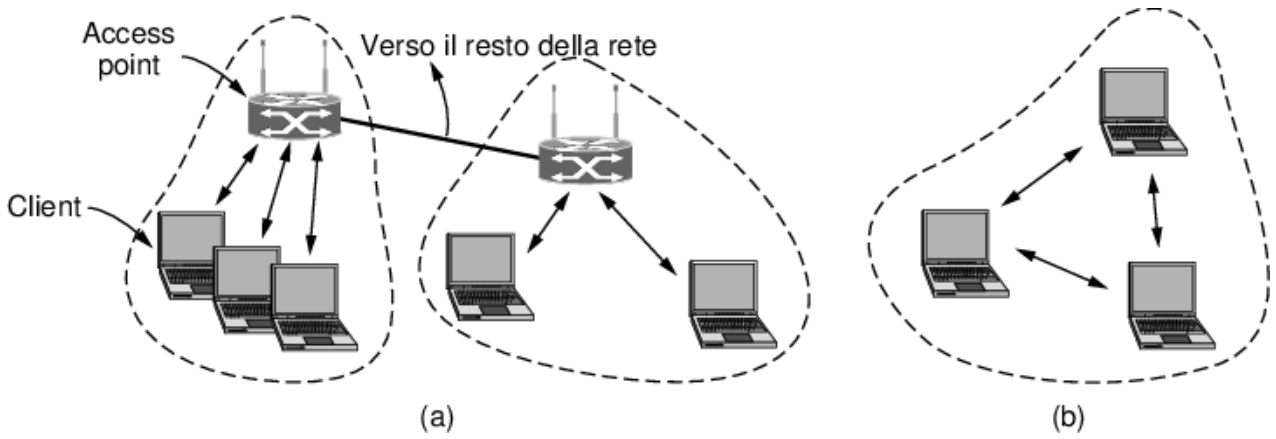


Figura 4.23 Architettura 802.11. (a) Modalità con infrastruttura, (b) Modalità ad hoc.

L'altra modalità, mostrata nella Figura 4.23(b), è una **rete ad hoc**. Questa modalità consiste in una collezione di computer associati tra loro che possono spedirsi direttamente i frame. Non esiste alcun access point. Dal momento che l'accesso a Internet è la killer application per le reti wireless, le reti ad hoc non sono molto popolari.

Passiamo ora ai protocolli. Tutti i protocolli 802, inclusi 802.11 e Ethernet, presentano una certa somiglianza nella struttura. Una rappresentazione parziale dello stack di protocolli di 802.11 è visibile nella Figura 4.24. Lo stack è lo stesso per client e AP. Il livello fisico corrisponde abbastanza bene al livello fisico OSI, ma il livello data link nei protocolli 802 è diviso in due o più sottolivelli. In 802.11 il sottolivello MAC determina com'è allocato il canale, cioè a chi tocca trasmettere. Sopra c'è il sottolivello **LLC** (*logical link control*) il cui compito è nascondere le differenze tra i differenti 802 e renderli indistinguibili al livello di rete. Questo poteva avere una responsabilità significantiva, ma oggi LLC è un livello di raccordo che identifica il protocollo (ad esempio IP) trasportato all'interno di un frame 802.11.

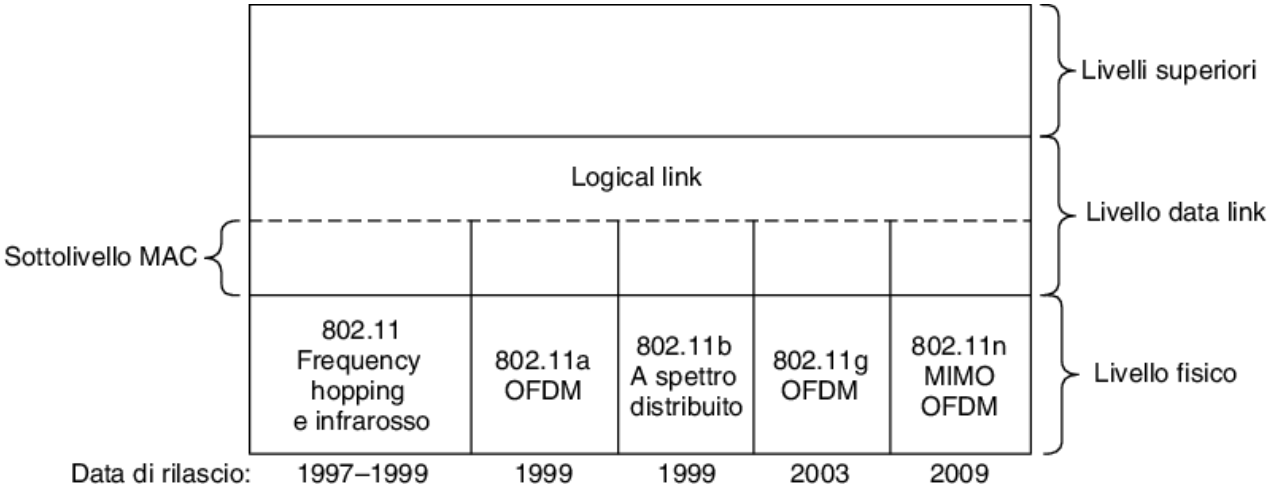


Figura 4.24 Una parte della pila di protocolli 802.11.

Fin dalla sua comparsa nel 1997 diverse tecniche di trasmissione sono state aggiunte al livello fisico di 802.11 mentre questo si evolveva. Due delle prime tecniche, gli infrarossi come quelli dei telecomandi televisivi e il frequency hopping nella banda dei 2,4 GHz, sono ormai scomparse. La terza tecnica iniziale, spettro distribuito a sequenza diretta a 1 o 2 Mbps nella banda dei 2,4 GHz, fu estesa per raggiungere tassi di trasmissione di 11 Mbps ed è diventata rapidamente un successo. È conosciuta oggi come 802.11b.

Per conferire ai dipendenti da wireless la tanto desiderata spinta sulla velocità, nel 1999 e nel 2003 furono introdotte nuove tecniche di trasmissione basate su OFDM, (schema che abbiamo già descritto nel Paragrafo 2.5.3). La prima è chiamata 802.11a e utilizza una differente banda di frequenze: 5 GHz. La seconda è rimasta agganciata ai 2,4 GHz e alla compatibilità; viene chiamata 802.11g. Entrambe raggiungono tassi trasmissivi di 54 Mbps.

Più recentemente, nell'ottobre del 2009, sono state messe a punto tecniche di trasmissione che utilizzano simultaneamente più antenne su trasmittenti e ricevitori per ottenere una maggiore velocità; queste sono note come 802.11n. Con quattro antenne e canali più ampi, lo standard 802.11 definisce ora tassi trasmissivi che raggiungono anche i 600 Mbps.

Di seguito esamineremo brevemente ognuna di queste tecniche di trasmissione; tratteremo tuttavia solo quelle attuali, tralasciando le vecchie metodologie 802.11. Tecnicamente queste appartengono al livello fisico e avremmo dovuto esaminarle nel Capitolo 2, tuttavia la spiegazione è stata inserita in questo capitolo, poiché sono strettamente collegate alle LAN in generale e al sottolivello MAC di 802.11 in particolare.

Livello fisico di 802.11

Ognuna delle tecniche di trasmissione permette l'invio di un frame MAC da una stazione a un'altra, ma le soluzioni differiscono sia per la tecnologia utilizzata sia per le velocità supportate. La descrizione dettagliata di queste tecnologie non rientra negli obiettivi del volume, tuttavia spendere qualche parola su ognuna di esse le metterà in relazione con il materiale trattato nel Paragrafo 2.5 e fornirà ai lettori interessati uno spunto dal quale dare avvio a una ricerca più approfondita.

Tutte le tecniche 802.11 utilizzano apparati radio a corto raggio per trasmettere nelle bande di frequenze dei 2,4 GHz o dei 5 GHz ISM (*industrial, scientific, medical*), entrambe descritte nel Paragrafo 2.3.3. Queste bande hanno il vantaggio di non richiedere una licenza e quindi sono liberamente disponibili a ogni emittente disposta a rispettare alcune restrizioni, come una potenza massima irradiata di 1 W (contro i 50 mW più tipici degli apparati radio per LAN). Sfortunatamente ciò è noto anche ai produttori di apricancelli per garage, telefoni cordless, forni a microonde e innumerevoli altri apparecchi, tutti in competizione con i laptop per lo stesso spettro. La banda a 2,4 GHz tende a essere più affollata di quella a 5 GHz, per cui i 5 GHz possono andare meglio per alcune applicazioni, anche se hanno un raggio di copertura inferiore a causa della frequenza più elevata.

Tutti i metodi di trasmissione definiscono anche più tassi di trasmissione. L'idea è che velocità differenti possono essere utilizzate a seconda delle condizioni del momento. Se il segnale wireless è debole, può essere utilizzato un tasso di invio basso; se il segnale è nitido, può essere utilizzato quello più alto. Tale accorgimento è detto **rate adaptation** (*adattamento del tasso di trasmissione*). Poiché le velocità di trasmissione variano di un fattore 10 o più, un buon adattamento è importante per ottenere buone prestazioni; naturalmente, poiché non è richiesto per l'interoperabilità, gli standard non indicano come dovrebbe essere fatto.

Il primo metodo di trasmissione che prenderemo in esame è **802.11b**. Si tratta di un metodo a spettro distribuito che utilizza tassi di trasmissione di 1, 2, 5,5 e 11 Mbps, anche se, in pratica, la velocità di funzionamento è quasi sempre 11 Mbps. È simile al sistema CDMA esaminato nel Paragrafo 2.5, eccetto che vi è un unico codice condiviso da tutti gli utenti. La codifica CDMA è utilizzata per soddisfare il requisito di FCC per cui la potenza trasmissiva sia distribuita all'interno della banda ISM. La sequenza di diffusione utilizzata da 802.11b è una **sequenza di Barker**; questa ha una proprietà per cui la sua autocorrelazione è bassa tranne quando le sequenze sono allineate, permettendo a un ricevitore di agganciarsi all'inizio di una trasmissione. Per inviare dati a un tasso

di 1 Mbps, la sequenza di Barker è utilizzata con una modulazione BPSK per spedire 1 bit ogni 11 chip. Le chip sono trasmesse a un tasso di 11 Mchip/s. Per inviare a 2 Mbps, la stessa sequenza è utilizzata con una modulazione QPSK per spedire 2 bit ogni 11 chip. I tassi trasmissivi più elevati sono diversi e anziché una sequenza di Barker utilizzano una tecnica chiamata **CCK** (*complementary code keying*) per costruire il codice. A 5,5 Mbps vengono spediti 4 bit in ogni codice da 8 chip e a 11 Mbps vengono spediti 8 bit in ogni codice da 8 chip.

Passiamo a **802.11a**, che arriva fino a 54 Mbps nella banda ISM a 5 GHz. Vi aspettereste che 802.11a preceda 802.11b, ma non è così. Anche se il gruppo di lavoro per 802.11a è stato istituito per primo, lo standard 802.11b fu approvato per primo e arrivò sul mercato molto prima di 802.11a, in parte a causa della difficoltà di operare nella banda a 5 GHz, più alta.

Il metodo di trasmissione 802.11a è basato su OFDM, dato che questo utilizza lo spettro in maniera efficiente e resiste alla degradazione del segnale wireless. I bit sono inviati su 52 sottoportanti in parallelo; 48 per trasportare i dati e 4 per la sincronizzazione. Ogni simbolo dura 4ms e invia 1, 2, 4 o 6 bit. I bit sono innanzitutto codificati per la correzione degli errori con un codice di convoluzione binario, quindi solo 1/2, 2/3 o 3/4 dei bit non sono ridondanti. Con diverse combinazioni, 802.11a può raggiungere otto diverse velocità, che vanno da 6 a 54 Mbps. Questi tassi di trasmissione sono significativamente più veloci di quelli di 802.11b e ci sono anche meno interferenze nella banda a 5 GHz. Tuttavia, 802.11b ha un raggio di copertura sette volte più grande di quello di 802.11a, il che è più importante in molte situazioni.

Anche con una gamma di frequenze maggiore, il popolo di 802.11b non aveva alcuna intenzione di permettere a questo nuovo venuto di vincere il campionato di velocità. Fortunatamente, a maggio del 2002, FCC fece decadere il suo vincolo di vecchia data che richiedeva che tutte le apparecchiature wireless operanti nelle bande ISM negli Stati Uniti utilizzassero uno spettro ampio, quindi si misero a lavorare su **802.11g**, approvato da IEEE nel 2003. Questo copia i metodi di modulazione OFDM di 802.11a, ma opera nella banda stretta ISM a 2,4 GHz con lo standard 802.11b. Inoltre, offre gli stessi tassi trasmissivi di 802.11a (da 6 a 54 Mbps) oltre che, naturalmente, la compatibilità con tutti gli apparecchi 802.11b vicini. Una tale diversità di scelte potrebbe essere fonte di confusione per un cliente, perciò di solito i prodotti commerciali supportano 802.11a/b/g sulla stessa scheda di rete.

Non accontentandosi, il comitato IEEE iniziò a lavorare su un livello fisico ad alta capacità chiamato **802.11n**, ratificato nel 2009. L'obiettivo per 802.11n era un throughput di almeno 100 Mbps dopo la rimozione di tutti gli overhead. Questo obiettivo richiese un aumento dell'intera velocità di un fattore quattro. Perché ciò accadesse, il comitato raddoppiò i canali da 20 MHz a 40 MHz e ridusse gli overhead dei frame, permettendo a un gruppo di frame di essere spedito congiuntamente. Più significativamente, 802.11n utilizza fino a quattro antenne per trasmettere fino a quattro flussi d'informazione contemporaneamente. I segnali di questi flussi interferiscono col ricevitore, ma possono essere separati utilizzando le tecniche di comunicazione **MIMO** (*multiple input multiple output*). L'utilizzo di più antenne dà una grande spinta alla velocità oppure migliora raggio di copertura e affidabilità. MIMO, come OFDM, è una di quelle idee brillanti nel campo della comunicazione che stanno rivoluzionando il modo di progettare le reti wireless e di cui probabilmente sentiremo molto parlare in futuro. Per una breve introduzione alle antenne multiple utilizzate in 802.11 si veda Halperin et al. (2010).

Protocollo del sottolivello MAC di 802.11

È il momento di abbandonare il terreno dell'ingegneria elettronica per tornare su quello dell'informatica. Il protocollo del sottostrato MAC 802.11 è decisamente diverso da quello Ethernet a causa di due fattori fondamentali nella comunicazione wireless.

Primo, le interfacce radio sono quasi sempre half duplex, il che vuol dire che non possono trasmettere e controllare picchi di rumore nello stesso istante su una sola frequenza. Il segnale ricevuto può facilmente essere un milione di volte più debole di quello trasmesso, quindi non può essere sentito in contemporanea. Con Ethernet, una stazione attende finché il mezzo trasmissivo non diventa libero e inizia a trasmettere. Se non riceve alcun picco di rumore mentre spedisce i primi 64 byte, il frame è quasi certamente stato consegnato correttamente. Con il wireless, questo meccanismo di rilevamento delle collisioni non funziona.

Invece, 802.11 prova a evitare le collisioni con un protocollo chiamato **CSMA/CA** (*CSMA with collision avoidance*).

Questo protocollo è concettualmente simile a CSMA/CD di Ethernet, con il rilevamento della portante prima dell'invio e il backoff esponenziale dopo le collisioni. Tuttavia, una stazione con un frame da spedire inizia con un backoff casuale (a meno che non abbia utilizzato il canale di recente e il canale sia libero) e non aspetta che avvenga una collisione. Il numero di slot di backoff è scelto nell'intervallo tra 0 e 15 nel caso del livello fisico OFDM. La stazione attende finché il canale è libero, rilevando che non c'è segnale per un certo periodo di tempo (chiamato DIFS, come spiegheremo più avanti) e conta alla rovescia gli slot inattivi, facendo delle pause quando vengono inviati dei frame. La stazione spedisce il suo frame quando il contatore arriva a 0. Se il frame arriva a destinazione il destinatario spedisce immediatamente un piccolo frame di acknowledgement, la mancanza del quale è indice di un errore, che sia una collisione o altro. In questo caso, il mittente raddoppia il periodo di backoff e prova ancora, continuando con backoff esponenziali come in Ethernet, finché il frame sia stato trasmesso con successo o sia stato raggiunto il numero massimo di ritrasmissioni.

Nella Figura 4.25 si vede un esempio di diagramma temporale. La stazione *A* è la prima a spedire un frame. Mentre *A* spedisce, le stazioni *B* e *C* diventano pronte per spedire. Vedono che il canale è occupato e attendono che si liberi. Poco dopo che *A* riceve un acknowledgement, il canale diventa libero. Tuttavia, invece di spedire subito un frame e farlo collidere, entrambe *B* e *C* eseguono un random backoff. *C* sceglie un backoff breve, perciò spedisce per primo. *B* mette in pausa il conto alla rovescia quando rileva che *C* sta utilizzando il canale e ricomincia quando *C* ha ricevuto un acknowledgement. *B* completa presto il backoff e spedisce il suo frame.

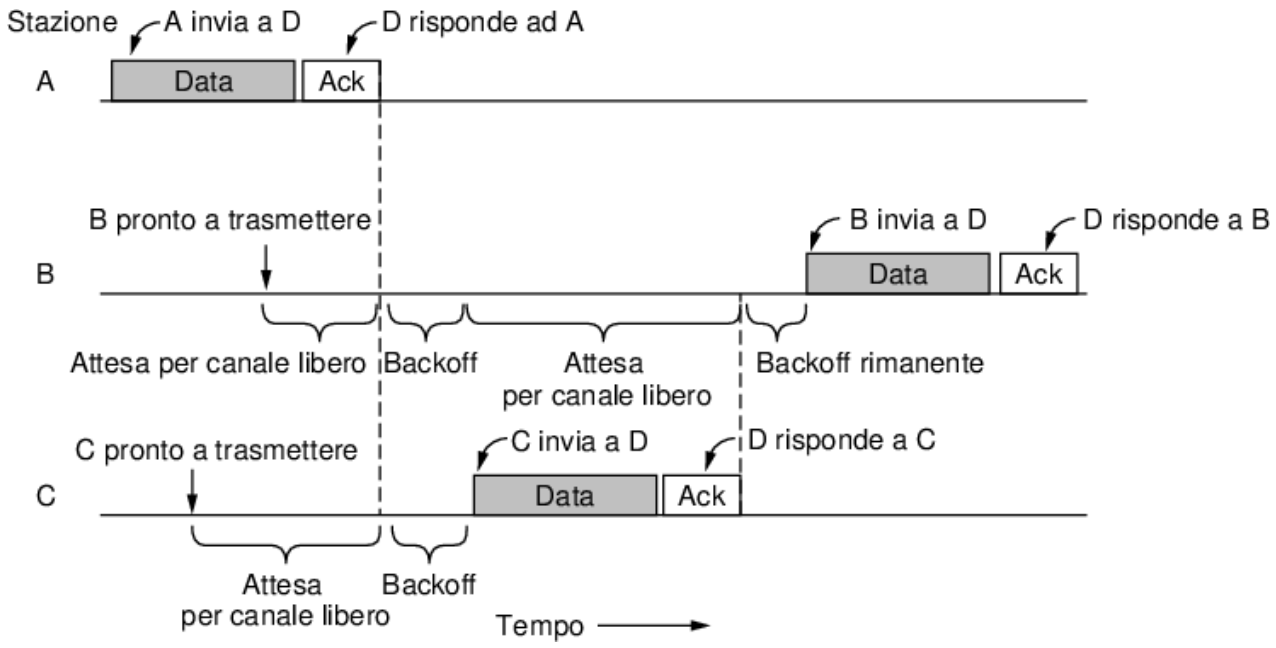


Figura 4.25 Invio di un frame con CSMA/CA.

Dal confronto con Ethernet emergono due differenze principali. Primo, far partire presto i backoff aiuta a evitare le collisioni. Questa elusione è utile in quanto le collisioni sono costose, poiché anche se se ne verifica una, l'intero frame viene trasmesso. Secondo, poiché non possono essere rilevate, gli acknowledgement sono usati per dedurre le collisioni.

Questo modo di operare è chiamato **DCF** (*distributed coordination function*) perché ogni stazione agisce indipendentemente, senza alcun controllo centrale. Lo standard include anche un modo di operare opzionale chiamato **PCF** (*point coordination function*) in cui l'AP controlla tutta l'attività nella sua cella, proprio come la stazione base di una rete cellulare. Tuttavia, PCF non è utilizzata nella pratica, perché normalmente non c'è modo di prevenire la trasmissione di traffico concorrente da parte di stazioni in una rete vicina.

Il secondo problema è che i raggi di trasmissione di stazioni diverse possono essere differenti. Con un cavo, il sistema è progettato in maniera tale che tutte le stazioni possano sentirsi a vicenda. Con la complessità della propagazione elettromagnetica questa situazione non vale per le stazioni wireless. Di conseguenza, possono sorgere situazioni come il problema del terminale nascosto menzionato precedentemente e illustrato ancora nella Figura 4.26(a). Poiché non tutte le stazioni sono a reciproca portata radio, le trasmissioni che avvengono in una parte di una cella potrebbero non essere ricevute in qualche altro punto nella stessa cella. In questo esempio, la stazione *C* sta trasmettendo alla stazione *B*. Se *A* rileva la portante sul canale, non sentirà nulla e dedurrà erroneamente di poter iniziare a trasmettere a *B*. Questa decisione genera una collisione.

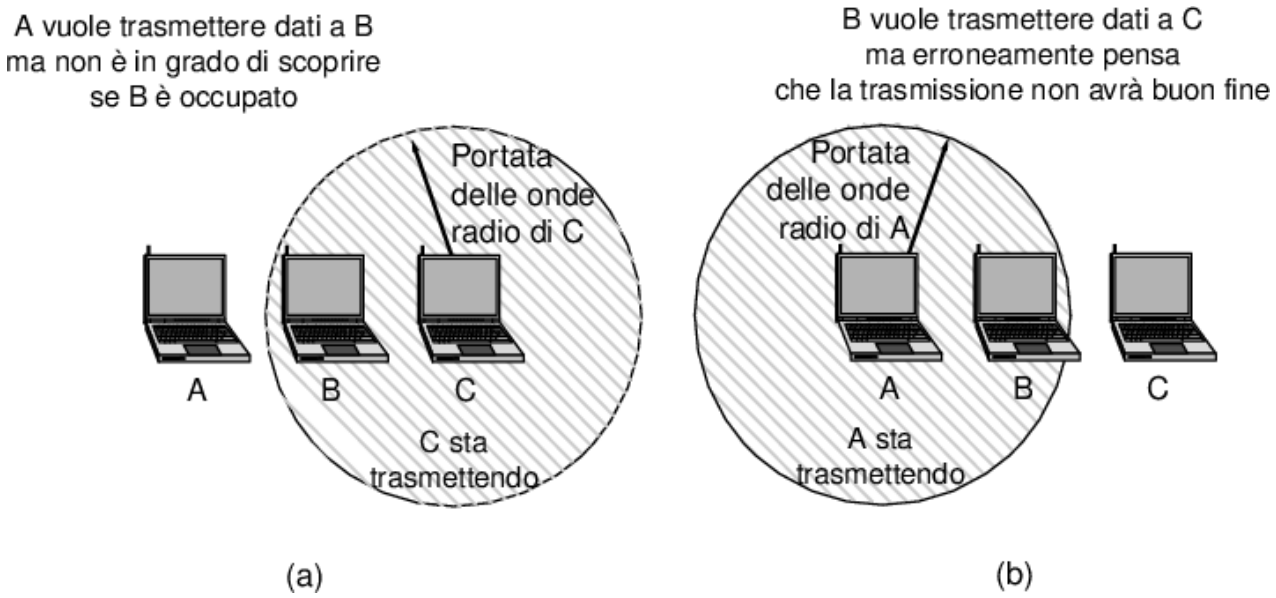


Figura 4.26 (a) Il problema del terminale nascosto. (b) Il problema del terminale esposto.

La situazione inversa è il problema del terminale esposto, illustrato nella Figura 4.26(b). Qui, *B* vuole inviare dati a *C*, quindi ascolta il canale; quando sente una trasmissione, conclude erroneamente di non poterlo fare anche se *A* potrebbe in effetti trasmettere a *D* (non visibile). Questa decisione spreca un'opportunità di trasmissione.

Per ridurre ambiguità su quale stazione stia inviando dati, 802.11 definisce la rilevazione del canale come composta sia dalla rilevazione fisica sia da quella virtuale. La rilevazione fisica semplicemente controlla il mezzo trasmissivo per capire se c'è un segnale valido. Con la rilevazione virtuale ogni stazione registra quando il canale è occupato tenendo traccia dei **NAV** (*network allocation vector,* vettore di allocazione della rete). Ogni frame possiede un campo NAV che indica quanto tempo sia necessario per completare la sequenza di cui il frame fa parte. Le stazioni che vedono il frame sapranno che il canale resterà occupato per il periodo di tempo indicato dal NAV, indipendentemente dal fatto che sia rilevato un segnale fisico. Per esempio, il NAV di un frame di dati include il tempo necessario a spedire un acknowledgement. Tutte le stazioni che vedono il frame di dati ritarderanno l'invio di dati durante il periodo di acknowledgement, che riescano o meno a vedere il messaggio di acknowledgement.

Un meccaniscmo opzionale di RTS/CTS utilizza il NAV per impedire che i terminali spediscano frame contemporaneamente a terminali nascosti, come rappresentato nella Figura 4.27. In questo esempio, *A* vuole spedire dati a *B*. *C* è una stazione nel raggio di copertura di *A* (e probabilmente anche in quello di *B*, ma non importa). *D* è una stazione nel raggio di copertura di *B,* ma non in quello di *A*. Il protocollo inizia quando *A* decide di voler spedire dati a *B*. *A* inizia spedendo un frame RTS a *B* per richiedere il permesso di spedirgli un frame. Se *B* riceve la richiesta, risponde con un frame CTS per indicare che il canale è libero per la spedizione. Una volta ricevuto il CTS, *A* spedisce il frame e fa partire un timer per l'acknowledgement (*ACK timer*). Dopo aver ricevuto

correttamente il frame di dati, *B* risponde con un acknowledgement, completando lo scambio. Se l'ACK timer di *A* scade prima dell'arrivo dell'acknowledgement viene ipotizzata una collisione e l'intero protocollo ricomincerà di nuovo dopo un backoff.

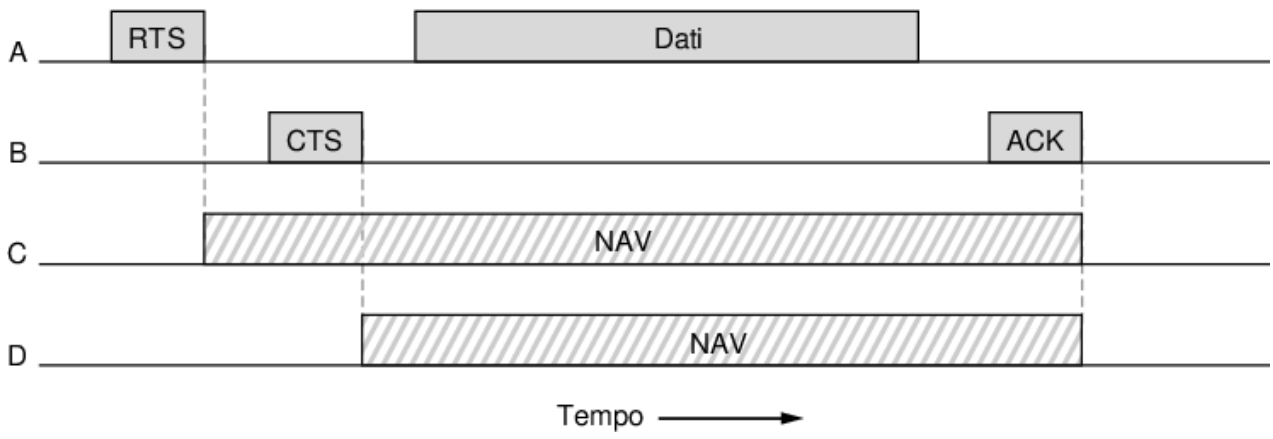


Figura 4.27 Il controllo del canale virtuale mediante CSMA/CA.

Consideriamo ora questo scambio dal punto di vista di *C* e *D*. *C* è nel raggio di copertura di *A*, quindi potrebbe ricevere il frame RTS. Se lo riceve, capisce che qualcuno presto spedirà dati. Dalle informazioni fornite dalla richiesta RTS potrà fare una stima del tempo necessario alla sequenza, incluso l'acknowledgement finale. Quindi, per il bene di tutti, desiste dallo spedire qualsiasi cosa finché non sarà terminato lo scambio. Lo fa aggiornando le sue informazioni interne sui NAV per indicare che il canale è occupato, come si vede dalla Figura 4.27. *D* non vede il frame RTS, ma vede quello CTS, quindi anche lui aggiorna il suo stato interno sui NAV. Si noti che i segnali NAV non sono trasmessi; sono solo note interne per rimanere inattivi per un certo periodo di tempo.

Tuttavia, mentre RTS/CTS sembra in teoria valida, è una di quelle architetture che hanno dimostrato di valere poco nella pratica. Sono diversi i motivi per cui viene utilizzata raramente. Non è efficace con i frame corti (che vengono spediti al posto degli RTS) e con gli AP (che chiunque può raggiungere, per definizione). Nelle altre situazioni, rallenta solo le operazioni. RTS/CTS in 802.11 è un po' diversa rispetto al protocollo MACA che abbiamo visto nel Paragrafo 4.2, perché chiunque veda l'RTS o il CTS rimane inattivo per tutto il tempo per permettere all'acknowledgement di arrivare a destinazione senza collisioni. Per questo motivo non è efficace con i terminali esposti come lo era MACA, ma solo con i terminali nascosti. Il più delle volte i terminali nascosti sono pochi e CSMA/CA si occupa già di loro rallentando le stazioni che trasmettono senza successo, qualunque ne sia la causa, per rendere più probabile che le trasmissioni vadano a buon fine.

CSMA/CA con rilevazione fisica e virtuale è il cuore del protocollo 802.11. Tuttavia, ci sono diversi altri meccanismi sviluppati per funzionare con esso. Ognuno di essi fu ispirato dalle necessità di situazioni reali, quindi li guarderemo brevemente.

La prima necessità che prenderemo in considerazione è l'affidabilità. Al contrario delle reti cablate, le reti wireless sono soggette a rumore e inaffidabili a causa dell'interferenza da parte di altri tipi di apparecchiature, come i forni a microonde, poiché anche loro utilizzano le bande ISM. L'uso di acknowledgement e ritrasmissioni è di poco aiuto se la probabilità di consegna di un frame è bassa già di per sè.

La strategia principale adottata per aumentare le trasmissioni che avvengono con successo è di diminuire il tasso di trasmissione. Tassi più bassi utilizzano modulazioni più robuste e che più probabilmente verranno ricevute correttamente per un dato rapporto segnale-rumore. Se vengono persi troppi frame una stazione può abbassare il tasso di trasmissione. Se i frame vengono consegnati con poche perdite una stazione può provare occasionalmente un tasso più elevato per vedere se può essere utilizzato.

Un'altra strategia per migliorare la probabilità di un frame di raggiungere integro la destinazione è spedire frame più corti. Se la probabilità di ogni bit di essere soggetto a un errore è p, la probabilità di un frame di n bit di essere ricevuto correttamente è $(1 - p)^n$. Per esempio, con $p = 10^{-4}$ la probabilità di ricevere correttamente un frame Ethernet completo (12,144 bit) è meno del 30%. La maggior parte dei frame andrà persa. Se i frame sono lunghi solo un terzo (4048 bit) due terzi di essi saranno ricevuti correttamente. Molti frame arriveranno a destinazione e saranno necessarie meno ritrasmissioni.

Frame più corti possono essere implementati riducendo la dimensione massima del messaggio accettato dal livello di rete. In alternativa, 802.11 permette di dividere i frame in pezzi più piccoli, chiamati **frammenti**, ognuno col proprio checksum. La dimensione di un frammento non è indicata dallo standard, ma è un parametro che può essere aggiustato dall'AP. I frammenti sono numerati individualmente e soggetti ad acknowledgement tramite un protocollo *stop and wait* (cioè, il mittente non può spedire il fammento $k + 1$ finché non ha ricevuto l'acknowledgement per il frammento k). Una volta che il controllo del canale è stato acquisito, molteplici frammenti vengono spediti in una rapida sequenza. Partono uno dopo l'altro con in mezzo un acknowledgement (e forse ritrasmissioni), finché l'intero frame non sia stato spedito con successo o sia stato raggiunto il tempo massimo di trasmissione permesso. Il meccanismo NAV rende le altre stazioni inattive solo fino al successivo acknowledgement, ma un altro meccanismo (si veda in seguito) è utilizzato per consentire la spedizione di una raffica di frammenti senza che le altre stazioni spediscano un frame nel bel mezzo.

La seconda esigenza di cui si discuterà è il risparmio energetico. La durata della batteria è sempre un problema con i dispositivi mobili wireless. Lo standard 802.11 fa attenzione al problema della gestione dell'energia, per cui i client non hanno bisogno di usare energia quando non hanno informazioni né da spedire né da ricevere.

Il meccanismo base per il risparmio energetico si basa sui **beacon frame**. I beacon sono trasmissioni broadcast periodiche lanciate dall'AP (per esempio ogni 100 ms). I frame rendono pubblica la presenza dell'AP ai client e trasportano parametri di configurazione del sistema, come l'identificatore dell'AP, l'ora, quanto tempo passerà prima del prossimo beacon e impostazioni di sicurezza.

I client possono impostare un bit di gestione energetica nei frame che spediscono all'AP per indicare che stanno per entare in **modalità power-save** (*modalità a consumo ridotto/a conservazione di energia*). In questo modo, il client può riposare e l'AP metterà in un buffer il traffico a esso destinato. Per controllare l'eventuale traffico in entrata, il client si risveglia a ogni beacon e controlla una mappa di traffico inviata come parte del beacon. Questa mappa indica al client se ci sia traffico per lui nel buffer. In caso affermativo, il client spedisce un messaggio sonda all'AP, che spedisce il traffico nel buffer. Il client può quindi tornare a dormire finché non sarà spedito il successivo beacon.

Un altro meccanismo di risparmo energetico, chiamato **APSD** (*automatic power save delivery*), fu aggiunto a 802.11 nel 2005. Con questo nuovo meccanismo, l'AP mette i frame in un buffer e li spedisce al client appena il client ha spedito dei frame all'AP. Il client può quindi rimanere inattivo finché non avrà altro traffico da spedire (e ricevere). Questo meccanismo funziona bene con applicazioni come VoIP che ha traffico frequente in entrambe le direzioni. Per esempio, un telefono wireless VoIP potrebbe usarlo per spedire e ricevere frame ogni 20 ms, molto più frequentemente dell'intervallo dei beacon di 100 ms, mentre nel frattempo riposa.

La terza e ultima esigenza che esamineremo è la **qualità del servizio** (QoS, *quality of service*). Quando il traffico VoIP dell'esempio precedente compete con quello peer-to-peer, il traffico VoIP ne soffrirà: sarà ritardato a causa della contesa con il traffico peer-to-peer a larga banda, anche se l'ampiezza di banda di VoIP è bassa. Questi ritardi probabilmente danneggerebbero le chiamate vocali; per prevenire questo problema sarebbe meglio che il traffico VoIP potesse scavalcare il traffico peer-to-peer, in quanto con priorità più alta.

IEEE 802.11 ha un meccanismo intelligente per fornire questo tipo di qualità di servizio, introdotto nel 2005 come un insieme di estensioni con il nome di 802.11e. Funziona estendendo CSMA/CA con intervalli scelti con cura tra i frame. Una volta che il frame è stato spedito è richiesto un certo intervallo di tempo libero prima che una stazione possa spedire un frame per verificare che il canale non sia più utilizzato. Il trucco sta nel definire diversi intervalli di tempo per diversi tipi di frame.

Nella Figura 4.28 sono illustrati cinque intervalli. L'intervallo tra frame di dati è chiamato **DIFS** (*DCF interframe spacing*). Ogni stazione può cercare di acquisire il controllo del canale per spedire un nuovo frame dopo che il mezzo trasmissivo è rimasto libero per DIFS. Valgono le normali regole di contesa e in caso di collisione potrebbe essere necessario il backoff esponenziale binario. L'intervallo più breve è **SIFS** (*short interframe spacing*); viene usato per permettere a coloro che partecipano a uno scambio di messaggi di poter partire per primi. Alcuni esempi includono il permettere al ricevente l'invio di acknowledgement, altre sequenze di frame di controllo come RTS e CTS o il consentire al mittente di trasmettere una raffica di frammenti. Spedire il frammento successivo dopo aver aspettato un solo SIFS impedisce che un'altra stazione possa intromettersi con un frame nel bel mezzo di uno scambio.

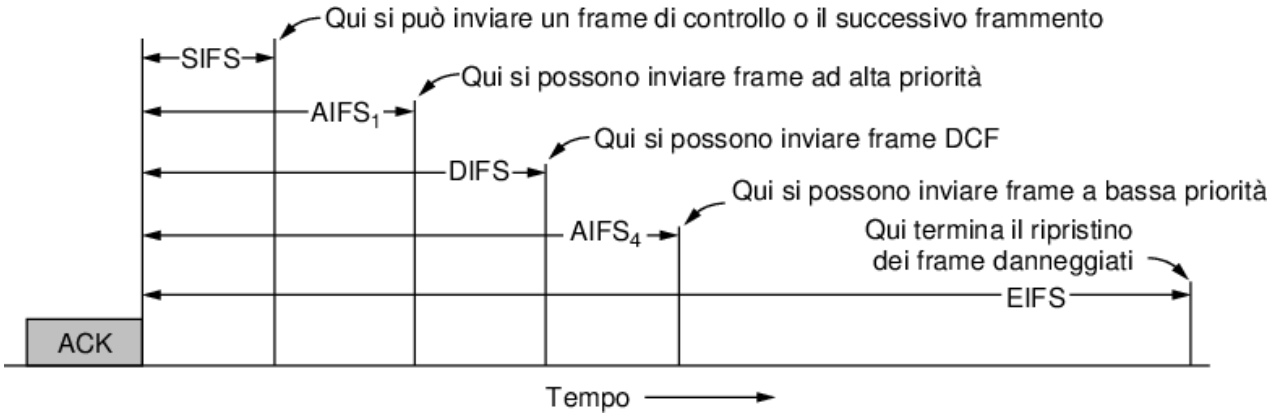


Figura 4.28 Intervalli tra frame in 802.11.

I due intervalli **AIFS** (*arbitration interframe space*) mostrano esempi di due differenti livelli di priorità. L'intervallo breve, $AIFS_1$, è minore di DIFS, ma più lungo di SIFS. Può essere utilizzato dall'AP per spostare voce o altro traffico ad alta priorità all'inizio della coda. L'AP attenderà un periodo più breve prima di spedire il traffico voce e quindi lo spedirà prima del normale traffico. Il lungo intervallo, $AIFS_4$, è maggiore di DIFS ed è utilizzato per il traffico di sottofondo (*traffico di background*) che può essere differito finché non sia stato smaltito il traffico regolare. L'AP aspetterà per un intervallo più lungo prima di spedire tale traffico, dandogli la possibilità di essere trasmesso prima. L'implementazione completa della qualità di servizio definisce quattro diversi livelli di priorità con diversi parametri di backoff, come pure diversi parametri di inattività.

L'ultimo intervallo di tempo, **EIFS** (*extended interframe spacing*), è utilizzato solo dalle stazioni che hanno ricevuto un frame difettoso o sconosciuto per segnalare il problema. L'idea è che, poiché il ricevente non è a conoscenza di ciò che succede, deve aspettare un po' per evitare di interferire con uno scambio di messaggi già in atto tra due stazioni.

Un altro risvolto delle estensioni della qualità di servizio è la nozione di **TXOP** o **opportunità di trasmissione**. Il CSMA/CA originale permette alle stazioni di spedire un frame per volta. Questa architettura andava bene finché non crebbe la gamma dei tassi di trasmissione. Con 802.11a/g, una stazione può inviare a 6 Mbps e un'altra a 54 Mbps. Entrambe hanno la possibilità di inviare un frame, ma la stazione a 6 Mbps impiega nove volte il tempo (ignorando gli overhead fissi) trascorso dalla stazione a 54 Mbps per spedire il suo frame. Questa disparità ha lo sfortunato effetto di ridurre la velocità di un mittente veloce, in competizione con un mittente lento, a circa la velocità del mittente lento. Per esempio, ancora ignorando gli overhead fissi, quando inviano da soli i mittenti a 6 Mbps e a 54 Mbps raggiungeranno il loro tasso di trasferimento dati, ma quando inviano insieme entrambi raggiungeranno in media i 5,4 Mbps. È una grande penalizzazione per il mittente veloce. Questo problema è conosciuto come **rate anomaly** (Heusse et al., 2003).

Con le opportunità di trasmissione ogni stazione ottiene un uguale quantità di tempo di trasmissione (*airtime*), non un uguale numero di frame. Le stazioni che inviano a un tasso più elevato per il loro tempo di trasmissione otterranno una capacità totale più elevata. Nel nostro

esempio, quando spediscono insieme i mittenti a 6 Mbps e a 54 Mbps otterranno ora rispettivamente 3 Mbps e 27 Mbps.

Struttura del frame di 802.11

Lo standard 802.11 definisce tre diverse classi di frame: dati, controllo e gestione. Ognuna ha un'intestazione composta da una varietà di campi utilizzati all'interno del sottolivello MAC. Inoltre, ci sono anche delle intestazioni usate dal livello fisico, ma hanno per lo più a che fare con le tecniche di modulazione adottate, perciò non saranno analizzate nel corso di questo paragrafo.

Prenderemo come esempio il formato dei frame di dati, rappresentato nella Figura 4.29. Prima compare il campo *Frame control*, costituito da 11 sottocampi. Il primo di questi è *Protocol version*, impostato a 00. È lì per permettere alle future versioni di 802.11 di operare contemporaneamente nella stessa cella. Quindi ci sono i campi *Type* (tipo) (con possibili valori dati, controllo e gestione) e *Subtype* (sottotipo) (ad esempio RTS o CTS); per un frame di dati regolare (senza qualità di sevizio) sono fissati a 10 e 0000 (in binario). I bit *To DS* (verso DS) e *From DS* (da DS) sono fissati a indicare se il frame stia andando o venendo dalla rete connessa all'AP, chiamata sistema di distribuzione. Il bit *More fragments* (ulteriori frammenti) significa che seguiranno più fammenti. Il bit *Retry* (riprova) indica la ritrasmissione di un frame spedito in precedenza. Il bit *Power management* (gestione energia) indica che il mittente sta andando in modalità power-save. Il bit *More data* (ulteriori dati) indica che il mittente ha altri dati per il ricevente. Il bit *Protected Frame* (frame protetto) indica che il corpo del frame è stato criptato per sicurezza. Discuteremo la sicurezza brevemente nel paragrafo successivo. Infine, il bit *Order* (in ordine) indica al ricevente che il livello superiore si aspetta che la sequenza di frame arrivi rigorosamente in ordine.

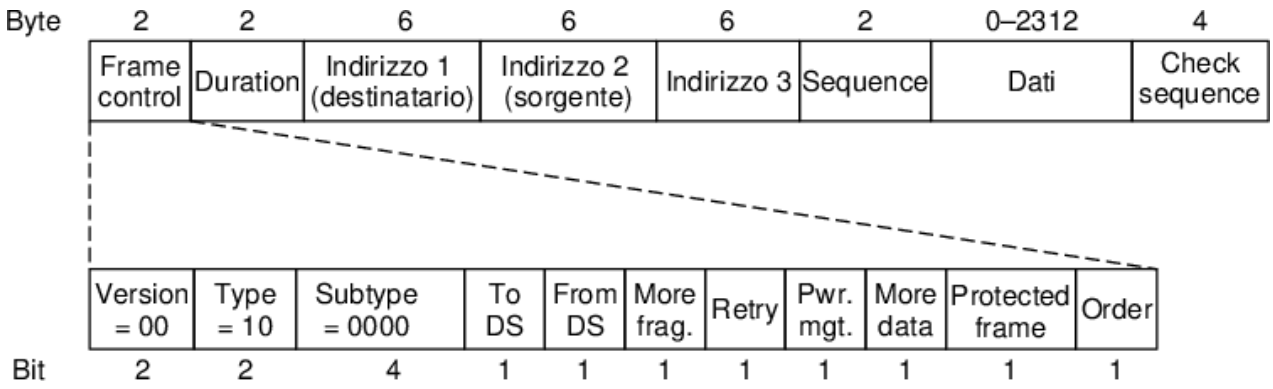


Figura 4.29 Formato del frame di dati 802.11.

Il secondo campo del frame di dati, *Duration* (durata), indica per quanto tempo il frame e il suo acknowledgement occuperanno il canale, misurato in microsecondi. È presente in tutti i tipi di frame, inclusi i frame di controllo, ed è ciò che le stazioni utilizzano per gestire il meccanismo del NAV.

Poi ci sono gli indirizzi. I frame di dati spediti da o per un AP hanno tre indirizzi, tutti in formato standard IEEE 802. Il primo indirizzo è il ricevente e il secondo è il mittente. Sono ovviamente necessari, ma a che cosa serve il terzo? Ricordate che un AP è semplicemente un punto di staffetta per i frame quando viaggiano tra un client e un altro punto della rete. Il terzo indirizzo corrisponde a questo distante punto finale.

Il campo *Sequence* (sequenza) numera i frame in modo da poter individuare duplicati. Dei 16 bit disponibili, 4 identificano il frammento e 12 trasportano un numero che viene incrementato con ogni nuova trasmissione. Il campo *Dati* contiene il payload, fino a 2312 byte. I primi byte di questo campo sono in formato conosciuto come **LLC** (*logical link control*). Questo livello è il collegamento che identifica il protocollo del livello superiore (ad esempio IP) a cui va passato il contenuto del segmento. Infine c'è la *Frame check sequence*, che è lo stesso CRC a 32 bit visto nel Paragrafo 3.2.2.

I frame di gestione hanno lo stesso formato dei frame di dati, più un formato per la porzione dati che varia a seconda di *Subtype* (ad esempio, parametri nei frame di beacon). I frame di controllo sono brevi. Come tutti i frame hanno i campi *Frame control*, *Duration*, e *Frame check sequence*. Tuttavia, potrebbero anche avere un solo indirizzo e nessuno spazio per i dati. La maggior parte delle informzioni chiave viene trasmessa nel campo *Subtype* (ACK, RTS e CTS).

Servizi

Lo standard 802.11 definisce i servizi che i client, gli access point e la rete che li connette devono gestire per essere una LAN wireless conforme allo standard. Questi servizi si possono classificare in diversi gruppi.

Il **servizio di associazione** viene usato dalle stazioni mobili per connettersi agli AP. Tipicamente è utilizzato non appena una stazione si muove entro la portata radio dell'AP. Quando arriva, la stazione viene a conoscenza dell'identità e delle funzionalità dell'AP dai frame di beacon o interrogando direttamente l'AP. Le funzionalità comprendono i tassi trasmissivi supportati, le modalità di sicurezza, la gestione energetica, supporto per la qualità di servizio e altro ancora. La stazione invia una richiesta per associarsi con l'AP; questo ha la facoltà di accettare o rifiutare la richiesta.

Le **riassociazioni** permettono a una stazione di cambiare il suo AP preferenziale. Questo servizio è utile alle stazioni mobili che si spostano da un AP all'altro nella stessa LAN 802.11 estesa, come un handover in una rete cellulare. Se utilizzato correttamente, non sarà perso alcun dato come conseguenza di un passaggio. (Ma 802.11, come Ethernet, è solo un servizio *best-effort*).[3] Sia la stazione che l'AP si possono *disassociare*, rompendo le loro relazioni. Una stazione dovrebbe utilizzare questo servizio prima di spegnersi o di lasciare la rete. L'AP può utilizzarlo prima di disattivarsi per manutenzione.
Le stazioni si devono anche *autenticare* prima di poter spedire frame attraverso l'AP, ma l'**autenticazione** viene gestita in modi diversi a seconda dello schema di sicurezza scelto. Se la rete 802.11 è "aperta", a chiunque è permesso di utilizzarla. Diversamente, per autenticarsi sono necessarie delle credenziali. Lo schema raccomandato, chiamato **WPA2** (*WiFi protected access* 2), implementa la sicurezza come definito nello standard 802.11i. (Il semplice WPA è uno schema provvisorio che implementa un sottoinsieme di 802.11i; lo salteremo per andare direttamente alla

versione completa). Con WPA2 l'AP può parlare con un server di autenticazione che ha un database di utenti e password per determinare se la stazione abbia il pemesso di accesso alla rete. In alternativa potrà essere configurata una chiave segreta condivisa, un termine estroso per indicare una password di rete. Vengono scambiati molti frame tra stazione e AP in un botta e risposta che permette alla stazione di dimostrare di avere le credenziali giuste. Questo scambio avviene dopo l'associazione.

Lo schema utilizzato precedentemente a WPA è chiamato **WEP** (*wired equivalent privacy*). In questo schema l'autenticazione con una chiave condivisa avviene prima dell'associazione. Tuttavia, il suo utilizzo è sconsigliato a causa dei difetti di progettazione che rendono WEP facile da compromettere. La prima dimostrazione pratica dell'inviolabilità di WEP si ebbe quando Adam Stubblefield partecipò a uno stage estivo presso AT&T (Stubblefield et al., 2002). In una sola settimana, in gran parte spesa per ottenere il pemesso dalla dirigenza per l'acquisto di schede WiFi necessarie per gli esperimenti, fu in grado di implementare e provare un attacco. Il software per compromettere le password WEP è ora liberamente disponibile.

Una volta che dei frame raggiungono l'AP, il **servizio di distribuzione** determina come instradarli. Se la destinazione è vicina all'AP, i frame possono essere spediti direttamente via etere. Altrimenti, dovranno essere trasmessi attraverso la rete cablata. Il **servizio di integrazione** gestisce ogni traduzione necessaria a un frame per essere spedito all'esterno della LAN 802.11 o per farlo arrivare da fuori della LAN 802.11. Il caso più comune qui è quello in cui si connette la LAN wireless a Internet.

La trasmissione dei dati è il punto centrale, quindi 802.11 provvede naturalmente al **servizio di spedizione** dei dati. Questo servizio permette alle stazioni di trasmettere e ricevere dati utilizzando i protocolli descritti precedentemente in questo capitolo. Poiché 802.11 prende come modello Ethernet e le trasmissioni di Ethernet non sono garantite affidabili al 100%, neanche la trasmissione su 802.11 lo è. I livelli superiori devono occuparsi di individuare e correggere gli errori.

Wireless è un sistema broadcast. Se informazioni spedite su una LAN wireless devono rimanere confidenziali devono essere cifrate. Questo obiettivo viene ottenuto per mezzo di un **servizio di privacy** che ne gestisce i dettagli. L'algoritmo di cifratura WPA2 si basa su **AES** (*advanced encryption standard*), uno standard governativo americano approvato nel 2002. Le chiavi utilizzate per la cifratura sono determinate durante la procedura di autenticazione.

Per gestire traffico con diverse priorità c'è un servizio di **schedulazione di traffico con QoS** (*QoS traffic scheduling*). Utilizza i protocolli che abbiamo descritto per dedicare un trattamento preferenziale al traffico voce e video rispetto al traffico best-effort e di background. Un servizio accessorio prevede anche la sincronizzazione del clock a un livello superiore; in questo modo le stazioni possono coordinare le loro azioni, il che può essere utile per la gestione di contenuti multimediali.

Infine ci sono due servizi che permettono alle stazioni di gestire il loro utilizzo dello spettro. Il **servizio di controllo dell'energia trasmissiva** (*transmit power control*) dà alle stazioni le informazioni necessarie per rispettare i limiti normativi sull'energia trasmessa, limiti che variano da regione a regione. Il **servizio di selezione dinamica della frequenza** (*dynamic frequency selection*) dà alle stazioni le informazioni necessarie per evitare di trasmettere nelle frequenze all'interno della banda dei 5 GHz utilizzate da radar nelle vicinanze.

Con questi servizi 802.11 fornisce un ricco insieme di funzionalità per connettere client mobili a Internet. È stato un enorme successo e lo standard è stato ripetutamente modificato per aggiungere ulteriori funzionalità. Per orientarsi sul presente e sulle prospettive future dello standard si veda Hiertz et al. (2010).

4.5 Wireless a banda larga

Siamo rimasti al chiuso per troppo tempo; usciamo all'aperto dove si svolgono un sacco di attività interessanti riguardo all'ultimo miglio. Dopo la deregolamentazione del sistema telefonico avvenuta in molti paesi, i concorrenti delle aziende telefoniche consolidate hanno ora la possibilità di offrire servizi di telefonia locale e di accesso a Internet ad alta velocità. La domanda è sicuramente abbondante, ma il problema è che stendere fibre ottiche o cavi coassiali per milioni di abitazioni e uffici ha un costo proibitivo. Che cosa può fare un concorrente a questo punto?

La risposta è il wireless a banda larga. Innalzare una grande antenna su una collina fuori città è molto più semplice ed economico che scavare numerose buche e tirare cavi. Per questo motivo le aziende hanno cominciato a sperimentare la possibilità di un servizio di comunicazione wireless multimegabit in grado di trasmettere voce, Internet, film e così via.

Per stimolare il mercato, IEEE formò un gruppo per standardizzare una rete metropolitana wireless a banda larga. Il successivo numero disponibile nella numerazione 802 era **802.16**, quindi lo standard ottenne questo numero. Informalmente la tecnologia è chiamata **WiMAX** (*worldwide interoperability for microwave access*). Useremo i termini 802.16 e WiMAX come sinonimi.

Il primo standard 802.16 fu approvato nel dicembre 2001. Le prime versioni fornivano un ultimo miglio wireless tra punti fissi tutti visibili (*line-of-sight*) a vicenda. Quest'architettura cambiò presto per far diventare WiMAX un'alternativa più competitiva con cable e DSL per l'accesso a Internet. A gennaio del 2003 802.16 fu revisionato per supportare collegamenti senza linea visibile (*non-line-of-sight*) utilizzando la tecnologia OFDM a frequenze tra 2 GHz e 10 GHz. Questo cambiamento ne facilitò lo sviluppo, sebbene le stazioni erano ancora punti fissi. L'avvento delle reti cellulari 3G rappresentava un minaccia, promettendo elevati tassi di trasmissione per i dati *e* mobilità. Nel dicembre 2005, sulla scia di questa sollecitazione, 802.16 fu nuovamente migliorato per consentire la mobilità a velocità veicolare. L'accesso a Internet mobile a banda larga è l'obiettivo dello standard corrente: IEEE 802.16-2009.

Come gli altri standard 802, 802.16 fu pesantemente influenzato dal modello OSI, inclusi (sotto)livelli, terminologia, servizi di base e altro. Sfortunatamente, proprio come OSI, esso è piuttosto complicato. Il **WiMAX Forum** fu quindi creato per definire sottoinsiemi interoperabili dello standard per offerte commerciali. Nei paragrafi successivi faremo una breve descrizione di alcune linee guida delle forme più comuni di interfaccia aerea per 802.16, ma questa trattazione è lontana dall'essere completa e tralascia molti dettagli. Per maggiori informazioni su WiMAX e wireless a banda larga in generale si consulti Andrews et al. (2007).

Confronto tra 802.16, 802.11 e 3G

A questo punto starete pensando: perché progettare un nuovo standard? Perché non utilizzare 802.11 o 3G? In effetti, WiMAX combina aspetti sia di 802.11 sia di 3G, rendendolo molto più simile alla tecnologia 4G.

Come 802.11, WiMAX si occupa unicamente di collegare dei dispositivi a Internet a una velocità nell'ordine dei megabit al secondo in modalità wireless, invece di utilizzare cable o DSL. Gli apparecchi possono essere mobili o almeno portatili. WiMAX non ha iniziato aggiungendo dati a bassa velocità a fianco di voce come le reti cellulari; 802.16 fu progettato per trasportare pacchetti IP nell'etere e per connettersi a una rete cablata basata su IP facendo meno confusione possibile. I pacchetti possono portare traffico peer-to-peer, chiamate VoIP o contenuti multimediali in streaming per supportare una gamma di applicazioni. Ancora come 802.11 esso è basato sulla tecnologia OFDM per assicurare buone prestazioni a dispetto del degrado del segnale wireless come il multipath fading e sulla tecnologia MIMO per ottenere elevate capacità trasmissive.

Tuttavia, WiMAX è più simile a 3G (e quindi diverso da 802.11) in diversi aspetti chiave. Il principale problema tecnico è ottenere una grande capacità di trasferimento grazie a un uso efficiente dello spettro, in maniera tale da permettere a un gran numero di abbonati in una determinata zona di copertura di avere un elevato throughput. Le distanze tipiche sono almeno dieci volte maggiori di quelle di una rete 802.11; quindi, le stazioni base WiMAX sono più potenti degli AP di 802.11. Per gestire segnali più deboli su distanze più lunghe la stazione base utilizza più energia, antenne migliori e fa più elaborazioni per la gestione degli errori. Per massimizzare il throughput, le trasmissioni sono accuratamente pianificate dalla stazione base per ogni specifico abbonato; l'utilizzo dello spettro non è lasciato al caso con CSMA/CA, che potrebbe sprecare risorse con delle collisioni.

L'uso di spettro che necessita di una licenza è il caso più probabile per WiMAX, tipicamente attorno ai 2,5 GHz negli Stati Uniti. L'intero sistema è sostanzialmente più ottimizzato di 802.11. Questa complessità è giustificata, considerando le grandi quantità di denaro in gioco per via dello spettro con licenza. Diversamente da 802.11, il risultato è un servizio gestito e affidabile con un buon supporto per la qualità di servizio.

Alla luce di tutte queste caratteristiche, 802.16 è molto simile alle reti cellulari 4G che sono in via di standardizzazione sotto il nome di **LTE** (*long term evolution*). Mentre le reti cellulari 3G si basano su CDMA e supportano voce e dati, le reti cellulari 4G si baseranno su OFDM con MIMO e il loro obiettivo saranno i dati, mentre la voce sarà solo un'applicazione. Sembra che WiMAX e 4G siano in rotta di collisione in termini di tecnologia e applicazioni. Forse questa convergenza non sorprende, dato che Internet è la killer application e OFDM e MIMO sono le tecnologie meglio note per l'uso efficiente dello spettro.

Architettura e stack di protocolli di 802.16

L'architettura di 802.16 è rappresentata nella Figura 4.30. Le stazioni base si connettono direttamente alla dorsale del provider che a sua volta si connette a Internet. Le stazioni base comunicano con le altre stazioni attraverso l'interfaccia aerea. Esistono due tipi di stazioni: quelle dell'abbonato, fissa, (*subscriber station*) e quelle mobili (*mobile station*). Le stazioni degli abbonati rimangono in posizioni fisse, per esempio l'accesso a Internet a banda larga per le abitazioni. Le stazioni mobili possono ricevere servizio mentre si muovono, come nel caso di una macchina equipaggiata con WiMAX.

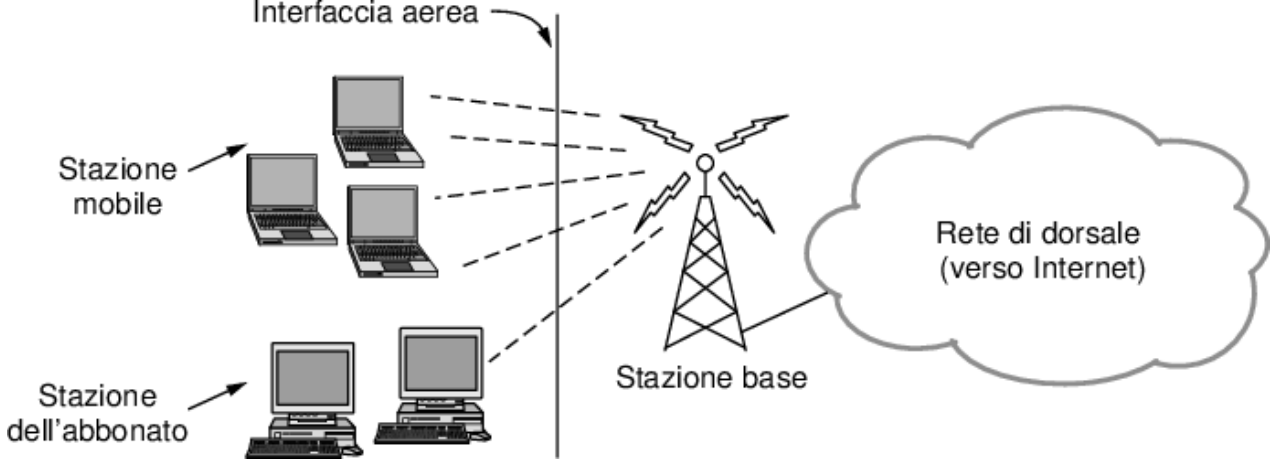


Figura 4.30 Architettura di 802.16.

Lo stack di protocolli di 802.16 utilizzato attraverso l'interfaccia aerea è mostrato nella Figura 4.31. La struttura generale è simile a quella delle reti 802, ma con più sottolivelli. Il livello inferiore gestisce la trasmissione; in questa figura abbiamo mostrato solo le due offerte più popolari di 802.16: WiMAX fisso e mobile; esiste un diverso livello fisico per ogni possibile offerta. Entrambi i livelli operano nello spettro con licenza al di sotto degli 11 GHz e utilizzano OFDM, anche se in modi diversi.

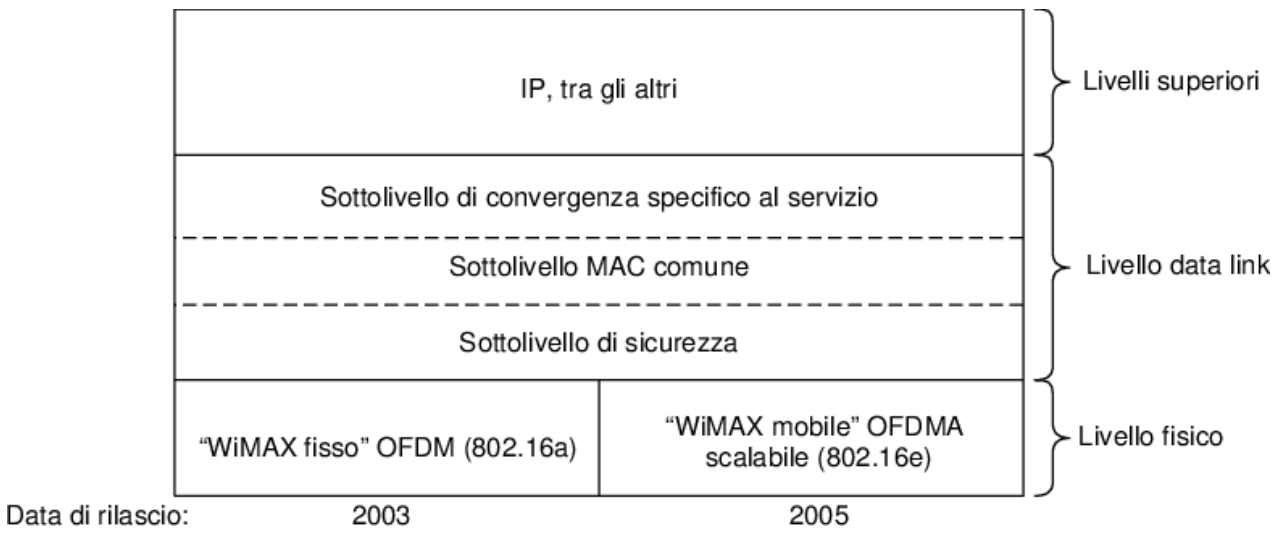


Figura 4.31 Stack di protocolli di 802.16.

Sopra al livello fisico, il livello data link consiste di tre sottolivelli. Quello più in basso si occupa di privacy e sicurezza, molto più cruciale per le reti pubbliche in esterni che per quelle private all'interno di una abitazione; si occupa di amministrare cifratura e decifratura dei dati e la gestione delle chiavi crittografiche, che vedremo più avanti nel Capitolo 8.

A seguire, troviamo un sottolivello MAC comune a tutti i servizi. In questa parte si trovano i protocolli principali, come la gestione del canale. Il modello di funzionamento qui è che la stazione base controlla completamente il sistema: può pianificare i canali di downlink (in ricezione, da base ad abbonato) molto efficientemente e ricopre un ruolo principale nella gestione dei canali di uplink (in trasmissione, da abbonato a base). Un aspetto insolito di questo sottolivello MAC è che, diversamente da quelli degli altri protocolli 802, è completamente orientato alla connessione in modo da fornire le garanzie di qualità di servizio per comunicazioni telefoniche e multimediali.

Il sottolivello di convergenza specifico al servizio prende il posto del sottolivello LLC degli altri protocolli 802. La sua funzione è quella di fornire un'interfaccia al livello di rete. Per facilitare l'integrazione dei diversi tipi di livelli superiori vengono definiti diversi livelli di convergenza. La scelta importante è IP, anche se lo standard definisce anche associazioni per protocolli come Ethernet e ATM. Poiché IP è senza connessione e il sottolivello MAC di 802.16 è orientato alla connessione, questo livello potrà associare tra loro indirizzi e connessioni.

Livello fisico di 802.16

Molte delle installazioni WiMAX utilizzano uno spettro con licenza attorno ai 3,5 o ai 2,5 GHz. Come con 3G, trovare una parte di spettro disponibile è un problema chiave. Per dare una mano in questa direzione, lo standard 802.16 è progettato per essere flessibile: permette di operare tra i 2 e gli 11 GHz. Sono supportati canali di diverse dimensioni; per esempio 3,5 MHz per WiMAX fisso e da 1,25 a 20 MHz per WiMAX mobile.

Le trasmissioni sono spedite su questi canali appena citati usando OFDM, la tecnica descritta nel Paragrafo 2.5.3. Messo a confronto con 802.11, il sistema OFDM di 802.16 è ottimizzato per ottenere il massimo dallo spettro con licenza e trasmissioni su aree molto grandi. Il canale è diviso in più sottoportanti con un tempo di trasmissione per ogni simbolo più lungo, al fine di tollerare una maggiore degradazione del segnale; i parametri di funzionamento di WiMAX sono un numero circa 20 volte maggiori degli equivalenti di 802.11. Per esempio, in WiMAX mobile ci sono 512 sottoportanti per un canale a 5 MHz e il tempo per spedire un simbolo su ogni sottoportante è approssimativamente di 100 ms.

I simboli su ciascuna sottoportante sono inviati usando QPSK, QAM-16 o QAM-64, schemi di modulazione descritti nel Paragrafo 2.5.3. Quando la stazione mobile o dell'abbonato è vicina alla stazione base e il segnale ricevuto ha un elevato rapporto segnale-rumore (SNR) è possibile usare QAM-64 per spedire 6 bit per simbolo. Per raggiungere una stazione distante con un basso SNR è possibile usare QPSK per spedire 2 bit per simbolo. I dati sono innanzitutto codificati per correggere gli errori con un codice convoluzionali (o schemi migliori) come descritto nel Sottoparagrafo 3.2.1. Questa codifica è comune su canali soggetti a molto rumore per tollerare alcuni errori sui bit senza bisogno di ritrasmissioni. I metodi di modulazione e codifica dovrebbero ora suonare familiari, poiché sono utilizzati in molte reti che abbiamo studiato, inclusi 802.11, cable e DSL. Il risultato è che una stazione base può supportare fino a 12,6 Mbps di traffico in ricezione e 6,2 Mbps di traffico in trasmissione per ogni canale da 5 MHz e paio di antenne.

Una cosa che ai progettisti di 802.16 non piacque fu un certo aspetto del funzionamento di GSM e DAMPS: entrambi questi sistemi utilizzano una uguale frequenza di banda per il traffico in upstream e downstream. Essi cioè assumono implicitamente che ci sia tanto traffico in upstream

quanto in downstream. Per la voce, il traffico è per la maggior parte simmetrico, ma per gli accessi a Internet (e certamente per la navigazione Web) c'è n'è spesso molto di più in downstream che in upstream. Il rapporto è spesso di 2:1, 3:1 o anche oltre.

Quindi, per dividere il canale tra le stazioni i progettisti scelsero uno schema flessibile, chiamato **OFDMA** (*orthogonal frequency division multiple access*). Con OFDMA gruppi diversi di sottoportanti possono essere assegnati a stazioni distinte, così che più di una stazione possa spedire o ricevere contemporaneamente. Se stessimo usando 802.11 tutte le sottoportanti sarebbero usate da una sola stazione per trasmettere in qualsiasi momento. La maggiore flessibilità con cui la banda è assegnata può migliorare le prestazioni, poiché una certa sottoportante potrebbe essere troppo debole per un ricevente a causa della propagazione multipath, ma forte per un altro. Le sottoportanti possono essere assegnate alle stazioni che sono in grado di usarle nel modo migliore.

Oltre ad avere un traffico simmetrico, le stazioni di solito alternano spedizione e ricezione. Questo metodo è chiamato **TDD** (*time division duplex*). Il metodo alternativo, con cui una stazione invia e riceve contemporaneamente (su frequenze diverse di sottoportante) è chiamato **FDD** (*frequency division duplex*).

WiMAX permette entrambi i metodi anche se si preferisce usare TDD, poiché è più semplice da implementare e più flessibile.

La Figura 4.32 mostra un esempio di struttura di un frame ripetuta nel tempo. Comincia con un preambolo per sincronizzare tutte le stazioni, seguito da trasmissioni inviate dalla stazione base. Innanzitutto, la stazione base spedisce delle mappe che indicano a tutte le stazioni come sono assegnate sul frame le sottoportanti di ricezione e trasmissione; parleremo quindi di mappe di ricezione e mappe di trasmissione. La stazione base gestisce le mappe, quindi può allocare una diversa ampiezza di banda alle stazioni da un frame al successivo, a seconda delle necessità.

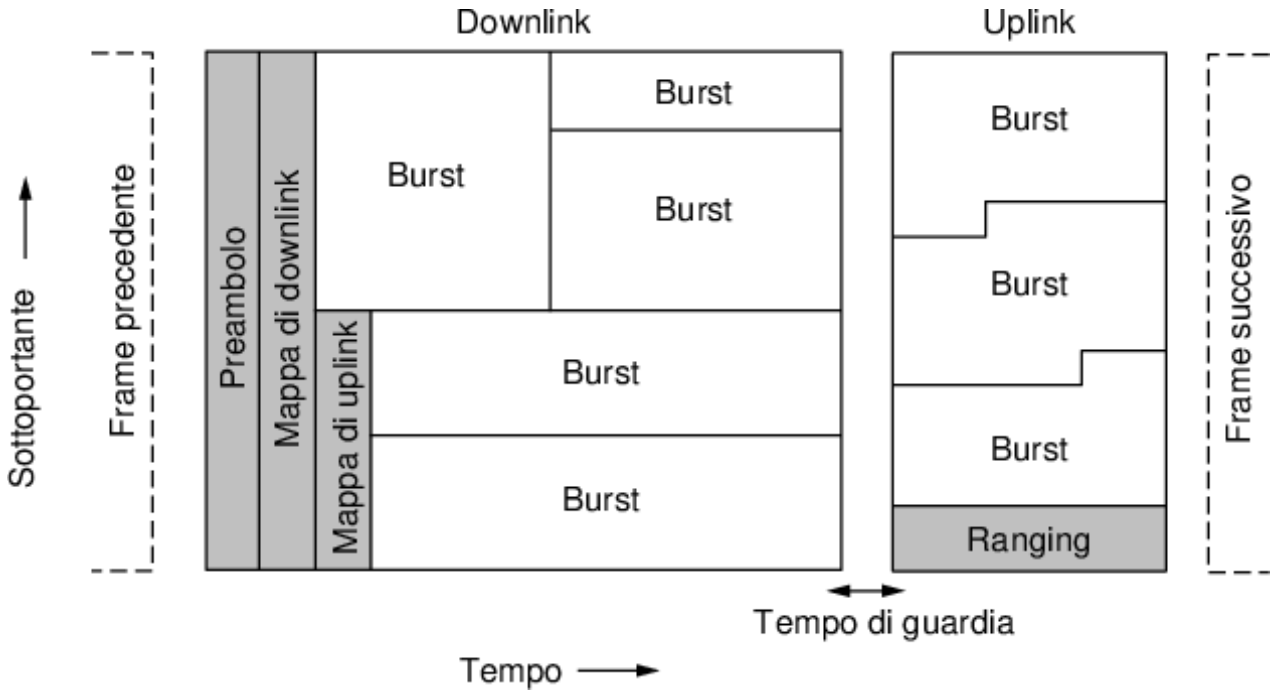


Figura 4.32 Struttura del frame per OFDMA con TDD.

Inoltre la stazione base spedisce burst di traffico alle varie stazioni degli abbonati e a quelle mobili sulle sottoportanti e con le tempistiche date nella mappa. Le trasmissioni verso le stazioni finiscono con un tempo di guardia per permettere loro di passare da riceventi a trasmittenti. Infine, le stazioni spediscono i loro burst di traffico alla stazione base nelle posizioni di trasmissione riservate loro nella mappa. Uno di questi burst sul canale di trasmissione è riservato al **ranging**, cioè il processo attraverso il quale le nuove stazioni si sincronizzano e richiedono della banda iniziale per connettersi alla stazione base. Dal momento che non si instaura una connessione in questa fase, le nuove stazioni trasmettono sperando di evitare collisioni.

Protocollo del sottolivello MAC di 802.16

Il livello data link è diviso in tre sottolivelli (Figura 4.31). Poiché la crittografia sarà studiata solo nel Capitolo 8 è difficile spiegare adesso il funzionamento del sottolivello di protezione. Basti sapere che la cifratura serve a tenere segreti tutti i dati trasmessi. Solo il campo dati del frame è cifrato, non l'intestazione. Questo significa che un ficcanaso può scoprire chi sta parlando e con chi, ma non può sapere che cosa i due si stiano dicendo.

Segue ora qualche indicazione sul funzionamento del sottolivello di sicurezza, utile a chi ha già qualche idea di crittografia, ma che richiederà di essere riletto dopo aver completato il Capitolo 8, soprattutto da parte di coloro digiuni in materia.

Quando un abbonato si connette a una stazione base entrambi si autenticano con criptografia a chiave pubblica RSA utilizzando certificati X.509. I dati inviati sono criptati utilizzando un sistema a chiavi simmetriche, AES (Rijndael) oppure DES con cifratura a blocco concatenata. Il controllo di integrità utilizza SHA-1. Fine delle indicazioni.

Andiamo ora a vedere la parte comune del sottolivello MAC. Il sottolivello MAC è orientato alla connessione e usa comunicazioni punto-multipunto, il che significa che una stazione base comunica con più abbonati. Molti concetti vengono presi in prestito dai cable modem, in cui l'headend controlla le trasmissioni di diversi modem nelle case degli utenti.

La ricezione è abbastanza semplice: la stazione base controlla i burst a livello fisico usati per mandare informazioni alle varie stazioni degli abbonati. Il sottolivello MAC infila semplicemente i suoi frame in questa struttura. Per ridurre l'overhead ci sono diverse opzioni. Per esempio, i frame MAC potranno essere spediti individualmente o impacchettati uno in coda all'altro per essere spediti in gruppo.

Il canale di trasmissione è più complicato, poiché ci sono abbonati non coordinati che competono tra loro per accedervi; la sua allocazione è strettamente legata alla problematica di qualità del servizio. Vengono definite quattro classi di servizio:

1. servizio constant bit rate (*a tasso costante*)
2. servizio real-time variable bit rate (*in tempo reale a tasso variabile*)
3. servizio non-real-time variable bit rate (*non in tempo reale a tasso variabile*)
4. servizio best-effort (*secondo disponibilità di risorse*).

Tutti i servizi in 802.16 sono orientati alla connessione e ogni connessione riceve una delle precedenti classi di servizio quando viene instaurata. Questa architettura è molto diversa da quella di 802.11 o di Ethernet, che non hanno il concetto di connessione nel sottolivello MAC.

Il servizio constant bit rate è stato pensato per trasmettere voce non compressa. Questo servizio ha bisogno di inviare una quantità di dati predeterminata a intervalli di tempo predeterminati ed è fornito dedicando certi intervalli temporali (*burst*) a ogni connessione di questo tipo. Una volta allocata la banda, gli intervalli temporali sono automaticamente disponibili senza che sia necessario richiederli singolarmente.

Il servizio real-time variable bit rate è utilizzato per i dati multimediali compressi e altre applicazioni real-time, dove la quantità di banda richiesta può variare in ogni istante. La stazione base interroga l'abbonato a intervalli fissi per sapere quanta banda gli serve in quel momento.

Il servizio non-real-time variable bit rate è utilizzato per trasmissioni pesanti che non avvengono in tempo reale, per esempio per trasferire file di grandi dimensioni. Per questo servizio la stazione base interroga spesso l'abbonato, ma non a intervalli imposti rigidamente. Connessioni con questo tipo di servizio possono usare anche il servizio best-effort, descritto di seguito per richiedere banda.

Il servizio best-effort è utilizzato per tutto il resto del traffico. Non viene eseguita alcuna interrogazione e l'abbonato deve competere per la banda insieme a tutti gli altri abbonati best-effort. Le richieste di banda vengono effettuate nei burst che nella mappa di trasmissione sono contrassegnati come disponibili per la contesa. Se una richiesta ha successo ciò sarà annotato nella successiva mappa di ricezione; in caso di insuccesso l'abbonato dovrà tentare di nuovo più tardi. Per limitare il numero di collisioni si usa l'algoritmo di backoff esponenziale binario.

Struttura del frame 802.16

Tutti i frame MAC iniziano con un'intestazione generica. L'intestazione è seguita dal payload e da un checksum (CRC), entrambi facoltativi, come mostrato nella Figura 4.33. Il payload non serve nei frame di controllo come quelli che, per esempio, richiedono intervalli di canale. Anche il checksum (sorprendentemente) è opzionale: questo perché viene effettuata una correzione degli errori nel livello fisico, oltre al fatto che non avviene mai alcun tentativo di ritrasmettere frame in tempo reale. Se non verrà tentata alcuna ritrasmissione, che senso avrà perdere tempo con il checksum? Tuttavia, se il checksum è presente, allora si tratta dello standard IEEE 802 CRC e per garantire l'affidabilità si usano acknowledgement e ritrasmissioni.

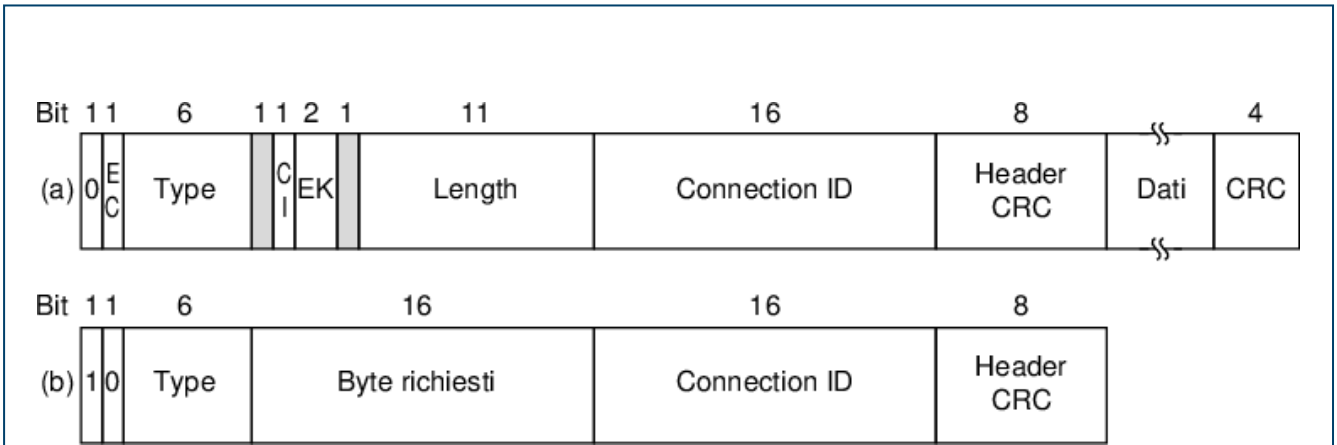


Figura 4.33 (a) Un frame generico. (b) Un frame di richiesta di banda.

Segue una rapida descrizione dei campi presenti nell'intestazione del frame rappresentato nella Figura 4.33(a). Il bit *EC* indica se il payload è stato cifrato. Il campo *Type* (tipo) identifica il tipo di frame e segnala se sono stati utilizzati meccanismi di impacchettamento e frammentazione. Il campo *CI* indica la presenza o l'assenza del codice di controllo finale. Il campo *EK* indica le chiavi di codifica utilizzate (in caso di cifratura). Il campo *Lenght* riporta la lunghezza totale del frame, inclusa l'intestazione. *Connection ID* (identificatore della connessione) definisce la connessione di appartenenza del frame. Infine, il campo *Header CRC* è un checksum dedicato solo all'intestazione e basato sul polinomio $x^8 + x^2 + x + 1$.

Il protocollo 802.16 ha molti tipi di frame; un esempio di frame utilizzato per richiedere intervalli di canale è mostrato nella Figura 4.33(b). Inizia con un bit 1 invece che con un bit 0 ed è simile all'intestazione generica, se non per il fatto che il secondo e il terzo byte formano un numero di 16 bit che indica la banda richiesta per trasportare il numero specificato di byte. Questo tipo di frame non trasporta payload e CRC.

Ci sarebbe ancora molto da dire su 802.16, ma questo non è il luogo adatto; per maggiori informazioni è possibile consultare lo standard IEEE 802.16-2009.

4.6 Bluetooth

Nel 1994 la L. M. Ericsson cominciò a interessarsi alla connessione senza fili dei suoi telefoni cellulari con altri dispositivi (per esempio i computer portatili). Insieme ad altre quattro aziende (IBM, Intel, Nokia e Toshiba), formò un SIG *(special interest group,* ossia un consorzio) per lo sviluppo di uno standard wireless che avrebbe dovuto permettere il collegamento tra dispositivi di calcolo, di comunicazione e accessori vari mediante un sistema radio wireless a basso costo, a bassa potenza e portata ridotta. Il progetto venne chiamato **Bluetooth**, in onore di Harald Blaatand (Bluetooth) II (940-981), il re vichingo che unificò (cioè conquistò) la Danimarca e la Norvegia, anche in questo caso senza cavi.

Bluetooth 1.0 fu reso pubblico nel luglio del 1999 e da allora il SIG non si è più fermato. Ogni possibile tipo di apparecchiatura elettronica utilizza Bluetooth: dai cellulari ai computer portatili, agli auricolari, stampanti, tastiere, mouse, piattaforme di gioco, orologi, riproduttori audio, navigatori e altro. I protocolli Bluetooth permettono a questi apparecchi di individuarsi a vicenda e connettersi, un'azione chiamata **pairing** (*associazione*), per poi trasferire dati in sicurezza.

I protocolli si sono evoluti anche durante lo scorso decennio. Dopo che i primi protocolli raggiunsero una certa stabilità, nel 2004 venne aggiunto un tasso di trasmissione più alto con Bluetooth 2.0. Con la versione 3.0, rilasciata nel 2009, Bluetooth può essere utilizzato per l'associazione di apparecchiature in combinazione con 802.11 per ottenere un trasferimento dati con elevato throughput. La versione 4.0, del dicembre 2009, specificò operazioni a bassa energia; ciò è utile per tutti coloro che non hanno voglia di sostituire regolarmente la batteria in tutte le apparecchiature domestiche. Ci occuperemo di seguito degli aspetti principali di Bluetooth.

Architettura Bluetooth

Cominciamo il nostro studio del sistema Bluetooth con una rapida panoramica di contenuti e funzionalità. L'unità base di un sistema Bluetooth è la **piconet**, composta da un nodo master e da fino a sette nodi slave attivi situati entro un raggio di 10 metri. Più piconet possono trovarsi nella stessa stanza (molto grande) e possono anche essere collegate attraverso un bridge, come mostrato nella Figura 4.34. Un insieme di piconet interconnesse è chiamato **scatternet**.

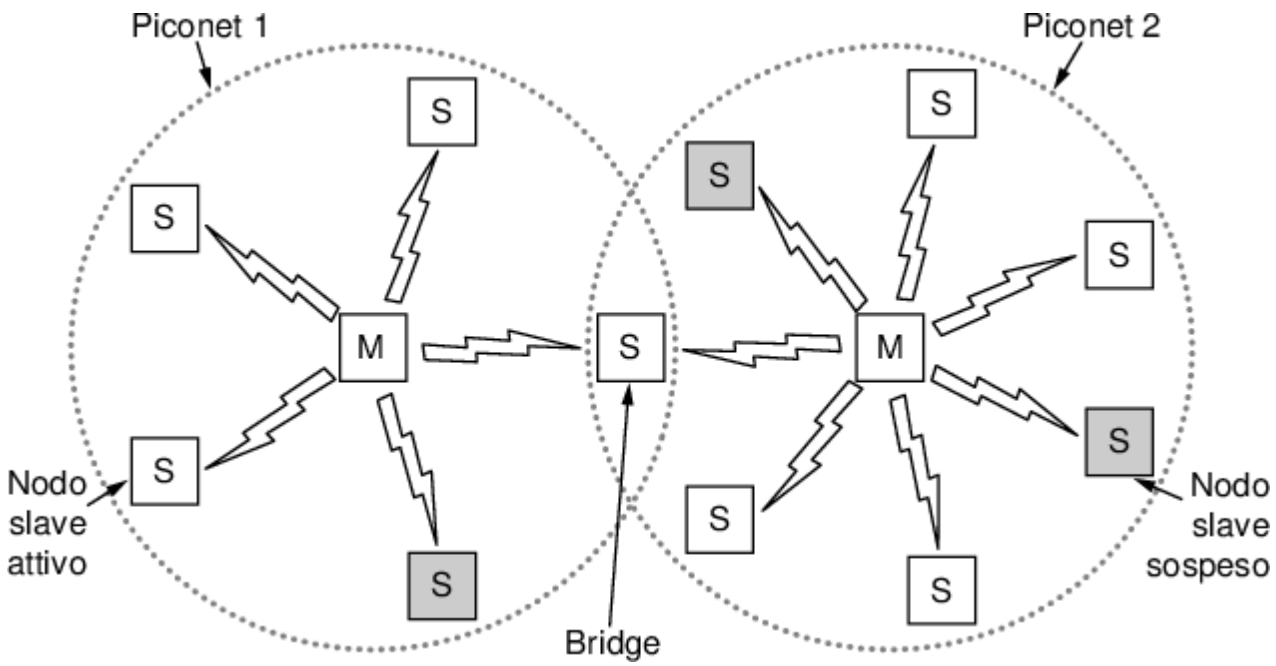


Figura 4.34 Due piconet possono essere collegate per formare una scatternet.

Oltre ai sette nodi slave attivi presenti in una piconet, la rete può contenere fino a 255 nodi sospesi (*parked*, in parcheggio): dispositivi a cui il nodo master ha imposto di attivare uno stato di basso consumo energetico per ridurre l'uso delle batterie. Quando si trova in stato di sospensione, un dispositivo può solo rispondere a una richiesta di attivazione o a un segnale di beacon trasmesso dal nodo master. Esistono anche altri due stati di alimentazione intermedi, chiamati hold e sniff, che però al momento non c'interessano.

La scelta dei progettisti di un'architettura basata su nodi master e slave facilita l'implementazione di chip Bluetooth completi per meno di 5 dollari. La conseguenza di questa decisione è che i nodi slave non sono indipendenti, fanno sostanzialmente solo ciò che il nodo master indica loro di fare. Il cuore della piconet è costituito da un sistema TDM centralizzato: il nodo master controlla il clock e decide quale dispositivo possa comunicare in ogni intervallo temporale. Tutta la comunicazione avviene tra il nodo master e un nodo slave; non è ammessa alcuna comunicazione diretta tra nodi slave.

Applicazioni Bluetooth

La maggior parte dei protocolli di rete fornisce i canali tra le entità comunicanti e lascia agli sviluppatori il compito di decidere come utilizzarli. Per esempio, 802.11 non specifica se gli utenti debbano usare i loro computer portatili per leggere la posta elettronica, navigare sul Web o fare altro. Al contrario, il Bluetooth SIG specifica particolari applicazioni da supportare e fornisce differenti stack di protocolli per ognuna. Al momento della stesura di questo libro ci sono 25 applicazioni, chiamate **profili**. Sfortunatamente, questo approccio porta a un grande livello di complessità. Qui non ci occuperemo di questo aspetto, ma guarderemo brevemente i profili per vedere più chiaramente che cosa stia cercando di ottenere il SIG Bluetooth.

Sei dei profili sono destinati a vari utilizzi di audio e video. Per esempio, il profilo *intercom* permette a due telefoni di connettersi come fossero walkie-talkies. I profili *headset* e *hands-free* permettono la comunicazione tra un auricolare e la sua stazione base, in quanto possono essere usati per avere le mani libere mentre si guida la macchina. Gli altri profili sono per flussi audio e video di qualità stereo, ad esempio, da un riproduttore portatile a degli auricolari o da una telecamera digitale a una televisione.

Il profilo *human interface device* serve a connettere tastiere e mouse a computer. Altri profili permettono a un cellulare o a un computer di ricevere immagini da una telecamera o spedire immagini a una stampante. Forse sarebbe più interessante un profilo per l'impiego di un cellulare come telecomando per la televisione (abilitata al Bluetooth).

Ulteriori profili forniscono funzionalità di rete. Il profilo *personal area network* permette a dispositivi Bluetooth di formare una rete ad hoc o accedere in modo remoto a un'altra rete, come una LAN 802.11, attraverso un access point. Il profilo *dial-up networking* fu in realtà la motivazione iniziale per l'intero progetto: permette a un computer portatile di connettersi a un cellulare con un modem integrato senza utilizzare fili.

Sono stati definiti anche profili per lo scambio di infomazioni con i livelli superiori. Il profilo *synchronization* è pensato per caricare dati su un telefono cellulare quando lascia l'abitazione e per raccogliere dati da esso quando ci ritorna.

Salteremo i restanti profili, con l'eccezione di menzionare che alcuni di essi sono la base di partenza su cui sono costruiti i profili già citati. Il profilo *generic access*, su cui sono basati tutti gli altri profili, fornisce un modo per stabilire e mantenere collegamenti sicuri (canali) tra il master e gli slave. Gli altri profili generici definiscono le basi per lo scambio di oggetti e il trasferimento audio e video. Profili di servizio sono ampiamente utilizzati per funzionalità quali l'emulazione di una linea seriale, utile specialmente per molte applicazioni che devono offrire compatibilità con vecchi sistemi.

Era veramente necessario indicare in modo esplicito e dettagliato tutte queste applicazioni e fornire a ognuna una specifica pila di protocolli? Probabilmente no, ma le varie parti dello standard sono state concepite da gruppi di lavoro diversi e ogni gruppo, concentrandosi solo su un problema specifico, ha generato un proprio profilo. Questo risultato può essere considerato come una dimostrazione della legge di Conway: nel numero di aprile 1968 della rivista Datamation, Melvin Conway commentò che incaricando *n* persone della scrittura di un compilatore si ottiene un compilatore con *n* passi o, più in generale, la struttura del software rispecchia quella del gruppo che lo ha prodotto. Probabilmente sarebbe stato possibile cavarsela con due stack di protocolli invece di 25, uno per il trasferimento dei file e l'altro per la gestione dei flussi di comunicazione in tempo reale.

Stack di protocolli Bluetooth

Lo standard Bluetooth ha molti protocolli raggruppati liberamente nei livelli rappresentati nella Figura 4.35. La prima osservazione da fare è che la struttura non segue il modello OSI, il modello TCP/IP, il modello 802, o qualsiasi altro modello.

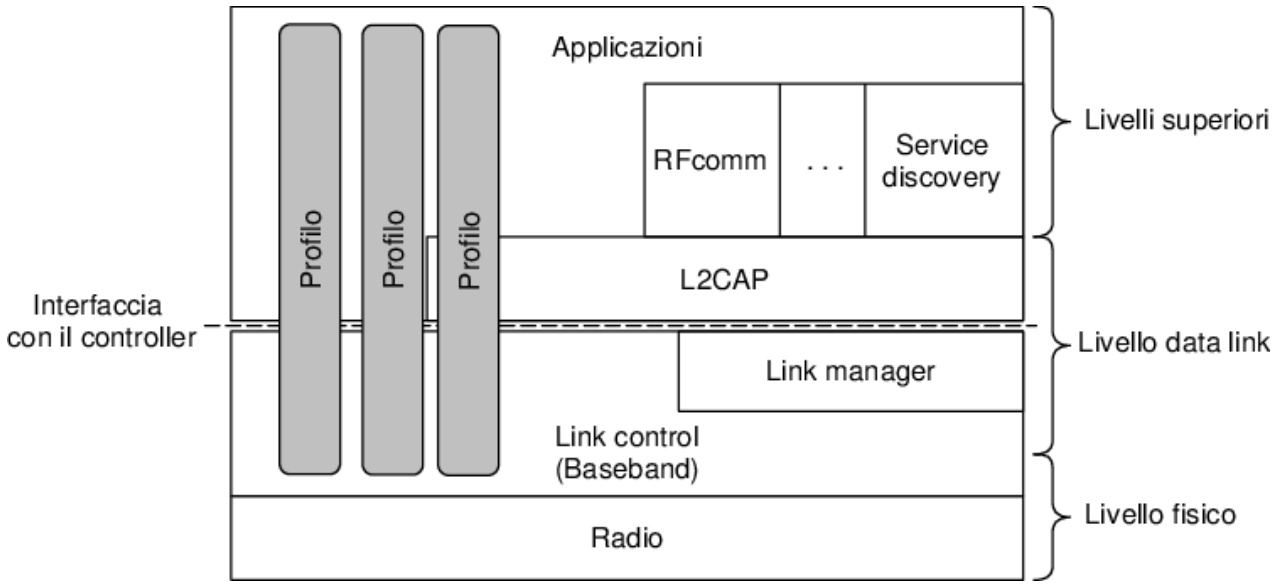


Figura 4.35 Architettura del protocollo Bluetooth.

Il livello più in basso è quello della trasmissione radio, che corrisponde piuttosto bene al livello fisico dei modelli OSI e 802. Questo livello si occupa della trasmissione radio e della modulazione. Molte delle preoccupazioni riguardanti questo livello dipendono dall'obiettivo di rendere il sistema economico, così da renderlo interessante per il mercato su larga scala.

Il livello link control (o baseband) è per certi versi analogo al sottolivello MAC, ma include anche elementi del livello fisico; gestisce il modo in cui il master controlla gli intervalli temporali e il raggruppamento di questi intervalli in frame.

Seguono due protocolli che utilizzano il protocollo link control. Il link manager gestisce la creazione di canali logici tra dispositivi, inclusa la gestione dell'energia, del pairing, della cifratura e la qualità del servizio; questo si posiziona sotto la linea che demarca l'interfaccia con il controller. Tale interfaccia facilita l'implementazione: tipicamente, i protocolli sotto la linea vengono implementati su di un chip Bluetooth e i protocolli al di sopra vengono implementati nel dispositivo Bluetooth che ospita il chip.

Il protocollo di livello link sopra la linea è **L2CAP** (*logical link control adaptation protocol*); suddivide in frame i messaggi di lunghezza variabile e quando necessario fornisce affidabilità. Molti protocolli utilizzano L2CAP, come i due protocolli di servizio illustrati; il protocollo di *service discovery* è utilizzato per localizzare servizi all'interno della rete, il protocollo **RFcomm** (*radio frequency communication*) emula le porte seriali standard che si trovano sui PC per connettere, per esempio, tastiera, mouse e modem.

Ai livelli superiori si trovano le applicazioni. I profili sono rappresentati da rettangoli verticali, poiché ognuno di essi definisce una sezione dello stack di protocolli adibita a uno specifico scopo. Profili specifici, come il profilo headset, contengono di solito solo i protocolli necessari a quell'applicazione e nessun altro. Per esempio, i profili possono includere L2CAP se hanno pacchetti da spedire, ma lo salteranno se hanno solo un flusso costante di campioni audio.

Nei paragrafi successivi esamineremo il livello radio di Bluetooth e diversi protocolli del livello link, poiché questi corrispondono grossomodo ai sottolivelli fisico e MAC degli altri stack di protocolli che abbiamo studiato.

Livello radio di Bluetooth

Il livello radio sposta i bit da master a slave e viceversa. È un sistema a bassa potenza che ha una portata di 10 metri e opera nella banda ISM dei 2,4 GHz, come 802.11. La banda è divisa in 79 canali da 1 MHz ciascuno. Per coesistere con le altre reti che utilizzano la banda ISM viene utilizzata la tecnica di *frequency hopping spread spectrum* (spettro distribuito a frequenza variabile). Ci possono essere fino a 1600 hop/s su slot con una durata (che prende il nome di *dwell time*) di 625 ms. Tutti i nodi in una piconet cambiano frequenza simultaneamente, seguendo la temporizzazione e una sequenza pseudocasuale dettate dal master.

Sfortunatamente saltò fuori che le prime versioni di Bluetooth e 802.11 interferivano abbastanza da rovinarsi a vicenda le trasmissioni. Alcune aziende reagirono bandendo completamente Bluetooth, ma alla fine si trovò una soluzione tecnica: che Bluetooth adattasse la sua sequenza di hop in modo da escludere canali su cui ci fossero altri segnali RF. Questo processo riduce le interferenze dannose ed è chiamato **frequency hopping adattativo** (*adaptive frequency hopping*).

Per spedire bit su un canale vengono utilizzate tre forme di modulazione. Lo schema di base è di utilizzare la modulazione di frequenza per spedire un simbolo da 1 bit ogni microsecondo, dando un tasso di trasmissione approssimativo di 1 Mbps. Tassi più alti furono introdotti con la vesione 2.0 di Bluetooth; questi usano la modulazione di fase per spedire 2 o 3 bit per simbolo, per arrivare a circa 2 o 3 Mbps. Tali tassi maggiori sono comunque utilizzati solo nella parte dati dei frame.

Livelli link di Bluetooth

Il livello link control (o baseband) è quanto di più simile Bluetooth abbia rispetto a un sottolivello MAC, infatti trasforma il flusso di bit grezzi in frame e definisce alcuni formati chiave. Nella sua forma più semplice, il master di ogni piconet definisce una serie di slot da 625 ms; le trasmissioni del nodo master iniziano negli slot pari, mentre le trasmissioni dei nodi slave iniziano in quelli dispari. Questo è un meccanismo multiplexing a divisione di tempo tradizionale, con il nodo master che usa la metà degli slot e i nodi slave che condividono l'altra metà. I frame possono occupare 1, 3 oppure 5 slot. Ogni frame ha un overhead di 126 bit per l'intestazione e il codice di accesso, più un tempo di assestamento di 250-260 ms per hop, in modo tale da permettere a circuiti radio economici di raggiungere stabilità. Il payload del frame può essere cifrato per confidenzialità con una chiave scelta quando master e slave si connettono. Gli hop avvengono solo tra frame, non durante un frame. Il risultato è che un frame da 5 slot è molto più efficiente di un frame da 1 slot, poiché l'overhead è costante, ma vengono spediti più dati.

Il protocollo link manager crea canali logici, chiamati **link** (da non confondersi con i link fisici), per portare i frame tra il master e un dispositivo slave che si sono scoperti a vicenda. Segue una procedura di pairing per assicurarsi che i due dispositivi possano comunicare prima di utilizzare il link. Il vecchio metodo di pairing consiste nel configurare entrambi i dispositivi con lo stesso PIN (*personal identification number*) di quattro cifre. La corrispondenza del PIN permette a ogni

dispositivo di avere conferma di essere stato connesso al giusto apparecchio remoto. Tuttavia, utenti senza immaginazione e PIN con default come “0000” e “1234” hanno reso questo metodo poco sicuro in pratica.

Il nuovo metodo **secure simple pairing** (*pairing semplice e sicuro*) permette agli utenti di confermare che entrambi gli apparecchi mostrino la stessa chiave di accesso o di vedere la chiave su un apparecchio e immetterla nel secondo. Questo metodo è più sicuro, perché gli utenti non devono scegliere o creare un PIN. Essi semplicemente confermano una chiave d’accesso più lunga, ma generata dal dispositivo. Naturalmente tale metodo non può essere utilizzato su dispositivi con un input/output limitato, come un auricolare.

Una volta completato il pairing, il protocollo link manager crea i link. Per trasportare i dati utente esistono due tipi principali di link. Il primo è il link **SCO** (*synchronous connection oriented*, “sincrono e orientato alla connessione”); è utilizzato per dati in tempo reale, come per le connessioni telefoniche. A questo tipo di link è allocato uno specifico slot in ogni direzione. Uno slave può avere fino a tre link SCO col suo master e ognuno di essi può trasmettere un canale audio PCM (non compresso) a 64.000 bps. A causa della natura time-critical dei link SCO i frame trasmessi su di essi non verranno mai ritrasmessi; invece, la correzione anticipata degli errori (FEC) può essere utilizzata per migliorare l’affidabilità.

L’altro tipo è il link **ACL** (*asynchronous connectionless*, asincrono senza connessione), utilizzato per dati commutati a pacchetto e disponibili a intervalli irregolari. Il traffico ACL è consegnato con una politica best-effort, non vengono date garanzie: i frame possono essere persi e potrebbero dover essere ritrasmessi. Uno slave può avere solo un link ACL col suo master.

I dati spediti su link ACL provengono dal livello L2CAP; questo livello ha quattro funzioni principali. Primo, accetta pacchetti fino a 64 KB dai livelli superiori e li spezza in frame per la trasmissione; all’altro capo i frame sono riassemblati in pacchetti. Secondo, gestisce il multiplexing e demultiplexing di più sorgenti; quando un pacchetto è stato riassemblato il livello L2CAP determina a quale protocollo di livello superiore deve essere passato, per esempio RFcomm o service discovery. Terzo, L2CAP gestisce il controllo degli errori e le ritrasmissioni; trova gli errori e rispedisce i pacchetti per cui non si è ricevuto un acknowledgement. Infine, L2CAP impone requisiti di qualità di servizio tra link multipli.

Struttura del frame di Bluetooth

Bluetooth definisce diversi formati di frame, il principale dei quali è mostrato in due formati nella Figura 4.36. Inizia con un codice di accesso che di solito identifica il nodo master; questa informazione consente ai nodi slave che si trovano nel raggio d’azione di due master di identificare il traffico di rispettiva competenza. Segue un’intestazione di 54 bit contenente i tipici campi del sottolivello MAC. Se il frame è spedito al tasso di base quello che segue è il campo dati; ha fino a 2744 bit per una trasmissione di cinque slot. Per un solo slot, il formato è lo stesso, tranne per il campo dati che è di 240 bit.

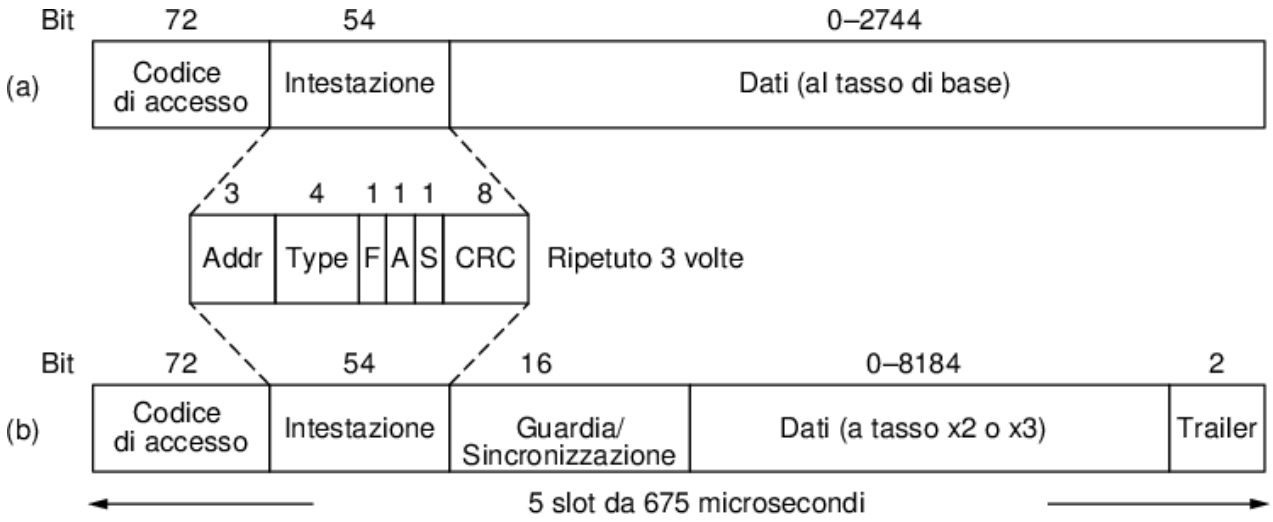


Figura 4.36 Un tipico frame di dati Bluetooth a tassi (a) base e (b) aumentati.

Se il frame è spedito a un tasso più elevato, la porzione dei dati può avere un numero di bit fino a due o tre volte superiore, perché ogni simbolo porta 2 o 3 bit al posto di uno. Questi dati sono preceduti da un campo di protezione e da una sequenza di sinconizzazione usata per passare al tasso di trasmissione più alto; il codice di accesso e l'intestazione sono trasportati al tasso di base e solo la porzione dei dati è trasportata a un tasso di trasmissione più alto. I frame spediti a tassi aumentati finiscono con un breve trailer.

Diamo ora un rapido sguardo all'intestazione comune. Il campo *Address* (indirizzo) identifica il destinatario del frame tra gli otto dispositivi attivi. Il campo *Type* identifica il tipo di frame (ACL, SCO, interrogazione oppure nullo), il tipo di correzione degli errori utilizzato nel campo dati e il numero di intervalli occupati dal frame. Il bit *Flow* (flusso), è attivato da un nodo slave quando non può ricevere altri dati, perché ha il buffer pieno; questa è una forma primitiva di controllo di flusso. Il bit *Acknowledgement* è utilizzato per aggiungere al frame un ACK. Il bit *Sequence* (sequenza) è utilizzato per numerare i frame in modo da rilevare le ritrasmissioni; il protocollo adotta un approccio stop-and-wait, perciò è sufficiente 1 bit. Seguono gli 8 bit utilizzati dal checksum dell'intestazione. L'intera intestazione, lunga 18 bit, è ripetuta tre volte per formare l'intestazione di 54 bit mostrata nella Figura 4.36. Dalla parte del destinatario, un semplice circuito esamina tutte e tre le copie di ogni bit; se le tre copie sono identiche il bit è accettato, altrimenti si va a maggioranza. Di conseguenza, per trasmettere 10 bit di intestazione servono 54 bit di capacità di trasmissione. Trasmettere dati in modo affidabile in un ambiente soggetto a rumore usando dispositivi economici, che operano a bassa potenza (2,5 mW) e che sono dotati di una limitata capacità di elaborazione richiede un elevato livello di ridondanza.

Ci sono vari formati per il campo *Data* (dati) di frame ACL e SCO. I frame SCO al tasso di trasmissione di base sono un esempio semplice da studiare: il loro campo dati è sempre di 240 bit. Sono definite tre varianti che consentono 80, 160 o 240 bit di payload, il resto è usato per la correzione degli errori. Nella versione più affidabile (80 bit di payload), il contenuto è ripetuto tre volte, proprio come l'intestazione.

Siamo in grado di ricavare la capacità del canale in questo ultimo caso procedendo come segue: poiché può utilizzare solo i dispari, lo slave ha a disposizione 800 slot/s, esattamente come il nodo master. Con 80 bit di payload, la capacità del canale che parte dal nodo slave è di 64.000 bps, come pure la capacità del canale che parte dal nodo master, giusto quello che serve per un singolo canale vocale PCM full duplex (ecco perché è stata scelta una frequenza di hopping di 1600 hop/s). Questo significa che, nonostante la banda grezza da 1 Mbps, un singolo canale vocale non complesso full-duplex può completamente saturare la piconet. L'efficenza del 13% è il risultato dello spendere il 41% della capacità in tempo di assestamento, il 20% nelle trasmissioni delle intestazioni e il 26% nella codifica a ripetizione. Questa mancanza sottolinea il valore di tassi più alti e frame costituiti da più di un singolo slot.

C'è ancora molto da dire su Bluetooth. Per i curiosi, le specifiche di Bluetooth 4.0 contengono tutti gli altri dettagli.

4.7 RFID

Abbiamo analizzato le architetture MAC dalle LAN su verso le MAN e giù verso le PAN. Come ultimo esempio studieremo una categoria di dispositivi wireless di basso livello normalmente non identificabile come costituente di una rete di calcolatori: i tag e i lettori **RFID** (*radio frequency identification*), descritti nel Paragrafo 1.5.4.

La tecnologia RFID prende molte forme viene utilizzata nelle smartcard, per la tracciabilità degli animali, nei passaporti, nei libri e molto altro. La forma che osserviamo qui fu sviluppata durante la ricerca per un **EPC** (*electronic product code*) che cominciò con l'Auto-ID Center del M.I.T. nel 1999. Un EPC è un sostituto del codice a barre che però ha una maggiore capacità di contenimento delle informazioni ed è leggibile elettronicamente fino a una distanza di 10 metri, anche quando non è visibile. È una tecnologia diversa, per esempio, dal RFID utilizzato nei passaporti, che per attuare una transazione deve essere posto abbastanza vicino a un lettore. L'abilità di comunicare a distanza rende gli EPC più significativi per i nostri studi.

EPCglobal fu costituito nel 2003 per commercializzare la tecnologia RFID sviluppata dall'Auto-ID Center. La svolta si verificò nel 2005 quando Walmart[4] richiese ai suoi 100 principali fornitori di etichettare tutte le spedizioni con tag RFID. L'applicazione su larga scala è stata ostacolata dalla difficoltà di competere con i più economici codici a barre stampati, ma i nuovi utilizzi, come nelle patenti (negli Stati Uniti), sono in crescita. Descriveremo qui la seconda generazione di questa tecnologia, informalmente chiamata **EPC Gen 2** (EPCglobal, 2008).

Architettura di EPC Gen 2

L'architettura di una rete RFID EPC Gen 2 è mostrata nella Figura 4.37 e possiede due componenti chiave: tag e lettori. I tag RFID sono dispositivi piccoli ed economici, con un unico identificatore EPC a 96 bit e una piccola quantità di memoria che può essere letta e scritta da un lettore RFID. La memoria può essere utilizzata per registrare lo storico della posizione di un oggetto, per esempio, i suoi spostamenti attraverso la catena di approvigionamento.

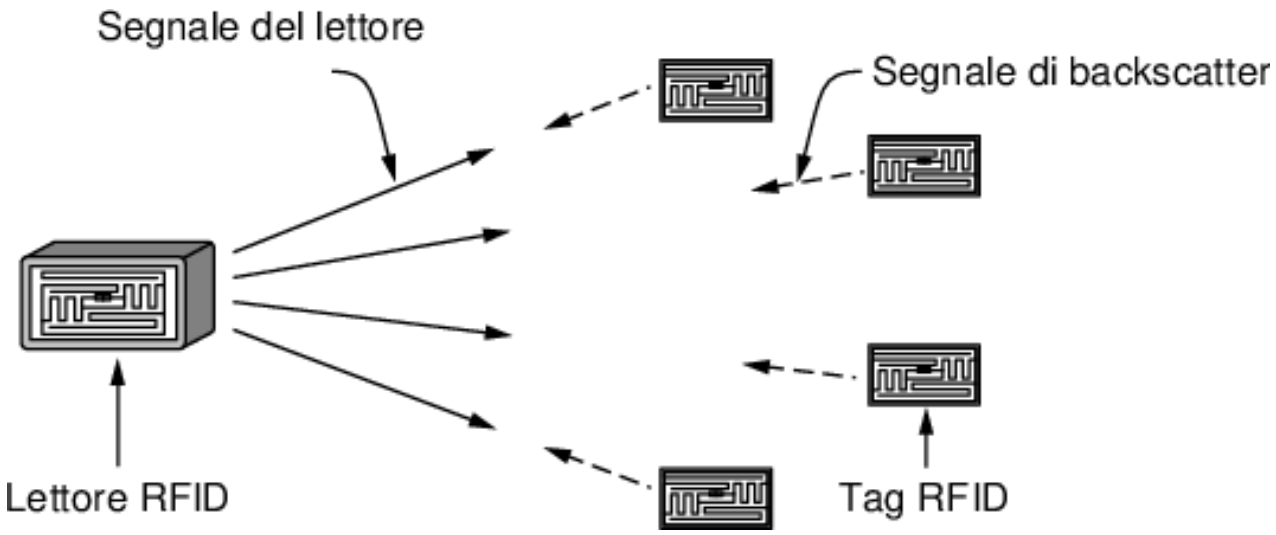


Figura 4.37 Architettura RFID.

Spesso i tag sono simili ad adesivi e possono essere posizionati, per esempio, su un paio di jeans sugli scaffali di un negozio. Gran parte dell'adesivo è occupato da un'antenna serigrafata. Un piccolo punto nel centro corrisponde al circuito integrato RFID; in alternativa, i tag RFID possono essere integrati in un oggetto, come una patente. In entrambi i casi, i tag non hanno batteria e per funzionare devono ottenere energia dalle trasmissioni radio di un lettore RFID nelle vicinanze; questo tipo di tag è chiamato tag di "Classe 1" per distinguerlo dai tag più avanzati e forniti di batterie.

Analoghi alle stazioni base e agli access point nelle reti cellulari e WiFi, i lettori sono la parte intelligente del sistema. Molto più potenti dei tag, hanno una loro sorgente di energia, spesso dispongono di molteplici antenne e stabiliscono l'invio e la ricezione dei messaggi da parte dei tag. Poiché di solito ci sono diversi tag all'interno del raggio di lettura, i lettori devono risolvere il problema dell'accesso multiplo; ci possono anche essere più lettori a competere tra loro nella stessa area.

Il compito principale dei lettori è di fare un inventario dei tag nelle vicinanze, cioè di trovare gli identificatori dei tag vicini; tale inventario è compilato grazie al protocollo di livello fisico e al protocollo di identificazione dei tag delineati nei paragrafi seguenti.

Livello fisico di EPC Gen 2

Il livello fisico definisce come avviene la spedizione dei bit tra il lettore RFID e i tag. Principalmente si utilizzano metodi già visti in precedenza per i segnali wireless. Negli Stati Uniti le trasmissioni avvengono nella banda IMS senza licenza a 902-928 MHz, che cade nell'intervallo UHF (*ultra high frequency*), per cui i tag sono indicati come tag UHF RFID. Il lettore cambia frequenza almeno ogni 400 millisecondi per diffondere il suo segnale sul canale, limitare

l'interferenza e soddisfare i requisiti normativi. Per codificare i bit, lettori e tag utilizzano delle forme di modulazione di tipo **ASK** (*amplitude shift keying*), descritte nel Paragrafo 2.5.2. Inoltre, fanno a turno per spedire i bit, quindi il link è half-duplex.

Sono due le differenze principali rispetto agli altri livelli fisici che abbiamo studiato. La prima è che il lettore trasmette sempre un segnale, indipendentemente che sia il lettore o il tag a comunicare. Naturalmente, il lettore trasmette un segnale per spedire dei bit ai tag. Per permettere al tag di spedire bit al lettore, il lettore trasmette un segnale portante fisso che non trasporta bit. I tag raccolgono questo segnale per ottenere l'energia di cui hanno bisogno per funzionare; altrimenti, un tag non sarebbe capace di trasmettere. Per spedire dati, un tag cambia come se stesse riflettendo il segnale proveniente dal lettore, come un segnale radar che può rimbalzare o essere assorbito da un bersaglio.

Questo metodo è chiamato **backscatter**. È diverso da ogni altra situazione wireless esaminata finora, in cui mittente e destinatario non trasmettono mai nello stesso istante. Il backscatter è un approccio a bassa energia che permette al tag di creare autonomamente un debole segnale che arriva fino al lettore. Il lettore, per decodificare il segnale in entrata, deve filtrare il segnale in uscita che sta trasmettendo. Poiché il segnale del tag è debole, i tag possono avere verso il lettore solo bassi tassi di trasmissione e non possono ricevere o anche solo percepire segnali provenienti da altri tag.

La seconda differenza è che vengono utilizzate forme molto semplici di modulazione, che quindi possono essere implementate su tag che funzionano con una energia minima e la cui produzione costa solo pochi centesimi. Per spedire dati ai tag, il lettore utilizza due livelli di ampiezza: i bit sono riconosciuti come 0 o 1 a seconda di quanto il lettore aspetta prima di un periodo a bassa energia. Il tag misura il tempo tra i periodi a bassa energia e lo confronta con uno di riferimento misurato durante il preambolo della comunicazione. Come si vede nella Figura 4.38, gli 1 sono più lunghi degli 0.

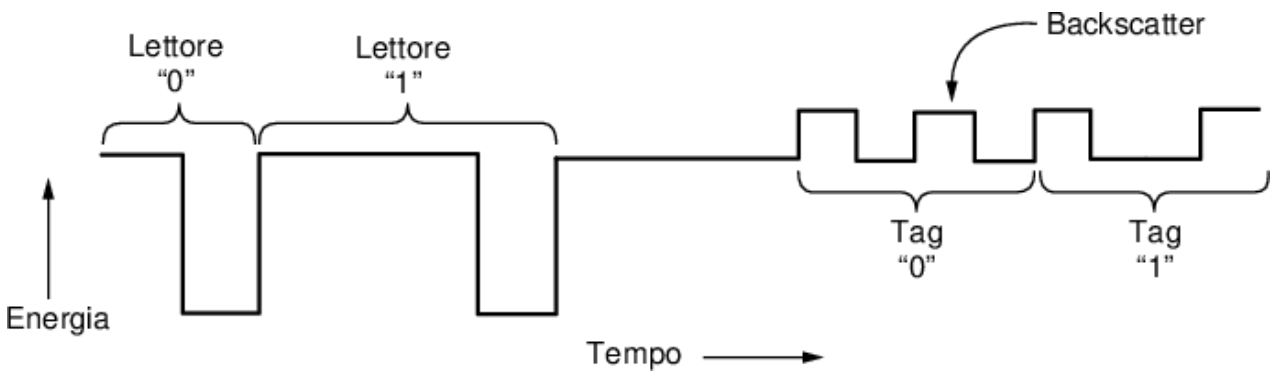


Figura 4.38 Lettore e segnali di backscatter dei tag.

Le risposte dei tag consistono nell'alternare il proprio stato di backscatter a intervalli fissati per creare una serie di impulsi nel segnale. Sequenze da uno a otto impulsi possono essere usate per decodificare 0 o 1 a seconda dell'affidabilità richiesta. Gli 1 hanno meno transizioni degli 0, come si vede nell'esempio di codifica di un periodo di due impulsi nella Figura 4.38.

Livello di identificazione dei tag di EPC Gen 2

Per fare l'inventario dei tag nelle vicinanze, il lettore deve ricevere da ciascuno un messaggio che fornisca il proprio identificatore. Questa situazione presenta un problema di accesso multiplo per il quale il numero di tag, in generale, non è noto a priori. Il lettore potrebbe trasmettere in broadcast una richiesta (*query*) a tutti i tag di spedire i propri identificativi. Tuttavia, i tag che risponderanno subito collideranno allo stesso modo in cui collidono le stazioni collegate tramite una Ethernet classica.

Abbiamo già visto molti modi per affrontare il problema dell'accesso multiplo; il protocollo più adatto a questa situazione, in cui i tag non possono sentire le rispettive trasmissioni, è lo slotted ALOHA, uno dei primi protocolli analizzati. Questo protocollo è appropriato per l'uso in Gen 2 RFID.

Nella Figura 4.39 è visibile la sequenza di messaggi utilizzati per identificare un tag; nel primo slot (0), il lettore spedisce un messaggio *Query* per iniziare il processo. Ogni messaggio di *QRepeat* serve ad avanzare verso lo slot successivo continuando a emettere segnale. Il lettore indica anche ai tag l'intervallo di slot su cui distribuire casualmente le trasmissioni. È necessario usare un intervallo, poiché il lettore sincronizza i tag quando inizia il processo; diversamente da stazioni su Ethernet, i tag non si risvegliano con un messaggio inviato in un momento scelto da loro.

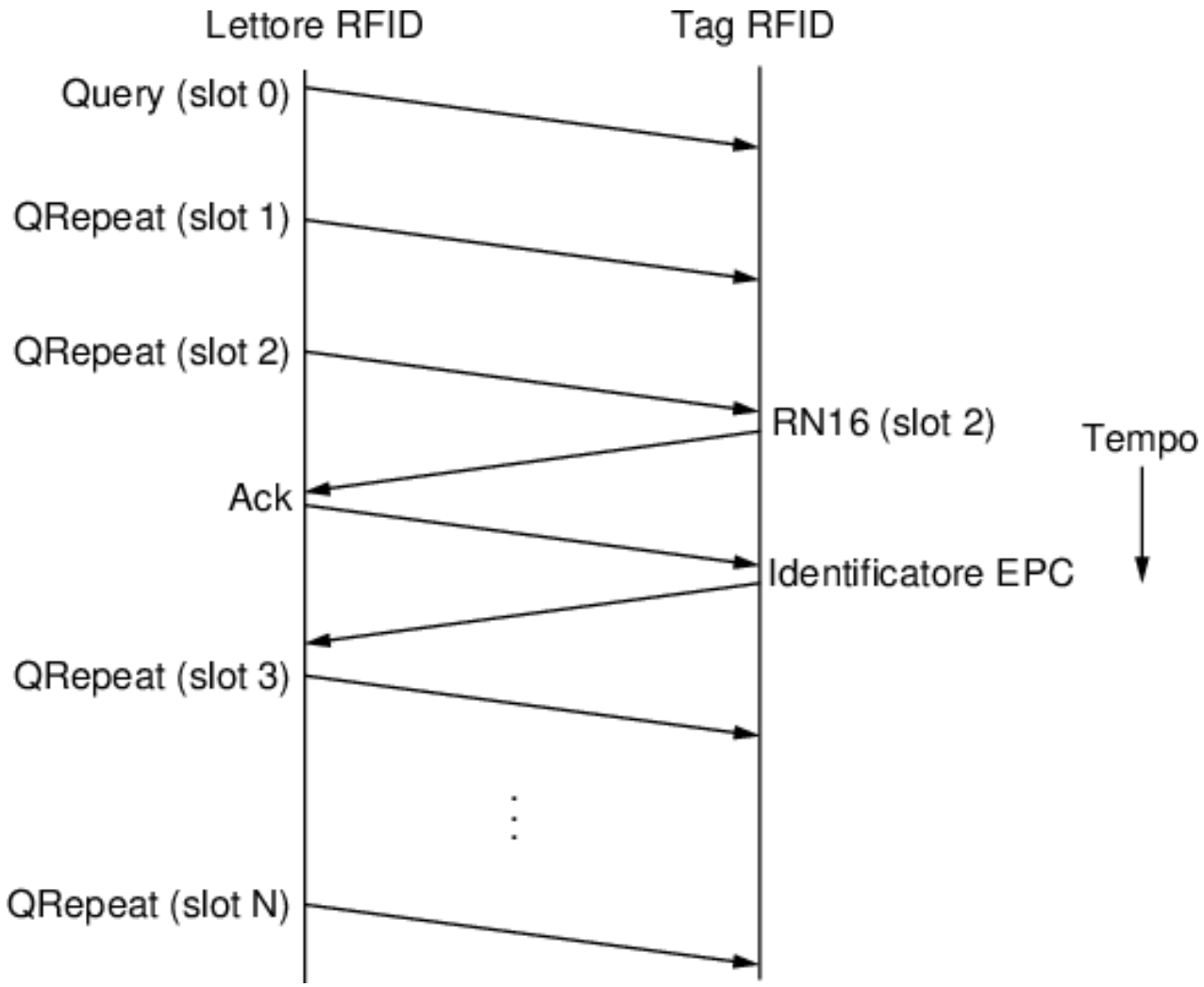


Figura 4.39 Esempio di scambio di messaggi per identificare un tag.

I tag scelgono uno slot casuale nel quale rispondere. Nella Figura 4.39 il tag risponde nello slot 2. Tuttavia, i tag non spediscono i loro identificativi alla prima risposta; spediscono invece un messaggio *RN16* contenente un numero casuale da 16 bit. Se non ci sono collisioni, il lettore riceve questo messaggio e spedisce un proprio messaggio *ACK*. A questo punto, il tag ha acquisito lo slot e spedisce il proprio identificatore *EPC*.

La ragione di questo scambio è che gli identificatori EPC sono lunghi e quindi collisioni su questi messaggi sarebbero costose. Invece, un messaggio breve viene usato per verificare se il tag può utilizzare lo slot per spedire il proprio identificativo. Una volta che l'identificativo è stato trasmesso con successo il tag cessa temporaneamente di rispondere a messaggi *Query,* così che tutti i restanti tag possano essere identificati.

Un problema chiave per il lettore è quello di adeguare il numero di slot per evitare collisioni, ma senza utilizzarne troppi, perché ciò penalizzerebbe le prestazioni. Questo adeguamento è analogo al backoff esponenziale binario di Ethernet: se il lettore vede troppi slot senza risposta o troppi slot con collisioni può spedire un messaggio *QAdjust* per diminuire o aumentare l'intervallo degli slot su cui i tag rispondono.

Il lettore RFID può eseguire altre operazioni sui tag. Per esempio, può selezionare un sottoinsieme di tag prima di iniziare l'inventario, permettendo così di raccogliere risposte, ad esempio, da parte dei blue jeans, ma non da parte delle camicie. Una volta identificati, il lettore può anche scrivere dati sui tag: questa funzionalità può essere utilizzata per registrare il punto vendita o altre informazioni di rilievo.

Formato dei messaggi di identificazione dei tag

Il formato del messaggio *Query* è visibile nella Figura 4.40 sotto forma di esempio di messaggio da lettore a tag. Il messaggio è breve, perché la banda in downlink è limitata da 27 kbps a 128 kbps. Il campo *Command* (comando) porta il codice 1000 per identificare il messaggio come *Query*.

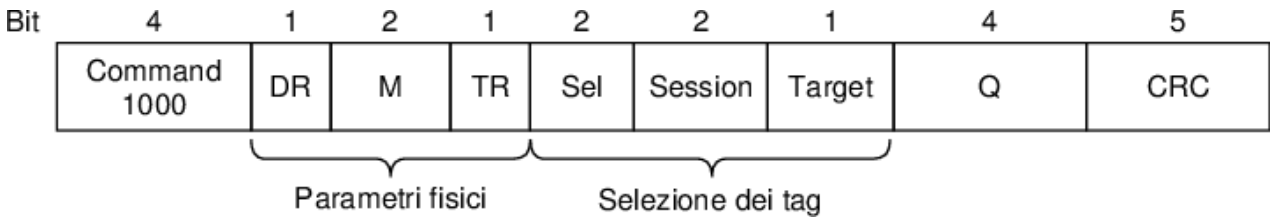


Figura 4.40 Formato del messaggio Query.

Gli indicatori seguenti, *DR*, *M* e *TR* determinano i parametri del livello fisico per le trasmissioni del lettore e le risposte del tag. Per esempio, il tasso di trasmissione per la risposta può andare da 5 a 640 kbps. Tralasceremo i dettagli di questi spazi.

Seguono tre campi, *Sel*, *Session* e *Target*, che selezionano i tag che risponderanno. Come il lettore è in grado di identificare un sottoinsieme di identificatori, i tag monitorano fino a quattro sessioni concorrenti e tengono traccia di quando essi stessi sono stati identificati; in questo modo diversi lettori possono operare in aree sovrapposte utilizzando sessioni diverse.

Successivamente si trova il parametro più importante per questo comando: Q. Questo campo definisce l'intervallo degli slot su cui i tag risponderanno, da 0 a $2^Q - 1$. Infine, c'è un CRC di 5 bit che protegge i campi del messaggio; è più breve di molti CRC che abbiamo visto, ma anche il messaggio *Query* è molto più breve di molti pacchetti.

I messaggi tag-lettore sono più semplici. Dal momento che il lettore ha il controllo della situazione sa che tipo di messaggio aspettarsi in risposta a ogni sua trasmissione; le risposte del tag contengono semplicemente dati, come l'identificativo EPC.

Originariamente i tag avevano solo uno scopo identificativo. Tuttavia sono cresciuti nel tempo assumendo le sembianze di computer molto piccoli. Alcuni tag usati per la ricerca sono dotati di sensori e sono capaci di eseguire brevi programmi per raccogliere ed elaborare i dati (Sample et al.,

2008). Una prospettiva del futuro di questa tecnologia è rappresentata nella “Internet of things” (ITU, 2005) che collega oggetti del mondo fisico a Internet (Welbourne et al., 2009; Gershenfeld et al., 2004).

4.8 Commutazione a livello data link

Molte aziende hanno diverse LAN che desidererebbero connettere tra loro. Non sarebbe utile se potessimo semplicemente unire insieme le LAN per crearne una più grande? Possiamo farlo quando le connessioni avvengono con dispositivi chiamati **bridge**. Gli switch Ethernet descritti nel Paragrafo 4.3.4 sono un modo attuale di chiamare i bridge; forniscono funzionalità che vanno oltre quelle di Ethernet classica e gli hub Ethernet per facilitare l'unione di più LAN in una rete più grande. Useremo i termini "bridge" e "switch" indifferentemente.

I bridge operano a livello data link, quindi esaminano gli indirizzi del livello data link per inoltrare i frame. Dal momento che non sono incaricati di esaminare il payload dei frame che inviano, possono manipolare i pacchetti IP come ogni altro pacchetto, come avviene per i pacchetti AppleTalk.

In questo paragrafo vedremo il funzionamento dei bridge e il loro impiego per unire diverse LAN fisiche in una singola LAN logica. Vedremo inoltre come fare il contario e gestire una LAN fisica come diverse LAN logiche, chiamate **VLAN** (*virtual LAN*). Entrambe le tecnologie permettono un'utile flessibilità per la gestione delle reti. Per avere una visione completa su bridge, switch e argomenti correlati, si vedano Seifert e Edwards (2008) e Perlman (2000).

Vari utilizzi dei bridge

Prima di iniziare l'esame della tecnologia dei bridge conviene descrivere alcune situazioni comuni in cui sono usati; illustreremo tre motivi che possono spingere una singola azienda a creare più LAN.

Primo, molte facoltà universitarie e divisioni aziendali dispongono di una propria LAN per collegare personal computer, server e dispositivi, come le stampanti. Poiché ogni azienda o ente persegue un obiettivo diverso, uffici differenti installano LAN differenti senza tener conto delle scelte adottate da altri. Presto o tardi, però, si presenta la necessità di interagire ed ecco che i bridge diventano indispensabili. In questo esempio la creazione delle diverse LAN è causata dall'autonomia dei loro proprietari.

Secondo, l'azienda può essere distribuita geograficamente su più edifici molto lontani tra loro. In questo caso può risultare più economico avere una LAN in ogni edificio e collegarle poi mediante dei bridge e qualche collegamento in fibra ottica a lunga distanza per far convergere tutti i cavi verso un singolo switch centrale. Sebbene posare i cavi sia facile, ci sono limiti alla loro lunghezza (come i 200 metri per i doppini di Gigabit Ethernet). La rete non funzionerebbe con cavi più lunghi a causa dell'eccessiva attenuazione del segnale o del ritardo di propagazione in andata e ritorno (*round-trip delay*). L'unica soluzione è la divisione della LAN e l'installazione di bridge per unire le parti e aumentare la distanza fisica totale che può essere coperta.

Terzo, per reggere il carico di lavoro può essere necessario dividere una singola LAN logica in più LAN fisiche (collegate da bridge). In molte grandi università, per esempio, migliaia di stazioni di lavoro sono a disposizione di studenti e docenti. Anche le aziende possono avere migliaia di

impiegati. L'ordine di grandezza di questi sistemi preclude di mettere tutte le stazioni di lavoro in una singola LAN (i computer sono più numerosi sia delle porte che è possibile trovare su un singolo hub Ethernet sia del numero massimo di stazioni consentite in una singola Ethernet classica).

Anche se fosse possibile cablare insieme tutte le workstation, aggiungendo un certo numero di stazioni su un hub Ethernet o una Ethernet classica, questo non aumenterebbe la capacità del sistema. Tutte le stazioni condividono la stessa quantità fissata di banda; più stazioni ci sono, meno sarà la banda media disponibile per ognuna.

Tuttavia, due LAN separate hanno il doppio della capacità di una LAN singola; i bridge permettono di unire le LAN mantenendo questa capacità. La soluzione è di non spedire traffico su porte dove non è richiesto, perciò ogni LAN può viaggiare a piena velocità. Questo comportamento aumenta anche l'affidabilità, dato che con una singola LAN un nodo difettoso che produce un flusso ininterrotto di spazzatura può intasare l'intera LAN. Decidendo che cosa inoltrare e che cosa no, i bridge agiscono come porte tagliafuoco in un edificio: impediscono che un singolo nodo impazzito metta in crisi l'intero sistema.

Per rendere questi vantaggi facilmente disponibili, i bridge ideali dovrebbero essere completamente trasparenti.[5] Dovrebbe essere possibile uscire a comprare dei bridge, collegare i cavi delle LAN ai bridge e avere tutto perfettamente funzionante in un istante. Non ci dovrebbero essere modifiche né all'hardware, né ai software richiesti, né impostazioni di indirizzi sugli switch, né si dovrebbero scaricare tabelle di instradamento o parametri, niente di niente; solo collegare i cavi e via. Inoltre, il funzionamento delle LAN esistenti non dovrebbe essere influenzato minimamente dai bridge. Per quanto interessa le stazioni, non dovrebbe esserci alcuna differenza osservabile che siano o meno parte di una LAN che fa uso di bridge. Dovrebbe essere facile spostare stazioni in una LAN con bridge tanto quanto spostarle in una singola LAN. Abbastanza sorprendentemente è realmente possibile creare bridge trasparenti. Vengono usati due algoritmi: un algoritmo di backward learning (*apprendimento a ritroso*) per evitare che il traffico sia spedito dove non è necessario, e un algoritmo di spanning tree (*albero di copertura*) per interrompere i cicli che possono formarsi quando degli switch vengono cablati insieme, volenti o nolenti. Diamo ora uno sguardo a questi algoritmi per imparare come si compie una simile magia.

Bridge con apprendimento

La topologia di due LAN collegate tramite bridge è illustrata nella Figura 4.41 per due casi. A sinistra, due LAN multidrop, come Ethernet classiche, sono unite da una stazione speciale (il bridge) che sta su entrambe le LAN. A destra, LAN cablate punto a punto, incluso un hub, sono unite insieme. I bridge sono i dispositivi a cui stazioni e hub sono agganciati. Se la tecnologia LAN è Ethernet, i bridge sono meglio conosciuti come switch Ethernet.

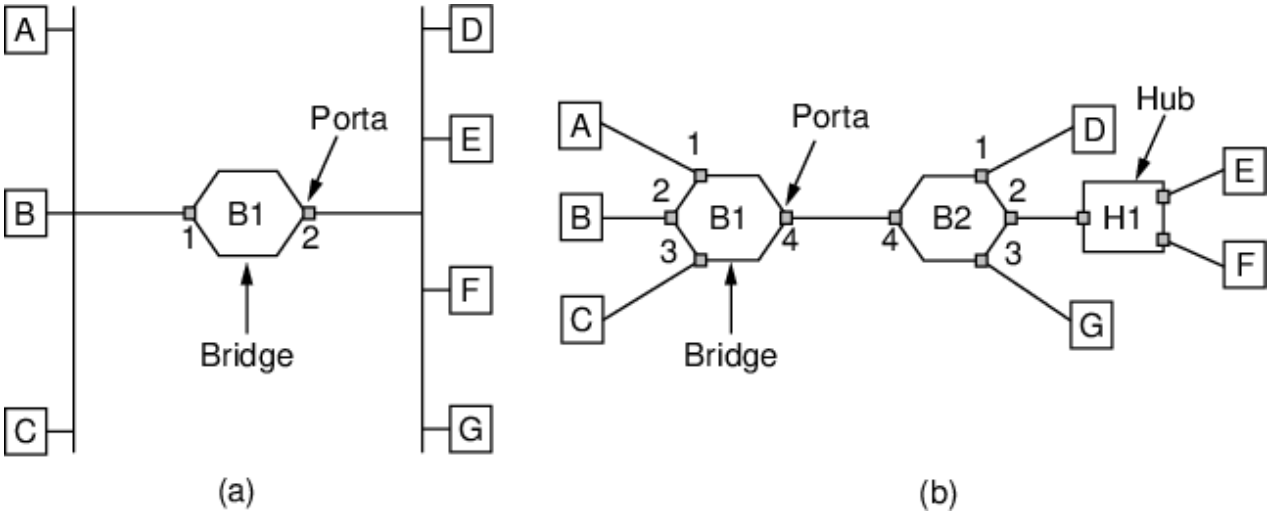


Figura 4.41 (a) Un bridge che connette due LAN multidrop. (b) Due bridge (e un hub) che connettono sette stazioni punto a punto.

I bridge si diffusero ai tempi di Ethernet classica, quindi vengono usati spesso in presenza di cavi multidrop, come nella Figura 4.41(a). Tuttavia, tutte le topologie che si incontrano oggi comprendono cavi punto a punto e switch; i bridge funzionano allo stesso modo in entrambi gli ambienti. Tutte le stazioni collegate alla stessa porta su di un bridge appartengono allo stesso dominio di collisione, diverso dal dominio di collisione delle altre porte. Se c'è più di una stazione, come in Ethernet classica, un hub o un collegamento half-duplex, il protocollo CSMA/CD viene utilizzato per spedire i frame.

C'è una differenza, tuttavia, in come sono costruite le LAN con bridge. Per collegare delle LAN multidrop, un bridge è aggiunto come una nuova stazione su ognuna delle LAN multidrop, come nella Figura 4.41(a). Per collegare LAN punto a punto, gli hub o sono connessi al bridge o, preferibilmente, sono rimpiazzati con un bridge per aumentare le prestazioni. Nella Figura 4.41(b) i bridge hanno rimpiazzato tutti gli hub tranne uno.

A un bridge possono essere collegati diversi tipi di cavo. Per esempio, il cavo che connette il bridge *B*1 al bridge *B*2 nella Figura 4.41(b) può essere un collegamento in fibra ottica a lunga distanza, mentre il cavo che connette i bridge alle stazioni può essere una linea a breve distanza realizzata con un doppino. Questa disposizione è utile per collegare delle LAN in edifici diversi.

Consideriamo ora che cosa succede dentro ai bridge. Ogni bridge opera in modalità promiscua, cioè, accetta ogni frame trasmesso dalle stazioni collegate a ognuna delle sue porte. Il bridge deve decidere se inoltrare o scartare ogni frame e, se lo inoltra, su quale porta far uscire il frame. Questa decisione è presa utilizzando l'indirizzo di destinazione. Per esempio, consideriamo la topologia della Figura 4.41(a). Se la stazione *A* spedisce un frame alla stazione *B*, il bridge *B*1 riceverà il frame sulla porta 1. Questo frame può essere immediatamente scartato senza indugi, perché è già sulla porta corretta. Tuttavia, nella topologia della Figura 4.41(b) supponiamo che *A* spedisca un frame a *D*. Il bridge *B*1 riceverà il frame sulla porta 1 e lo farà uscire sulla porta 4. Il bridge *B*2 riceverà quindi il frame sulla sua porta 4 e lo farà uscire sulla sua porta 1.

Un modo semplice per implementare questo schema è di avere una grande tabella (hash) dentro il bridge. La tabella può contenere tutte le possibili destinazioni e le porte a queste associate. Per esempio, nella Figura 4.41(b), la tabella di *B*1 indica *D* come appartenente alla porta 4, poiché tutto ciò che *B*1 deve sapere è su quale porta deve mettere i frame per raggiungere *D*; successivamente, infatti, non interessa a *B*1 se averranno altri inoltri quando il frame arriverà a *B*2.

Quando i bridge vengono collegati per la prima volta, tutte le tabelle hash sono vuote. Nessuno dei bridge sa dove siano le varie destinazioni, quindi essi devono utilizzare un algoritmo di flooding (*inondazione*): ogni frame in entrata per una destinazione sconosciuta viene fatto uscire da tutte le porte su cui il bridge è connesso, eccetto quello da cui è arrivato. Col passare del tempo i bridge imparano dove sono le destinazioni. Una volta che la destinazione è nota, i frame destinati a essa sono semplicemente posti sulla porta corretta e non più replicati.

L'algoritmo utilizzato dai bridge è il **backward learning**. Come accennato in precedenza, i bridge operano in modo promiscuo, quindi vedono ogni frame spedito su ognuna delle loro porte. Guardando l'indirizzo sorgente, essi possono dire quali macchine sono accessibili su quali porte. Per esempio, se il bridge *B*1 nella Figura 4.41(b) vede un frame sulla porta 3 che arriva da *C*, sa che *C* deve essere raggiungibile attraverso la porta 3, quindi lo segna sulla sua tabella hash. Tutti i frame successivi indirizzati a *C* che arrivano a *B*1 su qualsiasi altra porta saranno inoltrati attraverso la porta 3.

La topologia può cambiare, poiché le macchine e i bridge vengono accesi, spenti e spostati. Per gestire topologie dinamiche, ogni volta che un elemento viene aggiunto alla tabella hash, viene preso nota anche del tempo di arrivo del frame; ogni qualvolta arriva un frame la cui sorgente è già nella tabella, questo tempo viene aggiornato. Quindi, il tempo associato a ogni elemento della tabella ci indica l'ultima volta che è stato visto un frame da una certa macchina.

Periodicamente un processo interno al bridge scandisce la tabella di hash ed elimina tutti gli elementi più vecchi di qualche minuto. In questo modo, se un computer viene scollegato dalla propria LAN, spostato nell'edificio e collegato ancora da qualche altra parte, sarà subito di nuovo funzionante, senza alcun intervento manuale. Questo algoritmo significa anche che se una macchina non produce traffico per alcuni minuti, il traffico a essa indirizzato non dovrà essere replicato su tutte le porte finché essa stessa non invierà un frame.

La procedura di routing[6] per un frame in ingresso dipende dalla porta da cui arriva (la porta sorgente) e dall'indirizzo a cui è destinato (l'indirizzo di destinazione) ed è la seguente:

1. se la porta per raggiungere l'indirizzo di destinazione è la stessa della porta sorgente, scarta il frame;
2. se la porta per raggiungere l'indirizzo di destinazione e la porta sorgente sono diverse, inoltra il frame sulla porta di destinazione;
3. se la porta di destinazione è sconosciuta, utilizza il flooding e spedisce il frame su tutte le porte tranne la porta sorgente.

Ci si potrebbe chiedere se il primo caso si possa verificare con collegamenti punto a punto. La risposta è che può succedere se gli hub sono utilizzati per connettere un gruppo di calcolatori a un bridge. Un esempio è illustato nella Figura 4.41(b), dove le stazioni *E* e *F* sono connesse nell'hub *H*1, connesso a sua volta al bridge *B*2. Se *E* invia un frame a *F*, l'hub lo ritrasmetterà a *B*2 come a

F. Questo è ciò che fanno gli hub: collegano tutte le porte insieme, quindi un frame in input su una porta è semplicemente un output su tutte le altre. Il frame arriverà a *B*2 sulla porta 4, che è già la porta di output giusta per raggiungere la destinazione. Il bridge *B*2 deve solo scartare il frame.

Questo algoritmo deve essere applicato all'arrivo di ogni frame, viene quindi implementato di solito con degli appositi chip VLSI. I chip interrogano e aggiornano gli elementi della tabella, tutto in pochi microsecondi. Poiché i bridge guardano solo l'indirizzo MAC per decidere come inoltrare il frame è possibile iniziare a inoltrare non appena è arrivato il campo destinazione all'interno dell'intestazione e prima che il resto del frame sia arrivato (ovviamente a condizione che la linea di output sia disponibile). Questo approccio riduce la latenza introdotta dal passare attraverso il bridge, così come il numero di frame che il bridge deve essere in grado di mantenere nella memoria interna. Si tratta di **cut-through switching** o **wormhole routing** e di solito è gestito via hardware.

Possiamo guardare al funzionamento di un bridge in termini di stack di protocolli per capire che cosa voglia dire essere un dispositivo a livello data link. Consideriamo un frame spedito da una stazione *A* a una stazione *D* nella configurazione della Figura 4.41(a), in cui le LAN sono Ethernet. Il frame passerà attraverso il bridge e il processo dal punto di vista dello stack di protocolli è illustrato nella Figura 4.42.

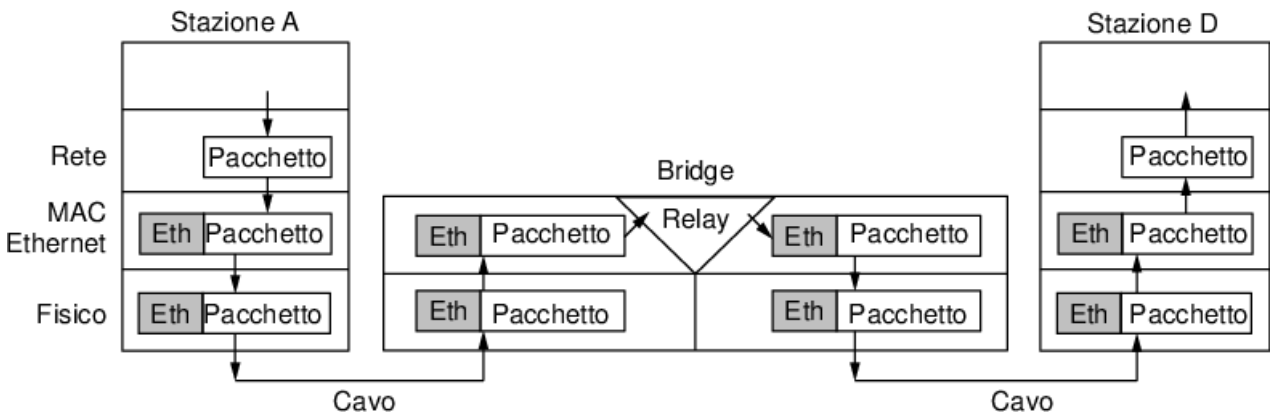


Figura 4.42 Elaborazione dei protocolli in un bridge.

Il pacchetto arriva dal livello superiore e scende nel livello MAC di Ethernet. Acquisisce un header Ethernet (e anche un trailer, che non si vede nella figura). L'unità dati risultante viene passata al livello fisico, viene spedita tramite il cavo e raccolta dal bridge.

Nel bridge il frame viene passato dal livello fisico al livello MAC di Ethernet, livello che ha delle capacità di elaborazione migliori dello stesso livello su una stazione. Passa il frame a un ripetitore (*relay*), ancora all'interno del livello MAC. La funzione di ripetitore del bridge usa solo l'intestazione MAC di Ethernet per determinare come gestire il frame. In questo caso passa il frame al livello MAC di Ethernet della porta usata per raggiungere la stazione *D* e il frame continua sulla sua strada.

In generale, i ripetitori di un dato livello possono sostituire le intestazioni per quel livello: le VLAN forniranno un breve esempio. In nessun caso il bridge dovrà guardare all'interno del frame e capire che cosa sta trasportando un pacchetto IP; ciò è irrilevante per l'elaborazione del bridge e violerebbe la stratificazione dei protocolli. Si noti anche che un bridge con k porte avrà k istanze di livelli MAC e fisici; il valore di k è 2 per il nostro semplice esempio.

Bridge con spanning tree

Per aumentare l'affidabilità possono essere utilizzati collegamenti ridondanti tra i bridge. Nell'esempio della Figura 4.43 ci sono due collegamenti in parallelo tra un paio di bridge. Questa architettura assicura che, se fosse tagliato un collegamento, la rete non sarebbe divisa in due gruppi di calcolatori che non si possono parlare.

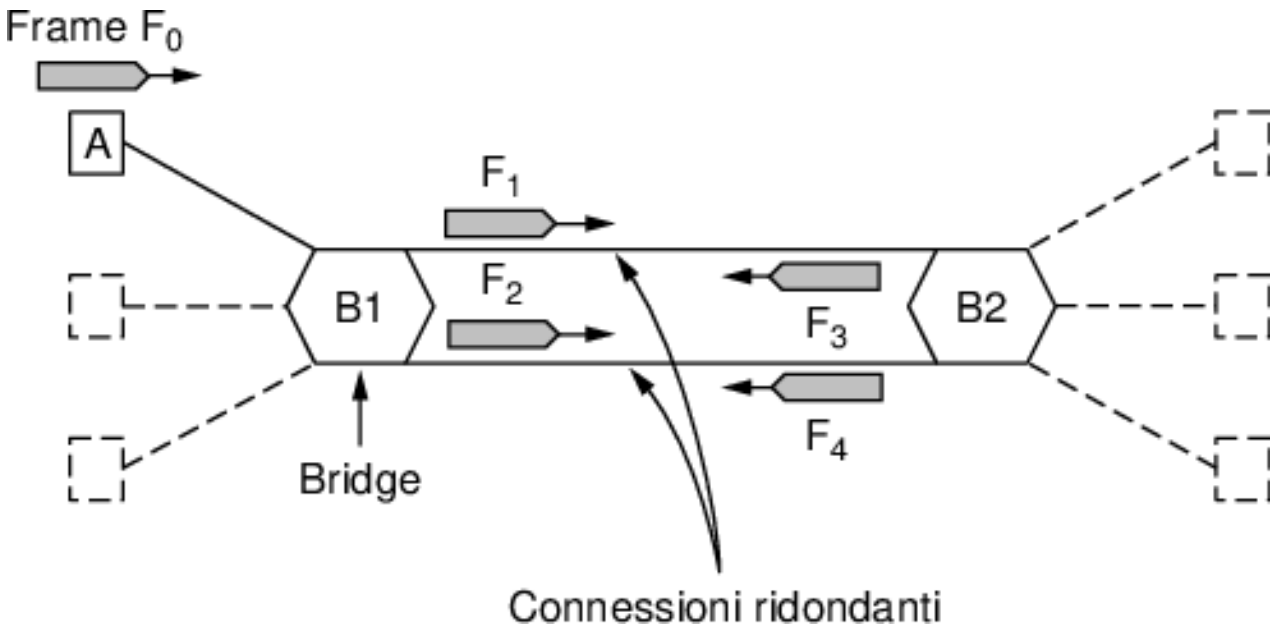


Figura 4.43 Bridge con due collegamenti paralleli.

Tuttavia, questa ridondanza introduce ulteriori problemi, perché crea dei cicli nella topologia. Un esempio di questi problemi può essere visto nella Figura 4.43 guardando come viene gestito un frame spedito da A verso una destinazione mai osservata prima. Ogni bridge segue la normale regola per gestire destinazioni sconosciute, cioè il flooding. Chiamiamo il frame proveniente da A che raggiunge il bridge $B1$ frame F_0. Il bridge spedisce copie di questo frame da tutte le sue altre porte. Considereremo solo le porte del bridge che connettono $B1$ a $B2$ (anche se il frame verrà inviato a tutte le altre porte). Dato che ci sono due collegamenti tra $B1$ e $B2$, due copie del frame (F_1 e F_2) raggiungeranno $B2$, come mostato nella Figura 4.43.

Poco dopo, il bridge *B*2 riceve questi frame. Tuttavia, non sa (e non può sapere) che sono copie dello stesso frame piuttosto che due frame diversi spediti uno dopo l'altro. Quindi il bridge *B*2 prende F_1 e ne spedisce copie a ogni sua altra porta; prende anche F_2 e ne spedisce copie a ogni sua altra porta. Questo produce F_3 ed F_4 che vengono spediti lungo i collegamenti a *B*1. Il bridge *B*1 vede due nuovi frame con destinazione sconosciuta e li copia ancora; questo ciclo va avanti per sempre.

La soluzione a questo problema è che i bridge comunichino l'un l'altro e che sovrappongano alla topologia fisica un albero di copertura in grado di raggiungere ogni bridge. Nella pratica alcuni collegamenti potenziali tra bridge vengono ignorati per poter costruire una topologia virtuale esente da cicli che sia un sottoinsieme di quella fisica.

Per esempio, nella Figura 4.44 vediamo cinque bridge connessi e che hanno anche delle stazioni a loro connesse (ogni stazione si connette a un solo bridge). Ci sono alcuni collegamenti ridondanti tra i bridge, quindi se vengono usati tutti i collegamenti i frame saranno inoltrati su dei cicli. Questa topologia può essere pensata come un grafo in cui i bridge sono i nodi e i collegamenti punto a punto sono gli archi. Il grafo può essere ridotto a uno spanning tree, che per definizione non ha cicli, ignorando i collegamenti indicati con linee tratteggiate nella Figura 4.44. Usando lo spanning tree c'è sempre esattamente un percorso tra ogni coppia di stazioni. Una volta che i bridge hanno raggiunto un accordo sullo spanning tree tutte le operazioni di inoltro si svolgono su di esso; siccome c'è un unico percorso da ogni sorgente a ogni destinazione i cicli sono impossibili.

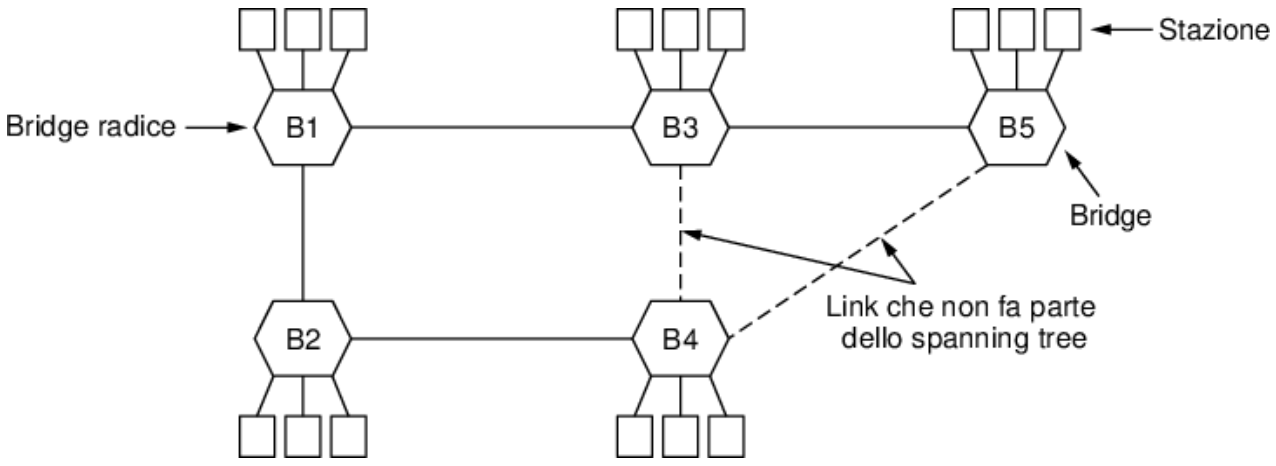


Figura 4.44 Uno spanning tree che collega cinque bridge. Le linee tratteggiate sono link che non fanno parte dell'albero.

Per costruire lo spanning tree i bridge eseguono un algoritmo distibuito: ogni bridge, periodicamente, trasmette un messaggio di configurazione da tutte le sue porte ai suoi vicini ed elabora i messaggi che riceve dagli altri bridge, come descritto in seguito. Questi messaggi non sono inoltrati, perché il loro obiettivo è quello di costruire l'albero, che può poi essere utilizzato per inoltrare i frame.

I bridge devono prima scegliere un bridge per costituire la radice dello spanning tree. A questo scopo ognuno include un identificatore basato sul proprio indirizzo MAC nel messaggio di configurazione, così come l'identificatore del bridge ritenuto la radice. Gli indirizzi MAC sono installati dal produttore e garantiti essere unici in tutto il mondo, il che rende questi identificatori pratici e unici. I bridge scelgono il bridge con l'identificatore più basso come radice. Una volta scambiati abbastanza messaggi per diffondere la notizia, tutti i bridge concorderanno su quale di loro sarà la radice. Nella Figura 4.44 il bridge *B*1 ha l'identificatore più basso e diventa la radice.

Viene quindi costruito un albero con i percorsi più brevi dalla radice a ogni bridge. Nella Figura 4.44 i bridge *B*2 e *B*3 possono entrambi essere raggiunti direttamente da *B*1 in un solo passaggio (detto anche *hop*), che è il percorso più breve. Il bridge *B*4 può essere raggiunto in due hop attraverso sia *B*2 sia *B*3; per risolvere questa indecisione viene scelto il percorso attraverso il bridge con l'identificativo più basso, quindi *B*4 è raggiunto attraverso *B*2. Il bridge *B*5 può essere raggiunto in due hop attraverso *B*3.

Per trovare questi percorsi minimi i bridge includono la loro distanza dalla radice nei messaggi di configurazione. Ogni bridge ricorda il percorso più breve individuato verso la radice e spengono le porte che non fanno parte del tragitto più breve.

Sebbene l'albero si estenda a tutti i bridge, non tutti i collegamenti (o anche i bridge) sono necessariamente presenti. Questo succede perché spegnendo le porte si tagliano alcuni collegamenti della rete per prevenire cicli; anche dopo che lo spanning tree è stato istituito, l'algoritmo continua a essere eseguito durante il normale funzionamento per scoprire automaticamente cambi di topologia e aggiornare l'albero.

L'algoritmo per costruire lo spanning tree fu inventato da Radia Perlman; il suo compito fu quello di risolvere il problema di come collegare delle LAN senza fare dei cicli. Le diedero una settimana per farlo, ma se ne uscì con l'idea dell'algoritmo dello spanning tree in un giorno. Fortunatamente, questo le lasciò abbastanza tempo per comporlo in poesia (*Perlman*, 1985):

> *I think that I shall never see*
> *A graph more lovely than a tree. A tree whose crucial property Is loop-free connectivity.*
> *A tree which must be sure to span. So packets can reach every LAN. First the Root must be selected By ID it is elected.*
> *Least cost paths from Root are traced In the tree these paths are placed. A mesh is made by folks like me*
> *Then bridges find a spanning tree.*

L'algoritmo dello spanning tree fu quindi standardizzato come IEEE 802.1D e utilizzato per molti anni. Nel 2001 fu revisionato per individuare più rapidamente un nuovo spanning tree dopo il cambio di topologia. Per una trattazione dettagliata dei bridge si veda Perlman (2000).

Repeater, hub, bridge, switch, router e gateway

I paragrafi precedenti hanno descritto diverse tecniche che consentono di far passare frame e pacchetti da un computer all'altro. Abbiamo già accennato a repeater, bridge, switch, hub, router e gateway. Tutti questi dispositivi sono di uso comune, ma differiscono l'uno dall'altro in modo più o meno sottile. Poiché sono numerosi, vale probabilmente la pena di esaminarli tutti insieme per metterli a confronto.

L'aspetto principale è la comprensione di come essi operino su livelli diversi, come mostrato nella Figura 4.45(a). Il livello è importante perché dispositivi diversi utilizzano differenti porzioni di informazione per stabilire come eseguire la commutazione. In uno scenario tipico, l'utente genera alcuni dati da inviare a una macchina remota; quei dati vengono passati al livello di trasporto che aggiunge un'intestazione, per esempio un'intestazione TCP, e passa il risultato al sottostante livello di rete. Questo aggiunge la propria intestazione, in modo da costruire il pacchetto del livello di rete, per esempio un pacchetto IP. Nella Figura 4.45(b) il pacchetto IP è evidenziato in grigio. Il pacchetto viene quindi trasferito al livello data link, che aggiunge la propria intestazione e il proprio checksum (CRC) e passa il frame risultante al livello fisico che gestisce la trasmissione, per esempio attraverso una LAN.

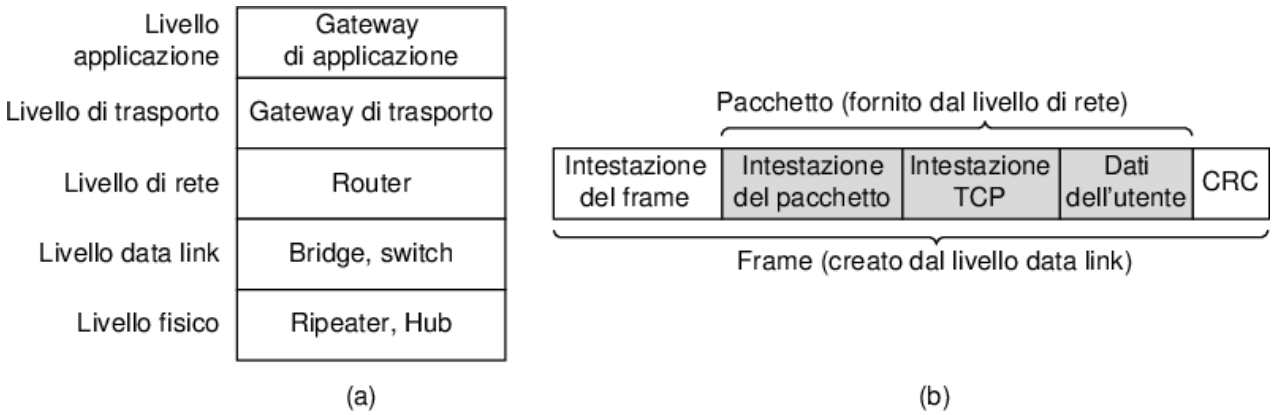


Figura 4.45 (a) Posizione dei dispositivi. (b) Frame, pacchetti e intestazioni.

Diamo ora uno sguardo ai dispositivi di commutazione e vediamo come si relazionano con pacchetti e frame. I repeater si trovano sul fondo, nel livello fisico. Sono dispositivi analogici che gestiscono i segnali sui (due) cavi a cui sono collegati. Un segnale che appare su un cavo è ripulito dal rumore, amplificato e trasmesso sull'altro. I repeater non sanno nulla dei frame, dei pacchetti e delle intestazioni; essi comprendono solo i simboli che codificano i bit in volt. Ethernet classica, per esempio, è stata progettata per supportare quattro repeater per potenziare il segnale ed estendere la lunghezza massima del cavo da 500 a 2500 metri.

Passiamo agli hub. Un hub ha diverse linee di input collegate elettricamente. I frame che arrivano su una di queste linee sono trasmessi attraverso tutte le altre. Due frame che arrivano contemporaneamente collidono, proprio come su un cavo coassiale. Tutte le linee che si collegano all'hub devono operare alla stessa velocità. A differenza dei repeater, gli hub non amplificano (di solito) i segnali in ingresso e sono progettati per avere diverse linee di input, ma le differenze sono lievi. Come i repeater, gli hub sono dispositivi di livello fisico che non esaminano né utilizzano in alcun modo gli indirizzi di livello data link.

Spostiamoci ora verso l'alto al livello data link, dove troviamo bridge e switch. Abbiamo appena analizzato i bridge abbastanza approfonditamente. Un bridge collega due o più LAN; come un hub, un bridge moderno ha molte porte, di solito in numero sufficiente per soddisfare da 4 a 48 linee di input di un dato tipo. Diversamente da un hub, ogni porta è isolata per avere un suo proprio dominio di collisione; se la porta ha una linea full duplex punto a punto, l'algoritmo CSMA/CD non è necessario. Quando arriva un frame, il bridge estrae l'indirizzo di destinazione dall'intestazione del frame e lo cerca in una tabella per vedere dove spedire il frame. Per Ethernet, questo indirizzo è l'indirizzo di destinazione di 48 bit che abbiamo visto nella Figura 4.14. Il bridge mette solo il frame sulla porta di output necessaria e può spedire più frame contemporaneamente.

I bridge offrono prestazioni molto migliori degli hub e l'isolamento tra le porte dei bridge significa anche che le linee di input possono viaggiare a velocità diverse, eventualmente anche su tipi di rete diversi. Un esempio comune è un bridge con porte connesse a Ethernet da 10, 100 e 1000 Mbps. È neccessario un buffer all'interno del bridge per accettare un frame da una porta e trasmetterlo su di una porta diversa. Se i frame arrivano più velocemente di quanto possano essere ritrasmessi, il bridge può esaurire lo spazio nel buffer e deve cominciare a scartarne. Per esempio, se una gigabit Ethernet sta riversando bit in una Ethernet da 10 Mbps alla massima velocità, il bridge dovrà metterli nel suo buffer sperando di non esaurire la memoria. Questo problema sussiste anche se tutte le porte vanno alla stessa velocità, perché è possibile che più di una porta stia inviando frame a una data porta di destinazione.

I bridge furono ideati originariamente per essere capaci di unire differenti tipi di LAN, per esempio una Ethernet e una Token Ring. Tuttavia la cosa non funzionò mai bene a causa delle differenze tra le LAN. Formati di frame diversi richiedono di essere copiati e riformattati, il che richiede tempo alla CPU, richiede un nuovo calcolo per il checksum e introduce la possibilità di errori non trovati a causa di bit danneggiati nella memoria del bridge. Anche le differenti lunghezze massime dei frame rappresentano un serio problema, per cui non si hanno ancora soluzioni efficaci; fondamentalmente, i frame troppo grandi per essere inoltrati devono essere scartati e allora addio trasparenza.

Sicurezza e qualità di servizio sono altre due aree in cui le LAN possono differire. Alcune, come 802.11, sono dotate di cifratura a livello data link e altre, come Ethernet, invece no. Alcune LAN, per esempio 802.11, hanno funzionalità per la qualità di servizio come le priorità, mentre altre no, come Ethernet. Di conseguenza, quando un frame deve viaggiare tra queste LAN potrebbe non essere possibile fornire la sicurezza o la qualità di servizio attesi dal mittente. Per tutti questi motivi, i bridge moderni di solito funzionano per un solo tipo di rete e i router, di cui parleremo presto, sono invece utilizzati per collegare reti di tipo diverso.

Gli switch sono bridge moderni, ma con un nome diverso. Le differenze sono più di marketing che di tecnica, ma ci sono alcuni aspetti che val la pena chiarire. I bridge si sono evoluti quando veniva usata Ethernet classica, quindi essi tendono a unire relativamente poche LAN e perciò ad avere relativamente poche porte. Oggi il termine "switch" è quello più popolare; inoltre, tutte le installazioni moderne utilizzano collegamenti punto a punto, come i doppini; i singoli computer si collegano direttamente a uno switch, per cui gli switch tendono ad avere molte porte. Infine, "switch" è anche utilizzato come termine generico. La funzionalità di un bridge è chiara; con uno switch, invece, ci si può riferire a uno switch Ethernet o a un dispositivo completamente diverso che prende decisioni nell'inoltro, come un permutatore telefonico.

Dopo aver esaminato repeater e hub, abbastanza simili tra loro, e poi bridge e switch, che sono ancora più simili, è giunto il momento di analizzare i router, che non hanno nulla in comune con i precedenti. Quando un pacchetto raggiunge un router, l'intestazione e la coda del frame vengono eliminati e il pacchetto contenuto nel payload del frame (evidenziato in grigio nella Figura 4.45) è passato al software di instradamento (*routing*), che sfrutta l'intestazione del pacchetto per scegliere la linea di output. Nel caso di un pacchetto IP, l'intestazione del pacchetto contiene un indirizzo a 32 bit (IPv4) o un indirizzo a 128 bit (IPv6), ma non un indirizzo IEEE 802 a 48 bit. Il software di routing non vede gli indirizzi del frame e non sa nemmeno se il pacchetto arriva da una LAN o da una linea punto a punto. Studieremo i router e il routing nel Capitolo 5.

Al livello ancora superiore troviamo i gateway di trasporto, che connettono due computer che usano protocolli di trasporto orientati alle connessioni differenti. Si supponga per esempio che un computer che utilizza il protocollo TCP/IP orientato alla connessione debba comunicare con un computer che usa un diverso protocollo di trasporto orientato alla connessione chiamato SCTP. Il gateway di trasporto può copiare i pacchetti provenienti da una connessione all'altra, modificando il formato secondo necessità.

Infine, i gateway a livello di applicazione comprendono il formato e il contenuto dei dati e possono tradurre i messaggi da un formato all'altro. Un gateway per la posta elettronica, per esempio, potrebbe tradurre i messaggi Internet in messaggi SMS per i telefoni cellulari. Come "switch", "gateway" è in qualche modo un termine generale: si riferisce a un processo di inoltro eseguito a un livello superiore.

Virtual LAN

Ai primi esordi delle reti locali, grossi cavi gialli si snodavano attraverso i condotti di molti palazzi di uffici; ogni computer vicino al quale passavano vi era collegato. Non si pensò a quale computer appartenesse a quale LAN; tutte le persone in uffici vicini erano messe sulla stessa LAN, sia che fossero nello stesso gruppo sia in gruppi differenti. La geografia trionfava sugli organigrammi aziendali.

Con l'avvento del doppino e degli hub negli anni '90 tutto questo cambiò. Gli edifici furono cablati di nuovo (con grandi spese) per sradicare tutti i cavi gialli e installare doppini che collegavano ogni ufficio a una centralina collocata alla fine di ogni corridoio o in una sala macchine centrale, come mostrato nella Figura 4.46. Se chi gestiva il cablaggio era un attento pianificatore, nell'edificio venivano installati doppini di Categoria 5; chi invece preferiva risparmiare utilizzava i cavi telefonici esistenti (di Categoria 3) solo per sostituirli qualche anno più tardi alla comparsa di fast Ethernet.

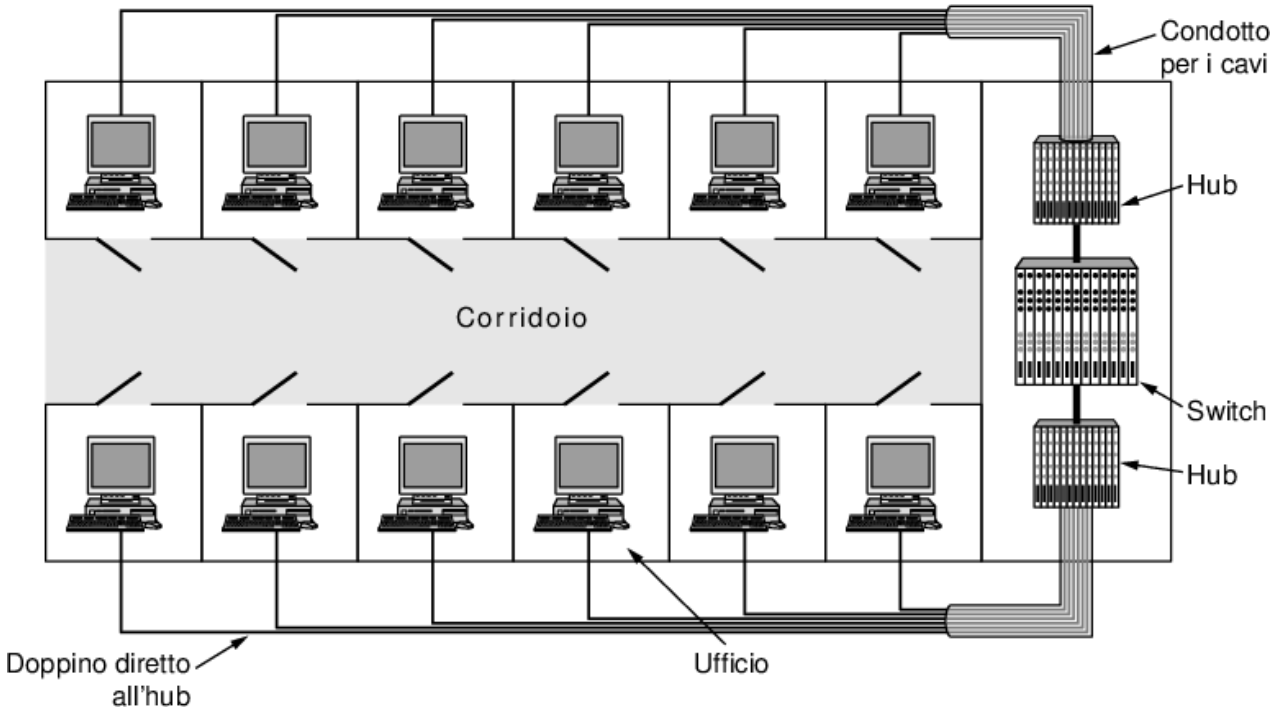


Figura 4.46 Un edificio con cablaggio centralizzato che utilizza hub e switch.

Oggi i cavi sono cambiati e gli hub sono diventati switch, ma lo schema di cablaggio è sempre lo stesso. Questo schema rende possibile configurare LAN logiche piuttosto che fisiche. Per esempio, se un'azienda vuole *k* LAN può decidere di comprare *k* switch. Scegliendo con attenzione quale connettore inserire in quale switch gli utilizzatori di una LAN possono essere scelti in base all'organigramma aziendale e non alla geografia.

Ha importanza sapere chi è collegato a quale LAN? Dopo tutto, in qualsiasi azienda tutte le LAN sono connesse. La risposta è affermativa: spesso è importante. Per molte ragioni gli amministratori di rete amano raggruppare gli utenti su LAN che riflettano la struttura dell'azienda, invece dello schema fisico dell'edificio. La prima ragione è la sicurezza; una LAN può ospitare dei Web server e altri computer destinati a un utilizzo pubblico. Un'altra LAN può ospitare dei computer contenenti gli archivi del dipartimento delle Risorse Umane che non devono arrivare all'esterno. In questa situazione ha senso mettere tutti i computer in una sola LAN e non permettere ad alcun server esterno alla LAN di accedervi. I dirigenti tendono a infastidirsi quando sentono dire che una configurazione simile è impossibile.

Un secondo problema è il carico della rete. Alcune LAN sono utilizzate più intensamente di altre e talvolta può essere utile separarle. Per esempio, se chi si occupa di ricerca esegue gli esperimenti più strani che qualche volta sfuggono di mano saturando la LAN, chi lavora in direzione potrebbe non essere entusiasta di cedere un po' della propria capacità di rete, che utilizzava ad esempio per videoconferenze.

Un terzo aspetto critico è quello del traffico broadcast (cioè, diretto a tutti i presenti sulla rete). I bridge trasmettono traffico broadcast quando la posizione della destinazione non è nota; anche i protocolli di livello superiore utilizzano il broadcast. Per esempio, quando un utente vuole spedire

un pacchetto a un indirizzo IP *x*, come fa a sapere quale indirizzo MAC deve inserire nel frame? Studieremo questo problema nel Capitolo 5 ma, per sommi capi, la risposta è che trasmette in broadcast un frame contenente la domanda "chi possiede l'indirizzo IP *x*?" e attende la risposta. Al crescere del numero dei computer in una LAN cresce anche il numero di messaggi broadcast. Ogni frame broadcast consuma la capacità trasmissiva della LAN più di un frame normale, perché è inviato a ogni computer sulla LAN. Tenendo le LAN non più grandi del necessario, l'impatto del traffico broadcast viene ridotto.

Un problema correlato alla trasmissione broadcast si presenta quando, occasionalmente, una scheda di rete si rompe o viene configurata male e inizia a generare un flusso senza fine di frame broadcast. Se la rete è davvero sfortunata, alcuni di questi frame susciteranno risposte che porteranno ad ancora più traffico. Il risultato di questa **tempesta di broadcast** (*broadcast storm*) è che (1) l'intera capacità della LAN è occupata da questi frame e che (2) tutte le macchine su tutte le LAN connesse restano paralizzate, a causa delle operazioni di elaborazione ed eliminazione di tutti i frame trasmessi.

A prima vista si potrebbe pensare di confinare le tempeste broadcast separando le LAN con bridge o switch, ma poiché l'obiettivo è la trasparenza (ossia le macchine devono poter essere spostate da una LAN all'altra senza che si noti la differenza), questi dispositivi devono inoltrare i frame broadcast.

Dopo aver visto perché le aziende potrebbero voler creare LAN distinte e dedicate, è meglio tornare al problema relativo al disaccoppiamento della topologia logica da quella fisica. Costruire una topologia fisica per riflettere la struttura organizzativa può comportare lavoro e costi, anche con un cablaggio centalizzato e switch. Per esempio, se due persone dello stesso dipartimento lavorano in edifici diversi, potrebbe essere più facile collegarli a switch diversi che fanno parte di LAN diverse. Anche se non è questo il caso, un utente può essere spostato all'interno dell'azienda da un dipartimento a un altro senza cambiare ufficio o può cambiare ufficio senza cambiare dipartimento. Ciò potrebbe comportare che l'utente si trovi sulla LAN sbagliata finché un amministratore cambia il connettore dell'utente da un switch a un altro. Inoltre, il numero di computer che appartengono ai diversi dipartimenti può non essere una buona combinazione per il numero di porte sugli switch; alcuni dipartimenti potrebbero essere troppo piccoli e altri così grandi da richiedere switch multipli. Ciò si traduce in uno spreco di porte non utilizzate sugli switch.

In molte aziende le modifiche organizzative si verificano continuamente costringendo gli amministratori di sistema passino un sacco di tempo staccando cavi da una porta e inserendoli in un'altra. Inoltre, in alcuni casi, la modifica non può essere apportata del tutto, perché il doppino che collega la macchina dell'utente è troppo lontano dal giusto switch (per esempio, si trova nell'edificio sbagliato) o le porte disponibili sono sulla LAN sbagliata.

In risposta ai clienti che chiedevano maggiore flessibilità, i produttori di dispositivi di rete cominciarono a lavorare su un metodo per ricablare gli edifici interamente via software. Il concetto risultante è stato chiamato **VLAN** (*virtual LAN*), standardizzato dal comitato IEEE 802, e oggi largamente utilizzato. Ne vedremo per sommi capi il funzionamento; per ulteriori informazioni sulle VLAN si consulti Seifert e Edwards (2008).

Le VLAN si basano su appositi switch (*VLAN-aware*). Per creare una rete basata su VLAN l'amministratore di rete decide quante VLAN dovranno esserci, quali computer collegare e quale nome assegnare a ogni VLAN. Spesso alle VLAN vengono (informalmente) attribuiti nomi di colori, poiché in questo modo è possibile stampare diagrammi a colori che mostrano lo schema fisico delle macchine, con i membri della LAN rossa rappresentati in rosso, quelli della LAN verde rappresentati in verde e così via. In questo modo, con un solo schema è possibile rappresentare sia la disposizione fisica che quella logica della rete.

Come esempio, consideriamo la LAN facente uso di bridge della Figura 4.47, in cui nove delle macchine appartengono alla VLAN G (in grigio) e cinque appartengono alla VLAN N (in nero). Le macchine della VLAN in grigio sono distribuite su due switch, incluse due macchine connesse a uno switch attraverso un hub.

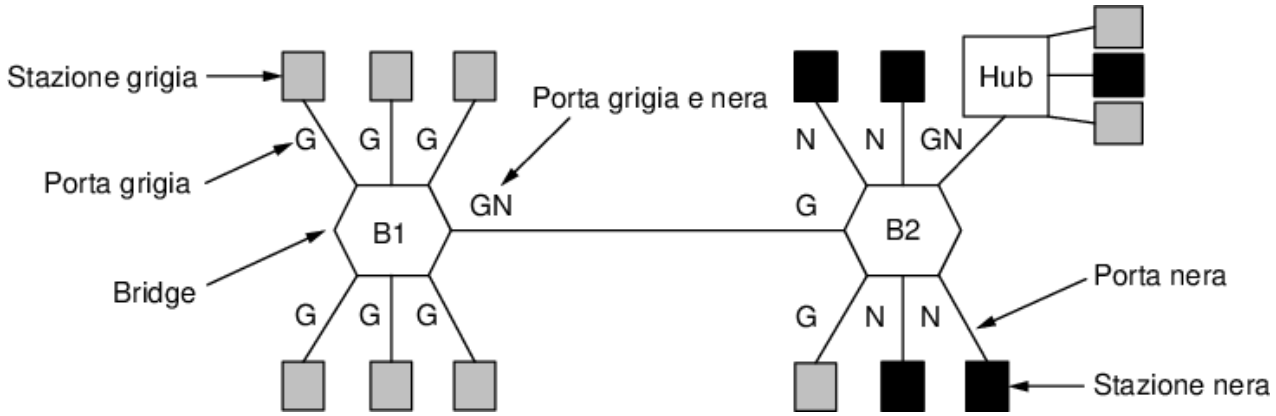


Figura 4.47 Due VLAN, grigie e nere, su una LAN che fa uso di bridge.

Per far funzionare correttamente le VLAN nei bridge devono essere impostate delle tabelle di configurazione. Queste tabelle indicano quali VLAN siano accessibili attraverso quali porte (linee). Quando arriva un frame, per esempio dalla VLAN in grigio, deve essere inoltrato su tutte le porte contrassegnate dalla lettera G. Questo vale per il traffico ordinario (unicast) per il quale i bridge non hanno appreso la posizione della destinazione, così come per il traffico multicast e broadcast. Si noti che una porta può essere etichettata con più di un colore.

Come esempio, supponete che una delle stazioni in grigio collegata al bridge *B*1 nella Figura 4.47 spedisca un frame a una destinazione che non è ancora stata osservata. Il bridge *B*1 riceverà il frame e vedrà che arriva da una macchina della VLAN in grigio, quindi farà flooding del frame a tutte le porte etichettate con G (tranne la porta da cui è arrivato il frame). Il frame sarà spedito alle altre cinque stazioni in grigio collegate a *B*1, così come sul collegamento da *B*1 a *B*2. Nel bridge *B*2 il frame è inoltrato in modo analogo su tutte le porte etichettate con G. Questo spedisce il frame a un'altra stazione e all'hub (che trasmetterà il frame a tutte le sue stazioni). L'hub ha entrambe le

etichette perché si connette a macchine di entrambe le VLAN. Il frame non è spedito a porte che non abbiano G nell'etichetta, perché il bridge sa che non ci sono macchine sulla VLAN in grigio che possano essere raggiunte attraverso queste porte.

Nel nostro esempio, il frame è spedito solo dal bridge *B*1 al bridge *B*2, perché ci sono macchine nella VLAN in grigio connesse a *B*2. Guardando la VLAN in nero, possiamo vedere che la porta del bridge *B*2 che lo connette al bridge *B*1 non è etichettata con N. Questo vuol dire che un frame sulla VLAN in nero non sarebbe inoltrato da *B*2 al bridge *B*1. Un simile comportamento è corretto perché non ci sono stazioni nella VLAN in nero connesse a *B*1.

Standard IEEE 802.1Q

Per implementare questo schema i bridge hanno bisogno di sapere a quale VLAN appartiene un frame in entrata. Senza questa infomazione, quando per esempio il bridge *B*2 riceve un frame dal bridge *B*1 nella Figura 4.47 tale bridge non può sapere se inoltrare il frame sulla VLAN grigia o su quella nera. Se stessimo progettando un nuovo tipo di LAN sarebbe abbastanza facile aggiungere un campo VLAN nell'intestazione. Ma che cosa fare con Ethernet, che è la LAN dominante, e non ha campi liberi in giro per l'identificatore della VLAN?

Al comitato IEEE 802 questo problema è stato presentato nel 1995. Dopo molte discussioni è avvenuto l'impensabile: è stata cambiata l'intestazione di Ethernet. Il nuovo formato è stato pubblicato nel 1998 nello standard IEEE **802.1Q**. Il nuovo formato contiene un'etichetta VLAN che verrà descritta fra poco. Com'è prevedibile, cambiare una cosa così ben radicata come l'intestazione di Ethernet non è stato banale. Alcune domande sorgono spontanee e sono le seguenti.

1. Dopo la modifica sarà necessario sbarazzarsi di centinaia di milioni di schede di rete esistenti?
2. Se no, chi genererà i nuovi campi?
3. Che cosa accadrà ai frame che hanno già la dimensione massima?

Naturalmente, il comitato 802 era (anche troppo) consapevole di questi problemi e dovette trovare alcune soluzioni.

La chiave di tutto sta nel comprendere che i campi VLAN sono effettivamente utilizzati solo dai bridge e dagli switch, non dalle macchine degli utenti. Quindi nella Figura 4.47 non è realmente necessaria la loro presenza sulle linee in uscita collegate alle stazioni finali, a patto che siano presenti sulla linea che collega i bridge. Inoltre, per usare le VLAN i bridge devono essere consci dell'esistenza di 802.1Q (VLAN-aware) e questo rende il progetto realizzabile.

Per quanto riguarda la domanda relativa alla sostituzione di tutte le vecchie schede di rete, la risposta è no. Si ricordi che il comitato 802.3 non era nemmeno riuscito a convincere le persone a trasformare il campo *Type* in *Length*; è quindi facile immaginare la reazione che avrebbe generato l'annuncio della necessità di sostituire tutte le schede Ethernet esistenti. Tuttavia, le nuove schede di rete Ethernet sono compatibili con 802.1Q e possono gestire correttamente i campi VLAN.

Poiché ci possono essere computer (e switch) che non sono VLAN-aware, il primo bridge VLAN-aware che tocca un frame aggiunge i campi VLAN e l'ultimo lungo la strada li rimuove. Un esempio di topologia mista è illustrata nella Figura 4.48, dove i computer VLAN-aware generano frame etichettati (come 802.1Q) direttamente e la permutazione da lì in avanti usa queste etichette. Gli elementi in grigio sono VLAN-aware, mentre quelli vuoti non lo sono.

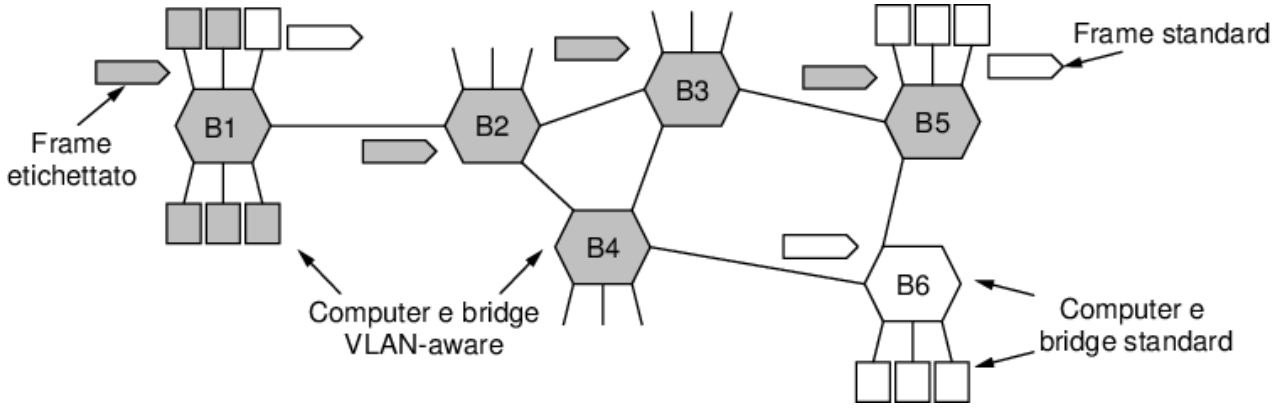


Figura 4.48 Una LAN che fa uso di bridge solo parzialmente VLAN-aware. I simboli evidenziati sono VLAN-aware, gli altri non lo sono.

Con 802.1Q i frame sono colorati a seconda della porta su cui sono ricevuti. Perché questo metodo funzioni, tutte le macchine su un porta devono appartenere alla stessa VLAN, il che riduce la flessibilità. Per esempio, nella Figura 4.48 questa proprietà vale per tutte le porte in cui un singolo computer si connette a un bridge, ma non per la porta in cui l'hub si connette al bridge *B*2.

Inoltre, il bridge può usare il protocollo di livello superiore all'interno del frame per selezionare il colore. In questo modo, i frame che arrivano a una porta possono essere posizionati su diverse VLAN a seconda se stiano trasportando pacchetti IP o frame PPP.

Sono possibili altri metodi, ma non supportati da 802.1Q. Per esempio, l'indirizzo MAC può essere utilizzato per selezionare il colore della VLAN. Questo può essere utile per frame in ingresso da una LAN 802.11 vicina e in cui i laptop spediscono frame attraverso porte diverse non appena si muovono. Un indirizzo MAC può quindi essere associato a una VLAN prefissata, indipendentemente della porta attraverso la quale sia entrato nella LAN.

Per quanto riguarda il problema dei frame più lunghi di 1518 byte, 802.1Q si è limitato ad aumentare il limite a 1522 byte. Fortunatamente, solo i computer e switch VLAN-aware devono supportare questi frame più lunghi.

Diamo ora uno sguardo al formato del frame di 802.1Q, mostrato nella Figura 4.49. L'unica modifica è l'aggiunta di un paio di campi lunghi 2 byte. Il primo campo è *VLAN protocol ID* (*identificativo del protocollo della VLAN*) e il suo valore è sempre 0x8100; poiché questo numero è maggiore di 1500, tutte le schede Ethernet lo interpretano come un tipo e non come una lunghezza. Quello che fanno le vecchie schede di rete con un frame di questo tipo è imprevedibile, in quanto questi frame non sono stati progettati per essere inviati a schede non compatibili.

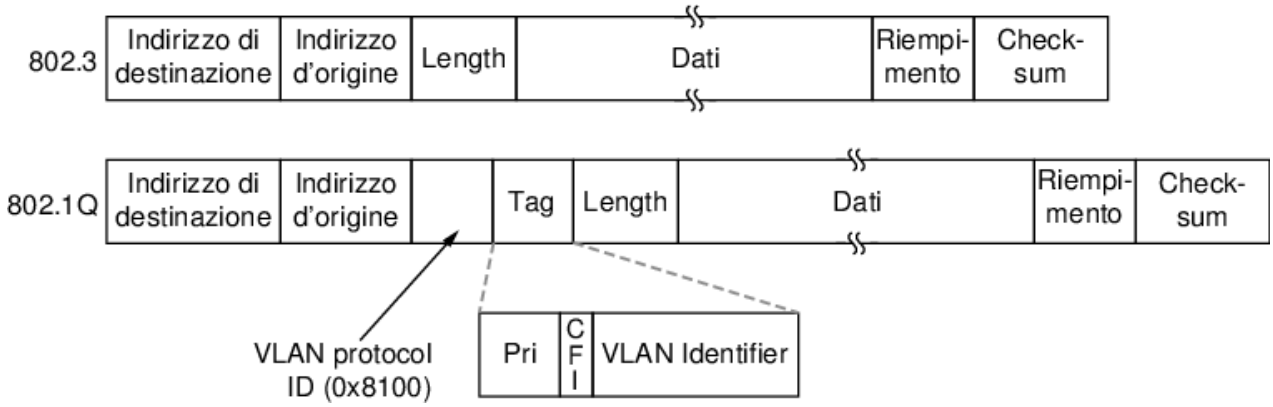


Figura 4.49 I formati di frame di Ethernet 802.3 (originale) e di Ethernet 802.1Q.

Il secondo campo di 2 byte contiene tre sottocampi. Il principale è *VLAN Identifier* (*identificatore della VLAN*) che occupa i 12 bit meno significativi. Il nocciolo della questione è il seguente: il colore della VLAN a cui appartiene il frame. Il campo di 3 bit *Priority* non ha nulla a che fare con le VLAN, ma visto che la modifica dell'intestazione di Ethernet è un evento straordinario che ha richiesto tre anni di lavoro e l'impegno di centinaia di persone, perché non aggiungerci altre cose utili già che ci stiamo lavorando? Questo campo permette di distinguere il traffico hard real-time da quello soft real-time e da quello non sensibile al tempo, in maniera da fornire un migliore supporto per la qualità di servizio su Ethernet. È necessario per la trasmissione della voce su Ethernet (anche se, in tutta sincerità, IP ha avuto un campo simile per un quarto di secolo e nessuno lo ha mai utilizzato).

L'ultimo campo, CFI (*canonical format indicator*), in origine doveva indicare l'ordine dei bit nell'indirizzo MAC (little-endian o big-endian), ma quell'uso si è perso nei meandri di altre controversie. La sua presenza ora indica che il payload contiene un frame 802.5 "liofilizzato" che spera di trovare un'altra LAN 802.5 alla destinazione passando attraverso una Ethernet. Tutta questa struttura, naturalmente, non ha nulla a che fare con le VLAN. Ma la politica del comitato che lavora sugli standard funziona purtroppo come la politica tradizionale: se voti per il mio bit io voterò per il tuo.

Com'è stato detto precedentemente, quando un frame etichettato raggiunge uno switch VLAN-aware, lo switch utilizza l'ID della VLAN come indice nella sua tabella per trovare la porta da utilizzare per la trasmissione. Ma da dove arriva la tabella? Una compilazione manuale della tabella ci riporterebbe punto a capo: comporterebbe una configurazione diretta dei bridge. La bellezza del bridge trasparente è che si tratta di un dispositivo plug-and-play che non richiede alcuna configurazione manuale, quindi perdere questa proprietà sarebbe un vero peccato. Per fortuna anche i bridge VLAN-aware sono in grado di autoconfigurarsi in base all'osservazione delle etichette di passaggio. Se un frame etichettato come VLAN 4 entra dalla porta 3, allora apparentemente qualche macchina collegata alla porta 3 si trova sulla VLAN 4. Lo standard 802.1Q spiega come costruire dinamicamente le tabelle, soprattutto referenziando parti appropriate dello standard 802.1D.

Prima di concludere l'argomento dell'instradamento delle VLAN è utile fare un'ultima osservazione. Molte persone che lavorano nei settori di Internet ed Ethernet sostengono in modo fanatico la tecnologia non orientata alla connessione e si oppongono violentemente a qualunque tipo di connessione nei livelli data link e di rete. Tuttavia le VLAN introducono qualcosa che è sorprendentemente simile a una connessione. Per utilizzare correttamente le VLAN ogni frame trasporta un nuovo identificatore speciale che, utilizzato come indice di una tabella memorizzata nello switch, permette di scoprire la destinazione del frame. Questo è esattamente ciò che accade nelle reti orientate alla connessione. Nelle reti non orientate alla connessione l'instradamento utilizza l'indirizzo di destinazione, non un identificatore della connessione. Questo argomento controverso verrà ripreso nel Capitolo 5.

4.9 Sommario

Alcune reti hanno un solo canale impiegato per tutta la comunicazione. In queste reti il problema chiave è l'assegnazione del canale alle diverse stazioni che competono per poterlo utilizzare. FDM e TDM sono semplici ed efficienti schemi di allocazione quando il numero delle stazioni è piccolo e prefissato e il traffico è continuo. Sono entrambi ampiamente utilizzati in queste circostanze, per esempio, per dividere la lunghezza di banda sui collegamenti telefonici tra centrali. Tuttavia, quando il numero di stazioni è grande e variabile o il traffico è piuttosto irregolare (la norma nelle reti di calcolatori) FDM e TDM sono scelte scadenti.

Sono stati ideati numerosi algoritmi per l'allocazione dinamica di canali. Il protocollo ALOHA, con e senza slot, è utilizzato in molte varianti nei sistemi reali, quali cable modem e RFID. Come miglioramento, quando può essere rilevato lo stato del canale, le stazioni possono evitare di iniziare a trasmettere mentre un'altra stazione lo sta già facendo. Questa tecnica, rilevamento della portante, ha generato una varietà di protocolli CSMA per le LAN e le MAN. È la base di Ethernet classica e delle reti 802.11.

È ben nota anche una classe di protocolli che elimina completamente la contesa o almeno la riduce in modo considerevole. Il protocollo a mappa di bit elimina completamente la contesa, mentre il protocollo tree walk la riduce dividendo dinamicamente le stazioni in due gruppi disgiunti di dimensioni diverse e permettendo la contesa solo all'interno di un gruppo; idealmente il gruppo è scelto in modo da avere una sola stazione pronta a trasmettere quando arriva il momento.

Le LAN wireless hanno dei problemi aggiuntivi: è difficile rilevare le trasmissioni soggette a collisione e le regioni di copertura delle stazioni possono essere diverse tra loro. Nella LAN wireless dominante, IEEE 802.11, le stazioni utilizzano CSMA/CA per mitigare il primo problema lasciando piccoli intervalli liberi per evitare collisioni. Le stazioni possono anche usare il protocollo RTS/CTS per combattere i terminali nascosti che derivano dal secondo problema. IEEE 802.11 è comunemente utilizzata per connettere laptop e altri dispositivi ad access point wireless, ma può anche essere utilizzata tra dispositivi. Si può usare uno qualsiasi dei molti livelli fisici, incluso FDM multicanale con e senza antenne multiple e spettro distribuito.

Come 802.11, anche i lettori e i tag RFID utilizzano un protocollo di accesso casuale per comunicare degli identificatori. Le altre PAN e MAN wireless hanno architetture diverse. Il sistema Bluetooth connette al computer auricolari e molti tipi di periferiche senza bisogno di fili. IEEE 802.16 fornisce su vasta area un servizio wireless di connettività dati per computer fissi e mobili. Entrambe queste reti utilizzano un'architettura centralizzata e orientata alla connessione in cui il master per Bluetooth e la stazione base per WiMAX decidono quando ogni stazione può inviare o ricevere dati. L'architettura di 802.16 supporta diverse qualità di servizio per il traffico in tempo reale come telefonate e traffico interattivo quale la navigazione Web. Per Bluetooth, confinare la complessità nel master genera dispositivi slave economici.

Ethernet è la forma dominante di LAN cablate. Ethernet classica usava CSMA/CD per l'allocazione dei canali su un cavo giallo della dimensione di una manichetta per annaffiare il giardino che serpeggiava da una macchina all'altra. L'architettura è cambiata mano a mano che la

velocità è aumentata da 10 Mbps a 10 Gbps e continua a salire. Ora, collegamenti punto a punto come i doppini sono collegati a hub e switch. Con gli switch moderni e collegamenti full-duplex non ci sono contese sui collegamenti e lo switch può inoltrare in parallelo i frame di porte diverse.

Con edifici pieni di LAN è necessario un modo per inerconnetterle tutte. I bridge plug-and-play sono utilizzati a questo proposito. I bridge sono costruiti con un algoritmo di backward learning e un algoritmo di spanning tree. Poiché questa funzionalità è integrata negli switch moderni, i termini “bridge” e “switch” sono diventati sinonimi. Per facilitare la gestione delle LAN che fanno uso di bridge, le VLAN permettono alla topologia fisica di essere suddivisa in diverse topologie logiche. Lo standard VLAN, IEEE 802.1Q, introduce un nuovo formato per i frame Ethernet.

Problemi

1. Per questo problema si utilizzi una formula di questo capitolo, che è necessario indicare. I frame arrivano in modo casuale a un canale a 100 Mbps per la trasmissione. Se un frame in arrivo trova il canale occupato, il frame attende il proprio turno in coda. La lunghezza del frame ha distribuzione esponenziale con una media di 10.000 bit per frame. Per ognuna delle seguenti frequenze di arrivo dei frame, indicare il ritardo medio incontrato dal frame includendo nel calcolo sia il tempo di accodamento sia quello di trasmissione.
   (a) 90 frame/s.
   (b) 900 frame/s.
   (c) 9000 frame/s.

2. Un gruppo di *N* stazioni condivide un canale ALOHA puro a 56 kbps. Ogni stazione trasmette un frame da 1000 bit in media una volta ogni 100 secondi, anche se il precedente non è stato ancora trasmesso (quindi le stazioni possono memorizzare in un buffer i frame in uscita). Qual è il valore massimo di *N*?

3. Si consideri il ritardo di ALOHA puro rispetto a slotted ALOHA nel caso di basso carico. Qual è il minore? Motivate la risposta.

4. Un gran numero di utenti ALOHA genera 50 richieste al secondo, comprendendo sia quelle originali sia le ritrasmissioni. Il tempo è diviso in unità di 40 ms.
   (a) Qual è la probabilità di un successo al primo tentativo?
   (b) Qual è la probabilità di avere esattamente *k* collisioni prima di un successo?
   (c) Qual è il numero atteso di tentativi richiesti?

5. In un sistema slotted ALOHA a popolazione infinita, il numero medio di intervalli che una stazione deve attendere tra una collisione e la ritrasmissione è 4. Disegnate la curva del ritardo in base alla capacità di trasporto di questo sistema.

6. Qual è la lunghezza di uno slot di contesa in CSMA/CD per (a) un doppino da 2 km (la velocità di propagazione del segnale è l'82% della velocità di propagazione del segnale nel vuoto) e (b) una cavo in fibra ottica multimodale da 40 km (la velocità di propagazione del segnale è il 65% della velocità di propagazione del segnale nel vuoto)?

7. Quanto tempo deve attendere una stazione, *s*, nel peggiore dei casi, prima di iniziare la trasmissione del suo frame attraverso una LAN che utilizza il protocollo a mappa di bit?

8. Nel protocollo con binary countdown si spieghi come una stazione con un numero basso possa essere ritardata indefinitamente nello spedire un pacchetto.

9. Sedici stazioni, numerate da 1 a 16, si contendono l'uso di un canale condiviso mediante il protocollo tree walk adattivo. Se tutte le stazioni che hanno per indirizzo un numero primo si attivano contemporaneamente, quanti intervalli di bit servono per risolvere la contesa?

10. Si considerino cinque stazioni wireless, *A*, *B*, *C*, *D* ed *E*. La stazione *A* può comunicare con tutte le altre stazioni. *B* può comunicare con *A*, *C* ed *E*. *C* può comunicare con *A*, *B* e *D*. *D* può comunicare con *A*, *C* ed *E*. *E* può comunicare con *A*, *D* e *B*.

(a) Quando *A* sta inviando a *B*, quali altre comunicazioni sono possibili?

(b) Quando *B* sta inviando a *A*, quali altre comunicazioni sono possibili?

(c) Quando *B* sta inviando a *C*, quali altre comunicazioni sono possibili?

11. Sei stazioni, da *A* a *F,* comunicano usando il protocollo MACA. È possibile che due trasmissioni avvengano contemporaneamente? Motivate la risposta.

12. Un edificio di sette piani ha 15 uffici adiacenti per ogni piano. Ogni ufficio contiene una presa a muro per un terminale, perciò le prese formano una griglia rettangolare sul piano verticale con una distanza di 4 m tra ogni coppia di prese, sia orizzontalmente sia verticalmente. Supponendo che sia possibile stendere un cavo dritto tra ogni coppia di prese, orizzontalmente, verticalmente o diagonalmente, quanti metri di cavo servono per collegare tutte le prese usando:

(a) una configurazione a stella con un singolo router nel mezzo

(b) una LAN 802.3 classica.

13. Qual è il baud rate di Ethernet classica a 10 Mbps?

14. Disegnate la codifica Manchester su una Ethernet classica relativa al flusso di bit 0001110101.

15. Una LAN CSMA/CD (non 802.3) da 10 Mbps e lunga 1 Km ha una velocità di propagazione pari a 200 m/ms. I repeater non sono ammessi in questo sistema. I frame di dati sono lunghi 256 bit, inclusi i 32 bit utilizzati per l'intestazione, il checksum e gli altri codici di controllo. Il primo intervallo di bit dopo una trasmissione che ha avuto successo è riservato al ricevente, che può così catturare il canale e inviare un frame di acknowledgement lungo 32 bit. Qual è il tasso di invio dei dati, escludendo il codice di controllo, supponendo che non ci siano collisioni?

16. Due stazioni CSMA/CD stanno tentando di trasmettere lunghi file (multiframe). Dopo aver trasmesso ogni frame, le stazioni si contendono il canale usando l'algoritmo di backoff esponenziale binario. Qual è la probabilità che la contesa termini in *k* round? Qual è il numero medio di round per periodo di contesa?

17. Un pacchetto IP che sta per essere trasmesso da Ethernet è lungo 60 byte, incluse tutte le sue intestazioni. Se non si usa LLC è necessario inserire del padding? Se sì, quanti byte devono essere aggiunti?

18. I frame di Ethernet devono essere lunghi almeno 64 byte per essere certi che il trasmittente sia ancora attivo in caso di collisione all'altro capo del cavo. Fast Ethernet ha la stessa dimensione minima di 64 byte, ma può trasmettere i bit dieci volte più velocemente. Com'è possibile mantenere la stessa dimensione minima del frame?

19. Alcuni testi riportano la dimensione massima di un frame Ethernet come 1522 byte invece di 1500 byte. È un errore? Spiegate la risposta.

20. Quanti frame al secondo può gestire una gigabit Ethernet? Fate molta attenzione e considerate tutti i casi rilevanti. Suggerimento: è importante il fatto che si tratti di gigabit Ethernet.

21. Citate il nome di due reti che consentono di impacchettare i frame uno in fila all'altro. Perché avere questa funzionalità è importante?

22. Nella Figura 4.27 sono mostrate quattro stazioni, *A*, *B*, *C* e *D*. Quale delle ultime due stazioni ritenete più vicina ad *A* e perché?

23. Si faccia un esempio per spiegare che RTS/CTS nel protocollo 802.11 è un po' diverso dal protocollo MACA.

24. Una LAN wireless con un AP ha 10 stazioni client. Quattro stazioni hanno un tasso di trasmissione di 6 Mbps, quattro stazioni hanno un tasso di trasmissione di 18 Mbps e le ultime due stazioni hanno un tasso di trasmissione di 54 Mbps. Qual è la capacità trasmissiva sperimentata da ogni stazione quando tutte le dieci stazioni spediscono dati contemporaneamente e:
    (a) TXOP non è utilizzato?
    (b) TXOP è utilizzato?

25. Si supponga che una LAN 802.1lb a 11 Mbps stia trasmettendo frame da 64 byte uno dopo l'altro attraverso un canale radio con una frequenza di errore pari a 10^{-7}. Quanti frame al secondo verranno danneggiati in media?

26. Una rete 802.16 ha un'ampiezza di canale di 20 MHz. Quanti bit/s possono essere inviati a una stazione utente?

27. Indicate due motivi per cui le reti potrebbero utilizzare un codice di correzione degli errori invece di rilevare gli errori ed effettuare ritrasmissioni.

28. Si elenchino due aspetti in cui WiMAX è simile a 802.11 e due aspetti in cui è diverso da 802.11.

29. La Figura 4.34 mostra che un dispositivo Bluetooth può trovarsi contemporaneamente in due piconet. C'è un motivo che impedisce a un dispositivo di essere contemporaneamente il nodo master di entrambe le piconet?

30. Qual è la dimensione massima del campo dati di un frame Bluetooth che occupa 3 slot al tasso di base ? Spiegate la risposta.

31. La Figura 4.24 mostra diversi protocolli di livello fisico. Quale di questi è il più vicino al protocollo di livello fisico Bluetooth? Qual è la principale differenza tra i due?

32. È stato detto nel Paragrafo 4.6.6 che l'efficienza di un frame da 1 slot con codifica a ripetizione è di circa il 13% al tasso di base. Quale sarebbe l'efficenza se invece venisse utilizzato un frame da 5 slot con codifica a ripetizione al tasso di base?

33. Si supponga ci siano 10 tag RFID attorno a un lettore RFID. Qual è il valore migliore di Q? Qual è la probabilità che un tag risponda senza collisioni in un dato slot?

34. Elencate alcune problematiche di sicurezza di un sistema RFID.

35. Uno switch progettato per funzionare con fast Ethernet ha un backplane che può funzionare a 10 Gbps. Quanti frame al secondo può gestire nel peggiore dei casi?

36. Descrivete brevemente la differenza tra gli switch store-and-forward e cut-through.

37. Si consideri la LAN estesa connessa usando i bridge *B*1 e *B*2 nella Figura 4.41(b). Supponendo che le tabelle di hash nei due bridge siano vuote, si elenchino tutte le porte su cui un pacchetto sarà inoltrato per la sequenza di trasmissioni seguente.

(a) *A* spedisce un pacchetto a *C*.
(b) *E* spedisce un pacchetto a *F*.
(c) *F* spedisce un pacchetto a *E*.
(d) *G* spedisce un pacchetto a *E*.
(e) *D* spedisce un pacchetto a *A*.
(f) *B* spedisce un pacchetto a *F*.

38. Gli switch store-and-forward hanno un vantaggio rispetto agli switch cut-through relativamente ai frame danneggiati, qual è?

39. Nel Paragrafo 4.8.3 è indicato che alcuni bridge potrebbero non essere presenti nello spanning tree. Descrivete uno scenario dove questo può accadere.

40. Per far funzionare le VLAN servono tabelle di configurazione nei bridge. Che cosa accade se le VLAN mostrate nella Figura 4.47 utilizzano hub al posto di switch? Anche gli hub hanno bisogno di tabelle di configurazione? Perché?

41. Nella Figura 4.48 lo switch nel vecchio dominio sulla destra è uno switch VLAN-aware. Sarebbe possibile utilizzare uno switch classico in quel punto? Se sì, come funzionerebbe il sistema? Se no, perché?

42. Scrivete un programma che simuli il comportamento del protocollo CSMA/CD su Ethernet quando ci sono *N* stazioni pronte a trasmettere mentre un frame è già in trasmissione. Il programma dovrebbe indicare il momento in cui ogni stazione inizia a trasmettere con successo il proprio frame. Si supponga che il clock avanzi a ogni intervallo di tempo (51,2 ms) e che il rilevamento di una collisione e la trasmissione di una sequenza di disturbo richiedano un intervallo di tempo. Tutti i frame hanno una lunghezza pari alla lunghezza massima consentita.

CAPITOLO 5
Il livello di rete

Il livello di rete si occupa del trasporto dei pacchetti lungo tutto il percorso dall'origine alla destinazione finale, che per essere raggiunta può richiedere molti salti attraverso router intermedi lungo il percorso. Questa funzione è chiaramente distinta da quella del livello data link, che ha il più modesto obiettivo di spostare semplicemente i frame da un capo all'altro di un cavo. Di conseguenza il livello di rete è il livello più basso a occuparsi della trasmissione punto a punto.

Per raggiungere i propri obiettivi, il livello di rete deve conoscere la topologia della rete di comunicazione (ossia l'insieme dei router e dei collegamenti) e scegliere i percorsi appropriati attraverso di essa, anche nel caso sia estesa. Nel compiere questa scelta deve anche evitare di

sovraccaricare alcune linee di comunicazione lasciando altre completamente libere. Deve infine occuparsi dei nuovi problemi che nascono quando la sorgente e la destinazione si trovano in reti diverse. Il presente capitolo analizza tutti questi argomenti, usando come riferimento Internet e il suo protocollo a livello di rete: IP.

5.1 Problematiche nella progettazione del livello di rete

I prossimi paragrafi introducono alcuni problemi che vanno affrontati dai progettisti del livello di rete, tra cui il servizio fornito al livello di trasporto e la struttura interna della rete.

Commutazione di pacchetto store-and-forward

Prima di introdurre i dettagli del livello di rete è bene ricordare il contesto, rappresentato nella Figura 5.1, in cui operano i protocolli di questo livello. I componenti principali del sistema sono le apparecchiature dell'ISP (router collegati tra loro da linee di trasmissione) mostrate dentro l'ovale ombreggiato e i dispositivi degli utenti (disegnati all'esterno dell'ovale). L'host *H*1, ad esempio un computer domestico connesso con un modem DSL, è collegato direttamente a uno dei router dell'ISP *A*. Al contrario, *H*2 si trova su una LAN, ad esempio una rete Ethernet aziendale che ha un router *F* posseduto e gestito dal cliente; a questo router fa capo una linea affittata diretta alle apparecchiature dell'ISP. *F* è disegnato all'esterno dell'ovale perché non appartiene all'ISP, ma per gli scopi di questo capitolo i router nei locali dei clienti sono considerati come parte della rete, perché utilizzano gli stessi algoritmi dei router dell'ISP (e il tema principale di questo capitolo riguarda proprio gli algoritmi).

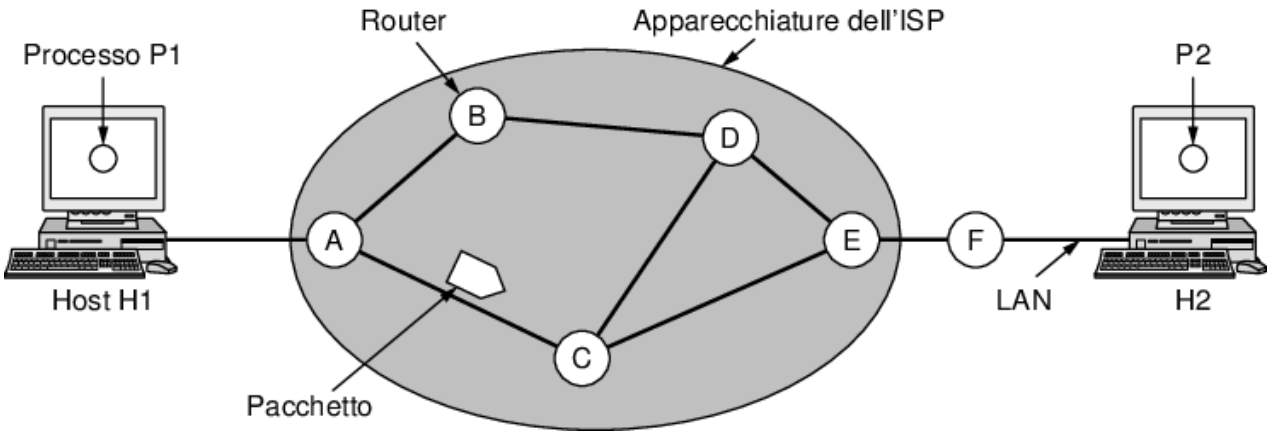


Figura 5.1 Ambiente dei protocolli del livello di rete.

Le apparecchiature sono utilizzate nel modo seguente: un host con un pacchetto da trasmettere invia i dati al router più vicino attraverso la sua stessa LAN, oppure attraverso un collegamento punto a punto con l'operatore di telecomunicazioni. Qui il pacchetto viene memorizzato finché non è interamente arrivato per verificare il checksum; quindi viene inoltrato al router successivo

che si trova lungo il percorso, finché non raggiunge l'host di destinazione dove viene consegnato. Tale meccanismo, com'è stato spiegato nei capitoli precedenti, è detto commutazione di pacchetto store-and-forward.

Servizi forniti al livello di trasporto

Il livello di rete fornisce servizi al livello di trasporto attraverso l'interfaccia livello di rete/livello di trasporto. È importante capire quali servizi fornisca il livello di rete al livello di trasporto. I servizi del livello di rete devono essere progettati attentamente tenendo in mente i seguenti obiettivi:

1. i servizi non dovrebbero essere legati alla tecnologia dei router
2. al livello di trasporto dovrebbero essere nascosti dettagli quali numero, tipo e topologia dei router
3. gli indirizzi di rete disponibili al livello di trasporto dovrebbero utilizzare uno schema di numerazione uniforme, anche attraverso le LAN e le WAN.

Dati questi obiettivi, i progettisti del livello di rete hanno piena libertà di definire le specifiche dettagliate dei servizi da offrire al livello di trasporto. Tale libertà degenera in una furiosa battaglia tra due fazioni in lotta; il nocciolo della questione è se il livello di rete dovrebbe fornire un servizio orientato alla connessione oppure no.

Una fazione (rappresentata dalla comunità Internet) afferma che il lavoro dei router è spostare i pacchetti da un punto all'altro e niente di più. Nella loro visione (basata su 40 anni di esperienza con una rete di calcolatori reale e operativa) la rete è intrinsecamente inaffidabile, indipendentemente da come è stata progettata, quindi gli host dovrebbero accettare questo assunto ed eseguire da soli il controllo (rilevamento e correzione) degli errori e il controllo di flusso.

Questo punto di vista conduce rapidamente alla conclusione che il servizio di rete non dovrebbe essere orientato alle connessioni, con primitive come SEND PACKET e RECEIVE PACKET e poco più. In particolare, non si dovrebbe eseguire alcuna operazione di ordinamento dei pacchetti e di controllo di flusso: ci sono già gli host che si occupano di questo aspetto e di solito fare due volte la stessa operazione non aiuta un granché. Questo ragionamento è un esempio di **visione punto a punto** (*end-to-end vision*), un principio di progettazione fondamentale nel dare forma a Internet (Saltzer et al., 1984). Inoltre ogni pacchetto deve contenere l'intero indirizzo di destinazione, perché ciascun pacchetto trasmesso viene trasportato in modo indipendente dai suoi eventuali predecessori.

L'altra fazione (rappresentata dalle compagnie telefoniche) sostiene che la rete dovrebbe fornire un servizio affidabile orientato alla connessione e che l'esperienza garantita da 100 anni di successi del sistema telefonico mondiale offra una guida eccellente. In questa visione la qualità del servizio è il fattore dominante, ma se nella rete non sono presenti connessioni è molto difficile da ottenere, specialmente per il traffico in tempo reale (per esempio audio e video).

Anche dopo alcuni decenni questa controversia è sempre attuale: le prime reti per trasporto dati largamente utilizzate, come X.25 negli anni '70[1] e successivamente Frame Relay negli anni '80 erano orientate alla connessione. Tuttavia dai giorni di Arpanet e della prima Internet, la diffusione di livelli di rete non orientata alla connessione è cresciuta enormemente. Il protocollo IP è il

simbolo onnipresente di questo successo. Tale protocollo non venne nemmeno scoraggiato da una tecnologia orientata alla connessione, ATM, sviluppata negli anni '80 proprio per demolirlo; anzi, ora è ATM a essere sempre meno usata, mentre IP si sta diffondendo anche nelle reti telefoniche. Per effetto della crescente necessità di garantire la qualità del servizio, Internet si sta comunque evolvendo per acquisire proprietà tipicamente associate a un servizio orientato alla connessione. Due esempi di tecnologie molto usate e orientate alla connessione sono **MPLS** (*multiprotocol label switching*), descritto in questo capitolo, e le reti VLAN viste nel Capitolo 4.

Implementazione del servizio senza connessione

Dopo aver indicato le due classi di servizio che il livello di rete può fornire ai propri utenti è giunto il momento di vedere in che modo funziona questo livello al suo interno. Sono possibili due diverse organizzazioni, che dipendono dal tipo di servizio offerto. Se il servizio è senza connessione, i pacchetti sono inoltrati nella rete individualmente e instradati indipendentemente l'uno dall'altro; non occorre configurazione anticipata. In questo contesto i pacchetti sono chiamati spesso **datagrammi o datagram** (è evidente l'analogia con i telegrammi) e la rete è chiamata **rete datagram**. Se il servizio è orientato alla connessione, prima di inviare i pacchetti si deve stabilire un percorso che colleghi il router sorgente al router destinazione. Questa connessione è chiamata **VC** (*virtual circuit*, circuito virtuale), in analogia con i circuiti fisici configurati dal sistema telefonico e la rete è chiamata **rete a circuito virtuale**. Questo paragrafo esamina le reti datagram, mentre il successivo si occuperà di quelle a circuito virtuale.

Vediamo come funziona una rete datagram. Si supponga che il processo *P*1 della Figura 5.2 abbia un lungo messaggio per *P*2. Il processo sorgente porge il messaggio al livello di trasporto dando istruzioni di inoltrarlo al processo *P*2 che si trova sull'host *H*2. Il codice del livello di trasporto in esecuzione in *H*1, tipicamente dentro il sistema operativo, aggiunge un'intestazione di trasporto all'inizio del messaggio e passa il risultato al livello di rete, rappresentato probabilmente da un'altra procedura del sistema operativo.

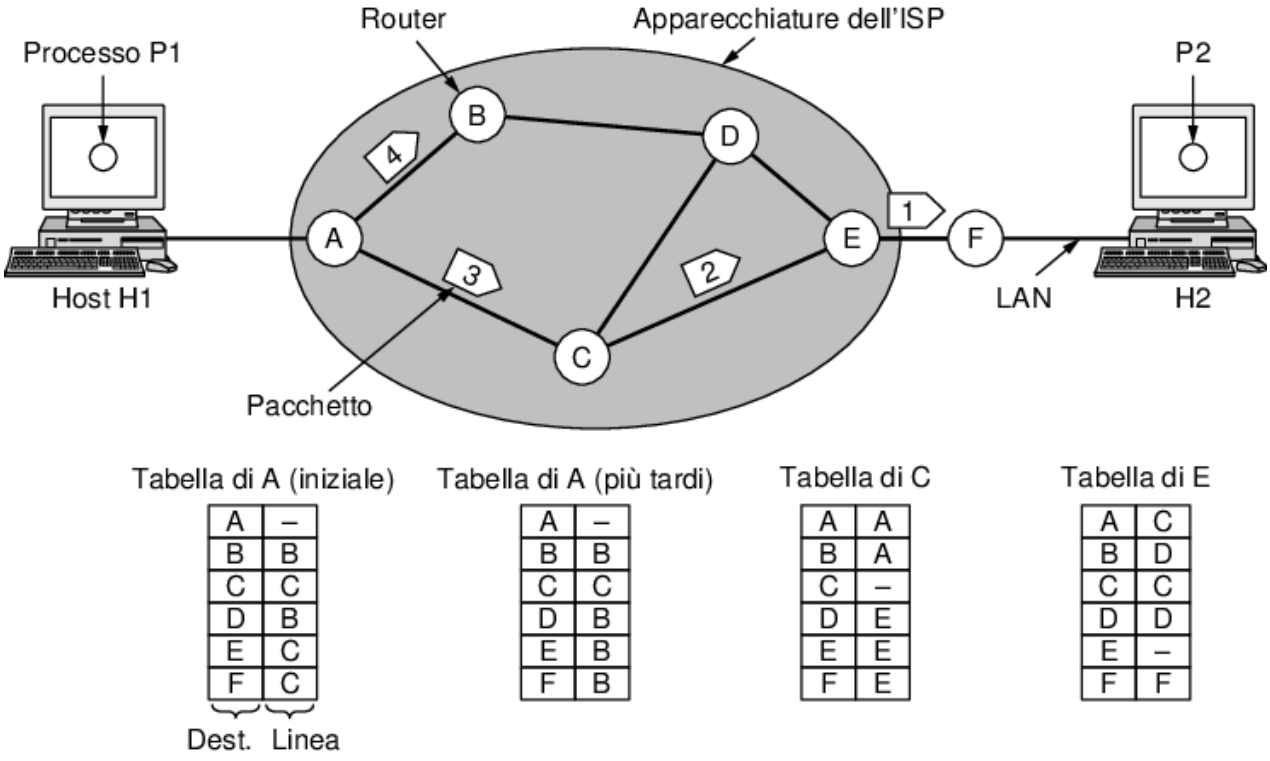


Figura 5.2 Routing in una rete datagram.

Si supponga che il messaggio sia lungo quattro volte la dimensione massima dei pacchetti; in questo caso il livello di rete deve suddividere il messaggio in quattro pacchetti, 1, 2, 3 e 4, e inviare un pacchetto dopo l'altro al router *A* usando un protocollo punto a punto, per esempio PPP. A questo punto l'ISP assume il controllo dell'operazione. Ogni router ha una tabella interna che indica dove devono essere inviati i pacchetti diretti a ogni possibile destinazione. Ogni voce della tabella è costituita da una coppia di valori, che rappresentano una destinazione e la linea di trasmissione da utilizzare per raggiungere tale destinazione. Possono essere utilizzate solo linee collegate direttamente. Per esempio nella Figura 5.2, *A* ha solo due linee in uscita, una porta a *B* e l'altra a *C,* perciò ogni pacchetto in arrivo deve essere inviato a uno di questi due router, anche se la destinazione finale si trova su qualche altro router.

Non appena raggiungono *A*, i pacchetti 1, 2 e 3 sono archiviati temporaneamente (per verificarne il checksum) e poi ogni pacchetto viene inoltrato verso *C* secondo la tabella di *A* all'interno di un nuovo frame. Il pacchetto 1 viene quindi trasmesso a *E* e poi a *F.* Quando arriva a *F,* il pacchetto è incapsulato in un frame e inviato ad *H*2 attraverso la LAN. I pacchetti 2 e 3 seguono lo stesso percorso.

Al pacchetto 4, invece, accade qualcosa di diverso. Quando raggiunge *A* viene inviato al router *B,* anche se la destinazione finale è ancora *F.* Per qualche motivo, *A* ha deciso di trasmettere il pacchetto 4 attraverso un percorso diverso da quello seguito dai primi 3 pacchetti. Forse, dopo aver rilevato un ingorgo di traffico da qualche parte lungo il percorso *A*-*C*-*E,* ha aggiornato la sua tabella di routing, come indicato dai valori riportati sotto l'intestazione "Più tardi". L'algoritmo che

gestisce le tabelle e prende le decisioni di instradamento è chiamato **algoritmo di routing**. Gli algoritmi di routing sono uno degli argomenti principali trattati in questo capitolo e possono essere di molti tipi diversi.

Il protocollo **IP** (*internet protocol,* protocollo internet), base di tutta Internet, è l'esempio più diffuso di servizio di rete non orientato alla connessione. Ogni pacchetto possiede l'indirizzo di rete della destinazione che i router utilizzano per inoltrarlo individualmente dagli altri. Gli indirizzi sono di 32 bit nei pacchetti IPv4 e di 128 bit nei pacchetti IPv6. Il protocollo IP verrà ampiamente descritto in questo capitolo.

Implementazione del servizio orientato alla connessione

Nel caso di un servizio orientato alla connessione serve una rete a circuito virtuale; vediamo come funziona. L'idea alla base dei circuiti virtuali è quella di evitare di dover scegliere una nuova strada per ogni pacchetto inviato, come nella Figura 5.2. Invece, quando è stabilita una connessione, il percorso dal computer sorgente a quello di destinazione viene scelto durante l'impostazione della connessione e archiviato nelle tabelle dei router. Quel percorso è utilizzato per tutto il traffico che scorre attraverso la connessione, con funzionamento identico a quello del sistema telefonico. Quando la connessione viene rilasciata, anche il circuito virtuale viene terminato. Con un servizio orientato alla connessione, ogni pacchetto contiene un identificatore che indica il circuito virtuale di appartenenza.

Come esempio si consideri la situazione mostrata nella Figura 5.3. In questo caso l'host *H*1 ha stabilito la connessione 1 con l'host *H*2. La connessione è memorizzata nella prima voce delle tabelle di routing. La prima linea della tabella di *A* indica che, se da *H*1 arriva un pacchetto contenente l'identificatore di connessione numero 1, allora i dati devono essere inviati al router *C* assegnando ancora l'identificatore 1. In modo analogo la prima voce della tabella di *C* instrada il pacchetto a *E,* ancora con l'identificatore di connessione numero 1. Che cosa accade se anche *H*3 cerca di stabilire una connessione con *H*2? L'host sceglie l'identificatore di connessione numero 1 (perché sta iniziando la connessione e questa è la sua unica connessione) e richiede alla rete di stabilire un circuito virtuale. Ciò conduce alla seconda riga della tabella: in questo caso si verifica un conflitto, perché sebbene *A* possa facilmente distinguere i pacchetti della connessione 1 provenienti da *H*1 dai pacchetti della connessione 1 provenienti da *H*3*, C* non è in grado di farlo. Per questo motivo *A* assegna un diverso identificatore di connessione al traffico in uscita relativo alla seconda connessione.

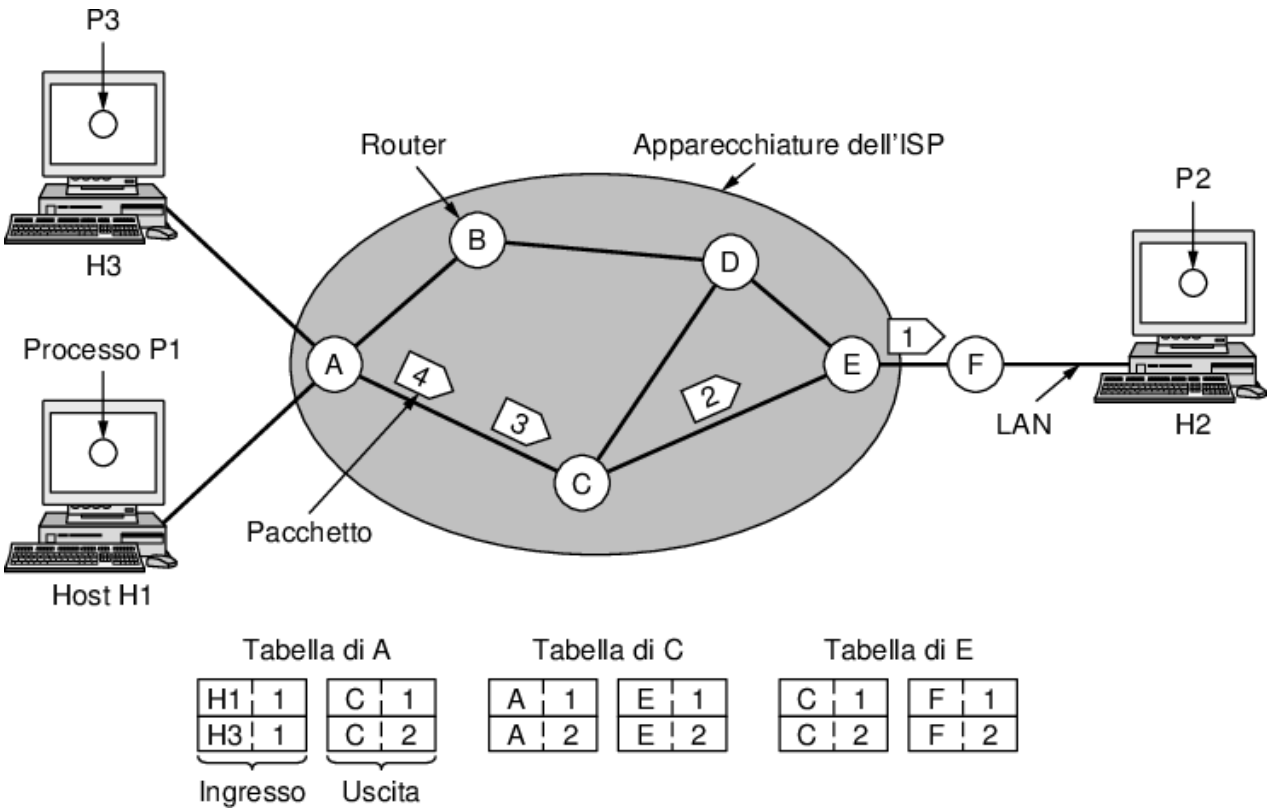


Figura 5.3 Routing in una rete a circuito virtuale.

Ai router è stata data la capacità di sostituire gli identificatori di connessione nei pacchetti in uscita proprio per evitare conflitti di questo tipo.

In alcuni contesti questa capacità è chiamata **commutazione di etichetta** (*label switching*). Un esempio di servizio orientato alla connessione è **MPLS** (*multiprotocol label switching*), usato in Internet all'interno delle reti degli ISP; con MPLS i pacchetti IP sono incapsulati in un'intestazione MPLS avente un identificatore di connessione a 20 bit. Il protocollo MPLS è spesso invisibile ai clienti, mentre l'ISP stabilisce connessioni di lunga durata per trasferire grandi volumi di traffico, ma viene sempre più usato sia quando è importante garantire la qualità del servizio sia per motivi di gestione del traffico. Torneremo sull'argomento più avanti in questo capitolo.

Confronto tra reti a circuito virtuale e reti datagram

Circuiti virtuali e datagrammi hanno i loro sostenitori e detrattori; tenteremo ora di riepilogare le motivazioni di entrambi. I concetti principali sono elencati nella Figura 5.4, anche se i puristi, probabilmente, potrebbero trovare un contro esempio per ciascun elemento indicato.

Problema	Rete datagram	Rete a circuito virtuale

Problema	Rete datagram	Rete a circuito virtuale
Impostazione del circuito	Non necessaria	Necessaria
Indirizzamento	Ogni pacchetto contiene gli indirizzi completi di sorgente e destinazione	Ogni pacchetto contiene un breve identificatore di circuito virtuale
Informazioni di stato	I router non mantengono informazioni di stato sulle connessioni	Ogni circuito virtuale richiede spazio nella tabella del router per la connessione
Routing	Ogni pacchetto è inoltrato in maniera indipendente	Il percorso viene scelto durante l'impostazione del circuito e tutti i pacchetti lo seguono
Effetto del malfunzionamento di un router	Nessuno, tranne i pacchetti persi perché in transito durante il malfunzionamento	Tutti i circuiti virtuali che passano dal router vengono chiusi
Qualità di servizio	Difficile	Facile se abbastanza risorse possono essere allocate in anticipo a ogni circuito virtuale
Controllo della congestione	Difficile	Facile se abbastanza risorse possono essere allocate in anticipo a ogni circuito virtuale

Figura 5.4 Confronto tra reti datagram e a circuito virtuale.

All'interno della rete esistono diversi compromessi tra circuiti virtuali e datagrammi. Un compromesso è tra il tempo per la configurazione del circuito e il tempo di analisi dell'indirizzo. L'utilizzo di circuiti virtuali richiede una fase di configurazione che esige tempo e consuma risorse, dopo però risulta facile capire che cosa si debba fare con un pacchetto dati in una rete a circuito virtuale: il router usa semplicemente il numero di circuito come indice nella tabella per trovare la destinazione dei dati. In una rete datagram non è necessaria alcuna procedura di configurazione ma, per individuare la voce relativa alla destinazione, serve una procedura di ricerca più complicata.

Un problema correlato è che gli indirizzi di destinazione nelle reti datagram sono più lunghi dei numeri di circuito usati nelle reti a circuito virtuale, in quanto questi hanno un significato globale. Se i pacchetti tendono a essere corti, un indirizzo di destinazione completo in ogni pacchetto può aumentare in modo significativo l'overhead e di conseguenza sprecare banda.

Un altro problema è la quantità di spazio richiesto dalla tabelle nella memoria del router. Una rete datagram ha bisogno di una voce per ogni possibile destinazione, mentre una rete a circuito virtuale si accontenta di una voce per ogni circuito virtuale. In realtà questo vantaggio è un po' illusorio, poiché anche i pacchetti utilizzati per impostare la connessione devono essere instradati usando l'indirizzo di destinazione, proprio come avviene con i datagrammi.

I circuiti virtuali sono in vantaggio quando si tratta di dare garanzie sulla qualità del servizio ed evitare le congestioni all'interno della rete, perché le risorse (per esempio i buffer, la banda e i cicli di CPU) possono essere riservate in anticipo, al momento di stabilire la connessione. Una volta che i pacchetti iniziano ad arrivare, la banda necessaria e la capacità del router sono già disponibili. Nella rete a datagrammi è più difficile evitare la congestione del traffico.

Per i sistemi che elaborano transazioni (come quelli utilizzati dai negozi per verificare gli acquisti con carta di credito), l'overhead richiesto per impostare e rilasciare un circuito virtuale può facilmente superare quello di utilizzo vero e proprio del circuito. Se si pensa che la maggior parte del traffico sarà di questo tipo, ha poco senso utilizzare i circuiti virtuali all'interno della rete. D'altra parte per utilizzi a lungo termine, come il traffico VPN tra due uffici di una multinazionale, possono dimostrarsi molto utili dei circuiti virtuali permanenti, impostati manualmente che durano mesi o anni.

I circuiti virtuali hanno anche un problema di vulnerabilità. Se un router ha un problema hardware o software e perde tutti i dati nella sua memoria, tutti i circuiti virtuali che passano attraverso tale apparecchiatura devono essere terminati, anche se questo si riattiva un solo secondo dopo. Al contrario se un router per datagrammi si blocca soffrono solo gli utenti che in quel momento avevano pacchetti accodati nel router e forse nemmeno tutti, perché verosimilmente la sorgente li ritrasmetterà a breve. La perdita della linea di comunicazione è fatale per i circuiti virtuali che la utilizzano, ma può essere facilmente compensata quando si usano datagrammi, che inoltre permettono ai router di bilanciare il traffico attraverso la rete, poiché i percorsi possono essere cambiati in corsa durante una lunga sequenza di trasmissione.

5.2 Algoritmi di routing

La funzione principale del livello di rete è quella di inoltrare i pacchetti dal computer sorgente al computer di destinazione. Nella maggior parte delle reti per raggiungere la meta i pacchetti compiono una sequenza di salti (*hop*); l'unica eccezione degna di nota riguarda le reti broadcast, ma anche in questo caso, se sorgente e destinazione non si trovano sulla stessa rete, il routing è un problema. Gli algoritmi che scelgono i percorsi e le strutture dati che usano sono una parte fondamentale dell'architettura del livello di rete.

L'**algoritmo di routing** (o *algoritmo di instradamento*) è quella parte del software del livello di rete che si preoccupa di scegliere lungo quale linea di uscita vanno inoltrati i pacchetti in arrivo. Se la rete fa uso di datagrammi, questa decisione va ripetuta per ogni pacchetto di dati in arrivo, poiché il percorso migliore può cambiare nel tempo. Se la rete utilizza i circuiti virtuali, le decisioni che riguardano il routing vengono prese solo quando viene impostato un nuovo circuito virtuale. Da quel momento in poi i pacchetti con i dati seguono semplicemente la rotta prestabilita. Il secondo caso è qualche volta chiamato **routing di sessione**, perché un percorso rimane valido per tutta la sessione utente (per esempio durante l'accesso a un terminale in una VPN).

Qualche volta è utile distinguere tra routing (che sceglie la strada da seguire) e forwarding, (che indica l'operazione eseguita all'arrivo di un pacchetto). È come se il router contenesse due processi al suo interno. Uno dei due gestisce ogni pacchetto in arrivo, cercando nella tabella di routing la linea di trasmissione più adatta; questo processo è chiamato **forwarding** (o *inoltro*). L'altro processo si occupa di riempire e aggiornare le tabelle di forwarding ed è qui che entrano in gioco gli algoritmi di routing.

Alcune proprietà dell'algoritmo di routing sono auspicabili indipendentemente dal fatto che i percorsi siano scelti per ogni pacchetto o stabiliti durante l'impostazione di una nuova connessione: correttezza, semplicità, robustezza, stabilità, imparzialità e ottimizzazione. Correttezza e semplicità non richiedono commenti, ma la necessità di robustezza può essere meno ovvia a prima vista. Ci si aspetta che ogni nuova grande rete continui a funzionare per anni, senza che si verifichino guasti capaci di coinvolgere l'intero sistema. Durante questo periodo si verificheranno guasti hardware e software di tutti i tipi; host, router e linee si guasteranno ripetutamente e la topologia subirà molte modifiche. L'algoritmo di routing dovrebbe essere in grado di far fronte ai cambiamenti di topologia e di traffico, senza che sia necessario interrompere tutto il lavoro in tutti gli host. Immaginate lo scompiglio se la rete dovesse essere riavviata ogni volta che un router si blocca.

Anche la stabilità costituisce un obiettivo importante per l'algoritmo di routing. Esistono algoritmi di routing che non convergono mai all'equilibrio, anche se eseguiti per molto tempo. Un algoritmo stabile raggiunge l'equilibrio e lì rimane. Dovrebbe anche convergere rapidamente, prima che cessi la comunicazione.

Imparzialità ed efficienza possono sembrare qualità ovvie (sicuramente nessuno cercherebbe ragionevolmente di opporsi a queste caratteristiche), ma spesso sono obiettivi contraddittori. Come semplice esempio di questo conflitto si osservi la Figura 5.5. Si supponga che ci sia abbastanza

traffico tra A e A', tra B e B' e tra C e C' da saturare i collegamenti orizzontali. Per massimizzare il flusso totale, il traffico tra X e X' dovrebbe essere interrotto del tutto. Sfortunatamente, X e X' potrebbero non vederla nello stesso modo. Evidentemente sono necessari alcuni compromessi tra efficienza globale e imparzialità verso le singole connessioni.

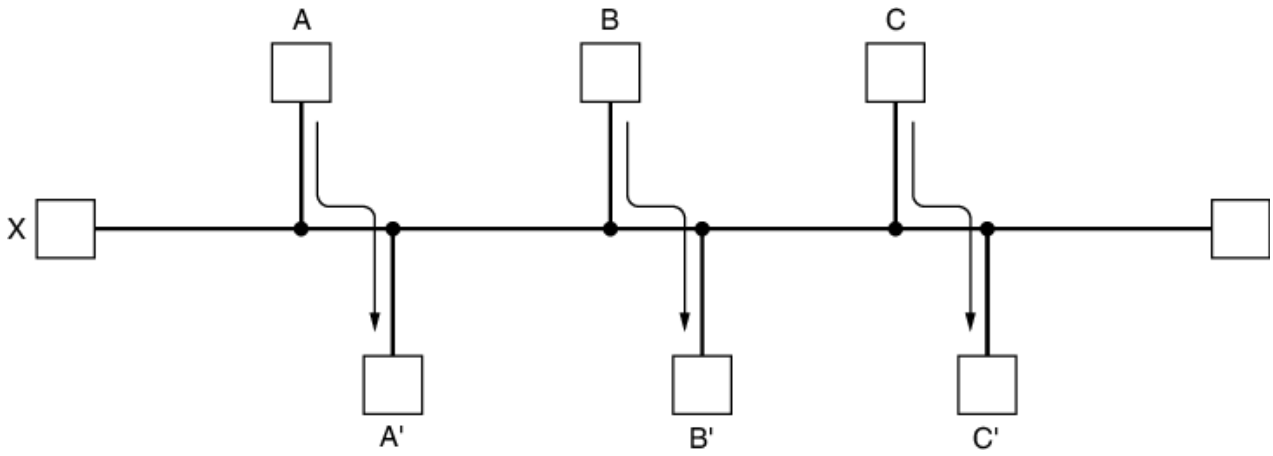


Figura 5.5 Rete con un conflitto tra imparzialità ed efficienza.

Ancor prima di poter trovare compromessi tra imparzialità ed efficienza è necessario scegliere che cosa ottimizzare. Un'ovvia candidata è la riduzione al minimo possibile del ritardo medio dei pacchetti, così come la massimizzazione della capacità di carico totale della rete. Anche questi due obiettivi sono però in conflitto, poiché far lavorare un sistema ad accodamento vicino alla sua capacità massima crea un lungo tempo di attesa. Come compromesso, molte reti tentano di minimizzare la distanza da percorrere o il numero dei salti che il pacchetto deve compiere, perché in questo modo si migliora il tempo di attesa, si riduce la banda consumata e di conseguenza si migliora anche la capacità di carico.

Gli algoritmi di routing si possono suddividere in due gruppi principali: adattativi e non adattativi. Gli **algoritmi non adattativi** non basano le proprie decisioni su misure o stime del traffico e della topologia corrente. Il percorso utilizzato per collegare I a J (per tutti gli I e J) è invece calcolato in anticipo e scaricato nei router all'avvio della rete. Questa procedura si chiama anche **routing statico**. Poiché non è reattivo agli errori, il routing statico è utile quando le scelte di routing sono chiare. Per esempio il router F nella Figura 5.3 dovrebbe trasmettere i pacchetti al router E qualunque sia la loro destinazione finale.

Al contrario gli **algoritmi adattativi** cambiano le proprie decisioni secondo le modifiche apportate alla topologia e di solito anche al traffico. Questi algoritmi di **routing dinamico** si distinguono per la fonte da cui traggono le loro informazioni (ossia localmente, da router adiacenti o da tutti i router), per il momento in cui modificano i percorsi (per esempio ogni DT secondi,

quando il carico cambia o quando la topologia cambia) e per il tipo di metrica utilizzata nell'operazione di ottimizzazione (per esempio la distanza, il numero di salti o il tempo di transito stimato).

I paragrafi seguenti descrivono diversi algoritmi di routing statici e dinamici. Oltre che occuparsi dell'invio di pacchetti da una sorgente a una destinazione gli algoritmi trattano di modelli di consegna. A volte l'obiettivo è spedire pacchetti ad alcuni, tutti o un insieme di destinazioni. Tutti gli algoritmi che verranno presentati si basano sulla topologia della rete: quelli basati sul traffico verranno analizzati nel Paragrafo 5.3.

Il principio di ottimalità

Prima di esaminare algoritmi specifici può essere utile notare che è possibile fare un'ipotesi generale riguardo ai percorsi ottimali indipendentemente dalla topologia di rete o dal traffico. Tale assioma, conosciuto come **principio di ottimalità** (Bellman, 1957), afferma che se il router *J* si trova sul percorso ottimale che collega *I* a *K,* allora anche il percorso ottimale da *J* a *K* segue la stessa sequenza di router. Per dimostrarlo è sufficiente chiamare r_1 la parte del percorso che unisce *I* a *J* e r_2 la parte restante del tracciato. Se tra *J* e *K* esistesse un percorso migliore di r_2 esso potrebbe essere concatenato con r_1 per migliorare la strada da *I* a *K,* ma ciò negherebbe l'ipotesi iniziale che r_1 r_2 sia ottimale.

Come diretta conseguenza del principio di ottimalità si ricava che la serie di percorsi ottimali che collegano tutte le sorgenti a una data destinazione forma una struttura ad albero dove il nodo principale è costituito dalla destinazione. In questo albero, chiamato **sink tree** (Figura 5.6b), la distanza è calcolata in base al numero di salti. L'obiettivo di tutti gli algoritmi di routing è scoprire e utilizzare i sink tree per tutti i router.

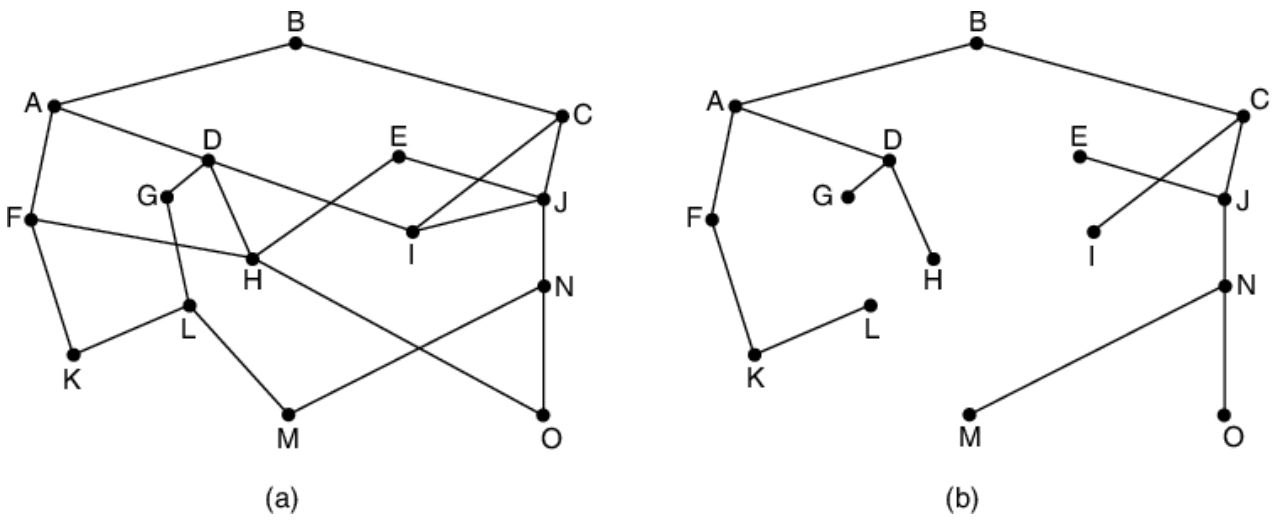


Figura 5.6 (a) Una rete. (b) Il sink tree per il router B.

Si noti che un sink tree non è necessariamente unico; possono esistere altri alberi con le stesse lunghezze di percorso. Se siamo autorizzati a scegliere tutti i possibili percorsi allora l'albero diventa una struttura più generale chiamata **DAG** (*directed acyclic graph*, grafo diretto senza cicli). In entrambi i casi useremo i sink tree basandoci sull'assunzione tecnica che i percorsi non interferiscano tra loro: per esempio, una congestione di traffico su di un percorso non ne fa deviare un altro.

Un sink tree è una struttura ad albero e pertanto non contiene alcun ciclo, quindi ogni pacchetto verrà inoltrato in un numero finito e limitato di salti. In realtà la vita non è così semplice. I collegamenti e i router possono disattivarsi e attivarsi durante il funzionamento della rete, perciò router diversi possono avere idee differenti sulla topologia corrente. Inoltre occorre ancora decidere come ciascun router deve acquisire le informazioni su cui basa il calcolo del sink tree: se devono essere recuperate individualmente dai singoli router oppure in qualche altro modo. Tra poco conosceremo la risposta, ma resta assodato che il principio di ottimalità e la struttura sink tree forniscono un modo per valutare le prestazioni degli algoritmi di routing.

Algoritmo di cammino minimo

Cominciamo l'analisi dei possibili algoritmi di routing con una semplice tecnica per calcolare i percorsi ottimali conoscendo la topologia di tutta la rete. Questi sono i percorsi che vogliamo che un algoritmo di routing distribuito trovi, sebbene non tutti i router abbiano una conoscenza completa della rete.

L'idea è quella di costruire un grafo della rete dove ogni nodo rappresenti un router e ogni arco rappresenti una linea di comunicazione (collegamento o link). Per scegliere un percorso tra una coppia di router, l'algoritmo trova semplicemente il cammino minimo che collega i due nodi del grafo.

Il concetto di **cammino minimo** (*shortest path*) merita una spiegazione: la lunghezza del percorso può essere misurata con il numero di salti; usando questa metrica i percorsi *ABC* e *ABE* nella Figura 5.7 sono uguali. Un'altra metrica è basata sulla distanza geografica espressa in chilometri; in questo caso il percorso *ABC* è chiaramente più lungo di *ABE* (supponendo che la figura sia disegnata in scala).

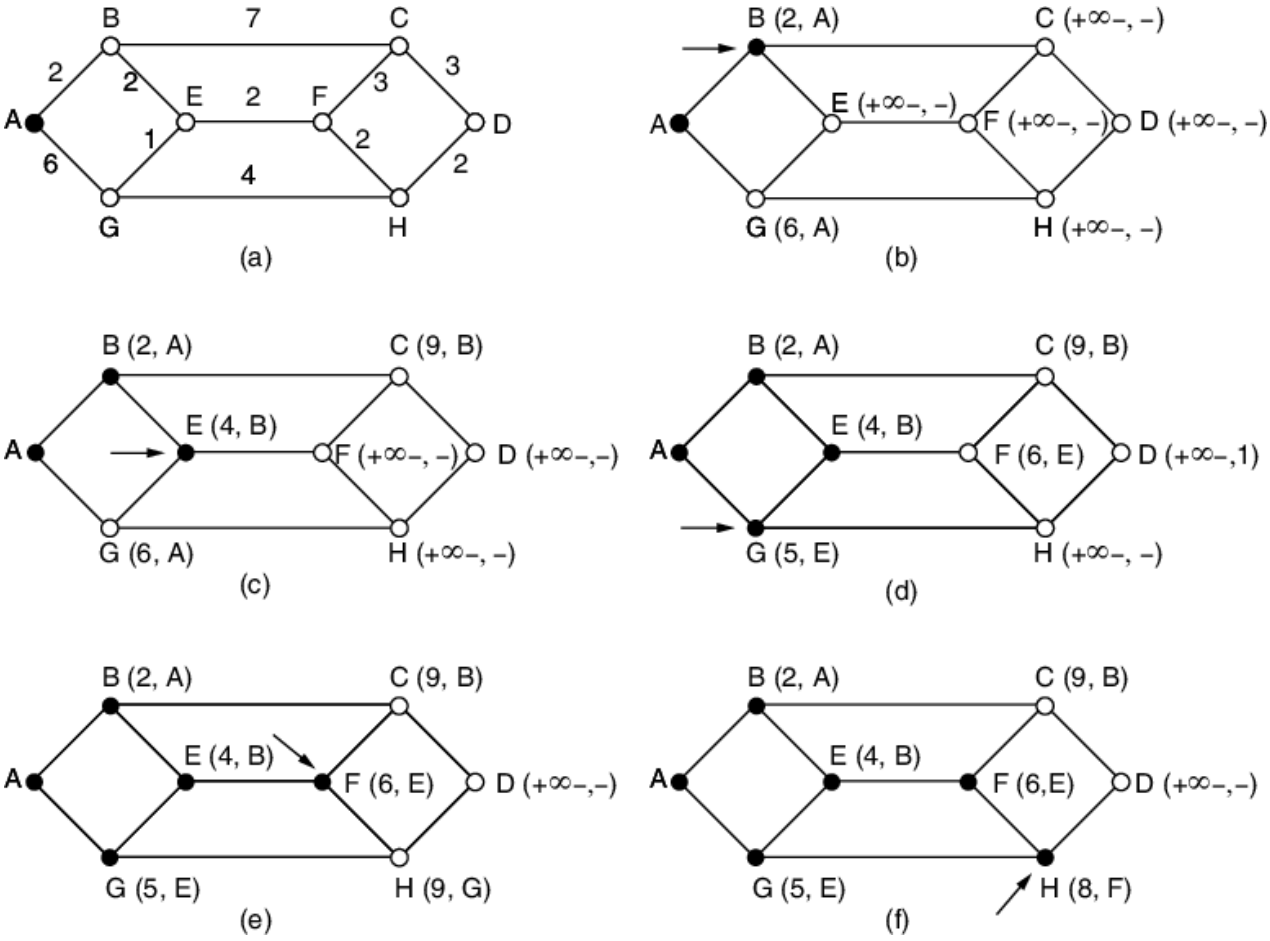


Figura 5.7 I primi sei passi usati per calcolare il percorso più breve da A a D. Le frecce indicano il working node.

Oltre alla distanza fisica e al numero di salti sono molte le metriche utilizzabili. Per esempio ogni arco potrebbe essere etichettato con i valori che rappresentano i valori medi di ritardo della trasmissione, calcolati mediante test eseguiti ogni ora su alcuni pacchetti di prova standard. Con questo tipo di classificazione il percorso più breve è rappresentato dal percorso più veloce e non da quello con il numero minore di archi o di chilometri.

Nel caso generale le etichette sugli archi possono essere calcolate come una funzione della distanza, della banda, del traffico medio, del costo della comunicazione, della lunghezza media della coda, del ritardo e di altri fattori. Modificando la funzione che attribuisce i pesi, l'algoritmo calcolerebbe il percorso "più breve" misurato secondo qualunque numero di criteri o loro combinazione.

Per elaborare il percorso più breve tra due nodi di un grafo ci sono diversi algoritmi. Quello descritto è stato ideato da Dijkstra (1959) e trova i cammini minimi tra una sorgente e tutte le destinazioni nella rete. A ogni nodo è associata un'etichetta (racchiusa tra parentesi) che riporta la sua distanza dal nodo di origine lungo il miglior percorso conosciuto. Le distanze devono essere non negative, come avviene quando ci si basa su grandezze fisiche quali la banda e il ritardo. Inizialmente nessun percorso è noto, perciò tutti i nodi sono etichettati con +∞–. Mano a mano che l'algoritmo procede e si trovano i percorsi, le etichette cambiano rispecchiando i percorsi

migliori. Un'etichetta può essere provvisoria o permanente. Inizialmente tutte le etichette sono provvisorie e quando si scopre l'etichetta che rappresenta il percorso più breve possibile dall'origine a quel nodo essa viene resa permanente e mai più modificata.

Per illustrare il funzionamento dell'algoritmo basato sulle etichette, si osservi il grafo pesato non orientato rappresentato nella Figura 5.7(a), dove i pesi rappresentano, per esempio, una distanza. Si desidera trovare il percorso più breve che collega *A* a *D.* Si comincia contrassegnando il nodo *A* come permanente; per farlo è sufficiente rappresentarlo con un cerchio pieno. Poi si esaminano, uno dopo l'altro, tutti i nodi adiacenti ad *A* (chiamato anche *working node*), etichettando nuovamente ogni nodo con la distanza da *A.* Ogni volta che si etichetta di nuovo un nodo, si prende nota anche del nome del nodo dal quale è partita l'analisi, in modo da poter successivamente ricostruire il percorso finale. Se esiste più di un cammino minimo tra *A* e *D* e vogliamo trovarli tutti, dobbiamo memorizzare tutti i nodi di partenza che possono raggiungere un dato nodo con la stessa distanza.

Dopo aver esaminato ogni nodo adiacente ad A si osservano tutti i nodi etichettati dell'intero grafo e si rende permanente quello con l'etichetta più piccola, come mostrato nella Figura 5.7(b). Questo diventa il nuovo working node.

Ora si comincia dal nuovo working node (*B*) e si esaminano tutti i nodi che gli sono adiacenti. Se l'etichetta su *B* sommata alla distanza da *B* al nodo considerato è minore dell'etichetta di quel nodo, allora è stato trovato un percorso più breve e il nodo è nuovamente etichettato.

Dopo aver esaminato tutti i nodi adiacenti al working node e aver modificato dove possibile le etichette provvisorie, si cerca nell'intero grafo il nodo etichettato provvisoriamente contenente il valore più piccolo. Questo nodo è reso permanente e diventa il working node del passaggio successivo. La Figura 5.7 mostra i primi sei passi dell'algoritmo.

Per capire perché l'algoritmo funziona, si osservi la Figura 5.7(c). In questo istante il nodo *E* è appena stato reso permanente. Si supponga che ci sia un percorso più breve di *ABE,* per esempio *AXYZE.* Ci sono due possibilità: o il nodo *Z* è già stato reso permanente oppure non lo è stato. Nel primo caso, *E* è stato già esaminato (nel passaggio successivo a quello in cui *Z* è stato reso permanente), perciò il percorso *AXYZE* non è sfuggito all'attenzione e non può essere un percorso più breve.

Ora si consideri il caso in cui *Z* ha ancora un'etichetta temporanea. O l'etichetta di *Z* è maggiore o uguale a quella di *E,* nel qual caso *AXYZE* non può essere un percorso più corto di *ABE,* oppure è minore di quella di *E,* nel qual caso *Z* e non *E* diventerà permanente per primo, permettendo a *E* di essere esaminato partendo da *Z.*

L'algoritmo è riportato nella Figura 5.8. Le variabili globali *n* e *dist* descrivono il grafo e sono inizializzate prima della chiamata di *shortest_path.* L'unica differenza tra il programma e l'algoritmo appena descritto è che nella Figura 5.8 il percorso più breve è calcolato iniziando dal nodo terminale *t* e non dal nodo di origine *s.*

```
#define MAX_NODES 1024                /* Numero massimo di nodi */
#define INFINITY 1000000000           /* Un numero più grande di qualunque percorso massimo */
int n, dist[MAX_NODES][MAX_NODES]; /* dist[i][j] rappresenta la distanza da i a j */

void shortest3path(int s, int t, int path[])
{ struct state {                      /* Il percorso elaborato */
     int predecessor;                 /* nodo precedente */
     int length;                      /* distanza dall'origine a questo nodo */
     enum {permanent, tentative} label;   /* etichetta lo stato */
} state[MAX_NODES];

int i, k, min;
struct state *p;

for (p = &state[0]; p < &state[n]; p++) { /* inizializza lo stato */
     p->predecessor = -1;
     p->length = INFINITY;
     p->label = tentative;
}
state[t].length = 0; state[t].label = permanent;
k = t;                                        /* k è il working node iniziale */
do {                                          /* C'è un percorso migliore che parte da k? */
     for (i = 0; i < n; i++)                  /* Questo grafo ha n nodi */
          if (dist[k][i] != 0 && state[i].label == tentative) {
              if (state[k].length + dist[k][i] < state[i].length) {
                  state[i].predecessor = k;
                  state[i].length = state[k].length + dist[k][i];
              }
          }

     /* Trova il nodo etichettato provvisoriamente con l'etichetta più piccola. */
     k = 0; min = INFINITY;
     for (i = 0; i < n; i++)
          if (state[i].label == tentative && state[i].length < min) {
              min = state[i].length;
              k = i;
          }
     state[k].label = permanent;
  } while (k != s);

  /* Copia il percorso nel vettore di output. */
  i = 0; k = s;
  do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}
```

Figura 5.8 Implementazione dell'algoritmo di Dijkstra per calcolare i cammini minimi in un grafo.

Poiché il percorso più breve che va da *t* a *s* in un grafo non orientato è identico al percorso più breve che va da *s* a *t*, l'estremità dalla quale si parte non ha importanza. La ricerca viene eseguita al contrario, perché ogni nodo è etichettato con il suo predecessore e non con il suo successore.

Quando il percorso finale viene copiato nella variabile di output, *path,* il percorso risulta perciò invertito; invertendo la ricerca, i due effetti si annullano e la risposta è prodotta nell'ordine corretto.

Flooding

Quando viene implementato un algoritmo di routing, ogni router deve effettuare le scelte in base alla sua conoscenza locale e non in base alla conformazione dell'intera rete. Una semplice tecnica locale è il **flooding** (letteralmente, *inondazione*), in cui ogni pacchetto in arrivo è inviato a tutte le linee in uscita tranne quella da cui proviene.

Ovviamente questo meccanismo genera un elevato numero di pacchetti duplicati; anzi, teoricamente il numero è infinito, a meno che non si prendano delle misure per attenuare il fenomeno. Una possibile tecnica è utilizzare un contatore di salti inserito nell'intestazione di ogni pacchetto e decrementare il suo valore a ogni salto, in modo da scartare il pacchetto quando il contatore raggiunge lo zero. Idealmente il valore iniziale da assegnare al contatore dei salti dovrebbe corrispondere alla lunghezza del percorso dalla sorgente alla destinazione. Se la sorgente non conosce la lunghezza del percorso, può assegnare al contatore il valore che rappresenta il caso peggiore, vale a dire il diametro dell'intera rete.

Il flooding con il conteggio dei salti può produrre una crescita esponenziale del numero di pacchetti duplicati quando il contatore cresce e i router duplicano pacchetti già visti prima. Usando una tecnica migliore i router tengono traccia dei pacchetti già trasmessi, in modo da evitare una seconda trasmissione. Per ottenere questo risultato, il router di origine deve inserire in ogni pacchetto che riceve dai suoi host un numero di sequenza. Ogni router ha quindi bisogno di una lista per ciascuno dei router sorgenti, che indichi i numeri di sequenza già trasmessi da quella sorgente. Se appare nella lista, il pacchetto in arrivo non viene trasmesso.

Per impedire una crescita illimitata ogni lista dovrebbe essere integrata con un contatore k, a indicare che sono stati visti tutti i numeri di sequenza fino a k. Quando arriva un pacchetto è facile controllare se è un duplicato paragonando il suo numero di sequenza con k; in caso affermativo viene scartato. Inoltre non è necessaria tutta la lista al di sotto di k, poiché in pratica k la riassume totalmente.

Il flooding non è un metodo pratico per inviare pacchetti, ma ha alcune importanti applicazioni. In primo luogo garantisce che un pacchetto venga consegnato a tutti i nodi della rete. Sarebbe uno spreco se la destinazione fosse unica, ma è efficace per informazioni indirizzate a tutti (*broadcast*). Nelle reti wireless tutti i messaggi trasmessi da una stazione possono essere ricevuti da tutte le altre stazioni che si trovano nel raggio di trasmissione della sorgente radio; si tratta in effetti di flooding e alcuni algoritmi utilizzano proprio questa proprietà.

In secondo luogo, il flooding è totalmente robusto. Anche se un gran numero di router saltasse (per esempio in una rete militare in una zona di guerra), il flooding troverebbe un percorso, se esiste, per portare il messaggio a destinazione. Il flooding, inoltre, richiede poco in termini di configurazione; i router devono solo conoscere i loro vicini; quindi può essere usato come componente base di altri algoritmi più efficienti che però richiedono più tempo per la configurazione. Infine, l'algoritmo di flooding può essere utilizzato anche come metrica di confronto per altri algoritmi di routing; il flooding sceglie sempre il percorso più breve, perché

sceglie ogni possibile percorso in parallelo; di conseguenza nessun altro algoritmo può produrre un ritardo più breve (se si ignora il tempo di elaborazione dati generato dal processo stesso di flooding).

Routing basato sul vettore delle distanze

Le reti di calcolatori utilizzano generalmente algoritmi di routing dinamici più complessi ma più efficienti del flooding, perché trovano il cammino più breve per la topologia corrente. I due algoritmi dinamici più popolari sono il routing basato sul vettore delle distanze e il routing basato sullo stato dei collegamenti (*link state routing*); questo paragrafo esamina il primo algoritmo, il seguente esaminerà il secondo.

Gli algoritmi di routing basati sul **vettore delle distanze** (*distance vector routing*) operano facendo in modo che ogni router conservi una tabella (ossia un vettore) che definisce la migliore distanza conosciuta per ogni destinazione e il collegamento che conduce a tale destinazione. Queste tabelle sono aggiornate scambiando informazioni con i router vicini. Alla fine del processo ogni router conosce il collegamento migliore per raggiungere qualsiasi destinazione.

Il routing basato sul vettore delle distanze qualche volta è chiamato anche con altri nomi; tra i più comuni vi è l'algoritmo di routing **Bellman-Ford** distribuito, dai nomi dei ricercatori che lo hanno sviluppato (Bellman, 1957; Ford e Fulkerson, 1962). È l'algoritmo di routing originale di ARPANET ed è tuttora utilizzato anche in Internet con il nome di RIP.

Nel routing basato sul vettore delle distanze ciascun router conserva una tabella di routing che usa come indice dei router della rete e dove è memorizzata una voce per ognuno di essi. Questa voce è composta da due parti: la linea di trasmissione preferita da utilizzare per quella destinazione e una stima del tempo o della distanza associata a quella destinazione. La distanza potrebbe essere misurata in termini del numero di salti o di un'altra metrica, com'è stato discusso trattando i cammini minimi.

Si presume che il router conosca la "distanza" che lo separa da ognuno dei suoi vicini. Se è misurata in salti, la distanza è pari a un solo salto. Se la metrica è il ritardo di propagazione, il router può misurare direttamente questo valore mediante speciali pacchetti ECHO che il ricevente rimanda indietro il più velocemente possibile dopo aver registrato al loro interno data e ora di trasmissione.

Come esempio si supponga che la distanza sia misurata con il ritardo e che il router conosca il ritardo di ogni vicino. Una volta ogni T ms, ogni router invia ai suoi vicini una lista delle proprie stime di ritardo relative a ciascuna destinazione. Il router riceve inoltre una lista simile da ognuno dei suoi vicini. Si immagini che una di queste tabelle sia appena arrivata dal vicino X, e che X_i rappresenti la stima del tempo impiegato da X per raggiungere il router i. Se sa che il ritardo per arrivare a X è pari a m ms, allora il router sa anche che può raggiungere il router i passando attraverso X in $X_i + m$ ms. Eseguendo questo calcolo per ognuno dei vicini un router può determinare la stima migliore e utilizzarla nella sua nuova tabella di routing assieme all'indicazione della linea corrispondente. Si noti che la vecchia tabella di routing non è utilizzata nel calcolo.

Questo processo di aggiornamento è illustrato nella Figura 5.9. La parte (a) mostra una rete; le prime quattro colonne della parte (b) mostrano i vettori di ritardo ricevuti dai vicini del router *J*. *A* dichiara un ritardo di 12 ms verso *B,* un ritardo di 25 ms verso *C*, un ritardo di 40 ms verso *D* e

così via. Si supponga che *J* abbia misurato o stimato il suo ritardo verso i suoi vicini *A*, *I*, *H* e *K* rispettivamente pari a 8, 10, 12 e 6 ms.

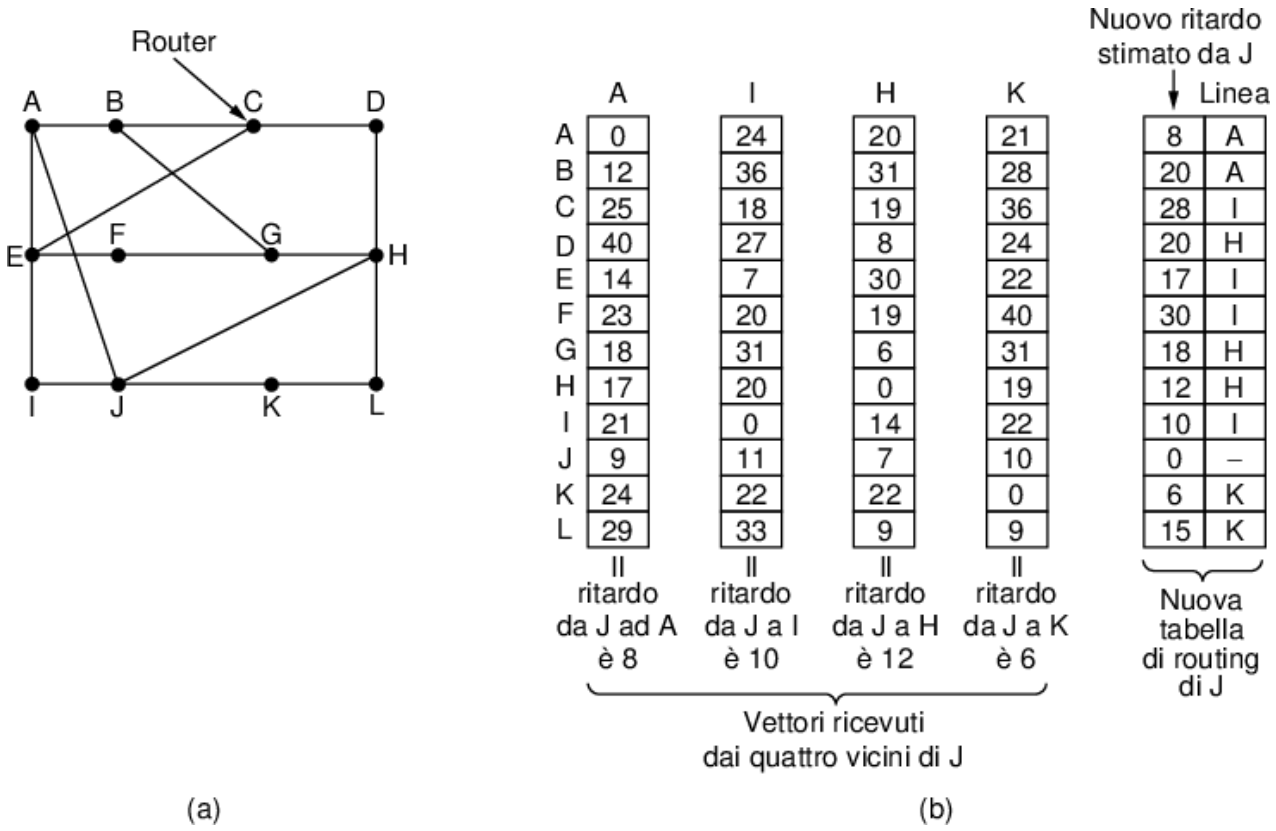


Figura 5.9 (a) Una rete. (b) Input da A, I, H, K e la nuova tabella di routing di J.

Vediamo come *J* calcola il suo nuovo percorso per il router *G*. Poiché sa di poter raggiungere *A* in 8 ms e poiché *A* dichiara di poter arrivare a *G* in 18 ms, sa che trasmettendo i suoi pacchetti verso *A* potrà raggiungere *G* in 26 ms. In modo analogo, la distanza da *G* calcolata passando attraverso *I, H* e *K* è rispettivamente di 41 (31 + 10), 18 (6 + 12) e 37 (31 + 6) ms. Il valore migliore è 18, perciò *J* crea nella sua tabella di routing una voce associata a *G* e vi registra il ritardo 18 ms e la linea di trasmissione *H*. Lo stesso calcolo è eseguito per tutte le altre destinazioni; la nuova tabella di routing è mostrata nell'ultima colonna della figura.

Il problema del conteggio all'infinito

Il processo di assestamento dei percorsi verso la configurazione dei cammini ottimi è chiamato anche **convergenza**. In teoria il routing basato sul vettore delle distanze è utile, perché è una tecnica semplice con cui i router possono collettivamente calcolare i cammini minimi, ma ha un serio difetto pratico: sebbene converga verso la risposta corretta, può raggiungere l'obiettivo molto lentamente. In particolare reagisce rapidamente alle buone notizie, ma troppo lentamente a quelle cattive. Per esempio si supponga che il percorso migliore da un router a una destinazione *X* sia molto lungo. Se uno degli scambi successivi con il vicino *A* improvvisamente indica un ritardo

breve verso *X*, il router inizia a utilizzare la linea che punta ad *A* per inoltrare il traffico verso *X*. Le buone notizie sono elaborate in un solo scambio di vettori. Per vedere con quale velocità si propaghi la buona notizia, si consideri la rete (lineare) composta da cinque nodi mostrata nella Figura 5.10; in questo caso la distanza è rappresentata dal numero di salti. Si supponga che *A* inizialmente non sia attivo e che tutti i router ne siano a conoscenza. In altre parole, tutti i router hanno registrato un ritardo infinito verso *A*.

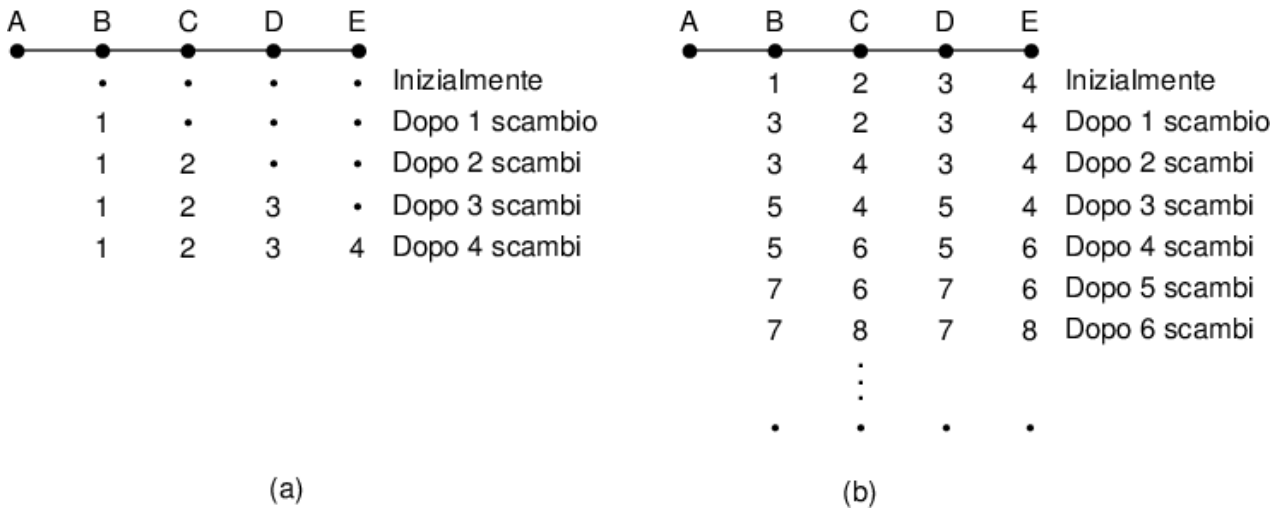


Figura 5.10 Il problema del conteggio all'infinito.

Quando *A* entra in azione, gli altri router si accorgono della sua presenza grazie allo scambio di vettori. Per semplicità si supponga che da qualche parte ci sia un gigantesco gong, colpito periodicamente per iniziare uno scambio di vettori contemporaneo su tutti i router. Al primo scambio *B* scopre che il suo vicino di sinistra dista 0 da *A*; *B* crea quindi una voce nella sua tabella di routing registrando che *A* si trova a un salto di distanza a sinistra. Tutti gli altri router credono ancora che *A* non sia attivo. Le voci della tabella di routing relative ad *A* sono riportate nella seconda riga mostrata nella Figura 5.10(a). Allo scambio successivo *C* scopre che *B* ha un percorso di lunghezza 1 diretto ad *A,* perciò aggiorna la sua tabella di routing per indicare un percorso di lunghezza 2, ma *D* ed *E* non sentono ancora le buone notizie. Logicamente la buona notizia si diffonde alla velocità di un salto per ciascuno scambio. In una rete il cui percorso più lungo richiede *N* salti, entro *N* scambi tutti vengono a conoscenza dei nuovi router e delle nuove linee.

Si consideri ora la situazione mostrata nella Figura 5.10(b), in cui tutte le linee e i router sono inizialmente attivi. La distanza tra *A* e i router *B*, *C*, *D* ed *E* è rispettivamente di 1, 2, 3 e 4. Improvvisamente *A* si spegne oppure la linea tra *A* e *B* si interrompe (ciò è tecnicamente la stessa cosa, dal punto di vista di *B*).

Al primo scambio di pacchetti *B* non riceve nulla da *A*. Fortunatamente *C* dice: "Non ti preoccupare, io ho un percorso che dista 2 da *A*." *B* non può sapere che il percorso di *C* passa da *B* stesso. Per quanto ne sa *B, C* potrebbe avere dieci linee, tutte con percorsi separati diretti ad *A* di

lunghezza 2. Di conseguenza, *B* pensa di poter raggiungere *A* attraverso *C* con un percorso di lunghezza 3. *D* ed *E* non aggiornano le loro voci relative ad *A* durante il primo scambio.

Al secondo scambio *C* nota che tutti i suoi vicini affermano di essere a distanza 3 da *A;* sceglie quindi a caso uno dei vicini e imposta a 4 la sua nuova distanza da *A,* come mostrato nella terza riga della Figura 5.10(b). Gli scambi seguenti producono i risultati mostrati nelle altre righe della Figura 5.10(b).

Dalla figura dovrebbe essere chiaro perché le cattive notizie viaggiano lentamente: nessun router ha mai un valore che supera di oltre un'unità quello minimo di tutti i vicini. Lentamente tutti i router trovano la loro strada per l'infinito, ma il numero di scambi richiesti dipende dal valore numerico utilizzato per rappresentarlo. Per questo motivo è meglio rappresentare infinito tramite il valore corrispondente al percorso più lungo all'interno della rete più uno.

Non c'è da stupirsi molto che questo problema sia noto come **problema del conteggio all'infinito** (*count-to-infinity*). Sono stati fatti diversi tentativi per risolverlo, per esempio impedendo ai router di dichiarare i cammini ottimi ai vicini dai quali li avevano ricevuti con una tecnica discussa nell'RFC 1058. Tuttavia nessuno di questi espedienti funziona bene in pratica. Il nocciolo del problema è che quando *X* dice a *Y* che ha un percorso che punta da qualche parte, *Y* non ha modo di sapere se fa parte di quel percorso.

Routing basato sullo stato dei collegamenti

Il routing basato sul vettore delle distanze è stato utilizzato in ARPANET fino al 1979, quando fu sostituito dal routing basato sullo stato dei collegamenti. Il problema principale fu che l'algoritmo spesso impiegava troppo tempo a raggiungere la convergenza dopo un cambio di topologia (per il problema del conteggio all'infinito). Per questo motivo venne sostituito da un algoritmo completamente nuovo, che oggi è chiamato **routing basato sullo stato dei collegamenti** (*link state routing*). I due algoritmi più usati attualmente nelle reti di grandi dimensioni e in Internet sono due sue varianti: IS-IS e OSPF.

L'idea alla base di questo algoritmo è semplice e può essere riassunta in cinque passaggi. Ogni router deve:

1. scoprire i propri vicini e i relativi indirizzi di rete
2. misurare la distanza o la metrica di costo di ogni vicino
3. costruire un pacchetto contenente tutte le informazioni raccolte
4. inviare tale pacchetto a tutti gli altri router e ricevere da loro i pacchetti
5. elaborare il percorso più breve verso tutti gli altri router.

In pratica, la topologia completa viene distribuita a ogni router; successivamente viene eseguito l'algoritmo di Dijkstra per trovare il percorso più breve associato a ogni altro router. Il paragrafo successivo esamina in dettaglio ognuno dei cinque passaggi.

Scoperta dei vicini

Quando viene acceso, il router prima di tutto cerca di scoprire chi sono i propri vicini. Il dispositivo raggiunge il suo obiettivo inviando uno speciale pacchetto HELLO su ogni linea punto a punto; il router all'altro capo della linea deve rispondere fornendo il proprio nome. Questi nomi

devono essere globalmente unici perché, quando in seguito un router lontano rileva che tre router sono collegati a *F*, è essenziale che possa determinare se tutti e tre intendono lo stesso *F*.

Quando due o più router sono connessi tramite un collegamento broadcast (per esempio, uno switch, un anello o una Ethernet classica) la situazione è leggermente più complicata. La Figura 5.11(a) mostra una LAN broadcast a cui sono direttamente collegati tre router *A*, *C* e *F* e ognuno di questi è collegato a uno o più router aggiuntivi come indicato.

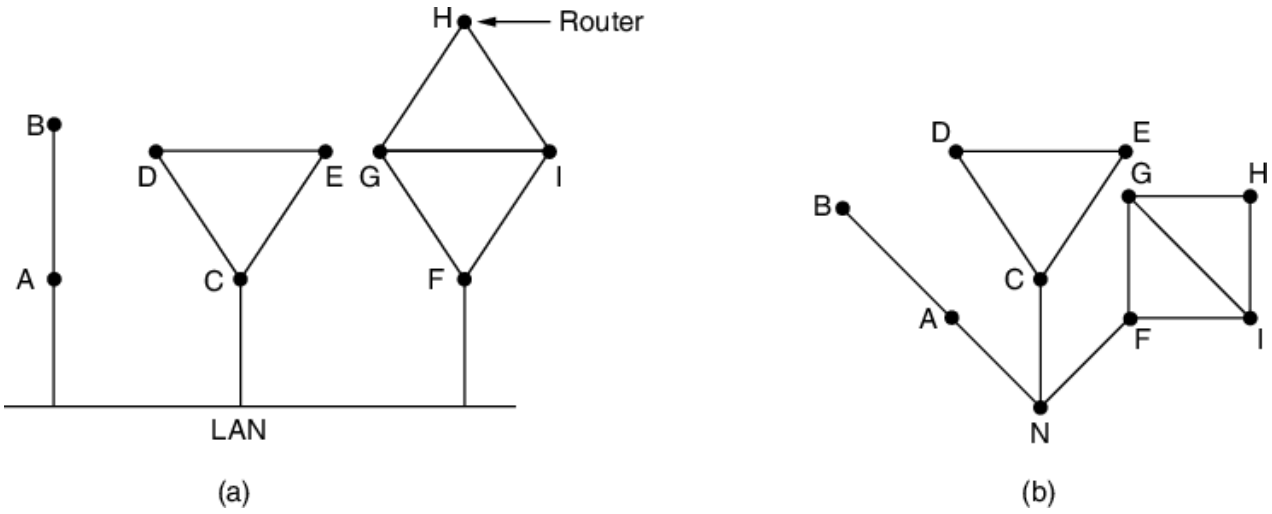


Figura 5.11 (a) Nove router e una LAN broadcast. (b) Un modello di (a) tramite un grafo.

La LAN broadcast fornisce connettività tra ogni coppia di router connessi. Tuttavia, modellare la LAN come molti collegamenti punto a punto aumenta la dimensione della topologia e causa messaggi non necessari. Un modo migliore per modellare la LAN è considerarla nell'insieme come un solo nodo, come avviene nella Figura 5.1l(b). In questa figura è stato introdotto un nuovo nodo artificiale, *N*, al quale si collegano *A*, *C* e *F*. Un **router designato** (*designated router*) nella LAN viene selezionato per avere il ruolo di *N* nel protocollo di routing. Il fatto che sia possibile andare da *A* a *C* sulla LAN è rappresentato dal percorso *ANC*.

Misurazione del costo dei collegamenti

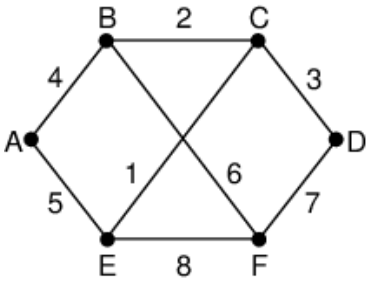
Per calcolare i cammini minimi, l'algoritmo *link state routing* richiede che ogni collegamento abbia una metrica di costo o distanza. Il costo per raggiungere i vicini può essere configurato automaticamente o dall'operatore di rete. Per esempio, una linea Ethernet a 1 Gbps può avere costo 1 e una a 100 Mbps costo 10. Questa scelta rende i cammini con capacità più alte la scelta migliore.

Se la rete è geograficamente estesa, il ritardo dei collegamenti può essere incluso nella funzione di costo, in modo da favorire i cammini con collegamenti corti. Il modo più diretto per determinare questo ritardo consiste nell'inviare attraverso la linea uno speciale pacchetto ECHO, al quale l'altra

parte deve rispondere immediatamente. Misurando il tempo di andata e ritorno e dividendo il risultato per due, il router trasmittente può ottenere una stima ragionevole del ritardo.

Costruzione dei pacchetti che contengono lo stato dei collegamenti

Dopo aver raccolto le informazioni necessarie per lo scambio, ogni router deve costruire un pacchetto contenente tutti i dati. Il pacchetto inizia con l'identità del trasmittente, seguita da un numero di sequenza, dall'età (*Age*, descritta più avanti) e da una lista dei vicini. Per ogni vicino è riportato il ritardo misurato. La Figura 5.12(a) mostra un esempio di rete; i ritardi sono rappresentati da etichette associate alle linee. I pacchetti che contengono lo stato dei collegamenti per tutti e sei i router sono mostrati nella Figura 5.12(b).



(a)

Coll. Stato Pacchetti

A		B		C		D		E		F	
Seq.		Seq.		Seq.		Seq.		Seq.		Seq.	
Età		Età		Età		Età		Età		Età	
B	4	A	4	B	2	C	3	A	5	B	6
E	5	C	2	D	3	F	7	C	1	D	7
		F	6	E	1			F	8	E	8

(b)

Figura 5.12 (a) Una rete. (b) I pacchetti con le informazioni di stato dei collegamenti di questa rete.

Costruire questi pacchetti è facile; difficile è determinare quando costruirli. I pacchetti che contengono le informazioni sullo stato dei collegamenti potrebbero essere costruiti periodicamente, ossia a intervalli regolari, oppure potrebbero essere creati quando accade un evento significativo: si interrompe una linea, un vicino si spegne, torna di nuovo attivo o cambia le sue proprietà.

Distribuzione dei pacchetti che contengono lo stato dei collegamenti

La parte più delicata dell'algoritmo è quella relativa alla distribuzione rapida e affidabile dei pacchetti che descrivono lo stato dei collegamenti. Se router diversi utilizzano versioni differenti della topologia, si rischia di creare inconsistenze come cicli, computer irraggiungibili e così via.

Questo paragrafo descrive l'algoritmo di distribuzione di base; più avanti ne saranno analizzati alcuni miglioramenti. L'idea fondamentale è di utilizzare il flooding per distribuire i pacchetti contenenti le informazioni sullo stato dei collegamenti. Per tenere sotto controllo il flusso di dati, in ogni pacchetto è inserito un numero di sequenza, incrementato per ogni nuovo pacchetto inviato. I router tengono traccia di tutte le coppie (router sorgente, sequenza) rilevate. Quando arriva un nuovo pacchetto contenente informazioni sullo stato del collegamento, il router

confronta i dati con quelli già visti. Se è nuovo, il pacchetto è inoltrato attraverso tutte le linee tranne quella di ricezione; se è un duplicato, viene scartato. Se arriva un pacchetto con un numero di sequenza inferiore al numero più alto visto fino a quel momento, i dati sono rifiutati in quanto obsoleti poiché il router possiede informazioni più recenti.

Questo algoritmo ha alcuni difetti che sono tuttavia gestibili. Prima di tutto, numeri di sequenza ripetitivi potrebbero generare il caos. Per risolvere questo problema si utilizzano numeri di sequenza lunghi 32 bit; trasmettendo ogni secondo un pacchetto di dati sullo stato dei collegamenti, i numeri di sequenza si esaurirebbero non prima di 137 anni, perciò questa possibilità può essere ignorata.

Secondo, quando un router si blocca perde traccia dei suoi numeri di sequenza. Se il conteggio ricomincia da 0, il pacchetto successivo viene rifiutato perché ritenuto un duplicato.

Terzo, se un numero di sequenza arriva danneggiato e il router legge 65.540 al posto del numero 4 (è un errore di un solo bit), i pacchetti marcati con i numeri che vanno da 5 a 65.540 saranno considerati obsoleti, poiché il router crede che il numero di sequenza sia 65.540.

Per risolvere tutti questi problemi è necessario includere l'età di ogni pacchetto subito dopo il numero di sequenza e decrementare il suo valore una volta al secondo. Quando l'età raggiunge lo zero, le informazioni provenienti da quel router vengono scartate. Normalmente un nuovo pacchetto arriva (per esempio) ogni 10 secondi, perciò le informazioni del router scadono quando un router è spento (o sei pacchetti consecutivi vengono perduti, evento piuttosto raro). Il campo *Age* è anche decrementato da ogni router durante il processo di flooding iniziale per garantire che nessun pacchetto possa perdersi nella rete e durare un periodo indefinito di tempo (un pacchetto la cui età è zero viene scartato).

Alcuni miglioramenti apportati a questo algoritmo rendono il processo ancora più robusto. Quando un pacchetto contenente i dati sullo stato dei collegamenti raggiunge un router per il flooding esso non viene accodato immediatamente per la trasmissione. I dati vengono prima inseriti in un'area di parcheggio e qui rimangono per un breve periodo in caso altri collegamenti si interrompano o diventino attivi. Se prima della trasmissione arriva dalla stessa sorgente un altro pacchetto contenente dati sullo stato dei collegamenti, il router confronta i numeri di sequenza; se i valori sono uguali il duplicato viene scartato, altrimenti viene eliminato il più vecchio. Per prevenire errori sulle linee da router a router, tutti i pacchetti contenenti informazioni relative allo stato dei collegamenti ricevono un acknowledgement.

La struttura dati utilizzata dal router *B* per la rete mostrata nella Figura 5.12(a) è descritta nella Figura 5.13. Ogni riga corrisponde a un pacchetto contenente informazioni sullo stato dei collegamenti arrivato di recente, ma non ancora elaborato. La tabella contiene l'origine del pacchetto, il suo numero di sequenza, l'età e i dati; inoltre esistono flag per l'invio e la ricezione dell'acknowledgement per ognuna delle tre linee di *B* (rispettivamente per *A*, *C* e *F).* La presenza del flag di invio indica che il pacchetto deve essere trasmesso su quella linea; il flag ACK indica se si è in attesa di un acknowledgement.

Origine	Seq.	Età	Flag di invio A	Flag di invio C	Flag di invio F	Flag di ACK A	Flag di ACK C	Flag di ACK F	Dati
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

Figura 5.13 Il buffer dei pacchetti per il router B nella **Figura 5.12** (a).

Nella Figura 5.13 il pacchetto contenente dati sullo stato dei collegamenti proveniente da *A* arriva direttamente, perciò deve essere inviato a *C* e a *F*, mentre ad *A* va inviato un acknowledgement, come indicato dai bit di flag. In modo analogo, il pacchetto proveniente da *F* deve essere inoltrato ad *A* e a *C,* mentre l'acknowledgement va inviato a *F*.

Tuttavia, la situazione cambia con il terzo pacchetto, quello proveniente da *E.* I dati arrivano due volte, la prima attraverso *EAB* e l'altra attraverso *EFB.* Di conseguenza, il pacchetto deve essere trasmesso solo a *C*, ma l'acknowledgement va inviato sia ad *A* sia a *F,* come indicato dai bit.

Se arriva un duplicato mentre l'originale è ancora nell'area di parcheggio i bit devono essere modificati. Per esempio, se una copia dello stato di *C* arriva da *F* prima dell'inoltro della quarta voce della tabella, i sei bit devono diventare 100011 per indicare che a *F* va inviato l'acknowledgement del pacchetto, ma non il pacchetto vero e proprio.

Calcolo di nuovi percorsi

Dopo aver accumulato una serie completa di pacchetti sullo stato dei collegamenti della rete, il router può costruire il grafo dell'intera rete in cui viene rappresentato ogni collegamento. In realtà ogni collegamento è rappresentato due volte, una per ogni direzione. Poiché il collegamento può avere un costo diverso a seconda della direzione, si possono trovare cammini minimi diversi per andare dal router *A* a *B* rispetto che da *B* ad *A*.

A questo punto ogni router può eseguire localmente l'algoritmo di Dijkstra per costruire il percorso più breve verso tutte le possibili destinazioni. I risultati di questo algoritmo, che indicano al router che linea usare per raggiungere qualsiasi destinazione, possono essere installati nelle tabelle di routing, prima di riprendere il normale funzionamento.

Il link state routing richiede più memoria e tempo di calcolo rispetto al distance vector routing. Per una rete con *n* router, ognuno dei quali ha *k* vicini, la memoria richiesta per conservare i dati in input è proporzionale a *kn*, che è grande almeno come una tabella di routing contenente tutte le destinazioni. Inoltre il tempo di calcolo cresce più velocemente di *kn*, persino utilizzando le

strutture dati più efficienti possibili; per reti grandi questo può diventare un problema. Ciò nonostante in molte situazioni pratiche il link state routing funziona bene perché non presenta problemi di convergenza lenta.

Il link state routing è molto utilizzato nelle reti reali, perciò è utile spendere qualche parola esemplificando alcuni protocolli che lo utilizzano. Molti ISP usano il protocollo link state **IS-IS** (*intermediate system-intermediate system)* (Oran, 1990). Fu progettato per una delle prime reti, chiamata DECnet, e successivamente modificato per gestire anche altri protocolli, tra i quali il principale è IP. L'altro importante protocollo link state è **OSPF** (*open shortest path first*). Fu progettato da IETF alcuni anni dopo IS-IS, da cui prese molti aspetti innovativi. Tra questi troviamo un metodo per il flooding degli aggiornamenti sullo stato dei collegamenti in grado di raggiungere autonomamente uno stato stabile, il concetto di router predefinito su una LAN e un metodo per calcolare e supportare la divisione dei percorsi e metriche multiple. Di conseguenza, le differenze tra IS-IS e OSPF sono lievi; la più importante è che IS-IS è codificato in modo tale da semplificare il trasporto simultaneo di informazioni relative a più protocolli di livello di rete (per esempio, IP, IPX e AppleTalk), funzionalità non supportata da OSPF. Questo vantaggio è prezioso soprattutto in ambienti di grandi dimensioni dove vanno supportati più protocolli. OSPF sarà illustrato nel Paragrafo 5.6.6.

Facciamo ora un commento generale sugli algoritmi di routing. L'algoritmo basato sullo stato dei collegamenti, quello basato sul vettore delle distanze e altri prevedono che tutti i router calcolino i cammini. Problemi causati da hardware o da software anche su pochi router possono provocare disastri sulla rete. Per esempio, se un router afferma di avere una linea che in realtà non ha, o dimentica di indicare una linea che invece esiste, il grafo della rete risulterà sbagliato. Se un router non riesce a inoltrare i pacchetti o li danneggia durante l'inoltro, il percorso non funzionerà come previsto. Infine, se un router esaurisce la memoria o esegue un calcolo di routing errato, la rete avrà dei problemi. Quando la rete inizia a essere composta da decine o centinaia o migliaia di nodi, la probabilità che si verifichi un guasto non è più trascurabile. Il trucco sta nel limitare i danni quando si verifica un inevitabile guasto. Perlman (1988) discute queste problematiche e ne descrive in dettaglio alcune possibili soluzioni.

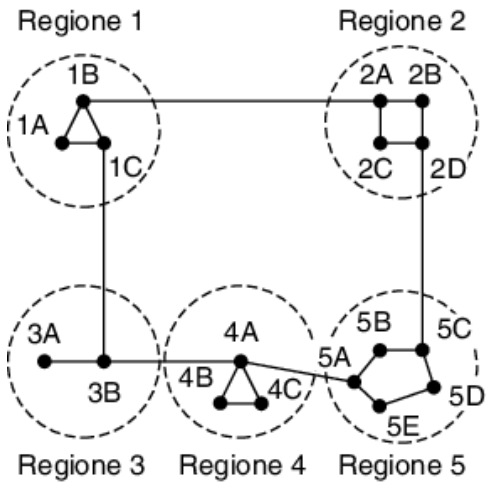
Routing gerarchico

La dimensione delle tabelle di routing cresce proporzionalmente alla dimensione della rete. La crescita delle tabelle non soltanto consuma la memoria del router ma aumenta il tempo che la CPU impiega ad analizzare i dati e la banda necessaria per inviare le informazioni sul loro stato. La rete può crescere al punto che non è più possibile per i router avere una voce (*entry*) per ogni altro router; in questo caso il routing dovrà avvenire gerarchicamente, come in una rete telefonica.

Quando si utilizza il routing gerarchico, i router sono divisi in **regioni**: ogni router conosce tutti i dettagli relativi al routing dei pacchetti diretti a destinazioni nella stessa regione, ma non sa nulla della struttura interna delle altre. Quando si interconnettono reti diverse è naturale considerare ogni rete come una regione separata; in questo modo i router di una rete non sono costretti a conoscere la struttura topologica delle altre. Per reti enormi può essere insufficiente una struttura gerarchica a due livelli; può essere necessario raggruppare le regioni in cluster, i cluster in zone, le zone in gruppi e così via, fino a esaurire tutti i nomi delle aggregazioni. Come esempio di gerarchia multilivello si consideri il modo in cui un pacchetto potrebbe essere inoltrato da

Berkeley, California, a Malindi, città del Kenya. Il router di Berkeley conosce i dettagli della topologia della California, ma invia al router di Los Angeles tutto il traffico diretto all'esterno dello stato. Il router di Los Angeles è in grado di instradare il traffico diretto agli altri router statunitensi, ma invia al router di New York i dati diretti all'estero. Il router di New York è programmato per dirigere tutto il traffico al router del paese di destinazione responsabile della gestione del traffico estero, che in questo caso potrebbe trovarsi a Nairobi. Infine il pacchetto da Nairobi troverà la strada per raggiungere il Kenya e quindi Malindi.

La Figura 5.14 fornisce un esempio quantitativo di routing in una gerarchia a due livelli contenente cinque regioni. La tabella di routing completa del router 1*A* contiene 17 voci, come mostrato nella Figura 5.14(b). Quando il routing è fatto gerarchicamente, come nella Figura 5.14(c), ci sono voci per tutti i router locali (come prima), ma tutte le altre regioni sono condensate in un singolo router, perciò tutto il traffico per la regione 2 viene trasmesso attraverso la linea 1*B*-2*A*, ma il resto del traffico remoto passa per la linea 1*C*-3*B*. Il routing gerarchico ha ridotto il numero di voci nella tabella da 17 a 7. Lo spazio risparmiato nelle tabelle aumenta al crescere del rapporto tra numero di regioni e numero di router per regione.



(a)

Tabella completa per 1A

Dest.	Linea	Salti
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Tabella gerarchica per 1A

Dest.	Linea	Salti
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

Figura 5.14 Routing gerarchico.

Sfortunatamente questi risparmi di spazio hanno un costo: la penalità da pagare è rappresentata dalla crescita della lunghezza dei percorsi. Per esempio, il percorso migliore che collega 1*A* a 5*C* passa attraverso la regione 2 ma, con il routing gerarchico, tutto il traffico diretto alla regione 5

viene inviato alla regione 3, perché quella è la strada migliore per la maggior parte delle destinazioni nella regione 5.

Quando una singola rete diventa molto grande è lecito domandarsi quanti livelli dovrebbe avere la gerarchia. Per esempio, si consideri una rete composta da 720 router. Se non ci fosse alcuna gerarchia, ogni router dovrebbe gestire una tabella composta da 720 voci. Se la rete fosse divisa in 24 regioni ognuna contenente 30 router, ogni router dovrebbe gestire una tabella composta da 30 voci locali, più 23 voci remote, per un totale di 53 voci. Scegliendo una gerarchia a tre livelli, con otto cluster ognuno contenente 9 regioni di 10 router, le tabelle dei router conterrebbero 10 voci per i router locali, 8 voci per i router delle altre regioni nello stesso cluster e 7 voci per cluster distanti, per un totale di 25 voci. Kamoun e Kleinrock (1979) hanno scoperto che il numero ottimale di livelli per una rete di N router è uguale a $ln(N)$, per un totale di $e * ln(N)$ voci per router. I due hanno anche dimostrato che l'aumento della lunghezza del percorso medio reale causato dal routing gerarchico è sufficientemente piccolo da renderlo il più delle volte accettabile.

Routing broadcast

In alcune applicazioni gli host hanno bisogno di inviare messaggi a molti o a tutti gli altri host. Per esempio un servizio di distribuzione di rapporti meteorologici, di aggiornamenti del mercato azionario o programmi radio dal vivo potrebbero funzionare meglio trasmettendo a tutti i computer e lasciando che quelli interessati leggano i dati.

La trasmissione contemporanea di un pacchetto a tutte le destinazioni è chiamata **broadcasting**. Per ottenere questo risultato sono stati proposti diversi metodi.

Un metodo di trasmissione broadcast che non richiede l'implementazione di alcuna funzionalità speciale nella rete è quello in cui la sorgente invia un pacchetto distinto a ogni destinazione. Questo metodo non solo spreca banda, ma obbliga la sorgente a possedere una lista completa di tutte le destinazioni. Anche se può essere adatto in molte situazioni, questo metodo non è molto popolare nella pratica.

Un metodo migliore è il **multidestination routing** (*routing a più destinazioni*), in cui ogni pacchetto contiene una lista delle destinazioni o una mappa di bit che indica le destinazioni desiderate. Quando riceve un pacchetto, il router controlla tutte le destinazioni per determinare l'insieme di linee di trasmissione richieste (una linea di trasmissione è necessaria se rappresenta il percorso migliore verso almeno una delle destinazioni). Il router genera una nuova copia del pacchetto per ogni linea di output da usare e include in ogni pacchetto solo quelle destinazioni che si trovano su quella linea; in altri termini, l'insieme delle destinazioni viene diviso tra le linee di trasmissione. Dopo un numero sufficiente di salti, ogni pacchetto conterrà una sola destinazione e potrà essere trattato come un pacchetto normale. È come se il routing a più destinazioni utilizzasse pacchetti indirizzati separatamente, ma quando diversi pacchetti devono seguire lo stesso percorso, uno solo paga il prezzo del biglietto, mentre gli altri viaggiano gratis. In questo modo la banda è usata in modo più efficiente. Tuttavia anche questo metodo richiede che la sorgente conosca tutte le destinazioni; inoltre un router deve effettuare più calcoli per determinare dove trasmettere un pacchetto con più destinazioni piuttosto che per molteplici pacchetti distinti.

Abbiamo già visto una tecnica broadcast migliore: il flooding. Se implementata con un numero di sequenza per sorgente, usa in modo efficiente i collegamenti con una regola di decisione adottata dai router relativamente semplice. Sebbene non si adatti bene alla comunicazione ordinaria punto a punto, nel caso della trasmissione broadcast potrebbe essere preso in seria considerazione. Tuttavia, una volta che i percorsi minimi per i pacchetti ordinari sono stati calcolati, è possibile fare ancora di meglio.

L'idea del **reverse path forwarding** (*instradamento sul percorso inverso*) una volta capita, è elegante e straordinariamente semplice (Dalal e Metcalfe, 1978). Quando riceve un pacchetto broadcast, il router verifica se il pacchetto è giunto attraverso la linea che normalmente è utilizzata per inviare i pacchetti *alla* sorgente della trasmissione broadcast. In caso affermativo, c'è una forte probabilità che il pacchetto broadcast stesso abbia seguito il percorso migliore dal router e che perciò sia la prima copia arrivata al router. In questo caso il router inoltra le copie del pacchetto attraverso tutte le linee, esclusa quella di input. Se al contrario il pacchetto broadcast è giunto attraverso una linea diversa da quella preferita per raggiungere la sorgente, il pacchetto è scartato, in quanto è probabile che si tratti di un duplicato.

La Figura 5.15 mostra un esempio di reverse path forwarding. La parte (a) mostra una rete, la parte (b) mostra un sink tree per il router *I* della rete e la parte (c) mostra in che modo funziona l'algoritmo del percorso inverso. Al primo salto, *I* invia pacchetti a *F*, *H*, *J* e a *N*, come indicato nella seconda riga della struttura ad albero. Ognuno di questi pacchetti giunge attraverso il percorso preferito associato a *I* (supponendo che il percorso preferito cada lungo il sink tree) ed è perciò indicato da un cerchio intorno alla lettera. Al secondo salto vengono generati otto pacchetti: ogni router che ha ricevuto un pacchetto al primo salto ne genera due. Tutti e otto arrivano a router non visitati precedentemente e cinque di questi giungono attraverso la linea preferita. Dei sei pacchetti generati al terzo salto, solo tre giungono attraverso il percorso preferito (*C*, *E* e *K*); gli altri sono duplicati. Dopo cinque salti e 24 pacchetti la trasmissione broadcast termina, mentre il sink tree impiega esattamente quattro salti e 14 pacchetti.

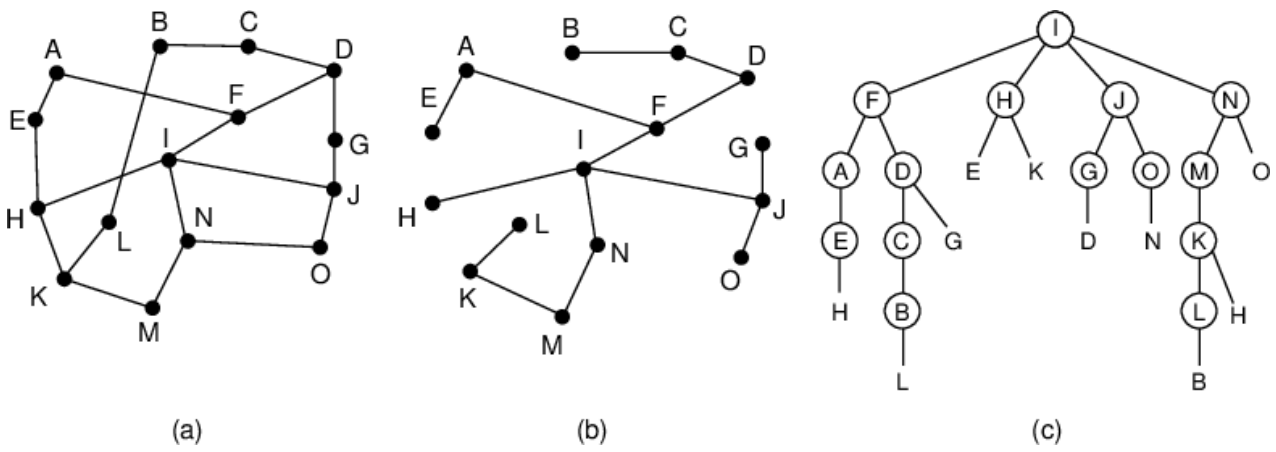


Figura 5.15 Reverse path forwarding. (a) Una rete. (b) Un sink tree. (c) L'albero costruito tramite reverse path forwarding.

Il vantaggio principale del reverse path forwarding è che si tratta di un sistema ragionevolmente efficiente e facile da implementare. Come il flooding, trasmette i pacchetti broadcast su ogni collegamento solo una volta in ogni direzione; inoltre richiede che i router sappiano solo come raggiungere tutte le destinazioni, senza che debbano memorizzare i numeri di sequenza (o usare altri meccanismi per fermare il flooding) o la lista di tutte le destinazioni del pacchetto.

L'ultimo algoritmo di trasmissione broadcast che consideriamo migliora il comportamento del reverse path forwarding. Fa esplicito uso del sink tree del router che ha iniziato la trasmissione (o di qualunque altro spanning tree utile). Uno **spanning tree** è un sottoinsieme che comprende tutti i router, ma che non contiene cicli. Se ogni router sa quali delle sue linee appartengono allo spanning tree, allora può copiare un pacchetto broadcast in arrivo su tutte le linee dello spanning tree esclusa quella d'ingresso. Questo metodo usa in modo eccellente la banda e genera in assoluto il minimo numero di pacchetti necessari per svolgere il lavoro. Per esempio, nella Figura 5.15, se il sink tree della parte (b) venisse usato come spanning tree, il pacchetto broadcast verrebbe inviato con il minimo uso di risorse: 14 pacchetti. L'unico problema sta nel fatto che, per rendere applicabile il metodo, ogni router deve conoscere uno spanning tree. Qualche volta questa informazione è disponibile (per esempio nel caso del link state routing nel quale tutti i router conoscono l'intera topologia della rete e possono calcolare lo spanning tree), altre volte invece non lo è (per esempio, nel caso del distance vector routing).

Routing multicast

Alcune applicazioni, come i giochi multiplayer o le dirette video di eventi sportivi trasmessi a più punti di visione, inviano i pacchetti a più destinazioni. A meno che il gruppo sia molto piccolo, inviare pacchetti distinti a ogni destinazione è costoso. D'altro canto, l'utilizzo del broadcast per informare 1000 macchine in una rete composta da milioni di nodi è inefficiente, perché la maggior parte dei ricevitori non è interessata a quel messaggio (o peggio ancora, potrebbe essere molto interessata al contenuto del messaggio, ma non dovrebbe ricevere quei dati). È quindi necessario trovare il modo di inviare messaggi a gruppi ben definiti che, pur essendo numericamente grandi, sono piccoli se confrontati con l'intera rete.

La trasmissione di un messaggio a un gruppo di questo tipo è chiamata **multicast** e il suo algoritmo di routing è chiamato **routing multicast**. Tutti gli schemi di trasmissione multicast richiedono un metodo che consenta di creare e distruggere i gruppi e identificare quali router appartengono a un gruppo. Il modo in cui queste operazioni sono eseguite non riguarda l'algoritmo di routing. Per ora assumiamo che ogni gruppo sia identificato con un indirizzo multicast e che i router conoscano i gruppi a cui appartengono.[2] Approfondiremo l'appartenenza ai gruppi nel Paragrafo 5.6, quando verrà descritto il livello di rete di Internet.
Gli schemi di routing multicast sono costruiti sugli schemi di routing broadcast che abbiamo già visto e inviano pacchetti lungo gli spanning tree per consegnarli ai membri di un gruppo facendo un uso efficiente della banda. Tuttavia, il miglior spanning tree da usare dipende dal fatto che il gruppo sia denso, con destinatari sparsi in tutta la rete, o sparso, con la maggior parte della rete non appartenente al gruppo. Considereremo entrambi i casi.

Se il gruppo è denso, il broadcast è un buon punto di partenza perché invia efficientemente i pacchetti a tutte le parti della rete, ma raggiunge anche router che non sono membri del gruppo generando degli sprechi. La soluzione esplorata da Deering e Cheriton (1990) è di "potare l'albero"

(*pruning*), riducendo lo spanning tree broadcast per tagliare i collegamenti che non portano a membri del gruppo. Il risultato è uno spanning tree multicast efficiente.

Per esempio nella rete della Figura 5.16(a) appaiono due gruppi, 1 e 2. Alcuni router sono collegati agli host che appartengono a uno o a entrambi i gruppi. La Figura 5.16(b) rappresenta uno spanning tree relativo al router posto più a sinistra. Questo albero può essere usato per il broadcast, ma è troppo abbondante per il multicast, come si può vedere dalle due versioni tagliate mostrate successivamente. Nella Figura 5.16(c) tutte le linee che non conducono agli host che sono membri del gruppo 1 sono tagliate. Il risultato è lo spanning tree multicast per il router più a sinistra associato al gruppo 1. I pacchetti vengono inoltrati lungo questo spanning tree che è molto più efficiente di quello broadcast, perché ha 7 collegamenti invece di 10. La Figura 5.16(d) mostra lo spanning tree multicast ridotto associato al gruppo 2. Anche questo è efficiente, avendo solo 5 collegamenti; inoltre mostra come gruppi multicast diversi hanno spanning tree differenti.

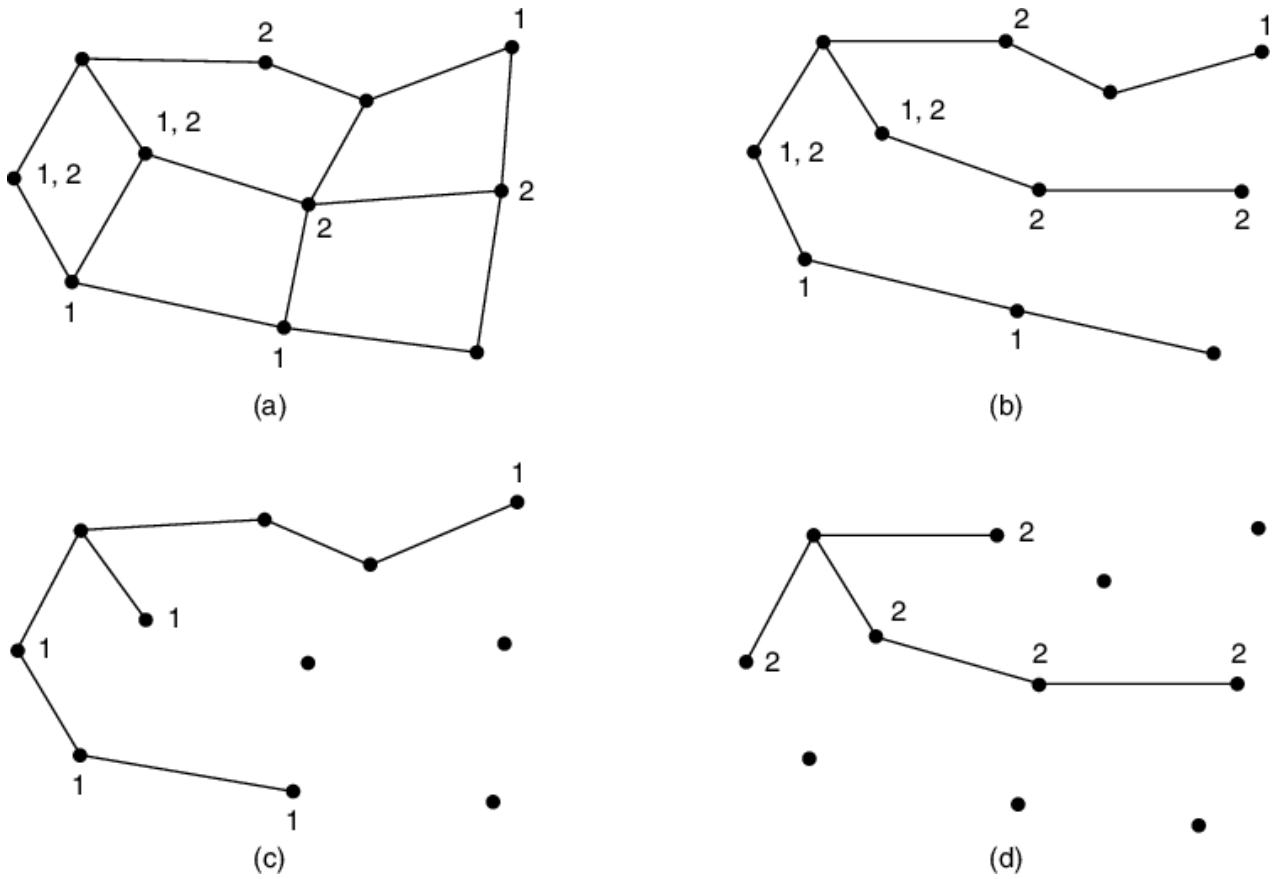


Figura 5.16 (a) Una rete. (b) Uno spanning tree per il router più a sinistra. (c) Un albero multicast per il gruppo 1. (d) Un albero multicast per il gruppo 2.

È possibile ridurre lo spanning tree in molti modi. Quello più semplice può essere utilizzato se si adotta il link state routing e se ogni router conosce la topologia completa, quindi sa quali host appartengono a ciascun gruppo. Ogni router costruisce il proprio spanning tree ridotto per ogni

sorgente verso il gruppo in questione, costruendo l'usuale sink tree e poi rimuovendo tutti i collegamenti che non connettono membri del gruppo alla radice dell'albero. **MOSPF** (*multicast* OSPF) è un esempio di protocollo link state che opera in questo modo (Moy, 1994).

Quando si usa il routing basato sul vettore delle distanze si deve seguire una strategia di riduzione diversa. L'algoritmo di base è quello del reverse path forwarding, ma ogni volta che un router che non ha host interessati a un particolare gruppo (e non ha connessioni dirette ad altri router) riceve un messaggio multicast diretto a quel gruppo, risponde con un messaggio PRUNE che avvisa il trasmittente di non inviare più alcun messaggio multicast per quel gruppo. Può rispondere con un messaggio PRUNE anche un router che riceve messaggi di questo tipo su tutte le sue linee e non ha host appartenenti a quel gruppo. In questo modo lo spanning tree viene ridotto in modo ricorsivo. **DVMRP** (*distance vector multicast routing protocol*) è un esempio di protocollo di routing multicast che opera in questo modo (Waitzman et al., 1988).

L'operazione di pruning porta a spanning tree efficienti che usano solo i collegamenti effettivamente necessari a raggiungere i membri del gruppo. Uno svantaggio potenziale di questo algoritmo è che fa svolgere ai router molti calcoli, adattandosi quindi male alle reti di grandi dimensioni. Si supponga che la rete abbia n gruppi, ognuno composto da un numero medio di m nodi. A ogni router e per ogni gruppo si devono memorizzare m spanning tree ridotti, per un totale di $m \cdot n$ alberi. Per esempio la Figura 5.16(c) mostra lo spanning tree del router più a sinistra per effettuare una trasmissione multicast al gruppo 1. Lo spanning tree per il router più a destra verso il gruppo 1 (non mostrato) sarebbe molto diverso, perché i pacchetti raggiungerebbero direttamente i membri del gruppo senza passare dalla parte sinistra del grafo. Questo significa che i router devono inoltrare i pacchetti destinati al gruppo 1 in direzioni diverse a seconda di quale nodo è la sorgente. Quando esistono molti gruppi di grosse dimensioni, l'archiviazione di tutte queste strutture dati può richiedere molta memoria.

Per calcolare un singolo spanning tree per gruppo, un'architettura alternativa utilizza strutture chiamate **core-based tree** (Ballardie et al., 1993). Tutti i router si mettono d'accordo su qual è la radice (chiamata *core* o *rendezvous point*) e costruiscono l'albero inviando pacchetti da ogni membro alla radice. L'albero è l'unione dei cammini tracciati da questi pacchetti. La Figura 5.17(a) mostra il core-based tree del gruppo 1. Per spedire un messaggio a questo gruppo, una sorgente invia il pacchetto al core che lo inoltra lungo l'albero. Il processo è mostrato nella Figura 5.17(b) per la sorgente sul lato destro della rete. Per ottimizzare le performance i pacchetti destinati al gruppo non devono raggiungere il core prima della trasmissione multicast. Non appena un pacchetto raggiunge il core può essere inoltrato su fino alla radice o giù nei rami dell'albero. Questo è il caso della sorgente in cima alla Figura 5.17(b).

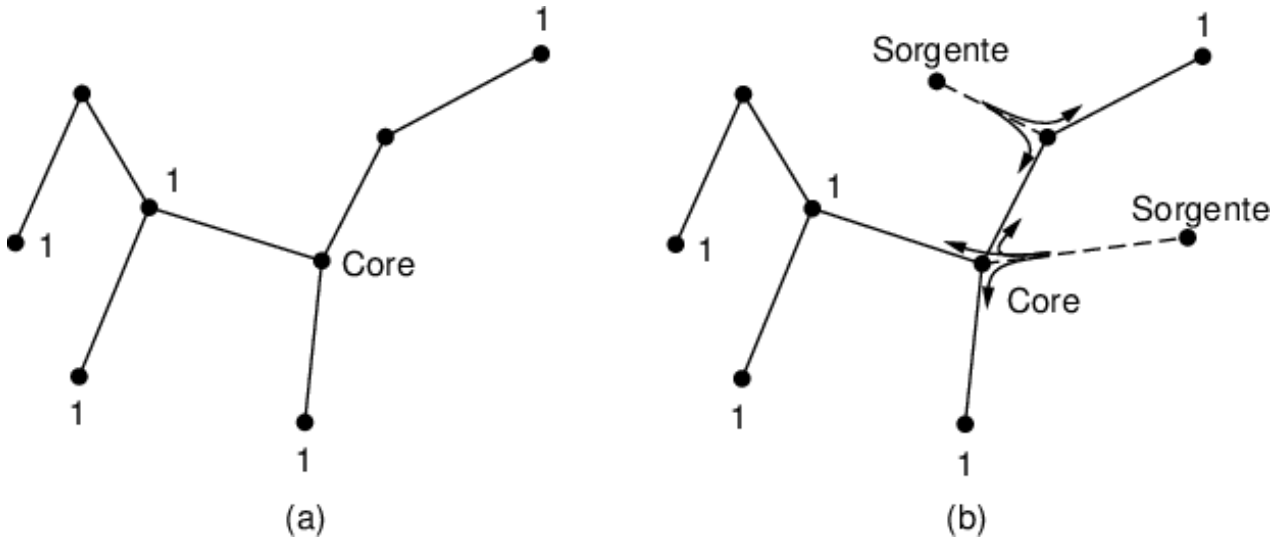


Figura 5.17 (a) Core-based tree per il gruppo 1. (b) Spedizione al gruppo 1.

Avere un albero condiviso non è ottimale per tutte le sorgenti. Per esempio nella Figura 5.17(b) il pacchetto inviato dalla sorgente sulla destra raggiunge i membri del gruppo in alto a destra in tre salti, invece che direttamente. Il grado di inefficienza del metodo dipende da dove sono ubicati la sorgente e il core, ma è ragionevole se il core si trova al centro. Quando si ha una sola sorgente, come un video mandato in streaming a un gruppo, è ideale usare la sorgente come core.

Una caratteristica rilevante dell'uso di un albero condiviso è il grande risparmio in termini di costi di memorizzazione, numero di messaggi inviati e costo di computazione. Ogni router deve tenere solo un albero per gruppo, invece di *m* alberi. Inoltre i router non appartenenti al gruppo non devono effettuare alcuna operazione. Per questa ragione gli approcci ad albero condiviso come il core-based tree sono usati per le trasmissioni multicast a gruppi sparsi su Internet in protocolli molto noti quali **PIM** (*protocol independent multicast*) (Fenner et al., 2006).

Routing anycast

Finora abbiamo trattato modelli di trasmissione nei quali una sorgente trasmette a una singola destinazione (**unicast**), a tutte le destinazioni (broadcast) o a un gruppo di destinazioni (multicast). A volte un altro modello, chiamato **anycast**, può risultare utile. Nell'anycast un pacchetto è consegnato al membro più vicino di un gruppo (Partridge et al., 1993). Gli schemi che trovano questi cammini sono chiamati **routing anycast**.

Perché dovremmo volere una trasmissione anycast? A volte i nodi forniscono un servizio, quale l'ora del giorno o la distribuzione di contenuti, per i quali l'aspetto più importante è ottenere la giusta informazione e non il nodo che viene contattato. Per esempio l'anycast è usato in Internet come parte del servizio DNS, come vedremo nel Capitolo 7.

Per fortuna non dovremo pensare a nuovi schemi di routing, perché gli usuali algoritmi basati sullo stato dei collegamenti o sul vettore delle distanze calcolano i percorsi anycast. Supponiamo di voler effettuare una trasmissione anycast ai membri del gruppo 1, aventi tutti indirizzo "1". L'algoritmo basato sul vettore delle distanze distribuirà i vettori nel modo usuale e i nodi sceglieranno i cammini più brevi per la destinazione 1. I percorsi sono mostrati nella Figura 5.18(a). Questa procedura funziona perché il protocollo di routing non si accorge che esistono più istanze della destinazione 1, quindi crede che tutte le istanze del nodo 1 siano lo stesso nodo, come nella topologia raffigurata nella Figura 5.18(b).

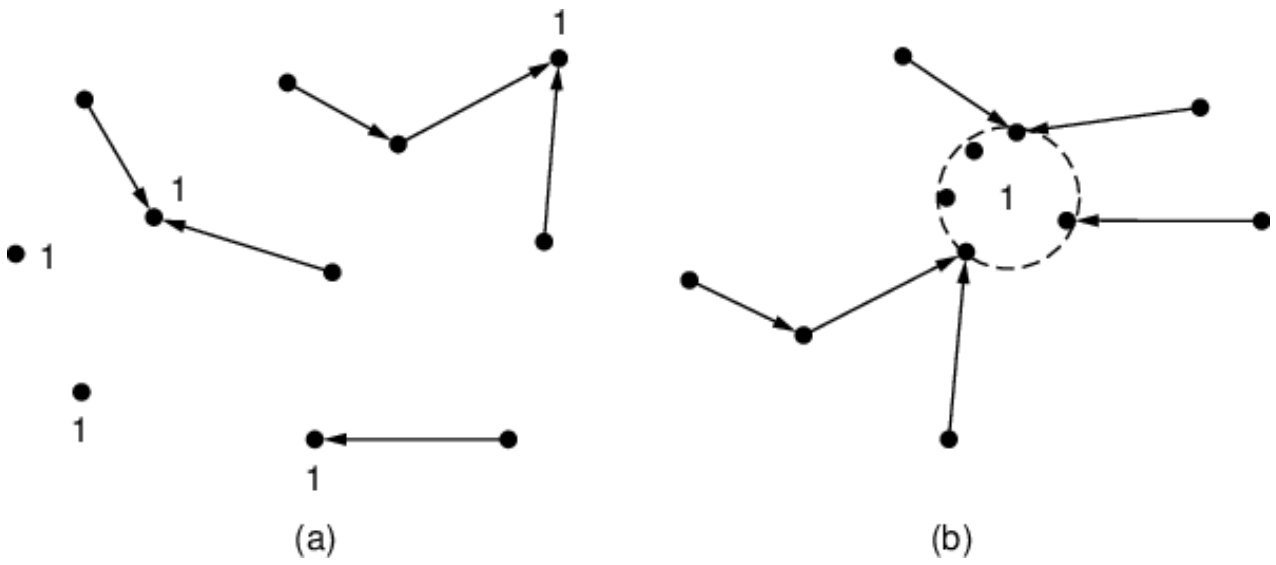


Figura 5.18 (a) Percorsi anycast verso il gruppo 1. (b) La topologia osservata dal protocollo di routing.

Questa procedura funziona anche per il link state routing, sebbene occorre considerare che il protocollo non deve trovare i cammini minimi che passino *attraverso* il nodo 1. Questo porterebbe a salti attraverso l'iperspazio perché le istanze del nodo 1 sono nodi ubicati nella realtà in differenti parti della rete. Tuttavia i protocolli link state già distinguono i router dagli host.

Routing per host mobili

Oggi milioni di persone utilizzano i computer durante gli spostamenti, sia in situazioni di mobilità completa, come il caso di dispositivi wireless su auto in movimento, sia parziale, come l'impiego di computer portatili in una serie di posti diversi. Useremo il termine **host mobile** (*mobile host*) per indicare entrambe le categorie, per distinguerli dagli host fissi che non si muovono mai. La gente richiede sempre più spesso di rimanere facilmente connessa come a casa ovunque si trovi. Questi host mobili introducono un nuovo problema: per inoltrare un pacchetto a un host mobile, la rete deve prima di tutto capire dov'è.

Nel modello che consideriamo tutti gli host hanno una **home location** (*LAN di appartenenza*) fissa che non cambia mai. Gli host hanno anche un indirizzo sulla LAN di appartenenza (*home address*) permanente, utilizzabile per determinare la loro home location, proprio come il numero di telefono 1-212-5551212 indica gli Stati Uniti (prefisso del paese 1) e Manhattan (prefisso 212). Il routing nei sistemi con host mobili ha come obiettivo l'invio dei pacchetti a host mobili usando i loro home address, consentendo ai pacchetti di raggiungere gli host mobili in modo efficiente, ovunque si trovino. La parte difficile consiste ovviamente nel trovarli.

È innanzitutto opportuno discutere questo modello. Si potrebbe utilizzare invece un modello in cui i percorsi vengono ricalcolati ogni volta che l'host mobile si muove e la topologia cambia, usando gli algoritmi di routing descritti nel paragrafo precedente. Tuttavia il numero sempre crescente di host mobili porterebbe presto l'intera rete a calcolare continuamente nuovi percorsi in un processo senza fine. L'uso di un home address riduce notevolmente questo onere.

Un'alternativa potrebbe essere quella di fornire mobilità sopra il livello di rete, come tipicamente avviene oggi con i laptop. Quando si muovono in una nuova postazione Internet, i laptop acquisiscono un nuovo indirizzo di rete. Non esiste alcuna correlazione tra l'indirizzo nuovo e quello vecchio: la rete non sa che appartengono allo stesso laptop. In questo modello un laptop può essere usato per navigare sul Web, ma non può ricevere pacchetti da altri host (per esempio una chiamata) senza costruire un servizio di rilevazione della posizione a più alto livello, come autenticarsi *di nuovo* su Skype dopo uno spostamento. Tuttavia la connessione non può essere mantenuta mentre l'host si muove; occorre iniziare una nuova connessione. La mobilità a livello di rete è utile per risolvere questi problemi.

L'idea di base utilizzata per il routing in mobilità su Internet e nelle reti cellulari è che l'host mobile indichi a un host nella LAN di appartenenza dove si trova. Questo host, che agisce a nome dell'host mobile, viene chiamato **home agent**. Una volta che l'home agent sa dove l'host mobile si trova al momento, può inoltrare i pacchetti in modo che gli siano consegnati.

La Figura 5.19 mostra come funziona il mobile routing. In questo caso il mittente, situato a Nord Ovest della città di Seattle, desidera inviare un pacchetto a un host che normalmente si trova negli Stati Uniti, nella città di New York. I pacchetti trasmessi verso la LAN dell'host mobile sono intercettati dall'home agent di New York; il caso interessante è quando l'host mobile non si trova nella home location, ma temporaneamente a San Diego.

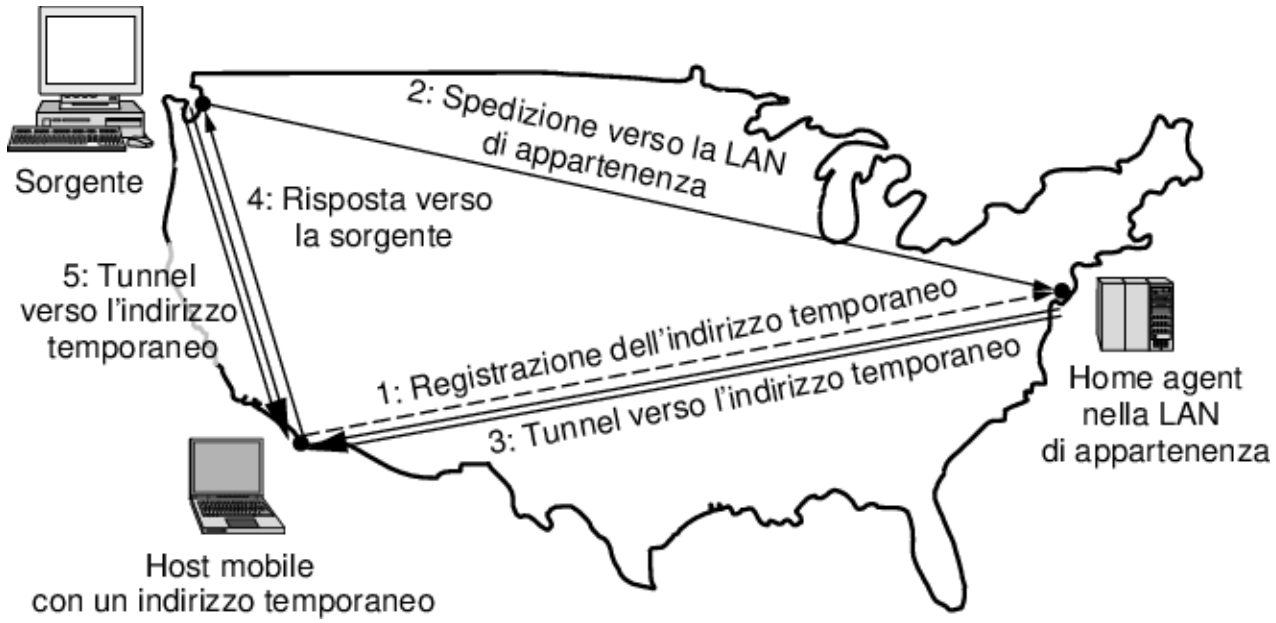


Figura 5.19 Routing di pacchetti per host mobili.

L'host mobile a San Diego deve acquisire un indirizzo della rete locale prima di poterla usare. La procedura di registrazione è quella usuale, ma ne parleremo più tardi per quanto riguarda Internet. L'indirizzo locale e temporaneo è definito come un **care of address**.[3] L'host mobile, una volta acquisito tale indirizzo, può dire all'home agent dove si trova, inviando un messaggio di registrazione (passo 1) contenente il suo indirizzo temporaneo. Nella Figura 5.19 il messaggio è mostrato con una linea tratteggiata per indicare che è un messaggio di controllo, non di dati. Quindi il mittente invia un pacchetto di dati all'host mobile usando il suo indirizzo permanente (passo 2). Il pacchetto viene inoltrato dalla rete alla home location dell'host mobile. A New York l'home agent intercetta il pacchetto perché sa che l'host mobile è via. Quindi incapsula il pacchetto con una nuova intestazione e lo invia all'indirizzo temporaneo (passo 3). Questo meccanismo viene chiamato **tunneling**, analizzato in dettaglio più avanti essendo molto importante in Internet.

Quando il pacchetto incapsulato arriva all'indirizzo temporaneo, l'host mobile recupera il pacchetto originale. L'host mobile invia quindi il suo pacchetto di risposta direttamente al mittente (passo 4). Il percorso complessivo è chiamato **routing triangolare** (*triangle routing*), in quanto il percorso potrebbe essere più lungo del percorso ottimale se la posizione attuale è lontana dalla home location. Come parte del passo 4 il mittente può imparare qual è l'attuale indirizzo temporaneo del destinatario. I pacchetti successivi possono essere indirizzati direttamente all'host mobile facendo tunneling (passo 5) e saltando completamente la home location. Se la connettività per qualsiasi motivo viene persa mentre l'host mobile è in movimento è sempre possibile usare l'home address.

Un aspetto importante omesso da questa descrizione è la sicurezza. In generale, quando un host o un router riceve un messaggio del tipo "Da questo momento si prega di inviare a me tutte le mail indirizzate a Stephany", potrebbero sorgergli dei dubbi. Nei messaggi vengono incluse

informazioni di sicurezza, in modo che la loro validità sia controllabile con i protocolli crittografici che studieremo nel Capitolo 8.

Ci sono molte varianti rispetto al mobile routing. Lo schema di cui sopra è modellato sulla mobilità di IPv6, la forma di mobilità utilizzata in Internet (Johnson et al., 2004), e come parte di reti cellulari basate su IP, come l'UMTS. Per semplicità abbiamo mostrato il mittente come nodo fermo, ma entrambi i nodi possono essere mobili. In alternativa l'host potrebbe essere parte di una rete mobile, come il caso di un computer su un aeroplano. Esistono estensioni di questo schema di base che supportano reti mobili in cui non viene richiesta agli host alcuna attività (Devarapalli et al., 2005).

Alcuni schemi fanno uso di un agente esterno (remoto), simile all'home agent, ma situato nella locazione remota o analogo al VLR (*visitor location register*) delle reti cellulari. Tuttavia, nei sistemi più recenti, l'agente esterno non è necessario; gli host mobili agiscono come propri agenti esterni. In ogni caso la conoscenza della locazione temporanea dell'host mobile è limitata a un numero ridotto di host (il dispositivo mobile, l'home agent e i mittenti), in modo che molti router in una rete di grandi dimensioni non debbano ricalcolare i percorsi.

Per ulteriori informazioni sul mobile routing si vedano Perkins (1998, 2002) e Snoeren e Balakrishnan (2000).

Routing nelle reti ad hoc

Nel paragrafo precedente abbiamo visto come funziona il routing quando gli host sono mobili, ma i router sono fissi. Un caso ancora più estremo è quello in cui i router stessi sono mobili. Tra i possibili scenari in cui ciò avviene troviamo squadre di soccorso che operano dopo un terremoto, veicoli militari su un campo di battaglia, una flotta di navi in mare o un raduno di persone dotate di computer portatili in un'area senza copertura 802.11.

In tutti questi casi (e anche in altri) ogni nodo comunica in modo wireless e agisce sia da router sia da host. Le reti costituite da nodi collocati l'uno vicino all'altro sono chiamate **reti ad hoc** o **MANET** (*mobile ad hoc network*) e sono brevemente esaminate in questo paragrafo; per ulteriori dettagli si può consultare (Perkins, 2001).

Ciò che rende le reti ad hoc diverse dalle reti cablate è che la topologia viene all'improvviso gettata fuori dalla finestra. I nodi possono andare e venire o apparire in nuovi luoghi in un battito di ciglia. Con una rete cablata, se un router ha un percorso valido diretto verso una particolare destinazione, il percorso continua a essere valido indefinitamente (a meno di guasti al sistema, fortunatamente rari). Con una rete ad hoc, la topologia può cambiare in qualunque momento, perciò il valore e anche la validità dei percorsi possono cambiare spontaneamente senza alcun preavviso. Inutile dirlo, queste circostanze rendono il routing nelle reti ad hoc una sfida molto maggiore del routing nelle controparti fisse.

Per le reti ad hoc sono stati proposti diversi algoritmi di routing. Tuttavia, poiché le reti ad hoc sono state poco utilizzate in pratica rispetto a quelle mobili, non è ben chiaro quali siano i protocolli migliori. Uno dei più interessanti è l'algoritmo di routing **AODV** (*ad hoc on-demand distance vector*), (Perkins e Royer, 1999). È un lontano parente dell'algoritmo basato sul vettore delle

distanze, adattato per funzionare in un ambiente mobile; l'algoritmo tiene conto della banda limitata e della scarsa durata delle batterie dei dispositivi che operano all'interno di questo ambiente. Vediamo ora come vengono individuati e gestiti i percorsi.

Identificazione dei percorsi

AODV è un algoritmo a richiesta (*on demand*), ossia determina un percorso diretto a una destinazione solo quando qualcuno tenta di inviare pacchetti verso quella destinazione. In questo modo si risparmia del lavoro che andrebbe sprecato nel caso in cui la topologia cambi prima che il cammino venga usato. In qualunque istante, una rete ad hoc può essere descritta da un grafo di nodi connessi. Due nodi sono collegati (ossia hanno un arco che li unisce) se possono comunicare direttamente usando la loro radio. Un modello semplice ma adeguato ai nostri scopi prevede che ogni nodo possa comunicare con tutti quelli compresi nel suo raggio di copertura. Le reti reali sono più complesse, perché ci possono essere edifici, colline o altri ostacoli che bloccano la comunicazione e perché è possibile che *A* sia collegato a *B* ma che *B* non sia collegato ad *A*, avendo *A* un trasmittente più potente di *B*. Assumeremo, tuttavia, per semplicità, che tutte le connessioni siano simmetriche.

Per descrivere questo algoritmo si consideri la rete ad hoc mostrata nella Figura 5.20. Si supponga che un processo del nodo *A* cerchi di inviare un pacchetto al nodo *I*. L'algoritmo AODV mantiene in ogni nodo la tabella dei vettori delle distanze, indicizzata per destinazione, che fornisce informazioni su quella destinazione; tra esse c'è l'indicazione del vicino a cui trasmettere i pacchetti per raggiungerla. Si supponga che *A* cerchi nella propria tabella e non trovi una voce per *I*, pertanto debba scoprire un percorso diretto a *I*. La proprietà di scoprire i percorsi solo quando si ha la necessità di raggiungerli rende l'algoritmo "a richiesta".

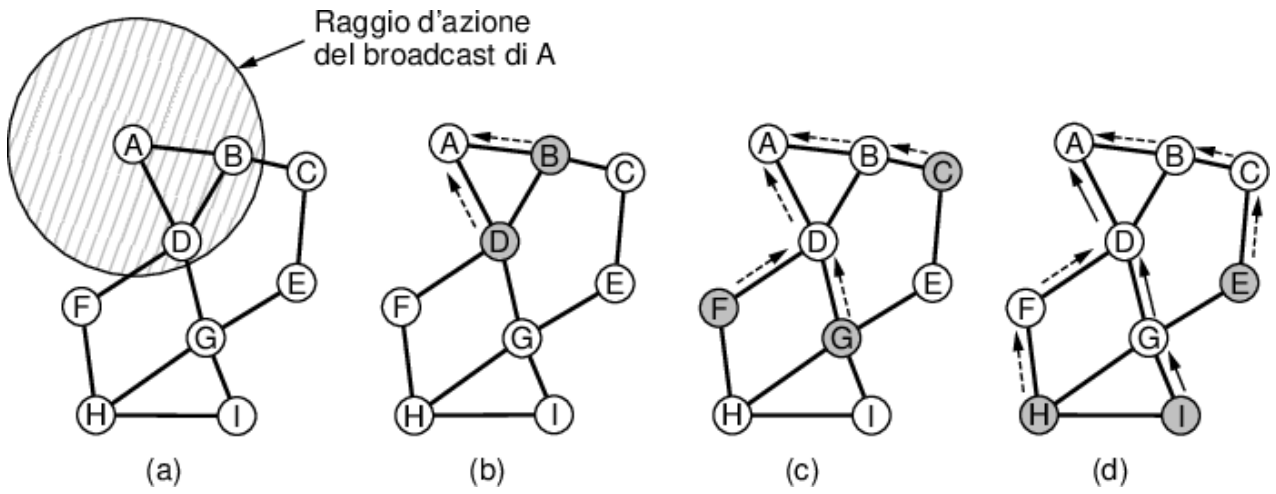


Figura 5.20 (a) Raggio d'azione del broadcast di *A*. (b) Dopo la ricezione da parte di *B* e *D*. (c) Dopo la ricezione da parte di *C*, *F* e *G*. (d) Dopo la ricezione da parte di *E*, *H*, e *I*. I nodi ombreggiati rappresentano nuovi destinatari. Le frecce tratteggiate indicano nuovi possibili percorsi inversi. Le frecce a tratto continuo indicano il percorso trovato.

Per individuare *I*, *A* costruisce un pacchetto ROUTE REQUEST e lo trasmette in modalità broadcast, usando il flooding come descritto nel Paragrafo 5.2.3. Il pacchetto raggiunge *B* e *D*, come mostrato nella Figura 5.20(a). Ogni nodo esegue nuovamente il broadcast della richiesta che continua a raggiungere i nodi *F*, *G* e *C* nella Figura 5.20(c) e i nodi *H*, *E* e *I* nella Figura 5.20(d). Un insieme di numeri di sequenza alla sorgente è usato per scartare i duplicati del flooding. Per esempio *D* scarta la trasmissione da *B* nella Figura 5.20(c) perché ha già inoltrato la richiesta.

Infine la richiesta raggiunge il nodo *I* che costruisce un pacchetto ROUTE REPLY. Questo pacchetto è trasmesso in modalità unicast al nodo di origine del pacchetto, seguendo il percorso inverso. Per far ciò ogni nodo intermedio deve memorizzare da quale nodo ha ricevuto la richiesta. Le frecce nelle Figure 5.20(b–d) mostrano le informazioni sul percorso inverso che vengono memorizzate. Ogni nodo intermedio inoltre incrementa un contatore del numero di salti ogni volta che inoltra una risposta. Questo indica ai nodi quanto siano distanti dalla destinazione. Le risposte dicono a ogni nodo intermedio quale vicino usare per raggiungere la destinazione: è quello da cui hanno ricevuto la risposta. I nodi intermedi *G* e *D* memorizzano il percorso migliore che hanno ricevuto nella loro tabella di routing quando processano la risposta. Quando la risposta raggiunge *A*, un nuovo percorso, *ADGI*, è stato creato.

In una rete di grandi dimensioni l'algoritmo genera molte trasmissioni broadcast, anche per destinazioni che sono vicine. Per ridurre si usa il campo *time to live* (*tempo di vita*) del pacchetto IP, inizializzato dalla sorgente e decrementato a ogni salto. Non appena il contatore raggiunge lo 0, il pacchetto invece di essere trasmesso in modalità broadcast viene scartato. Il processo di scoperta è quindi modificato: per individuare la destinazione, il trasmittente invia un pacchetto ROUTE REQUEST broadcast contenente il campo *time to live* impostato a 1. Se non arriva alcuna risposta in un tempo ragionevole, il trasmittente invia un altro pacchetto, questa volta con il campo *time to live* impostato a 2. I tentativi successivi usano i valori 3, 4, 5 e così via. In tal modo, la ricerca viene eseguita prima localmente e poi in cerchi sempre più ampi.

Aggiornamento di un percorso

Poiché i nodi si possono spostare o spegnere, la topologia può cambiare spontaneamente. Per esempio, se nella Figura 5.20*G* si spegne, *A* non si accorge che il percorso che sta utilizzando per raggiungere *I* (*ADGI*) non è più valido. L'algoritmo deve essere in grado di gestire anche queste situazioni. Periodicamente ogni nodo trasmette in modalità broadcast un messaggio HELLO; ognuno dei vicini deve rispondere a tale messaggio. Se un vicino non trasmette la risposta, il trasmittente sa che quel dispositivo si è spostato al di fuori del suo raggio di copertura e non è più connesso a lui. In modo analogo, se tenta di inviare un pacchetto a un vicino che non risponde, il trasmittente scopre che il destinatario non è più disponibile.

Questa informazione è utilizzata per eliminare i percorsi che non sono più validi. Per ogni possibile destinazione, ogni generico nodo *N* tiene traccia dei vicini che gli hanno inviato un pacchetto per quella destinazione durante gli ultimi ΔT secondi.

Quando uno dei vicini di *N* diventa irraggiungibile, il nodo controlla la propria tabella di routing per vedere quali destinazioni sono associate ai percorsi che usano il vicino appena scomparso. Per ognuno di questi percorsi i vicini attivi sono informati che il loro percorso attraverso *N* non è più valido e che deve essere eliminato dalle tabelle di routing. Nel nostro esempio, *D* scarta le voci per

G e *I* dalla propria tabella di routing e avverte *A*, che elimina la sua voce per *I*. Nel caso generale, i vicini attivi avvisano i loro vicini attivi, e così via, ricorsivamente, fino a quando tutti i percorsi dipendenti dal nodo scomparso vengono eliminati da tutte le tabelle di routing.

A questo punto, i percorsi non validi sono stati eliminati dalla rete e le sorgenti possono trovare nuovi percorsi validi, utilizzando il meccanismo di scoperta descritto prima. Tuttavia vi è una complicazione. Ricordiamo che i protocolli basati sul vettore delle distanze possono soffrire di convergenza lenta o problemi di conteggio all'infinito dopo un cambiamento di topologia, perché confondono vecchi percorsi non validi con nuovi percorsi validi. Per garantire una rapida convergenza, i percorsi includono un numero di sequenza controllato dalla destinazione. Il numero di sequenza di destinazione è come un orologio logico (logical clock);[4] la destinazione lo incrementa ogni volta che invia un nuovo pacchetto ROUTE REPLY. Le sorgenti richiedono un percorso aggiornato includendo nella ROUTE REQUEST il numero di sequenza della destinazione dell'ultimo percorso usato, che potrà essere il numero di sequenza del percorso che è stato appena eliminato o il valore iniziale 0.
La richiesta sarà trasmessa in broadcast fino a un percorso con un numero di sequenza più alto. I nodi intermedi memorizzano i percorsi che hanno un numero di sequenza più alto, o il minor numero di salti per il numero di sequenza corrente. Nello spirito di un protocollo su richiesta, i nodi intermedi memorizzano solo i percorsi che sono in uso. Altre informazioni apprese nel corso delle trasmissioni broadcast vengono scartate dopo un breve ritardo. Scoprire e conservare solo i percorsi che vengono utilizzati migliora il risparmio di banda e durata della batteria rispetto a un usuale protocollo basato sul vettore delle distanze che aggiorna periodicamente trasmissioni. Finora abbiamo considerato solo un unico percorso, da *A* a *I*. Per risparmiare ulteriormente risorse, scoperta e manutenzione dei percorsi sono condivise quando questi si sovrappongono. Per esempio, se anche *B* vuole inviare i pacchetti a *I*, effettuerà il procedimento di scoperta del percorso. Tuttavia, in questo caso, la richiesta in primo luogo raggiunge *D*, che ha già un cammino verso *I*. Il nodo *D* può quindi generare una risposta per indicare il cammino a *B*, senza alcuna operazione supplementare.

Ci sono molti altri schemi di routing ad hoc. Un altro ben noto schema a richiesta è DSR (*dynamic source routing)* (Johnson et al., 2001). Una differente strategia basata sulla geografia è esplorata da GPSR (*greedy perimetral stateless routing*) (Karp e Kung, 2000). Se tutti i nodi conoscono la loro posizione geografica, l'inoltro a una destinazione può procedere senza calcolo del percorso semplicemente andando nella direzione giusta e tornando indietro di nuovo per evitare i vicoli ciechi. Quali protocolli si riveleranno vincenti dipenderà dal tipo di reti ad hoc che si dimostreranno utili nella pratica.

5.3 Algoritmi per il controllo della congestione

Quando in (una parte) della rete sono presenti troppi pacchetti, si verificano ritardi e perdite di pacchetti che causano un degrado delle prestazioni. Questa situazione è chiamata **congestione**. I livelli di rete e trasporto condividono la responsabilità di gestire la congestione. Dal momento che la congestione si verifica all'interno della rete, è il livello di rete che la sperimenta direttamente e che deve infine determinare come agire sui pacchetti in eccesso. Tuttavia il modo più efficace di effettuare un controllo della congestione è quello di ridurre il carico che il livello di trasporto sta immettendo nella rete. Ciò richiede che i livelli di rete e di trasporto lavorino insieme. In questo capitolo prenderemo in considerazione gli aspetti della congestione legati al livello di rete, mentre nel Capitolo 6 completeremo l'argomento, coprendone gli aspetti riguardanti il livello di trasporto.

La Figura 5.21 descrive il verificarsi di una congestione di rete. Quando il numero di pacchetti inviati in rete dagli host è al di sotto della sua capacità di carico, il numero di pacchetti consegnati è proporzionale al numero di inviati; se il doppio dei pacchetti viene inviato, ne verrà consegnato il doppio. Tuttavia, quando il carico immesso si avvicina alla capacità di carico, picchi di traffico riempiono occasionalmente i buffer dei router e alcuni pacchetti vanno persi. Questi pacchetti persi consumano parte della capacità di rete portando il numero di pacchetti consegnati al di sotto della curva ideale. Ora la rete è congestionata.

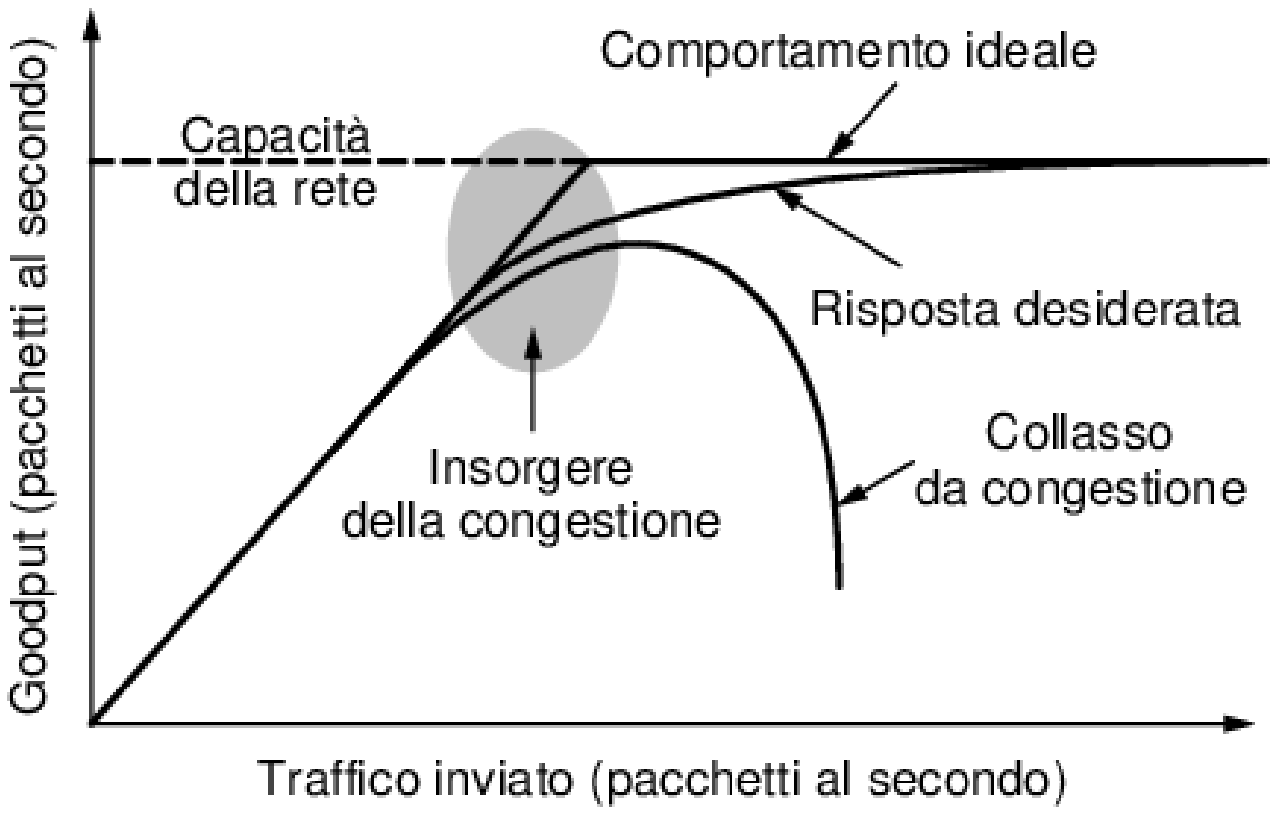


Figura 5.21 Con troppo traffico le prestazioni decadono rapidamente.

Se la rete non è ben progettata è possibile che si verifichi un **collasso da congestione** (*congestion collapse*), durante il quale le prestazioni crollano all'aumentare del carico immesso oltre le capacità di rete. Questo può accadere perché i pacchetti subiscono un ritardo tale in rete da renderli inutili quando la lasciano. Ad esempio, agli esordi di Internet, il tempo speso da un pacchetto nell'attesa che l'accumulo di dati che lo precedeva fosse inviato su una linea a 56 kbps poteva raggiungere il limite massimo consentitogli per rimanere in rete. In questo caso il pacchetto doveva essere gettato via. Un problema di diverso tipo si verifica quando le sorgenti ritrasmettono i pacchetti fortemente in ritardo, dandoli per persi; le copie dello stesso pacchetto verranno quindi consegnate dalla rete, sprecando anche in questo caso della capacità di trasferimento. Per catturare tali fattori, l'asse y nella Figura 5.21 è dato in termini di **goodput**, e cioè il tasso al quale i pacchetti utili sono consegnati dalla rete.

Vorremmo progettare reti possibilmente in grado di evitare le congestioni e il collasso se la congestione si verifica. Sfortunatamente le congestioni sono inevitabili. Se improvvisamente diversi flussi di pacchetti cominciano ad arrivare attraverso tre o quattro linee di input e tutti i pacchetti necessitano della stessa linea di output, inizia a formarsi una coda. Se la memoria non è sufficiente, alcuni pacchetti andranno persi. Aggiungere nuova memoria può aiutare fino a un certo punto, ma Nagle (1987) ha scoperto che, se i router avessero memoria infinita, la congestione peggiorerebbe invece di migliorare, perché quando i pacchetti usciranno della coda saranno già (ripetutamente) scaduti e quindi saranno già stati trasmessi dei duplicati. Tutto ciò peggiora la situazione invece di migliorarla, portando la rete al collasso.

Collegamenti con bassa capacità di banda o router che elaborano i pacchetti più lentamente del tasso di trasmissione della linea possono congestionarsi. In questo caso la situazione può essere migliorata allontanando parte del traffico dal punto di congestione. Alla fine, tuttavia, tutta la rete sarà congestionata. A questo punto non vi è alternativa, a meno di diminuire il carico o costruire una rete più veloce.

Vale la pena richiamare l'attenzione sulla differenza tra controllo della congestione e controllo di flusso, in quanto la distinzione è sottile. Controllare la congestione significa garantire che la rete sia in grado di trasportare il traffico immesso: è un problema globale, che coinvolge il comportamento di tutti gli host e di tutti i router. Il controllo del flusso, invece, ha a che fare con il traffico tra un singolo trasmittente e un dato ricevente fissato; il suo compito è evitare che una sorgente veloce trasmetta continuamente una quantità di dati maggiore di quella che il ricevente è in grado di assorbire.

Per comprendere la differenza tra i due concetti, si consideri una rete in fibra ottica con una capacità di 100 Gbps attraverso la quale un supercomputer tenta di trasferire un file a un personal computer con una capacità di 1 Gbps. Sebbene non ci sia congestione (la rete in sé non ha problemi) è necessario utilizzare il controllo di flusso per costringere il supercomputer a interrompere spesso la trasmissione, in modo da dare al PC il tempo di respirare.

All'altro estremo, si consideri una rete composta da linee a 1 Mbps e da 1000 grandi computer, metà dei quali tenta di trasferire file a 100 kbps all'altra metà. In questo caso il problema non è causato da trasmettitori veloci che opprimono ricevitori lenti, ma dal traffico totale immesso, che

supera quello che la rete è in grado di gestire.

Il motivo per cui spesso il controllo della congestione viene confuso con il controllo di flusso è che il modo migliore per gestire entrambi i problemi è di rallentare gli host. Per questo motivo un host può ricevere un messaggio "rallenta" sia perché il ricevente non è in grado di ricevere il carico a quella velocità, sia perché la rete non è in grado di gestirlo. Questo punto verrà discusso nel Capitolo 6.

Lo studio del controllo della congestione inizierà descrivendo gli approcci praticabili su varie scale temporali, per poi esaminare gli approcci per prevenire la congestione e infine descrivere come affrontare il problema quando essa si presenta.

Approcci al controllo della congestione

La presenza di una congestione indica che il carico è (temporaneamente) maggiore di quello che può essere gestito dalle risorse (in una parte del sistema). Vengono in mente due soluzioni: aumentare le risorse o diminuire il carico. Come mostrato nella Figura 5.22, queste soluzioni sono solitamente applicate su diverse scale temporali, sia per prevenire congestioni sia per reagire quando se ne verifica una.

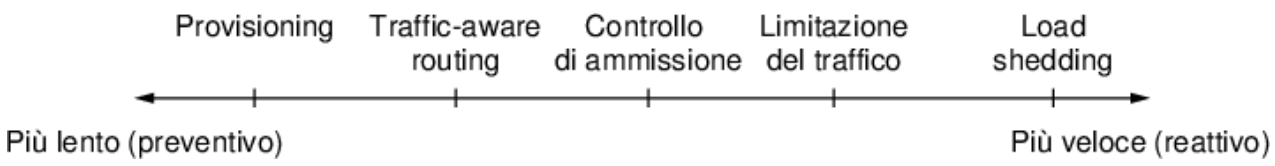


Figura 5.22 Approcci per il controllo della congestione rispetto al tempo di reazione.

Il modo più semplice per evitare una congestione è quello di costruire una rete che ben si adatti al traffico sopportato. Se vi è un collegamento con una larghezza di banda ridotta lungo il percorso dove la maggior parte del traffico è diretto, è probabile che si verificherà una congestione. A volte, quando vi è una grave congestione, le risorse possono essere aggiunte dinamicamente, ad esempio deviando il traffico su router di riserva, attivando linee normalmente utilizzate solo come backup (per rendere la rete tollerante agli errori) o acquistando banda. Capita spesso che gli amministratori di rete aggiornino, alla prima occasione possibile, router e collegamenti molto utilizzati. Questo metodo è chiamato **provisioning**, accade su una scala temporale di mesi ed è guidato dalle tendenze di traffico a lungo termine.

Per sfruttare al massimo la capacità della rete esistente, i percorsi possono essere personalizzati in modo da tenere conto dei cambiamenti che avvengono durante la giornata, dato che gli utenti della rete sono svegli e dormono a ore diverse a causa dei diversi fusi orari. Per esempio i cammini sono modificabili per alleggerire il traffico da quelli pesantemente utilizzati modificando i pesi usati dall'algoritmo di cammino minimo. Alcune stazioni radio locali dispongono di elicotteri che

sorvolano la città per monitorare l'evoluzione degli ingorghi stradali, permettendo agli ascoltatori mobili di indirizzare i propri pacchetti (cioè le automobili) aggirando i punti caldi della rete. Questo metodo è chiamato **traffic-aware routing** (*routing con conoscenza del traffico*).

Tuttavia a volte non è possibile aumentare la capacità e l'unico modo per eliminare una congestione è quello di diminuire il carico. In una rete a circuito virtuale si può rifiutare una connessione se la sua attivazione causi la congestione della rete. Questa procedura è chiamata **controllo di ammissione** (*admission control*).

A un livello di granularità più fine, quando la congestione è imminente, la rete può offrire un feedback alle sorgenti i cui flussi di traffico sono responsabili del problema, richiedendo l'interruzione delle trasmissioni o il rallentamento del traffico.

Le due difficoltà riscontrate in questo approccio sono l'identificazione dell'insorgenza di congestioni e l'informazione alla sorgente della necessità di rallentare. Per contrastare il primo problema, i router possono monitorare il carico medio, il ritardo di accodamento o la perdita di pacchetti. In tutti i casi tali numeri, se crescenti, indicano un aumento di congestione.

Per contrastare il secondo problema, i router devono mandare un feedback alle sorgenti. Perché un sistema funzioni correttamente, la scala temporale deve essere condizionata adeguatamente. Se ogni volta che due pacchetti arrivano consecutivamente un router dice STOP e ogni volta che un router è inattivo per 20 μs dice GO il sistema oscillerà come un pazzo e non convergerà mai. D'altra parte se per sicurezza esso attende 30 minuti prima di esprimersi, il meccanismo di controllo della congestione avrà un tempo di reazione troppo lungo per essere utile. Fornire un feedback tempestivo non è un problema banale. A tutto ciò si aggiunge il fatto che in questo modo anche i router inviano più messaggi quando la rete è già congestionata. Infine, quando tutto il resto fallisce, la rete è costretta a scartare i pacchetti che non può consegnare. Il termine generico per questa operazione è **load shedding (***eliminazione del carico***)**. Una buona politica per scegliere quali pacchetti scartare può aiutare a prevenire il collasso della rete.

Traffic-aware routing

Il primo metodo che trattiamo è l'algoritmo di traffic-aware. Gli schemi di routing visti nel Paragrafo 5.2 usano pesi fissi per i collegamenti. Questi schemi si adattano a cambiamenti di topologia, ma non di carico. L'obiettivo nel tenere conto del carico quando si calcolano i percorsi è di dirottare il traffico lontano dai punti nevralgici che saranno i primi a congestionarsi.

Il modo più diretto per farlo è fare in modo che i pesi siano una funzione della banda e del ritardo di propagazione (fissi), più il ritardo misurato o il ritardo medio di accodamento (variabili). I percorsi con peso minore favoriranno quindi i cammini con meno carico, essendo tutto il resto uguale.

Il traffic-aware routing è stato usato in Internet all'inizio, secondo il modello descritto in (Khanna e Zinky, 1989). Tuttavia vi è un rischio: si consideri la rete della Figura 5.23 che è divisa in due parti, Est e Ovest, collegate da due linee, *CF* ed *EI*. Supponiamo che la maggior parte del traffico tra Est e Ovest usi *CF* portando questo collegamento a essere fortemente utilizzato e con lunghi ritardi. Includendo il ritardo di accodamento nel peso utilizzato per il calcolo del percorso minimo, *EI* risulterà migliore. Dopo che le nuove tabelle di routing saranno state memorizzate, la maggior

parte del traffico Est-Ovest percorrerà *EI*, caricando questa linea. Nell'aggiornamento successivo, *CF* risulterà essere il percorso più breve. Di conseguenza le tabelle di routing potrebbero oscillare molto, causando un instradamento irregolare e altri potenziali problemi.

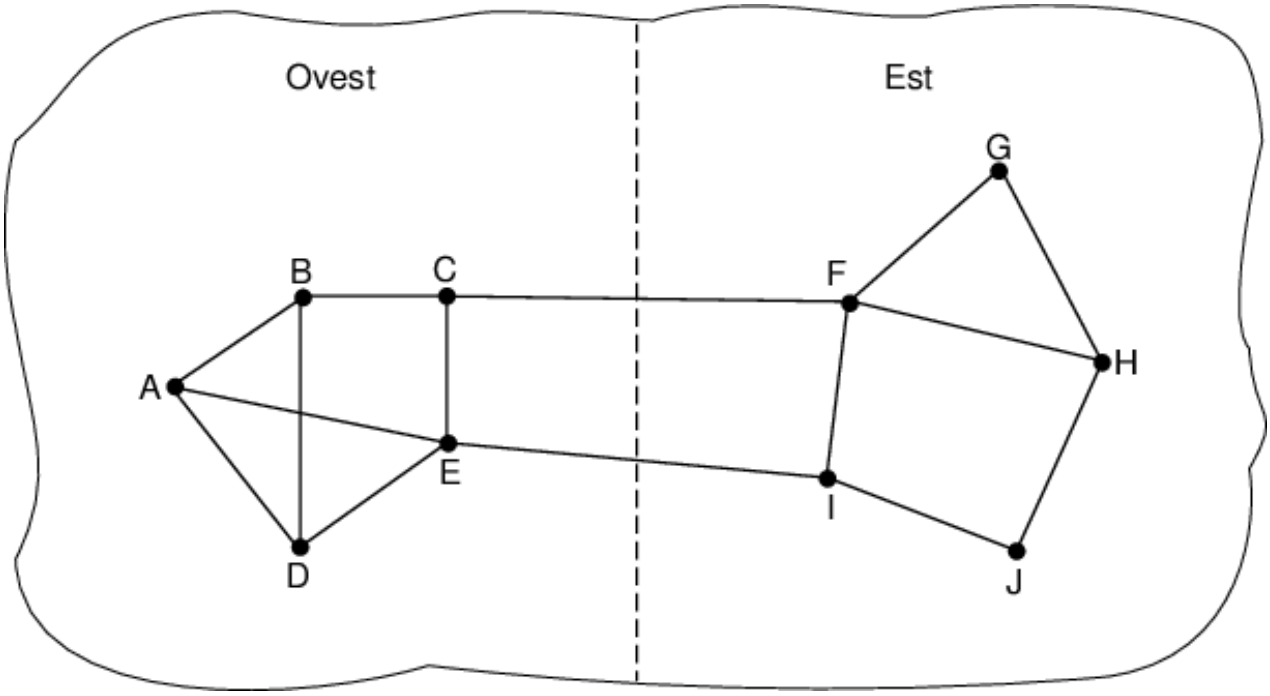


Figura 5.23 Una rete in cui le parti Est e Ovest sono collegate tramite due connessioni.

Se non si considera il carico, ma solo la larghezza di banda e il ritardo di propagazione, questo problema non si verifica. I tentativi di comprendere il carico cambiando poco i pesi porta solo a rallentare le oscillazioni del routing. Due tecniche possono contribuire a una soluzione funzionante. La prima è il multipath routing, in cui vi possono essere più percorsi da una sorgente a una destinazione. Nel nostro esempio questo significa che il traffico si distribuisce su entrambi i collegamenti da Est a Ovest. La seconda è spostare il traffico sui percorsi abbastanza lentamente da riuscire a convergere, come nel metodo di Gallagher (1977).

Tenuto conto di tali difficoltà, in Internet i protocolli di routing in genere non sono in funzione del carico. Invece gli adattamenti sono effettuati al di fuori del protocollo di routing cambiando lentamente il suo input. Questa metodologia è detta **ingegneria del traffico** (*traffic engineering*).

Controllo di ammissione

Una tecnica largamente utilizzata per evitare una congestione si chiama **controllo di ammissione**. L'idea è semplice: nessun circuito virtuale viene impostato se la rete non è in grado di supportare il carico che questo aggiungerebbe. Quindi ciascun tentativo di impostare una nuova connessione può fallire. Meglio questo che lasciare che le persone si colleghino peggiorando la situazione.

Benché sia rozzo, tale approccio è semplice e facile da implementare. Per analogia nei sistemi telefonici sono i commutatori che, quando sovraccaricati, praticano il controllo di ammissione non dando il segnale di libero.

Il problema di questo approccio è capire quando un nuovo circuito virtuale porta a una congestione. Tale compito è facile nelle reti telefoniche dove la banda è fissa (64 kbps per audio non compresso), ma i circuiti virtuali nelle reti di calcolatori variano per tipo e dimensione. Quindi, per applicare il controllo di ammissione, i circuiti devono conoscere il loro traffico.

Il traffico viene spesso descritto in termini di tasso di trasmissione e forma. Il problema di come esprimere il traffico in modo semplice ma significativo è difficile, perché il traffico è tipicamente a picchi per cui il tasso medio è solo una metà della storia. Ad esempio, il traffico che varia durante la navigazione sul Web è più difficile da gestire di un film in streaming con lo stesso throughput a lungo termine, perché i picchi di traffico sul Web hanno più probabilità di congestionare i router della rete. Due modelli comuni che catturano questo effetto sono il **leaky bucket** e il **token bucket**. Il leaky bucket prevede due parametri che limitano il tasso medio del traffico passante e la dimensione istantanea dei picchi di traffico. Dal momento che è molto usato per la sua qualità del servizio lo analizzeremo in dettaglio nel Paragrafo 5.4.

Avendo a disposizione le descrizioni del traffico, la rete può decidere se ammettere il nuovo circuito virtuale. Una possibilità è che la rete riservi, lungo i cammini di ciascuno dei propri circuiti virtuali, capacità sufficiente a non far verificare una congestione. In questo caso la descrizione del traffico è un contratto di servizio con cui la rete fornisce garanzie ai suoi utenti. Abbiamo impedito la congestione, ma introdotto l'argomento della qualità del servizio un po' troppo presto; ci torneremo sopra nel prossimo paragrafo.

Anche senza fornire garanzie, per effettuare il controllo di ammissione la rete può utilizzare le descrizioni del traffico. Il compito è ora quello di stimare quanti circuiti rientrino nella capacità di carico della rete senza provocare una congestione. Supponiamo che i circuiti virtuali che possono supportare tassi di trasmissione fino a 10 Mbps passino tutti attraverso la stessa linea fisica a 100 Mbps. Quanti circuiti potrebbero essere ammessi? Chiaramente possono essere ammessi 10 circuiti senza rischiare una congestione, ma sarebbe uno spreco nel caso normale, perché raramente accade che tutti e 10 trasmettano a pieno ritmo nello stesso istante. Nelle reti reali le misurazioni del comportamento passato che catturano le statistiche di trasmissioni possono essere utilizzate per stimare il numero ammissibile di circuiti, e raggiungere un compromesso tra prestazioni e rischi accettabili.

Il controllo di ammissione può anche essere combinato con il traffic-aware routing considerando nella procedura di setup percorsi che girino attorno ai punti con traffico più intenso. Per esempio si consideri la rete mostrata nella Figura 5.24(a) in cui due router sono congestionati come indicato.

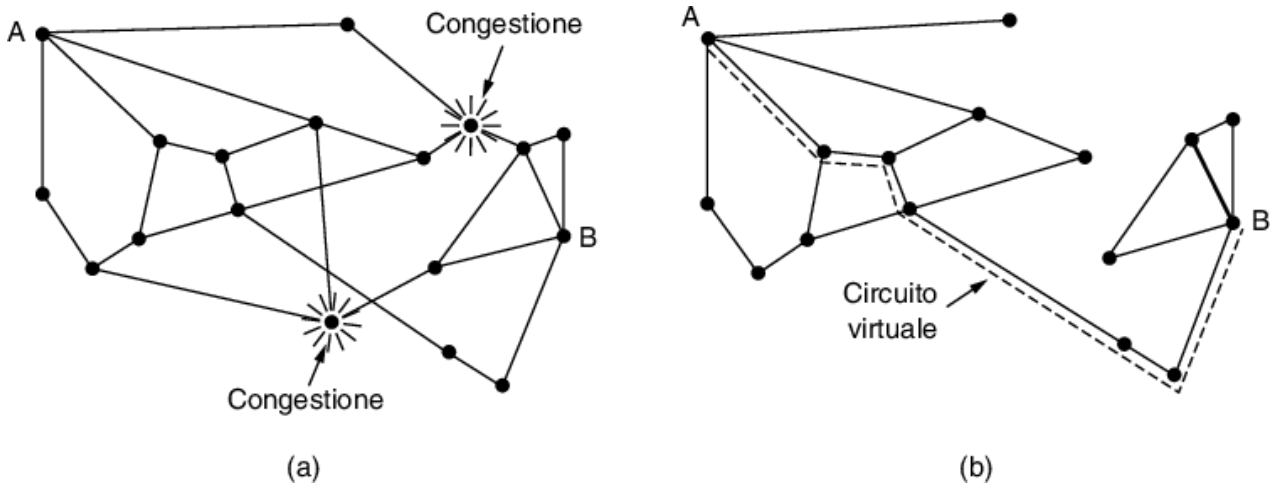


Figura 5.24 (a) Una rete congestionata. (b) La porzione di rete che non è congestionata. Viene inoltre mostrato un circuito virtuale che collega *A* a *B*.

Si supponga che un host collegato al router *A* voglia impostare una connessione con un host collegato al router *B*. Normalmente questa connessione passerebbe attraverso uno dei due router congestionati. Per evitare questa situazione si può ridisegnare la rete come mostrato nella Figura 5.24(b), omettendo i router congestionati e tutte le loro linee. La linea tratteggiata rappresenta un possibile percorso del circuito virtuale che eviti i router congestionati. Shaikh et al. (1999) propongono un'architettura per questo tipo di routing sensibile al carico.

Limitazione del traffico

In Internet e in molte altre reti di calcolatori i mittenti regolano le loro trasmissioni in modo da inviare quanto più traffico la rete può facilmente trasportare. In questa situazione la rete si propone di operare fino a poco prima dell'inizio della congestione; quando la congestione è imminente, essa avverte i mittenti di ridurre (*strozzare*) le loro trasmissioni e rallentare. Tale feedback è più la norma che l'eccezione. Il termine **prevenzione della congestione** (*congestion avoidance*) è talvolta usato per contrapporre questa condizione di lavoro a quella in cui la rete è diventata (troppo) congestionata.

Vediamo ora alcuni approcci alla limitazione del traffico (*traffic throttling*) utilizzati sia nelle reti datagram sia nelle reti a circuito virtuale. Ogni approccio deve risolvere due problemi. In primo luogo, i router devono determinare quando la congestione si avvicina, idealmente prima che sopraggiunga. A questo scopo ogni router può monitorare costantemente le risorse utilizzate. Le tre possibili grandezze da controllare sono: l'utilizzo dei collegamenti in uscita, lo stato del buffer dei pacchetti in coda all'interno del router e il numero di pacchetti persi a causa di buffer insufficiente; tra queste possibilità, la più utile è la seconda. Le medie di utilizzo non tengono direttamente in conto la maggior parte del traffico; un utilizzo del 50% può essere basso per un traffico senza picchi, ma troppo alto per traffico molto variabile. Il conteggio dei pacchetti persi è una metrica tardiva perché la perdita si ha quando la congestione è già iniziata.

Il ritardo di accodamento all'interno dei router cattura direttamente ogni congestione sperimentata dai pacchetti. Dovrebbe essere basso per la maggior parte del tempo, ma aumenta improvvisamente quando c'è un picco di traffico che genera un accumulo di pacchetti. Per mantenere una buona stima del ritardo di accodamento *d*, periodicamente si potrebbe prendere un campione della lunghezza istantanea della coda, *s*, aggiornando *d* secondo questa funzione:

$$d_{nuovo} = ad_{vecchio} + (1 - a)s$$

dove la costante α determina la velocità con cui il router dimentica la statistica recente. Questo metodo si chiama **EWMA** (*exponentially weighted moving average*, media mobile esponenziale ponderata); esso smorza le oscillazioni e si comporta come un filtro passa-basso. Ogni volta che *d* si sposta sopra la soglia di guardia, il router si accorge dell'insorgere di una congestione.

Il secondo problema è che i router devono fornire un feedback tempestivo alle sorgenti che stanno causando la congestione. È la rete a subire la congestione, ma per alleviarla è richiesto l'intervento dei mittenti che la stanno usando. Per fornire feedback, il router deve identificare i mittenti giusti e avvertirli facendo attenzione a non inviare troppi pacchetti sulla rete già congestionata. Come vedremo ora, schemi distinti utilizzano diversi meccanismi di feedback.

Choke packet

Il modo più diretto per notificare una congestione a una sorgente è comunicarglielo direttamente. In questo approccio il router seleziona un pacchetto congestionato e invia all'host sorgente un **choke packet** (letteralmente, *pacchetto di strozzamento*) assegnandogli la destinazione trovata nel pacchetto congestionato. Il pacchetto originale viene etichettato (un bit dell'intestazione è impostato a 1) in modo da impedire la generazione di altri choke packet lungo il percorso e poi è inoltrato nel solito modo. Per evitare di accrescere ulteriormente il carico durante la congestione, il router dovrebbe inviare i choke packet con un tasso molto basso.

Quando riceve il choke packet, l'host sorgente deve ridurre il traffico inviato alla destinazione specificata, per esempio del 50%. In una rete datagram raccogliere semplicemente a caso i pacchetti quando c'è una congestione può provocare l'invio di choke packet a mittenti veloci, perché questi avranno più pacchetti nella coda. Il feedback implicito in questo protocollo può aiutare a prevenire la congestione, ma non ferma alcuna sorgente, a meno che non provochi problemi. Per lo stesso motivo è facile che più choke packet vengano inviati alla stessa sorgente o destinazione. L'host dovrebbe ignorare questi choke packet aggiuntivi per l'intervallo di tempo necessario affinché la sua riduzione di traffico abbia effetto. Dopo tale termine ulteriori choke packet indicano che la rete è ancora congestionata.

Un esempio di choke packet impiegato ai primi esordi di Internet è il messaggio SOURCE-QUENCH (Postel, 1981). Non ha mai preso piede, parzialmente perché le circostanze in cui venne generato e i suoi effetti non sono mai stati specificati chiaramente. Internet, attualmente, usa un metodo di notifica alternativo che ora descriveremo.

Notifica esplicita di congestione

Invece di generare pacchetti aggiuntivi per avvertire di una congestione un router può etichettare i pacchetti che inoltra (mettendo a 1 un bit nell'intestazione) per segnalare che c'è una congestione in atto. Quando la destinazione riceve il pacchetto può accorgersi che c'è stata una congestione e informarne la sorgente quando le manda il pacchetto di risposta. La sorgente può quindi limitare le sue trasmissioni.

Questo metodo è chiamato **ECN** (*explicit congestion notification,* notifica esplicata di congestione) e viene usato in Internet (Ramakrishnan et al., 2001). Si tratta di un perfezionamento dei primi protocolli di segnalazione della congestione, in particolare dello schema di feedback binario di Ramakrishnan e Jain (1988) utilizzato nell'architettura DECNET. Due bit nell'intestazione del pacchetto IP sono utilizzati per registrare se il pacchetto ha sperimentato congestione. Questa segnalazione è assente dai pacchetti nel momento in cui vengono inviati, come mostrato nella Figura 5.25; se uno qualsiasi dei router che attraversano è congestionato, il router contrassegna il pacchetto come avente subìto una congestione quando inoltrato. La destinazione restituirà al mittente un segnale esplicito di congestione nel suo pacchetto di risposta successiva; nella figura ciò è illustrato con una linea tratteggiata, per indicare che ciò accade al di sopra del livello IP (ad esempio, in TCP). Il mittente deve quindi ridurre le sue trasmissioni, come nel caso dei choke packet.

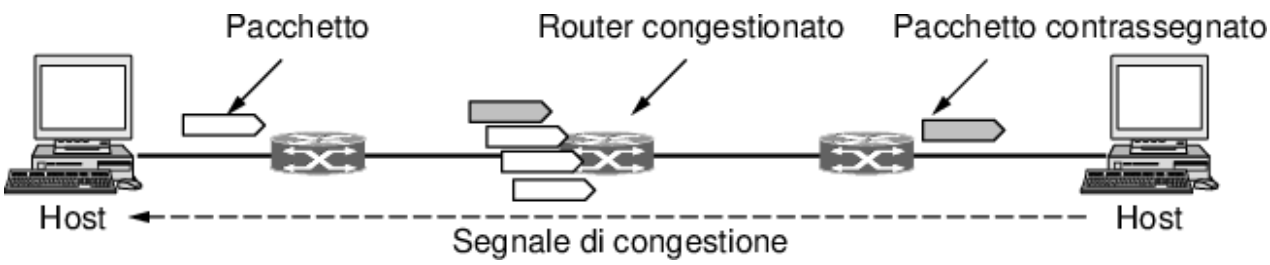


Figura 5.25 Notifica esplicita di congestione.

Choke packet hop-by-hop

Ad alte velocità o su lunghe distanze molti nuovi pacchetti possono essere trasmessi dopo la segnalazione di una congestione a causa della lenta diffusione del segnale. Si consideri, per esempio, un host a San Francisco (router *A* nella Figura 5.26) che trasmette il traffico a un host che si trova a New York (router *D* nella Figura 5.26) alla velocità OC-3 di 155 Mbps. Se l'host di New York comincia a esaurire il buffer, il choke packet che rallenterà la trasmissione impiegherà circa 40 ms per raggiungere San Francisco. Un'indicazione ECN ci mette ancora di più. La propagazione del choke packet è mostrata nella terza e nella quarta fase della Figura 5.26(a). In quei 40 ms potrebbero essere trasmessi altri 6,2 megabit di dati. Anche se l'host di San Francisco interrompesse immediatamente la trasmissione, i 6,2 megabit nel canale continuerebbero ad affluire e ciò richiederebbe comunque di doverli gestire. Solo nel settimo diagramma nella Figura 5.26(a) il router di New York noterebbe un rallentamento del flusso.

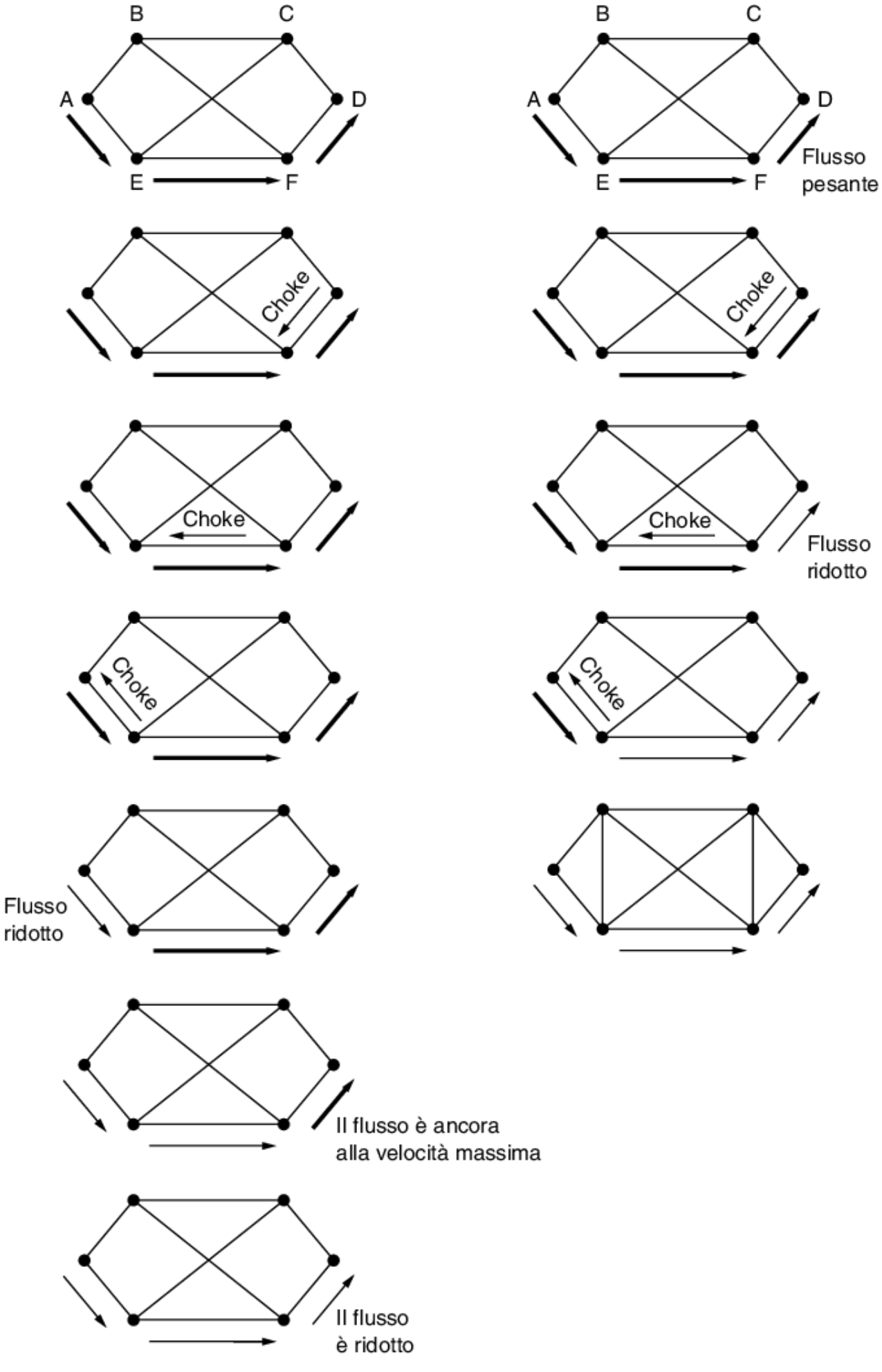

B
C
A
D
E
F
Flusso pesante
Choke
Choke
Flusso ridotto
Choke
Choke
Flusso ridotto
Il flusso è ancora alla velocità massima
Il flusso è ridotto
(a)
(b)

Figura 5.26 (a) Un choke packet che influenza solo la sorgente. (b) Un choke packet che influenza tutti i router attraversati.

In un approccio alternativo, il choke packet ha effetto su tutti i salti (*hop*) attraversati, come mostrato nella sequenza descritta nella Figura 5.26(b). In questo caso non appena il choke packet raggiunge *F,* il router *F* riduce il flusso diretto a *D.* Quando si agisce in questo modo, *F* deve dedicare più buffer al flusso, poiché la sorgente continua a inviare i dati a piena velocità; Tuttavia *D* riceve un sollievo immediato, proprio come accade nelle pubblicità televisive dei rimedi contro il mal di testa. Nella fase successiva il choke packet raggiunge *E* e il router riduce il flusso dei dati diretto verso *F.* Questa azione incrementa l'utilizzo dei buffer di *E,* ma dà un sollievo immediato a *F.* Infine il choke packet raggiunge *A* e il flusso rallenta realmente.

Questo schema, che considera la sequenza degli hop attraversati (hop-by-hop), ha l'effetto di alleviare rapidamente la congestione nel punto dove si forma, al prezzo di un incremento dell'utilizzo dei buffer di trasmissione nella parte di percorso precedente. In questo modo la congestione può essere stroncata sul nascere senza perdere alcun pacchetto. L'idea è descritta in dettaglio in Mishra et al. (1996).

Load shedding

Quando nessuno dei metodi descritti precedentemente riesce a eliminare la congestione, i router possono ricorrere all'artiglieria pesante: l'eliminazione del carico. La tecnica di **load shedding** (*eliminazione del carico*) è un modo estroso per indicare l'azione di eliminazione intrapresa dai router inondati da troppi pacchetti. Il termine deriva dal mondo della generazione dell'energia elettrica e si riferisce alla pratica di oscurare intenzionalmente certe aree per evitare il collasso dell'intera rete durante i giorni estivi più caldi, quando il consumo di elettricità supera enormemente la disponibilità.

La questione chiave è come scegliere i pacchetti da scartare. La scelta del pacchetto da scartare può dipendere dalle applicazioni in esecuzione. Per il trasferimento dei file, un pacchetto vecchio è più prezioso di uno nuovo, perché eliminare il pacchetto 6 e tenere i pacchetti 7, 8, 9 e 10 costringe il destinatario a tenere nel buffer pacchetti che non può usare. Al contrario nel caso dei dati multimediali un pacchetto nuovo è più importante di un pacchetto vecchio. Infatti tali pacchetti risultano inutili se subiscono un ritardo e non arrivano nel tempo in cui l'utente deve riceverli.

La prima delle due politiche (il vecchio è migliore del nuovo) è spesso chiamata **wine** (vino), mentre la seconda (il nuovo è migliore del vecchio) è spesso chiamata **milk** (latte) perché si beve il latte fresco e il vino d'annata.

Un ulteriore passo in avanti richiede la cooperazione dei mittenti. Un esempio sono i pacchetti che trasportano informazioni di routing. Tali pacchetti sono più importanti di quelli di dati, perché stabiliscono i percorsi e in caso di loro perdita la rete perderebbe connettività. Un altro esempio sono gli algoritmi di compressione video, come MPEG, che trasmettono periodicamente un intero

frame e poi inviano i frame successivi come differenza dall'ultimo frame completo. In questo caso è preferibile eliminare un pacchetto che fa parte di una differenza, piuttosto che scartare un pacchetto che contiene un fotogramma completo.

Per implementare un criterio di eliminazione intelligente, le applicazioni devono contrassegnare i loro pacchetti con classi di priorità, in modo da indicare la loro importanza. Se lo fanno, i router possono scartare i pacchetti partendo da quelli di classe più bassa. Naturalmente, senza un adeguato incentivo tutti utilizzeranno il contrassegno "MOLTO IMPORTANTE — NON CANCELLARE MAI". L'incentivo potrebbe essere economico, per esempio la rete potrebbe fornire un tasso di trasmissione superiore a quello comprato se i pacchetti sono segnati a bassa priorità. Questa strategia, in effetti, è buona: rende più efficiente l'utilizzo delle risorse poiché permette agli host di adoperarle quando nessun altro è interessato, ma senza stabilire un diritto di utilizzo quando la situazione si fa difficile.

Random early detection

È risaputo che risulta più semplice gestire la congestione appena viene rilevata piuttosto che cercare di porvi rimedio dopo averle dato il tempo di bloccare tutto. Questa osservazione conduce all'idea di scartare i pacchetti prima che l'intero spazio del buffer sia completamente esaurito.

La motivazione alla base di tale idea è che la maggior parte degli host di Internet non riceve segnali di congestione dai router sotto forma di ECN. L'unica indicazione affidabile di congestione ricevuta è la perdita di pacchetti. In fondo è difficile costruire un router che non getti via i pacchetti se sovraccarico. In alcuni protocolli di trasporto quali TCP essi sono implementati per reagire alla perdita di pacchetti come per una congestione, causando il rallentamento della sorgente. Il ragionamento alla base di questa logica è che TCP è stato progettato per le reti cablate e tali reti sono molto affidabili, perciò la perdita dei pacchetti è causata più che altro dall'esaurimento dei buffer e non dagli errori di trasmissione. Per funzionare bene con TCP, le linee wireless reagiscono agli errori di trasmissione a livello data link (quindi non arrivano a livello di rete).

Questo fatto può essere sfruttato per aiutare a ridurre la congestione. Facendo in modo che i router scartino i pacchetti prima che la situazione diventi senza speranza, è possibile bloccare il problema sul nascere, prima che sia troppo tardi. Un celebre algoritmo usato per mettere in pratica questo schema è chiamato **RED** (*random early detection*) (Floyd e Jacobson, 1993). Per stabilire quando è il momento giusto per iniziare a scartare i pacchetti, i router mantengono una media mobile delle lunghezze delle code. Quando la lunghezza media della coda su una linea supera una soglia di guardia, la linea è considerata congestionata e una piccola frazione di pacchetti viene scartata in modo casuale. La scelta dei pacchetti in modo casuale rende più probabile lo scarto di quelli delle sorgenti più veloci; questa è l'opzione migliore, poiché il router probabilmente non è in grado di segnalare quale sia la sorgente che causa la maggior parte dei guai in una rete datagram. La sorgente noterà la perdita quando non c'è acknowledgement e il protocollo di trasporto rallenterà. Il pacchetto perso, quindi, trasmetterà lo stesso messaggio di un choke packet, ma implicitamente, senza che il router invii un segnale esplicito.

Rispetto a quelli che scartano i pacchetti quando hanno i buffer pieni, i router RED permettono di migliorare le prestazioni della rete, ma per funzionare correttamente richiedono una fase di inizializzazione. Per esempio il numero ideale di pacchetti da scartare dipende dal numero di

sorgenti a cui notificare la congestione. ECN rimane comunque l'opzione da preferire quando disponibile: funziona allo stesso modo, ma consegna un esplicito segnale di congestione piuttosto che una perdita; RED viene usato quando gli host non possono ricevere segnali espliciti.

5.4 Qualità del servizio

Le tecniche descritte nei precedenti paragrafi sono state progettate per ridurre la congestione e migliorare le prestazioni della rete. Tuttavia alcune applicazioni (e alcuni clienti) richiedono garanzie di prestazioni più rassicuranti de "il meglio che si può fare date le circostanze". In particolare le applicazioni multimediali, per poter funzionare, spesso richiedono alla rete una capacità minima e un ritardo massimo. In questo paragrafo proseguiamo lo studio delle prestazioni della rete, concentrando però l'attenzione sui modi che consentono di fornire la qualità di servizio richiesta dalle applicazioni. Questo è un campo in cui Internet sta sperimentando un rinnovamento sul lungo periodo.

Una soluzione semplice per fornire una buona qualità del servizio è quella di costruire una rete con capacità sufficiente per qualsiasi traffico venga immesso. Il nome di tale soluzione è **overprovisioning** (*fornitura in eccesso*). La rete risultante sarebbe in grado di supportare il traffico delle applicazioni, senza perdite significative e, ipotizzando un algoritmo di routing decente, di consegnare i pacchetti con una bassa latenza; tali prestazioni sarebbero ottimali.

Il problema di questa soluzione sono i costi. Si tratta essenzialmente di risolvere un problema investendoci molto denaro. I meccanismi di qualità del servizio permettono a una rete con minore capacità di soddisfare i requisiti delle applicazioni altrettanto bene e a un costo inferiore. Inoltre l'overprovisioning si basa sul traffico previsto. Tutte le scommesse vengono perse se il profilo di traffico cambia troppo. Con i meccanismi di qualità di servizio la rete può onorare le garanzie di prestazioni fatte anche quando vi sono picchi di traffico, a costo di abbassare il livello di alcune richieste.

Per garantire la qualità del servizio devono essere affrontate le seguenti quattro questioni.

1. Di che cosa le applicazioni hanno bisogno dalla rete.
2. Come regolare il traffico che entra in rete.
3. Come prenotare le risorse dei router per garantire le prestazioni.
4. Capire se la rete può tranquillamente accettare più traffico.

Non esiste una singola tecnica che tratti efficientemente tutti questi problemi, ma è stata sviluppata una varietà di tecniche da usare a livello di rete e di trasporto. Nella pratica le soluzioni di qualità del servizio combinano molte tecniche; a tal fine descriveremo due versioni della qualità del servizio per Internet chiamate *servizi integrati* e *servizi differenziati*.

Requisiti delle applicazioni

Un flusso di pacchetti diretto da una sorgente a una destinazione è chiamato semplicemente **flusso** (*flow*) (Clarke, 1988). Un flusso può essere l'insieme dei pacchetti di una connessione in una rete orientata alle connessioni o l'insieme dei pacchetti da un processo a un altro in una rete senza connessione. Le esigenze di ogni flusso possono essere caratterizzate da quattro parametri primari: affidabilità, ritardo, jitter e banda. Insieme questi parametri determinano la **QoS** (*quality of service*), ossia la qualità del servizio richiesta dal flusso.

La Figura 5.27 elenca diverse applicazioni note e la rigidità dei corrispondenti requisiti. Si noti che, nei casi in cui le applicazioni possono migliorare il servizio offerto dalla rete, i requisiti da parte di quest'ultima sono meno stringenti di quelli delle applicazioni. In particolare le reti non hanno bisogno di essere prive di perdite per effettuare un trasferimento di file affidabile né di consegnare pacchetti con ritardi identici per riprodurre video e audio. Un certo quantitativo di dati persi può essere recuperato tramite ritrasmissioni e il jitter può essere ridotto facendo uso di buffer al ricevente. Tuttavia non c'è nulla che le applicazioni possano fare per migliorare la situazione se la rete fornisce troppa poca banda o ha troppo ritardo.

Applicazione	Banda	Ritardo	Jitter	Perdita di dati
Posta elettronica	Bassa	Bassa	Bassa	Media
Condivisione file	Alta	Bassa	Bassa	Media
Accesso al Web	Media	Media	Bassa	Media
Terminale remoto	Bassa	Media	Media	Media
Audio on demand	Bassa	Bassa	Alta	Bassa
Video on demand	Alta	Bassa	Alta	Bassa
Telefonia	Bassa	Alta	Alta	Bassa
Videoconferenza	Alta	Alta	Alta	Bassa

Figura 5.27 Rigidità dei requisiti di qualità del servizio da parte delle applicazioni.

Le applicazioni differiscono nei requisiti di banda: posta elettronica, audio in tutte le sue forme e login da remoto ne vogliono poca, file sharing e video molta.

I requisiti sul ritardo sono più interessanti. Le applicazioni di trasferimento file, inclusi posta elettronica e video, non sono sensibili al ritardo. Se tutti i pacchetti subiscono un ritardo uniforme di pochi secondi non accade nulla di male. Applicazioni interattive, quali esplorazione del Web e accesso remoto, sono sensibili al ritardo. Applicazioni in tempo reale, quali telefonia e videoconferenza, hanno requisiti severi nei confronti del ritardo. Se tutte le parole di una chiamata telefonica fossero ritardate esattamente di 2 secondi, gli utenti considererebbero la connessione inaccettabile. D'altra parte la riproduzione di file audio e video scaricati da un server non richiede un basso ritardo.

La variazione (cioè deviazione standard) nel ritardo o nel tempo di arrivo di un pacchetto è chiamata **jitter**. Le prime tre applicazioni non sono sensibili ai pacchetti che arrivano a intervalli irregolari, il login remoto è un po' più sensibile a questo problema, poiché se la connessione soffre di un elevato jitter i caratteri sullo schermo potrebbero essere aggiornati a scatti. Il video e soprattutto l'audio sono estremamente sensibili allo jitter. Se un utente sta guardando un video

trasmesso attraverso la rete e i fotogrammi hanno un ritardo di esattamente 2 secondi, non accade nulla di male, ma se il tempo di trasmissione varia in modo casuale tra 1 e 2 secondi, il risultato sarà terribile a meno che l'applicazione non intervenga. Nel caso dell'audio anche un jitter di pochi millisecondi risulta chiaramente udibile.

Le prime quattro applicazioni hanno requisiti più rigidi riguardo all'affidabilità rispetto ad audio e video; tutti i bit devono essere consegnati correttamente. Questo obiettivo di solito è raggiunto ritrasmettendo i pacchetti persi a livello di trasporto; questo è uno spreco, sarebbe meglio rifiutare i pacchetti che verosimilmente verranno persi. Le applicazioni audio e video possono tollerare perdite senza ritrasmissione, perché non ci si accorge di un'occasionale perdita di un frame.

Per poter ospitare una varietà di applicazioni, le reti supportano differenti categorie di QoS. Un esempio importante viene dalle reti ATM concepite con un'ambiziosa visione e diventate una nicchia tecnologica. Esse supportano:

1. un tasso di trasmissione costante (per esempio, telefonia)
2. un tasso di trasmissione variabile in tempo reale (per esempio, videoconferenza compressa)
3. un tasso di trasmissione variabile non in tempo reale (per esempio, la visione di un film on demand)
4. un tasso di trasmissione disponibile (per esempio, il trasferimento di file).

Queste categorie sono utili anche per altre funzioni e altre reti. Il tasso di trasmissione costante tenta di simulare un cavo, fornendo una banda e un ritardo uniformi. Il tasso di trasmissione variabile è usato in presenza di video compresso; alcuni fotogrammi si comprimono meglio di altri, perciò la trasmissione di un fotogramma contenente tanti dettagli può richiedere molti bit, mentre la trasmissione della ripresa di un muro bianco può essere compressa molto bene. I film on demand non sono in realtà real-time, perché si può memorizzare nel buffer qualche secondo di video prima che inizi la visualizzazione; quindi il jitter sulla rete causa una variazione della quantità di video nel buffer. La trasmissione al tasso disponibile (la capacità di rete libera in un dato momento) è per le applicazioni, quali la posta elettronica, che non sono sensibili né al ritardo né allo jitter e usano la banda che possono avere.

Traffic shaping

Prima di dare garanzie di QoS, la rete deve conoscere quale traffico deve garantire. Nella rete telefonica la caratterizzazione è semplice. Per esempio una chiamata vocale (in formato non compresso) richiede 64 kbps e consiste di un campione di 8 bit ogni 125 μs. Ma il traffico in una rete di dati è irregolare con dei picchi. Arriva a tassi non uniformi al variare del traffico (per esempio, le videoconferenze compresse), gli utenti interagiscono con le applicazioni (per esempio, la navigazione di una nuova pagina Web) e i computer passano da un'attività all'altra. I picchi di traffico sono più difficili da gestire del traffico costante, perché riempiono i buffer causando perdite di pacchetti.

Il **traffic shaping** regola il tasso medio della trasmissione dei dati e le irregolarità del flusso di dati che entra in rete. L'obiettivo è permettere alle applicazioni di trasmettere una grande varietà di traffico adatto alle loro necessità, inclusi i picchi, continuando ad avere un modo semplice e utile di descrivere le caratteristiche del traffico stesso. All'inizio di un flusso l'utente e la rete (cioè cliente e

fornitore)[5] si mettono d'accordo su una certa caratteristica di traffico (cioè la forma, o *shape*) di quel flusso. In effetti è come se il cliente dicesse al fornitore: "Il mio modello di trasmissione è questo. Sai gestirlo?".
Qualche volta questo accordo è chiamato **SLA** (*service level agreement*, accordo sul livello di servizio), specialmente quando è fatto su flussi aggregati e per un lungo periodo di tempo, come nel caso di tutto il traffico di un dato cliente. Fintanto che il cliente si attiene alla sua parte dell'accordo e invia solo pacchetti secondo il contratto concordato, il fornitore promette di consegnare tutti i dati in modo tempestivo.

Il traffic shaping riduce la congestione e aiuta così la rete a tener fede alle sue promesse. Per funzionare, però, il fornitore deve avere il modo di verificare se il cliente stia seguendo l'accordo e deve sapere che cosa fare in caso negativo. I pacchetti in eccesso rispetto al contratto potrebbero essere scartati o segnati come a bassa priorità. La supervisione del flusso di traffico è chiamata **traffic policing**.

Le operazioni di shaping e di policing non sono importanti per applicazioni peer to peer e altri trasferimenti che usano tutta la banda disponibile, ma lo diventano per le applicazioni real-time, come connessioni audio e video, che hanno stringenti requisiti di qualità del servizio.

Leaky e token bucket

Abbiamo già visto un modo per limitare la quantità di dati che un'applicazione invia: le finestre scorrevoli, che usano un parametro per limitare la quantità di dati in transito in ogni istante, limitando indirettamente il tasso di trasmissione. Vediamo ora un modo più generale per caratterizzare il traffico con gli algoritmi leaky bucket e token bucket. Sono leggermente diversi, ma danno risultati equivalenti.

Si immagini un secchio con un piccolo buco sul fondo, come quello disegnato nella Figura 5.28(b). Qualunque sia la velocità d'ingresso dell'acqua nel secchio, l'efflusso avrà una velocità costante R quando ci sarà acqua nel secchio e avrà una velocità pari a 0 quando il secchio sarà vuoto. Inoltre, una volta che il secchio è pieno, l'acqua aggiuntiva versata nel contenitore traboccherà e andrà perduta.

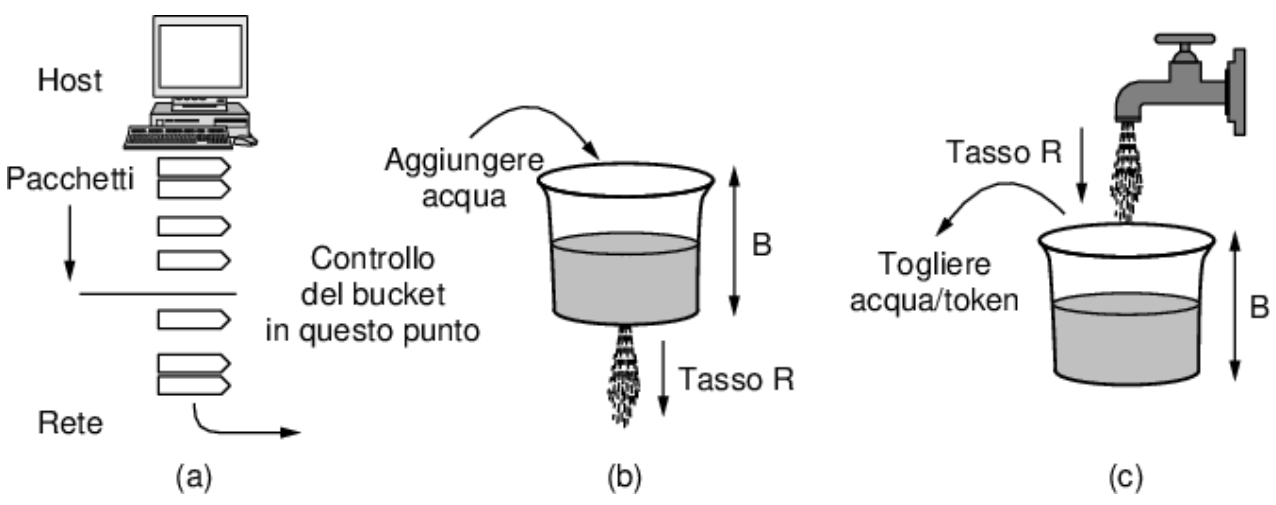


Figura 5.28 (a) Traffic shaping sui pacchetti. (b) Un leaky bucket. (c) Un token bucket.

La stessa idea può essere applicata anche ai pacchetti, come mostrato nella Figura 5.28(a). Concettualmente, ogni host è collegato alla rete da un'interfaccia che contiene un leaky bucket (letteralmente, *secchio bucato*). Per immettere un pacchetto in rete deve essere possibile mettere ancora acqua nel secchio. Se il pacchetto arriva quando il secchio è pieno deve essere accodato finché abbastanza acqua non sia uscita o scartata. La prima opzione può essere svolta dal sistema operativo di un host per fare traffic shaping dei dati in uscita, la seconda può essere implementata via hardware nell'interfaccia di rete del fornitore che fa traffic policying dei dati verso la rete. Questa tecnica è stata proposta per la prima volta da Turner (1986) ed è chiamata **algoritmo leaky bucket**.

Una formulazione diversa ma equivalente è quella di immaginare l'interfaccia di rete come un secchio che si riempie, come mostrato nella Figura 5.28(c). Il rubinetto lavora a tasso R e il secchio ha una capacità B, come prima. Per inviare un pacchetto dobbiamo essere in grado di togliere l'acqua o i token, come i contenuti sono comunemente chiamati, dal secchio (invece di mettere l'acqua nel secchio). Non più di un numero fisso di token, B, può accumularsi nel secchio e, se il secchio è vuoto, dobbiamo aspettare che più token arrivino prima di poter inviare un altro pacchetto. Questo algoritmo è chiamato **algoritmo token bucket**.

Gli algoritmi leaky e token bucket limitano il tasso di trasmissione a lungo termine di un flusso, ma permettono a brevi picchi con una lunghezza massima regolata di passare inalterati senza subire ritardi aggiuntivi. Grandi picchi vengono smussati da un traffic shaper facente uso di leaky bucket per ridurre la congestione della rete. Per esempio immaginiamo che un computer sia in grado di produrre dati fino a 1000 Mbps (125 milioni di byte/s) e che il primo collegamento della rete funzioni a questa velocità. Il modello di traffico generato dall'host è mostrato nella Figura 5.29(a). Questa sequenza presenta dei picchi; il tasso medio su un secondo è di 200 Mbps, anche se l'host manda un picco di 16.000 KB alla velocità massima di 1000 Mbps (per un ottavo di secondo).

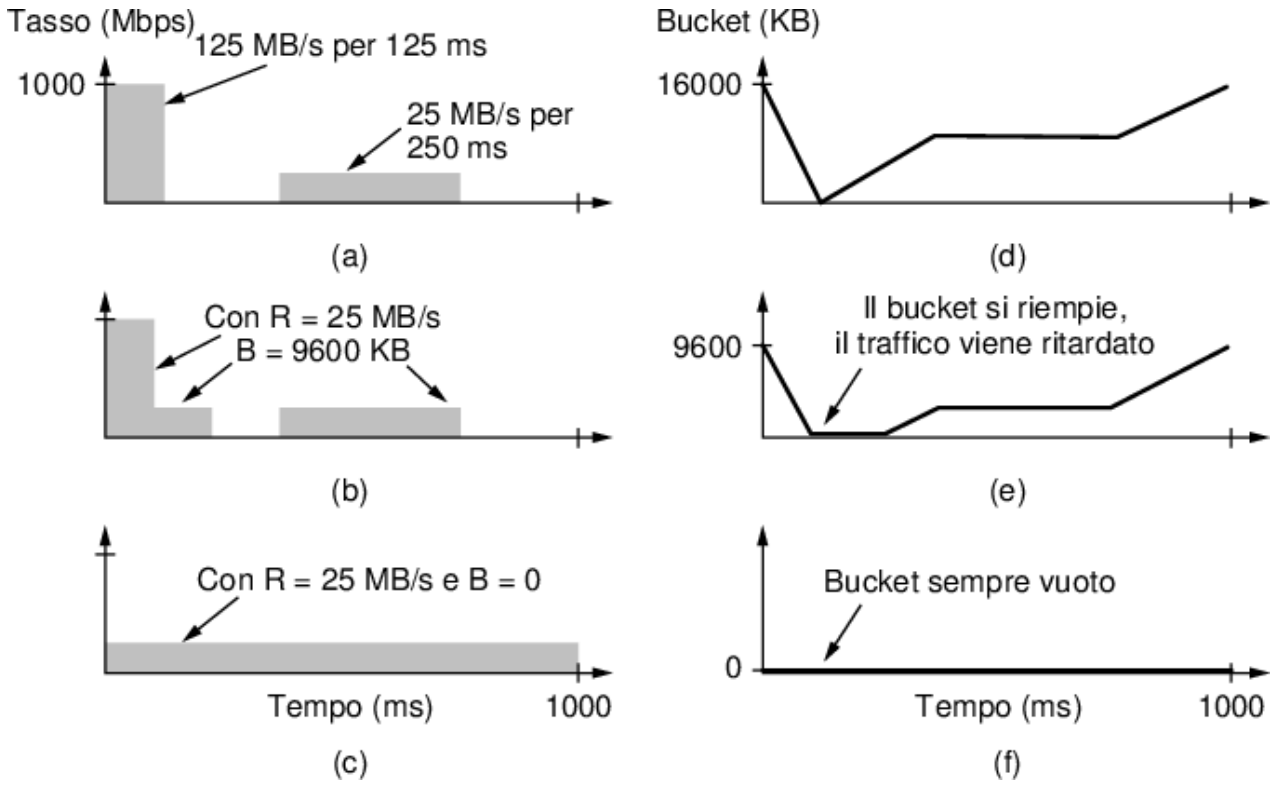


Figura 5.29 (a) Traffico proveniente da un host. Traffico in uscita regolato da un token bucket con tasso 200 Mbps e capacità di (b) 9600 KB e (c) 0 KB. Il livello del bucket per fare controllo di flusso con un tasso di 200 Mbps e capacità di (d) 16.000 KB, (e) 9600 KB e (f) 0 KB.

Ora supponiamo che il router sia in grado di accettare dati alla velocità massima solo per brevi intervalli, fino al riempimento del buffer. La dimensione del buffer è di 9600 KB, più piccola rispetto al picco di traffico. In lunghi intervalli i router funzionano meglio a tassi non superiori ai 200 Mbps (per esempio, perché questa è tutta la banda data al cliente). L'implicazione è che se il traffico viene inviato in questa sequenza alcuni pacchetti verranno scartati, perché non rientrano nel buffer dei router.

Per evitare questa perdita di pacchetti si può rimodellare il traffico a livello di host con un token bucket. Se utilizziamo un tasso, *R*, di 200 Mbps e una capacità, *B*, di 9600 KB, il traffico ricade nei limiti che la rete può gestire. L'output di questo token bucket è mostrato nella Figura 5.29(b). L'host può inviare a pieno regime a 1000 Mbps per un breve periodo fino al prosciugamento del bucket. Poi deve ridurre a 200 Mbps fino alla conclusione dell'invio del picco di dati. L'effetto è quello di spalmare il picco nel corso del tempo, perché troppo grande da gestire tutto in una volta. Il livello del token bucket è mostrato nella Figura 5.29(e): comincia pieno e si esaurisce con il picco iniziale; quando arriva a zero i nuovi pacchetti possono essere inviati solo alla velocità con cui il buffer si sta riempiendo; non ci possono essere altri picchi finché il bucket non è pronto. Il bucket si riempie quando non c'è traffico e rimane costante quando il traffico viene inviato al tasso di riempimento.

Possiamo anche rimodellare il traffico in modo da avere meno picchi. La Figura 5.29(c) mostra l'output di un token bucket con *R* = 200 Mbps e una capacità di 0. Questo è il caso estremo in cui il traffico è stato completamente smussato. Non sono ammessi picchi e il traffico ha un tasso fisso di

trasmissione. Il corrispondente livello del bucket, mostrato nella Figura 5.29(f), è sempre vuoto. Il traffico è accodato all'host ed esiste sempre un pacchetto che aspetta di essere inviato quando gli è permesso.

Infine, la Figura 5.29(d) indica il livello del bucket per un token bucket di R = 200 Mbps e una capacità di B = 16.000 KB. Questo è il più piccolo token bucket attraverso il quale il traffico passa inalterato. Potrebbe essere utilizzato in un router della rete per fare traffic policing del traffico inviato dall'host. Se l'host invia traffico conforme al token bucket concordato con la rete, il traffico si adatta allo stesso token bucket eseguito dal router al bordo della rete. Se l'host invia a un tasso più alto o con più picchi, il token bucket terminerà l'acqua e non ce la farà; in questo caso chi di dovere saprà che il traffico non è quello descritto. Il router potrà quindi scartare i pacchetti in eccesso o diminuire le loro priorità a seconda del tipo di rete. Nel nostro esempio il bucket si svuota, solo momentaneamente, al termine del picco iniziale e poi recupera abbastanza per quello successivo.

Leaky bucket e token bucket sono facili da implementare. Descriveremo ora come ottenere un token bucket. Anche se abbiamo descritto uno scorrimento continuo dell'acqua dentro e fuori dal bucket, le implementazioni reali devono lavorare con quantità discrete. Un token bucket è implementato con un contatore per il livello del bucket. Il contatore viene incrementato di $R/\Delta T$ unità a ogni ciclo di clock di ΔT secondi. Si tratterebbe di 200 Kbit ogni 1 ms nel nostro esempio precedente. Ogni volta che un'unità di traffico viene immessa in rete, il contatore viene decrementato e il traffico può essere inviato finché il contatore non raggiunga lo zero.

Quando i pacchetti sono tutti della stessa dimensione, il livello del bucket può solo essere conteggiato in pacchetti (per esempio, 200 Mbit sono 20 pacchetti di 1250 byte). Tuttavia, spesso i pacchetti sono di dimensione variabile. In questo caso, il livello del bucket viene conteggiato in byte. Se il conteggio dei byte residui è troppo basso per inviare un pacchetto di grandi dimensioni, il pacchetto deve attendere il prossimo ciclo di clock (o anche di più, se il tasso di riempimento è basso).

Calcolare la lunghezza del picco massimo (fino allo svuotamento del bucket) può essere leggermente complicato. Non è sufficiente dividere 9600 KB per 125 MB/s, perché arrivano altri token mentre il picco viene trasmesso. Sia S la durata del picco in secondi, M byte/s la velocità di output massima, B la capacità del bucket, R byte/s la velocità di arrivo dei token; quindi un picco di output può essere costituito da un massimo di $B + RS$ byte. Si sa anche che il numero di byte in un picco di lunghezza S secondi è MS. Quindi:

$$B + RS = MS$$

Si può risolvere l'equazione per ottenere $S = B/(M - R)$. Per i parametri B = 9600 KB, M = 125 MB/s e R = 25 MB/s si ottiene un tempo di picco di circa 94 ms.

Un problema potenziale dell'algoritmo token bucket è che questo riduce i grandi picchi al tasso sul lungo periodo R. Spesso si desidera ridurre il tasso di picco, ma senza tornare al basso valore del leaky bucket originale. Si può ottenere un traffico più uniforme inserendo un secondo token bucket più veloce. Il primo caratterizza il traffico, fissando il tasso medio, ma permettendo piccoli picchi; il secondo riduce il tasso di trasmissione massimo dei picchi che vengono immessi nella

rete. Per esempio, se il tasso del secondo token bucket è di 500 Mbps e la capacità è 0, il picco iniziale entra in rete con un tasso di picco di 500 Mbps, che è più piccolo dei 1000 Mbps che avevamo prima.

L'impiego di tutti questi bucket può risultare un po' complesso. Quando i token bucket sono usati per fare shaping di traffico dagli host, i pacchetti sono accodati e ritardati finché i bucket non permettono di mandarli. Quando, invece, sono usati dai router nella rete per fare traffic policing, essi devono simulare l'algoritmo e assicurarsi che non vengano trasmessi più pacchetti di quelli consentiti. Ciò nonostante, questi strumenti permettono di dare al traffico di rete forme più maneggevoli che aiutano a soddisfare i requisiti di qualità del servizio.

Scheduling dei pacchetti

Poter regolare la forma del traffico offerto è un buon punto di partenza per garantire la qualità del servizio. Tuttavia, per fornire buone garanzie di prestazioni, bisogna prenotare le risorse sufficienti lungo tutto il percorso. Assumiamo che tutti i pacchetti di un flusso seguano lo stesso percorso; sparpagliare i pacchetti su router scelti a caso rende difficile garantire qualunque cosa. Di conseguenza, è necessario impostare qualcosa che assomigli a un circuito virtuale tra sorgente e destinazione e tutti i pacchetti che appartengono al flusso dovranno seguire questo percorso.

Gli algoritmi che allocano le risorse dei router tra i pacchetti di un flusso e tra flussi in competizione si chiamano **algoritmi di scheduling dei pacchetti** (*packet scheduling algorithm*). Potenzialmente si possono prenotare tre diversi tipi di risorse:

1. banda
2. spazio nei buffer
3. cicli di CPU.

La banda è la risorsa più ovvia. Se un flusso richiede 1 Mbps e la linea di trasmissione ha una capacità di 2 Mbps, tentare di spingere tre flussi attraverso quella linea non è possibile: prenotare la banda significa non sovraccaricare alcuna linea di output.

Una seconda risorsa che spesso scarseggia è lo spazio nei buffer. Quando arrivano, i pacchetti di solito sono depositati nel buffer del router finché la linea scelta non si libera. Lo scopo del buffer è assorbire i piccoli picchi di traffico quando c'è concorrenza tra i flussi. Se non è disponibile alcun buffer, un pacchetto in arrivo deve essere scartato, perché non c'è posto dove tenerlo. Per una buona qualità di servizio, alcuni buffer possono essere dedicati a un flusso specifico, in modo che quel flusso non debba competere con altri: ci sarà sempre un buffer disponibile quando il flusso ne avrà bisogno, fino a raggiungere un valore massimo.

Infine anche i cicli di CPU sono una risorsa limitata. L'elaborazione di un pacchetto consuma tempo di CPU del router, perciò un'apparecchiatura può elaborare solo un certo numero di pacchetti al secondo. I router moderni sono in grado di elaborare velocemente la maggior parte dei pacchetti, ma alcuni tipi di pacchetti necessitano più tempo di CPU; un esempio è dato dai pacchetti ICMP che verranno descritti nel Paragrafo 5.6. Accertandosi che la CPU non sia sovraccaricata si può garantire un'elaborazione tempestiva di ogni pacchetto.

Gli algoritmi di scheduling dei pacchetti allocano la banda e le altre risorse di un router determinando quali pacchetti del buffer inviare sulla linea di uscita di volta in volta. Quando abbiamo spiegato il funzionamento dei router abbiamo già descritto lo scheduler più semplice. Ogni router mantiene nei buffer i pacchetti in coda per ogni linea di uscita finché questi possano essere inviati, cosa che avviene nello stesso ordine di arrivo. Tale algoritmo è noto come **FIFO** (*first-in first-out*) o, in modo equivalente, **FCFS** (*first-come-first-serve*).

I router FIFO scartano i pacchetti in arrivo quando la coda è piena. Dal momento che il pacchetto appena arrivato sarebbe stato posto alla fine della coda, questo comportamento è chiamato **tail drop**. È intuitivo e vi chiederete come possano esistere alternative. In realtà, l'algoritmo RED descritto nel Paragrafo 5.3.5 scartava in modo casuale un pacchetto appena arrivato quando la lunghezza media della coda cresceva troppo. Gli altri algoritmi di scheduling che descriveremo useranno altri metodi per decidere quali pacchetti scartare quando i buffer sono pieni.

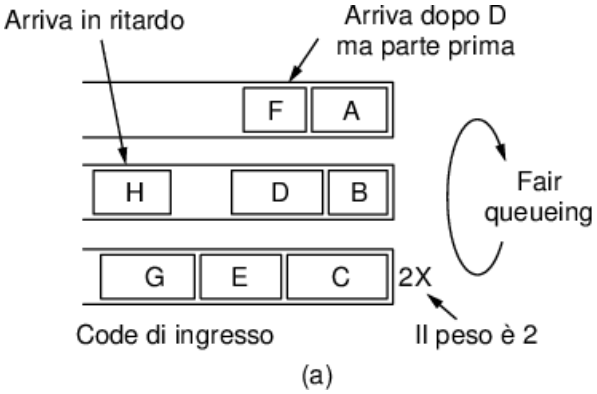
Lo scheduling FIFO è semplice da implementare, ma non è adatto a fornire una buona qualità del servizio perché, quando ci sono flussi multipli un flusso può facilmente influire sulle prestazioni degli altri. Se il primo flusso è aggressivo e manda numerosi picchi di traffico, i suoi pacchetti prenderanno posto in coda. Elaborare i pacchetti nell'ordine di arrivo significa che il mittente aggressivo può prendersi la maggior parte della capacità dei router, danneggiando gli altri flussi e riducendo la loro qualità del servizio. Per aggiungere al danno la beffa, i pacchetti degli altri flussi che trovano posto nel buffer possono essere ritardati, perché devono aspettare in coda dietro ai molti pacchetti dal mittente aggressivo.

Sono stati sviluppati diversi algoritmi di pianificazione dei pacchetti che forniscono un forte isolamento tra flussi e non ne permettono l'interferenza (Bhatti e Crowcroft, 2000). Uno dei primi è stato l'algoritmo di **fair queuing** (*accodamento equo*) di Nagle (1987). L'essenza di questo algoritmo sta nell'attribuire ai router code separate per ogni flusso su ogni linea di trasmissione. Quando una linea si libera, il router esamina le code in modalità round-robin, come mostrato nella Figura 5.30, prendendo ogni volta il primo pacchetto della coda successiva. In questo modo con n host che concorrono per una data linea di trasmissione, ogni host riesce a trasmettere uno dei suoi pacchetti ogni n pacchetti. L'invio di un numero maggiore di pacchetti non aumenta il tasso di trasmissione dal router.

Figura 5.30 Fair queueing con round-robin.

Anche se ottiene un primo risultato, l'algoritmo presenta un problema: offre più banda agli host che usano pacchetti grandi. Demers et al. (1990) hanno suggerito un miglioramento in cui il round-robin è fatto in modo da simulare una sequenza byte per byte, invece che pacchetto per pacchetto. Il problema è calcolare un tempo virtuale pari al numero di cicli necessari per inviare ogni pacchetto. Ogni ciclo prende un byte da tutte le code con byte da spedire. I pacchetti sono quindi ordinati in base all'istante in cui l'ultimo byte viene prelevato dalla coda (istante finale) e vengono trasmessi esattamente in quest'ordine.

Questo algoritmo e un esempio di tempi di elaborazione di pacchetti che arrivano in tre flussi distinti sono illustrati nella Figura 5.31. Se un pacchetto ha una lunghezza *L*, il suo istante finale è semplicemente *L* istanti dopo l'istante iniziale. Come istante iniziale consideriamo l'istante finale del pacchetto precedente o l'istante di arrivo del pacchetto se trova la coda vuota.



Pacchetto	Tempo di arrivo	Lunghezza	Istante finale	Ordine di uscita
A	0	8	8	1
B	5	6	11	3
C	5	10	10	2
D	8	9	20	7
E	8	8	14	4
F	10	6	16	5
G	11	10	19	6
H	20	8	28	8

(b)

Figura 5.31 (a) Weighted fair queueing. (b) Tempo di termine per i pacchetti.

Dalla tabella della Figura 5.31(b), e guardando solo i primi due pacchetti delle due code in alto, i pacchetti arrivano nell'ordine *A*, *B*, *D* e *F*. Il pacchetto *A* arriva all'istante 0 ed è lungo 8 byte, quindi il suo istante finale è il numero 8. Allo stesso modo l'istante finale del pacchetto *B* è 11. Il pacchetto *D* arriva mentre *B* viene inviato. Il suo istante finale è di 9 istanti dopo la fine di *B*, o 20. Allo stesso modo l'istante finale di *F* è 16. In assenza di nuovi arrivi l'ordine relativo di invio è *A*, *B*, *F*, *D*, anche se *F* è arrivato dopo *D*. È possibile che un altro piccolo pacchetto arrivi sul flusso in cima e ottenga un istante finale inferiore a *D*. Salterà davanti a *D* se la trasmissione di tale pacchetto non è ancora iniziata. L'algoritmo di fair queing non altera i pacchetti in corso di trasmissione. Dato che i pacchetti vengono inviati nella loro interezza, fair queuing è solo un'approssimazione dello schema ideale byte per byte. Ma è una approssimazione molto buona, rimanendo sempre all'interno della trasmissione di un pacchetto nel modello ideale.

Un problema di questo algoritmo nella pratica è che dà a tutti gli host la stessa priorità. In molte situazioni è auspicabile assegnare, per esempio, ai server che distribuiscono video più banda che non ai server che rendono disponibili dei file. Ciò è fattibile semplicemente dando ai server video

due byte per round. L'algoritmo modificato, largamente utilizzato, è stato chiamato **WFQ** (*weighted fair queueing,* accodamento equo ponderato). Considerando il numero di byte per istante come il peso del flusso W, l'istante finale è:

$$F_i = \max(A_i, F_{i-1}) + L_i W$$

dove A_i è l'istante di arrivo, F_i è l'istante finale e L_i è la lunghezza del pacchetto i.

La coda in fondo nella Figura 5.31(a) ha un peso di 2, quindi i suoi pacchetti vengono inviati più velocemente, come si può vedere dai tempi finali riportati nella Figura 5.31(b).

Un'altra considerazione pratica è la complessità di implementazione. WFQ richiede che i pacchetti vengano inseriti in una coda ordinata dai loro istanti finali. Con N flussi si hanno nella migliore delle ipotesi $O(\log N)$ operazioni per ogni pacchetto, difficile da ottenere con molti flussi nei router ad alta velocità. Shreedhar e Varghese (1995) descrivono un'approssimazione chiamata **deficit round-robin** (*round-robin deficitario*) che può essere implementata in maniera molto efficiente, con solo $O(1)$ operazioni per pacchetto. WFQ è ampiamente usato grazie a questa approssimazione.

Esistono anche altri tipi di algoritmi di scheduling. Un semplice esempio è lo scheduling con priorità, in cui ogni pacchetto è marcato con una priorità. I pacchetti ad alta priorità sono sempre inviati prima dei pacchetti a bassa priorità. All'interno di una stessa classe di priorità i pacchetti vengono inviati secondo la politica FIFO. Tuttavia lo scheduling a priorità ha lo svantaggio per cui un picco di pacchetti ad alta priorità può far bloccare per un tempo indefinito i pacchetti a bassa priorità. WFQ fornisce spesso una soluzione migliore; dando alla coda ad alta priorità un grosso peso, diciamo 3, i pacchetti ad alta priorità spesso attendono poco tempo in coda (supponendo che ci siano relativamente pochi pacchetti con priorità alta) permettendo a una frazione di pacchetti a bassa priorità di continuare a essere inviati anche in presenza di traffico ad alta priorità. Un sistema di alta e bassa priorità è essenzialmente un sistema WFQ con due code, in cui la alta priorità ha un peso infinito.

Come ultimo esempio di scheduler, i pacchetti potrebbero portare con sé una indicazione temporale (*timestamp*) ed essere inviati in ordine cronologico. Clark et al. (1992) descrivono un'architettura in cui il timestamp memorizza quanto il pacchetto sia in ritardo o in anticipo rispetto alla tempistica prevista. I pacchetti che sono stati in coda dietro altri pacchetti in un router tenderanno a essere in ritardo, mentre i pacchetti che sono stati serviti prima tenderanno a essere in anticipo sui tempi previsti. L'invio di pacchetti in ordine temporale ha il benefico effetto di accelerare i pacchetti lenti e allo stesso tempo rallentare i pacchetti veloci. Il risultato è che tutti i pacchetti vengono consegnati dalla rete con un ritardo più uniforme.

Controllo di ammissione

Abbiamo finora esaminato tutti gli elementi necessari per la QoS. È tempo quindi di metterli assieme per realizzarla effettivamente. Le garanzie di QoS saranno stabilite attraverso il processo di controllo di ammissione. Prima abbiamo visto il controllo di ammissione usato per controllare la congestione, che è una garanzia di prestazioni, sebbene debole; le garanzie che consideriamo ora sono più forti, ma il modello è lo stesso. L'utente sottopone alla rete un flusso unitamente ai

requisiti di QoS, questa deve quindi stabilire se accettare o rifiutare il flusso basandosi sulla propria capacità e sugli impegni già presi con altri flussi. Se accetta, la rete riserva in anticipo parte della capacità sui router per garantire la QoS quando il traffico viene mandato sul nuovo flusso.

La prenotazione deve essere fatta a tutti i router del percorso che i pacchetti attraverseranno in rete. Ogni router sul percorso che non abbia ricevuto una prenotazione potrebbe diventare congestionato e anche un singolo router congestionato fa fallire la garanzia di QoS. Alcuni algoritmi di routing trovano il cammino migliore tra ogni sorgente e ogni destinazione e vi inviano tutto il traffico. Questo metodo porta a rigettare alcuni flussi in assenza di capacità sufficiente lungo il cammino migliore. Le garanzie di QoS possono ancora essere soddisfatte su un percorso diverso con capacità in eccesso. Questo metodo è chiamato **routing con QoS**. Chen e Nahrstedt (1998) descrivono in dettaglio queste tecniche. È possibile dividere il traffico per ogni destinazione su più cammini per avere più facilmente a disposizione capacità in eccesso. Un metodo semplice è che i router scelgano cammini di uguale costo e dividano il traffico uniformemente su di essi o in proporzione alla capacità dei loro collegamenti in uscita. Tuttavia esistono anche algoritmi più sofisticati (Nelakuditi e Zhang, 2002).

Dato un percorso, la decisione di accettare o rifiutare un flusso non dipende solo dall'esito del confronto tra le risorse richieste (banda, buffer e cicli di CPU) e la capacità in eccesso del router, ma è un po' più complicato. Prima di tutto, sebbene alcune applicazioni possano conoscere i loro requisiti di banda, solo alcune sanno valutare i requisiti relativi ai buffer e ai cicli di CPU, perciò è necessario come minimo un sistema di descrizione dei flussi diverso e un modo per tradurlo in risorse del router; parleremo di questo tra breve.

In secondo luogo, alcune applicazioni sono molto più tolleranti di altre nei confronti di limiti che occasionalmente non vengono mantenuti. Le applicazioni devono scegliere tra i tipi di garanzia offerti dalla rete, siano esse sempre sicure o valide per il massimo tempo possibile. A parità di tutto il resto, tutti vorrebbero forti garanzie, ma sono costose perché obbligano a riferirsi al caso peggiore. Garanzie per la maggior parte di pacchetti sono spesso sufficienti per le applicazioni e con tali garanzie sono supportabili più flussi, data una certa capacità.

Infine, alcune applicazioni possono essere disposte a contrattare i parametri del flusso, altre invece no. Per esempio, un riproduttore video che normalmente visualizza 30 fotogrammi al secondo può accettare di scendere a 25 fotogrammi al secondo, se la banda disponibile non è in grado di supportare 30 fotogrammi al secondo; in modo analogo è possibile regolare il numero di pixel per fotogramma, la banda audio e altre proprietà.

Poiché nella negoziazione del flusso possono essere coinvolte molte parti (il trasmittente, il ricevente e tutti i router lungo il percorso che collega i due punti), i flussi devono essere descritti accuratamente in termini di parametri specifici che possono essere negoziati. Un insieme di parametri di questo tipo è chiamato **specifica di flusso** (*flow specification*). In genere il trasmittente (per esempio il server che distribuisce il video) produce una specifica di flusso per proporre i parametri da utilizzare. Quando la specifica si propaga lungo il percorso, ogni router la esamina e modifica i parametri in base alle sue necessità. Le modifiche possono solo ridurre il flusso, non possono aumentarlo (ossia, si può diminuire la banda, ma non aumentarla). Quando arriva all'altro capo, i parametri possono essere fissati.

Come esempio di quello che ci può essere in una specifica di flusso si consideri la Figura 5.32, che si riferisce a un caso basato sugli RFC 2210 e 2211 per servizi integrati di cui parleremo nel prossimo paragrafo. Ci sono cinque parametri; i primi due, *tasso token bucket* e *dimensione del token bucket*, usano un token bucket per dare il massimo tasso di trasmissione della sorgente, mediato su un lungo intervallo di tempo, e il picco massimo trasmettibile in poco tempo.

Parametro	**Unità**
Tasso del token bucket	Byte/s
Dimensione del token bucket	Byte
Tasso di picco	Byte/s
Dimensione minima del pacchetto	Byte
Dimensione massima del pacchetto	Byte

Figura 5.32 Un esempio di specifica di flusso.

Il terzo parametro, *tasso di picco,* è la velocità di trasmissione massima tollerata, anche per brevi intervalli di tempo; il trasmittente non deve mai superarla.

Gli ultimi due parametri specificano la dimensione minima e massima del pacchetto, incluse le intestazioni dei livelli di rete e di trasporto (per esempio, TCP e IP). La dimensione minima è importante, perché l'elaborazione di ogni pacchetto richiede un tempo prefissato, qualunque sia la lunghezza. Un router può essere pronto a gestire 10.000 pacchetti al secondo grandi 1 KB, ma non 100.000 pacchetti al secondo grandi 50 byte ognuno, anche se questo valore rappresenta una velocità dati più bassa. La dimensione massima del pacchetto è importante a causa delle limitazioni interne della rete che non possono essere superate. Per esempio, se parte del percorso attraversa una Ethernet, la dimensione di ogni pacchetto non potrà superare i 1.500 byte anche se il resto della rete supporta dimensioni maggiori.

Una questione interessante riguarda il modo in cui un router trasforma una specifica di flusso in una serie di prenotazioni di risorse specifiche. A prima vista si potrebbe credere che, se il router ha un collegamento a 1 Gbps e il pacchetto medio è di 1000 bit, può elaborare 1 milione di pacchetti al secondo. Questa osservazione non è vera, perché ci saranno sempre periodi di inattività causati da fluttuazioni statistiche nel carico. Se il collegamento ha bisogno di ogni singolo bit di capacità per svolgere il suo lavoro, anche la perdita di pochi bit crea un arretrato che non potrà mai essere recuperato.

Anche con un carico leggermente sotto la capacità teorica, le code possono incrementarsi e i ritardi aumentare. Si consideri una situazione in cui i pacchetti arrivano con una distribuzione casuale con un tasso medio di l pacchetti al secondo; i pacchetti hanno lunghezza casuale e possono essere

inviati sul collegamento con un tasso di servizio medio di m pacchetti al secondo. Supponendo che la distribuzione di servizio e quella di arrivo siano distribuzioni di Poisson (chiamata M/M/1 nella teoria delle code, dove "M" sta per Markov, cioè Poisson), usando la teoria delle code si può dimostrare che il ritardo medio *T* osservato da un pacchetto è:

$$T = \frac{1}{\mu} \times \frac{1}{1 - \lambda/\mu} = \frac{1}{\mu} \times \frac{1}{1 - \rho}$$

dove r = l/μ rappresenta l'utilizzo della CPU. Il primo fattore, 1/μ, rappresenta il tempo di servizio in assenza di competizione. Il secondo fattore indica il rallentamento causato dalla competizione con altri flussi. Per esempio, se *X* = 950.000 pacchetti al secondo e μ = 1.000.000 pacchetti al secondo, allora r = 0,95 e il ritardo medio osservato da ogni pacchetto sarà di 20 μs invece di 1 μs. Questo tempo comprende sia il tempo di accodamento sia il tempo di servizio, come si può osservare quando il carico è molto basso (l/μ ≈ 0). Per esempio se lungo il percorso del flusso ci fossero 30 router, il solo ritardo di accodamento causerebbe un ritardo di 600 μs.

Un metodo per correlare le specifiche del flusso alle risorse del router che corrispondono a garanzie di prestazioni sulla larghezza di banda e sul ritardo è dato da Parekh e Gallagher (1993, 1994). Si basa su sorgenti di traffico che fanno traffic shaping con token bucket (*R*, *B*) e WFQ ai router. A ogni flusso è dato un peso WFQ *W*, abbastanza grande da esaurire il tasso *R* del token bucket, come mostrato nella Figura 5.33. Per esempio, se il flusso ha un tasso di 1 Mbps e se il router e i collegamenti in uscita hanno una capacità di 1 Gbps, il peso per il flusso deve essere maggiore di 1/1000 del totale dei pesi di tutti quei flussi che transitano dal router per andare verso quel collegamento in uscita. Ciò assicura al flusso una banda minima garantita. Se non gli si può dare un tasso sufficientemente alto, il flusso non può essere ammesso.

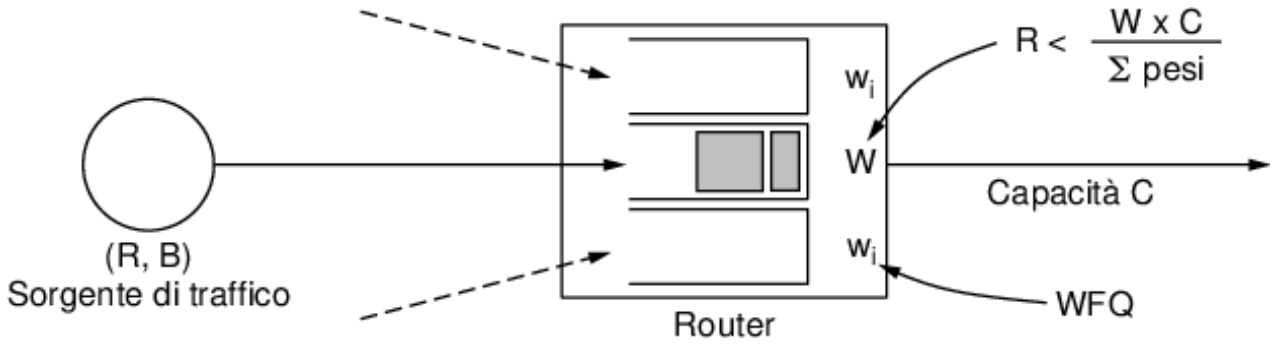


Figura 5.33 Garanzie di banda e ritardo con token bucket e WFQ.

Il ritardo di accodamento più grande che il flusso vedrà è una funzione della dimensione del picco del token bucket. Consideriamo i due casi estremi. Se il traffico è regolare, senza picchi, i pacchetti verranno drenati dal router man mano che arrivano. Non ci sarà alcun ritardo di accodamento (ignorando gli effetti di suddivisione in pacchetti). D'altra parte, se il traffico ha un profilo irregolare, un picco di dimensione massima, *B*, può arrivare improvvisamente al router; in questo

caso il ritardo di accodamento massimo, *D*, sarà il tempo impiegato per scaricare questo picco sulla banda garantita, o *B*/*R* (di nuovo ignorando gli effetti di suddivisione in pacchetti). Se questo ritardo è troppo grande, il flusso deve richiedere più banda alla rete.

Queste garanzie sono vincolanti; i token bucket limitano i picchi della sorgente e il fair queueing isola la larghezza di banda data a flussi distinti. Questo significa che il flusso avrà la garanzia di banda e ritardo, indipendentemente dal comportamento degli altri flussi concorrenti sul router; tali flussi non possono rompere il contratto, neppure inviando meno per poi spedire tutto in una volta.

Inoltre il risultato vale per un percorso attraverso più router in qualsiasi topologia di rete. Ogni flusso ottiene una capacità minima, perché tale capacità è garantita a ogni router. La ragione per cui ciascun flusso ottiene un ritardo massimo è più sottile: nel caso peggiore in cui un picco di traffico colpisca il primo router e sia in concorrenza con il traffico di altri flussi, sarà ritardato fino al ritardo massimo di *D*. Tuttavia questo ritardo smusserà il picco. A sua volta questo significa che il picco non subirà ulteriori ritardi di accodamento ai router successivi. Il ritardo di accodamento complessivo può quindi essere massimo.

Servizi integrati

Tra il 1995 e il 1997 IETF dedicò molte energie allo sviluppo di un'architettura per la trasmissione dei flussi di dati multimediali. Il risultato di questo lavoro fu raccolto in più di venti RFC, a partire dagli RFC 2205-2212 con il nome generico di **servizi integrati** (*integrated services* o *int-serv*) ed era rivolto alle applicazioni unicast e multicast. Un esempio di applicazione unicast è rappresentato dal singolo utente che scarica un video clip da un sito di notizie; un esempio di applicazione multicast è rappresentato da un gruppo di stazioni televisive digitali che trasmettono i loro programmi, sotto forma di flussi di pacchetti IP, a molti ricevitori sparsi in più locazioni. Questo paragrafo si concentra soprattutto sulla trasmissione multicast, poiché quella unicast può essere considerata un caso speciale della multicast.

In molte applicazioni multicast i gruppi possono cambiare dinamicamente; per esempio possono esserci persone che, dopo essersi unite a una videoconferenza, prese dalla noia, decidono di guardare un telefilm o il canale sportivo. In queste condizioni l'approccio che fa riservare in anticipo la banda alle sorgenti non funziona bene, perché sarebbe necessario che ogni sorgente tenesse traccia di tutti gli ingressi e le uscite del pubblico. In un sistema progettato per trasmettere la televisione a milioni di abbonati non funzionerebbe affatto.

RSVP – resource reservation protocol

La parte più importante dell'architettura dei servizi integrati visibile all'utente è **RSVP** (*resource reservation protocol*), descritto negli RFC 2205-2210 e in altri documenti. Questo protocollo si occupa delle prenotazioni; altri protocolli sono utilizzati per trasmettere i dati. RSVP permette a più sorgenti di trasmettere a diversi gruppi di destinatari e a singoli destinatari di cambiare canale liberamente; inoltre ottimizza l'uso della banda, eliminando contemporaneamente le congestioni.

Nella sua forma più semplice il protocollo utilizza il routing multicast basato su strutture di tipo spanning tree (descritte precedentemente). Ogni gruppo riceve un indirizzo di gruppo. Per trasmettere a un gruppo, una sorgente inserisce l'indirizzo del gruppo nei suoi pacchetti; in questo modo l'algoritmo di routing multicast (che non fa parte di RSVP) costruisce uno spanning tree che

copre tutti i membri del gruppo. L'unica differenza rispetto alla normale trasmissione multicast è rappresentata da qualche informazione in più, trasmessa periodicamente in modalità multicast al gruppo, per avvertire i router lungo la struttura ad albero di aggiornare alcune strutture dati nella loro memoria.

Come esempio si consideri la rete mostrata nella Figura 5.34(a). Gli host 1 e 2 sono sorgenti multicast, gli host 3, 4 e 5 sono destinatari multicast. In questo esempio le sorgenti e i destinatari sono disgiunti, ma in generale i due gruppi possono sovrapporsi. Le Figure 5.34(b) e 5.34(c) mostrano le strutture ad albero relative agli host 1 e 2.

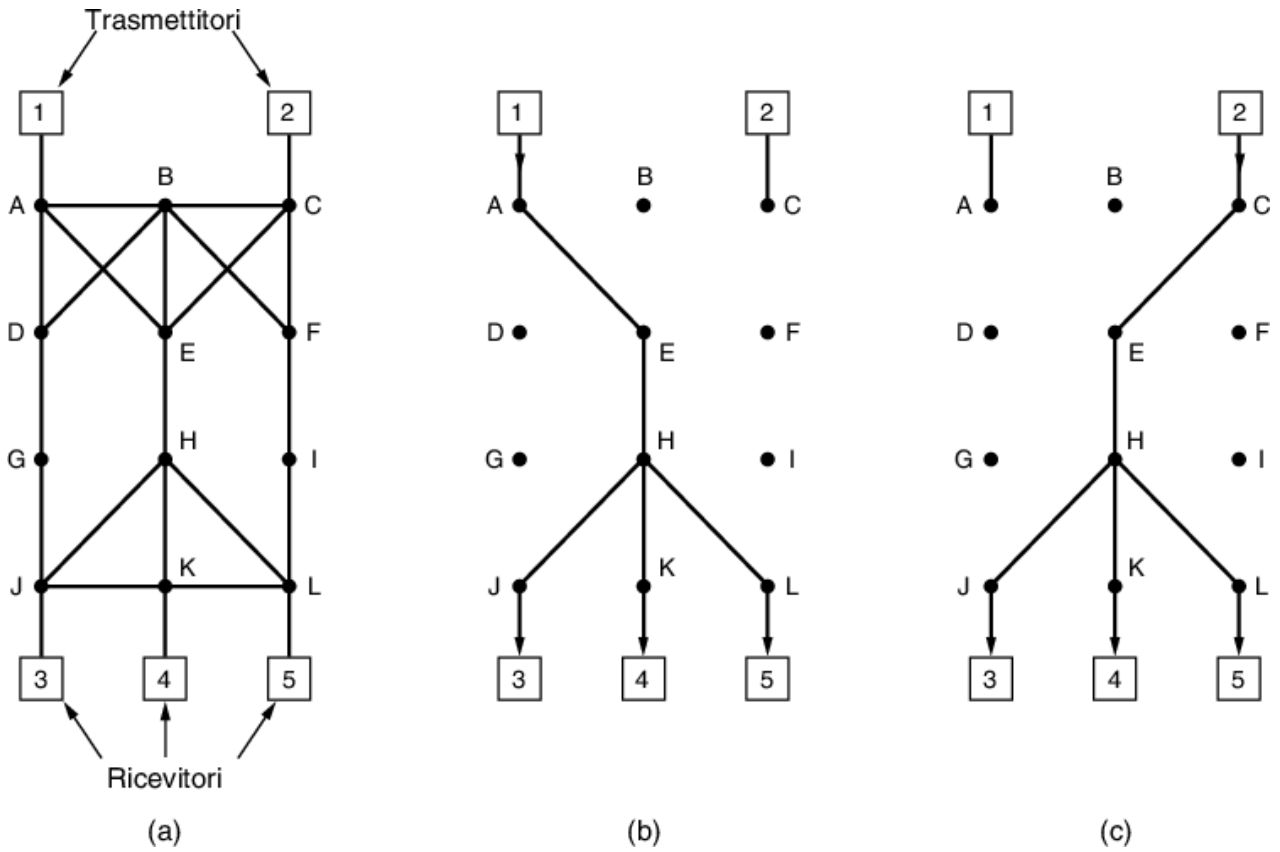


Figura 5.34 (a) Una rete. (b) Lo spanning tree multicast dell'host 1. (c) Lo spanning tree multicast dell'host 2.

Per avere una migliore ricezione ed eliminare le congestioni, tutti i destinatari di un gruppo possono inviare alla sorgente un messaggio di prenotazione attraverso la struttura ad albero. Il messaggio è propagato usando l'algoritmo di reverse path forwarding descritto precedentemente. A ogni salto il router prende nota della prenotazione e riserva la banda necessaria; se non è disponibile una banda sufficiente, il dispositivo risponde comunicando il fallimento. Quando il messaggio ritorna alla sorgente, la banda è stata prenotata su tutto il percorso.

Un esempio di prenotazione di questo tipo è mostrato nella Figura 5.35(a). L'host 3 ha richiesto un canale all'host 1; una volta attivato, i pacchetti possono scorrere da 1 a 3 senza creare congestioni. Si supponga che in seguito l'host 3 prenoti un canale di collegamento verso un'altra sorgente, per esempio l'host 2, per dare all'utente la possibilità di guardare contemporaneamente due programmi televisivi. In questo caso viene riservato un secondo percorso, come mostrato nella Figura 5.35(b). Si noti che dall'host 3 al router *E* occorrono due canali separati, perché stanno per essere trasmessi due flussi indipendenti.

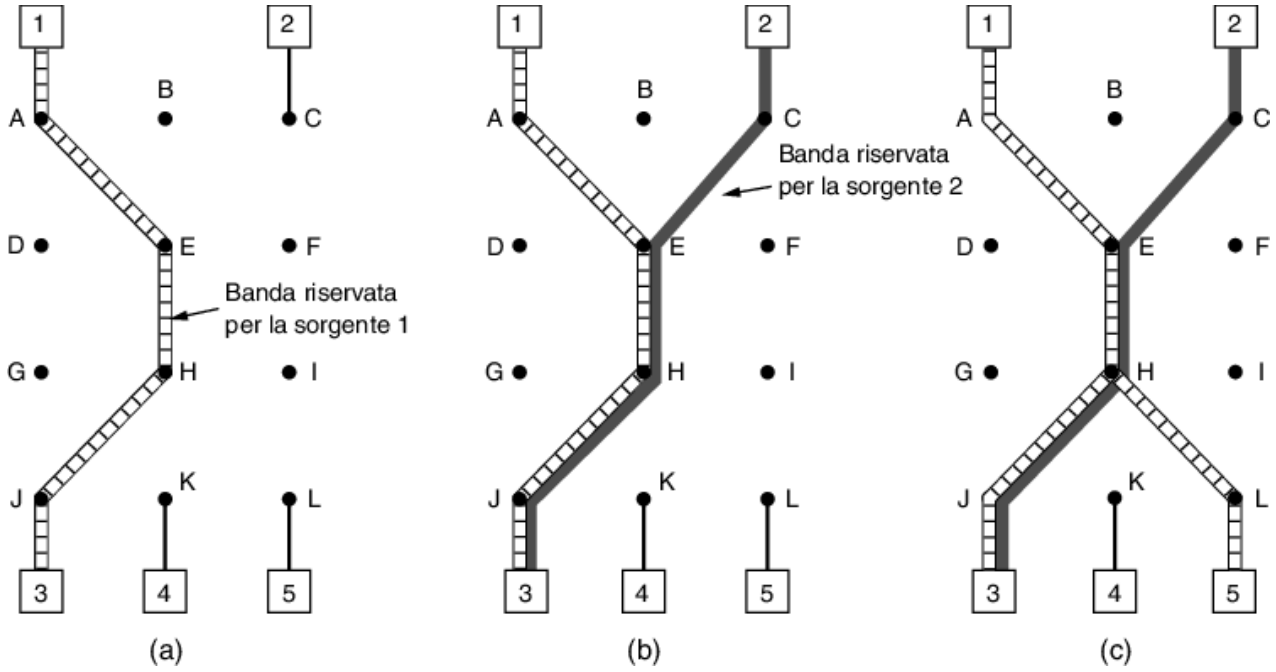


Figura 5.35 (a) L'host 3 richiede un canale verso l'host 1. (b) Successivamente l'host 3 richiede un secondo canale verso l'host 1. (c) L'host 5 richiede un canale verso l'host 1.

Infine, nella Figura 5.35(c), l'host 5 decide di guardare il programma trasmesso dall'host 1e fa un'altra prenotazione. Prima di tutto la banda dedicata è riservata fino al router *H,* ma questo router si accorge di essere già alimentato dall'host 1; perciò se la banda necessaria è già stata riservata non è necessario riservarne altra. Si noti che gli host 3 e 5 potrebbero aver chiesto bande diverse (per esempio l'host 3 potrebbe utilizzare un piccolo schermo in bianco e nero e quindi non aver bisogno delle informazioni relative al colore), perciò la capacità riservata deve essere abbastanza larga da soddisfare il destinatario più esigente. Quando fa una prenotazione, il destinatario può (facoltativamente) specificare una o più sorgenti dalle quali desidera ricevere i dati; può anche specificare se queste scelte sono fisse per la durata della prenotazione o se preferisce aver la possibilità di modificare le sorgenti in un secondo momento. I router utilizzano queste informazioni per ottimizzare la pianificazione della banda. In particolare, due ricevitori sono configurati per condividere un percorso solo se entrambi accettano di non cambiare le risorse in un secondo tempo.

Il motivo di questa strategia nel caso completamente dinamico è che la banda riservata è disaccoppiata dalla scelta della sorgente. Una volta che ha riservato la banda, il destinatario può passare a un'altra sorgente e mantenere la parte del percorso esistente che è valida per la nuova sorgente. Se, per esempio, l'host 2 sta trasmettendo diversi flussi video, l'host 3 può passare dall'uno all'altro senza cambiare la sua prenotazione: ai router non interessa sapere quale programma sta guardando il ricevente.

Servizi differenziati

I servizi integrati hanno la capacità di offrire una buona qualità del servizio a uno o più flussi perché riservano tutte le risorse necessarie lungo il percorso. Tuttavia hanno anche un lato negativo: richiedono la preventiva impostazione di ogni flusso, approccio che non si adatta bene a sistemi dove circolano migliaia o milioni di flussi. Mantengono inoltre nei router uno stato interno per ogni flusso, cosa che li rende vulnerabili ai guasti dei router. Infine le modifiche da apportare al codice del router sono considerevoli e coinvolgono complessi scambi da router a router per l'impostazione dei flussi. È per questo motivo che oggi esistono poche implementazioni di RSVP.

IETF ha pertanto concepito anche un approccio più semplice mirato alla qualità del servizio, che può essere implementato per lo più localmente in ogni router, senza richiedere l'impostazione anticipata e senza coinvolgere l'intero percorso. Questo approccio è chiamato **qualità di servizio basata sulle classi** (in contrapposizione al sistema di QoS dei servizi integrati, basata sui flussi). IETF ha standardizzato un'architettura per questa soluzione, chiamata **servizi differenziati** (*differentiated services* o *diff-serv*) e descritta negli RFC 2474, 2475 e in molti altri.

I servizi differenziati possono essere offerti da un insieme di router che costituiscono un dominio amministrativo (per esempio, un ISP o una compagnia telefonica). L'amministrazione definisce una serie di classi di servizio e le corrispondenti regole di inoltro. Se un cliente firma un contratto per servizi differenziati, i pacchetti del cliente che entrano nel dominio sono marcati con la classe a cui appartengono. Questa informazione è riportata nel campo *Differentiated services* dei pacchetti IPv4 e IPv6 (descritti nel Paragrafo 5.6). Le classi sono definite tramite un **per hop behaviour** (*comportamento a ogni salto*) perché corrispondono a come il pacchetto verrà trattato a ogni router e non a una garanzia su tutta la rete. Alcune classi avranno diritto a un servizio migliore (servizio premium) di altre (servizio normale). Il traffico appartenente a una data classe può essere obbligato a seguire un profilo specifico, per esempio un leaky bucket con un preciso tasso di servizio. Un operatore abile negli affari potrebbe richiedere un pagamento per ogni pacchetto premium aggiuntivo trasportato, oppure potrebbe consentire l'invio di massimo N pacchetti premium al mese con un contributo mensile aggiuntivo. È bene notare che questo schema, a differenza di quello adottato dai servizi integrati, non richiede alcuna impostazione anticipata, alcuna prenotazione di risorse e alcuna pesante negoziazione punto a punto per ogni flusso; ciò rende i servizi differenziati relativamente semplici da implementare.

Il servizio basato sulle classi è utilizzato anche in altri settori. Per esempio i corrieri spesso offrono un servizio in nottata e servizi di recapito in due o in tre giorni; le linee aeree offrono un servizio di prima classe, classe business e classe economica; treni su lunghe distanze spesso hanno più classi di servizio. Per i pacchetti le classi possono differire, tra l'altro, per il ritardo, il jitter e la probabilità di essere scartati in caso di congestione (ma probabilmente non per frame Ethernet più comodi).

Per sottolineare ulteriormente la differenza tra la qualità del servizio basata sui flussi e quella basata sulle classi, si consideri l'esempio seguente: la telefonia via Internet. Con uno schema basato sui flussi, ogni chiamata telefonica riceve le sue risorse e garanzie; con uno schema basato sulla classi, tutto l'insieme delle chiamate telefoniche riceve le risorse prenotate per la classe della telefonia. Queste risorse non possono essere sottratte dai pacchetti che appartengono alla classe associata alla navigazione Web o ad altre classi, ma nessuna singola chiamata telefonica riceve risorse private riservate solo a essa.

Expedited forwarding

La scelta delle classi di servizio spetta a ogni operatore, ma poiché i pacchetti sono spesso inoltrati tra reti gestite da operatori diversi, IETF sta cercando di definire classi di servizio indipendenti dalle reti. La classe più semplice e la prima esaminata in questo paragrafo si chiama **expedited forwarding** (*inoltro accelerato*) ed è descritta nell'RFC 3246.

L'idea alla base è molto semplice: sono disponibili due classi di servizio: una regolare e una accelerata. La maggior parte del traffico dovrà essere considerata regolare, mentre una piccola frazione di pacchetti potrà essere accelerata. I pacchetti accelerati dovrebbero poter attraversare la rete come se non fosse presente alcun altro pacchetto. In questo modo perdite, ritardo e jitter saranno bassi, proprio come necessita il VoIP. Una rappresentazione simbolica di questo sistema "a due condotte" è visibile nella Figura 5.36. Si noti che c'è ancora una sola linea fisica. Le due condotte logiche mostrate nella figura rappresentano un modo di riservare la banda e non una seconda linea fisica.

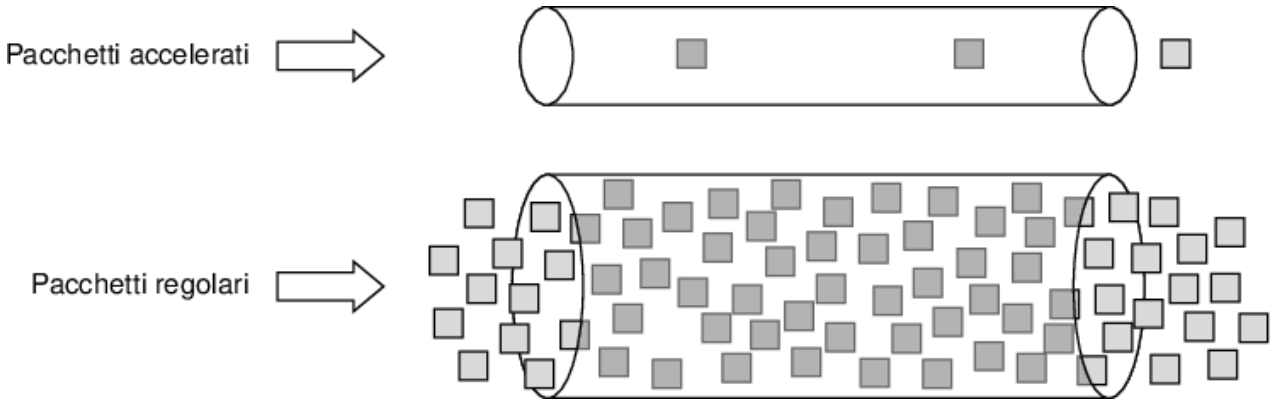


Figura 5.36 I pacchetti accelerati osservano una rete senza traffico.

Un modo per attuare questa strategia è il seguente: i pacchetti sono classificati come accelerati o regolari e marcati di conseguenza. Questo passaggio potrebbe essere fatto sul'host sorgente o nel router di ingresso alla rete. Il vantaggio di fare la classificazione all'host sorgente è che le informazioni sul flusso di appartenenza disponibili sono maggiori. Questa operazione può essere effettuata dal software di rete o anche dal sistema operativo, per evitare di dover modificare le applicazioni esistenti. Per esempio sta diventando comune che i pacchetti VoIP vengano marcati

per il servizio accelerato dagli host. Se i pacchetti passano attraverso una rete aziendale o un ISP che supporta il servizio accelerato, riceveranno un trattamento preferenziale. Se la rete non supporta il servizio accelerato, non viene comunque fatto alcun danno.

Naturalmente, se la marcatura è fatta dall'host, il router di ingresso che verosimilmente attua il policing si deve assicurare che i clienti non inviino più traffico accelerato di quello per cui hanno pagato. All'interno della rete i router possono avere due code per ogni linea in uscita, una per i pacchetti accelerati e una per quelli regolari. Quando un pacchetto arriva, viene messo nella coda appropriata. La coda accelerata ha priorità rispetto a quella regolare, gestita per esempio da uno scheduler. In questo modo i pacchetti accelerati vedono una rete scarica anche quando vi è, in effetti, un pesante carico di traffico regolare.

Assured forwarding

Assured forwarding (*inoltro assicurato*) è il nome di uno schema di gestione delle classi di servizio un po' più elaborato e descritto nell'RFC 2597. Il documento specifica quattro classi di priorità, dove ogni classe ha a disposizione le proprie risorse. Le tre classi superiori possono essere chiamate oro, argento e bronzo (più una quarta tipologia di traffico alla quale non viene data alcuna garanzia di qualità di servizio); inoltre, vengono definite tre classi di eliminazione per i pacchetti soggetti a congestione: bassa, media e alta. Nel loro complesso questi due fattori definiscono 12 classi di servizio.

La Figura 5.37 mostra un modo in cui i pacchetti potrebbero essere elaborati nello schema di inoltro assicurato. Durante la prima fase i pacchetti sono classificati in una delle quattro classi di priorità. Questa operazione potrebbe essere fatta sull'host trasmittente (come mostrato nella figura) oppure nel router di ingresso alla rete e il numero di pacchetti a priorità più alta viene limitato dall'operatore come parte del servizio.

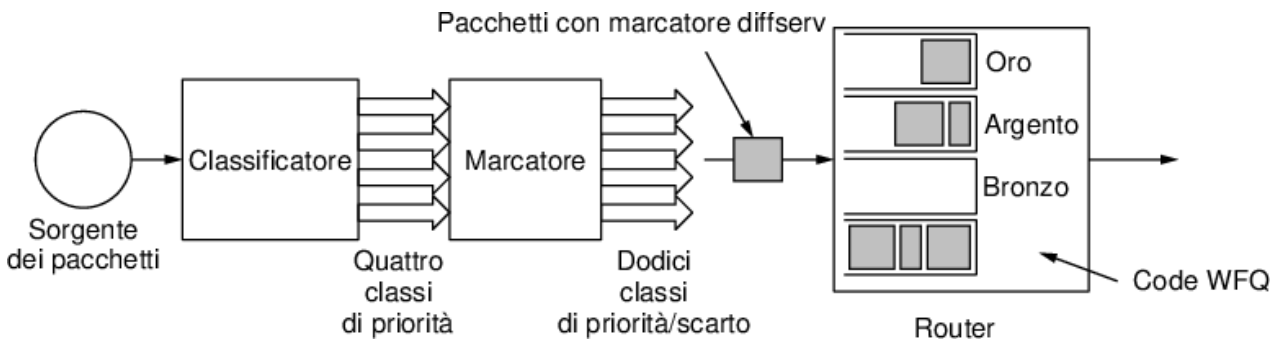


Figura 5.37 Una possibile implementazione di assured forwarding.

Il passo successivo è quello di determinare la classe di scarto di ogni pacchetto. Ciò avviene facendo passare i pacchetti di ciascuna classe di priorità attraverso un sistema di traffic policing come un token bucket, il quale permette a tutto il traffico di passare, ma identifica i pacchetti che

ricadono sotto il profilo di piccoli picchi di traffico come a bassa priorità di scarto, quelli che eccedono i piccoli picchi di traffico come a media priorità di scarto e quelli che seguono il profilo di grandi picchi di traffico con priorità di scarto elevata. La combinazione di classe di priorità e classe di scarto è quindi codificata in ogni pacchetto. Infine i pacchetti sono gestiti dai router della rete con uno scheduler che distingue le diverse classi. Una scelta comune è quella di utilizzare il WFQ per le quattro classi di priorità, fornendo alle classi più alte un peso maggiore. In questo modo, le classi superiori otterranno la maggior parte della banda, ma le classi inferiori non ne rimarranno del tutto prive. Per esempio se il peso di una classe è la metà di quello della classe immediatamente superiore, la classe superiore otterrà il doppio di banda. All'interno di una classe di priorità i pacchetti con una classe di scarto più alta saranno eliminati in maniera preferenziale tramite algoritmi come il RED, che abbiamo visto nel Paragrafo 5.3.5. RED inizierà a scartare i pacchetti all'inizio della congestione, ma prima che il router esaurisca lo spazio nel buffer; in questa fase c'è ancora spazio in cui accettare pacchetti con bassa priorità di scarto mentre si eliminano quelli che ne hanno una alta.

5.5 Internetworking

Fino a questo momento è stata implicitamente presupposta l'esistenza di una singola rete omogenea in cui ciascuna macchina utilizza lo stesso protocollo in ogni livello. Sfortunatamente questa premessa è esageratamente ottimistica; infatti esistono molte reti diverse, tra cui PAN, LAN, MAN e WAN. Abbiamo descritto Ethernet, le reti telefoniche mobili e fisse, 802.11, 802.16 e molte altre ancora. Inoltre esistono numerosi protocolli utilizzati comunemente in ogni livello. Nei paragrafi seguenti esamineremo con attenzione i problemi che sorgono quando si collegano tra loro due o più reti per formare una **internetwork** o, semplicemente, **internet**.

Se si usasse una sola tecnologia sarebbe più semplice unire le reti, come accade quando c'è un tipo dominante di rete quale Ethernet. Alcuni ipotizzano che la molteplicità di tecnologie andrà avanti finché tutti capiranno quanto [aggiungete il nome della vostra rete favorita] sia meravigliosa. Non contateci; la storia ha mostrato che è solo una speranza. Tipi differenti di rete hanno problemi diversi; per esempio Ethernet e le reti satellitari saranno sempre diverse. Il riutilizzo di sistemi esistenti, come il cavo televisivo, la rete telefonica e la rete elettrica aggiungono vincoli che portano alla divergenza delle caratteristiche di rete; l'eterogeneità è qui per rimanere.

Se pensiamo che ci saranno sempre due reti diverse tra loro, sarebbe più semplice se non dovessimo connetterle, ma anche questo non è verosimile. Bob Metcalfe ha postulato che il valore di una rete di N nodi è pari al numero di connessioni che possono essere fatte: N^2 (Gilder, 1993). Questo significa che le reti grandi hanno più valore di quelle piccole perché permettono più connessioni; ci sarà sempre un incentivo a unire le reti più piccole. Internet è il primo esempio di interconnessione. Scriveremo Internet con la "I" maiuscola iniziale per distinguerla dalle altre internet o reti interconnesse. Lo scopo di unire queste reti è permettere ai loro utenti di comunicare con gli utenti di tutte le altre reti. Quando pagate un operatore per avere il servizio Internet il conto è funzione della banda della vostra linea, ma in realtà state pagando per avere la possibilità di scambiare pacchetti con qualunque altro host connesso in Internet. Dopo tutto Internet non sarebbe così apprezzata se si potessero mandare pacchetti solo agli altri host della stessa città.

Data la grande differenza tra le reti, non è sempre semplice scambiare pacchetti tra di loro. Dobbiamo affrontare il problema dell'eterogeneità e i problemi derivanti dalla scala della rete risultante che è in continua crescita. Cominciamo col vedere come le reti differiscono tra di loro per poi considerare come superare queste differenze; quindi analizzeremo l'approccio usato da un protocollo di grande successo, IP, il protocollo di livello di rete di Internet, analizzando anche le tecniche per fare tunneling tra le reti, fare routing nelle internetwork e frammentare i pacchetti.

Differenze tra le reti

Le reti possono differire in molti modi. Alcune di queste differenze, come le diverse tecniche di modulazione o i formati dei frame, si trovano nei livelli fisico e data link e non saranno affrontate in questo capitolo. Nella Figura 5.38 vediamo invece alcune delle differenze che possono presentarsi nel livello di rete: sono proprio queste che rendono la comunicazione che avviene tra reti diverse più difficile di quella all'interno di una singola rete.

Elemento	Alcune possibilità
Servizio offerto	Orientato alla connessione oppure no
Indirizzamento	Diverse dimensioni, lineare o gerarchico
Trasmissione broadcast	Presente o assente (come pure il multicast)
Dimensione del pacchetto	Ogni rete ha un suo valore massimo
Ordinamento	Consegna ordinata oppure no
Qualità del servizio	Presente o assente, molti tipi diversi
Affidabilità	Vari livelli di perdita delle informazioni
Sicurezza	Regole di riservatezza, cifratura dei dati e altro
Parametri	Scadenze diverse, specifiche di flusso e altro
Contabilizzazione dei costi	Per tempo di connessione, pacchetto, byte o nessuna

Figura 5.38 Alcuni dei molti modi in cui le reti possono essere diverse.

Quando i pacchetti inviati da una sorgente su una rete devono attraversare una o più reti estranee prima di giungere a quella di destinazione possono presentarsi molti problemi sulle interfacce che collegano le diverse reti. Prima di tutto la sorgente deve essere in grado di definire l'indirizzo della destinazione. Che cosa si fa se la sorgente è su una rete Ethernet e la destinazione su una rete WiMAX? Assumendo che si possa specificare la destinazione WiMAX da una rete Ethernet, i pacchetti dovrebbero passare dalla rete non orientata alla connessione a una orientata alla connessione. Questo richiederebbe di impostare una nuova connessione con poco preavviso, generando un ritardo e un grande overhead se la connessione viene usata per pochi pacchetti.

Bisognerebbe anche superare molte differenze specifiche. Come si può effettuare il multicast di un pacchetto a un gruppo di cui alcuni membri stanno su una rete che non supporta il multicast? La differenza nella dimensione massima che un pacchetto può avere nelle differenti reti può essere una bella seccatura. Si può far passare un pacchetto di 8000 byte attraverso una rete che ha come dimensione massima 1500 byte? Se i pacchetti di una rete orientata alla connessione transitano su una rete non orientata alla connessione potrebbero arrivare in un ordine diverso da quello in cui sono stati inviati. Questo è qualcosa che la sorgente verosimilmente non si aspetta e che sarebbe una spiacevole sorpresa anche per il destinatario.

Questi tipi di differenze sono risolvibili con qualche sforzo. Per esempio un gateway che unisce due reti potrebbe generare pacchetti separati per ogni destinazione invece di offrire un supporto migliore per il multicast. Un pacchetto grande potrebbe essere suddiviso, inviato a pezzi e quindi riunito. I riceventi potrebbero memorizzare i pacchetti nei loro buffer e consegnarli in ordine.

Le reti possono differire anche in aspetti più difficili da conciliare. L'esempio più evidente è la qualità del servizio. Se una rete offre una qualità del servizio valida e affidabile, mentre le altre offrono solo un servizio best effort, risulterà impossibile fornire garanzie di banda e ritardo al traffico real-time da sorgente a destinazione. Ci si riuscirebbe solo qualora la rete best effort stia operando a basso carico, che non è certo l'obiettivo della maggior parte degli operatori. I meccanismi di sicurezza sono problematici, ma la crittografia per la riservatezza e l'integrità dei dati potrebbe essere implementata sopra la rete, che attualmente non la include. Infine le differenze nella procedura di contabilizzazione del traffico possono causare richieste di pagamento esagerate in cui un normale uso diventa improvvisamente costoso, come hanno scoperto gli utenti di cellulari in roaming con un piano dati.

Come connettere le reti

Per connettere reti diverse le possibili scelte di base sono due: possiamo costruire dispositivi che traducano o convertano i pacchetti da un tipo di rete a un altro o, da bravi informatici, possiamo tentare di risolvere il problema aggiungendo un livello di indirettezza per costruire un livello comune al di sopra delle reti diverse. In ogni caso i dispositivi verranno posti al confine tra le reti.

Cerf e Kahn (1974) argomentarono che il modello comune avrebbe nascosto le differenze tra le reti esistenti. Questo approccio ha riscosso un incredibile successo e il livello da loro proposto è stato infine separato in due parti: i protocolli TCP e IP. Dopo circa quattro decenni, IP è diventato il fondamento della moderna Internet. Per questo risultato Cerf e Kahn si aggiudicarono nel 2004 il Turing Award, informalmente noto come il premio Nobel dell'informatica. Il protocollo IP fornisce un formato universale per i pacchetti, riconosciuto da tutti i router e adatto a ogni rete. IP si è esteso dalle reti di calcolatori alle reti telefoniche, a quelle di sensori e ad altri minuscoli dispositivi che una volta si presumeva fossero troppo vincolati alle risorse per supportarlo.

Abbiamo visto diversi dispositivi che connettono reti, quali ripetitori, hub, switch, bridge, router e gateway. I ripetitori e gli hub si limitano a trasportare bit da un cavo all'altro. Sono per la maggior parte dispositivi analogici, che non hanno nulla a che vedere con i protocolli dei livelli più alti. Bridge e switch operano a livello data link. Possono essere usati per costruire reti, ma solo con traduzione di protocolli minori durante il processo, per esempio switch Ethernet 10, 100 e 1000 Mbps. In questo paragrafo ci concentriamo sui dispositivi di interconnessione che operano a livello di rete, vale a dire i router. Per il momento lasceremo perdere i gateway, dispositivi di interconnessione a livello più alto.

Cominciamo a capire ad alto livello come l'interconnessione con un livello di rete comune possa essere usata per interconnettere reti diverse. Una internetwork composta di reti 802.11, MPLS ed Ethernet è mostrata nella Figura 5.39(a). Si supponga che la macchina sorgente sulla rete 802.11 invii un pacchetto alla macchina di destinazione sulla rete Ethernet. Poiché sono tecnologie diverse e separate da un altro tipo di rete (MPLS) è necessario aggiungere una fase di elaborazione al confine tra le reti.

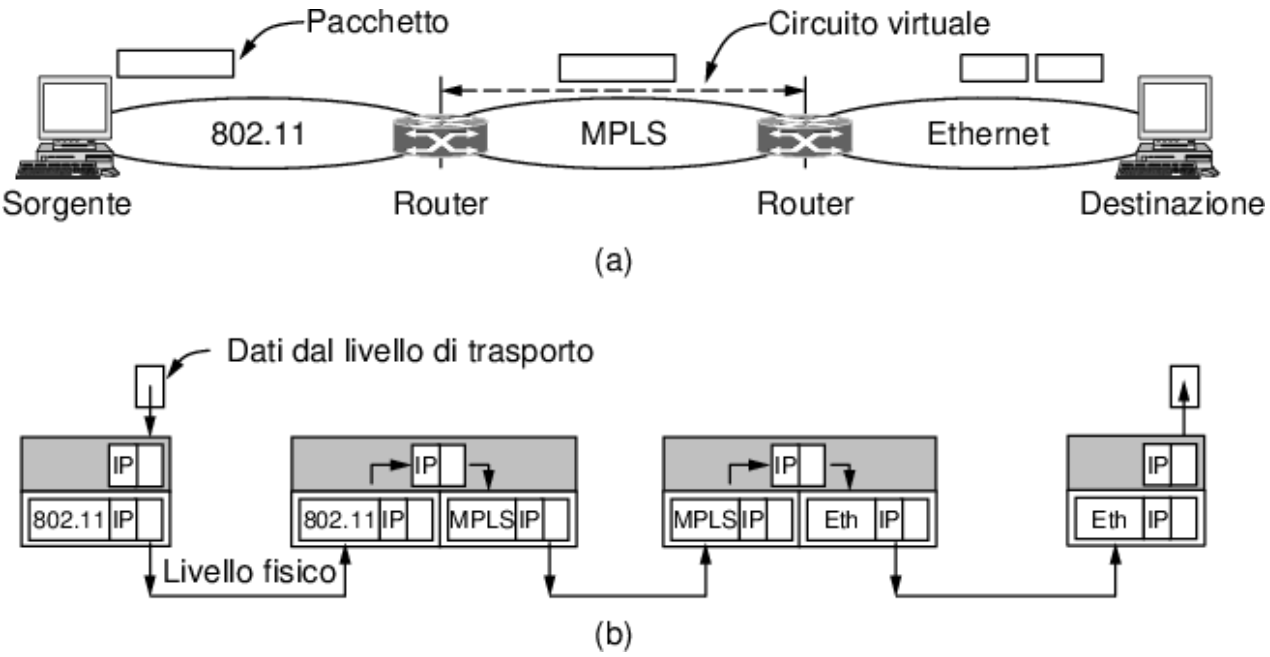


Figura 5.39 (a) Un pacchetto che attraversa reti diverse. (b) Elaborazione dei protocolli ai livelli di rete e data link.

Poiché in generale reti diverse hanno differenti forme di indirizzamento, il pacchetto porta un indirizzo a livello di rete che può identificare qualunque host attraverso le tre reti. Il primo confine raggiunto dal pacchetto si trova tra la rete 802.11 e la rete MPLS. 802.11 fornisce un servizio non orientato alla connessione, mentre MPLS ha il servizio orientato alla connessione. Per attraversare la rete è necessario impostare un circuito virtuale. Una volta che il pacchetto ha viaggiato lungo il circuito virtuale, raggiunge la rete Ethernet. A questo confine il pacchetto potrebbe essere troppo grande per essere trasportato, poiché 802.11 può usare frame più grandi di Ethernet. Per gestire tale problema, il pacchetto viene suddiviso in frammenti ognuno dei quali è spedito separatamente. Quando i frammenti raggiungono la destinazione vengono riassemblati. Ora il pacchetto ha completato il suo viaggio.

L'elaborazione a livello di protocollo per questo percorso è mostrata nella Figura 5.39(b). La sorgente accetta i dati dal livello di trasporto e genera un pacchetto con un'intestazione del livello di rete comune, che in questo caso è IP. L'intestazione contiene l'indirizzo della destinazione ultima, usato per capire che il pacchetto va inviato tramite il primo router. Quindi il pacchetto viene incapsulato in un frame 802.11 la cui destinazione è il primo router e viene trasmesso. Quando arriva al router, il pacchetto viene rimosso dal campo dati del frame e l'intestazione del frame 802.11 viene eliminata. Il router ora esamina l'indirizzo IP nel pacchetto e lo cerca nella sua tabella di routing. Basandosi su questo indirizzo, decide di inviare il pacchetto al secondo router. Per questa parte del percorso è necessario impostare un circuito virtuale MPLS verso il secondo router e il pacchetto deve essere incapsulato in un'intestazione MPLS e viaggiare lungo il circuito. Alla fine l'intestazione MPLS verrà eliminata e l'indirizzo di rete verrà consultato nuovamente per trovare il salto successivo di livello di rete. Questo non è altro che la destinazione. Poiché il pacchetto è troppo lungo per essere inviato in Internet, viene suddiviso in due parti, ognuna delle

quali viene inserita nel campo dati di un frame Ethernet e inviata all'indirizzo Ethernet della destinazione. Qui l'intestazione Ethernet è tolta da tutti i frame e i contenuti vengono riassemblati. Finalmente il pacchetto ha raggiunto la sua destinazione.

Si osservi che tra il funzionamento con il router e quello con uno switch (o bridge) c'è una differenza essenziale. Con un router il pacchetto viene estratto dal frame e l'indirizzo di rete del pacchetto è usato per decidere dove inviarlo; con uno switch (o bridge) l'intero frame viene trasportato sulla base del suo indirizzo MAC. Al contrario dei router, gli switch non devono necessariamente capire il protocollo a livello di rete usato per fare lo switch dei pacchetti.

Sfortunatamente connettere reti non è così facile come sembra. Infatti, quando i bridge vennero introdotti, furono pensati per connettere differenti tipi di reti o al massimo differenti tipi di LAN. Avrebbero dovuto farlo traducendo i frame da una LAN ai frame di un'altra LAN. Tuttavia ciò non funziona per la stessa ragione per cui connettere reti è difficile: le differenze tra le caratteristiche delle LAN, come la dimensione massima dei pacchetti e la presenza o meno di classi di priorità, sono difficili da mascherare. Attualmente i bridge sono usati soprattutto per connettere reti di ugual tipo a livello data link, mentre i router connettono reti differenti a livello di rete.

L'internetworking tra reti ha avuto molto successo nel costruire reti grandi, ma funziona solo se esiste un livello di rete comune, e nel tempo ne sono stati sviluppati molti. Mettere d'accordo tutti su un singolo formato risulta difficile, perché le aziende percepiscono come un vantaggio commerciale avere un formato proprietario che possono controllare. Altri esempi oltre a IP, che è ora il protocollo di rete quasi universale, erano IPX, SNA e Appletalk. Nessuno di questi protocolli è più molto diffuso, ma altri protocolli esisteranno sempre. L'esempio più rilevante attualmente è IPv4 e IPv6. Benché entrambi siano versioni di IP, non sono compatibili tra di loro (altrimenti non sarebbe stato necessario creare IPv6).

Un router in grado di gestire protocolli di rete multipli è chiamato **router multiprotocollo**. Questo deve tradurre i protocolli o lasciare la gestione della connessione a un protocollo di livello più alto; nessun approccio risulta completamente soddisfaciente. La connessione a livello più alto, per esempio usando TCP, richiede che tutte le reti implementino TCP (che non sempre è il caso), per cui limita la sua applicabilità ad applicazioni che usano TCP (tra queste non sono incluse, per esempio, alcune applicazioni real-time).

L'alternativa è tradurre i pacchetti tra le reti; tuttavia, a meno che i formati dei pacchetti siano parenti stretti con gli stessi campi di informazioni, tale conversione risulta sempre incompleta e spesso destinata a fallire. Per esempio gli indirizzi IPv6 sono a 128 bit; non si possono adattare a quelli di 32 bit di IPv4, indipendentemente dagli sforzi del router. Fare in modo che IPv4 e IPv6 funzionino sulla stessa rete si è dimostrato un ostacolo maggiore rispetto all'installazione di IPv6 (per essere onesti lo è stato anche far capire ai clienti perché dovrebbero scegliere IPv6). I problemi maggiori si presentano quando si tenta di tradurre tra protocolli completamente differenti, come il passaggio da un protocollo di rete orientato alla connessione a uno che non lo è. Date tutte queste difficoltà, raramente si tenta di effettuare la conversione. Il fatto che IP abbia funzionato così bene perché è servito da denominatore comune a più basso livello è anche discutibile. Richiede poco dalle reti su cui viene eseguito, ma offre solo un servizio best effort.

Tunneling

Trattare il caso generale di una connessione tra due reti diverse è estremamente difficile. Tuttavia esiste un caso particolare, molto frequente, che fortunatamente è gestibile: un host sorgente e un host di destinazione che si trovano sullo stesso tipo di rete, ma che sono separati da una rete differente. Come esempio, si consideri una banca internazionale: una filiale a Parigi usa una rete IPv6, una sede a Londra usa una rete IPv6, e le due reti sono separate da una rete Internet IPv4 come mostrato nella Figura 5.40.

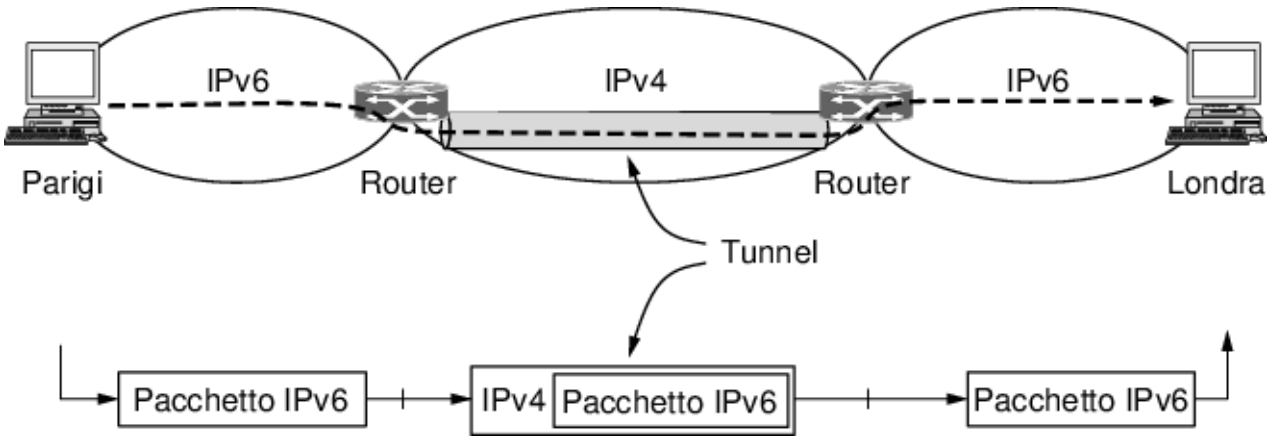


Figura 5.40 Tunneling di un pacchetto da Parigi a Londra.

La soluzione di questo problema è la tecnica di **tunneling**. Per inviare un pacchetto IP a un host nell'ufficio di Londra, un host nell'ufficio di Parigi costruisce il pacchetto contenente l'indirizzo IPv6 di Londra e lo manda al router multiprotocollo che connette la rete IPv6 di Parigi alla Internet IPv4. Quando questo router prende il pacchetto IPv6, lo incapsula in un'intestazione IPv4 indirizzata al lato IPv4 del router multiprotocollo che è connesso alla rete IPv6 di Londra. Quindi il router mette un pacchetto IPv6 all'interno di un pacchetto IPv4. Quando questo pacchetto incapsulato arriva, il router di Londra estrae il pacchetto IPv6 originale e lo invia all'host di destinazione.

Il percorso attraverso Internet IPv4 può essere visto come un lungo tunnel che si estende da un router multiprotocollo a un altro. Il pacchetto IPv6 si limita a viaggiare da un capo del tunnel all'altro nella sua graziosa scatola. Non si deve assolutamente preoccupare di avere a che fare con IPv4, così come non lo devono fare gli host di Parigi e Londra. Solo i router multiprotocollo devono capire sia i pacchetti IPv4 sia quelli IPv6. In effetti l'intero percorso da un router multiprotocollo a un altro avviene come se fosse un salto su un singolo collegamento.

Il ricorso a un'analogia potrebbe rendere il tunneling più chiaro. Si consideri una persona che guida da Parigi a Londra. In Francia la macchina si muove autonomamente, ma quando arriva al canale sulla Manica viene caricata su un treno ad alta velocità e trasportata in Inghilterra attraverso un tunnel sottomarino (alle auto non è permesso attraversare autonomamente il tunnel); nei fatti la

macchina viene trasportata, come mostrato nella Figura 5.41. Alla fine la macchina viene rimessa in libertà a percorrere le strade inglesi. Il tunneling dei pacchetti attraverso una rete estranea funziona allo stesso modo.

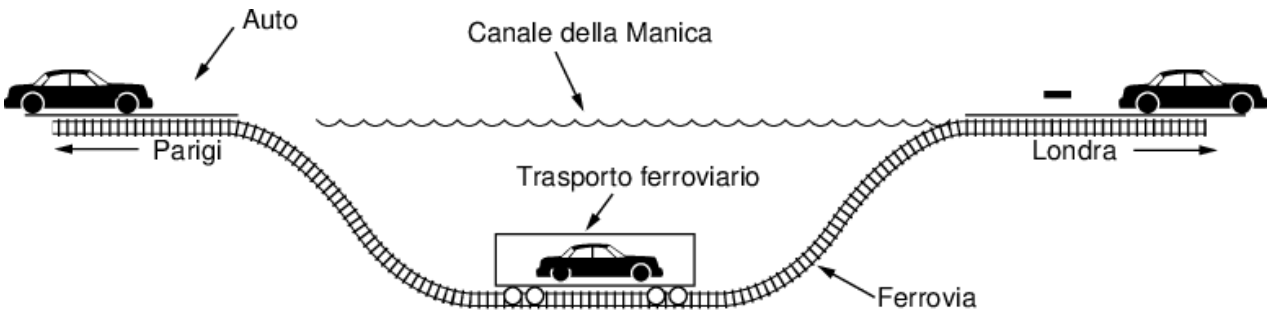


Figura 5.41 Passaggio sotterraneo di un auto dalla Francia all'Inghilterra.

Il tunneling è spesso usato per connettere host e reti isolati, anche usando altre reti, tipicamente per permettere l'installazione di un protocollo con caratteritiche innovative, come nel caso di "IPv6 su IPv4". La rete risultante è chiamata **rete di overlay** (*rete di copertura*), perché in pratica viene posta al di sopra della rete di partenza. Lo svantaggio del tunneling è che nessuno degli host della rete su cui viene fatto è raggiungibile, perché i pacchetti non possono scappare nel bel mezzo del tunnel. Tuttavia questa limitazione del tunnel è stata trasformata in un vantaggio con le **VPN** (*virtual private networks*, reti private virtuali). Una VPN è semplicemente una rete di overlay usata per fornire sicurezza. Ne parleremo nel Capitolo 8.

Routing tra reti distinte

Il routing attraverso una internet pone gli stessi problemi di base del routing attraverso una singola rete, con l'aggiunta di alcune complicazioni. Innanzitutto le reti possono usare al loro interno algoritmi di routing differenti. Per esempio una rete potrebbe usare un link state routing e un'altra un distance vector routing. Poiché gli algoritmi link state hanno bisogno di conoscere l'intera topologia, mentre a quelli distance vector non occorre, diventa difficile capire come trovare i percorsi più brevi attraverso l'intera internetwork.

Le reti gestite da operatori diversi pongono problemi ancora più gravi. Innanzitutto, gli operatori possono avere idee diverse su che cosa sia un buon cammino attraverso la rete. Un operatore potrebbe preferire il percorso con ritardo minimo, mentre un altro quello meno costoso dal punto di vista economico. Quindi gli operatori usano differenti metriche di costo per i percorsi ottimi (per esempio, millisecondi di ritardo o costo in denaro); i pesi attraverso l'intera rete non sarebbero comparabili e quindi i percorsi ottimi non sarebbero ben definiti.

Peggio, un operatore potrebbe non volere che un altro operatore conosca i dettagli dei suoi percorsi nella rete, perché pesi e percorsi potrebbero riflettere informazioni sensibili (come il costo in denaro), che rappresentano un vantaggio competitivo.

Infine la internetwork potrebbe essere molto più grande di ognuna delle reti che la compongono, richiedendo algoritmi di routing che scalino bene man mano che il numero di nodi aumenta, facendo uso di una gerarchia, anche se le singole reti non ne hanno bisogno.

Queste considerazioni portano a un algoritmo di routing a due livelli. All'interno di ogni rete viene usato un **protocollo di routing intradominio** o *interior gateway protocol* (protocollo con gateway interno), dove "gateway" è un termine un po' vecchio per "router" e potrebbe essere un protocollo link state del tipo già descritto. Attraverso le reti che formano la internetwork viene invece usato un **protocollo di routing interdominio** o *exterior gateway protocol* (protocollo con gateway esterno). Le reti al loro interno possono usare diversi protocolli intradominio, ma devono usare lo stesso protocollo interdominio. In Internet il protocollo di routing interdominio è chiamato **BGP** (*border gateway protocol*), che verrà descritto nel prossimo paragrafo.

C'è un altro fattore importante di cui parlare: ogni rete viene gestita in modo indipendente da tutte le altre e viene spesso chiamata **AS** (*autonomous system*, sistema autonomo). Un tipico esempio di AS è la rete di un ISP. In effetti, la rete di un ISP può essere composta da più di un AS se è gestita, o è stata acquistata, sotto forma di più reti, anche se di solito la differenza non è significativa.

Spesso i due livelli inter e intra dominio non sono strettamente gerarchici perché se una grande rete internazionale e una piccola rete regionale venissero pensate entrambe come una singola rete, si potrebbero avere dei percorsi altamente sub-ottimali. Tuttavia relativamente poche informazioni riguardo i percorsi all'interno delle reti vengono divulgate per trovare i percorsi attraverso l'internetwork. Questo fatto aiuta a superare tutte le complicazioni: migliora la scalabilità, permette agli operatori di scegliere liberamente i percorsi all'interno delle loro reti usando un protocollo di loro scelta e non richiede di paragonare pesi attraverso le reti o di mostrare informazioni sensibili all'esterno.

Per ora abbiamo detto poco su come vengono scelti i percorsi attraverso le reti. In Internet un fattore determinante è rappresentato dagli accordi commerciali tra gli ISP: ogni ISP può pagare, o essere pagato, da un altro ISP per trasportare il traffico. Un altro fattore consiste nel fatto che se i percorsi della internetwork richiedono l'attraversamento di confini internazionali, bisogna tenere conto che intervengono diverse legislazioni, come la legge svedese che è molto severa riguardo l'esportazione di dati personali di cittadini svedesi fuori dalla Svezia stessa. Tutti questi fattori non tecnologici sono riassunti nel concetto di **routing policy** (*politica di routing*), che governa il modo in cui i sistemi autonomi selezionano i percorsi da usare. Torneremo alle politiche di routing quando descriveremo BGP.

Frammentazione dei pacchetti

Ogni rete impone una dimensione massima ai suoi pacchetti. Questi limiti possono dipendere (tra le varie cose):

1. dall'hardware (come nel caso dei frame Ethernet)
2. dal sistema operativo (per esempio se tutti i buffer sono di 512 byte)

3. dai protocolli (dal numero di bit del campo che definisce la lunghezza di un pacchetto)
4. dalla conformità con qualche standard (inter)nazionale
5. dal desiderio di ridurre a un certo livello le ritrasmissioni causate dagli errori
6. dal desiderio di impedire a un pacchetto di occupare il canale per troppo tempo.

Il risultato di tutti questi fattori è che i progettisti di rete non sono liberi di scegliere la dimensione massima che preferiscono per i pacchetti. I payload massimi variano da 1500 byte (Ethernet) a 2272 byte (802.11). IP permette pacchetti più grandi da 65.515 byte.

Gli host, di solito, preferiscono trasmettere pacchetti grandi per ridurre l'overhead dato, per esempio, dallo spreco di banda usato per trasmettere le intestazioni. Un problema ovvio si presenta quando un pacchetto grande tenta di viaggiare attraverso una rete che supporta pacchetti di piccole dimensioni.

Una soluzione è evitare l'insorgere del problema a monte, ma la sorgente, non conoscendo il cammino, non conosce nemmeno la dimensione massima dei pacchetti, chiamata **path MTU** (*path maximum transmission unit,* massima unità trasferibile sul percorso). Se anche la sorgente la conoscesse, il problema non verrebbe risolto perché in una rete non orientata alla connessione come Internet i pacchetti sono indipendenti e i percorsi possono cambiare e quindi far cambiare anche il path MTU.

Fondamentalmente, il problema può essere risolto consentendo ai router di suddividere i pacchetti in **frammenti**, inviando poi ogni frammento separatamente come pacchetto a livello di rete. Tuttavia, come sanno tutti i genitori di figli piccoli, è molto più facile convertire un oggetto grande in piccoli frammenti che eseguire l'operazione inversa (i fisici hanno anche dato un nome a questo effetto, lo hanno chiamato "seconda legge della termodinamica"). Anche le reti a commutazione di pacchetto hanno problemi a rimettere insieme i frammenti.

Per ricostruire il pacchetto originale partendo dai frammenti si adottano due opposte strategie. In base alla prima strategia, la frammentazione che è eseguita da una rete "a pacchetti piccoli" viene resa trasparente a ogni altra rete situata più a valle lungo il percorso che il pacchetto deve attraversare per raggiungere la destinazione. Questa scelta è mostrata nella Figura 5.42(a). Quando riceve un pacchetto troppo grande, il router *G1* suddivide i dati in frammenti. Ogni frammento è indirizzato allo stesso router di uscita, *G2*, dove i pezzi sono riassemblati. In questo modo il passaggio attraverso la rete a pacchetti piccoli diventa trasparente. Le reti successive non si accorgono nemmeno che è avvenuta la frammentazione.

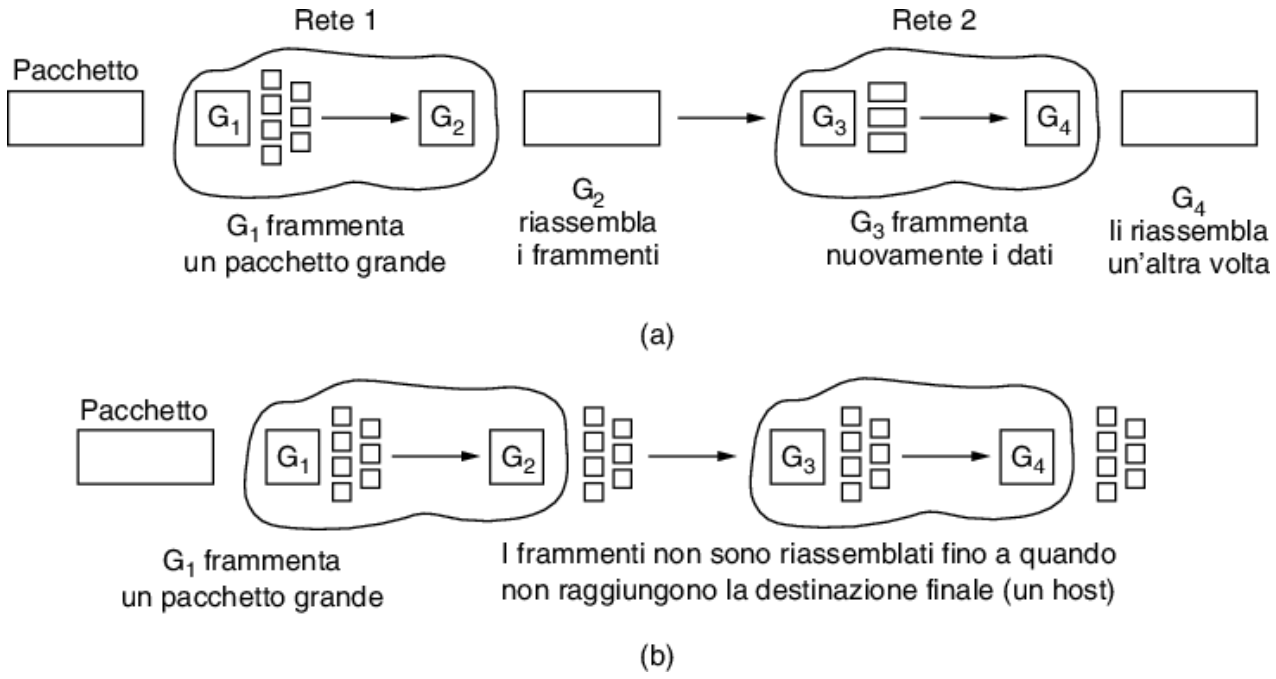


Figura 5.42 (a) Frammentazione trasparente. (b) Frammentazione non trasparente.

La frammentazione trasparente è semplice ma ha alcuni problemi. Tanto per cominciare, il router di uscita deve sapere quando ha ricevuto tutti i pezzi, perciò è necessario fornire un campo contatore o un campo che indichi la fine del pacchetto. Poi, poiché tutti i pacchetti devono uscire attraverso lo stesso router per essere riassemblati, i loro percorsi sono vincolati e, non dando la possibilità ai frammenti di seguire percorsi completamente disgiunti verso la destinazione finale, si ha un calo di prestazioni. Un problema più significativo è rappresentato dal tempo di elaborazione richiesto per memorizzare nel buffer i frammenti mentre arrivano e decidere di scartarli qualora ne mancasse qualcuno; parte di questo lavoro è inutile qualora il pacchetto attraversi più reti con dimensione massima piccola e venisse più volte frammentato e riassemblato.

L'altra strategia di frammentazione si astiene dal ricombinare i frammenti a ogni router intermedio: una volta che il pacchetto è stato suddiviso, ogni frammento è trattato come se fosse un pacchetto originale. Tutti i frammenti passano attraverso il router di uscita (o i gateway), come mostrato nella Figura 5.42(b) e la ricostruzione viene eseguita solo dall'host di destinazione.

Il vantaggio principale della frammentazione non trasparente è che i router devono fare meno lavoro; IP funziona in questo modo. Lo schema completo richiede che i frammenti siano numerati in modo tale che il flusso originale di dati possa essere ricostruito; IP assegna a ogni frammento un numero di pacchetto, uno spiazzamento assoluto all'interno del pacchetto e un flag di fine pacchetto come mostrato nella Figura 5.43. Sebbene semplice, questo schema ha alcune proprietà interessanti. I frammenti possono essere memorizzati nel buffer di destinazione nel modo giusto per essere riassemblati anche se non arrivano in ordine. I frammenti possono essere ulteriormente frammentati quando passano in una rete con un MTU ancora più piccolo, come mostrato nella Figura 5.43(c). Se non tutti i frammenti sono stati ricevuti la ritrasmissione del pacchetto può essere effettuata in pezzi diversi. Infine, i frammenti possono essere di dimensione arbitraria, fino a

un singolo byte più l'intestazione del pacchetto. In tutti i casi la destinazione semplicemente usa il numero di pacchetto e lo spiazzamento del frammento per mettere nella giusta posizione i dati; il flag di fine pacchetto determina quando il pacchetto è completo.

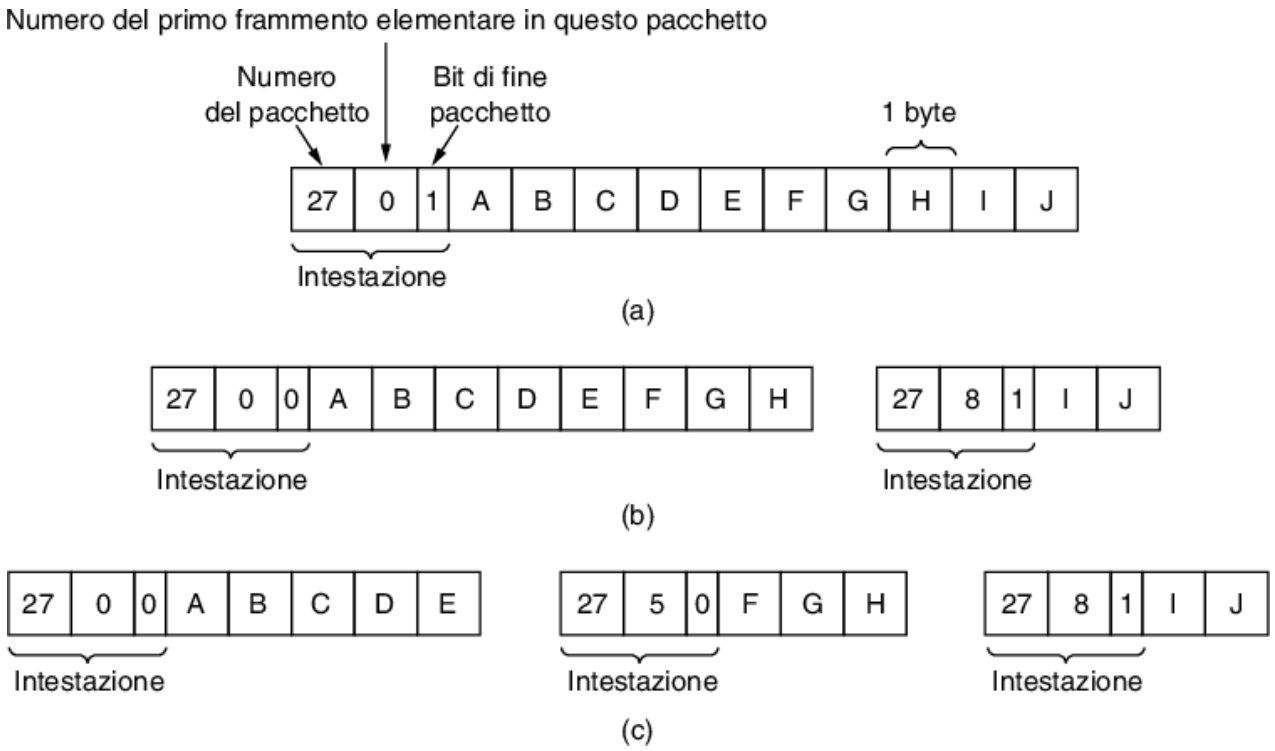


Figura 5.43 Frammentazione quando la dimensione dei dati elementari è di 1 byte. (a) Pacchetto originale contenente 10 byte di dati. (b) Frammenti dopo il passaggio attraverso una rete che supporta una dimensione massima di pacchetto di 8 byte di payload, più l'intestazione. (c) Frammenti dopo il passaggio attraverso un gateway che supporta una dimensione massima di 5.

Sfortunatamente anche questo schema pone dei problemi. L'overhead può essere più alto che con la frammentazione trasparente, poiché le intestazioni dei frammenti transitano ora anche su alcuni collegamenti dove non sono necessarie. Ma il vero problema consiste nell'esistenza dei frammenti in sé; Kent e Mogul (1987) mostrarono che la frammentazione è nociva alle prestazioni perché, esattamente come l'overhead delle intestazioni, un intero pacchetto viene perso se anche un solo frammento non arriva e la frammentazione rappresenta per gli host un fardello maggiore di quanto ci si immaginasse.

Questo ci riporta alla soluzione originale: liberarsi della frammentazione nella rete, la strategia usata nella moderna Internet. Tale processo si chiama **path MTU discovery** (*identificazione dell'MTU sul percorso*) (Mogul e Deering, 1990) e funziona nel seguente modo: ogni pacchetto IP viene mandato con l'indicazione nella sua intestazione che non è permesso effettuare alcuna frammentazione; se un router riceve un pacchetto troppo grande, genera un messaggio di errore da inviare alla sorgente[6] e scarta il pacchetto, come mostrato nella Figura 5.44. Quando la sorgente

riceve il messaggio di errore, usando le informazioni al suo interno, frammenta il pacchetto in pezzi abbastanza piccoli da essere gestiti dal router; se un router più avanti nel percorso ha un MTU ancora più piccolo, il processo viene iterato.

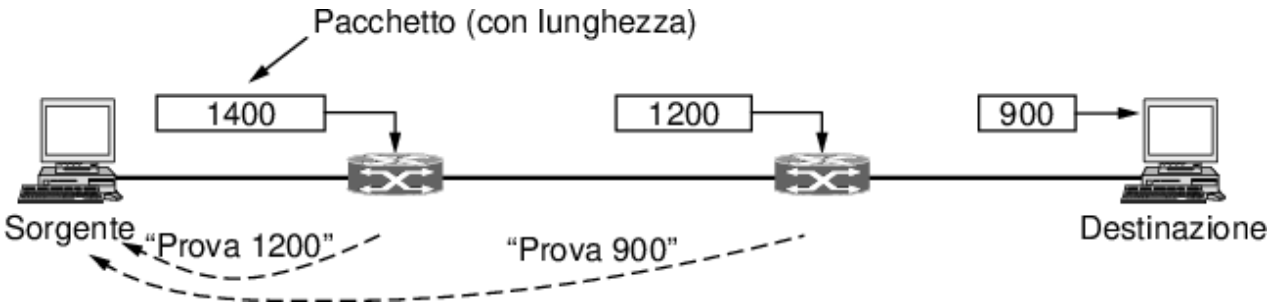


Figura 5.44 Determinazione dell'MTU su un percorso.

Il vantaggio di conoscere il path MTU è che la sorgente ora è consapevole della lunghezza dei pacchetti da inviare. Se la tabella di inoltro e di conseguenza il path MTU cambiano, la sorgente riceverà nuovi pacchetti di errore e si adatterà al nuovo percorso. Tuttavia la frammentazione è ancora necessaria alla sorgente e alla destinazione, a meno che i livelli più alti imparino il path MTU e passino il giusto quantitativo di dati a IP. TCP e IP sono usualmente implementati insieme (come TCP/IP), in modo che siano in grado di passare questo tipo di informazione. Anche se non viene fatto per tutti protocolli, si cerca comunque di muovere la frammentazione dalla rete verso gli host.

Lo svantaggio di usare questo tipo di approccio è semplicemente che, per inviare anche un pacchetto, si aggiunge un tempo di ritardo iniziale. Potrebbero essere necessari più di un ritardo di andata e ritorno per sondare il percorso e trovare l'MTU prima che qualunque dato possa essere consegnato alla destinazione; questo solleva la domanda se esistano schemi migliori. La risposta è probabilmente "sì"; si consideri uno schema in cui ogni router semplicemente tronca i pacchetti che eccedono il proprio MTU; questo assicurerebbe che la destinazione apprenda l'MTU il più rapidamente possibile (dal quantitativo di dati consegnati) e riceva comunque parte dei dati.

5.6 Il livello di rete in Internet

Prima di studiare in dettaglio il livello di rete di Internet vale la pena esaminare brevemente i principi che in passato hanno guidato lo sviluppo della sua architettura e che hanno decretato il suo successo odierno: troppo spesso, di questi tempi, la gente sembra dimenticare gli esordi. Questi principi sono elencati e descritti nell'RFC 1958, che vale la pena leggere (e dovrebbe essere obbligatorio per tutti i progettisti di protocolli, con verifica finale). Questo RFC fa largo uso delle idee riportate in (Clark, 1988; e Saltzer et al., 1984). L'elenco seguente riepiloga i dieci principi fondamentali, ordinati dal più importante al meno importante.

1. **Assicurarsi che funzioni.** Non dichiarare terminato il progetto o lo standard fino a quando diversi prototipi non abbiano comunicato con successo gli uni con gli altri. Troppo spesso i progettisti scrivono uno standard di 1000 pagine, lo fanno approvare e poi, dopo aver scoperto che è profondamente sbagliato e non funziona, ne scrivono la versione 1.1. Questo non è il modo di procedere.
2. **Mantenerlo semplice.** Nel dubbio, si utilizzi la soluzione più semplice. Guglielmo di Occam ha formulato questo principio (detto "rasoio di Occam") nel quattordicesimo secolo. Tradotto in termini moderni: lotta contro le funzionalità. Le funzionalità non strettamente necessarie vanno eliminate, specialmente se lo stesso effetto può essere ottenuto combinando altre caratteristiche.
3. **Fare scelte chiare.** Se la stessa cosa può essere fatta in modi diversi, sceglierne uno. Fare la stessa cosa in due o più modi significa cercare guai. Gli standard spesso hanno più opzioni o modalità o parametri, perché membri influenti insistono nell'affermare che il loro è il sistema migliore. I progettisti dovrebbero resistere accanitamente a questa ridondanza. Basta dire di no.
4. **Sfruttare la modularità.** Questo principio conduce direttamente all'idea di avere pile di protocolli con ciascuno dei livelli indipendente da tutti gli altri. In tal modo, se le circostanze richiedono la modifica di un modulo o di un livello, l'intervento apportato a un elemento non modificherà gli altri elementi.
5. **Aspettarsi l'eterogeneità.** In qualsiasi rete di grandi dimensioni si usano diversi tipi di hardware, di attrezzature di trasmissione e di applicazioni. Per gestire tutti questi elementi, l'architettura di rete deve essere semplice, generale e flessibile.
6. **Evitare opzioni e parametri statici.** Se i parametri sono inevitabili (come nel caso della dimensione massima del pacchetto), invece di definire scelte fisse è meglio fare in modo che il trasmettente e il ricevente concordino un valore.
7. **Mirare a un buon progetto; non è necessario che sia anche perfetto.** Spesso i progettisti hanno un buon progetto che però non riesce a gestire alcuni misteriosi casi speciali. Piuttosto che mettere da parte il progetto, i progettisti dovrebbero procedere con la loro idea e passare il fardello delle modifiche a quelli che hanno le esigenze strane.
8. **Essere rigorosi nella trasmissione e tolleranti nella ricezione.** In altre parole, inviare solo pacchetti che si attengono rigorosamente agli standard, ma aspettarsi che i pacchetti in arrivo non siano completamente compatibili, tentando in ogni caso di gestirli.
9. **Pensare alla scalabilità.** Se il sistema deve gestire milioni di host e miliardi di utenti in modo efficace, non è tollerabile alcun tipo di database centralizzato e il carico deve essere il più possibile distribuito tra tutte le risorse disponibili.

10. **Considerare le prestazioni e i costi.** Se una rete ha scarse prestazioni o costi oltraggiosi, nessuno la utilizzerà.

È giunto il momento di abbandonare i principi generali e iniziare a esaminare i dettagli del livello di rete di Internet. Nel livello di rete, Internet può essere vista come un insieme di reti o di **AS** (*autonomous system*) interconnessi. Non esiste alcuna vera struttura, esistono solo molte dorsali principali formate da linee a banda larga e router veloci. Le più grandi di queste dorsali sono chiamate **reti di livello 1** (*tier 1 network*), a cui si collega chiunque voglia connettersi per raggiungere il resto di Internet. Attaccati alle dorsali ci sono gli **ISP** (*Internet service provider*) che forniscono accesso Internet e abitazioni, uffici, data center, centri che ospitano batterie di server e reti regionali di medio livello. I datacenter gestiscono la maggior parte dei contenuti inviati su Internet. Connessi alle reti regionali ci sono altri ISP, LAN di molte università, aziende e altre reti di confine. La Figura 5.45 mostra uno schema di questa organizzazione quasi gerarchica.

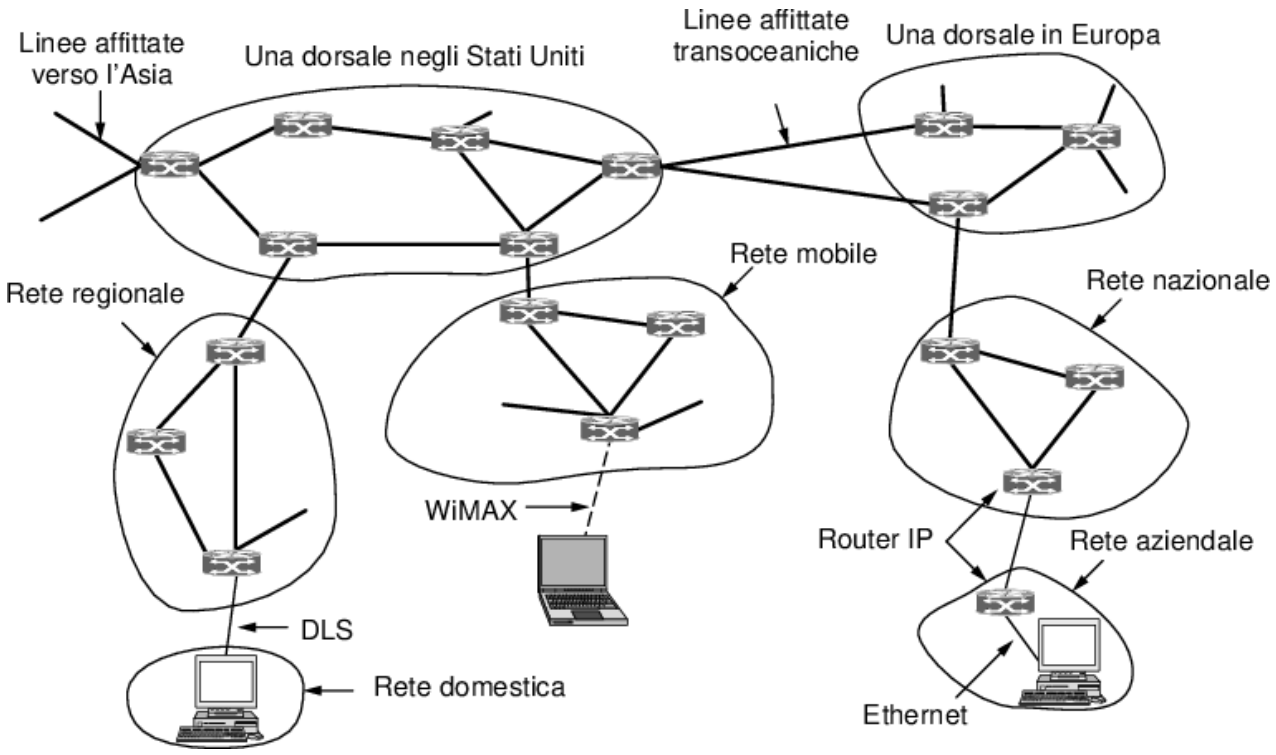


Figura 5.45 Internet è una collezione di molte reti interconnesse tra loro.

Il collante che tiene unita Internet è il protocollo a livello di rete: **IP** (*Internet protocol*). A differenza della maggior parte dei protocolli di livello di rete più vecchi, IP è stato progettato fin dall'inizio pensando alla comunicazione tra reti. Un buon modo di pensare al livello di rete è il seguente: il suo compito è fornire un servizio di tipo best effort (perciò non garantito) per trasportare i datagrammi inviati da una sorgente a una destinazione, senza tener conto della posizione delle macchine che potrebbero trovarsi su reti diverse e della eventuale presenza di reti intermedie poste tra loro.

La comunicazione in Internet funziona in questo modo: il livello di trasporto prende i flussi di dati e li divide in modo tale da poter essere inviati in pacchetti IP. In teoria, ogni pacchetto può essere grande 64 KB, ma all'atto pratico la loro dimensione di solito non supera i 1500 byte (così possono essere inseriti in un frame Ethernet). I router inoltrano ogni pacchetto attraverso Internet lungo il percorso da un router all'altro finché non viene raggiunta la destinazione, dove il livello di rete passa i dati al livello di trasporto, che li dà al processo destinatario.

Nell'esempio della Figura 5.45 un pacchetto generato dall'host della rete domestica deve attraversare quattro reti e molti router IP per raggiungere la rete aziendale dell'host di destinazione. Questo non è inusuale nella pratica, anzi, spesso il numero di reti attraversate è maggiore. C'è anche molta ridondanza di connettività in Internet, perché le dorsali e gli ISP si connettono tra di loro in più punti. Questo significa che ci sono molti percorsi possibili tra due host e sono i protocolli di routing per IP che devono decidere quali utilizzare.

Protocollo IP versione 4

Il punto giusto per iniziare lo studio del livello di rete in Internet è l'esame del formato dei datagrammi IP. Un datagramma IPv4 è costituito da una parte di intestazione e da un corpo (il payload). L'intestazione ha una parte fissa di 20 byte e una parte opzionale di lunghezza variabile; la Figura 5.46 mostra il suo formato. I bit vengono trasmessi in ordine da sinistra a destra e dall'alto verso il basso, a partire dal bit di ordine più elevato del campo *Version* (l'ordine dei byte nella rete è "big endian", su macchine "little endian"; come per i computer che usano processori Intel x86 è necessario eseguire una conversione software sia in ricezione sia in trasmissione). A posteriori il little endian sarebbe stato una scelta migliore, ma quando IP venne progettato nessuno immaginava che sarebbe diventato dominante.

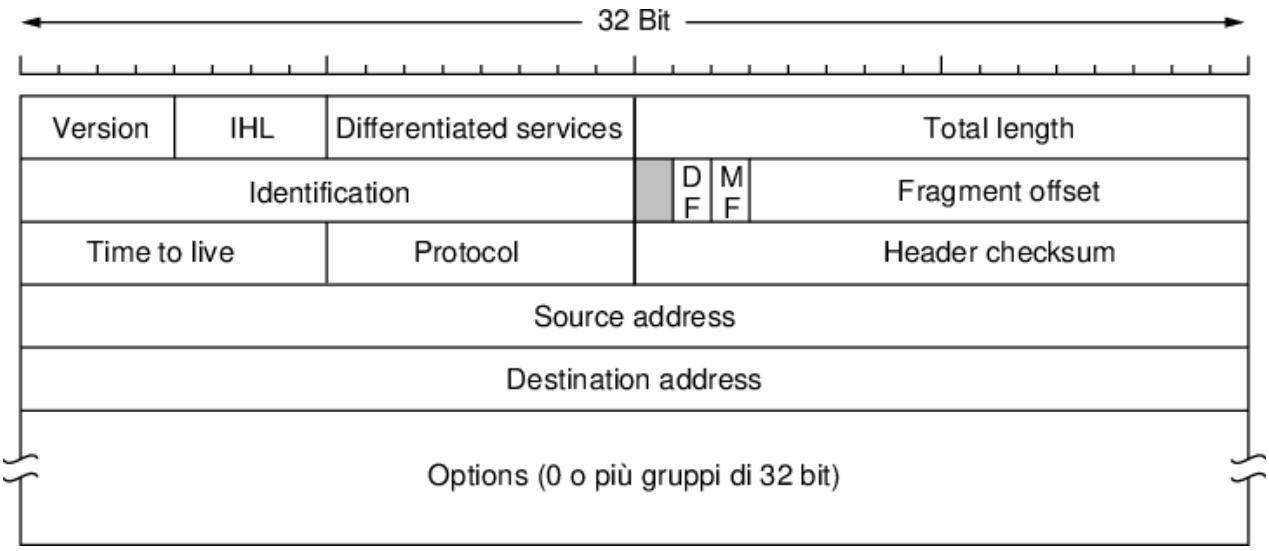


Figura 5.46 Intestazione di IPv4 (Internet Protocol).

Il campo *Version* (versione) tiene traccia della versione del protocollo usato per il datagramma. La versione 4 attualmente domina Internet, motivo per cui iniziamo la discussione da essa. Includere la versione all'inizio di ogni datagramma rende possibile effettuare una transizione tra versioni in un lungo periodo di tempo.[7] In effetti IPv6, la versione successiva di IP, venne definita più di 10 anni fa, ma solo ora inizia a essere adottata; la descriveremo nel seguito di questo paragrafo. Alla fine il suo impiego diventerà obbligatorio quando ognuno dei circa 2^{31} abitanti della Cina avrà un computer da tavolo o un portatile e un telefono IP.
Poiché la lunghezza dell'intestazione non è costante, l'intestazione contiene un campo *IHL* che indica la lunghezza dell'intestazione espressa in parole di 32 bit: il valore minimo, 5, si applica quando non sono presenti altre opzioni; il valore massimo di questo campo a 4 bit è 15, quindi la dimensione massima dell'intestazione è 60 byte e il campo *Options* (opzioni) non può occupare più di 40 byte. Alcune opzioni, come quella che registra il percorso seguito dal pacchetto, hanno bisogno di più di 40 byte e quindi non possono usare questo campo.

Il campo *Differientated services* (servizi differenziati) è uno dei pochi campi che nel corso degli anni ha (leggermente) cambiato il suo significato. Originariamente era chiamato campo *Type of service* (tipo di servizio). Era (ed è ancora) usato per distinguere diverse classi di servizio; sono ammesse svariate combinazioni di affidabilità e velocità. Per la voce digitalizzata la velocità di trasmissione prevale sulla precisione di consegna, mentre nel trasferimento dei file la trasmissione senza errori è una qualità più importante della velocità. Il campo *Type of service* forniva 3 bit per segnalare le priorità e altri tre permettevano all'host di indicare l'interesse nei confronti del ritardo, capacità di tra-sporto e affidabilità. Tuttavia, poiché nessuno veramente conosceva che cosa fare con questi bit ai router, per molti anni rimasero inutilizzati. Quando vennero progettati i servizi differenziati, IETF ha riutilizzato questo campo. Ora i sei bit sono utilizzati per indicare la classe di servizio di appartenenza del pacchetto; abbiamo già descritto in questo capitolo i servizi accelerati e quelli assicurati. Gli ultimi 2 bit sono usati per trasportare informazioni esplicite su possibili congestioni, come quando un pacchetto vede una congestione durante il percorso. Abbiamo già descritto la notifica esplicita di congestione come parte del controllo di congestione in questo stesso capitolo.

Total length (lunghezza totale) tiene conto di tutto il contenuto del datagramma, intestazione e dati; la lunghezza massima è 65.535 byte. Oggi questo limite superiore è tollerabile, ma le reti gigabit del futuro potrebbero richiedere datagrammi più grandi.

Il campo *Identification* (identificazione) serve all'host di destinazione per determinare a quale datagramma appartiene il frammento appena arrivato. Tutti i frammenti di un datagramma contengono lo stesso valore di *Identification.*

Subito dopo c'è un bit inutilizzato, il che è sorprendente, in quanto lo spazio disponibile nell'intestazione IP è estremamente scarso. Come pesce d'aprile Bellovin (2003) ha proposto di usarlo per rilevare il traffico dannoso. Questo semplificherebbe notevolmente la sicurezza, perché i pacchetti col bit "cattivo" sarebbero riconosciuti come inviati per un attacco di sicurezza e semplicemente scartati. Purtroppo la sicurezza della rete non è così semplice.

Seguono poi due campi lunghi 1 bit per la frammentazione. *DF* è l'acronimo di "*don't fragment",* ossia "non frammentare"; rappresenta un ordine che impone ai router di non dividere in frammenti il datagramma. Originariamente venne pensato per host non in grado di rimettere insieme i pezzi. Ora è parte del processo di scoperta dell'MTU di un percorso: il pacchetto più

grande che può essere trasmesso lungo un certo percorso senza essere frammentato. Contrassegnando il datagramma con il bit *DF*, il trasmittente sa che i dati arriveranno in un unico pezzo o gli verrà inviato un messaggio di errore.

MF è l'acronimo di "*more fragments",* ossia "altri frammenti". Tutti i frammenti, tranne l'ultimo, hanno questo bit impostato a 1. È fondamentale sapere quando sono arrivati tutti i frammenti di un datagramma.

Fragment offset (spiazzamento del frammento) indica la posizione del frammento nel datagramma corrente. Tutti i frammenti tranne l'ultimo di un datagramma devono essere multipli di 8 byte, che è la dimensione del frammento elementare. Poiché sono disponibili 13 bit, ci possono essere al massimo 8192 frammenti per ogni datagramma, di conseguenza la lunghezza massima di un datagramma è pari al limite del campo *Total length.* Insieme a *Identification*, *MF* e *Fragment offset* sono usati per implementare la frammentazione come descritto nel Paragrafo 5.5.5. Il campo *Time to Live* (tempo di vita) è un contatore utilizzato per limitare la vita di un pacchetto. Originariamente contava il tempo in secondi, permettendo una vita massima di 255 secondi. Il contatore deve essere decrementato a ogni salto e dovrebbe esserlo più volte quando un pacchetto rimane per lungo tempo accodato in un router. Nella realtà, però, il valore riflette solo il numero di salti. Quando raggiunge lo zero, il pacchetto è scartato e un pacchetto di avviso è inviato all'host sorgente. Questa funzionalità impedisce ai datagrammi di girare a vuoto per sempre, evento che potrebbe accadere in caso di danneggiamento delle tabelle di routing.

Quando ha assemblato un datagramma completo, il livello di rete ha bisogno di sapere che cosa farne. Il campo *Protocol* (protocollo) indica quale processo di trasporto è in attesa di quei dati. TCP è una possibilità, ma ci sono anche UDP e altre soluzioni. La numerazione dei protocolli è valida globalmente in Internet. I numeri di protocollo e gli altri numeri assegnati sono stati elencati in passato nel documento RFC 1700, ma oggi sono contenuti in un database che può essere scaricato dal sito *www.iana.org*. Dato che l'intestazione trasporta informazioni vitali come gli indirizzi, ha quindi un proprio checksum di protezione: il campo *Header checksum* (checksum dell'intestazione). L'algoritmo somma tutti i gruppi di 16 bit appena arrivano usando l'aritmetica in complemento a uno e poi prende il complemento a uno del risultato. Per gli scopi di questo algoritmo, si suppone che *Header checksum* sia zero all'arrivo. Questo checksum aiuta a rilevare gli errori durante il percorso del pacchetto. Si noti che *Header checksum* deve essere ricalcolato a ogni salto, perché almeno un campo cambia sempre (il campo *Time to Live*), ma alcuni trucchi permettono di accelerarne l'elaborazione.

Source address (indirizzo sorgente) e *Destination address* (indirizzo di destinazione) indicano gli indirizzi IP delle interfacce di rete di partenza e arrivo; gli indirizzi Internet sono esaminati nel prossimo paragrafo.

Il campo *Options* (opzioni) era stato pensato come via di fuga per dare alle versioni successive del protocollo la possibilità di includere informazioni non presenti nel progetto originale, agli sperimentatori di collaudare nuove idee e ai progettisti di evitare di assegnare bit di intestazione alle informazioni usate più di rado. Le opzioni sono di lunghezza variabile. Ognuna inizia con un codice di 1 byte che la identifica; alcune sono seguite da un campo *Option length* (lunghezza dell'opzione) di 1 byte che precede uno o più byte di dati. Il campo *Options* è riempito con multipli di quattro byte. In origine erano definite cinque opzioni, elencate nella Figura 5.47.

Opzione	Descrizione
Security	Specifica il livello di segretezza del datagramma
Strict source routing	Definisce il percorso completo da seguire
Loose source routing	Elenco dei router che devono fare parte del percorso
Record route	Fa sì che ogni router aggiunga il proprio indirizzo IP
Timestamp	Fa sì che ogni router aggiunga indirizzo e ora

Figura 5.47 Alcune opzioni di IP.

L'opzione *Security* (sicurezza) indica il livello di segretezza delle informazioni. In teoria, un router militare potrebbe utilizzare questo campo per impedire il routing dei dati attraverso certi paesi considerati ostili; in realtà tutti i router lo ignorano, perciò la sua unica funzione pratica è quella di aiutare le spie a trovare più facilmente le informazioni più interessanti.

L'opzione *Strict source routing* (routing alla sorgente rigoroso) definisce il percorso completo dalla sorgente alla destinazione, attraverso una sequenza di indirizzi IP. Il datagramma deve seguire quel percorso preciso. Per lo più è utilizzato dagli amministratori di sistema per inviare pacchetti di emergenza in caso di danneggiamento dei router o per eseguire misure di tempi.

L'opzione *Loose source routing* (routing alla sorgente lasco) costringe il pacchetto ad attraversare un elenco di router specificati, nell'ordine indicato; il pacchetto può comunque attraversare anche altri router durante il suo viaggio. Normalmente, per forzare un particolare percorso, questa opzione indica solo pochi router. Per esempio, per costringere un pacchetto proveniente da Londra e diretto a Sydney a viaggiare in direzione Ovest invece che Est, questa opzione potrebbe specificare i router di New York, Los Angeles e Honolulu. Questa opzione è utilizzata soprattutto quando fattori politici ed economici impongono di attraversare o di evitare particolari paesi.

L'opzione *Record route* (registra il percorso) costringe ogni router lungo la strada ad aggiungere il proprio indirizzo IP al campo delle opzioni. Questo permette agli amministratori di sistema di individuare gli errori negli algoritmi di routing ("Come mai i pacchetti provenienti da Houston e diretti a Dallas passano prima per Tokyo?"). Ai tempi di ARPANET nessun pacchetto superava più di nove router, perciò 40 byte di opzioni erano più che sufficienti. Com'è stato spiegato precedentemente, oggi questo spazio è inadeguato.

Infine l'opzione *Timestamp* (contrassegno temporale) è simile all'opzione *Record route,* ma costringe ogni router a registrare sia i 32 bit che rappresentano l'indirizzo IP sia i 32 bit che indicano la data e l'ora di elaborazione. Anche questa opzione è usata per misure di rete.

Oggigiorno le opzioni IP non sono più considerate. Molti router le ignorano o non le elaborano in modo efficiente; sono supportate solo parzialmente e raramente utilizzate.

Indirizzi IP

La caratteristica di IPv4 è che i suoi indirizzi sono lunghi 32 bit. Ogni host o router in Internet ha un indirizzo IP utilizzato nei campi *Source address* e *Destination address* dei pacchetti IP. È importante notare che un indirizzo IP non si riferisce veramente a un host, ma a una scheda di rete, perciò quando un host è collegato a due reti, deve avere due indirizzi IP. Tuttavia la maggior parte degli host è collegata a un'unica rete e perciò ha un solo indirizzo IP, mentre i router hanno più indirizzi IP.

Prefissi

Al contrario degli indirizzi Ethernet, gli indirizzi IP sono gerarchici. Ogni indirizzo a 32 bit è composto di una parte di rete di lunghezza variabile nei primi bit e di una parte per l'host negli ultimi. La parte di rete ha lo stesso valore per tutti gli host di una singola rete, come in una LAN Ethernet. Questo significa che una rete corrisponde a un blocco contiguo di indirizzi IP; tale blocco è chiamato **prefisso**.

Gli indirizzi di rete sono scritti in **notazione decimale puntata**. In questo formato ognuno dei 4 byte è rappresentato in forma decimale con un numero che varia tra 0 e 255. Per esempio l'indirizzo esadecimale di 32 bit 80D00297 è scritto come 128.208.2.151. I prefissi sono scritti dando l'indirizzo IP più basso del blocco e la grandezza del blocco. La dimensione è determinata dal numero di bit nella porzione di rete; i bit rimanenti nella parte dedicata all'host possono variare. Questo significa che la dimensione deve essere una potenza di due. Per convenzione viene scritta dopo il prefisso dell'indirizzo IP come una barra obliqua / (slash) seguito dalla lunghezza in bit della parte dedicata alla rete. Nel nostro esempio se il prefisso contiene 2^8 indirizzi IP e quindi 24 bit sono dedicati alla rete, è scritto come 128.208.0.0/24.

Poiché la lunghezza del prefisso non può essere dedotta semplicemente dall'indirizzo IP, i protocolli di routing devono portare i prefissi ai router. A volte i prefissi sono descritti semplicemente dalla loro lunghezza, come in "/16". La lunghezza del prefisso corrisponde a una maschera binaria di 1 nella parte di rete. Quando viene scritta in questo modo è chiamata **subnet mask** (*maschera di sottorete*). Si può effettuare un'operazione di AND logico con l'indirizzo IP per estrarre la porzione di rete. Nel nostro esempio la subnet mask è 255.255.255.0. La Figura 5.48 mostra un prefisso e una subnet mask.

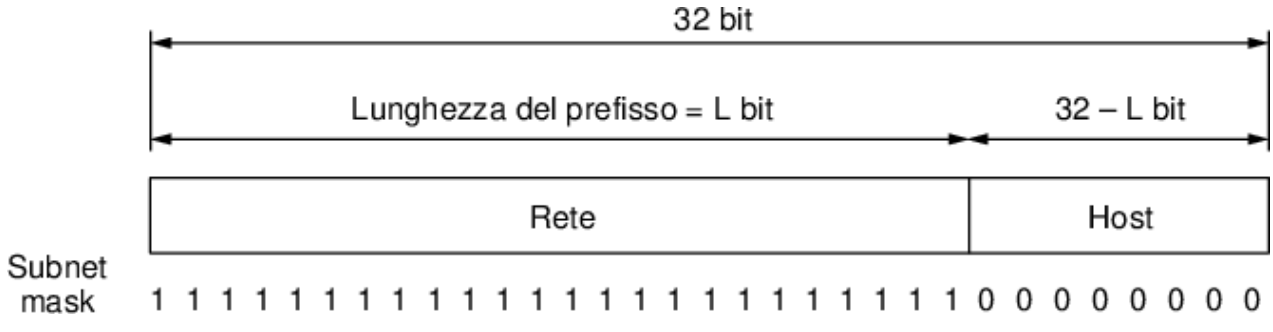


Figura 5.48 Un prefisso IP e una subnet mask.

Gli indirizzi gerarchici presentano vantaggi e svantaggi significativi. Il vantaggio principale dei prefissi è che i router possono inoltrare i pacchetti basandosi solo sulla parte di rete dell'indirizzo, fintanto che ogni rete abbia un unico blocco di indirizzi. La parte host non importa ai router, perché i pacchetti indirizzati a tutti gli host della stessa rete verranno inviati nella medesima direzione. Solo quando i pacchetti raggiungono la rete di destinazione vengono inoltrati all'host corretto. Questo schema rende le tabelle di routing molto più piccole. Si consideri che il numero di host in Internet si avvicina al miliardo. I router dovrebbero mantenere tabelle di routing veramente molto grandi. Tuttavia, usando la gerarchia, i router devono memorizzare percorsi solo per circa 300.000 prefissi.

Da una parte usare la gerarchia permette una buona scalabilità nel routing su Internet, dall'altra ha due svantaggi. Innanzitutto l'indirizzo IP di un host dipende da dove è localizzato nella rete. Un indirizzo Ethernet può essere usato ovunque nel mondo, mentre un indirizzo IP appartiene a una rete specifica e i router sono in grado di consegnare pacchetti solo a quell'indirizzo della rete. Progetti quali IP mobile sono necessari per supportare host che si muovono tra le reti, ma che vogliono mantenere gli stessi indirizzi IP.

Il secondo svantaggio consiste nel fatto che la gerarchia spreca indirizzi a meno che non sia gestita in modo opportuno. Se gli indirizzi vengono assegnati alle reti in blocchi (troppo) larghi, esisteranno (molti) indirizzi che vengono allocati, ma non vengono usati. Questa locazione non importerebbe a nessuno se ci fosse abbondanza di indirizzi. Tuttavia, già due decenni fa ci si accorse che la straordinaria crescita di Internet stava rapidamente consumando lo spazio libero di indirizzi. IPv6 è la soluzione per questo problema ma, finché non verrà completamente implementato, si continuerà a cercare di allocare gli indirizzi IP in modo estremamente efficiente.

Sottoreti

Per evitare conflitti, i numeri di rete sono gestiti da un ente no profit chiamato **ICANN** (*Internet corporation for assigned names and numbers*). ICANN ha affidato la gestione di alcune parti dello spazio di indirizzamento ad autorità regionali, che distribuiscono con parsimonia gli indirizzi IP agli ISP e alle altre aziende. Questo è il processo con cui un'azienda ottiene un blocco di indirizzi IP.

Tuttavia questo processo è solo l'inizio, perché l'assegnazione degli indirizzi IP cambia man mano che le aziende crescono. Abbiamo visto che gli algoritmi di routing basati sul prefisso richiedono che tutti gli host di una rete abbiano lo stesso numero di rete. Questa proprietà porta ad avere dei problemi quando le reti crescono. Si consideri per esempio una università che parte con un prefisso /16 da usare per i computer nella sua Ethernet del dipartimento di informatica. Un anno dopo anche il dipartimento di ingegneria elettronica vuole connettersi a Internet e subito dopo anche il dipartimento di storia dell'arte. Quali indirizzi IP dovrebbero usare questi dipartimenti? Ottenere altri blocchi richiederebbe di uscire dall'università e potrebbe risultare costoso o scomodo. Tuttavia il prefisso /16 già allocato ha indirizzi sufficienti per circa 60.000 host. Potrebbe essere inteso per consentire crescite significative ma, finché ciò non si verifica, sarebbe uno spreco allocare ulteriori blocchi di indirizzi IP alla stessa università. È necessario pensare a un'organizzazione diversa.

La soluzione consiste nel suddividere il blocco di indirizzi in più parti per usarle internamente come reti multiple, ma presentandole all'esterno come una singola rete. Questa procedura è chiamata **subnetting** e le reti (come le LAN Ethernet) che risultano dalla divisione di una rete più

grande vengono chiamate **subnet** (*sottoreti*). Come già detto nel Capitolo 1, si faccia attenzione che questo nuovo uso del termine è in conflitto con quello associato precedentemente che aveva il significato di insieme di tutti i router e linee di comunicazione di una rete. La Figura 5.49 mostra le subnet del nostro esempio. Il singolo prefisso /16 è stato suddiviso in più parti. Questa suddivisione non deve essere uniforme, ma ogni parte deve essere allineata in modo che tutti i bit nella parte più bassa, dedicata all'host, possano essere usati. In questo caso, metà del blocco (uno /17) è allocato per il dipartimento di informatica, un quarto è allocato per il dipartimento di ingegneria elettronica (uno /18) e un ottavo (uno /19) al dipartimento di storia dell'arte. I rimanenti ottavi non vengono allocati. Un modo diverso per capire come il blocco sia stato diviso è analizzare i prefissi risultanti, scritti in notazione binaria:

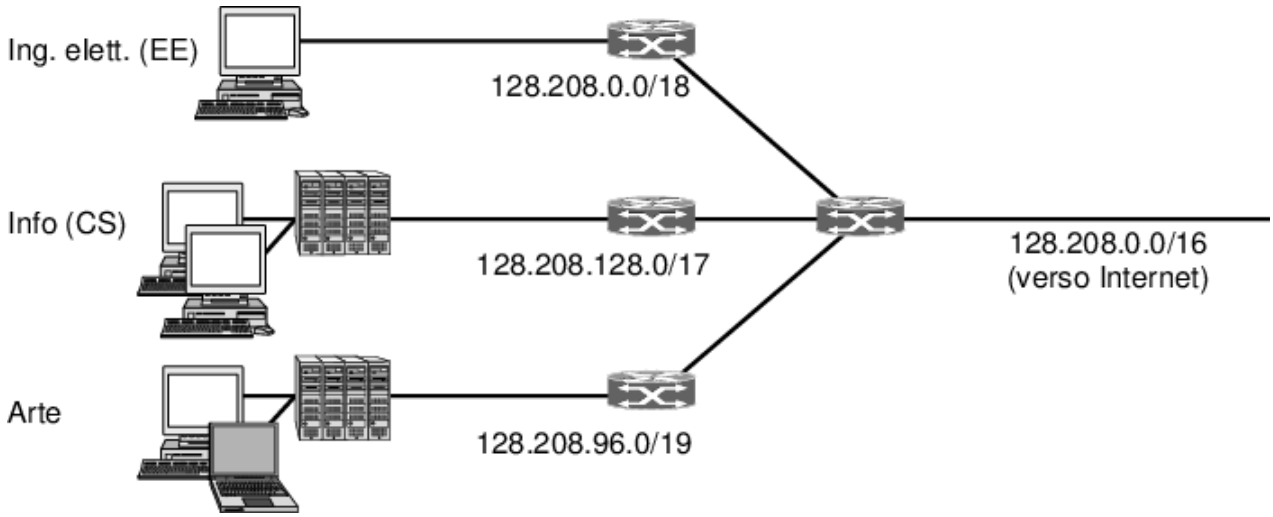


Figura 5.49 Suddivisione di un prefisso IP in reti separate tramite subnetting.

```
Informatica:    10000000 11010000 1|xxxxxxx xxxxxxxx

Ing. elettronica:    10000000 11010000 00|xxxxxx xxxxxxxx

Storia dell'arte:    10000000 11010000 011|xxxxx xxxxxxxx
```

La barra verticale (|) mostra il confine tra il numero della subnet e la parte dell'host.

Quando un pacchetto arriva al router principale, come fa questo a sapere a quale subnet deve consegnarlo? Qui entrano in gioco i dettagli dei prefissi. Un modo potrebbe essere quello che ogni router abbia una tabella con 65.536 elementi che gli indichi quale linea di uscita usare per ogni host del campus. Ma tale metodo eliminerebbe il grande vantaggio della scalabilità ottenuto usando la gerarchia. Al contrario i router semplicemente hanno bisogno di conoscere la subnet mask delle reti del campus.

Quando un pacchetto arriva, il router guarda l'indirizzo di destinazione del pacchetto e controlla a quale subnet appartiene. Per capirlo, effettua un AND logico dell'indirizzo di destinazione con la maschera di tutte le subnet e controlla se il risultato corrisponde al prefisso. Si consideri per esempio un pacchetto destinato all'indirizzo IP 128.208.2.151. Per vedere se appartiene al dipartimento di informatica, effettuiamo l'AND con 255.255.128.0, per ottenere i primi 17 bit (che sono 128.208.0.0) e vedere se corrispondono al prefisso dell'indirizzo (128.208.128.0). Non c'è corrispondenza. Controllando quindi i primi 18 bit del dipartimento di ingegneria elettronica ed effettuando l'AND con la subnet mask, si ottengono 128.208.0.0. Ora c'è corrispondenza con il prefisso dell'indirizzo, quindi il pacchetto viene inoltrato all'interfaccia che porta alla rete del dipartimento di ingegneria elettronica.

La divisione in subnet può sempre essere cambiata aggiornando le subnet mask ai router dell'università. Al di fuori della rete il *subnetting* non è visibile; in questo modo allocare una nuova subnet non richiede di contattare ICANN né di cambiare alcun database esterno.

CIDR – classless interdomain routing

Anche se i blocchi di indirizzi IP vengono allocati in maniera efficiente rimane tuttavia un problema: l'esplosione delle dimensioni delle tabelle di routing.

I router di enti e aziende ai bordi di una rete, quali le università, hanno bisogno di una informazione per ognuna delle loro subnet; queste informazioni indicano al router quale linea usare per andare verso una specifica rete. Per percorsi la cui destinazione si trovi fuori dell'organizzazione dove opera il router essi possono semplicemente usare la regola standard di inviare i pacchetti sulla linea verso l'ISP che connette tale organizzazione al resto di Internet.

Gli ISP e le dorsali al centro di Internet non godono di questo lusso; devono conoscere come raggiungere ogni rete e nessuna regola standard funziona. Questi core router sono detti essere nella **zona default-free** (zona senza default) di Internet. Nessuno sa veramente quante reti sono connesse a Internet, ma sono tante, probabilmente almeno 1 milione. Quindi le tabelle di routing possono essere veramente molto grandi. Potrebbero non sembrare così grandi per gli standard di un computer, ma si pensi che i router devono effettuare una ricerca in questa tabella per inoltrare ogni pacchetto e che quelli di grandi ISP possono inoltrare anche milioni di pacchetti al secondo. Con questi numeri sono necessari hardware specializzato e memoria veloce; un computer normale non è sufficiente.

Inoltre vari algoritmi di routing richiedono che i router si scambino informazioni sugli indirizzi che possono raggiungere. Più sono grandi le tabelle, più informazioni devono essere comunicate ed elaborate; l'aumento dei tempi di elaborazione cresce almeno linearmente con la dimensione della tabella. Più sono i dati da comunicare, più è probabile che alcune parti si perdano lungo la strada, generando instabilità.

Il problema della tabella di routing potrebbe essere risolto creando una gerarchia più ramificata, come nella rete telefonica. Per esempio si poteva immaginare di inserire in ogni indirizzo IP dei campi per indicare paese, stato o provincia, città, rete e host. In quel caso, un router avrebbe dovuto conoscere solamente come raggiungere ogni paese, gli stati o le provincie nel proprio

paese, le città nella sua provincia o stato, e le reti nella sua città. Sfortunatamente questa soluzione richiederebbe un numero di bit maggiore dei 32 usati dagli indirizzi IP e li utilizzerebbe in modo inefficiente (il Liechtenstein avrebbe lo stesso numero di bit degli Stati Uniti).

Fortunatamente per ridurre la dimensione delle tabelle di routing si può fare qualcosa: possiamo applicare lo stesso meccanismo del subnetting; i router in posizioni differenti possono riconoscere un fissato indirizzo IP come appartenente a prefissi di dimensione diversa. Tuttavia, invece di suddividere un blocco di indirizzi in subnet, in questo caso si aggregano prefissi piccoli in un unico prefisso più grande. Questo processo è chiamato **route aggregation** (*aggregazione di percorsi*). Il risultante prefisso, più grande, è chiamato a volte **supernet**, in contrasto con le subnet che sono invece una suddivisione dei blocchi di indirizzo.

Con questo metodo di aggregazione gli indirizzi IP sono contenuti in prefissi di dimensione variabile. Lo stesso indirizzo IP che un router tratta come parte di uno /22 (un blocco contenente 2^{10} indirizzi) può essere trattato da un altro router come parte di un più grande /20 (che contiene 2^{12} indirizzi). Compito di ogni router è avere la corrispondente informazione sul prefisso. Questo schema opera con il subnetting ed è chiamato **CIDR** (*classless interdomain routing*). La versione più recente è specificata nell'RFC 4632 (Fuller e Li, 2006). Il nome sottolinea il contrasto con gli indirizzi che codificano la gerarchia in classi, come spiegheremo tra breve.

Per agevolare la comprensione del funzionamento di CIDR, si consideri una situazione di esempio in cui un blocco di 8192 indirizzi IP sia disponibile a partire da 194.24.0.0. Si supponga che all'Università di Cambridge, che necessita di 2048 indirizzi, siano assegnati gli indirizzi compresi tra 194.24.0.0 e 194.24.7.255, insieme alla maschera 255.255.248.0; questo è un prefisso /21. Successivamente l'Università di Oxford chiede 4096 indirizzi; poiché un blocco di 4096 indirizzi deve trovarsi allineato a 4096 byte non è possibile assegnare al richiedente indirizzi che partono da 194.24.8.0. L'università riceve perciò gli indirizzi compresi tra 194.24.16.0 e 194.24.31.255 insieme alla subnet mask 255.255.240.0. A questo punto all'Università di Edimburgo che chiede 1024 indirizzi sono assegnati gli indirizzi compresi tra 194.24.8.0 e 194.24.11.255 insieme alla maschera 255.255.252.0. Queste assegnazioni sono riepilogate nella Figura 5.50.

Università	Primo indirizzo	Ultimo indirizzo	Totale indirizzi	Prefisso
Cambridge	194.24.0.0	194.24.7.255	2.048	194.24.0.0/21
Edimburgo	194.24.8.0	194.24.11.255	1.024	194.24.8.0/22
(disponibili)	194.24.12.0	194.24.15.255	1.024	194.24.12.0/22
Oxford	194.24.16.0	194.24.31.255	4.096	194.24.16.0/20

Figura 5.50 Una serie di assegnazioni di indirizzi IP.

Tutti i router nella zona default free ora conoscono gli indirizzi IP nelle tre reti. I router vicino alle università possono dover effettuare l'inoltro su una linea di uscita differente per ogni prefisso; quindi nella loro tabella di routing hanno bisogno di una voce per ogni prefisso. Un esempio è il router di Londra mostrato nella Figura 5.51.

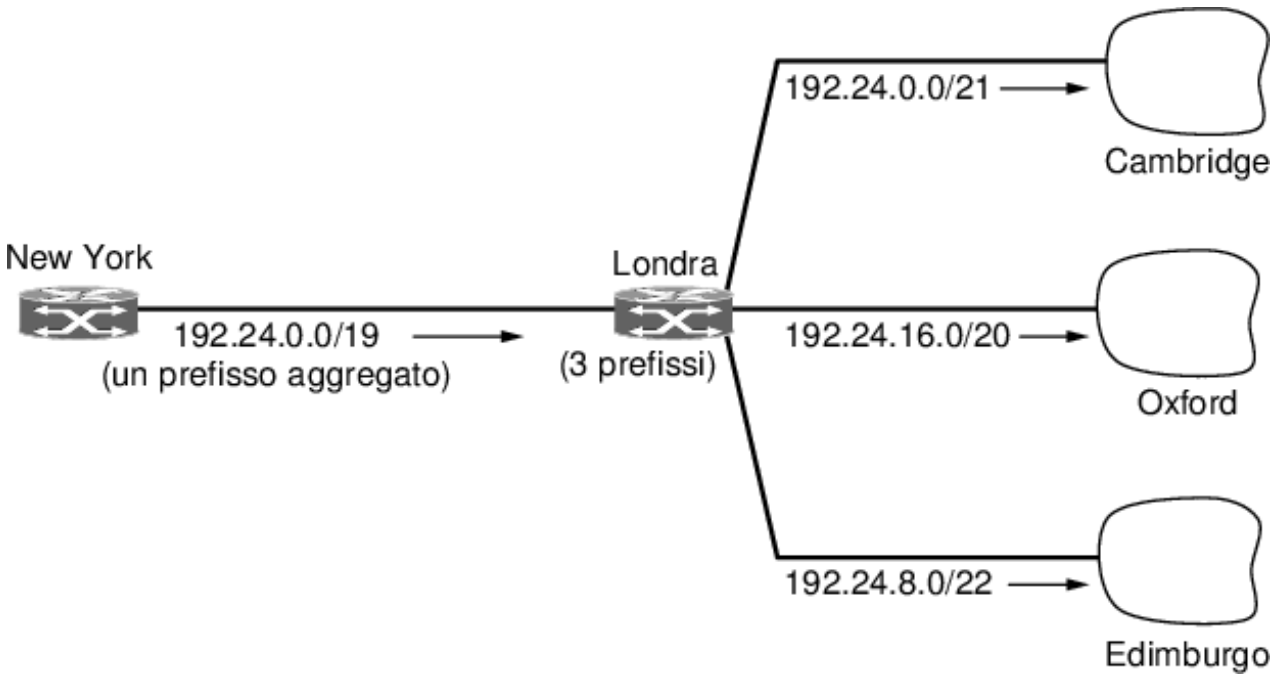


Figura 5.51 Aggregazione di prefissi IP.

Guardiamo ora queste tre università dal punto di vista di un router nella lontana New York. Gli indirizzi IP nei tre prefissi dovrebbero essere inviati da New York (dagli Stati Uniti in generale) a Londra. Il processo di routing a Londra se ne accorge e combina i tre prefissi in un'unica voce aggregata per il prefisso 194.24.0.0/19 che trasmette al router di New York. Questo prefisso contiene 8192 indirizzi, copre le tre università e gli altri 1024 indirizzi, altrimenti non allocati. Usando l'aggregazione i tre prefissi sono stati ridotti a uno solo, riducendo i prefissi che il router di New York deve comunicare e le voci della sua tabella di routing.

Quando l'aggregazione viene fatta partire, diventa un processo automatico. Dipende da quali prefissi sono allocati in Internet e non dalle azioni di un amministratore che assegna gli indirizzi alle reti. Il processo di aggregazione è pesantemente usato su tutta Internet e riduce la dimensione delle tabelle dei router a circa 200.000 prefissi.

I prefissi possono anche sovrapporsi. La regola su cui ci si basa è che i pacchetti vengono inviati nella direzione del percorso più specifico o **longest matching prefix** (*più lungo prefisso corrispondente*) che ha meno indirizzi IP. Questa strategia di routing fornisce un utile grado di flessibilità, come si vede nel comportamento del router di New York nella Figura 5.52. Questo router utilizza ancora un singolo prefisso aggregato per trasmettere traffico alle tre università di Londra. Tuttavia il blocco di indirizzi prima disponibile all'interno di questo prefisso è stato ora

allocato alla rete di San Francisco. Una possibilità sarebbe che il router di New York mantenesse quattro prefissi, inoltrando i pacchetti di tre di questi a Londra e i pacchetti del quarto a San Francisco. Invece il routing con longest matching prefix può gestire questo processo di inoltro con i due prefissi mostrati. Un prefisso è usato per dirigere il traffico di un intero blocco a Londra, mente un prefisso più specifico è usato per trasmettere una parte più grande del prefisso a San Francisco. Con la regola del prefisso più lungo gli indirizzi IP della rete di San Francisco verranno inviati su una linea di uscita verso San Francisco, mentre gli altri indirizzi IP nel prefisso più grande saranno inviati a Londra.

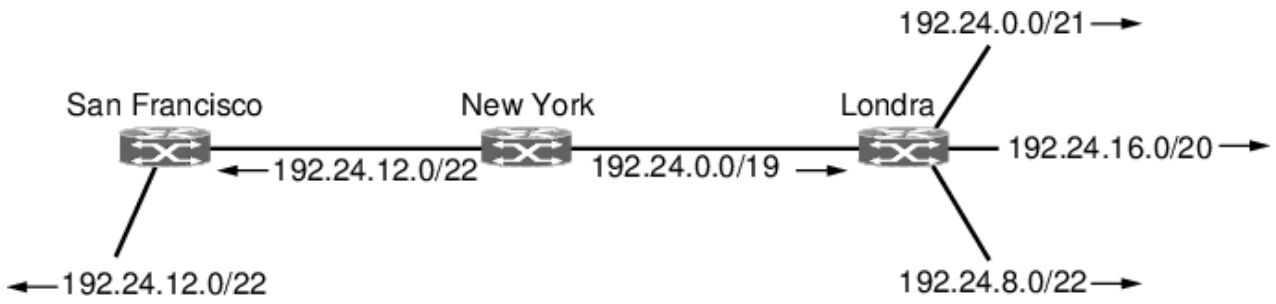


Figura 5.52 Routing basato sul più lungo prefisso corrispondente nel router di New York.

Concettualmente CIDR lavora in questo modo. All'arrivo di un pacchetto, la tabella di routing viene esaminata per determinare se la destinazione sta all'interno del prefisso. È possibile che più voci con lunghezze di prefisso differenti corrispondano; in questo caso viene usata la voce con il prefisso più lungo. Quindi, se c'è una corrispondenza con una maschera /20 e una maschera /24, per guardare quale linea di uscita usare per il pacchetto si usa la voce con /24. Tuttavia questo processo sarebbe lungo se veramente la tabella dovesse essere esaminata voce per voce.

Algoritmi complessi sono stati pensati per rendere più veloce il processo di ricerca (Ruiz-Sanchez et al., 2001). I router commerciali usano chip VLSI appositi, in cui questi algoritmi sono implementati via hardware.

Indirizzamento per classi e indirizzi speciali

Per comprendere meglio perché il CIDR sia così utile, descriviamo brevemente gli schemi usati in precedenza. Prima del 1993 gli indirizzi IP erano suddivisi nelle cinque categorie mostrate nella Figura 5.53, classificazione chiamata **classfull addressing** (*indirizzamento per classi*).

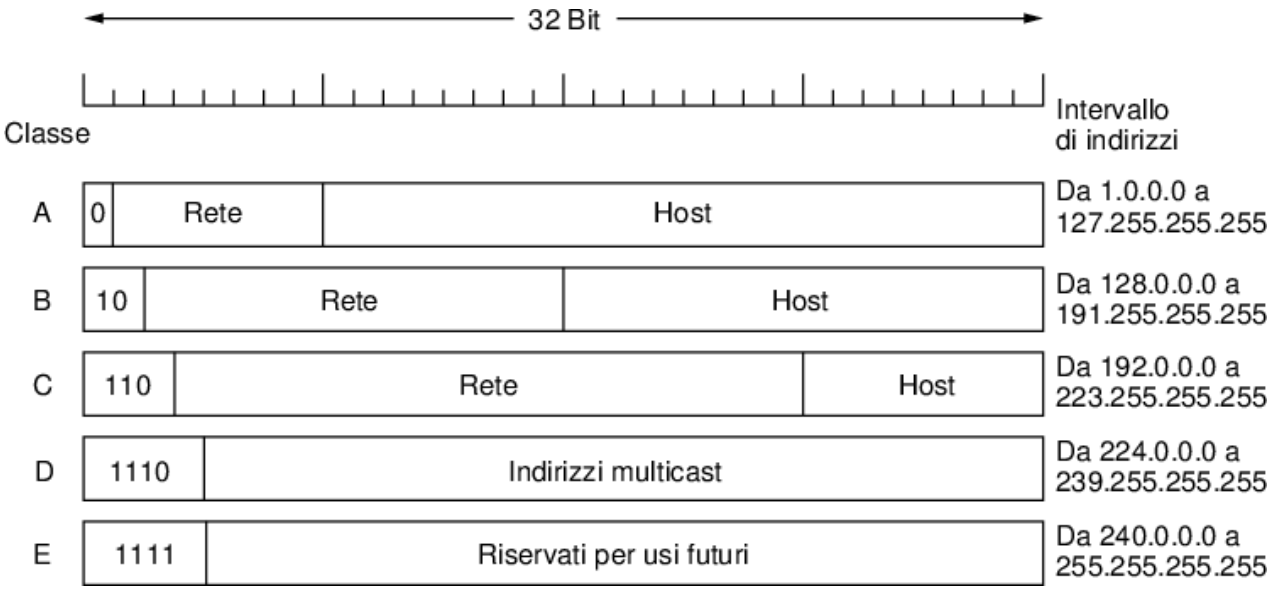


Figura 5.53 Formati degli indirizzi IP.

Le classi A, B e C supportano, rispettivamente, fino a 128 reti ognuna con 16 milioni di host, 16.384 reti con ognuna 65.536 host e 2 milioni di reti con fino a 256 host ciascuna (sebbene alcune di queste siano speciali). Anche il multicast trova il suo spazio nella classe D, dove un datagramma viene inviato a più host. Infine, gli indirizzi che iniziano con 1111 sono riservati a usi futuri. Sarebbe molto utile usarli in questo momento, data la scarsità di indirizzi IPv4.

Sfortunatamente, molti host non accetterebbero questi indirizzi come validi, perché sono stati dichiarati non utilizzabili per molto tempo ed è difficile insegnare nuovi trucchi a vecchi host.

Anche questo è uno schema gerarchico ma, al contrario di CIDR, la dimensione dei blocchi di indirizzi è fissa. Esistono più di 2 milioni di indirizzi, ma organizzare gli indirizzi in classi ne spreca milioni. In particolare, il vero cattivo della situazione è rappresentato dalla rete di classe B: per la maggior parte di enti e aziende, una rete di classe A con 16 milioni di indirizzi è troppo grande e una rete di classe C con 256 indirizzi è troppo piccola. Una rete di classe B (con 65.536 indirizzi) invece è perfetta. In Internet questa situazione è chiamata **three bears problem** (problema dei tre orsi) [si veda la fiaba *Riccioli d'oro e i tre orsi* (Southey, 1848)].

In realtà, un indirizzo di classe B è ancora di gran lunga troppo grande per la maggior parte delle aziende. Le ricerche hanno dimostrato che più della metà di tutte le reti di classe B hanno meno di 50 host. Una rete di classe C sarebbe stata sufficiente, ma senza dubbio ogni azienda che ha chiesto un indirizzo di classe B pensava che un giorno gli 8 bit del campo host non sarebbero più stati sufficienti. In retrospettiva, sarebbe stato meglio assegnare al numero di host delle reti di classe C 10 bit invece di 8, in modo da supportare 1022 host per rete. Con una scelta del genere, la maggior parte delle aziende avrebbe richiesto una rete di classe C e gli indirizzi di questo tipo sarebbero stati mezzo milione (mentre le reti di classe B sono solo 16.384).

È difficile criticare i progettisti di Internet per non aver fornito un numero maggiore di indirizzi di classe B (più piccoli). Quando venne presa la decisione di creare le tre classi, Internet era una rete di ricerca che collegava le principali università degli Stati Uniti (più un ristretto numero di aziende e di siti militari impegnati in ricerche sulla comunicazione in rete). Nessuno pensava che Internet sarebbe diventata un sistema di comunicazione di massa in grado di competere con la rete telefonica. Allora qualcuno senza dubbio pensò: "Gli Stati Uniti dispongono di circa 2000 college e università; anche se si collegassero tutte a Internet e anche se le università di altri paesi si unissero alla rete, non sarà mai possibile raggiungere il valore limite di 16.000, poiché in tutto il mondo non ci sono così tante università. Inoltre, usando un numero intero di byte per indicare l'host si accelera l'elaborazione dei pacchetti." (Che veniva fatta interamente via software). Forse, un giorno, le persone guarderanno indietro e si lamenteranno dei progettisti che hanno stabilito il sistema di numerazione telefonico pensando "Che idioti, perché non hanno incluso il prefisso del pianeta nel numero di telefono?". Ma a quel tempo non sembrava necessario.

Per gestire questi problemi vennero introdotte le subnet per assegnare in modo flessibile i blocchi di indirizzi all'interno di un ente o azienda. Successivamente venne introdotto il CIDR per ridurre la dimensione delle tabelle di routing. Attualmente i bit che indicano se un indirizzo appartiene a una delle classi non vengono più usati, anche se i riferimenti alle classi sono ancora abbastanza comuni in letteratura.

Per capire come l'eliminazione delle classi abbia complicato l'inoltro dei pacchetti, si consideri come era semplice il vecchio sistema. Quando un pacchetto raggiungeva un router, una copia dell'indirizzo IP veniva fatta scorrere a destra di 28 bit in modo da ottenere i 4 bit del numero di classe. Poi una diramazione a sedici percorsi ordinava i pacchetti in A, B e C (e D ed E), con otto casi per la classe A, quattro casi per la classe B, due casi per la classe C. Il codice relativo a ogni classe identificava poi il numero di rete a 8, 16 o 24 bit e lo allineava a destra in una sequenza di 32 bit. Il numero di rete era quindi cercato nella tabella A, B o C, di solito per indice nel caso di reti di classe A e B e per hash nel caso di reti di classe C. Una volta trovata la voce, si identificava la linea di output e si inoltrava il pacchetto. Questa procedura è molto più semplice dell'attuale che non può più servirsi di una semplice ricerca in una tabella perché gli indirizzi IP possono avere prefissi di qualsiasi lunghezza.

Gli indirizzi di classe D continuano a essere usati in Internet per il multicast. In realtà sarebbe più preciso dire che stanno iniziando a essere usati per il multicast, perché multicast in Internet non è stato largamente adottato in passato.

Ci sono ulteriori indirizzi che hanno un significato speciale, come mostrato nella Figura 5.54. L'indirizzo IP 0.0.0.0 è utilizzato dagli host al momento del boot e significa "questa rete" o "questo host". Gli indirizzi IP che hanno lo 0 come numero di rete si riferiscono alla rete corrente. Grazie a questi indirizzi, i computer possono utilizzare la rete locale senza conoscerne il numero (devono però conoscere la sua maschera di rete, per sapere quanti zeri includere). L'indirizzo composto da tutti 1, o 255.255.255.255, viene utilizzato per indicare tutti gli host di una rete. Permette la trasmissione broadcast sulla rete locale, in genere una LAN. Gli indirizzi con un numero di rete opportuno e tutti 1 nel campo host permettono ai computer l'invio di pacchetti broadcast a LAN distanti collegate a Internet; tuttavia, molti amministratori di rete disattivano questa funzionalità per sicurezza. Infine tutti gli indirizzi espressi nella forma 127 sono riservati per effettuare controlli

all'interno dell'host stesso (*loopback*). I pacchetti diretti a quell'indirizzo non sono immessi nel cavo, ma elaborati localmente e trattati come pacchetti in arrivo. Questo permette di inviare pacchetti all'host locale, senza che chi trasmette conosca il suo numero, cosa utile per effettuare dei test.

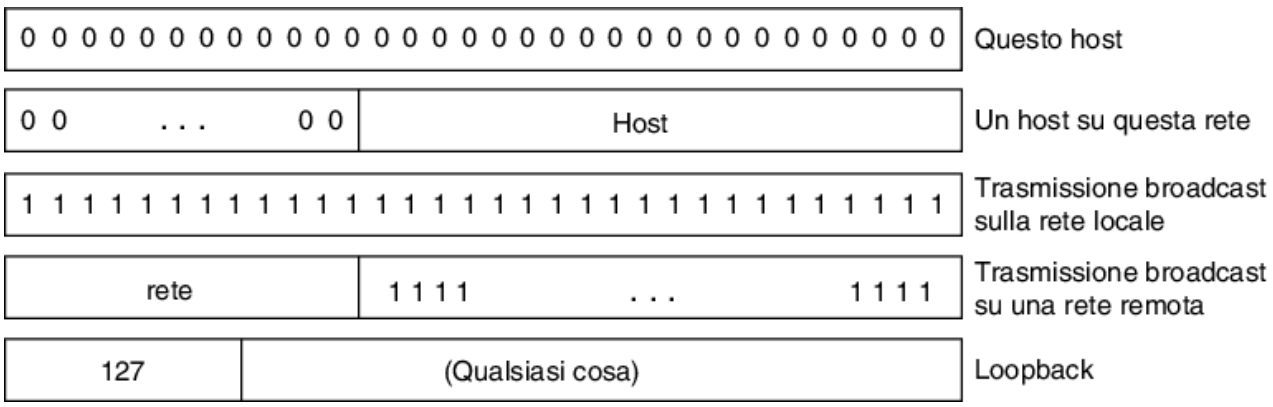


Figura 5.54 Indirizzi IP speciali.

NAT – network address translation

Gli indirizzi IP sono scarsi; un ISP potrebbe avere un indirizzo /16 che consente di gestire 65.534 host, ma se il numero dei clienti supera questo valore, sorge un problema.

Questa scarsità ha portato alla creazione di tecniche per usare gli indirizzi IP cercando di fare economia. Un approccio è quello di assegnare gli indirizzi IP ai computer in modo dinamico al momento della connessione e recuperarli al termine della sessione per essere assegnati ad altri computer che si connettono. In questo modo un singolo indirizzo /16 permette di gestire fino a 65.534 utenti attivi.

Questa strategia funziona bene per reti in dial-up o in mobilità o con computer non sempre connessi, ma non è adatta ad ambiti aziendali dove i computer prevedono di essere continuamente attivi. Alcuni computer sono i PC degli impiegati a cui viene fatto il backup di notte, altri possono essere dei server Web che devono rispondere alle richieste. Queste aziende hanno una linea di accesso a Internet che fornisce costantemente connettività.

Il numero degli utenti domestici che sottoscrivono abbonamenti ADSL o Internet via cavo è in continua ascesa perché il costo dell'abbonamento non dipende dalla durata della connessione (l'utente paga solo un addebito fisso mensile); molti di questi utenti hanno in casa due o più computer, spesso uno per ogni membro della famiglia, e tutti vogliono essere sempre collegati. Una soluzione è collegare tutti i PC in una LAN domestica e installare un router (wireless) che si collega all'ISP. Dal punto di vista dell'ISP, la famiglia è simile a un piccolo ufficio contenente una manciata di computer; benvenuti alla Rossi s.n.c.! Con le tecniche viste finora, ogni computer deve

possedere per tutto il giorno un proprio indirizzo IP. Per un ISP con migliaia di clienti, in particolar modo commerciali o domestici che agiscono come piccole aziende, la richiesta di indirizzi IP può velocemente superare il blocco di indirizzi disponibile.

La questione dell'esaurimento degli indirizzi IP non è un problema teorico che potrebbe presentarsi in un lontano futuro: sta accadendo proprio qui e ora. La soluzione a lungo termine è che tutta Internet passi a IPv6, protocollo che adotta indirizzi a 128 bit. Questa transizione sta procedendo lentamente e ci vorranno anni prima che si completi. Di conseguenza alcune persone si sono rese conto che era necessario trovare una soluzione rapida attuabile in tempi brevi. La soluzione adottata si chiama **NAT** (*network address translation*), è descritta nell'RFC 3022 ed è esaminata brevemente in questo paragrafo. Per informazioni più dettagliate si può consultare (Dutcher, 2001)

L'idea di base di NAT è assegnare a ogni azienda o casa un singolo indirizzo IP (o, al massimo, un piccolo numero di indirizzi) per il traffico di Internet. Dentro la rete del cliente, ogni computer riceve un indirizzo IP unico, utilizzato per instradare il traffico interno alla rete locale. Tuttavia quando un pacchetto sta per lasciare la rete locale per dirigersi verso l'ISP viene eseguita una traduzione di indirizzo dall'unico indirizzo IP interno a quello pubblico condiviso. Questa traslazione usa tre intervalli di indirizzi IP dichiarati privati, che le aziende possono utilizzare internamente come preferiscono. L'unica regola è che nessun pacchetto contenente questi indirizzi possa apparire su Internet. I tre intervalli riservati sono:

```
10.0.0.0    – 10.255.255.255/8    (16.777.216 host)

172.16.0.0    – 172.31.255.255/12    (1.048.576 host)

192.168.0.0    – 192.168.255.255/16    (65.536 host)
```

Il primo intervallo permette di gestire 16.777.216 host (sono esclusi, come sempre, i valori 0 e l) ed è la scelta comune in molte aziende, anche in quelle che non hanno bisogno di così tanti indirizzi.

Il funzionamento di NAT è mostrato nella Figura 5.55. Dentro i locali dell'azienda, ogni macchina ha un unico indirizzo espresso nella forma 10.*x.y.z*, ma quando un pacchetto lascia la rete locale, passa attraverso un **apparato NAT** che converte l'indirizzo IP interno (10.0.0.1 nella figura) nel vero indirizzo IP assegnato all'azienda (198.60.42.12 in questo esempio). L'apparato NAT è spesso abbinato a un firewall, inserito all'interno di un singolo apparecchio che protegge la rete locale, controllando attentamente tutti i dati che entrano e che escono dalla LAN (studieremo i firewall nel Capitolo 8). Si può anche integrare l'apparato NAT nel router o nel modem ADSL aziendale.

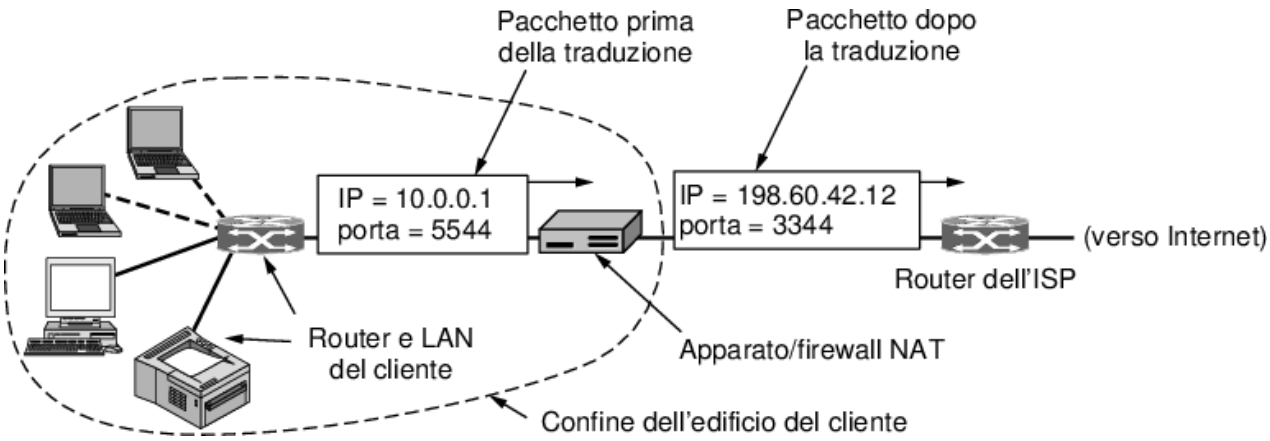


Figura 5.55 Collocazione e funzionamento di un apparato NAT.

Fino a questo punto è stato trascurato un piccolo dettaglio: quando la risposta ritorna (per esempio da un Web server), i dati sono naturalmente indirizzati a 198.60.42.12; in che modo, allora, l'apparato NAT sceglie l'indirizzo di destinazione interno? È qui che si presenta il problema di NAT. Se ci fosse un campo di riserva nell'intestazione IP lo si potrebbe utilizzare per tener traccia del trasmittente reale, ma è rimasto inutilizzato solo 1 bit. In linea di principio si sarebbe potuta creare una nuova opzione che conservasse il vero indirizzo sorgente; la gestione di questo meccanismo, però, avrebbe richiesto la modifica del codice per IP di tutti i computer di Internet, quindi non sarebbe stata rapida.

Ecco che cosa è successo realmente: i progettisti di NAT hanno osservato che la maggior parte dei pacchetti IP trasporta al suo interno TCP o UDP. Quando studieremo TCP e UDP, nel Capitolo 6, vedremo che entrambi i protocolli hanno intestazioni che contengono una porta sorgente e una porta di destinazione. La descrizione che segue, relativa alle porte TCP, vale in modo analogo per le porte UDP. Le porte sono numeri interi lunghi 16 bit che indicano dove inizia e finisce la connessione TCP e offrono il campo che permette di far funzionare NAT.

Quando un processo desidera stabilire una connessione TCP con un processo remoto, si lega a una porta TCP inutilizzata sulla sua macchina; questa porta è chiamata **porta sorgente** e indica al codice di TCP dove devono essere inviati i pacchetti in arrivo appartenenti alla connessione. Il processo fornisce anche una **porta di destinazione** che indica chi deve ricevere i pacchetti sulla parte remota. Le porte da 0 a 1023 sono riservate a servizi noti; per esempio, la porta 80 è utilizzata dai server Web, e ciò consente ai client remoti di individuare il servizio. Ogni messaggio TCP in uscita contiene sia la porta sorgente sia la porta di destinazione e insieme permettono di identificare i processi che utilizzano la connessione.

Il ricorso a un'analogia può chiarire il funzionamento delle porte. Si immagini un'azienda che dispone di un unico numero di telefono. Quando qualcuno chiama il numero principale, risponde un operatore che chiede il numero dell'interno con cui la persona desidera parlare e poi inoltra la chiamata all'interno richiesto. Il numero principale equivale all'indirizzo IP assegnato all'azienda e i numeri degli interni rappresentano le porte. Le porte sono altri 16 bit di indirizzamento che identificano il processo al quale è destinato il pacchetto in arrivo.

L'utilizzo del campo *Source port* (porta sorgente) permette di risolvere il problema dell'associazione. Ogni volta che un pacchetto diretto verso l'esterno raggiunge l'apparato NAT, l'indirizzo sorgente 10.*x.y.z* è sostituito dall'indirizzo IP dell'azienda. Inoltre, il campo *Source port* è sostituito da un indice che punta alla tabella di traduzione da 65.536 voci dell'apparato NAT. Questa voce della tabella contiene l'indirizzo IP originale e la porta sorgente originale. Infine i codici di controllo delle intestazioni IP e TCP sono ricalcolati e inseriti nel pacchetto. È necessario sostituire il campo *Source port* perché, per esempio, due connessioni distinte dalle macchine 10.0.0.1 e 10.0.0.2 possono entrambe utilizzare la porta 5000; perciò la porta sorgente non basta, da sola, a identificare il processo che sta trasmettendo.

Quando un pacchetto trasmesso dall'ISP raggiunge l'apparato NAT, la *Source port* nell'intestazione TCP viene estratta e utilizzata come indice nella tabella di associazione dell' apparato. Dalla voce individuata, l'apparato estrapola l'indirizzo IP interno e la *Source port* TCP originale, quindi inserisce entrambe le informazioni nel pacchetto; poi ricalcola e inserisce nel pacchetto sia il checksum IP sia quello TCP. Alla fine il pacchetto viene passato al router interno per il normale inoltro basato sull'indirizzo 10.*x.y.z.*

Sebbene questo schema in un certo senso risolva il problema, molti nella comunità IP lo considerano un vero e proprio abominio. In breve ecco alcune delle obiezioni.

Primo, NAT viola il modello gerarchico di IP, che afferma che ogni indirizzo IP identifica in modo univoco a livello mondiale una singola macchina. L'intera struttura del software di Internet è costruita su questa verità. Con NAT migliaia di macchine potrebbero utilizzare l'indirizzo 10.0.0.1 (e lo fanno realmente).

Secondo, NAT rompe il modello di connettività end-to-end di Internet, che afferma che ogni host può inviare un pacchetto in ogni momento a ogni altro host. Poiché l'associazione viene effettuata dai pacchetti in uscita, i pacchetti in entrata non possono essere accettati se non dopo quelli in uscita. In pratica questo significa che un utente domestico con una NAT può effettuare una connessione TCP/IP a un server Web remoto, ma un utente remoto non può connettersi a un server sulla rete domestica. Per supportare queste situazioni è necessario avere speciali configurazioni o tecniche di **NAT traversal** (*attraversamento NAT*).

Terzo, NAT trasforma Internet da una rete non orientata alla connessione in un tipo particolare di rete orientata alla connessione, perché l'apparato NAT deve conservare le informazioni (le associazioni) relative a ogni connessione che lo attraversa. Il mantenimento dello stato della connessione è una proprietà delle reti orientate alle connessioni. Se l'apparato NAT si blocca e la sua tabella di mappatura si perde, tutte le sue connessioni TCP vanno distrutte. In assenza di NAT, i malfunzionamenti dei router non hanno effetto su TCP, perché il processo di trasmissione dopo alcuni secondi ritrasmette tutti i pacchetti che non hanno ricevuto acknowledgement. Con NAT Internet diventa vulnerabile come una rete a commutazione di circuito.

Quarto, NAT viola la principale regola della stratificazione dei protocolli: il livello k non deve essere costretto a formulare alcuna ipotesi su ciò che il livello $k + 1$ ha inserito nel payload. Questo principio fondamentale serve a rendere i livelli indipendenti. Se in futuro TCP venisse aggiornato a TCP-2, con uno schema di intestazione diverso (porte a 32 bit, per esempio), NAT non

funzionerebbe più. L'idea centrale del concetto di protocollo stratificato è quella di assicurare che la modifica apportata a uno livello non renda necessario modificare gli altri e NAT distrugge questa indipendenza.

Quinto, i processi su Internet non sono obbligati a utilizzare TCP e UDP. Se un utente sulla macchina *A* decidesse di utilizzare qualche nuovo protocollo di trasporto per parlare con un utente seduto davanti alla macchina *B* (per esempio, attraverso un'applicazione multimediale), l'introduzione di un apparato NAT non farebbe funzionare l'applicazione perché quest'ultimo non sarebbe in grado di individuare in modo corretto il campo *Source port* TCP.

Sesto, alcune applicazioni usano più connessioni TCP o porte UDP con modalità prefissate. Per esempio **FTP** (*file transfer protocol*), il protocollo standard di trasferimento file, inserisce indirizzi IP nel corpo del pacchetto dal quale il ricevente li estrae e li usa. Poiché NAT non conosce il formato non può gestirlo. Sia FTP sia altre applicazioni come il protocollo telefonico di Internet H.323 (descritto nel Capitolo 7) possono fallire in presenza di NAT, a meno che non vengano prese speciali precauzioni. Si potrebbe correggere NAT, ma essere obbligati ad aggiornare il codice dell'apparato NAT ogni volta che nasce una nuova applicazione non è una buona idea.

Infine, poiché il campo *Source port* di TCP è grande 16 bit, a un indirizzo IP si possono associare al massimo 65.536 macchine. In realtà il numero è un po' più basso, perché le prime 4.096 porte sono riservate per usi speciali.

Questi e altri problemi legati a NAT sono discussi nell'RFC 2993. Nonostante ciò, NAT è molto usato nella pratica, specialmente per reti domestiche o in piccole aziende, in quanto è l'unica tecnica che riesca a gestire il problema della mancanza di indirizzi IP. È diventato una cosa unica con firewall e privacy, perché di norma blocca i pacchetti in arrivo non richiesti. Per questa ragione è inverosimile che sparisca, anche qualora IPv6 fosse ampiamente adottato.

IP versione 6

IP è stato usato pesantemente per decenni e ha funzionato estremamente bene, come dimostra la crescita esponenziale di Internet. Sfortunatamente IP è diventato una vittima della sua stessa popolarità: sta esaurendo gli indirizzi. Anche se CIDR e NAT ne fanno un uso oculato ci si aspetta che gli ultimi indirizzi IPv4 siano assegnati da ICANN prima della fine del 2012.

In questo paragrafo descriveremo sia il problema sia alcune delle soluzioni proposte. L'unica soluzione a lungo termine è quella di adottare indirizzi più grandi; **IPv6** (*IP versione 6*) è un progetto che fa proprio questo: usa indirizzi a 128 bit. Che anche questi indirizzi si esauriscano in un futuro prossimo non è verosimile; tuttavia IPv6 si è dimostrato molto difficile da implementare. È un protocollo a livello di rete differente, che non interagisce con IPv4 nonostante le molte similarità. Inoltre sia le aziende sia gli utenti non sono veramente sicuri del perché dovrebbero adattare IPv6. Il risultato è che IPv6 è implementato e usato solo su una piccola frazione di Internet (stimata nell'1%), benché sia uno standard Internet dal 1998. I prossimi anni saranno interessanti, in quanto verranno allocati gli ultimi pochi indirizzi IPv4 rimanenti. La gente comincerà a vendere i propri indirizzi IPv4 su eBay? Nascerà un mercato nero? Chi vivrà vedrà.

Oltre a questi problemi di indirizzi esistono altre problematiche nascoste nell'ombra. Nei primi anni Internet era molto usata dalle università, dalle aziende high-tech e dal governo degli Stati Uniti (specialmente il dipartimento della difesa). Con l'esplosione di interesse suscitata a partire da metà degli anni '90, Internet ha cominciato a essere usata da gruppi diversi di persone spesso con esigenze distinte. Da una parte molte di loro usano gli smart phone per rimanere in contatto con le proprie abitazioni; dall'altra con la convergenza di computer, comunicazioni e industria dell'intrattenimento non manca molto a quando ogni telefono e televisione al mondo saranno un nodo di Internet, portando a 1 miliardo le macchine usate per audio e video on demand. In queste circostanze diventa evidente che IP deve evolversi e conquistare più flessibilità.

Scorgendo questi problemi all'orizzonte, nel 1990 IETF cominciò a lavorare su una nuova versione di IP che non avrebbe mai esaurito gli indirizzi, avrebbe risolto una grande varietà di altre questioni e sarebbe stata più flessibile ed efficiente. Gli obiettivi più importanti erano i seguenti.

1. Supportare miliardi di host anche con un'allocazione di indirizzi inefficiente.
2. Ridurre la dimensione delle tabelle di routing.
3. Semplificare i protocolli per permettere ai router di elaborare i pacchetti più velocemente.
4. Fornire un grado di sicurezza maggiore (autenticazione e privacy).
5. Prestare più attenzione al tipo di servizio, particolarmente per i dati in tempo reale.
6. Aiutare il multicasting permettendo di specificarne la validità degli indirizzi.
7. Permettere a un host di spostarsi senza cambiare il suo indirizzo.
8. Permettere al protocollo di evolversi in futuro.
9. Permettere al protocollo vecchio e nuovo di coesistere per anni.

La progettazione di IPv6 offriva una grande opportunità di migliorare tutte le caratteristiche di IPv4, insufficienti per le attuali esigenze. Per sviluppare un protocollo che corrispondesse a tutti questi requisiti, IETF presentò un bando e una discussione nell'RFC 1550. All'inizio le risposte furono 21 e nel dicembre 1992 sul tavolo c'erano sette proposte serie. Si andava dal fare modifiche minori a IP al buttarlo via completamente e usare un protocollo completamente differente.

Una proposta consisteva nell'usare TCP su CLNP, il protocollo a livello di rete progettato per OSI. Con i suoi indirizzi a 160 bit, CLNP avrebbe fornito per sempre sufficienti indirizzi, in quanto avrebbe potuto fornire a ogni molecola di acqua nell'oceano abbastanza indirizzi (circa 2^5) per fare una piccola rete. Questa scelta avrebbe anche unificato due importanti protocolli a livello di rete. Tuttavia in molti pensavano che sarebbe stata un'ammissione che qualcosa nel mondo OSI era effettivamente stata fatta bene, una dichiarazione considerata politicamente scorretta negli ambienti di Internet. CLNP era stato progettato in modo simile a IP, quindi i due non differiscono molto l'uno dall'altro. Il protocollo scelto alla fine differisce da IP molto di più di quanto lo faccia CLNP. Un'altra tesi sfavorevole a CLNP è che supporta poco i tipi di servizio, necessari per trasmettere contenuti multimediali in modo efficiente.

Tre delle proposte migliori vennero pubblicate in *IEEE Network* (Deering, 1993; Francis, 1993; e Katz e Ford, 1993). Dopo molte discussioni, revisioni e manovre per conquistare una posizione, venne selezionata una versione che combinava le proposte fatte da Deering e Francis, oggi chiamata **SIPP** (*simple internet protocol plus*) e indicata come **IPv6**.[8]
IPv6 raggiunge gli obiettivi di IETF con successo. Mantiene le buone caratteristiche di IP, abbandona le cattive e ne aggiunge di nuove dove necessario. In generale IPv6 non è compatibile con IPv4, ma è compatibile con tutti gli altri protocolli di Internet ausiliari, come TCP, UDP,

ICMP, IGMP, OSPF, BGP e DNS, aggiungendo solo piccole modifiche necessarie per trattare indirizzi più lunghi. Le caratteristiche più importanti di IPv6 verranno discusse tra poco. Per approfondire l'argomento si consulti dall'RFC 2460 fino all'RFC 2466.

Innanzitutto IPv6 ha indirizzi più lunghi di IPv4. Sono lunghi 128 bit e questo risolve il problema per cui IPv6 era stato progettato: fornire un numero effettivamente illimitato di indirizzi Internet.

La seconda importante miglioria apportata da IPv6 è la semplificazione dell'intestazione: contiene solo sette campi (rispetto ai 13 di IPv4). Questo cambiamento permette ai router di elaborare i pacchetti più velocemente, migliorando throughput e ritardo. Di questo parleremo a breve.

Il terzo importante progresso è che supporta meglio le opzioni. Questo cambiamento era essenziale con la nuova intestazione, perché i campi una volta obbligatori ora sono opzionali (perché non vengono usati molto spesso). Inoltre il modo in cui le opzioni sono rappresentate è differente e rende semplice ai router non considerarle, quando non è necessario. Questa caratteristica riduce il tempo di elaborazione dei pacchetti.

Una quarta area in cui IPv6 rappresenta un grande passo avanti è la sicurezza. IETF enumerava molte testimonianze di adolescenti particolarmente precoci che con i loro personal computer entravano in banche e basi militari in tutta Internet e urgevano contromisure per migliorare la sicurezza. Autenticazione e privacy sono punti chiave del nuovo IP. Tuttavia, in tempi successivi, IPv4 fu aggiornato di queste caratteristiche, quindi nell'area della sicurezza le differenze non sono più così grandi.

Infine si è prestata più attenzione alla qualità del servizio; nel passato vi furono sporadici tentativi di migliorarla ma adesso, con la crescita delle applicazioni multimediali in Internet, si è sviluppata una particolare sensibilità in questo campo.

Intestazione principale di IPv6

L'intestazione di IPv6 è mostrata nella Figura 5.56. Il campo *Version* è sempre 6 per IPv6 (e 4 per IPv4). Durante il periodo di transizione da IPv4 a IPv6, iniziato da più di un decennio, i router sono stati in grado di esaminare questo campo per determinare il tipo di pacchetto. Come nota a margine, questo test spreca alcune istruzioni sul percorso critico, poiché l'intestazione del livello data link indica di solito il tipo di protocollo di rete per il demultiplexing; quindi alcuni router potrebbero evitare questo controllo. Per esempio, il campo *Type* di Ethernet ha valori diversi per indicare un payload IPv4 e uno IPv6. La discussione riguardo alla scelta tra "fare bene" o "fare in fretta" sarà lunga e vivace.

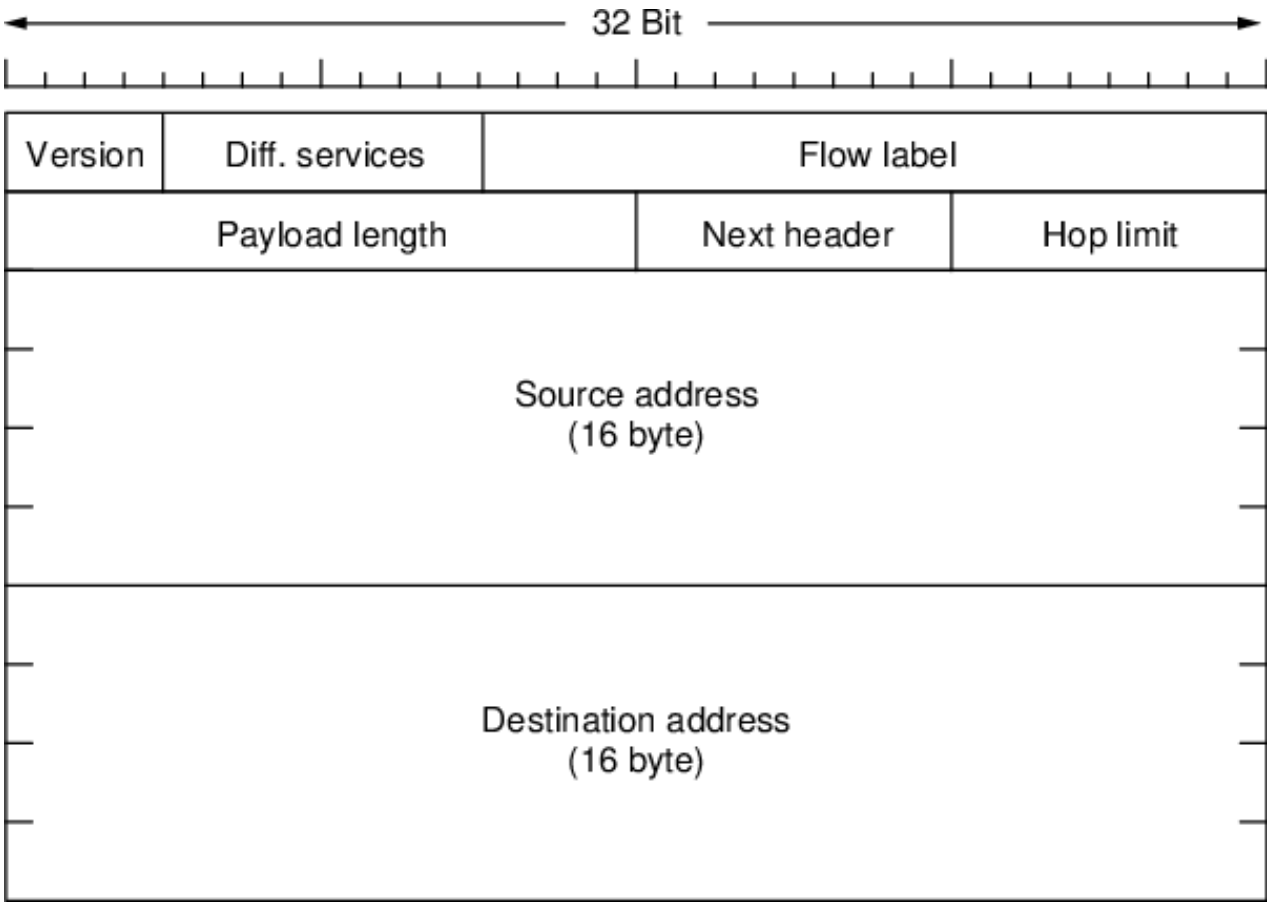


Figura 5.56 L'intestazione fissa di IPv6 (obbligatoria).

Il campo *Differentiated services* (*servizi differenziati*, originariamente chiamato *Traffic class* o *classe di traffico*) è usato per distinguere le classi di servizio dei pacchetti con differenti richieste di consegna in tempo reale. Viene usato nell'architettura a servizi differenziati per la qualità del servizio nello stesso modo dell'analogo campo nei pacchetti IPv4. Inoltre i 2 bit di ordine più basso sono usati per segnalare esplicitamente la presenza di congestione, esattamente come in IPv4.

Il campo *Flow label* (etichetta di flusso) consente a una sorgente e a una destinazione di marcare un gruppo di pacchetti che, avendo gli stessi requisiti, devono essere trattati allo stesso modo, formando una pseudoconnessione. Per esempio un flusso di pacchetti generato da un processo su un particolare host sorgente e diretto a un particolare processo su un particolare host di destinazione potrebbe avere rigorosi requisiti di ritardo e quindi aver bisogno di banda riservata. Il flusso può essere impostato in anticipo e gli si può assegnare un identificatore. Quando appare un pacchetto con *Flow label* diversa da zero, tutti i router cercano l'etichetta nelle loro tabelle interne per scoprire il tipo di trattamento speciale richiesto dal pacchetto. In realtà i flussi sono un tentativo di combinare la flessibilità della rete a datagrammi e le garanzie di una rete a circuito virtuale.

Ai fini della qualità del servizio, ogni flusso è rappresentato dall'indirizzo sorgente, dall'indirizzo di destinazione e dal numero di flusso, perciò tra una data coppia di indirizzi IP possono essere attivi anche 2^{20} flussi. Inoltre, in questo modo, anche se due flussi provenienti da host diversi ma con la stessa etichetta di flusso passano attraverso il medesimo router, potranno essere distinti grazie agli

indirizzi di sorgente e di destinazione. Poiché ci si aspetta che le etichette di flusso siano scelte a caso e non assegnate in modo sequenziale partendo da 1, ci si attende che i router le memorizzino tramite una lista hash.

Il campo *Payload length* (lunghezza del payload) indica il numero di byte che segue l'intestazione di 40 byte mostrata nella Figura 5.56. È stato abbandonato il nome del campo *Total length* adottato da IPv4 perché il significato è leggermente cambiato: i 40 byte di intestazione non sono più conteggiati nel calcolo della lunghezza. Questo cambiamento implica che il payload possa essere ora di 65.535 byte invece che di 65.515 byte.

Il campo *Next header* (intestazione successiva) fa trapelare un segreto. Il motivo per cui è stato possibile semplificare l'intestazione è che ci possono essere altre intestazioni estese (opzionali). Questo campo indica quale delle sei (per il momento) intestazioni estese, se presente, segue l'intestazione corrente. Se questa intestazione è l'ultima intestazione IP, il campo *Next header* indica il gestore del protocollo di trasporto (TCP o UDP) al quale va passato il pacchetto.

Il campo *Hop limit* (limite di hop) è utilizzato per impedire ai pacchetti di vivere per sempre. In pratica funziona come il campo *Time to live* di IPv4, vale a dire che è decrementato a ogni salto del pacchetto. In teoria in IPv4 il valore rappresentava una durata espressa in secondi, ma nessun router utilizzava l'etichetta in quel modo, perciò il nome è stato modificato per riflettere il modo in cui è realmente utilizzato.

Gli ultimi campi rappresentano l'indirizzo sorgente (*source address*) e l'indirizzo di destinazione (*destination address*). La proposta originale di Deering (SIPP) utilizzava indirizzi a 8 byte, ma durante il processo di revisione in molti pensavano che IPv6 avrebbe esaurito gli indirizzi entro pochi decenni usando 8 byte, mentre con indirizzi a 16 byte non si sarebbero mai esauriti. Altri sostenevano che 16 byte fossero troppi, mentre altri ancora avrebbero preferito utilizzare indirizzi a 20 byte compatibili con il protocollo datagram di OSI. Un'altra fazione proponeva indirizzi di lunghezza variabile. Dopo un lungo dibattito vennero scelti gli indirizzi a 16 byte di lunghezza fissa, il miglior compromesso tra tutte le possibilità.

Per scrivere gli indirizzi a 16 byte è stata escogitata una nuova notazione. Gli indirizzi sono scritti come otto gruppi di quattro cifre esadecimali separate da due punti:

```
8000:0000:0000:0000:0123:4567:89AB:CDEF
```

Poiché molti indirizzi possono contenere numerosi zero sono state autorizzate tre ottimizzazioni. Primo, è possibile omettere gli zero iniziali di un gruppo, perciò 0123 può essere scritto come 123. Secondo, uno o più gruppi contenenti 16 bit a zero possono essere sostituiti da una coppia di due punti. Perciò l'indirizzo precedente diventa:

```
8000::123:4567:89AB:CDEF
```

Infine, gli indirizzi IPv4 possono essere scritti in notazione decimale a punti dopo una coppia di due punti:

```
::192.31.20.46
```

Forse non è necessario essere così espliciti, ma esistono moltissimi indirizzi a 16 byte. Per la precisione, ci sono 2^{128} indirizzi, ossia circa 3×10^{38}. Se l'intero pianeta, terraferma e acqua, fosse coperto di computer, IPv6 permetterebbe di utilizzare 7×10^{23} indirizzi IP per metro quadrato. Gli studenti di chimica noteranno che questo numero è più grande del numero di Avogadro. Anche se nessuno aveva intenzione di dare un indirizzo IP a ogni molecola della superficie terrestre, non siamo troppo lontani dal farlo. In pratica, lo spazio di indirizzamento non sarà utilizzato efficientemente, proprio come non lo è lo spazio di indirizzamento dei numeri telefonici (il prefisso di Manhattan, 212, è quasi esaurito, ma quello del Wyoming, 307, è quasi vuoto). Nell'RFC 3194 Durand e Huitema hanno calcolato che, usando come guida l'allocazione dei numeri telefonici, anche nello scenario più pessimistico ci saranno ancora più di 1.000 indirizzi IP per ogni metro quadrato di superficie terrestre (terraferma e acqua). In uno scenario probabile ci saranno trilioni di indirizzi per metro quadrato. In breve, è molto improbabile che nell'immediato futuro gli indirizzi si esauriscano.

È istruttivo confrontare l'intestazione IPv4 (Figura 5.46) con l'intestazione IPv6 (Figura 5.56) per vedere che cosa è stato eliminato in IPv6. Il campo *IHL* è stato tolto, perché l'intestazione IPv6 ha una lunghezza fissa. Il campo *Protocol* è stato tolto, perché il campo *Next header* indica che cosa c'è dopo l'ultima intestazione IP (per esempio un segmento UDP o TCP).

Tutti i campi che riguardano la frammentazione sono stati rimossi, perché IPv6 gestisce la frammentazione in modo diverso. Tanto per cominciare, tutti gli host compatibili con IPv6 devono determinare dinamicamente la dimensione del datagramma da utilizzare. Lo fanno servendosi della procedura MTU discovery descritta nel Paragrafo 5.5.5. Quando un host invia un pacchetto IPv6 che è troppo grande, il router che non è in grado di inoltrarlo restituisce un messaggio di errore invece di frammentarlo. Questo messaggio indica all'host di suddividere tutti i pacchetti futuri diretti a quella destinazione. Fare in modo che l'host invii pacchetti della dimensione corretta è in definitiva più efficiente che costringere i router a frammentare i pacchetti al volo. La dimensione minima è stata portata da 576 a 1280, per consentire 1024 byte di dati e molte intestazioni.

Infine il campo *Checksum* è stato eliminato, perché l'elaborazione di questa informazione riduce enormemente le prestazioni. Con le reti affidabili utilizzate oggi, e visto che i livelli data link e di trasporto hanno i loro codici di controllo, il valore di un altro checksum non varrebbe il prezzo delle prestazioni richieste per estrarlo. La rimozione di tutte queste funzionalità ha permesso di ottenere un protocollo a livello di rete leggero e non invadente. Perciò l'obiettivio di IPv6, un protocollo veloce e flessibile, con una miriade di indirizzi, è stato raggiunto dal progetto.

Intestazione estesa

Alcuni dei campi mancanti di IPv4 sono ancora saltuariamente necessari, perciò IPv6 ha introdotto il concetto di **intestazione estesa** (opzionale), chiamata anche **extension header**. Queste intestazioni possono essere fornite per offrire informazioni aggiuntive codificate in modo efficiente. Al momento sono definiti sei tipi di intestazioni estese, elencati nella Figura 5.57. Ogni estensione è opzionale, ma quando ne è presente più di una devono apparire direttamente dopo l'intestazione fissa e preferibilmente nell'ordine elencato.

Intestazione estesa	Descrizione
Hop-by-hop options	Informazioni varie per i router
Destination options	Informazioni addizionali per la destinazione
Routing	Lista di router da visitare
Fragmentation	Gestione dei frammenti del datagramma
Authentication	Verifica dell'identità della sorgente
Encrypted security payload	Informazioni relative al contenuto cifrato

Figura 5.57 Le intestazioni estese di IPv6.

Alcune intestazioni hanno un formato fisso, altre contengono un numero variabile di campi di lunghezza variabile. Per queste ultime ogni elemento è codificato come un vettore di tre elementi: *Type, Length* e *Value. Type* è un campo lungo 1 byte che indica l'opzione; i valori di *Type* sono stati scelti in modo tale che i primi due bit istruiscano i router che non sanno come elaborare l'opzione su ciò che devono fare. Le possibilità sono: non considerare l'opzione, scartare il pacchetto, scartare il pacchetto inviando un pacchetto ICMP di risposta, scartare il pacchetto e non inviare pacchetti ICMP per gli indirizzi multicast (in modo da impedire ai pacchetti multicast danneggiati di generare milioni di risposte ICMP).

Anche *Length* occupa 1 byte; questo campo indica l'estensione del valore (da 0 a 255 byte). *Value* è una qualsiasi informazione necessaria, lunga fino a 255 byte.

L'intestazione *Hop-by-hop options* è utilizzata per le informazioni che tutti i router lungo il percorso devono esaminare. Finora è stata definita una sola opzione: supporto di datagrammi che superano i 64 KB. La Figura 5.58 mostra il formato di questa intestazione. Quando è utilizzata, il campo *Payload length* nell'intestazione fissa è impostato a zero.

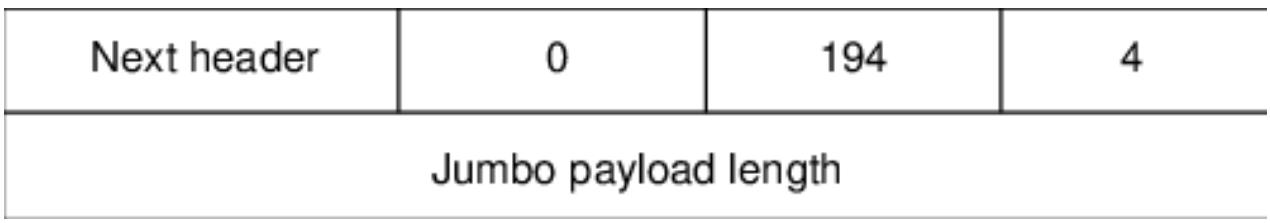


Figura 5.58 L'intestazione estesa hop-by-hop per datagrammi grandi (jumbogrammi).

Come nel caso di tutte le intestazioni estese, anche questa inizia con un byte che indica il tipo di intestazione che segue. Questo byte è seguito da un altro byte che indica la lunghezza, espressa in byte, dell'intestazione *Hop-by-hop options*, esclusi i primi 8 byte che sono obbligatori. Tutte le intestazioni iniziano in questo modo.

I successivi due byte indicano che questa opzione definisce la dimensione del datagramma (codice 194) e che la dimensione è un numero a 4 byte. Gli ultimi 4 byte rappresentano la dimensione del datagramma. Dimensioni minori di 65.536 byte non sono consentite e, se utilizzate, costringono il primo router a scartare il pacchetto e a restituire un messaggio di errore ICMP. I datagrammi che usano questa estensione di intestazione sono chiamati **jumbogrammi**. L'uso dei jumbogrammi è importante per applicazioni di supercomputer che devono trasferire gigabyte di dati attraverso Internet in modo efficiente.

L'intestazione *Destination options* (opzioni di destinazione) è utilizzata per i campi che devono essere interpretati dall'host di destinazione. Nella versione originale di IPv6 le uniche opzioni definite sono opzioni nulle usate per riempire questa intestazione con multipli di 8 byte, perciò attualmente non viene utilizzata. È stata inclusa per essere certi che il software di routing e quello sull'host siano in grado di gestirla in caso qualcuno un giorno pensi a un'opzione di destinazione.

L'intestazione *Routing* elenca uno o più router che devono essere visitati lungo la strada che porta alla destinazione. È molto simile al *Loose source routing* di IPv4, in quanto tutti gli indirizzi elencati devono essere visitati nell'ordine indicato, ma in mezzo possono essere visitati anche altri router non elencati. Il formato dell'intestazione di routing è mostrato nella Figura 5.59.

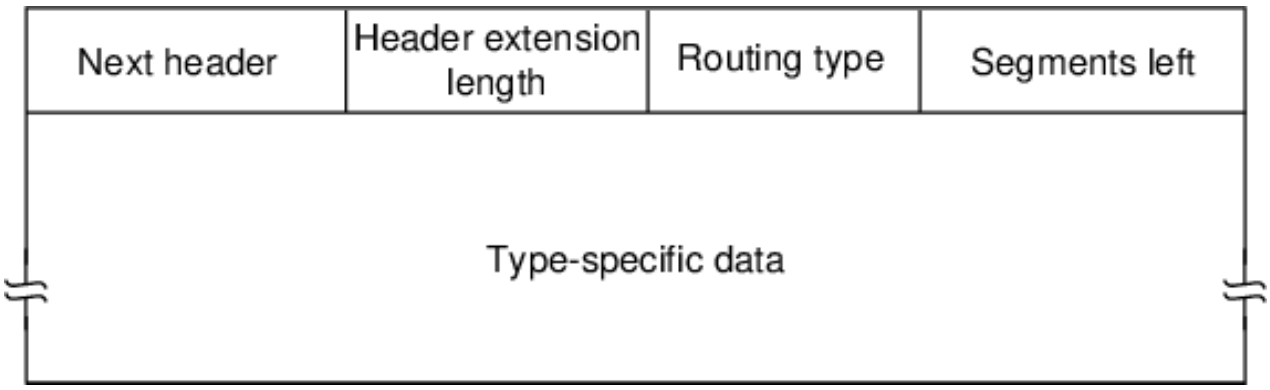


Figura 5.59 Intestazione estesa per il routing.

I primi 4 byte dell'intestazione estesa di routing contengono quattro valori interi lunghi 1 byte. I campi *Next header* e *Header extension length* sono stati già descritti precedentemente. Il campo *Routing type* (tipo di routing) indica il formato del resto dell'intestazione. Il tipo 0 indica che un gruppo specifico di 32 bit segue il primo, seguito da una serie di indirizzi IPv6. Altri tipi potranno essere inventati in futuro a seconda delle necessità. Infine, il campo *Segments left* (segmenti rimasti) tiene traccia del numero di indirizzi nell'elenco che non sono ancora stati visitati, ed è

decrementato ogni volta che ne viene visitato uno. Quando il valore raggiunge lo 0, il pacchetto è lasciato libero e può seguire il percorso che desidera. Di solito a questo punto è talmente vicino alla destinazione che il percorso migliore appare ovvio.

L'intestazione *Fragmentation* si occupa delle informazioni di frammentazione in maniera simile alla gestione di IPv4. L'intestazione contiene l'identificatore di datagramma, la posizione del frammento all'interno del datagramma originale e un bit che indica se il frammento corrente è seguito da altri. In IPv6, a differenza di IPv4, solo l'host sorgente può frammentare un pacchetto; i router lungo il percorso non possono farlo. Sebbene tale approccio rappresenti una rottura con la filosofia del passato, semplifica il lavoro dei router e rende più rapido il processo di routing. Come è stato spiegato precedentemente, se riceve un pacchetto troppo grande, il router scarta i dati e invia alla sorgente un pacchetto di risposta ICMP. Tale informazione permette all'host sorgente di spezzare il pacchetto in pezzi più piccoli, usando questa intestazione e tentando di nuovo la trasmissione.

L'intestazione *Authentication* fornisce un meccanismo che garantisce al ricevente di un pacchetto l'autenticità del mittente, mentre *Encrypted security payload* permette di cifrare il contenuto del pacchetto in modo da renderlo leggibile solo al destinataro desiderato. Queste intestazioni utilizzano, per raggiungere lo scopo, le tecniche crittografiche che verranno descritte nel Capitolo 8.

Controversie

Il processo di sviluppo di tipo aperto e le opinioni difese con vigore da molte delle persone coinvolte hanno reso naturale il sorgere di controversie su molte delle scelte adottate per IPv6. Questo paragrafo riassume brevemente alcune di queste discussioni; per maggiori dettagli si consultino i vari RFC.

È già stata menzionata la disputa legata alla lunghezza dell'indirizzo. Il risultato è stato un compromesso: indirizzo di lunghezza fissa a 16 byte.

Un'altra lotta si è consumata intorno alla lunghezza del campo *Hop limit*: una fazione pensava che limitare il numero massimo di salti a 255 (scelta implicita se si utilizza un campo lungo 8 bit) sarebbe stato un madornale errore; dopo tutto i percorsi di 32 salti oggi sono comuni e tra dieci anni potranno essere comuni percorsi molto più lunghi. Queste persone sostenevano che l'utilizzo di un enorme dimensione per l'indirizzo significava vedere lontano, mentre l'adozione di un conteggio di salti minuscolo voleva dire essere miopi. Secondo loro, il più grande peccato che un informatico può commettere è mettere a disposizione troppi pochi bit da qualche parte.

La risposta fu che era possibile trovare argomenti che giustificassero l'aumento di ogni campo, cosa che avrebbe gonfiato enormemente l'intestazione. Inoltre la funzione del campo *Hop limit* è impedire ai pacchetti di girovagare per troppo tempo e 65.536 salti sono decisamente eccessivi. Infine, al crescere di Internet aumenterà il numero dei collegamenti a lunga distanza, il che permetterà di raggiungere qualunque paese da qualunque altro in non più di una mezza dozzina di salti. Se la sorgente o la destinazione impiegano più di 125 salti per raggiungere i rispettivi gateway internazionali, c'è qualcosa di sbagliato nelle dorsali nazionali. I promotori degli 8 bit hanno quindi vinto lo scontro.

Un'altra patata bollente era la dimensione massima del pacchetto. La comunità dei supercomputer voleva pacchetti che superassero i 64 KB. Quando si utilizza un supercomputer per trasferire dati la faccenda diventa seria ed è meglio non interrompere l'operazione ogni 64 KB. Il ragionamento contro i pacchetti di grandi dimensioni è semplice: se un pacchetto da 1 MB raggiunge una linea T1 a 1,5 Mbps, quel pacchetto occupa la linea per 5 secondi, producendo un ritardo molto evidente agli utenti interattivi che condividono la linea. È stato quindi raggiunto un compromesso: i pacchetti normali non possono superare i 64 KB, ma l'intestazione estesa *Hop-by-hop options* può essere utilizzata per consentire il trasferimento di jumbogrammi.

Un terzo tema scottante è stato la rimozione del checksum di IPv4. Alcuniparagonavano l'operazione alla rimozione dei freni da un'automobile. Agendo in questo modo si ottiene un'auto più leggera e veloce, ma in caso di eventi inaspettati si va incontro a problemi.

L'argomento contro il checksum era che ogni applicazione che si preoccupa realmente dell'integrità dei suoi dati deve avere comunque un checksum a livello di trasporto, perciò averne un altro in IP (oltre a quello presente nel livello data link) è eccessivo. Inoltre l'esperienza ha mostrato che l'elaborazione del checksum IP rappresentava una delle maggiori perdite di tempo in IPv4. Vinse la fazione che si opponeva al checksum e IPv6 non ne era dotato.

Gli host mobili rappresentavano un altro punto di contesa: se un computer portatile si sposta da una parte all'altra del mondo, può continuare a operare alla destinazione con lo stesso indirizzo IPv6 o deve utilizzare uno schema basato sugli home agent? Alcuni volevano realizzare in IPv6 un supporto esplicito per gli host mobili, ma il tentativo fallì quando non fu possibile trovare consenso per alcuna proposta specifica.

Probabilmente la battaglia più grande fu quella relativa alla sicurezza. Tutti concordavano che fosse essenziale, il problema era definire il dove e il come. Prima il dove: chi proponeva di inserirla nel livello di rete faceva notare che in questo modo la sicurezza sarebbe diventata un servizio standard che tutte le applicazioni avrebbero potuto utilizzare senza fare alcuna pianificazione in anticipo. Chi si opponeva a questa scelta affermava che le applicazioni sono veramente sicure, solo quando si applica una crittografia end-to-end, con l'applicazione sorgente che codifica i dati e quella di destinazione che li decodifica. Qualunque altro meccanismo avrebbe lasciato l'utente in balia di implementazioni del livello di rete potenzialmente errate e sulle quali non avrebbe potuto avere alcun controllo. Si ribattè a queste argomentazioni affermando che queste applicazioni possono semplicemente astenersi dall'utilizzare le funzionalità di protezione di IP e fare il lavoro da sole. La risposta a questa affermazione fu che coloro che non si fidano della rete non vogliono pagare il prezzo di lente e ingombranti implementazioni IP che hanno questa funzionalità, anche se disattivata.

Un altro aspetto relativo al dove inserire la sicurezza fa riferimento al fatto che molti paesi (ma non tutti) hanno leggi di esportazione molto severe sulla crittografia. Alcuni paesi, specialmente la Francia e l'Iraq,[9] ne limitano addirittura l'utilizzo domestico. Di conseguenza, qualunque implementazione IP che utilizza sistemi crittografici abbastanza forti da essere molto efficaci non potrebbe essere esportata dagli Stati Uniti (e da molti altri paesi) verso i clienti di tutto il mondo. Il mantenimento di due categorie di software, una per uso domestico e l'altra per l'esportazione, è contrastato dalla maggior parte dei produttori di computer.

Un punto sul quale non c'è stata alcuna controversia è quello dei tempi di transizione: nessuno, infatti, si aspetta che il passaggio da Internet IPv4 a Internet IPv6 possa avvenire da un giorno all'altro. Al contrario, saranno convertite a IPv6 singole "isole", messe inizialmente in comunicazione attraverso dei tunnel, come discusso nel Paragrafo 5.5.3. Quando le isole IPv6 incominceranno a crescere, si fonderanno tra loro e alla fine Internet sarà totalmente convertita.

Almeno, questo era il piano; l'installazione si è dimostrata il tallone di Achille di IPv6: è rimasto poco utilizzato anche se i sistemi operativi maggiori lo supportano pienamente. La maggior parte delle installazioni avviene in situazioni per cui un operatore di rete, per esempio un operatore di telefonia cellulare, necessita di un gran numero di indirizzi IP. Per aiutare la transizione vennero definite molte strategie. Tra queste troviamo metodi per configurare automaticamente i tunnel che portano IPv6 su IPv4 e strategie che consentano agli host di trovare automaticamente i punti terminali dei tunnel. Gli host dotati di doppio stack di rete (*dual-stack*) hanno implementazioni sia IPv4 che IPv6, in modo che possano selezionare quale protocollo usare a seconda della destinazione del pacchetto. Quando gli indirizzi IPv4 saranno esauriti queste strategie renderanno fluida l'istallazione sostanziale che sembra ormai inevitabile. Per maggiori informazioni su IPv6 si veda Davies (2008).

Protocolli di controllo di Internet

Oltre a IP, utilizzato per il trasferimento dei dati, Internet ha diversi protocolli di controllo utilizzati nel livello di rete, tra cui ICMP, ARP, e DHCP. In questo paragrafo li esamineremo descrivendo le versioni che corrispondono a IPv4 in quanto sono quelle comunemente usate. ICMP e DHCP hanno versioni simili per IPv6; l'equivalente di ARP per IPv6 è chiamato NDP (*neighbor discovery protocol*).

ICMP – Internet control message protocol

Il funzionamento di Internet è monitorato attentamente dai router. Quando avviene qualcosa di imprevisto, l'evento è comunicato da **ICMP** (*internet control message protocol*) utilizzato anche per fare dei test su Internet. È stata definita circa una dozzina di tipi di messaggi ICMP, di cui i più importanti sono elencati nella Figura 5.60.

Tipo di messaggio	Descrizione
DESTINATION UNREACHABLE	Non è stato possibile consegnare il pacchetto
TIME EXCEEDED	Il campo Time to live ha raggiunto il valore 0
PARAMETER PROBLEM	Campo dell'intestazione non valido
SOURCE QUENCH	Pacchetto choke
REDIRECT	Insegna a un router la geografia
ECHO e ECHO REPLY	Controlla se una macchina è funzionante
TIMESTAMP REQUEST/REPLY	Come Echo, ma con un timestamp

Tipo di messaggio	Descrizione
ROUTER ADVERTISEMENT/SOLICITATION	Trovare un router nelle vicinanze

Figura 5.60 Principali tipi di messaggi ICMP.

Il messaggio DESTINATION UNREACHABLE è utilizzato quando il router non è in grado di individuare la destinazione o quando un pacchetto con il bit *DF* non può essere inoltrato perché lungo il percorso si trova una rete con MTU più piccolo.

Il messaggio TIME EXCEEDED è inviato quando un pacchetto è scartato perché il suo contatore *TtL* (*time to live*) ha raggiunto il valore zero. Questo evento indica che i pacchetti si trovano in un ciclo o che i valori assegnati al contatore sono troppo bassi.

Un impiego intelligente di questo messaggio di errore è rappresentato dall'utility di **traceroute**, sviluppata da Van Jacobson nel 1987. Essa trova i router che stanno sul percorso dall'host all'indirizzo IP di destinazione, senza alcun tipo di supporto privilegiato di rete. Il metodo consiste semplicemente nell'inviare una sequenza di pacchetti alla destinazione, prima con un *TtL* di 1, quindi di 2, 3 e così via. Il contatore di questi pacchetti raggiungerà lo 0 in corrispondenza di router successivi lungo il percorso. Questi router restituiscono all'host un messaggio di TIME EXCEEDED. L'host può determinare da questi messaggi gli indirizzi IP dei router lungo il percorso, come pure può tenere statistiche e tempistiche di parti del cammino. Anche se non è questo lo scopo per cui il messaggio TIME EXCEEDED venne pensato, è diventato il tool di debugging più utilizzato in rete.

Il messaggio PARAMETER PROBLEM indica che è stato rilevato un valore scorretto in un campo dell'intestazione. Questo problema può dipendere da un errore nel software che si occupa di IP nell'host trasmittente o nel software di un router attraversato.

Il messaggio SOURCE QUENCH era utilizzato in passato per rallentare gli host che trasmettevano troppi pacchetti; quando riceveva questo messaggio l'host doveva rallentare. Oggi è usato raramente perché in caso di congestione questi pacchetti tendono a gettare benzina sul fuoco. Il controllo della congestione in Internet è ora gestito in larga misura nel livello di trasporto, usando la perdita di pacchetti come segnale di congestione (i dettagli sono descritti nel Capitolo 6).

Il messaggio REDIRECT è utilizzato quando un router si accorge che un pacchetto sembra essere stato inoltrato in modo errato. È usato dal router per comunicare all'host trasmittente di aggiornare la sua tabella di routing.

I messaggi ECHO e ECHO REPLY sono utilizzati per scoprire se una certa destinazione sia raggiungibile e attiva. Non appena riceve il messaggio ECHO, la destinazione deve rispondere con un messaggio ECHO REPLY. Questi messaggi sono usati nell'utility **ping** per vedere se un host è connesso a Internet.

I messaggi TIMESTAMP REQUEST e TIMESTAMP REPLY funzionano in modo simile a quelli appena visti, ma registrano nella risposta l'ora di arrivo del messaggio e l'ora di invio della risposta. Queste informazioni sono utilizzate per misurare le prestazioni della rete.

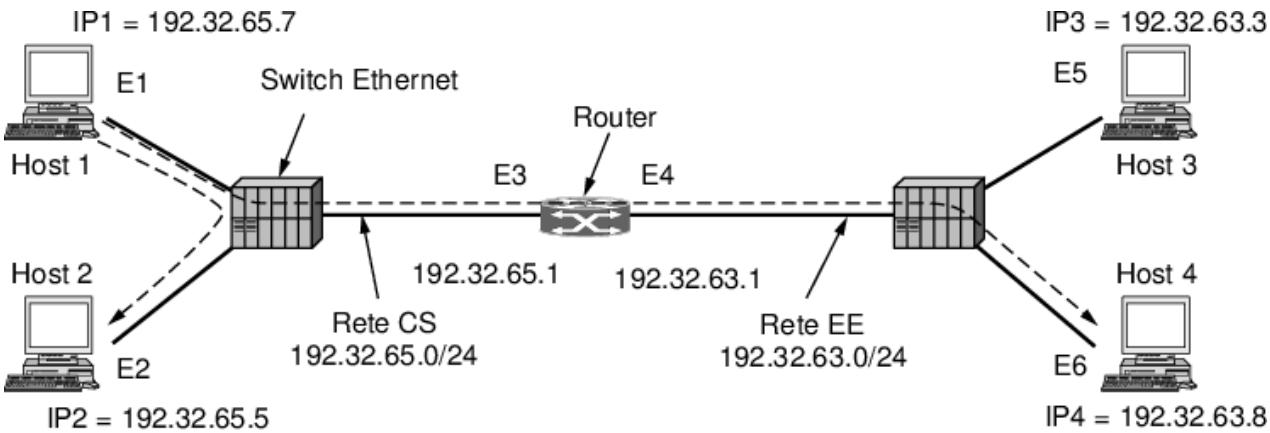
I messaggi ROUTER ADVERTISEMENT e ROUTER SOLICITATION sono usati dagli host per trovare i router vicini. Un host deve conoscere l'indirizzo IP di almeno un router per uscire dalla rete locale.

Oltre a questi messaggi ne sono stati definiti anche altri; la lista completa si può trovare a *http://www.iana.org/assignments/icmp-parameters.*

ARP – address resolution protocol

Anche se ogni macchina di Internet ha uno o più indirizzi IP, in realtà questi non possono essere utilizzati per inviare pacchetti, in quanto le schede di rete, come le schede Ethernet che operano a livello data link, non comprendono gli indirizzi Internet. Per esempio nel caso di Ethernet qualsiasi scheda di rete incorpora un indirizzo Ethernet a 48 bit. Ogni produttore richiede a IEEE un blocco di indirizzi per garantire che nessuna coppia di schede utilizzi lo stesso indirizzo (per evitare conflitti se dovessero capitare sulla stessa LAN). Le schede di rete inviano e ricevono frame basati sugli indirizzi Ethernet a 48 bit e non sanno nulla degli indirizzi IP a 32 bit.

La domanda che sorge spontanea è: in che modo gli indirizzi IP vengono associati agli indirizzi del livello data link (per esempio agli indirizzi Ethernet)? Per spiegare come funziona il meccanismo è utile l'esempio rappresentato nella Figura 5.61, che mostra una piccola università contenente due reti di tipo /24. Una rete (CS) è una Ethernet commutata nel dipartimento di informatica con prefisso 192.32.65.0/24 e l'altra LAN (EE), sempre Ethernet commutata, nel dipartimento di ingegneria elettronica associata a 192.32.63.0/24. Queste sono connesse tramite un router IP; ogni macchina su Ethernet e ogni interfaccia del router hanno un unico indirizzo Ethernet, indicati da *E1* a *E6* e un indirizzo IP unico sulla rete CS o EE.



Frame	IP sorgente	Eth. sorgente	IP destinazione	Eth. destinazione
Da host 1 a host 2 sulla rete CS	IP1	E1	IP2	E2
Da host 1 a host 4 sulla rete CS	IP1	E1	IP4	E3
Da host 1 a host 4 sulla rete EE	IP1	E4	IP4	E6

Figura 5.61 Due LAN con Ethernet commutata unite tramite un router.

Vediamo prima di tutto in che modo l'utente dell'host 1 invia un pacchetto all'utente dell'host 2 sulla rete CS. Si supponga che il trasmittente conosca il nome del ricevente desiderato, nome che potrebbe essere per esempio *eagle.cs.uni.edu.* Il primo passo consiste nel cercare l'indirizzo IP. Questa ricerca è eseguita dal DNS (*domain name system*) che esamineremo nel Capitolo 7; per il momento è sufficiente supporre che il sistema restituisca l'indirizzo IP dell'host 2 (192.31.65.5).

Il software del livello superiore sull'host 1 costruisce un pacchetto che riporta nel campo *Destination address* l'indirizzo IP 192.32.65.5 e lo passa al software che implementa IP per la trasmissione. Il software esamina l'indirizzo e scopre che la destinazione si trova sulla rete CS, la sua stessa, ma ha bisogno di trovare in qualche modo l'indirizzo Ethernet di destinazione per inviare il frame. Il problema potrebbe essere risolto mediante un file di configurazione, collocato da qualche parte nel sistema, che associ gli indirizzi IP agli indirizzi Ethernet. Questa soluzione è certamente possibile, ma in un'azienda con migliaia di macchine l'aggiornamento di questi file sarebbe soggetta a errori e richiederebbe molto tempo.

Esiste una soluzione migliore: l'host 1 trasmette attraverso la Ethernet un pacchetto broadcast che chiede chi sia il proprietario dell'indirizzo IP 192.32.65.5, la trasmissione broadcast arriva a tutte le macchine della Ethernet CS e ognuna controlla il proprio indirizzo IP. Solo l'host 2 risponde inviando il proprio indirizzo Ethernet (*E2*). In questo modo l'host 1 scopre che l'indirizzo IP 192.32.65.5 è assegnato all'host che ha l'indirizzo Ethernet *E2.* Il protocollo utilizzato per fare questa domanda e ottenere una risposta si chiama **ARP** (*address resolution protocol*) e quasi tutte le macchine di Internet lo utilizzano. ARP è definito nell'RFC 826.

Utilizzare ARP al posto dei file di configurazione rende tutto più semplice. Il gestore del sistema non deve fare altro che assegnare a ogni macchina un indirizzo IP e stabilire le maschere di sottorete. ARP si occupa di tutto il resto.

A questo punto il software che si occupa di IP sull'host 1 costruisce un frame Ethernet indirizzato a *E2,* inserisce il pacchetto IP (indirizzato a 192.32.65.5) nel payload e scarica il tutto sulla Ethernet. Gli indirizzi IP ed Ethernet di questo pacchetto sono rappresentati nella Figura 5.61. La scheda Ethernet dell'host 2 rileva il frame, si accorge di essere il destinatario della comunicazione, preleva i dati e genera un interrupt. Il driver Ethernet estrae il pacchetto IP dal payload e lo passa al software che verifica la correttezza dell'indirizzo ed elabora i dati.

Varie ottimizzazioni permettono ad ARP di funzionare in modo più efficiente. Tanto per cominciare, una volta utilizzato ARP, il computer memorizza in una cache il risultato, caso mai dovesse essere necessario ricontattare lo stesso computer. La volta successiva, trovando l'informazione desiderata nella cache, il computer non dovrà generare alcun messaggio broadcast. In molti casi l'host 2 potrebbe aver bisogno di inviare una richiesta per determinare grazie ad ARP l'indirizzo Ethernet del trasmittente; per evitare questa trasmissione ARP broadcast è sufficiente che l'host 1 includa la propria associazione IP Ethernet nel pacchetto ARP. Quando la trasmissione ARP raggiunge l'host 2, la coppia di valori del tipo (192.32.65.7, *E1*) viene inserita nella cache ARP dell'host 2 in previsione di un utilizzo futuro. In realtà, tutti i computer sulla Ethernet possono inserire queste associazioni nelle proprie cache ARP.

Per permettere cambiamenti nelle associazioni, per esempio quando un host è configurato per utilizzare un nuovo indirizzo IP, mantenendo quello Ethernet, le informazioni nella cache ARP dovrebbero scadere in pochi minuti. Un modo più intelligente per tenere le informazioni nella cache e ottimizzare le prestazioni è che ogni computer trasmetta in broadcast la propria associazione durante l'accensione. Questa trasmissione broadcast è generalmente eseguita sotto forma di una ricerca ARP mirata al proprio indirizzo IP. L'operazione, che non dovrebbe ricevere alcuna risposta, ottiene l'effetto di inserire una voce nella cache ARP di tutti gli altri computer. Questa operazione è chiamata **gratuitous ARP** (*ARP gratuito*). Se (inaspettatamente) arriva una risposta, significa che due computer hanno ricevuto lo stesso indirizzo IP; l'errore deve essere risolto dall'amministratore di rete prima che le due macchine possano usare la rete.

Si osservi nuovamente la Figura 5.61 e si supponga che questa volta l'host 1 voglia inviare un pacchetto all'host 4 (192.32.63.8) sulla rete EE. L'host 1 vedrà che l'indirizzo IP di destinazione non si trova sulla rete CS. Sa che deve inviare tutto questo traffico fuori rete al router che prende anche il nome di **default gateway** e che, per convenzione, ha l'indirizzo più basso sulla rete (198.31.65.1). L'host 1 per inviare un frame al router deve comunque conoscere l'indirizzo Ethernet della sua scheda di rete sulla rete CS. Lo scopre inviando un broadcast ARP per 198.31.65.1 dal quale impara *E3* e poi può inviare il frame. Lo stesso meccanismo viene usato per inviare un pacchetto da un router a un altro su un percorso.

Quando la scheda di rete del router riceve il frame passa il pacchetto al software che implementa IP, il quale capisce dalla maschera di sottorete che deve essere inoltrato sulla rete EE dove potrà raggiungere l'host 4. Se il router non conosce l'indirizzo Ethernet di host 4 userà ARP un'altra volta. La tabella nella Figura 5.61 elenca gli indirizzi Ethernet e IP di sorgente e destinazione presenti nei frame che transitano sulle reti CS ed EE. Si osservi come gli indirizzi Ethernet cambino su ogni rete mentre gli IP rimangono costanti (in quanto indicano i punti terminali attraverso tutte le reti interconnesse).

È anche possibile mandare un pacchetto dall'host 1 all'host 4 senza che l'host 1 sappia che l'host 4 si trova su una rete diversa. La soluzione è di usare il router sulla rete CS per rispondere alle richieste ARP per l'host 4 e fornire il suo indirizzo Ethernet, *E3*. Non è possibile far rispondere l'host 4 direttamente, perché questo non vedrà la richiesta ARP in quanto i router non inoltrano i messaggi broadcast a livello Ethernet. Il router, poi, riceverà i frame indirizzati a 192.32.63.8 e li inoltrerà sulla rete EE. Questa soluzione prende il nome di **proxy ARP** ed è utilizzata in casi in cui un host voglia apparire su una rete anche se effettivamente risiede in un'altra. Una situazione comune, ad esempio, è quando una stazione mobile vuole che qualche altro nodo raccolga il proprio traffico quando non si trova nella sua LAN di appartenenza.

DHCP – dynamic host configuration protocol

ARP, come altri protocolli di Internet, assume che gli host siano configurati con alcune informazioni di base, come il proprio indirizzo IP. Come fanno gli host a ottenere questa informazione? Si potrebbe effettuare una configurazione manuale su ogni computer, ma sarebbe tedioso e suscettibile di errori. La soluzione è chiamata **DHCP** (*dynamic host configuration protocol*).

Con DHCP ogni rete deve avere un server DHCP responsabile della configurazione. Quando un computer viene avviato ha un indirizzo Ethernet o altro di livello data link all'interno della scheda di rete, ma non un indirizzo IP. Analogamente ad ARP, il computer invia sulla rete una richiesta

broadcast tramite un pacchetto DHCP DISCOVER per ottenere un indirizzo IP. Il pacchetto deve raggiungere il server DHCP. Se il server non è direttamente collegato alla stessa rete, il router deve essere configurato per ricevere il messaggio DHCP e inoltrarlo al server, ovunque esso sia.

Quando il server riceve il pacchetto alloca un indirizzo IP libero e lo invia all'host in un pacchetto DHCP OFFER (che potrebbe dover essere inoltrato nuovamente dal router). Per essere in grado di eseguire questa operazione anche quando gli host non hanno indirizzo IP, il server identifica un host tramite il suo indirizzo Ethernet (contenuto nel pacchetto DHCP DISCOVER).

Un problema che si presenta quando si assegnano automaticamente indirizzi IP estratti da una riserva comune riguarda la durata dell'allocazione. Se un host abbandona la rete e non restituisce il proprio indirizzo IP al server DHCP, quell'indirizzo andrà definitivamente perso, perciò dopo un po' di tempo molti indirizzi IP potrebbero non essere più disponibili. Per impedire che ciò accada, l'assegnazione dell'indirizzo IP può essere fissata per un periodo di tempo, mediante una tecnica chiamata **leasing**. Poco prima della scadenza del leasing, l'host deve chiedere al server DHCP il rinnovo dell'indirizzo. Se non riesce a fare la richiesta o la richiesta viene rifiutata, l'host non può più utilizzare l'indirizzo IP che gli era stato assegnato precedentemente.

DHCP è descritto negli RFC 2131 e 2132. È molto usato in Internet per configurare ogni sorta di parametro oltre a fornire indirizzi IP agli host. Oltre alle reti aziendali e domestiche DHCP è usato anche dagli ISP per configurare i parametri dei dispositivi di accesso a Internet, in modo che i clienti non debbano telefonare all'ISP per avere tale informazione. Esempi comuni sono la maschera di rete, l'indirizzo IP del default gateway, del DNS e del server per la sincronizzazione dell'orologio interno (*time server*). DHCP ha quasi del tutto sostituito protocolli precedenti (RARP e BOOTP) che avevano funzionalità più limitate.

Commutazione di etichetta e MPLS

Finora, nel nostro tour del livello di rete di Internet, ci siamo concentrati esclusivamente sui pacchetti come datagrammi inoltrati dai router IP. Esiste anche un'altra tecnologia che comincia a essere largamente utilizzata, specialmente dagli ISP, per trasferire il traffico di Internet attraverso le loro reti. Questa tecnologia è chiamata **MPLS** (*multiprotocol label switching*) ed è pericolosamente simile alla commutazione di circuito. Nonostante il fatto che molte persone della comunità Internet provino una forte antipatia per le reti orientate alla connessione, sembra che l'idea continui a ripresentarsi. Tuttavia ci sono differenze fondamentali tra il modo in cui Internet gestisce la costruzione del percorso e il modo in cui funzionano le reti orientate alla connessione, perciò la tecnica è senza alcun dubbio diversa dalla commutazione di circuito tradizionale.

MPLS aggiunge un'etichetta davanti a ogni pacchetto, in modo da consentire ai router di inoltrare i dati in base all'etichetta, invece che in base all'indirizzo di destinazione. Usando l'etichetta come un indice associato a una tabella interna, per determinare la linea di output corretta è sufficiente eseguire una ricerca. Attraverso questa tecnica, l'inoltro può essere fatto molto velocemente. Questo vantaggio fu la motivazione originale di MPLS, che iniziò come tecnologia proprietaria nota con diversi nomi tra cui **tag switching** (*commutazione di etichetta*). Alla fine, IETF ha standardizzato l'idea con l'RFC 3031 e molti altri. I benefici maggiori offerti nel tempo sono un routing flessibile e un inoltro adatto alla qualità del servizio, oltre che veloce.

Il primo problema è dove inserire l'etichetta. Poiché i pacchetti IP non sono stati progettati per i circuiti virtuali, nell'intestazione IP non ci sono campi a disposizione della numerazione dei circuiti virtuali. Per questo motivo è stato necessario aggiungere una nuova intestazione MPLS prima dell'intestazione IP. Su una linea tra due router che usa PPP come protocollo di framing, il formato del frame è quello indicato nella Figura 5.62, che mostra le intestazioni PPP, MPLS, IP e TCP.

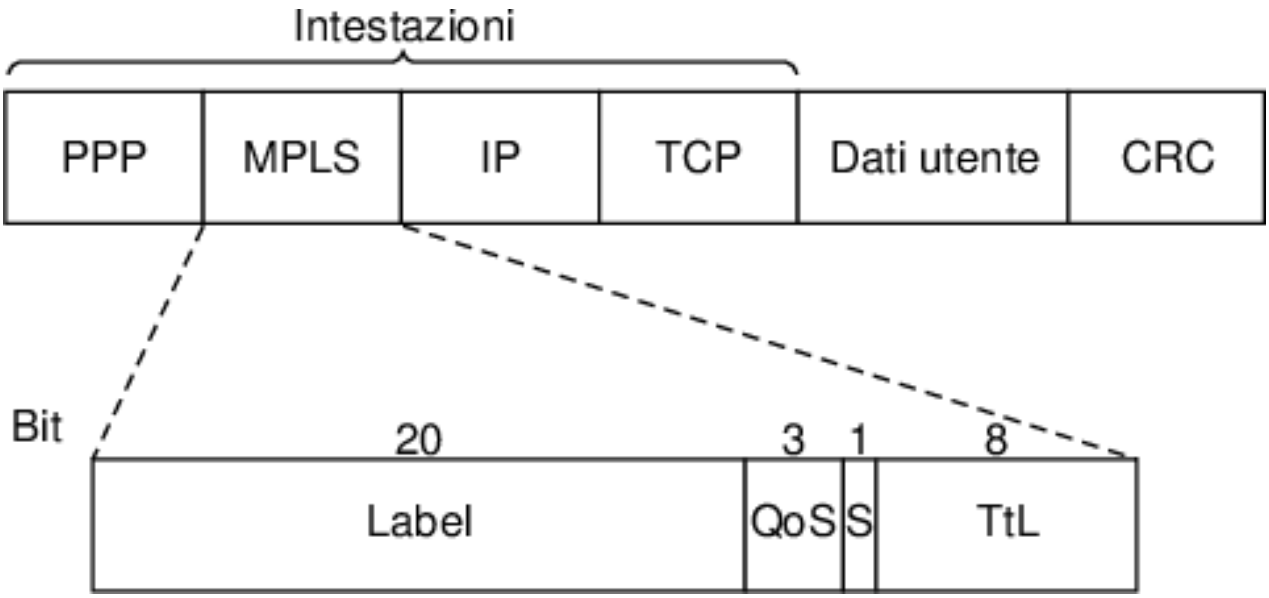


Figura 5.62 Trasmissione di un segmento TCP usando IP, MPLS e PPP.

L'intestazione MPLS generica è lunga 4 byte e ha quattro campi, il principale dei quali è il campo *Label* (etichetta), che contiene l'indice. Il campo *QoS* indica la classe di servizio. Il campo *S* si riferisce all'accatastamento (*stacking*) di più etichette, che verrà discusso a breve. Il campo *TtL* indica quante volte il pacchetto deve essere ancora inoltrato. Viene decrementato a ogni router e quando raggiunge 0, il pacchetto è scartato. Questa funzionalità impedisce la formazione di cicli infiniti nei casi di instabilità nel routing.

MPLS cade tra il protocollo di livello di rete IP e il protocollo del livello data link. Non è propriamente un protocollo di livello 3, perché per impostare le etichette dei percorsi dipende da IP o da altri indirizzi a livello di rete. Non è nemmeno propriamente un protocollo di livello 2, perché inoltra i pacchetti attraverso più salti e non attraverso un singolo collegamento. Per questa ragione, MPLS è a volte descritto come un protocollo di livello 2,5. È un esempio di come protocolli reali non sempre coincidano perfettamente con il nostro modello ideale di protocolli stratificati.

Poiché le intestazioni MPLS non fanno parte del pacchetto del livello di rete o del frame del livello data link, MPLS è in larga misura indipendente da entrambi. Tra le altre cose, questa proprietà significa che è possibile costruire switch MPLS in grado di inoltrare sia pacchetti IP che non, a

seconda di ciò che si presenta. Da questa funzionalità deriva il termine "multiprotocollo" che appare nel nome di MPLS. MPLS può anche trasportare pacchetti IP su reti non IP.

Quando un pacchetto potenziato da MPLS raggiunge un router **LSR** (*label switching router*, router a commutazione di etichetta), l'etichetta è utilizzata come indice in una tabella per determinare la linea di trasmissione e la nuova etichetta da utilizzare. Questo scambio di etichette è utilizzato in tutte le reti a circuito virtuali. Le etichette hanno un significato soltanto locale e due router possono inviare allo stesso router dei pacchetti non correlati tra loro, che però usano la stessa etichetta per essere trasmessi sulla stessa linea di uscita. Per essere distinguibili all'altro capo della linea, le etichette devono essere riassegnate a ogni salto. Il meccanismo è già stato visto in azione nella Figura 5.3; MPLS utilizza la stessa tecnica.

Per inciso, alcune persone distinguono tra *inoltro* e *commutazione.* L'inoltro è il processo di ricerca della migliore corrispondenza dell'indirizzo di destinazione in una tabella e stabilisce la destinazione dei pacchetti. Un esempio è l'algoritmo di prefisso più lungo usato per l'inoltro di pacchetti IP. Al contrario, la commutazione usa come indice della tabella di routing un'etichetta presa dal pacchetto ed è più semplice e veloce. Queste definizioni, però, non sono per niente universali.

Poiché molti host e router non capiscono MPLS, dobbiamo anche chiederci quando e come le etichette vengono attaccate ai pacchetti. Questo succede quando un pacchetto IP raggiunge il bordo di una rete MPLS. Il **LER** (*label egde router*, router di bordo a etichetta) ispeziona l'indirizzo della destinazione IP e gli altri campi, per vedere quale percorso MPLS il pacchetto dovrebbe seguire e attaccare la giusta etichetta in cima al pacchetto. Questa etichetta viene usata dalla rete MPLS per l'inoltro del pacchetto. Quando il pacchetto raggiunge l'altro bordo della rete MPLS, l'etichetta ha svolto il suo compito e quindi viene rimossa, rivelando di nuovo i pacchetti IP per la rete successiva; questo processo è mostrato nella Figura 5.63. Una differenza dai circuiti virtuali tradizionali è il livello di aggregazione: è certamente possibile per ogni flusso avere il proprio insieme di etichette attraverso la rete MPLS; tuttavia è più comune che i router aggreghino più flussi che finiscono in un router o in una LAN e usino una sola etichetta per tutti.

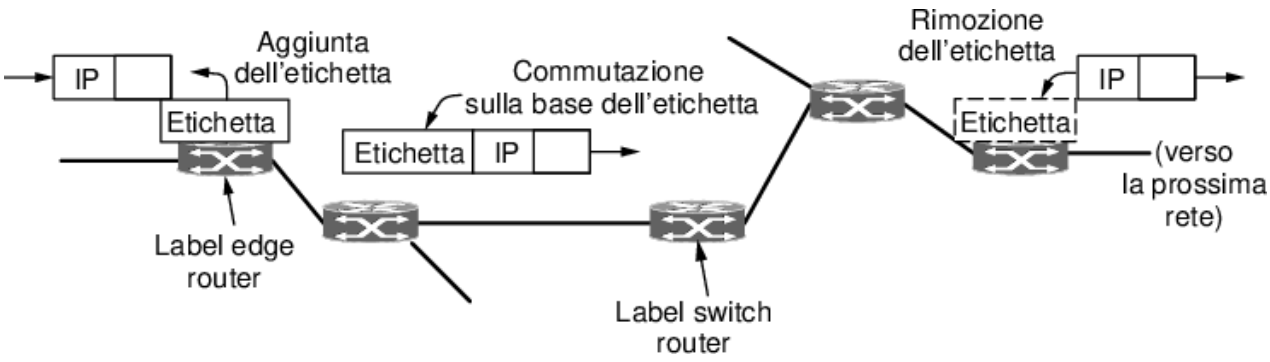


Figura 5.63 Inoltro di un pacchetto IP attraverso una rete MPLS.

I flussi raggruppati insieme sotto una singola etichetta sono detti appartenenti alla stessa **FEC** (*forwarding equivalence class*, classe equivalente di inoltro). Questa classe copre non solo la destinazione dei pacchetti, ma anche la loro classe di servizio (nel senso inteso dai servizi differenziati) perché tutti i pacchetti di un flusso sono trattati nello stesso modo durante l'inoltro.

Con il routing basato sui circuiti virtuali non è possibile raggruppare sotto lo stesso identificatore di circuito virtuale più percorsi distinti con punti finali diversi, perché non esiste un modo per distinguere l'uno dall'altro una volta arrivati alla destinazione finale. Con MPLS i pacchetti contengono ancora il loro indirizzo di destinazione finale oltre all'etichetta, perciò alla fine del percorso etichettato è possibile rimuovere l'intestazione di etichetta e continuare l'inoltro nel solito modo, usando l'indirizzo di destinazione del livello di rete.

Effettivamente, MPLS si spinge ancora più in là: può operare su più livelli contemporaneamente aggiungendo più di un'etichetta all'inizio del pacchetto. Si supponga, per esempio, che ci siano già molti pacchetti con etichette diverse (perché da qualche parte nella rete vogliamo trattare i pacchetti diversamente) che dovrebbero seguire un percorso comune per la destinazione. Invece di configurare molti percorsi a commutazione di etichetta, uno per ogni differente etichetta, possiamo configurarne uno solo. Quando pacchetti già etichettati arrivano all'inizio del percorso, un'altra etichetta viene aggiunta in testa, creando una pila di etichette. L'etichetta più esterna guida i pacchetti lungo il percorso e viene rimossa alla fine del cammino; le etichette rimaste, se ci sono, vengono usate per gli inoltri successivi. Il bit *S* mostrato nella Figura 5.62 permette a un router che rimuove un'etichetta di sapere se dietro ce ne sono altre; il suo valore è 1 per l'etichetta posta più in basso e 0 per tutte le altre.

La domanda finale che ci poniamo è come configurate le tabelle di inoltro delle etichette in modo che i pacchetti le seguano. Questa è una grande differenza tra MPLS e le architetture a circuito virtuale convenzionali. Nelle reti a circuito virtuale tradizionali, quando un utente vuole stabilire una connessione, viene immesso nella rete un pacchetto di impostazione per creare il percorso e aggiungere le voci nelle tabelle di inoltro. MPLS non coinvolge gli utenti nella configurazione; chiedere agli utenti di fare qualcosa di più che inviare datagrammi richiederebbe la modifica di gran parte del software esistente su Internet.

Al contrario, le informazioni di inoltro sono configurate da protocolli che sono una combinazione di protocolli di routing e protocolli di configurazione della connessione. Questi protocolli di controllo sono chiaramente separati dalle operazioni di inoltro, il che permette di usare molti protocolli di controllo diversi. Una delle possibili varianti funziona in questo modo: quando un router viene avviato cerca di capire quali sono i percorsi per i quali esso rappresenta la destinazione finale (per esempio, determina quali prefissi appartengono alle sue interfacce); quindi crea una o più FEC per queste destinazioni, assegna un'etichetta a ognuna di esse e le passa ai suoi vicini. Questi, a loro volta, inseriscono le etichette nelle loro tabelle di inoltro e inviano nuove etichette ai loro vicini, finché tutti i router non abbiano acquisito il percorso. Per garantire una qualità del servizio appropriata, anche le risorse possono essere riservate durante la costruzione del percorso.

Sebbene l'idea alla base di MPLS sia semplice, i dettagli sono estremamente complicati a causa di molte variazioni e casi d'uso che sono attualmente oggetto di studio. Per maggiori informazioni si possono consultare Davie e Farrel (2008) e Davie e Rekhter (2000).

OSPF – Protocollo di routing per gateway interni

Dopo aver concluso lo studio dei protocolli di controllo di Internet è giunto il momento di passare all'argomento successivo: il routing in Internet. Come è stato detto precedentemente, Internet è una rete composta da un gran numero di reti indipendenti o **AS** (*autonomous system,* sistema autonomo) gestite da diverse organizzazioni, di solito aziende, università o ISP. All'interno della propria rete, un ente può adottare un proprio algoritmo di routing interno, o **routing intradominio**, com'è più noto. C'è tuttavia solo un pugno di protocolli standard comunemente usati. Questo paragrafo esamina il routing intradominio e il protocollo OSPF, largamente adottato nella pratica. Un protocollo di routing intradominio è detto anche **interior gateway protocol** *(protocollo di routing per gateway interno)*. Nel prossimo paragrafo studieremo il routing tra reti indipendenti o **routing interdominio**. Tutte le reti devono usare lo stesso protocollo di routing interdominio o **exterior gateway protocol** (*protocollo di routing per gateway esterno*). Il protocollo di routing interdominio usato in Internet è BGP (*border gateway protocol*).

I primi protocolli di routing intradominio usavano un approccio distance vector, fondato sull'algoritmo distribuito di Bellman-Ford ereditato da ARPANET. RIP (*routing information protocol*) è il principale esempio usato fino a oggi. Funzionava bene nei sistemi piccoli, meno bene nelle reti più grandi; soffre anche del problema del conteggio all'infinito e di una convergenza lenta. In ARPANET, nel maggio 1979, venne sostituito da un protocollo link state. Nel 1988 IETF iniziò a lavorare su un routing intradominio link state, chiamato **OSPF** (*open shortest path first*), diventato uno standard nel 1990. Questo influenzò anche un protocollo chiamato **IS-IS** (*intermediate system to intermediate system*) che divenne uno standard OSI. Data la radice comune, i due protocolli sono più simili che diversi; per una storia completa si può consultare l'RFC 2328. OSPF e IS-IS sono i protocolli di routing intradominio dominanti e oggi sono supportati dalla maggior parte dei produttori di router; OSPF è più usato nelle reti aziendali, mentre IS-IS è più usato dagli ISP. Questo paragrafo spiega brevemente come funziona OSPF.

Data la lunga esperienza con gli altri protocolli di routing, il gruppo che stava progettando il nuovo protocollo aveva un lungo elenco di requisiti che doveva soddisfare. Prima di tutto, l'algoritmo doveva essere pubblicato in letteratura senza vincoli di brevetto, da qui la lettera "O" (open, aperto) dell'acronimo OSPF. Una soluzione proprietaria posseduta da un'azienda non sarebbe stata accettabile. Secondo, il nuovo protocollo doveva supportare diverse metriche di distanza, per esempio la distanza fisica, il ritardo e così via. Terzo, doveva essere un algoritmo dinamico, in grado di modificarsi rapidamente e automaticamente in base alla topologia.

Quarto (e nuovo per OSPF) doveva supportate il routing basato sul tipo di servizio. Il nuovo protocollo doveva essere in grado di instradare il traffico in tempo reale in un modo e il traffico di altro tipo diversamente. A quel tempo il protocollo IP aveva un campo *Type of service,* ma nessun protocollo di routing esistente lo utilizzava; questo campo fu incluso in OSPF, ma poiché continuava a essere inutilizzato, alla fine venne rimosso. Forse questa richiesta precorse i tempi perché venne prima del lavoro di IETF sui servizi differenziati, che ha dato nuova vita alle classi di servizio.

Quinto (e collegato a quanto appena detto), il nuovo protocollo doveva eseguire il bilanciamento del carico, ossia suddividere il carico su più linee. La maggior parte dei protocolli precedenti inviava tutti i pacchetti lungo il percorso migliore, anche se c'erano due percorsi ugualmente

buoni, il secondo non era mai utilizzato. In molti casi, suddividere il carico su più linee porta a prestazioni migliori.

Sesto, era necessario supportare i sistemi gerarchici. Nel 1988 Internet era diventata così grande che nessun router poteva conoscerne l'intera topologia. Il nuovo protocollo di routing doveva essere progettato in modo che nessun router avrebbe dovuto farlo.

Settimo, era necessario implementare un po' di sicurezza per impedire agli studenti in caccia di facile divertimento di imbrogliare i router inviando loro false informazioni di routing. Infine, era necessario provvedere alla gestione dei router collegati a Internet tramite tunnel. I protocolli precedenti non gestivano bene questa situazione.

OSPF supporta sia collegamenti punto a punto (per esempio SONET) sia reti broadcast (per esempio, la maggior parte delle LAN). In effetti, supporta reti con più router che possono comunicare direttamente tra di loro (chiamate **reti multiaccesso**) anche se questi non sono in grado di fare trasmissioni broadcast. I protocolli precedenti non gestivano bene questo caso.

La Figura 5.64(a) mostra la rete di un AS. Gli host sono stati omessi in quanto generalmente non hanno alcun ruolo in OSPF, mentre i router e le reti (che contengono gli host) lo hanno. La maggior parte dei router nella Figura 5.64(a) è connessa ad altri router da collegamenti punto a punto e a reti di cui raggiungere gli host. Tuttavia, i router *R3*, *R4* e *R5* sono connessi da una LAN broadcast come una Ethernet comutata.

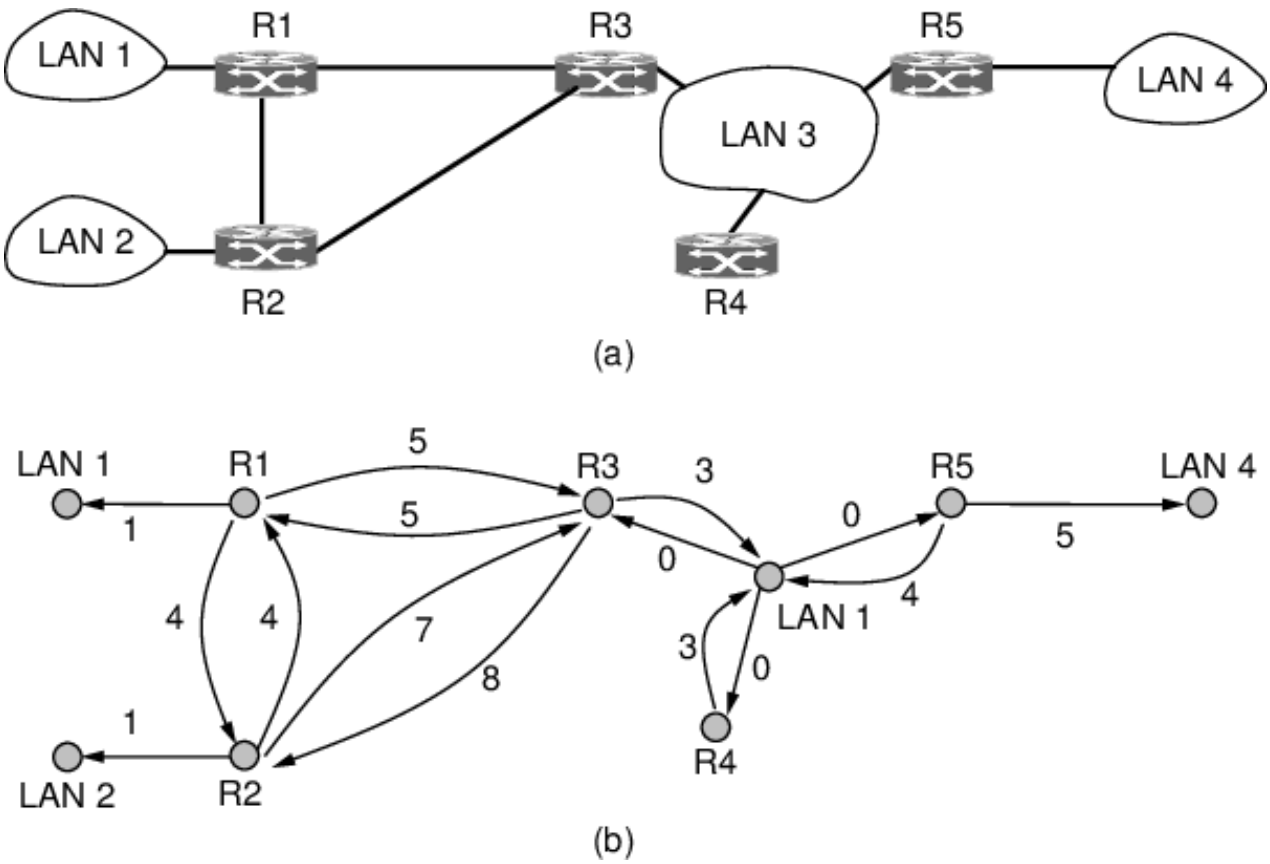


Figura 5.64 (a) Un AS. (b) Una rappresentazione di (a) tramite un grafo.

OSPF opera riassumendo l'insieme di reti reali, router e linee in un grafo orientato in cui a ogni arco è assegnato un costo (distanza, ritardo e così via), quindi calcola il percorso più breve in base al peso degli archi. Una connessione punto a punto tra due router è rappresentata da una coppia di archi, uno per ogni direzione; i loro pesi possono essere diversi. Una rete broadcast è rappresentata da un nodo per la rete stessa, più un nodo per ogni router. Gli archi dal nodo di rete ai router hanno peso 0 e sono senza dubbio importanti, perché altrimenti non esisterebbero percorsi attraverso la rete. Altre reti, che hanno solo host, hanno solo archi che li raggiungono e non che partano da essi; questa struttura definisce percorsi verso gli host, ma non attraverso essi.

La Figura 5.64(b) mostra il grafo della rete rappresentata nella Figura 5.64(a). Ciò che OSPF fondamentalmente fa è rappresentare la rete reale come un grafo simile a questo e poi elaborare il percorso più breve da ogni router a ogni altro router tramite un algoritmo di link state. Sono reperibili più percorsi della stessa lunghezza. In questo caso OSPF ricorda l'intero insieme dei percorsi più brevi e durante l'inoltro del pacchetto il traffico viene suddiviso su di essi. Questa procedura aiuta a bilanciare il carico ed è chiamata **ECMP** (*equal cost multipath*).

Molti AS di Internet sono a loro volta grandi e difficili da gestire. OSPF permette di dividere tali AS in più **aree** numerate, dove ogni area è una rete o un insieme di reti contigue. Le aree non si sovrappongono, ma non hanno bisogno di coprire tutta la rete; cioè alcuni router (o parte delle schede di un router) possono non appartenere ad alcuna area. I router che stanno completamente in un'area sono detti **router interni** (*internal router*). Un'area è una generalizzazione di una rete individuale: dall'esterno, sono visibili le sue destinazioni ma non la topologia e questo contribuisce alla scalabilità del routing.

Ogni AS possiede un'**area di dorsale** (*backbone area*), chiamata area 0 e i router in questa area sono chiamati **router di dorsale** (*backbone router*). Tutte le altre aree sono collegate alla dorsale, per esempio con un tunnel, perciò è possibile passare da un'area a qualunque altra area dello stesso AS attraverso la dorsale. Un tunnel è rappresentato nel grafo come un arco che ha un costo. Come per le altre aree, la topologia della dorsale non è visibile al di fuori dell'area di dorsale.

Ogni router connesso a due o più aree viene chiamato **router di area di confine** (*area border router*) e deve anche fare parte della dorsale. Il suo compito è elencare le destinazioni di un'area e inviarle alle aree a cui è connesso. Questo elenco include anche informazioni di costo, ma non i dettagli della topologia all'interno dell'area. Le informazioni di costo permettono agli host nelle altre aree di trovare il miglior router di confine da usare per entrare in un'area. Non passare informazioni sulla topologia riduce il traffico e semplifica l'elaborazione dei percorsi più brevi verso i router delle altre aree. Tuttavia, se esiste un solo router di confine fuori da un'area, anche tale elenco non ha bisogno di essere trasmesso: i percorsi verso destinazioni fuori dall'area iniziano sempre con l'indicazione "va al router di confine". Questo tipo di area è chiamato **stub area**.

L'ultimo tipo di router è il **router di confine dell'AS** (*AS boundary router*). Trasmette nell'area i percorsi verso destinazioni esterne poste in altri AS. I percorsi esterni, quindi, appaiono come le destinazioni e possono essere raggiunti tramite il router di confine dell'AS con un costo. Un cammino esterno può essere trasmesso da uno o più router di confine dell'AS. Le relazioni tra AS, aree e i vari tipi di router sono mostrati nella Figura 5.65. Un router può avere molti ruoli; per esempio, un router di confine è anche un router dorsale.

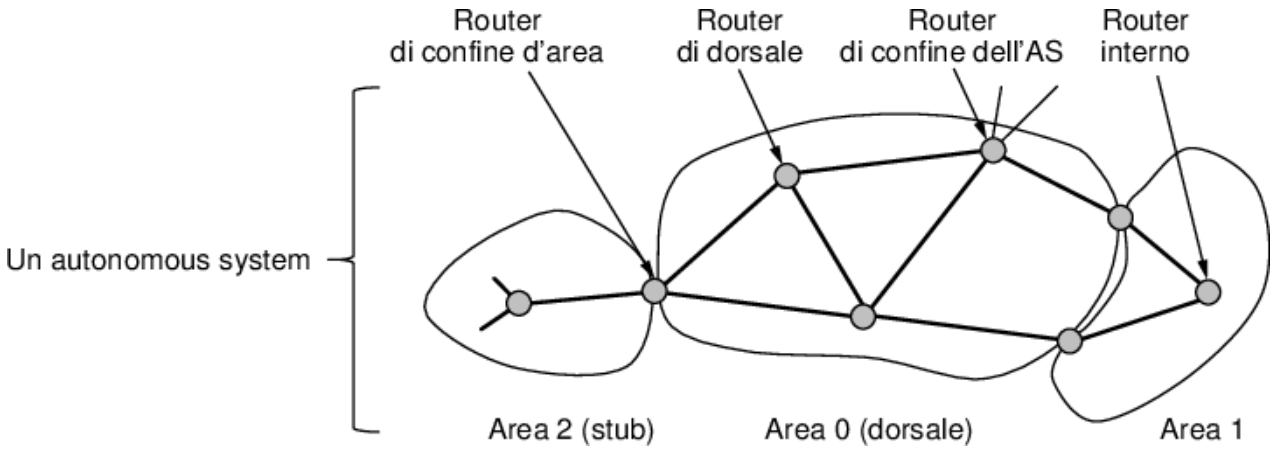


Figura 5.65 La relazione tra AS, dorsali e aree in OSPF.

Durante il funzionamento normale, ogni router ha lo stesso database degli stati dei collegamenti ed esegue lo stesso algoritmo di individuazione del percorso più breve; il suo compito principale è calcolare il percorso più breve da sé stesso a ogni altro router nell'AS. Un router di confine dell'area ha bisogno dei database di tutte le aree a cui è connesso e deve eseguire separatamente un diverso algoritmo di ricerca del percorso più breve per ogni area.

Il cammino migliore viene scelto per una sorgente e una destinazione nella stessa area (intra-area). Per una sorgente e una destinazione in aree diverse il cammino inter-area va dalla sorgente alla dorsale, attraverso la dorsale fino all'area di destinazione e infine alla destinazione. Questo algoritmo crea su OSPF una configurazione a stella, con la dorsale che svolge la funzione di punto centrale e le altre aree che ne sono le derivazioni. Poiché viene scelto il cammino a costo inferiore, i router in parti diverse della rete possono usare diversi router di confine dell'area per entrare nella dorsale e nell'area di destinazione. I pacchetti sono inoltrati dalla sorgente alla destinazione così come sono; non sono incapsulati né trasmessi attraverso tunnel, a meno che l'area di destinazione non ne usi uno per collegarsi alla dorsale. Inoltre i percorsi verso le destinazioni esterne possono includere il costo esterno dal router di confine sul cammino esterno, se si vuole, o solo il costo interno fino all'AS.

All'avvio il router trasmette messaggi HELLO attraverso tutte le sue linee punto a punto e comunica questi messaggi in modalità multicast sulle LAN al gruppo composto da tutti gli altri router. Sulle WAN, per individuare chi deve essere contattato, il router ha bisogno di alcune informazioni di configurazione. In base alle risposte, ogni router scopre chi sono i suoi vicini; i router sulla stessa LAN sono tutti vicini.

OSPF funziona scambiando informazioni tra router adiacenti, il che non equivale a dire tra router vicini. In particolare, è inefficiente che ogni router su una LAN comunichi con ogni altro router sulla LAN. Per evitare questa situazione, un router è nominato **router designato** (*designated router*). Il router designato viene detto **adiacente** agli altri router della sua LAN e scambia informazioni con gli altri apparati. In effetti, agisce come un singolo nodo che rappresenta la LAN.

I router confinanti che non sono adiacenti non scambiano informazioni gli uni con gli altri. Un router di backup viene costantemente aggiornato per facilitare la transizione in caso di blocco del router principale designato.

Durante le normali operazioni, ogni router invia periodicamente in flooding messaggi LINK STATE UPDATE a ognuno dei suoi router adiacenti. Questi messaggi comunicano lo stato del dispositivo e i costi utilizzati nel database della topologia. La ricezione di acknowledgement rende questi messaggi affidabili. Ogni messaggio ha un numero di sequenza, perciò un router può scoprire se un LINK STATE UPDATE appena arrivato sia più vecchio o più nuovo di quello già posseduto. I router inviano questi messaggi anche quando una linea si attiva o si disattiva e quando il suo costo varia.

I messaggi DATABASE DESCRIPTION contengono i numeri di sequenza di tutte le voci relative allo stato dei collegamenti possedute in quel momento dal trasmittente. Confrontando i propri valori con quelli inviati dal router trasmittente, il ricevente può scoprire chi possiede i valori più recenti. Questi messaggi sono utilizzati quando una linea viene attivata.

Ognuno dei due partecipanti in uno scambio può richiedere all'altro alcune informazioni sullo stato dei collegamenti, usando i messaggi LINK STATE REQUEST. Il risultato di questo algoritmo è che ogni coppia di router adiacenti controlla chi dei due abbia i dati più recenti; in questo modo le nuove informazioni si propagano attraverso l'area. Tutti questi messaggi sono inviati direttamente come pacchetti IP. La Figura 5.66 riepiloga i cinque tipi di messaggi.

Tipo di messaggio	Descrizione
Hello	Utilizzato per scoprire chi sono i vicini
Link state update	Comunica ai vicini i costi del trasmittente
Link state ack	Conferma l'aggiornamento dello stato del collegamento
Database description	Annuncia gli aggiornamenti posseduti dal trasmittente
Link state request	Richiede informazioni al partner

Figura 5.66 I cinque tipi di messaggi OSPF.

Finalmente possiamo mettere insieme tutti i pezzi. Il flooding permette a ogni router di comunicare a tutti gli altri router della sua area chi sono i vicini e i costi utilizzati. Queste informazioni consentono a ogni router di costruire il grafo dell'area e di calcolare il percorso più breve. Anche l'area dorsale fa lo stesso. Inoltre i router della dorsale accettano informazioni dai router di confine d'area in modo da poter calcolare il percorso migliore diretto da ogni router di dorsale a ogni altro router. Queste informazioni si propagano indietro verso i router di confine

d'area, che le annunciano dentro le loro aree. Usando questi dati, un router interno può selezionare il percorso migliore verso una destinazione esterna all'area e quindi il router di uscita verso la dorsale.

BGP – Protocollo di routing per gateway esterni

Dentro un singolo AS i protocolli di routing comunemente usati sono OSPF e IS-IS. Tra i vari AS si utilizza un protocollo diverso chiamato **BGP** (*border gateway protocol*), necessario perché gli obiettivi sono differenti. Un protocollo per gateway interni non deve far altro che spostare i pacchetti nel modo più efficiente possibile dalla sorgente alla destinazione; non deve preoccuparsi degli aspetti politici.

I router che gestiscono i protocolli di routing per gateway esterni devono, di contro, preoccuparsi parecchio degli aspetti politici (Metz, 2001). Per esempio, un AS aziendale potrebbe voler inviare pacchetti a qualunque sito di Internet e ricevere pacchetti da qualunque altro sito di Internet; ma potrebbe anche non essere disposto a trasportare pacchetti generati in un AS estraneo e diretti a un altro AS estraneo, anche quando il suo AS si trova sul percorso più corto tra i due ("Il problema è loro, non nostro"). D'altro canto, potrebbe essere pronto a trasportare il traffico di passaggio per i suoi vicini o anche per altri AS specifici che hanno pagato per questo servizio. Le aziende telefoniche, per esempio, potrebbero essere felici di svolgere la funzione di carrier per i loro clienti, ma non per i clienti di altre aziende. I protocolli di routing interdominio in generale, e BGP in particolare, sono stati progettati per consentire il rispetto di molti tipi di criteri di routing nel traffico tra AS.

Criteri tipici coinvolgono considerazioni politiche, di sicurezza o economiche. Ecco di seguito alcuni esempi di vincoli applicati al routing.

1. Nessun traffico commerciale su reti per la ricerca.
2. Mai porre l'Iraq su un percorso che inizia dal Pentagono.
3. Usare TeliaSonera invece di Verizon perché meno cara.
4. Non usare AT&T in Austrialia perché ha basse prestazioni.
5. Il traffico generato o in arrivo a Apple non dovrebbe passare attraverso Google.

Come si può immaginare da questa lista, le politiche di routing possono essere molto individuali. Sono spesso proprietarie perché contengono informazioni economiche sensibili. Tuttavia, ne possiamo descrivere alcune caratteristiche che rendono conto delle ragioni delle aziende e che sono spesso usate come punto di partenza.

Una politica di routing viene implementata decidendo quale traffico può fluire su quali linee tra AS. Una politica comune consiste nel fatto che un cliente di un ISP paghi un altro ISP per consegnare pacchetti a qualunque altra destinazione di Internet e ricevere i pacchetti inviati da qualunque altra destinazione. Si dice che l'ISP cliente compra un **servizio di transito** (*transit service*) da un ISP fornitore. È esattamente come quando un cliente domestico compra un servizio di accesso Internet da un ISP. Perché funzioni, il provider dovrebbe diffondere i percorsi per tutte le destinazioni di Internet al cliente sulle linee che li collegano. In questo modo il cliente avrà un percorso da usare per spedire pacchetti ovunque voglia. Dall'altra parte, il cliente dovrebbe diffondere i percorsi solo delle destinazioni sulla propria rete al provider. Questo permetterebbe al fornitore di inviare traffico al cliente solo per questi indirizzi; il cliente non vuole dover gestire traffico per altre destinazioni.

Un esempio di servizio di transito è illustrato nella Figura 5.67. Ci sono quattro AS connessi. La connessione spesso viene effettuata tramite una linea a un **IXP** (*internet exchange point*) a cui molti ISP si collegano per collegarsi ad altri ISP. *AS2*, *AS3* e *AS4* sono clienti di *AS1* perché comprano un servizio di transito da esso. Quindi, quando la sorgente *A* trasmette alla destinazione *C*, i pacchetti viaggiano da *AS2* ad *AS1* e finalmente arrivano ad *AS4*. La diffusione del routing viaggia in direzione opposta ai pacchetti. *AS4* avverte *C* come destinazione al suo provider di transito, *AS1*, di lasciare che le sorgenti raggiungano *C* attraverso *AS1*. Dopo *AS1* diffonde un percorso verso *C* a tutti gli altri clienti, incluso *AS2*, perché i clienti sappiano che possono inviare traffico a *C* attraverso *AS1*.

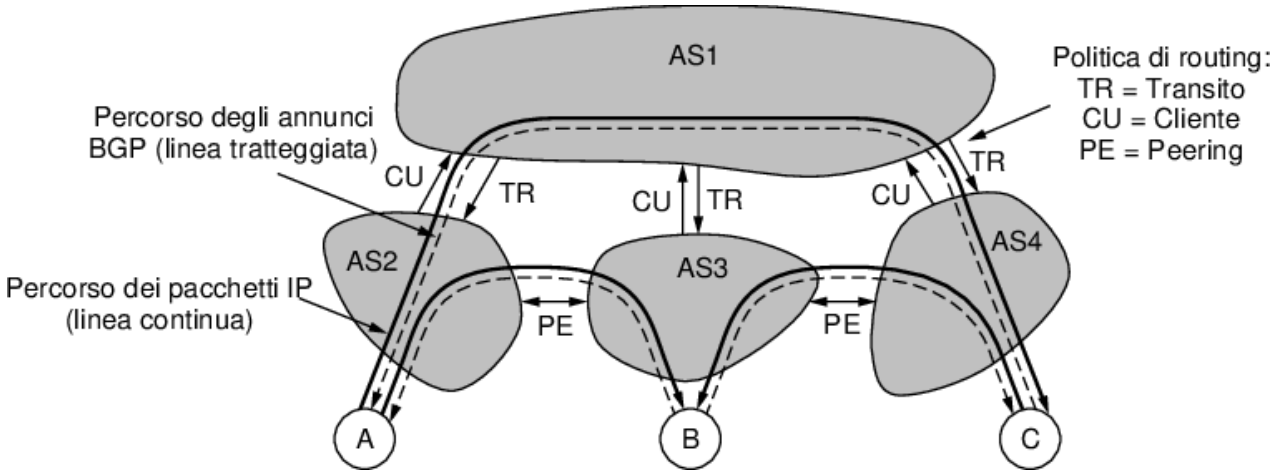


Figura 5.67 Politiche di routing tra quattro autonomous system.

Nella Figura 5.67 tutti gli altri AS comprano un servizio di transito da *AS1*. Questo fornisce connettività in modo da poter interagire con qualunque host di Internet. Tuttavia per avere questo privilegio devono pagare. Si supponga che *AS2* e *AS3* si scambino molto traffico. Dato che le loro reti sono già connesse, se lo vogliono, possono usare una differente politica: possono scambiarsi traffico direttamente tra di loro senza pagare. Questa procedura riduce la quantità di traffico che *AS1* consegna al loro posto e si spera riduca il loro conto da pagare. Questa politica è chiamata **peering**.

Per implementare il peering due AS si scambiano le tabelle di routing per gli indirizzi che stanno nelle loro reti. Ciò permette ad *AS2* di inviare pacchetti di *AS3* da *A* a *B* e viceversa. Tuttavia si noti che il peering non è transitivo. Nella Figura 5.67 anche *AS3* e *AS4* fanno peering tra di loro, permettendo che il traffico da *C* a *B* venga inviato direttamente ad *AS4*. Ma che cosa accade se *C* manda un pacchetto ad *A*? *AS3* è stato avvisato solo del percorso a *B* ad *AS4*, non ad *A*, di conseguenza il traffico non passerà da *AS4* ad *AS3* ad *AS2*, anche se esiste un percorso fisico. Questa restrizione è esattamente ciò che vuole *AS3*, che fa peering con *AS4* per scambiare traffico, ma non vuole trasportare traffico da *AS4* ad altre parti di Internet perché non è pagato per farlo. Al contrario *AS4* prende servizio di transito da *AS1*, quindi sarà *AS1* a trasportare il pacchetto da *C* ad *A*.

Ora che conosciamo il transito e il peering possiamo anche vedere che *A*, *B* e *C* hanno degli accordi di transito. Per esempio, *A* deve comprare un accesso a Internet da *AS2*; *A* potrebbe essere un unico computer domestico oppure una rete aziendale con molte LAN. Tuttavia, non ha bisogno di eseguire BGP perché è una rete periferica connessa al resto di Internet da una sola linea. Quindi l'unico modo per inviare pacchetti fuori dalla rete è usare il collegamento verso *AS2*. Non c'è altro modo per farlo. Questo percorso può essere implementato semplicemente definendolo come percorso di default. Per questa ragione *A*, *B* e *C* non sono state mostrate come AS che partecipano in un routing interdominio.

D'altra parte, alcune reti aziendali sono connesse a più ISP. Questa tecnica è usata per aumentare l'affidabilità, in quanto se un cammino attraverso un ISP cade, l'azienda può usare un percorso attraverso un altro ISP. Questa tecnica è chiamata **multihoming**. In questo caso la rete aziendale probabilmente esegue un protocollo di routing interdominio (per esempio BGP) per dire ad altri AS quali indirizzi dovrebbero essere raggiunti attraverso quali collegamenti.

Sono possibili molte varianti di queste politiche di transito e di peering; quelle viste fin qui bastano già a dimostrare come le relazioni commerciali e i controlli sui percorsi intrapresi dagli annunci possono implementare molti tipi di politiche. Consideriamo ora più in dettaglio come i router che eseguono la diffusione BGP dei cammini tra di loro selezionano i percorsi su cui inoltrare i pacchetti.

BGP è un protocollo di tipo distance vector, ma è abbastanza diverso dai protocolli interdominio basati anch'essi su distance vector, quale RIP. Abbiamo già visto che le politiche e non la distanza vengono usate per selezionare i percorsi da impiegare. Un'altra grande differenza è che i router BGP, invece di tenere traccia solo del costo di un cammino verso ogni destinazione, memorizzano tutto il cammino usato. Questo approccio è chiamato **path vector protocol** (*protocollo a vettore di percorso*). Il cammino consiste nel router del prossimo salto (che potrebbe trovarsi dall'altra parte dell'ISP, non adiacente) e la sequenza di AS, chiamata anche **AS path**, che il percorso ha seguito (data in ordine inverso). Infine coppie di router BGP comunicano tra di loro stabilendo connessioni TCP. Questo modo di operare fornisce comunicazioni affidabili e nasconde tutti i dettagli della rete che si attraversa.

Un esempio di diffusione dei percorsi BGP è mostrato nella Figura 5.68. Ci sono tre AS e quello al centro fornisce un servizio di transito agli ISP di destra e di sinistra. Una notifica di percorso verso il prefisso *C* inizia in *AS3*; è stata propagata attraverso la linea verso *R2c* in cima alla figura e ha un AS path formato semplicemente da *S3* e dal prossimo router dopo *R3a*. Nella parte bassa della figura la notifica ha lo stesso AS path, ma con un differente salto successivo, perché è arrivata da un collegamento diverso. Questa notizia continua propagarsi e attraversa i confini di *AS1*. Al router *R1a*, in cima alla la figura, l'AS path è: *AS2*, *AS3* e il salto successivo è *R2a*.

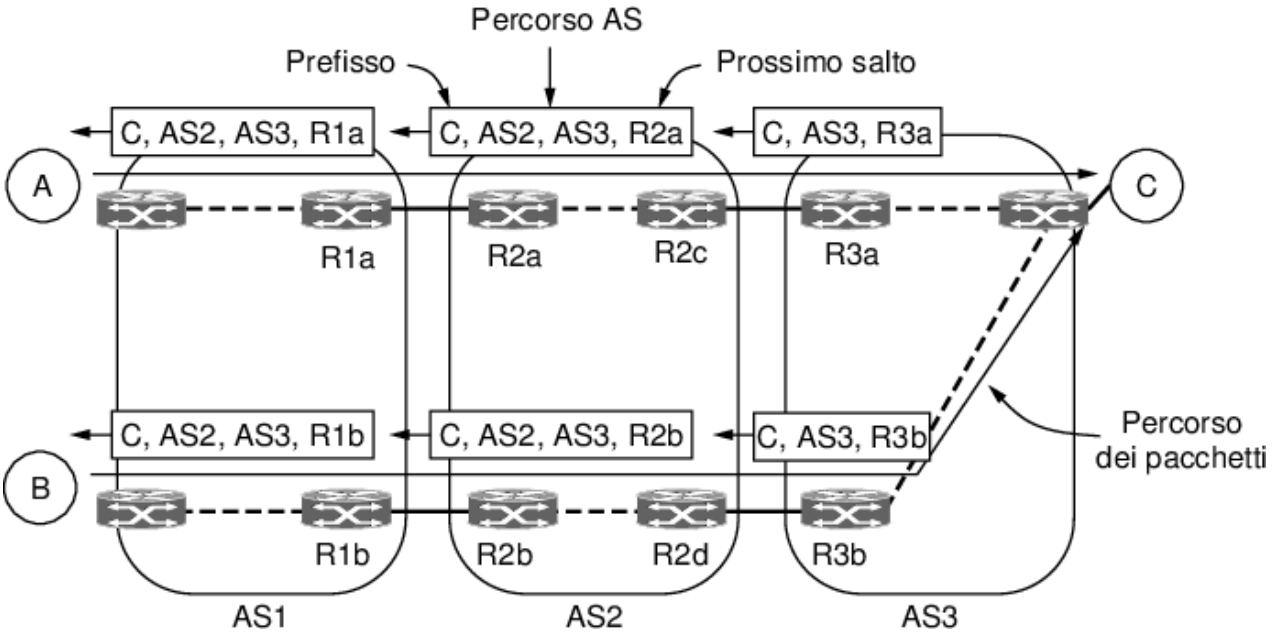


Figura 5.68 Propagazione degli annunci di percorsi BGP.

Far arrivare il percorso completo con le informazioni di routing rende facile al router ricevente rilevare e rompere i cicli. La regola è che ogni router che invia una notifica di percorso fuori dall'AS mette il numero del suo AS in cima alla lista; questo è il motivo per cui la lista è in ordine inverso. Quando un router riceve un percorso, controlla se il numero del suo AS è già nell'AS path. Se ciò accade viene rilevato un ciclo e la notizia viene scartata. Tuttavia, e in modo ironico, nei tardi anni '90 si realizzò che, nonostante questa precauzione, BGP soffriva di una variante del problema del conteggio all'infinito (Labovitz et al., 2001); i cicli non hanno una vita lunga, ma i percorsi a volte convergono lentamente e presentano cicli transitori.

Formulare una lista di AS è un modo molto grezzo per specificare un percorso. Un AS potrebbe essere una piccola azienda o una dorsale internazionale. Non c'è modo di capirlo dal percorso e BGP non ci tenta nemmeno, perché AS diversi possono usare diversi protocolli intradominio i cui costi potrebbero non essere comparabili. Se anche potessero esserlo, un AS potrebbe non voler rilevare le proprie metriche interne. Questo è uno dei motivi per cui i protocolli di routing interdominio differiscono da quelli intradominio.

Finora abbiamo visto come la notifica di un percorso venga inviata attraverso il collegamento tra due ISP. Dobbiamo ancora trovare un modo per propagare i percorsi BGP da un lato dell'ISP all'altro, in modo che possano essere spediti all'ISP successivo. Questo compito potrebbe essere dato al protocollo interdominio ma, poiché BGP presenta una scalabilità molto buona sulle reti grandi, viene spesso usata una variante BGP chiamata **iBGP** (*internal BGP*, BGP interno) per distinguerla dall'uso regolare di BGP chiamato anche **eBGP** (*external BGP*, BGP esterno).

La regola per propagare i percorsi all'interno di un ISP è che i router di confine dell'ISP imparino, per consistenza, tutti i cammini visti da tutti gli altri router di confine. Se un router di confine di un ISP viene a conoscenza di un prefisso verso IP 128.208.0.0/16, gli altri router verranno a

conoscenza di questo prefisso. Il prefisso sarà quindi raggiungibile da tutti i punti dell'ISP, indipendentemente da come i pacchetti siano entrati nell'ISP da altri AS.

Per evitare confusione nella Figura 5.68 non abbiamo mostrato questa propagazione ma, per esempio, il router *R2b* saprà che può raggiungere *C* sia attraverso il router *R2c* in alto sia attraverso il router *R2d* in fondo alla figura. Il salto successivo è aggiornato quando il cammino attraversa l'ISP, in modo che i router sul lato lontano dell'ISP sappiano quale router usare per uscire dall'ISP dall'altro lato. Questo può essere visto come il cammino più a sinistra in cui il salto successivo porta a un router nello stesso ISP e non a un router nell'ISP successivo.

Possiamo ora descrivere il punto chiave mancante: come i router BGP scelgono quale percorso usare per ogni destinazione. Ogni router BGP può venire a conoscenza di un percorso per una certa destinazione da un router connesso all'ISP successivo e da tutti i router di confine (che hanno avuto notizie di cammini differenti dai router connessi ad altri ISP). Il router deve decidere quale percorso tra tutti è il migliore da utilizzare. La risposta finale è che è l'ISP a decidere la policy con cui scegliere i percorsi preferenziali. Tuttavia, questa spiegazione è molto generale e non del tutto soddisfacente, quindi descriviamo almeno alcune strategie comuni.

La prima strategia afferma che i percorsi tramite reti in peering sono preferibili a quelli tramite provider di transito; i primi non costano nulla, mentre i secondi sono a pagamento. Una strategia simile indica che i percorsi dei clienti hanno la preferenza più alta. Spedire il traffico dei clienti paganti direttamente è una buona politica commerciale.

Una strategia di tipo diverso è rappresentata dalla regola per cui gli AS path sono i migliori. Questa strategia è opinabile, dato che un AS potrebbe essere una rete di qualunque dimensione, quindi un cammino attraverso tre piccoli AS potrebbe in pratica essere più breve di un cammino attraverso un solo grande AS. Tuttavia, quelli brevi in media tendono a essere migliori e questa regola è un modo comune di vincere le indecisioni.

La strategia finale è preferire il percorso con un costo più basso all'interno dell'ISP. Questa strategia è implementata nella Figura 5.68. I pacchetti inviati da *A* a *C* escono da *S1* tramite router in cima, *R1a*. I pacchetti inviati da *B* escono dal router in basso, *R1b*. La ragione è che sia *A* sia *B* stanno prendendo il percorso a costo minimo o quello più veloce per uscire da *AS1*. Poiché sono in punti differenti dell'ISP, l'uscita più veloce per ognuno è diversa. La stessa cosa accade quando i pacchetti passano attraverso *AS2*. Nell'ultima parte, *AS3* deve trasportare il pacchetto di *B* attraverso la sua stessa rete.

Questa strategia è chiamata **early exit** (*uscita precoce*) o **hot-potato routing** (*routing della patata bollente*) e ha il curioso effetto collaterale di rendere i percorsi asimmetrici. Si consideri ad esempio il percorso intrapreso quando *C* invia un pacchetto indietro a *B*; tale pacchetto uscirà da *AS3* velocemente, dal router in cima, per non sprecare risorse. In modo simile resterà nella parte alta della figura quando *AS2* lo passerà ad *AS1* più velocemente possibile, quindi il pacchetto dovrà fare un lungo viaggio in *AS1*. Questa è l'immagine speculare del percorso preso da *B* a *C*.

La discussione appena presentata dovrebbe rendere chiaro che ogni router BGP sceglie la propria strada migliore tra quelle possibili che conosce. Non capita, come ci si sarebbe aspettato, che BGP scelga un percorso da seguire a livello AS e OSPF scelga i percorsi all'interno di ogni AS. BGP e il protocollo di routing per gateway interni sono integrati in modo molto profondo. Questo significa che, per esempio, BGP può trovare il punto di uscita migliore da un ISP al successivo e

che questo vari attraverso l'ISP, come nel caso della politica hot-potato. Significa anche che router BGP posti in parti diverse di un AS possono scegliere cammini diversi per raggiungere la stessa destinazione. L'ISP deve stare attento a configurare tutti i router BGP in modo che, nonostante tutta questa libertà, effettuino scelte compatibili, ma questa è una cosa fattibile.

Sorprendentemente, abbiamo solo scalfito la superficie di BGP. Per ulteriori informazioni si possono vedere le specifiche di BGP versione 4 nell'RFC 4271 e quelli a esso correlati. Tuttavia, si noti che la maggior parte della complessità di questo protocollo è racchiusa nelle politiche che non sono descritte nelle specifiche del protocollo.

Multicast su Internet

La normale comunicazione IP avviene tra un trasmittente e un ricevente, ma per alcune applicazioni è utile che un processo possa comunicare contemporaneamente con un gran numero di destinazioni. Alcuni esempi sono lo streaming di eventi sportivi, la consegna di aggiornamenti software a un gruppo di server e la gestione di audioconferenze digitali in chiamate telefoniche.

IP supporta la trasmissione uno a molti (o multicast), usando indirizzi di classe D. Ogni indirizzo di classe D identifica un gruppo di host dove 28 bit sono utilizzati per identificare i gruppi, perciò possono esistere nello stesso momento più di 250 milioni di gruppi. Quando un processo invia un pacchetto a un indirizzo di classe D viene fatto un tentativo best effort per trasmettere i dati a tutti i membri del gruppo di destinazione, ma non viene data alcuna garanzia. Alcuni membri possono non ricevere il pacchetto.

Gli indirizzi IP 224.0.0.0/24 sono riservati per il multicast su rete locale. In questo caso non c'è bisogno di alcun protocollo di routing. I pacchetti vengono trasmessi in multicast, semplicemente con un broadcast sulla LAN con un indirizzo multicast. Tutti gli host ricevono i pacchetti broadcast e solo gli host membri del gruppo lo elaborano. I router non inoltrano il pacchetto fuori dalla LAN. Alcuni esempi di indirizzi multicast locali sono:

```
224.0.0.1    tutti gli apparati su una LAN

224.0.0.2    tutti i router su una LAN

224.0.0.5    tutti i router OSPF su una LAN

224.0.0.251    tutti i server DNS su una LAN.
```

Altri indirizzi multicast possono avere membri su reti diverse. In questo caso c'è bisogno di un protocollo di routing. Per prima cosa i router multicast devono sapere quali host sono membri di un gruppo. Il processo richiede al proprio host di unirsi a (o lasciare) un gruppo specifico. Ogni host tiene traccia a quali gruppi attualmente i propri processi appartengono. Quando l'ultimo processo di un host lascia un certo gruppo, l'host non è più parte di quel gruppo. Circa una volta al minuto ogni router multicast manda una richiesta a tutti gli host sulla sua LAN (usando l'indirizzo multicast 224.0.0.1, naturalmente) in cui chiede alle macchine di indicare i gruppi a cui

appartenengono. Ogni host risponde comunicando tutti gli indirizzi di classe D che gli interessano. Questi pacchetti di interrogazione e risposta utilizzano un protocollo chiamato **IGMP** (*internet group management protocol*), descritto nell'RFC 3376.

Qualunque routing multicast può essere usato per costruire gli spanning tree che danno i percorsi dalle sorgenti a tutti i membri di un gruppo. Gli algoritmi utilizzati sono quelli descritti nel Paragrafo 5.2.8. All'interno di un AS, il protocollo principale usato è **PIM** (*protocol independent multicast*), con alcune possibili varianti. In modalità densa (*dense mode PIM*) viene creato un albero potato di reverse path forwarding; è adatto a situazioni in cui i membri sono ovunque nella rete, come nel caso si voglia distribuire file a molti server all'interno della rete di un data center. Gli spanning tree nella modalità sparsa (*sparse mode PIM*) sono costruiti in modo simile a quelli core based e sono adatti a situazioni quali quella di un fornitore di contenuti che fa multicast di contenuti televisivi agli iscritti nella propria rete IP. Una variante, chiamata *source-specific multicast PIM*, è ottimizzata per il caso in cui vi sia una sola sorgente nel gruppo. Infine, è necessario utilizzare estensioni multicast di BGP o tunnel per creare percorsi multicast quando i membri del gruppo sono sparsi in più AS.

Mobile IP

Molti utenti di Internet usano computer portatili e vogliono rimanere sempre collegati quando si allontanano da casa e pure durante il loro ritorno. Sfortunatamente, il sistema di indirizzamento IP rende il lavoro lontano da casa più semplice in teoria che in pratica. Quando gli utenti hanno iniziato a chiedere comunque questa possibilità, IETF ha creato un gruppo di lavoro per trovare una soluzione. Il gruppo di lavoro ha rapidamente formulato diversi obiettivi considerati auspicabili in ogni situazione. I principali sono i seguenti.

1. Ogni host mobile deve poter utilizzare il proprio indirizzo IP sulla LAN di appartenenza ovunque si trovi.
2. Non apportare modifiche al software degli host fissi.
3. Non apportare modifiche al software dei router e alle loro tabelle.
4. Non consentire di fare deviazioni alla maggior parte dei pacchetti diretti agli host mobili.
5. Quando si trova a casa, l'host mobile non dovrebbe incorrere in alcun overhead.

La soluzione scelta è quella descritta nel Paragrafo 5.2.10. In breve, ogni organizzazione locale che vuole permettere ai propri utenti di fare roaming, deve mettere a disposizione un supporto chiamato **home agent** (*agente domestico*). Quando un host mobile compare in un sito remoto ottiene un nuovo indirizzo IP temporaneo (chiamato anche *care of address*) dal sito remoto. Quindi, il dispositivo mobile indica all'home agent dove si trova in quel momento, dandogli l'indirizzo temporaneo. Quando un pacchetto per l'host mobile arriva all'indirizzo di casa mentre è da qualche altra parte, l'home agent prende il pacchetto e lo invia via tunnel all'host mobile facendo riferimento al suo indirizzo temporaneo. L'host mobile può inviare pacchetti di risposta direttamente a chi vuole comunicare con esso, ma usando l'indirizzo sulla rete di appartenenza come indirizzo sorgente. Questa soluzione soddisfa tutti i requisiti elencati prima, tranne quello per cui i pacchetti per l'host mobile non dovrebbero fare deviazioni.

Ora che abbiamo trattato il livello di rete di Internet, possiamo approfondire questa soluzione. Il bisogno di un supporto per la mobilità deriva innanzitutto dal sistema di indirizzamento di IP. Ogni indirizzo IP contiene un numero di rete e un numero di host. Si consideri, per esempio, la

macchina associata all'indirizzo IP 160.80.40.20/16. Il numero di rete è rappresentato da 160.80 mentre 40.20 rappresenta la parte host. Tutti i router del mondo hanno tabelle di routing che indicano quale linea utilizzare per raggiungere la rete 160.80. Ogni volta che un router riceve un pacchetto destinato a un indirizzo IP espresso nella forma 160.80.xxx.yyy sa su che linea inviare i dati. Se, improvvisamente, la macchina con quell'indirizzo si spostasse in un sito lontano, i pacchetti diretti a essa continuerebbero a essere instradati verso la sua LAN (o router) di appartenenza.

A questo punto ci sono due opzioni, entrambe poco attraenti. La prima è che potremmo creare un percorso verso un prefisso più specifico. Cioè se il sito distante segnala un percorso a 160.80.40.20/32, i pacchetti inviati alla destinazione cominceranno di nuovo ad arrivare nel posto giusto. Questa opzione dipende dall'algoritmo usato sui router che trova la corrispondenza sul prefisso più lungo. Tuttavia, abbiamo aggiunto un percorso a un prefisso IP con un solo indirizzo IP dentro e tutti gli ISP del mondo apprenderanno questo prefisso. Se tutti cambiassero i percorsi a livello globale in questo modo quando spostano il proprio computer, ogni router avrebbe milioni di voci in tabella, con un costo astronomico per Internet. Questa opzione non è praticabile.

La seconda opzione è cambiare l'indirizzo IP dell'host mobile. È vero che i pacchetti inviati all'indirizzo IP sulla rete di appartenenza non verranno più consegnati finché tutti i database, programmi e persone rilevanti saranno informate del cambiamento, ma l'host mobile può ancora usare Internet nella nuova posizione per navigare sul Web ed eseguire altre applicazioni; questa opzione gestisce la mobilità a un livello più alto. È quello che accade tipicamente quando un utente porta il proprio computer portatile in un bar e usa Internet tramite la rete wireless locale. Lo svantaggio di questa opzione è che non permette ad alcune applicazioni di funzionare e non mantiene la connettività mentre l'host mobile si muove.

La mobilità può anche essere gestita al livello più basso: il livello data link. Questo accade quando si usa un computer portatile su una singola rete wireless 802.11: l'indirizzo IP non cambia e il percorso sulla rete rimane lo stesso; è il collegamento wireless a fornire mobilità. Tuttavia, il grado di mobilità è limitato; se il computer portatile si muove troppo lontano, dovrà connettersi a Internet tramite un'altra rete e con un indirizzo IP diverso.

La soluzione per IP mobile con IPv4 è data nell'RFC 3344. Funziona sfruttando il routing già esistente di Internet e permette agli host di rimanere connessi con il proprio indirizzo IP mentre si muovono. Perché funzioni l'host mobile deve essere in grado di capire quando si è mosso; ciò si ottiene tramite i messaggi ICMP di segnalazione e sollecito dei router. Gli host mobili ascoltano gli avvisi periodici dai router, poi inviano un sollecito per scoprire il router più vicino; se questo router non è al solito indirizzo nella rete di appartenenza, allora si trovano in una rete remota. Se il router è cambiato dall'ultima volta, l'host mobile si è mosso in una rete remota diversa. Questo stesso meccanismo permette agli host mobili di trovare i loro home agent.

Per avere un indirizzo IP temporaneo sulla rete remota l'host mobile può semplicemente utilizzare DHCP. Alternativamente, se gli indirizzi IPv4 sono scarsi, può spedire e ricevere i pacchetti tramite un agente remoto (*foreign agent*) che ha già un indirizzo IP su quella rete. L'host mobile trova il foreign agent usando lo stesso meccanismo ICMP impiegato per trovare l'home agent. Dopo che il mobile ha ottenuto un indirizzo IP o ha trovato un foreign agent, esso è in grado di usare la rete per inviare un messaggio al suo home agent e informarlo della sua posizione attuale.

L'home agent ha bisogno di poter intercettare i pacchetti inviati all'host mobile solo quando questo non è a casa. ARP fornisce un meccanismo adatto: per inviare un pacchetto su Ethernet a un host IP, il router ha bisogno di conoscere l'indirizzo Ethernet di quell'host. Il meccanismo attuale è che il router invii una richiesta ARP per chiedere, per esempio, qual è l'indirizzo Ethernet di 160.80.40.20. Quando l'host mobile è a casa, risponde alle richieste ARP per il suo indirizzo IP con il suo stesso indirizzo Ethernet. Quando è via, l'agente domestico risponde a questa richiesta dando il suo indirizzo Ethernet. Il router quindi invia pacchetti per 160.80.40.20 all'home agent. Se ricordate, questo è un proxy ARP.

Per aggiornare rapidamente le associazioni ARP quando l'host mobile lascia l'abitazione o vi ritorna, un'altra tecnica ARP è chiamata **gratuitous ARP** (*ARP gratuito*); fondamentalmente, l'host mobile o l'home agent mandano a se stessi una richiesta ARP per l'indirizzo IP mobile che fornisce la risposta giusta per permettere al router di aggiornare le sue associazioni.

L'operazione di tunneling per inviare un pacchetto dall'home agent all'indirizzo temporaneo dell'host mobile è fatta incapsulando il pacchetto con un'altra intestazione IP destinata all'indirizzo temporaneo. Quando un pacchetto incapsulato arriva all'indirizzo temporaneo, l'intestazione IP più esterna viene rimossa per rivelare il pacchetto.

Come con molti protocolli di Internet, il problema è nei dettagli e in particolare nella compatibilità con gli altri protocolli utilizzati. In questo caso siamo di fronte a due complicazioni. In primo luogo i dispositivi NAT dipendono dall'intestazione IP per vedere le intestazioni TCP o UDP; la versione originale di tunnel per IP mobile non usa queste intestazioni, quindi non funziona con il NAT. La soluzione è stata cambiare l'incapsulamento per includere un'intestazione UDP.

La seconda complicazione è che alcuni ISP controllano l'indirizzo IP sorgente del pacchetto per vedere se corrisponde a dove il protocollo di routing crede che la sorgente dovrebbe essere. Questa tecnica è chiamata **ingress filtering** (*filtro in ingresso*) ed è una misura di sicurezza per eliminare il traffico con indirizzi apparentemente scorretti che potrebbero rappresentare attacchi alla sicurezza. Tuttavia, i pacchetti inviati da un host mobile ad altri host su Internet quando si trova su una rete remota avranno un indirizzo IP sorgente fuori luogo, quindi saranno scartati. Per aggirare il problema, l'host mobile può usare l'indirizzo temporaneo come sorgente per fare tunnelling dei pacchetti indietro verso l'home agent. Da qui possono essere spediti all'interno di Internet da quella che appare essere la posizione corretta. Il prezzo da pagare è che il percorso diventa più tortuoso.

Un ulteriore problema riguarda la sicurezza. Quando l'home agent riceve un messaggio che gli chiede di inoltrare tutti i pacchetti di Roberta a un altro indirizzo IP, esso dovrebbe assecondare la richiesta solo quando sia assolutamente certo che la richiesta viene da Roberta, e nessun altro stia tentando di impersonare quell'utente. Per gestire queste situazioni si utilizzano protocolli di autenticazione crittografica (descritti nel Capitolo 8).

I protocolli di mobilità costruiti per IPv6 si fondano su IPv4. Lo schema precedente soffre di un problema di routing triangolare in cui i pacchetti spediti all'host mobile fanno una curva a gomito attraverso un home agent distante. In IPv6 i percorsi sono ottimizzati per seguire un cammino diretto tra host mobile e gli altri indirizzi IP, dopo che il pacchetto iniziale ha seguito la strada lunga. IPv6 mobile è descritto nell'RFC 3775.

C'è un altro tipo di mobilità definita per Internet. Alcuni aerei hanno una rete wireless all'interno e i passeggeri possono usarla per connettere i loro computer portatili a Internet. L'aereo ha un router che connette al resto di Internet tramite un collegamento wireless (vi aspettavate forse un collegamento cablato?). Quindi si ha un router volante, il che vuol dire che l'intera rete è mobile. I progetti di mobilità delle reti supportano questa situazione senza che i computer portatili capiscano che l'aereo è mobile; per quanto ci concerne è semplicemente un'altra rete. Naturalmente, alcuni computer portatili potrebbero usare l'IP mobile per mantenere il proprio indirizzo sulla rete di appartenenza mentre sono in aereo; quindi abbiamo due livelli di mobilità. La mobilità della rete per IPv6 è definita nell'RFC 3963.

5.7 Sommario

Il livello di rete fornisce servizi al livello di trasporto. Può basarsi su circuiti virtuali o datagrammi; in entrambi i casi, il suo compito principale è instradare i pacchetti dalla sorgente alla destinazione. Nelle reti a circuito virtuale la decisione di routing è presa durante l'impostazione del circuito virtuale, nelle reti a datagrammi la decisione è presa per ogni pacchetto.

Nelle reti di calcolatori si usano molti algoritmi di routing. Il flooding è un semplice algoritmo per inviare i pacchetti su tutti i percorsi. La maggior parte degli algoritmi trova il percorso più breve e si adatta ai cambiamenti della topologia di rete. Gli algoritmi di routing principali sono quello basato su distance vector e quello basato su link state; la maggior parte delle reti moderne usa uno di questi due. Altri punti importanti nel routing sono l'uso di gerarchie nelle reti di grandi dimensioni, il routing degli host mobili, il broadcast, il multicast e l'anycast.

Le reti possono facilmente congestionarsi, aumentando il ritardo e diminuendo la capacità di trasporto. I progettisti di rete tentano di evitare le congestioni progettando la rete in modo che abbia abbastanza capacità, scelga percorsi non congestionati, rifiuti di accettare più traffico del dovuto, segnali alle sorgenti di rallentare ed eliminando il carico.

Il passo successivo alla gestione della congestione è tentare di ottenere realmente la qualità di servizio promessa. Alcune applicazioni si preoccupano di più del throughput, mentre altre si preoccupano di più di ritardo e jitter. I metodi che possono essere utilizzati per fornire diverse qualità del servizio includono una combinazione di profilazione di traffico, prenotazione delle risorse sui router e controllo di ammissione. Gli approcci pensati per ottenere una buona qualità di servizio includono i servizi integrati (incluso RSVP) e i servizi differenziati IETF.

Le reti possono differire per molti aspetti, perciò quando molte reti vengono interconnesse possono nascere problemi. Quando reti diverse hanno una differente dimensione massima, per il pacchetto potrebbe essere necessario effettuare una frammentazione. Reti diverse possono usare internamente protocolli di routing differenti, ma hanno bisogno di eseguire un protocollo comune all'esterno. Qualche volta i problemi si possono ridurre inviando attraverso un tunnel i pacchetti che devono transitare per una rete ostile, ma l'approccio non funziona se la rete sorgente e quella di destinazione sono diverse.

Internet supporta una gran varietà di protocolli collegati al livello di rete. Tra questi, il protocollo di trasporto dati IP, ma anche i protocolli di controllo ICMP, ARP e DHCP. Un protocollo orientato alla connessione chiamato MPLS trasporta pacchetti IP attraverso alcune reti. Uno dei principali protocolli di routing utilizzati all'interno delle reti è OSPF; quello usato tra reti è BGP. Internet sta esaurendo rapidamente gli indirizzi IP, perciò è stata sviluppata una nuova versione di IP chiamata IPv6 che sta lentamente prendendo piede.

Problemi

1. Fornite due esempi di applicazioni di computer che si avvantaggiano di un servizio orientato alla connessione. Fornite poi due esempi dove è preferibile il servizio non orientato alla connessione.

2. Le reti a datagrammi instradano ogni pacchetto come unità separata, indipendente dalle altre. Le reti a circuito virtuale non hanno l'obbligo di seguire lo stesso approccio, poiché ogni pacchetto di dati segue un percorso predeterminato. Questa osservazione significa che le reti a circuito virtuale non hanno bisogno della capacità di instradare pacchetti isolati da una sorgente arbitraria a una destinazione arbitraria? Motivate la risposta.

3. Fornite tre esempi di parametri di protocollo che potrebbero essere negoziati durante l'impostazione di una connessione.

4. Supponendo che tutti i router e gli host funzionino correttamente e che tutto il software al loro interno non contenga errori, c'è qualche possibilità, seppur piccola, che un pacchetto venga trasmesso alla destinazione sbagliata?

5. Fornite una semplice euristica per trovare due percorsi attraverso una rete, tra una sorgente e una destinazione assegnate, che possano resistere alla perdita di una qualsiasi linea di comunicazione (supponete che esistano due percorsi di questo tipo). I router sono considerati abbastanza affidabili, perciò non è necessario preoccuparsi di possibili malfunzionamenti.

6. Si consideri la rete mostrata nella Figura 5.12(a). È utilizzato il distance vector routing e nel router *C* sono stati inseriti i vettori elencati di seguito. Da *B:* (5, 0, 8, 12, 6, 2); da *D:* (16,12, 6, 0, 9,10); da *E:* (7, 6, 3, 9, 0,4). I ritardi misurati associati da *C* a *B*, *D* ed *E*, sono rispettivamente 6, 3 e 5. Quale sarà la nuova tabella di routing di *C*? Indicate la linea di trasmissione da utilizzare e il costo previsto.

7. Se i costi sono memorizzati come numeri di 8 bit in una rete di 50 router e i vettori delle distanze sono scambiati due volte al secondo, quanta banda per una linea (full duplex) è erosa da un algoritmo di routing distribuito? Assumete che ogni router abbia tre linee verso gli altri router.

8. Nella Figura 5.13 l'OR logico delle due serie di bit ACF produce 111 in ogni riga. Il risultato è solo un caso o vale per tutte le reti in tutte le circostanze?

9. Per il routing gerarchico con 4.800 router, quali dimensioni di cluster e regione dovrebbero essere scelte per ridurre al minimo la dimensione della tabella di routing in una gerarchia a tre livelli? Un buon punto di partenza è l'ipotesi che una soluzione con k cluster di k regioni di k router sia quasi ottimale, quindi k è circa la radice cubica di 4.800 (circa 16). Controllate per tentativi le combinazioni quando tutti e tre i parametri sono vicini a 16.

10. Nel testo si afferma che quando un host mobile non è a casa, i pacchetti inviati alla sua LAN domestica sono intercettati dal suo agente fisso presente sulla LAN. Per una rete IP su una LAN 802.3, in che modo l'agente fisso compie questa intercettazione?

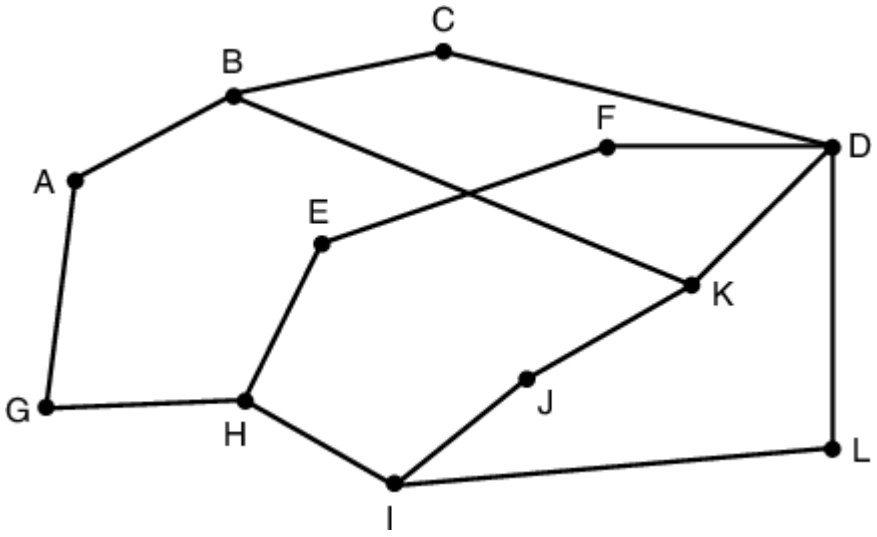
11. Osservando la rete mostrata nella Figura 5.6, quanti pacchetti sono generati da una trasmissione broadcast di *B*, nel caso di:

(a) un reverse path forwarding

(b) una struttura sink tree.

12. Si consideri la rete mostrata nella Figura 5.15(a). Si immagini di aggiungere una nuova linea tra *F* e *G* senza cambiare la struttura sink tree mostrata nella Figura 5.15(b). Quali modifiche devono essere apportate alla Figura 5.15(c)?

13. Calcolate uno spanning tree multicast per il router *C* nella rete seguente, per un gruppo con membri ai router *A*, *B*, *C*, *D*, *E*, *F*, *I* e *K*.



14. Si supponga che il nodo *B* nella Figura 5.20 si sia appena riavviato e che non abbia informazioni di routing nelle sue tabelle. Improvvisamente ha bisogno di un percorso per raggiungere *H*. Invia pacchetti broadcast con *TtL* impostati a 1, 2, 3 e così via. Dopo quanti round trova un percorso?

15. Come meccanismo di controllo della congestione in una rete che usa internamente i circuiti virtuali, un router potrebbe astenersi dall'inviare l'acknowledgement in seguito all'arrivo di un pacchetto finché (1) non sa che la sua ultima trasmissione attraverso il circuito virtuale è stata ricevuta con successo e (2) non ha un buffer vuoto. Per semplicità, si supponga che i router utilizzino un protocollo stop-and-wait e che ogni circuito virtuale abbia un buffer dedicato per ciascuna direzione del traffico. Se ci vogliono *T* secondi per trasmettere un pacchetto (di dati o di acknowledgement) e se ci sono *n* router sul percorso, a quale velocità i pacchetti sono consegnati all'host di destinazione? Si supponga che gli errori di trasmissione siano rari e che la connessione host-router sia infinitamente veloce.

16. Una rete datagram permette ai router di scartare pacchetti ogni volta che serve. La probabilità che un router scarti un pacchetto è *p*. Si consideri il caso di un host sorgente collegato al router sorgente, che è collegato al router di destinazione, che a sua volta è collegato all'host di destinazione. Se entrambi i router scartano un pacchetto, l'host sorgente alla fine va in timeout e ritenta. Contando la linea host-router e quella router-router come un salto ciascuna, qual è il numero medio di:

(a) salti compiuti da un pacchetto per ogni trasmissione

(b) trasmissioni di un pacchetto

(c) salti richiesti per ogni pacchetto ricevuto.

17. Si descrivano due differenze importanti tra il metodo ECN e il metodo RED per evitare le congestioni.

18. Lo schema token bucket è usato per profilare il traffico. Un nuovo token viene messo nel bucket ogni cinque microsecondi. Ogni token va bene per un pacchetto piccolo che contenga 48 byte di dati, qual è il massimo tasso sostenibile per i dati?

19. Un computer su una rete a 6 Mbps è regolato da un token bucket. Il token bucket è riempito a una velocità di 1 Mbps e inizialmente contiene 8 MB. Per quanto tempo il computer può trasmettere a 6 Mbps?

20. La rete mostrata nella Figura 5.34 utilizza RSVP con alberi multicast per gli host 1 e 2, come indicato. Si supponga che l'host 3 richieda un canale di banda 2 MB/s per un flusso proveniente dall'host 1 e un altro canale di banda 1 MB/s per un flusso proveniente dall'host 2. Contemporaneamente, l'host 4 richiede un canale di banda 2 MB/s per un flusso proveniente dall'host 1 e l'host 5 richiede un canale di banda 1 MB/s per un flusso proveniente dall'host 2. Qual è la banda totale riservata a queste richieste dai router *A*, *B*, *C*, *E*, *H*, *J*, *K* e *L*?

21. Un router può elaborare 2 milioni di pacchetti al secondo. Il carico offerto al processore è pari a 1,5 milioni di pacchetti al secondo. Se un percorso dalla sorgente alla destinazione contiene 10 router, quanto tempo dura l'accodamento e l'elaborazione eseguita dalle CPU?

22. Si consideri l'utente di servizi differenziati con inoltro accelerato. C'è una garanzia che i pacchetti accelerati subiscano un ritardo più piccolo dei pacchetti regolari? Perché sì o perché no?

23. Si supponga che l'host *A* sia collegato a un router *R* 1, che *R* 1 sia collegato a un altro router *R* 2 e che *R* 2 sia collegato all'host *B.* Si supponga che un messaggio TCP che contiene 900 byte di dati e 20 byte di intestazione TCP sia passato al codice IP dell'host *A* per la trasmissione a *B.* Mostrate i campi *Total length, Identification, DF, MF* e *Fragment offset* dell'intestazione IP in ogni pacchetto trasmesso attraverso i tre collegamenti. Si supponga che il collegamento *A-R1* possa supportare una dimensione massima di frame pari a 1024 byte inclusi 14 byte di intestazione del frame, che il collegamento *R1-R2* possa supportare una dimensione massima di frame pari a 512 byte, inclusi 8 byte dell'intestazione del frame, e che il collegamento *R2-B* possa supportare una dimensione massima di frame di 512 byte inclusi 12 byte di intestazione.

24. Un router sta emettendo un flusso di pacchetti IP la cui lunghezza totale (dati più intestazione) è di 1024 byte. Supponendo che i pacchetti abbiano una vita di 10 secondi, qual è la velocità massima della linea che il router può utilizzare, senza correre il rischio di ripetere il numero di ID dei datagrammi IP?

25. Un datagramma IP che utilizza l'opzione *Strict source routing* deve essere frammentato. L'opzione va copiata in ogni frammento oppure è sufficiente inserirla solo nel primo frammento? Motivate la risposta.

26. Si supponga che invece di utilizzare 16 bit per la parte di rete di un indirizzo di classe B originale fossero stati utilizzati 20 bit. Quante reti di classe B ci sarebbero state in questo caso?

27. Convertite in notazione decimale a punti l'indirizzo IP la cui rappresentazione esadecimale è C22F1582.

28. Una rete su Internet ha la maschera di sottorete 255.255.240.0. Quanti host può gestire al massimo?

29. Gli indirizzi IP sono legati a una rete specifica, gli indirizzi Ethernet non lo sono. Riuscite a pensare a una buona ragione perché non lo sono?

30. Un gran numero di indirizzi IP consecutivi sono disponibili a partire da 198.16.0.0. Si supponga che quattro aziende, *A*, *B*, *C* e *D*, richiedano rispettivamente 4000, 2000, 4000 e 8000 indirizzi. Per ognuna di loro, date il primo e l'ultimo indirizzo IP assegnato e la maschera nella notazione *w.x.y.z/s.*

31. Un router ha appena ricevuto i seguenti nuovi indirizzi IP: 57.6.96.0/21, 57.6.104.0/21, 57.6.112.0/21 e 57.6.120.0/21. Se usano tutti la stessa linea di trasmissione in uscita, possono essere aggregati? Se sì, in che modo? Se no, perché?

32. L'insieme di indirizzi IP compreso tra 29.18.0.0 e 29.18.128.255 è stato aggregato in 29.18.0.0/17. Tuttavia, c'è un intervallo di 1.024 indirizzi non assegnati che va da 29.18.60.0 a 29.18.63.255 che improvvisamente viene assegnato a un host che usa una diversa linea di trasmissione. È necessario dividere l'indirizzo aggregato nei suoi blocchi costituenti, aggiungere il nuovo blocco alla tabella e poi vedere se è possibile eseguire qualche aggregazione? Se no, che cosa si può fare?

33. Un router ha nella sua tabella di routing le seguenti voci (CIDR):

Indirizzo/maschera **Hop successivo**

135. 46.56.0/22Interfaccia 0

135. 46.60.0/22Interfaccia 1

192. 53.40.0/23Router 1

predefinito Router 2

Per ognuno dei seguenti indirizzi IP, che cosa fa il router se arriva un pacchetto con quell'indirizzo?

(a) 135.46.63.10
(b) 135.46.57.14
(c) 135.46.52.2
(d) 192.53.40.7
(e) 192.53.56.7

34. Molte aziende adottano il criterio di avere due (o più) router che collegano la società a Internet in modo da fornire ridondanza nel caso che uno dei router si guasti. Questa politica è ancora attuabile con NAT? Motivate la risposta.

35. Dopo aver spiegato a un amico come funziona il protocollo ARP, questa persona commenta: "Ho capito. ARP fornisce un servizio al livello di rete, quindi fa parte del livello data link." Che cosa gli rispondete?

36. Descrivete un sistema per riassemblare i frammenti IP giunti a destinazione.

37. La maggior parte degli algoritmi di ricostruzione dei datagrammi IP ha un timer che serve a evitare che i frammenti perduti tengano impegnati in eterno i buffer di ricostruzione. Si supponga che un datagramma sia frammentato in quattro frammenti. I primi tre arrivano, l'ultimo invece è in ritardo. Alla fine, il timer scade e i tre frammenti vengono scartati dalla memoria del ricevente. Un po' più tardi, sopraggiunge l'ultimo frammento. Che cosa si deve fare in questo caso?

38. In IP il checksum controlla solo l'intestazione e non i dati. Perché è stata fatta questa scelta?

39. Una persona che vive a Boston viaggia fino a Minneapolis portandosi dietro il computer portatile. La LAN di destinazione a Minneapolis è una LAN IP wireless, perciò l'utente non deve collegare alcun cavo. È ancora necessario passare attraverso l'intero processo basato su home agent e foreign agent per far arrivare correttamente la posta elettronica e il resto del traffico?

40. IPv6 utilizza indirizzi a 16 byte. Se un blocco di 1 milione di indirizzi venisse assegnato ogni picosecondo, quanto tempo durerebbero gli indirizzi?

41. Il campo *Protocol* utilizzato nell'intestazione IPv4 non è presente nell'intestazione fissa IPv6. Come mai?

42. Quando è stato introdotto il protocollo IPv6 è stato necessario apportare modifiche al protocollo ARP? Se sì, le modifiche sono state concettuali o tecniche?

43. Scrivete un programma che simuli il routing eseguito tramite flooding. Ogni pacchetto dovrebbe contenere un contatore decrementato a ogni salto. Quando il contatore raggiunge lo zero, il pacchetto è scartato. Il tempo è discreto, ogni linea gestisce un pacchetto per intervallo di tempo. Create tre versioni del programma: tutte le linee inondate, tutte le linee tranne quella di input, solo le k linee migliori (scelte in modo statico). Confrontate il routing basato sul flooding con il routing deterministico ($k = 1$) in termini di ritardo e banda usata.

44. Scrivete un programma che simuli una rete di calcolatori a tempo discreto. Il primo pacchetto su ogni coda di router compie un salto ogni intervallo di tempo. Ogni router ha un numero finito di buffer. Se un pacchetto arriva e non c'è spazio per accoglierlo i dati sono scartati e non ritrasmessi, esiste però un protocollo end-to-end, completo di scadenze e pacchetti di acknowledgement, che in questi casi rigenera il pacchetto dal router sorgente. Disegnate la capacità di trasporto della rete come funzione dell'intervallo di scadenza end-to-end, parametrizzato in base alla frequenza di errore.

45. Scrivete una funzione che esegua l'inoltro in un router IP. La procedura ha un parametro, l'indirizzo IP, e accede a una tabella globale composta da una matrice di triplette. Ogni tripletta contiene tre valori interi: un indirizzo IP, una maschera di sottorete e la linea di trasmissione da utilizzare. La funzione cerca l'indirizzo IP nella tabella usando CIDR e restituisce la linea da utilizzare come suo valore.

46. Utilizzate i programmi *traceroute* (UNIX) o *tracert* (Windows) per tracciare il percorso dal proprio computer a varie università che si trovano in altri continenti. Create la lista dei collegamenti transoceanici scoperti. Ecco alcuni siti da provare:

http://www.berkeley.edu (California)
http://www.mit.edu (Massachusetts)
http://www.vu.nl (Amsterdam)

http://www.ucl.ac.uk (London)
http://www.usyd.edu.au (Sydney)
http://www.u-tokyo.ac.jp (Tokyo)
http://www.uct.ac.za (Cape Town)

CAPITOLO 6
Il livello di trasporto

Insieme al livello di rete, il livello di trasporto è il cuore della gerarchia del protocollo. Il livello di rete provvede alla consegna di pacchetti end-to-end usando datagrammi o circuiti virtuali. Il livello di trasporto si basa sul livello di rete per fornire il trasporto dei dati da un processo su una macchina sorgente a un processo su una macchina destinazione con un livello di affidabilità desiderato e indipendente dalle reti fisiche attualmente in uso. Fornisce le astrazioni necessarie alle applicazioni per utilizzare la rete. Senza il livello di trasporto, l'intero concetto di protocolli a livelli avrebbe poco senso. In questo capitolo studieremo il livello di trasporto in dettaglio, compresi i suoi servizi e le scelte architetturali per le interfacce, per affrontare problemi di affidabilità, connessioni e controllo di congestione, prestazioni e protocolli come TCP e UDP.

6.1 Il servizio di trasporto

Nei paragrafi seguenti forniremo un'introduzione al servizio di trasporto e osserveremo quale tipo di servizio viene fornito al livello applicazione. Per rendere più concrete le problematiche del servizio di trasporto esamineremo due gruppi di primitive: per prima cosa sarà illustrato un insieme semplice (ma ipotetico) per mostrare le idee di base, dopodiché sarà presentata l'interfaccia utilizzata comunemente su Internet.

6.1.1 Servizi forniti ai livelli superiori

L'obiettivo finale del livello di trasporto è fornire un servizio di trasmissione dati efficace, affidabile ed efficiente in termini di costi ai suoi utenti, che normalmente sono processi nel livello applicazione. Per raggiungere tale obiettivo, il livello di trasporto utilizza i servizi forniti dal livello di rete. L'hardware e/o il software all'interno del livello di trasporto che svolge il lavoro è chiamato **entità di trasporto** (*transport entity*). L'entità di trasporto può essere situata nel kernel del sistema operativo, in un processo utente separato, in librerie associate alle applicazioni di rete o addirittura nella scheda di rete. Le prime due opzioni sono molto comuni in Internet. La relazione (logica) tra i livelli di rete, trasporto e applicazione è mostrata nella Figura 6.1.

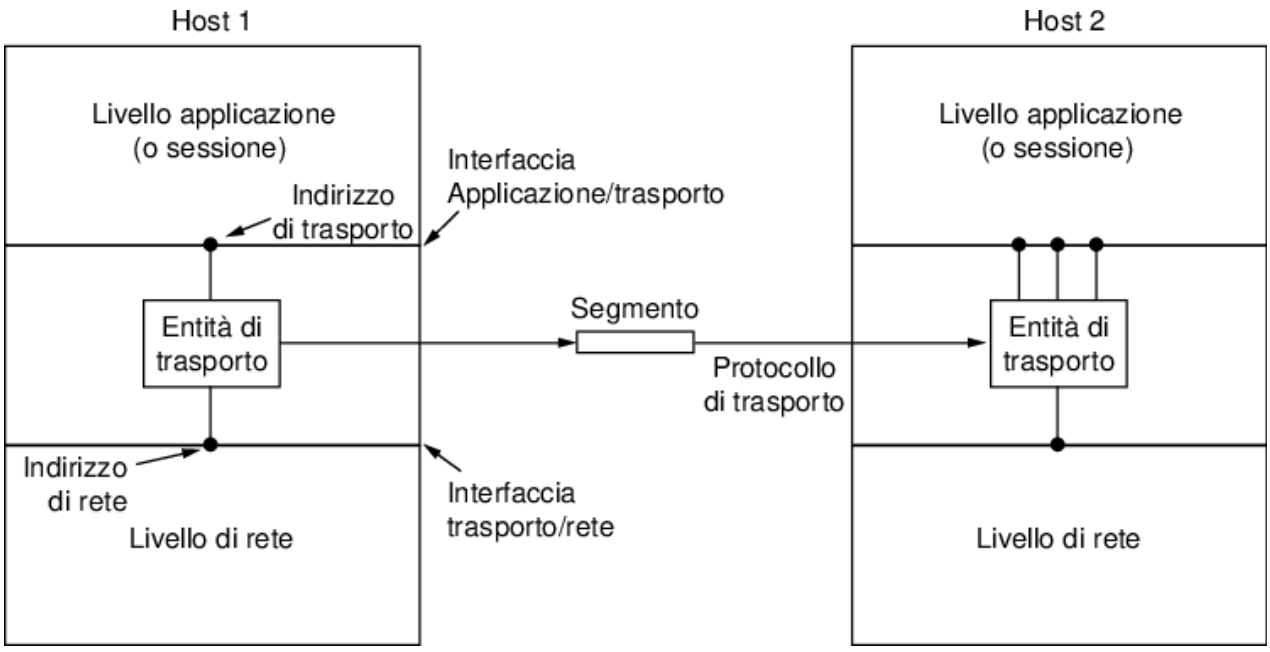


Figura 6.1 Livelli di rete, trasporto e applicazione.

Proprio come esistono due tipi di servizi di rete, orientati alla connessione e non, esistono anche due tipi di servizi di trasporto. Il servizio di trasporto orientato alla connessione è per molti aspetti simile al servizio di rete orientato alla connessione. In entrambi i casi, le connessioni seguono tre fasi: impostazione, trasferimento dati e rilascio. Anche l'indirizzamento e il controllo del flusso sono simili in entrambi i livelli. Inoltre il servizio di trasporto senza connessione è molto simile al servizio di rete senza connessione. Notate, tuttavia, che può essere difficile fornire un trasporto dati senza connessione su un servizio di rete orientato alla connessione, poiché è inefficiente creare una connessione per inviare un singolo pacchetto e tirarla giù subito dopo.

La domanda che sorge spontanea è quindi la seguente: se il servizio del livello di trasporto è così simile al servizio del livello di rete, perché esistono due livelli distinti? Perché un solo livello non sarebbe adeguato? La risposta è sottile ma cruciale. Il codice del livello di trasporto viene eseguito interamente sulle macchine degli utenti, mentre quello per il livello di rete è eseguito per la maggior parte sui router, gestiti dall'operatore di telecomunicazioni (per lo meno nelle reti estese). Che cosa accade se il livello di rete offre un servizio inadeguato? Magari se perde frequentemente i pacchetti? Che cosa accade se i router subiscono dei malfunzionamenti di tanto in tanto?

I problemi esistono, questa è la verità. Gli utenti non hanno un reale controllo sul livello di rete, pertanto non possono risolvere il problema di un servizio scadente utilizzando router migliori o migliorando la gestione degli errori nel livello data link perché non sono i possessori dei router. L'unica possibilità è inserire sopra il livello di rete un altro livello che migliori la qualità del servizio. Se, in una rete senza connessione, vengono persi o alterati pacchetti, l'entità di trasporto può rilevare il problema e compensarlo tramite ritrasmissioni. Se, in una rete orientata alla connessione, un'entità di trasporto è informata a metà di una lunga trasmissione che la sua connessione è stata bruscamente interotta, senza indicazioni su ciò che è accaduto ai dati in transito, può impostare una nuova connessione di rete verso l'entità di trasporto remota. Utilizzando questa nuova connessione di rete, può inviare un'interrogazione al suo pari (*peer*) chiedendo quali dati sono arrivati e quali no, riprendendo poi la trasmissione da dove era stata interrotta.

In sostanza, l'esistenza del livello di trasporto consente al servizio di trasporto di essere più affidabile del sottostante servizio di rete. Inoltre, le primitive del servizio di trasporto possono essere implementate come chiamate a procedure di libreria, per renderle indipendenti dalle primitive del servizio di rete. Le chiamate del servizio di rete possono variare considerevolmente da rete a rete (ad esempio, chiamate basate su Ethernet senza connessione possono essere molto diverse da chiamate su rete WiMAX orientata alla connessione). Nascondere il servizio di rete dietro l'insieme di primitive del servizio di trasporto assicura che cambiare la rete richiede solo la sostituzione di un insieme di procedure di libreria con un altro insieme capace di svolgere le stesse operazioni con un diverso servizio sottostante.

Grazie al livello di trasporto, i programmatori di applicazioni possono scrivere codice usando un insieme di primitive standard e ottenere programmi funzionanti su molte reti diverse, senza doversi preoccupare della gestione delle diverse interfacce verso i servizi di rete e dei vari livelli di affidabilità. Se tutte le reti reali fossero prive di difetti e avessero le stesse e immutabili primitive di servizio, il livello di trasporto potrebbe non essere necessario, ma nel mondo reale esso svolge la funzione chiave di isolare i livelli superiori dalla tecnologia, dalla struttura e dalle imperfezioni della rete.

Per questo motivo molte persone hanno creato una distinzione tra i primi quattro livelli e quelli superiori. I quattro livelli inferiori possono essere visti come i **fornitori del servizio di trasporto**, mentre i livelli superiori sono gli **utenti del servizio di trasporto**. Questa distinzione tra fornitore e utente ha un impatto considerevole sulla struttura dei livelli e pone quello di trasporto in una posizione chiave, dal momento che forma il confine principale tra il fornitore e l'utente di un servizio di trasmissione dati affidabile. Questo è il livello visto dalle applicazioni.

6.1.2 Primitive del livello di trasporto

Per consentire agli utenti di accedere al servizio di trasporto, il livello di trasporto deve fornire alcune funzionalità ai programmi applicativi, vale a dire un'interfaccia per il servizio di trasporto. Ogni servizio di trasporto ha la propria interfaccia. In questo paragrafo esamineremo per prima cosa un semplice servizio di trasporto (ipotetico) e la sua interfaccia per vedere lo stretto indispensabile, mentre nel paragrafo successivo studieremo un esempio reale.

Il servizio di trasporto è simile al servizio di rete, ma con alcune importanti differenze. La principale sta nel fatto che il servizio di rete è pensato per modellare il servizio offerto dalle reti reali, nel bene e nel male. Le reti reali possono perdere pacchetti, pertanto il servizio di rete è generalmente inaffidabile.

Al contrario il servizio di trasporto (orientato alla connessione) è affidabile. Naturalmente le reti reali non sono prive di errori, ma questo è proprio lo scopo del livello di trasporto: fornire un servizio affidabile appoggiandosi su una rete inaffidabile.

Come esempio si possono considerare due processi sulla stessa macchina UNIX connessi da una pipe (o un'altra modalità di comunicazione tra processi), che presuppongono una connessione perfetta. Non vogliono sapere nulla su acknowledgement, pacchetti persi, congestione o questioni simili; ciò che vogliono è una connessione affidabile al 100%. Il processo *A* inserisce i dati a un'estremità della pipe, mentre il processo *B* li preleva all'altro capo. Il servizio di trasporto orientato alla connessione non è altro che questo: nasconde le imperfezioni del servizio di rete in modo che i processi utente possano presumere l'esistenza di un flusso di bit privo di errori anche quando sono su macchine diverse.

Tra parentesi, il livello di trasporto può anche fornire un servizio (datagram) inaffidabile, ma c'è relativamente poco da dire sull'argomento, a parte che "è datagram", pertanto in questo capitolo ci concentreremo principalmente sul servizio di trasporto orientato alla connessione. Ciò nonostante, esistono alcune applicazioni, per esempio per l'elaborazione client/server e lo streaming di contenuti multimediali, che sfruttano un servizio di trasporto senza connessione. Di queste parleremo brevemente in seguito.

Una seconda differenza tra il servizio di rete e il servizio di trasporto riguarda i destinatari del servizio. Il servizio di rete è utilizzato solo dalle entità di trasporto; pochi utenti scrivono delle loro entità di trasporto e quindi pochi utenti o programmi hanno visibilità diretta del servizio di rete. Al contrario molti programmi (e i loro programmatori) vedono le primitive di trasporto. Di conseguenza, il servizio di trasporto deve essere conveniente e facile da utilizzare.

Per farsi un'idea dell'aspetto di un servizio di trasporto è possibile considerare le cinque primitive elencate nella Figura 6.2. Questa interfaccia di trasporto è realmente essenziale, ma offre il senso di ciò che un'interfaccia di trasporto orientata alla connessione deve fare: consentire ai programmi applicativi di stabilire, utilizzare e rilasciare le connessioni; ciò è sufficiente per molte applicazioni.

Primitiva	Pacchetto inviato	Significato
LISTEN	(nessuno)	Si blocca fino a quando un processo cerca di connettersi
CONNECT	CONNECTION REQ.	Tenta di stabilire una connessione
SEND	DATA	Invia informazioni
RECEIVE	(nessuno)	Si blocca fino all'arrivo di un pacchetto DATA
DISCONNECT	DISCONNECTION REQ.	Questo lato desidera rilasciare la connessione

Figura 6.2 Le primitive per un semplice servizio di trasporto.

Per vedere come queste primitive possano essere utilizzate, consideriamo un'applicazione con un server e diversi client remoti. Per iniziare, il server esegue una primitiva LISTEN, chiamando generalmente una procedura di libreria, che esegue una chiamata di sistema la quale blocca il server fino al sopraggiungere di un client. Quando un client desidera comunicare con il server, esegue una primitiva CONNECT. L'entità di trasporto esegue questa primitiva bloccando il chiamante e inviando un pacchetto al server. Incapsulato nel payload di questo pacchetto troviamo un messaggio del livello di trasporto per l'entità di trasporto del server.

Ora è necessaria una breve nota sulla terminologia. In mancanza di un termine migliore, chiameremo **segmento** un messaggio spedito da un'entità di trasporto all'altra. TCP, UDP e altri protocolli Internet usano questo termine, mentre alcuni protocolli più vecchi usavano lo sgraziato termine **TPDU** (*transport protocol data unit*). Quel termine ora non è più molto utilizzato, seppure si trovi ancora in vecchi articoli e libri.

Di conseguenza, i segmenti (scambiati dal livello di trasporto) sono contenuti in pacchetti (scambiati dal livello di rete). A loro volta i pacchetti sono contenuti in frame (scambiati dal livello data link). Quando arriva un frame, il livello data link elabora l'intestazione del frame e, se l'indirizzo di destinazione corrisponde alla scheda di rete locale, passa il contenuto del campo payload del frame all'entità di rete. L'entità di rete elabora l'intestazione del pacchetto e passa il payload del pacchetto all'entità di trasporto. Questa nidificazione è illustrata nella Figura 6.3.

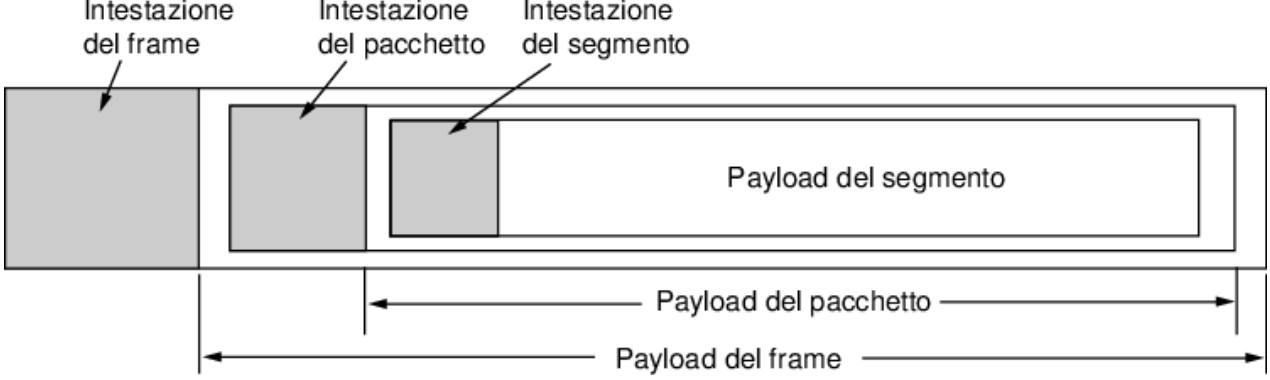


Figura6.3 Nidificazione di segmenti, pacchetti e frame.

Tornando all'esempio client/server, la chiamata CONNECT del client provoca l'invio al server di un segmento CONNECTION REQUEST. Quando arriva, l'entità di trasporto controlla se il server è bloccato su una chiamata LISTEN (vale a dire se è interessato a gestire delle richieste). In tal caso sblocca il server e invia un segmento CONNECTION ACCEPTED al client. Quando questo segmento arriva, il client viene sbloccato e la connessione è stabilita.

Ora i dati possono essere scambiati con le primitive SEND e RECEIVE. Nella forma più semplice, ogni lato può inviare una primitiva RECEIVE (bloccante) per attendere un SEND da parte della controparte. Quando il segmento arriva il destinatario viene sbloccato, può quindi elaborarlo e inviare una risposta. Finché entrambi i lati riescono a tenere traccia di a chi tocca inviare, lo schema funziona bene.

Occorre notare che nel livello di trasporto persino un semplice scambio di dati unidirezionale è più complicato che non nel livello di rete. Ogni pacchetto dati inviato viene (prima o poi) confermato con un acknowledgement. Anche i pacchetti che portano i segmenti di controllo sono confermati in modo implicito o esplicito. Questi acknowledgement sono gestiti dalle entità di trasporto utilizzando il protocollo del livello di rete e non sono visibili agli utenti del livello di trasporto. Allo stesso modo, le entità di trasporto dovranno preoccuparsi di timer e ritrasmissioni. Per gli utenti del livello di trasporto una connessione è un canale affidabile per i bit: un utente inserisce i bit ed essi appaiono magicamente all'altra estremità nello stesso ordine. Questa capacità di nascondere la complessità è il motivo per cui i protocolli stratificati sono uno strumento così potente.

Quando una connessione non è più necessaria, deve essere rilasciata per liberare spazio nelle tabelle all'interno delle due entità di trasporto. La disconnessione presenta due varianti: asimmetrica e simmetrica. Nella variante asimmetrica, ogni utente del trasporto può emettere una primitiva DISCONNECT che provoca l'invio di un segmento DISCONNECT all'entità di trasporto remota. Al suo arrivo la connessione viene rilasciata.

Nella variante simmetrica ogni direzione viene chiusa separatamente, indipendentemente dall'altra. Quando un lato usa la primitiva DISCONNECT, significa che non ha altri dati da inviare, ma è ancora in grado di accettare dati dal suo partner. In questo modello, una connessione viene rilasciata quando entrambi i lati hanno inviato un segmento DISCONNECT.

Un diagramma di stato per la costituzione e il rilascio della connessione tramite queste semplici primitive è mostrato nella Figura 6.4. Ogni transizione è innescata da un evento, cioè una primitiva eseguita dall'utente locale del trasporto o un pacchetto in ingresso. Per semplicità, presumiamo che ogni segmento riceva un acknowledgement separato. Supponiamo inoltre di impiegare un modello di disconnessione simmetrica in cui il client agisce per primo. Occorre notare che questo modello è piuttosto semplice; osserveremo modelli più realistici in seguito quando descriveremo come funziona TCP.

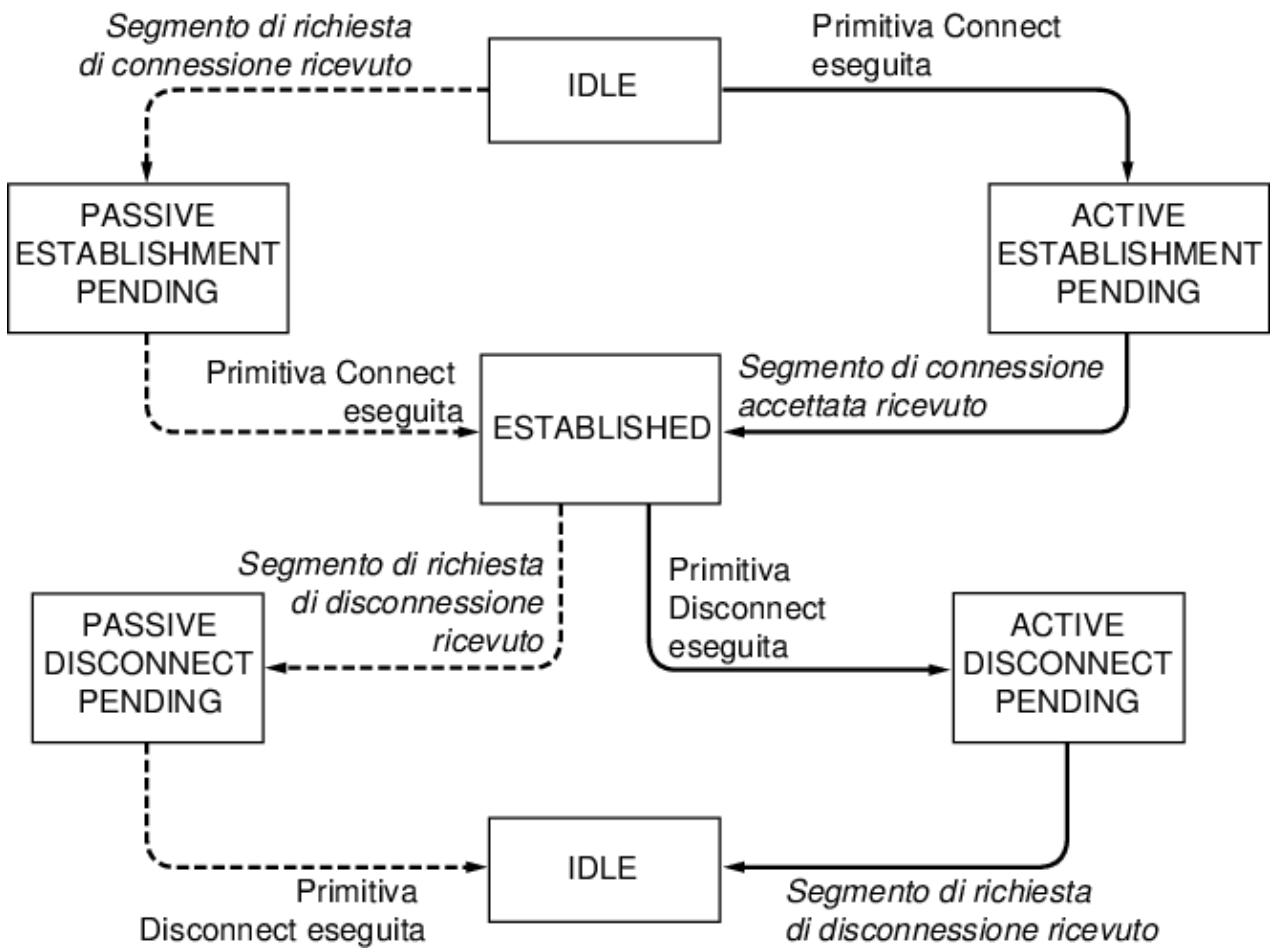


Figura 6.4 Un diagramma di stato per un semplice schema di gestione della connessione. Le transizioni in corsivo sono provocate dall'arrivo dei pacchetti. Le linee continue mostrano la sequenza di stato del client; le linee tratteggiate mostrano la sequenza di stato del server.

6.1.3 Le socket di Berkeley

Esaminiamo ora brevemente un altro gruppo di primitive di trasporto, le primitive delle socket utilizzate per TCP. Le socket furono distribuite inizialmente nel 1983 come parte del software di UNIX Berkeley 4.2BSD e divennero presto popolari. Le primitive sono ora ampiamente utilizzate per programmi che hanno a che fare con Internet su molti sistemi operativi, specialmenti quelli basati su UNIX; esistono delle API con lo stesso stile delle socket per Windows chiamate "winsock".

Queste primitive, elencate nella Figura 6.5, seguono all'incirca il modello del nostro primo esempio, ma offrono maggiori caratteristiche e flessibilità. Non esamineremo i segmenti corrispondenti qui; l'argomento sarà affrontato più avanti nel capitolo.

Primitiva	Significato
SOCKET	Crea un nuovo punto finale do comunicazione
BIND	Associa un indirizzo locale a una socket
LISTEN	Annuncia la capacità di accettare connessioni; fornisce la dimensione della coda
ACCEPT	Blocca il chiamante fino all'arrivo di un tentativo di connessione
CONNECT	Tenta in modo attivo di stabilire una connessione
SEND	Invia alcuni dati sulla connessione
RECEIVE	Riceve alcuni dati sulla connessione
CLOSE	Rilancia la connessione

Figura6.5 Le primitive delle socket per TCP.

Le prime quattro primitive dell'elenco sono eseguite in quello stesso ordine dai server. La primitiva SOCKET crea un nuovo endpoint di comunicazione e alloca il corrispondente spazio nella tabella all'interno dell'entità di trasporto. I parametri della chiamata specificano il formato dell'indirizzo da utilizzare, il tipo di servizio desiderato (per esempio, flusso di byte affidabile) e il protocollo. Una chiamata a SOCKET eseguita con successo restituisce un normale descrittore di file da utilizzare nelle chiamate successive, allo stesso modo in cui opera una chiamata OPEN su di un file.

Le socket appena create non hanno indirizzi di rete, che vengono assegnati utilizzando la primitiva BIND. I client remoti possono connettersi a una socket solo dopo che il server vi ha associato un indirizzo. Il motivo per cui non si utilizza la chiamata SOCKET per creare direttamente un indirizzo è che alcuni processi fanno attenzione ai propri indirizzi (per esempio, se hanno utilizzato lo stesso indirizzo per anni e tutti lo conoscono), mentre altri no.

A questo punto viene la chiamata LISTEN, che alloca spazio per accodare le chiamate in ingresso nel caso in cui diversi client provino a connettersi contemporaneamente. Al contrario della LISTEN del primo esempio, nelle socket LISTEN non è una chiamata bloccante.

Per bloccarsi in attesa di una connessione in ingresso il server esegue una primitiva ACCEPT. Quando arriva un segmento che richiede una connessione, l'entità di trasporto crea una nuova socket con le stesse proprietà di quella originale e restituisce un corrispondente descrittore di file. Il server può quindi generare un nuovo processo o un thread per gestire la connessione sulla nuova

socket, e tornare ad attendere la connessione successiva sulla socket originale. ACCEPT restituisce un descrittore di file che può essere utilizzato per leggere e scrivere in modo analogo a come si fa con i file.

Osserviamo ora il lato client. Anche qui è necessario creare una socket utilizzando la primitiva SOCKET, ma BIND non è richiesto perché l'indirizzo utilizzato non è importante per il server. La primitiva CONNECT blocca il chiamante e avvia il processo di connessione. Una volta completato (vale a dire quando il segmento appropriato viene ricevuto dal server), il processo del client è sbloccato e viene stabilita la connessione. Entrambi i lati ora possono utilizzare SEND e RECEIVE per trasmettere e ricevere dati sulla connessione full-duplex. Se non sono richieste le opzioni speciali di SEND e RECEIVE si possono utilizzare anche le chiamate di sistema standard di UNIX: READ e WRITE.

Con le socket il rilascio della connessione è simmetrico: le risorse vengono rilasciate quando entrambi i lati hanno eseguito una primitiva CLOSE.

Le socket si sono dimostrate incredibilmente popolari e sono lo standard di fatto per l'astrazione dei servizi di trasporto. Le API delle socket sono spesso usate con il protocollo TCP per fornire un servizio orientato alla connessione chiamato **flusso di byte affidabile** (*reliable byte stream*). Tuttavia altri protocolli potrebbero essere usati per implementare questo servizio utilizzando le stesse API; dovrebbe essere del tutto uguale per gli utenti del servizio di trasporto.

Un punto di forza delle API delle socket è che sono utilizzabili da un'applicazione per altri servizi di trasporto. Per esempio le socket possono essere utilizzate con un servizio di trasporto non orientato alla connessione. In questo caso CONNECT imposta l'indirizzo del corrispondente (*peer*) remoto a livello di trasporto e SEND e RECEIVE spediscono e ricevono datagrammi da e per il peer remoto. (È anche di uso comune usare un insieme esteso di chiamate, per esempio SENDTO e RECEIVEFROM, che enfatizzano i messaggi e non limitano un'applicazione a un solo peer a livello di trasporto). Le socket possono essere anche usate con protocolli di trasporto che forniscono un flusso di messaggi invece di un flusso di byte e che effettuano o meno controllo di congestione. Per esempio **DCCP** (*datagram congestion controlled protocol*) è una versione di UDP con controllo di congestione (Kohler et al., 2006). Sta agli utenti del livello di trasporto capire che servizio stanno ottenendo.

Tuttavia le socket non sembrano essere l'ultima parola in fatto di interfacce di trasporto. Per esempio le applicazioni spesso utilizzano un gruppo di flussi collegati, come un Web browser fa quando richiede diversi oggetti dallo stesso server. Con le socket la forma più naturale per le applicazioni è utilizzare un flusso per ogni oggetto. Questa struttura significa che il controllo di congestione è applicato separatamente per ogni flusso e non su tutto il gruppo, il che non è ottimale. Inoltre passa all'applicazione l'onere di gestire il gruppo. Nuovi protocolli e interfacce sono stati ideati per supportare gruppi di flussi correlati tra loro in modo più efficace e semplice per le applicazioni. Due esempi sono **SCTP** (*stream control transmission protocol*) definito nell'RFC 4960 e **SST** (*stream control transmission protocol*) (Ford, 2007). Questi protocolli devono modificare leggermente le API delle socket per ottenere i benefici di gruppi di flussi correlati; inoltre supportano caratteristiche quali un misto di traffico orientato alla connessione e non, e la possibilità di avere più percorsi di rete. Il tempo ci dirà se avranno successo.

6.1.4 Un esempio di programmazione con le socket: un file server per Internet

Come esempio di utilizzo delle chiamate socket, si consideri il codice di client e server nella Figura 6.6, dove abbiamo un file server molto primitivo e un esempio di client che lo utilizza. Il codice ha molte limitazioni (discusse in seguito), ma in linea di principio il codice del server può essere compilato ed eseguito su qualsiasi sistema UNIX connesso a Internet. Il codice del client può quindi essere compilato ed eseguito su qualsiasi altra macchina UNIX connessa a Internet, ovunque nel mondo; con gli opportuni parametri è possibile prelevare qualsiasi file a cui il server ha accesso sul proprio computer. Il file viene scritto sullo standard output, che naturalmente può essere reindirizzato su un file o una pipe.

```
/* Programma client che può richiedere un file al programma
* server, listato nella pagina seguente. Il server risponde inviando l'intero file.
*/

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345          /* arbitrario, ma client e server devono essere d'accordo */
#define BUF_SIZE 4096              /* dimensione del blocco di trasferimento */

int main(int argc, char **argv)
{
     int c, s, bytes;
     char buf[BUF_SIZE];           /* buffer per il file in arrivo */
     struct hostent *h;            /* informazioni sul server */
     struct sockaddr_in channel;   /* contiene l'indirizzo IP */

     if (argc != 3) fatal("Usage: client server-name file-name");
     h = gethostbyname(argv[1]);   /* cerca l'indirizzo IP dell'host */
     if (!h) fatal("gethostbyname failed");

     s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
     if (s <0) fatal("socket");
     memset(&channel, 0, sizeof(channel));
     channel.sin_family= AF_INET;
     memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
     channel.sin_port= htons(SERVER_PORT);

     c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
     if (c < 0) fatal("connect failed");

     /* La connessione è stabilita. Invia il nome file includendo un byte a 0 alla fine. */
     write(s, argv[2], strlen(argv[2])+1);

     /* Prende il file e lo scrive nello standard output. */
     while (1) {
          bytes = read(s, buf, BUF_SIZE);      /* legge dalla socket */
          if (bytes <= 0) exit(0);             /* controlla se il file è finito */
          write(1, buf, bytes);                /* scrive sullo standard output */
     }
}

fatal(char *string)
{
     printf("%s\n", string);
     exit(1);
}
```

```
#include <sys/types.h>                  /* Questo è il codice del server */
#include <sys/fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345               /* arbitrario, ma client e server devono essere d'accordo */
#define BUF_SIZE 4096                   /* dimensione del blocco di trasferimento */
#define QUEUE_SIZE 10

int main(int argc, char *argv[])
{
    int s, b, l, fd, sa, bytes, on = 1;
    char buf[BUF_SIZE];                 /* buffer per il file in uscita */
    struct sockaddr_in channel;         /* contiene l'indirizzo IP */

    /* Costruisce la struttura degli indirizzi da associare al socket. */
    memset(&channel, 0, sizeof(channel));       /* canale zero */
    channel.sin_family = AF_INET;
    channel.sin_addr.s_addr = htonl(INADDR_ANY);
    channel.sin_port = htons(SERVER_PORT);

    /* Apertura passiva. Attende una connessione. */
    s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);        /* crea una socket */
    if (s < 0) fatal("socket failed");
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));

    b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
    if (b < 0) fatal("bind failed");

    l = listen(s, QUEUE_SIZE);          /* specifica la dimensione della coda */
    if (l < 0) fatal("listen failed");

    /* La socket è impostata e associata. Attende una connessione e la elabora. */
    while (1) {
        sa = accept(s, 0, 0);           /* blocco per la richiesta di connessione */
        if (sa < 0) fatal("accept failed");

        read(sa, buf, BUF_SIZE);        /* legge il nome del file dalla socket */

        /* Ottiene e restituisce il file. */
        fd = open(buf, O_RDONLY);       /* apre il file da inviare in risposta */
        if (fd < 0) fatal("open failed");

        while (1) {
            bytes = read(fd, buf, BUF_SIZE);    /* legge dal file */
            if (bytes <= 0) break;              /* controlla se il file è finito */
            write(sa, buf, bytes);              /* scrive i byte sulla socket */
        }
        close(fd);                              /* chiude il file */
        close(sa);                              /* chiude la connessione */
    }
}
```

Figura6.6 Codice client facente uso di socket. Il codice del server è nella pagina successiva.

Per prima cosa osserviamo il codice del server. Inizia con l'inclusione di alcuni header standard, di cui gli ultimi tre contengono le principali definizioni e strutture dati correlate a Internet. Subito dopo è presente la definizione di *SERVER_PORT* come 12345: questo numero è stato scelto in modo arbitrario, qualsiasi numero compreso tra 1024 e 65.535 funzionerà alla perfezione, purché non utilizzato da altri processi, mentre le porte sotto il 1023 sono riservate a utenti privilegiati.

Le due righe successive nel codice del server definiscono due costanti necessarie: la prima determina la dimensione del blocco utilizzato per il trasferimento di file, la seconda il numero di connessioni che possono essere tenute in sospeso prima di scartare le successive quando arrivano.

Il codice del server comincia dopo le dichiarazioni delle variabili locali, con l'inizializzazione di una struttura dati che conterrà l'indirizzo IP del server. Questa struttura dati sarà presto associata alla socket del server. La chiamata a *memset* imposta la struttura dati con tutti i valori a zero, le tre assegnazioni seguenti riempiono altrettanti campi; l'ultima rappresenta la porta del server. Le funzioni *htonl* e *htons* hanno a che fare con la conversione dei valori in un formato standard, in modo che il codice sia eseguito correttamente sia sulle macchine little-endian (come Intel x86) sia su quelle big-endian (come leSPARC). La loro esatta semantica non è rilevante per questa discussione.

Il server crea quindi una socket e controlla gli errori (indicati da $s < 0$). In una versione più professionale del codice il messaggio di errore potrebbe essere più esplicativo. La chiamata a *setsockopt* è necessaria per consentire il riutilizzo della porta, in modo che il server possa essere eseguito indefinitamente, rispondendo a ogni richiesta. Arrivati a questo punto l'indirizzo IP viene associato alla socket e si esegue un controllo per vedere se la chiamata a *bind* ha avuto successo. Il passaggio finale dell'inizializzazione è la chiamata a *listen* per annunciare la disponibilità del server ad accettare chiamate in ingresso e comunicare al sistema di conservarne una quantità massima pari al valore *QUEUE_SIZE* nel caso arrivino nuove richieste mentre il server sta ancora elaborando quella attuale. Se la coda è piena e arrivano altre richieste, verranno scartate senza inviare alcuna segnalazione all'applicazione.

Adesso il server entra nel suo ciclo principale, che non abbandonerà mai: l'unico modo per fermarlo è interromperlo dall'esterno. La chiamata ad *accept* blocca il server finché qualche client non tenti di stabilire una connessione. Se la chiamata ad *accept* ha successo, questa restituisce un descrittore di file che può essere utilizzato per lettura e scrittura, in modo analogo all'utilizzo dei descrittori di file per lettura e scrittura da/su pipe. Tuttavia, a differenza delle pipe (che sono unidirezionali), le socket sono bidirezionali, per cui *sa* (la socket accettata) può essere usata sia per leggere dalla connessione sia per scrivervi.

Dopo avere stabilito la connessione, il server la usa per leggere il nome del file. Se il nome non è ancora disponibile, il server si blocca in attesa. Dopo avere ottenuto il nome del file, il server lo apre ed entra in un ciclo che, alternativamente, legge blocchi dal file e li scrive sulla socket fino a copiare l'intero file. Al termine il server chiude il file e la connessione, attendendo la comparsa della richiesta di connessione successiva. Il ciclo è ripetuto all'infinito.

Osserviamo ora il codice del client. Per comprenderne il funzionamento, è necessario capire come viene invocato. Supponendo di chiamarlo *client*, una chiamata tipica è la seguente:

```
client flits.cs.vu.nl /usr/tom/nomefile >f
```

Questa chiamata funziona solo se il server è già in esecuzione su *flits.cs.vu.nl*, il file */usr/tom/nomefile* esiste e il server può accedervi in lettura. Se la chiamata ha successo, il file viene trasferito su Internet e scritto sul file *f*, dopo di che il programma client termina. Dal momento che il server prosegue l'esecuzione dopo un trasferimento, il client può essere avviato di nuovo per ottenere altri file.

Il codice del client inizia con alcune inclusioni e dichiarazioni. L'esecuzione inizia controllando se il programma è stato chiamato con il giusto numero di argomenti (*argc = 3* significa che il nome del programma ha più di due argomenti). Si noti che *argv*[1] contiene il nome del server (per esempio *flits.cs.vu.nl*) ed è convertito in un indirizzo IP da *gethostbyname*. Questa funzione utilizza il DNS per cercare il nome. Studieremo il DNS nel Capitolo 7.

Successivamente viene creata e inizializzata una socket. Il client tenta poi di stabilire una connessione TCP con il server utilizzando *connect*. Se il server è in esecuzione sulla macchina invocata, è collegato a *SERVER_PORT* e sia che non stia facendo nulla sia che possieda spazio nella coda di attesa, la connessione verrà (prima o poi) stabilita. Utilizzando la connessione, il client invia il nome del file scrivendo sulla socket. Il numero di byte inviati è superiore di uno al numero di byte del nome, perché deve essere inviato anche un byte a 0 che termina la stringa in modo che il server ne individui la fine.

Ora il client entra in un ciclo, nel quale legge il file blocco per blocco dalla socket e lo copia sullo standard output. Al termine semplicemente esce.

La procedura *fatal* stampa un messaggio di errore ed esce. Il server necessita della stessa procedura, ma essa è stata omessa a causa della mancanza di spazio sulla pagina. Dal momento che client e server sono compilati separatamente e normalmente sono eseguiti su computer diversi, non possono condividere il codice di *fatal*.

Questi due programmi (insieme ad altro materiale correlato al libro) possono essere prelevati dal sito Web del volume originale americano:

`http://www.pearsonhighered.com/tanenbaum`

Per la cronaca questo server non è il massimo. Il suo controllo degli errori è scarso e la loro segnalazione mediocre. Dal momento che gestisce tutte le richieste in modo rigorosamente sequenziale (perché dispone di un singolo thread), le prestazioni sono scadenti. Chiaramente non ha mai sentito parlare della sicurezza e l'utilizzo di semplici chiamate di sistema UNIX non è la soluzione migliore per ottenere indipendenza dalla piattaforma. Fa anche ipotesi tecnicamente scorrette: per esempio assume che il nome del file possa essere contenuto nel buffer e trasmesso in modo atomico. Nonostante tali carenze, è un file server funzionante. Negli esercizi il lettore è invitato a perfezionarlo. Per ulteriori informazioni sulla programmazione con le socket consultate Donahoo e Calvert (2008, 2009).

6.2 Elementi dei protocolli di trasporto

Il servizio di trasporto è implementato da un **protocollo di trasporto** utilizzato tra le due entità di trasporto. Per certi aspetti i protocolli di trasporto ricordano i protocolli data link studiati in dettaglio nel Capitolo 3. Entrambi hanno a che fare, tra le altre cose, con il controllo degli errori, l'ordinamento e il controllo di flusso.

Tuttavia tra i due esistono differenze significative, dovute alle diversità tra gli ambienti in cui operano i protocolli, come mostrato nella Figura 6.7. Nel livello data link due router comunicano direttamente tramite un canale fisico sia esso cablato o wireless, mentre nel livello di trasporto il canale fisico è sostituito dall'intera rete. Questa differenza porta con sé molte importanti implicazioni per i protocolli.

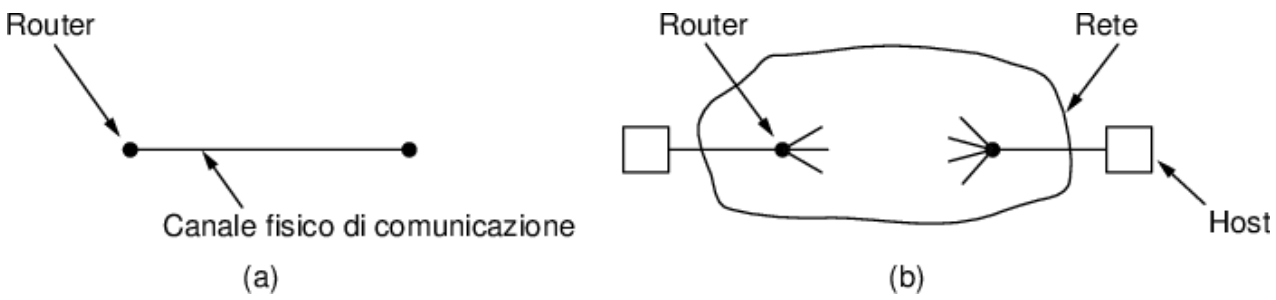


Figura6.7 (a) Ambiente del livello data link. (b) Ambiente del livello di trasporto.

Da una parte, in collegamenti punto a punto come cavi o fibre ottiche non è normalmente necessario che un router specifichi con quale router desidera comunicare: ogni linea in uscita ne specifica in modo univoco uno in particolare. Nel livello di trasporto è invece richiesto l'indirizzamento esplicito delle destinazioni.

Sotto un altro aspetto, il processo d'impostazione di una connessione sul filo della Figura 6.7(a) è semplice: l'altra estremità è sempre presente (a meno che non subisca un malfunzionamento, nel qual caso non è disponibile) e non c'è molto da fare. Anche su linee wireless, il processo non è molto differente: spedire un messaggio è sufficiente perché trovi tutte le altre destinazioni. Se il messaggio non riceve acknowledgement a causa di un errore, può essere rispedito. Nel livello di trasporto la costituzione della connessione iniziale è più complicata, come vedremo.

Un'altra fastidiosissima differenza tra il livello data link e il livello di trasporto è la potenziale esistenza di capacità di memorizzazione nella rete. Quando un router invia un pacchetto su una linea, questo potrebbe arrivare o essere perso, ma non può rimbalzare avanti e indietro, nascondersi in un remoto angolo del mondo e riemergere improvvisamente dopo altri pacchetti spediti molto più tardi. Se la rete usa datagrammi, instradati indipendentemente al suo interno, c'è una possibilità

non trascurabile che il pacchetto possa prendere la via panoramica e che arrivi in ritardo e fuori dall'ordine atteso, o anche che arrivino duplicati. Le conseguenze dell'abilità della rete nel ritardare e duplicare pacchetti possono a volte essere disastrose e richiedere l'uso di protocolli speciali per trasportare correttamente le informazioni.

L'ultima differenza tra i livelli data link e di trasporto è di tipo quantitativo. Il buffering e il controllo di flusso sono necessari in entrambi i livelli, ma la presenza nel livello di trasporto di un numero alto e variabile di connessioni (con un impiego di risorse fluttuante, a causa dei flussi in competizione tra loro) può richiedere un approccio diverso da quello utilizzato nel livello data link. Alcuni protocolli visti nel Capitolo 3 allocano un quantitativo fisso di buffer per ogni linea, in modo che quando arriva un frame vi sia sempre buffer disponibile. Nel livello di trasporto il gran numero di connessioni da gestire rende meno attraente l'idea di dedicare un ampio spazio al buffer di ogni linea. Nei paragrafi seguenti esamineremo queste e altre importanti questioni.

6.2.1 Indirizzamento

Quando un processo applicativo (per esempio un utente) vuole creare una connessione verso un processo applicativo remoto, deve specificare a quale intende connettersi. Il trasporto non orientato alla connessione presenta lo stesso problema: a chi bisogna inviare ogni singolo messaggio? Il metodo normalmente utilizzato consiste nel definire indirizzi di trasporto su cui i processi possono restare in ascolto delle richieste di connessione. Su Internet questi *endpoint* (o punti terminali) sono chiamati **porte** (*port*). Useremo il termine generico **TSAP** (*transport service access point*) riferendoci a uno specifico endpoint nel livello di trasporto. Gli analoghi endpoint nel livello di rete (come gli indirizzi del livello di rete) sono banalmente chiamati **NSAP** (*network service access point*). Gli indirizzi IP sono esempi di NSAP.

La Figura 6.8 illustra la relazione tra NSAP, TSAP e una connessione di trasporto. I processi applicativi, client e server, possono collegarsi a un TSAP locale per stabilire una connessione con un TSAP remoto. Queste connessioni scorrono attraverso gli NSAP su ogni host. Il motivo per cui servono i TSAP è che in alcune reti ogni computer possiede un singolo NSAP, pertanto è necessario distinguere in qualche modo i diversi endpoint di trasporto che condividono lo stesso NSAP.

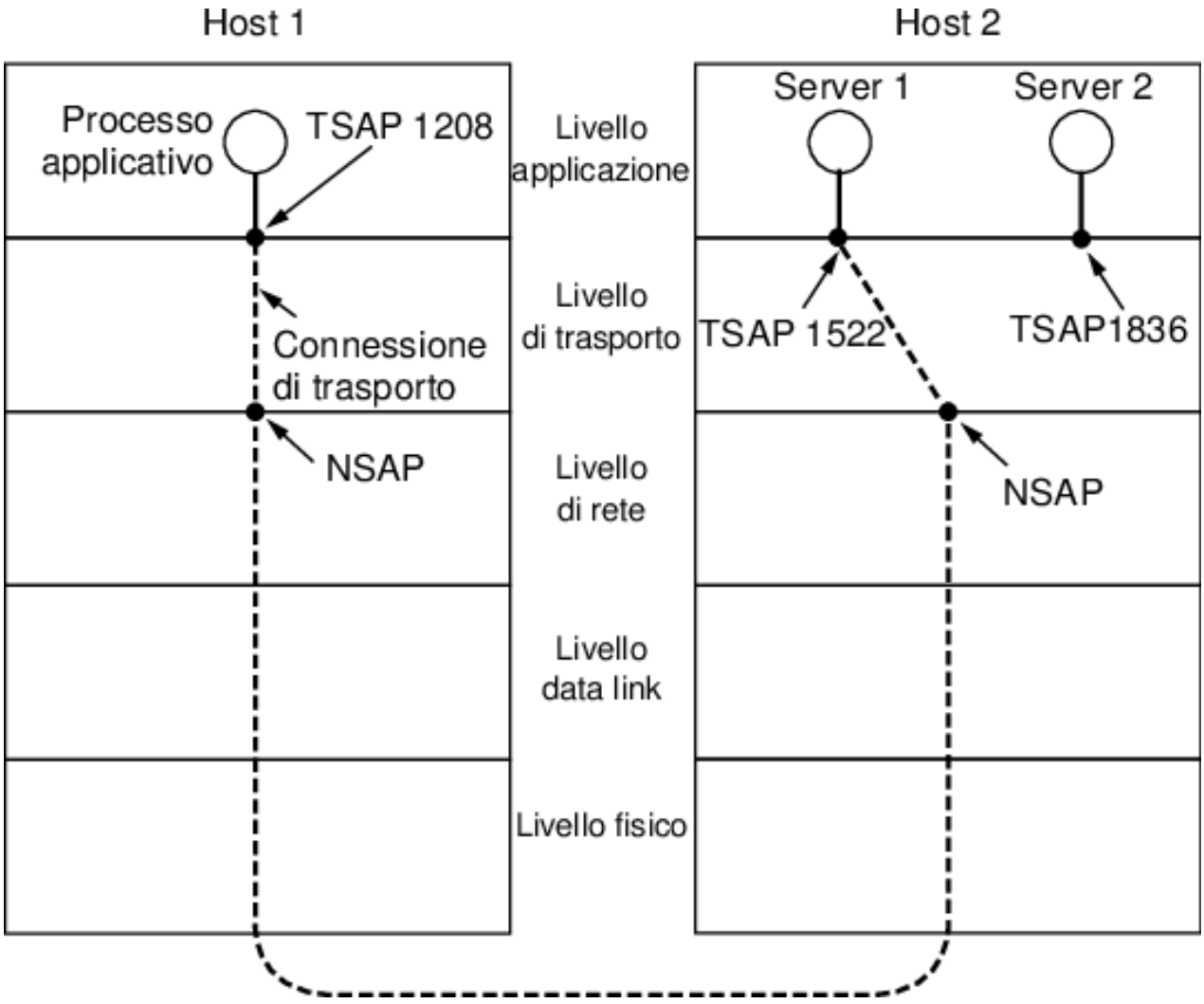


Figura 6.8 TSAP, NSAP e connessioni di trasporto.

Un possibile scenario per una connessione di trasporto è il seguente.

1. Il processo di un mail server si collega al TSAP 1522 sull'host 2 aspettando una chiamata in entrata. Il modo in cui un processo si collega al TSAP non fa parte del modello di rete e dipende interamente dal sistema operativo locale. Potrebbe essere utilizzata una chiamata LISTEN, per esempio.
2. Un processo applicativo sull'host 1 vuole spedire un messaggio di posta elettronica, quindi si collega a TSAP 1208 ed emette una richiesta CONNECT. La richiesta specifica il TSAP 1208 sull'host 1 come sorgente e TSAP 1522 sull'host 2 come destinazione. Questa azione ha come risultato finale una connessione di trasporto stabilita tra i due processi: quello dell'applicazione e quello del server.
3. Il processo applicativo spedisce il messaggio di posta elettronica.
4. Il mail server risponde per avvertire che consegnerà il messaggio.
5. La connessione a livello di trasporto viene rilasciata.

Notiamo che potrebbero esserci altri server sull'host 2 collegati ad altri TSAP e in attesa di connessioni in ingresso in arrivo tramite lo stesso NSAP.

Lo scenario appena visto è corretto, tranne per il fatto che abbiamo nascosto sotto il tappeto un piccolo problema: come fa il processo utente sull'host 1 a sapere che il mail server è collegato al TSAP 1522? Una possibilità è che il mail server si sia collegato a TSAP 1522 per anni, e che gradualmente gli utenti della rete lo abbiano imparato. In questo modello i servizi dispongono di indirizzi TSAP stabili elencati in file salvati in posizioni ben note, come il file */etc/services* nei sistemi UNIX che elenca quali servizi dovrebbero essere permanentemente collegati a quali porte, includendo il fatto che il mail server dovrebbe trovarsi sulla porta 25 TCP.

Sebbene gli indirizzi TSAP stabili funzionino bene per un numero ridotto di servizi chiave che non cambiano mai (per esempio un server Web), i processi utente spesso vogliono comunicare con altri processi utente che possiedono un indirizzo TSAP noto a priori o che potrebbero esistere solo per un breve periodo di tempo.

Per gestire questa situazione è possibile usare un approccio alternativo in cui esiste uno speciale processo chiamato **portmapper**. Per trovare il TSAP corrispondente a un dato nome di servizio, ad esempio "BitTorrent", un utente stabilisce una connessione al portmapper (che ascolta su un TSAP noto) e poi spedisce un messaggio in cui specifica il nome del servizio; il portmapper risponde con l'indirizzo TSAP. Dopo che l'utente ha rilasciato la connessione con il portmapper può stabilirne una nuova con il servizio desiderato.

In questo modello, quando un servizio viene creato deve registrarsi con il portmapper fornendo sia il nome del servizio (tipicamente una stringa ASCII) sia il suo TSAP. Il portmapper registra questa informazione nel suo database interno, in modo tale che quando gli verranno inoltrate delle interrogazioni saprà rispondere.

La funzione del portmapper è analoga a quella del centralinista in un sistema telefonico: fornisce un'associazione tra nomi e numeri. Esattamente come nel sistema telefonico è necessario che l'indirizzo del TSAP usato dal portmapper sia davvero ben noto. Se non si conosce il numero del centralino non è possibile chiamare il centralino stesso per saperlo. Se pensate che il numero per ottenere informazioni sia ovvio, provate qualche volta a comporlo quando siete all'estero.

Molti dei processi che svolgono la funzione di server su una stessa macchina saranno usati solo raramente. È inutile averli tutti attivi e in ascolto su un indirizzo TSAP stabile per tutto il giorno. Uno schema alternativo è mostrato nella Figura 6.9 in una forma semplificata ed è noto come **protocollo di connessione iniziale** (*initial connection protocol*). Invece di avere tutti i possibili server in ascolto su di un TSAP noto, ogni macchina che voglia offrire servizi a utenti remoti ha uno speciale **process server** che agisce da proxy per i servizi utilizzati meno pesantemente. Questo server, chiamato *inetd* sui sistemi UNIX, ascolta contemporaneamente su un insieme di porte aspettando una richiesta di connessione. I potenziali utenti di un servizio iniziano facendo una richiesta CONNECT, specificando l'indirizzo TSAP del servizio che vogliono e ottenendo una connessione al process server, come si vede nella Figura 6.9(a).

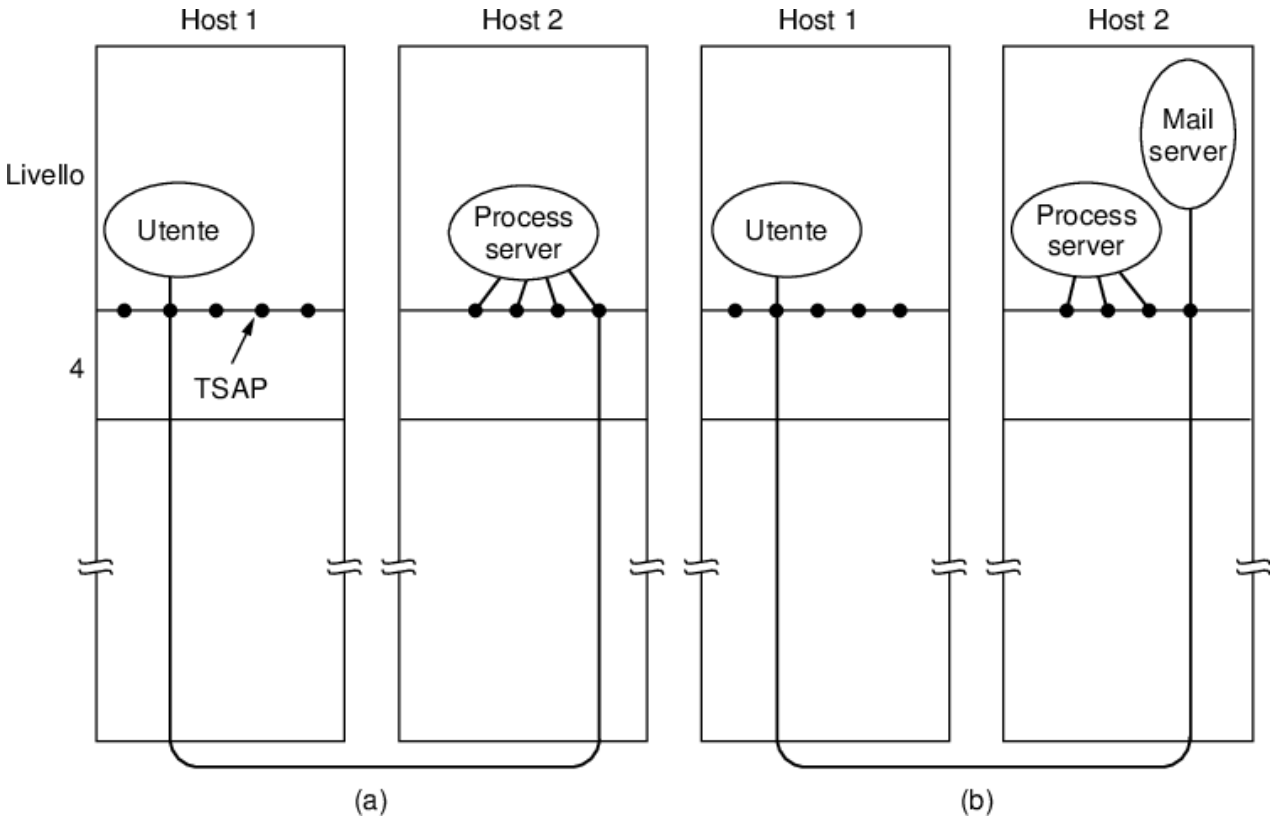


Figura6.9 Modalità di connessione di un processo utente nell'host 1 con un mail server nell'host 2.

Dopo aver ricevuto la richiesta, il process server genera il server richiesto, permettendogli di ereditare la connessione esistente con l'utente. Il nuovo server svolge il lavoro richiesto, mentre il process server torna ad attendere nuove richieste, come si vede nella Figura 6.9(b). Questo metodo è applicabile solo quando i server possono essere creati a richiesta.

6.2.2 Stabilire una connessione

Sembra facile, ma in realtà stabilire una connessione è un'operazione sorprendentemente complessa. A prima vista pare sufficiente che un'entità di trasporto invii un segmento CONNECTION REQUEST alla destinazione e attenda una risposta CONNECTION ACCEPTED. Il problema si verifica se la rete può perdere, ritardare, corrompere e duplicare i pacchetti. Questo comportamento provoca serie complicazioni.

Si immagini una rete così congestionata che gli acknowledgement non riescono quasi mai a tornare indietro in tempo: ogni pacchetto subisce un timeout e viene ritrasmesso due o tre volte. Si supponga che la rete usi i datagrammi al proprio interno e che ogni pacchetto segua una via diversa. Alcuni pacchetti potrebbero restare bloccati in una congestione di traffico all'interno della rete e richiedere molto tempo per arrivare; possono essere ritardati dalla rete e saltare fuori molto più tardi, quando il mittente pensa che siano stati perduti.

Il peggior incubo immaginabile è il seguente: un utente stabilisce una connessione con una banca e invia messaggi che comunicano alla banca il trasferimento di un'elevata quantità di denaro sul conto di una persona non completamente affidabile. Sfortunatamente i pacchetti decidono di prendere la strada panoramica per la destinazione ed esplorano qualche angolo remoto della rete. Il mittente, quindi, va in timeout e li spedisce di nuovo tutti. Questa volta i pacchetti prendono la via più breve e vengono consegnati in fretta, quindi il mittente rilascia la connessione.

Sfortunatamente, prima o poi il primo gruppo di pacchetti esce dal nascondiglio e arriva a destinazione in ordine, chiedendo alla banca di stabilire una nuova connessione e trasferire la somma di denaro (di nuovo). La banca non ha modo di sapere che sono duplicati; deve assumere che questa sia una seconda transazione indipendente e trasferisce di nuovo l'importo.

Questo scenario può sembrare inverosimile, o anche poco plausibile, ma il punto è che i protocolli devono essere progettati per essere corretti in ogni situazione. Solo i casi comuni devono essere implementati in maniera efficiente per ottenere delle buone prestazioni di rete, ma il protocollo deve essere capace di affrontare i casi insoliti senza rompersi. Se non è in grado di farlo abbiamo costruito una rete solo per le situazioni tranquille che può fallire senza preavviso quando le condizioni peggiorano.

Come promemoria di questo paragrafo studieremo il problema dei duplicati in ritardo, concentrandoci sugli algoritmi che servono a stabilire connessioni in maniera affidabile, per cui incubi come quello appena visto non possono verificarsi. Il punto cruciale del problema è che i duplicati in ritardo sono interpretati come pacchetti nuovi. Non possiamo prevenire che questo accada ma, nel caso, i pacchetti devono essere rifiutati come duplicati e non elaborati come pacchetti nuovi.

Il problema può essere affrontato in molti modi, nessuno dei quali è molto soddisfacente. Un modo è utilizzare indirizzi di trasporto usa e getta; in questo modo viene generato un nuovo indirizzo di trasporto ogni volta sia necessario; quando una connessione viene rilasciata l'indirizzo viene scartato e mai più utilizzato. I pacchetti duplicati in ritardo non troveranno mai la loro strada per un processo e non potranno fare danni. Tuttavia questo approcio rende molto più difficile connettersi a un processo.

Un'altra possibilità è quella di assegnare a ogni connessione un identificatore (cioè un numero sequenziale incrementato a ogni connessione che viene stabilita), scelto dalla parte che inizia la connessione e inserito in ogni segmento, compresa quella che richiede la connessione. Dopo il rilascio di ogni connessione, le entità di trasporto possono aggiornare una tabella che elenca le connessioni obsolete sotto forma di coppie (entità di trasporto remota, identificatore di connessione). Ogni volta che giunge una richiesta di connessione, è possibile confrontarla con la tabella per vedere se apparteneva a una connessione già rilasciata in precedenza.

Sfortunatamente questo schema presenta un difetto di base: richiede che ogni entità di trasporto mantenga indefinitamente una certa quantità di informazioni storiche. Questo archivio storico deve essere persistente su entrambe le macchine, sia sorgente che destinazione, altrimenti, se una macchina ha un malfunzionamento e perde il contenuto della sua memoria, non saprà più quali identificativi di connessione siano già stati usati.

Abbiamo invece bisogno di intraprendere un'altra strada per semplificare il problema. Anziché consentire ai pacchetti di vivere per sempre all'interno della rete escogitiamo un meccanismo per eliminare i pacchetti obsoleti che sono ancora in circolo. Con questa restrizione il problema diventa più gestibile.

La vita del pacchetto può essere limitata a un valore massimo prefissato utilizzando una (o più) delle seguenti tecniche.

1. Progettazione di reti soggette a restrizioni.
2. Inserimento di un contatore di salti in ogni pacchetto.
3. Applicazione di un timestamp a ogni pacchetto.

La prima tecnica combina due elementi: una qualsiasi soluzione che impedisce ai pacchetti di percorrere dei cicli e un modo per contenere il ritardo, anche dovuto alla congestione, sul percorso più lungo possibile (ora noto). Il secondo metodo consiste nell'inizializzare il contatore dei salti a un valore appropriato e decrementarlo ogni volta che il pacchetto viene inoltrato. Il protocollo di rete scarta tutti i pacchetti in cui il contatore sia arrivato a zero. Il terzo metodo richiede che ogni pacchetto porti con sé l'orario in cui è stato creato e i router scartino qualsiasi pacchetto più vecchio di un tempo stabilito. Questo ultimo metodo richiede che gli orologi dei router siano sincronizzati, che è un'operazione per nulla semplice.

In pratica occorre garantire non solo che un pacchetto sia eliminato, ma anche che tutti i suoi acknowledgement siano stati rimossi; per questo motivo introdurremo un intervallo di tempo T, un piccolo multiplo della reale vita massima di un pacchetto. La vita massima di un pacchetto è una costante della rete; per Internet è stata arbitrariamente fissata a 120 secondi. Il multiplo dipende dal protocollo e ha il semplice effetto di rendere T più lungo. Se si attende un tempo T dopo l'invio di un pacchetto, è possibile essere certi che tutte le sue tracce siano scomparse e che né il pacchetto né i suoi acknowledgement possano apparire improvvisamente a complicare le cose.

Limitando la vita dei pacchetti si può escogitare un metodo pratico e infallibile per rigettare pacchetti duplicati in ritardo. Il metodo che viene descritto di seguito è dovuto a Tomlinson (1975), poi raffinato da Sunshine e Dalai (1978) e le sue varianti sono largamente utilizzate nella pratica, per esempio nel protocollo TCP.

Il cuore del metodo è che la sorgente etichetti i segmenti con numeri di sequenza che non saranno riutilizzati entro T secondi. Il periodo T e il tasso dei pacchetti al secondo determinano la dimensione dei numeri di sequenza. In questo modo solo un pacchetto con una dato numero di sequenza può essere in sospeso in un dato istante. Possono ancora verificarsi dei duplicati di questo pacchetto che devono essere scartati dalla destinazione; tuttavia, non può più capitare che il duplicato in ritardo di un vecchio pacchetto possa battere sul tempo un nuovo pacchetto con lo stesso numero di sequenza ed essere accettato dal destinatario al suo posto.

Per aggirare il problema di una macchina che, dopo un malfunzionamento, perde ogni informazione, una possibilità sta nel richiedere alle entità di trasporto di rimanere inattive per T secondi dopo il ripristino. Il periodo di inattività permetterà a tutti i vecchi segmenti di scadere, così che il mittente possa iniziare di nuovo con qualsiasi numero di sequenza. Tuttavia, poiché in una rete complessa, T può essere molto grande, questa strategia è poco conveniente.

Tomlinson propose invece di equipaggiare ogni host con un orologio. Gli orologi dei differenti host non hanno bisogno di essere sincronizzati; si assume che ogni orologio sia implementato come un contatore binario che si incrementi a intervalli regolari. Inoltre il numero di bit nel contatore deve essere uguale o maggiore al numero di bit nel numero di sequenza. Ultimo e più importante si assume che l'orologio continui a misurare il tempo anche se l'host non sta funzionando.

Quando viene instaurata una connessione, i k bit di ordine più basso dell'orologio sono usati come primo numero di sequenza. Perciò, diversamente dai protocolli del Capitolo 3, ogni connessione inizia numerando i suoi segmenti in modo diverso. Lo spazio della sequenza può essere così grande che, quando i numeri della sequenza ricominciano da capo, i vecchi segmenti con lo stesso numero di sequenza sono già spariti da molto. Questa relazione lineare tra il tempo e i numeri iniziali delle sequenze viene illustrata nella Figura 6.10(a). La regione proibita mostra i tempi per i quali i numeri di sequenza dei segmenti non sono ammissibili per essere utilizzati. Se un segmento è spedito con numero di sequenza all'interno di questa regione, potrebbe venire ritardato ed essere scambiato per un diverso pacchetto con lo stesso numero di sequenza emesso più tardi. Per esempio, l'host, se si blocca e si riavvia nell'istante in cui l'orologio segna 70 secondi, utilizzerà numeri per iniziare le sequenze in base all'istante in cui torna a funzionare e non utilizzerà numeri all'interno della regione proibita.

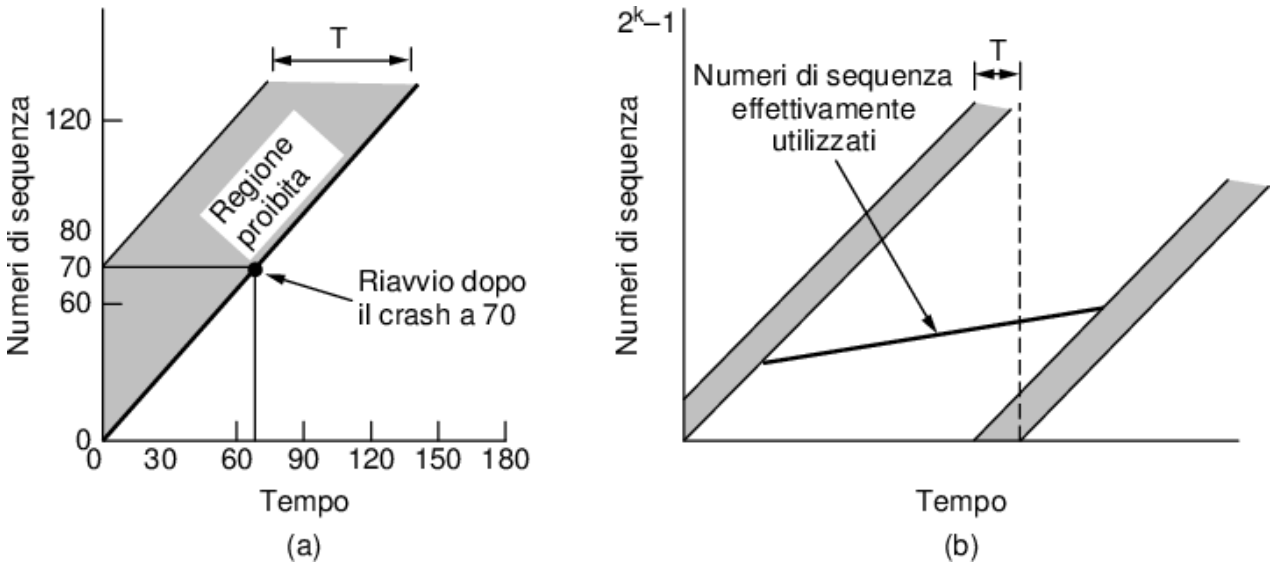


Figura6.10 (a) I segmenti non dovrebbero entrare all'interno della regione proibita. (b) Il problema della risincronizzazione.

Una volta che entrambe le entità di trasporto si sono messe d'accordo sul numero di sequenza iniziale, può essere utilizzato qualsiasi protocollo a finestra scorrevole per controllare il flusso dei dati. Questo protocollo a finestra troverà e scarterà i duplicati dei pacchetti nel caso in cui siano già

stati accettati. In realtà, l'andamento della sequenza numerica iniziale (evidenziata dalla linea spessa) non è lineare, ma presenta dei gradini, in quanto l'orologio procede a passi discreti. Per semplicità, ignoreremo questo dettaglio.

Per garantire che i numeri di sequenza dei pacchetti siano fuori dalla regione proibita abbiamo bisogno di fare attenzione a due aspetti. Se un host spedisce troppo velocemente troppi dati su una connessione appena aperta, l'andamento del numero di sequenza attuale rispetto al tempo può crescere più rapidamente del numero di sequenza iniziale, sempre rispetto al tempo, facendo entrare il numero di sequenza nella regione proibita. Per evitare che questo accada, il massimo tasso di invio dei dati per qualsiasi connessione è di un segmento per ogni ciclo di clock. Questo vuol dire anche che l'entità di trasporto dovrà aspettare fino al prossimo avanzamento dell'orologio prima di aprire una nuova connessione dopo il riavvio dovuto a un malfunzionamento, altrimenti lo stesso numero potrebbe essere utilizzato due volte. Entrambe queste argomentazioni sono in favore di un ciclo di clock molto breve (1 ms o meno), ma l'orologio non può procedere troppo velocemente rispetto al numero di sequenza. Per una velocità C dell'orologio e uno spazio dei numeri di sequenza S, si deve avere $S/C > T$ così che la sequenza numerica non possa ricominciare da capo troppo velocemente.

Entrare dal basso nella regione proibita inviando pacchetti troppo velocemente non è il solo modo per mettersi nei guai. Dalla Figura 6.10(b) vediamo che, per ogni tasso di trasmissione dati inferiore a quello di avanzamento dell'orologio, l'andamento dei numeri di sequenza utilizzati rispetto al tempo andrà a finire nella regione proibita da sinistra non appena i numeri di sequenza ricominceranno da zero. Più forte è l'inclinazione dell'andamento dei numeri di sequenza più grande sarà il ritardo di questo evento. Evitare tale situazione vuol dire imporre una velocità minima alla progressione dei numeri su una connessione (e quanto a lungo può durare la connessione).

Il metodo basato sull'orologio risolve il problema di non riuscire a distinguere i segmenti duplicati da quelli nuovi. Tuttavia c'è un intoppo pratico nell'utilizzarlo per stabilire delle connessioni: poiché normalmente non ricordiamo i numeri di sequenza tra una connessione e l'altra, non abbiamo modo di sapere se un segmento CONNECTION REQUEST contenente un numero di sequenza iniziale sia un duplicato di una connessione recente. Questo problema non sussiste durante una connessione perché il protocollo a finestra scorrevole ricorda il numero di sequenza corrente.

Per risolvere questo problema, Tomlinson (1975) ha introdotto l'**handshake a tre vie** (*three-way handshake*). Questo protocollo per l'impostazione di una connessione richiede la verifica reciproca da parte dei peer che l'attuale richiesta di connessione non sia un duplicato. La normale procedura per stabilire la connessione quando è l'host 1 a iniziare è mostrata nella Figura 6.11(a). L'host 1 sceglie un numero di sequenza, x, e invia un segmento CONNECTION REQUEST contenente x all'host 2, che risponde con un segmento ACK per confermare x e annunciare il suo numero di sequenza iniziale, y. Per finire l'host 1 conferma la scelta del numero di sequenza iniziale dell'host 2 con il primo segmento dati inviato.

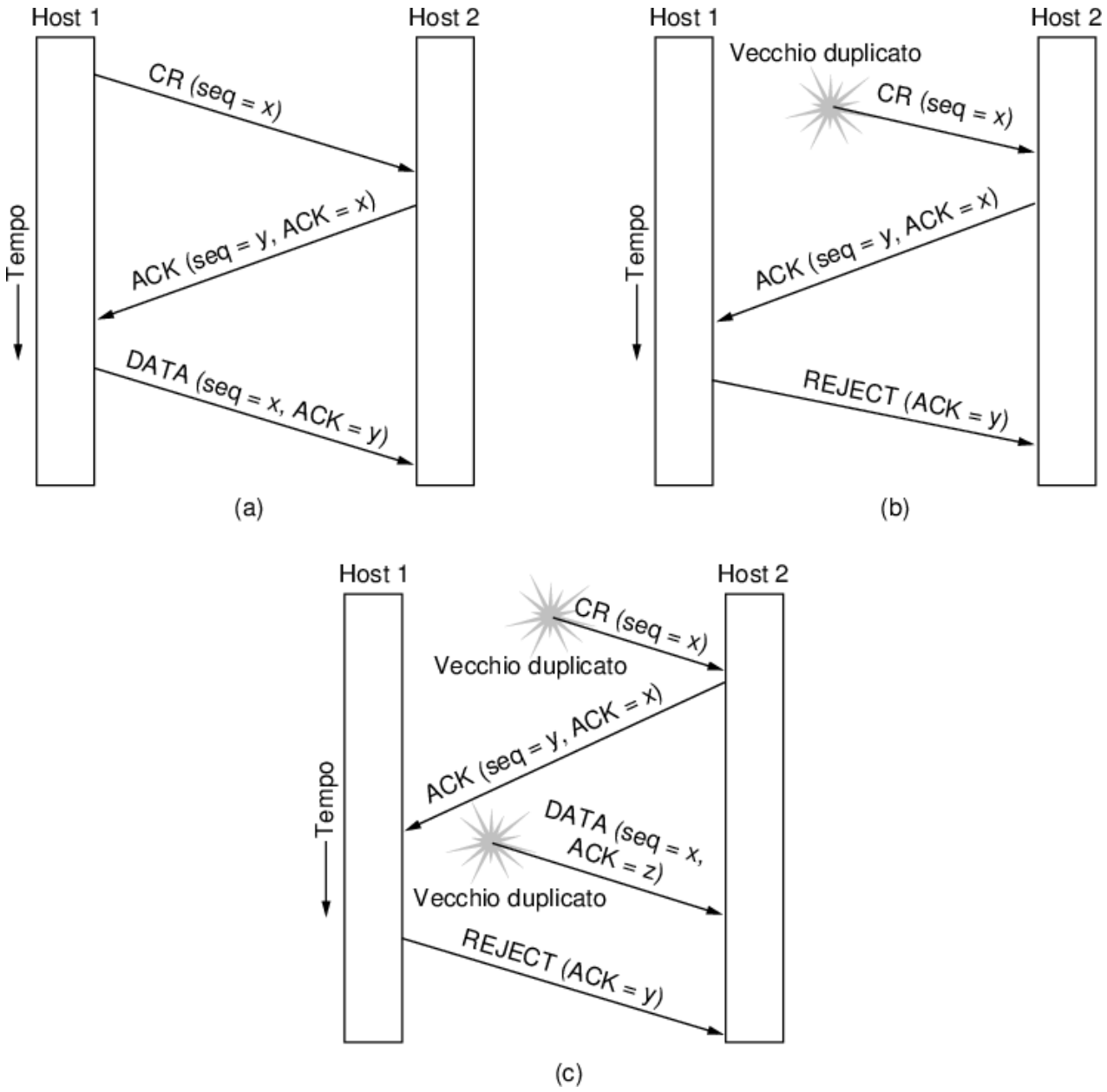


Figura6.11 Tre scenari relativi all'istituzione di una connessione con l'handshake a tre vie. CR indica CONNECTION REQUEST. (a) Operazioni normali. (b) Una vecchia CONNECTION REQUEST duplicata appare dal nulla. (c) Con presenza di CONNECTION REQUEST e ACK duplicati.

Vediamo ora come funziona l'handshake a tre vie in presenza di segmenti di controllo duplicati e ritardati. Nella Figura 6.11(b) il primo segmento è una CONNECTION REQUEST duplicata e ritardata che si riferisce a una vecchia connessione. Questo segmento arriva all'host 2 senza che l'host 1 lo sappia. L'host 2 reagisce inviando all'host 1 un segmento ACK, chiedendo in definitiva una conferma del fatto che l'host 1 stia tentando di creare una nuova connessione. Quando l'host 1 rifiuta il tentativo dell'host 2 di stabilire una connessione, l'host 2 comprende di essere stato ingannato da un duplicato in ritardo e abbandona la connessione; in questo modo un duplicato in ritardo non fa danni.

Il caso peggiore, mostrato nella Figura 6.11(c), si verifica quando nella rete circolano sia una CONNECTION REQUEST ritardata sia un ACK. Come nell'esempio precedente, l'host 2 riceve una CONNECTION REQUEST ritardata e risponde a essa. A questo punto è fondamentale comprendere che l'host 2 ha proposto l'utilizzo di y come numero di sequenza iniziale per il traffico dall'host 2 all'host 1, sapendo bene che nessun segmento contenente il numero di sequenza y o l'acknowledgement a y è ancora in circolazione. Quando il secondo segmento ritardato giunge all'host 2, il fatto che sia stato confermato z al posto di y indica all'host 2 che anche questo è un vecchio duplicato. È importante osservare che non esiste una combinazione di vecchi segmenti che può provocare il fallimento del protocollo o l'instaurarsi accidentale di una connessione indesiderata.

Per stabilire le connessioni TCP utilizza questo handshake a tre vie. All'interno di una connessione un marcatore temporale (*timestamp*) è usato per estendere la sequenza numerica di 32 bit così che non ritorni a zero prima del tempo di vita massimo del pacchetto, anche con connessioni da gigabit al secondo. Questo meccanismo è una correzione a TCP necessaria da quando viene utilizzato su linee molto veloci. È descritto nell'RFC 1323 e prende il nome di **PAWS** (*protection against wrapped sequence numbers*, protezione dai numeri di sequenza azzerati). Tra connessioni diverse, prima che fosse introdotto PAWS, per iniziare le sequenze numeriche TCP utilizzava lo schema appena descritto basato sull'orologio. Tuttavia questo approccio presentò una problematica di sicurezza: l'orologio rende facile a un aggressore prevedere il successivo numero iniziale di sequenza e spedire pacchetti che imbrogliano l'handshake a tre vie, stabilendo così una connessione contraffatta. Nella pratica per risolvere questo problema di sicurezza vengono utilizzate sequenze numeriche iniziali pseudocasuali; tuttavia è importante che i numeri della sequenza iniziale non si ripetano per un certo periodo anche se a un osservatore appaiono casuali, altrimenti i duplicati in ritardo possono essere devastanti.

6.2.3 Rilascio della connessione

Rilasciare una connessione è più facile che stabilirla, però le trappole sono più di quante se ne possano immaginare. Come affermato in precedenza, esistono due modi per terminare una connessione: il rilascio asimmetrico e quello simmetrico. Il rilascio asimmetrico è il metodo utilizzato dal sistema telefonico: quando una delle due parti riattacca la connessione viene interrotta. Il rilascio simmetrico tratta la connessione come se fosse composta da due connessioni unidirezionali separate e richiede il rilascio separato di ognuna di esse.

Il rilascio asimmetrico è improvviso e può provocare una perdita di dati. Si consideri lo scenario nella Figura 6.12. Dopo avere stabilito la connessione, l'host 1 invia un segmento che arriva correttamente nell'host 2 e successivamente invia un altro segmento. Sfortunatamente, l'host 2 emette un segmento DISCONNECT prima dell'arrivo del secondo segmento. Il risultato è che la connessione è rilasciata e i dati vanno persi.

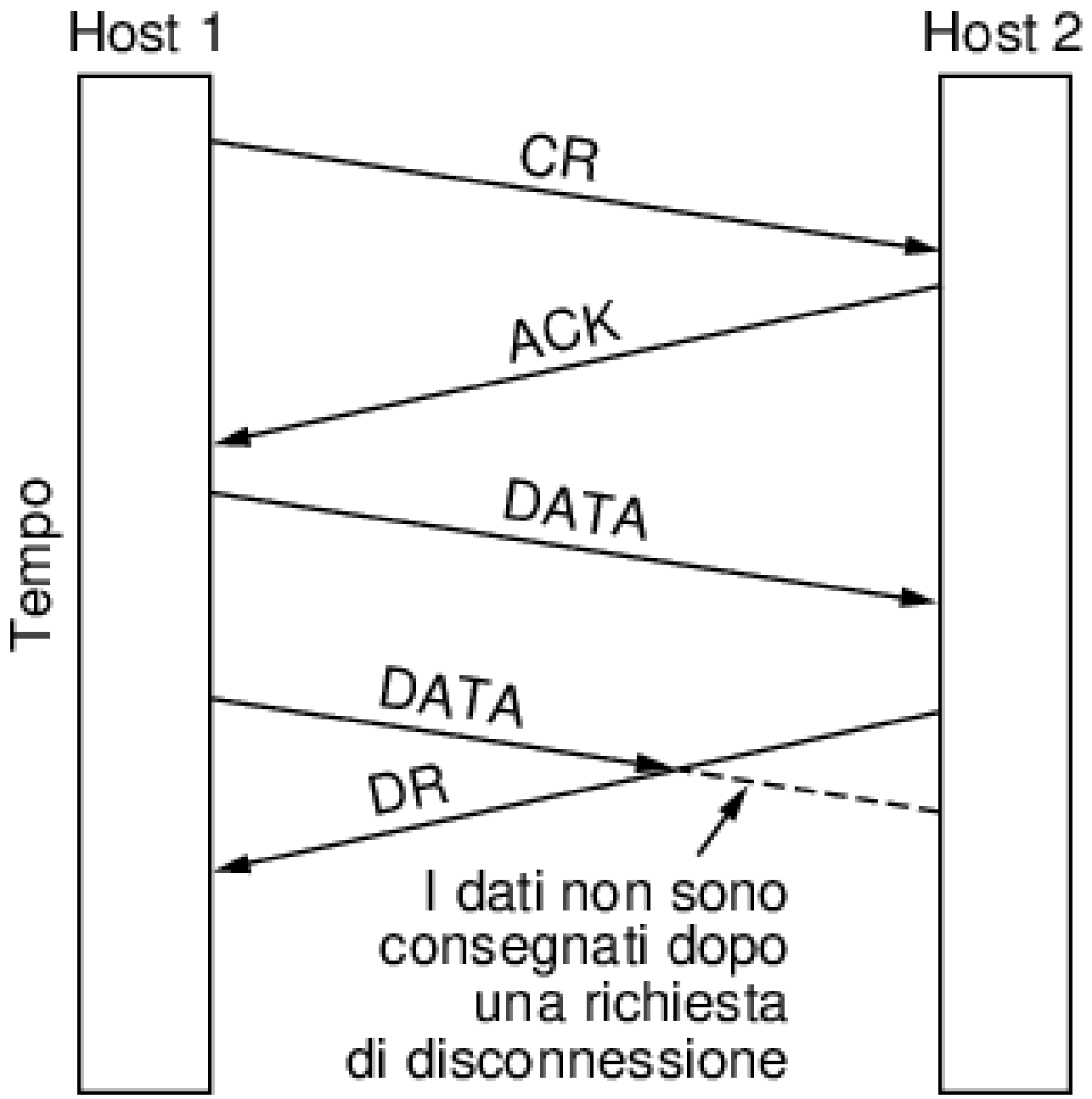


Figura6.12 Una disconnessione improvvisa con perdita di dati.

Per evitare la perdita di dati è chiaramente necessario un protocollo di rilascio più sofisticato. Un metodo prevede l'utilizzo del rilascio simmetrico, in cui ogni direzione è rilasciata indipendentemente dall'altra. Un host può continuare a ricevere dati anche dopo avere inviato un segmento DISCONNECT.

Il rilascio simmetrico svolge il suo compito quando ogni processo presenta una quantità fissa di dati da inviare e sa chiaramente quando li ha inviati; ma in certe situazioni stabilire che tutto il lavoro è stato svolto e che la connessione dovrebbe essere terminata non è un'operazione così ovvia. Qualcuno potrebbe immaginare un protocollo in cui l'host 1 afferma: "Ho finito. Hai finito anche tu?" Se l'host 2 rispondesse: "Ho finito anch'io. Ciao", la connessione potrebbe essere rilasciata in sicurezza.

Sfortunatamente questo protocollo non funziona sempre. Esiste un famoso problema che illustra la questione: è chiamato **problema dei due eserciti**. Supponiamo che l'esercito bianco sia accampato in una valle, come mostrato nella Figura 6.13. Sulle colline a entrambi i fianchi si trovano gli eserciti blu. L'esercito bianco è più grande di ciascuno degli eserciti blu, ma l'insieme degli eserciti blu è più grande dell'esercito bianco. Se un esercito blu lancia l'attacco da solo sarà sconfitto, ma se i due eserciti blu attaccano contemporaneamente saranno vittoriosi.

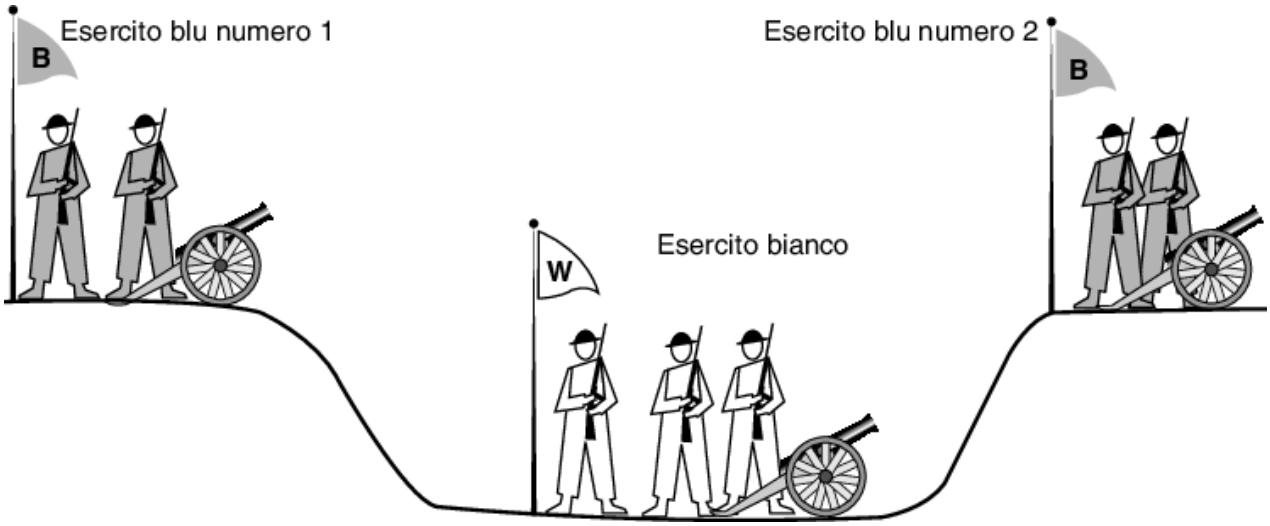


Figura6.13 Il problema dei due eserciti.

Gli eserciti blu vogliono sincronizzare i loro attacchi. Tuttavia il loro unico mezzo di comunicazione è l'invio di messaggeri a piedi nella valle, dove potrebbero essere catturati perdendo il messaggio (significa che devono utilizzare un canale di comunicazione inaffidabile). La domanda è: esiste un protocollo che consenta agli eserciti blu di vincere?

Supponiamo che il comandante del primo esercito blu invìi questo messaggio: "Propongo di attaccare all'alba del 29 marzo. Che cosa ne pensate?". Supponiamo ora che il messaggio arrivi: il comandante del secondo esercito blu è d'accordo e la sua risposta viene consegnata al primo esercito blu. L'attacco avverrà? Probabilmente no, perché il comandante del secondo esercito non sa se la sua risposta è giunta a destinazione. Se non arrivasse, il primo esercito blu non attaccherebbe perché sarebbe insensato sostenere la battaglia.

Proviamo a migliorare il protocollo con un handshake a tre vie: l'autore della proposta originale deve dare l'acknowledgement alla risposta. Supponendo che non vengano persi messaggi, il secondo esercito blu riceverà l'acknowledgement, ma il comandante del primo esercito blu esiterà ancora. Dopo tutto, non sa se il suo acknowledgement sia giunto a destinazione: se non lo fosse, il secondo esercito blu non attaccherebbe. Potremmo provare a creare un protocollo di handshake a quattro vie, ma non aiuterebbe comunque.

In effetti si può dimostrare che non esiste alcun protocollo funzionante. Supponiamo che esista un protocollo di questo tipo: l'ultimo messaggio del protocollo potrebbe essere o meno fondamentale. Se non lo è, occorre rimuoverlo (insieme a tutti gli altri messaggi non fondamentali) fino a ottenere un protocollo in cui ogni messaggio è fondamentale. Che cosa accade se il messaggio finale non giunge a destinazione? Abbiamo già detto che è essenziale per cui, se viene perso, l'attacco non può avvenire. Dal momento che il mittente del messaggio finale non può essere certo del suo arrivo, non rischierà l'attacco. Peggio ancora, l'altro esercito blu è a conoscenza di questo fatto e non lancerà l'attacco.

Per capire che il problema dei due eserciti è rilevante per il rilascio delle connessioni basta sostituire il verbo "disconnettere" al verbo "attaccare". Se entrambe le parti esigono la certezza che anche l'altra sia pronta a terminare la connessione, la disconnessione non avverrà mai.

In pratica possiamo evitare questa situazione non imponendo la necessità di un accordo e demandando il problema a chi usa il livello di trasporto, facendo decidere indipendentemente ogni partecipante. La Figura 6.14 illustra quattro scenari di rilascio utilizzando un handshake a tre vie; anche se questo protocollo non è infallibile, di solito è adeguato.

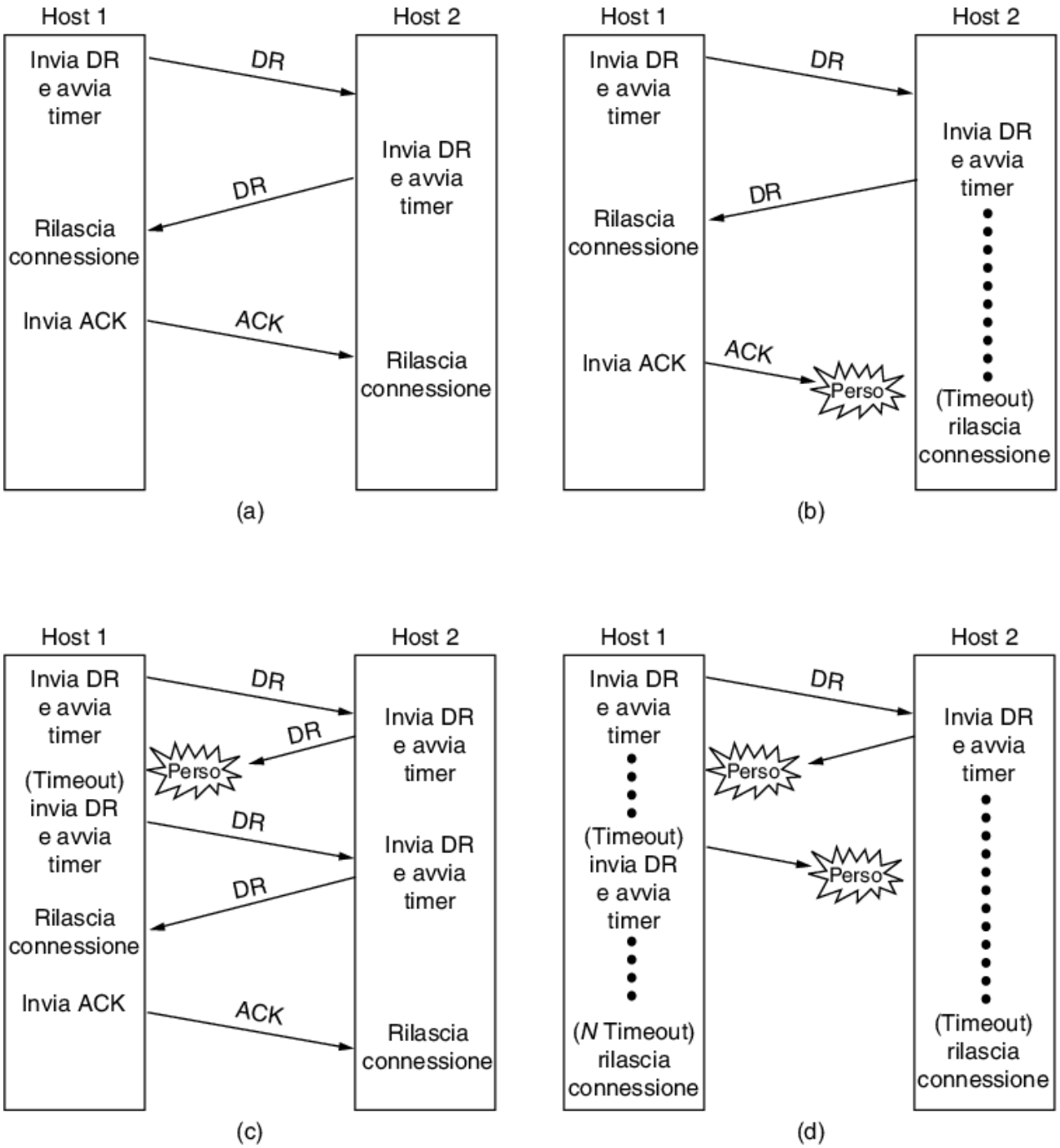


Figura6.14 Quattro scenari per il rilascio di una connessione. (a) Caso normale di handshake a tre vie. (b) ACK finale perduto. (c) Risposta perduta. (d) Risposta perduta e DR successivo perduti

Nella Figura 6.14(a) è mostrato il caso normale in cui uno degli utenti invia un segmento DR (DISCONNECTION REQUEST) per iniziare il rilascio di una connessione. Quando arriva, il destinatario restituisce un segmento DR e avvia un timer (servirà nel caso il suo DR vada perso). All'arrivo di questo DR, il mittente originale invia un segmento ACK e rilascia la connessione. Per finire, quando arriva il segmento ACK, il ricevente rilascia la connessione. Rilasciare una connessione significa che l'entità di trasporto rimuove le informazioni sulla connessione dalla propria tabella delle connessioni aperte e lo segnala al proprietario della connessione (l'utente del livello di trasporto). Questa azione è diversa da un utente del livello di trasporto che fa uso di una primitiva DISCONNECT.

Se il segmento ACK finale viene perso, come mostrato nella Figura 6.14(b), la situazione viene risolta dal timer, alla cui scadenza la connessione viene comunque rilasciata.

Consideriamo ora il caso della perdita del secondo DR. L'utente che inizia la disconnessione non riceve la risposta prevista, si verifica un timeout e tutto ricomincia; la sequenza degli eventi è visibile nella Figura 6.14(c), facendo l'ipotesi che la seconda volta nessun segmento venga perso e che tutti i segmenti siano consegnati correttamente e in tempo.

L'ultimo scenario, mostrato nella Figura 6.14(d), equivale a quello della Figura 6.14(c), tranne per il fatto che ora si presume che tutti i tentativi di ritrasmettere il DR falliscano a causa di segmenti persi. Dopo *N* tentativi il mittente si arrende e rilascia la connessione; nel frattempo si verifica un timeout anche nel ricevente che a sua volta esce.

Questo protocollo è generalmente sufficiente, ma in teoria può fallire se vengono persi il DR iniziale e le *N* ritrasmissioni. Il mittente abbandonerà i tentativi e rilascerà la connessione, mentre l'altro lato non saprà nulla di tutti i tentativi di disconnessione e sarà pienamente attivo. Questa situazione genera una connessione aperta a metà.

Avremmo potuto evitare il problema non consentendo al mittente di abbandonare dopo *N* tentativi, ma obbligandolo a proseguire fino al ricevimento di una risposta. Tuttavia, se si consente alla controparte di terminare per timeout, il mittente procederà all'infinito perché non riceverà mai risposta. Se non consentiamo il timeout del lato ricevente, il protocollo rimarrà in sospeso, come nella Figura 6.14(d).

Un modo per interrompere tutte le connessioni aperte a metà può essere la definizione di una regola: se non arrivano segmenti per un certo numero di secondi, la connessione viene automaticamente terminata. In questo modo, quando un lato si disconnette, l'altro lato rileva la mancanza di attività ed esegue a sua volta la disconnessione. Questa regola si prende anche cura del caso in cui la connessione viene interrotta (quando la rete non è più in grado di consegnare pacchetti ai due host) e nessuno dei due utenti ha richiesto la disconnessione. Naturalmente, se viene introdotta questa regola, è necessario che ogni entità di trasporto abbia un timer che venga fermato e fatto ripartire a ogni invio di un nuovo segmento. Se il timer scade, viene trasmesso un segmento fittizio solo per impedire la disconnessione dell'altro lato. D'altra parte se viene utilizzata la regola di disconnessione automatica e si perdono troppi segmenti fittizi consecutivi su una connessione altrimenti inattiva, prima un lato e poi l'altro si disconnetteranno automaticamente.

Dovrebbe quindi essere chiaro che il rilascio di una connessione senza perdita di dati non è così semplice come potrebbe sembrare. La morale è che l'utente del livello di trasporto deve essere coinvolto nel decidere quando disconnettersi (il problema non può essere pienamente risolto dalle stesse entità di trasporto). Per valutare l'importanza dell'applicazione, considerate che mentre TCP si chiude normalmente in modo simmetrico (in cui ogni parte chiude indipendentemente la propria metà della connessione con un pacchetto FIN una volta spediti i propri dati), molti Web server spediscono al client un pacchetto RST che provoca una chiusura brusca della connessione molto simile a una chiusura asimmetrica. Questo funziona solo perché il Web server conosce lo schema dello scambio di dati. Innanzitutto riceve una richiesta dal client, che è tutto ciò che il client spedirà e quindi spedisce una risposta al client. Quando il Web server avrà finito di inviare la sua risposta, tutti i dati saranno stati spediti in entrambe le direzioni. Il server può spedire al client

un avvertimento e chiudere bruscamente la connessione. Se il client ottiene questo avviso, rilascerà la sua connessione su due piedi. Viceversa, realizzerà col tempo che il server non sta più parlando con lui e rilascerà la connessione. In entrambi i casi i dati saranno stati trasferiti con successo.

6.2.4 Controllo degli errori e controllo di flusso

Dopo avere esaminato in dettaglio la costituzione e il rilascio della connessione, vediamo come vengono gestite le connessioni durante il loro utilizzo. I punti chiave del problema sono il controllo degli errori e il controllo di flusso. Il controllo degli errori si assicura che i dati vengano consegnati con il livello di affidabilità desiderato; normalmente ciò vuol dire che tutti i dati siano consegnati senza alcun errore. Il controllo del flusso impedisce che un trasmittente veloce sovraccarichi un ricevente lento.

Entrambi questi problemi si sono già visti prima quando abbiamo studiato il livello data link; le soluzioni utilizzate dal livello di trasporto utilizzano gli stessi meccanismi studiati nel Capitolo 3. Ricapitoliamo brevemente.

1. Un frame trasporta un codice di rilevamento di errore (come un CRC o un checksum) usato per verificare se l'informazione sia stata ricevuta correttamente.
2. Un frame trasporta un numero di sequenza che lo identifica e viene ritrasmesso dal mittente finché non riceve dal destinatario un acknowledgement di ricezione riuscita. Questo procedimento è chiamato **ARQ** (*automatic repeat request*).
3. Esiste un numero massimo di frame a cui il mittente permetterà di restare in sospeso in ogni momento (*finestra*), facendo pausa se il destinatario non manderà acknowledgement abbastanza velocemente. Se questo massimo è di un solo pacchetto il protocollo è chiamato **stop-and-wait**. Finestre più grandi consentono il pipelining e migliorano le prestazioni su linee lunghe e veloci.
4. Un protocollo a **finestra scorrevole** (*sliding window*) combina queste caratteristiche ed è anche utilizzato per trasferimenti di dati bidirezionali.

Visto che questi meccanismi sono utilizzati su frame a livello data link, è naturale chiedersi perché vengano utilizzati anche su segmenti a livello di trasporto. Tuttavia, nella pratica c'è una piccola duplicazione tra i livelli di trasporto e data link; anche se vengono utilizzati gli stessi meccanismi ci sono differenze a livello di funzionalità e grado.

Per quanto riguarda la differenza funzionale consideriamo il rilevamento dell'errore. Il checksum del livello data link protegge un frame mentre percorre un solo collegamento, mentre il checksum del livello di trasporto protegge un segmento mentre attraversa l'intero percorso di rete. Si tratta di un controllo end-to-end, che è diverso da un controllo su ogni collegamento. Saltzer et al. (1984) descrivono una situazione in cui i pacchetti sono stati corrotti in un router. I checksum del livello data link proteggevano i pacchetti solo mentre passavano attraverso un collegamento, non mentre erano all'interno del router. Perciò i pacchetti erano consegnati in modo errato anche se erano corretti relativamente ai controlli su ogni singolo collegamento.

Questo e altri esempi hanno portato Saltzer et al. ad articolare un **end-to-end argument** (*argomentazione end-to-end*). Secondo questo ragionamento, il controllo del livello di trasporto che funziona in modo end-to-end è essenziale per la correttezza; i controlli a livello data link non sono essenziali, ma sono preziosi per assicurare le prestazioni (poiché senza di loro un pacchetto danneggiato può essere inviato inutilmente lungo l'intero percorso).

Come differenza in grado si considerino le ritrasmissioni e il protocollo a finestra scorrevole. Molte linee wireless e collegamenti satellitari possono avere un solo frame in sospeso da un mittente in un dato istante. Cioè, il prodotto tra ritardo e larghezza di banda per il collegamento è così piccolo che neanche un frame può esservi memorizzato. In questo caso, una finestra piccola è sufficiente per delle buone prestazioni. Per esempio, 802.11 utilizza un protocollo stop-and-wait, trasmettendo o ritrasmettendo ogni frame e aspettando l'acknowledgement prima di spostarsi a quello successivo. Avere una finestra più grande di un frame aggiungerebbe complessità senza migliorare le prestazioni. Per linee cablate, come Ethernet (commutata) o dorsali di ISP, la percentuale d'errore è abbastanza bassa da poter omettere le ritrasmissioni del livello data link, poiché le ritrasmissioni end-to-end porranno rimedio alle minime perdite di frame.

D'altra parte molte connessioni TCP hanno un prodotto banda-ritardo molto maggiore di un singolo segmento. Si consideri una connessione che invia dati attraverso gli Stati Uniti a 1 Mbps con un tempo di andata e ritorno (*round-trip time*) di 100 ms. Anche con questa connessione lenta, 200 Kbit di dati saranno memorizzati dal destinatario nel tempo che occorre per spedire un segmento e ricevere un acknowledgement. Per queste situazioni deve essere usata una finestra grande. Lo stop-and-wait azzopperà le prestazioni. Nel nostro esempio le limiterà a un segmento ogni 200 ms o 5 segmenti al secondo, indipendentemente da quanto sia veloce in realtà la rete.

Dato che i protocolli di trasporto utilizzano generalmente finestre grandi, esamineremo con più cura il problema dell'immagazzinamento dei dati in un buffer (*buffering*). Poiché un host può avere molte connessioni, ognuna delle quali è trattata separatamente, potrebbe essere necessaria una notevole quantità di buffer per le finestre scorrevoli. I buffer sono necessari sia al mittente che al destinatario; certamente sono necessari al mittente per contenere tutti i segmenti che sono stati trasmessi, ma che non hanno ancora ricevuto acknowledgement; gli sono necessari perché questi segmenti potrebbero andare persi e dovrebbero essere ritrasmessi.

Tuttavia, siccome il mittente fa uso di buffer, il destinatario può o meno dedicare dei buffer specifici a singole connessioni. Il destinatario può, per esempio, mantenere un solo buffer condiviso da tutte le connessioni. Quando arriva un segmento viene fatto un tentativo per acquisire dinamicamente un nuovo buffer; se ce n'è disponibile uno, il segmento è accettato, altrimenti viene scartato. Siccome il mittente è pronto a ritrasmettere i segmenti persi dalla rete, il fatto che il destinatario scarti dei segmenti non provoca un danno permanente, sebbene vengano spercate delle risorse. Il mittente continua a tentare fintanto che non riceve un acknowledgement.

Il miglior compromesso tra l'uso di un buffer alla sorgente e l'uso alla destinazione dipende dal tipo di traffico sulla connessione. Per un traffico irregolare e con bassa richiesta di banda, come quello prodotto da un terminale interattivo, è ragionevole non dedicare alcun buffer, ma acquisirli dinamicamente a entrambe le estremità, affidandosi ai buffer del mittente se i segmenti dovessero essere occasionalmente scartati. D'altro canto per il trasferimento di file o l'invio di altro traffico che richiede grande disponibilità di banda è meglio se il destinatario dedica un'intera finestra di buffer per permettere ai dati di andare alla massima velocità. Questa è la strategia usata da TCP.

Resta da risolvere il problema riguardante come organizzare i buffer. Se la maggior parte dei segmenti ha una dimensione simile, è naturale raggrupparli per dimensione identica e usare un buffer per ogni segmento, come mostrato nella Figura 6.15(a). Tuttavia, se esiste una variazione notevole nella dimensione dei segmenti (da richieste brevi per pagine Web a pacchetti di grandi dimensioni per trasferimenti di file peer-to-peer), un insieme di buffer con dimensioni fisse genera

dei problemi. Se la dimensione del singolo buffer è scelta in base al segmento con la massima dimensione possibile, si verifica uno spreco di spazio ogni volta che viene ricevuto un segmento breve. Se la dimensione del buffer è inferiore alla dimensione massima dei segmenti, saranno necessari più buffer per i segmenti lunghi, incrementando la complessità della gestione della memoria.

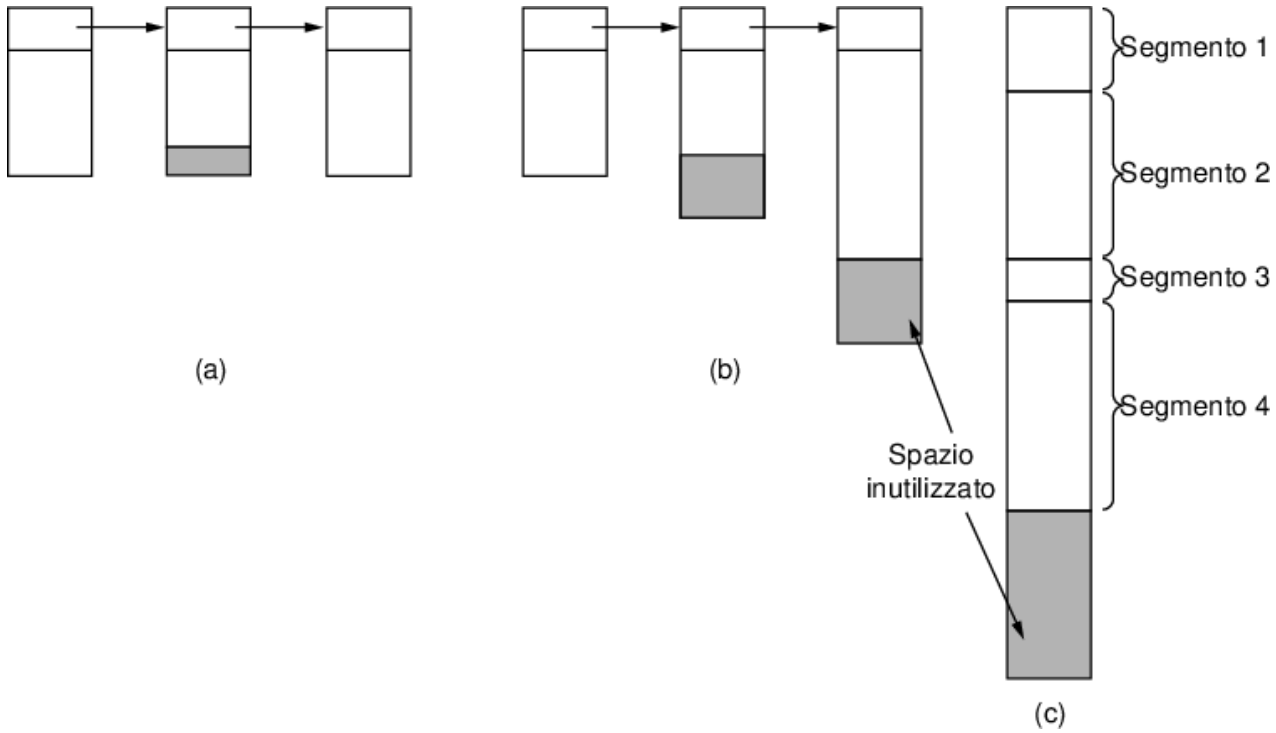


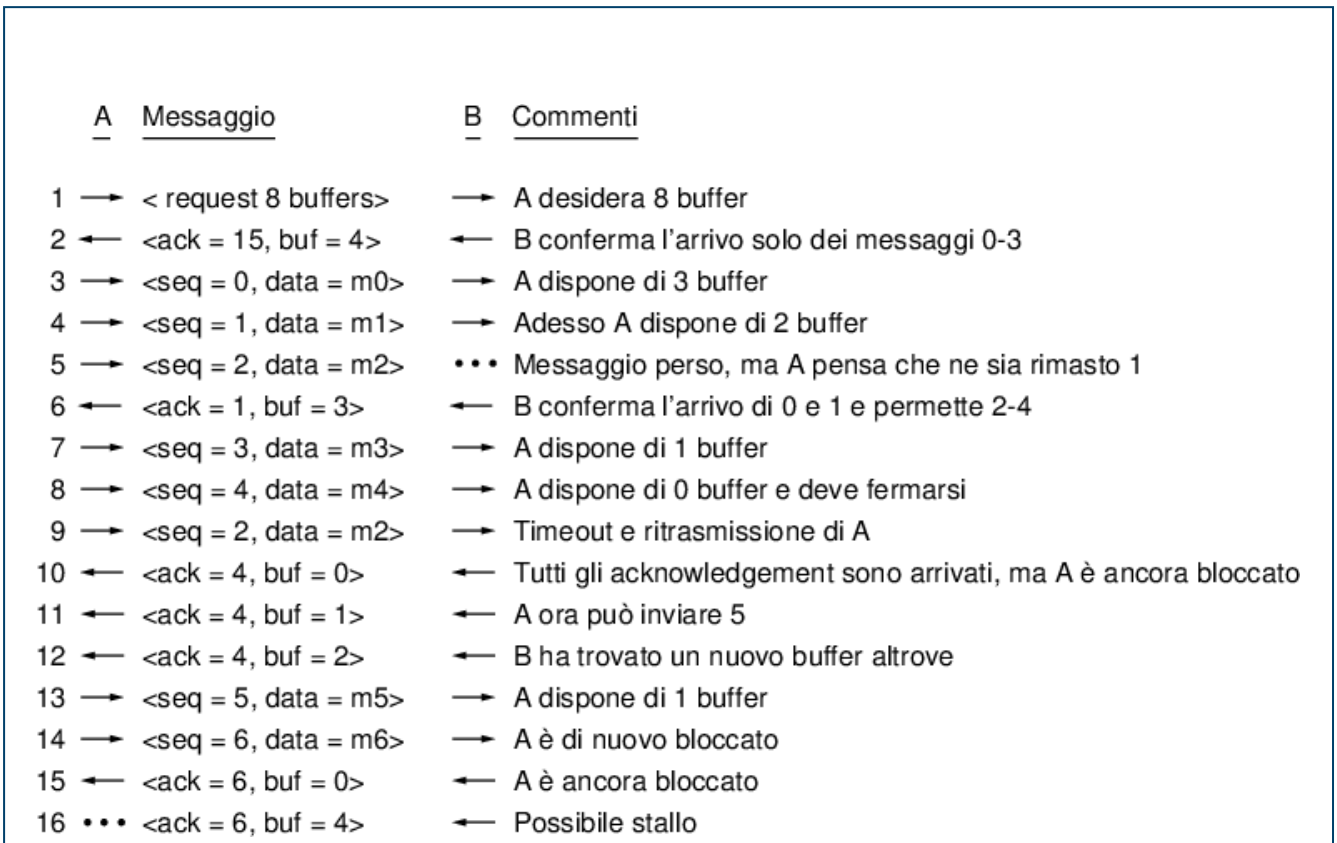
Figura 6.15 (a) Buffer concatenati a dimensione fissa. (b) Buffer concatenati a dimensione variabile. (c) Un grosso buffer circolare per ogni connessione.

Un altro approccio al problema della dimensione del buffer è l'utilizzo di buffer con dimensioni variabili, come mostrato nella Figura 6.15(b). Il vantaggio è un utilizzo migliore della memoria, al prezzo di una gestione più complicata. Una terza possibilità prevede di dedicare un singolo buffer circolare di grandi dimensioni a ogni connessione, come mostrato nella Figura 6.15(c). Questo sistema è semplice ed elegante e non dipende dalla dimensione dei segmenti, ma fa un buon utilizzo della memoria solo quando tutte le connessioni sono pesantemente utilizzate.

Con l'aprirsi e il chiudersi delle connessioni e il variare del profilo del traffico, il mittente e il destinatario devono regolare dinamicamente le allocazioni dei buffer. Di conseguenza, il protocollo di trasporto dovrebbe consentire a un host mittente di richiedere spazio nel buffer all'altra estremità. I buffer possono essere allocati per singola connessione, oppure collettivamente per tutte le connessioni in atto tra i due host. In alternativa il destinatario che conosce la situazione dei suoi buffer (ma non il traffico offerto) potrebbe comunicare al mittente "Ti ho riservato *X* buffer". Se il numero di connessioni aperte aumenta può essere necessario ridurre l'allocazione; pertanto il protocollo deve prevedere questa possibilità.

A differenza dei protocolli a finestra scorrevole del Capitolo 3, un metodo generale per gestire l'allocazione dinamica dei buffer consiste nel separare il buffering dagli acknowledgement. La gestione dinamica dei buffer si traduce in definitiva nell'uso di una finestra di dimensione variabile. Inizialmente il mittente richiede un certo numero di buffer, basato sulla previsione delle proprie esigenze e il destinatario ne concede il numero che può permettersi. Ogni volta che il mittente trasmette un segmento deve decrementare la sua allocazione, fermandosi completamente quando raggiunge il valore zero. Il destinatario può quindi aggiungere acknowledgement e informazioni sulle allocazioni del buffer in coda al traffico che scorre nella direzione opposta. TCP usa questo approccio, trasportando le allocazioni di buffer in un campo dell'intestazione chiamato *Window size*.

La Figura 6.16 mostra un esempio di come potrebbe funzionare la gestione dinamica delle finestre in una rete datagram con numeri di sequenza a 4 bit. Nell'esempio i dati viaggiano in segmenti dall'host *A* all'host *B,* mentre acknowledgement e informazioni sui buffer viaggiano in segmenti nella direzione opposta. Inizialmente *A* vuole otto buffer, ma gliene vengono garantiti solo quattro. Successivamente invia tre segmenti, di cui il terzo viene perso. Il segmento 6 fornisce l'acknowledgement a tutti i segmenti fino a quello, con numero di sequenza 1 incluso, consentendo quindi ad *A* di rilasciare i buffer coinvolti; inoltre informa *A* che ha il permesso di inviare altri tre segmenti con numero di sequenza superiore a 1 (vale a dire i segmenti 2, 3 e 4). *A* sa di avere già inviato il numero 2, pertanto pensa che potrebbe inviare i segmenti 3 e 4 e procede a farlo; a questo punto *A* è bloccato e deve aspettare un'ulteriore allocazione dei buffer. Durante il blocco possono comunque avvenire ritrasmissioni indotte dal timeout (riga 9), dal momento che utilizzano buffer già allocati. Nella riga 10, *B* conferma la ricezione di tutti i segmenti fino a 4 compreso, ma impedisce ad *A* di continuare. Tale situazione sarebbe impossibile con i protocolli a finestra fissa del Capitolo 3. Il segmento successivo da *B* ad *A* alloca un altro buffer e consente ad *A* di proseguire. Questo accadrà quando *B* ha spazio nel buffer, probabilmente perché l'utente del livello di trasporto ha accettato altri dati.



	A	Messaggio		B	Commenti
1	→	< request 8 buffers>	→		A desidera 8 buffer
2	←	<ack = 15, buf = 4>	←		B conferma l'arrivo solo dei messaggi 0-3
3	→	<seq = 0, data = m0>	→		A dispone di 3 buffer
4	→	<seq = 1, data = m1>	→		Adesso A dispone di 2 buffer
5	→	<seq = 2, data = m2>	• • •		Messaggio perso, ma A pensa che ne sia rimasto 1
6	←	<ack = 1, buf = 3>	←		B conferma l'arrivo di 0 e 1 e permette 2-4
7	→	<seq = 3, data = m3>	→		A dispone di 1 buffer
8	→	<seq = 4, data = m4>	→		A dispone di 0 buffer e deve fermarsi
9	→	<seq = 2, data = m2>	→		Timeout e ritrasmissione di A
10	←	<ack = 4, buf = 0>	←		Tutti gli acknowledgement sono arrivati, ma A è ancora bloccato
11	←	<ack = 4, buf = 1>	←		A ora può inviare 5
12	←	<ack = 4, buf = 2>	←		B ha trovato un nuovo buffer altrove
13	→	<seq = 5, data = m5>	→		A dispone di 1 buffer
14	→	<seq = 6, data = m6>	→		A è di nuovo bloccato
15	←	<ack = 6, buf = 0>	←		A è ancora bloccato
16	• • •	<ack = 6, buf = 4>	←		Possibile stallo

Figura6.16 Allocazione dinamica dei buffer. Le frecce mostrano la direzione della trasmissione. I puntini di sospensione (...) indicano un segmento perso.

Nelle reti datagram, se i segmenti di controllo possono essere persi (e lo fanno) potrebbero verificarsi problemi con gli schemi di allocazione dei buffer di questo tipo. Si osservi la riga 16. Ora *B* ha allocato altri buffer per *A*, ma il segmento di allocazione è andato perso. Dal momento che i segmenti di controllo non sono associati a numeri di sequenza o timeout, *A* è in stallo. Per impedire questa situazione, ogni host dovrebbe periodicamente inviare segmenti di controllo per fornire acknowledgement e stato del buffer di ogni connessione. In questo modo prima o poi lo stallo verrà superato.

Finora abbiamo tacitamente presunto che l'unico limite imposto sulla velocità dei dati del mittente fosse la quantità di spazio del buffer disponibile nel destinatario. Spesso non è questo il caso. La memoria una volta era costosa, ma i prezzi si sono abbassati drasticamente. Gli host possono essere dotati di sufficiente memoria, per cui la scarsità di buffer è raramente (se mai) un problema anche per connessioni su grandi distanza. Certamente questo dipende dal fatto che il buffer sia impostato a una dimensione sufficientemente grande, il che non è sempre stato il caso di TCP (Zhang et al., 2002).

Quando lo spazio per i buffer non limita più il flusso massimo, nasce un altro collo di bottiglia: la capacità di trasporto della rete. Se router adiacenti possono scambiare al massimo x pacchetti al secondo ed esistono k percorsi disgiunti tra una coppia di host, non vi è modo per tali host di scambiare più di kx segmenti al secondo, indipendentemente dalla quantità di buffer disponibile a ogni estremità. Se il mittente invia troppo rapidamente (più di kx segmenti al secondo), la rete si congestiona perché non è in grado di consegnare i segmenti con la stessa velocità con cui li riceve.

Quello che serve è un meccanismo che limita le trasmissioni del mittente basato sulla capacità di trasmissione della rete anziché sulla capacità di buffering del destinatario. Belsnes (1975) ha proposto l'utilizzo di uno schema di controllo di flusso a finestra scorrevole in cui il mittente regola dinamicamente la dimensione della finestra per farla corrispondere alla capacità di trasporto della rete. Ciò significa che una finestra scorrevole dinamica può implementare sia il controllo di flusso sia il controllo di congestione. Se la rete può gestire c segmenti al secondo e il round-trip time (comprensivo di trasmissione, propagazione, accodamento, elaborazione nel destinatario e consegna dell'acknowledgement[1] è r, la dimensione della finestra del mittente dovrebbe essere cr. Con una finestra di questa dimensione, il mittente opera normalmente utilizzando tutte le risorse del canale (la *pipeline* è piena) e un piccolo calo delle prestazioni di rete potrebbe provocarne il blocco. Dal momento che la capacità di rete disponibile per un dato flusso varia nel tempo, la dimensione della finestra dovrebbe essere regolata frequentemente per tenere traccia dei cambiamenti nella capacità di trasporto. Come vedremo in seguito, TCP utilizza uno schema simile.

6.2.5 Multiplexing

Il multiplexing di più conversazioni su connessioni, circuiti virtuali e collegamenti fisici svolge un ruolo importante in diversi livelli dell'architettura di rete. Nel livello di trasporto l'esigenza del multiplexing può sorgere in molti modi. Per esempio se un host ha a disposizione un solo indirizzo di rete, tutte le connessioni di trasporto del computer devono utilizzare tale indirizzo. Quando arriva un segmento, è necessario stabilire in qualche modo a quale processo passarlo. Questa situazione, chiamata **multiplexing**, è mostrata nella Figura 6.17(a). In questa figura quattro connessioni di trasporto distinte utilizzano la stessa connessione di rete (e cioè l'indirizzo IP) con l'host remoto.

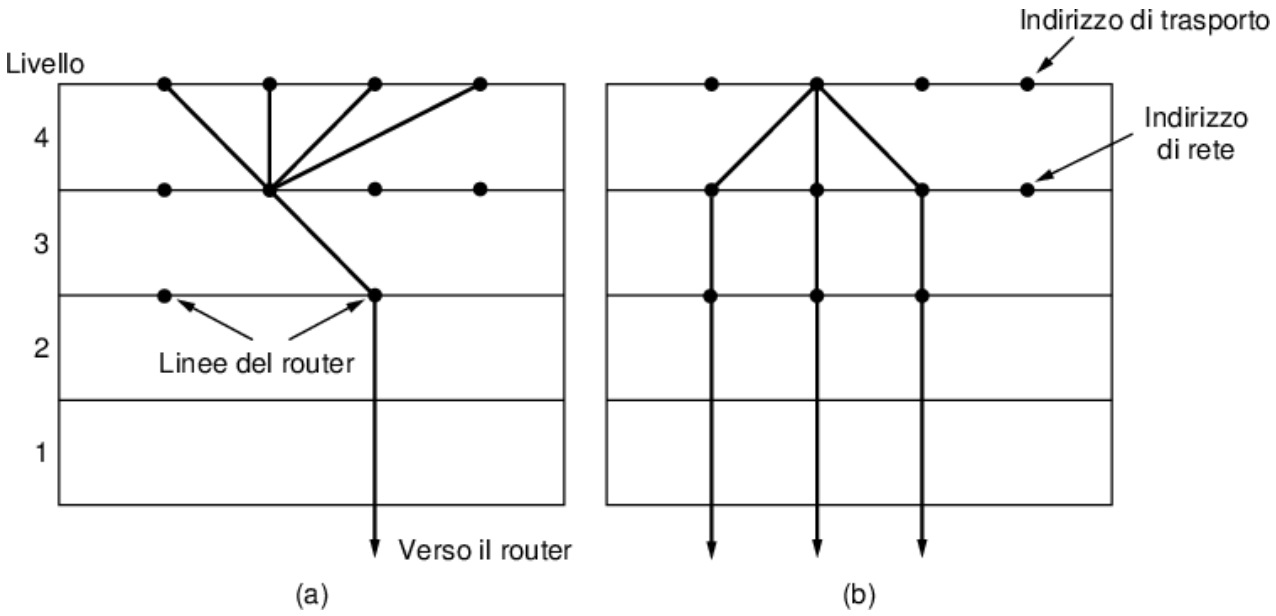


Figura6.17 (a) Multiplexing. (b) Multiplexing inverso.

Il multiplexing può essere utile nel livello di trasporto anche per un altro motivo. Si supponga, per esempio, che un host abbia diversi percorsi di rete che può usare. Se un utente ha bisogno di più ampiezza di banda o più affidabilità di quella fornita da uno dei percorsi di rete, un modo per risolvere la situazione è avere una connessione che distribuisca il traffico su diversi percorsi di rete facendo uso di round-robin, come indicato nella Figura 6.17(b). Questo modo di operare è definito **multiplexing inverso** (*inverse multiplexing*); con k connessioni di rete aperte, la larghezza di banda effettiva è aumentata di un fattore k. Un esempio di multiplexing inverso è **SCTP** (*stream control transmission protocol*), che può gestire una connessione usando più interfacce di rete. Al contrario, TCP usa un singolo endpoint di rete. Il multiplexing inverso si trova anche nel livello data link, quando molte linee a bassa capacità sono usate parallelamente come una sola linea ad alta capacità.

6.2.6 Ripristino dopo un crash

Se host e router sono soggetti a malfunzionamenti e smettono di funzionare improvvisamente (gergalmente diciamo che "vanno in crash") mentre sono in atto delle connessioni che durano molto (come download di grossi software o file multimediali), il ripristino può diventare un problema. Quando l'entità di trasporto è interamente all'interno degli host, il ripristino dopo un crash di rete o del router è banale. Le entità di trasporto prevedono sempre la perdita di segmenti e sanno come gestirla tramite ritrasmissioni. Un problema più complesso riguarda il ripristino dopo un crash dell'host. In particolare sarebbe auspicabile che i client fossero in grado di continuare a lavorare quando i server subiscono un crash e vengono rapidamente riavviati. Per illustrare la difficoltà, supponiamo che un host, il client, stia inviando un lungo file a un altro host, il file server, utilizzando un semplice protocollo stop-and-wait. Il livello di trasporto sul server si limita a passare i segmenti in arrivo all'utente del livello di trasporto uno per uno. Durante la trasmissione il server subisce un crash; dopo il riavvio le sue tabelle vengono reinizializzate, pertanto non sa più precisamente a che punto era arrivato.

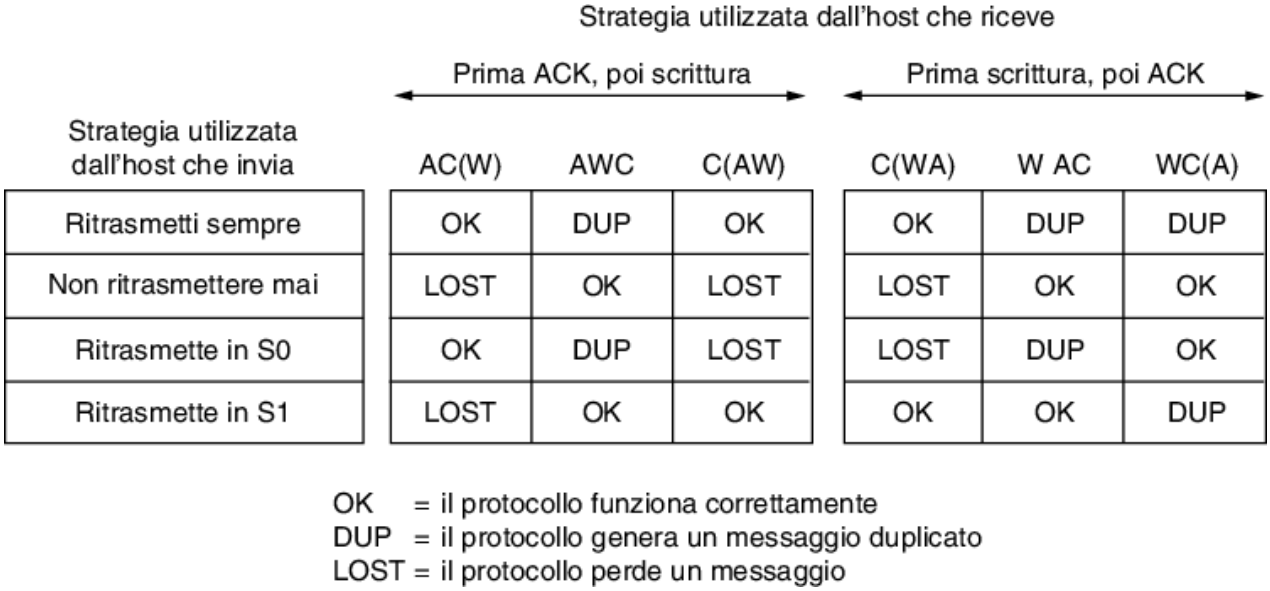
In un tentativo di recuperare il suo stato precedente, il server potrebbe inviare un segmento broadcast a tutti gli altri host, annunciando di aver subìto un crash e richiedendo che tutti i suoi client lo informino dello stato delle connessioni aperte. Ogni client può trovarsi in uno dei due stati: *S1*, con un segmento in circolazione o *S0*, senza segmenti in circolazione. In base solo a queste informazioni di stato il client deve decidere se ritrasmettere il segmento più recente.

A prima vista il risultato appare ovvio: il client dovrebbe ritrasmettere se e solo se, quando viene informato del crash, ha in circolazione un segmento che non ha ancora ricevuto acknowledgement (stato *S1*). Tuttavia un'analisi più attenta evidenzia alcune difficoltà di tale approccio. Si consideri, per esempio, la situazione in cui l'entità di trasporto del server invia prima l'acknowledgement e in seguito, dopo l'invio, passa i dati al processo applicativo. La scrittura di un segmento nel flusso di output e l'invio di un acknowledgement sono due eventi distinti che non possono essere svolti contemporaneamente. Se si verifica un crash dopo l'invio dell'acknowledgement ma prima della scrittura, il client riceverà l'acknowledgement e pertanto si troverà nello stato *S0* quando giunge l'annuncio di ripristino dal crash. Il client non eseguirà di nuovo la trasmissione, pensando (erroneamente) che il segmento sia arrivato. Questa decisione del client genera un segmento mancante.

A questo punto i lettori potrebbero pensare: "Il problema può essere risolto facilmente. È sufficiente riprogrammare l'entità di trasporto per svolgere prima la scrittura e poi inviare l'acknowledgement". In altre parole, si supponga che la scrittura sia stata eseguita, ma che il crash avvenga prima dell'invio dell'acknowledgement. Il client sarà nello stato *S1* ed eseguirà di nuovo la trasmissione, generando un segmento duplicato non rilevato nel flusso di output verso il processo applicativo del server.

Indipendentemente dall'implementazione di client e server, esistono sempre situazioni in cui il protocollo fallisce il corretto ripristino. Il server può essere implementato in due modi: prima l'acknowledgement o prima la scrittura. Il client può essere programmato in quattro modi: per ritrasmettere sempre l'ultimo segmento, per non ritrasmettere mai l'ultimo segmento, per ritrasmetterlo solo nello stato *S0* o per ritrasmetterlo solo nello stato *S1*. Si ottengono così otto combinazioni: come potremo vedere, per ogni combinazione esiste un insieme di eventi che provocano il fallimento del protocollo.

Per il server sono possibili tre eventi: invio di un acknowledgement (*A*), passaggio dei dati al processo di output (*W*) e crash (*C*). I tre eventi possono avvenire in sei ordini diversi: *AC*(*W*), *AWC*, *C*(*AW*), *C*(*WA*), *WAC* e *WC*(*A*); le parentesi sono usate per indicare che né *A* né *W* possono seguire *C* (una volta che l'host ha subìto un crash non c'è più nulla da fare). La Figura 6.18 mostra le otto combinazioni della strategia per client e server e le sequenze di eventi valide per ognuno. Occorre notare che per ogni strategia esiste una sequenza di eventi che provoca il fallimento del protocollo. Per esempio, se il client ritrasmette sempre la ritrasmissioni, l'evento *AWC* genera un duplicato non rilevato, anche se gli altri due eventi funzionano correttamente.



Strategia utilizzata dall'host che invia	Strategia utilizzata dall'host che riceve: Prima ACK, poi scrittura			Strategia utilizzata dall'host che riceve: Prima scrittura, poi ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Ritrasmetti sempre	OK	DUP	OK	OK	DUP	DUP
Non ritrasmettere mai	LOST	OK	LOST	LOST	OK	OK
Ritrasmette in S0	OK	DUP	LOST	LOST	DUP	OK
Ritrasmette in S1	LOST	OK	OK	OK	OK	DUP

OK = il protocollo funziona correttamente
DUP = il protocollo genera un messaggio duplicato
LOST = il protocollo perde un messaggio

Figura 6.18 Le diverse combinazioni di strategie per client e server.

Rendere il protocollo più complicato non aiuta. Anche se il client e il server scambiano diversi segmenti prima che il server tenti di eseguire la scrittura, in modo che il client sappia esattamente che cosa stia per accadere, il client non ha modo di sapere se un crash sia avvenuto appena prima o subito dopo la scrittura. La conclusione è inevitabile: se postuliamo la mancata simultaneità degli eventi, il crash e il ripristino dell'host non possono essere resi trasparenti ai livelli superiori.

In termini più generali questo risultato può essere esposto come segue: "Il ripristino di crash nel livello *N* può essere svolto solo dal livello *N* + 1" e solo se il livello superiore dispone di informazioni di stato sufficienti a ricostruire la situazione prima dell'insorgere del problema. Come affermato in precedenza, il livello di trasporto può ripristinare gli errori nel livello di rete, purché ciascun estremo di una connessione tenga traccia del punto in cui è arrivato.

Questo problema ci porta ad affrontare la questione del vero significato di un acknowledgement end-to-end. In linea di principio, il protocollo di trasporto è di tipo end-to-end e non concatenato come i livelli inferiori. Ora si consideri il caso di un utente che inserisce richieste di transazioni su un database remoto. Si supponga che l'entità di trasporto remota sia programmata per passare prima i segmenti al livello superiore e poi inviare l'acknowledgment. Anche in questo caso, la

ricezione di un acknowledgement sulla macchina dell'utente non significa necessariamente che l'host remoto sia rimasto attivo per il tempo sufficiente ad aggiornare il database. Un vero acknowledgement end-to-end, la cui ricezione indica che il lavoro è stato svolto e la cui mancanza indica il contrario, è praticamente impossibile da ottenere. Questo punto è discusso in maggiore dettaglio da Saltzer et al. (1984).

6.3 Controllo di congestione

Se le entità di trasporto di molte macchine inviano troppo velocemente troppi pacchetti nella rete, questa si congestionerà con decadimento delle prestazioni, in quanto i pacchetti vengono ritardati e persi. Controllare la congestione per evitare questo problema è una responsabilità combinata dei livelli di rete e di trasporto. La congestione avviene nei router, quindi viene individuata dal livello di rete. Tuttavia la congestione è in ultima analisi causata dal traffico inviato in rete dal livello di trasporto. L'unico modo efficace per controllare la congestione è che i protocolli di trasporto spediscano i pacchetti in rete più lentamente.

Nel Capitolo 5 abbiamo studiato i meccanismi di controllo della congestione nel livello di rete. In questo paragrafo studieremo l'altra metà del problema, i meccanismi di controllo della congestione nel livello di trasporto. Dopo aver descritto gli obiettivi del controllo di congestione, descriveremo come gli host possano regolare il tasso a cui spedire i pacchetti in rete. Internet si affida pesantemente al livello di trasporto per il controllo della congestione tramite algoritmi specifici all'interno di TCP e altri protocolli.

6.3.1 Allocazione auspicabile di banda

Prima di descrivere come regolare il traffico dobbiamo capire che cosa cerchiamo di realizzare utilizzando un algoritmo di controllo della congestione. Dobbiamo cioè specificare la condizione in cui verrà portata la rete da un buon algoritmo di controllo della congestione. L'obiettivo è molto più che evitare semplicemente la congestione; si tratta di trovare una buona allocazione di banda per le entità di trasporto che utilizzano la rete. Una buona allocazione darà buone prestazioni, perché utilizzerà tutta l'ampiezza di banda disponibile ma eviterà congestioni, sarà equa tra entità di trasporto in competizione tra loro e terrà velocemente traccia dei cambiamenti delle richieste di risorse da parte del traffico.

Efficienza ed energia

Un'allocazione di banda efficente tra entità di trasporto utilizzerà tutta la capacità di rete disponibile. Tuttavia non è completamente corretto pensare che se c'è un collegamento da 100 Mbps, cinque entità di trasporto avranno 20 Mbps ciascuna. Di solito avranno meno di 20 Mbps per avere delle buone prestazioni. Il motivo è che spesso il traffico è irregolare. Ricordiamo che nel Paragrafo 5.3 abbiamo descritto il **goodput** (o il tasso con cui pacchetti utili arrivano al destinatario) come funzione del traffico offerto alla rete. Questa curva e una curva corrispondente per il ritardo in funzione del carico offerto sono riportate nella Figura 6.19.

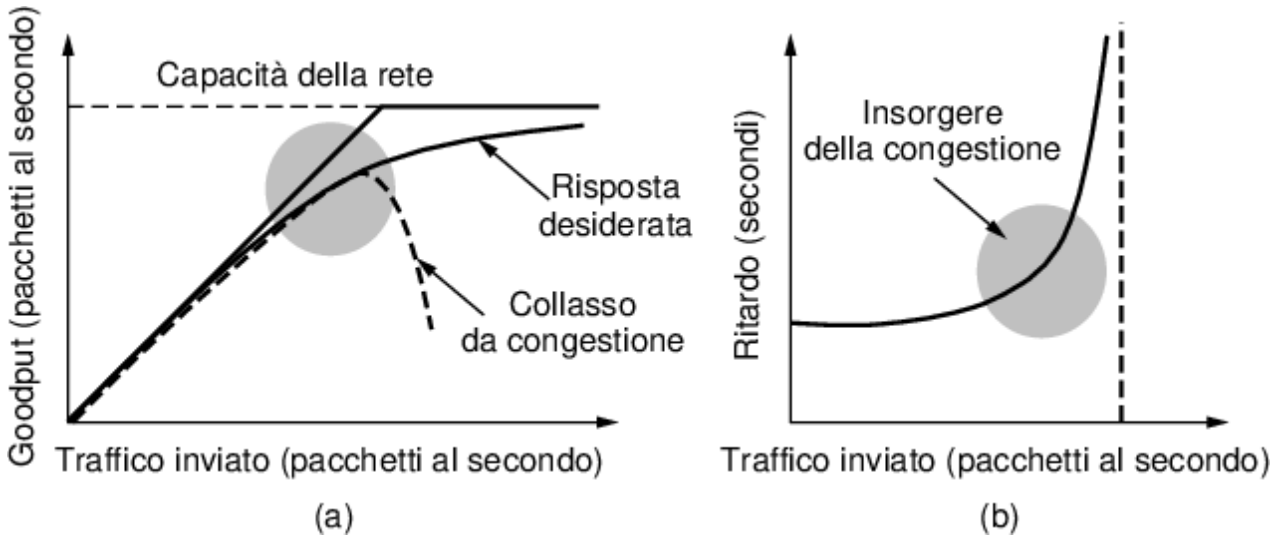


Figura6.19 (a) Goodput e (b) ritardo in funzione del traffico inviato.

Nella Figura 6.19(a) quando il carico inizia ad aumentare, il goodput aumenta con lo stesso tasso, ma come il carico si avvicina alla capacità, il goodput cresce più gradualmente. Questa diminuzione è dovuta a picchi occasionali di traffico che provocano alcune perdite nei buffer nella rete. Se il protocollo di trasporto è progettato male e ritrasmette pacchetti in ritardo ma non persi, la rete può collassare per congestione. In questo caso i mittenti spediscono pacchetti furiosamente, ma ottengono sempre meno risultati.

Il ritardo corrispondente è riportato nella Figura 6.19(b). Inizialmente il ritardo è fisso ed equivale al ritardo di propagazione nella rete. Come il carico si avvicina alla capacità, il ritardo cresce, all'inizio lentamente e poi sempre più rapidamente. Ciò accade, nuovamente, a causa di picchi di traffico il cui numero aumenta in caso di carichi elevati. Il ritardo non può in realtà aumentare all'infinito, tranne in un modello in cui i router abbiano buffer infiniti, ma i pacchetti saranno persi dopo aver subìto il ritardo massimo possibile nei buffer.

Sia per il goodput sia per il ritardo, le prestazioni iniziano a degradare all'insorgenza della congestione. Intuitivamente, otterremmo dalla rete le prestazioni migliori se assegnassimo larghezza di banda finché il ritardo inizi a crescere rapidamente. Questo punto è sotto la capacità e per identificarlo, Kleinrock (1979) propose una metrica basata sull'energia:

$$energia = \frac{carico}{ritardo}$$

L'energia crescerà inizialmente con il carico, poiché il ritardo rimane piccolo e abbastanza costante, ma raggiungerà il massimo e crollerà quando il ritardo crescerà rapidamente. Il carico con maggiore energia rappresenta per l'entità di trasporto un carico efficiente da mettere in rete.

Max-min fairness

Nella discussione precedente non abbiamo parlato di come dividere la banda tra diversi mittenti a livello di trasporto. Sembra una domanda semplice a cui rispondere (basta dare a ogni mittente la stessa porzione di banda), ma occore fare alcune considerazioni.

Forse la prima considerazione è chiedersi che cos'abbia a che fare questo problema con il controllo della congestione. Dopo tutto, se la rete dà al mittente una certa quantità di banda da utilizzare, il mittente potrà solo che usare quella quantità. Tuttavia, spesso succede che le reti non abbiano una rigorosa quantità di banda riservata a ogni flusso o connessione. Possono farlo, per alcuni flussi, se è supportata la qualità di servizio; molte connessioni, comunque, cercheranno di usare quanta più banda possibile o saranno raccolte sotto una allocazione condivisa. Per esempio i servizi differenziati IETF separano il traffico in due classi e le connessioni competono per la banda all'interno di ognuna di esse. I router IP hanno spesso tutte le connessioni in competizione per la stessa ampiezza di banda. In questa situazione è il meccanismo di controllo della congestione che distribuisce la banda alle connessioni in competizione tra loro.

Una seconda considerazione è che cosa significhi una giusta porzione per i flussi in una rete. Se N flussi usano un solo collegamento è abbastanza semplice: in questo caso possono avere tutti $1/N$ della larghezza di banda totale (anche se l'efficienza detterà che ne usino un po' meno nel caso in cui il traffico presenti dei picchi). Ma che cosa succede se i flussi hanno percorsi di rete diversi, ma sovrapposti? Per esempio un flusso può attraversare tre collegamenti e gli altri flussi possono attraversarne uno solo. Il flusso su tre collegamenti consuma più risorse di rete; potrebbe essere meglio, in un certo senso, dargli una minore larghezza di banda rispetto ai flussi su un solo collegamento, perché sarebbe certamente possibile supportarne di più. Questo fatto dimostra un conflitto intrinseco tra equità ed efficienza.

Tuttavia adotteremo una nozione di equità che non dipende dalla lunghezza del percorso di rete. Anche con questo semplice modello, attribuire a tutte le connessioni la stessa ampiezza di banda è un po' complicato, perché queste prenderanno percorsi distinti nella rete e i percorsi avranno a loro volta capacità diverse. In questo caso è possibile che un flusso sia strozzato su un collegamento in downlink e che prenda una porzione più piccola rispetto ad altri flussi su un collegamento in uplink; ridurre l'ampiezza di banda degli altri flussi li rallenterà, ma non agevolerà per niente il flusso strozzato.

La forma di equità spesso desiderata per l'utilizzo delle reti è **max-min fairness** (*equità max-min*). Un'allocazione è "max-min equa" se l'ampiezza di banda attribuita a un flusso non può essere aumentata senza diminuire l'ampiezza di banda data a un altro flusso a cui è stata assegnata un'allocazione minore o uguale. Quindi aumentare l'ampiezza di banda di un flusso peggiora la situazione per i flussi già meno abbienti.

Vediamo un esempio: un'allocazione max-min equa è illustrata per una rete con i quattro flussi *A*, *B*, *C* e *D* nella Figura 6.20. Ogni collegamento tra router ha la stessa capacità, che diremo di 1 unità, anche se nel caso generale i collegamenti avranno capacità diverse. Tre flussi competono per il collegamento in basso a sinistra tra i router *R4* e *R5*. Ognuno di questi, quindi, ottiene 1/3 del collegamento. Il flusso restante, *A*, compete con *B* sul collegamento tra *R2* e *R3*. Poiché *B* ha un'allocazione di 1/3, *A* ottiene i restanti 2/3. Si noti che tutti gli altri collegamenti hanno della capacità residua inutilizzata. Tuttavia questa capacità non può essere data ad alcun flusso senza diminuire la capacità di un altro flusso inferiore. Per esempio se una maggiore ampiezza di banda sul collegamento tra *R2* e *R3* viene data al flusso *B*, ce ne sarà meno per il flusso *A*. Ciò ha un senso, in quanto *A* possiede già una banda maggiore. Tuttavia, la capacità dei flussi *C* o *D* (o entrambi) deve essere diminuita per attribuire più banda a *B*, e questi flussi avranno meno banda di *B*. Quindi l'allocazione è max-min equa.

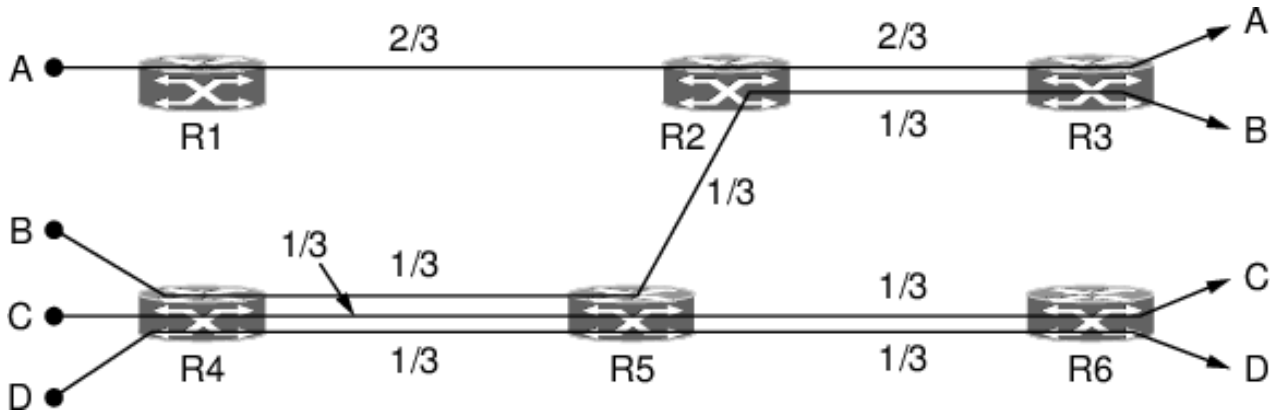


Figura6.20 Allocazione di banda max-min per quattro flussi.

Data una conoscenza globale della rete, si possono calcolare le allocazioni max-min. Un modo intuitivo è quello di immaginare che il tasso di trasmissione di tutti i flussi inizi a zero e cresca lentamente. Quando il tasso raggiunge il limite di uno dei flussi quel flusso smette di aumentare. Gli altri, invece, continuano a crescere, condividendo equamente la capacità residua, finché anch'essi raggiungeranno i rispettivi massimi.

Una terza considerazione riguarda il livello su cui considerare l'equità. Una rete può essere equa a livello di connessioni, connessioni tra coppie di host o di tutte le connessioni degli host. Abbiamo esaminato questo problema quando abbiamo discusso di WFQ (*weighted fair queueing*) nel Paragrafo 5.4 e abbiamo concluso che ognuna di queste definizioni ha dei problemi. Per esempio definire l'equità a livello di singolo host significa che un server molto occupato non se la passerà meglio di un telefono cellulare, mentre definire l'equità a livello di connessione incoraggia gli host ad aprire più connessioni. In assenza di risposte chiare l'equità è spesso considerata per connessione, ma non ci si pone il problema di ottenerla in modo preciso: è più importante nella pratica che

nessuna connessione rimanga senza risorse rispetto all'assicurarsi che tutte le connessioni abbiano precisamente la stessa quantità di banda. Infatti con TCP è possibile aprire connessioni multiple e competere più aggressivamente per la banda.

Convergenza

L'ultimo criterio da applicare all'algoritmo di controllo della congestione è che questo converga velocemente verso una allocazione della banda equa ed efficiente. La precedente discussione riguardo la condizione di lavoro auspicabile per la rete presuppone un ambiente statico; le connessioni, invece, vengono create e rilasciate di continuo e anche la banda necessaria a una data connessione varia nel tempo per esempio quando un utente visualizza pagine Web e occasionalmente scarica video di grandi dimensioni.

A causa della variazione della domanda, la condizione di lavoro ideale varia nel tempo. Un buon algoritmo di controllo della congestione dovrebbe convergere rapidamente al punto ideale e seguirlo mentre varia nel tempo. Se la convergenza è troppo lenta, l'algoritmo non sarà mai vicino all'obiettivo, che cambierà. Se l'algoritmo non è stabile, in alcuni casi può fallire nel convergere al punto esatto o anche oscillarvi attorno.

Un esempio di allocazione di banda che cambia nel tempo e converge velocemente è mostrato nella Figura 6.21. All'inizio il flusso 1 possiede tutta la banda. Un secondo dopo inizia il flusso 2, anch'esso desideroso di banda. L'allocazione cambia velocemente per dare a ogni flusso metà dell'ampiezza di banda. A 4 secondi si unisce un terzo flusso, il quale usa solo il 20% di banda, che è meno della sua quota equa (che sarebbe un terzo). I flussi 1 e 2 si regolano rapidamente, dividendo la banda disponibile per averne ognuno il 40%. A 9 secondi, il secondo flusso cessa e il terzo flusso rimane invariato. Il primo flusso si accaparra velocemente l'80% di banda. In ogni momento la banda totale allocata è approssimativamente il 100%, quindi la rete è completamente utilizzata e i flussi in competizione ottengono un trattamento equo (ma non devono usare più banda di quella di cui hanno bisogno).

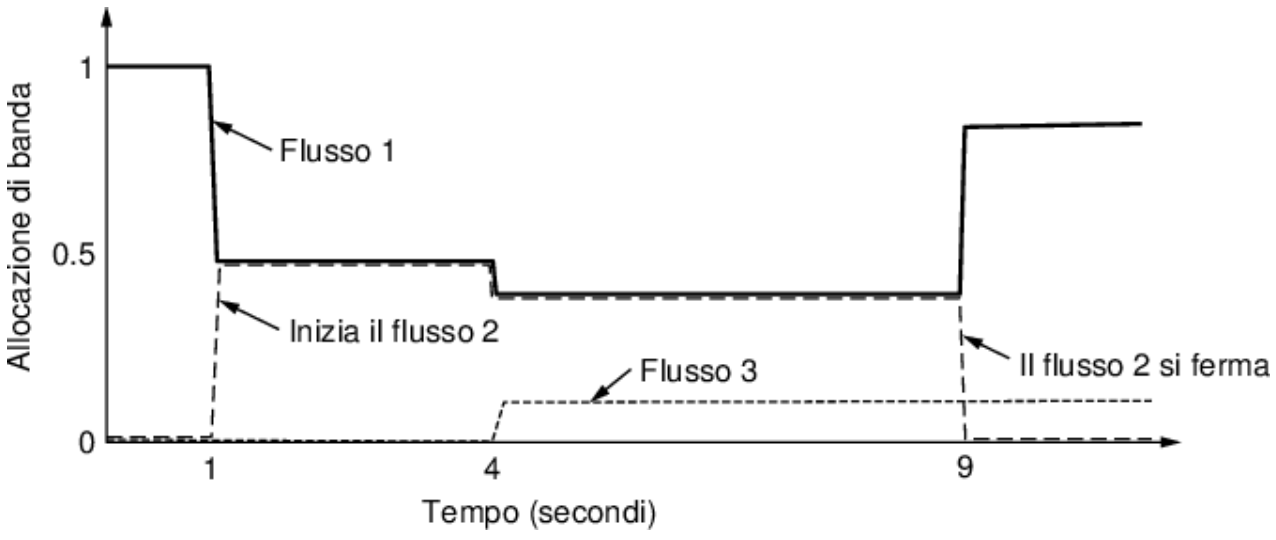


Figura6.21 La variazione dell'allocazione di banda nel tempo.

6.3.2 Regolazione del tasso di spedizione

Ora arriva il piatto forte: come regoliamo il tasso di spedizione per ottenere una buona allocazione di banda? Il tasso di spedizione può essere limitato da due fattori. Il primo è il controllo di flusso nel caso in cui il destinatario abbia buffer insufficiente. Il secondo è la congestione, nel caso in cui ci sia insufficiente capacità nella rete. Nella Figura 6.22 vediamo questo problema illustrato dal punto di vista idraulico; nella Figura 6.22(a) vediamo un grosso tubo che porta a un ricevente di scarsa capacità. Questa è una situazione limitata dal controllo di flusso limitato: finché il mittente non spedisce più acqua di quella che può essere contenuta dal recipiente, non verrà perso liquido. Nella Figura 6.22(b) il fattore di limitazione non è la capienza del contenitore, ma la capacità interna di trasporto della rete. Se l'acqua che arriva è troppa e troppo veloce, tornerà indietro e in parte andrà persa (facendo in questo caso straripare l'imbuto).

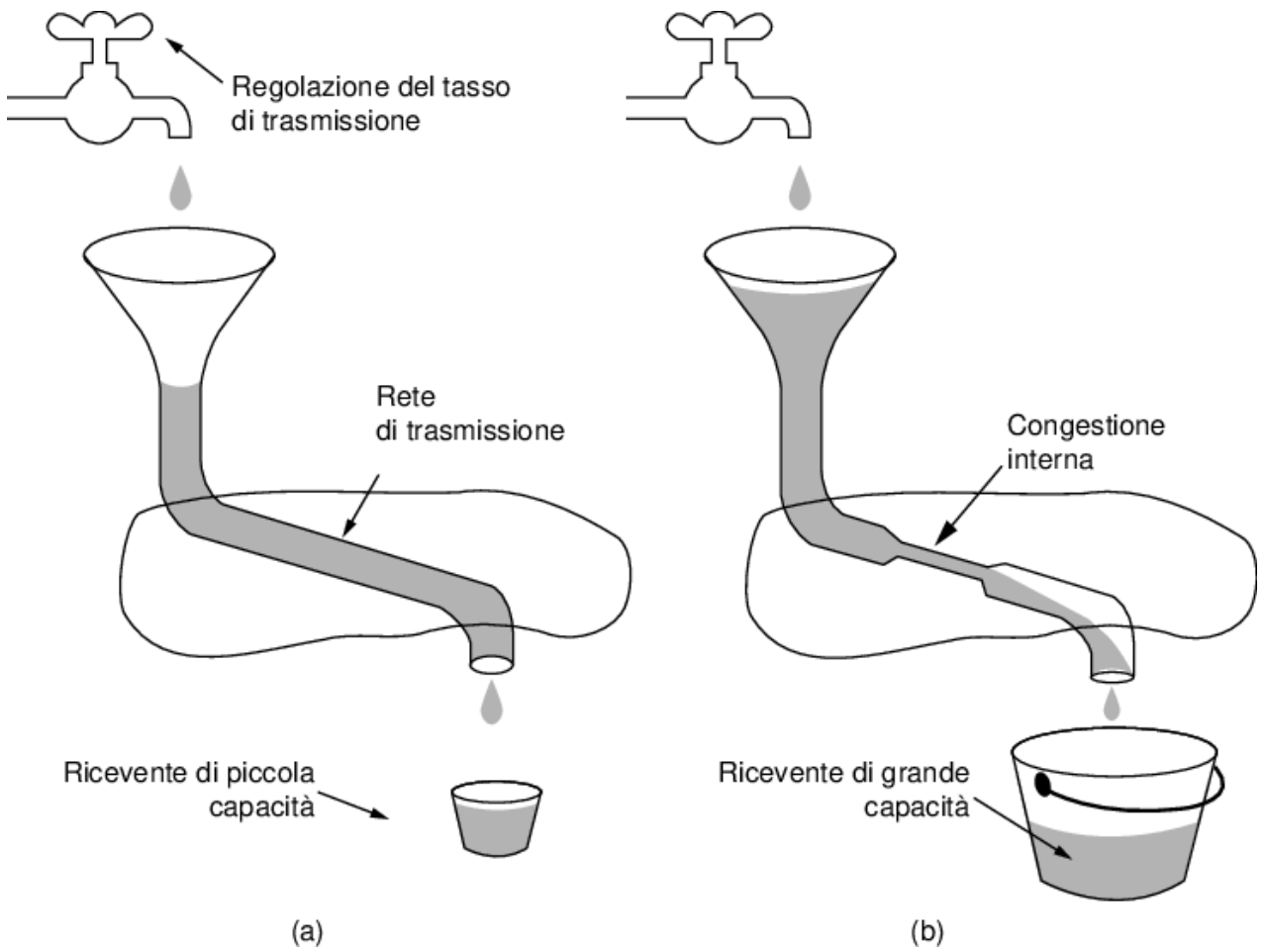


Figura 6.22 (a) Una rete veloce che alimenta un ricevente a bassa capacità. (b) Una rete lenta che alimenta un ricevente ad alta capacità.

Questi casi possono apparire simili a quelli che accadono al mittente, poiché trasmettere troppo velocemente può far perdere dei pacchetti. Tuttavia hanno cause diverse e soluzioni distinte. Abbiamo già parlato di una soluzione per il controllo di flusso tramite una finestra a dimensione

variabile; ora considereremo una soluzione basata sul controllo di congestione. Dato che possono verificarsi entrambi i problemi, in generale il protocollo di trasporto avrà bisogno di attuare entrambe le soluzioni e rallentare quando se ne verifica uno.

Il modo in cui un protocollo di trasporto dovrà regolare il tasso di spedizione dipende dal feedback restituito dalla rete. Differenti livelli di rete possono restituire differenti tipi di feedback: il feedback può essere esplicito o implicito, preciso o impreciso.

Un esempio di architettura esplicita e precisa è quando i router indicano alle sorgenti il tasso al quale possono spedire. Alcune architetture presenti in letteratura come **XCP** (*explicit congestion protocol*) operano in questo modo (Katabi et al., 2002). Un'architettura esplicita e imprecisa è rappresentata dall'uso di **ECN** (*explicit congestion notification*) di TCP; i router impostano dei bit nei pacchetti che sono soggetti a congestione per avvertire il mittente di rallentare, ma non dicono di quanto.

In altre architetture un segnale esplicito non c'è. FAST TCP misura il ritardo di round-trip e lo usa come segnale per evitare una congestione (Wei et al., 2006). Infine nella forma di controllo di congestione prevalente oggi in Internet (TCP con *drop-tail* o router RED), la perdita di pacchetti viene utilizzata per segnalare che la rete è diventata congestionata. Ci sono molte varianti di questa forma di TCP, inclusa CUBIC TCP che viene utilizzata da Linux (Ha et al., 2008). Sono possibili anche delle combinazioni. Per esempio, Windows fa uso di Compound TCP che utilizza sia la perdita sia il ritardo dei pacchetti come feedback (Tan et al., 2006). Tutte queste architetture sono riassunte nella Figura 6.23.

Protocollo	Segnale	Esplicito?	Preciso?
XCP	Velocità da usare	Sì	Sì
TCP con ECN	Avviso di congestione	Sì	No
FAST TCP	Ritardo end-to-end	No	Sì
Compound TCP	Perdita di pacchetti e ritardo end-to-end	No	Sì
CUBIC TCP	Perdita di pacchetti	No	No
TCP	Perdita di pacchetti	No	No

Figura6.23 Segnali di alcuni protocolli di controllo della congestione.

Se viene dato un segnale preciso ed esplicito, l'entità di trasporto può usare quel segnale per portare il suo tasso verso un nuovo punto di funzionamento. Per esempio, se XCP dà ai mittenti il tasso da utilizzare, i mittenti potranno semplicemente utilizzarlo. Negli altri casi, tuttavia, vengono fatte delle ipotesi: in assenza di segnale di congestione i mittenti possono aumentare il proprio tasso, ma

quando ricevono tale segnale devono diminuirlo. Il modo in cui i tassi vengono aumentati o diminuiti è dato da una **legge di controllo** (*control law*) che ha un effetto molto forte sulle prestazioni.

Chiu e Jain (1989) hanno studiato il caso di feedback di congestione binario e hanno concluso che **AIMD** (*additive increase multiplicative decrease*) è la legge di controllo appropriata per arrivare a un equo ed efficiente punto di funzionamento. Per sostenere tale discorso, hanno impostato il ragionamento attraverso un grafico per il semplice caso di due connessioni in competizione per la banda di un singolo collegamento. Il grafico nella Figura 6.24 mostra la banda allocata all'utente 1 sull'asse x e all'utente 2 sull'asse y. Quando l'allocazione è equa, entrambi gli utenti ricevono la stessa quantità di banda; questo è illustrato dalla linea di fairness tratteggiata. Quando la somma delle allocazioni arriva al 100% (la capacità del collegamento), l'allocazione è efficiente. Questo è illustrato dalla linea di efficienza tratteggiata. La rete dà un segnale di congestione a entrambi gli utenti quando la somma delle loro allocazioni supera questa linea. L'intersezione di queste linee è il punto di funzionamento ottimale, quello in cui entrambi gli utenti hanno la stessa ampiezza di banda e tutta la banda della rete è utilizzata.

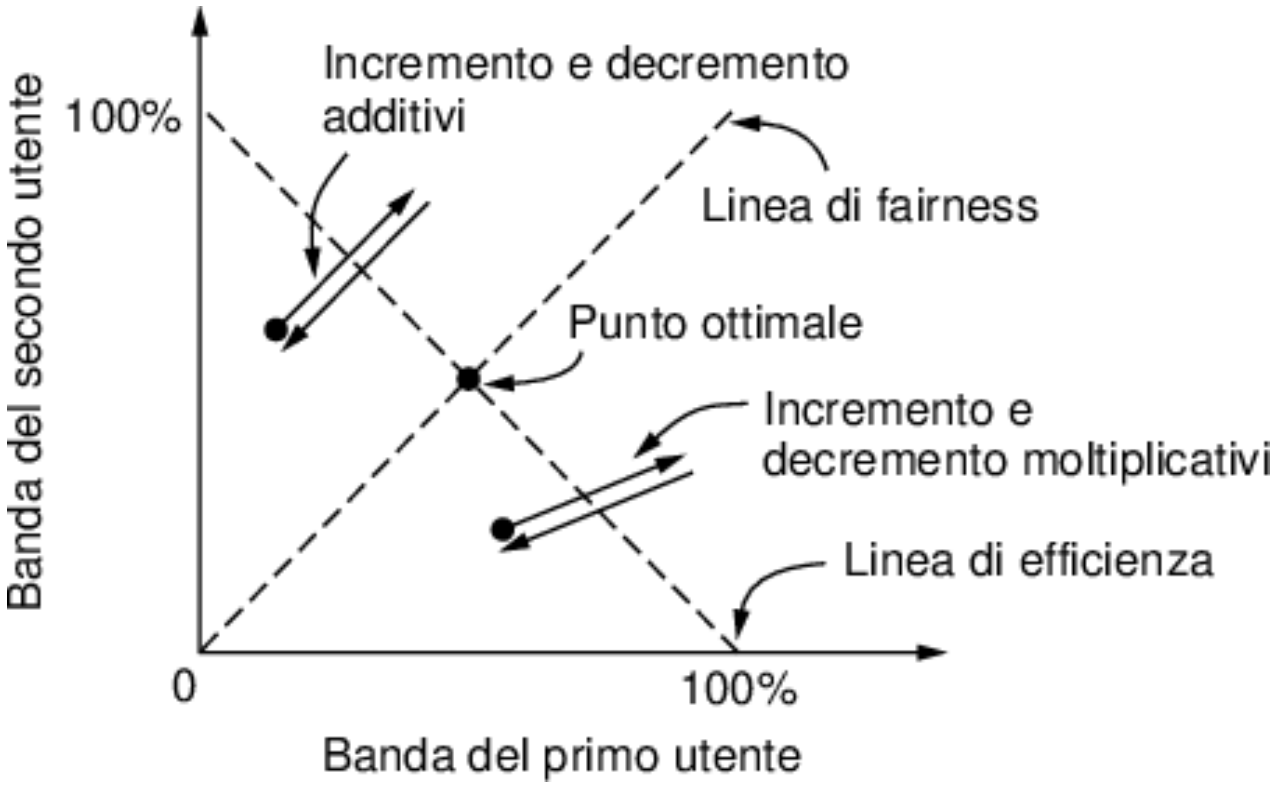


Figura6.24 Regolazioni di banda additiva e moltiplicativa.

Consideriamo che cosa accadrebbe se, partendo da una situazione iniziale sia l'utente 1 che l'utente 2 aumentassero additivamente nel tempo il loro uso di risorse. Per esempio ogni utente potrebbe aumentare il proprio tasso di spedizione di 1 Mbps ogni secondo. Col tempo il punto di funzionamento oltrepasserà la linea di efficienza ed entrambi gli utenti riceveranno un segnale di

congestione dalla rete; a questo punto dovranno ridurre le loro allocazioni. Tuttavia una diminuzione additiva li porterà solo a oscillare lungo una linea "additiva". Questa situazione è illustrata nella Figura 6.24. Tale comportamento porterà il punto di funzionamento vicino all'efficienza, senza però essere necessariamente equo.

Analogamente, consideriamo il caso in cui entrambi gli utenti moltiplicano nel tempo il loro uso di risorse finché non ricevono un segnale di congestione. Per esempio, gli utenti potrebbero aumentare il loro tasso di spedizione del 10% ogni secondo. Se quindi diminuissero per multipli i loro tassi di spedizione, il punto operativo degli utenti semplicemente oscillerebbe lungo una linea "moltiplicativa". Questo comportamento è anch'esso illustrato nella Figura 6.24. La linea moltiplicativa ha una pendenza diversa rispetto a quella additiva (punta verso l'origine, mentre la linea additiva presenta un angolo di 45 gradi), ma la situazione non migliora. In nessun caso gli utenti convergeranno a un tasso di invio ottimale che sia equo ed efficiente.

Consideriamo ora il caso in cui gli utenti aumentino additivamente l'uso delle risorse e lo diminuiscano moltiplicativamente quando viene segnalata una congestione. Questo comportamento è la legge di controllo AIMD, mostrata nella Figura 6.25. Si può vedere che il percorso tracciato da questo comportamento converge al punto ottimale che è sia equo sia efficiente. Tale convergenza avviene indipendentemente da quale sia il punto iniziale, rendendo AIMD utile in tutti i casi. Per la stessa motivazione l'unica altra combinazione, aumento moltiplicativo e diminuzione additiva, divergerà dal punto ottimale.

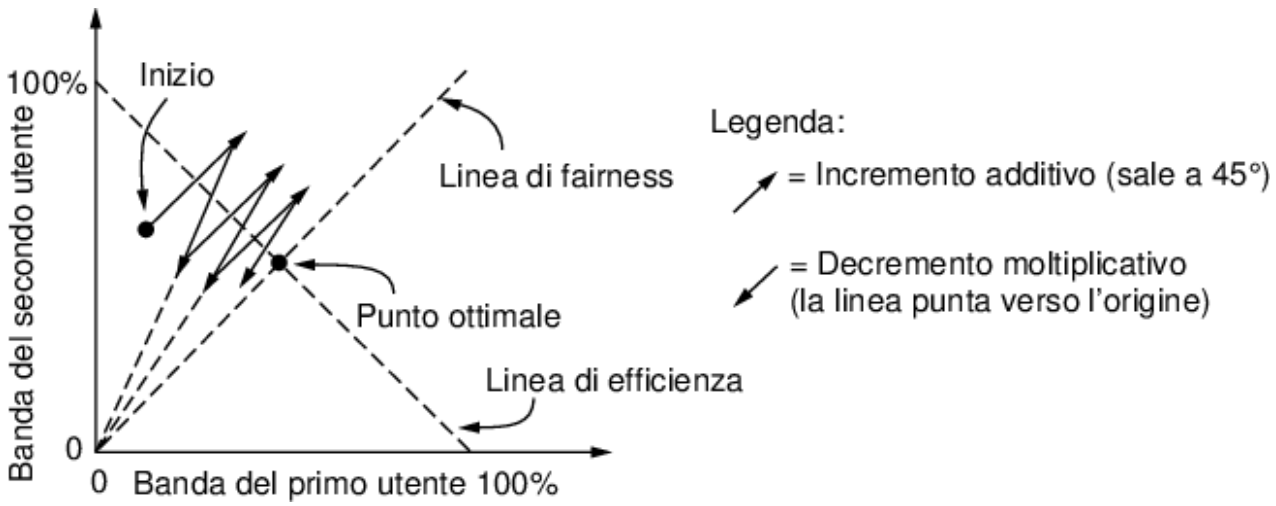


Figura6.25 Legge di controllo con incremento additivo e decremento moltiplicativo (AIMD).

AIMD è la legge di controllo utilizzata da TCP, basata su questo ragionamento e altri riguardanti la stabilità; in particolare, è facile portare la rete alla congestione ma difficile farla tornare a funzionare normalmente; quindi la politica di aumento dovrà essere gentile e la politica di diminuzione dovrà essere aggressiva. Non è però così equo: siccome le connessioni TCP aggiustano le dimensioni delle loro finestre di un quantitativo noto per ogni round-trip time,

connessioni diverse avranno tempi di round-trip distinti. Questo genera una distorsione in cui le linee verso gli host più vicini ricevono più banda di quelle verso gli host lontani, lasciando eque tutte le altre.

Nel Paragrafo 6.5 descriveremo in dettaglio come TCP implementi la legge di controllo AIMD per aggiustare il tasso di spedizione e fornire controllo di congestione. Questo compito è più difficile di quel che sembra, in quanto i tassi sono misurati su intervalli e il traffico è irregolare. Invece di aggiustare direttamente il tasso di invio, una strategia spesso utilizzata nella pratica è aggiustare la dimensione di una finestra scorrevole; TCP usa questa strategia. Se la dimensione della finestra è W e il round-trip time è RTT, il tasso equivalente è W/RTT. Questa strategia è facile da combinare con il controllo del flusso, che già usa una finestra e presenta il vantaggio che il mittente stabilisce il ritmo dei pacchetti usando gli acknowledgement e quindi rallenta in un RTT se smette di ricevere segnalazioni.

Affrontiamo come problema finale il fatto che ci possono essere molti protocolli di trasporto diversi che spediscono traffico nella rete. Che cosa succederebbe se i diversi protocolli competessero con diverse leggi di controllo per evitare la congestione? Allocazioni di banda diseguali, ecco che cosa. Siccome TCP è la forma dominante di controllo di congestione in Internet, c'è una significativa pressione della comunità scientifica perché vengano progettati nuovi protocolli di trasporto in grado di convivere con questo protocollo. I primi protocolli di streaming multimediale causarono problemi riducendo eccessivamente il throughput di TCP, perché non competevano correttamente. Ciò ha condotto alla nozione di controllo di congestione **TCP-friendly**, in cui i protocolli di trasporto TCP e non-TCP possono essere liberamente mischiati senza effetti negativi (Floyd et al., 2000).

6.3.3 Problemi con le reti wireless

Protocolli di trasporto come TCP, che implementano il controllo della congestione, dovrebbero essere indipendenti dalle tecnologie di rete sottostante e dal livello data link. Questa è una buona teoria, ma in pratica sorgono problemi con le reti wireless. Il principale è che la perdita di pacchetti viene spesso usata come segnale di congestione, da alcuni protocolli, incluso TCP come abbiamo appena discusso. Le reti wireless, però, perdono sempre pacchetti a causa di errori di trasmissione.

Con la legge di controllo AIMD, alti livelli di throughput richiedono tassi molto bassi di pacchetti persi. Analisi di Padhye et al. (1998) mostrano che il throughput aumenta come l'inverso della radice quadrata del tasso di perdita di pacchetti. Questo significa in pratica che il tasso di perdita delle connesioni TCP veloci è molto piccolo; 1% è un tasso di perdita moderato e quando raggiunge il 10% la connessione ha praticamente smesso di funzionare. Tuttavia per reti wireless come le LAN 802.11 tassi di perdita dei frame di almeno il 10% sono comuni; questa differenza significa che, senza misure di protezione, gli schemi di controllo di congestione che usano la perdita di pacchetti come segnale schiacceranno inutilmente le connessioni che viaggiano su collegamenti wireless a velocità molto basse.

Per funzionare bene, la sola perdita di pacchetti che l'algoritmo di controllo di congestione dovrebbe osservare è quella dovuta a banda insufficiente, non perdite dovute a errori di trasmissione. Una soluzione a questo problema è mascherare le perdite wireless usando ritrasmissioni sul collegamento wireless. Per esempio 802.11 usa un protocollo stop-and-wait per

consegnare ogni frame, riprovando le trasmissioni molte volte se necessario, prima di comunicare la perdita del pacchetto al livello superiore. Nel caso normale ogni pacchetto viene consegnato nonostante gli errori di trasmissione transitori invisibili ai livelli superiori.

La Figura 6.26 mostra un percorso con una linea cablata e una wireless per cui è utilizzata la strategia di mascheramento. Ci sono due aspetti da notare. Primo, il mittente non sa necessariamente che il percorso include una linea wireless, poiché tutto ciò che vede è la linea cablata a cui è attaccato. I percorsi su Internet sono eterogenei e non c'è un metodo generale che permetta al mittente di capire che tipi di linee comprenda il percorso. Questo complica il problema di controllo della congestione, poiché non c'è un modo facile per usare un protocollo per le linee wireless e un altro protocollo per quelle cablate.

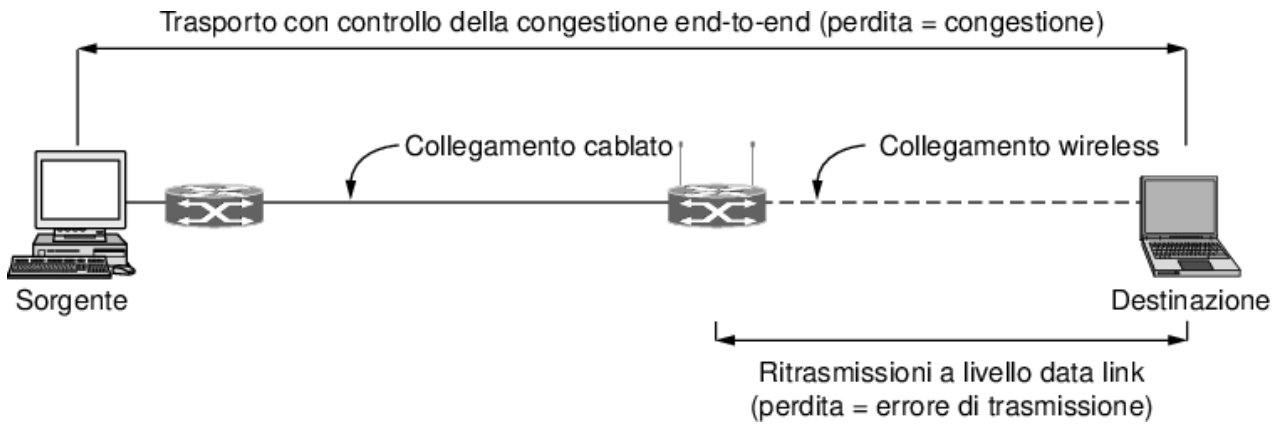


Figura6.26 Controllo di congestione su un percorso con un collegamento wireless.

Il secondo aspetto è un enigma. La figura mostra due meccanismi guidati dalla perdita: la ritrasmisione del frame da parte del livello data link e il controllo di congestione da parte del livello di trasporto. L'enigma è come questi due meccanismi possano coesistere senza confondersi. Dopotutto una perdita dovrebbe provocare la reazione di un solo meccanismo, perché sarebbe o un errore di trasmissione o un segnale di congestione, ma non entrambi. Se agissero entrambi i meccanismi (ritrasmettendo il frame e rallentando il tasso di spedizione) saremmo tornati al problema iniziale di protocolli di trasporto che viaggiano troppo lentamente su linee wireless. Soffermatevi su questo enigma per un momento e vedete se riuscite a risolverlo.

La soluzione è che i meccanismi agiscano a differenti scale temporali. Le ritrasmissioni del livello data link accadono tra microsecondi e millisecondi per le linee wireless come 802.11, mentre nei protocolli di trasporto i timer di perdita si attivano tra i millisecondi e i secondi; la differenza è di tre ordini di grandezza. Ciò permette ai collegamenti wireless di individuare le perdite di frame e ritrasmettere i frame per riparare gli errori di trasmissione molto prima che la perdita di pacchetti sia rilevata dall'entità di trasporto.

La strategia di mascheramento è sufficiente per permettere a molti protocolli di trasporto di funzionare bene sulla maggior parte delle linee wireless. Tuttavia non è sempre una soluzione adatta: alcuni collegamenti wireless come i satelliti hanno round-trip time molto lunghi. Per queste linee devono essere usate altre tecniche per mascherare le perdite, come **FEC** (*forward error correction*); altrimenti il protocollo di trasporto deve usare una segnalazione di non-perdita per il controllo della congestione.

Un secondo problema del controllo della congestione sui collegamenti wireless è rappresentato dalla loro capacità variabile: la capacità di una linea wireless cambia nel tempo, a volte bruscamente, quando si muovono i nodi e il rapporto segnale-rumore (SNR) varia col cambiare delle condizioni del canale. Nei collegamenti cablati invece la capacità è fissa. Il protocollo di trasporto si deve adattare alla mutevole capacità delle linee wireless, altrimenti potrebbe congestionare la rete o non riuscire a utilizzare la capacità disponibile.

Una possibile soluzione a questo problema è semplicemente di non preoccuparsene. Questa strategia è praticabile in quanto gli algoritmi di controllo della congestione devono già gestire i casi di nuovi utenti che entrano in rete o di utenti esistenti che cambiano i loro tassi di spedizione. Anche se la capacità dei collegamenti cablati è fissa, un diverso comportamento di altri utenti si presenta come una variabilità della banda disponibile a un dato utente. Così è possibile semplicemente utilizzare TCP su un percorso con una linea wireless 802.11 e ottenere prestazioni ragionevoli.

Tuttavia, quando c'è molta variabilità wireless, i protocolli di trasporto pensati per linee cablate possono avere problemi a tenere il passo e quindi ottenere basse prestazioni. La soluzione in questo caso è un protocollo di trasporto progettato per collegamenti wireless. Un ambiente che presenta sfide particolarmente forti è quello delle reti di tipo mesh (*mesh network*) in cui più linee wireless interferenti tra loro devono essere attraversate, i percorsi cambiano a causa della mobilità e ci sono molte perdite di dati. La ricerca in quest'area sta andando avanti. Potete consultare Li et al. (2009) per un esempio di architettura di protocollo di trasporto wireless.

6.4 Protocolli di trasporto di Internet: UDP

Nel livello di trasporto Internet possiede due protocolli principali: un protocollo non orientato alla connessione e uno orientato alla connessione. I protocolli si completano l'un l'altro. Il protocollo non orientato alla connessione è UDP; non fa praticamente altro che spedire pacchetti tra le applicazioni, permettendo loro di costruirci sopra i propri protocolli come necessario. Il protocollo orientato alla connessione è TCP. Fa praticamente tutto: crea connessioni e aumenta l'affidabilità con le ritrasmissioni, implementa anche il controllo di flusso e il controllo di congestione, tutto per conto delle applicazioni che lo usano.

Nei paragrafi seguenti studieremo UDP e TCP. Inizieremo con UDP perché è più semplice. Vedremo anche due diversi usi di UDP perché UDP è un protocollo del livello di trasporto eseguito tipicamente all'interno del sistema operativo e i protocolli che usano UDP vengono eseguiti a livello utente. Questi utilizzi possono essere considerati applicazioni. Tuttavia le tecniche che usano sono utili per molte applicazioni ed è meglio considerarle come appartenenti più a un servizio di trasporto, per cui ce ne occuperemo qui.

6.4.1 Introduzione a UDP

La suite di protocolli di Internet supporta un protocollo di trasporto non orientato alla connessione chiamato **UDP** (*user datagram protocol*, protocollo per datagrammi utente), descritto nell'RFC 768. UDP offre alle applicazioni un modo per inviare datagrammi senza dover stabilire una connessione.

UDP trasmette **segmenti** costituiti da un'intestazione di 8 byte seguita dal payload. L'intestazione è mostrata nella Figura 6.27. Le due **porte** servono per identificare gli endpoint all'interno dei computer di sorgente e destinazione. Quando arriva un pacchetto UDP, il suo payload è consegnato al processo associato alla porta di destinazione. Questa associazione avviene con l'utilizzo della primitiva BIND o qualcosa di simile, come abbiamo osservato nella Figura 6.6 per TCP (il processo di associazione è lo stesso per UDP). Pensate alle porte come a delle caselle postali che le applicazioni possono noleggiare per ricevere pacchetti. Avremo molto di più da dire quando descriveremo TCP, visto che anch'esso fa uso di porte. In effetti il vantaggio principale dell'utilizzo di UDP rispetto a IP è l'aggiunta delle porte di sorgente e destinazione. Senza i campi per le porte, il livello di trasporto non saprebbe che cosa farsene dei pacchetti in arrivo. Grazie a tali campi, invece, esso consegna il segmento all'interno del frame alla corretta applicazione.

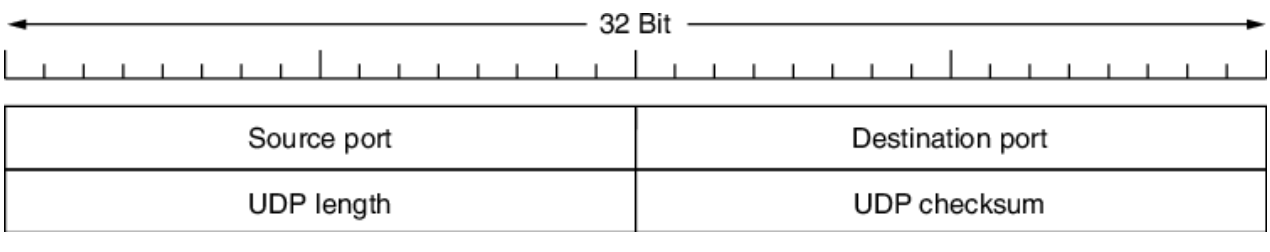


Figura 6.27 L'intestazione di UDP.

La porta sorgente serve principalmente quando si deve inviare una risposta al mittente. Copiando dal segmento in ingresso il campo *Source port* (porta sorgente) nel campo *Destination port* (porta destinazione) del segmento in uscita, il processo che invia la risposta può specificare il processo sulla macchina sorgente che deve riceverla.

Il campo *UDP length* (lunghezza UDP) include l'intestazione di 8 byte e tutti i dati. La lunghezza minima, per comprendere l'intestazione, è di 8 byte. La lunghezza massima è di 65.515 byte, che è inferiore al numero più grande esprimibile con 16 bit a causa del limite di dimensione imposto sui pacchetti IP.

È fornito anche un campo *Checksum* opzionale per migliorare l'affidabilità. Verifica la somma dell'intestazione, dei dati e di una pseudointestazione concettuale IP.[2] Quando viene fatta questa somma, il campo *Checksum* è impostato a zero e il campo dati viene prolungato con un ulteriore byte a zero se il numero di byte della sua lunghezza è dispari. L'algoritmo per il calcolo del checksum prevede semplicemente di sommare tutti i gruppi di 16 bit in complemento a uno e prenderne il risultato. Di conseguenza, quando un destinatario fa il calcolo sull'intero segmento includendo il campo *Checksum*, il risultato dovrebbe essere 0. Se il checksum non è calcolato, è salvato come uno 0, perché per una felice coincidenza dell'aritmetica in complemento a uno un risultato calcolato che ha valore 0 viene memorizzato come tutti bit a 1. Tuttavia non calcolarlo è stupido, a meno che la qualità dei dati sia poco importante (come per la voce digitalizzata).
La pseudointestazione per il caso di IPv4 è mostrata nella Figura 6.28. Contiene l'indirizzo IPv4 di 32 bit delle macchine sorgente e destinazione, il numero di protocollo per UDP (17) e il numero di byte del segmento UDP (inclusa l'intestazione). È diverso ma analogo a IPv6. Includere la pseudointestazione di UDP nel calcolo del checksum aiuta a trovare i pacchetti consegnati in modo scorretto, ma includendola si viola anche la gerarchia dei protocolli, in quanto l'indirizzo IP al suo interno appartiene al livello IP e non al livello UDP. Anche TCP usa la stessa pseudointestazione per il suo checksum.

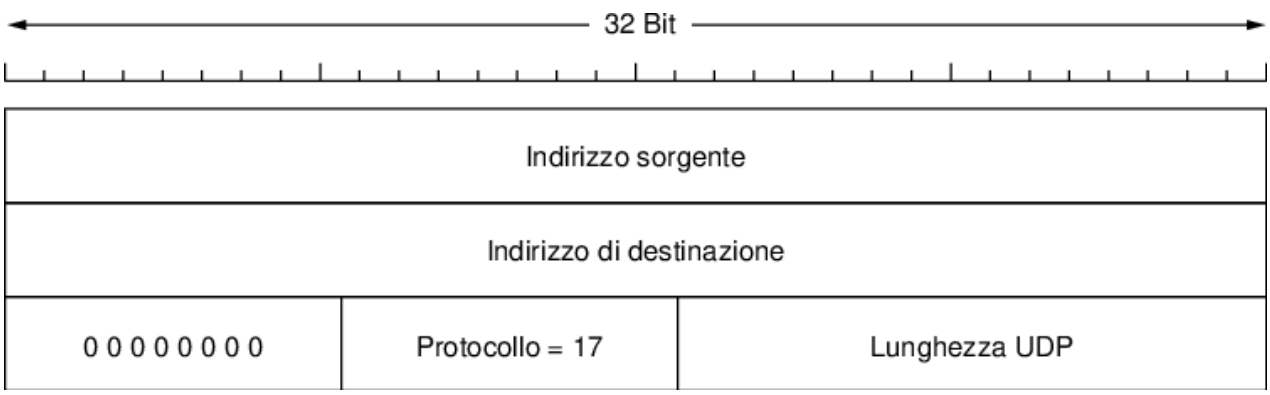


Figura 6.28 La pseudointestazione IPv4 inclusa nel checksum UDP.

Vale probabilmente la pena menzionare esplicitamente alcune delle cose che UDP *non* può fare. Il protocollo non si occupa del controllo di flusso, del controllo di congestione o della ritrasmissione dopo la ricezione di un segmento errato; questi compiti sono lasciati ai processi utente. UDP si occupa invece di fornire un'interfaccia al protocollo IP, con la caratteristica aggiunta del demultiplexing di più processi utilizzando le porte e opzionalmente del rilevamento di errori end-to-end; questo è tutto ciò che fa.

Per le applicazioni che necessitano di esercitare un controllo preciso sul flusso di pacchetti, il controllo degli errori o la temporizzazione, UDP fornisce solo ciò che ha ordinato il dottore. Un'area in cui è particolarmente utile è in alcune comunicazioni client-server. Spesso il client spedisce una richiesta breve al server e si aspetta di ritorno una risposta breve. Se la richiesta o la risposta vengono perse, il client può solo aspettare lo scadere di un timer e provare di nuovo. Non solo il codice è più semplice da scrivere, ma sono richiesti meno messaggi (uno in ogni direzione) rispetto a protocolli che richiedono un setup iniziale, come TCP.

Un'applicazione che utilizza UDP in questo modo è **DNS** (*domain name system*), che studieremo nel Capitolo 7. In breve un programma che deve cercare l'indirizzo IP corrispondente a un host, per esempio *www.cs.berkeley.edu*, può inviare un pacchetto UDP contenente il nome dell'host a un server DNS. Il server risponde con un pacchetto UDP contenente l'indirizzo IP dell'host. Non è necessaria una preparazione della connessione e nemmeno un successivo rilascio; sono sufficienti due messaggi trasmessi sulla rete.

6.4.2 Remote procedure call

In un certo senso, l'invio di un messaggio a un host remoto e la ricezione di una risposta corrispondono all'esecuzione di una chiamata a una funzione in un linguaggio di programmazione. In entrambi i casi si inizia con uno o più parametri, ottenendo alla fine un risultato. Questa osservazione ha portato al tentativo di organizzare le interazioni richiesta/risposta sulle reti per implementarle come chiamate a procedure. Tale disposizione facilita la programmazione e la gestione delle applicazioni di rete. Si immagini per esempio una procedura denominata *get_IP_address (host_name)* che funziona tramite l'invio di un pacchetto UDP a un server DNS, l'attesa di una risposta, il timeout e un nuovo tentativo se la risposta non arriva abbastanza rapidamente. In questo modo tutti i dettagli della rete possono essere nascosti al programmatore.

Un lavoro fondamentale in questa area è stato svolto da Birrell e Nelson (1984). In breve, Birrell e Nelson hanno suggerito di consentire ai programmi di chiamare delle procedure situate su host remoti. Quando un processo sulla macchina 1 chiama una procedura sulla macchina 2, il processo chiamante su 1 viene sospeso e l'esecuzione della procedura chiamata avviene su 2. Le informazioni possono essere trasportate dal chiamante al chiamato nei parametri e possono tornare indietro nel risultato della procedura. Il passaggio dei messaggi non è visibile al programmatore. Questa tecnica è nota come **RPC** (*remote procedure call*, chiamata a procedura remota) ed è diventata la base per molte applicazioni di rete. Tradizionalmente la procedura chiamante è nota come client, mentre la procedura chiamata è nota come server; anche noi utilizzeremo questa terminologia.

L'idea alla base di RPC è eseguire una chiamata a procedura remota in maniera più simile possibile a una locale. Nella forma più semplice, per chiamare una procedura remota, il programma client deve essere associato a una piccola procedura di libreria, chiamata **client stub**, che rappresenta la procedura del server nello spazio di indirizzamento del client. Allo stesso modo il server è associato a una procedura chiamata **server stub**. Queste procedure nascondono il fatto che la chiamata a procedura dal client al server non sia locale.

I passaggi effettivi per creare una RPC sono mostrati nella Figura 6.29. Il passaggio 1 rappresenta il client che chiama il suo stub. Questa è una chiamata a procedura locale, con i parametri inseriti nello stack secondo il metodo usuale. Il passaggio 2 rappresenta il client stub che inserisce i parametri in un messaggio ed effettua una chiamata di sistema per inviare il messaggio. L'inserimento dei parametri nel pacchetto prende il nome di **marshaling**. Il passaggio 3 mostra il kernel che invia il messaggio dalla macchina client alla macchina server. Il passaggio 4 rappresenta il kernel che passa il pacchetto in ingresso al server stub. Per finire, il passaggio 5 mostra il server stub che chiama la procedura del server con i parametri decodificati. La risposta segue lo stesso percorso nell'altra direzione.

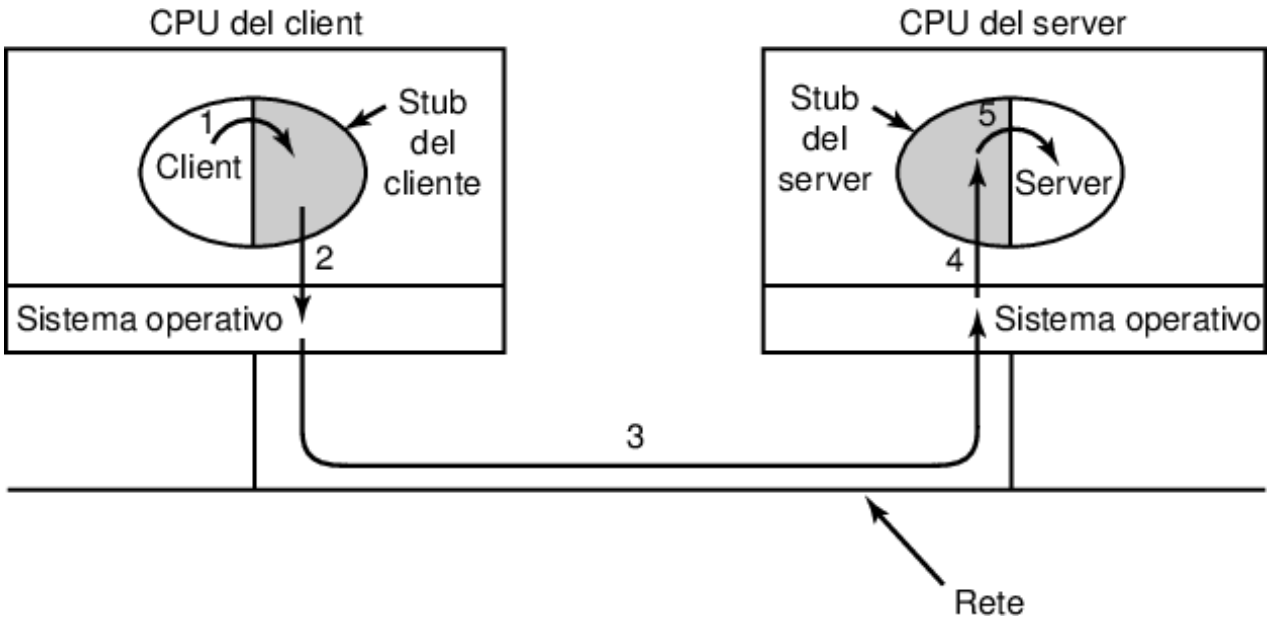


Figura6.29 I passaggi per effettuare una chiamata a procedura remota. Gli stub sono ombreggiati.

L'elemento chiave da osservare è che la procedura del client, scritta dall'utente, esegue una chiamata a procedura normale (quindi locale) al client stub, che possiede lo stesso nome della procedura del server. Dal momento che la procedura e il client stub si trovano nello stesso spazio di indirizzamento, i parametri vengono passati nel modo solito. Allo stesso modo la procedura del server viene chiamata da una procedura nello spazio di indirizzamento locale con i parametri previsti. Per la procedura del server niente è insolito. In questo modo, al posto dell'I/O tramite socket, la comunicazione di rete viene svolta mimando una normale chiamata a procedura.

Nonostante l'eleganza concettuale di RPC, vi sono pericoli in agguato. Uno dei principali riguarda l'utilizzo dei parametri puntatore. Normalmente il passaggio di un puntatore a una procedura non è un problema. La procedura chiamata può utilizzare il puntatore in modo simile alla chiamante, perché entrambe risiedono nello stesso spazio di indirizzamento virtuale. Con RPC il passaggio dei puntatori è impossibile perché il client e il server si trovano in spazi di indirizzamento diversi.

In alcuni casi è possibile sfruttare qualche trucco per consentire l'uso di puntatori. Supponiamo che il primo parametro sia un puntatore a un intero, *k*. Il client stub può eseguire il marshaling di *k* e inviarlo al server. Il server stub crea poi un puntatore a *k* e lo passa alla procedura del server, come previsto. Quando la procedura del server restituisce il controllo al server stub, quest'ultimo invia *k* al client dove il nuovo *k* viene copiato sopra il vecchio nel caso il server lo abbia modificato. In effetti la chiamata standard per riferimento è stata sostituita da una chiamata con copia e ripristino. Sfortunatamente questo trucco non sempre funziona: per esempio fallisce quando il puntatore fa riferimento a un grafo o a un'altra struttura dati complessa. Per questo motivo, come vedremo, occorre porre alcune limitazioni sui parametri delle procedure chiamate in remoto.

Un secondo problema riguarda il fatto che nei linguaggi con tipizzazione debole come C è perfettamente legale scrivere una procedura che elabora il prodotto interno di due vettori (array) senza specificarne la grandezza. Entrambi potrebbero essere terminati da un valore speciale noto solo alle procedure chiamante e chiamata. In queste circostanze è praticamente impossibile pretendere che il client stub esegua il marshaling dei parametri: non c'è infatti modo di determinarne la loro dimensione.

Un terzo problema risiede nel fatto che non è sempre possibile dedurre i tipi dei parametri, nemmeno da una specifica formale o dal codice stesso. Un esempio riguarda *printf*, che può avere qualsiasi numero di parametri (almeno uno): inoltre, i parametri possono essere un misto arbitrario di interi (di varie dimensioni), caratteri, stringhe, numeri in virgola mobile di varie lunghezze o altri tipi. Sarebbe praticamente impossibile chiamare *printf* come procedura remota, vista la permissività del C. Tuttavia decidere che RPC si può usare solo se non si programma in C (o C++) non riscuoterebbe il favore di molti programmatori.

Un quarto problema riguarda l'utilizzo delle variabili globali. Normalmente le procedure chiamante e chiamata possono comunicare tramite variabili globali oltre che mediante i parametri. Se la procedura chiamata viene spostata su una macchina remota, il codice fallisce perché le variabili globali non sono più condivise.

Questi problemi non intendevano suggerire che RPC sia senza speranza. Infatti è ampiamente usato, ma sono necessarie alcune restrizioni per farlo funzionare bene nella pratica.

In termini di protocolli di livello di trasporto, UDP è una buona base su cui implementare RPC. Sia le richieste che le risposte possono essere spedite come un solo pacchetto UDP nel caso più semplice e l'operazione può essere veloce. Tuttavia un'implementazione dovrà includere anche altri meccanismi; poiché la richiesta o la risposta possono andare perse, il client deve tenere un timer per ritrasmettere la richiesta. Notate che una risposta funziona come un acknowledgement implicito per una richiesta, che non ha bisogno di un riscontro separato. A volte i parametri o i risultati possono essere più grandi della dimensione massima dei pacchetti UDP, nel qual caso è

necessario un protocollo per consegnare grossi messaggi. Se più richieste e risposte possono sovrapporsi (come nel caso della progammazione concorrente), è necessario un identificativo per associare richiesta e risposta.

Una preoccupazione di livello superiore è che l'operazione possa non essere idempotente (cioè, che possa essere ripetuta). Il caso semplice è rappresentato da operazioni idempotenti come richieste e risposte DNS. Il client può ritrasmettere tranquillamente le richieste più e più volte se non tornano indietro risposte. Non importa se il server non ha mai ricevuto la richiesta o se la risposta è andata persa; la risposta, quando finalmente arriverà, sarà la stessa (assumendo che il database del DNS non sia stato aggiornato nel frattempo). Tuttavia non tutte le operazioni sono idempotenti, perché per esempio potrebbero avere importanti effetti collaterali come incrementare un contatore. L'uso di RPC per queste operazioni richiede semantiche più forti, in maniera tale che quando un programmatore richiama una procedura non venga eseguita più volte. In questo caso può essere necessario impostare una connessione TCP e spedire la richiesta su di essa invece che usare UDP.

6.4.3 Protocolli di trasporto real-time

La comunicazione RPC client/server è una delle aree dove UDP è molto diffuso; un'altra riguarda le applicazioni multimediali real-time. In particolare, man mano che radio e telefonia su Internet, musica e video-on-demand, videoconferenze e altre applicazioni multimediali si diffondevano, la gente scopriva che ogni applicazione reinventava più o meno lo stesso protocollo di trasporto real-time. Gradualmente si è capito che l'utilizzo di un protocollo di trasporto generico real-time per più applicazioni sarebbe stato una buona idea.

Nacque così **RTP** (*real-time transport protocol*).[3] È descritto nell'RFC 3350 e ormai è ampiamente utilizzato per le applicazioni multimediali. Descriveremo due aspetti del trasporto real-time. Il primo è il protocollo RTP per il trasporto di dati audio e video. Il secondo è l'elaborazione che viene eseguita, principalmente dal ricevente, per riprodurre l'audio e il video nel momento giusto. Queste funzioni si inseriscono nello stack di protocolli come illustrato nella Figura 6.30.

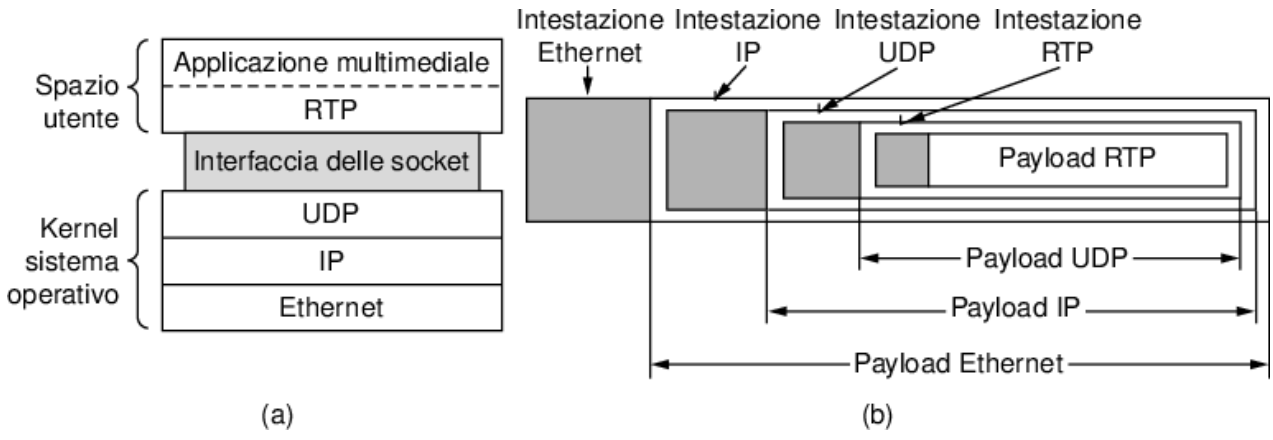


Figura6.30 (a) La posizione di RTP nello stack dei protocolli. (b) La nidificazione dei pacchetti.

RTP viene normalmente eseguito nello spazio utente appoggiandosi su UDP (nel sistema operativo). Un'applicazione multimediale è composta di più flussi audio, video, di testo e forse di altro tipo. Questi flussi vengono passati alla libreria RTP che si trova nello spazio utente insieme all'applicazione; la libreria esegue il multiplexing dei flussi e li codifica in pacchetti RTP, che vengono poi inseriti in una socket. Nella parte di kernel che si occupa della socket vengono generati pacchetti UDP che incapsulano quelli RTP, che vengono poi passati al livello IP per essere trasmessi. Il processo inverso ha luogo al ricevente e l'applicazione multimediale riceve i dati dalla libreria RTP e si occupa di rappresentarli. La pila di protocolli per questa situazione è mostrata nella Figura 6.30(a); la nidificazione dei pacchetti nella Figura 6.30(b).

In conseguenza di questa struttura è difficile affermare in quale livello si trovi RTP. Dal momento che viene eseguito nello spazio dell'utente ed è collegato al programma applicativo, certamente è simile a un protocollo del livello applicazione. D'altra parte si tratta di un protocollo generico indipendente dall'applicazione e che fornisce funzionalità di trasporto; pertanto somiglia anche a un protocollo di trasporto. Probabilmente la descrizione migliore è che si tratta di un protocollo di trasporto implementato nel livello applicazione e questo è il motivo per cui lo stiamo trattando in questo capitolo.

RTP – real-time transport protocol

La funzione base di RTP è eseguire il multiplexing di flussi di dati real-time in un singolo flusso di pacchetti UDP. Il flusso UDP può essere inviato a una singola destinazione (unicasting) o a più destinazioni (multicasting). Dal momento che RTP utilizza il normale UDP, i suoi pacchetti non sono trattati in modo speciale dai router, a meno che siano attivate alcune delle normali funzionalità per la qualità del servizio di IP. In particolare non vi sono garanzie speciali sulla consegna e i pacchetti possono essere persi, avere ritardi o essere danneggiati.

Il formato RTP contiene molte caratteristiche che aiutano i destinatari a lavorare con informazioni multimediali. A ogni pacchetto inviato in un flusso è assegnato un numero incrementato di 1 rispetto al suo predecessore; la numerazione serve alla destinazione per scoprire se mancano pacchetti. In assenza di un pacchetto, l'azione migliore che la destinazione può compiere dipende dall'applicazione: potrebbe essere quella di saltare un fotogramma se i pacchetti trasportano dati video o di approssimare il valore mancante per interpolazione in caso di dati audio. La ritrasmissione non è un'opzione praticabile, perché il pacchetto ritrasmesso arriverebbe probabilmente troppo tardi per essere utile. Come conseguenza, RTP non fa uso di acknowledgement o di meccanismi per richiedere la ritrasmissione.

Ogni payload RTP può contenere più campioni, codificati nel modo desiderato dall'applicazione. Per consentire la cooperazione, RTP definisce diversi profili (per esempio un singolo flusso audio) e può consentire l'utilizzo di più formati di codifica per ogni profilo. Per esempio un singolo flusso audio potrebbe essere codificato in campioni PCM di 8 bit a 8 kHz, oppure con codifica delta, predittiva, GSM, MP3 e così via. RTP offre un campo intestazione in cui la sorgente può specificare la codifica, ma non è coinvolto nell'esecuzione della codifica.

Un'altra caratteristica necessaria a molte applicazioni real-time è l'aggiunta di un contrassegno temporale (timestamp). L'idea sta nel consentire alla sorgente di associare un timestamp al primo campione di ogni pacchetto. I timestamp sono relativi all'inizio del flusso, per cui sono significative

solo le differenze tra essi e non i loro valori assoluti. Come vedremo tra poco, questo meccanismo consente alla destinazione di gestire un piccolo buffer e di riprodurre ogni campione nell'esatto istante richiesto, indipendentemente dall'istante di arrivo del pacchetto contenente il campione.

Non solo l'uso dei timestamp riduce gli effetti della variazione del ritardo di rete, ma permette anche a più flussi di essere sincronizzati tra loro. Per esempio un programma televisivo digitale potrebbe avere un flusso video e due flussi audio; i due flussi audio potrebbero essere l'audio stereofonico del programma oppure le colonne sonore in lingua originale e doppiate di un film per dare allo spettatore una possibilità di scelta. Ogni flusso proviene da un dispositivo fisico diverso ma, se possiede un timestamp generato dallo stesso contatore centralizzato, può essere riprodotto in modo sincrono anche quando i flussi vengono trasmessi e/o ricevuti in modo sconnesso.

L'intestazione RTP è illustrata nella Figura 6.31. Consiste di tre gruppi di 32 bit e di alcune estensioni facoltative. Il primo gruppo contiene il campo *Version*, che attualmente corrisponde a 2. Speriamo che questa versione sia molto vicina a quella definitiva, visto che resta un solo numero di versione a disposizione (anche se in futuro si potrebbe decidere che il numero 3 vuol dire che il numero di versione è in uno dei gruppi di estensione).

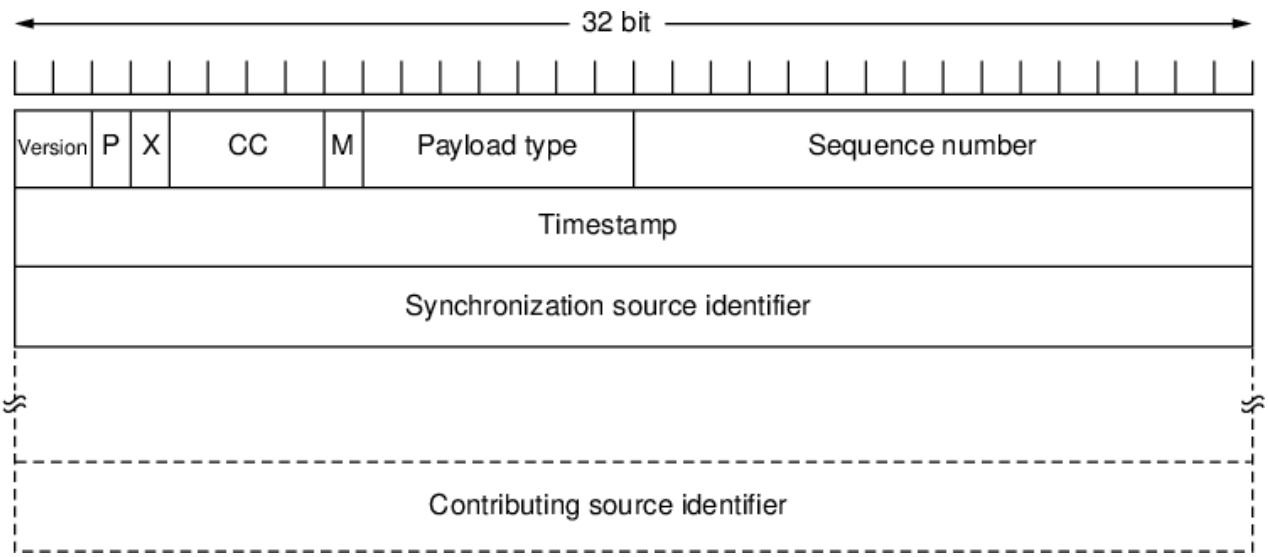


Figura6.31 L'intestazione RTP.

Il bit *P* indica che il pacchetto è stato riempito fino a ottenere un multiplo di 4 byte. L'ultimo byte del riempimento indica quanti byte sono stati aggiunti. Il bit *X* indica che è presente un'intestazione estesa. Il formato e il significato dell'intestazione estesa non sono definiti; l'unica regola è che il primo gruppo dell'estensione indica la lunghezza. Si tratta di una via di fuga per qualsiasi requisito non ancora previsto.

Il campo *CC* indica il numero di sorgenti attive presenti, da 0 a 15 (si veda di seguito). Il bit *M* è un bit di contrassegno specifico dell'applicazione. Può essere utilizzato per contrassegnare l'inizio di un fotogramma per il video, l'inizio di una parola in un canale audio o qualcos'altro che

l'applicazione sia in grado di comprendere. Il campo *Payload type* (tipo di payload) indica quale algoritmo di codifica è stato utilizzato (per esempio audio a 8 bit non compresso, MP3 e così via). Dal momento che ogni pacchetto contiene questo campo, la codifica può cambiare durante la trasmissione. *Sequence number* (numero di sequenza) è un contatore incrementato a ogni invio di un pacchetto RTP; serve a rilevare i pacchetti persi.

Timestamp è prodotto da chi genera il flusso per indicare quando è stato creato il primo campione nel pacchetto. Questo valore può aiutare a ridurre la variabilità della temporizzazione, disaccoppiando l'istante di riproduzione da quello di arrivo del pacchetto. Il campo *Synchronization source identifier* (identificatore di sincronizzazione di flusso) specifica a quale flusso appartiene il pacchetto e rappresenta l'informazione utilizzata per il multiplexing e il demultiplexing di più flussi dati in un singolo flusso di pacchetti UDP. Per finire, i campi facoltativi *Contributing source identifier* (identificatore di sorgente attiva) sono utilizzati da apparecchiature di studio quali i mixer: il mixer è la sorgente della sincronizzazione e l'elenco dei flussi combinati si trova qui.

RTCP – real-time transport control protocol

RTP si accompagna a un procotollo chiamato **RTCP** (*real-time transport control protocol*). È definito insieme a RTP nell'RFC 3550 e gestisce le retroazioni verso la sorgente, la sincronizzazione e l'interfaccia utente. Non trasporta campioni multimediali.

La prima funzione può essere utilizzata per fornire alla sorgente del flusso una retroazione (*feedback*) su ritardi, jitter, larghezza di banda, congestione e altre proprietà di rete. Queste informazioni possono servire al processo di codifica per aumentare la velocità dei dati (e fornire una qualità superiore) quando la rete funziona bene e per ridurla quando vi sono dei problemi. Fornendo un feedback continuo, gli algoritmi di codifica si possono adattare costantemente per fornire la migliore qualità permessa dalle circostanze. Per esempio, se la larghezza di banda aumenta o diminuisce durante la trasmissione, la codifica può passare da MP3 a PCM 8 bit, fino alla codifica delta secondo necessità. Il campo *Payload type* viene utilizzato per comunicare alla destinazione l'algoritmo di codifica utilizzato per il pacchetto corrente, in modo che sia possibile variarlo su richiesta. Un problema nel fornire feedback è che i rapporti RTCP vengono spediti a tutti i partecipanti. Per un'applicazione multicast con un folto gruppo di partecipanti, la quantità di banda utilizzata da RTCP potrebbe aumentare velocemente. Per prevenirlo, i mittenti RTCP diminuiscono il tasso dei loro rapporti per consumare collettivamente non più di, più o meno, il 5% della banda utilizzata per i dati multimediali. Per farlo, ogni partecipante ha bisogno di conoscere la banda utilizzata che gli viene data dal mittente e il numero dei partecipanti che stima in base agli altri rapporti RTCP.

RTCP gestisce anche la sincronizzazione tra flussi. Il problema è che flussi diversi possono utilizzare clock differenti, con granularità e velocità di deriva diverse. RTCP può essere utilizzato per mantenere la sincronia.

Infine, RTCP fornisce un metodo per denominare le sorgenti (per esempio con testo ASCII). Queste informazioni possono essere visualizzate sullo schermo del destinatario per indicare chi sta parlando al momento.

Ulteriori informazioni su RTP sono disponibili in (Perkins, 2003).

Riproduzione con buffering e controllo del jitter

Una volta che le informazioni multimediali raggiungono il destinatario dovranno essere riprodotte nel momento giusto. In generale non sarà il momento in cui i pacchetti RTP arrivano al destinatario, perché i pacchetti impiegheranno degli intervalli di tempo leggermente diversi per attraversare la rete. Anche se i pacchetti sono spediti con esattamente i giusti intervalli dal mittente, raggiungeranno il destinatario con dei diversi tempi relativi. Questa variazione nel ritardo è chiamata **jitter**. Anche una piccola quantità jitter nei pacchetti può creare fastidiosi artefatti multimediali, come video a scatti e audio incomprensibile.

La soluzione per ridurre il jitter è fare uso di un **buffer** per i pacchetti al destinatario prima che siano riprodotti. Come esempio, nella Figura 6.32 vediamo un flusso di pacchetti consegnati con un sostanziale valore di jitter. Il pacchetto 1 è spedito dal server al tempo $t = 0$ secondi e arriva al client a $t = 1$ secondi. Il pacchetto 2 subisce più ritardo e impiega 2 secondi ad arrivare. Quando arrivano, i pacchetti sono memorizzati in un buffer sulla macchina client.

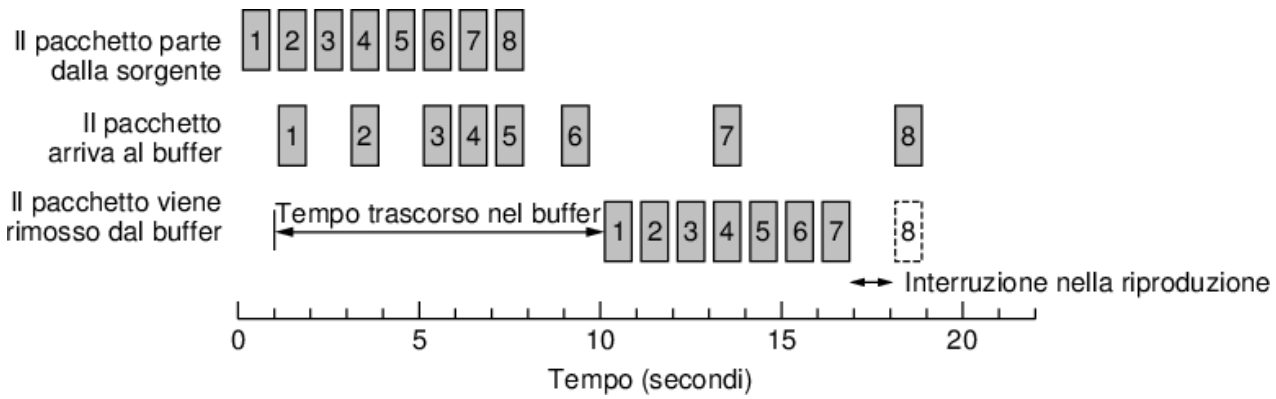


Figura6.32 Regolarizzazione del flusso in uscita tramite buffer.

A $t = 10$ secondi inizia la riproduzione. In questo momento i pacchetti da 1 a 6 sono nel buffer, quindi possono essere tolti da lì a intervalli uniformi per una riproduzione fluida. Nel caso generale non è necessario usare intervalli uniformi, perché i timestamp RTP indicano quando il contenuto multimediale dovrà essere riprodotto.

Sfortunatamente possiamo constatare che il pacchetto 8 è in ritardo, quindi non è disponibile quando arriva il suo turno di riproduzione. Ci sono due opzioni: il pacchetto 8 può essere tralasciato e il riproduttore procede verso i pacchetti successivi oppure il playback può fermarsi finché non arriva il pacchetto 8, creando un fastidioso salto nella musica o nel film. In un'applicazione multimediale interattiva come una chiamata vocale su IP (*voice-over-IP*, *VoIP*) normalmente il pacchetto verrà tralasciato; le applicazioni interattive non funzionano bene se messe in attesa. In un'applicazione di streaming multimediale, invece, il riproduttore potrebbe andare in pausa. Questo problema può essere alleviato ritardando ancor di più il tempo di inizio utilizzando un buffer più grande. Per un riproduttore di streaming audio o video sono spesso usati

buffer di circa 10 secondi per assicurarsi che il riproduttore riceva tutti i pacchetti (che non sono stati persi dalla rete) in tempo. Per le applicazioni interattive come le videoconferenze sono necessari piccoli buffer per assicurare la reattività.

Una considerazione chiave per la riproduzione fluida è il **punto di playback** (*playback point*) o quanto a lungo il destinatario deve attendere i dati multimediali prima di riprodurli. Decidere quanto attendere dipende dal jitter; la differenza tra una connessione a basso jitter e una ad alto jitter è mostrata nella Figura 6.33. Il ritardo medio non deve differire molto tra i due, ma se c'è un valore di jitter alto, il punto di playback può aver bisogno di essere molto più lontano per catturare il 99% dei pacchetti, rispetto a se ci fosse un basso valore di jitter.

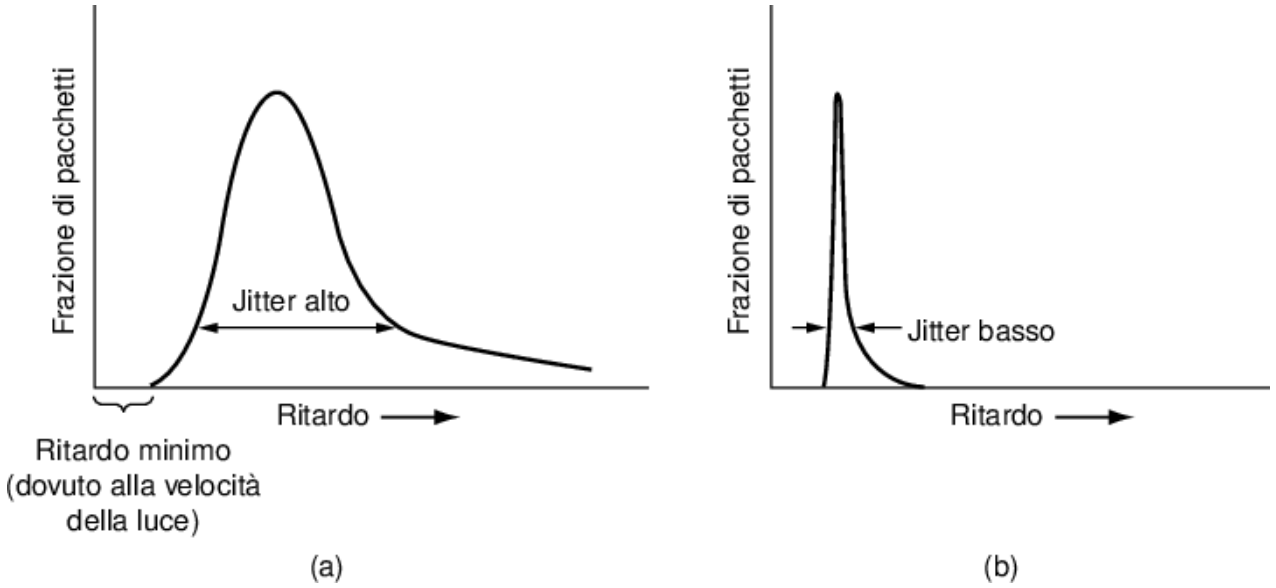


Figura6.33 (a) Jitter alto. (b) Jitter basso.

Per scegliere un buon punto di playback, l'applicazione può misurare il jitter guardando la differenza tra i timestamp RTP e il tempo di arrivo. Ogni differenza dà un campione di ritardo (più uno spiazzamento fisso arbitrario). Tuttavia il ritardo può cambiare nel tempo a causa di altro traffico in competizione e di cambi di percorso. Per accomodare questo cambiamento, le applicazioni possono adattare i loro punti di playback durante l'esecuzione. Tuttavia, se non fatto correttamente, cambiare il punto di playback può produrre un artefatto visibile all'utente. Un modo per evitare questo problema per l'audio è adattare il punto di playback nelle pause di silenzio all'interno di una conversazione. Nessuno noterà la differenza tra un breve silenzio e uno leggermente più lungo. RTP permette alle applicazioni di stabilire i bit marcatori *M* per indicare l'inizio di una nuova sequenza di suoni (**talkspurt**).

Se il ritardo assoluto prima che il contenuto multimediale venga riprodotto è troppo lungo, le applicazioni interattive ne soffriranno. Se si sta già utilizzando un percorso diretto, non può essere fatto nulla per ridurre il ritardo di propagazione. Il punto di playback può essere anticipato, semplicemente accettando che una grossa porzione di pacchetti arrivino troppo tardi per essere

riprodotti. Se questo non è accettabile, l'unico modo per anticipare il punto di playback è ridure il jitter usando una qualità del servizio migliore, quale per esempio l'inoltro accelerato nel caso di servizi differenziati. Quindi è necessaria una rete migliore.

6.5 Protocolli di trasporto di Internet: TCP

UDP è un protocollo semplice con alcuni utilizzi molto importanti, come le interazioni client-server e le trasmissioni multimediali, ma per la maggior parte delle applicazioni Internet è necessaria una consegna affidabile e in sequenza che UDP non può fornire, pertanto è richiesto un altro protocollo. È chiamato TCP ed è il principale motore di Internet.

6.5.1 Introduzione a TCP

TCP (*transmission control protocol*, protocollo di controllo della trasmissione) è stato progettato appositamente per fornire un flusso di byte affidabile end-to-end su una internetwork inaffidabile. Una internetwork differisce da una singola rete perché le diverse parti possono avere topologie, larghezze di banda, ritardi, dimensioni di pacchetti e altri parametri completamente differenti tra loro. TCP è stato progettato per adattarsi dinamicamente alle proprietà della internetwork e per continuare a offrire solide prestazioni in presenza di molti tipi di errore.

TCP fu definito formalmente nell'RFC 793 nel settembre del 1981. Col passare del tempo furono fatti molti miglioramenti e corretti molti errori e inconsistenze. Per darvi una misura dell'estensione di TCP, gli RFC importanti sono ora il 793 con l'aggiunta di: chiarimenti e correzioni di bug nell'RFC 1122; estensioni per alte prestazioni nell'RFC 1323; acknowledgement selettivi nell'RFC 2018; controllo di congestione nell'RFC 2581; campi dell'intestazione riproposti per qualità del servizio nell'RFC 2873; timer di ritrasmissione migliorati nell'RFC 2988 e notifiche esplicite di congestione nell'RFC 3168. La collezione completa è ancora più grande, il che porta a una guida dei molti RFC, pubblicata ovviamente come un altro documento: l'RFC 4614.

Ogni computer che supporta TCP dispone di un'entità di trasporto TCP, che può essere una procedura di libreria, un processo utente o una parte del kernel, ma in tutti i casi gestisce i flussi TCP e si interfaccia con il livello IP. Un'entità TCP accetta dai processi locali i flussi dati dell'utente; li suddivide in pezzi di dimensione non superiore a 64 KB (nelle applicazioni pratiche sono quasi sempre 1.460 byte di dati, in modo che stiano in un singolo frame Ethernet con le intestazioni IP e TCP); invia infine ogni pezzo in un datagramma IP autonomo. I datagrammi contenenti i dati TCP che arrivano a un computer vengono consegnati all'entità TCP che ricostruisce i flussi di byte originali. Per semplicità si utilizzerà a volte il termine generico "TCP" per fare riferimento all'entità di trasporto TCP (un componente software) oppure al protocollo TCP (un insieme di regole); dal contesto dovrebbe essere chiaro che cosa si intende. Per esempio, nell'affermazione "L'utente consegna i dati a TCP" si fa chiaramente riferimento all'entità di trasporto TCP.

Il livello IP non garantisce la consegna senza errori dei datagrammi come pure non fornisce alcuna indicazione sulla velocità con cui è possibile inviarli. È compito di TCP spedire i datagrammi abbastanza velocemente da utilizzare la capacità di trasferimento senza creare congestione e ritrasmettere tutti i datagrammi che non sono stati consegnati. I datagrammi potrebbero anche arrivare nell'ordine sbagliato; è sempre compito di TCP riassemblare i messaggi nella giusta sequenza. Riassumendo, TCP deve fornire delle buone prestazioni con l'affidabilità che la maggior parte delle applicazioni desidera e che IP non offre.

6.5.2 Il modello di servizio di TCP

Il servizio TCP è ottenuto con la creazione di punti terminali di un sistema di comunicazione da parte di mittente e ricevente, chiamati **socket**, come già visto nel Paragrafo 6.1.3. Ogni socket possiede un numero (un indirizzo) composto dall'indirizzo IP dell'host e da un numero di 16 bit locale all'host, chiamato **porta**. Una porta è il nome TCP per un TSAP. Per ottenere il servizio TCP si deve stabilire esplicitamente una connessione tra una socket su una macchina e una socket su un'altra macchina. Le chiamate alle socket sono elencate nella Figura 6.5.

Una socket può essere usata per più connessioni contemporaneamente. In altre parole, due o più connessioni possono terminare alla stessa socket. Le connessioni sono individuate dagli identificatori di socket a entrambe le estremità, vale a dire dalla coppia (*socket1, socket2*). Non vengono utilizzati numeri di circuito virtuale o altri identificatori.

I numeri di porta minori di 1024 sono riservati per servizi standard, che normalemente possono essere erogati solo da utenti privilegiati (come *root* nei sistemi UNIX). Le porte in questo sottoinsieme prendono il nome di **well-known port** (*porte ben note*). Per esempio, qualsiasi processo che desidera stabilire una connessione a un host per recuperare la mail può connettersi alla porta 143 per contattare il server (che, in gergo, prende il nome di demone, *daemon*) IMAP. L'elenco delle porte ben note si può ottenere da *www.iana.org*; ne sono state assegnate più di 700. Alcune tra le più conosciute sono elencate nella Figura 6.34.

Porta	Protocollo	Utilizzo
20, 21	FTP	Trasferimento di file
22	SSH	Login remoto, rimpiazzamento di Telnet
25	SMTP	Posta elettronica
80	HTTP	World Wide Web
110	POP-3	Accesso remoto alla posta elettronica
143	IMAP	Accesso remoto alla posta elettronica
443	HTTPS	Web sicuro (HTTP su SSL/TLS)
543	RTSP	Controllo di riproduttori multimediali
631	IPP	Condivisione di stampanti

Figura6.34 Alcune porte assegnate.

Altre porte, dalla 1024 alla 49151 possono essere registrate presso la IANA per essere utilizzate da utenti non privilegiati. Per esempio l'applicazione peer-to-peer di file-sharing BitTorrent utilizza le porte da 6881 a 6887.[4]

In fase di avvio del sistema operativo sarebbe certamente possibile associare il demone FTP alla porta 21, il demone SSH alla porta 22 e così via; tuttavia in questo modo si occupa la memoria con demoni che rimangono inattivi per la maggior parte del tempo. In genere si preferisce invece avviare un singolo demone che, per motivi storici, nei sistemi UNIX viene chiamato **inetd** (*internet daemon*), associato a più porte per attendere la prima connessione in ingresso. Quando questa si verifica, *inetd* genera un nuovo processo ed esegue il demone appropriato, consentendogli di gestire la richiesta. In questo modo i demoni diversi da *inetd* sono attivi solo quando hanno del lavoro da compiere. *Inetd* apprendeva quali porte utilizzare da un file di configurazione; di conseguenza l'amministratore di sistema poteva configurare il computer perché disponesse di demoni permanenti sulle porte più usate (per esempio la porta 80) mentre *inetd* si occupava delle altre.

Tutte le connessioni TCP sono di tipo full-duplex punto a punto. Full-duplex indica che il traffico può procedere in entrambe le direzioni contemporaneamente. Punto a punto significa che ogni connessione ha esattamente due punti terminali. TCP non supporta il multicast o il broadcast.

Una connessione TCP è un flusso di byte, non un flusso di messaggi. I confini dei messaggi non vengono conservati da un'estremità all'altra della connessione. Per esempio, se il processo mittente esegue quattro invii da 512 byte su un flusso TCP, questi dati possono essere consegnati al processo ricevente come quattro blocchi di 512 byte, due blocchi di 1024 byte, un solo blocco di 2048 byte (Figura 6.35) o in altri modi ancora. Il ricevente non può sapere le unità con cui sono stati inviati i dati.

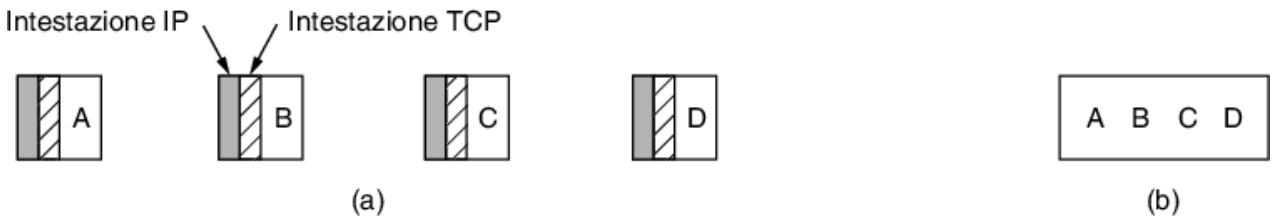


Figura 6.35 (a) Quattro segmenti di 512 byte inviati come datagrammi IP separati. (b) I 2.048 byte di dati consegnati all'applicazione in risposta a una singola chiamata READ.

Anche i file in UNIX possiedono questa proprietà: chi legge un file non può capire se è stato scritto un blocco alla volta, un byte alla volta o tutto insieme. Come con un file UNIX, il software TCP non ha idea del significato dei byte e non è interessato a trovarlo: un byte è solo un byte.

TCP, quando un'applicazione gli passa i dati a sua discrezione può inviarli immediatamente o inserirli in un buffer per raccoglierne una quantità maggiore da inviare tutta insieme. A volte può succedere che l'applicazione desideri realmente un invio immediato dei dati. Si supponga, per esempio, che l'utente di un gioco interattivo voglia mandare un flusso di aggiornamenti. È essenziale che gli aggiornamenti siano spediti immediatamente e non inseriti in un buffer. Per forzare l'uscita dei dati, TCP possiede un flag PUSH in ogni pacchetto il cui intento originale era

permettere alle applicazioni di indicare alle varie implementazioni di TCP di non ritardare la trasmissione; tuttavia le applicazioni non sono in grado di impostare il flag PUSH quando inviano i dati e vari sistemi operativi hanno sviluppato diverse opzioni per accelerare la trasmissione (come TCP_ NODELAY in Windows e Linux).

Per gli archeologi di Internet menzioneremo anche una caratteristica interessante del servizio TCP prevista nel protocollo, ma usata raramente: i **dati urgenti**. Quando un'applicazione ha dati ad alta priorità che devono essere elaborati immediatamente, come accade, per esempio, se un utente interattivo digita CTRL-C per interrompere un processo di calcolo remoto già iniziato, l'applicazione di spedizione può mettere delle informazioni di controllo nel flusso di dati e darlo a TCP con il flag URGENT. Questa azione ferma l'accumulo di dati di TCP e fa trasmettere immediatamente tutto ciò che ha per quella connessione.

Quando i dati urgenti vengono ricevuti dalla destinazione, l'applicazione ricevente viene fermata (per esempio gli viene dato un segnale, secondo la terminologia UNIX), in modo che possa interrompere ciò che stava facendo e leggere il flusso dei dati per trovare quelli urgenti. La fine dei dati urgenti è contrassegnata in modo che l'applicazione sappia dove terminano, mentre l'inizio dei dati urgenti non è contrassegnato ed è compito dell'applicazione calcolarlo.

Questo schema fornisce un meccanismo di segnalazione basilare lasciando tutto il resto all'applicazione. Tuttavia, sebbene l'opzione di contrassegnare i dati come urgenti è potenzialmente utile, nei primi tempi non trovò alcuna applicazione accattivante e cadde in disuso. Il suo uso è ora scoraggiato a causa di differenze implementative, lasciando le applicazioni a gestire le proprie segnalazioni. Forse futuri protocolli di trasporto forniranno un sistema di segnalazione migliore.

6.5.3 Il protocollo TCP

In questo paragrafo viene fornita una panoramica generale del protocollo TCP, nel successivo sarà analizzata l'intestazione del protocollo campo per campo.

Una funzionalità vitale di TCP, che guida la struttura del protocollo, consiste nel fatto che ogni byte in una connessione TCP ha un proprio numero di sequenza a 32 bit. Agli esordi di Internet le linee tra i router erano principalmente linee affittate a 56 kbps, pertanto un host che inviava ininterrottamente alla velocità massima impiegava circa una settimana per utilizzare tutti i numeri di sequenza. Alle moderne velocità di rete, i numeri di sequenza possono essere consumati a velocità preoccupanti, come vedremo in seguito. Numeri di sequenza distinti vengono inviati per la posizione della finestra scorrevole in una direzione e per gli acknowledgement nella direzione opposta.

Le entità TCP di invio e ricezione scambiano i dati sotto forma di segmenti. Un **segmento TCP** consiste di un'intestazione fissa di 20 byte (più una parte facoltativa) seguita da zero o più byte di dati. Il software TCP decide la dimensione dei segmenti e può accumulare in un segmento i dati provenienti da più invii oppure dividere i dati di un invio in più segmenti. I limiti sulla dimensione del segmento sono due: ogni segmento, compresa l'intestazione TCP, deve essere contenuto nel payload IP di 65.515 byte e ogni collegamento ha una **MTU** (*maximum transfer unit*, unità di trasferimento massima). Ogni segmento deve essere contenuto nella MTU sia lato

mittente sia lato ricevente, in maniera tale che possa essere spedito e ricevuto in un singolo pacchetto non frammentato. In pratica, la MTU è generalmente lunga 1500 byte (la dimensione del payload di Ethernet) e ciò definisce il limite superiore per la dimensione del segmento.

Tuttavia è ancora possibile per i pacchetti IP che trasportano segmenti TCP venire frammentati quando attraversano un percorso di rete, sul quale alcuni collegamenti hanno un MTU piccolo; ciò degrada le prestazioni e causa altri problemi (Kent e Mogul, 1987). Invece le implementazioni moderne di TCP fanno uso di **path MTU discovery** (*identificazione dell'MTU sul percorso*) attraverso la tecnica delineata nell'RFC 1191 descritta nel Paragrafo 5.5.5. Questa tecnica usa i messaggi d'errore ICMP per trovare il più piccolo MTU per ogni collegamento sul percorso. TCP riduce quindi la dimensione del segmento per evitare la frammentazione.

Il protocollo di base utilizzato dalle entità TCP è il protocollo a finestra scorrevole con una dimensione dinamica della finestra. Quando un mittente trasmette un segmento, avvia anche un timer. Quando il segmento arriva a destinazione, l'entità TCP ricevente invia un segmento (con i dati, se esistono, oppure senza) contrassegnato da un numero di acknowledgement uguale al numero di sequenza successivo che prevede di ricevere la dimensione della finestra disponibile. Se il timer del mittente scade prima della ricezione dell'acknowledgement, questi ritrasmette il segmento.

Anche se questo protocollo sembra semplice, esistono pro e contro (magari poco evidenti) che saranno illustrati in seguito. I segmenti possono arrivare in ordine sbagliato; in questo caso i byte da 3072 a 4095 potrebbero arrivare ma non essere confermati, perché i byte da 2048 a 3071 non si sono ancora visti. Durante il transito i segmenti possono anche essere ritardati così tanto che il mittente va in timeout ed è costretto a ripetere la trasmissione. Le ritrasmissioni possono includere intervalli di byte diversi rispetto alla trasmissione originale; ciò costringe a un'attenta gestione per tenere traccia di quali byte sono stati ricevuti correttamente fino a quel momento, ma l'operazione può essere svolta perché ogni byte nel flusso ha uno spiazzamento (*offset*) univoco.

TCP deve essere preparato a gestire questi problemi e risolverli in modo efficiente. Una quantità considerevole di sforzi è stata spesa per l'ottimizzazione delle prestazioni dei flussi TCP, anche in presenza di problemi di rete. Nel seguito saranno discussi diversi algoritmi utilizzati da molte implementazioni di TCP.

6.5.4 Intestazione del segmento TCP

La Figura 6.36 mostra la struttura di un segmento TCP. Ogni segmento inizia con un'intestazione di 20 byte con formato fisso; l'intestazione fissa può essere seguita da alcune opzioni dell'intestazione. Dopo le opzioni seguono fino a 65.535 – 20 – 20 = 65.495 byte di dati, dove i primi 20 fanno riferimento all'intestazione IP e i secondi a quella TCP. Sono ammessi segmenti senza dati, ampiamente usati per acknowledgement e messaggi di controllo.

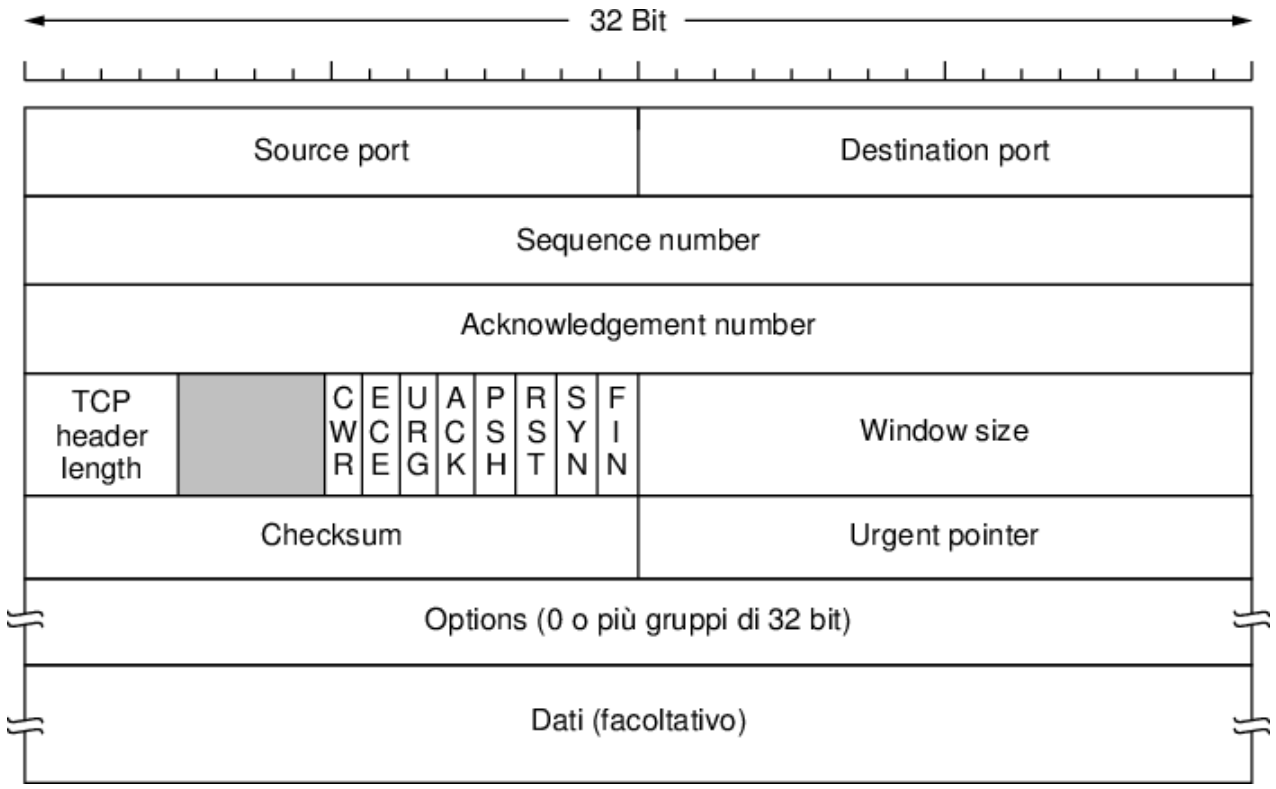


Figura6.36 L'intestazione TCP.

Analizziamo l'intestazione TCP campo per campo. I campi *Source port* (porta sorgente) e *Destination port* (porta destinazione) identificano gli estremi locali della connessione. Una porta TCP più l'indirizzo IP del suo host formano un unico punto terminale a 48 bit. I punti terminali di sorgente e destinazione insieme identificano la connessione. Questo identificatore di connessione è una quintupla perché consiste di cinque pezzi di informazione: il protocollo (TCP), l'IP e porta della sorgente e l'IP e porta della destinazione.

I campi *Sequence number* (numero di sequenza) e *Acknowledgement number* (numero di acknowledgement) svolgono le loro solite funzioni. Occorre notare che il secondo specifica il successivo byte previsto, non l'ultimo byte ricevuto correttamente; si tratta di un **acknowledgement cumulativo**, perché riassume i dati ricevuti con un solo numero e non va al di là dei dati persi. Entrambi sono numeri di 32 bit, perché ogni byte di dati è numerato un uno stream TCP.

Il campo *TCP header length* (*lunghezza intestazione TCP*) indica quanti gruppi di 32 bit sono contenuti nell'intestazione TCP. L'informazione è necessaria perché il campo *Options* (opzioni) ha una lunghezza variabile e, di conseguenza, anche l'intestazione. Tecnicamente, questo campo indica l'inizio dei dati all'interno del segmento, misurato in gruppi di 32 bit; ma questo numero rappresenta praticamente la lunghezza dell'intestazione, che è poi la stessa cosa.

Segue un campo di 4 bit inutilizzato. Il fatto che questi bit siano rimasti inutilizzati per 30 anni (e solo 2 dei 6 originali siano stati riutilizzati) testimonia quanto TCP sia ben studiato. Un protocollo studiato male li avrebbe già sfruttati per correggere bug presenti nella struttura originale.

Seguono otto flag di un bit. *CWR* e *ECE* sono usati per segnalare la congestione quando viene utilizzata **ECN** (*explicit congestion notification*), come specificato nell'RFC 3168. *ECE* è regolata per mandare un *ECN-Echo* a un mittente TCP per indicargli di rallentare quando il destinatario TCP riceve un'indicazione di congestione dalla rete. *CWR* è usato per segnalare una condizione di *congestion window reduced* (riduzione della finestra di congestione) dal mittente TCP al destinatario TCP così che esso sappia che il mittente ha rallentato e possa smettere di spedire degli *ECN-Echo*. Il ruolo di ECN nel controllo di congestione TCP viene discusso nel Paragrafo 6.5.10.

URG è impostato a 1 quando si usa *Urgent pointer* (puntatore urgente), che indica lo spiazzamento in byte (partendo dal numero di sequenza corrente) in cui si trovano i dati urgenti.[5] Questa funzionalità è utilizzata al posto dei messaggi di interrupt e, come affermato in precedenza, è un metodo rudimentale per consentire al mittente di inviare segnali al ricevente senza coinvolgere TCP nel motivo dell'interrupt, ma è usato raramente.
Il bit *ACK* è impostato a 1 per indicare che *Acknowledgement number* (numero di acknowledgement) è valido e questo è il caso della quasi totalità dei pacchetti. Se *ACK* è 0, il segmento non contiene un acknowledgement, pertanto il campo *Acknowledgement number* viene ignorato.

Il bit *PSH* segnala la presenza di dati *PUSH*. Al ricevente viene gentilmente chiesto di consegnare i dati all'applicazione all'arrivo e di non archiviarli nel buffer per poi trasmetterle un buffer completo.

Il bit *RST* viene utilizzato per reimpostare una connessione che è diventata confusa a causa di un malfunzionamento dell'host o per altre ragioni. È anche utilizzato per rifiutare un segmento non valido o un tentativo di aprire una connessione. In generale, se si riceve un segmento con il bit *RST* attivato, si è di fronte a un problema.

Il bit *SYN* viene utilizzato per stabilire le connessioni. La richiesta di connessione presenta *SYN* = 1 e *ACK* = 0 per indicare che il campo *Acknowledgement* non è utilizzato. La risposta alla richiesta di connessione porta un acknowledgement, pertanto possiede *SYN* = 1 e *ACK* = 1. In pratica il bit *SYN* è utilizzato per segnalare entrambi CONNECTION REQUEST e CONNECTION ACCEPTED, mentre il bit *ACK* distingue tra le due possibilità.

Il bit *FIN* viene utilizzato per rilasciare una connessione. Specifica che il mittente non ha altri dati da trasmettere. Tuttavia, dopo avere chiuso una connessione, il processo di chiusura potrebbe continuare a ricevere dati all'infinito. Entrambi i segmenti *SYN* e *FIN* possiedono numeri di sequenza ed è quindi garantito che verranno elaborati nell'ordine corretto.

Il controllo di flusso in TCP è gestito con una finestra scorrevole a dimensione variabile. Il campo *Window size* (dimensione finestra) indica quanti byte possono essere inviati a partire da quello che ha ricevuto acknowledgement. Un campo *Window size* con valore 0 è ammesso e afferma che i byte fino ad *Acknowledgement number* – 1 compreso sono stati ricevuti, ma che il ricevente non ha avuto modo di consumarli e non desidera altri dati per il momento. Il ricevente può in seguito dare l'autorizzazione all'invio trasmettendo un segmento con lo stesso *Acknowledgement number* e un campo *Window size* diverso da zero.

Nei protocolli del Capitolo 3 gli acknowledgement dei frame ricevuti e le autorizzazioni per inviare nuovi frame erano uniti, in conseguenza della dimensione fissa della finestra per ogni protocollo. In TCP gli acknowledgement e le autorizzazioni per inviare dati aggiuntivi sono

completamente separati. In effetti un ricevente può affermare: “Ho ricevuto i byte fino a *k* compreso ma non ne voglio altri per ora”. Questa divisione (di fatto una finestra a dimensione variabile) offre una flessibilità aggiuntiva, che studieremo in dettaglio in seguito.

Per aumentare l’affidabilità viene anche fornito un *Checksum*; questo fa un controllo dell’intestazione, dei dati e di una pseudointestazione concettuale esattamente come UDP, tranne che la speudointestazione ha un numero di protocollo per TCP (6) e il checksum è obbligatorio. Si veda il Paragrafo 6.4.1 per dettagli.

Il campo *Options* (*opzioni*) fornisce un modo per aggiungere funzionalità aggiuntive non previste dall’intestazione standard. Sono state definite molte opzioni e diverse di queste sono utilizzate comunemente. Le opzioni sono di lunghezza variabile, riempiono un multiplo di 32 bit occupando lo spazio extra con degli zeri e possono estendersi a 40 byte per accogliere la più lunga intestazione TCP che possa essere specificata. Alcune opzioni sono aggiunte quando viene stabilita una connessione per effettuare la negoziazione o per informare l’altro capo sulle funzionalità disponibili. Altre opzioni sono trasportate nei pacchetti lungo la durata della connessione. Ogni opzione ha una codifica tipo-lunghezza-valore.

Un’opzione ampiamente utilizzata è quella che permette a ogni host di specificare il **MSS** (*maximum segment size*) che è disposto ad accettare. Usare grossi segmenti è più efficiente che usarne di piccoli perché l’intestazione di 20 byte può essere ammortizzata su più dati, ma gli host di scarsa capacità possono non essere in grado di gestire segmenti grandi. Durante l’impostazione della connessione, ogni lato può annunciare il proprio massimo e vedere quello del proprio partner. Se un host non usa questa opzione, il valore predefinito sarà 536 byte di payload. A tutti gli host di Internet viene richiesto di accettare segmenti TCP di 536 + 20 = 556 byte. La dimensione massima di segmento non ha bisogno di essere la stessa nelle due direzioni.

Per le linee con elevata ampiezza di banda, grande ritardo, o entrambi, la finestra da 64 KB corrispondente a un campo da 16 bit è un problema. Per esempio una linea OC-12 (di circa 600 Mbps) impiega meno di 1 ms per produrre una finestra intera da 64 KB. Se il round-trip time è di 50 ms (tipico per fibre transcontinentali), il mittente rimarrà inattivo per circa il 98% del tempo aspettando gli acknowledgement. Una finestra più grande permetterebbe al mittente di continuare a spedire fuori i dati. L’opzione **windows scale** (*fattore di scala della finestra*) permette a mittente e destinatario di negoziare un fattore di scala per la finestra all’inizio di una connessione. Entrambe le parti usano il fattore di scala per spostare il campo *Window size* fino a 14 bit a sinistra, ammettendo così finestre fino a 2^{30} byte. Molte implementazioni di TCP supportano questa opzione.

L’opzione **timestamp** porta un marcatore temporale spedito dal mittente e rimbalzato poi dal destinatario. Una volta che durante la configurazione della connessione si è stabilito di farne uso viene inclusa in ogni pacchetto; è usata per calcolare i campioni dei round-trip time usati per stimare quando è stato perso un pacchetto. È usata anche come un’estensione logica del numero di sequenza di 32 bit. Su una connessione veloce la sequenza numerica può tornare a zero in fretta, generando possibile confusione tra dati vecchi e nuovi. Lo schema **PAWS** (*protection against wrapped sequence numbers*, protezione dai numeri di sequenza azzerati) per prevenire questo problema scarta i segmenti in arrivo con timestamp vecchi.

Infine l'opzione **SACK** (*selective acknowledgement*) permette a un ricevente di indicare al mittente gli intervalli di numeri di sequenza che ha ricevuto. Si aggiunge al campo *Acknowledgement number* ed è utilizzata dopo che un pacchetto è stato perso ma sono arrivati dati successivi (o duplicati). Il valore del campo *Acknowledgement number* nell'intestazione non rispecchia i nuovi dati, perché quel campo dà solo il bit successivo in ordine numerico che ci si aspetta di ricevere. Con SACK il mittente è avvertito esplicitamente su quali dati il destinatario possiede e quindi può determinare quali dati debbano essere ritrasmessi. SACK è definito negli RFC 2108 e 2883 ed è usato sempre di più. Descriveremo l'uso di SACK insieme al controllo di congestione nel Paragrafo 6.5.10.

6.5.5 Instaurazione della connessione TCP

Le connessioni in TCP vengono stabilite mediante l'handshake a tre vie trattato nel Paragrafo 6.2.2. Per stabilire una connessione, un lato (per esempio il server) attende in modo passivo una connessione in ingresso eseguendo nell'ordine le primitive LISTEN e ACCEPT che possono indicare una sorgente specifica oppure nessuna in particolare.

L'altro lato (diciamo il client) esegue una primitiva CONNECT, specificando l'indirizzo IP e la porta a cui vuole connettersi, la dimensione massima del segmento TCP che è intenzionato ad accettare e, facoltativamente, alcuni dati utente (per esempio una password). La primitiva CONNECT invia un segmento TCP con il bit *SYN* a 1 e il bit *ACK* a 0, poi attende una risposta.

Quando questo segmento arriva a destinazione, l'entità TCP controlla se esiste un processo che ha eseguito una LISTEN sulla porta indicata nel campo *Destination port* e in caso negativo invia una risposta con il bit *RST* a 1 per rifiutare la connessione.

Se un processo è in ascolto sulla porta, gli viene dato il segmento TCP in ingresso; quindi può accettare o rifiutare la connessione. Se accetta, viene restituito al mittente un segmento di acknowledgement. La sequenza dei segmenti TCP inviata nel caso normale è mostrata nella Figura 6.37 (a). Si noti che un segmento *SYN* consuma un byte nello spazio delle sequenze, così da poter ricevere un acknowledgement non ambiguo.

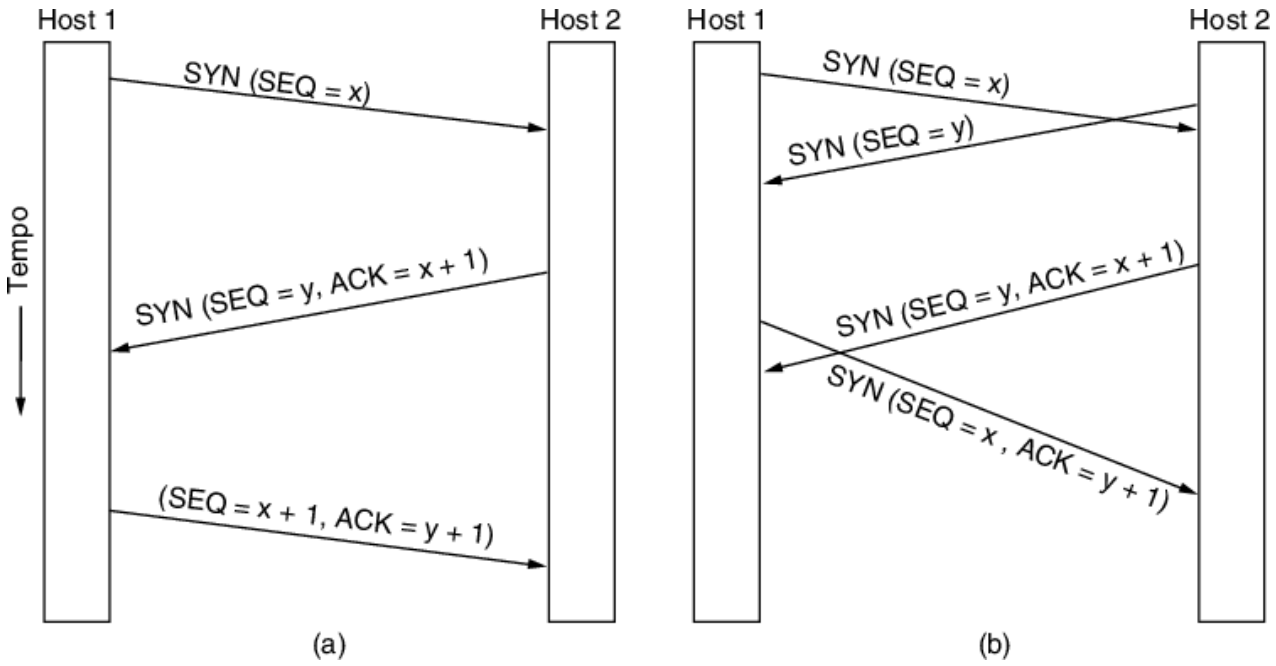


Figura6.37 (a) Instaurazione di una connessione TCP nel caso normale. (b) Instaurazione simultanea da entrambi i lati.

Nel caso che due host tentino contemporaneamente di stabilire una connessione tra le stesse due socket, la sequenza di eventi è illustrata nella Figura 6.37(b). Il risultato di questi eventi è la costituzione di una sola connessione, non due, perché le connessioni sono identificate dai loro punti terminali. Se sia la prima attivazione che la seconda generano una connessione identificata da (x, y), viene comunque creata una sola voce nella tabella per (x, y).

Si ricordi che il numero di sequenza iniziale, scelto da ciascun host, dovrebbe variare lentamente invece di essere una costante, come 0. Questa regola serve a proteggere dai pacchetti in ritardo e duplicati, come detto nel Paragrafo 6.2.2. Originariamente ciò si otteneva con schemi basati su clock che scattavano ogni 4 ms.

Tuttavia, una vulnerabilità nell'implementazione dell'handshake a tre vie è che il processo in ascolto deve ricordare il proprio numero di sequenza dal momento in cui risponde con il proprio segmento *SYN*. Questo significa che un mittente malintenzionato può bloccare le risorse su un host spedendo un flusso di segmenti *SYN* e non completando mai la connessione. Questo attacco è chiamato **SYN flood** (*inondazione di SYN*) e ha "azzoppato" molti Web server negli anni '90.

Un modo per difendersi da questo attacco è utilizzare i **SYN cookie**. Invece di ricordare il numero di sequenza, un host sceglie un numero di sequenza, generato facendo uso di un algoritmo tipico della criptografia, lo mette nel segmento in uscita e se lo dimentica. Se l'handshake a tre vie arriva a termine, questo numero di sequenza (più 1) verrà restituito all'host che potrà quindi rigenerarlo eseguendo la stessa funzione criptografica, fintantoché siano noti gli input di quella funzione; per esempio gli indirizzi IP le porte dell'altro host e un segreto locale. Questa procedura permette all'host di verificare che una numero di sequenza per cui si riceve un acknowledgement è corretto senza doverselo ricordare separatamente. Ci sono alcune limitazioni,

come l'impossibilità di gestire le opzioni TCP; quindi i SYN cookie dovrebbero essere usati solo quando l'host è soggetto a un SYN flood. Tuttavia sono una variante interessante durante l'impostazione di una connessione. Per maggiori informazioni si consultino l'RFC 4987 e Lemon (2002).

6.5.6 Rilascio di una connessione TCP

Anche se le connessioni TCP sono full-duplex, per comprendere come vengono rilasciate è meglio immaginarle come una coppia di connessioni simplex, in cui ogni connessione è rilasciata in modo indipendente dalla sua gemella. Per rilasciare una connessione, entrambe le parti possono inviare un segmento TCP con il bit *FIN* a 1, per indicare che non hanno più dati da trasmettere. Quando il *FIN* riceve l'acknowledgement, la direzione viene chiusa ai nuovi dati; tuttavia i dati possono continuare a fluire indefinitamente nell'altra direzione. Quando entrambe le direzioni saranno state chiuse la connessione sarà rilasciata. Normalmente per rilasciare una connessione sono necessari quattro segmenti TCP, un *FIN* e un *ACK* per ogni direzione; tuttavia è possibile inserire il primo *ACK* e il secondo *FIN* nello stesso segmento, riducendo il totale a tre.

Proprio come nelle conversazioni telefoniche dove entrambe le persone si salutano e riagganciano contemporaneamente, entrambe le estremità di una connessione TCP possono inviare segmenti *FIN* contemporaneamente; questi riceveranno acknowledgement nel modo abituale e la connessione verrà chiusa. In effetti non c'è una vera e propria differenza tra due host che rilasciano in sequenza o contemporaneamente.

Per evitare il problema dei due eserciti (descritto nel Paragrafo 6.2.3) si usano i timer: se una risposta a *FIN* non arriva entro il tempo di vita massimo di due pacchetti, il mittente del *FIN* rilascia la connessione. L'altro lato noterà prima o poi che nessuno sembra più ascoltarlo e andrà anch'esso in timeout. Anche se questa soluzione non è perfetta, non resta che accettarla, poiché una soluzione perfetta è teoricamente impossibile e nella pratica raramente nascono problemi.

6.5.7 Modello di gestione della connessione TCP

I passi richiesti per stabilire e rilasciare le connessioni possono essere rappresentati in una macchina a stati finiti con gli 11 stati elencati nella Figura 6.38. In ogni stato sono ammessi alcuni eventi; quando si verifica un evento ammesso è possibile svolgere alcune azioni, mentre se si verificano altri eventi viene segnalato un errore.

Stato	Descrizione
CLOSED	Nessuna connessione è attiva o in sospeso
LISTEN	Il server è in attesa di una chiamata in ingresso
SYN RCVD	È arrivata una richiesta di connessione; in attesa di ACK
SYN SENT	L'applicazione ha iniziato ad aprire una connessione

Stato	Descrizione
ESTABLISHED	Il normale stato di trasferimento dei dati
FIN WAIT 1	L'applicazione ha detto di aver terminato
FIN WAIT 2	L'altro lato ha accettato il rilascio
TIME WAIT	Attende la scadenza di tutti i pacchetti
CLOSING	Entrambi i lati hanno cercato di chiudere contemporaneamente
CLOSE WAIT	L'altro lato ha iniziato il rilascio
LAST ACK	Attende la scadenza di tutti i pacchetti

Figura6.38 Gli stati utilizzati nella macchina a stati finiti per la gestione della connessione TCP.

Ogni connessione inizia nello stato *CLOSED*. Lo stato viene lasciato quando la connessione esegue un'apertura passiva (LISTEN) o un'apertura attiva (CONNECT). Se l'altro lato esegue l'operazione opposta, viene stabilita una connessione e lo stato diventa *ESTABLISHED*. Il rilascio della connessione può essere iniziato da una qualunque delle parti. Una volta completato, lo stato torna a essere *CLOSED*.

La macchina a stati finiti vera e propria è mostrata nella Figura 6.39. Il caso tipico di un client connesso in modo attivo a un server passivo è mostrato con linee spesse (continue per il client, tratteggiate per il server). Le linee sottili rappresentano sequenze di eventi insolite. Ogni linea nella Figura 6.39 è contrassegnata da una coppia evento/azione. L'evento può essere una chiamata di sistema da parte dall'utente (CONNECT, LISTEN, SEND o CLOSE), l'arrivo di un segmento (*SYN*, *FIN*, *ACK* o *RST*) oppure un timeout pari al doppio del massimo tempo di vita del pacchetto. L'azione è l'invio di un segmento di controllo (*SYN*, *FIN* o *RST*) oppure nulla (indicato con –). I commenti sono mostrati tra parentesi.

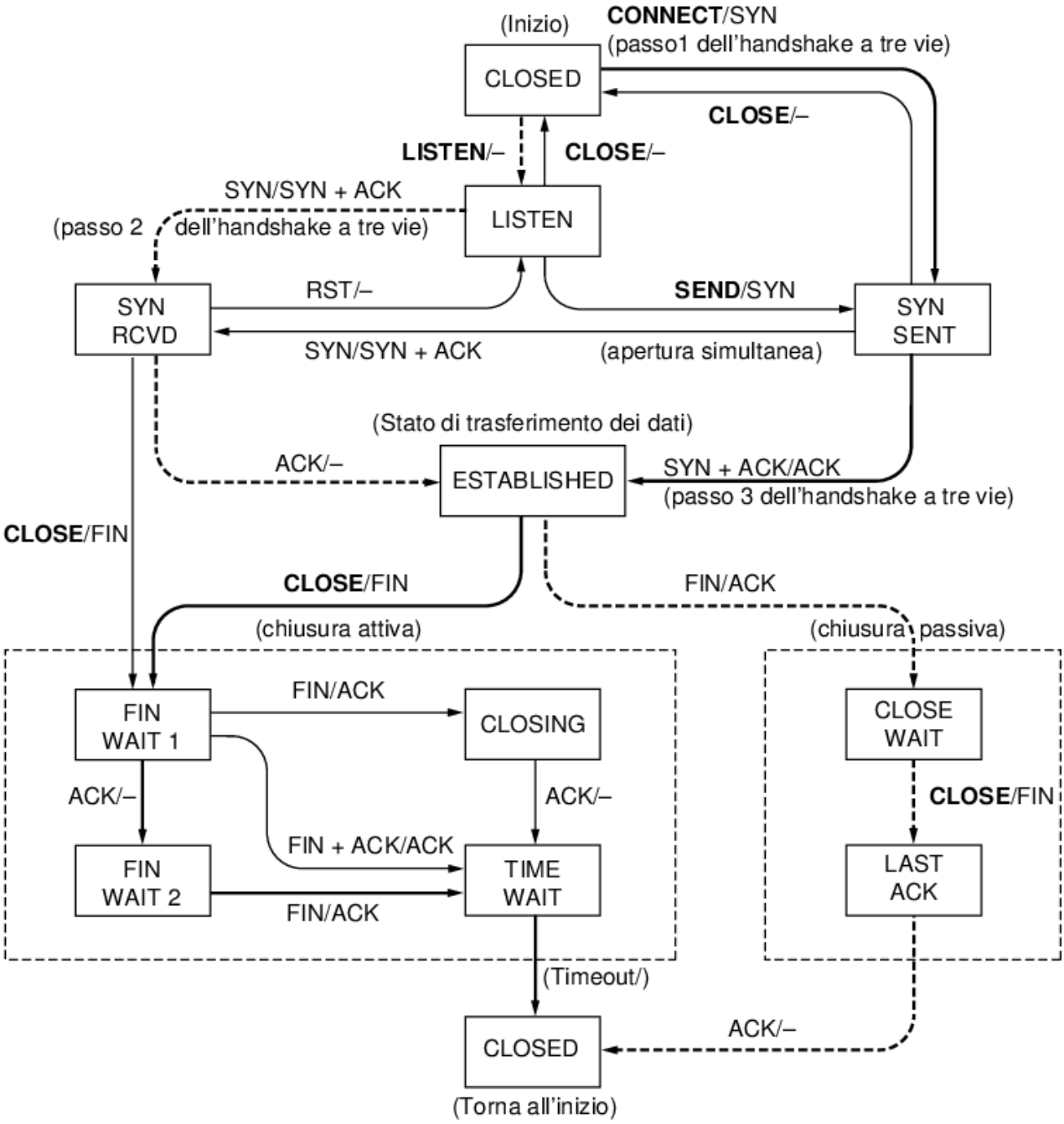


Figura6.39 La macchina a stati finiti per la gestione di una connessione TCP. La linea continua spessa è il percorso normale per un client; la linea tratteggiata spessa è il percorso normale per un server. Le linee sottili rappresentano eventi insoliti. Ogni transizione è etichettata con l'evento che la provoca e l'azione risultante, separati da una barra.

Potrebbe essere più facile comprendere il diagramma seguendo prima il percorso di un client (la linea continua spessa) e poi il percorso di un server (la linea tratteggiata spessa). Quando un programma applicativo sulla macchina client genera una richiesta CONNECT, l'entità TCP locale crea un record per la connessione, la contrassegna nello stato *SYN SENT* e invia un segmento *SYN*. Si noti che molte connessioni potrebbero essere aperte (o venire aperte) contemporaneamente da più applicazioni, pertanto lo stato è relativo alla singola connessione ed è registrato nel relativo record. Quando arriva il *SYN* + *ACK*, TCP invia l'*ACK* finale dell'handshake a tre vie e passa nello stato *ESTABLISHED*. Ora è possibile inviare e ricevere i dati.

Quando un'applicazione ha finito, esegue una primitiva CLOSE che provoca l'invio da parte dell'entità TCP locale di un segmento *FIN* e l'attesa dell'*ACK* corrispondente (il riquadro tratteggiato indicato come "chiusura attiva"). Quando arriva l'*ACK*, viene eseguita una transizione allo stato *FIN WAIT 2* e una direzione della connessione si chiude. Quando anche l'altro lato esegue la chiusura, viene ricevuto un *FIN* che richiede l'invio di un acknowledgment. Ora entrambi i lati sono chiusi, ma TCP attende un tempo uguale al doppio del tempo di vita massimo del pacchetto per garantire che tutti i pacchetti della connessione siano scaduti (nel caso in cui l'acknowledgement fosse andato perso). Alla scadenza del timer, TCP elimina il record della connessione.

Esaminiamo ora la gestione della connessione dal punto di vista del server. Il server esegue una LISTEN e attende per vedere che cosa succede. Quando al server arriva un *SYN,* questo manda un acknowledgement e passa allo stato *SYN RCVD*; nel momento in cui anche il *SYN* del server riceve un acknowledgement, l'handshake a tre vie è completo e il server passa allo stato *ESTABLISHED*. Da questo momento in poi può avvenire il trasferimento dei dati.

Quando il client ha terminato di trasmettere i propri dati effettua una CLOSE, che provoca l'invio da parte del client di un *FIN* al server (il riquadro tratteggiato indicato come "chiusura passiva"). Il server, ricevuto il segnale *FIN*, esegue a sua volta una CLOSE; operazione che prevede l'invio di un *FIN* al client. All'arrivo dell'acknowledgement relativo a quest'ultimo *FIN* da parte del client, il server rilascia la connessione e ne elimina il relativo record.

6.5.8 La finestra scorrevole di TCP

Come detto in precedenza, la gestione della finestra in TCP disaccoppia il problema degli acknowledgement relativi alla corretta ricezione dei segmenti e l'allocazione dei buffer lato ricevente. Si supponga per esempio che il ricevente abbia un buffer di 4096 byte, come mostrato nella Figura 6.40. Se il mittente trasmette un segmento di 2048 byte che viene correttamente ricevuto, il ricevente darà un acknowledgement al segmento. Tuttavia, dal momento che ora possiede solo 2048 byte di spazio nel buffer (fino a quando l'applicazione rimuove dei dati dal buffer), "pubblicizza" una finestra di 2048 byte partendo dal successivo byte atteso.

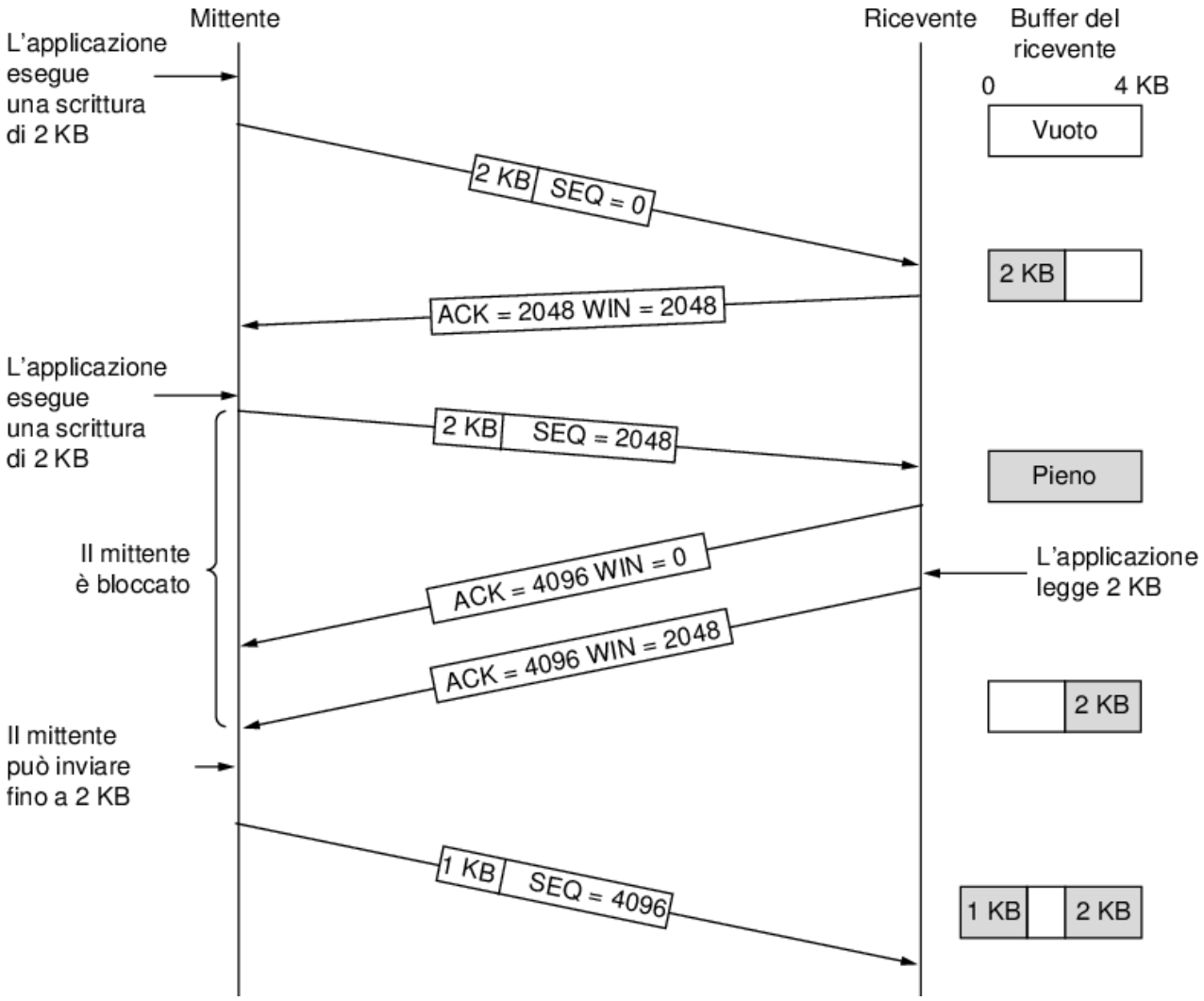


Figura 6.40 Gestione della finestra in TCP.

Ora il mittente trasmette altri 2048 byte che ricevono acknowledgement; la finestra pubblicizzata ha però dimensione 0. Il mittente deve fermarsi finché il processo applicativo sull'host ricevente non abbia rimosso dei dati dal buffer; in quel momento TCP potrà pubblicizzare una finestra più grande e potranno essere spediti altri dati.

Quando la finestra è 0, il mittente non può di norma inviare segmenti, ma vi sono due eccezioni. Primo: è possibile inviare dati urgenti, per esempio per consentire all'utente di interrompere il processo in esecuzione sulla macchina remota. Secondo: il mittente può inviare un segmento di 1 byte, per fare in modo che il ricevente annunci di nuovo il successivo byte atteso e la dimensione della finestra; questo pacchetto prende il nome di **window probe** (*sonda per la finestra*). Lo standard TCP fornisce esplicitamente questa possibilità per prevenire un blocco nel caso dovesse essere perso un aggiornamento della dimensione della finestra.

I mittenti non sono obbligati a trasmettere i dati appena li ricevono dall'applicazione e i riceventi non hanno l'obbligo di inviare gli acknowledgement il prima possibile. Nell'esempio della Figura 6.40 all'arrivo dei primi 2 KB di dati, TCP, sapendo di avere a disposizione una finestra di 4 KB, si

sarebbe comportato in modo legittimo se avesse inserito dati nel buffer fino all'arrivo di altri 2 KB, in modo da trasmettere un segmento con payload di 4 KB. Questa libertà può essere sfruttata per migliorare le prestazioni.

Si consideri una connessione a un terminale remoto tramite SSH o telnet che reagisce a ogni pressione di un tasto. Nel caso peggiore, quando un carattere arriva all'entità TCP di invio, TCP crea un segmento TCP di 21 byte che passa a IP affinché lo invìi come datagramma IP di 41 byte. Dal lato ricevente TCP invia immediatamente un acknowledgement di 40 byte (20 byte di intestazione TCP e 20 byte di intestazione IP). In seguito, quando il terminale remoto ha letto il byte, TCP invia un aggiornamento della finestra, spostandola di 1 byte a destra; anche questo pacchetto è di 40 byte. Per finire, il terminale remoto, quando ha elaborato il carattere, lo restituisce per poterlo visualizzare localmente tramite un pacchetto di 41 byte. In tutto vengono utilizzati 162 byte di banda e quattro segmenti per ogni carattere digitato. Se la banda è scarsa, questo metodo non è il più conveniente.

Un approccio usato da molte implementazioni di TCP per ottimizzare questa situazione prende il nome di **delayed acknowledgement** (*acknowledgement ritardato*). Utilizzato da molte implementazioni di TCP, prevede di ritardare gli acknowledgement e gli aggiornamenti della finestra di 500 ms, nella speranza di acquisire alcuni dati che possano "scroccare un passaggio". Supponendo che il terminale visualizzi il carattere entro i 500 ms, occorre spedire un solo pacchetto di 41 byte verso il mittente, dimezzando il numero di pacchetti e l'utilizzo della banda.

Anche se l'acknowledgement ritardato riduce il carico di rete al destinatario, un mittente che spedisce molti pacchetti piccoli (come pacchetti da 41 byte contenenti 1 byte di dati) fa ancora un uso inefficiente della rete. Un modo per ridurre questo spreco è chiamato **algoritmo di Nagle** (Nagle, 1984). Ciò che Nagle suggerì è semplice: quando i dati arrivano al mittente in piccoli gruppi, è sufficiente inviare il primo gruppo e inserire il resto nel buffer finché arriva l'acknowledgement. A questo punto si possono inviare tutte le informazioni nel buffer usando un solo segmento TCP e iniziare di nuovo a inserire i dati nel buffer fino all'arrivo del prossimo acknowledgement. Questo vuol dire che solo un pacchetto (di piccole dimensioni) può essere in circolazione in ogni dato momento. Se molti dati vengono spediti dall'applicazione in un round-trip time, l'algoritmo di Nagle li metterà tutti in un segmento, riducendo notevolmente la banda utilizzata. L'algoritmo consente inoltre l'invio di un nuovo segmento se sono arrivati abbastanza dati da riempirne uno con dimensione massima.

L'algoritmo di Nagle è ampiamente utilizzato dalle implementazioni TCP, ma a volte è preferibile disattivarlo. In particolare nei giochi interattivi che usano Internet di solito i giocatori vogliono un flusso veloce di piccoli pacchetti contenenti gli aggiornamenti. Raccogliere gli aggiornamenti per spedirli in blocchi fa rispondere il gioco in maniera inpredicibile, il che rende insoddisfatti gli utenti. Un problema più subdolo è che l'algoritmo di Nagle può interagire a volte con gli acknowledgement ritardati causando un blocco temporaneo: il destinatario aspetta dei dati a cui accodare un acknowledgement e il mittente aspetta un acknowledgement per spedire altri dati; questa interazione può ritadare i download di pagine Web. A causa di questi problemi l'algoritmo di Nagle può essere disabilitato (opzione *TCP NO_DELAY*). Mogul e Minshall (2001) discutono questa e altre soluzioni.

Un altro problema che può degradare le prestazioni di TCP è la **silly window syndrome** (*sindrome della finestra stupida*) (Clark, 1982). Questo problema si verifica quando i dati vengono passati all'entità TCP di invio in blocchi grandi, ma un'applicazione interattiva dal lato ricevente legge i dati 1 byte alla volta. Per comprendere il problema si osservi la Figura 6.41. Inizialmente il buffer TCP lato ricevente è pieno (ha una finestra con dimensione 0) e il mittente ne è a conoscenza. L'applicazione interattiva legge quindi un carattere dal flusso TCP. Questa azione fa felice il TCP di ricezione, che invia un aggiornamento della finestra al mittente comunicandogli che può inviare 1 byte. Il mittente obbedisce e invia 1 byte. Ora il buffer è pieno, pertanto il ricevente manda un acknowledgement per il segmento di 1 byte, ma imposta la finestra a 0. Questo comportamento può andare avanti per sempre.

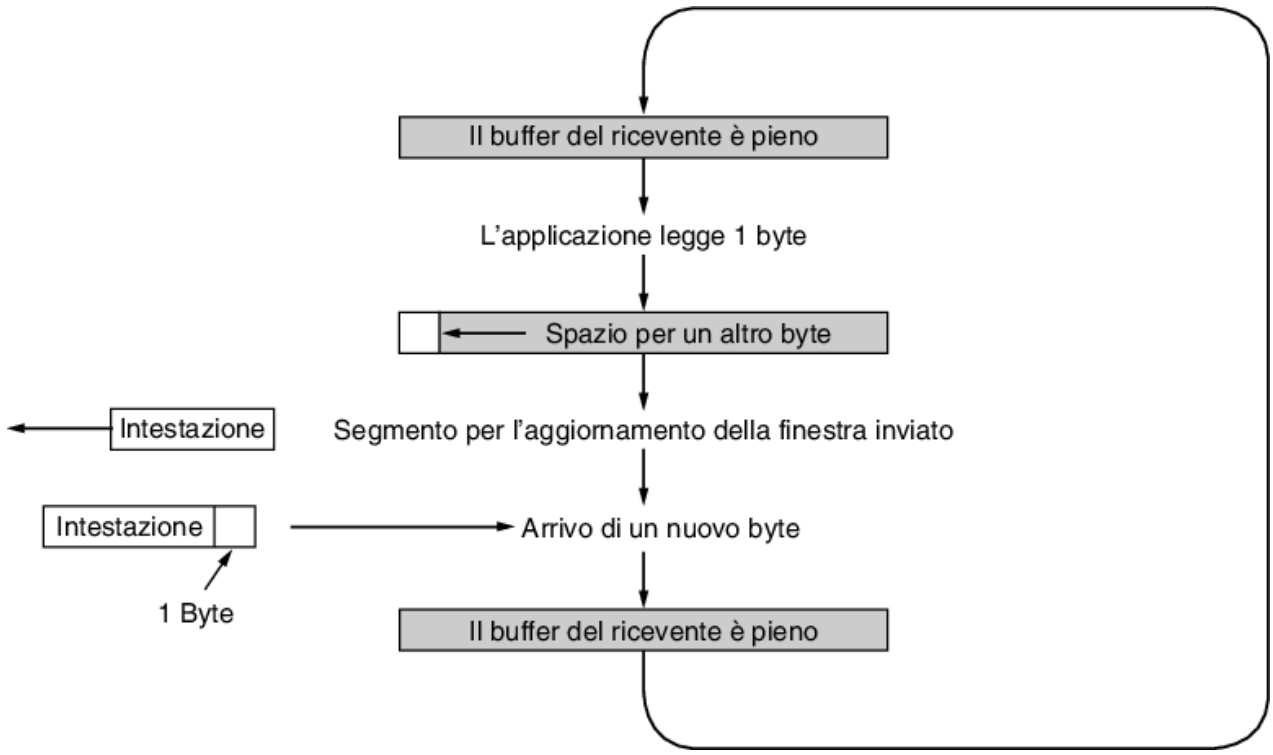


Figura6.41 Silly window syndrome.

La soluzione di Clark sta nell'impedire che il ricevente invìi un aggiornamento della finestra per 1 byte; invece è obbligato ad attendere la disponibilità di una certa quantità di spazio. Nello specifico il ricevente può inviare un aggiornamento della finestra solo quando è in grado di gestire la dimensione massima del segmento (concordata quando la connessione è stata instaurata) o quando il suo buffer è vuoto per metà. Inoltre il mittente può essere di aiuto se non invia segmenti di piccole dimensioni; dovrebbe invece provare ad attendere fino a quando ha accumulato un segmento completo o almeno un segmento contenente dati per la metà della dimensione del buffer del ricevente.

L'algoritmo di Nagle e la soluzione di Clark al problema della silly window syndrome sono complementari. Nagle stava provando a risolvere il problema provocato dal fatto che l'applicazione di invio consegna i dati a TCP un byte alla volta. Clark stava provando a risolvere il problema di un'applicazione ricevente che legge i dati da TCP un byte alla volta. Entrambe le soluzioni sono valide e possono cooperare. Lo scopo è che il mittente non invii segmenti piccoli e che il ricevente non li richieda.

Il TCP di ricezione può andare oltre nel miglioramento delle prestazioni. Proprio come il TCP di invio, può fare uso di un buffer per tenere bloccata una richiesta READ dell'applicazione finché non possa fornirle un blocco di dati abbastanza grande. In questo modo si riducono il numero di chiamate a TCP e il corrispondente overhead. Naturalmente aumenta anche il tempo di risposta, ma per applicazioni non interattive come il trasferimento di file l'efficienza potrebbe essere più importante del tempo di risposta alle singole richieste.

Un altro problema che il destinatario deve gestire è che i segmenti possono arrivare in disordine. In destinatario deve memorizzare i dati in un buffer fino al momento in cui possono essere passati all'applicazione nel giusto ordine. In realtà se fossero scartati segmenti in disordine non succederebbe nulla di male, dal momento che alla fine sarebbero ritrasmessi dal mittente, ma sarebbe uno spreco di risorse.

Gli acknowledgement possono essere inviati solo quando sono stati ricevuti tutti i dati fino all'ultimo byte; questo prende il nome di **cumulative acknowledgement** (*acknowledgement cumulativo* o *ack cumulativo*). Se il destinatario riceve i segmenti 0, 1, 2, 4, 5, 6 e 7, può mandare acknowledgement fino all'ultimo byte del segmento 2 incluso. Al timeout del mittente, la trasmissione riprende dal segmento 3. Se il ricevente ha inserito nel buffer i segmenti da 4 a 7, dopo la ricezione del segmento 3 può mandare acknowledgement per tutti i byte fino alla fine del segmento 7.

6.5.9 Gestione dei timer di TCP

TCP utilizza più timer (almeno concettualmente) per svolgere il proprio lavoro. Il più importante è quello che fa scattare il **RTO** (*retransmission timeout*, timeout di ritrasmissione). Quando viene inviato un segmento, si avvia un timer di ritrasmissione. Se il segmento riceve un acknowledgement prima della scadenza del timer, TCP lo ferma; se invece il timer scade prima dell'arrivo dell'acknowledgement, il segmento viene ritrasmesso (e il timer riavviato). La questione che si pone è: quanto dovrebbe essere lungo l'intervallo di timeout?

Questo problema è molto più difficile nel livello di trasporto piuttosto che nei protocolli data link come 802.11, perché in questo ultimo caso il ritardo atteso si misura in microsecondi ed è altamente prevedibile (vale a dire che ha una varianza bassa), pertanto il timer può essere impostato per scadere appena dopo il tempo stimato di ricezione dell'acknowledgement, come mostrato nella Figura 6.42(a). Dal momento che nel livello data link gli acknowledgement vengono raramente ritardati (a causa della mancanza di congestione), l'assenza di un acknowledgement nel momento previsto indica generalmente che il frame o l'acknowledgement sono stati persi.

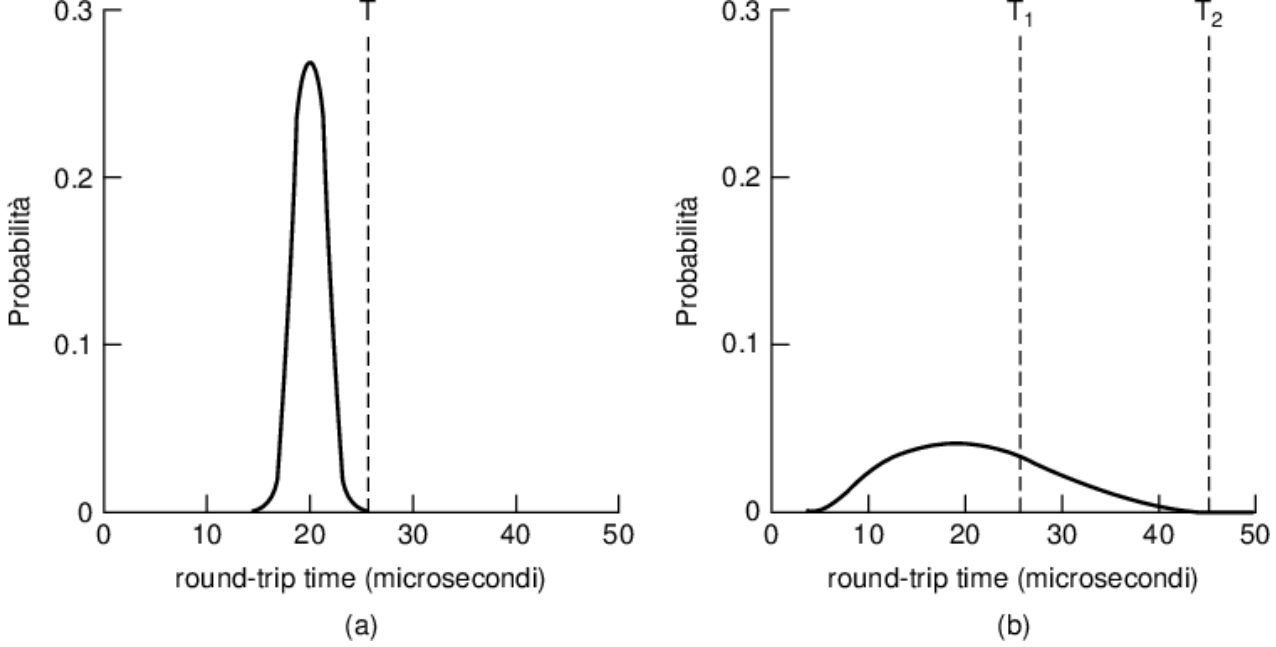


Figura6.42 (a) Funzione di densità di probabilità dei tempi di arrivo degli acknowledgement nel livello data link. (b) Funzione di densità di probabilità dei tempi di arrivo degli acknowledgement TCP.

TCP deve affrontare un ambiente radicalmente diverso. La funzione di densità di probabilità relativa al tempo richiesto da un acknowledgement TCP per tornare indietro è più simile a quella della Figura 6.42(b) che a quella della Figura 6.42(a): è più grande e ha una maggiore variabilità. Determinare un round-trip time tra sorgente e destinazione è complicato e, anche quando è noto, non è semplice decidere l'intervallo di timeout. Se il timeout impostato è troppo breve (T_1 nella Figura 6.42b), si verificheranno ritrasmissioni non necessarie che congestioneranno Internet con pacchetti inutili. Se è troppo lungo (cioè T_2), le prestazioni ne risentiranno a causa del lungo ritardo nella ritrasmissione ogni volta che un pacchetto viene perso. Inoltre la media e la varianza della distribuzione di arrivo degli acknowledgement possono cambiare rapidamente in pochi secondi, a causa della congestione che va e viene.

La soluzione è utilizzare un algoritmo dinamico che adatta costantemente l'intervallo di timeout in base a continue misurazioni delle prestazioni di rete. L'algoritmo utilizzato generalmente da TCP è dovuto a Jacobson (1988) e funziona come segue: per ogni connessione TCP mantiene una variabile, **SRTT** (*smoothed round-trip time*), che rappresenta la migliore stima corrente del round-trip time per la destinazione in questione. Quando viene inviato un segmento, si avvia un timer che serve a due scopi: per sapere quanto tempo richiede l'acknowledgement e per innescare un'eventuale ritrasmissione. Se l'acknowledgement torna indietro prima della scadenza del timer, TCP misura il tempo richiesto, diciamo *R,* e poi aggiorna *SRTT* secondo la formula

$$SRTT = a\ SRTT + (1 - a)\ R$$

dove a è un fattore di perequazione che determina quanto velocemente i vecchi valori vengono dimenticati; generalmente a corrisponde a 7/8.[6] Questo tipo di formula è una **EWMA** (*exponentially weighted moving average*, media mobile a peso esponenziale) o filtro passa basso e serve a togliere il rumore dalle campionature.

Anche con un buon valore di *SRTT* scegliere un timeout di ritrasmissione adatto non è banale. Le implementazioni iniziali di TCP usavano 2*xRTT*, ma l'esperienza ha dimostrato che un valore costante è troppo poco flessibile perché non riesce a intervenire quando aumenta la varianza. In particolare modelli di accodamento di traffico casuale (Poisson) prevedono che, quando il carico si avvicina alla capacità, il ritardo cresce e diventa altamente variabile. Questo può portare a far scattare il timer e alla ritrasmissione di una copia del pacchetto, nonostante il fatto che il pacchetto originale sia ancora in transito. Tutto ciò è molto più probabile che accada in condizioni di carico elevato, che è il momento peggiore in cui spedire pacchetti aggiuntivi in rete.

Per risolvere questo problema, Jacobson propose di rendere il valore di timeout sensibile alla variazione del round-trip time oltre che al valore filtrato dei tempi. Questo cambiamento richiede di tenere traccia di un'altra variabile: ***RTTVAR*** (*round-trip time variation*, variazione del tempo di round-trip) aggiornata con la formula

$$RTTVAR = b\ RTTVAR + (1 - b)\ |\ SRTT - R\ |$$

Questa è una EWMA come la precedente, e di solito b = 3/4. Il timeout di ritrasmissione, *RTO*, è impostato come

$$RTO = SRTT + 4 \times RTTVAR$$

La scelta del fattore 4 è in qualche modo arbitraria, ma la moltiplicazione per 4 può essere fatta con un solo scorrimento e meno dell'1% di tutti i pacchetti arrivano con un ritardo superiore a quattro volte la deviazione standard. Notate che *RTTVAR* non è esattamente equivalente alla deviazione standard (in realtà è la deviazione media), ma nella realtà gli è abbastanza vicina. L'articolo di Jacobson è pieno di trucchi intelligenti per calcolare i timeout usando solo somme, sottrazioni e scorrimenti di numeri interi. Questa economia non è necessaia agli host moderni, ma è diventata parte della cultura che permette a TCP di funzionare su tutti i tipi di apparecchiature, dai supercomputer fino ai dispositivi più piccoli. Finora nessuno l'ha messo su un chip RFID, ma un giorno forse qualcuno lo farà.

Maggiori dettagli su come calcolare questo timeout, comprese le impostazioni iniziali delle variabili, vengono date nell'RFC 2988. Il timer di ritrasmissione vincolato a un minimo di 1 secondo, indipendentemente dalle stime. È un valore scelto prudentemente per prevenire ritrasmissioni spurie a causa delle misurazioni (Allman e Paxson, 1999).

Un problema che si verifica con la raccolta dei campioni del round-trip time *R* è che cosa fare quando un segmento va in timeout e viene spedito di nuovo. Quando arriva l'acknowledgement non è chiaro se si riferisce alla prima trasmissione o a quella successiva. Un errore di valutazione potrebbe alterare seriamente il timeout di ritrasmissione. Phil Karn ha scoperto questo problema nel modo peggiore. Karn, un radioamatore interessato alla trasmissione di pacchetti TCP/IP tramite radio amatoriali, un mezzo notoriamente inaffidabile, ha fatto una semplice proposta: evitare di aggiornare le stime per ogni segmento che è stato ritrasmesso. Inoltre, propone che il timeout venga raddoppiato a ogni successiva ritrasmissione fino a quando i segmenti giungono a destinazione al primo tentativo. Questa correzione è chiamata **algoritmo di Karn** (Karn e Partridge, 1987) ed è utilizzata da molte implementazioni di TCP.

Il timer di ritrasmissione non è l'unico utilizzato da TCP; un altro è il **timer di persistenza.** È progettato per impedire la seguente situazione di stallo: il ricevente invia un acknowledgement con una dimensione della finestra pari a 0, chiedendo al mittente di aspettare. In seguito il mittente aggiorna la finestra, ma il pacchetto con l'aggiornamento viene perso. Ora sia il mittente sia il ricevente attendono la prossima mossa dell'altro. Quando il timer di persistenza scade, il mittente trasmette una sonda (*window probe*) al ricevente. La risposta alla sonda fornisce la dimensione della finestra; se la dimensione è ancora zero, il timer di persistenza viene reimpostato e il ciclo si ripete; se non è zero è ora possibile inviare i dati.

Un terzo timer utilizzato da alcune implementazioni è il **timer di keepalive** (*timer di mantenimento in vita*). Quando una connessione è stata inattiva per lungo tempo, il timer di keepalive scade in modo che una delle parti controlli se l'altra sia ancora presente. Se non ottiene risposta la connessione viene terminata. Questa funzionalità è controversa, perché aggiunge carico di lavoro e può terminare una connessione attiva a causa di una partizione temporanea della rete.

L'ultimo timer utilizzato da ogni connessione TCP è quello impiegato nello stato *TIMED WAIT* durante la chiusura. Il conteggio prosegue per un tempo pari al doppio del tempo di vita massimo del pacchetto, per garantire che alla chiusura di una connessione tutti i pacchetti creati siano stati rimossi.

6.5.10 Controllo della congestione di TCP

Abbiamo tenuto una delle funzioni chiave di TCP per ultima: il controllo della congestione. Quando il carico applicato a qualsiasi rete è più di quello che questa riesce a gestire, si crea una congestione, e Internet non fa eccezione. Il livello di rete rileva la congestione quando le code sui router aumentano e cerca di gestirla scartando dei pacchetti. È compito del livello di trasporto ricevere i segnali di congestione dal livello di rete e rallentare il tasso del traffico inviato sulla rete. In Internet TCP svolge il ruolo principale nel controllare la congestione, nonché il ruolo principale nel trasporto affidabile. Ecco perché è un protocollo così speciale.

Nel Paragrafo 6.3 abbiamo trattato la situazione generica di controllo di congestione. Un insegnamento chiave era che un protocollo di trasporto facente uso di una legge di controllo AIMD (*additive increase multiplicative decrease*, incremento additivo, decremento moltiplicativo) in risposta a segnali binari di congestione da parte della rete può convergere verso un'allocazione di banda equa ed efficiente. Il controllo della congestione di TCP è basato sull'implementazione di questo approccio, utilizzando una finestra e interpretando la perdita di pacchetti come segnali binari di congestione. Per farlo, TCP mantiene una **finestra di congestione** la cui dimensione è pari al numero di byte che il mittente può avere sulla rete in qualsiasi momento. Il tasso corrispondente è la dimensione della finestra diviso per il round-trip time della connessione. TCP aggiusta la dimensione della finestra secondo le regole AIMD.

Si ricordi che la finestra di congestione viene mantenuta *in aggiunta* alla finestra di controllo del flusso che specifica il numero di byte che il destinatario può inserire nel suo buffer. Entrambe le finestre vengono seguite in parallelo e il numero di byte che possono essere spediti è il più piccolo delle due finestre. Perciò la finestra effettiva è la più piccola di quelle che il mittente e il destinatario credono vada bene. Bisogna essere in due per ballare il tango. TCP smetterà di spedire dati se la finestra di controllo di congestione o quella di controllo di flusso sono temporaneamente piene. Se

il destinatario richiede “Spedisci 64 KB” ma il mittente sa che un picco di più di 32 KB intaserebbe la rete, spedirà 32 KB. D’altra parte, se il destinatario fa la stessa domanda e il mittente sa che picchi fino a 128 KB passeranno senza sforzo, spedirà l’intero 64 KB richiesto. Poiché la finestra di controllo di flusso è stata descritta in precedenza, di seguito descriveremo solo la finestra di congestione.

L’attuale controllo di congestione fu aggiunto a TCP in gran parte grazie agli sforzi di Van Jacobson (1988). Si tratta di una storia affascinante; a partire dal 1986, la crescente popolarità della prima Internet portò al primo avvenimento di ciò che divenne noto come **congestion collapse** (*collasso da congestione*), un periodo prolungato durante il quale il goodput diminuì in maniera veloce e improvvisa (di più di un fattore 100) a causa di una congestione nella rete. Jacobson (e molti altri) cercarono di capire che cosa stava succedendo e di porre rimedio alla situazione.

La soluzione ad alto livello implementata da Jacobson fu di approssimare una finestra di congestione AIMD. La parte interessante, oltre a molta della complessità del controllo della congestione di TCP, si dimostrò nell’aggiungere una implementazione esistente senza cambiare nulla dei formati di messaggio, il che la rese immediatamente distribuibile. Innanzitutto, Jacobson osservò che la perdita di un pacchetto era un segnale utilizzabile per la congestione. Questo segnale arriva poco dopo (siccome la rete è già congestionata) ma è abbastanza affidabile. Dopo tutto è difficile costruire un router che non perda pacchetti quando è sovraccarico, e questo è improbabile che cambierà. Anche quando useremo buffer con terabyte di memoria capaci di tenere un enorme numero di pacchetti, probabilmente avremo reti da terabit/s che li riempiranno.

Tuttavia l’uso della perdita di pacchetti come segnale di congestione viene influenzato da errori di trasmissione relativamente rari. Le linee wireless come 802.11 normalmente non lo fanno, per cui includono i loro meccanismi di ritrasmissione a livello data link; a causa di queste ritrasmissioni sulle reti wireless la perdita di pacchetti a livello di rete causata da errori di trasmissione viene normalmente mascherata. È raro anche su altre linee perché cavi e fibre ottiche di solito hanno un basso tasso di errore.

Tutti gli algoritmi di TCP usati su Internet assumono che i pacchetti persi siano causati da una congestione; si affidano ai timeout analogamente ai minatori con i canarini. È necessario un buon timer di ritrasmissione a rilevare i segnali di perdita di pacchetti in modo preciso e tempestivo. Abbiamo già discusso come il timer di ritrasmissione di TCP includa stime di media e varianza dei round-trip time. Fissare questo timer, includendo il fattore di variazione, è stato un passo importante nel lavoro di Jacobson. Dato un buon timeout di ritrasmissione, il mittente TCP può tracciare il numero di byte in sospeso che occupa la rete, cercando semplicemente al differenza tra le sequenze numeriche trasmesse e quelle che hanno ricevuto acknowledgement.

Ora il nostro compito sembra facile; tutto ciò che dobbiamo fare è tenere d’occhio la finestra di congestione, usando numeri di sequenze e acknowledgement e aggiustandone la dimensione usando una regola AIMD. Come probabilmente vi aspettavate, è più complicato di così. Una prima considerazione è che il modo in cui i pacchetti vengono spediti in rete, anche per brevi periodi di tempo, deve corrispondere al percorso di rete. Altrimenti il traffico causerà congestione. Per esempio, consideriamo un host con una finestra di congestione da 64 KB collegato a una Ethernet commutata da 1 Gbps. Se l’host spedisce l’intera finestra in una volta, questo picco di traffico potrebbe attraversare anche una linea ADSL da 1 Mbps lungo il percoso. Il picco, che prende solo mezzo millisecondo sulla linea da 1 Gbps, ingorgherà la linea da 1 Mbps per mezzo

secondo disturbando fortemente protocolli come il VoIP. Questo comportamento potrebbe essere positivo per un protocollo progettato per causare congestione, ma non per un protocollo concepito per controllarla.

Tuttavia ne risulta che possiamo usare piccoli picchi di pacchetti a nostro vantaggio. La Figura 6.43 mostra che cosa accade quando un mittente su una rete veloce (la linea a 1 Gbps) spedisce un gruppo di quattro pacchetti molto ravvicinati tra loro a un destinatario su una rete lenta (la linea a 1 Mbps) che è il collo di bottiglia, o la parte più lenta, sul percorso. Inizialmente i quattro pacchetti viaggiano sul collegamento alla velocità a cui possono essere spediti dal mittente. Nel router vengono accodati durante l'inoltro perché ci vuole più tempo per spedire un pacchetto su un collegamento lento rispetto che per ricevere il pacchetto successivo su un collegamento veloce. Ma la coda non è lunga, perché solo un piccolo numero di pacchetti è stato spedito in una volta. Da notare la lunghezza maggiore dei pacchetti sul collegamento lento: lo stesso pacchetto di 1 KB ora è più lungo perché si impiega più tempo per spedirlo.

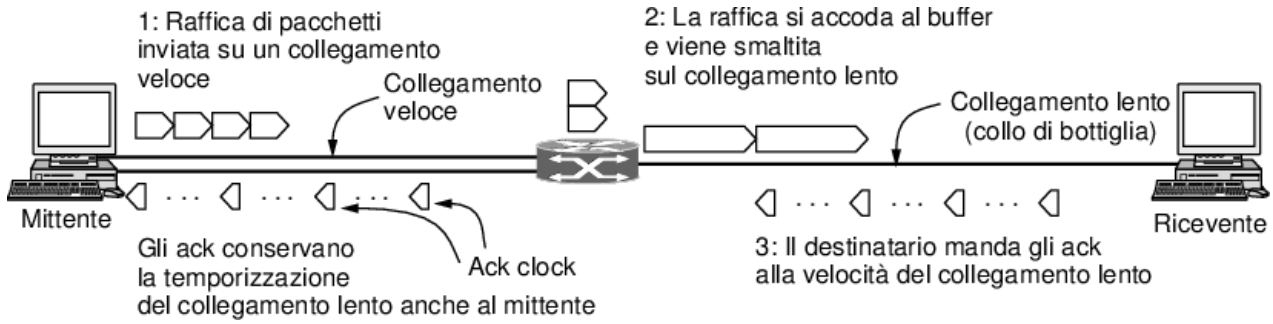


Figura 6.43 Una raffica di pacchetti da una sorgente e l'ack clock di ritorno.

Alla fine i pacchetti arrivano al destinatario, dove verrà generato un acknowledgement. Le tempistiche degli acknowledgement riflettono i tempi impiegati dai pacchetti per arrivare al destinatario dopo aver attraversato il collegamento più lento e risultano sparpagliati se confrontati ai pacchetti originali sul collegamento veloce. Nell'attraversare la rete per tornare al mittente gli acknowledgement mantengono inalterate queste tempistiche.

L'osservazione principale è questa: gli acknowledgement tornano al mittente a circa lo stesso tasso a cui i pacchetti possono essere spediti sul collegamento più lento lungo il percorso; questo è esattamente il tasso che il mittente desidera usare. Se inserisce nuovi pacchetti nella rete con questo tasso, essi saranno spediti alla velocità consentita dal collegamento più lento, ma non saranno messi in coda e non congestioneranno alcun router lungo il percorso. Questa temporizzazione è conosciuta come **ack clock** ed è una parte essenziale di TCP. Usando un ack clock, TCP rende più regolare il traffico ed evita code inutili ai router.

Una seconda considerazione è che una regola AIMD impiegherà molto tempo per ottenere un buon punto di funzionamento sulle reti veloci se la finestra di congestione parte da una dimensione piccola. Considerate un breve percorso di rete che può supportare 10 Mbps con un

round-trip time di 100 ms. La finestra di congestione appropriata è il prodotto banda-ritardo, cioè 1 Mbit o 100 pacchetti da 1250 byte ognuno. Se la finestra di congestione inizia con 1 pacchetto e aumenta di 1 pacchetto ogni round-trip time, ci vorranno 100 round-trip time o 10 secondi prima che la connessione viaggi all'incirca al giusto tasso di invio. È un tempo lungo da aspettare solo per raggiungere la giusta velocità di trasferimento. Possiamo ridurre questo tempo iniziale cominciando con una finestra più grande, ad esempio di 50 pacchetti. Questa finestra tuttavia sarà decisamente troppo grande per collegamenti lenti o brevi e causerebbe una congestione se venisse usata tutta insieme, proprio come abbiamo appena descritto.

Al contrario la soluzione scelta da Jacobson per gestire entrambe queste considerazioni è un insieme di aumento lineare e moltiplicativo. Quando viene stabilita una connessione, il mittente inizializza la finestra di congestione a un valore iniziale basso, di al massimo quattro segmenti; i dettagli sono descritti nell'RFC 3390, l'uso di quattro segmenti è un incremento, sulla base dell'esperienza, da un primo valore iniziale di un solo segmento. Il mittente, quindi, spedisce la finestra iniziale. I pacchetti impiegheranno un round-trip time per ottenere i relativi acknowledgement. Per ogni segmento che ha ricevuto l'acknowledgement prima dello scadere del timer di ritrasmissione il mittente aggiunge alla finestra di congestione il valore in byte di un segmento; in più, quando quel segmento ha ricevuto un acknowledgement ci sarà un segmento in meno in rete. Il risultato è che ogni acknowledgement, permette a due segmenti di essere inviati. La finestra di congestione raddoppia a ogni round-trip time.

Questo algoritmo è chiamato **slow start** (*partenza lenta*), ma non è lento per niente (ha una crescita esponenziale) tranne se confrontato con l'algoritmo precedente che permette a un'intera finestra di controllo di flusso di essere spedita in una volta. Lo slow start è illustrato nella Figura 6.44; nel primo round-trip time il mittente immette un pacchetto nella rete (e il destinatario riceve un pacchetto), due pacchetti sono spediti nel successivo round-trip time e quindi quattro nel terzo round-trip time.

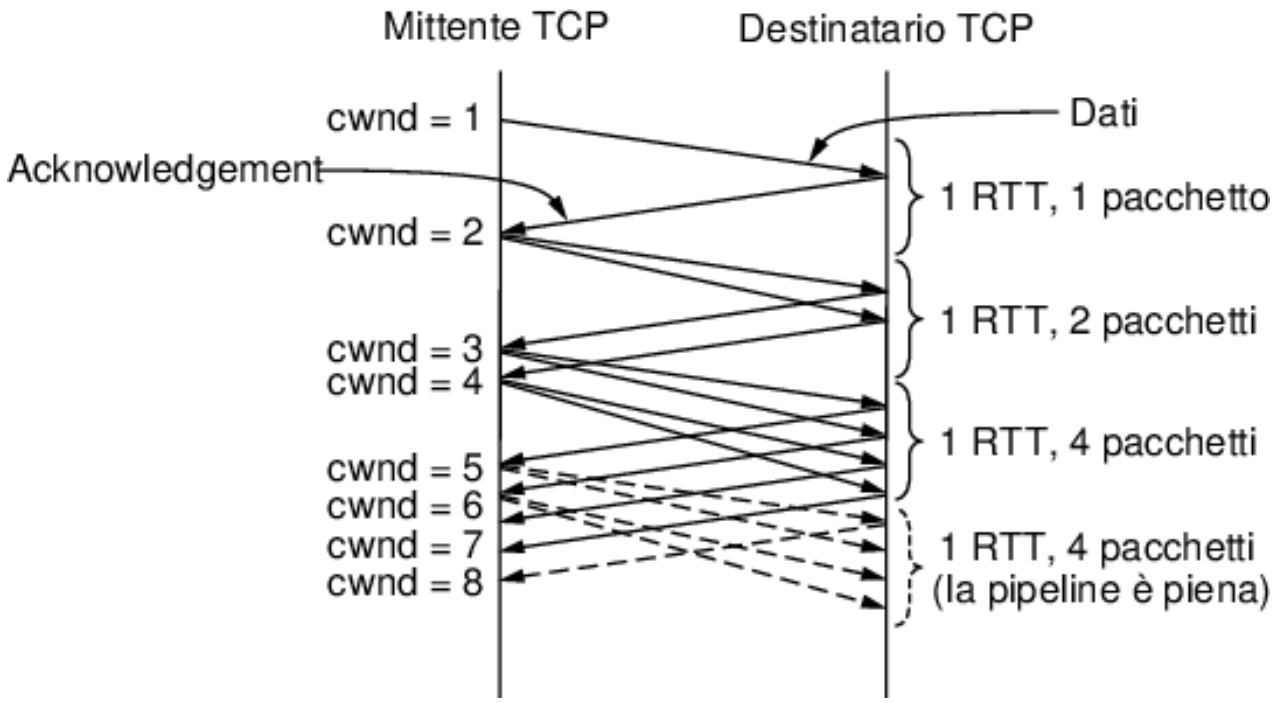


Figura 6.44 Slow start a partire da una finestra di congestione iniziale di un segmento.

Lo slow start funziona bene su una gamma di velocità e round-trip time e usa un ack clock per allineare il tasso delle trasmissioni del mittente al percorso di rete. Si guardi il modo in cui gli acknowledgement ritornano dal mittente al destinatario nella Figura 6.44. Quando un mittente riceve un acknowledgement aumenta la finestra di congestione di un'unità e immediatamente spedisce due pacchetti sulla rete: un pacchetto è per l'incremento, l'altro è un rimpiazzo per il pacchetto il cui acknoledgement è stato ricevuto; in ogni momento il numero di pacchetti senza acknowledgement è dato dalla finestra di congestione. Tuttavia questi due pacchetti non arriveranno necessariamente a destinazione così ravvicinati come quando sono stati spediti; supponiamo, per esempio, che il mittente sia su una Ethernet da 100 Mbps. Ogni pacchetto da 1250 byte impiega 100 ms per essere spedito, quindi il ritardo tra i pacchetti può essere al massimo 100 ms. La situazione cambia se questi pacchetti attraversano una linea ADSL da 1 Mbps da qualche parte lungo il percorso: ora ci vogliono 10 ms per spedire lo stesso pacchetto. Questo significa che lo spazio minimo tra i due pacchetti è cresciuto di un fattore 100; a meno che i pacchetti debbano aspettare insieme in coda su una linea successiva, il divario rimarrà ampio.

Nella Figura 6.44 questo effetto è mostato imponendo uno spazio minimo tra i pacchetti dati che arrivano al destinatario. Lo stesso spazio è mantenuto quando il destinatario spedisce gli acknowledgement e quindi anche quando il mittente li riceve. Se il percorso nella rete è lento, gli acknowledgement arrivano lentamente (dopo un ritardo di un RTT). Se il percorso di rete è veloce, gli acknowledgement arrivano velocemente (ancora, dopo un RTT). Tutto ciò che il mittente deve fare è seguire la tempistica dell'ack clock per inviare nuovi pacchetti, che è ciò che fa lo slow start.

Poiché lo slow start causa una crescita esponenziale, prima o poi (più prima che poi) spedirà in rete troppi pacchetti e troppo velocemente. Quando ciò avviene, si creano code nella rete e quando le code saranno piene uno o più pacchetti saranno persi. Dopo che questo accade, il mittente TCP andrà in timeout quando un acknowledgement non riuscirà ad arrivare in tempo. Nella Figura 6.44 viene data evidenza del fatto che lo slow start cresce troppo velocemente; dopo due RTT, ci sono quattro pacchetti in rete. Questi quattro pacchetti impiegano un intero RTT per arrivare al destinatario, quindi una finestra di quattro pacchetti ha la giusta dimensione per questa connessione.[7] Tuttavia, non appena vengono ricevuti gli acknowledgement di questi pacchetti, lo slow start continua a far crescere la finestra di congestione, raggiungendo gli otto pacchetti in un altro RTT. Solo quattro di questi pacchetti possono raggiungere il destinatario in un RTT, indipendentemente da quanti ne vengono spediti. In altre parole, la pipeline di rete è piena. Ulteriori pacchetti spediti in rete dal mittente si accumuleranno nelle code sui router, in quanto non potranno essere consegnati al destinatario abbastanza velocemente. Congestione e perdita di pacchetti sopraggiungeranno presto.
Per tenere sotto controllo lo slow start, il mittente mantiene una soglia per il collegamento chiamata **slow start threshold** (*soglia di slow start*). Inizialmente questo valore è fissato, in modo abitrariamente elevato, alla dimensione della finestra di controllo di flusso, in maniera tale da non limitare la connessione. TCP continua a far crescere la finestra di congestione nello slow start finché non si verifica un timeout o la finestra di congestione supera la soglia (o si è riempita la finestra del destinatario).

Ogni volta che viene rilevata la perdita di un pacchetto, per esempio attraverso un timeout, la soglia di slow start è fissata per essere la metà della finestra di congestione e l'intero processo ricomincia. L'idea è che la finestra corrente sia troppo grande perché ha già causato una congestione che solo ora viene rilevata da un timeout. Metà della finestra, usata con successo in un momento precedente, è probabilmente una stima migliore per una finestra di congestione vicina alla capacità del percorso, ma che non causerà perdite. Nel nostro esempio, nella Figura 6.44, la crescita della finestra di congestione a otto pacchetti può causare perdite, mentre la finestra di congestione di quattro pacchetti nel RTT precedente era il valore corretto. La finestra di congestione è quindi riportata al suo valore iniziale e lo slow start ricomincia.

Ogni volta che viene oltrepassata la soglia dello slow start, TCP passa dallo slow start all'incremento additivo; in questa modalità, la finestra di congestione viene aumentata di un segmento per ogni round-trip time. Questo, però, viene solitamente implementato con un aumento (più piccolo) per ogni segmento, per cui si riceve un acknowledgement invece di crescere a ogni RTT, come nel caso dello slow start. Indichiamo la finestra di congestione con *cwnd* e la grandezza massima di segmento con *MSS*. Un'approssimazione di uso comune consiste nell'aumentare *cwnd* di (*MSS* × *MSS*)/*cwnd* per ognuno dei *cwnd*/*MSS* pacchetti che potrebbero ricevere un acknowledgement. Questo incremento non ha bisogno di essere veloce; l'idea è che una connessione TCP passi molto tempo con la propria finestra di connessione vicina al valore ottimale (non così piccola da rendere il throughput basso e non così grande da provocare congestione).

L'incremento additivo è mostrato nella Figura 6.45 per la stessa situazione dello slow start. Alla fine di ogni RTT la finestra di congestione del mittente è cresciuta abbastanza per poter introdurre un pacchetto aggiuntivo in rete. Rispetto allo slow start, il tasso lineare di crescita è molto più lento. Non fa molta differenza per piccole finestre di congestione, come in questo caso, ma ne fa molta sul tempo impiegato per far crescere la finestra di congestione a 100 segmenti, ad esempio.

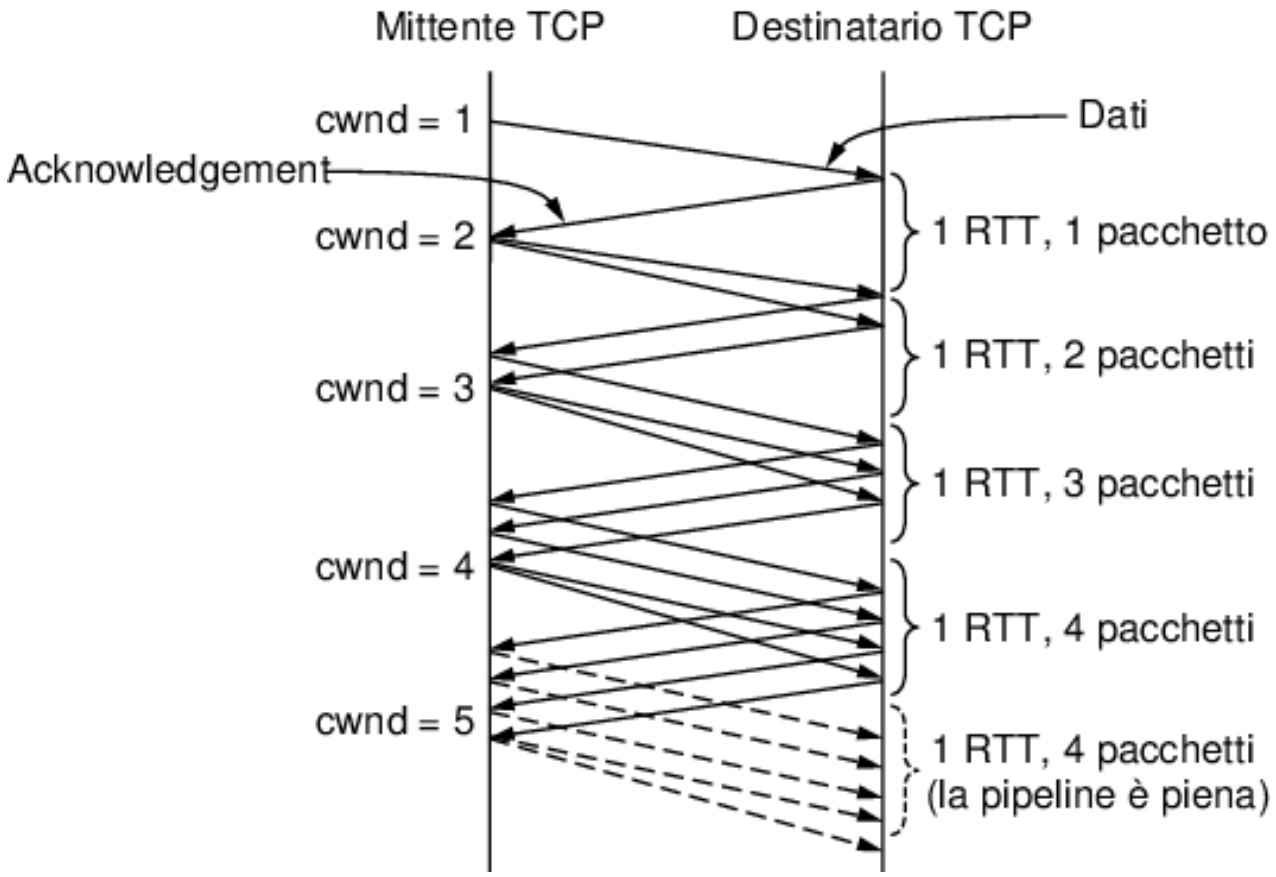


Figura 6.45 Incremento additivo a partire da una finestra di congestione iniziale di un segmento.

Per aumentare le prestazioni c'è ancora qualcos'altro che possiamo fare. Il difetto del sistema descritto finora è l'attesa di un timeout. I timeout sono relativamente lunghi per questioni di prudenza. Dopo che viene perso un pacchetto il destinatario non può mandare acknowledgement per quelli successivi, quindi il numero di acknowledgement rimarrà fisso e il mittente non sarà in grado di spedire alcun nuovo pacchetto sulla rete, perché la sua finestra di congestione rimarrà piena. Questa condizione può continuare per un periodo relativamente lungo finché scatta il timer e il pacchetto perso viene ritrasmesso. A questo punto TCP parte nuovamente con lo slow start.

Il mittente ha un modo veloce per riconoscere che uno dei suoi pacchetti è stato perso. Non appena i pacchetti successivi al pacchetto perduto arrivano a destinazione, innescano degli acknowledgement che ritornano al mittente. Questi acknowledgement portano lo stesso numero e sono chiamati **duplicate acknowledgement** (*acknowledgement duplicati*). Ogni volta che il mittente riceve un acknowledgement duplicato è probabile che un altro pacchetto sia arrivato al destinatario e il pacchetto perso non si sia ancora fatto vivo.

Poiché i pacchetti possono prendere diversi percorsi attraverso rete, possono arrivare fuori ordine generando acknowledgement duplicati anche se nessun pacchetto è stato perso. Tuttavia, nella maggior parte dei casi, questo non è comune su Internet; anche quando c'è un riordino dei pacchetti a causa di percorsi diversi, i pacchetti ricevuti non sono troppo fuori ordine. Quindi TCP assume in modo abbastanza arbitrario che tre acknowledgement duplicati implichino che un

pacchetto sia stato perso e l'identità del pacchetto perso si possa facilmente dedurre dal numero negli acknowledgement: è il prossimo pacchetto nella sequenza. Questo pacchetto può essere ritrasmesso immediatamente, prima che scatti il timeout di ritrasmissione.

Tale euristica prende il nome di **fast retransmission** (*ritrasmissione veloce*). Quando entra in funzione, la soglia di slow start viene di nuovo portata a metà della corrente finestra di congestione, proprio come con un timeout. Lo slow start può essere fatto ripartire portando la finestra di congestione a un pacchetto. Con questa dimensione della finestra, dopo il round-trip time necessario per mandare un acknowledgement relativo al pacchetto ritrasmesso, verrà spedito un nuovo pacchetto insieme a tutti i dati spediti prima del rilevamento della perdita.

Un'illustrazione dell'algoritmo di ritrasmissione che abbiamo creato fin qui è mostrata nella Figura 6.46. Questa versione di TCP è chiamata TCP Tahoe a seguito di 4.2BSD Tahoe,[8] rilasciata nel 1988 e in cui fu inclusa. La dimensione massima di un segmento nell'esempio è di 1 KB. All'inizio la finestra di congestione era 64 KB, ma si era verificato un timeout, quindi la soglia fu impostata a 32 KB e la finestra di congestione a 1 KB per la trasmissione 0. La finestra di congestione cresce esponenzialmente finché raggiunge la soglia (32 KB). Invece che in maniera continua, la finestra è incrementata ogni volta che arriva un acknowledgement, il che porta a un profilo con una serie di gradini. Superata la soglia la finestra aumenta linearmente e viene incrementata di un segmento ogni RTT.

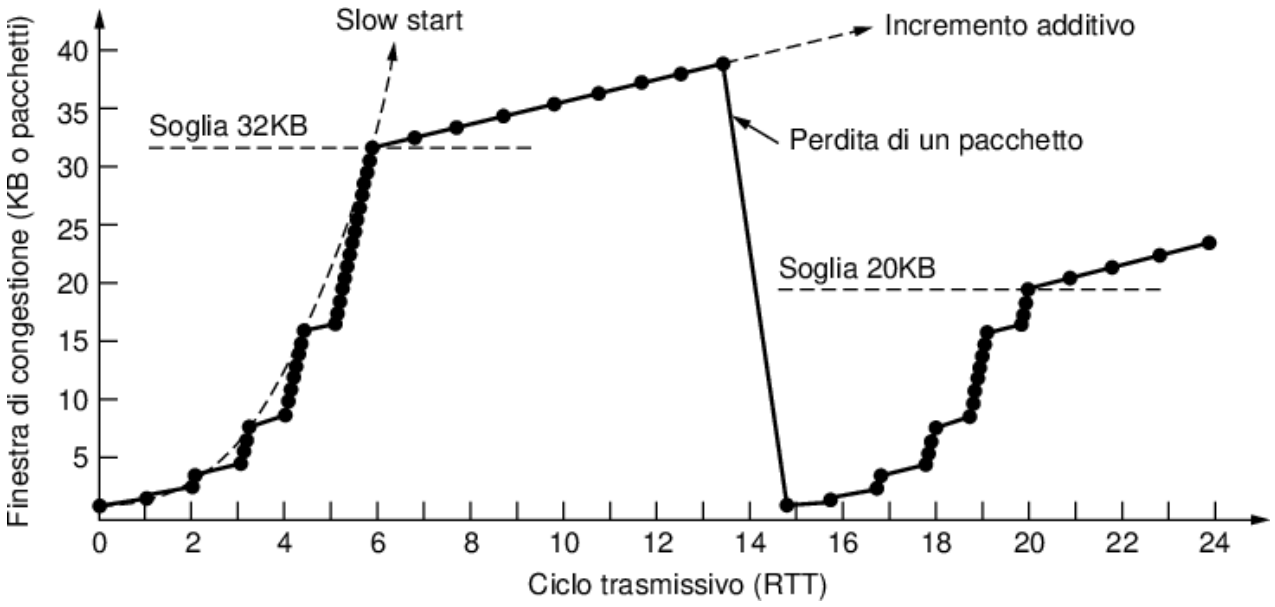


Figura 6.46 Slow start seguito da incremento additivo in TCP Tahoe.

Le trasmissioni al tredicesimo ciclo sono sfortunate (era da prevedere) e una di esse si perde nella rete. Questo fatto viene rilevato quando arrivano tre acknowledgement duplicati. In quel momento il pacchetto perso viene ritrasmesso, la soglia è portata a metà della finestra corrente (finora 40 KB, quindi la metà è 20 KB) e lo slow start comincia di nuovo, da capo. Ripartire con

una finestra di congestione di un pacchetto richiede un tempo di un round-trip time, perché tutti i precedenti dati trasmessi lascino la rete e ricevano un acknowledgement, incluso il pacchetto ritrasmesso. La finestra di congestione aumenta con lo slow start come ha fatto in precedenza, finché raggiunge la nuova soglia di 20 KB e l'incremento diventa nuovamente lineare. Continuerà in questo modo finché verrà rilevato un altro pacchetto perso attraverso acknowledgement duplicati o un timeout (o la finestra del destinatario diventa il limite).

TCP Tahoe (che include buoni timer di ritrasmissione) fornisce un algoritmo di controllo della congestione funzionante che risolve il problema del collasso per congestione. Jacobson capì che era possibile fare anche di meglio. Al momento della ritrasmissione veloce la connessione ha a disposizione una finestra di congestione troppo grande, ma anche un ack clock attivo. L'arrivo di ogni ulteriore acknowledgement duplicato vuol dire che un altro pacchetto ha lasciato la rete. Usare gli acknowledgement duplicati per contare i pacchetti in rete rende possibile spedire un nuovo pacchetto per ogni acknowledgement duplicato aggiuntivo.

Il **fast recovery** (*recupero veloce*) è l'euristica che implementa questo comportamento. È una modalità temporanea che cerca di mantenere l'ack clock in funzione con una finestra di congestione grande quanto la nuova soglia o nel momento della ritrasmissione veloce quanto metà del valore della finestra di congestione. Si rimane in modalità temporanea finché non ci si accorge che il numero di pacchetti in rete è sceso al di sotto della nuova soglia contando gli acknowledgement duplicati (inclusi i tre che hanno fatto scattare la ritrasmissione veloce); questa procedura impiega circa metà del round-trip time. Da quel momento in poi, un nuovo pacchetto può essere spedito per ogni acknowledgement duplicato ricevuto. Trascorso un round-trip time dopo la ritrasmissione veloce il pacchetto perduto avrà ricevuto un acknowledgement. A quel punto il flusso di acknowledgement duplicati cesserà e la modalità di fast recovery sarà abbandonata. La finestra di congestione sarà impostata alla nuova soglia di slow start e aumenterà con crescita lineare.

Il risultato di questa euristica è che TCP evita lo slow start, tranne all'inizio della connesione e quando si verifica un timeout. Quest'ultimo può ancora verificarsi quando va perso più di un pacchetto e la ritrasmissione veloce non recupera adeguatamente. Invece di slow start multipli, la finestra di congestione della connessione in atto segue un profilo a dente di sega per gli incrementi additivi (di un segmento ogni RTT) e i decrementi moltiplicativi (di metà in un RTT). Questa è esattamente la regola AIMD che cercavamo di implementare.

Tale andamento a dente di sega è mostrato nella Figura 6.47 ed è prodotto da TCP Reno, chiamato così a seguito di 4.3BSD Reno rilasciato nel 1990 e in cui era incluso. TCP Reno è essenzialmente TCP Tahoe con fast recovery. Dopo uno slow start iniziale, la finestra di congestione aumenta linearmente finché viene rilevata la perdita di un pacchetto per via di acknowledgement duplicati. Il pacchetto perso è ritrasmesso e il fast recovery è usato per mantenere attivo l'ack clock finché la ritrasmissione riceve un acknowledgement. A quel punto la finestra di congestione è riavviata dalla nuova soglia di slow start, invece che da 1. Questo comportamento continua all'infinito e la connessione passa la maggior parte del tempo con la sua finestra di congestione vicina al valore ottimale del prodotto banda-ritardo.

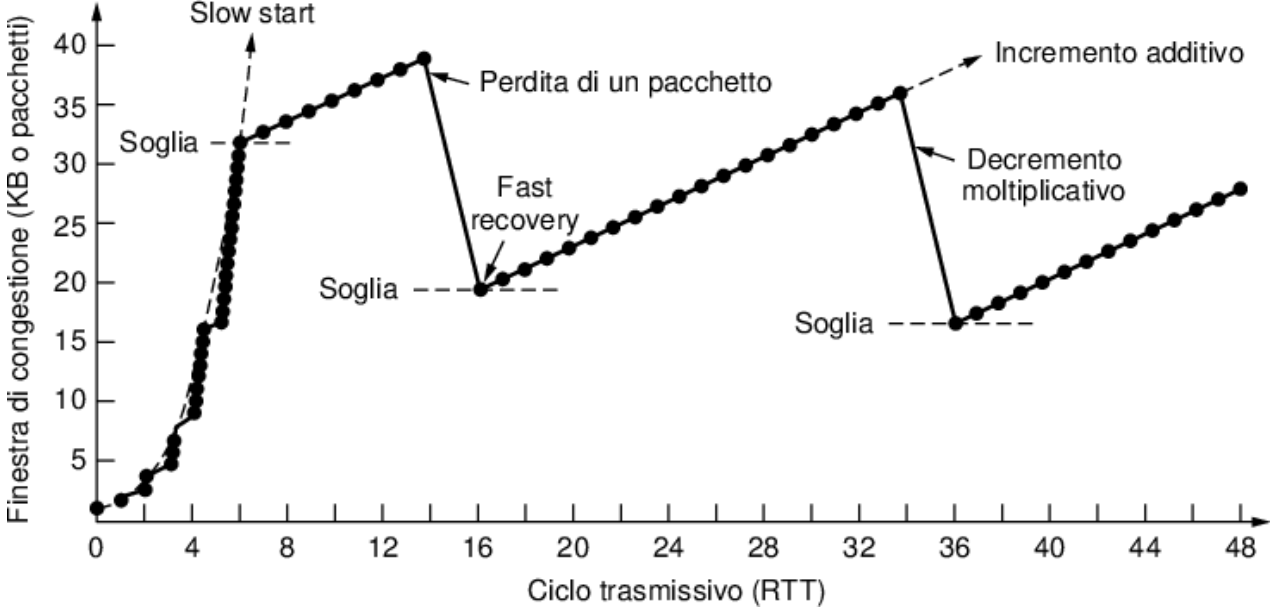


Figura 6.47 Fast recovery e sequenza a dente di sega di TCP Reno.

TCP Reno, con i suoi meccanismi di aggiustamento della finestra di congestione, ha costituito la base per il controllo di congestione di TCP per più di vent'anni. Molti dei cambiamenti negli anni successivi hanno modificato questi meccanismi in maniera minore cambiando, ad esempio, le scelte della finestra iniziale e rimuovendo varie ambiguità. Alcuni miglioramenti sono stati fatti per recuperare da due o più perdite in una finestra di pacchetti. Per esempio la versione TCP NewReno opera un avanzamento parziale del numero di acknowledgement dopo una ritrasmissione per trovare e rimediare a un'altra perdita (Hoe, 1996), come descritto nell'RFC 3782. A partire dalla metà degli anni '90 sono emerse molte varianti che seguono i principi descritti, ma che usano leggi di controllo leggermente diverse. Per esempio Linux usa una variante chiamata CUBIC TCP (Ha et al., 2008) e Windows include una variante chiamata Compound TCP (Tan et al., 2006).

Due grandi cambiamenti hanno ulteriormente influenzato le implementazioni di TCP. Primo, molta della complessità di TCP è la deduzione da un flusso di acknowledgement duplicati di quali pacchetti siano arrivati e quali andati persi; il numero di acknowledgement cumulativo non fornisce questa informazione. Una soluzione semplice è fare uso di **SACK** (*selective acknowledgement*), che elenca fino a tre intervalli di byte ricevuti. Con questa informazione il mittente può decidere più direttamente quali pacchetti ritrasmettere e tenere traccia dei pacchetti in viaggio per implementare la finestra di congestione.

Quando mittente e destinatario stabiliscono una connessione, ognuno invia l'opzione *SACK permitted* per segnalare che entrambi capiscono gli acknowledgement selettivi. Una volta abilitato, il SACK per una connessione, funziona come mostato nella Figura 6.48: un destinatario usa il campo TCP *Acknowledgement number* nel solito modo, come un acknowledgement cumulativo del più alto byte che sia stato ricevuto in ordine fino a quel momento. Quando riceve il pacchetto 3 fuori sequenza (perché il pacchetto 2 è stato perso), spedisce una *SACK option* per i dati ricevuti insieme all'acknowledgement cumulativo (duplicato) del pacchetto 1. La *SACK option* fornisce gli intervalli di byte ricevuti e superiori al numero dato attraverso l'acknowledgement cumulativo. Il primo intervallo è il pacchetto che ha innescato l'acknowledgement duplicato. Gli intervalli

successivi, se presenti, sono blocchi più vecchi e vengono usati comunemente fino a tre intervalli. Quando viene ricevuto il pacchetto 6, due intervalli di byte SACK vengono usati per indicare che il pacchetto 6 e i pacchetti da 3 a 4 sono stati ricevuti, in aggiunta a tutti i pacchetti fino al pacchetto 1. Dall'informazione di ciascuna *SACK option* che riceve, il mittente può decidere quali pacchetti ritrasmettere. In questo caso ritrasmettere i pacchetti 2 e 5 sarebbe una buona idea.

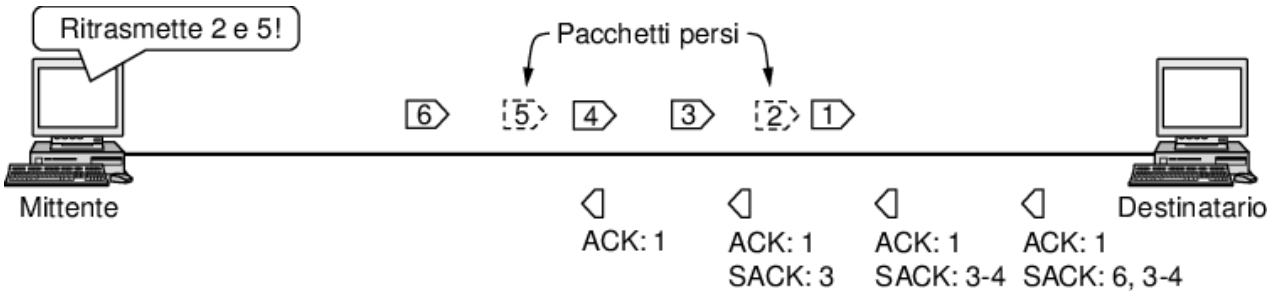


Figura 6.48 Acknowledgement selettivi.

SACK è strettamente un'informazione di avvertimento. Il rilevamento effettivo delle perdite utilizzando acknowledgement duplicati e adeguamenti della finestra di congestione procede esattamente come prima. Tuttavia, con SACK, TCP può uscire più facilmente da situazioni in cui più pacchetti vengono persi più o meno nello stesso momento, dato che il mittente TCP sa quali pacchetti non sono stati ricevuti. SACK è ora ampiamente utilizzato ed è descritto nell'RFC 2883, mentre il controllo della congestione di TCP tramite SACK è descritto nell'RFC 3517.

Il secondo cambiamento riguarda l'uso di **ECN** (*explicit congestion notification*) oltre alla perdita di pacchetti come segnale di congestione. ECN è un meccanismo di livello IP per notificare agli host una congestione descritta nel Paragrafo 5.3.4. Con esso il destinatario TCP può ricevere il segnale di congestione da IP.

L'uso di ECN è abilitato per una connessione TCP quando sia mittente sia destinatario indicano di essere in grado di usare ECN impostando i bit *ECE* e *CWR* durante la creazione della connessione. Se viene usato ECN, ogni pacchetto che trasporta un segmento TCP viene contrassegnato nell'intestazione IP per segnalare che può portare una segnalazione ECN. I router che supportano ECN imposteranno un segnale di congestione sui pacchetti che possono trasportare indicatori ECN quando si sta avvicinando una congestione, invece di scartarli dopo che la congestione ha avuto luogo.

Il destinatario TCP viene avvertito se qualche pacchetto in arrivo porta un segnale di congestione ECN. Il destinatario quindi usa l'indicatore **ECE** (*ECN echo*) per segnalare al mittente TCP che i suoi pacchetti hanno sperimentato della congestione. Il mittente avverte il destinatario di aver sentito il segnale usando l'indicatore **CWR** (*congestion window reduced*, finestra di congestione ridotta).

Il mittente TCP reagisce a questa notifica di congestione esattamente come alla perdita di pacchetti individuata tramite acknowledgement duplicati. Tuttavia la situazione è decisamente migliore: la congestione è stata individuata e nessun pacchetto è stato danneggiato in alcun modo. ECN è descritto nell'RFC 3168, richiede di essere supportato sia da host che da router e non è ancora largamente diffuso su Internet.

Per maggiori informazioni su tutti i comportamenti di controllo di congestione implementati in TCP si consulti l'RFC 5681.

6.5.11 Il futuro di TCP

TCP è stato usato per molte applicazioni ed esteso nel tempo per dare buone prestazioni su un'ampia gamma di reti. Sono molte le versioni adottate, anche se con implementazioni leggermente diverse rispetto agli algoritmi classici che abbiamo descritto, specialmente per quanto riguarda il controllo di congestione e la robustezza contro gli attacchi. È probabile che TCP continuerà a evolvere insieme a Internet. Menzioneremo ora peró due dei suoi problemi principali.

Il primo problema è che TCP non fornisce le semantiche di trasporto che tutte le applicazioni vogliono. Per esempio alcune di esse vogliono spedire messaggi o record i cui confini devono essere preservati. Altre funzionano con un gruppo di conversazioni correlate, come un Web browser che trasferisce molti oggetti dallo stesso server. Altre ancora puntano su un miglior controllo sui percorsi di rete che usano. TCP con la sua interfaccia socket standard non soddisfa molto queste necessità. Essenzialmente, l'applicazione ha l'onere di gestire qualsiasi problema non risolto da TCP. Questo ha portato a proporre nuovi protocolli che forniscono un'interfaccia leggermente diversa. Due esempi sono **SCTP** (*stream control transmission protocol*), definito nell'RFC 4960 e **SST** (*structured stream transport*) (Ford, 2007). Tuttavia ogni qual volta qualcuno propone di cambiare qualcosa che ha funzionato così bene e tanto a lungo, c'è sempre una grande battaglia tra le due fazioni "Gli utenti chiedono più funzioni" e "Se non è rotto, non lo aggiustare".

Il secondo problema è il controllo di congestione. Dopo le nostre lunghe discussioni e i meccanismi che sono stati sviluppati nel tempo vi sareste forse aspettati che questo fosse un problema risolto. Non è così. La forma di controllo della congestione di TCP descritta e ampiamente in uso è basata sulla perdita di pacchetti come segnale di congestione. Quando Padhye et al. (1998) hanno realizzato un modello del throughput di TCP basato sulla sequenza a dente di sega, scoprirono come il tasso di perdita di pacchetti dovesse scendere rapidamente con l'aumento della velocità. Per raggiungere un throughput di 1 Gbps con un round-trip time di 100 ms e pacchetti da 1500 byte un pacchetto può essere perso approssimativamente ogni 10 minuti: quindi con un tasso di perdita di pacchetti di 2×10^{-8}, che è incredibilmente basso. Sono eventi troppo poco frequenti per essere un buon segnale di congestione e qualsiasi altra causa di perdita (come un tasso di errori di trasmissione di 10^{-7}) può diventare dominante, limitanto il throughput.

Questa relazione non è stata un problema in passato, ma le reti stanno diventando sempre più veloci, obbligando molti ricercatori a rivedere il controllo di congestione. Una possibilità è di usare un controllo di congestione alternativo in cui il segnale che indica la congestione non è per nulla legato ai pacchetti persi; ne abbiamo dato alcuni esempi nel Paragrafo 6.2. Il segnale di

congestione potrebbe essere legato al round-trip time (visto che aumenta al crescere della congestione di rete) come in FAST TCP (Wei et al., 2006). Altri approcci sono possibili, solo il tempo potrà dire quale sarà il migliore.

6.6 Problemi di prestazioni

I problemi che riguardano le prestazioni sono molto importanti nelle reti di calcolatori. Quando centinaia o migliaia di computer sono connessi, si verificano spesso interazioni complesse con conseguenze impreviste. Spesso questa complessità porta a pessime prestazioni e nessuno sa il perché. Nei paragrafi seguenti verranno esaminate molte questioni che riguardano le prestazioni di rete per vedere quali tipi di problemi esistono e che cosa è possibile fare per risolverli.

Sfortunatamente la comprensione delle prestazioni di rete è più un'arte che una scienza. La scarsa teoria di base non ha quasi alcun valore nella pratica. Il meglio che si possa fare è fornire alcune regole generali acquisite con l'esperienza e presentare esempi presi dal mondo reale. Abbiamo ritardato questa discussione fino a dopo lo studio del livello di trasporto perché le prestazioni che le applicazioni ottengono dipendono dalle prestazioni combinate dei livelli di trasporto, rete e data link, nonché dal fatto di riuscire a usare TCP come esemplificazione in vari casi.

Nei prossimi paragrafi vedremo sei aspetti delle prestazioni di rete:

1. problemi di prestazioni
2. misurazione delle prestazioni di rete
3. progettazione di host per reti veloci
4. elaborazione rapida dei segmenti
5. compressione dell'intestazione
6. protocolli per reti ad alte prestazioni.

Questi aspetti considerano le prestazioni di rete sia sull'host sia attraverso la rete, nonché come le reti siano cresciute in velocità e dimensione.

6.6.1 Problemi di prestazioni nelle reti di calcolatori

Alcuni problemi riguardanti le prestazioni, come la congestione, sono provocati da sovraccarichi temporanei delle risorse. Se improvvisamente a un router giunge una quantità di traffico superiore a quella che può gestire, si verifica una congestione che influisce sulle prestazioni.

Le prestazioni degradano anche in presenza di uno squilibrio strutturale di risorse. Per esempio, se una linea di comunicazione con capacità di un gigabit è connessa a un PC dalle basse prestazioni, il povero host non sarà in grado di elaborare i pacchetti in ingresso abbastanza velocemente e ne perderà qualcuno. Questi pacchetti saranno eventualmente ritrasmessi, con conseguente incremento di ritardi, spreco di banda e in generale riduzione delle prestazioni.

I sovraccarichi si possono innescare anche in modo sincrono. Per esempio, se un segmento contiene un parametro errato (come la porta a cui è destinato), in molti casi il ricevente restituirà diligentemente una notifica di errore. Ora si pensi a che cosa potrebbe accadere se un segmento errato fosse trasmesso a 1000 macchine: ognuna potrebbe restituire un messaggio di errore. La **broadcast storm** (*tempesta di broadcast*) risultante potrebbe paralizzare la rete. UDP ha sofferto per questo problema finché il protocollo ICMP non è stato modificato in modo che gli host si

astenessero dal rispondere agli errori nei segmenti UDP inviati a indirizzi broadcast. Poiché la trasmissione avviene naturalmente in broadcast e la banda wireless è limitata, le reti wireless devono stare particolarmente attente per evitare risposte broadcast incontrollate.

Un secondo esempio di sovraccarico sincrono è quello che si verifica dopo una mancanza di energia elettrica. Al rispristino dell'alimentazione tutti i computer iniziano contemporaneamente il riavvio. Una tipica sequenza di riavvio potrebbe richiedere innanzitutto di interpellare un server (DHCP) per conoscere la propria vera identità e poi qualche file server per ottenere una copia del sistema operativo. Se centinaia di macchine svolgessero questa operazione contemporaneamente, il server probabilmente collasserebbe sotto il carico di lavoro.

Anche in assenza di sovraccarichi sincroni e in presenza di risorse sufficienti si potrebbero ottenere prestazioni scadenti a causa della scarsa ottimizzazione del sistema. Per esempio, se un computer ha una CPU potente e molta memoria, ma non è stata allocata memoria sufficiente per i buffer, il controllo di flusso rallenterà la ricezione dei segmenti e limiterà le prestazioni. Questo era il problema di molte connessioni TCP quando Internet divenne più veloce, mentre la dimensione predefinita della finestra di controllo di flusso rimase fissata a 64 KB.

Un altro problema di ottimizzazione riguarda l'impostazione corretta dei timeout. Quando viene inviato un segmento, viene normalmente avviato un timer per rilevarne la perdita. Se il timeout è troppo breve si verificheranno ritrasmissioni inutili che intaseranno i collegamenti. Se i timeout sono troppo lunghi, ci saranno invece ritardi inutili dopo la perdita di un segmento. Altri parametri regolabili comprendono il tempo di attesa di ulteriori dati prima di inviare un acknowledgement separato e il numero di ritrasmissioni dopo le quali abbandonare i tentativi.

Un altro problema relativo alle prestazioni, che interessa le applicazioni real-time come quelle che trasmettono audio e video, è il jitter. Avere mediamente abbastanza banda non è sufficiente per avere buone prestazioni; sono anche richiesti bassi ritardi di trasmissione. Ottenere coerentemente bassi ritardi richiede un attento studio del carico di rete o il supporto per la qualità di servizio ai livelli data link e di rete, o entrambi.

6.6.2 Misurazione delle prestazioni di rete

Quando le prestazioni di una rete sono scadenti, gli utenti si lamentano con gli amministratori chiedendo miglioramenti. Per migliorare le prestazioni, gli operatori devono prima determinare esattamente che cosa sta accadendo e per farlo devono eseguire delle misurazioni. Questo paragrafo esamina la misura delle prestazioni di rete. La discussione seguente è basata sul lavoro di Mogul (1993).

Le misurazioni possono essere fatte in diversi modi e in molti punti (sia nello stack di protocolli sia fisicamente). Il modo più semplice di effettuare una misurazione è avviare un timer quando inizia qualche attività e vedere quanto tempo impiega. Per esempio, sapere quanto tempo impiega un segmento per ricevere un acknowledgement è una delle misurazioni principali. Altre misurazioni sono fatte con contatori che registrano quanto spesso accadono alcuni eventi (come il numero dei segmenti persi). Infine qualcuno è spesso interessato a sapere la quantità di qualcosa, come il numero di byte elaborati in un dato intervallo di tempo.

La misurazione dei parametri e delle prestazioni di rete nasconde molti tranelli. Di seguito ne sono elencati alcuni. Qualsiasi tentativo sistematico di misurare le prestazioni di rete dovrebbe cercare attentamente di evitarli.

Assicurarsi che la dimensione del campione sia sufficientemente elevata

Non bisogna misurare il tempo di invio di un solo segmento, ma ripetere la misurazione per un milione di volte e calcolare la media. Gli effetti dovuti alla partenza, come nel caso di una scheda di rete 802.16 o cable modem che ottiene un'ampiezza di banda riservata dopo un periodo di inattività, possono rallentare il primo segmento, e gli accodamenti possono introdurre variabilità. Un campione grande ridurrà l'incertezza nella media e nella deviazione standard misurate. Questa incertezza può essere calcolata utilizzando formule statistiche standard.

Assicurarsi che i campioni siano rappresentativi

Idealmente, l'intera sequenza di un milione di misurazioni dovrebbe essere ripetuta in diversi momenti del giorno e della settimana per vedere l'effetto delle diverse condizioni di rete sulla quantità misurata. La misurazione della congestione, per esempio, è di poco conto se viene fatta in un momento in cui non ci sono congestioni. A volte i risultati possono sembrare poco intuitivi inizialmente, per esempio pesanti congestioni alle undici del mattino e all'una del pomeriggio, ma nessuna a mezzogiorno (quando tutti sono a pranzo).

Con le reti wireless la posizione è un variabile importante a causa della propagazione di segnale. Persino un nodo di misurazione posto vicino a un client wireless può non osservare gli stessi pacchetti del client a causa di differenze nelle antenne. È meglio prendere le misurazioni dal client wireless in esame per sapere che cosa vede. Se questo non è possibile si possono usare tecniche per combinare misurazioni wireless prese da diversi punti di vista per ottenere un'immagine più dettagliata di ciò che sta succedendo (Mahajan et al., 2006).

Le cache possono portare il caos nelle misurazioni

Ripetere una misurazione più volte restituirà una risposta inaspettatamente veloce se il protocollo fa uso di cache. Per esempio, reperire una pagina Web o risolvere un nome tramite DNS (per trovare l'indirizzo IP) può comportare uno scambio di dati in rete la prima volta e successivamente far restituire la risposta da una cache locale senza spedire alcun pacchetto in rete. I risultati di una misurazione del genere sono essenzialmente inutili (a meno che vogliate misurare la performance della cache).

L'uso di buffer può avere un effetto simile. Test di prestazioni di TCP/IP erano noti riportare che UDP poteva raggiungere prestazioni sostanzialmente maggiori di quelle permesse dalla rete. Come poteva succedere? Una chiamata a UDP restituiva normalmente il controllo non appena il messaggio veniva accettato dal kernel e aggiunto alla coda di trasmissione. Se esiste spazio sufficiente nel buffer, la temporizzazione di 1000 chiamate UDP non indica la spedizione di tutti i dati. La maggior parte potrebbe trovarsi ancora nel kernel, ma il programma che misura le prestazioni pensa che siano stati tutti trasmessi.

Assicurarsi che non accada nulla di imprevisto durante i test

Eseguire misurazioni nel momento in cui un utente ha deciso di fare una videoconferenza sulla vostra rete darà spesso risultati diversi rispetto a se non ci fosse una videoconferenza. È preferibile eseguire i test su un sistema non utilizzato e creare noi stessi l'intero carico di lavoro della rete. Anche questo approccio nasconde però dei tranelli. Si potrebbe pensare che nessuno usi la rete alle 3 del mattino, ma quello potrebbe essere il momento in cui i programmi di backup automatico iniziano a copiare tutti i dischi su nastro. Oppure ci potrebbe essere un pesante traffico verso le vostre stupende pagine Web proveniente da zone con fusi orari diversi.

Le reti wireless sono difficoltose da trattare in questo senso, perché spesso non è possibile separarle da tutte le fonti di interferenza. Anche in assenza di altre reti wireless nelle vicinanze che spediscono traffico, qualcuno potrebbe cuocere dei popcorn col microonde e inavvertitamente causare un'interferenza che degrada le prestazioni di 802.11. Per queste ragioni è buona norma monitorare l'intera attività della rete così che si possa almeno capire quando accade qualcosa di inaspettato.

Fare attenzione quando si usa un clock poco preciso

Gli orologi dei computer funzionano incrementando un contatore a intervalli regolari. Per esempio, un timer in millisecondi aggiunge 1 a un contatore ogni 1 ms. Utilizzare tale timer per misurare un evento che avviene in meno di 1 ms è possibile, ma l'operazione richiede attenzione. Naturalmente ci sono computer che hanno orologi più accurati, ma ci sono sempre eventi da misurare più brevi del ciclo di clock. Si noti che non sempre gli orologi sono accurati con la stessa precisione con cui viene dato il valore del tempo quando sono letti.

Per misurare il tempo per stabilie una connessione TCP, per esempio, il clock di sistema (supponiamo in millisecondi) dovrebbe essere letto all'ingresso nel codice del livello di trasporto e all'uscita da quest'ultimo. Se l'effettivo tempo per stabilire la connessione è 300 ms, la differenza tra le due letture sarà 0 o 1 (entrambe errate). Tuttavia, se la misurazione viene ripetuta un milione di volte e il totale di tutte le misurazioni sommato e diviso per un milione, il tempo medio sarà più accurato di 1 ms.

Prestare attenzione nel generalizzare i risultati

Si supponga di eseguire delle misurazioni con carichi di rete simulati da 0 (non utilizzata) a 0,4 (40% della capacità). Per esempio, il tempo di risposta per restituire un pacchetto VoIP su una rete 802.11 può essere mostrato dai punti uniti dalla linea continua nella Figura 6.49. Si potrebbe essere tentati di estrapolarla linearmente, come mostrato dalla linea punteggiata. Tuttavia, molti risultati ottenuti tramite la teoria delle code coinvolgono un fattore di $1/(1 - r)$, dove r è il carico, quindi il vero risultato potrebbe assomigliare molto più alla linea tratteggiata, che sale più veloce che non linearmente al crescere del carico. Quindi, fate attenzione agli effetti dovuti alla contesa, che diventano molto più pronunciati con carichi elevati.

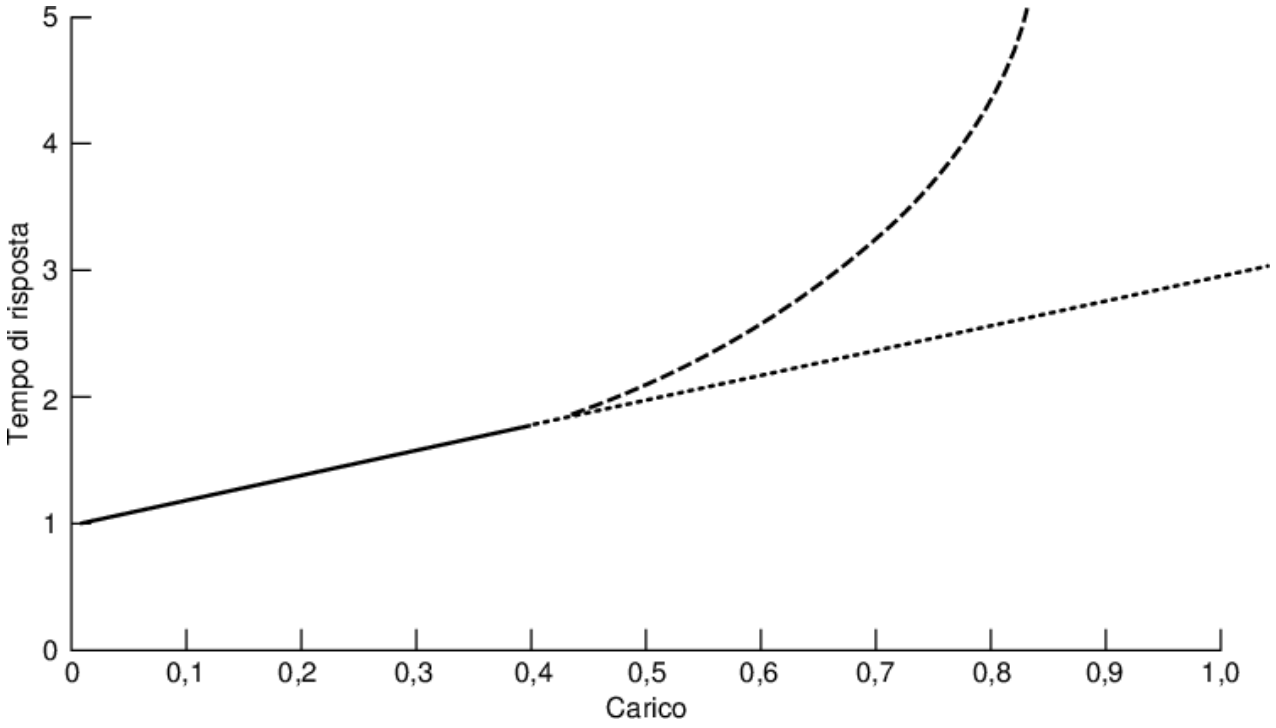


Figura 6.49 La risposta come funzione del carico.

6.6.3 Struttura degli host per reti veloci

Misurare e operare piccoli aggiustamenti può migliorare notevolmente le prestazioni, ma non può sostituire una valida progettazione iniziale. Una rete progettata in modo scadente può essere migliorata solo relativamente. In effetti, dovrebbe essere riprogettata da zero.

In questo paragrafo presenteremo alcune indicazioni pratiche per l'implementazione software di protocolli di rete sugli host. Sorprendentemente l'esperienza mostra che spesso l'implementazione sull'host è un collo di bottiglia per le prestazioni di reti altrimenti veloci per due ragioni. Primo, le interfacce di rete e i router sono già stati progettati (con supporto hardware) per funzionare alla velocità del mezzo trasmissivo. Questo significa che possono elaborare pacchetti tanto velocemente quanto questi possono eventualmente arrivare dalla linea. Secondo, le prestazioni che interessano sono quelle osservate dalle applicazioni (il throughput e il ritardo dopo le elaborazioni di rete e trasporto) e non la capacità della linea.

Ottimizzare il software migliora le prestazioni incrementando il throughput e diminuendo il ritardo. Inoltre, può anche ridurre l'enegia impiegata dalla rete, che è un fattore importante per i computer portatili. La maggior parte di queste idee sono da anni ben note ai progettisti di rete e sono state elencate in modo esplicito da Mogul (1993); la descrizione del prossimo paragrafo segue sostanzialmente la sua. Un'altra fonte rilevante è (Metcalfe, 1993).

La velocità dell'host è più importante di quella della rete

Una lunga esperienza ha dimostrato che, in quasi tutte le reti veloci, l'overhead (il carico di lavoro aggiuntivo) del sistema operativo e dei protocolli risulta dominante rispetto al tempo effettivo speso per la propagazione nel cavo. Per esempio, in teoria il tempo minimo per una RPC (*chiamata a procedura remota*) su una Ethernet da 1 Gbps è 1 ms, corrispondenti a una richiesta minima (512 byte) seguita da una risposta minima (512 byte). In pratica, a causa dell'inefficienza del software, ottenere un valore simile per una RPC è una conquista sensazionale e raramente capita di riuscirci.

Allo stesso modo, il problema maggiore quando si lavora a 1 Gbps riguarda come estrarre i bit dal buffer dell'utente per mandarli sulla rete abbastanza velocemente e come permettere al destinatario di elaborarli alla stessa velocità con cui arrivano. Raddoppiando la velocità dell'host (CPU e memoria) è spesso possibile raddoppiare il throughput. Al contrario, raddoppiando la capacità di rete non si ottengono effetti perché il collo di bottiglia si trova negli host.

Ridurre il numero di pacchetti per ridurre l'overhead

Ogni segmento ha una certa quantità di overhead (come l'intestazione) e di dati (come payload). La banda è richiesta per entrambi i componenti come pure è richiesta l'elaborazione (come l'elaborazione dell'intestazione e il calcolo del checksum). Quando vengono spediti 1 milione di byte il costo dei dati è lo stesso, non importa la dimensione del segmento. Tuttavia, usare un segmento da 128 byte significa avere un overhead 32 volte superiore per segmento che non se si usassero segmenti da 4 KB. Gli overhead di banda ed elaborazione di sovrappongono velocemente e riducono il throughput.

L'overhead di ogni singolo pacchetto nei livelli inferiori amplifica questo effetto. Ogni pacchetto in arrivo causa un nuovo interrupt nel sistema operativo se l'host sta tenendo il passo. In un moderno processore dotato di pipeline ogni interruzione spezza la pipeline della CPU, interferisce con la cache, richiede un cambiamento del contesto di gestione della memoria, annulla la tabella di predizione dei salti e costringe un consistente numero di registri CPU a essere salvati. Una riduzione di un fattore *n* del numero dei segmenti spediti riduce quindi l'interruzione e l'overhead dei pacchetti dello stesso fattore.

È possibile dire che sia le persone sia i computer sono deboli nel multitasking; questa osservazione sottolinea il desiderio di spedire pacchetti con MTU della grandezza giusta per passare senza frammentazione sui percosi di rete. Meccanismi come l'algoritmo di Nagle e la soluzione di Clark sono anche tentativi per evitare di spedire pacchetti piccoli.

Ridurre la copia dei dati in memoria

Il modo più semplice per implementare uno stack di protocolli è tramite un modulo software per ogni livello. Sfortunatamente, questo porta a duplicare il codice (o al massimo ad accedere ai dati in più passaggi) in quanto ogni livello svolge il proprio compito. Per esempio, dopo che un pacchetto è stato ricevuto dalla scheda di rete, di solito viene copiato in un buffer del kernel; da lì viene copiato in un buffer di livello di rete per essere elaborato dal livello di rete, poi in un buffer di livello di trasporto per essere elaborato dal livello di trasporto e alla fine nel processo destinatario. Non è insolito per un pacchetto in arrivo essere copiato tre o quattro volte prima che il segmento racchiuso in esso venga consegnato.

Tutte queste copie possono degradare enormemente le prestazioni, perché le operazioni in memoria sono un ordine di grandezza più lente di quelle tra registri. Per esempio, se il 20% delle istruzioni vanno in memoria (cioè, ci sono dei *cache miss*), il che è probabile quando si copiano dei pacchetti da una parte all'altra della memoria, il tempo medio di esecuzione delle istruzioni è rallentato di un fattore di 2,8 (0,8 × 1 + 0,2 × 10). L'hardware qui non ci aiuterà: il problema è effettuare troppe copie da parte del sistema operativo.

Un sistema operativo intelligente minimizzerà le operazioni di copia combinando le elaborazioni di più livelli. Per esempio, TCP e IP sono di solito implementati congiuntamente (come "TCP/IP") quindi non sarà necessario copiare il payload del pacchetto quando l'elaborazione passa dal livello di rete a quello di trasporto. Un altro trucco comune è fare più operazioni all'interno di un livello in un solo passaggio sui dati. Per esempio, il checksum è spesso calcolato durante le operazioni di copia dei dati (quando devono essere copiati) e il checksum appena calcolato viene aggiunto in coda.

Minimizzare i cambi di contesto

Una regola collegata al discorso appena fatto è che i cambi di contesto nel sistema operativo (ad esempio, passare dalla modalità kernel alla modalità utente) sono mortali: combinano le cattive proprietà sia degli interrupt sia della copia. Questo costo è il motivo per cui spesso i protocolli di trasporto sono implementati nel kernel. Come nel caso della riduzione del numero di pacchetti, i cambi di contesto possono essere ridotti richiedendo che la procedura di libreria che spedisce dati faccia uso di un buffer interno finché non avrà un quantitativo consistente di dati. Allo stesso modo, nel lato ricevente, piccoli segmenti in ingresso dovrebbero essere raccolti e passati all'utente in blocco anziché singolarmente, in modo da ridurre i cambi di contesto.

Nel migliore dei casi, un pacchetto in arrivo provoca un cambio di contesto dall'utente corrente al kernel e un passaggio al processo ricevente per fornirgli i dati appena arrivati. Sfortunatamente, con alcuni sistemi operativi avvengono ulteriori cambi di contesto. Per esempio, se il gestore della rete viene eseguito come un processo speciale in spazio utente, l'arrivo di un pacchetto è facile che provochi un cambio di contesto dall'utente corrente al kernel, poi un altro dal kernel al gestore di rete, seguito da un altro di ritorno al kernel e infine da un passaggio dal kernel al processo ricevente. Questa sequenza è mostrata nella Figura 6.50. Tutti questi cambi di contesto eseguiti provocano sprechi del tempo di CPU e hanno un effetto devastante sulle prestazioni della rete.

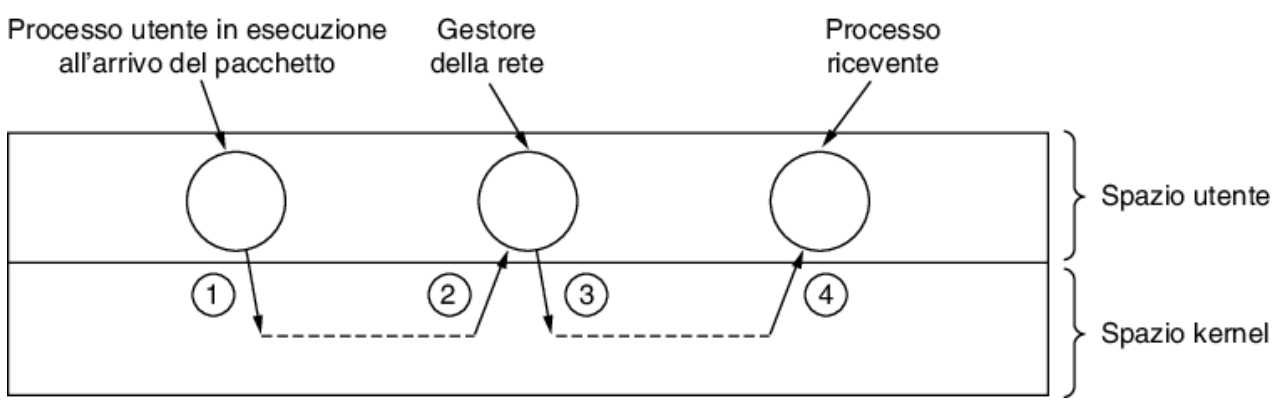


Figura 6.50 Quattro cambi di contesto per gestire un pacchetto con un gestore di rete nello spazio utente.

Evitare la congestione è meglio che porvi rimedio

Il vecchio detto per cui "prevenire è meglio che curare" vale certamente anche per la congestione di rete. Quando una rete è congestionata i pacchetti vengono persi, la banda sprecata, si introducono ritardi inutili e così via. Il recupero dalle congestioni richiede tempo e pazienza: è quindi preferibile evitarle del tutto. Evitare le congestioni è come vaccinarsi: fa un po' male all'inizio, ma impedisce che qualcosa faccia ancora più male in futuro.

Evitare i timeout

I timer sono necessari nelle reti, ma dovrebbero essere utilizzati raramente; anche i timeout dovrebbero essere minimizzati. Alla scadenza di un timer viene generalmente ripetuta qualche azione. Se è realmente necessario ripetere l'azione è giusto farlo, ma ripeterla senza necessità è uno spreco di risorse.

La strada per evitare lavoro extra consiste nell'assicurarsi di regolare i timer in modo leggermente accorto. Un timer che per scadere impiega troppo tempo aggiunge un ritardo extra a una connessione nell'evento (improbabile) che un segmento vada perso. Un timer che scade quando non dovrebbe utilizza delle risorse sull'host, spreca banda e impone un carico extra su dozzine di router senza una valida ragione.

6.6.4 Elaborazione rapida dei segmenti

Ora che abbiamo trattato le regole generali, analizzeremo alcuni metodi specifici per aumentare la velocità di elaborazione dei segmenti. Per maggiori informazioni si vedano Clark et al. (1989) e Chase et al. (2001).

L'overhead di elaborazione dei segmenti si compone di due elementi: overhead per segmento e overhead per byte, entrambi da ridurre. La chiave per elaborare rapidamente un segmento sta nell'isolare il caso normale (trasferimento di dati unidirezionale) ed elaborarlo in modo speciale. Molti protocolli tendono a enfatizzare che cosa fare quando qualcosa va storto (come un pacchetto andato perso), ma per fare andare velocemente i protocolli il progettista deve puntare a minimizzare i tempi di elaborazione quando tutto funziona. Minimizzare i tempi di elaborazione quando avviene un errore è secondario.

Anche se è necessaria una sequenza di segmenti speciali per entrare nello stato *ESTABLISHED*, una volta entratici, l'elaborazione del segmento resta ovvia fino a quando un lato inizia a chiudere la connessione. Iniziamo esaminando il lato di invio nello stato *ESTABLISHED* quando vi sono dati da trasmettere. Per chiarezza, qui dobbiamo presumere che l'entità di trasporto si trovi nel kernel, anche se le stesse idee valgono per un processo dello spazio dell'utente o per una libreria all'interno del processo di invio. Nella Figura 6.51 il processo di invio manda un segnale asincrono (trap) al kernel per eseguire SEND. La prima operazione svolta dall'entità di trasporto è valutare se questo è il caso normale: lo stato è *ESTABLISHED*, nessun lato sta tentando di chiudere la

connessione, è stato inviato un segmento regolare completo (non fuori banda) e nel ricevente è disponibile spazio sufficiente nella finestra. Se tutte le condizioni sono soddisfatte, non servono altri test e l'entità di trasporto può imboccare il percorso rapido per l'invio. Generalmente questo percorso viene seguito la maggior parte delle volte.

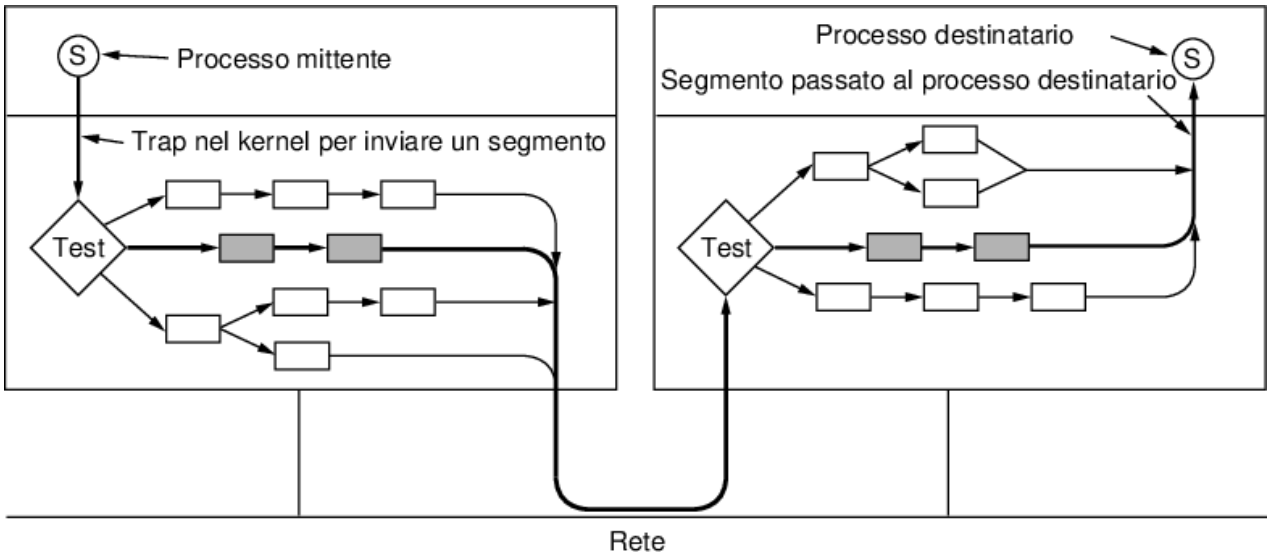


Figura 6.51 Il percorso rapido dal mittente al ricevente è mostrato con una linea spessa. I passaggi che richiedono elaborazione sono ombreggiati nel tracciato.

Normalmente le intestazioni di segmenti dati consecutivi sono quasi le stesse. Per trarne vantaggio un prototipo di intestazione viene memorizzato all'interno dell'entità di trasporto. All'inizio del percorso rapido viene copiato il più velocemente possibile in un buffer e i campi che cambiano da un segmento all'altro vengono sovrascritti nel buffer. Spesso questi campi vengono facilmente derivati dalle variabili di stato, come il numero di sequenza successivo. Al livello di rete viene quindi passato un puntatore all'intera intestazione del segmento, insieme a un puntatore ai dati dell'utente. A livello di rete si può adottare la stessa strategia (non mostrata nella Figura 6.51). Per finire, il livello di rete consegna il pacchetto risultante al livello data link per la trasmissione.

Come esempio del funzionamento pratico di questo principio consideriamo TCP/IP. La Figura 6.52(a) mostra l'intestazione TCP. I campi identici nei segmenti consecutivi del flusso unidirezionale sono ombreggiati. Tutto ciò che l'entità di trasporto in uscita deve fare è copiare i gruppi di bit cambiati rispetto al prototipo di intestazione nel buffer di output, inserire il numero di sequenza successivo (copiandolo dalla memoria), calcolare il checksum e incrementare il numero di sequenza in memoria. Può quindi passare l'intestazione e i dati a una speciale procedura IP per l'invio di un segmento standard con dimensione massima. IP copia poi il suo prototipo di intestazione (Figura 6.52b) nel buffer, compila il campo di identificazione e calcola il proprio checksum. Ora il pacchetto è pronto per la trasmissione.

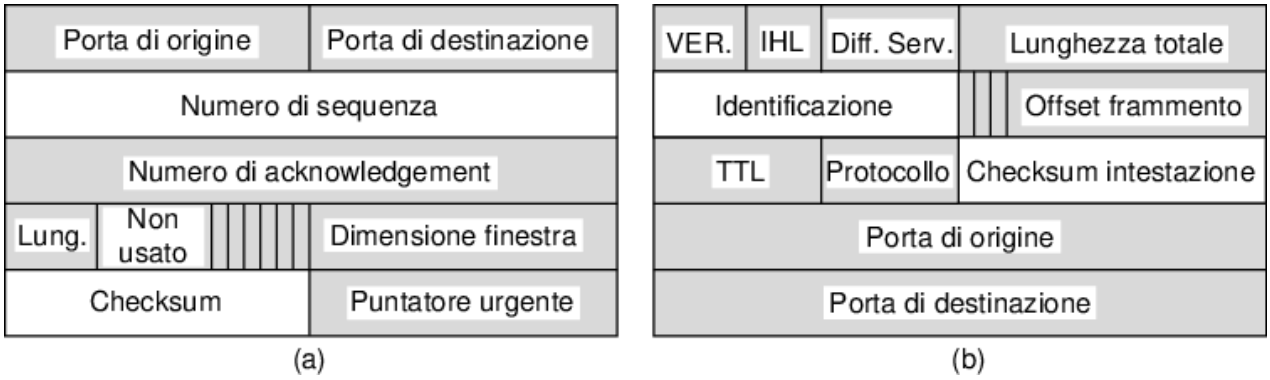


Figura6.52 (a) Intestazione TCP. (b) Intestazione IP. In entrambi i casi sono prese dal prototipo senza cambiamenti.

Ora osserviamo l'elaborazione del percorso rapido sul lato ricevente della Figura 6.51. Il primo passaggio è l'individuazione del record di connessione per il segmento in ingresso. Per TCP il record della connessione può essere memorizzato in una tabella hash la cui chiave è rappresentata dal risultato di una semplice funzione dei due indirizzi IP e delle relative porte. Una volta individuato il record della connessione, gli indirizzi e le porte devono essere confrontati per verificare di aver individuato il giusto record.

Un'ottimizzazione che spesso accelera ancor più la ricerca del record di connessione consiste nel mantenere un puntatore all'ultimo record utilizzato e provarlo per primo. Clark et al. (1989) hanno provato questo metodo e hanno osservato una frequenza di successo superiore al 90%.

Il segmento viene poi controllato per stabilire se è normale: lo stato è *ESTABLISHED*, nessuno dei due lati sta provando a chiudere la connessione, il segmento è completo, non sono impostati flag speciali e il numero di sequenza è quello previsto. Questi test richiedono solo una manciata di istruzioni. Se tutte le condizioni vengono soddisfatte, si esegue la chiamata a una speciale procedura TCP per il percorso rapido.

Il percorso rapido aggiorna il record della connessione e rende i dati disponibili all'utente. Durante la copia elabora anche il checksum, eliminando un passaggio extra sui dati. Se il checksum è corretto, il record della connessione viene aggiornato e viene restituito un acknowledgement. Lo schema generale che prevede di controllare prima se l'intestazione è quella prevista e poi di utilizzare una procedura speciale per la gestione di questo caso è chiamato **header prediction** (*predizione dell'intestazione*) e viene usato da molte implementazioni di TCP. Quando questa ottimizzazione e tutte le altre discusse nel capitolo vengono utilizzate insieme, è possibile eseguire TCP al 90% della velocità di una copia locale da memoria a memoria, nell'ipotesi che la rete sia abbastanza veloce.

Altre due aree dove sono possibili guadagni in termini di prestazioni sono le gestioni dei buffer e dei timer. Per la gestione del buffer si devono evitare copie non necessarie, come affermato in precedenza. La gestione dei timer è importante, perché quasi tutti i timer non scadono: sono

programmati per proteggere dalle perdite dei segmenti, ma la maggior parte dei segmenti arriva correttamente insieme ai relativi acknowledgement. Di conseguenza, è importante ottimizzare la gestione dei timer per il caso in cui la scadenza è un evento raro.

Uno schema comune consiste nell'usare una lista concatenata degli eventi dei timer ordinati per tempo di scadenza. La voce in testa contiene un contatore che specifica il numero di cicli di clock mancanti alla scadenza; ogni voce successiva contiene un contatore che specifica il numero di cicli di clock che mancano alla scadenza oltre a quelli della voce precedente. Pertanto, se i timer scadono a 3, 10 e 12 cicli di clock, i tre contatori presenteranno rispettivamente i valori 3, 7 e 2.

A ogni ciclo di clock il contatore nella voce in testa viene decrementato. Quando raggiunge lo zero, il suo evento viene elaborato e la successiva voce nell'elenco diventa quella in testa; il suo contatore non deve essere modificato. In questo schema inserimento ed eliminazione di timer sono operazioni costose con tempi di esecuzione proporzionali alla lunghezza della lista.

Un approccio più efficiente può essere utilizzato se l'intervallo massimo del timer ha un valore limitato e noto in anticipo. È possibile utilizzare un array chiamato **timing wheel** (*ruota temporizzatrice*), come mostrato nella Figura 6.53. Ogni slot corrisponde a un ciclo di clock. È mostrata la situazione al tempo $T = 4$. I timer sono pianificati per scadere a 3, 10 e 12 cicli dall'istante corrente. Se improvvisamente viene impostato un nuovo timer che deve scadere entro 7 cicli, si crea una voce nello slot 11. Allo stesso modo, se il timer impostato per $T + 10$ deve essere annullato, occorre eseguire la ricerca nella lista che inizia con lo slot 14 e rimuovere la voce interessata. È possibile notare che l'array della Figura 6.53 non può gestire timer che vadano oltre $T + 15$.

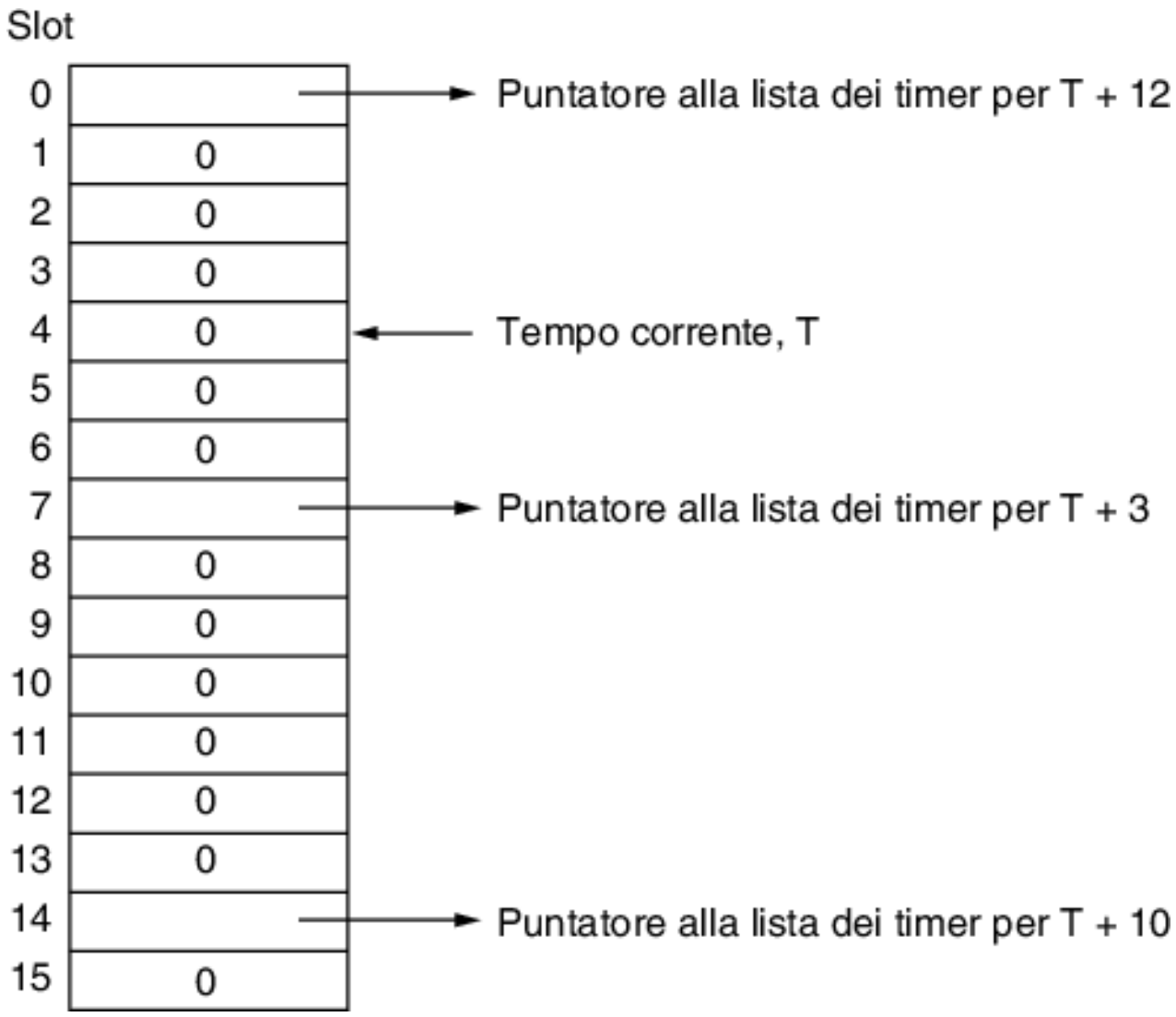


Figura 6.53 Un timing wheel.

A ogni ciclo di clock, il puntatore del tempo corrente avanza circolarmente di uno slot. Se la voce indicata è diversa da zero, tutti i suoi timer vengono elaborati. Molte varianti di questa idea di base sono discusse in Varghese e Lauck (1987).

6.6.5 Compressione dell'intestazione

Abbiamo trattato reti veloci per fin troppo tempo, c'è molto altro là fuori. Consideriamo ora le prestazioni su reti wireless e altre reti in cui la banda è limitata. Ridurre l'overhead del software può aiutare i computer portatili a funzionare in maniera più efficiente, ma non fa nulla per migliorare le prestazioni quando i collegamenti di rete sono il collo di bottiglia.

Per usare bene la banda, le intestazioni e i payload dei protocolli dovrebbero essere trasportati con il numero minimo di bit. Per il payload questo significa usare codifiche compatte per le informazioni, come immagini in formato JPEG invece che bitmap, o formati di documenti come PDF che includono compressione. Significa anche usare meccanismi di cache a livello di applicazioni, come le cache Web che riducono in primo luogo i trasferimenti.

E per le intestazioni del protocollo? A livello data link le intestazioni per le reti wireless di solito sono compatte, poiché furono progettate tenendo presente la poca banda disponibile. Per esempio le intestazioni di 802.16 hanno identificativi di connessione (corti) invece di indirizzi (lunghi). Tuttavia i protocolli a livelli più elevati, come IP, TCP e UDP sono disponibili in una sola versione per ogni possibile livello data link e non sono stati progettati con intestazioni compatte. Infatti, progettare per una elaborazione veloce porta spesso a intestazioni che non sono così compatte come altrimenti potrebbero essere (ad esempio IPv6 ha intestazioni meno compatte di IPv4).

Le intestazioni dei livelli più alti possono penalizzare significativamente le prestazioni. Consideriamo, per esempio, dati VoIP trasportati con una combinazione di IP, UDP e RTP. Questi protocolli richiedono 40 byte di intestazione (20 per IPv4, 8 per UDP e 12 per RTP). Con IPv6 la situazione è anche peggiore: 60 byte, inclusi i 40 byte dell'intestazione IPv6. Le intestazioni possono crescere fino a rappresentare la maggior parte dei dati trasmessi e consumare più di metà della banda.

La **compressione dell'intestazione** (*header compression*) è usata per ridurre la banda sottratta sui collegamenti dalle intestazioni dei livelli superiori. Schemi progettati specificamente vengono usati al posto di metodi a uso generico. Questo perché le intestazioni sono corte, quindi non si comprimono bene individulmente e la decompressione richiede che tutti i dati precedenti siano stati ricevuti, che non è il caso in cui un pacchetto viene perso.

La compressione dell'intestazione ottiene grandi vantaggi sfruttando la conoscenza del formato del protocollo. Uno dei primi schemi fu progettato da Van Jacobson (1990) per comprimere le intestazioni TCP/IP su linee seriali lente. Questo metodo è in grado di comprimere una tipica intestazione TCP/IP di 40 byte fino a una media di 3 byte; il trucco è accennato nella Figura 6.52: molti dei campi dell'intestazione non cambiano da pacchetto a pacchetto. Non c'è bisogno, per esempio, di spedire lo stesso TTL IP o gli stessi numeri di porta TCP in ogni pacchetto; possono essere omessi da parte del mittente e riempiti dal destinatario.

In maniera simile altri campi cambiano in maniera prevedibile. Per esempio, escludendo le perdite, il numero di sequenza TCP aumenta con i dati; in questi casi il ricevente può prevedere il possibile valore. Il numero effettivo ha bisogno di essere trasportato solo quando differisce da ciò che ci si aspetterebbe. Anche in quel caso può essere trasportato come un piccolo cambiamento dal valore precedente. Lo stesso discorso vale per il numero di acknowledgement nella direzione opposta.

Grazie alla compressione dell'intestazione è possibile avere intestazioni semplici in protocolli di alto livello e codifiche compatte su linee con banda limitata. **ROHC** (*robust header compression*, compressione robusta dell'intestazione) è una versione moderna della compressione dell'intestazione nell'RFC 5795. Questa compressione è progettata per tollerare le perdite che possono avere luogo sulle linee wireless. C'è un profilo per ogni gruppo di protocolli da comprimere, come IP/UDP/RTP. Le intestazioni compresse sono trasportate facendo riferimento a un contesto, che è essenzialmente una connessione; i campi dell'intestazione di pacchetti appartenenti alla stessa connessione possono essere facilmente pronosticati; la stessa cosa, però, non è attuabile per pacchetti di connessioni diverse. In condizioni normali ROHC riduce le intestazioni IP/UDP/RTP da 40 byte fino a 1 o al più 3 byte.

Mentre la compressione dell'intestazione è principalmente finalizzata alla riduzione delle necessità di banda, può essere utile anche per la riduzione del ritardo. Nel ritardo è compreso il ritardo di propagazione, che è fisso dato un certo percorso in rete, e il ritardo di trasmissione, che dipende dalla banda e dalla quantità di dati da spedire. Per esempio, un collegamento da 1 Mbps spedisce 1 bit in 1 ms. In caso di contenuti multimediali su reti wireless, la rete è relativamente lenta, quindi il ritardo di trasmissione può essere un fattore influente sul ritardo totale e un ritardo costantemente basso è influente per la qualità del servizio.

La compressione dell'intestazione può essere utile nella riduzione della quantità di dati spediti, con conseguente riduzione del ritardo di trasmissione. Lo stesso effetto lo si ottiene spedendo pacchetti più piccoli. Si otterrà quindi un aumento di overhead dovuto al software in cambio di una diminuzione del ritardo di trasmissione. Si noti che un'altra potenziale fonte di ritardo è il ritardo di accodamento per accedere al collegamento wireless. Questo può anche essere significativo, perché i collegamenti wireless sono spesso pesantemente utilizzati in quanto una risorsa limitata della rete. In questo caso, il collegamento wireless deve avere meccanismi per la qualità del servizio che diano un basso ritardo ai pacchetti real-time; la compressione dell'intestazione da sola non è sufficiente.

6.6.6 Protocolli per reti ad altà capacità su lunghi percorsi

A partire dagli agli anni '90 ci sono state delle linee con capacità dell'ordine del gigabit per trasmettere dati su lunga distanza. Queste reti, a causa della combinazione tra alta velocità, quindi una pipeline molto grande (*fat pipe*), e forte ritardo (*long delay*), vengono indicate come **LFN** (*long fat network*). Alla nascita di queste reti la prima reazione della gente fu di usare per esse i protocolli esistenti, ma saltarono fuori velocemente vari problemi. In questo paragrafo discuteremo alcuni dei problemi dei protocolli di rete nati con l'aumento progressivo di velocità e ritardo.

Il primo problema è che molti protocolli utilizzano numeri di sequenza a 32 bit. Agli esordi di Internet le linee tra i router erano per lo più linee affittate a 56 kbps, pertanto un host che inviava ininterrottamente alla velocità massima impiegava circa una settimana per utilizzare tutti i numeri di sequenza. Per chi progettò TCP, 2^{32} era un'approssimazione decente di infinito, perché il pericolo di avere in circolazione pacchetti trasmessi una settimana prima era davvero ridotto. Con Ethernet a 10 Mbps il tempo per effettuare un ciclo divenne 57 minuti, più breve ma ancora gestibile. Una Ethernet a 1 Gbps che riversa dati su Internet ha un tempo di ciclo di circa 34 secondi, molto inferiore al minimo tempo di vita dei pacchetti (120 secondi). Improvvisamente, 2^{32} non appare più una valida approssimazione di infinito, perché un mittente veloce può eseguire un intero ciclo dello spazio dei numeri di sequenza mentre sono ancora in circolo vecchi pacchetti.

Il problema è che molti progettisti di protocolli hanno semplicemente assunto, senza dichiararlo apertamente, che il tempo necessario a utilizzare l'intero spazio delle sequenza avrebbe ampiamente superato il tempo di vita massima dei pacchetti; di conseguenza, non c'era bisogno di preoccuparsi del problema dei vecchi duplicati rimasti in circolazione al termine di un ciclo di numeri di sequenza; a velocità dell'ordine del gigabit, questa assunzione viene meno. Fortunatamente è stato possibile estendere l'effettiva sequenza numerica considerando il timestamp che può essere

trasportato come un'opzione dell'intestazione TCP di ogni pacchetto; il timestamp viene usato come valore per i bit di ordine superiore del numero di sequenza. Questo meccanismo, chiamato **PAWS**, è descritto nell'RFC 1323.

Un secondo problema è che la dimensione della finestra di controllo di flusso deve essere incrementata notevolmente. Si consideri, per esempio, l'invio di un blocco da 64 KB di dati da San Diego a Boston con lo scopo di riempire il buffer di 64 KB del ricevente. Si supponga che il collegamento sia a 1 Gbps e che il ritardo relativo alla velocità della luce nella fibra in una direzione sia di 20 ms. Inizialmente, a $t = 0$ il canale è vuoto, come illustrato nella Figura 6.54(a). Appena 500 ms più tardi, nella Figura 6.54(b), tutti i segmenti sono sulla fibra. Il segmento di testa ora si trova nelle vicinanze di Brawley, nel sud della California; tuttavia, il trasmittente deve fermarsi fino a che non riceve un aggiornamento della finestra.

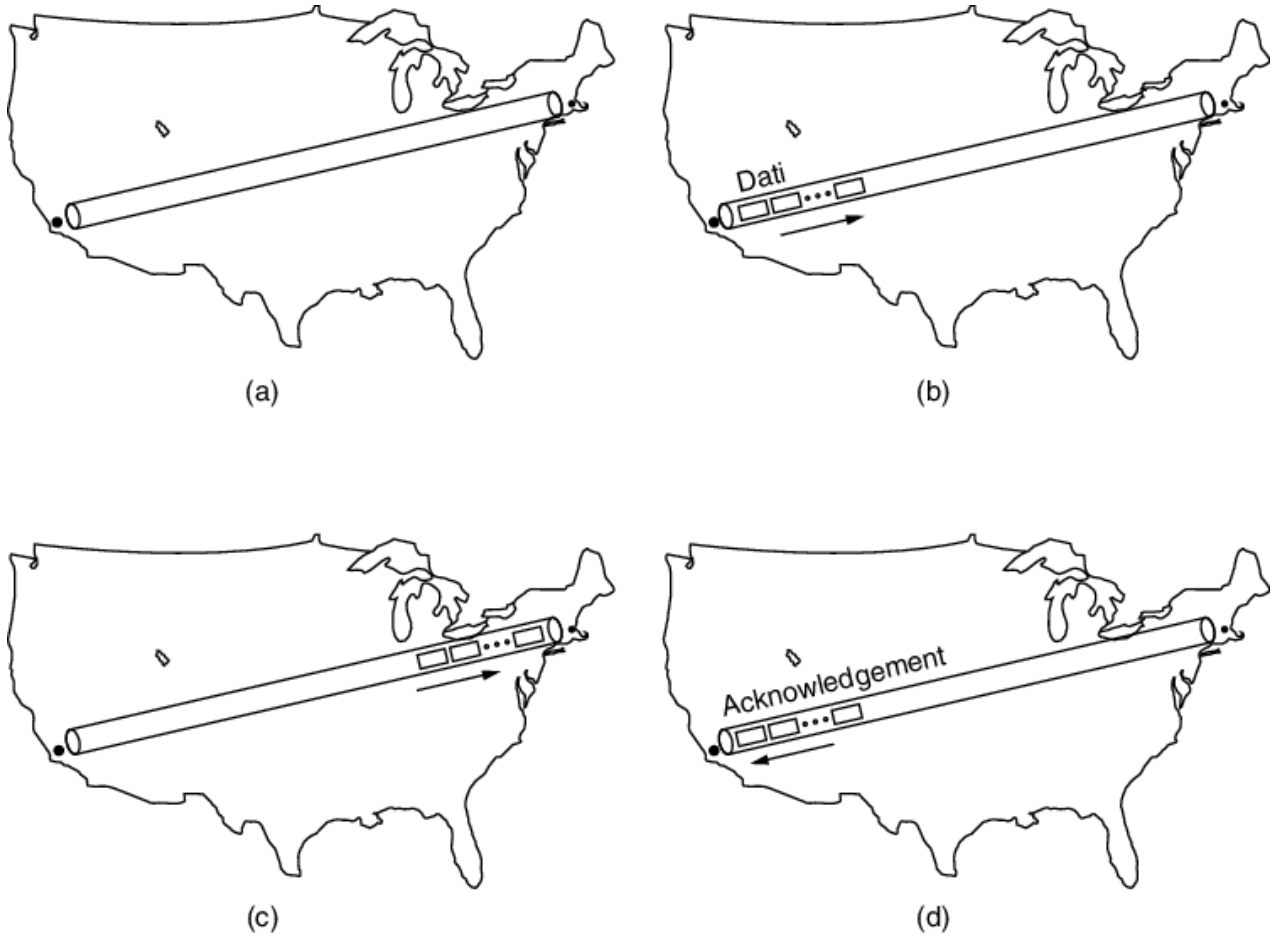


Figura 6.54 Lo stato della trasmissione di un megabit da San Diego a Boston. (a) A t = 0. (b) Dopo 0,5 ms. (c) Dopo 20 ms. (d) Dopo 40 ms.

Dopo 20 ms il segmento di testa raggiunge Boston, come mostrato nella Figura 6.54(c) e viene emesso il corrispondente acknowledgement. Per finire, 40 ms dopo l'inizio, il primo acknowledgement torna al mittente e il secondo blocco può essere trasmesso. Dal momento che la

linea di trasmissione è stata utilizzata per 1,25 ms su 100, l'efficienza è pari a 1,25%. Questa situazione è tipica dei vecchi protocolli in esecuzione su linee con capacità di gigabit.

Un valore da tenere ben presente durante l'analisi delle prestazioni di rete è il **prodotto banda-ritardo** (*bandwidth-delay product*), ottenuto moltiplicando la banda (in bit al secondo) per il round-trip time (in secondi). Il prodotto è la capacità della pipeline dal mittente al ricevente e viceversa (in bit).

Per l'esempio nella Figura 6.54 il prodotto banda-ritardo è 40 milioni di bit. In altre parole, il mittente dovrebbe trasmettere una raffica di 40 milioni di bit per essere in grado di continuare a trasmettere a piena velocità fino al ritorno del primo acknowledgement: sono questi i bit necessari per riempire la pipeline (in entrambe le direzioni). Ecco perché un blocco di mezzo milione di bit (64 KB) raggiunge un'efficienza pari ad appena 1,25%: viene utilizzata solo l'1,25% della capacità della pipeline.

La conclusione che possiamo trarre è la seguente: per avere buone prestazioni la finestra del ricevente deve essere grande almeno quanto il prodotto banda-ritardo e preferibilmente un po' di più perché il ricevente potrebbe non rispondere immediatamente. Per una linea intercontinentale con capacità dell'ordine del gigabit sono richiesti almeno 5 megabyte.

Un terzo problema collegato a quello appena descritto è che gli schemi semplici di ritrasmissione, come il protocollo go-back-n, funzionano male su linee con un valore elevato del prodotto banda-ritardo. Si consideri una linea transcontinentale da 1 Gbps con un round-trip time di 40 ms; un mittente può quindi trasmettere 5 MB in un round-trip time. Se viene rilevato un errore, trascorreranno 40 ms prima che il mittente sia avvisato. Usando il protocollo go-back-n, il mittente dovrà ritrasmettere non solo il pacchetto errato, ma anche i 5 MB di pacchetti seguenti, con un chiaro ed enorme spreco di risorse. Sono quindi necessari protocolli più complessi, come selective-repeat.

Un quarto problema è che le linee ad alta capacità sono fondamentalmente diverse dalle linee con capacità dell'ordine dei megabit, perché le prime sono limitate dal ritardo anziché dalla banda. Nella Figura 6.55 è mostrato il tempo richiesto per trasferire un file di 1 megabit lungo una linea di 4000 Km a differenti velocità di trasmissione. A velocità fino a 1 Mbps il tempo di trasmissione ha un peso minore del tempo di invio dei bit, mentre arrivati a 1 Gbps, il round-trip time di 40 ms assume un peso maggiore di 1 ms necessario per trasferire i bit sulla fibra. Ulteriori aumenti della banda non forniscono praticamente alcun aumento di prestazioni.

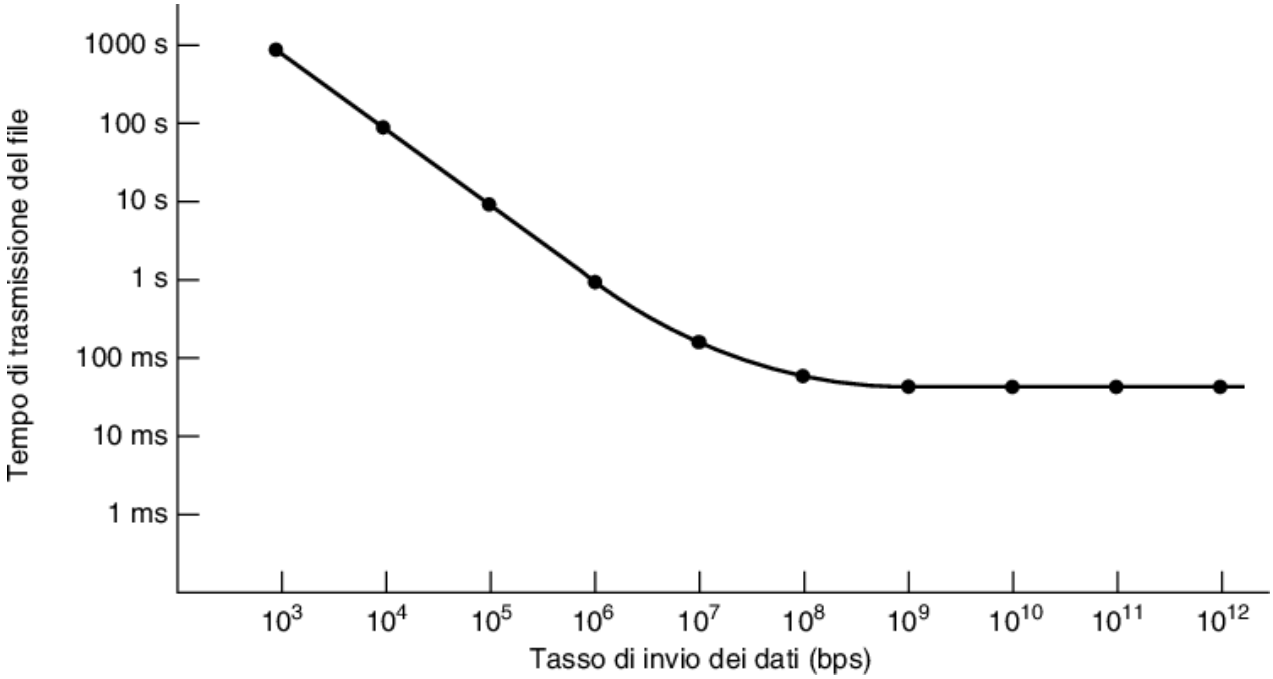


Figura 6.55 Il tempo per trasferire e dare l'acknowledgement a un file di 1 megabit su una linea di 4000 Km.

La Figura 6.55 presenta delle spiacevoli ricadute sui protocolli di rete: afferma che i protocolli stop-and-wait, come RPC, hanno un limite superiore intrinseco alle prestazioni. Questo limite è dettato dalla velocità della luce. Nessun progresso tecnologico potrà mai migliorare la situazione (ma nuove leggi fisiche *ad retem* potrebbero essere utili). A meno che possa essere trovato qualche altro uso per la linea mentre un host sta aspettando una risposta, la linea a 1 Gbps non è migliore di una linea a 1 Mbps, solo è più costosa.

Un quinto problema è che le velocità di comunicazione sono aumentate più rapidamente delle velocità di calcolo. (Nota per gli informatici, ingegneri e non: andate fuori e bastonate gli ingegneri delle telecomunicazioni! Contiamo su di voi.) Negli anni '70 ARPANET andava a 56 kbps e aveva dei computer a 1 MIPS. Si confrontino questi numeri con computer a 1000 MIPS che si scambiano pacchetti su una linea a 1 Gbps. Il numero di istruzioni per byte è sceso di più di un fattore 10. I numeri esatti sono discutibili a seconda di date e scenari, ma la conclusione è questa: c'è meno tempo disponibile per l'elaborazione dei protocolli di quanto ce ne fosse prima, quindi i protocolli devono semplificarsi.

Passiamo ora dai problemi alle soluzioni. Il principio di base che tutti i progettisti di reti ad alta velocità dovrebbero imparare a memoria è il seguente:

progettare per la velocità, non per ottimizzare la banda.

I vecchi protocolli sono stati spesso progettati per ridurre il numero di bit sul cavo utilizzando in molti casi piccoli campi compattati insieme in singoli byte. Questa preoccupazione è ancora valida per le reti wireless, ma non per quelle ad alta velocità: il problema è l'elaborazione dei protocolli, che di conseguenza dovrebbero essere progettati per minimizzarla. I progettisti di IPv6 hanno chiaramente compreso questo principio.

Un modo allettante per migliorare la velocità consiste nel costruire interfacce di rete con hardware veloce. La difficoltà di questa strategia sta nel fatto che, a meno che il protocollo sia davvero semplice, l'hardware è rappresentato da una scheda con una seconda CPU e un suo programma. Per assicurare che il coprocessore di rete sia più economico della CPU principale, viene di norma utilizzato un chip più lento. La conseguenza di questa struttura è che per la gran parte del tempo la CPU principale (veloce) è in attesa che la seconda CPU (lenta) svolga il lavoro importante. È una leggenda metropolitana che la CPU principale abbia altro lavoro da svolgere nell'attesa. Inoltre, quando due CPU generiche comunicano, si possono verificare situazioni di conflitto, pertanto si devono usare protocolli elaborati per sincronizzare correttamente i due processori. Solitamente l'approccio migliore prevede di creare protocolli semplici e di lasciare che la CPU principale svolga il lavoro.

La struttura dei pacchetti è una considerazione importante nelle reti ad alta velocità. L'intestazione dovrebbe contenere meno campi possibili per ridurre il tempo di elaborazione e questi campi dovrebbero essere abbastanza grandi per svolgere il loro compito ed essere allineati con il sistema di indirizzamento della CPU (*word-aligned*) per un'elaborazione veloce. In questo contesto, "abbastanza grande" significa che problemi quali numeri di sequenza che tornano a zero mentre sono ancora in circolazione vecchi pacchetti, destinatari non in grado di segnalare abbastanza spazio nella finestra perché il relativo campo è troppo piccolo e così via non si verificano.

Per ridurre l'overhead software e permettere operazioni efficienti la dimensione massima di dati dovrebbe essere grande. 1500 byte sono troppo pochi per una rete ad alta velocità, ragion per cui Gigabit Ethernet supporta i jumbo frame fino a 9 KB e IPv6 supporta jumbogrammi in eccesso di 64 KB.

Vediamo ora il problema del feedback nei protocolli ad alta velocità. A causa del (relativamente) lungo ciclo di ritardo, il feedback potrebbe essere evitato: il destinatario impiega troppo tempo a mandare un segnale al mittente. Un esempio di feedback è il caso in cui si gestisce il tasso di trasmissione usando un protocollo a finestra scorrevole. I futuri protocolli potrebbero basarsi sui tassi di invio delle informazioni per evitare i (lunghi) ritardi causati dal ricevente per la spedizione degli aggiornamenti riguardanti la finestra di spedizione al mittente. In un protocollo del genere il mittente può spedire tutto ciò che vuole, a condizione di non spedire più in fretta del tasso che mittente e destinatario hanno concordato in anticipo.

Un secondo esempio di feedback è l'algoritmo di slow start di Jacobson. Questo algoritmo effettua una serie di rilevamenti per capire quanto la rete possa gestire. Con reti ad alta velocità fare una mezza dozzina di piccoli rilevamenti per vedere come risponde la rete spreca una grande quantità di banda. Un sistema più efficiente consiste nella richiesta a mittente, destinatario e rete di prenotare le risorse necessarie nel momento della creazione della connessione. Riservare risorse in anticipo ha anche il vantaggio di rendere più facile la riduzione del jitter. In breve, andare verso velocità elevate spinge inesorabilmente la progettazione verso operazioni orientate alla connessione o a qualcosa di molto simile.

Un'altra caratteristica interessante sarebbe la capacità di spedire dei dati unitamente alla richiesta di connessione; in questo modo potrebbe essere risparmiato un round-trip time.

6.7 Reti con tolleranza al ritardo

Finiremo questo capitolo descrivendo un nuovo tipo di trasporto che un giorno potrebbe essere una componente importante di Internet. TCP e molti altri protocolli di trasporto sono basati sull'ipotesi che mittente e destinatario siano continuamente connessi da qualche percorso funzionante, altrimenti il protocollo fallisce e i dati non possono essere recapitati. In alcune reti spesso un percorso end-to-end non esiste. Un esempio è una rete spaziale come i satelliti LEO (*low-earth orbit*) che entrano ed escono dal raggio di azione delle stazioni a terra. Un dato satellite può essere in grado di comunicare con una stazione a terra solo in particolari momenti e due satelliti potrebbero non essere mai in grado di comunicare tra di loro, neanche attraverso una stazione a terra, perché uno dei satelliti è sempre fuori raggio d'azione. Un altro esempio di reti coinvolge i sottomarini, gli autobus, i cellulari e altri dispositivi con dei computer per i quali ci sia connettività intermittente a causa di mobilità o di condizioni estreme.

In queste reti occasionalmente connesse i dati possono ancora essere comunicati immagazzinandoli nei nodi e inoltrandoli più tardi quando c'è un collegamento funzionante: alla fine i dati saranno inviati alla destinazione. Tale tecnica è chiamata **message switching** (*commutazione di messaggio*). Una rete la cui architettura è basata su questo approccio è chiamata **DTN** (*delay-tolerant network* o *disruption-tolerant network*, rete con tolleranza di ritardo o rete con tolleranza alle rotture).

Il lavoro sulle DTN iniziò nel 2002 quando IETF organizzò un gruppo di ricerca sull'argomento. L'ispirazione per le DTN proveniva da una fonte improbabile: gli sforzi per inviare pacchetti nello spazio. Le reti spaziali devono avere a che fare con comunicazioni intermittenti e ritardi molto lunghi. Kevin Fall osservò che le idee per queste Internet Interplanetaria potevano essere applicate alla reti sulla Terra in cui la connettività intermittente era la norma (Fall, 2003). Questo modello dà un'utile generalizzazione di Internet, in cui l'immagazzinamento e il ritardo possono avvenire durante una comunicazione. La consegna dei dati è simile alla consegna in un sistema postale, o nella posta elettronica, invece che la commutazione di pacchetti nei router.

Dal 2002 l'architettura DTN è stata raffinata e le applicazioni del modello DTN sono aumentate. Come una delle principali applicazioni si considerino grandi insiemi di dati di molti terabyte prodotti da esperimenti scientifici, eventi multimediali o servizi basati sul Web che necessitano di essere copiati in datacenter in diversi parti del mondo. Gli operatori vorrebbero spedire questa massa di traffico nelle ore di minor congestione usando la banda già pagata per questo ma che nessuno sta usando, e sono disposti a tollerare un po' di ritardo. È come fare dei backup di notte quando le altre applicazioni non stanno facendo un uso pesante della rete. Il problema è che, per servizi globali, le ore di minor traffico sono diverse nelle diverse posizioni attorno al mondo. Potrebbe esserci poca sovrapposizione nei periodi in cui i datacenter di Boston e Perth hanno la rete scarica, perché la notte per una città è il giorno per l'altra.

Tuttavia i modelli DTN permettono l'immagazzinamento e il ritardo durante il trasferimento. Con questo modello diventa possibile spedire dati da Boston ad Amsterdam usando la banda nei momenti di minor utilizzo, poiché le città sono a sole sei ore di fuso orario di distanza. I dati saranno quindi immagazzinati ad Amsterdam finché non ci sarà poco traffico tra Amsterdam e Perth. Quindi verranno spediti a Perth per completare il trasferimento. Laoutaris et al. (2009)

hanno studiato questo modello scoprendo che può fornire notevole capacità a costi ridotti, e che l'uso del modello DTN spesso raddoppia quella capacità se confrontato con un modello tradizionale end-to-end.

Descriveremo nel seguito l'architettura e i protocolli DTN di IETF.

6.7.1 Architettura DTN

L'ipotesi principale in Internet che le reti DTN cercano di eliminare è che un percorso end-to-end tra una sorgente e una destinazione esiste per l'intera durata di una sessione di comunicazione; quando non è così, i normali protocolli di Internet falliscono. Le DTN aggirano la mancanza di connettività end-to-end con un'architettura basata sulla commutazione di messaggi, come mostrato nella Figura 6.56. Questa architettura, specificata nell'RFC 4838, è anche pensata per tollerare collegamenti con una bassa affidabilità e ampi ritadi.

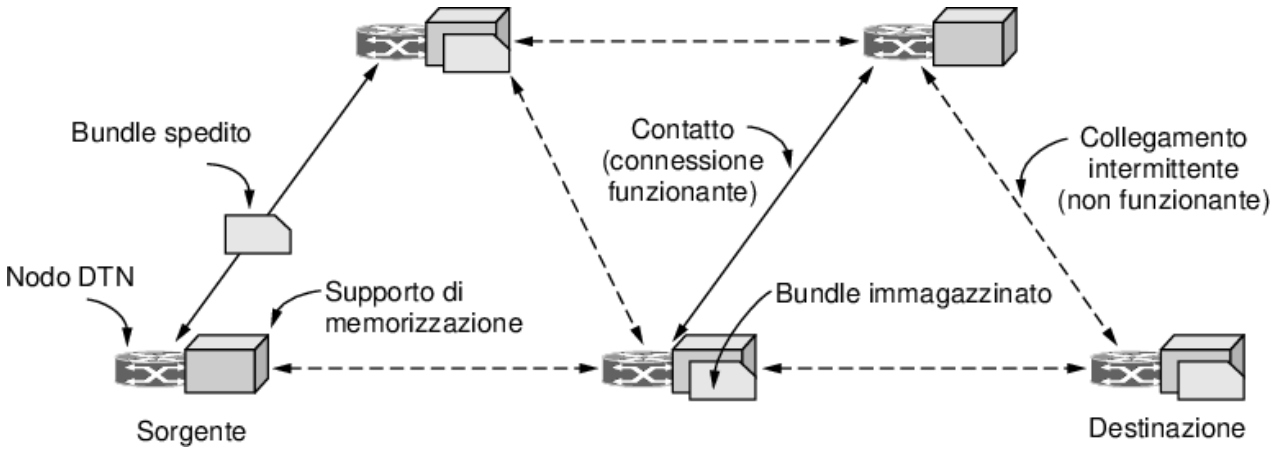


Figura6.56 Architettura di rete con tolleranza al ritardo.

Nella terminologia DTN un messaggio prende il nome di **bundle** (letteralmente, *fagotto* o *fardello*). I nodi DTN, tipicamente dotati di un sistema di memorizzazione persistente come un disco o memoria flash, memorizzano i bundle finché il collegamento diventa disponibile e quindi li inoltrano. La Figura 6.56 mostra cinque collegamenti intermittenti attualmente non in funzione e due collegamenti in funzione. Un collegamento funzionante è chiamato **contatto**. La Figura 6.56 mostra inoltre i bundle memorizzati in due nodi DTN che aspettano un contatto per inoltrare i bundle lungo il percorso. In questo modo i bundle sono inoltrati attraverso contatti dalla sorgente alla loro destinazione.

L'immagazzinamento e l'inoltro dei bundle da parte dei nodi DTN suona simile all'accodamento (nei buffer) e l'inoltro dei pacchetti nei router, ma ci sono differenze qualitative. Nei router in Internet gli accodamenti avvengono per millisecondi o al massimo secondi, mentre nei nodi DTN i bundle possono essere memorizzati per ore, finché un autobus non arriva in città, mentre un

aereoplano completa il volo, finché un sensore non raccoglie abbastanza energia solare per funzionare, finché un computer non si sveglia e così via. Questi esempi indicano una seconda differenza, cioè che i nodi potrebbero muoversi (con l'autobus o l'aereo) mentre tengono memorizzati dei dati, e questo movimento può anche essere una parte importante nella consegna dei dati. I router in Internet non possono muoversi. L'intero processo dei bundle che si muovono potrebbe essere meglio noto come "store-carry-forward" (*memorizza-trasporta-inoltra*).

Come esempio, si consideri lo scenario mostrato nella Figura 6.57 che era il primo uso di protocolli DTN nello spazio (Wood et al., 2008). La fonte dei bundle è un satellite LEO che registra immagini della Terra come parte di un costellazione di satelliti per la rilevazione di disastri. Le immagini devono essere convogliate verso un punto di raccolta. Tuttavia il satellite, essendo in orbita, ha solo contatti intermittenti con tre stazioni terrestri e vi entra in contatto a turno. Ognuno dei satelliti, delle stazioni a terra e il punto di raccolta funziona come un nodo DTN: a ogni contatto un bundle (o una parte di un bundle) viene spedito a una stazione a terra. I bundle sono quindi spediti tramite una rete terrestre al punto di raccolta per completare il trasferimento.

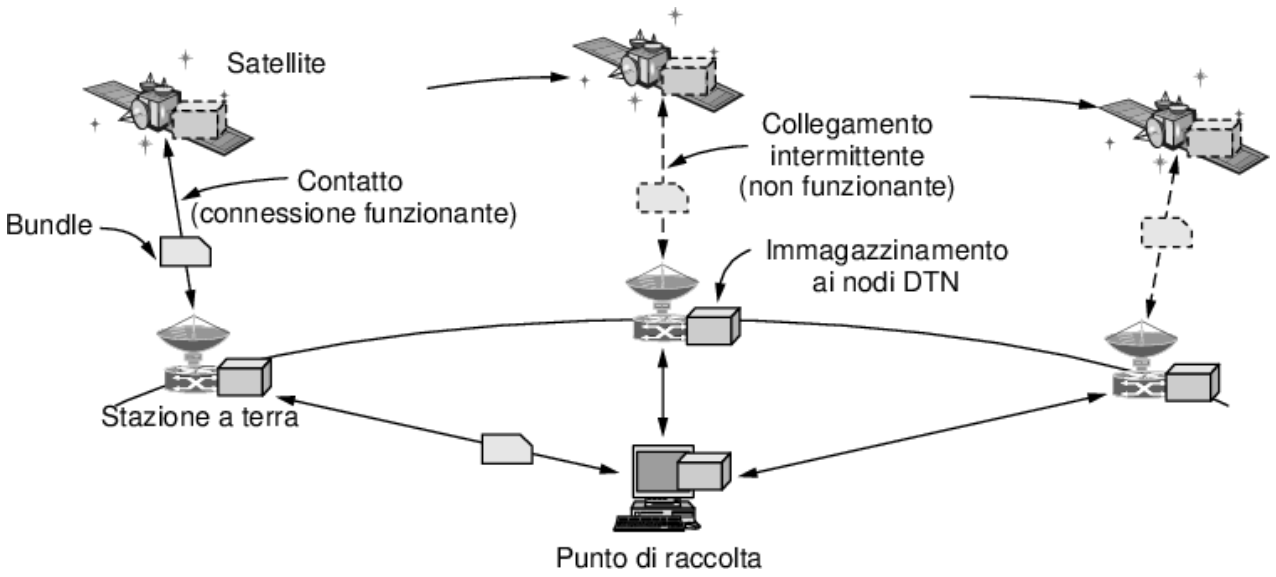


Figura6.57 Uso di una DTN nello spazio.

Il principale vantaggio dell'architettura DTN nell'esempio è che questa si adatta perfettamente alla situazione in cui il satellite ha bisogno di memorizzare le immagini, perché non c'è connettività nel momento in cui l'immagine viene scattata. Ci sono altri due vantaggi. Primo, potrebbe non esistere un singolo contatto abbastanza lungo per spedire le immagini; tuttavia, possono essere sparpagliate sui contatti con le tre stazioni terrestri. Secondo, l'uso del collegamento tra il satellite e una stazione terrestre è disaccoppiato dal collegamento della rete terrestre. Ciò significa che il download via satellite non è limitato da una collegamento terrestre lento, ma può procedere a piena velocità, con il bundle memorizzato nella stazione terrestre, finché questo non può essere inoltrato al punto di raccolta.

Una questione rilevante non affrontata dall'architettura è come trovare dei buoni percorsi attraverso i nodi DTN; questi dipendono dalla natura dell'architettura che vincola relativamente a quando spedire i dati e quali contatti usare. Alcuni contatti sono noti in anticipo; un buon esempio è il movimento dei corpi celesti nell'esempio dello spazio. Per l'esperimento spaziale si sapeva già quando sarebbero avvenuti i contatti, che l'intervallo di contatto andava da 5 a 14 minuti per passaggio con ogni stazione terrestre e che la capacità di downlink era di 8134 Mbps. Sapendo questo, il trasporto di un bundle contenente immagini può essere pianificato in anticipo.

In altri casi i contatti possono essere previsti, ma con meno certezza. Tra gli esempi troviamo gli autobus che hanno contatti in modo abbastanza regolare a causa di una programmazione, anche se con un po' di variabilità. All'altro estremo, i contatti sono occasionali e casuali. Un esempio è portare i dati da utente a utente su cellulare a seconda di quali di loro entrano in contatto durante il giorno. Quando c'è imprevedibilità dei contatti, una strategia di instradamento è di spedire copie del bundle su percorsi diversi nella speranza che una delle copie venga consegnata al destinatario prima che si raggiunga il loro tempo di vita massimo.

6.7.2 Bundle protocol

Per osservare da vicino il funzionamento delle DTN analizzeremo ora i protocolli di IETF. Le DTN sono un tipo di reti emergente e le DTN sperimentali hanno usato diversi protocolli, poiché non vi sono obblighi che siano usati quelli dell'IETF. Tuttavia, sono almeno un buon punto di partenza e mettono in evidenza molti dei problemi principali.

Lo stack di protocolli DTN è mostrato nella Figura 6.58. Il protocollo chiave è il **bundle protocol**, specificato nell'RFC 5050 e responsabile della ricezione dei messaggi dall'appliczione e della loro spedizione come uno o più bundle attraverso le operazioni di store-carry-forward verso il nodo DTN destinatario. È anche evidente dalla Figura 6.58 che il bundle protocol viene eseguito a un livello superiore di TCP/IP; in altre parole TCP/IP può essere usato a ogni contatto per spostare i bundle tra nodi DTN. Questo posizionamento solleva la questione se il bundle protocol sia un protocollo a livello di trasporto o un protocollo a livello applicazione. Proprio come con RTP, diremo che, pur funzionando sopra un protocollo di trasporto, il bundle protocol fornisce un servizio di trasporto a molte applicazioni, per cui trattiamo le DTN in questo capitolo.

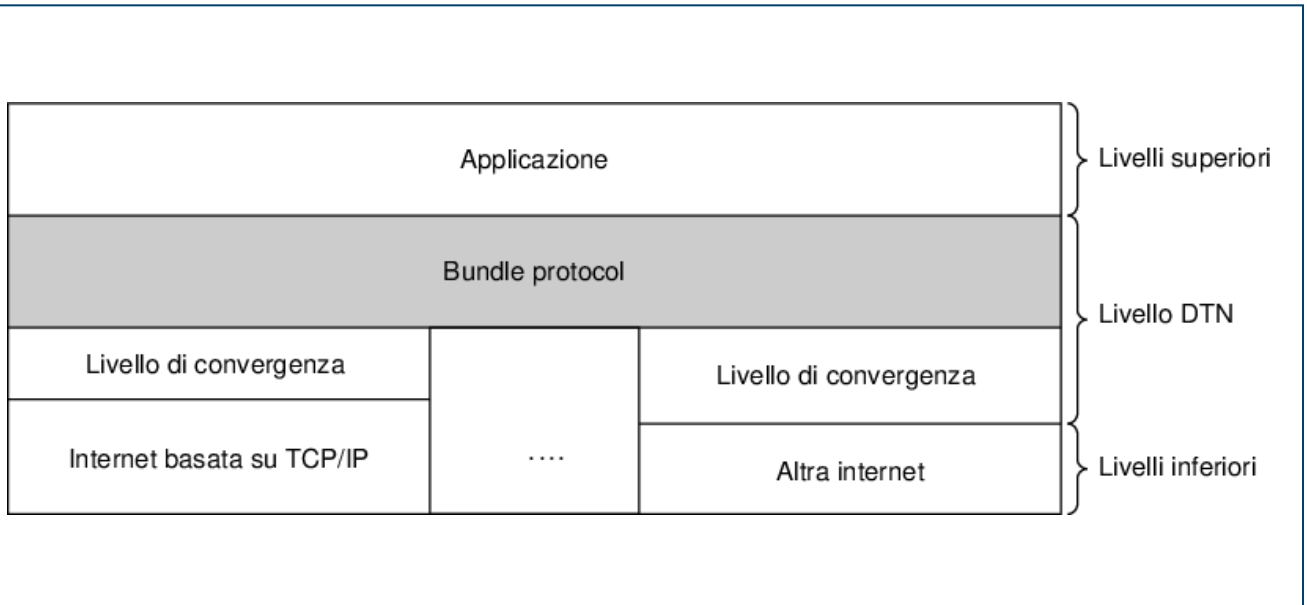


Figura 6.58 Stack di protocolli di una rete con tolleranza al ritardo.

Nella Figura 6.58 vediamo che il bundle protocol può funzionare su altri tipi di protocolli, come UDP, o anche altri tipi di internet. Per esempio in una rete spaziale i collegamenti possono avere ritardi molto lunghi. Il round-trip time tra la Terra e Marte può facilmente essere di 20 minuti a seconda della posizione relativa dei due pianeti. Immaginate quanto funzionerebbero bene gli acknowledgement e le ritrasmissioni TCP su quel collegamento, specialmente per messaggi relativamente brevi; decisamente male. Al contrario dovrebbe essere usato un altro protocollo che utilizza codici di correzione d'errore. Oppure, in reti di sensori con vincoli molto forti sulle risorse potrebbe essere usato un protocollo più leggero di TCP.

Siccome il bundle protocol è definitivo e pensato per funzionare su molti protocolli di trasporto, ci può essere una lacuna di funzionalità tra i protocolli. Quella lacuna è il motivo per cui viene incluso un livello di convergenza nella Figura 6.58. Il livello di convergenza è solo un livello collante che si accoppia alle interfacce dei protocolli che unisce. Per definizione esiste un diverso livello di convergenza per ogni diverso livello di trasporto inferiore. I livelli di convergenza si trovano di solito negli standard per unire protocolli nuovi e quelli esistenti.

Il formato dei messaggi del bundle protocol è mostrato nella Figura 6.59. I diversi campi in questi messaggi esprimono alcuni dei problemi principali gestiti dal bundle protocol.

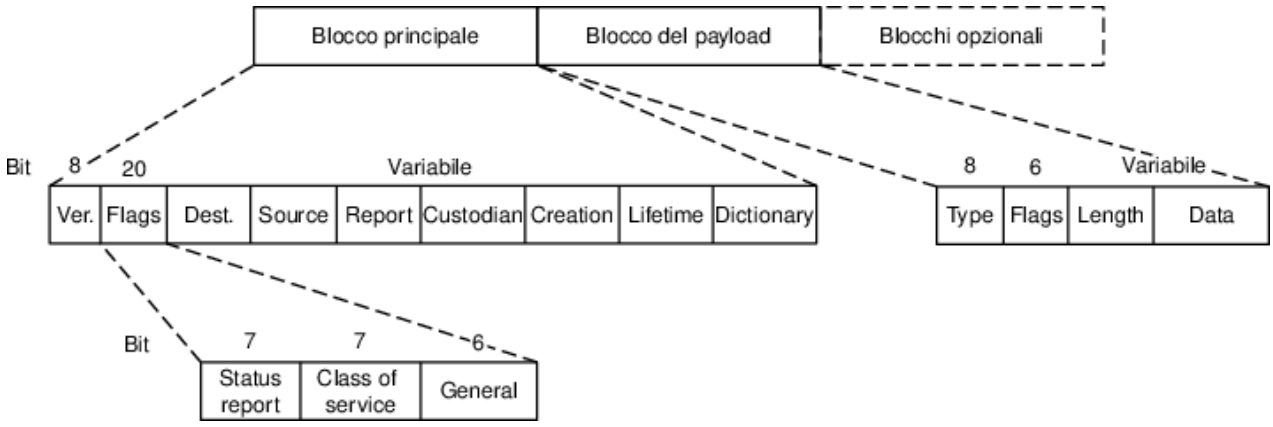


Figura6.59 Formato di un messaggio del bundle protocol.

Ogni messaggio consiste di un blocco primario, che può essere pensato come un'intestazione, un blocco di payload per i dati e facoltativamente altri blocchi per trasportare, ad esempio, parametri di sicurezza. Il blocco primario inizia con un campo *Version* (attualmente 6) seguito da un campo *Flags*. Tra le altre funzioni, i flag codificano una classe di servizio per permettere a una sorgente di marcare i propri bundle con alta o bassa priorità e altre richieste relative al trattamento, come se il destinatario dovesse mandare un acknowledgement.

Quindi arrivano gli indirizzi, che sottolineano tre parti interessanti del progetto. Oltre ai campi con gli identificativi di destinatario (*Destination*) e sorgente (*Source*), c'è un identificativo *Custodian* (custode). Il custode è il soggetto responsabile di assicurarsi che il bundle sia consegnato. In Internet il nodo sorgente è di solito anche il custode, come lo è il nodo che ritrasmette se alla fine i dati non sono stati consegnati al destinatario. Tuttavia, in una DTN, il nodo sorgente potrebbe non essere sempre connesso e potrebbe non avere modo di sapere se i dati sono stati consegnati. Le DTN affrontano questo problema usando la nozione di **custody transfer** (*trasferimento di custodia*) in cui un altro nodo, più vicino al destinatario, può prendersi la responsabilità di occuparsi che i dati vengano consegnati. Per esempio, se un bundle è memorizzato in un aereoplano per essere inoltrato successivamente in un altro posto, l'aereo può diventare il custode del bundle.

Il secondo aspetto interessante è che questi identificativi *non* sono indirizzi IP. Poiché il bundle protocol è pensato per funzionare su una varietà di protocolli di trasporto e di internet, definisce dei propri identificatori. Questi identificatori sono davvero molto più simili a nomi di alto livello, come gli URL di pagine Web, che a indirizzi di basso livello come indirizzi IP. Questi indirizzi danno alle DTN una connotazione di routing a livello applicazione, come la consegna di e-mail o la distribuzione di aggiornamenti software.

Il terzo aspetto interessante è il modo in cui sono codificati gli identificativi. C'è anche un identificativo *Report* (rapporto) per i messaggi di diagnostica. Tutti gli identificativi sono codificati come riferimenti a un campo *Dictionary* (dizionario) di lunghezza variabile. Questo fonisce compressione quando i nodi *Custodian* o *Report* coincidono con la sorgente o la destinazione. Infatti gran parte del formato del messaggio è stato progettato pensando sia all'estensibilità che all'efficienza tramite una rappresentazione compatta dei campi a lunghezza variabile. La rappresentazione compatta è importante per le linee wireless e i nodi con risorse limitate come in una rete di sensori.

Successivamente viene il campo *Creation* (creazione) che trasporta il tempo in cui è stato creato il bundle insieme a un numero di sequenza dato dalla sorgente, più un campo *Lifetime* che indica il tempo in cui i dati nel bundle non saranno più utili. Questi campi esistono perché i dati possono essere memorizzati per lunghi periodi nei nodi DTN e ci deve essere un modo per rimuovere i dati stagnanti dalla rete. Diversamente da Internet, questi campi richiedono che i nodi DTN abbiano orologi sincronizzati in modo lasco.

Il blocco primario è completo col campo *Dictionary*. Qundi c'è il blocco di payload. Questo blocco inizia con un breve campo *Type* che lo identifica come payload, seguito da un piccolo gruppo di flag che descrivono le opzioni di elaborazione. Quindi c'è il campo *Data* (dati) preceduto da un campo *Length* (lunghezza). Infine, ci può essere un altro blocco opzionale, come un blocco che trasporta parametri di sicurezza.

La comunità scientifica sta ancora esplorando molti aspetti delle DTN. Buone strategie di routing dipendono dalla natura dei contatti, come è stato già detto. Memorizzare dati in rete solleva altre questioni. Ora il controllo di congestione deve considerare la memorizzazione nei nodi come un altro tipo di risorsa che si può esaurire. La mancanza di comunicazioni end-to-end accentua i problemi di sicurezza: prima che un nodo DTN prenda in custodia un bundle potrebbe voler sapere se il mittente è autorizzato a usare la rete e che il bundle è probabilmente desiderato dal destinatario. Le soluzioni di questi problemi dipenderanno dal tipo di DTN, in quanto le reti spaziali sono diverse dalle reti di sensori.

6.8 Sommario

Il livello di trasporto è la chiave per capire i protocolli stratificati; fornisce molti servizi, il più importante dei quali è un flusso di byte affidabile, end-to-end e orientato alla connessione dal mittente al ricevente. È possibile accedervi con primitive di servizio che consentono la costituzione, l'utilizzo e il rilascio delle connessioni. Una comune interfaccia per il livello di trasporto è quella fornita dalle socket di Berkeley.

I protocolli di trasporto devono essere in grado di gestire connessioni su reti inaffidabili. La costituzione della connessione è complicata dall'esistenza di pacchetti duplicati e ritardati che possono ricomparire in momenti inopportuni. Per gestirli, servono handshake a tre vie per stabilire le connessioni. Rilasciare una connessione è più facile che stabilirne una, ma non è banale a causa del problema dei due eserciti.

Anche quando il livello di rete è completamente affidabile, il livello di trasporto ha molto lavoro da svolgere: deve gestire tutte le primitive di servizio, gestire le connessioni e i timer, allocare banda tramite il controllo di congestione e usare una finestra scorrevole di dimensione variabile per implementare il controllo di flusso.

Oltre a tenere traccia dei cambiamenti nell'uso della rete, il controllo di congestione dovrebbe allocare equamente tutta la banda disponibile ai flussi in competizione tra loro. La legge di controllo AIMD converge verso un'allocazione equa ed efficiente.

Internet possiede due protocolli di trasporto principali: UDP e TCP. UDP è un protocollo senza connessione che è principalmente un involucro per i pacchetti IP, con la funzionalità aggiuntiva di fare multiplexing e demultiplexing da e verso più processi usando un singolo indirizzo IP. UDP può essere utilizzato per le interazioni client-server, per esempio utilizzando RPC, e può anche essere impiegato per costruire protocolli real-time come RTP.

Il principale protocollo di trasporto Internet è TCP: fornisce un flusso di byte bidirezionale, affidabile e dotato di controllo di congestione con un'intestazione di 20 byte su tutti i segmenti. Molte energie sono state dedicate all'ottimizzazione delle prestazioni di TCP, che fa uso degli algoritmi di Nagle, Clark, Jacobson, Karn e altri.

Le prestazioni della rete sono generalmente dettate dagli overhead di protocollo e di elaborazione del segmento, ma questa situazione può peggiorare ad alte velocità. I protocolli dovrebbero essere progettati per ridurre il numero di segmenti e per funzionare su percorsi con elevati valori del prodotto banda-ritardo. Nelle reti ad alte capacità si dovrebbero usare protocolli semplici ed elaborazioni veloci.

Le reti con tolleranza di ritardo forniscono un servizio di consegna attraverso reti che hanno connettività occasionale o grossi ritardi sui collegamenti. I nodi intermedi memorizzano, trasportano e inoltrano bundle di informazione così che alla fine vengano consegnati anche se non ci sono mai percorsi attivi tra mittente e destinatario.

Problemi

1.Nelle primitive di trasporto dell'esempio della Figura 6.2 LISTEN è una chiamata bloccante. È strettamente necessaria? Se non lo è, spiegate come si potrebbe utilizzare una primitiva non bloccante. Quale vantaggio si avrebbe rispetto allo schema descritto nel testo?

2.Le primitive del servizio di trasporto presumono asimmetria tra i due punti terminali durante la creazione della connessione, un estremo (server) esegue LISTEN, mentre l'altro (client) esegue CONNECT. Tuttavia in applicazioni peer to peer come i sistemi di file sharing, tra cui BitTorrent, tutti gli estremi sono paritetici e non c'è funzionalità server o client. Come possono le primitive di servizio di trasporto essere utilizzate per creare tali applicazioni peer to peer?

3. Nel modello della Figura 6.4 si presume che i pacchetti possano essere persi dal livello di rete e debbano quindi ricevere individualmente un acknowledgement. Supponiamo che il livello di rete sia affidabile al 100% e non perda mai pacchetti. Quali eventuali modifiche sono necessarie alla Figura 6.4?

4. In entrambe le parti della Figura 6.6 esiste un commento che indica che il valore di *SERVER_PORT* deve essere lo stesso sul client e sul server. Perché è così importante?

5. Nell'esempio di File Server (Figura 6.6) può la chiamata a sistema connect( ) nel client fallire per qualche ragione diversa da quella per cui la coda di attesa è piena nel server? Presumete che la rete sia perfetta.

6. Un metodo per decidere se avere un server attivo tutto il tempo o farlo attivare a richiesta usando un process server si basa sulla frequenza di utilizzo del servizio fornito. Potete pensare a qualche altro criterio da adottare in questo caso?

7. Si supponga di usare uno schema per la generazione dei numeri di sequenza iniziali guidato da un clock a 15 bit. Un ciclo di clock dura 100 ms e la durata massima del pacchetto è 60 secondi. Con che frequenza è necessaria la risincronizzazione:

a) nel caso peggiore

b) quando i dati consumano 240 numeri di sequenza al minuto.

8. Perché il massimo tempo di vita di un pacchetto, *T*, deve essere abbastanza grande da assicurare che non solo il pacchetto ma anche i suoi acknowledgement siano svaniti?

9. Si immagini che per impostare le connessioni venga utilizzato un handshake a due vie anziché a tre. In altre parole, il terzo messaggio non è richiesto. Potrebbero verificarsi delle situazioni di stallo? Se ne dia un esempio o si dimostri che non ne esistono.

10. Si immagini un problema generalizzato di *n* eserciti, in cui l'accordo di qualunque coppia di eserciti blu è sufficiente per la vittoria. Esiste un protocollo che consente ai blu di vincere?

11. Si consideri il problema del ripristino dal crash di un host (Figura 6.18). Se l'intervallo tra la scrittura e l'invio di un acknowledgement (o viceversa) può essere reso relativamente piccolo, quali sono le due strategie mittente-ricevente migliori per minimizzare le probabilità di fallimento

del protocollo?

12. Nella Figura 6.20 si supponga di aggiungere un nuovo flusso *E* che usa il percoso da *R*1 a *R2* a *R6*. Come cambia l'allocazione di banda max-min per i cinque flussi?

13. Alcune altre politiche per l'equità nel controllo di congestione sono *additive increase additive decrease* (AIAD), *multiplicative increase additive decrease* (MIAD) e *multiplicative increase multiplicative decrease* (MIMD). Si discutano queste tre politiche in termini di convergenza e stabilità.

14. Perché esiste UDP? Non sarebbe stato sufficiente consentire ai processi utente di inviare pacchetti IP grezzi?

15. Si consideri un semplice protocollo a livello di applicazione costruito su UDP che consenta a un client di recuperare un file da un server remoto con un indirizzo ben noto. Il client invia prima una richiesta con il nome del file; il server risponde con una sequenza di pacchetti contenenti parti diverse del file richiesto. Per assicurare l'affidabilità e la consegna sequenziale, il client e il server utilizzano un protocollo stop-and-wait. Ignorando le ovvie questioni relative alle prestazioni, questo protocollo ha un problema? Pensate attentamente alle possibilità di crash dei processi.

16. Un client invia una richiesta di 128 byte a un server situato a 100 Km di distanza su una fibra ottica a 1 gigabit. Qual è l'efficienza della linea durante la chiamata a procedura remota?

17. Si consideri nuovamente la situazione del problema precedente. Calcolate il tempo minimo di risposta per la linea a 1 Gbps e per una linea a 1 Mbps. Quale conclusione è possibile trarre?

18. Sia UDP sia TCP utilizzano i numeri di porta per identificare l'entità di destinazione durante la consegna di un messaggio. Esponete due ragioni per cui questi protocolli hanno inventato un nuovo ID astratto (numeri di porta) anziché utilizzare gli ID di processo, che esistevano già nel momento in cui sono stati progettati.

19. Alcune implementazioni di RPC forniscono un'opzione al client per usare RPC implementata su UDP o su TCP. In quali condizioni un client preferirà usare RPC su UDP e in quali preferirà usare RPC su TCP?

20. Si considerino due reti, *N*1 e *N*2, con lo stesso ritardo medio tra una sorgente *A* e una destinazione *D*. In *N*1 il ritardo sperimentato dai diversi pacchetti è distribuito uniformemente con un ritardo massimo di 10 secondi, mentre in *N*2 il 99% dei pacchetti è soggetto a meno di un secondo di ritardo senza limite sul ritardo massimo. Si dica come RTP possa essere usato in questi due casi per trasmettere un flusso di dati audio/video real-time.

21. Qual è la dimensione totale minima della MTU di TCP, comprendendo l'overhead di TCP e IP ma non quello del livello data link?

22. La frammentazione e il riassemblaggio dei datagrammi sono gestiti da IP e invisibili a TCP. Questo significa che TCP non si deve preoccupare dell'arrivo dei dati in ordine errato?

23. RTP è utilizzato per trasmettere audio con qualità CD, composto da una coppia di campioni di 16 bit 44.100 volte al secondo, un campione per ogni canale stereo. Quanti pacchetti al secondo deve trasmettere RTP?

24. Sarebbe possibile inserire il codice RTP nel kernel del sistema operativo, insieme al codice UDP? Motivate la risposta.

25. Un processo sull'host 1 è stato assegnato alla porta *p*, mentre un processo sull'host 2 è stato assegnato alla porta *q*. È possibile stabilire due o più connessioni TCP tra queste due porte contemporaneamente?

26. Nella Figura 6.36 abbiamo visto che, oltre al campo *Acknowledgement* di 32 bit, esiste un bit *ACK*. Aggiunge realmente qualcosa? Perché (o perché no)?

27. Il massimo payload di un segmento TCP è 65.495 byte. Perché è stato scelto questo numero strano?

28. Si descrivano due modi per entrare nello stato *SYN RCVD* della Figura 6.39.

29. Si consideri l'effetto dello slow start su una linea con un round-trip time pari a 10 ms e nessuna congestione. La finestra di ricezione è di 24 KB e la dimensione massima del segmento è 2 KB. Quanto tempo è necessario prima che sia possibile inviare la prima finestra completa?

30. Si supponga che la finestra di congestione TCP sia impostata a 18 KB e che si verifichi un timeout. Quanto sarà grande la finestra se le quattro trasmissioni successive hanno successo? Si assuma che la dimensione massima del segmento sia 1 KB.

31. Se il round-trip time di TCP, *RTT*, è attualmente 30 ms e i seguenti acknowledgement vengono ricevuti rispettivamente in 26, 32 e 24 ms, qual è il nuovo *RTT* stimato utilizzando l'algoritmo di Jacobson? Si usi a = 0,9.

32. Un host TCP sta inviando finestre complete di 65.535 byte su un canale a 1 Gbps con un ritardo unidirezionale di 10 ms. Qual è la massima velocità dei dati raggiungibile? Qual è l'efficienza della linea?

33. Qual è la velocità massima della linea a cui un host può inviare payload TCP di 1500 byte con un tempo di vita massimo del pacchetto di 120 secondi, senza dover riutilizzare i numeri di sequenza? Si tenga conto dell'overhead TCP, IP ed Ethernet e si assuma che i frame Ethernet possano essere inviati continuamente.

34. Per occuparsi delle limitazioni della versione 4 di IP è stato fatto un grande sforzo attraverso IETF: ne è conseguita la progettazione della versione 6 di IP e permane una significativa riluttanza nell'adottare questa nuova versione. Tuttavia non è necessario uno sforzo così grande per le limitazioni di TCP. Si spieghi perché.

35. In una rete con una dimensione e tempo di vita massimi dei segmenti rispettivamente di 128 byte e 30 secondi e un numero di sequenza di 8 bit, qual è la velocità massima dei dati per ogni connessione?

36. Si supponga di misurare il tempo necessario per ricevere un segmento. Quando avviene un interrupt, viene letto l'orologio di sistema in millisecondi; dopo l'elaborazione del segmento l'orologio viene letto nuovamente. Misurate 0 ms per 270.000 volte e 1 ms per 730.000 volte. Quanto tempo è necessario per ricevere un segmento?

37. Una CPU esegue istruzioni alla velocità di 1000 MIPS. I dati possono essere copiati 64 bit alla volta e la loro copia richiede 10 istruzioni. Se un pacchetto in ingresso deve essere copiato quattro volte, questo sistema è in grado di gestire una linea a 1 Gbps? Per semplicità si assuma che tutte le istruzioni, anche quelle che leggono o scrivono in memoria, siano eseguite alla velocità di 1000 MIPS.

38. Per aggirare il problema del riutilizzo dei numeri di sequenza quando sono ancora in circolo vecchi pacchetti si potrebbero impiegare numeri di sequenza di 64 bit. Tuttavia una fibra ottica può lavorare teoricamente a 75 Tbps. Quale durata massima del pacchetto è necessaria per assicurare che le future reti a 75 Tbps non incontrino il suddetto problema anche con i numeri di sequenza a 64 bit? Presumete che ogni byte possieda il proprio numero di sequenza, come in TCP.

39. Per una rete a 1 Gbps operante su 4000 Km, il fattore limitante è il ritardo e non la banda. Si consideri una MAN con la sorgente e la destinazione distanti 20 Km. A quale velocità dei dati il round-trip time dovuto alla velocità della luce equivale al ritardo di trasmissione per un pacchetto di 1 KB?

40. Si calcoli il prodotto banda-ritardo per le seguenti reti: (1) TI (1,5 Mbps), (2) Ethernet (10 Mbps), (3) T3 (45 Mbps) e (4) STS-3 (155 Mbps). Si assuma un RTT pari a 100 ms ricordando che l'intestazione TCP riserva 16 bit alla dimensione della finestra. Quali sono le implicazioni di questo fatto alla luce dei calcoli?

41. Qual è il prodotto banda-ritardo per un canale a 50 Mbps su un satellite geostazionario? Se i pacchetti sono tutti di 1500 byte (compreso l'overhead), quanto deve essere grande la finestra in termini di numero di pacchetti?

42. Il file server della Figura 6.6 non è perfetto, pertanto si possono apportare alcuni miglioramenti. Si facciano le seguenti modifiche:

– fornire al client un terzo argomento che specifica un intervallo di byte

– aggiungere un flag -w al client che consenta la scrittura del file sul server.

43. Una funzione che tutti i protocolli di rete necessitano è quella relativa alla manipolazione dei messaggi. Si ricordi che i protocolli manipolano i messaggi aggiungendo/togliendo intestazioni. Alcuni protocolli possono spezzare un messaggio in più frammenti e più tardi unire questi diversi frammenti di nuovo in un solo messaggio. A tal fine, si progetti e implementi una libreria di gestione dei messaggi che fornisca supporto per la creazione di un nuovo messaggio, l'aggiunta e l'eliminazione di un'intestazione a/da un messaggio, la suddivisione di un messaggio in due parti con la relativa ricombinazione e il salvataggio di una copia di un messaggio. L'implementazione dovrà minimizzare il più possibile la copia di dati da un buffer a un altro ed è fondamentle che le operazioni che manipolano i messaggi non tocchino i dati del messaggio ma, invece, gestiscano solo puntatori.

44. Si progetti e implementi un sistema di messaggistica (chat) che permette a più gruppi di utenti di conversare. Il coordinatore della chat, che risiede a un indirizzo di rete ben noto, utilizza UDP per la comunicazione con i client, imposta i server per ogni sessione di conversazione e mantiene un elenco delle sessioni. Esiste un server di conversazione per ciascuna sessione e utilizza TCP per la comunicazione con i client. Un client consente agli utenti di creare, unirsi e abbandonare una sessione di conversazione. Si progetti e implementi il codice per il coordinatore, il server e il client.

CAPITOLO 7
Il livello applicazione

Terminate le necessarie premesse si può passare al livello in cui si trovano tutte le applicazioni. I livelli al di sotto del livello applicazione forniscono il trasporto affidabile, ma non svolgono alcun lavoro per gli utenti. In questo capitolo studieremo alcune applicazioni reali delle reti. Tuttavia anche nel livello applicazione sussiste l'esigenza dei protocolli di supporto che permettano alle applicazioni di funzionare. Per questo motivo studieremo uno di questi protocolli prima di iniziare l'esame delle applicazioni vere e proprie: il DNS, che gestisce i nomi all'interno di Internet. Successivamente esamineremo tre applicazioni reali: posta elettronica, World Wide Web e contenuti multimediali. Infine tratteremo la distribuzione di contenuti e in particolare le reti peer-to-peer.

7.1 DNS – domain name system

Anche se i programmi possono teoricamente fare riferimento a host, caselle di posta e altre risorse tramite i loro indirizzi di rete (per esempio, IP), le persone ricordano con difficoltà questi indirizzi. Inoltre aprire la pagina Web di un'azienda dall'indirizzo *128.111.24.41* implica che l'azienda, nel caso spostasse il server su una macchina con un indirizzo IP diverso, sarebbe costretta ad avvertire tutti del cambio di indirizzo. Per questi motivi sono stati introdotti nomi ad alto livello e leggibili per separare i nomi dei computer dai rispettivi indirizzi; in questo modo l'indirizzo dell'azienda potrebbe essere simile a *www.cs.washington.edu* indipendentemente dall'indirizzo IP. Ciò nonostante, poiché la rete in sé sa interpretare solo indirizzi numerici, sono necessari meccanismi per convertire i nomi in indirizzi di rete. Nei paragrafi successivi studieremo come viene realizzata su Internet questa associazione.

Ai tempi di ARPANET esisteva semplicemente un file, *host.txt,* che elencava tutti gli host e i loro indirizzi IP. Ogni notte tutti gli host lo prelevavano dal sito da cui era gestito. Per una rete composta da poche centinaia di grandi calcolatori questo approccio era abbastanza efficace.

Tuttavia, quando alla rete furono connessi milioni di PC, fu chiaro a tutti che questo metodo non poteva funzionare per sempre; da una parte la dimensione del file sarebbe diventata troppo elevata, mentre dall'altra c'era il rischio (decisamente più grave) di continui conflitti tra i nomi degli host, se questi non fossero stati gestiti centralmente; ma ciò è impensabile in una vasta rete internazionale, a causa del carico e della latenza. Per risolvere questi problemi nel 1983 fu inventato **DNS** (*domain name system,* sistema dei nomi di dominio), diventato una parte chiave di Internet.

L'essenza del DNS è l'introduzione di uno schema di denominazione gerarchico basato su domini e di un database distribuito per l'implementazione di questo schema di denominazione. È principalmente utilizzato per associare nomi di host a indirizzi IP, ma può essere utilizzato anche per altri scopi. DNS è definito negli RFC 1034, 1035, 2181 e in molti altri.

In breve, il DNS viene utilizzato come segue. Per associare un nome a un indirizzo IP un programma applicativo invoca una procedura di libreria chiamata **resolver** (*risolutore*), passando il nome come parametro. Abbiamo visto un esempio di uso di resolver, tramite *gethostbyname*, nella Figura 6.6. Il resolver invia un pacchetto UDP contenente la richiesta a un server DNS locale, che cerca il nome e restituisce l'indirizzo IP al DNS locale che a sua volta lo restituisce al resolver. Equipaggiato dell'indirizzo IP, il programma può quindi stabilire una connessione TCP con la destinazione oppure inviarle pacchetti UDP.

Lo spazio dei nomi DNS

La gestione di un insieme di nomi grande e in continuo cambiamento non è un problema da poco. Nel sistema postale la gestione dei nomi viene svolta imponendo che sulle lettere siano specificate (in modo implicito o esplicito) la nazione, lo stato o la provincia, la città e la via del destinatario.

Utilizzando questo tipo di indirizzo gerarchico non è possibile fare confusione tra Marvin Anderson residente in Main Street a White Plains, (New York) e Marvin Anderson residente in Main Street ad Austin, (Texas). Il DNS funziona allo stesso modo.

In Internet, in cima alla gerarchia di assegnazione dei nomi, c'è un'organizzazione chiamata **ICANN** (*Internet corporation for assigned names and numbers*), creata a questo scopo nel 1998 come parte della maturazione di Internet rispetto alle problematiche economiche mondiali. Concettualmente, Internet è divisa in oltre 250 **domini di primo livello** (*top level*), dove ogni dominio copre molti host. Ogni dominio è partizionato in sottodomini, che sono a loro volta divisi e così via. Tutti questi domini possono essere rappresentati con una struttura ad albero, come mostrato nella Figura 7.1. Le foglie rappresentano i domini che non hanno sottodomini (ma naturalmente contengono computer). Un dominio foglia può contenere un singolo host o può rappresentare una società e contenere migliaia di host.

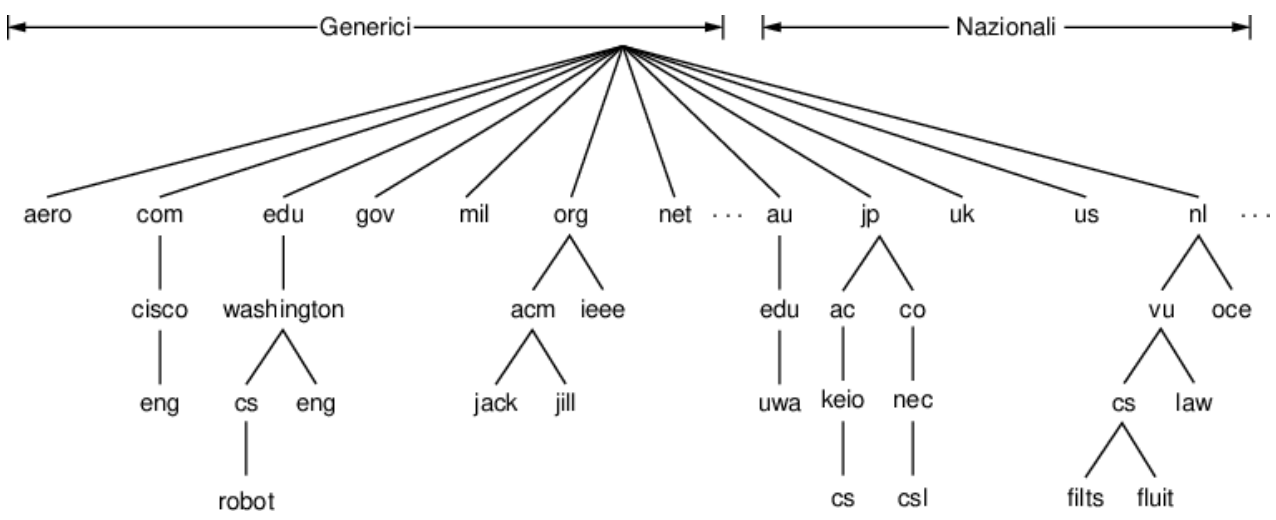


Figura 7.1 Una porzione dello spazio dei nomi di dominio di Internet.

I domini di primo livello sono di due tipi: generici e per nazioni. I domini generici, elencati nella Figura 7.2, sono sia i domini originali dal 1980 sia quelli introdotti richiedendoli a ICANN. Altri domini di primo livello generici saranno aggiunti in futuro.

Dominio	Uso previsto	Data di creazione	Ristretto?
com	Commerciale	1985	No
edu	Educazione e ricerca	1985	Sì
gov	Pubblica amministrazione	1985	Sì

Dominio	Uso previsto	Data di creazione	Ristretto?
int	Organizzazioni internazionali	1988	Sì
mil	Esercito	1985	Sì
net	Network provider	1985	No
org	Organizzazioni non-profit	1985	No
aero	Trasporto aereo	2001	Sì
biz	Commercio	2001	No
coop	Cooperative	2001	Sì
info	Informazioni	2002	No
museum	Musei	2002	Sì
name	Persone	2002	No
pro	Professionisti	2002	Sì
cat	Catalano	2005	Sì
jobs	Occupazione	2005	Sì
mobi	Dispositivi mobili	2005	Sì
tel	Contatti personali	2005	Sì
travel	Agenzie viaggi	2005	Sì
xxx	Industria del sesso	2010	No

Figura 7.2 Domini generici di primo livello.

I domini nazionali includono una voce per ogni nazione, come definito in ISO 3166. Nel 2010 sono stati introdotti nomi di domini di nazioni internazionalizzati, che usano alfabeti non latini. Questi domini permettono di dare nomi agli host in arabo, cirillico, cinese e altre lingue.

In generale, è facile ottenere un dominio di secondo livello come *nome-azienda.com*. I domini di primo livello sono gestiti da **registrar** (autorità di registrazione dei domini) stabiliti da ICANN. È solamente necessario contattare il registrar per il dominio di primo livello corrispondente *(com* in questo caso), per controllare se il nome prescelto sia disponibile e se altri non possiedano già il marchio. Se non vi sono problemi, al richiedente viene addebitata una modesta tariffa annuale e ottiene il nome.

Tuttavia, man mano che Internet diventa sempre più commerciale e internazionale, aumentano anche i contenziosi, soprattutto riguardo ai nomi. Queste controversie riguardano anche ICANN stesso. Per esempio, la creazione di un dominio *xxx* richiese alcuni anni e contenziosi da risolvere. Pubblicare coscientemente contenuti per adulti nel proprio dominio è una cosa buona o cattiva? Alcune persone non vogliono che esistano contenuti per adulti disponibili su Internet nel modo più assoluto, mentre altri vogliono che siano messi in un unico dominio in modo che i filtri per

bambini possano facilmente individuarli e bloccarli. Alcuni domini si organizzano autonomamente, mentre altri hanno restrizioni su chi possa ottenere un nome, come si può notare nella Figura 7.2. Ma quali restrizioni sono appropriate? Si prenda il dominio *pro,* per esempio. È pensato per i professionisti certificati. Ma chi sono i professionisti? Medici e avvocati, chiaramente; ma che cosa dire di fotografi freelance, insegnanti di pianoforte, maghi, idraulici, parrucchieri, disinfestatori, tatuatori, mercenari e prostitute? Si tratta di occupazioni professionali e quindi idonee? Chi lo stabilisce?

C'è anche una questione finanziaria legata ai nomi. La nazione del Tuvalu vendette un contratto di affitto sul suo dominio *tv* per 50 milioni di dollari, solo perché il codice della nazione era particolarmente adatto a pubblicizzare siti televisivi. Virtualmente qualsiasi parola (inglese) è stata utilizzata nel dominio *com*, insieme a molti errori linguistici comuni; provate con articoli casalinghi, animali, piante, parti del corpo e così via. Anche la pratica di registrare un dominio solo per venderlo alla parte interessata a un prezzo molto superiore ha un nome: **cybersquatting**. Molte aziende che si mossero lentamente per quanto riguardava il marchio ai primi esordi di Internet, trovarono il loro nome di dominio naturale già preso quando tentarono di acquistarlo. In generale, finché non vengono violati i diritti sul marchio e non c'è alcuna frode, la politica dei nomi è first-come, first-serve. Tuttavia, si stanno ancora definendo politiche per risolvere le dispute sui nomi.

Ogni dominio è denominato dal percorso compreso tra esso e la radice (non denominata). I componenti sono separati da punti ("dot"), di conseguenza la divisione ingegneristica di Cisco potrebbe utilizzare *eng.cisco.com.,* piuttosto che un nome in stile UNIX come */com/cisco/eng.* Occorre notare che questa denominazione gerarchica garantisce che *eng.cisco.com.* non sia in conflitto con un potenziale utilizzo di *eng* in *eng.washington.edu*., che potrebbe essere utilizzato dal dipartimento di lingua inglese dell'università di Washington.

I nomi di dominio possono essere assoluti o relativi. Un nome di dominio assoluto termina sempre con un punto (per esempio *eng.cisco.com.*), al contrario di un nome relativo. I nomi relativi devono essere interpretati in un contesto per determinarne univocamente il reale significato. In entrambi i casi, il dominio fa riferimento a un nodo specifico nella struttura e a tutti i nodi sottostanti.

I nomi di dominio non distinguono tra maiuscole e minuscole, pertanto *edu, Edu* ed *EDU* indicano la stessa cosa. I componenti del nome possono essere lunghi fino a 63 caratteri, mentre il percorso completo non deve superare 255 caratteri.

In teoria, i domini possono essere inseriti nell'albero sia nei domini generici sia in quelli delle nazioni. Per esempio, *cs.washington.edu* poteva essere elencato sotto il dominio per nazioni *us* come *cs.washington.wa.us.* In pratica, però, la maggior parte degli enti e delle istituzioni negli Stati Uniti utilizza un nome generico, mentre la maggior parte di quelli esterni agli Stati Uniti sceglie il dominio della sua nazione. Non esistono regole per la registrazione in due domini di primo livello, spesso le società grandi lo fanno (per esempio *sony.com*, *sony.net* e *sony.nl).*

Ogni dominio controlla come allocare i domini sottostanti. Per esempio, il Giappone possiede i domini *ac.jp* e *co.jp*[1] che rispecchiano *edu* e *com.* L'Olanda non fa questa distinzione e pone tutte le organizzazioni direttamente sotto *nl.* Di conseguenza, i tre nomi seguenti rappresentano dipartimenti di informatica delle rispettive università:

1. *cs.washington.edu* (University of Washington, Stati Uniti)
2. *cs.vu.nl* (Vrije Universiteit, Olanda)

3. *cs.keio.ac.jp* (Keio University, Giappone).

Per creare un nuovo dominio è necessaria l'autorizzazione del dominio in cui sarà incluso. Per esempio, se alla University of Washington viene avviato un gruppo VLSI che desidera essere conosciuto come *vlsi.cs.washington.edu*, deve richiedere l'autorizzazione a chi gestisce *cs.washington.edu.* Allo stesso modo, se viene concesso lo statuto a una nuova università, per esempio la University of Northern South Dakota, questa deve chiedere al gestore del dominio *edu* di assegnarle *unsd.edu* (sempre che sia ancora disponibile). In questo modo, i conflitti tra i nomi vengono evitati e ogni dominio può tenere traccia di tutti i suoi sottodomini. Una volta creato e registrato un nuovo dominio, questo può creare sottodomini, come *cs.unsd.edu,* senza bisogno di ottenere l'autorizzazione delle parti della struttura che si trovano al di sopra.

La denominazione segue i confini dell'organizzazione, non le reti fisiche. Per esempio, se le divisioni di informatica e ingegneria elettronica sono situate nello stesso edificio e condividono la medesima LAN, possono comunque utilizzare domini distinti. Allo stesso modo, se il dipartimento di informatica viene diviso in Babbage Hall e Turing Hall, gli host in entrambi gli edifici apparterranno di norma allo stesso dominio.

Record delle risorse di dominio

A ogni dominio, che sia rappresentato da un singolo host o sia un dominio di primo livello, può essere associato un insieme di **resource record** (*record di risorse*) che formano il database DNS. Per un singolo host il record delle risorse più comune è quello relativo all'indirizzo IP, ma possono esisterne molti altri tipi. Quando un resolver sottopone un nome di dominio a un server DNS (che indicheremo con *name server*), ciò che ottiene sono i record delle risorse associate a tale nome. Di conseguenza, la funzione principale del DNS è associare i nomi di dominio ai record delle risorse.

Un record delle risorse è una quintupla. Anche se sono codificati in binario per efficienza, nella maggior parte dei casi i record delle risorse sono presentati come testo ASCII, uno per riga. Il formato che useremo è il seguente:

```
Domain_name    Time_to_live    Class    Type    Value
```

Domain_name indica il dominio a cui si riferisce il record. Normalmente esistono molti record per ogni dominio e ogni copia del database contiene informazioni su più domini. Questo campo è pertanto la prima chiave di ricerca utilizzata per soddisfare le interrogazioni. L'ordine dei record nel database non è significativo.

Il campo *Time_to_live* offre un indicatore della stabilità del record. Alle informazioni altamente stabili viene assegnato un valore elevato, come 86.400 (il numero di secondi in un giorno). Alle informazioni di breve durata viene invece assegnato un valore piccolo, come 60 (1 minuto). Torneremo su questo punto in seguito quando parleremo dell'uso delle cache.

Il terzo campo di ogni record delle risorse è *Class*. Per le informazioni che riguardano Internet è sempre *IN*; per altre informazioni non riguardanti Internet possono essere utilizzati altri codici, che in pratica però sono utilizzati raramente.

Il campo *Type* specifica il tipo di record tra i molti di DNS. I tipi più importanti sono elencati nella Figura 7.3.

Tipo	Significato	Valore
SOA	Fonte delle informazioni	I parametri per questa zona
A	Indirizzo IPv4 di un host	Intero a 32 bit
AAAA	Indirizzo IPv6 di un host	Intero a 28 bit
MX	Mail exchanger	Principale host con cui il dominio accetta la posta elettronica
NS	Name server	Name server per questo dominio
CNAME	Nome del dominio	Nome canonico del dominio
PTR	Puntatore	Alias per un indirizzo IP
SPF	Politica per l'invio della posta elettronica	Politica per la codifica del testo nella posta in uscita
SRV	Servizio	Host che lo fornisce
TXT	Testo	Testo ASCII di descrizione

Figura 7.3 Principali tipi di record delle risorse.

Un record *SOA* fornisce il nome della fonte primaria di informazioni sulla zona del name server (descritta nelle righe successive), l'indirizzo di posta elettronica del suo amministratore, un numero di serie univoco oltre che diversi flag e timeout.

Il tipo di record più importante è il record *A*. Contiene l'indirizzo IPv4 a 32 bit di un'interfaccia di un host. Il record corrispondente *AAAA* (o quadrupla A) contiene l'indirizzo IPv6 a 128 bit. Ogni host di Internet deve avere almeno un indirizzo IP, affinché le altre macchine possano comunicare con lui. Alcuni host presentano due o più interfacce di rete e in questo caso possiedono un record delle risorse di tipo *A o AAAA* per ciascuna. Di conseguenza il DNS può fornire più indirizzi per un singolo nome.

Un tipo comune di record è il record *MX*; questo specifica il nome dell'host configurato per accettare la posta elettronica in ingresso per il dominio specificato. È utilizzato perché non tutte le macchine sono capaci di accettare posta elettronica. Se qualcuno desidera inviare messaggi a

bill@microsoft.com, per esempio, l'host di invio deve trovare un server di posta all'interno del dominio *microsoft.com* disposto ad accettare posta elettronica. Il record *MX* può fornire questa informazione.[2]
Un altro record importante è il record *NS* che specifica il name server per il dominio o sottodominio. È usato come parte del processo di ricerca dei nomi, che verrà descritto a breve.

I record *CNAME* consentono la creazione di alias. Per esempio, una persona che ha acquisito familiarità con la generale nomenclatura di Internet e desidera inviare un messaggio a qualcuno il cui nome di login è *paul* nel dipartimento di informatica del M.I.T., potrebbe eseguire un tentativo con *paul@cs.mit.edu.* In realtà, questo indirizzo non funzionerà perché il dominio del dipartimento di informatica di M.I.T. è *csail.mit.edu*. Tuttavia, come servizio per le persone che non lo sanno, il MIT potrebbe creare una voce *CNAME* che diriga persone e programmi sulla strada corretta. Una voce come la seguente potrebbe occuparsi della questione:

```
cs.mit.edu    86400    IN    CNAME    csail.mit.edu
```

Come *CNAME*, *PTR* punta verso un altro nome. Tuttavia, a differenza di *CNAME* che è soltanto una macro (un meccanismo che sostituisce a una stringa un'altra stringa), *PTR* è un tipo di dati DNS regolare la cui interpretazione dipende dal contesto in cui viene trovato. In pratica, è quasi sempre utilizzato per associare un nome a un indirizzo IP per consentire le ricerche che forniscono gli indirizzi IP e restituiscono i nomi delle macchine corrispondenti. Sono chiamate **reverse lookup** (*ricerca inversa*).

SRV è un tipo di record di recente introduzione che permette a un host di essere identificato per un dato servizio in un dominio. Per esempio, il server Web per *cs.washington.edu* potrebbe venire identificato come *cockatoo.cs.washington.edu*.[3] Questo record generalizza il record *MX* che esegue lo stesso compito, ma solo per i server di posta elettronica.
Anche *SPF* è un tipo di record più recente. Permette al dominio di codificare informazioni riguardanti quali macchine nel dominio manderanno le e-mail al resto di Internet. Questo aiuta le macchine che ricevono le e-mail a controllare che siano valide. Se una mail viene ricevuta da una macchina che si denomina *dodgy*, ma il record del dominio afferma che le mail possono essere spedite al di fuori solo da una macchina che si chiama *smtp*, probabilmente si tratta di spam.

Infine i record *TXT* furono pensati per consentire ai domini di identificare sé stessi in modo arbitrario. Oggi vengono usati per contenere informazioni interpretabili dai calcolatori, tipicamente quelle riguardanti *SPF*.

In conclusione è disponibile il campo *Value,* che può essere un numero, un nome di dominio o una stringa ASCII. La semantica dipende dal tipo di record. Una breve descrizione dei campi *Value* per ognuno dei tipi principali di record è data nella Figura 7.3.

Per un esempio del tipo di informazioni presenti nel database del DNS di un dominio si osservi la Figura 7.4. Questa figura mostra parte di un database (ipotetico) per il dominio *cs.vu.nl* mostrato nella Figura 7.1. Il database contiene sette tipi di record di risorse.

```
; Authoritative data for cs.vu.nl
cs.vu.nl.       86400   IN   SOA      star boss (9527,7200,7200,241920,86400)
cs.vu.nl.       86400   IN   MX       1 zephyr
cs.vu.nl.       86400   IN   MX       2 top
cs.vu.nl.       86400   IN   NS       star

star            86400   IN   A        130.37.56.205
zephyr          86400   IN   A        130.37.20.10
top             86400   IN   A        130.37.20.11
www             86400   IN   CNAME    star.cs.vu.nl
ftp             86400   IN   CNAME    zephyr.cs.vu.nl

flits           86400   IN   A        130.37.16.112
flits           86400   IN   A        192.31.231.165
flits           86400   IN   MX       1 flits
flits           86400   IN   MX       2 zephyr
flits           86400   IN   MX       3 top

rowboat                 IN   A        130.37.56.201
                        IN   MX       1 rowboat
                        IN   MX       2 zephyr

little-sister           IN   A        130.37.62.23

laserjet                IN   A        192.31.231.216
```

Figura 7.4 Una porzione di un possibile database DNS per *cs.vu.nl*.

La prima riga della Figura 7.4 è un commento, mentre la seconda offre alcune informazioni di base sul dominio, di cui non ci interesseremo. Dopo di che appaiono due voci che comunicano il primo e il secondo luogo dove tentare di consegnare la posta elettronica inviata a *persona@cs.vu.nl.* Per prima cosa è opportuno provare *zephyr* (una macchina specifica); in caso di fallimento, si dovrebbe provare *top.* La linea successiva identifica il nome del name server per il dominio come *star*.

Dopo la riga vuota, aggiunta a fini di leggibilità, appaiono le righe che indicano gli indirizzi IP di *star*, *zephir* e *top*. Successivamente appare un alias, *www.cs.vu.nl*. La creazione di questo alias consente a *cs.vu.nl* di cambiare il suo server World Wide Web senza invalidare gli indirizzi utilizzati dalle persone per raggiungerlo. Un discorso simile vale per *ftp.cs.vu.nl.*

La parte che descrive la macchina *flits* elenca per essa due indirizzi IP e tre scelte per la gestione della posta elettronica inviata a *flits.cs.vu.nl.* La prima scelta è naturalmente *flits* stesso, ma se non fosse disponibile si possono provare *zephyr* e *top.*

Le tre righe successive contengono una voce tipica per una workstation, in questo caso *rowboat.cs.vu.nl.* Le informazioni fornite contengono l'indirizzo *IP,* i server predefiniti per la consegna della posta (primario e secondario). Appare quindi una voce per un sistema di cui non si specifica il record *MX*,[4] seguita da una voce che fa presumibilmente riferimento a una stampante connessa a Internet.

Name server

In teoria, un singolo name server potrebbe contenere l'intero database DNS e rispondere a tutte le interrogazioni. In pratica, questo server sarebbe troppo sovraccarico per essere utile. Inoltre, se dovesse incontrare un problema, l'intera Internet sarebbe bloccata.

Per evitare i problemi associati alla disponibilità di una singola fonte di informazioni, lo spazio dei nomi DNS viene diviso in **zone** non sovrapposte. Un modo per dividere lo spazio dei nomi della Figura 7.1 è mostrato nella Figura 7.5. Ogni zona cerchiata contiene alcune parti della struttura ad albero.

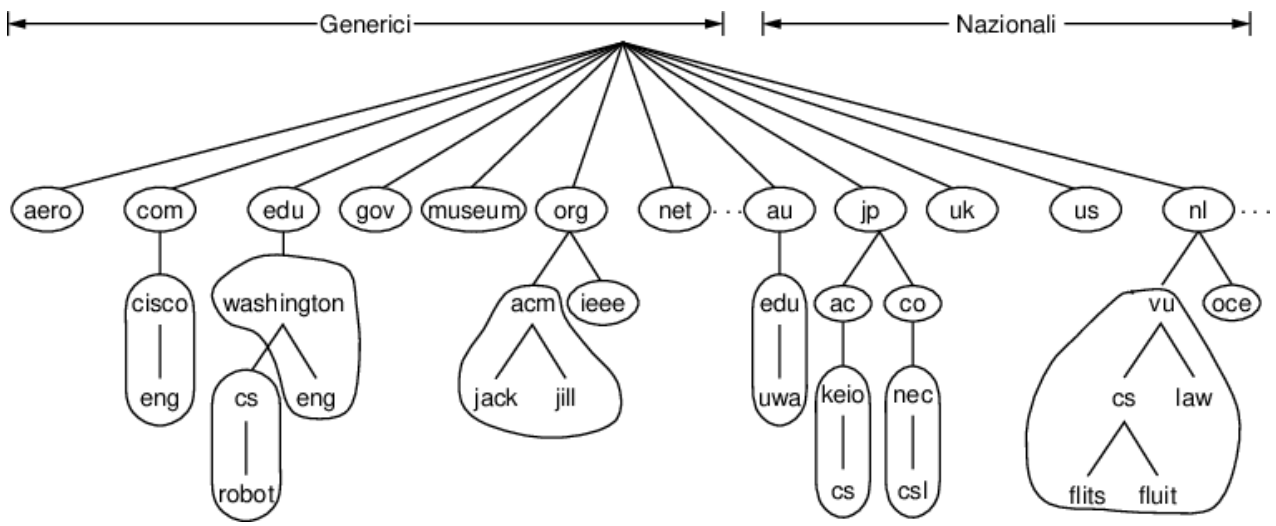


Figura 7.5 Una parte dello spazio dei nomi DNS diviso in zone (cerchiate).

La posizione dei confini di zona all'interno di una zona è una scelta lasciata all'amministratore. La decisione viene presa in gran parte sulla base del numero di name server desiderati e della loro posizione. Per esempio, nella Figura 7.5, l'università di Washington ha un server per *Washington.edu* che gestisce *eng.washington.edu* ma non *cs.washington.edu,* che è una zona separata con i suoi name server. Tale decisione può essere presa quando un dipartimento come quello di Inglese (eng) non desidera implementare il suo name server, mentre il dipartimento di informatica (cs) preferisce farlo.

Ogni zona è associata a uno o più name server. Questi sono host che tengono il database per la zona. Di norma, una zona avrà un name server primario, che ottiene le sue informazioni da un file su disco e uno o più name server secondari, che ottengono le loro informazioni dal name server primario. Per migliorare l'affidabilità, alcuni name server possono essere collocati all'esterno della zona.

Il processo di ricerca di un nome e di un indirizzo è chiamato **risoluzione del nome** (*name resolution*). Quando un resolver ha un'interrogazione su un nome di dominio, passa l'interrogazione a uno dei name server locali. Se il dominio è all'interno della giurisdizione del

name server (come *top.cs.vu.nl* che ricade in *cs.vu.nl*), restituisce i record autoritativi delle risorse. Un **record autoritativo** (*authoritative record*) è un record fornito direttamente dall'autorità che lo gestisce ed è pertanto sempre corretto. I record autoritativi si contrappongono ai **cached record** (*record archiviati nella cache*), che potrebbero non essere aggiornati.

Che cosa succede quando un dominio remoto, come *flits.cs.vu.nl*, desidera risolvere l'indirizzo IP di *robot.cs.washington.edu* presso la UW (University of Washington)? In questo caso, se non ci sono informazioni archiviate localmente sulla disponibilità del dominio, il name server inizia una ricerca in remoto che segue il processo mostrato nella Figura 7.6. Il passo 1 mostra come l'interrogazione venga inviata al name server locale. Questa interrogazione contiene il nome del dominio, il tipo (*A*) e la classe (*IN*). Il passo successivo è partire dalla cima della gerarchia dei nomi chiedendo a ognuno dei **root name server**. Questi name server hanno informazioni su ognuno dei domini di primo livello, come mostrato nel passo 2 nella Figura 7.6. Per contattare un root name server ogni name server deve avere informazioni su uno o più di essi. Questa informazione è normalmente presente nel file di configurazione del sistema che viene caricato nella cache quando il server viene avviato. È semplicemente un elenco di record *NS* con i corrispondenti record *A*.

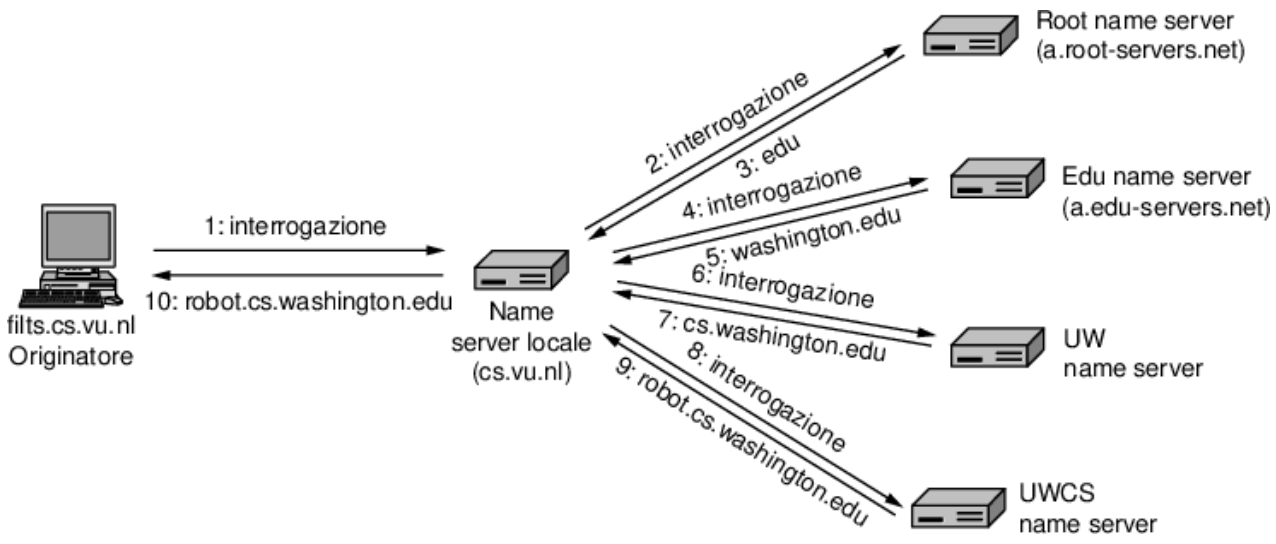


Figura 7.6 Esempio della risoluzione di un nome remoto in 10 passi.

Esistono 13 root DNS server, con molta immaginazione chiamati *a-root-server.net* fino a *m.root-servers.net*. Logicamente ogni root server potrebbe essere un singolo computer. Tuttavia, poiché l'intera Internet dipende da loro, essi sono computer molto potenti e con molte repliche. La maggior parte dei server è dislocata in più posizioni geografiche e raggiunta attraverso un routing anycast, nel quale un pacchetto viene consegnato all'istanza più vicina all'indirizzo di destinazione; abbiamo descritto anycast nel Capitolo 5. Le repliche aumentano affidabilità e performance del sistema.

Il root name server probabilmente non conosce l'indirizzo di una macchina di UW e probabilmente non ne conosce nemmeno il name server. Deve tuttavia conoscere il name server del dominio *edu*, nel quale si trova *cs.washington.edu*. Restituisce quindi il nome e l'indirizzo IP per la parte di interrogazione che riguarda il passo 3.

Il name server locale continua quindi la sua ricerca. Invia tutta l'interrogazione al name server del dominio *edu* (*a.edu-servers.net*). Questo restituisce il name server di UW, come mostrato nei passi 4 e 5. Ormai vicino, il name server locale invia l'interrogazione al name server di UW(passo 6). Se il nome del dominio oggetto della ricerca fosse il dipartimento di Inglese, la risposta sarebbe trovata, perché la zona include il dipartimento di Inglese. Ma il dipartimento di informatica ha scelto di avere il proprio name server e l'interrogazione restituisce nome e indirizzo IP del name server del dipartimento di informatica di UW (passo 7). Infine, il name server locale invia l'interrogazione al name server del dipartimento di informatica UW (passo 8). Il server è autoritativo per il dominio *cs.washington.edu*, quindi deve avere la risposta. Restituisce la risposta finale (passo 9) che il name server locale inoltra come risposta a *flits.cs.vu.nl* (passo 10) e il nome è stato risolto.

Potete esplorare questo processo usando un tool standard come il programma *dig* installato nella maggior parte dei sistemi UNIX. Per esempio, digitando

```
dig @ a.edu-servers.net robot.cs.washington.edu
```

si invia un'interrogazione riguardante *robot.cs.washington.edu* al name server *a.edu-servers.net* e ne viene stampato il risultato. Questo vi mostrerà le informazioni ottenute nel passo 4 dell'esempio e quindi apprenderete nome e indirizzo IP dei name server di UW.

Ci sono tre questioni tecniche da discutere in questo scenario. Primo, due differenti meccanismi di interrogazione sono all'opera nella Figura 7.6. Quando l'host *flits.cs.vu.nl* invia un'interrogazione al name server locale, il name server gestisce la risoluzione al posto di *flits* finché non viene data la risposta desiderata. Non restituisce risposte parziali perché, pur essendo utili, non sono ciò che l'interrogazione stava cercando. Questo meccanismo è chiamato **interrogazione ricorsiva** (*recursive query*).

D'altra parte, il root name server (e ogni name server seguente) non continua ricorsivamente l'interrogazione sul name server locale, ma restituisce solo una risposta parziale e passa all'interrogazione successiva. Il name server locale è responsabile per la continuazione del processo di risoluzione ponendo ulteriori domande. Questo meccanismo è chiamato **interrogazione iterativa** (*iterative query*).

La risoluzione di un nome può coinvolgere entrambi i meccanismi, come mostrato nell'esempio. Sembrerebbe sempre preferibile un'interrogazione ricorsiva, ma molti name server (specialmente i root) non sono in grado di gestirle, in quanto troppo occupati e le interrogazioni iterative scaricano quindi l'onere sulla sorgente. La logica del name server locale nel supportare

un'interrogazione ricorsiva è che sta fornendo un servizio agli host nel suo dominio che non devono essere configurati per svolgere essi stessi le funzioni di un name server, ma devono solo raggiungere quello locale.

Il secondo punto è il chaching. Tutte le risposte, comprese quelle parziali, vengono archiviate nella cache. In questo modo se un altro host *cs.vu.nl* fa un'interrogazione per *robot.cs.washington.edu* la risposta sarà già nota. Meglio, se un host fa un'interrogazione per un host diverso nello stesso dominio, come *galah.cs.washington.edu*, l'interrogazione può essere inviata direttamente al name server autoritativo. In modo simile, le interrogazioni per altri domini di *washington.edu* possono partire direttamente dal name server di *washington.edu*. L'uso delle risposte archiviate riduce notevolmente i passaggi di un'interrogazione e migliora le prestazioni. Lo scenario originario che abbiamo tracciato è in effetti il peggiore possibile, quando cioè non è archiviata alcuna informazione utile.

Tuttavia, le risposte archiviate non sono autoritative, perché i cambiamenti fatti a *washington.edu* non si propagheranno alle altre cache del mondo. Per questa ragione le informazioni non dovrebbero vivere troppo a lungo nelle cache; questo è il motivo per cui in ogni record delle risorse viene incluso il campo *Time_to_live*, che indica ai name server remoti per quanto tempo mantenere in cache i record. Se una certa macchina ha avuto lo stesso indirizzo IP per anni, sarebbe sicuro archiviare tale informazione per il tempo di un giorno. Per informazioni più volatili sarebbe meglio ripulire i record dopo pochi secondi o minuti.

Il terzo problema riguarda il protocollo di trasporto usato per le interrogazioni e le risposte: UDP. I messaggi DNS sono inviati in pacchetti UDP con un semplice formato per interrogazioni, risposte e dati dei name server utilizzabili per continuare la risoluzione del nome. Non descriveremo in dettaglio questo formato. Se non arriva alcuna risposta entro un limitato periodo di tempo, il client DNS ripeterà l'interrogazione, provando con un altro server del dominio dopo un piccolo numero di tentativi. Il processo è progettato per gestire il caso di server non funzionanti o di pacchetti di interrogazione o risposta che vadano persi. Un identificatore a 16 bit viene incluso in ogni interrogazione e copiato nella risposta, in modo che il name server possa associare le risposte all'interrogazione corrispondente, anche se vi sono più interrogazioni nello stesso istante.

Sebbene il suo scopo sia semplice, dovrebbe essere chiaro che DNS è un sistema distribuito grande e complesso, fatto di milioni di name server che lavorano insieme. Tutti insieme, formano il collegamento chiave tra i nomi di dominio leggibili dall'uomo e gli indirizzi IP delle macchine; esso precede anche replica e archiviazione per ottenere buone affidabilità e prestazioni; inoltre è progettato per essere molto robusto.

Non abbiamo trattato della sicurezza, ma come potete immaginare la possibilità di modificare l'associazione tra nomi e indirizzi può avere conseguenze devastanti se fatta con dolo. Per questa ragione è stata sviluppata un'estensione di sicurezza del DNS chiamata DNSSEC, descritta nel Capitolo 8.

C'è anche una richiesta da parte delle applicazioni di usare i nomi in modi più flessibili, per esempio, dando un nome ai contenuti e risolvendolo con l'indirizzo IP di un host vicino che ha quel contenuto. Questo è concettualmente ciò che succede quando si cerca e scarica un film: quel che conta è il film e non il computer che ne ha una copia; quindi ciò che si vuole è l'indirizzo IP di

qualunque computer nelle vicinanze che abbia una copia di quel film. Le reti per la distribuzione di contenuti sono indirizzate a ottenere questa associazione. Descriveremo come ciò avvenga tramite il DNS più avanti in questo capitolo, nel Paragrafo 7.5.

7.2 Posta elettronica

La posta elettronica, o più comunemente **e-mail**, è disponibile da più di tre decenni. Più veloce ed economica della posta tradizionale, l'e-mail è stata un'applicazione popolare fin dai primi esordi di Internet. Prima del 1990 era utilizzata principalmente nelle università. Negli anni '90 è divenuta nota al grande pubblico ed è cresciuta in modo esponenziale: basti pensare che il numero di e-mail attualmente spedite ogni giorno è di molto superiore al numero di lettere cartacee (chiamate anche **snail mail** o *posta lumaca* per il confronto sulla velocità). Benché altre forme di comunicazione in rete, quali la messaggistica istantanea e le chiamate su IP, si siano molto diffuse nell'ultimo decennio, l'e-mail rimane il cavallo di battaglia delle comunicazioni via Internet. È largamente utilizzata nelle aziende per le comunicazioni interne, per esempio per permettere a impiegati sparsi in tutto il mondo di cooperare su progetti complessi. Sfortunatamente, come per le lettere cartacee, la maggior parte delle e-mail, 9 su 10, è costituita da "messaggi spazzatura" o **spam** (McAfee, 2010).

La posta elettronica, come molte altre forme di comunicazione, presenta propri stili e convenzioni. In particolare è molto informale e ha una bassa soglia di utilizzo. Persone che non si sarebbero mai sognate di telefonare o scrivere una lettera a un personaggio famoso non esitano un secondo a spedirgli un messaggio di posta elettronica scritto in modo approssimativo. Eliminando molti dei pregiudizi associati allo stato sociale, all'età e al sesso, i dibattiti per posta elettronica tendono a concentrarsi sulle idee. Con la posta elettronica una brillante idea di uno studente può avere maggiore impatto di un'idea stupida di un vicepresidente.

I messaggi di posta elettronica contengono numerose "parole in codice", come l'italiano CMQ ("comunque") oppure gli inglesi BTW ("by the way", a proposito), ROTFL ("rolling on the floor laughing", mi sto rotolando sul pavimento dalle risate) e IMHO ("in my humble opinion", secondo la mia modesta opinione). Molti utilizzano nei loro messaggi anche piccoli simboli ASCII chiamati **smiley**, a partire dall'onnipresente **:-)**; questo simbolo e altre icone emozionali (**emoticon**) aiutano a veicolare il tono del messaggio e si sono estese anche ad altre forme di comunicazione, come la messaggistica istantanea.

I protocolli e-mail si sono evoluti nel corso degli anni. I primi sistemi di posta elettronica erano semplicemente protocolli di trasferimento per file testuali, con la convenzione che la prima riga di ogni messaggio (o file) conteneva l'indirizzo del destinatario. Con il trascorrere del tempo l'e-mail si è diversificata dal trasferimento file e sono state aggiunte molte proprietà, quali la capacità di inviare un messaggio a una lista di destinatari. Negli anni '90 divennero importanti le funzionalità multimediali per inviare messaggi contenenti immagini o altro materiale non testuale. Anche i programmi di lettura delle e-mail divennero più sofisticati, spostandosi da interfacce basate sul testo a interfacce grafiche e aggiungendo la possibilità di accedere alle proprie e-mail da computer portatili ovunque ci si trovi. Infine, con la prevalenza degli spam, i lettori di e-mail e i protocolli di trasferimento devono ora fare attenzione a trovare e rimuovere le e-mail indesiderate.

Nella nostra descrizione ci focalizzeremo sul modo in cui le e-mail vengono mandate e ricevute dagli utenti, piuttosto che sulle interfacce dei programmi di lettura. Tuttavia, dopo aver tracciato una panoramica dell'architettura, inizieremo con la parte vista dall'utente del sistema e-mail, perché

più familiare alla maggior parte dei lettori.

Architettura e servizi

In questo paragrafo forniremo una panoramica di ciò che possono fare i sistemi di posta elettronica e del modo in cui sono organizzati. L'architettura del sistema è mostrata nella Figura 7.7. Normalmente è composta da due sottosistemi: gli **user agent** (*agenti utente*), che consentono alle persone di leggere e inviare la posta elettronica, e i **message transfer agent** (*agenti di trasferimento dei messaggi*), che spostano i messaggi dalla sorgente alla destinazione. Questi ultimi sono informalmente chiamati **mail server**.

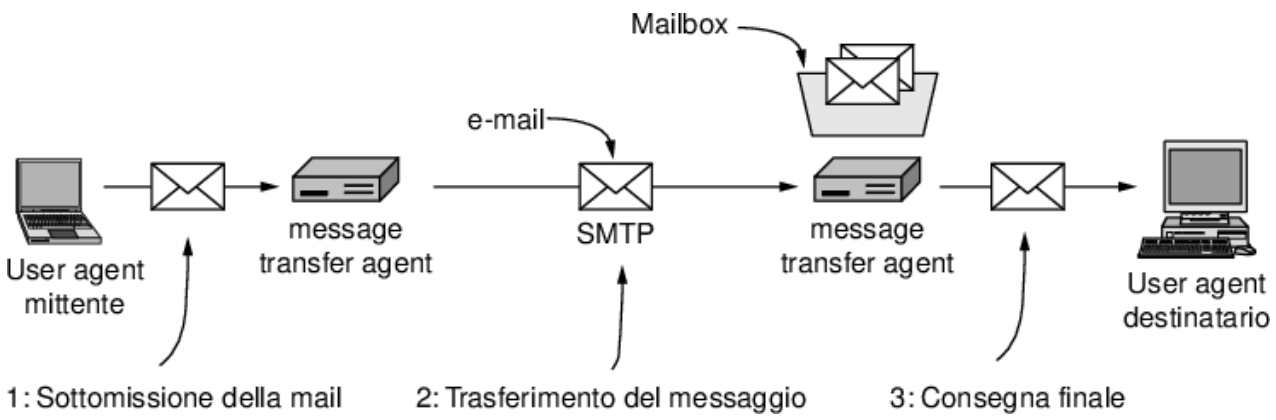


Figura 7.7 Architettura di un sistema di posta elettronica.

Uno user agent è un programma che fornisce un'interfaccia grafica o, talvolta, un'interfaccia testuale o basata su comandi, per interagire con il sistema di posta elettronica. Include gli strumenti per comporre, rispondere a, visualizzare e organizzare i messaggi riempiendoli, cercandoli e scartandoli.

Alcune elaborazioni dello user agent possono essere fatte automaticamente, anticipando i desideri degli utenti. Per esempio, le e-mail in ingresso possono essere filtrate per estrarre o attribuire una minore priorità ai messaggi che potrebbero essere spam. Alcuni user agent includono caratteristiche avanzate, come la compilazione di una e-mail automatica di risposta. Lo user agent è semplicemente un altro programma che può essere sporadicamente seguito.

I message transfer agent sono tipicamente processi di sistema. Vengono eseguiti come servizi sui mail server e sono pensati per essere sempre disponibili. Il loro compito è di spostare automaticamente le e-mail attraverso il sistema dalla sorgente al destinatario facendo uso di **SMTP** (*simple mail transfer protocol*).

SMTP venne originariamente specificato nell'RFC 821 e quindi rivisto per diventare quello attuale nell'RFC 5321. Invia le e-mail con un approccio orientato alla connessione e notifica lo stato della consegna e gli errori. Esistono numerose applicazioni in cui la conferma della consegna è importante e può anche avere significato legale ("Vostro onore, il mio sistema di e-mail non è molto affidabile, per cui immagino che la citazione elettronica si sia persa da qualche parte.").

I message transfer agent implementano anche le **mailing list**, in cui copie identiche di un messaggio sono consegnate a ogni iscritto alla lista. Altre funzionalità avanzate sono le copie per conoscenza (*carbon copy*, *cc*), le copie per conoscenza nascosta (*blind carbon copy*, *bcc*), i messaggi di posta elettronica ad alta priorità, la posta segreta cifrata, destinatari alternativi se il primo non è disponibile, la capacità da parte delle segretarie di leggere e rispondere alla posta del capo.

I concetti di mailbox e di formato standard dei messaggi e-mail sono il collegamento tra user agent e message transfer agent. Le **mailbox** (*caselle di posta*) memorizzano le e-mail ricevute dall'utente e sono mantenute dai mail server, mentre gli user agent semplicemente presentano agli utenti una visione dei contenuti delle loro mailbox. Per fare ciò, gli user agent inviano ai mail server i comandi per manipolare le mailbox, ispezionando i loro contenuti, cancellando messaggi e così via. Chi ritira la mail è il destinatario finale (passo 3) nella Figura 7.7. Con questa architettura per accedere a una mailbox un utente può usare differenti user agent su diversi computer.

Un'idea basilare nei sistemi di posta elettronica è la distinzione tra l'*involucro* (**envelope**) e il suo contenuto. L'involucro incapsula il messaggio e contiene tutte le informazioni necessarie per il suo trasporto, come l'indirizzo di destinazione, la priorità e il livello di protezione, che sono distinte dal messaggio stesso. I message transfer agent utilizzano l'involucro per il routing, come gli uffici postali tradizionali utilizzano le buste.

Il messaggio all'interno dell'involucro consiste di due parti: l'**intestazione** e il **corpo**. L'intestazione contiene le informazioni di controllo per gli user agent, mentre il corpo è dedicato interamente al destinatario umano. Involucri e messaggi sono illustrati nella Figura 7.8.

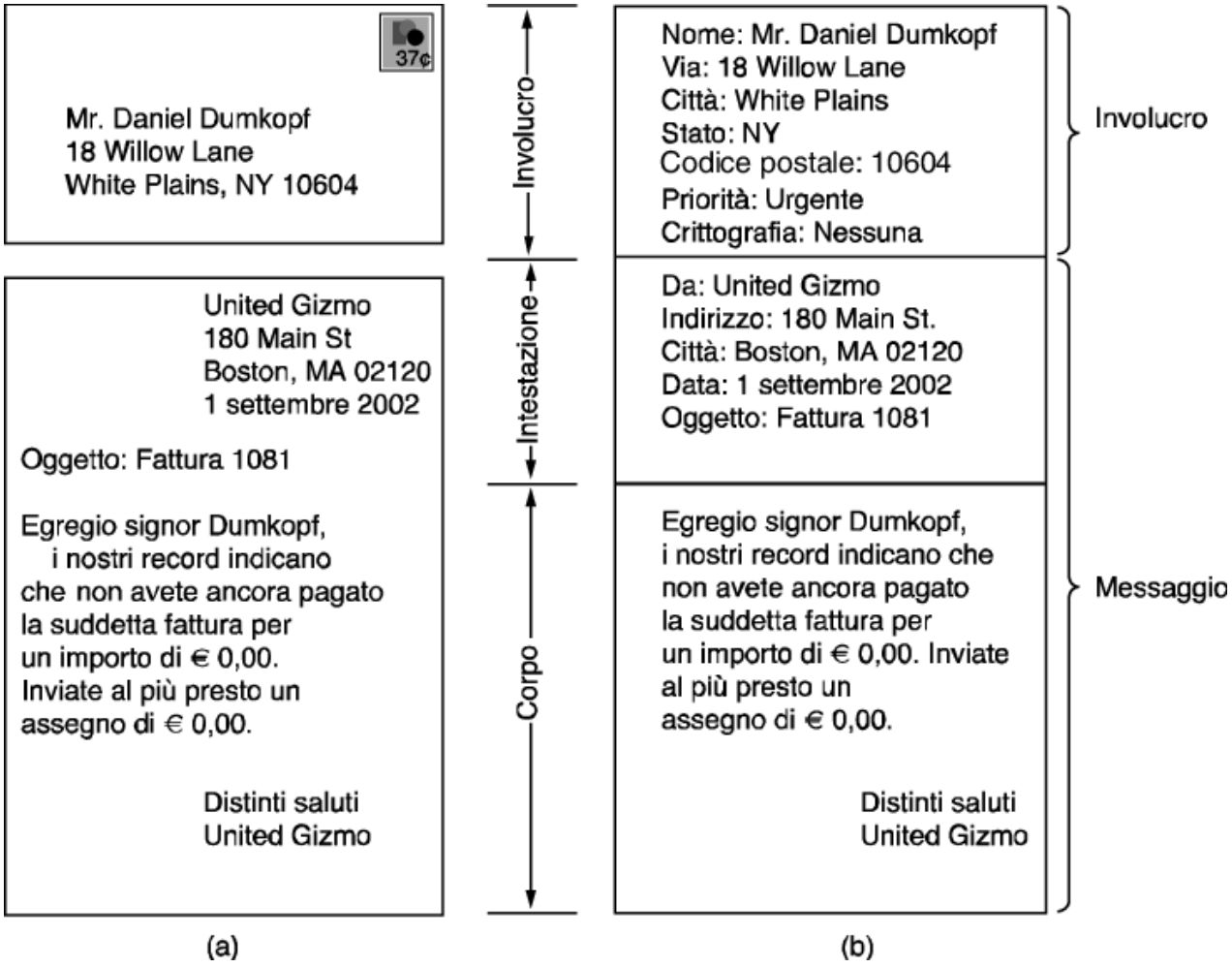


Figura 7.8 Involucri e messaggi. (a) Posta tradizionale. (b) Posta elettronica.

Esamineremo queste parti dell'architettura in dettaglio quando tratteremo i passaggi coinvolti nell'invio delle e-mail da un utente a un altro. Il nostro viaggio comincia con lo user agent.

User agent

Uno user agent è un programma (a volte chiamato **e-mail reader**, *lettore di e-mail* o, più colloquialmente, anche *client di posta*) che accetta una varietà di comandi per comporre, ricevere e rispondere ai messaggi, nonché per organizzare le caselle di posta. Ci sono molti user agent popolari quali Google Gmail, Microsoft Outlook, Mozilla Thunderbird e Apple mail. Alcuni user agent presentano un'interessante interfaccia grafica con menu e icone che richiede l'utilizzo di un mouse o di un'interfaccia touch per dispositivi mobili. Gli user agent più vecchi, come Elm, mh e Pine forniscono interfacce testuali e usano comandi via tastiera composti di un singolo carattere. A livello di funzionalità, tuttavia, questi due tipi di user agent si equivalgono, almeno per i messaggi di testo.

I tipici elementi dell'interfaccia di uno user agent sono mostrati nella Figura 7.9. Il vostro lettore di e-mail probabilmente è più accattivante, ma ha funzioni equivalenti. Quando uno user agent viene avviato, usualmente presenta un riepilogo dei messaggi che si trovano nella casella di posta

dell'utente. Spesso ha una linea per ogni messaggio elencato in qualche ordine. Evidenzia i campi chiave del messaggio che vengono estratti dall'involucro o dall'intestazione del messaggio.

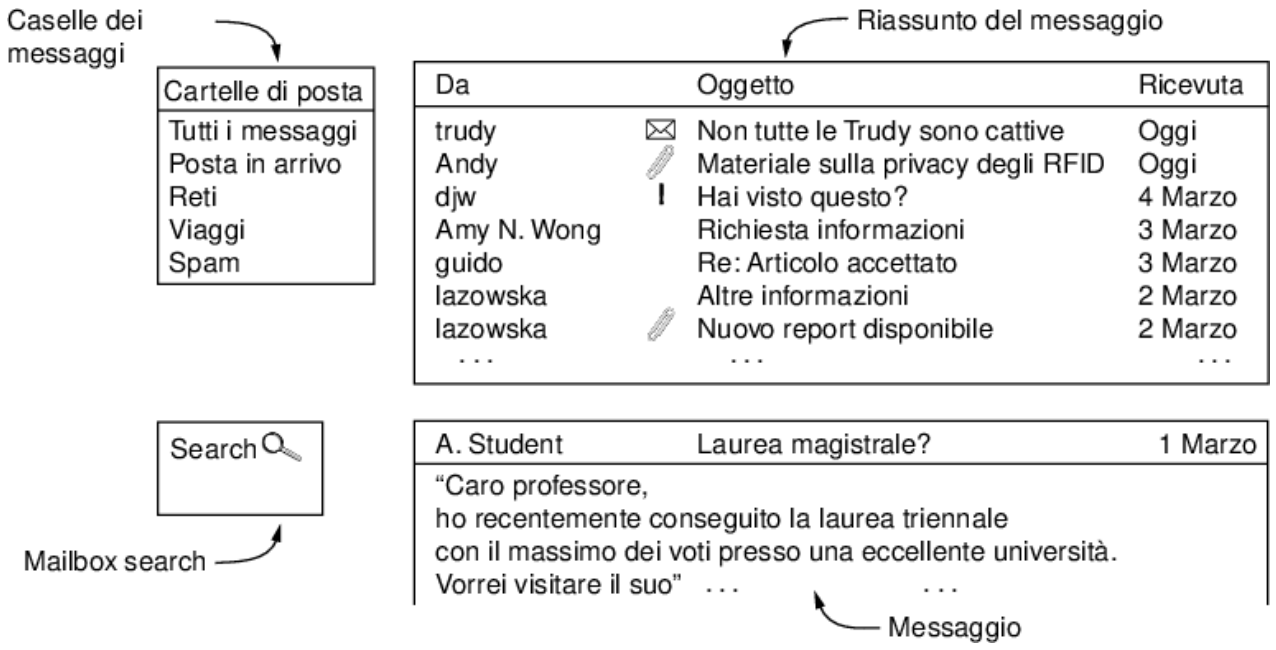


Figura 7.9 Tipici elementi dell'interfaccia di uno user agent.

Un esempio di sette linee di riepilogo è mostrato nella Figura 7.9. Le linee usano i campi *Da* (From), *Oggetto* (Subject), *Ricevuto* (Received), in questo ordine, per mostrare chi ha spedito il messaggio, di che cosa parla e quando è stato ricevuto. Piuttosto che visualizzare i contenuti letterali dei campi del messaggio su cui comunque i riepiloghi si basano, tutte le informazioni sono formattate in modo comprensibile all'utente, così chi dimentica di includere il campo *Oggetto* scopre che le sue e-mail tendono a non avere una grande priorità.

Sono disponibili anche molti altri campi o indicazioni. Le icone successive all'oggetto del messaggio nella Figura 7.9 possono indicare per esempio mail non ancora lette (busta), materiale allegato (graffetta) o una e-mail importante, almeno dal punto di vista di chi ha mandato il messaggio (con il punto esclamativo).

È possibile ordinare i messaggi in vari modi. Il più comune è quello in base al momento in cui sono stati ricevuti: per primi i più recenti con l'indicazione se il messaggio è nuovo o è già stato eletto dall'utente. Campi e ordine dei messaggi possono essere personalizzati dall'utente in base alle preferenze personali.

Gli user agent devono anche essere in grado di mostrare i messaggi in arrivo in modo che le persone possano leggere le proprie e-mail. Spesso viene fornita una piccola anteprima del messaggio, come nella Figura 7.9, per aiutare l'utente a decidere se leggerlo del tutto. Le anteprime

possono usare piccole icone o immagini per descrivere i contenuti del messaggio. Altre elaborazioni della presentazione includono la possibilità di cambiare il formato dei messaggi per essere ben visualizzati e quella di tradurre o convertire i contenuti in formati più convenienti.

Dopo aver letto un messaggio, l'utente può decidere che cosa farne. Alcune opzioni sono: distruggerlo, inviare una risposta, inoltrare il messaggio a un altro utente e conservare il messaggio. La maggior parte degli user agent può gestire una casella di posta per le e-mail in arrivo con più cartelle in cui salvarle. Le cartelle permettono agli utenti di salvare i messaggi a seconda della sorgente, dell'argomento o di altre categorie.

L'archiviazione può essere fatta automaticamente dallo user agent prima che l'utente legga i messaggi. Un esempio comune si ha quando i campi e i contenuti dei messaggi sono ispezionati e usati, insieme a quanto detto dall'utente riguardo ai messaggi precedenti, per determinare se un messaggio può essere spam. Molti ISP e aziende fanno uso di software che etichetta le e-mail come importanti o spam in modo che lo user agent possa archiviarle nella casella di posta corrispondente. In questo modo gli ISP e le aziende hanno il vantaggio di poter guardare le e-mail di molti utenti e avere una lista degli spam noti; se centinaia di utenti hanno appena ricevuto un messaggio uguale, probabilmente si tratta di spam. Etichettando le e-mail entranti come "probabilmente corrette" e "probabilmente spam", lo user agent farà risparmiare agli utenti molto lavoro per separare il materiale buono dalla spazzatura.

Quali sono gli spam più popolari? Generalmente i messaggi sono generati da un insieme di computer compromessi chiamati **botnet** e i loro contenuti dipendono dal luogo di residenza. I diplomi falsi sono tipici dell'Asia, mentre le droghe a basso prezzo e altri prodotti dubbi sono tipicamente offerti negli Stati Uniti. Vengono anche proposti conti bancari nigeriani e pillole per ingrossare varie parti del corpo sono popolari ovunque.

Gli utenti possono costruirsi altre regole di archiviazione. Ogni regola specifica una condizione e un'azione. Per esempio, una regola potrebbe dire che qualunque messaggio ricevuto dal proprio capo va in una cartella e deve essere letto immediatamente, mentre ogni messaggio di una particolare mailing-list deve andare in un'altra cartella per essere letto più tardi. Alcune cartelle sono mostrate nella Figura 7.9. Le cartelle più importanti sono la *Inbox* (posta in arrivo), per le e-mail in ingresso non archiviate da altre parti e la *Junk e-mail* (spam) per i messaggi che si suppone siano spam.

Ora gli user agent forniscono anche ricche possibilità di ricerca nella casella di posta. Questa proprietà è mostrata nella Figura 7.9. La capacità di ricerca permette agli utenti di individuare i messaggi velocemente come quello che riguarda "dove acquistare Vegemite",[5] inviato da qualcuno nell'ultimo mese.
Le e-mail hanno percorso un lungo viaggio dai tempi in cui erano semplicemente un trasferimento di file. Ora i provider supportano normalmente caselle di posta con fino a 1 GB di e-mail archiviate e con dettagli delle interazioni degli utenti su un lungo periodo di tempo. Quello che permette di gestire tali volumi di e-mail è la gestione sofisticata degli user agent con forme di elaborazione automatica e di ricerca delle e-mail. Questi tool sono inestimabili per coloro che inviano e ricevono migliaia di messaggi ogni anno.

Una caratteristica utile è la capacità di rispondere automaticamente ai messaggi. Una possibile risposta è inoltrare le e-mail in ingresso su indirizzi differenti, ad esempio verso computer di aziende in grado di mandare un SMS con l'oggetto della mail. Questi auto-risponditori devono essere eseguiti sul mail server perché lo user agent non è attivo sempre e riceve i messaggi solo occasionalmente. Per questi motivi lo user agent non può fornire una risposta automatica vera. Tuttavia un'interfaccia per la risposta automatica è usualmente offerta dallo user agent.

Un esempio differente di risposta automatica è un **vacation agent** (*agente per assenza*). Questo è un programma che esamina i messaggi in ingresso e invia alla sorgente una risposta neutra come "Sono in vacanza, tornerò il 24 agosto e vi risponderò." Queste repliche possono anche specificare come gestire le urgenze nel frattempo o quali persone contattare per problemi specifici. La maggior parte dei vacation agent tiene traccia di coloro a cui hanno mandato le repliche in modo da non inviarne alla stessa persona una seconda. Sorgono però problemi potenziali rispetto a questi agenti; ad esempio non è suggeribile mandare una risposta preconfezionata a un messaggio che arriva da una mailing list.

Torniamo ora allo scenario in cui un utente invia un messaggio a un altro utente. Una delle proprietà di base degli user agent che non abbiamo ancora discusso è come comporre la e-mail. Questa include la creazione di messaggi o risposte a messaggi e l'invio del tutto nel sistema di e-mail per la consegna. Sebbene qualunque editor di testo possa essere usato per creare il corpo del messaggio, di solito gli editor sono integrati nello user agent, in modo che questo possa fornire assistenza nel compilare l'indirizzo e i numerosi campi dell'intestazione attaccata a ogni messaggio. Per esempio, quando si risponde a un messaggio, il sistema può estrarre l'indirizzo della sorgente dalla e-mail in arrivo e inserirlo automaticamente dove appropriato nella risposta. Altre funzionalità comuni sono apporre in fondo al messaggio un **signature block** (*blocco contenente la firma dell'utente*), correggere gli errori di battitura e calcolare una firma digitale che mostri che il messaggio è valido.

I messaggi inviati hanno un formato standard che viene creato a partire dalle informazioni fornite dallo user agent. La parte più importante del messaggio per il trasferimento è l'involucro e la parte più importante dell'involucro è l'indirizzo della destinazione. Tale indirizzo deve essere in un formato gestibile dal message transfer agent.

La forma che ci si aspetta da un indirizzo è *user@indirizzo-dns*. Poiché abbiamo già affrontato l'argomento del DNS in questo capitolo non ripeteremo la discussione qui. Vale comunque la pena notare che ci sono altre forme di indirizzo. In particolare, gli indirizzi **X.400** hanno un aspetto radicalmente differente da quelli di DNS. X.400 era uno standard ISO per sistemi di gestione dei messaggi che un tempo era competitivo con SMTP. SMTP vinse, ma X.400 ha continuato a essere usato per lo più fuori dagli Stati Uniti. Gli indirizzi X.400 sono composti da coppie *attributo=valore* separate da barre oblique (*slash*), come nell'esempio seguente:

```
/C=US/ST=MASSACHUSETTS/L=CAMBRIDGE/PA=360 MEMORIAL DR./CN=KEN SMITH/
```

Questo indirizzo specifica una nazione, uno stato, una località, un indirizzo personale e un nome comune (Ken Smith). Sono consentiti molti altri attributi, pertanto potete inviare la posta elettronica a qualcuno di cui non conoscete l'esatto indirizzo di posta elettronica, purché conosciate altri attributi (per esempio la società e il titolo professionale).

Anche se i nomi X.400 sono considerevolmente meno pratici dei nomi DNS, la maggior parte dei sistemi di posta elettronica dispone di alias (o *nickname*) che permettono all'utente di inserire o selezionare il nome di una persona per ottenere il corretto indirizzo di posta elettronica. Di conseguenza anche con gli indirizzi X.400 solitamente non è necessario digitare queste stringhe.

La maggior parte dei sistemi di posta elettronica supporta le mailing list, pertanto un utente può inviare lo stesso messaggio a un elenco di persone con un singolo comando. Se la mailing list viene gestita localmente, lo user agent può inviare un messaggio separato a ogni destinatario previsto; se invece la lista è gestita in remoto, i messaggi saranno espansi in tale posizione. Per esempio, se un gruppo di amanti del bird watching possiede una mailing list chiamata *birders* installata su *meadowlark.arizona.edu,* qualsiasi messaggio inviato a *birders@meadowlark.arizona.edu* sarà instradato all'università dell'Arizona e qui espanso in singoli messaggi per tutti i membri della mailing list ovunque si trovino nel mondo. Gli utenti di questa mailing list non possono capire che ad avere originato il messaggio sia stata una mailing list, potrebbe anche essere stato inviato dalla casella di posta personale del professor Gabriel O. Birders.

Formato dei messaggi

Passiamo ora dall'interfaccia utente al formato dei messaggi di posta elettronica. I messaggi inviati dallo user agent devono essere in un formato standard gestibile dal message transfer agent. Per prima cosa osserveremo i messaggi di posta ASCII che utilizzano RFC 5322, l'ultima revisione del formato originale dei messaggi Internet descritto in RFC 822. Infine studieremo le estensioni multimediali.

RFC 5322 – Il formato dei messaggi di Internet

I messaggi sono costituiti da un semplice involucro (descritto come parte di SMTP nell'RFC 5321), alcuni campi di intestazione, una riga vuota e il corpo del messaggio. Ogni campo di intestazione consiste logicamente di una singola riga di testo ASCII contenente il nome del campo seguito da un carattere di due punti (:) e, per la maggior parte dei campi, un valore. L'RFC 822 è stato progettato decenni fa e non distingue chiaramente i campi dell'involucro da quelli dell'intestazione. Sebbene sia stato rivisto nell'RFC 5322, non è stato possibile riscriverlo completamente a causa del suo utilizzo diffuso. Di norma, lo user agent costruisce un messaggio e lo passa al message transfer agent che utilizza i campi di intestazione per costruire l'involucro effettivo, una sorta di ibrido "vecchio stile" tra messaggio e involucro.

I campi di intestazione principali correlati al trasporto del messaggio sono elencati nella Figura 7.10. Il campo *To:* indica l'indirizzo DNS di un destinatario principale; è consentito indicare più destinatari. Il campo *Cc:* indica gli indirizzi dei destinatari secondari. In termini di consegna non vi è una distinzione tra destinatari principali e secondari: si tratta solo di una attitudine psicologica differente che può rivestire un significato per le persone coinvolte, ma non per il sistema di posta elettronica. Il termine *Cc:* significa copia per conoscenza; (*carbon copy* o *copia carbone*), una

definizione un po' vecchio stile, ma sempre valida. Il campo *Bcc:* (*bling carbon copy*, o *copia per conoscenza nascosta*) è simile al campo *cc:*, tranne per il fatto che la riga viene eliminata da tutte le copie inviate ai destinatari principali e secondari. Questa funzionalità consente alle persone di inviare copie a terze parti senza che i destinatari principali e secondari lo sappiano.

Intestazione	Significato
To:	Gli indirizzi di posta elettronica dei destinatari primari
Cc:	Gli indirizzi di posta elettronica dei destinatari secondari
Bcc:	Gli indirizzi di posta elettronica per le copie per conoscenza nascoste
From:	La persona che ha creato il messaggio
Sender:	L'indirizzo di posta elettronica del mittente vero e proprio
Received:	Riga aggiunta da ogni transfer agent lungo il percorso
Return-path:	Può essere utilizzato per identificare un percorso di ritorno al mittente

Figura 7.10 Campi di intestazione dell'RFC 5322 relativi al trasporto del messaggio.

I due campi successivi, *From:* e *Sender:*, indicano rispettivamente chi ha scritto e inviato il messaggio. Non devono necessariamente avere lo stesso contenuto. Per esempio, un dirigente commerciale può scrivere un messaggio, ma potrebbe essere la sua segretaria a trasmetterlo. In questo caso, il dirigente sarebbe elencato nel campo *From:* e la sua segretaria nel campo *Sender:*. Il campo *From:* è obbligatorio, mentre il campo *Sender:* può essere omesso se equivale al campo *From:*. Questi campi sono necessari nel caso in cui il messaggio non sia recapitabile e debba essere restituito al mittente.

Una riga contenente *Received:* viene aggiunta da ogni message transfer agent lungo il percorso. La riga contiene identità dell'agente, data e ora di ricezione del messaggio e altre informazioni che possono essere utilizzate per trovare i bug nel sistema di routing.

Il campo *Return-Path:* viene aggiunto dal message transfer agent finale e ha lo scopo di spiegare come ricontattare il mittente. In teoria, questa informazione può essere raccolta da tutte le intestazioni *Received:* (tranne per il nome della casella di posta del mittente), ma viene raramente compilata in questo modo e di norma contiene solo l'indirizzo del mittente.

Oltre ai campi della Figura 7.10, i messaggi descritti dall'RFC 5322 possono contenere diversi campi di intestazione utilizzati dagli user agent o dai destinatari umani. I più comuni sono elencati nella Figura 7.11. La maggior parte è autoesplicativa, pertanto non saranno presentati tutti in dettaglio.

Intestazione	Significato
Date:	**La data e l'ora di invio del messaggio**
Reply-to:	L'indirizzo di posta elettronica a cui inviare le risposte
Message-id:	Identificatore del messaggio a cui si riferisce questa risposta
In-reply-to:	Identificatore del messaggio a cui si riferisce questa risposta
References:	Altri identificatori rilevanti
Keywords:	Parole chiave scelte dall'utente
Subject:	Può essere utilizzato per identificare un percorso di ritorno al mittente

Figura 7.11 Alcuni campi utilizzati nell'intestazione del messaggi RFC 5322.

Il campo *Message-Id*: è un numero generato automaticamente usato per collegare tra loro messaggi (ad esempio quando utilizzato nel campo *In-Reply-To:*) e per prevenire consegne duplicate.

Il campo *Reply-To:* viene a volte utilizzato quando né la persona che compone il messaggio né la persona che lo invia desiderano vedere la risposta. Per esempio, il direttore del marketing scrive un messaggio di posta elettronica presentando ai clienti un nuovo prodotto. Il messaggio viene inviato da una segretaria, ma il campo *Reply-To:* indica il capo del reparto vendite, che può rispondere alle domande e accettare ordini. Questo campo è utile anche quando il mittente dispone di due account di posta elettronica e desidera che tutte le risposte giungano a uno solo di essi.

L'RFC 5322 afferma in modo esplicito che gli utenti possono inventare nuove intestazioni per utilizzo privato, purché tali intestazioni inizino con la stringa *X-*. È garantito che nessuna intestazione futura userà nomi che iniziano con *X-*, per evitare conflitti con intestazioni private e ufficiali. A volte gli universitari sapientoni creano campi come *X-Frutto-del-giorno:* o *X-Malattia-della-settimana:,* che sono legali, ma non illuminanti. L'intestazione è seguita dal corpo del messaggio. Gli utenti possono inserire tutto quello che desiderano. Alcune persone terminano i loro messaggi con firme elaborate, comprese citazioni di personaggi più o meno famosi, citazioni politiche e postille di ogni tipo (per esempio, "La società XYZ non è responsabile delle mie opinioni; in effetti, non è nemmeno in grado di comprenderle.").

MIME – multipurpose Internet mail extension

Agli esordi di ARPANET la posta elettronica consisteva esclusivamente di messaggi di testo scritti in inglese ed espressi in ASCII. Per questo ambiente l'RFC 822 era perfetto: specificava le intestazioni e lasciava il contenuto agli utenti. Oggi sulla Internet mondiale la richiesta di poter inviare contenuti più complessi attraverso il sistema di mail ha reso questo approccio non più adeguato. I problemi che possono nascere comprendono invio e ricezione di messaggi scritti in

lingue con accenti (come il francese e il tedesco), alfabeti non latini (l'ebraico e il russo), alfabeti con ideogrammi (come il cinese e il giapponese) e messaggi che non contengono testo (come audio, immagini, documenti binari e programmi).

La soluzione fu lo sviluppo di **MIME** (*multipurpose Internet mail extension*). Oggi è ampiamente utilizzato per le e-mail spedite tramite Internet e per descrivere contenuti di altre applicazioni come la navigazione Web. È descritto negli RFC da 2045 a 2047, 4288, 4289 e 2049.

L'idea alla base di MIME è quella di continuare a utilizzare il formato RFC 822 (il precursore di RFC 5322 al tempo in cui MIME fu proposto), aggiungendo una struttura al corpo del messaggio e definendo le regole di codifica per i messaggi non ASCII. Visto che non si discosta dall'RFC 822, i messaggi MIME possono essere inviati utilizzando programmi e protocolli di posta esistenti (basati allora su RFC 821 e ora su RFC 5321). Tutto ciò che deve essere cambiato sono i programmi di invio e ricezione, un compito lasciato agli utenti.

Come mostrato nella Figura 7.12 MIME definisce cinque nuove intestazioni dei messaggi. La prima indica semplicemente allo user agent di ricevere il messaggio in questione come un messaggio MIME e specifica la versione utilizzata. Qualsiasi messaggio che non contiene l'intestazione *MIME-Version:* è considerato un messaggio di testo normale in lingua inglese (o perlomeno che usa solo caratteri ASCII) e viene elaborato come tale.

Intestazione	Significato
MIME-version:	**Identifica la versione di MIME**
Content-description:	**Una stringa leggibile che comunica che cosa contiene il messaggio**
Content-id	**Un identificatore univoco**
Content-transfer-encoding:	**Indica come il corpo del messaggio è stato preparato per la trasmissione**
Content-type:	**Il tipo e il formato del contenuto**

Figura 7.12 Intestazioni aggiunte da MIME.

L'intestazione *Content-Description:* è una stringa ASCII che comunica il contenuto del messaggio. Questa intestazione è necessaria per consentire al destinatario di decidere se vale la pena di decodificare e leggere il messaggio. Se la stringa afferma "Foto del criceto di Barbara" e la persona che riceve il messaggio non è un grande estimatore di criceti, il messaggio sarà probabilmente scartato anziché decodificato in una fotografia a colori ad alta risoluzione.

L'intestazione *Content-Id:* identifica il contenuto. Usa lo stesso formato dell'intestazione standard *Message-Id:*.

L'intestazione *Content-Transfer-Encoding:* spiega da che cosa è costituito l'involucro per la trasmissione in rete. Un problema chiave quando venne sviluppato MIME fu che il protocollo di trasferimento delle e-mail (SMTP) era sviluppato per messaggi ASCII con linee che non eccedevano i 1000 caratteri. I caratteri ASCII usano 7 degli 8 bit di un byte. Dati binari, programmi eseguibili e immagini usano tutti gli 8 bit di un byte come fanno i caratteri estesi. Non c'è alcuna garanzia che questi dati vengano trasferiti in modo affidabile; era quindi necessario un metodo per trasportare i dati binari facendoli sembrare normali messaggi di e-mail ASCII. Le estensioni di SMTP, a partire dall'introduzione di MIME, permettono di trasferire direttamente dati binari, anche se oggi i dati binari non sempre possono essere inviati correttamente tramite e-mail se non codificati.

Sono previsti cinque schemi di codifica, più una via di fuga per quelli nuovi, nel caso sia necessaria. Il più semplice è il testo ASCII. I caratteri ASCII usano 7 bit e possono essere trasportati direttamente dal protocollo di posta elettronica se le righe non superano la lunghezza di 1000 caratteri.

Lo schema successivo in ordine di semplicità è identico, salvo che utilizza caratteri di 8 bit, vale a dire tutti i valori da 0 a 255. I messaggi che utilizzano la codifica a 8 bit devono comunque aderire alla lunghezza massima standard delle righe.

Ancora più critici sono i messaggi che utilizzano la vera codifica binaria. Si tratta di file binari arbitrari che non solo utilizzano tutti gli 8 bit, ma non rispettano nemmeno il limite di 1000 caratteri per riga. I programmi eseguibili appartengono a questa categoria. Oggigiorno i mail server possono negoziare l'invio di dati in binario (8 bit) codificati, ricorrendo all'ASCII se entrambe le parti non supportano l'estensione.

La codifica ASCII di dati binari prevede l'utilizzo della **codifica base64** (*base64 encoding*). In questo schema gruppi di 24 bit sono suddivisi in quattro unità di 6 bit, ognuna inviata come carattere ASCII valido. La codifica è "A" per 0, "B" per 1 e così via, seguita dalle 26 lettere minuscole, dalle dieci cifre e infine da "+" e "/" rispettivamente per 62 e 63. Le sequenze "==" e "=" indicano che l'ultimo gruppo contiene rispettivamente solo 8 o 16 bit. I ritorni carrello e gli avanzamenti riga sono ignorati, pertanto possono essere inseriti a piacere per rispettare la lunghezza delle righe. I testi binari arbitrari possono essere inviati in modo sicuro usando questo schema, anche se inefficiente. Questa codifica era molto popolare prima che mail server capaci di trattare i binari fossero diffusi, comunque è tuttora in circolazione.

Per i messaggi quasi interamente ASCII, ma contenenti alcuni caratteri non ASCII, la codifica base64 è inefficiente. Al suo posto viene utilizzata una codifica nota come **codifica quoted-printable** (*coded-printable encoding*). Si tratta di ASCII a 7 bit, con tutti i caratteri superiori a 127 codificati come segni uguale seguiti dal valore del carattere espresso da due cifre esadecimali. Anche caratteri di controllo, punteggiatura, simboli matematici e spazi sono codificati.

Quando vi sono valide ragioni per non utilizzare questi schemi è possibile specificare una codifica definita dall'utente nell'intestazione *Content-Transfer-Encoding:.*

L'ultima intestazione mostrata nella Figura 7.12 è quella più interessante, che specifica la natura del corpo del messaggio e ha avuto un impatto al di là delle e-mail. Per esempio, i contenuti scaricati dal Web sono etichettati con un tipo MIME, in modo che il browser sappia come presentarli. Allo

stesso modo sono presentati i contenuti inviati su flussi multimediali e real-time, come la voce su IP.

L'RFC 1521 definì originariamente sette tipi MIME, per ognuno dei quali sono disponibili uno o più sottotipi. Il tipo e il sottotipo sono separati da una barra obliqua (*slash*) come in "*Content-type: video/mpeg*". Da allora, centinaia di sottotipi sono stati aggiunti insieme ad altri tipi. Ogni volta che viene sviluppato un nuovo tipo di contenuto si deve aggiungere una voce. La lista di tipi e sottotipi assegnati è mantenuta on-line da IANA all'indirizzo *http://www.iana.org/assignments/media-types*.

L'elenco dei tipi e dei sottotipi più comuni è mostrato nella Figura 7.13. Analizziamolo brevemente a partire da *text*. La combinazione *text/plain* è per i messaggi ordinari che possono essere visualizzati come vengono ricevuti, senza codifica o ulteriori elaborazioni. Questa opzione consente ai messaggi ordinari di essere trasportati in MIME con poche intestazioni aggiuntive. Il sottotipo *text/html* venne aggiunto (con l'RFC 2854) quando il Web divenne popolare per consentire alle pagine Web di essere inviate come posta elettronica aderente all'RFC 822. Un sottotipo per **XML** (*extensible markup language*), *text/xml*, è definito nell'RFC 3023. HTML e XML verranno studiati nel Paragrafo 7.3.

Tipo	Esempio di sottotipi	Descrizione
text	plain, html, xml, css	Testo in vari formati
image	gif, jpeg, tiff	Immagini
audio	basic, mpeg, mp4	Suoni
video	mpeg, mp4, quicktime	Filmati
model	vrml	Modelli 3D
application	octet-stream, pdf, javascript, zip	Dati prodotti da applicazioni
message	http, rfc822, partial	Messaggio incapsulato
multipart	mixed, alternative, parallel, digest	Combinazione di più tipi

Figura 7.13 Tipi di contenuti MIME ed esempi di sottotipi.

Il tipo MIME successivo è *image*, utilizzato per trasmettere immagini statiche. Oggi sono ampiamente utilizzati molti formati per archiviare e trasmettere le immagini, con o senza compressione. Alcuni di questi, come GIF, JPEG e TIFF, sono incorporati in quasi tutti i browser, ma esistono molti altri formati con i corrispettivi sottotipi.

I tipi *audio* e *video* sono dedicati rispettivamente al suono e alle immagini in movimento. Occorre notare che *video* include solo le informazioni visive, non la colonna sonora. Se diventa necessario trasmettere un filmato con audio, le porzioni audio e video devono essere trasmesse separatamente, a seconda del sistema di codifica utilizzato. Il primo formato video definito è stato quello inventato da *moving picture experts group* (MPEG), ma in seguito ne sono stati aggiunti altri. Oltre ad *audio/basic,* nell'RFC 3003 è stato aggiunto un nuovo tipo audio, *audio/mpeg,* per consentire alle persone di inviare per posta elettronica i file audio MP3. I tipi *video/mp4* e *audio/mp4* sono contenuti nel nuovo formato MPEG 4.

Il tipo *model* venne aggiunto dopo gli altri tipi di contenuto. È stato pensato per descrivere dati di modelli 3D. Tuttavia non è ancora molto utilizzato.

Il tipo *application* è un ripiego per i formati non compresi nelle altre tipologie, che richiedono un'applicazione che interpreti i dati. Abbiamo elencato i sottotipi PDF, Javascript e zip, come esempi di, rispettivamente, documenti PDF, programmi Javascript e archivi. Gli user agent che ricevono questi contenuti utilizzano una libreria di terze parti o un programma esterno per visualizzarne il contenuto; la visualizzazione può apparire o non apparire integrata nello user agent.

Usando i tipi MIME gli user agent acquistano un'ulteriore abilità di gestire nuovi tipi di contenuti applicativi appena questi vengono sviluppati; si tratta di un beneficio significativo. D'altra parte, molte nuove forme di contenuto sono eseguite o interpretate da applicazioni che presentano alcuni pericoli. Ovviamente, mandare in esecuzione un programma eseguibile arbitrario che è arrivato via e-mail da un "amico" è un rischio per la sicurezza. Il programma può fare qualsiasi sorta di danno a quella parte del computer a cui ha accesso, soprattutto se può leggere e scrivere file e usare la rete. È meno ovvio pensarlo, ma i formati dei documenti possono porre gli stessi rischi; questo succede perché i formati come il PDF sono linguaggi di programmazione travestiti. Anche se sono interpretati e ristretti rispetto a quanto possono fare, errori di implementazione nell'interprete, tuttavia, permettono spesso di sfuggire a queste restrizioni.

Ci sono inoltre molti altri sottotipi di applicazioni semplicemente perché ci sono molte altre applicazioni. Un sottotipo che si usa quando non ne è noto alcun altro più adeguato è *octet-stream*, che è semplicemente una sequenza di byte non interpretati. Dopo la ricezione di tale flusso, uno user agent deve probabilmente visualizzarlo suggerendo all'utente che deve essere copiato in un file. L'elaborazione successiva è lasciata all'utente che, probabilmente, conosce il contenuto.

Gli ultimi due tipi sono utili per comporre e manipolare i messaggi. Il tipo *message* consente a un messaggio di essere incapsulato in un altro. Questo schema è utile per l'inoltro della posta elettronica. Quando un messaggio RFC 822 completo viene incapsulato in un messaggio più esterno, deve essere utilizzato il sottotipo *rfc822*.

Il sottotipo *partial* permette di suddividere in più parti un messaggio incapsulato e di inviarle separatamente (per esempio se il messaggio incapsulato è troppo lungo). I parametri permettono di riassemblare tutte le parti nell'ordine corretto, una volta giunte a destinazione.

L'ultimo tipo è *multipart,* che consente di inserire più parti in un messaggio delimitando chiaramente l'inizio e la fine di ciascuna. Il sottotipo *mixed* consente a ogni parte di essere diversa, senza strutture aggiuntive imposte. Molti programmi di posta elettronica consentono all'utente di

fornire uno o più allegati a un messaggio di testo. Questi allegati sono inviati utilizzando il tipo *multipart.*

In contrasto con *mixed*, il sottotipo *alternative* consente allo stesso messaggio di essere incluso più volte, ma espresso con due o più supporti diversi. Per esempio, un messaggio potrebbe essere inviato come testo ASCII, HTML e PDF. Uno user agent correttamente progettato che riceve tale messaggio lo visualizza secondo le preferenze dell'utente, possibilmente in PDF, in subordine proverebbe HTML e se nessuna di queste soluzioni fosse disponibile verrebbe visualizzato il testo ASCII. Le parti dovrebbero essere ordinate dalla più semplice alla più complessa per aiutare i destinatari con user agent precedenti a MIME a dare un senso a un messaggio (anche uno user agent precedente a MIME può leggere il testo ASCII).

Il sottotipo *alternative* può essere utilizzato anche per più lingue. In questo contesto la stele di Rosetta può essere concepita come un vecchio messaggio *multipart/alternative.*

Il sottotipo *parallel* è utilizzato quando tutte le parti devono essere "viste" contemporaneamente; un esempio sono i filmati con un canale audio e un canale video. I filmati sono più efficaci se questi due flussi sono riprodotti in parallelo, anziché consecutivamente. Per finire, il sottotipo *digest* viene utilizzato quando molti messaggi sono uniti in un messaggio composito. Per esempio, alcuni gruppi di discussione su Internet raccolgono messaggi dagli iscritti e poi li inviano al gruppo come singolo messaggio *multipart/digest.*

Nella Figura 7.14 è mostrato un messaggio multimediale come esempio di impiego dei tipi MIME per i messaggi e-mail. Qui vengono trasmessi gli auguri di compleanno utilizzando sia HTML sia un file audio. Se il ricevitore dispone delle funzionalità audio, lo user agent analizzerà il file audio. In questo esempio il suono è come un sottotipo *external-body*, quindi per prima cosa lo user agent deve prendere il file *birthday.snd* via FTP. Se lo user agent non ha capacità audio, il suono verrà visualizzato sul video in silenzio. Le parti sono delimitate da due trattini seguiti da una stringa (generata dal software) specificata nel parametro *boundary.*

```
From: alice@cs.washington.edu
To: bob@ee.uwa.edu.au
MIME-Version: 1.0
Message-Id: <0704760941.AA00747@cs.washington.edu>
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
Subject: Earth orbits sun integral number of times

Questo è il preambolo. Lo user agent lo ignora. Buona giornata

--qwertyuiopasdfghjklzxcvbnm
Content-Type: text/html

<p>Tanti auguri a te<br>Happy
Tanti auguri a te<br>
Tanti auguri caro<b>Bob</b><br>
Tanti auguri a te</p>

--qwertyuiopasdfghjklzxcvbnm
Content-Type: message/external-body;
        access-type="anon-ftp";
        site="bicycle.cs.washington.edu";
        directory="pub";
        name="birthday.snd"

content-type: audio/basic
content-transfer-encoding: base64
--qwertyuiopasdfghjklzxcvbnm--
```

Figura 7.14 Un messaggio multipart contenente come alternative HTML e audio.

Occorre notare che l'intestazione *content-type* appare in tre posizioni all'interno di questo esempio. Nella parte superiore indica che il messaggio è diviso in più parti; all'interno di ogni parte restituisce il tipo e il sottotipo di tale parte. Per finire, all'interno del corpo della seconda parte viene richiesto di comunicare allo user agent quale tipo di file esterno deve analizzare. Per specificare questa leggera differenza nell'uso abbiamo usato le lettere minuscole, anche se le intestazioni non fanno distinzione tra maiuscole e minuscole. *Content-transfer-encoding* è richiesto anche per un corpo esterno che non sia codificato come ASCII a 7 bit.

Trasferimento dei messaggi

Ora che abbiamo descritto user agent e messaggi siamo pronti a descrivere come il message tranfer agent si occupi della trasmissione dei messaggi dal mittente al destinatario. Il trasferimento delle mail è fatto dal protocollo SMTP. Il modo più semplice per svolgere questa operazione è stabilire una connessione di trasporto dalla macchina sorgente a quella di destinazione per poi trasferire

semplicemente il messaggio. Così faceva SMTP originale. Tuttavia, nel corso degli anni, si sono differenziati due diversi tipi di uso di SMTP. Il primo è la **sottomissione della mail** (*mail submission*), passo numero uno nell'architettura delle e-mail descritta nella Figura 7.7. Questo è il modo in cui lo user agent invia messaggi nel sistema e-mail per la consegna. Il secondo uso è quello di trasferire messaggi ai message transfer agent (passo numero due nella Figura 7.7). Questa sequenza consegna le e-mail in un solo salto dal message transfer agent della sorgente a quello del destinatario. La consegna finale è effettuata da protocolli diversi descritti nel prossimo paragrafo, in cui descriveremo le basi del protocollo SMTP e i suoi meccanismi di estensione. Discuteremo come esso venga usato in modo diverso per la sottomissione delle e-mail e per il trasferimento dei messaggi.

SMTP e le sue estensioni

All'interno di Internet la posta elettronica viene consegnata costituendo una connessione tra la macchina sorgente e la porta 25 della macchina di destinazione. In ascolto su questa porta esiste un server di posta elettronica che utilizza **SMTP** (*simple mail transfer protocol*). Questo server accetta le connessioni in ingresso e copia nelle caselle di posta appropriate i messaggi ricevuti da queste connessioni. Se un messaggio non può essere consegnato, al mittente viene restituito un rapporto di errore (*undeliverable message*).

SMTP è un semplice protocollo ASCII. Questa non è una debolezza, ma una sua caratteristica. Usare testo ASCII facilita lo sviluppo dei protocolli e il relativo debug: possono essere validati inviando comandi composti manualmente e i record dei messaggi sono facili da leggere. La maggior parte dei protocolli Internet di livello applicazione funziona in questo modo (per esempio HTTP).

Vediamo ora come si effettua il trasferimento di un semplice messaggio tra server di posta elettronica che consegnano un messaggio. Stabilita la connessione TCP alla porta 25, la macchina trasmittente (che opera come client) attende la comunicazione da parte della macchina ricevente (che opera come server). Il server inizia inviando una riga di testo che comunica la sua identità e la possibilità di inviare la posta. Altrimenti il client rilascia la connessione e riprova in seguito.

Se il server è pronto ad accettare la posta elettronica, il client annuncia da chi proviene il messaggio e a chi è destinato. Se tale destinatario esiste, il server comunica al client di inviare il messaggio, dopo di che il client invia il messaggio e il server fornisce l'acknowledgement. Non sono necessari checksum, perché TCP offre un flusso di byte affidabile. Se vi sono più messaggi di posta elettronica ora vengono inviati tutti. Una volta scambiata tutta la posta elettronica in entrambe le direzioni, la connessione viene rilasciata. Una semplice finestra di dialogo per inviare il messaggio della Figura 7.14, comprensiva dei codici numerici utilizzati da SMTP, è mostrata nella Figura 7.15. Le linee inviate dal client, cioè chi trasmette, sono contrassegnate con *C:*. Quelle inviate dal server, cioè chi riceve, sono contrassegnate con *S:*.

```
                    S: 220 ee.uwa.edu.au SMTP service ready
C: HELO ee.uwa.edu.au
                    S: 250 cs.washington.edu says hello to ee.uwa.edu.au
C: MAIL FROM: <alice@cs.washington.edu>
                    S: 250 sender ok
C: RCPT TO: <bob@ee.uwa.edu.au>
                    S: 250 recipient ok
C: DATA
                    S: 354 Send mail; end with "." on a line by itself
C: From: alice@cs.washington.edu
C: To: bob@ee.uwa.edu.au
C: MIME-Version: 1.0
C: Message-Id: <0704760941.AA00747@ee.uwa.edu.au>
C: Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
C: Subject: Earth orbits sun integral number of times
C:
C: Questo è il preambolo. Lo user agent lo ignora. Buona giornata.
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: text/html
C:
C: <p>Tanti auguri a te
C: Tanti auguri a te
C: Tanti auguri caro <bold> Bob </bold>
C: Tanti auguri a te
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: message/external-body;
C: access-type="anon-ftp";
C: site="bicycle.cs.washington.edu";
C: directory="pub";
C: name="birthday.snd"
C:
C: content-type: audio/basic
C: content-transfer-encoding: base64
C: --qwertyuiopasdfghjklzxcvbnm
C: .
                    S: 250 message accepted
C: QUIT
                    S: 221 ee.uwa.edu.au closing connection
```

Figura 7.15 Spedizione di un messaggio da *alice@cs.washington.edu a bob@ee.uwa.edu.au*.

Il primo comando dal client è *HELO* da *"HELLO"* (*ciao*). Nella Figura 7.15 il messaggio è inviato a un solo destinatario, per cui viene utilizzato un solo comando *RCPT.* Questo comando permette l'invio di un singolo messaggio a più destinatari, dove per ognuno viene ricevuto un acknowledgement o un rifiuto. Anche se alcuni destinatari vengono rifiutati (perché non esistono nella destinazione), il messaggio può essere inviato agli altri.

Infine anche se la sintassi dei comandi di quattro caratteri è specificata dal client in modo rigido, la sintassi delle risposte lo è meno: in realtà contano solo i codici numerici e ogni implementazione può inserire la stringa che desidera dopo di essi.

SMTP nella sua forma di base funziona bene, ma è limitato in alcuni aspetti. Non include l'autenticazione. Questo significa che il comando *FROM* nell'esempio potrebbe dare qualunque indirizzo di sorgente, cosa assai utile per spedire spam. Un'altra limitazione è che SMTP trasferisce messaggi ASCII, non dati binari. Questo è il motivo per cui il MIME con base64 necessita il trasferimento di contenuti codificati. Tuttavia con questa codifica la trasmissione della posta usa la banda in modo inefficiente, il che è un problema per messaggi grandi. Una terza limitazione è che SMTP invia messaggi in chiaro: non dispone di cifratura per fornire una misura di sicurezza contro gli spioni.

Per risolvere questi e altri problemi legati all'elaborazione dei messaggi SMTP è stato rivisto per avere un meccanismo di estensioni. Tale meccanismo è parte obbligatoria dell'RFC 5321. L'uso di SMTP con tali estensioni è chiamato **ESMTP** (*extended SMTP*).

I client che desiderano utilizzarlo devono inviare un messaggio iniziale *EHLO* al posto di *HELO.* Se viene rifiutato, il server è un normale server SMTP e il client deve procedere nel modo solito. Se *EHLO* viene accettato, il server replica con le estensioni supportate.

Il client ora può usare qualunque di queste estensioni. Alcune estensioni comuni sono mostrate nella Figura 7.16. La figura mostra come le parole chiave siano usate nel meccanismo di estensione, insieme alla descrizione della nuova funzionalità. Non entreremo in ulteriori dettagli sulle estensioni.

Parola chiave	Descrizione
AUTH	Autenticazione del client
BINARYMIME	Il server accetta messaggi binary
CHUNKING	Il server accetta messaggi grandi suddivisi in parti
SIZE	Controllare la dimensione del messaggio prima di provare a inviare
STARTTLS	Passare a un trasporto sicuro (TLS, si veda il Capitolo 8)
UTF8SMTP	Indirizzo internazionale

Figura 7.16 Alcune estensioni SMTP.

Per comprendere meglio come funzionano SMTP e altri protocolli descritti in questo capitolo si può fare una prova. Per prima cosa occorre individuare una macchina connessa a Internet. Su un sistema UNIX (o Linux), digitate nella shell

```
telnet mail.isp.com 25
```

sostituendo alla scritta *mail.isp.com* dell'esempio il nome DNS del server di posta dell'ISP utilizzato. Su un sistema Windows XP, selezionate Start, Esegui e digitate il comando nella finestra di dialogo. Su macchine Vista o Windows 7 dovete prima installare il programma di telnet (o qualcosa di equivalente) e farlo partire. Questo comando stabilirà una connessione TCP alla porta 25 del computer indicato, quindi alla sua porta SMTP. Si veda la Figura 6.34 per le porte di alcuni protocolli comuni. Probabilmente otterrete una risposta come la seguente:

```
Trying 192.30.200.66...

Connessione a mail.isp.com

Connesso a mail.isp.com

Il carattere di escape è '^]'

220 mail.isp.com Smail #74 ready at Thu, 25 Sept 2002 13:26 +0200
```

Le prime tre righe provengono da telnet, che comunica che cosa sta facendo. L'ultima riga proviene dal server SMTP sulla macchina remota, che annuncia la sua disponibilità a comunicare e ad accettare la posta elettronica. Per scoprire i comandi accettati è possibile digitare:

```
HELP
```

Da questo punto in poi potete creare una sequenza di comandi come quella della Figura 7.16, se il server accetta mail da voi.

Sottomissione della e-mail

Originariamente gli user agent venivano eseguiti sullo stesso computer del message transfer agent della sorgente. In questo modo tutto ciò che era richiesto per inviare un messaggio allo user agent era di parlare ai server di posta locale usando il dialogo appena descritto. Tuttavia questo sistema non è più il caso standard.

Gli user agent vengono spesso eseguiti su computer portatili, computer domestici e telefoni mobili e non sono sempre connessi a Internet. I message transfer agent sono eseguiti su server di un ISP o aziendali e sono sempre connessi a Internet. Questo significa che uno user agent a Boston può aver bisogno di contattare il suo server di posta elettronica regolare a Seattle per inviare un messaggio, perché il suo utente è in viaggio.

Da sola questa comunicazione remota non pone alcun problema: è esattamente ciò per cui i protocolli TCP/IP sono progettati. Tuttavia, di solito un ISP o un'azienda non vogliono che un utente remoto qualsiasi possa sottoporre messaggi al proprio server di posta elettronica per essere consegnati altrove; l'ISP o l'azienda non stanno facendo funzionare i loro server come un servizio pubblico. Inoltre, questo tipo di **ritrasmissione libera della posta** (*open mail relay*) attrae chi invia lo spam; infatti in questo modo si fornisce un modo per mascherare il mittente originale e quindi rendere il messaggio meno facilmente identificabile come spam.

Date queste considerazioni, SMTP è normalmente usato per la sottomissione della e-mail con l'estensione *AUTH*, che permette al server di controllare le credenziali (nome utente e password) del client per confermare che il server possa fornire il servizio di posta elettronica.

Ci sono ulteriori differenze nel modo in cui SMTP è usato per la sottomissione della posta. Per esempio, la porta 587 è preferita alla porta 25 e il server SMTP può controllare e correggere il formato dei messaggi inviati dallo user agent. Per ulteriori informazioni riguardanti l'uso ristretto di SMTP per una sottomissione della posta si può consultare l'RFC 4409.

Trasferimento dei messaggi

Quando un message transfer agent sorgente riceve un messaggio da uno user agent, lo consegnerà al message transfer agent della posta ricevente usando SMTP. Per far ciò la sorgente usa l'indirizzo di destinazione. Si consideri il messaggio nella Figura 7.15 indirizzato a *bob@ee.uwa.edu.au*. A quale server di posta elettronica dovrebbe essere consegnato il messaggio?

Per determinare il corretto server di posta elettronica da contattare viene consultato il DNS. Nel paragrafo precedente abbiamo descritto come il DNS contenga molti tipi di record, incluso il record MX (o *mail exchanger*). In questo caso viene fatta un'interrogazione al DNS riguardante il record MX del dominio. Questa interrogazione restituisce una lista ordinata dei nomi degli indirizzi IP di uno o più server di posta elettronica.

Il message transfer agent sorgente stabilisce quindi una connessione TCP sulla porta 25 verso l'indirizzo IP del server di posta elettronica da raggiungere e usa SMTP per consegnare il messaggio. Il message transfer agent di posta elettronica ricevente quindi metterà la posta dell'utente Bob nella corretta casella di posta, in modo che Bob possa leggerla più tardi. Questo passaggio locale di consegne può includere il fatto di muovere il messaggio tra computer diversi in caso di una infrastruttura di posta complessa.

Con questo processo di consegna la posta viaggia dal message transfer agent iniziale a quello finale con un solo salto. Non ci sono server intermedi nel trasferimento dei messaggi. Tuttavia è possibile che questo processo di consegna avvenga in più passi. Un esempio che abbiamo già descritto è quando il message transfer agent implementa una mailing list. In questo caso un messaggio ricevuto per la lista viene espanso come un messaggio per ogni membro della lista e inviato agli indirizzi individuali.

Come ulteriore esempio di consegna Bob può essersi laureato al M.I.T. e può essere ancora raggiungibile all'indirizzo *bob@alum.mit.edu*. Piuttosto che leggere la posta su più account, Bob può fare in modo che la posta mandata a questo indirizzo venga inoltrata all'indirizzo *bob@ee.uwa.edu*. In questo caso la posta inviata a *bob@alum.mit.edu* subirà due consegne: prima verrà inviata al server di posta per *alum.mit.edu*, quindi verrà inviata al server di posta per *ee.uwa.edu*. Ognuna di queste parti è una consegna completa e separata, per quanto riguarda i message transfer agent.

Un'altra questione è quella dello spam: 9 messaggi su 10 inviati sono spam (McAfee, 2010). Sono sempre meno le persone interessate allo spam, ma è difficile evitarlo perché è mascherato da posta regolare. Prima di accettare un messaggio occorre effettuare ulteriori controlli per ridurre la possibilità che si tratti di spam. Il messaggio per Bob era inviato da *alice@cs.washington.edu*. Il message transfer agent ricevente può controllare il message transfer agent sorgente tramite il DNS. Questo gli permette di controllare se l'indirizzo IP all'altro capo della connessione TCP corrisponde al nome DNS del mittente. Generalmente l'agente ricevente può cercare il dominio di invio nel DNS per guardare se ha una politica particolare per l'invio della posta. Questa informazione è spesso presente nei record *TXT* e *SPF* e può indicare che sono necessari altri controlli. Per esempio, la posta proveniente da *cs.washington.edu* dovrebbe essere sempre inviata dall'host *june.cs.washington.edu*. Se il message transfer agent sorgente non è June c'è un problema.

Se uno di questi controlli fallisce probabilmente la posta è stata falsificata con un indirizzo di invio inesistente. In questo caso viene eliminata. Tuttavia passare questi controlli non implica che la mail non sia spam. I controlli dicono semplicemente che le mail sembrano provenire da una regione della rete da cui è verosimile provengano. L'idea è che chi genera spam dovrebbe essere forzato a usare il suo vero indirizzo quando manda e-mail. Questo renderebbe lo spam facilmente riconoscibile e cancellabile se indesiderato.

Consegna finale

La nostra e-mail è ora quasi al termine del suo percorso: è arrivata alla casella di posta di Bob. Ciò che rimane da fare è trasferire una copia del messaggio allo user agent di Bob per essere visualizzato. Questo è il passo 3 dell'architettura descritta nella Figura 7.7. All'inizio di Internet questa attività era molto semplice, quando cioè lo user agent e il message transfer agent venivano eseguiti sulla stessa macchina. Il message transfer agent scriveva semplicemente i nuovi messaggi in coda al file della casella di posta e lo user agent si limitava a controllare il file della casella di posta per vedere se c'erano nuovi messaggi.

Oggi lo user agent su PC, computer portatili o dispositivi mobili si trova verosimilmente su una macchina diversa dal server di posta elettronica dell'ISP o dell'azienda e gli utenti vogliono poter accedere alle proprie e-mail da remoto, ovunque siano: dal lavoro, da casa, mentre viaggiano e

dagli Internet caffé quando trascorrono le cosiddette vacanze. Vogliono anche lavorare senza essere connessi e quindi connettersi per ricevere i messaggi in arrivo e inviare a loro volta messaggi. Inoltre ogni utente può usare diversi user agent a seconda di quale computer è conveniente usare al momento e più user agent potrebbero essere attivi contemporaneamente.

In questo contesto il compito dello user agent è di visualizzare i contenuti della casella di posta e permettere che questa venga gestita da remoto. A questo scopo possono essere usati diversi protocolli differenti, ma non SMTP, che è un protocollo push:[6] prende un messaggio e si connette al server remoto per trasferirlo. La consegna finale non può essere eseguita in questo modo, sia perché la casella di posta deve continuare a essere memorizzata sul mail server sia perché lo user agent potrebbe non essere connesso a Internet nel momento in cui SMTP tenta di inoltrare i messaggi.

IMAP – Internet message access protocol

Uno dei protocolli principali usati per la consegna finale è **IMAP** (*Internet message access protocol*). La quarta versione del protocollo è definita nell'RFC 3501. Per fare uso di IMAP il mail server esegue un server IMAP che ascolta sulla porta 143. Lo user agent esegue un IMAP client che si connette al server e inizia a usare i comandi elencati nella Figura 7.17.

Comando	**Descrizione**
CAPABILITY	Elenca le funzionalità del server
STARTTLS	Passare a un trasporto sicuro (TLS, si veda il Capitolo 8)
LOGIN	Autenticazione sul server
AUTHENTICATE	Autenticazione con un altro metodo
SELECT	Seleziona una cartella
EXAMINE	Seleziona una cartella in sola lettura
CREATE	Crea una cartella
DELETE	Cancella una cartella
RENAME	Rinomina una cartella
SUBSCRIBE	Aggiunge una cartella a un archivio
UNSUBSCRIBE	Rimuove una cartella da un archivio
LIST	Elenca le cartelle disponibili
LSUB	Elenca le cartelle attive
STATUS	Controlla lo stato di una cartella
APPEND	Aggiunge un messaggio a una cartella
CHECK	Esegue un controllo di integrità su una cartella

Comando	Descrizione
FETCH	Recupera un messaggio da una cartella
SEARCH	Trova un messaggio in una cartella
STORE	Cambia i flag di un messaggio
COPY	Fa una copia di un messaggio in una cartella
EXPUNGE	Rimuove i messaggi marcati per la cancellazione
UID	Impartisce un comando usando un identificatore univoco
NOOP	Non fa nulla
CLOSE	Rimuove i messaggi marcati e chiude la cartella
LOGOUT	Si scollega e chiude la connessione

Figura 7.17 Comandi IMAP (versione 4).

Innanzitutto il client, qualora necessario, instaura una sessione di trasporto sicura (per tenere confidenziali i messaggi e i comandi) e quindi effettua il login o un'altra autenticazione sul server. Una volta autenticato esistono molti comandi per elencare le cartelle e i messaggi, per prendere i messaggi o parte di essi, etichettarli per una successiva cancellazione e organizzarli in cartelle. Per evitare confusione qui si fa uso del termine "cartella" per essere consistenti con il resto di questo paragrafo, nel quale un utente ha una singola casella di posta fatta da più cartelle. Tuttavia, nelle specifiche IMAP viene invece usato il termine "mailbox", quindi un utente può avere molte mailbox IMAP, ognuna delle quali viene tipicamente presentata all'utente come una cartella.

IMAP è un aggiornamento di un protocollo di consegna finale precedente, **POP3** (*post office protocol*, *version 3*) specificato nell'RFC 1939. POP3 è un protocollo più semplice, ma che supporta meno proprietà ed è meno sicuro. Invece di rimanere sul server i messaggi sono solitamente scaricati dallo user agent del computer. Questo facilita la vita ai server, ma la complica agli utenti. Non è facile leggere le e-mail su più computer; inoltre se lo user agent del computer si rompe, tutte le e-mail saranno perse in modo definitivo. Tuttavia, POP3 viene ancora usato. Possono essere usati anche protocolli proprietari, perché il protocollo può essere impiegato tra server di posta elettronica e user agent forniti dalla stessa azienda; Microsoft Exchange è un sistema di e-mail con un protocollo proprietario.

Webmail

Una sempre più popolare alternativa a IMAP e SMTP per fornire servizio di e-mail è l'impiego del Web come interfaccia per inviare e ricevere posta. I **Webmail** più utilizzati sono Google Gmail, Microsoft Hotmail e Yahoo! Mail. Quello di Webmail è un esempio di software (in questo caso uno user agent di posta) fornito come servizio usando il Web.

In questa architettura il provider gestisce un server di posta elettronica, come al solito, per accettare i messaggi degli utenti usando SMTP sulla porta 25. Tuttavia, lo user agent è diverso: invece di essere un programma indipendente è un'interfaccia utente che viene fornita tramite una pagina

Web. Questo significa che gli utenti possono usare qualunque browser vogliano per accedere alle loro e-mail e mandare nuovi messaggi.

Anche se il Web non è ancora stato esaminato, ne daremo qui una breve descrizione. Quando l'utente va alla pagina Web relativa alla posta elettronica, vede un form[7] che richiede un nome utente e una password che verranno inviati al server per la convalida. Se la procedura di autenticazione (*login*) ha successo, il server trova la casella di posta dell'utente e crea su due piedi una pagina Web che elenca il contenuto della casella di posta. La pagina Web viene quindi inviata al browser per la visualizzazione.
Molti degli elementi sulla pagina rispondo ai click del mouse per consentire lettura o eliminazione dei messaggi. Per avere un'interfaccia interattiva le pagine Web includono spesso programmi JavaScript. Questi programmi sono eseguiti localmente sul client in risposta a eventi locali (per esempio un click del mouse) e possono anche scaricare o caricare messaggi mentre l'utente fa altro. In questo modello la sottomissione di un messaggio avviene usando il normale protocollo Web, inviando i dati a un URL. Il server Web si preoccupa di inviare messaggi nel tradizionale sistema di consegna che abbiamo già descritto. Per quanto riguarda la sicurezza si possono usare i protocolli standard del Web; questi protocolli si preoccupano di cifrare pagine Web, non importa se il contenuto della pagina sia o meno un messaggio di posta elettronica.

7.3 Il World Wide Web

Il Web, com'è popolarmente chiamato il World Wide Web, è un'infrastruttura per l'accesso ai documenti collegati sparsi su milioni di macchine in Internet. In 10 anni si è evoluto da un modo per coordinare la progettazione di esperimenti di fisica delle alte energie usato in Svizzera a un'applicazione che milioni di persone considerano semplicemente come "Internet". La sua enorme popolarità è dovuta al facile utilizzo; inoltre fornisce moltissime informazioni su qualsiasi argomento, dagli abati agli Zulù, con una ricca interfaccia grafica.

Il Web nacque nel 1989 presso il CERN, il centro europeo per la ricerca nucleare. L'idea iniziale era aiutare grandi gruppi di scienziati provenienti da molte nazioni anche con diversi fusi orari a collaborare utilizzando una raccolta in costante cambiamento di resoconti, cianografie, documenti, disegni e fotografie prodotti dagli esperimenti di fisica delle particelle. La proposta iniziale per una rete di documenti collegati fu opera del fisico del CERN Tim Berners-Lee. Il primo prototipo (basato su testo) fu operativo 18 mesi dopo. Ne fu eseguita una presentazione pubblica alla conferenza Hypertext '91 a San Antonio, Texas. Questa dimostrazione pubblica attrasse l'attenzione di altri ricercatori, tra cui Marc Andreessen dell'Università dell'Illinois che iniziò a sviluppare il primo browser grafico. Venne chiamato Mosaic e fu reso disponibile al pubblico nel febbraio del 1993.

Il resto, come si suol dire, è storia. Mosaic divenne così popolare che, un anno dopo, Andreessen creò una nuova società, Netscape Communications Corp., il cui obiettivo era sviluppare programmi per il Web. Per i tre anni successivi, Netscape Navigator e Microsoft Internet Explorer ingaggiarono la "guerra dei browser", cercando di catturare clienti aggiungendo all'impazzata nuove caratteristiche (e quindi più bug) rispetto al prodotto concorrente.

Dal 1990 al 2010 circa i siti Web e le pagine Web, come sono chiamati i contenuti Web, sono cresciuti esponenzialmente fino a raggiungere milioni di siti e miliardi di pagine. Tra questi un piccolo numero di questi siti divenne incredibilmente popolare. Tali siti e le aziende che li hanno realizzati hanno definito il Web com'è adesso. Alcuni esempi sono: un negozio di libri (Amazon, iniziato nel 1994, ha una capitalizzazione di mercato di 50 miliardi di dollari), un mercatino delle pulci (eBay, 1995, 30 miliardi), un motore di ricerca (Google, 1998, 150 miliardi) e una rete sociale (Facebook, 2004, azienda privata valutata più di 15 miliardi). Anche il decennio intorno al 2000, quando molte aziende Web cominciarono a valere centinaia di milioni di dollari in una notte, solo per dissolversi praticamente il giorno successivo quando risultarono essere una montatura, ha un nome: **era del dot com**.[8] Nuove idee rendono tuttora un sacco di soldi sul Web e molte di queste provengono da studenti. Per esempio, Mark Zuckerberg era uno studente di Harvard quando Facebook partì e Sergey Brin e Larry Page erano studenti a Stanford quando diedero inizio a Google. Magari sarete voi i prossimi.
Nel 1994 CERN e M.I.T. firmarono un accordo per creare il **W3C** (*world wide web consortium*): un'organizzazione che ha come scopo l'ulteriore sviluppo del Web, lo standard dei protocolli e lo sviluppo dell'interoperabilità tra i siti. Berners-Lee divenne il direttore. Da allora, centinaia di università e società sono entrate a far parte del consorzio. Anche se ora esistono moltissimi libri sul Web, il luogo migliore dove trovare informazioni aggiornate è naturalmente il Web stesso. La

home page del consorzio si trova all'indirizzo *http://www.w3.org/*; i lettori interessati possono consultarla per seguire i collegamenti alle pagine che descrivono i suoi numerosi documenti e attività.

Panoramica dell'architettura

Dal punto di vista degli utenti il Web è una vasta raccolta mondiale di documenti o **pagine Web**, spesso definite per brevità solamente **pagine**. Ogni pagina può contenere collegamenti (*link*) ad altre pagine situate ovunque nel mondo. Gli utenti possono seguire un collegamento facendo click su di esso e raggiungendo così la pagina indicata; il processo può essere ripetuto indefinitamente. L'idea di fare in modo che ogni pagina puntasse a un'altra (un **ipertesto**, *hypertext*) venne nel 1945 a un visionario professore di ingegneria elettronica del M.I.T., Vannevar Bush, molto prima che fosse inventata Internet.

Le pagine sono visualizzate con un programma chiamato **browser**, di cui Firefox, Internet Explorer e Chrome sono le forme più popolari. Il browser preleva la pagina richiesta, interpreta il testo e i comandi di formattazione, quindi visualizza la pagina correttamente formattata sullo schermo. Il contenuto può essere un misto di testo, immagini e comandi di formattazione, come in un documento tradizionale oppure altre forme di contenuto come video o programmi che producono un'interfaccia grafica con cui l'utente può interagire.

Un esempio di pagina è mostrato in alto a sinistra nella Figura 7.18. È la pagina del dipartimento di informatica e ingegneria dell'Università di Washington. La pagina mostra elementi testuali e grafici. Alcune parti della pagina sono associate a collegamenti verso altre pagine. Un pezzo di testo, icona, immagine o altro collegati a un'altra pagina è chiamato **collegamento ipertestuale** (*hyperlink*). Per seguire un collegamento, l'utente posiziona il puntatore del mouse sull'area evidenziata e fa click con il mouse. Seguire un collegamento è un modo semplice per indicare al browser di prendere un'altra pagina. All'inizio i collegamenti erano spesso evidenziati da una sottolineatura o da un colore particolare. Oggigiorno i creatori delle pagine Web hanno modo di controllare come appaiono le parti con collegamenti, in modo che un collegamento possa apparire come un'icona o modificarsi quando il mouse ci passa sopra. Dipende dai creatori delle pagine fare in modo che i collegamenti siano ben distinguibili per fornire un'interfaccia fruibile agli utenti.

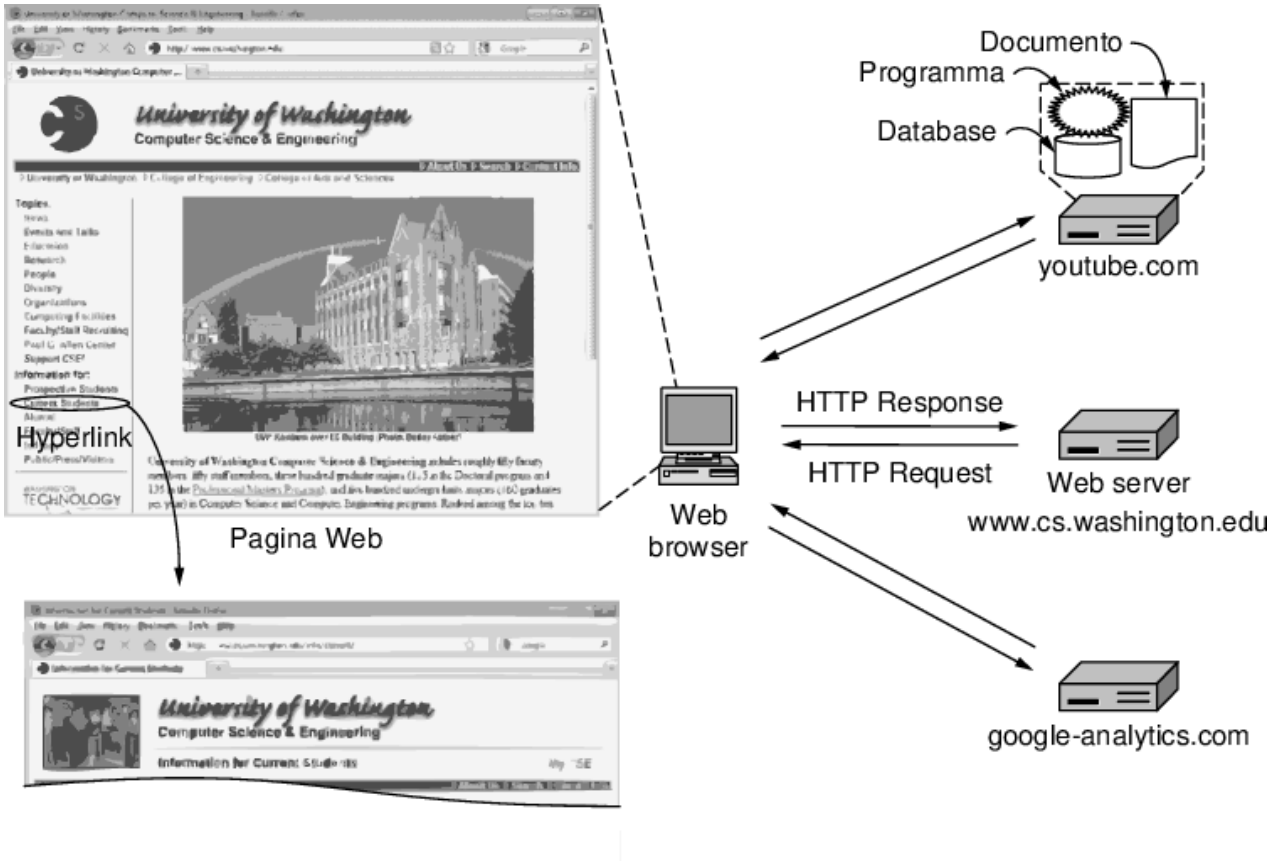


Figura 7.18 Architettura del Web.

Gli studenti di un dipartimento possono ricavare maggiori informazioni seguendo un collegamento a una pagina con informazioni a loro dedicate. Si accede a questo collegamento facendo click nell'area cerchiata nella figura. Il browser recupera la nuova pagina e la visualizza, come parzialmente mostrato in basso a a sinistra nella Figura 7.18. Ogni altra pagina può trovarsi sulla stessa macchina della prima o su una macchina dall'altra parte del globo; l'utente non può saperlo. Il recupero della pagina è svolto dal browser, senza alcun contributo da parte dell'utente: il movimento tra le macchine mentre si guardano contenuti è completamente trasparente all'utente.

Il modello di base del funzionamento del Web è mostrato nella Figura 7.18. Il browser visualizza una pagina Web sulla macchina client. Ogni pagina viene presa, inviando una richiesta a uno o più server che rispondono inviando i contenuti della pagina. Il protocollo di domanda e risposta per catturare le pagine è un semplice protocollo basato sul testo che funziona su TCP, esattamente come nel caso di SMTP. È chiamato **HTTP** (*hypertext transfer protocol*). Il contenuto può essere semplicemente un documento che viene letto da un disco, il risultato di una interrogazione a un database o l'esecuzione di un programma. Se il documento è sempre uguale siamo in presenza di una **pagina statica**, mentre se è generato su richiesta da un programma o contiene del codice eseguibile è chiamato **pagina dinamica**.

Una pagina dinamica si presenta in modo diverso ogni volta che viene visualizzata. Per esempio, la prima pagina di un negozio di elettronica può essere differente per ogni visitatore. Se il cliente di una libreria ha comprato nel passato romanzi polizieschi, quando visiterà la pagina principale del negozio probabilmente vedrà visualizzati prevalentemente romanzi polizieschi, mentre un cliente

più appassionato di gastronomia vedrà nuovi libri di cucina. Come un sito Web tiene traccia delle nostre passioni è una storia di cui parleremo fra poco. Per il momento diciamo che la risposta coinvolge degli oggetti chiamati *cookie*.

Nella figura il browser contatta tre server per prelevare tre pagine, *cs.washington.edu*, *youtube.com* e *google-analytics.com*. I contenuti derivanti da questi tre differenti server sono integrati dal browser per essere visualizzati. La visualizzazione coinvolge una serie di elaborazioni che dipendono dal tipo di contenuto. Oltre a elaborare il testo e la grafica può precedere la visualizzazione di un video o l'esecuzione di uno script che ha la propria interfaccia utente come parte della pagina. In questo caso il server *cs.washington.edu* fornisce la pagina principale, il server *youtube.com* il video integrato e *google-analytics.com* non dà niente all'utente, ma traccia i visitatori del sito. Avremo ancora molto da dire riguardo ai tracciatori più avanti.

Il lato client

Esaminiamo ora in dettaglio il lato client della Figura 7.18. In pratica, un browser è un programma che può visualizzare una pagina Web e rilevare i click del mouse sugli elementi in essa visualizzati. Quando viene selezionato un elemento, il browser segue il collegamento ipertestuale e preleva la pagina selezionata.

Quando fu creato il Web, divenne immediatamente chiaro che la disponibilità di pagine che facessero riferimento l'una all'altra richiedeva un meccanismo per denominare e individuare le pagine. In particolare occorreva rispondere a tre domande prima di poter visualizzare una pagina selezionata:

1. come si chiama la pagina
2. dov'è situata la pagina
3. com'è possibile accedere alla pagina.

Se a ogni pagina fosse assegnato un nome univoco non vi sarebbero ambiguità nell'identificazione; tuttavia il problema non sarebbe risolto. Si consideri un'analogia tra persone e pagine. In Italia tutti possiedono un codice fiscale, che è un identificatore univoco (due persone non dovrebbero avere mai lo stesso). Ciononostante, se si dispone solo del codice fiscale non c'è modo di individuare l'indirizzo del proprietario e ovviamente non c'è modo di sapere se a questa persona sa scrivere in italiano, spagnolo o cinese. Il Web presenta fondamentalmente gli stessi problemi.

La soluzione scelta identifica le pagine in un modo che risolve tutti e tre i problemi contemporaneamente. A ogni pagina è assegnato un **URL** (*uniform resource locator*) che serve effettivamente come nome mondiale per la pagina. Gli URL sono composti di tre parti: il protocollo (noto anche come schema), il nome DNS della macchina su cui è situata la pagina e un percorso che indica in modo univoco la pagina specifica (un file da leggere o un programma da eseguire). Nel caso generale il percorso ha una struttura gerarchica che rispecchia il modello della struttura gerarchica di directory e file. Tuttavia, l'interpretazione del percorso è compito del server; può riflettere o meno la vera struttura delle directory. Ad esempio l'URL della pagina mostrata nella Figura 7.18 è:

```
http://www.cs.washington.edu/index.html
```

Questo URL è composto di tre parti: il protocollo (*http*), il nome DNS dell'host (*www.cs.washington.edu*) e il nome del percorso (*index.htmt*).

Quando un utente fa click su un collegamento ipertestuale il browser esegue una serie di passaggi per recuperare la pagina indicata. Vediamo i passaggi seguiti alla selezione di questo collegamento.

1. Il browser determina l'URL (osservando che cosa è stato selezionato).
2. Il browser richiede al DNS l'indirizzo IP del server *www.cs.washington.edu*
3. Il DNS risponde con 128.208.3.88.
4. Il browser esegue una connessione TCP alla porta 80 su 128.208.3.88.
5. Invia una richiesta HTTP per la pagina */index.html.*
6. Il server *www.cs.washington.edu* invia la pagina come risposta HTTP inviando, per esempio, il file */index.html*.
7. Se la pagina include URL necessari alla visualizzazione il browser prende gli altri URL usando lo stesso procedimento. In questo caso, l'URL include più immagini, anche queste prese da *www.cs.washington.edu*, un video integrato da *youtube.com* e uno script da *google-analytics.com*.
8. Il browser visualizza la pagina */index.html* come appare nella Figura 7.18.
9. La connessione TCP viene rilasciata se non vi sono altre richieste agli stessi server per un breve periodo.

Molti browser visualizzano il passaggio che stanno eseguendo in una riga di stato nella parte inferiore dello schermo. In questo modo, se le prestazioni sono scadenti l'utente può vedere se la causa è il DNS che non risponde, il server che non risponde o che la rete è lenta o congestionata.

Questo degli URL è uno schema aperto, nel senso che consente ai browser di utilizzare più protocolli per raggiungere tipi diversi di risorse. In effetti sono stati definiti URL per altri protocolli comuni. Alcune forme semplificate dei più usati sono elencate nella Figura 7.19.

Nome	**Utilizzato per**	**Esempio**
http	Ipertesto (HTML)	http://www.ee.uwa.edu/~rob/
https	Ipertesto con sicurezza	https://www.bank.com/accounts/
ftp	Trasferimento file con FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	File locale	file:///usr/suzanne/prog.c
mailto	Spedire e-mail	mailto:JohnUser@acm.org
rtsp	Streaming multimediale	rtsp://youtube.com/montypython.mpg
sip	Chiamate multimediali	sip:eve@adversary.com
about	Informazioni sul browser	about:plugins

Figura 7.19 Alcuni comuni schemi di URL.

Diamo uno sguardo all'elenco. Il protocollo *http* è il linguaggio nativo del Web, quello "parlato" dai Web server. **HTTP** sta per *hypertext transfer protocol*. Lo esamineremo in dettaglio nel seguito nel paragrafo.

La modalità di accesso *ftp* è utilizzata per accedere ai file con FTP, il protocollo di trasferimento file di Internet. FTP è precedente al Web ed è stato in uso per più di tre decenni. Il Web facilita l'operazione di prendere file situati su numerosi server FTP nel mondo, fornendo una semplice interfaccia su cui fare click, invece che un'interfaccia a linea di comando. Questa facilitazione nell'accesso delle informazioni è uno dei motivi che ha portato alla crescita spettacolare del Web.

È possibile accedere a un file locale come se fosse una pagina Web, sia utilizzando la modalità di accesso *file* sia, più semplicemente, indicando solo il suo nome. Questo approccio non richiede la disponibilità di un server. Naturalmente funziona solo per i file locali, non per quelli remoti.

La modalità di accesso *mailto* in realtà non prende pagine Web, ma consente agli utenti di inviare posta elettronica da un browser. La maggior parte dei browser risponde a un collegamento *mailto* avviando uno user agent di e-mail che componga il messaggio con l'indirizzo già compilato.

Gli schemi di accesso *rtsp* e *sip* stabiliscono sessioni di media streaming e chiamate audio e video.

Infine la modalità di accesso *about* è una convenzione che fornisce informazioni sul browser. Per esempio, l'URL *about:plugins* fa in modo che la maggior parte dei browser mostri una pagina con l'elenco dei tipi MIME in grado di gestire insieme alle estensioni del browser chiamate, appunto, plug-in.

In breve gli URL sono stati progettati non solo per permettere agli utenti di navigare il Web, ma anche per implementare protocolli più vecchi come FTP, spedire e-mail, supportare protocolli più nuovi per audio e video e fornire un accesso conveniente ai file locali e alle informazioni del browser; questo approccio rende superflui i programmi specializzati di interfaccia verso tutti quegli altri servizi e integra praticamente tutti gli accessi a Internet in un solo programma: il Web browser. Se non fosse per il fatto che l'idea l'ha avuta un fisico inglese che lavorava in un laboratorio di ricerca in Svizzera potrebbe facilmente passare per un piano visionario della divisione pubblicitaria di un'azienda di software.

Nonostante tutte queste belle proprietà, l'utilizzo crescente del Web ha evidenziato una debolezza nello schema degli URL. Un URL fa riferimento a un host specifico, ma a volte è utile riferirsi a una pagina senza dover dire contemporaneamente dove sia. Per esempio, per le pagine visitate in modo "pesante" è utile disporre di più copie ben distanti per ridurre il traffico di rete. Non c'è alcun modo per dire: "Voglio la pagina *xyz*, ma non importa da dove venga presa".

Per risolvere questo problema gli URL sono stati generalizzati in URI (*uniform resource identifier*). Alcuni, come gli URL, dicono dove è situata una risorsa, mentre altri URI dicono il nome della risorsa, ma non dove si trova. Questi sono chiamati **URN** (*uniform resource name*). Le regole per

scrivere gli URI sono indicate nell'RFC 3986, mentre i diversi schemi di URI in uso sono tracciati da IANA. Oltre a quelli elencati nella Figura 7.19 ve sono molti altri, ma sono questi a dominare il Web in questo momento.

Tipi MIME

Per essere in grado di visualizzare la nuova pagina (o qualsiasi pagina) il browser deve comprenderne il formato. Per consentire a tutti i browser di comprendere tutte le pagine, le pagine Web sono scritte in un linguaggio standardizzato chiamato HTML, che per ora è la lingua franca del Web. Ne parleremo in dettaglio nel seguito di questo capitolo.

Anche se un browser è fondamentalmente un interprete HTML, la maggior parte offre pulsanti e funzioni per facilitare l'esplorazione del Web: un pulsante per tornare alla pagina precedente, un pulsante per passare alla successiva (operativo solo se l'utente è prima tornato indietro di una pagina) e un pulsante per passare direttamente alla pagina iniziale predefinita. Molti browser hanno un pulsante o una voce di menu per impostare un segnalibro su una data pagina e un altro per visualizzare l'elenco dei segnalibri, consentendo di rivisitare le pagine con pochi click del mouse.

Nel nostro esempio le pagine HTML possono contenere elementi più ricchi e non semplicemente testo e ipertesto. Non solo, non tutte le pagine devono contenere HTML. Una pagina può consistere di un video in formato MPEG, di un documento in formato PDF, di una fotografia in formato JPEG, di un brano in formato MP3 o di centinaia di altri tipi di file. Dal momento che le pagine HTML standard possono collegarsi a tutti questi elementi, il browser può avere problemi quando incontra una pagina che non è in grado di interpretare.

Anziché rendere i browser sempre più grandi incorporandovi interpreti per una raccolta di tipi di file in rapida crescita, la maggior parte dei produttori ha scelto una soluzione più generica. Quando un server restituisce una pagina, comunica anche alcune informazioni aggiuntive, che comprendono il tipo MIME della pagina (Figura 7.13). Le pagine di tipo *text/html* sono visualizzate direttamente, come le pagine di altri tipi incorporati. Se il tipo MIME non corrisponde a uno di quelli incorporati, il browser consulta la sua tabella di tipi MIME per capire come visualizzare la pagina. Questa tabella associa un tipo MIME a un visualizzatore.

Esistono due possibilità: plug-in e applicazioni di supporto (*helper application*). Un **plug-in** è un modulo di codice che il browser preleva da una directory speciale sul disco e installa come estensione a se stesso, come illustrato nella Figura 7.20(a). Esempi comuni sono quelli per PDF, Flash e Quicktime per visualizzare documenti e riprodurre audio e video. Dal momento che i plug-in sono eseguiti nel browser, hanno accesso alla pagina corrente e possono modificare il suo aspetto.

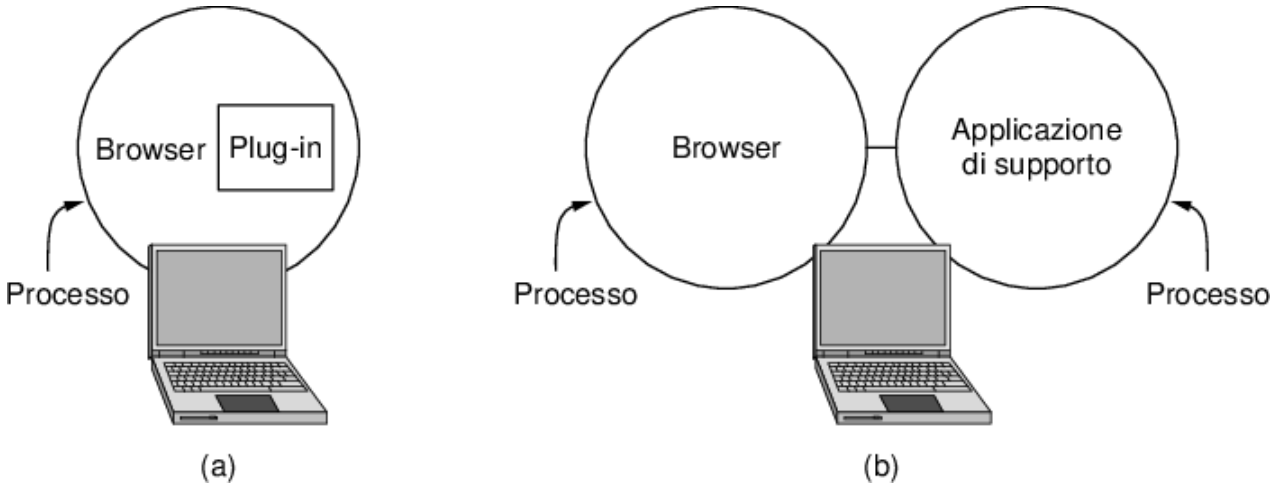


Figura 7.20 (a) Un plug-in di un browser. (b) Un'applicazione di supporto.

Ogni browser possiede un set di procedure che tutti i plug-in devono implementare affinché il browser possa richiamarli. Per esempio, solitamente il codice base del browser chiama una procedura per fornire al plug-in i dati da visualizzare. Questo set di procedure è l'interfaccia del plug-in ed è specifico del browser.

Inoltre il browser crea un set di proprie procedure per il plug-in, al fine di fornirgli dei servizi. Le procedure tipiche nell'interfaccia del browser riguardano allocazione e deallocazione della memoria, visualizzazione di un messaggio sulla riga di stato del browser e interrogazione del browser in merito ai parametri di funzionamento.

Prima di poter utilizzare un plug-in occorre installarlo. L'abituale procedura d'installazione prevede che l'utente si colleghi al sito Web del plug-in e scarichi un file di installazione. L'esecuzione del file di installazione spacchetta il plug-in e fa la chiamata appropriata per registrare il tipo MIME nel browser e associarlo al plug-in. I browser hanno di solito già inseriti i plug-in più comuni.

L'altro modo per estendere un browser consiste nell'utilizzare un'**applicazione di supporto**. Si tratta di un programma completo eseguito come un processo separato e illustrato nella Figura 7.20(b). Dal momento che l'helper è un programma separato, esso non offre un'interfaccia al browser. Al contrario, solitamente accetta il nome di un file temporaneo dove è memorizzato il contenuto, lo apre e lo visualizza. Generalmente, gli helper sono programmi complessi indipendenti dal browser, come Microsoft Word o PowerPoint.

Molte applicazioni di supporto utilizzano il tipo MIME *application*, per cui è stato definito un numero considerevole di sottotipi, per esempio *application/vnd.ms-powerpoint* per i file PowerPoint. *vnd* denota lo specifico formato del produttore (*vendor*). In questo modo un URL può puntare direttamente a un file PowerPoint; quando l'utente fa click viene automaticamente avviato PowerPoint. Le applicazioni di supporto non sono ristrette al tipo MIME *application*; Adobe Photoshop usa ad esempio *image/x-photoshop*.

Di conseguenza i browser possono essere configurati per gestire un numero virtualmente illimitato di tipi di documenti senza bisogno di modifiche. I server Web moderni sono spesso configurati con centinaia di combinazioni tipo/sottotipo; ne vengono aggiunte di nuove ogni volta che viene installato un nuovo programma.

Quando sono disponibili più plug-in ed helper per alcuni sottotipi, come *video/mpg*, nascono dei conflitti. In pratica, l'ultimo programma registrato sovrascrive le associazioni esistenti col tipo MIME, acquisendo la gestione del tipo. Di conseguenza, l'installazione di un nuovo programma può cambiare il modo in cui un browser gestisce i tipi esistenti.

I browser possono anche aprire i file locali, anziché prelevarli da server Web remoti. Dal momento che i file locali non possiedono tipi MIME, il browser ha bisogno di un qualche metodo per determinare il tipo MIME del file. Il metodo standard è che il sistema operativo associ l'estensione del file al tipo MIME. In una configurazione tipica, aprendo *foo.pdf*, il browser lo aprirà usando il plug-in *application/pdf*, mentre l'apertura di *bar.doc* farà in modo che il browser apra il file in Word tramite l'helper *application /msword*.

Anche qui possono sorgere conflitti, perché molti programmi desiderano (o meglio, bramano) appropriarsi della gestione di particolari file, per esempio *.mpg.* Durante l'installazione, i programmi destinati ai professionisti spesso visualizzano caselle di controllo per i tipi MIME e le estensioni che sono in grado di gestire, consentendo all'utente di scegliere quelle appropriate e di non sovrascrivere accidentalmente le associazioni esistenti. I programmi mirati al mercato dei consumatori presumono che l'utente non sappia che cos'è un tipo MIME e acquisiscono tutto ciò che possono senza riguardo per quanto fatto dai programmi installati in precedenza.

La capacità di estendere il browser con un elevato numero di nuovi tipi è conveniente, ma genera anche alcuni problemi. Quando Internet Explorer preleva un file con estensione *exe,* comprende che questo file è un programma eseguibile e di conseguenza non ha un helper. L'azione ovvia è eseguire il programma. Tuttavia, questo potrebbe generare un enorme falla nella protezione. Un sito Web maligno potrebbe produrre una pagina Web con immagini di stelle dello spettacolo o dello sport, collegate a un virus. Un singolo click su un'immagine potrebbe provocare lo scaricamento del file e l'esecuzione sul computer dell'utente di un programma eseguibile sconosciuto e potenzialmente ostile. Per impedire ospiti indesiderati come questo, Firefox e altri browser sono configurati per segnalare l'esecuzione di programmi sconosciuti, ma non tutti gli utenti capiscono quali scelte sono sicure piuttosto che utili.

Il lato server

Finora abbiamo presentato il lato client; adesso osserviamo il lato server. Come affermato in precedenza, quando l'utente digita un URL o fa click su un collegamento ipertestuale, il browser analizza sintatticamente l'URL e interpreta la parte tra *http://* e lo slash successivo come il nome DNS da ricercare. Armato dell'indirizzo IP del server, il browser stabilisce una connessione TCP sulla relativa porta 80. Invia poi un comando contenente il resto dell'URL, vale a dire il nome di un file sul server, che quindi restituisce il file per la visualizzazione da parte del browser.

In prima approssimazione un server Web è simile al server della Figura 6.6. A tale server, come un vero server Web, viene fornito il nome di un file da cercare e restituire. In entrambi i casi, i passaggi eseguiti dal server nel suo ciclo principale sono i seguenti:

1. accettare una connessione TCP da un client (un browser)
2. ottenere il percorso della pagina che è il nome del file richiesto
3. ottenere il file (dal disco)
4. inviare il contenuto del file al client
5. rilasciare la connessione TCP.

I server Web moderni hanno anche altre funzionalità, ma in sostanza le operazioni sono queste, per il semplice caso di contenuti di un file. Per contenuti dinamici, il terzo passaggio viene sostituito dall'esecuzione di un programma (determinato dal percorso) che restituisce i contenuti.

Tuttavia, i Web server hanno diversi schemi per soddisfare molte richieste al secondo. Un problema di questo semplice schema è che l'accesso al disco è spesso il collo di bottiglia. La lettura da disco è molto lenta rispetto all'esecuzione di un programma e gli stessi file possono essere letti ripetutamente dal disco usando chiamate al sistema operativo. Un altro problema è che può essere elaborata solo una richiesta alla volta. I file possono essere grandi e quindi bloccare le altre richieste mentre vengono trasferiti.

Un miglioramento ovvio (utilizzato da tutti i server Web) consiste nel mantenere in memoria in una cache gli n file utilizzati più di recente o un certo numero di gigabyte di contenuti. Prima di accedere al disco per recuperare un file, il server controlla la cache. Se il file è presente, può essere inviato direttamente dalla memoria eliminando l'accesso al disco. Anche se l'utilizzo effettivo della cache richiede un'elevata quantità di memoria principale e un tempo di elaborazione extra per controllare la cache e gestirne il contenuto, il risparmio in termini di tempo vale sempre l'overhead e la spesa.

Per ovviare al problema di servire una singola richiesta alla volta una strategia è implementare il server con un approccio **multithread**.[9] In una possibile struttura il server consiste di un modulo di front end che accetta tutte le richieste in ingresso e k moduli di elaborazione come mostrato nella Figura 7.21. I $k + 1$ thread appartengono tutti allo stesso processo, in modo che tutti i moduli di elaborazione abbiano accesso alla cache all'interno dello spazio di indirizzamento del processo. Quando arriva una richiesta, il front end l'accetta e costruisce un piccolo record che la descrive. Passa quindi il record a uno dei moduli di elaborazione per l'erogazione del servizio.

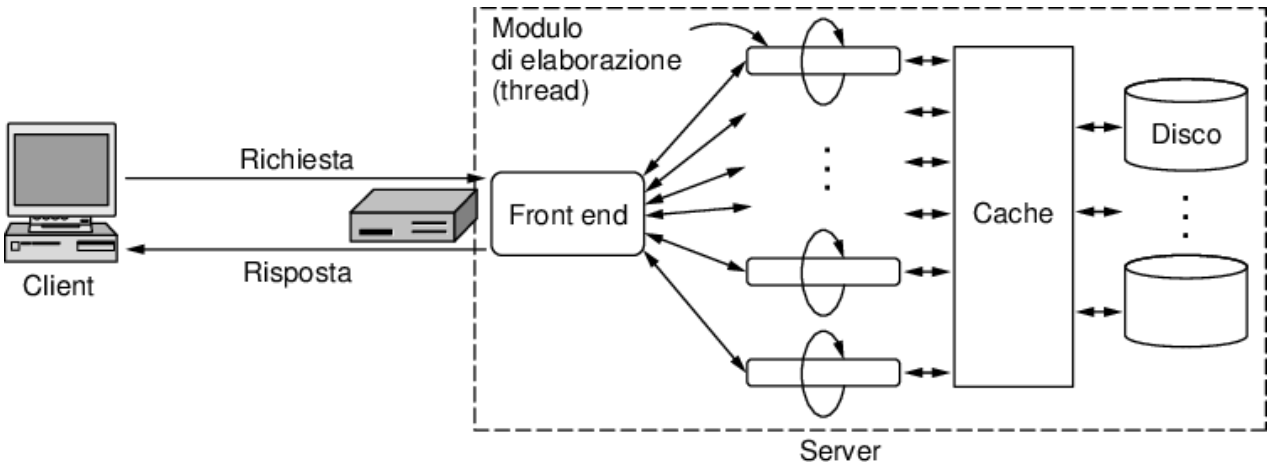


Figura 7.21 Un server web con multithread con un front end e moduli di elaborazione.

Il modulo di elaborazione controlla per prima cosa la cache per vedere se contiene il file necessario. In questo caso, aggiorna il record per includervi un puntatore al file. Se non è presente, il modulo di elaborazione avvia un'operazione su disco per la lettura (scartando, eventualmente, altri file nella cache per creare spazio). Quando il file viene recuperato dal disco, esso viene inserito nella cache e restituito al client.

Il vantaggio di questo schema è il seguente: mentre uno o più moduli di elaborazione sono bloccati in attesa del completamento di un'operazione su disco o di rete (senza consumare tempo della CPU), altri moduli possono lavorare attivamente su altre richieste. Con *k* moduli di elaborazione, la velocità dei dati può essere *k* volte superiore a quella ottenibile con un server dotato di un singolo thread. Naturalmente quando il disco o la rete sono i fattori limitanti, è necessario avere più dischi o una rete più veloce per ottenere un reale miglioramento sul modello a singolo thread.

Oltre ad accettare nomi e a restituire i file i moderni server Web svolgono altre operazioni, per cui l'elaborazione effettiva di ogni richiesta può divenire complessa. Per questo motivo in molti server ogni modulo di elaborazione esegue una serie di passaggi. Il front end passa ogni richiesta in ingresso al primo modulo disponibile, che la soddisfa utilizzando un sottoinsieme dei seguenti passi (che dipende da quali sono necessari per quella particolare richiesta). Questi passi avvengono dopo che è stata stabilita la connessione TCP e i meccanismi di trasporto sicuri (come SSL/TLS, che verrà descritto nel Capitolo 8):

1. risolvere il nome della pagina Web richiesta
2. eseguire il controllo di accesso sulla pagina Web
3. controllare la cache
4. prelevare la pagina richiesta dal disco o eseguire un programma che la costruisca
5. determinare il resto della risposta (per esempio, il tipo MIME da includere)
6. restituire la risposta al client
7. aggiungere un elemento al registro delle operazioni svolte dal server.

Il passo 1 è necessario perché la richiesta in ingresso potrebbe non contenere il nome effettivo del file come stringa letterale. Può contenere scorciatoie che devono essere tradotte. Per esempio, si consideri l'URL *http:/vww.cs.vu.nl/*, che presenta un nome di file vuoto da espandere in qualche nome di file predefinito, solitamente *index.html*. Un'altra regola comune è associare *~user/* a una directory nella home dell'utente.[10] Queste regole possono essere usate insieme. Quindi la home page di uno degli autori di questo libro (Tanenbaum, AST) può essere raggiunta a

```
http://www.cs.vu.nl/~ast/
```

anche se il vero nome del file è *index.html* posto in una directory predefinita. In generale l'espansione del nome non è così semplice come può sembrare, a causa di numerose convenzioni su come associare il percorso alle directory e ai programmi.

Inoltre i browser moderni possono specificare informazioni di configurazione, come la lingua predefinita dell'utente (italiano o inglese). In questo modo il server può selezionare una pagina Web con figure piccole per un dispositivo mobile e la lingua preferita, se è disponibile.

Il passo 2 controlla se ci sono restrizioni di accesso associate alla pagina incontrata. Non tutte le pagine sono disponibili per il grande pubblico; il fatto che un client possa accedere a una pagina può dipendere dall'identità del client (per esempio, dati il nome utente e la password), da dove è situato il client relativamente al DNS o all'IP. Per esempio, una pagina potrebbe essere disponibile solo agli utenti interni a un'azienda. Come ciò venga realizzato dipende da come è stato progettato il server. Per il server Apache, per esempio, la convenzione è quella di porre un file chiamato *.htaccess* che elenca le restrizioni di accesso alla directory dove sono situate le pagine con autorizzazione ristretta.

I passi 3 e 4 riguardano il recupero della pagina. Il fatto che possa essere presa dalla cache dipende dalle regole di elaborazione. Per esempio, le pagine che vengono create dall'esecuzione di programmi non sempre possono essere archiviate nella cache, perché tali programmi potrebbero produrre un risultato differente ogni volta che vengono eseguiti. Si dovrebbe controllare occasionalmente se anche i contenuti dei file siano cambiati, in modo che i contenuti vecchi possano essere rimossi. Se una pagina richiede di eseguire un programma c'è anche il problema di definire i parametri del programma o l'input. Questi dati vengono estrapolati dal percorso o da altre parti della richiesta.

Il passo 5 riguarda la determinazione di altre parti della risposta che accompagnano i contenuti della pagina. Il tipo MIME è un esempio. Può venire dall'estensione del file, i primi byte del file o dell'output del programma, un file di configurazione e altre possibili fonti.

Il passo 6 invia la pagina attraverso la rete. Per migliorare le prestazioni, una singola connessione TCP può essere usata da un client e da un server per prendere più pagine. Questo riutilizzo implica l'esistenza di regole per associare una richiesta a una connessione condivisa e dare ogni risposta in modo che sia associata alla domanda corrispondente.

Il passo 7 aggiunge un'informazione nel log di sistema a scopi amministrativi e tiene traccia di altri importanti statistiche. Queste informazioni possono essere usate successivamente per ottenere informazioni importanti sui comportamenti degli utenti, quali l'ordine in cui le persone accedono alle pagine.

I cookie

Navigare per il Web come descritto finora comporta il recupero di una serie di pagine indipendenti. Non esiste il concetto di sessione: il browser invia una richiesta a un server e ottiene il file, dopo di che il server dimentica di aver mai visto quel particolare client.

All'inizio, quando il Web era utilizzato solo per recuperare documenti disponibili pubblicamente, questo modello era perfettamente adeguato. Tuttavia esso non è adatto per fornire pagine differenti a utenti diversi a seconda di quello che hanno già fatto con il server. Questo

comportamento è necessario per molte interazioni che prevedano una sequenza logica di scambi di informazioni con i siti Web. Per esempio, alcuni siti Web (come quelli dei quotidiani) richiedono ai clienti di registrarsi (e magari di pagare) per l'utilizzo. Questo fa nascere il problema di come possano i server distinguere tra le richieste degli utenti registrati e quelle di altri utenti. Un secondo esempio è relativo al commercio elettronico. Se gli utenti visitano un negozio elettronico, aggiungendo elementi al carrello di volta in volta, come può il server tenere traccia del contenuto del carrello? Un terzo esempio è dato dai portali Web personalizzati come Yahoo!. Gli utenti possono impostare una pagina iniziale dettagliata con le informazioni desiderate (per esempio le loro azioni e le loro squadre sportive preferite); come può il server visualizzare la pagina corretta se non sa chi è l'utente?

A prima vista si potrebbe pensare che i server possano tenere traccia degli utenti osservandone gli indirizzi IP. Tuttavia questa idea non funziona. Prima di tutto, molti utenti lavorano su computer condivisi, specialmente a casa: gli indirizzi IP identificano solamente il computer, non l'utente. In secondo luogo, molte aziende utilizzano NAT, pertanto tutti i pacchetti in uscita provenienti da tutti gli utenti contengono lo stesso indirizzo IP. Dal punto di vista del server, tutti i computer dietro una NAT sembrano uguali. Inoltre, molti ISP assegnano gli indirizzi IP ai clienti con DHCP. Gli indirizzi IP cambiano nel tempo, quindi per un server potreste all'improvviso apparire come il vostro vicino di casa. Per tutti questi motivi i server non possono usare gli indirizzi IP per tenere traccia degli utenti.

Il problema è stato risolto con un meccanismo spesso criticato, che è quello dei **cookie**. Il nome deriva da un antico gergo dei programmatori, in cui un programma chiama una procedura e ottiene qualcosa che potrebbe in seguito dover presentare per svolgere un lavoro. In questo senso, un descrittore di file UNIX o il riferimento (handle) a un oggetto Windows possono essere considerati dei cookie. I cookie sono stati inventati da Netscape nel 1994 e sono ora specificati nell'RFC 2109.

Quando un client richiede una pagina Web il server può fornire informazioni aggiuntive sotto forma di cookie insieme alla pagina richiesta. Un cookie è un piccolo file o stringa (che non supera i 4 KB) che il server può associare a un browser. Questa associazione non è la stessa cosa di un utente, ma è più simile e più utile di un indirizzo IP. I browser archiviano i cookie in un'apposita directory sul disco rigido del client, a meno che l'utente non li abbia disattivati. I cookie sono solo file o stringhe, non programmi eseguibili; potrebbero teoricamente contenere un virus, ma visto che sono trattati come dati, non esiste un modo ufficiale per mandare in esecuzione l'ipotetico virus e quindi causare danni. Tuttavia non si può escludere che qualche hacker sfrutti un bug del browser per provocarne l'attivazione.

Come mostrato nella Figura 7.22, un cookie può contenere fino a cinque campi. *Domain* (dominio) indica da dove proviene il cookie. I browser dovrebbero verificare che i server non mentano in merito al loro dominio. Su ogni client non può essere mantenuto un numero superiore a venti cookie provenienti dallo stesso dominio. *Path* (percorso) è un percorso nella struttura delle directory del server che identifica quale parte della struttura di file del server può utilizzare il cookie. Spesso corrisponde a "/", che indica l'intera struttura.

Dominio	Percorso	Contenuto	Scadenza	Sicuro
toms-casino.com	/	CustomerID=497793521	15-10-10 17:00	Sì
jills-store.com	/	Cart=1-00501;1-07031;2-13721	11-1-11 14:22	No
aportal.com	/	Prefs=Stk:CSCO+ORCL;Spt:Jets	31-12-20 23:59	No
sneaky.com	/	UserID=3627239101	31-12-19 23:59	No

Figura 7.22 Alcuni esempi di cookie.

Il campo *Content* (contenuto) assume la forma *nome* = *valore*. *Nome* e *valore* sono due stringhe a discrezione del server e rappresentano il contenuto del cookie.

Il campo *Expires* (scadenza) specifica quando scade il cookie. Se il campo è assente, il browser scarta il cookie alla sua chiusura; un cookie di questo tipo è definito **cookie non persistente**. Se vengono fornite una data e un'ora, il cookie è **persistente** ed è conservato fino alla scadenza. I tempi di scadenza sono forniti secondo l'ora di Greenwich. Per rimuovere un cookie dal disco rigido del client un server deve inviarlo di nuovo, ma con un tempo di scadenza già trascorso.

Per finire, il campo *Secure* (sicuro) può essere impostato per indicare che il browser può restituire il cookie a un server solo usando un sistema di trasporto sicuro come SSL/TLS, che descriveremo nel Capitolo 8. Questa funzionalità è utilizzata per il commercio elettronico, il remote banking e altre applicazioni sicure.

Abbiamo visto come avviene l'acquisizione dei cookie, ma come vengono utilizzati? Il browser, prima di inviare una richiesta di una pagina di qualche sito Web, controlla nella directory dei cookie se sono presenti cookie inseriti dal dominio a cui rivolge la richiesta. In questo caso, tutti i cookie inseriti da tale dominio vengono inclusi nel messaggio di richiesta. Quando il server li riceve, può interpretarli come desidera.

Esaminiamo alcuni possibili utilizzi dei cookie. Nella Figura 7.22 il primo cookie è impostato da *toms-casino.com* ed è utilizzato per identificare il cliente. Quando il cliente esegue il login la settimana successiva per buttare via altri soldi, il browser invia il cookie in modo che il server riconosca l'utente. Armato dell'ID del cliente, il server può ricercare il suo record in un database e utilizzare l'informazione per costruire una pagina Web appropriata. A seconda delle abitudini del cliente, la pagina potrebbe proporre una mano di poker, un elenco delle corse ippiche del giorno o una slot machine.

Il secondo cookie proviene da *.jillss-store.com*. In questo caso lo scenario è un cliente che visita il negozio, cercando gli articoli da acquistare. Quando trova un affare e fa click su di esso, il server aggiunge l'articolo al carrello (sul server) e costruisce un cookie che fa riferimento al carrello e lo spedisce al client. Il cookie viene nuovamente restituito a ogni richiesta di pagina durante la visita al negozio; all'accumularsi di altri acquisti, il server li aggiunge al carrello a cui si riferisce il cookie. Per finire, quando il cliente fa click su *Procedi con il pagamento*, il cookie viene inviato insieme alla richiesta; in questo modo il server sa esattamente che cosa vuole comprare il cliente.

Il terzo cookie è per un portale Web. Quando il cliente fa click su un collegamento del portale, il browser invia il cookie, che comunica al portale di costruire una pagina contenente i prezzi delle azioni di Cisco e Oracle, nonché i risultati dei New York Jets. Dal momento che un cookie può raggiungere una dimensione di 4 KB, c'è molto spazio per preferenze dettagliate relative a titoli di testate, previsioni del tempo, offerte speciali e così via.

Un impiego più controverso dei cookie è quello di tracciare il comportamento on-line degli utenti. Questo permette agli operatori di siti Web di capire come gli utenti navigano nel loro sito; le aziende pubblicitarie costruiscono profili di pubblicità o siti che un particolare utente ha visionato. Il punto focale della controversia è che gli utenti tipicamente non sono consci che la loro attività in quel momento viene tracciata, anche con profili dettagliati e attraverso siti Web apparentemente non correlati. Nonostante ciò, il tracciamento sul Web è un grande affare. DoubleClick, che fornisce e traccia pubblicità, è considerato dalla compagnia di monitoraggio del Web Alexa tra i 100 siti più trafficati al mondo. Google analytics che traccia l'uso del sito per gli operatori è usato da più della metà dei 100.000 siti più trafficati del Web.

Grazie ai cookie è semplice per un server tenere traccia delle attività degli utenti. Si supponga che un server voglia tenere traccia del numero di singoli visitatori e del numero di pagine osservate da ognuno prima di lasciare il sito. Quando giunge la prima richiesta questa non è accompagnata da un cookie, pertanto il server restituisce un cookie contenente *counter* = 1. I click successivi in tale sito restituiranno il cookie al server. Ogni volta il contatore viene incrementato e restituito al client. Tenendo traccia dei contatori, il server può sapere quante persone hanno abbandonato il sito dopo avere visto la prima pagina, quanti hanno guardato due pagine e così via.

Tracciare il comportamento di utenti su più siti è solo leggermente più complicato e funziona nel seguente modo: un'agenzia di pubblicità, per esempio Sneaky Ads, contatta i principali siti Web e pone banner pubblicitari per i prodotti delle aziende partner sulle loro pagine, pagando una piccola quota ai proprietari del sito. Anziché fornire al sito un file GIF da inserire in ogni pagina, comunicano un URL da aggiungere alle pagine. Ogni URL contiene un numero univoco nella parte relativa al file, per esempio

```
http://www.sneaky.com/382674902342.gif
```

Quando un utente visita per la prima volta una pagina *P* contenente tale annuncio, il browser preleva il file HTML e lo analizza, trovando il collegamento al file d'immagine su *www.sneaky.com*, pertanto invia una richiesta per l'immagine. Viene restituito un file GIF insieme a un cookie contenente un ID utente univoco: 4627239101 della Figura 7.22. Sneaky in questo modo registra il fatto che l'utente con questo ID ha visitato la pagina *P*; ciò è facilmente realizzabile perché il file richiesto (*382674902342.gif*) è referenziato solo dalla pagina *P*. Naturalmente, l'annuncio effettivo può apparire su centinaia di pagine, ma ogni volta con un diverso nome di file. Sneaky raccoglie probabilmente un paio di centesimi dal produttore dell'articolo ogni volta che mostra l'annuncio.

In seguito, quando l'utente visita un'altra pagina Web contenente uno degli annunci di Sneaky, il browser preleva il file HTML dal server, vede il collegamento a *http://www.sneaky.com/493654919923.gif* (per esempio) e richiede il file. Visto che possiede già un cookie dal dominio *sneaky.com*, il browser include il cookie di Sneaky contenente l'ID utente. Ora Sneaky conosce una seconda pagina visitata dall'utente.

Nel tempo Sneaky può costruire un profilo completo delle abitudini di esplorazione dell'utente, anche se non ha mai fatto click su uno degli annunci. Naturalmente ancora non possiede il nome dell'utente, ma dispone dell'indirizzo IP che può essere sufficiente per dedurre il nome da altri database. Se l'utente fornisce il suo nome a un qualsiasi sito che coopera con Sneaky diventa disponibile un profilo completo di nome che viene venduto a chiunque voglia acquistarlo. La vendita di queste informazioni può essere abbastanza proficua da consentire a Sneaky di inserire altri annunci su più siti Web e quindi di raccogliere più informazioni.

Se Sneaky volesse diventare supersneaky, l'annuncio non apparirebbe nemmeno come un classico banner. Un "annuncio" costituito da un singolo pixel nel colore di sfondo (e quindi invisibile) ha esattamente lo stesso effetto di un banner pubblicitario: richiede al browser di prelevare l'immagine GIF di 1 × 1 pixel e di inviare tutti i cookie originati dal dominio del pixel.

I cookie sono diventati un punto focale nel dibattito sulla privacy on-line a causa della possibilità di tracciare i comportamenti come descritto sopra. La parte più insidiosa dell'intero business è che molti utenti sono completamente inconsapevoli di questa collezione di informazioni e possono addirittura pensare di essere sicuri, perché non fanno click su nessun annuncio. Per questa ragione i cookie che tracciano gli utenti attraverso i siti sono considerati da molti come **spyware**.[11]
Provate a guardare i cookie già memorizzati dal vostro browser; la maggior parte dei browser visualizza queste informazioni insieme alle preferenze sulla privacy. Potreste sorprendervi nel trovare nomi, indirizzi e-mail o password insieme a identificatori con nomi non interpretabili. Speriamo che non troviate numeri di carta di credito, ma i potenziali abusi sono evidenti.
Per mantenere una parvenza di privacy alcuni utenti configurano i propri browser in modo che rifiutino tutti i cookie. Tuttavia questo modo di procedere causa dei problemi, perché molti siti Web non funzionano bene senza. Alternativamente molti browser permettono agli utenti di bloccare i **cookie provenienti da terze parti**. Sono cookie di siti differenti da quelli della pagina principale che state prendendo; per esempio il cookie *sneaky.com*, usato quando interagite con la pagina su di un sito completamente differente. Il blocco di questi cookie aiuta a prevenire il tracciamento su più siti Web. Possono anche essere installate estensioni del browser per fornire un controllo a grana fine su come i cookie vengono usati (o piuttosto non usati). Poiché il dibattito continua, molte aziende stanno sviluppando politiche di privacy che limitano la condivisione di informazione per prevenirne l'abuso. Naturalmente tali politiche riguardano come le aziende dicono di gestire l'informazione. Per esempio: "Potremmo usare le informazioni raccolte su di voi per condurre le nostre attività commerciali" (che potrebbero consistere nel vendere le informazioni).

Pagine Web statiche

L'idea alla base del Web è trasferire pagine Web dal server al client. Nella loro forma più semplice le pagine Web sono statiche. Sono semplicemente file, situati su alcuni server, che si presentano nello stesso modo ogni volta che vengono prese e visualizzate. Solo perché sono statiche non

significa che le pagine siano inerti quando visualizzate sul browser; una pagina contenente un video può essere una pagina Web statica.

Come detto prima, la lingua franca del Web, in cui sono scritte la maggior parte delle pagine, è HTML. Le home page dei docenti sono usualmente pagine HTML statiche. Le home page delle aziende sono solitamente pagine dinamiche create da un'azienda di progettazione di siti Web. In questo paragrafo descriveremo brevemente le pagine Web statiche come base per la trattazione successiva. I lettori che già conoscono HTML possono saltare il prossimo paragrafo e andare direttamente a quello successivo che descrive i contenuti dinamici e i servizi Web.

HTML – hypertext markup language

Il linguaggio **HTML** (*hypertext markup language*) nacque con il Web. HTML consente agli utenti di produrre pagine Web che contengono testo, grafica, video, puntatori ad altre pagine Web e altro ancora. HTML è un linguaggio di markup, ovvero un linguaggio per descrivere come devono essere formattati i documenti. Il termine "markup" deriva da tempi lontani, quando gli impaginatori contrassegnavano (*marking-up*) i documenti per comunicare allo stampatore quali tipi di caratteri utilizzare e così via. I linguaggi di markup contengono quindi comandi espliciti di formattazione. Per esempio, in HTML, `<b>` indica l'inizio della modalità grassetto, mentre `</b>` indica di terminarne l'utilizzo. LaTeX e TeX sono altri esempi di linguaggio di markup ben noti alla maggior parte degli accademici.

Il vantaggio di un linguaggio di markup rispetto a uno senza markup esplicito è che separa i contenuti dalla forma in cui devono essere presentati. Scrivere un browser risulta facile: il browser, infatti, deve semplicemente comprendere i comandi di markup e applicarli ai contenuti. Incorporare tutti i comandi di markup all'interno di ogni file HTML e standardizzarli, permette a qualsiasi browser Web di leggere e riformattare qualsiasi pagina Web. Questo è fondamentale, perché una pagina potrebbe essere stata prodotta in una finestra 1600 × 1200 pixel con colore a 24 bit su un computer di altro profilo, ma potrebbe dover essere visualizzata in una finestra di 640 × 320 di un telefono cellulare.

Anche se è possibile scrivere documenti HTML con un editor standard, come fanno in molti, si possono utilizzare editor HTML specializzati o elaboratori di testo che svolgono la maggior parte del lavoro (offrendo però un minore controllo sul risultato finale).

Una semplice pagina Web scritta in HTML e la sua visualizzazione in un browser sono mostrati nella Figura 7.23 Una pagina Web consiste di un'intestazione e un corpo, entrambi racchiusi dai **tag** `<html>` e `</html>` (comandi di formattazione), tuttavia molti browser non si lamentano se questi mancano. Come è possibile osservare nella Figura 7.23(a), l'intestazione è racchiusa tra i tag `<head>` e `</head>` mentre il corpo è racchiuso tra i tag `<body>` e `</body>`. Le stringhe all'interno dei tag sono chiamate **direttive.** La maggior parte, ma non tutti, dei tag HTML utilizza questo formato, vale a dire che utilizzano `<qualcosa>` per segnalare l'inizio di qualcosa e `</qualcosa>` per indicarne la fine.

```
<html>
<head> <title> AMALGAMATED WIDGET, INC. </title> </head>
<body> <h1> Benvenuti nella home page di AWI </h1>
<img src="http://www.widget.com/images/logo.gif" ALT="Logo AWI"> <br>
Siamo lieti che abbiate scelto di visitare la home page di <b> Amalgamated
Widget</b>. Speriamo che <i> voi </i> possiate trovare tutte le informazioni necessarie.
<p>Di seguito trovate collegamenti alle informazioni sui nostri prodotti.
Potete ordinarli per via elettronica (WWW), per telefono o via e-mail. </p>
<hr>
<h2> Informazioni sui prodotti </h2>
<ul>
        <li> <a href="http://widget.com/products/big"> Aggeggi grandi </a> <li>
        <li> <a href="http://widget.com/products/little"> Aggeggi piccoli </a> <li>
</ul>
<h2>Contatti</h2>
<ul>
        <li>Telefono: 1-800-WIDGETS</li>
        <li>E-mail: info@amalgamated-widget.com</li>
</ul>
</body>
</html>
```

(a)

(b)

Figura 7.23 (a) Il codice HTML per una pagina Web di esempio. (b) La pagina formattata.

I tag possono essere scritti sia in maiuscolo sia in minuscolo. Di conseguenza, `<head>` e `<HEAD>` indicano la stessa cosa, ma le minuscole sono da preferire per questioni di compatibilità. La disposizione effettiva del documento HTML è irrilevante. Gli analizzatori sintattici HTML ignorano spazi e ritorni a capo extra, perché devono riformattare il testo per adattarlo all'area di visualizzazione corrente, pertanto è possibile aggiungere spazi per rendere i documenti HTML più leggibili (visto che la maggior parte dei documenti è piuttosto carente da questo punto di vista). Come altra conseguenza, non si possono utilizzare righe vuote per separare i paragrafi, perché vengono ignorate; è richiesto un tag esplicito.

Alcuni tag hanno parametri (dotati di nome), chiamati **attributi**. Per esempio, il tag `<img>` nella Figura 7.23 è usato per includere un'immagine allineata al testo. Ha due attributi: *src* e *alt.* Il primo attributo fornisce l'URL dell'immagine. Lo standard HTML non specifica quali formati di immagini sono ammessi. In pratica tutti i browser supportano file GIF e JPEG. I browser sono liberi di supportare altri formati, ma questa estensione è un'arma a doppio taglio. Se un utente usa un browser che supporta per esempio i file TIFF, egli può includerli nelle sue pagine Web per essere poi sorpreso che altri browser ignorino tutta la sua stupenda arte.

Il secondo attributo fornisce un testo alternativo da usare se l'immagine non può essere visualizzata. Per ogni tag lo standard HTML fornisce una lista dei parametri permessi, se ce ne sono, e che cosa significano. Poiché ogni parametro ha un nome, l'ordine nel quale sono specificati i parametri non è significativo.

Tecnicamente, i documenti HTML sono scritti con il set di caratteri ISO 8859-1 Latin-1; tuttavia, per gli utenti le cui tastiere supportano solo ASCII, sono fornite sequenze di escape per caratteri speciali, come "è". L'elenco di caratteri speciali è fornito nello standard; tutti iniziano con una "&" (e commerciale) e terminano con un punto e virgola. Per esempio ` ` produce uno spazio, `è` produce "è" e `é` produce "é". Visto che <, > e & hanno significati speciali, possono essere espressi solo con le loro sequenze di escape, rispettivamente `<`, `>` e `&`.

L'elemento principale dell'intestazione è il titolo, delimitato da `<title>` e `</title>`, ma possono essere presenti anche altri tipi di meta-informazioni, sebbene nessuna sia presente nel nostro esempio. Il titolo vero e proprio non è visualizzato sulla pagina: alcuni browser lo utilizzano per titolare la finestra dove viene visualizzata la pagina.

Tutti i tag utilizzati sono mostrati nella Figura 7.23. Le intestazioni sono generate da un tag `<hn>`, dove *n* è una cifra compresa tra 1 e 6. Di conseguenza, `<hl>` è il titolo più importante, mentre `<h6>` è quello meno importante. Sta al browser rappresentarli in modo appropriato sullo schermo. Generalmente, i titoli con numerazione inferiore sono visualizzati con un carattere più grande e più spesso. Il browser può anche scegliere di utilizzare colori diversi per ogni livello di titolo. Di norma, i titoli `<hl>` sono grandi e in grassetto, con almeno una riga vuota sopra e sotto. I titoli `<h2>` invece sono in un carattere più piccolo con meno spazio sopra e sotto.

I tag `<b>` e `<i>` sono utilizzati rispettivamente per attivare le modalità grassetto e corsivo. Il tag `<hr>` forza una pausa e disegna una linea orizzontale attraverso lo schermo.

Il tag `<p>` inizia un paragrafo. Il browser può visualizzarlo inserendo una linea vuota o qualche indentazione. È interessante notare che `</p>`, che esiste per segnalare la fine di un paragrafo, è spesso omesso dai compositori di HTML pigri.

HTML fornisce diversi meccanismi per creare elenchi, anche nidificati. Gli elenchi, come quelli nella Figura 7.23, iniziano con `<ul>`, mentre `<li>` è utilizzato per indicare l'inizio degli elementi. Una variazione di questo meccanismo è `<ol>`, dedicato agli elenchi ordinati. Il tag `<ul>` apre un elenco non ordinato. I singoli elementi, in un elenco non ordinato, appaiono preceduti da punti elenco (•). Gli elementi di un elenco ordinato sono numerati dal browser.

Per finire dobbiamo parlare dei collegamenti ipertestuali, mostrati nella Figura 7.23 che utilizzano i tag `<a>` (*ancora*, *anchor*) e `</a>`. Il tag `<a>` presenta numerosi parametri, di cui il più importante è *href* che indica l'URL collegato. Il testo tra `<a>` e `</a>` è quello che viene visualizzato. Se selezionato, si visualizza la nuova pagina. E anche permesso collegare altri elementi. Per esempio, un'immagine può essere data tra `<a>` e `</a>` usando il tag `<img>`. In questo caso l'immagine viene visualizzata e facendo click su di essa si attiva il collegamento.

Esistono molti altri tag HTML e attributi non visti in questo semplice esempio. Per esempio, il tag `<a>` può avere il parametro *name* per fare un collegamento a un punto arbitrario della pagina. Queste è utile, per esempio, per le pagine Web che iniziano con una tabella di contenuti su cui è possibile fare click su uno di essi, in modo che l'utente salti alla sezione corrispondente della stessa pagina. Un altro esempio di tag è `<br>`, che forza il browser a interrompere la linea corrente e iniziarne una nuova.

Probabilmente il miglior modo per capire i tag è quello di guardarli in azione. Prendete una pagina Web e guardate il testo HTML nel vostro browser per vedere come è stata realizzata. La maggior parte dei browser ha una voce del menu "visualizza sorgente". Selezionando tale voce viene visualizzato il codice sorgente HTML della pagina invece che l'output formattato.

Abbiamo brevemente descritto i tag esistenti dalla nascita del Web. HTML continua a evolversi. La Figura 7.24 mostra alcune caratteristiche aggiunte nelle versioni successive di HTML. HTML 1.0 si riferisce alla versione usata all'inizio del Web. Le versioni 2.0, 3.0 e 4.0 apparvero in rapida successione nello spazio di pochi anni, quando il Web esplose. Dopo la versione 4.0 trascorsero circa 10 anni prima della standardizzazione della successiva importante versione, HTML 5.0. Poiché è un aggiornamento fondamentale che consolida le modalità con cui i browser gestiscono contenuti complessi, HTML 5.0 è ancora in fase di studio e, nella migliore delle ipotesi, non ci si aspetta di produrre uno standard prima del 2012. A parte gli standard, i browser più importanti già supportano le funzionalità di HTML 5.0.

Elemento	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
Collegamenti ipertestuali	x	x	x	x	x
Immagini	x	x	x	x	x
Liste	x	x	x	x	x
Mappe e immagini attive		x	x	x	x
Form		x	x	x	x
Equazioni			x	x	x

Elemento	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
Barre degli strumenti			x	x	x
Tabelle			x	x	x
Funzioni di accessibilità				x	x
Incorporazione di oggetti				x	x
Fogli di stile				x	x
Scripting				x	x
Video e audio					x
Grafica vettoriale					x
Rappresentazione di XML					x
Elaborazione in background					x
Registrazione dati nel browser					x
Aree per disegnare					x

Figura 7.24 Alcune differenze tra le versioni di HTML.

Lo sviluppo delle nuove versioni di HTML consistette nell'aggiungere nuove caratteristiche desiderate dagli utenti, ma che erano gestite in modi non standard (per esempio, plug-in) finché non divennero uno standard. Per esempio, HTML 1.0 e HTML 2.0 non possedevano le tabelle, aggiunte in HTML 3.0. Una tabella HTML è composta da una o più righe, ognuna composta da una o più celle. Le celle possono contenere una vasta gamma di materiale, compresi testo, figure, immagini e persino altre tabelle. Prima di HTML 3.0 per realizzare una tabella bisognava ricorrere a un espediente, come includere un'immagine contenente una tabella.

In HTML 4.0 sono state aggiunte ulteriori caratteristiche, che comprendono le funzioni di accessibilità per utenti disabili, l'incorporamento di oggetti (una generalizzazione del tag `<img>` che permette di incorporare nelle pagine anche gli oggetti), il supporto per i linguaggi di scripting (per consentire il contenuto dinamico) e altro ancora.

HTML 5.0 include molte caratteristiche per la gestione di contenuti ormai di uso comune sul Web. Video e audio possono essere inclusi nelle pagine ed eseguiti dal browser senza richiedere all'utente di installare un plug-in. La grafica può essere costruita nel browser come grafica vettoriale, piuttosto che usare un formato basato su bitmap (come JPEG e GIF). C'è anche molto più supporto nell'eseguire script nel browser come thread in background per l'elaborazione e l'accesso ai supporti di memorizzazione. Tutte queste caratteristiche aiutano a supportare le pagine Web che sono più vicine ad applicazioni tradizionali con un'interfaccia utente piuttosto che a dei documenti. È questa la tendenza del Web.

Inserimento dati e form

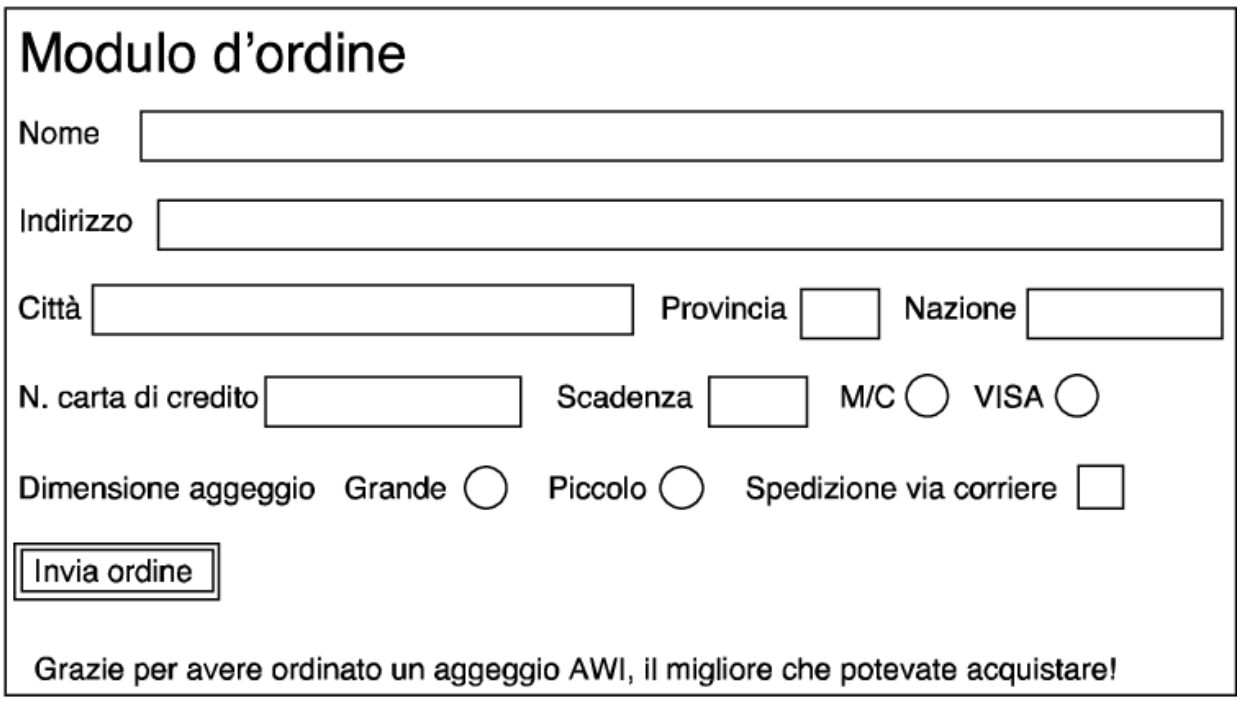
C'è una caratteristica di cui non abbiamo ancora parlato: l'inserimento dei dati (*input*). HTML 1.0 era fondamentalmente unidirezionale. Gli utenti potevano richiamare le pagine dai fornitori di informazioni, ma era difficile inviare informazioni nell'altra direzione. Apparve subito chiaro che c'era la necessità di avere traffico bidirezionale per raccogliere ordini tramite le pagine Web, chiedere ai clienti di compilare on-line le schede di registrazione, permettere l'inserimento di parole chiave nelle ricerche e molto altro.

Inviare input dall'utente al server (via browser) richiede due tipi di supporto. Il primo richiede che HTTP sia capace di trasportare dati in quella direzione, nel prossimo paragrafo descriveremo come. Si usa il metodo *POST*. Il secondo requisito è di poter presentare elementi dell'interfaccia utente che riuniscano e organizzino i dati inseriti Queste richieste portarono all'inclusione dei **form** (*modulo*, *formulario*) a partire da HTML 2.0.

I form contengono caselle e pulsanti che permettono agli utenti di inserire informazioni o compiere scelte che vengono poi inviate al proprietario della pagina. Sono scritti come le altre parti di HTML (Figura 7.25). Si noti che sono statici, non cambiano a seconda dell'utente. I contenuti dinamici, di cui parleremo in seguito, forniscono modi più sofisticati di raccolta dell'input inviando un programma il cui comportamento dipende dall'ambiente del browser.

```
<html>
<head> <title> MODULO D'ORDINE AWI </title> </head>
<body>
<h1> Modulo d'ordine </h1>
<form ACTION="http://widget.com/cgi-bin/order.cgi" method=POST>
<p> Nome <input name="customer" size=46> </p>
<p> Indirizzo <input name="address" size=40> </p>
<p> Città <input name="city" size=20> Provincia <input name= "county" size =4>
Nazione <input name="country" size=10> </p>
<p> N. carta di credito <input name="cardno" size=10>
Scadenza <input name="expires" size=4>
M/C <input name="cc" type=radio value="mastercard">
VISA <input name="cc" type=radio value="visacard"> </p>
<p> Dimensione aggeggio Grande <input name="product" type=radio value="expensive">
Piccolo <input name="product" type=radio value="cheap">
Spedizione via corriere <input name="express" type=checkbox> </p>
<p><input type=submit value="Invia ordine"> </p>
Grazie per avere ordinato un aggeggio AWI, il migliore che potevate acquistare!
</form>
</body>
</html>
```

(a)



(b)

Figura 7.25 (a) Il codice HTML per un modulo d'ordine. (b) La pagina formattata.

Come tutti i moduli, anche quello dell'esempio è racchiuso tra i tag `<form>` e `</form>`. Gli attributi di questo tag indicano che cosa fare con i dati in input, in questo caso usando il metodo *POST* per inviare i dati all'URL specificato. Il testo non racchiuso in un tag è solamente

visualizzato. Nel form possono essere utilizzati tutti i soliti tag (per esempio <b>) per controllarne l'aspetto sullo schermo.

In questo form sono usati tre tipi di caselle di input, ognuno dei quali usa il tag <input>; questo ha una grande varietà di parametri per definire grandezza, natura e uso della casella visualizzata. I form più comuni sono campi vuoti che l'utente può riempire con testo, caselle di controllo e pulsanti *submit* che restituiscono i dati al server.

Il primo tipo è una casella di testo che segue il testo "Nome". La casella è larga 46 caratteri e prevede che l'utente digiti una stringa, che è memorizzata nella variabile *customer*.

La riga successiva del modulo chiede l'indirizzo dell'utente, offrendo 40 caratteri su una singola riga. Viene poi una riga che richiede città, provincia e nazione. Poiché non vengono utilizzati tag <p> tra questi campi, il browser li visualizza tutti su una riga (se ci stanno). Per il browser questo paragrafo contiene sei elementi: tre stringhe alternate a tre caselle. La riga seguente richiede il numero della carta di credito e la data di scadenza. La trasmissione dei numeri di carta di credito su Internet dovrebbe essere fatta solo dopo aver preso adeguate misure di protezione. Alcune sono discusse nel Capitolo 8.

Dopo la data di scadenza appare una nuova funzionalità: i pulsanti di scelta (*radio button*), utilizzati quando occorre compiere una scelta tra due o più alternative. Il modello concettuale è quello di un'autoradio con alcuni pulsanti per la scelta delle stazioni. Il browser visualizza queste caselle in un modulo che permette all'utente di selezionarle e deselezionarle con un click (o utilizzando la tastiera). Selezionando un pulsante vengono disattivati tutti gli altri pulsanti dello stesso gruppo; la presentazione visiva è lasciata al browser. Anche per specificare la dimensione dell'aggeggio da acquistare si utilizzano due pulsanti di scelta; i due gruppi sono distinti dal campo *name,* non dalla posizione o dall'utilizzo di qualcosa del tipo <radiobutton> ... </radiobutton>.

I parametri *value* vengono utilizzati per indicare quale pulsante di opzione è selezionato. A seconda dell'opzione scelta per la carta di credito, la variabile *cc* viene impostata con la stringa "mastercard" o "visacard".

Dopo i due gruppi di pulsanti di scelta arriviamo alle opzioni per la spedizione rappresentate da un pulsante di tipo *checkbox* diversamente dai casi precedenti, in cui solo una delle opzioni poteva essere attiva in ogni dato istante: ogni pulsante di questo tipo può essere attivo o meno indipendentemente da tutti gli altri.

Infine abbiamo il pulsante *Invia ordine*. Quando l'utente fa click sul questo pulsante il browser organizza tutte le informazioni raccolte in una singola e lunga riga di testo e le invia al server dell'URL fornito come parte del tag <form> usando un semplice sistema di codifica. Il carattere "&" viene utilizzato per separare i campi, mentre "+" rappresenta lo spazio. Nel form di esempio la riga potrebbe rispecchiare il contenuto della Figura 7.26. È compito del server dare un senso a questa stringa, probabilmente passando l'informazione a un programma che la elabora; spiegheremo come sia possibile farlo nel prossimo paragrafo.

```
customer=John+Doe&address=100+Main+St.&city=White+Plains&state=NY&country=USA&
cardno=1234567890&expires=6/98&cc=mastercard&product=cheap&express=on
```

Figura 7.26 Una possibile risposta dal browser al server con le informazioni compilate dall'utente.

Vi sono anche altri tipi di input non mostrati in questo semplice esempio, come *password* e *textarea.* Una casella *password* equivale a una casella *text,* tranne per il fatto che i caratteri non vengono visualizzati durante la digitazione. Anche una casella *textarea* equivale a una casella *text*, tranne per il fatto che può contenere più righe.

Per le liste lunghe su cui bisogna fare una scelta, sono forniti i tag `<select>` e `</select>` per dare una lista di alternative. Questa lista viene spesso rappresentata con un menu a tendina. La semantica è quella dei pulsanti di scelta, a meno che non venga fatto uso del parametro *multiple*, nel qual caso la semantica è quella dei check box.

Infine è possibile indicare i valori iniziali che l'utente può modificare. Per esempio, se nella casella *text* viene dato il campo *value* i contenuti sono visualizzati nel modulo in modo che l'utente possa scrivere o cancellare.

CSS – cascading style sheet

Lo scopo originale di HTML era specificare la struttura del documento, non il suo aspetto. Per esempio,

```
<h1> Le foto di Deborah </h1>
```

indica al browser di enfatizzare l'intestazione, ma non dice niente riguardo al tipo di carattere, alla grandezza o al colore. Queste caratteristiche sono lasciate al browser, che conosce le proprietà del monitor (per esempio, quanti pixel ha). Tuttavia alcuni progettisti di pagine Web volevano l'assoluto controllo sull'aspetto delle loro pagine, per cui vennero aggiunti ad HTML nuovi tag per controllare l'aspetto, quali

```
<font face="helvetica" size="24" color="red"> Le foto di Deborah </font>
```

Vennero anche aggiunti metodi per il controllo della posizione sullo schermo in modo accurato. Il problema di questo approccio è che è noioso e produce un HTML gonfiato, che non è portabile: una pagina può adattarsi perfettamente al browser su cui è stata sviluppata, ma potrebbe essere molto confusa in un altro browser o in un'altra versione dello stesso browser o su un video con risoluzione differente.

Un'alternativa migliore è l'uso dei fogli di stile. I fogli di stile negli editor di testo permettono agli autori di associare il testo a uno stile logico, invece che a uno stile fisico: per esempio, "paragrafo iniziale" invece di "testo in corsivo". L'aspetto di ogni stile è definito separatamente. In questo modo, se l'autore decide di cambiare i paragrafi iniziali da un corsivo a 14 punti blu a un grassetto a 18 punti rosa shocking, deve solo cambiare una definizione per riformattare l'intero documento.

CSS (*cascading style sheet*) introdusse i fogli di stile nel Web con HTML 4.0, sebbene non vennero usati comunemente e non vennero supportati dai browser fino al 2000. Il CSS definisce un semplice linguaggio per descrivere le regole che controllano l'aspetto dei contenuti con i tag. Vediamone un esempio. Si supponga che AWI voglia una pagina Web trendy, con un testo di colore blu marino, font Arial su sfondo bianco e i livelli di intestazione con una larghezza aggiuntiva rispettivamente del 100% e del 50% per ogni livello. La definizione CSS nella Figura 7.27 dichiara queste regole.

```
body {background-color:linen; color:navy; font-family:Arial;}
h1 {font-size:200%;}
h2 {font-size:150%;}
```

Figura 7.27 Esempio di CSS.

Come si può vedere, le definizioni di stile possono essere compatte. Ogni linea seleziona un elemento al quale applicare e dare valori delle proprietà; le proprietà di un elemento si applicano di solito a tutti gli altri elementi HTML contenuti. Quindi, lo stile per il corpo definisce lo stile dei paragrafi di testo nel corpo. Esistono anche comode scorciatoie per i nomi dei colori (per esempio, `red`, rosso). Se un parametro di stile non è definito esplicitamente, è definito con i parametri di default del browser. Questa metodologia rende opzionali le definizioni dei fogli di stile; anche senza di essi si ottiene una ragionevole rappresentazione.

I fogli di stile possono essere messi in un file HTML (per esempio usando il tag `<style>`), ma è più comune metterli in un file separato e fare riferimento a esso. Per esempio, il tag `<head>` della pagina AWI può essere modificato in modo da riferirsi al foglio di stile *awistyle.css*, come mostrato nella Figura 7.28. L'esempio illustra anche il tipo MIME del file: *text/css*.

```
<head>
<title> AMALGAMATED WIDGET, INC. </title>
<link rel="stylesheet" type="text/css" href="awistyle.css" />
</head>
```

Figura 7.28 Inclusione di un foglio di stile CSS.

Questa strategia offre due vantaggi. Innanzitutto permette a un insieme di fogli di stile di essere applicati a molte pagine sul sito Web. Questo tipo di organizzazione conferisce un aspetto consistente alle pagine, anche se vengono sviluppate da persone diverse in tempi diversi e permette di cambiare l'aspetto dell'intero sito cambiando il file CSS, invece dell'HTML. Questo metodo può essere paragonato alla direttiva `#include` di un programma in C. Il secondo vantaggio è che i file HTML che vengono scaricati rimangono piccoli. Infatti il browser può scaricare una copia del file CSS per tutte le pagine che fanno riferimento al testo; non è necessario che scarichi una nuova copia delle definizioni per ogni pagina Web.

Documenti Web dinamici e applicazioni Web

Il modello statico che abbiamo appena trattato gestisce le pagine come documenti multimediali collegati tra loro in modo conveniente. Era un modello adatto ai primi tempi del Web, quando una grande quantità di informazioni vennero messe on-line. Attualmente, la maggior parte dell'eccitazione intorno al Web è usarne le applicazioni e i servizi. Per esempio, acquistare prodotti su siti di commercio on-line, ricercare sui cataloghi delle librerie, esplorare mappe, leggere e inviare e-mail e collaborare su documenti.

Queste nuove applicazioni sono simili ai software applicativi tradizionali (per esempio, lettori di e-mail ed elaboratori di testo). Il cambiamento consiste nel fatto che ora queste applicazioni vengono eseguite all'interno del browser, con i dati degli utenti memorizzati sui server in Internet nei data center. Esse usano i protocolli Web per accedere alle informazioni via Internet e il browser per visualizzare un'interfaccia utente. Il vantaggio di tale approccio è che gli utenti non devono installare separatamente programmi applicativi, i dati degli utenti sono accessibili da computer diversi e ne viene fatta una copia di backup dal fornitore del servizio. Questo approccio ha riscosso un tale successo da diventare un rivale dei software applicativi tradizionali. Ovviamente aiuta anche il fatto che tali applicazioni vengano offerte gratis dai grandi provider. Questo modello è la forma prevalente di **cloud computing**, nel quale l'elaborazione si sposta dai computer individuali a cluster di server condivisi in Internet.

Come applicazioni le pagine Web non potevano più essere statiche; era necessario avere contenuti dinamici. Per esempio, una pagina di un catalogo di una libreria dovrebbe mostrare quali libri sono al momento disponibili e quali invece no. In modo analogo, la pagina di uno mercato azionario dovrebbe permettere all'utente di interagire per vedere come cambiano i prezzi delle azioni nei

diversi periodi di tempo ed elaborare profitti e perdite. Come suggerito da questi esempi, i contenuti dinamici possono essere generati da programmi eseguiti sul server o sul browser (o su entrambi).

In questo paragrafo esamineremo entrambi i casi, uno alla volta. La situazione generale è quella mostrata nella Figura 7.29. Si consideri come esempio un servizio di mappe che permette all'utente digitando l'indirizzo di una via di ottenere la mappa stradale corrispondente. Data la richiesta di una località, il server Web deve usare un programma che crei una pagina che mostri la mappa del posto prendendola da un database di informazioni sulle strade e altro, come mostrato nei passi da uno a tre. La domanda (passo 1) causa l'esecuzione di un programma su un server. Il programma consulta un database per generare la pagina appropriata (passo 2) e la restituisce al browser (passo 3).

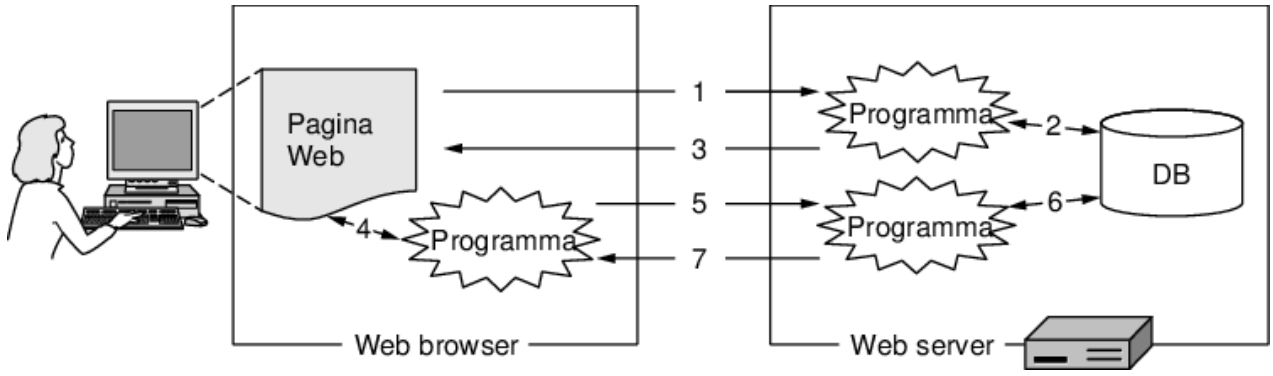


Figura 7.29 Pagine dinamiche.

Tuttavia gli argomenti relativi ai contenuti dinamici non si esauriscono qui. La pagina che viene restituita può contenere a sua volta programmi che vengono eseguiti nel browser. Nel nostro esempio delle mappe, il programma potrebbe permettere all'utente di trovare i percorsi ed esplorare aree vicine a livelli differenti di dettaglio.

Potrebbe aggiornare la pagina, ingrandirla o rimpicciolirla come stabilito dall'utente (passo 4). Per gestire le interazioni il programma potrebbe aver bisogno di ulteriori dati dal server. In questo caso, il programma invia una richiesta al server (passo 5) che recupera ulteriori informazioni dal database (passo 6) e restituisce la risposta (passo 7). Il programma continuerà ad aggiornare la pagina (passo 4). Le richieste e le risposte avvengono in background; l'utente potrebbe persino non accorgersi di esse, perché tipicamente l'URL e il titolo non cambiano. Includendo programmi lato client, la pagina può offrire un'interfaccia più interattiva piuttosto che usando solo programmi lato server.

La generazione di pagine Web dinamiche lato server

Trattiamo ora più in dettaglio il caso della generazione di contenuti lato server. Un caso semplice in cui l'elaborazione lato server è necessaria è l'uso dei form. Si consideri un utente che deve riempire un modulo d'ordine AWI, come mostrato nella Figura 7.25(b), e debba fare click sul pulsante di sottomissione dell'ordine. Quando ciò avviene, la richiesta viene inviata al server all'URL specificato con il form (in questo caso *POST* a *http://widget.com/cgi-bin/order.cgi*) insieme ai contenuti del form, come riempiti dall'utente. Questi dati devono essere dati a un programma per essere elaborati; quindi l'URL identifica il programma che deve essere eseguito e i dati vengono forniti in input a tale programma. In questo caso il processo di elaborazione dovrebbe provocare la sottomissione dell'ordine nel sistema interno di AWI, aggiornare i record del cliente ed effettuare l'addebito sulla carta di credito. La pagina che viene restituita da questa richiesta dipenderà dall'output dell'elaborazione, non è fissa come una pagina statica. Se l'ordine ha avuto successo la pagina restituita dovrebbe mostrare la data ipotetica di consegna. Altrimenti la pagina dovrebbe segnalare che i prodotti richiesti non sono disponibili o che la carta di credito non è valida per qualche ragione.

Il metodo esatto con cui il server esegue un programma, invece che scaricare un file, dipende dalla progettazione del Web server; non è specificato dai protocolli Web perché l'interfaccia potrebbe essere proprietaria e il browser potrebbe non aver bisogno di conoscere i dettagli. Per quanto riguarda il browser è semplicemente una questione di fare una richiesta e prendere una pagina.

Nonostante ciò sono state sviluppate API standard in modo da permettere ai server Web di invocare programmi. L'esistenza di queste API permette agli sviluppatori di aggiungere facilmente ai server delle applicazioni Web. Per dare un senso a quello che possono fare daremo uno sguardo a due API.

La prima API è un metodo per gestire le richieste di pagine dinamiche ed è stato disponibile dall'inizio del Web. È chiamato **CGI** (*common gateway interface*) ed è definito nell'RFC 3875. Fornisce un'interfaccia che consente ai server Web di comunicare con programmi di back-end, che possono accettare l'input (per esempio da form) e generare pagine HTML in risposta. Questi programmi possono essere scritti in qualunque linguaggio conveniente allo sviluppatore, di solito un linguaggio di scripting, perché ne facilita lo sviluppo: Python, Ruby, Perl o quello che preferite.

Per convenzione i programmi invocati da CGI risiedono in una directory chiamata *cgi-bin,* visibile nel'URL. Il server associa una richiesta a questa directory al nome di un programma che esegue come processo separato. Il server fornisce al programma come input qualsiasi dato inviato insieme alla richiesta. L'output del programma produce una pagina Web che viene restituita al browser.

Nel nostro esempio il programma *order.cgi* viene invocato fornendogli in input il contenuto codificato del form, come mostrato nella Figura 7.26. Fa l'analisi dei parametri ed elabora l'ordine. Una convenzione utile è che il programma restituisca l'HTML per il modulo d'ordine qualora non venga fornito alcun input. In questo modo il programma sarà sicuro di conoscere la conformazione del modulo.

La seconda API che trattiamo è molto diversa; l'approccio consiste nell'incorporare piccoli script nelle pagine HTML e nel farli eseguire dal server stesso per generare la pagina. Un celebre linguaggio utilizzato per la scrittura degli script è **PHP** (*hypertext preprocessor*). Per utilizzarlo il

server deve comprendere PHP, proprio come un browser deve comprendere CSS per interpretare le pagine scritte coi fogli di stile. Solitamente i server si aspettano che le pagine Web contenenti PHP abbiano un'estensione *php* anziché *html* o *htm.*

PHP è più semplice di CGI. Come esempio del suo funzionamento rispetto ai form si consideri la Figura 7.30(a), contenente nella parte superiore una normale pagina HTML al cui interno si trova un form. Il tag `<form>` specifica che il file *action.php* deve essere invocato per gestire i parametri dopo che l'utente ha compilato e inviato il modulo. La pagina visualizza due caselle di testo: la prima richiede un nome e la seconda un'età. Dopo la compilazione delle due caselle e l'invio del form il server analizza la stringa restituita (simile a quella della Figura 7.30) inserendo il nome nella variabile *name* e l'età nella variabile *age*; subito dopo avvia l'elaborazione del file *action.php*, mostrato nella Figura 7.30(b). Durante l'elaborazione del file vengono eseguiti i comandi PHP. Se l'utente ha inserito "Barbara" e "32" nelle caselle, il file HTML restituito sarà quello mostrato nella Figura 7.30(c). Di conseguenza, utilizzando PHP la gestione dei form diventa estremamente semplice.

```
<html>
<body>
<form action="action.php" method="post">
<p> Come ti chiami: <input type="text" name="name"> </p>
<p> Quanti anni hai: <input type="text" name="age"> </p>
<input type="submit">
</form>
</body>
</html>
```

(a)

```
<html>
<body>
<h1> Risposta: </h1>
Ciao <?php echo $name; ?>.
Previsione: l'anno prossimo avrai <?php echo $age + 1; ?> anni
</body>
</html>
```

(b)

```
<html>
<body>
<h1> Risposta: </h1>
Ciao Barbara.
Previsione: l'anno prossimo avrai 33 anni
</body>
</html>
```

(c)

Figura 7.30 (a) Una pagina Web contenente un form. (b) Uno script PHP per la gestione dell'output del form. (c) Output dello script PHP quando i valori di input sono rispettivamente "Barbara" e "32".

Anche se PHP è facile da utilizzare, si tratta sempre di un potente linguaggio di programmazione orientato all'interfacciamento tra il Web e un database sul server. Possiede variabili, stringhe, array e la maggior parte delle strutture di controllo presenti in C, ma dispone di funzioni di I/O più potenti di *printf.* PHP è un progetto open source disponibile gratuitamente ed è molto usato. È progettato in modo specifico per funzionare bene con Apache (il software che implementa il server Web più utilizzato al mondo) che è anch'esso open source. Per ulteriori informazioni su PHP si veda (Valade, 2009).

Abbiamo visto due diversi modi per generare pagine HTML dinamiche: gli script CGI e il codice PHP incorporato. Esiste anche una terza tecnica, chiamata **JSP** (*javaserver page*) e simile a PHP, tranne per il fatto che la parte dinamica è scritta nel linguaggio di programmazione Java, anziché in PHP. Le pagine che utilizzano questa tecnica presentano un file con estensione *jsp.* Una quarta tecnica, **ASP.NET** (*active server pages .NET*), è la versione Microsoft di PHP e JavaServer Pages. Per la generazione di contenuto dinamico utilizza programmi scritti facendo uso del framework applicativo *.NET* proprietario di Microsoft. Le pagine che utilizzano questa tecnica hanno l'estensione *aspx.* La scelta tra *PHP*, *JSP* e *ASP.NET* di solito ha a che fare più con la politica (open source contro Microsoft) che con la tecnologia, perché i tre linguaggi sono in un certo senso equivalenti.

La generazione di pagine Web dinamiche lato client

Gli script PHP e CGI risolvono il problema della gestione di moduli e interazioni con i database sul server. Possono accettare le informazioni in ingresso dai form, cercare le informazioni in uno o più database e generare pagine HTML con i risultati. Ciò che non possono fare è rispondere ai movimenti del mouse o interagire direttamente con gli utenti. A questo scopo è necessario disporre di script incorporati nelle pagine HTML, eseguiti sulla macchina client anziché sulla macchina server. Iniziando da HTML 4.0, tali script possono essere utilizzati con il tag `<script>`. L'insieme di tecnologie per la generazione di contenuto dinamico a volte è chiamata **HTML dinamico**.

Il linguaggio di scripting più popolare per il lato client è **JavaScript**, a cui ora daremo un rapido sguardo. Nonostante il nome simile, JavaScript non ha quasi niente a che vedere con il linguaggio di programmazione Java. Come altri linguaggi di scripting è un linguaggio molto ad alto livello. Per esempio, con una sola riga di codice JavaScript è possibile aprire una finestra di dialogo, attendere l'input di testo e archiviare la stringa risultante in una variabile. Le funzioni di alto livello come questa rendono JavaScript ideale per la progettazione di pagine Web interattive. D'altra parte, il fatto che non sia standardizzato e stia mutando più rapidamente di un moscerino della frutta intrappolato in una macchina a raggi X complica la scrittura di programmi JavaScript capaci di funzionare su tutte le piattaforme, ma forse un giorno si stabilizzerà.

Come esempio di programma in JavaScript si consideri la Figura 7.31. Come nel caso della Figura 7.30 mostra un form che richiede un nome e un'età, prevedendo quanti anni avrà la persona in questione l'anno successivo. Il testo è quasi equivalente all'esempio di PHP; la differenza principale sta nella dichiarazione del pulsante di invio e nell'istruzione di assegnazione al suo interno. Questa istruzione di assegnazione comunica al browser di invocare lo script *response* al click di un pulsante e di passargli come parametro il form stesso.

```
<html>
<head>
<script language="javascript" type="text/javascript">
function response(test_form) {
      var person = test_form.name.value;
      var years = eval(test_form.age.value) + 1;
      document.open();
      document.writeln("<html> <body>");
      document.writeln("Ciao " + person + ".<br>");
      document.writeln("Previsione: l'anno prossimo avrai " + years + " anni.");
      document.writeln("</body> </html>");
      document.close();
}
</script>
</head>

<body>
<form>
Come ti chiami <input type="text" name="name">
<p>
Quanti anni hai <input type="text" name="age">
<p>
<input type="button" value="submit" onclic="response(this.form)">
</form>
</body>
</html>
```

Figura 7.31 Uso di JavaScript per l'elaborazione di un form.

La parte completamente nuova riguarda la dichiarazione della funzione JavaScript *response* nell'intestazione del file HTML, un'area normalmente riservata a titoli, colori di sfondo e così via. Questa funzione estrae il valore del campo *name* dal modulo e lo memorizza nella variabile *person* come stringa. Estrae anche il valore del campo *age*, lo converte in un intero utilizzando la funzione *eval*, vi somma 1 e memorizza il risultato in *years*. Apre quindi un documento per l'output, esegue quattro scritture utilizzando il metodo *writeln* e chiude il documento. Il documento è un file HTML, com'è possibile osservare dai tag HTML che contiene. Il browser visualizza quindi il documento sullo schermo.

È molto importante comprendere che anche se PHP e JavaScript sembrano simili, perché entrambi incorporano codice in file HTML, sono elaborati in modo completamente diverso. Nell'esempio relativo a PHP nella Figura 7.30, dopo che l'utente ha fatto click sul pulsante di invio, il browser raccoglie le informazioni in una lunga stringa e la invia al server sotto forma di una richiesta di una

pagina PHP. Il server carica il file PHP ed esegue lo script PHP incorporato per produrre una nuova pagina Web. La pagina viene restituita per la visualizzazione al browser che potrebbe persino non sapere che è stata prodotta da un programma. Questa elaborazione è mostrata nei passaggi da uno a quattro nella Figura 7.32(a).

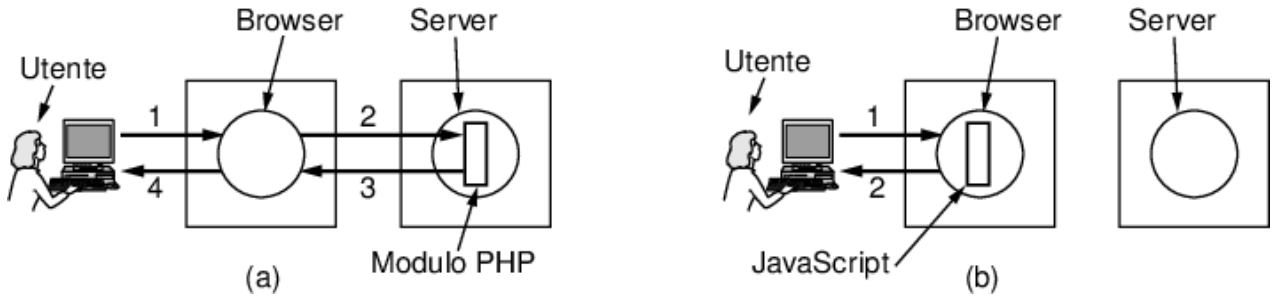


Figura 7.32 (a) Scripting lato server con PHP. (b) Scripting lato client con JavaScript.

Nell'esempio relativo a JavaScript della Figura 7.31, quando l'utente fa click sul pulsante di sottomissione, il browser interpreta una funzione JavaScript contenuta nella pagina. Tutto il lavoro viene svolto in locale, all'interno del browser. Non esistono contatti con il server. Questa elaborazione è mostrata nei passaggi da 1 a 2 nella Figura 7.32(b). Di conseguenza, il risultato è visualizzato quasi immediatamente; al contrario, con PHP può esserci un ritardo di diversi secondi prima che il codice HTML risultante arrivi al client. Questa differenza non significa che JavaScript sia migliore di PHP; i loro utilizzi sono completamente diversi. PHP (e di conseguenza JSP e ASP) è utilizzato quando è necessaria l'interazione con un database remoto; JavaScript (e altri linguaggi lato client, che menzioneremo, come VBScript) viene utilizzato quando l'interazione avviene con l'utente sul computer client. È certamente anche possibile combinarle, come vedremo tra breve.

JavaScript non è l'unico mezzo per rendere le pagine Web altamente interattive. Un'alternativa su piattaforma Windows è **VBScript**, basato su Visual Basic. Un altro metodo popolare prevede l'utilizzo delle **applet**, piccoli programmi Java compilati nel codice macchina per un computer virtuale chiamato **JVM** (*java virtual machine*). Le applet possono essere incorporate nelle pagine HTML (tra `<applet>` e `</applet>`) e interpretate dai browser che includono una JVM. Dal momento che le applet Java sono interpretate e non eseguite direttamente, l'interprete Java può impedire loro di svolgere operazioni nocive. Questo almeno in teoria; in pratica gli autori di applet hanno trovato nelle librerie di I/O di Java un'infinità di bug da sfruttare.

Microsoft ha risposto alle applet Java di Sun permettendo alle pagine Web di contenere **controlli ActiveX**, vale a dire programmi compilati nel linguaggio macchina x86 ed eseguiti direttamente dall'hardware. Questa caratteristica li rende particolarmente veloci e più flessibili delle applet Java interpretate, perché possono svolgere qualsiasi operazione eseguita da un programma. Quando Internet Explorer vede un controllo ActiveX in una pagina lo scarica, ne verifica l'identità e lo esegue. Tuttavia, con il download e l'esecuzione di programmi esterni sorgono problemi di protezione, trattati nel Capitolo 8.

Dal momento che quasi tutti i browser possono interpretare sia i programmi Java sia JavaScript, un Webmaster che desidera produrre una pagina Web molto interattiva può scegliere tra almeno due tecniche; se la portabilità su piattaforme diverse non è importante, può prendere in considerazione anche ActiveX. Come regola generale, i programmi JavaScript sono facili da scrivere, le applet Java vengono eseguite velocemente e i controlli ActiveX sono eseguiti ancora più rapidamente. Inoltre, visto che tutti i browser implementano esattamente la stessa JVM, ma non la stessa versione di JavaScript, le applet Java sono più portabili dei programmi JavaScript. Per ulteriori informazioni su JavaScript esistono molti libri, tutti con un elevato numero di pagine (spesso più di 1000) come Flanagan (2010).

AJAX – asyncronous javascript e XML

Forzare le applicazioni Web richiede interfacce utente reattive e un accesso ai dati memorizzati nei server Web remoti trasparente all'utente. Gli script lato client (per esempio, con JavaScript) e sul lato server (per esempio, con PHP) sono le tecnologie base che forniscono parti della soluzione. Queste tecnologie sono normalmente usate in combinazione con altre e insieme vengono chiamate **AJAX** (*asyncronous javascript and XML*). Molte delle applicazioni Web più complete, come Google Gmail, Maps e Docs sono scritte con AJAX.

AJAX è un po' un ibrido, perché non è un linguaggio: è un insieme di tecnologie che lavorano insieme per produrre applicazioni Web, reattive e potenti come le applicazioni tradizionali su desktop. Le tecnologie sono:

1. HTML e CSS per presentare le informazioni come pagine
2. DOM (*document object model*) per cambiare parti delle pagine mentre sono aperte
3. XML (*estensibile markup language*) per permettere ai programmi di scambiare i dati delle applicazioni con il server
4. Una modalità asincrona per permettere ai programmi di inviare e ricevere dati XML
5. JavaScript come linguaggio che mette insieme tutte queste funzionalità.

Essendo una collezione di tecnologie ne vedremo una alla volta per capire quale contributo apporta. Abbiamo già visto HTML e CSS: sono standard per descrivere i contenuti e come visualizzarli. Qualsiasi programma in grado di produrre HTML e CSS può usare un browser Web come motore per la visualizzazione.

DOM (*document object model*) è una rappresentazione di una pagina HTML accessibile al programma. Questa rappresentazione è strutturata con un albero che riflette la struttura degli elementi HTML. Per esempio, l'albero DOM dell'HTML nella Figura 7.30(a) è dato nella Figura 7.33. La radice è un elemento *html* che rappresenta l'intero blocco HTML. Questo elemento è il padre dell'elemento *body* che a sua volta è il padre dell'elemento *form*. Il form ha due attributi disegnati sulla destra: uno per il metodo del modulo (un POST) e uno per l'azione del form (l'URL da richiedere). Questo elemento ha tre figli: i due tag di paragrafo e i tag di input contenuti nel modulo. In fondo all'albero ci sono le foglie contenenti sia gli elementi sia le costanti, come stringhe di testo. Il significato del modello DOM è di fornire ai programmi un metodo diretto per cambiare parti di una pagina senza dover riscrivere l'intera pagina. Solo il nodo che contiene il cambiamento deve essere sostituito. Quando viene fatto il cambiamento il browser aggiorna la proprio visualizzazione in modo corrispondente. Per esempio, se viene cambiata in DOM un'immagine in una parte della pagina, il browser la aggiornerà senza cambiare nient'altro. Abbiamo già visto DOM in azione nell'esempio relativo a JavaScript nella Figura 7.31, dove erano

state aggiunte linee all'elemento *document* che avevano causato la visualizzazione di nuove linee di testo in fondo alla finestra del browser. DOM è un metodo potente per produrre pagine in grado di evolvere.

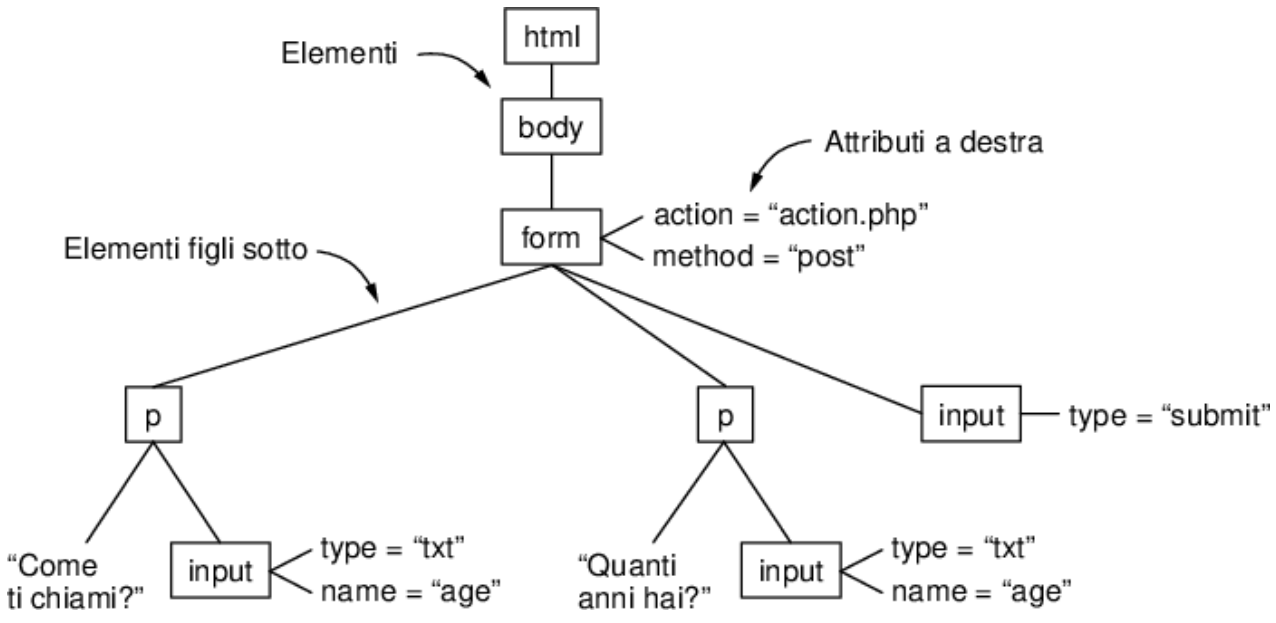


Figura 7.33 L'albero DOM per l'HTML nella **Figura 7.30** (a).

La terza tecnologia, **XML** *(extensible markup language)*, è un linguaggio che descrive del contenuto strutturato. HTML mescola contenuti con formattazione, perché si preoccupa della presentazione delle informazioni. Tuttavia, man mano che le applicazioni Web divengono sempre più comuni, cresce il bisogno di strutturare le pagine Web e separare il contenuto dalla formattazione. Si consideri, per esempio, un programma che ricerca sul Web il prezzo migliore di un libro; deve analizzare molte pagine Web alla ricerca del titolo e del prezzo di tale articolo. Con le pagine Web in HTML è molto difficile per un programma capire dove si trovino il titolo e il prezzo.

Per questo motivo W3C ha sviluppato XML (Bray et al., 2006), che consente la strutturazione delle pagine Web per l'elaborazione automatica. Al contrario di HTML, in XML non ci sono tag predefiniti, ma ogni utente può definire i propri. Si consideri il documento XML di esempio della Figura 7.34. Definisce una struttura chiamata `book_list`, vale a dire una lista di elementi di tipo `book` (*libri*). Ogni libro dispone di tre campi: titolo, autore e anno della pubblicazione; queste strutture sono molto semplici. È possibile utilizzare strutture con campi ripetuti (per esempio più autori), campi facoltativi (per esempio il titolo del CD-ROM incluso) e campi alternativi (per esempio l'URL di una libreria se è disponibile la copia stampata oppure l'URL di un sito di aste se il libro è esaurito).

```
<?xml version="1.0" ?>
<book_list>
<book>
  <title> Human Behavior and the Principle of Least Effort </title>
  <author> George Zipf </author>
  <year> 1949 </year>
</book>
<book>
  <title> The Mathematical Theory of Communication </title>
  <author> Claude E. Shannon </author>
  <author> Warren Weaver </author>
  <year> 1949 </year>
</book>
<book>
  <title> Nineteen Eighty-Four </title>
  <author> George Orwell </author>
  <year> 1949 </year>
</book>
</book_list>
```

Figura 7.34 Un semplice documento XML.

In questo esempio i tre campi sono un'entità indivisibile, ma è comunque possibile dividerli ulteriormente. Per esempio, il campo *author* (*autore*) potrebbe essere rappresentato come segue per consentire migliori funzioni di ricerca e formattazione:

```
<author>
   <first_name> George </first_name>
   <last_name> Zipf </last_name>
</author>
```

Ogni campo può essere diviso in campi secondari, nidificandoli arbitrariamente.

Il file della Figura 7.34 non fa altro che definire un elenco di libri contenente tre volumi. È adatto a trasportare le informazioni tra i programmi eseguiti nei browser e quelli nei server, ma non dice nulla riguardo a come visualizzare il documento come pagina Web. Per fare questo, un programma che consuma le informazioni e pensa che il 1949 sia una buona annata per i libri può produrre dell'HTML in cui i titoli sono evidenziati in corsivo. In alternativa si può usare un linguaggio chiamato **XSLT** *(extensible stylesheet language transformation*), per definire come XML dovrebbe essere trasformato in HTML. È come CSS, ma molto più potente, ma ve ne risparmieremo i dettagli.

Il vantaggio di esprimere i dati in XML invece che in HTML è che è più facile per i programmi analizzarli. HTML veniva originariamente scritto manualmente (e spesso è ancora così) e quindi è poco rigoroso. A volte i tag di chiusura, come `</p>`, sono tralasciati e altri tag non hanno un tag di chiusura associato, come `<br>`. Altri tag possono essere indentati in modo improprio e i nomi dei tag e degli attributi possono essere sia minuscoli che maiuscoli. La maggior parte dei browser fa del proprio meglio per visualizzare quello che probabilmente si voleva. XML è più rigido e più chiaro nelle sue definizioni. I tag e gli attributi sono sempre minuscoli, i tag devono essere sempre chiusi nell'ordine inverso in cui sono stati aperti (oppure è necessario indicare chiaramente se sono tag vuoti senza chiusura corrispondente), i valori degli attributi devono essere racchiusi tra virgolette. Questa precisione rende l'analisi più chiara e non ambigua.

HTML può essere definito in termini di XML. Questo approccio è chiamato **XHTML** *(extended hypertext markup language).* Fondamentalmente è una versione molto pignola di HTML. Le pagine XHTML devono conformarsi strettamente alle regole di XML, altrimenti non vengono accettate dal browser. Non ci saranno più pagine Web mal fatte e inconsistenze tra i browser. L'intento, come in XML, è produrre pagine che programmi (in questo caso applicazioni Web) possono elaborare più comodamente. Benché XHTML sia in circolazione dal 1998, esso si è affermato lentamente. Chi produceva HTML non vedeva la necessità di XHTML e i browser hanno ritardato nel supportarlo. Ora è stato definito HTML 5.0, cosicché una pagina può essere rappresentata sia in HTML sia in XHTML per aiutare la transizione. Prima o poi, XHTML dovrebbe sostituire HTML, ma passerà molto tempo prima che questa transizione sia completa.

XML si è rilevato un linguaggio di comunicazione tra programmi molto popolare. Quando questa comunicazione viene trasportata dal protocollo HTTP, che verrà descritto nel prossimo paragrafo, viene definita un *Web service*. In particolare, **SOAP** *(simple object access protocol)* è un modo di implementare Web service che eseguono RPC tra applicazioni in modo indipendente dal linguaggio e dal sistema. Il client costruisce la richiesta come messaggio XML e la invia al server usando il protocollo HTTP. Il server invia una risposta come messaggio formattato in XML. In questo modo, applicazioni su piattaforme eterogenee possono comunicare.

Tornando ad AJAX, il punto è semplicemente che XML è un formato utile per scambiare dati tra programmi eseguiti nei browser e sui server. Tuttavia, per fornire un'interfaccia reattiva nel browser durante l'invio e la ricezione di dati, deve essere permesso agli script di effettuare un **I/O asincrono**, che non blocchi la visualizzazione mentre si attende la risposta alla richiesta. Si consideri, per esempio, una mappa che l'utente possa fare scorrere nel browser. Quando viene notificata l'azione di scorrimento, lo script della pagina della mappa può richiedere al server ulteriori dati sulla mappa qualora la visualizzazione rischi di andare oltre il bordo attualmente coperto dai dati. L'interfaccia non dovrebbe fermarsi mentre tali dati vengono ricevuti; un'interfaccia che non è in grado di fare ciò non sarebbe certo gradita agli utenti; al contrario, lo

scorrimento dovrebbe continuare in modo continuo. L'arrivo dei dati viene notificato allo script in modo che esso possa usarli. Se tutto funziona, i nuovi dati della mappa verrebbero ricevuti prima del momento in cui sarebbero necessari. I browser moderni devono supportare questo modello di comunicazione.

L'ultima tessera di questo puzzle è il linguaggio di script che tiene insieme AJAX, fornendo accesso all'elenco di tecnologie di cui abbiamo parlato prima. Nella maggior parte dei casi questo linguaggio è Javascript, ma vi sono alternative quali VBScript. Abbiamo presentato prima un semplice esempio di JavaScript, ma non fatevi prendere in giro da questa semplicità: JavaScript ha molti cavilli, ma è un linguaggio di programmazione completo con tutta la potenza del C o di Java. Ha variabili, stringhe, vettori, oggetti, funzioni e tutte le strutture di controllo. Ha anche interfacce specifiche per i browser e le pagine Web. JavaScript può tracciare il movimento del mouse sugli oggetti del video, in modo da semplificare l'apparire istantaneo di un menu e avere pagine Web animate. Può usare DOM per accedere alle pagine, manipolare HTML e XML ed effettuare comunicazioni HTTP asincrone.

Prima di abbandonare l'argomento delle pagine dinamiche riassumiamo brevemente le tecnologie illustrate fin qui, mettendole in relazione tra loro in una singola figura. Le pagine Web complete possono essere generate al volo da vari script sulla macchina server. Gli script possono essere scritti in linguaggi estesi sul server come PHP, JSP e ASP.NET o essere eseguiti come processi CGI separati che possono essere scritti in qualunque linguaggio. Tutte queste opzioni sono mostrate nella Figura 7.35.

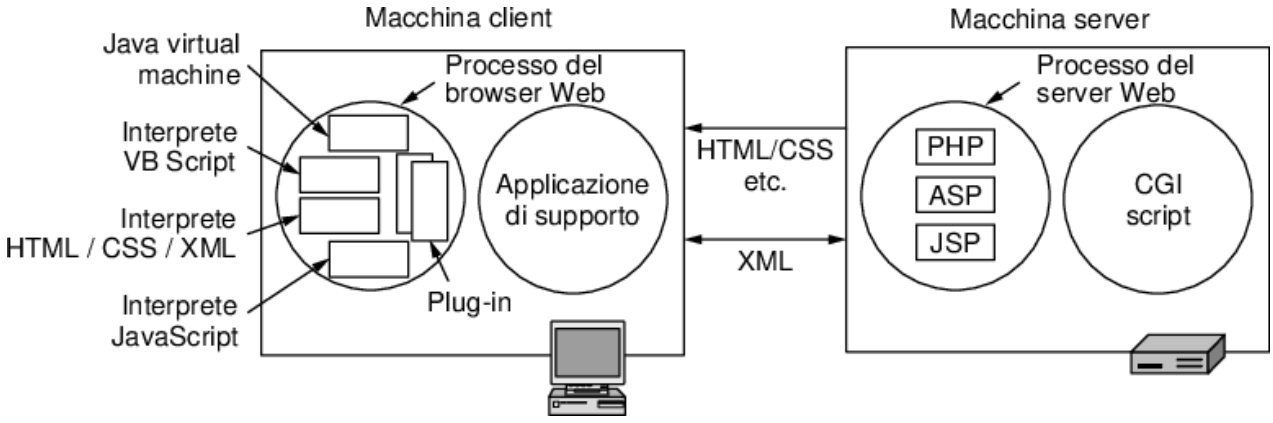


Figura 7.35 Varie tecnologie usate per generare pagine dinamiche.

Quando il browser riceve queste pagine Web le tratta come normali pagine in HTML, CSS e altri tipi MIME e semplicemente le visualizza. I plug-in vengono eseguiti nel browser e le applicazioni eseguite fuori dal browser possono essere installate per estendere i tipi MIME supportati.

La generazione di contenuti dinamici è possibile anche lato client. I programmi contenuti nelle pagine Web possono essere scritti in JavaScript, VBScript, Java e altri linguaggi. Questi programmi possono effettuare elaborazioni arbitrarie e aggiornare il contenuto visualizzato. Con AJAX i programmi nelle pagine Web possono scambiare dati XML o di altro tipo in modo asincrono con il server. Questo modello supporta ricche applicazioni Web che appaiono come applicazioni tradizionali, ma vengono eseguite all'interno del browser e accedono a informazioni memorizzate su server in Internet.

HTTP – hypertext transfer protocol

Illustrati i contenuti Web e le applicazioni è tempo di studiare il protocollo usato per trasportare tutte queste informazioni tra server Web e client: il protocollo **HTTP** (*hypertext transfer protocol*), come specificato nell'RFC 2616.

HTTP è un protocollo semplice basato sul modello di domanda e risposta (*request-response*), normalmente implementato su TCP. Specifica quali messaggi i client possono inviare ai server e quali risposte vengono restituite. Esattamente come in SMTP, le intestazioni delle domande e delle risposte sono formulate in ASCII. I contenuti sono dati in un formato simile a quello MIME, ancora come in SMTP. Questo semplice modello è stato parzialmente responsabile dell'iniziale successo del Web, perché semplificò sviluppo e installazione.

In questo paragrafo ci occuperemo delle proprietà principali di HTTP. Prima di entrare nei dettagli vedremo che il modo in cui è usato in Internet si sta evolvendo. Tuttavia, in un altro senso, HTTP sta diventando sempre più un protocollo di trasporto, che fornisce una possibilità ai processi per comunicare contenuti attraversando i confini di reti differenti. Questi processi potrebbero anche non essere un browser Web o un server Web. Un lettore di contenuti multimediali potrebbe usare HTTP per parlare a un server e richiedere informazioni su un album. Un software antivirus potrebbe usare HTTP per scaricare gli ultimi aggiornamenti. Gli sviluppatori potrebbero impiegare HTTP per il recupero di file di progetti. I prodotti di consumo elettronico come le cornici digitali spesso hanno incorporato un server HTTP, usato come interfaccia verso il mondo esterno. Le comunicazioni tra calcolatori avvengono sempre più con HTTP; per esempio, il server di una compagnia aerea potrebbe usare SOAP per contattare il server di un autonoleggio e prenotare un automobile come parte di un pacchetto vacanze. Questa tendenza, probabilmente, continuerà a crescere insieme all'uso sempre più vasto di HTTP.

Connessioni

Il modo solitamente utilizzato da un browser per contattare un server prevede di stabilire una connessione TCP alla porta 80 sulla macchina server, anche se questa procedura non è formalmente richiesta. Il vantaggio dell'utilizzo di TCP sta nel fatto che né i browser né i server devono preoccuparsi di messaggi lunghi, affidabilità o controllo di congestione. Tutti questi dettagli sono gestiti dall'implementazione di TCP.

All'inizio del Web, con HTTP 1.0, dopo avere stabilito la connessione, veniva inviata una singola richiesta e restituita una singola risposta; a questo punto la connessione TCP veniva rilasciata. In un mondo in cui la tipica pagina Web era composta interamente da testo HTML, questo metodo

era adeguato. Nel giro di pochi anni la pagina Web media iniziò a contenere sempre più collegamenti a contenuti quali icone e altri elementi per migliorarne l'aspetto, ma stabilire una connessione TCP per trasportare una singola icona diventava un modo molto costoso di operare.

Questa osservazione ha portato ad HTTP 1.1, che supporta **connessioni persistenti**. Con questa tecnologia è possibile stabilire una connessione TCP, inviare una richiesta, ottenere una risposta e poi inviare altre richieste e ottenere nuove risposte. Questa strategia è detta **riutilizzo della connessione**. Ammortizzando l'impostazione e il rilascio di TCP su più richieste il corrispondente overhead è inferiore. È inoltre possibile concatenare le richieste, vale a dire inviare la richiesta 2 prima dell'arrivo della risposta alla richiesta 1.

La differenza nelle prestazioni tra questi tre casi è mostrata nella Figura 7.36. La parte (a) mostra tre richieste, una successiva all'altra e ognuna fatta in una connessione separata. Si supponga che questo rappresenti una pagina Web con due immagini incorporate sullo stesso server. Gli URL delle immagini sono determinati quando la pagina principale viene presa, quindi vengono recuperate dopo di essa. Oggigiorno una pagina tipica ha circa 40 altri oggetti che devono essere presi per visualizzarla, ma poiché questo renderebbe la figura troppo pesante, useremo solo due oggetti incorporati.

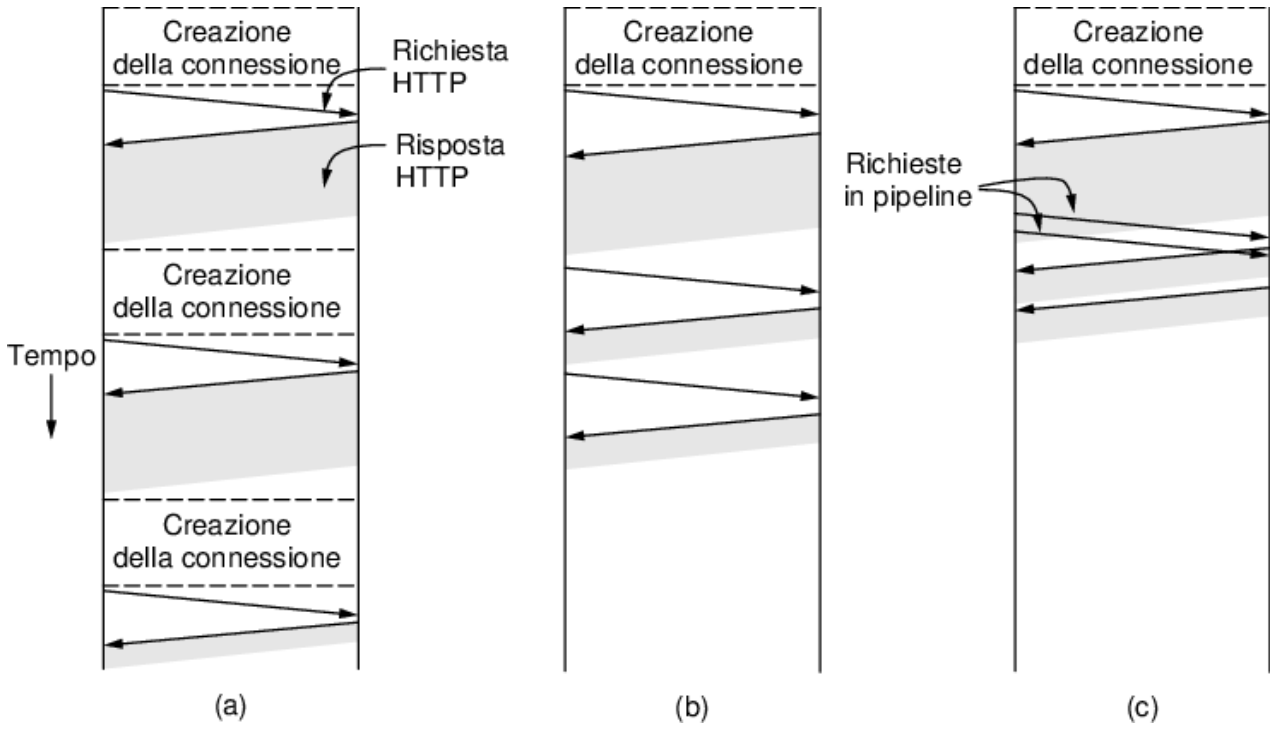


Figura 7.36 HTTP con (a) connessioni multiple e richieste sequenziali. (b) Una connessione persistente e richieste sequenziali. (c) Una connessione persistente e richieste in pipeline.

Nella Figura 7.36(b) la pagina viene presa con una connessione persistente. La connessione TCP viene aperta all'inizio, quindi vengono inviate le stesse tre richieste, una di seguito all'altra come prima, e solo dopo la connessione viene chiusa. Si osservi che il recupero viene completato più

rapidamente per due ragioni. La prima è che non si spreca tempo nel creare connessioni aggiuntive; ogni connessione TCP richiede almeno un tempo di andata e ritorno per essere stabilita. La seconda è che il trasferimento delle stesse immagini procede più velocemente grazie al controllo di congestione di TCP. All'inizio della connessione TCP usa la procedura di slow start per aumentare il throughput, finché non ha appreso il comportamento del percorso nella rete. La conseguenza di questo periodo di riscaldamento è che tante connessioni TCP brevi hanno un tempo di trasferimento sproporzionalmente più lungo di una singola, lunga connessione TCP perché ognuna deve ripetere la procedura di slow start dall'inizio.

Infine nella Figura 7.36(c) c'è una connessione persistente e le richieste sono parallelizzate (in *pipeline*). Più specificatamente, la seconda e la terza richiesta sono inviate in rapida successione non appena sono arrivati abbastanza dati sulla pagina principale da identificare cosicché le immagini possano essere recuperate. Quindi seguono le risposte a queste richieste. Tale metodo riduce drasticamente il tempo in cui il server è inattivo, aumentando le prestazioni.

Tuttavia anche le connessioni persistenti devono pagare un prezzo; un problema che nasce da loro è quando chiudere la connessione. La connessione a un server dovrebbe rimanere aperta mentre la pagina viene caricata. C'è però una buona probabilità che l'utente faccia click su un collegamento che richiede un'altra pagina al server; se la connessione rimane aperta, la richiesta successiva verrà immediatamente inviata. Tuttavia, non vi è alcuna garanzia che il client farà un'ulteriore richiesta al server a breve. In pratica, i client e i server di solito tengono le connessioni persistenti aperte finché sono rimaste inattive per un breve periodo di tempo (per esempio, 60 secondi) o hanno un gran numero di connessioni aperte e necessitano di chiuderne qualcuna.

Il lettore attento potrebbe essersi accorto che abbiamo lasciato fuori una possibile combinazione. È anche possibile inviare una richiesta di connessione TCP, ma eseguire più connessioni TCP in parallelo. Questo metodo di **connessione parallela** era molto usato dai browser prima dell'avvento delle connessioni persistenti. Ha lo stesso svantaggio delle connessioni sequenziali, un carico extra, ma prestazioni molto migliori, perché le connessioni in parallelo nascondono parte della latenza. Nel nostro esempio, le connessioni per entrambe le immagini incorporate potrebbero essere definite nello stesso tempo. Tuttavia, si scoraggia l'esecuzione di più connessioni TCP allo stesso server, perché TCP effettua un controllo di congestione indipendente per ogni connessione. Quindi le connessioni competono tra di loro, causando una perdita aggiuntiva di pacchetti e, aggregate, sono per la rete utenti più aggressive rispetto a una connessione individuale. Le connessioni persistenti sono migliori e preferite alle connessioni parallele, perché evitano il sovraccarico e non soffrono di problemi di congestione.

Metodi

Anche se HTTP fu progettato per l'utilizzo sul Web è stato intenzionalmente reso più generico del necessario con un occhio alle future applicazioni orientate agli oggetti. Per questo motivo sono supportate varie operazioni (**metodi**), al di la della richiesta di una pagina Web; chiamate **metodi**; questa generalità ha permesso la nascita di SOAP.

Ogni richiesta consiste di una o più righe di testo ASCII, con la prima parola della prima riga che rappresenta il nome del metodo richiesto. I metodi standard sono elencati nella Figura 7.37. I nomi sono *case sensitive*, pertanto *GET* è un metodo legale, ma *get* non lo è.

Metodo	Descrizione
GET	Legge una pagina Web
HEAD	Legge l'intestazione di una pagina Web
POST	Aggiunge a una pagina Web
PUT	Memorizza una pagina Web
DELETE	Rimuove la pagina Web
TRACE	Visualizza la richiesta in ingresso
CONNECT	Si collega tramite un proxy
OPTIONS	Richiede le opzioni di una pagina

Figura 7.37 I metodi di richiesta HTTP incorporati.

Il metodo *GET* richiede al server di inviare la pagina, che nel caso più generale è un oggetto, ma in pratica è un file. La pagina viene codificata in MIME. La maggior parte delle richieste ai server Web è di tipo *GET.* La forma usuale di *GET* è

```
GET nomefile HTTP/1.1
```

dove *nomefile* è il nome della pagina da prelevare mentre 1.1 è la versione del protocollo utilizzata.

Il metodo *HEAD* richiede solamente l'intestazione del messaggio, senza la pagina effettiva. Questo metodo può essere utilizzato per ottenere la data dell'ultima modifica di una pagina, per raccogliere informazioni a scopo di indicizzazione o per verificare la validità di un URL.

Il metodo *POST* è usato quando viene sottomesso un form. Insieme a *GET* è usato nei Web service SOAP. Come *GET*, genera un URL, ma invece di limitarsi a caricare la pagina, invia dati al server (per esempio, i contenuti di un modulo o dei parametri RPC). Il server effettua qualche operazione sui dati a seconda dell'URL, concettualmente accodando i dati all'oggetto. L'effetto potrebbe essere, per esempio, quello di acquistare un articolo o richiamare una procedura. Infine, il metodo restituisce una pagina con il risultato.

I metodi rimanenti non sono molto usati per navigare il Web. Il metodo *PUT* è il contrario di *GET*: invece di leggere una pagina, la scrive. Questo metodo permette di fare una raccolta di pagine Web su un server remoto; il corpo della richiesta contiene la pagina e può essere codificato utilizzando MIME. In questo caso, le righe che seguono *PUT* possono contenere intestazioni di autenticazione per provare che il chiamante ha l'autorizzazione per eseguire l'operazione richiesta.

DELETE fa proprio ciò che ci si aspetta: rimuove la pagina o almeno indica che il server Web è d'accordo a rimuovere la pagina. Come con *PUT*, autenticazione e autorizzazioni giocano un ruolo fondamentale.

Il metodo *TRACE* è utilizzato per il debug. Indica al server di mostrare la richiesta. Questo metodo è utile quando le richieste non vengono elaborate correttamente e il client desidera sapere quale richiesta è stata effettivamente ricevuta dal server.

Il metodo *CONNECT* permette all'utente di effettuare una connessione attraverso un dispositivo intermedio, come una Web cache.

Il metodo *OPTIONS* offre al client un modo per interrogare il server su una pagina e ottenere metodi e intestazioni utilizzabili con quella pagina.

Ogni richiesta ottiene una risposta costituita da una riga di stato e magari da altre informazioni (per esempio una pagina Web o parte di essa). La riga di stato contiene un codice di stato di tre cifre che indica se la richiesta è stata soddisfatta e, in caso contrario, la causa del fallimento. Come mostrato nella Figura 7.38 la prima cifra è utilizzata per dividere le risposte in cinque gruppi principali. I codici 1xx sono utilizzati raramente nella pratica. I codici 2xx indicano che la richiesta è stata gestita con successo e che il contenuto viene restituito. I codici 3xx indicano al client di prendere la pagina altrove, sia in un URL differente sia nella propria cache (discussa in seguito). I codici 4xx indicano che la richiesta è fallita a causa di un errore del client, come una richiesta non valida o una pagina inesistente. Infine, gli errori 5xx indicano che il server stesso ha un problema, dovuto a un errore nel proprio codice o a un sovraccarico temporaneo.

Codice	Significato	Esempi
1xx	Informazione	100 = il server accetta di soddisfare la richiesta del client
2xx	Successo	200 = richiesta eseguita con successo; 204 = nessun contenuto presente
3xx	Reindirizzamento	301 = pagina spostata; 304 = pagina nella cache ancora valida
4xx	Errore del client	403 = pagina vietata; 404 = pagina non trovata
5xx	Errore del server	500 = errore interno del server; 503 = riprovare più tardi

Figura 7.38 Gruppi di risposte e corrispondenti codici di stato.

Intestazioni del messaggio

La riga della richiesta (per esempio la riga con il metodo *GET*) può essere seguita da altre righe con ulteriori informazioni. Sono chiamate **intestazioni di richiesta** (*request header*). Queste informazioni si possono assimilare ai parametri della chiamata di una procedura. Anche le risposte

possono avere **intestazioni di risposta** (*response header*). Alcune intestazioni sono utilizzabili in entrambe le direzioni; un riepilogo delle principali è mostrato nella Figura 7.39. Come vedete questa lista è piuttosto lunga; potete quindi ben immaginare che spesso nelle richieste e nelle risposte ci sono molte intestazioni.

Intestazione	**Tipo**	**Contenuto**
User-Agent	Richiesta	Informazioni sul browser e sulla piattaforma
Accept	Richiesta	Il tipo di pagine che il client può gestire
Accept-Charset	Richiesta	I set di caratteri accettabili per il client
Accept-Encoding	Richiesta	Le codifiche di pagina che il client può gestire
Accept-Language	Richiesta	Le lingue che il client può gestire
If-Modified-Since	Richiesta	Data e ora per controllare la validità di una pagina in cache
If-None-Match	Richiesta	Tag per controllare la validità di una pagina in cache
Host	Richiesta	Il nome DNS del server
Authorization	Richiesta	Una lista di credenziali per il client
Referer	Richiesta	Il precedente URL da cui arriva la richiesta
Cookie	Richiesta	Invia al server un cookie impostato in precedenza
Set-Cookie	Risposta	Un cookie che il client deve salvare
Server	Risposta	Informazioni sul server
Content-Encoding	Risposta	Com'è codificato il contenuto (ad esempio *gzip*)
Content-Language	Risposta	La lingua usata nella pagina
Content-Length	Risposta	La lunghezza della pagina in byte
Content-Type	Risposta	Il tipo MIME della pagina
Content-Range	Risposta	Identifica una porzione del contenuto della pagina
Last-Modified	Risposta	Data e ora in cui la pagina è stata modificata
Expires	Risposta	Data e ora in cui la pagina non sarà più valida
Location	Risposta	Indica al client dove inviare la sua richiesta
Accept-Ranges	Risposta	Il server è in grado di accettare richieste di intervalli di byte
Date	Entrambe	Data e ora in cui il messaggio è stato spedito
Range	Entrambe	Identifica una porzione di una pagina
Cache-Control	Entrambe	Direttive su come gestire la cache
ETag	Entrambe	Tag per il contenuto della cache
Upgrade	Entrambe	Il protocollo a cui il mittente vuole passare

Figura 7.39 Alcune intestazioni HTTP.

L'intestazione *User-Agent* è usata dal client per passare al server informazioni relative al proprio browser (per esempio *Mozilla/5.0* o *Chrome/5.0.375.125*). È utile ai server per personalizzare le loro risposte ai browser, in quanto browser diversi hanno funzionalità e comportamenti che possono variare molto l'uno dall'altro.

Le quattro intestazioni *Accept* comunicano al server ciò che il client è in grado di accettare, se dispone di un insieme limitato di elementi accettabili. La prima intestazione specifica i tipi MIME accettati (per esempio *text/html*); la seconda indica il set di caratteri (per esempio *ISO-8859-5* o *Unicode-1-1*). La terza ha a che fare con il metodo di compressione (per esempio *gzip*), mentre la quarta indica una lingua naturale (per esempio lo spagnolo). Se il server ha più pagine tra cui scegliere, può utilizzare questa informazione per fornire ciò che il client sta cercando. Se non è in grado di soddisfare la richiesta viene restituito un codice di errore e la richiesta fallisce.

Le intestazioni *If-Modified-Since* e *If-None-Match* sono usate quando si fa uso di una cache. Permettono al client di richiedere una pagina solo se la copia memorizzata nella cache è meno recente di quella presente sul server.

L'intestazione *Host* denomina il server. È presa dall'URL ed è obbligatoria.[12] Viene utilizzata perché alcuni indirizzi IP possono essere associati a più nomi DNS e il server deve in qualche modo capire quale host deve gestire la richiesta.
L'intestazione *Authorization* è necessaria per le pagine protette. In questa situazione il client può avere l'obbligo di dimostrare che possiede i diritti per vedere la pagina richiesta.

L'intestazione *Referer*[13] viene usata dal client per segnalare l'URL che ha fatto riferimento all'URL richiesto ora. Molto spesso è l'URL della pagina precedente. Questa intestazione è particolarmente utile per tracciare la navigazione Web, perché indica ai server come un client arriva alla pagina. Anche se i cookie sono trattati nell'RFC 2109 anziché nell'RFC 2616 dispongono anch'essi di intestazioni. *Set-Cookie* permette ai server di inviare i cookie ai client. Il client deve salvare il cookie e restituirlo al server nelle richieste successive, usando l'intestazione *Cookie*. (Si noti che c'è una specifica più recente per i cookie con intestazioni più nuove, RFC 2965, ma è stata per lo più respinta dalle aziende e non è molto implementata).

Nelle risposte vengono usate molte altre intestazioni. L'intestazione *Server* consente al server di affermare la sua identità, mentre le cinque intestazioni successive, che iniziano tutte con *Content*, permettono al server di descrivere le proprietà della pagina che sta inviando.

L'intestazione *Last-Modified* comunica quando è stata modificata per l'ultima volta una pagina, mentre *Expires* indica per quanto tempo una pagina rimarrà valida. Entrambe giocano un ruolo importante per la gestione nella cache della pagina.

L'intestazione *Location* è utilizzata dal server per informare il client che dovrebbe provare un URL diverso. Può essere utilizzata se la pagina è stata spostata o per consentire che più URL facciano riferimento alla stessa pagina (possibilmente su server diversi). È utilizzata anche dalle società che

hanno una pagina Web principale nel dominio *.com*, ma che reindirizzano i client verso una pagina nazionale o regionale in base all'indirizzo IP o alla lingua preferita.

Se una pagina è molto grande, un client di capacità limitate potrebbe non volere vederla tutta in una volta. Alcuni server accettano richieste di intervalli di byte, in modo che la pagina possa essere prelevata in più unità piccole. L'intestazione *Accept-Ranges* annuncia la disponibilità del server a gestire questo tipo di richieste di pagine parziali.

Veniamo ora alle intestazioni che possono essere usate in entrambe le direzioni.

L'intestazione *Date* è utilizzabile in entrambe le direzioni e contiene la data e l'ora di invio del messaggio, mentre *Range* indica l'intervallo di byte della pagina che può essere fornito dalla risposta.

L'intestazione *ETag* fornisce una piccola etichetta che serve come nome per il contenuto della pagina ed è usato per la gestione della cache. L'intestazione *Cache-Control* fornisce altre istruzioni esplicite su come mettere (o, più usualmente, come non mettere) le pagine nella cache.

Infine, l'intestazione *Upgrade* viene utilizzata per facilitare la transizione a un nuovo protocollo di comunicazione, come un futuro protocollo HTTP o un protocollo di trasporto sicuro. Consente al client di annunciare che cosa sia in grado di supportare e al server di affermare che cosa stia utilizzando.

Caching

Gli utenti tornano spesso alle pagine Web già viste precedentemente e pagine Web correlate hanno spesso incorporate le stesse risorse. Alcuni esempi sono le immagini usate per la navigazione attraverso un sito, così come fogli di stile comuni e script. Sarebbe un grave spreco dover recuperare tutte queste risorse ogni volta che queste pagine vengono visualizzate, perché il browser ne ha già una copia.

La procedura di memorizzare in una cache delle pagine recuperate per farne un uso successivo viene chiamata **caching**. Il vantaggio è che quando una pagina è memorizzata nella cache, non è necessario ripetere il trasferimento. HTTP ha un supporto integrato che aiuta i client a identificare quando possono riusare le pagine in modo sicuro. Questo supporto aumenta le prestazioni, riducendo il traffico e la latenza della rete. Ora però il browser deve memorizzare le pagine, ma la memorizzazione locale non è costosa. Queste pagine sono usualmente tenute su disco, in modo che siano riutilizzabili quando il browser viene eseguito in un momento successivo.

Il problema difficile da risolvere con il caching HTTP è come determinare che la copia di una pagina precedentemente memorizzata nella cache sia la stessa che verrebbe recuperata nuovamente. Questa decisione non può essere presa solamente sulla base dell'URL. Per esempio, l'URL potrebbe dare una pagina che visualizza le ultime notizie. I contenuti di questa pagina vengono aggiornati frequentemente, anche se l'URL rimane uguale. Alternativamente, i contenuti della pagina potrebbero essere un elenco degli dei della mitologia greca e romana. Questa pagina dovrebbe cambiare un po' meno frequentemente.

Per affrontare questo problema HTTP usa due strategie. Sono mostrate nella Figura 7.40 come forme di elaborazione tra la richiesta (passo 1) e la risposta (passo 5). La prima strategia consiste nella validazione della pagina (passo 2). La cache viene consultata e, se ha una copia della pagina dell'URL richiesto che si sa essere ancora valida, non è necessario prenderne un'altra dal server: viene quindi restituita direttamente la pagina memorizzata nella cache. L'intestazione *Expires* è trasmessa al momento del recupero della pagina nella cache e la data e l'ora correnti vengono usate per prendere questa decisione.

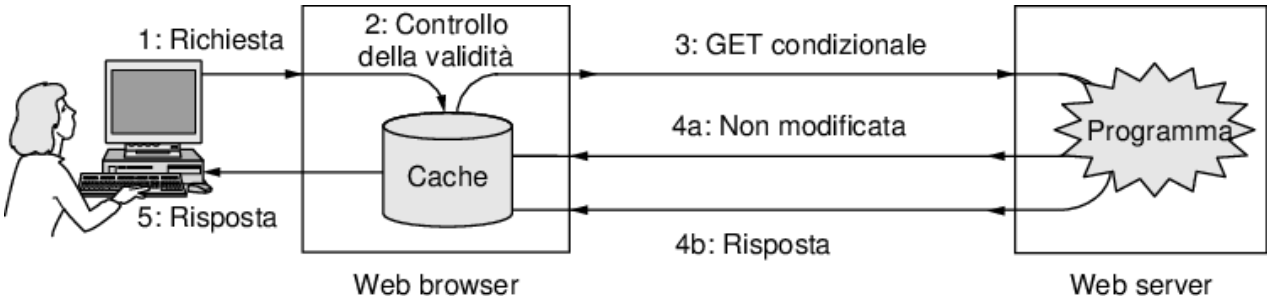


Figura 7.40 Cache HTTP.

Tuttavia, non tutte le pagine arrivano con un'intestazione *Expires* che indichi quando la pagina debba essere nuovamente recuperata. Dopotutto, fare previsioni è molto difficile, specialmente riguardo al futuro. In questo caso, i browser possono usare una euristica. Per esempio, se la pagina non è stata modificata nell'ultimo anno (come dato dall'intestazione *Last-Modified*) potremmo scommettere con abbastanza sicurezza che non cambierà nella prossima ora. Non c'è tuttavia garanzia che questa non sia una pessima scommessa. Per esempio, il mercato azionario potrebbe essere chiuso e la pagina non cambiare per ore, ma cambierà rapidamente non appena il mercato azionario riaprirà. Quindi il caching di una pagina può variare moltissimo nel tempo. Per questa ragione, le euristiche andrebbero usate con attenzione, anche se spesso nella pratica funzionano bene.

Trovare pagine che non siano scadute vuol dire aver fatto un buon uso del caching, perché significa che il server non necessita di essere contattato. Sfortunatamente non sempre succede. I server devono usare l'intestazione *Expires* in modo conservativo, perché non possono essere sicuri di quando una pagina verrà aggiornata; quindi le copie memorizzate nella cache possono essere ancora valide, ma il client non lo sa.

In questo caso viene usata la seconda strategia: si richiede al server se la copia memorizzata nella cache sia ancora valida. Questa richiesta è un **GET condizionale** (*conditional GET*) ed è mostrato nella Figura 7.40 al passo 3. Se il server sa che la copia nella cache è ancora valida, invia una breve replica per dirlo (passo 4a), altrimenti invia la risposta completa (passo 4b).

Per permettere ai server di controllare se una copia nella cache sia ancora valida vengano usati altri campi di intestazione. Il client ottiene l'informazione su quando una pagina nella cache è stata aggiornata dall'intestazione *Last-Modified*. Può inviare questa informazione al server usando l'intestazione *If-Modified-Since* per richiedere la pagina, solo se nel frattempo è cambiata.

Alternativamente il server può restituire insieme alla pagina l'intestazione *ETag*. Questa intestazione fornisce una etichetta che è il nome breve del contenuto della pagina, in modo simile al checksum, ma migliore (potrebbe essere una hash crittografica che verrà descritta nel Capitolo 8). Il client valida la copia nella cache, inviando al server l'intestazione *If-None-Match* che elenca le etichette delle copie nella cache. Se tutte queste etichette corrispondono al contenuto che il server dovrebbe inviare, la copia in cache può essere usata. Si ricorre a questo metodo quando capire la validità di una copia non è conveniente o utile. Per esempio, un server può restituire contenuti diversi dallo stesso URL a seconda di quale linguaggio e quale tipo MIME sono preferiti. In questo caso la semplice data modificata non aiuta il server a capire se la pagina nella cache è valida.

Infine si noti che entrambe le strategie di caching sono annullate dalle direttive portate dall'intestazione *Cache-Control* che possono essere usate per restringerlo (per esempio con *no-cache*) quanto non appropriato. Un esempio è una pagina dinamica che sarà diversa la volta successiva che verrà recuperata. Neanche le pagine che richiedono autorizzazione sono memorizzate in cache.

Ci sarebbe molto da dire sul caching, ma diamo spazio solo a due punti importanti. In primo luogo, il caching può essere effettuato in altri luoghi oltre che nel browser. Nel caso generale, le richieste HTTP possono essere instradate lungo una serie di cache. L'uso di una cache esterna al browser è chiamata **proxy caching**. Ogni livello raggiunto nel caching aiuta a ridurre le richieste lungo la catena. È comune per organizzazioni come ISP e aziende gestire un proxy caching per ottenere il vantaggio di fare caching su più utenti. Discuteremo di questo tema quando tratteremo l'argomento più vasto della distribuzione di contenuti nel Paragrafo 7.5 alla fine di questo capitolo.

In secondo luogo, le cache danno un'importante spinta alle prestazioni, anche se non quanto si sperava. La ragione è che mentre alcuni documenti sono popolari nel Web, ce n'è un gran numero di non popolari che le persone richiedono, molti dei quali sono molto lunghi (per esempio, i video). La "coda lunga" di documenti non popolari occupa spazio nelle cache e il numero di richieste che possono essere gestite da una cache cresce lentamente rispetto alla dimensione della cache stessa. Le Web cache sono in grado di gestire, nella maggior parte dei casi, meno della metà delle richieste. Si veda Breslau et al. (1999) per ulteriori informazioni.

Sperimentare con HTTP

Visto che HTTP è un protocollo ASCII, è abbastanza facile per qualcuno con un terminale (in opposizione a un browser) comunicare direttamente con un server Web. È sufficiente una connessione TCP alla porta 80 del server. Si incoraggiano i lettori a sperimentare questo scenario personalmente. Funzionerà nella maggior parte degli interpreti di comandi UNIX e nella finestra comandi di Windows (una volta abilitato il programma `telnet`).

```
telnet www.ietf.org 80
```

```
GET/rfc.html HTTP/1.1

Host: www.ietf.org
```

La sequenza di comandi avvia una connessione TCP alla porta 80 del server Web di IETF, *www.ietf.org*. Segue il comando *GET* che indica di dare il percorso dell'URL e il protocollo. La riga successiva è l'intestazione *Host* obbligatoria. È richiesta anche una riga vuota successiva all'ultima intestazione per segnalare al server che non vi sono altre intestazioni di richiesta (in pratica, per terminare la richiesta occorre premere due volte consecutive il tasto Invio). Il server invierà la risposta. A seconda del server e dell'URL si possono osservare differenti tipi di intestazioni e pagine.

Web mobile

Il Web è usato da tutti tipi di computer, inclusi i telefoni cellulari. Navigare sul Web da una rete wireless mentre si è in movimento può essere molto utile. Presenta anche problemi tecnici, perché la maggior parte dei contenuti Web venne progettato per presentazioni vistose su computer desktop con connettività a larga banda. In questo paragrafo descriveremo come si stia sviluppando l'accesso al Web da dispositivi mobili o **Web mobile** (*mobile Web*).

Se paragonati ai computer desktop i telefoni cellulari presentano alcune difficoltà nella navigazione sul Web.

1. Il fatto di avere schermi relativamente piccoli preclude la possibilità di visualizzare pagine e immagini di grandi dimensioni.
2. Le limitate capacità di input rendono difficoltosa la scrittura di un URL o altre informazioni lunghe.
3. La banda sui collegamenti wireless è limitata, in maniera particolare sulle reti cellulari (3G), dove è spesso anche costosa.
4. La connettività può essere intermittente.
5. La potenza di elaborazione è limitata per ragioni di durata della batteria, dimensione, dissipazione del calore e costo.

Queste difficoltà significano semplicemente che usare contenuti pensati per i desktop porterà probabilmente a un'esperienza frustrante per gli utenti mobili.

I primi approcci al Web mobile condussero alla progettazione di una nuova pila di protocolli creata apposta per i dispositivi wireless con capacità limitate: **WAP** (*wireless application protocol*) è l'esempio più noto di questa strategia. Lo sforzo per sviluppare WAP fu iniziato nel 1997 dai più importanti produttori di telefoni cellulari come Nokia, Ericsson e Motorola. Tuttavia nel tempo successe qualcosa di inaspettato. Nel successivo decennio banda di rete e capacità dei dispositivi crebbero incredibilmente con la messa in esercizio di servizi dati 3G, di telefoni cellulari con schermi a colori più grandi, di processori più veloci e della possibilità di usare 802.11. Improvvisamente fu possibile per i dispositivi mobili eseguire semplici browser Web. C'è ancora un divario tra questi dispositivi e quelli desktop che non verrà mai colmato, ma molti dei problemi tecnologici che avevano dato spinta allo sviluppo di una pila di protocolli diversa svanirono.

L'approccio che sta prendendo piede è quello di usare gli stessi protocolli Web sia per dispositivi mobili sia per quelli fissi e che i siti Web consegnino contenuti pensati per dispositivi mobili quando l'utente ne sta usando uno. I server Web sono in grado di discriminare se restituire la versione di una pagina per dispositivo mobile o desktop guardando le intestazioni della richiesta. L'intestazione *User-Agent* è utile specialmente in questo caso, perché identifica il browser. Così, quando un server Web riceve la richiesta, può controllare le intestazioni e restituire a un iPhone una pagina con immagini piccole, meno testo e navigazione semplice e una pagina completa a un utente su un laptop.

W3C sta incoraggiando in molti modi questo approccio. Uno è quello di standardizzare le regole riguardanti i contenuti per il Web mobile. Una lista di 60 suggerimenti è fornita nella prima specificazione (Rabin e McCathieNevile, 2008). La maggior parte di queste regole ha fatto fare passi significativi nel ridurre la dimensione delle pagine, quali l'uso della compressione, poiché il costo della comunicazione è maggiore di quello di elaborazione e massimizzazione dell'efficacia del caching. Questo approccio incoraggia i siti, specialmente quelli grandi, a creare versioni dei loro contenuti per il Web mobile, perché è tutto ciò che serve per attirare nuovi utenti.

Un altro strumento utile è una versione ridotta di HTML, chiamata **XHTML Basic**; è un sottoinsieme di XHTML pensato per l'uso su telefoni cellulari, televisioni, PDA, distributori automatici, cercapersone, automobili e persino orologi. Per questo motivo non supporta fogli di stile, script e frame, ma accetta la maggior parte dei tag standard, che vengono raggruppati in 11 moduli, alcuni obbligatori e altri facoltativi, ma tutti definiti in XML. Questi moduli e alcuni tag di esempio sono elencati nella Figura 7.41.

Modulo	**Obbligatorio?**	**Funzione**	**Esempi**
Structure	Sì	Struttura del documento	body, head, html, title
Text	Sì	Informazione	br, code, dfn, em, h*n*, kbd, p, strong
Hypertext	Sì	Collegamenti ipertestuali	a
List	Sì	Elenchi	dl, dt, dd, ol, ul, li
Forms	No	Form	form, input, label, option, textarea
Tables	No	Tabelle rettangolari	caption, table, td, th, tr
Image	No	Immagini	img
Object	No	Applet, mapppe	object, param
Meta-information	No	Informazioni aggiuntive	meta
Link	No	Simile ad <a>	link
Base	No	Punto di partenza per l'URL	base

Figura 7.41 Moduli e tag per XHTML basic.

Tuttavia non tutte le pagine sono progettate per funzionare bene sul Web mobile. Un approccio complementare è l'uso della **trasformazione dei contenuti** o **transcoding**. In questo approccio un computer fa da intermediario tra il dispositivo mobile e il server, prende le richieste dal mobile, recupera i contenuti del server e li trasforma in contenuti per il Web mobile. Una trasformazione semplice consiste nel ridurre la dimensione delle immagini grandi ricodificandole a una risoluzione più bassa. Sono possibili molte altre trasformazioni piccole ma utili. Questa tecnica è stata usata con successo sin dai primi esordi del Web mobile; si veda per esempio Fox et al. (1996). Tuttavia, quando vengono usati entrambi gli approcci si creano delle relazioni difficili tra le decisioni riguardo il contenuto mobile fatte dal server e quelle fatte dal transcoder. Un sito Web potrebbe per esempio selezionare una particolare combinazione di immagini e testo per l'utente sul dispositivo mobile, solo per trovarsi il transcoder che cambia il formato dell'immagine.

Finora la nostra discussione si è concentrata sui contenuti, non sui protocolli, perché i contenuti sono il problema maggiore nella realizzazione del Web mobile. Tuttavia menzioneremo brevemente i problemi riguardanti i protocolli. I protocolli HTTP, TCP e IP usati dal Web possono consumare una quantità significativa di banda sotto forma di overhead di protocollo, come le intestazioni. Per ovviare a questo problema WAP e altre soluzioni definirono protocolli appositi. Ciò si è rivelato assolutamente superfluo. Le tecniche di compressione delle intestazioni, come ROHC (*robust header compression*), descritte nel Capitolo 6, possono ridurre l'overhead di questi protocolli. In tal modo è possibile avere un insieme di protocolli (HTTP, TCP, IP) e usarli su collegamenti sia con alta sia con bassa disponibilità di banda. L'uso su collegamenti con poca banda richiede semplicemente che la compressione delle intestazioni sia attiva.

Ricerca sul Web

Nella nostra descrizione del Web discuteremo quella che è probabilmente la sua applicazione di maggior successo: la ricerca. Nel 1998 Sergey Brin e Larry Page, allora studenti a Stanford, lanciarono una startup chiamata Google per costruire un motore di ricerca sul Web migliore di quelli esistenti. Erano armati dell'idea allora innovativa che un algoritmo di ricerca che contasse il numero di richiami di una pagina da parte di altre fosse in un certo senso più importante di quante volte ricorreva la parola chiave da ricercare. Per esempio, molte pagine si collegano alla pagina principale di Cisco, il che rende questa pagina più importante per un utente che stia ricercando Cisco rispetto a una pagina estranea all'azienda in cui per caso la parola Cisco sia menzionata più volte.

Avevano ragione. Dimostrarono che era possibile costruire un motore di ricerca a cui il pubblico affluì in massa. Sostenuto da venture capital, Google crebbe incredibilmente. Divenne una società per azioni nel 2004 con un capitale di 23 miliardi di dollari. Nel 2010 fu stimato avere più di 1 milione di server sparsi in data center in tutto il mondo.

In un certo senso le ricerche sono semplicemente un'altra applicazione Web, sebbene sia una delle più mature perché in continuo sviluppo fin dagli esordi del Web. Tuttavia la ricerca sul Web si è dimostrata indispensabile nella vita di tutti giorni. Si stima che più di un milione di ricerche venga

effettuato ogni giorno; le persone che ricercano tutti i tipi di informazioni usano Google come punto di partenza. Per esempio, per trovare dove comprare del Vegemite a Seattle, ovviamente non esiste un sito Web da usare come punto di partenza, ma è possibile che un motore di ricerca conosca una pagina con le informazioni desiderate e possa portarvi velocemente alla risposta.

Per effettuare una ricerca sul Web nel modo tradizionale l'utente dirige il proprio browser all'URL di un sito di ricerca Web. I principali sono Google, Yahoo! e Bing. Quindi, l'utente sottopone i termini della ricerca usando un form. Questo fa in modo che il motore di ricerca effettui una richiesta al suo database delle pagine o di immagini rilevanti o di qualunque altro tipo di risorsa che si sta ricercando e restituisca il risultato come pagina dinamica. L'utente quindi segue i link alle pagine che sono state trovate.

La ricerca sul Web è un interessante argomento di discussione, perché ha implicazioni per la progettazione e l'uso delle reti. In primo luogo ci si domanda come le pagine vengano reperite. Un motore di ricerca Web deve avere un database di pagine su cui eseguire la ricerca. Ogni pagina HTML può contenere collegamenti ad altre pagine e qualsiasi cosa interessante (o almeno ricercabile) è segnalata da qualche parte. Questo significa che teoricamente è possibile iniziare con una manciata di pagine e trovare tutte le altre pagine sul Web con una ricerca trasversale di tutte le pagine e collegamenti. Tale processo è chiamato **Web crawling**. Tutti i motori di ricerca sul Web usano i Web crawler.

Un tema legato al crawling è quello del tipo di pagine che si possono trovare. Prendere dei documenti statici e seguirne i collegamenti è facile. Tuttavia molte pagine Web contengono programmi che visualizzano pagine diverse a seconda dell'interazione con l'utente. Un esempio è il catalogo on-line di un negozio che può contenere pagine dinamiche create da un database e presentare prodotti sempre diversi. Come fa un Web crawler a trovare le pagine dinamiche? La risposta, nella maggior parte dei casi, è che non lo fa; questo tipo di contenuto nascosto è chiamato **deep Web** (*Web profondo*) e la ricerca su di esso è un problema aperto che i ricercatori stanno ora affrontando. Si veda per esempio Madhavan et al. (2008). Esistono anche delle convenzioni secondo le quali i siti fanno una pagina (*robots.txt*) che indica ai crawler quali parti del sito debbano essere visitate o meno.

Una seconda considerazione è come elaborare i dati scaricati. Per permettere agli algoritmi di indicizzazione di essere eseguiti su questa massa di dati le pagine devono essere memorizzate da qualche parte. Le stime variano, ma si pensa che i più importanti motori di ricerca abbiano un indice di decine di miliardi di pagine prese dalla parte visibile del Web. La dimensione media di una pagina è stimata essere di 320 KB. Questo significa che una copia scaricata del Web occupa circa 20 petabyte o 2×10^{16} byte di spazio. Benché sia veramente una grande quantità di dati, può essere tranquillamente memorizzata ed elaborata in un data center (Chang et al., 2006). Per esempio, se lo spazio su disco costa 20 dollari al TB allora 2×10^4 TB costano 400.000 dollari, che non è poi tanto per aziende della dimensione di Google, Microsoft e Yahoo!. E, mentre il Web si espande il costo dei dischi è in picchiata, per cui registrare l'intero Web potrebbe continuare a essere possibile per grandi aziende nel prossimo futuro.

Dare un senso a questi dati è un'altra questione. Si può apprezzare come XML aiuti i programmi a estrarre facilmente la struttura dei dati, mentre i formati particolari richiedono più congetture. C'è anche il problema della conversione tra formati e la traduzione tra linguaggi. Ma anche la conoscenza della struttura dei dati è solo una parte del problema, il punto critico è capirne il

significato. Da qui si può trarre più valore e partire dai risultati delle pagine più rilevanti per una ricerca. L'obiettivo finale è quello di rispondere a delle domande del tipo: dove comprare un tostapane economico, ma decente nella nostra città?

Un terzo aspetto della ricerca sul Web è che è arrivata a dare una nomenclatura a livello più alto. Non è necessario ricordare un lungo URL se è altrettanto affidabile (o forse di più) cercare una pagina Web tramite il nome di una persona, assumendo che siate più bravi a ricordare i nomi che non gli URL. Questa strategia ha sempre più successo. Nello stesso modo in cui i nomi DNS relegarono gli indirizzi IP ai computer le ricerche sul Web stanno relegando gli URL ai computer.

Infine, la ricerca sul Web ci mostra qualcosa che ha poco a che fare con la progettazione di rete, ma molto con la crescita dei servizi di Internet: c'è un gran giro d'affari nella pubblicità, il motore economico che ha spinto la crescita della ricerca su Web. Il grande cambiamento rispetto alle inserzioni su carta è la possibilità di fare pubblicità specifica mediata dalla domanda del mercato. Variazioni sul meccanismo delle aste sono usate per associare una ricerca alla pubblicità più efficace (Edelman et al, 2007). Naturalmente questo modello innovativo ha generato nuovi problemi, come le **click fraud** (*frodi di click*) in cui programmi che imitano gli utenti fanno click su pubblicità che generano guadagni illeciti.

7.4 Streaming audio e video

Le applicazioni Web e il Web mobile non sono gli unici sviluppi interessanti dell'uso delle reti. Sono molti a giudicare l'audio e il video il Santo Gral delle reti. Quando viene pronunciata la parola "multimedia" sia i nerd che i commerciali cominciano a sbavare come se fosse un segnale. I primi vedono grandi sfide tecniche per fornire voce su IP e video on demand a ogni computer, i secondi intravedono la possibilità di grandi profitti.

Sebbene l'idea di inviare audio e video su Internet sia esistita almeno dagli anni '70 è circa solo dal 2000 che il traffico di **audio** e **video in tempo reale** è cresciuto a oltranza. Il traffico in tempo reale differisce dal traffico Web, perché per essere utile deve essere gestito a tassi fissati; dopotutto vedere un video che si muove lento con dei sobbalzi non farà certo divertire lo spettatore. Invece il Web può prevedere brevi interruzioni, il caricamento di una pagina può impiegare più o meno tempo, purché entro certi limiti, senza che sia un grande problema.

Questa crescita è stata possibile grazie a due fattori. In primo luogo, i computer sono diventati molto più potenti e sono ora dotati di microfoni e videocamere; sono quindi in grado di avere in input, elaborare e mandare in output dati audio e video con facilità; in secondo luogo, si è resa disponibile una banda enorme su Internet. I collegamenti a lunga distanza su Internet viaggiano a molti gigabit al secondo, mentre sui bordi gli utenti hanno a disposizione banda larga e wireless 802.11. Questi sviluppi permettono agli ISP di trasportare quantità incredibili di traffico attraverso le loro dorsali e agli utenti ordinari di potersi connettere a Internet 100 o 1000 volte più velocemente rispetto al modem telefonico a 56 kbps.

La grande disponibilità di banda sta alla base della crescita del traffico audio e video, per differenti ragioni. Le chiamate telefoniche prendono una banda relativamente piccola (in teoria 64 kbps, ma anche meno se compresse), ma il servizio telefonico è sempre stato costoso. Le aziende videro l'opportunità di trasportare il traffico voce su Internet usando la banda esistente per risparmiare sulle bollette telefoniche. Startup come Skype videro un modo per permettere ai clienti di fare telefonate gratuite usando le connessioni a Internet. Le compagnie telefoniche immaginarono di poter trasportare la voce tradizionale a costi contenuti usando le apparecchiature di rete IP. Il risultato fu un'esplosione del trasporto di dati vocali sulla rete Internet chiamato **voice over IP** (*VoIP*) o **telefonia Internet**.

Diversamente dall'audio i video occupano una grande quantità di banda. I video di qualità ragionevole su Internet sono codificati con una compressione a tassi di circa un Mbps e, con questa codifica, un tipico film in DVD occupa 2 GB di dati. Prima che esistesse l'accesso a Internet su banda larga inviare film sulla rete era proibitivo, ora non più. Con la diffusione della banda larga è diventato per la prima volta possibile che gli utenti guardino un decoroso video in streaming a casa propria: la gente lo adora. Si stima che circa un quarto degli utenti di Internet consulti ogni giorno YouTube, il popolare sito di condivisione video. Il business del noleggio dei film si è spostato on-line e il solo volume dei dati ha trasformato l'intero aspetto del traffico di Internet, la maggior parte del quale è già video e si stima che diventerà il 90% in pochi anni (Cisco, 2010).

Assumendo che ci sia abbastanza banda per trasportare audio e video, il problema principale nel progettare applicazioni di streaming e di teleconferenza è il ritardo di rete. Audio e video richiedono presentazione in tempo reale, il che significa che per essere utili devono essere inviati sulla rete a tassi fissati. Grandi ritardi implicano che le chiamate che dovrebbero essere interattive non lo sono più. Chi ha provato ad avere una conversazione su un telefono satellitare ha ben presente questo problema, perché anche il ritardo fino a mezzo secondo complica la conversazione. Per l'esecuzione di musica o film sulla rete il valore assoluto del ritardo non è importante, perché ha come unico effetto di ritardare il momento di inizio esecuzione. Rimane invece cruciale la variazione sul ritardo, chiamata *jitter*, che deve in qualche modo essere mascherata dall'esecutore, altrimenti l'audio risulterà inintelligibile e il video si vedrà a scatti.

In questo paragrafo discuteremo alcune strategie per gestire il problema del ritardo e i protocolli per instaurare sessioni audio e video. Dopo un'introduzione ad audio e video digitali, la nostra trattazione si dividerà in tre casi per i quali si usano differenti approcci. Il primo caso, e anche più semplice, è la gestione di streaming di contenuti registrati (*stored media*), come vedere un video su YouTube. Il caso successivo, in termini di difficoltà, è lo streaming di contenuti prodotti in tempo reale (*live media*). Radio su Internet e IPTV sono due esempi nei quali radio e televisione vengono inviati in broadcast dalle stazioni a molti utenti su Internet. L'ultimo e più difficile caso è una chiamata come potrebbe essere quella inoltrata usando Skype o più in generale una audio o videoconferenza interattiva.

Per inciso, il termine **multimedia** è spesso usato in Internet per indicare video e audio. Letteralmente multimedia significa semplicemente due o più tipi di contenuti (*media*) diversi; questa definizione rende questo libro una presentazione multimediale, perché di fatto contiene testo ed elementi grafici (le figure). Tuttavia probabilmente voi non avete in mente questo, quindi useremo il termine multimedia per indicare due o più **media continui**[14] (*continuous media*), vale a dire media che devono essere fruiti durante un intervallo di tempo ben definito. I due media sono normalmente video con audio, cioè figure in movimento con un suono. Molti chiamano multimedia anche il puro audio, come la telefonia o la radio su Internet, ma è una definizione non chiara. In effetti, in questo caso un termine migliore sarebbe **media in streaming** (*streaming media*). Ciononostante, noi seguiremo il gregge e considereremo l'audio in tempo reale come multimedia.

Audio digitale

Un'onda audio (sonora) è un'onda acustica (di pressione) monodimensionale. Quando un'onda acustica penetra nell'orecchio il tamburo vibra provocando la successiva vibrazione delle piccole ossa interne all'orecchio, che inviano impulsi nervosi al cervello percepiti come suoni dall'ascoltatore. Analogamente, quando un'onda acustica colpisce un microfono, esso genera un segnale elettrico, che rappresenta l'ampiezza del suono come funzione del tempo.

L'intervallo di frequenze percepite dall'orecchio umano è compreso tra 20 e 20.000 Hz. Alcuni animali, in particolare i cani, possono udire frequenze superiori. L'orecchio "ascolta" in modo logaritmico, pertanto il rapporto tra due suoni con potenze A e B è espresso convenzionalmente in **dB** (*decibel*) come $10\ log_{10}(A/B)$. Se definiamo il limite inferiore di udibilità (una pressione di circa

20 microPascal) per un'onda sinusoidale di 1 kHz come 0 dB, una conversazione ordinaria avviene a circa 50 dB, mentre la soglia del dolore è a circa 120 dB: l'intervallo dinamico[15] presenta un fattore di più di un milione.
L'orecchio è sorprendentemente sensibile alle variazioni del suono, anche se di pochi millisecondi. L'occhio, al contrario, non nota le modifiche del livello di luminosità che avvengono per pochi millisecondi. Il risultato di questa osservazione è che un jitter di pochi millisecondi durante una trasmissione multimediale influisce sulla qualità percepita del suono più di quanto influenzi la qualità percepita dell'immagine.

L'audio digitale è una rappresentazione digitale di un'onda sonora che può essere usata per ricrearlo. Le onde audio possono essere convertite in forma digitale da un **convertitore analogico/digitale** (ADC, *analog-digital converter).* Un ADC riceve in input una tensione elettrica e genera in output un numero. Nella Figura 7.42(a) è mostrato un esempio di onda sinusoidale. Per rappresentare questo segnale in forma digitale è possibile eseguire un campionamento ogni D*T* secondi, come mostrato dalle barre nella Figura 7.42(b). Se un'onda audio non è un'onda sinusoidale pura, ma una sovrapposizione lineare di onde sinusoidali dove la componente di frequenza più alta è *f*, il teorema di Nyquist (si veda il Capitolo 2) afferma che è sufficiente creare campioni alla frequenza di 2*f*. Un campionamento più rapido non ha alcun senso, perché non sono presenti le frequenze, più alte, che questo campionamento potrebbe rilevare.

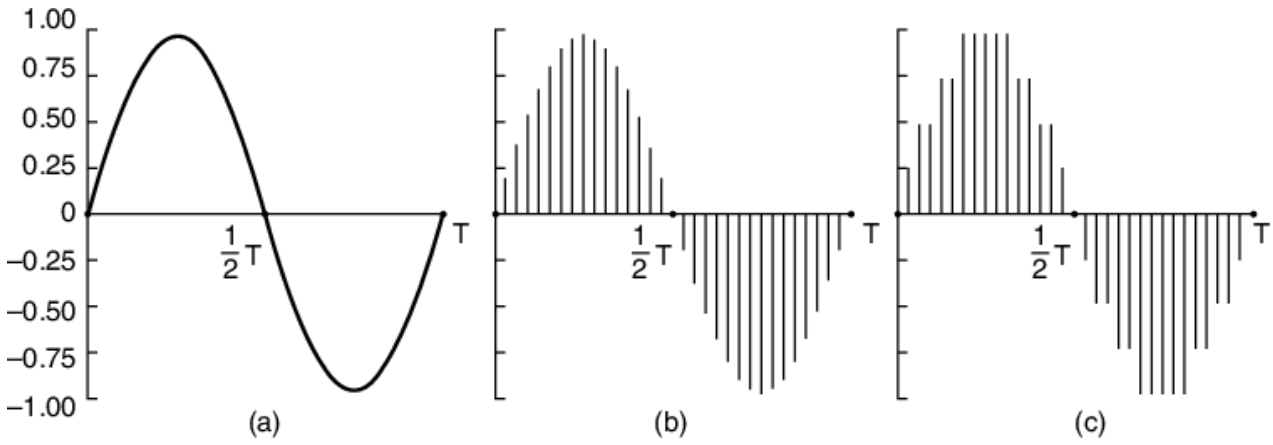


Figura 7.42 (a) Un'onda sinusoidale. (b) Campionamento dell'onda sinusoidale. (c) Quantizzazione dei campioni a 4 bit.

Il processo inverso prende valori digitali e produce una differenza di potenziale elettrica analogica; è fatto da un **convertitore digitale/analogico** (DAC, *digital to analog converter*). Un altoparlante può poi convertire la differenza di potenziale analogica in onde sonore, in modo che si possa sentire il suono. I campioni digitali non sono mai esatti. I campioni della Figura 7.42(c) consentono solo nove valori, da –1,00 a +1,00 con passi di 0,25. Un campione di 8 bit consentirebbe 256 valori distinti. Un campione di 16 bit consentirebbe 65.536 valori distinti. L'errore introdotto dal numero finito di bit per campione è chiamato **rumore di quantizzazione**. Se è troppo grande, l'orecchio è in grado di rilevarlo.

Il telefono e i CD audio sono due esempi ben noti di utilizzo dei suoni campionati. La modulazione a impulsi impiegata dal sistema telefonico utilizza campioni di 8 bit presi 8000 volte al secondo. La scala è non lineare per minimizzare la percezione della distorsione e, con solo 8000 campioni al secondo, le frequenze superiori a 4 kHz vengono perse. In Nord America e in Giappone viene usata la codifica m**-law**, in Europa e internazionalmente la **A-law**; entrambe offrono un tasso dei dati di 64.000 bps.

I CD audio sono digitali, con una frequenza di campionamento pari a 44.100 campioni al secondo, sufficienti per acquisire le frequenze fino a 22.050 Hz: sono sufficienti per le persone, ma non per i cani amanti della musica. I campioni sono di 16 bit ognuno e sono lineari nella gamma delle ampiezze. Occorre notare che i campioni di 16 bit permettono solo 65.536 valori distinti, anche se l'intervallo dinamico dell'orecchio è più di un milione. Di conseguenza, anche se la qualità dell'audio CD è molto migliore di quella dell'audio telefonico, l'utilizzo di soli 16 bit per campione introduce un certo rumore di quantizzazione (anche se l'intero intervallo dinamico non è coperto, i CD non sono pensati per arrivare alla soglia del dolore). Alcuni fanatici dell'audio preferiscono ancora gli LP a 33 giri ai CD, perché non c'è la frequenza di taglio a 22 kHz e non c'è il rumore di quantizzazione (ma si graffiano, a meno che non vengano maneggiati con la massima cura). Con 44.100 campioni al secondo di 16 bit ognuno, un CD audio richiede una banda di 705,6 kbps per l'audio mono e 1,411 Mbps per l'audio in stereo.

Compressione audio

Anche se i tassi di trasmissione dei dati audio sono di molto inferiori a quelli video l'audio viene spesso compresso per ridurre la necessità di banda e il tempo di trasferimento. Tutti i sistemi di compressione richiedono due algoritmi: uno per la compressione dei dati alla sorgente e l'altro per la decompressione alla destinazione. Questi algoritmi sono definiti rispettivamente **algoritmo di codifica** (*encoding*) e **algoritmo di decodifica** (*decoding*). Useremo questa terminologia nella parte restante del capitolo.

Questi algoritmi presentano alcune asimmetrie che è importante comprendere sia per l'audio che per il video. Per molte applicazioni un documento multimediale viene codificato una volta sola, quando viene memorizzato sul server multimediale, ma sarà decodificato migliaia di volte, quando viene visto dai clienti. Questa asimmetria segnala che è accettabile che l'algoritmo di codifica sia lento e richieda hardware costoso, purché l'algoritmo di decodifica sia rapido e non richieda hardware costoso. Dopo tutto l'operatore di un popolare server multimediale può essere disposto a comprare un cluster di computer per codificare la sua intera libreria, ma chiedere ai consumatori di fare la stessa cosa per guardare un video o ascoltare musica non è certo una buona idea. Molti sistemi di compressione usati nella realtà hanno reso la decodifica semplice e veloce, anche al prezzo di una codifica lenta e complessa.

Tuttavia per gli audio e video live, come le chiamate voice over IP, una codifica lenta non è più accettabile. La codifica deve avvenire all'istante, in tempo reale. Di conseguenza, il multimedia in tempo reale utilizza algoritmi o parametri diversi da quelli impiegati per memorizzare i video su disco, spesso con una compressione notevolmente inferiore.

Una seconda asimmetria riguarda il fatto che non è necessario rendere reversibile il processo di codifica/decodifica. Quando si comprime un file, lo si trasmette e lo si decomprime e l'utente si aspetta di ricevere l'originale, accurato fino all'ultimo bit. Con le applicazioni multimediali questo

requisito non esiste. Solitamente è accettabile che il segnale video che è stato codificato e poi decodificato sia leggermente diverso dall'originale, purché il suono sembri uguale. Quando l'output decodificato non è esattamente uguale all'input originale si dice che il sistema è di tipo **lossy** (*con perdite*). Se l'input e l'output sono identici il sistema è **lossless** (*senza perdite*). I sistemi con perdite sono importanti perché accettando una piccola perdita di informazioni si può ottenere un rapporto di compressione elevato.

La banda sui collegamenti a lunga distanza nelle reti telefoniche era molto costosa e a ciò si deve un notevole lavoro svolto sui **vocoder** (*voice coder*, codificatori vocali) che comprimono l'audio nel caso speciale del parlato. La voce umana si colloca tendenzialmente tra i 600 e i 6000 Hz ed è prodotta da un processo meccanico che dipende da estensione vocale, lingua e mascella di chi parla. Alcuni vocoder fanno uso di modelli di sistemi vocali per ridurre il parlato a pochi parametri (per esempio la dimensione e forma delle varie cavità) e a un tasso di trasmissione di circa 2,4 kbps. Tuttavia il loro impiego va al di là degli scopi di questo libro.

Ci concentreremo ora sull'audio inviato su Internet, tipicamente vicino alla qualità dei CD. È anche auspicabile ridurre il tasso di invio per questo tipo di audio. A 1,411 Mbps l'audio stereo impegnerebbe la banda non lasciando spazio per il traffico video e altro traffico Web. Il suo tasso con compressione può essere ridotto di un ordine di grandezza con una conseguente perdita di qualità piccola o non percepibile.

Compressione e decompressione richiedono l'elaborazione del segnale; fortunatamente i suoni e i film digitalizzati possono essere facilmente elaborati da software. Esistono dozzine di programmi che permettono agli utenti di memorizzare, visualizzare, modificare, combinare e archiviare media da più sorgenti. Questo ha generato una grande quantità di musica e film disponibili su Internet, non tutti legali, quindi anche numerose denunce da parte di artisti e proprietari di copyright.

Sono stati sviluppati molti algoritmi di compressione audio. I formati più popolari sono **MP3** (MPEG *audio layer 3*) e **AAC** (*adavanced audio coding*) memorizzati in file di tipo **MP4** (MPEG-4).[16] Per evitare ambiguità si tenga presente che MPEG fornisce compressione a audio e video. MP3 si riferisce alla parte di compressione audio (parte 3) di MPEG-1 standard, non alla terza versione di MPEG che, infatti, non è stata rilasciata. Esistono solo MPEG-1, MPEG-2 e MPEG-4. AAC è il successore di MP3 ed è la codifica audio di default usata in MPEG-4. MPEG-2 permette l'audio sia MP3 che AAC. L'aspetto positivo degli standard è che ce ne sono molti tra cui scegliere e, se non ve ne piace uno, dovete solo aspettare un anno o due.
La compressione audio può essere eseguita in due modi. Nella **codifica per forma d'onda** (*waveform coding*) il segnale viene convertito matematicamente da una trasformata di Fourier nelle sue componenti nel dominio della frequenza. L'esempio della Figura 2.1(a) mostra una funzione nel dominio del tempo e le sue ampiezze di Fourier. L'ampiezza di ogni componente viene poi codificata con la minima quantità di bit; lo scopo è quello di riprodurre accuratamente la forma d'onda all'altro capo con il minor numero di bit possibile.

L'altro metodo, la **codifica percettiva** (*perceptual coding*), sfrutta alcuni limiti del sistema uditivo umano per codificare un segnale in modo tale che a un ascoltatore umano sembri lo stesso, pur apparendo piuttosto diverso su un oscilloscopio. La codifica percettiva è basata sulla scienza della **psicoacustica**, che studia il modo in cui le persone percepiscono i suoni. MP3 e AAC si basano sulla codifica percettiva.

La proprietà fondamentale della codifica percettiva riguarda il fatto che alcuni suoni possono **mascherarne** altri. Si supponga di voler trasmettere un concerto per flauto dal vivo, in un caldo giorno d'estate. Immediatamente una squadra di operai attiva i martelli pneumatici e inizia a rompere la strada. Nessuno potrebbe più udire i flauti: il loro suono verrebbe mascherato dai martelli pneumatici. Ai fini della trasmissione ora è sufficiente codificare solo la banda di frequenza utilizzata dai martelli pneumatici, perché gli ascoltatori non possono comunque udire i flauti. La tecnica è chiamata **mascheramento di frequenza** (*frequency masking*) e indica la capacità di un suono forte in una banda di frequenza di nasconderne uno più tenue in un'altra banda di frequenza, che sarebbe stato udibile in assenza del suono più forte. In effetti, anche dopo che i martelli pneumatici avranno terminato il loro lavoro, i flauti non saranno udibili per un breve periodo di tempo, perché all'avviarsi dei martelli pneumatici l'orecchio deve abbassare il suo guadagno e ciò richiede un tempo finito per riportarlo a valori normali. Tale effetto è definito **mascheramento temporale** (*temporal masking*).

Per rendere gli effetti più quantitativi eseguiamo un primo esperimento. Una persona in una stanza silenziosa indossa le cuffie connesse alla scheda audio del computer. Il computer genera un'onda sinusoidale pura a 100 Hz, inizialmente tenue, ma che aumenta gradualmente di potenza. La persona deve premere un tasto quando riesce a udire il tono, il computer registra il livello di potenza corrente e poi ripete l'esperimento a 200 Hz, 300 Hz e a tutte le altre frequenze, fino al limite dell'udito umano. Dopo avere ripetuto l'esperimento su diverse persone per calcolare una media, il grafico su scala bilogaritmica della potenza necessaria per rendere udibile un tono assume l'aspetto della Figura 7.43(a). Una conseguenza diretta di questa curva è che non è mai necessario codificare le frequenze la cui potenza non ricade all'interno della soglia di udibilità. Per esempio, se la potenza a 100 Hz fosse di 20 dB nella Figura 7.43(a), potrebbe essere omessa dall'output con una perdita di qualità non percettibile, perché 20 dB a 100 Hz è un valore che si trova sotto alla soglia di udibilità.

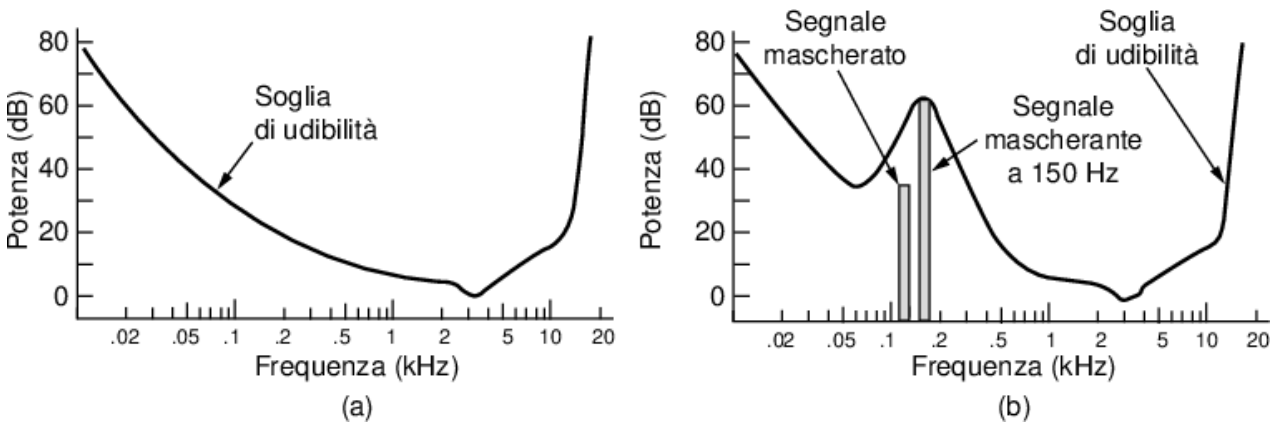


Figura 7.43 (a) La soglia di udibilità in funzione della frequenza. (b) L'effetto del mascheramento.

Vediamo ora un secondo esperimento. Il computer ripete il primo esperimento, questa volta con un'onda sinusoidale costante a 150 Hz, sovrimposta alla frequenza di test. È possibile scoprire che la soglia di udibilità per le frequenze vicine a 150 Hz è aumentata, come mostrato nella Figura 7.43(b).

La conseguenza di questa nuova osservazione è che, tenendo traccia di quali segnali vengono mascherati dai segnali più potenti in bande di frequenza vicine, è possibile risparmiare bit omettendo frequenze nel segnale codificato. Nella Figura 7.43 il segnale di 125 Hz può essere completamente omesso dall'output e nessuno sarà in grado di notare la differenza. Anche dopo che in qualche banda di frequenza viene a cessare un segnale potente, la conoscenza delle sue proprietà di mascheramento temporali consente di continuare a omettere le frequenze mascherate per il tempo di recupero dell'orecchio. L'idea alla base di MP3 e AAC sta nell'eseguire una trasformata di Fourier del suono per ottenere la potenza a ogni frequenza, quindi trasmettere solo le frequenze non mascherate, codificandole con il minor numero di bit possibile.

Con queste informazioni di base possiamo vedere come viene eseguita la codifica. La compressione audio viene svolta campionando la forma d'onda con un tasso da 8 a 96 kHz per AAC, spesso a 44,1 kHz per imitare il suono CD. Il campionamento può essere fatto usando un canale (*mono*) o due (*stereo*). Successivamente viene scelto il numero di bit al secondo. MP3 può comprimere un CD stereo di rock'n'roll fino a 96 kbps con una perdita di qualità accettabile. Per un concerto per pianoforte AAC richiede almeno 128 kbps. La differenza è data dal fatto che il rapporto segnale-rumore del rock'n'roll è (in senso ingegneristico) molto più alto di quello del concerto per pianoforte. E anche possibile scegliere di avere un tasso del segnale codificato minore e accettare una perdita di qualità.

I campioni sono elaborati in piccoli lotti, ognuno dei quali passa attraverso filtri digitali per ottenere le bande di frequenza. L'informazione sulla frequenza è data in input a un modello psichico acustico per determinare le frequenze mascherate. Quindi, i bit disponibili vengono divisi tra le bande, allocando più bit a quelle con la maggior parte dello spettro di potenza non mascherato, meno bit a quelle non mascherate con minore spettro di potenza e nessun bit alle bande mascherate. Infine, i bit vengono codificati secondo la codifica di Huffman, che assegna codici brevi ai numeri che appaiono più frequentemente e lunghi a quelli rari. Il lettore curioso che vuole saperne di più può leggere Brandenburg (1999).

Video digitale

Ora che sappiamo tutto sull'orecchio possiamo passare all'occhio, ma non vi preoccupate, questo paragrafo non sarà seguito da una trattazione sul naso. L'occhio umano ha la proprietà che, quando un'immagine appare sulla retina, viene trattenuta per alcuni millisecondi prima di decadere. Se la sequenza di immagini viene disegnata al tasso di 50 al secondo, l'occhio non si accorge che sta guardando immagini discrete. Tutti i sistemi video sfruttano questo principio per produrre immagini in movimento. La rappresentazione digitale più semplice di un video è una sequenza di *frame*,[17] ognuna consistente in una griglia rettangolare di elementi dell'immagine, chiamati **pixel**. Ogni pixel potrebbe essere un singolo bit per rappresentare bianco o nero. Tuttavia, la qualità di questi sistemi è terribile. Provate a convertire i pixel di un'immagine a colori in un'immagine in bianco e nero (e non in toni di grigio), usando il vostro editor di immagini preferito.

Il successivo passo è usare 8 bit per pixel per rappresentare 256 livelli di grigio. Questo schema conferisce ai video in bianco e nero un'alta qualità. Per i video a colori molti sistemi usano 8 bit per le componenti dei colori primari, rosso, verde e blu (RGB). Questa rappresentazione è possibile, perché ogni colore può essere costruito come una combinazione lineare di rosso, verde e blu con intensità appropriata. Con 24 bit per pixel ci sono circa 16 milioni di colori, più di quanti l'occhio umano possa distinguere. Sui monitor e televisori LCD a colori ogni pixel discreto è fatto di sottopixel poco spaziati rossi, verdi e blu. I frame sono visualizzati definendo l'intensità dei sottopixel e gli occhi mescolano le componenti dei colori.

I frame rate (il tasso, o frequenza, con cui i frame vengono rappresentati) più comuni sono 24 frame al secondo (ereditati dai film a 35mm), 30 frame al secondo (ereditati dalla televisione NTSC degli Stati Uniti) e 25 frame al secondo (ereditati dal sistema televisivo PAL usato in quasi tutto il resto del mondo).[18] A dire il vero la televisione a colori NTSC funziona a 29,97 frame al secondo; il sistema originale in bianco e nero funzionava a 30 frame al secondo, ma quando venne introdotto il colore gli ingegneri avevano bisogno di 1 bit extra per la banda di segnalazione, così ridussero il frame rate a 29,97 (anche se i video NTSC pensati per i computer in realtà ne usano 30), mentre il sistema PAL, inventato dopo NTSC, usa esattamente 25 frame al secondo. Per completare il quadro, un terzo sistema, SECAM, viene usato in Francia, nell'Africa francofona e nell'Europa dell'Est; venne per primo introdotto nell'Europa dell'Est dalla allora comunista Germania Est, in modo da impedire alle persone di vedere la televisione della Germania Ovest (PAL) e farsi venire brutte idee. Tuttavia molti di questi paesi stanno passando al PAL. Il meglio di tecnologia e politica.
Effettivamente, per la televisione broadcast, 25 frame al secondo non sono abbastanza efficaci per avere un movimento fluido; quindi le immagini vengono divise in due **semiquadri** (*field*): uno per le linee dispari e uno per quelle pari. I due campi (con una risoluzione che è la metà di quella di un frame) sono mandati in broadcast in sequenza, per un totale di circa 60 (NTSC) o esattamente 50 (PAL) semiquadri al secondo, un sistema chiamato **scansione interlacciata** (*interlacing*). I video pensati per essere visti su un computer sono **progressivi**, vale a dire non usano la scansione interlacciata; infatti, i monitor dei computer hanno un buffer a disposizione sulle loro schede grafiche, in modo che la CPU possa mettere nuove immagini nel buffer 30 volte al secondo, ma la scheda grafica ridisegna lo schermo 50 o persino 100 volte al secondo per eliminare gli sfarfallii. La televisione analogica non ha un buffer per i frame come i computer; quando un video interlacciato con movimenti rapidi viene visualizzato su un computer, saranno visibili corte linee orizzontali in corrispondenza di contorni netti: questo fenomeno prende il nome di **effetto pettine** (*combing*).

La dimensione dei frame usati per i video inviati su Internet varia molto per la semplice ragione che frame più grandi richiedono più banda, che non sempre è disponibile. I video a bassa risoluzione possono avere 320 × 240 pixel e video pensati per essere visti "a tutto schermo" 640 × 480 pixel; queste dimensioni approssimano rispettivamente quelle dei primi monitor dei computer e della televisione NTSC con un rapporto larghezza-altezza (*aspect ratio*), di 4:3, che è lo stesso della televisione standard. I video **HDTV** (*high definition television*, TV ad alta definizione) possono essere scaricati con 1280 × 720 pixel; queste immagini hanno un aspect ratio di 16:9 per avvicinarsi maggiormente a quello dei film. Per fare un paragone, i video DVD standard sono usualmente di 720 × 480 pixel e i Blu-ray disc sono usualmente HDTV a 1080 × 720 pixel.

Su Internet il numero di pixel è solo una parte della storia, perché i riproduttori multimediali possono rappresentare la stessa immagine a dimensioni diverse. Un video è semplicemente un'altra finestra sullo schermo di un computer che può allargarsi o rimpicciolirsi. Il motivo per avere più pixel è la crescita della qualità dell'immagine in modo che non sembri sfuocata quando viene espansa. Tuttavia, molti monitor possono mostrare immagini e quindi video anche con più pixel dell'HDTV.

Compressione video

Dovrebbe risultare ovvio dalla nostra discussione sui video digitali che la compressione è fondamentale per inviare video su Internet. Anche un video di qualità standard con frame di 640 × 480 pixel, 24 bit per pixel per il colore e 30 frame al secondo richiede circa 200 Mbps. Questo eccede di molto la banda delle connessioni Internet della maggior parte delle aziende, per non parlare degli utenti domestici, e tutto questo per un singolo flusso video. Poiché trasmettere video non compressi è assolutamente fuori questione, almeno sulle WAN, l'unica speranza è una compressione significativa. Fortunatamente, negli ultimi decenni molte ricerche sono state fatte su questo argomento e hanno portato a varie tecniche e algoritmi di compressione che permettono la trasmissione video.

Per il video su Internet vengono usati molti formati, alcuni proprietari e altri standard. La codifica più popolare è MPEG nelle sue varie forme; è un formato pubblico e standard che si può trovare nei file con estensioni mpg e mp4, così come in altri tipi di contenitori. In questo paragrafo studieremo MPEG per capire come viene effettuata la compressione video. Innanzitutto vedremo la compressione delle immagini fisse con JPEG. Un video è semplicemente una sequenza di immagini più il suono. Un modo per comprimere un video è codificare ogni immagine in successione. In prima approssimazione, MPEG è semplicemente la codifica JPEG di ogni frame, con aggiunte alcune caratteristiche extra per rimuovere la ridondanza tra i frame.

Lo standard JPEG

Lo standard **JPEG** (*joint photographic experts group*) per la compressione di immagini statiche a toni continui (come le fotografie) è stato sviluppato da esperti di fotografia sotto gli auspici di ITU, ISO e IEC (altro gruppo per la standardizzazione). È molto usato (provate a guardare i file con estensione jpg) e spesso fornisce un rapporto di compressione di 10:1 o anche migliore nelle immagini non create da un computer.

JPEG è definito nello standard internazionale 10918. In realtà è più simile a una lista della spesa che a un singolo algoritmo; presenta quattro modalità ma, per i nostri scopi, è rilevante solo la modalità sequenziale con perdite. Inoltre, ci concentreremo sul modo in cui JPEG viene normalmente utilizzato per codificare le immagini video RGB a 24 bit e tralasceremo alcuni dettagli e opzioni per semplicità.

L'algoritmo è illustrato nella Figura 7.44. Il primo passo è la preparazione dei blocchi. Supponiamo che l'input JPEG sia un'immagine RGB 640 × 480 con 24 bit per pixel, come mostrato nella Figura 7.44(a). RGB non è il migliore modello cromatico da usare per la compressione. Gli occhi sono molto più sensibili alla **luminanza** o luminosità che alla **crominanza** o colore dei segnali video. Quindi, calcoleremo prima la luminanza, Y, e poi le due crominanze, Cb e Cr dalle componenti R, G e B. Le seguenti formule sono calcolate per valori di 8 bit che variano da 0 a 255:

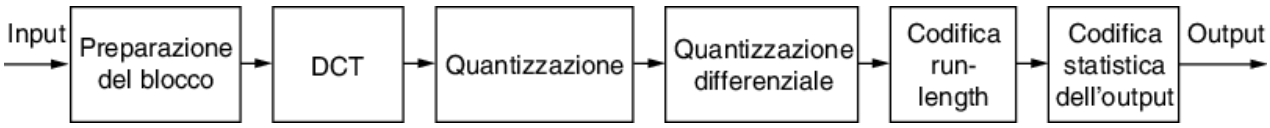


Figura 7.44 Passi della codifica sequenziale JPEG con perdita di dati.

1. Y = 16 + 0,26R + 0,50G + 0,09B
2. Cb = 128 + 0,15R – 0,29G – 0,44B
3. Cr = 128 + 0,44R – 0,37G + 0,07B

Per *Y, Cb* e *Cr* vengono costruite matrici separate. Successivamente viene calcolata la media per blocchi quadrati di 4 pixel nelle matrici *Cb* e *Cr*, al fine di ridurle a 320 × 240. Questa riduzione è con perdite, ma l'occhio lo nota poco, perché apprezza la luminanza più della crominanza. Ciononostante il totale dei dati viene compresso di un fattore pari a due. Ora viene sottratto 128 da ogni elemento di tutte le tre matrici per centrare l'intervallo dei valori sullo 0. Infine ogni matrice è divisa in blocchi di 8 × 8. La matrice *Y* ha 4800 blocchi e le altre due contengono 1200 blocchi ognuna, come appare nella Figura 7.45(b).

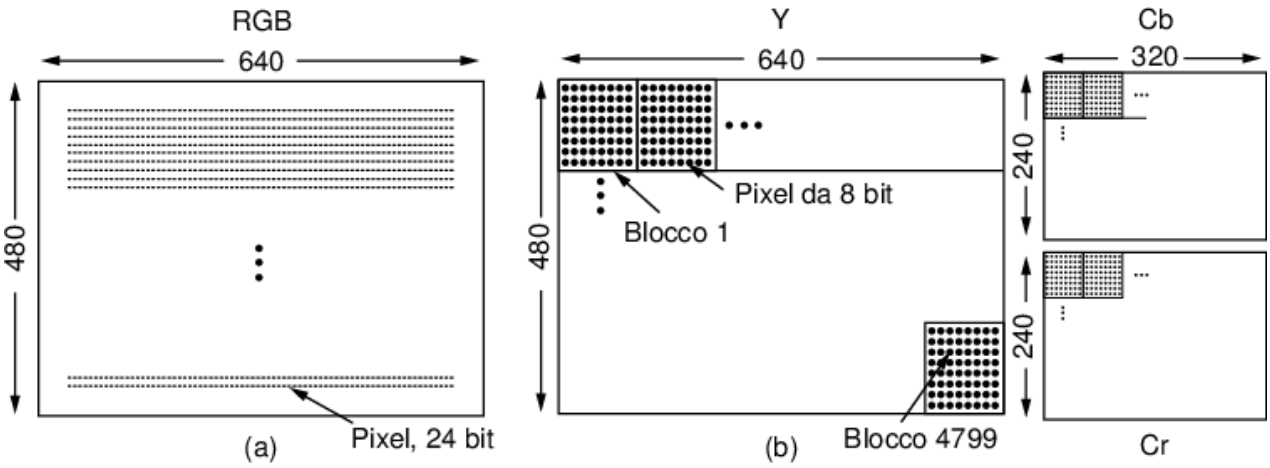


Figura 7.45 (a) Dati in ingresso RGB. (b) Dopo la preparazione del blocco.

La seconda fase di JPEG consiste nell'applicare una **DCT** (*discrete cosine transformation*, trasformata coseno discreta) separatamente a ognuno dei 7200 blocchi. L'output di ogni DCT è una matrice di 8 × 8 coefficienti DCT, dove l'elemento DCT (0, 0) è il valore medio del blocco e gli altri elementi indicano la quantità di potenza spettrale presente per ogni frequenza spaziale. Normalmente questi elementi diminuiscono rapidamente all'aumento della distanza dall'origine (0, 0), come suggerito dalla Figura 7.46.

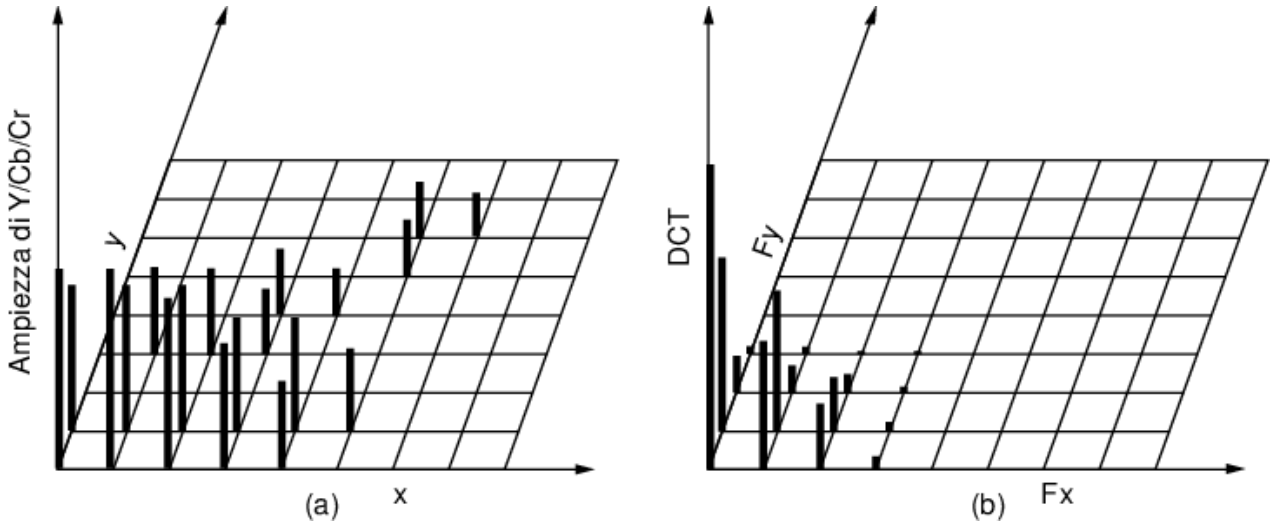


Figura 7.46 (a) Un blocco della matrice *Y*. (b) I coefficienti DCT.

Una volta completata la DCT, JPEG passa alla fase 3, chiamata **quantizzazione**, in cui vengono eliminati i coefficienti DCT meno importanti. Questa trasformazione con perdite viene eseguita dividendo ogni coefficiente nella matrice DCT di 8 × 8 per un peso estratto da una tabella. Se tutti i pesi corrispondono a 1, la trasformazione non ha effetti. Tuttavia, se i pesi aumentano in modo netto a partire dall'origine, le frequenze spaziali alte vengono eliminate rapidamente.

Un esempio di questo passaggio è mostrato nella Figura 7.47. Qui vediamo la matrice DCT iniziale, la tabella di quantizzazione e il risultato ottenuto dividendo ogni elemento DCT per il corrispondente elemento della tabella di quantizzazione. I valori nella tabella di quantizzazione non fanno parte dello standard **JPEG**; ogni applicazione deve fornire i propri e ciò le dà la possibilità di trovare il giusto compromesso tra compressione e perdita di informazione.

Coefficienti DCT

150	80	40	14	4	2	1	0
92	75	36	10	6	1	0	0
52	38	26	8	7	4	0	0
12	8	6	4	2	1	0	0
4	3	2	0	0	0	0	0
2	2	1	1	0	0	0	0
1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Tabella di quantizzazione

1	1	2	4	8	16	32	64
1	1	2	4	8	16	32	64
2	2	2	4	8	16	32	64
4	4	4	4	8	16	32	64
8	8	8	8	8	16	32	64
16	16	16	16	16	16	32	64
32	32	32	32	32	32	32	64
64	64	64	64	64	64	64	64

Coefficienti quantizzati

150	80	20	4	1	0	0	0
92	75	18	3	1	0	0	0
26	19	13	2	1	0	0	0
3	2	2	1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Figura 7.47 Calcolo dei coefficienti DCT quantizzati.

Il passaggio 4 riduce il valore (0, 0) di ogni blocco (quello nell'angolo in alto a sinistra) sostituendolo con la quantità di cui differisce dall'elemento corrispondente nel blocco precedente. Visto che questi elementi sono la media dei rispettivi blocchi, dovrebbero cambiare lentamente, pertanto la scelta dei valori differenziali dovrebbe ridurne la maggior parte a valori piccoli. Nessun differenziale è elaborato per gli altri valori.

La fase 5 linearizza i 64 elementi e applica una **codifica run-length** all'elenco. La scansione del blocco da sinistra a destra e poi dall'alto verso il basso non concentrerà insieme i valori nulli, pertanto viene utilizzato uno schema di scansione a zig zag, come mostrato nella Figura 7.48. In questo esempio lo schema a zig zag produce 38 zero consecutivi alla fine della matrice. Questa stringa può essere ridotta a un singolo conteggio che afferma la presenza di 38 zero.

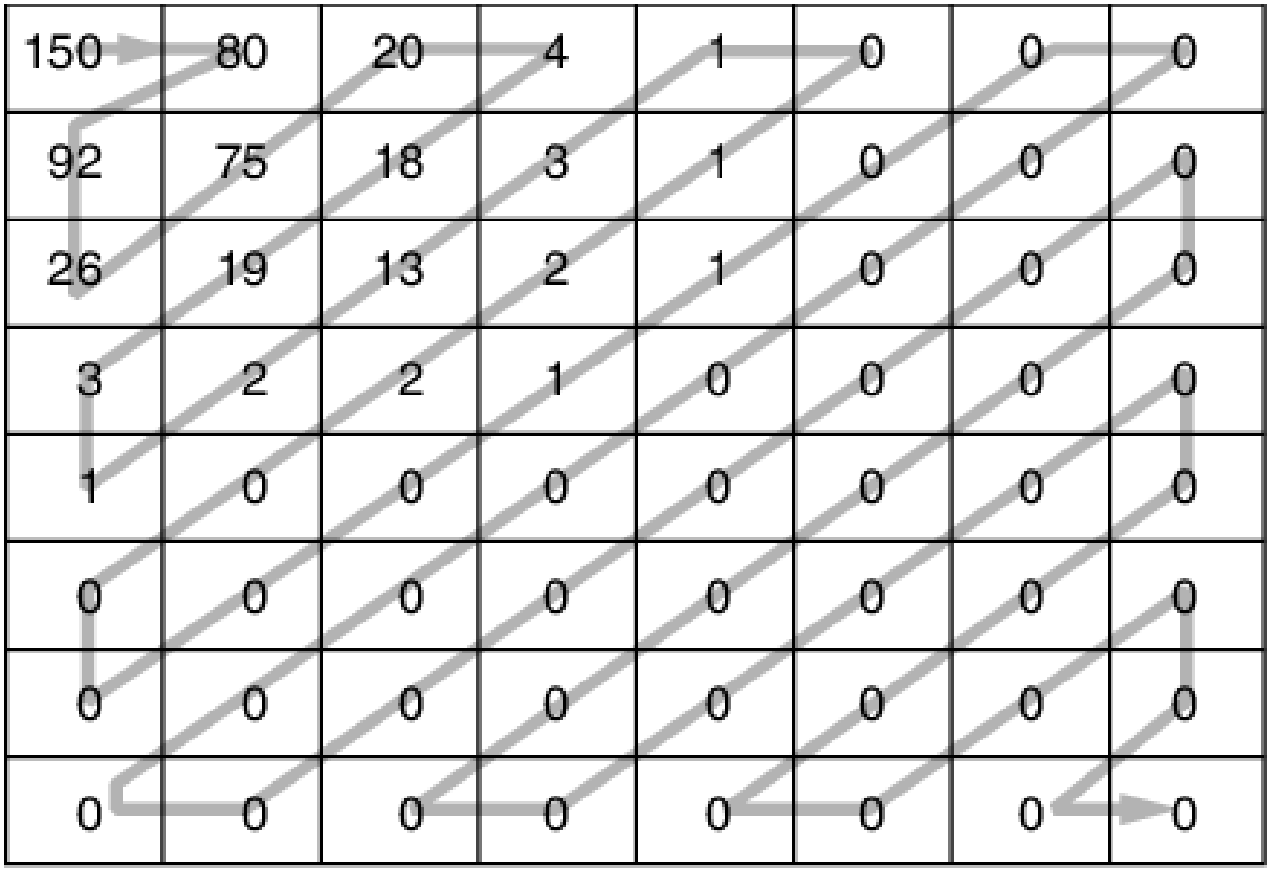


Figura 7.48 L'ordine in cui i valori quantizzati vengono trasmessi.

Ora abbiamo un elenco di numeri che rappresentano l'immagine (nello spazio della trasformata). Il passaggio 6 applica ai numeri la codifica di Huffman per l'archiviazione o la trasmissione, assegnando ai numeri più frequenti codici più brevi di quelli meno frequenti.

JPEG può sembrare complicato e in effetti lo è, tuttavia è anche molto diffuso perché spesso raggiunge una compressione con rapporto 20:1 o anche migliore. La decodifica di un'immagine JPEG richiede l'esecuzione inversa dell'algoritmo. JPEG è approssimativamente simmetrico: la

decodifica richiede circa lo stesso tempo della codifica. Questa proprietà non vale per tutti gli algoritmi di compressione, come vedremo tra poco.

Lo standard MPEG

Eccoci al cuore della questione: gli standard **MPEG** (*motion picture experts group*). Si tratta degli algoritmi principali utilizzati per comprimere i video e sono divenuti standard internazionali dal 1993. Dal momento che i filmati possono contenere immagini e suoni, MPEG può comprimere sia l'audio sia il video. Abbiamo già esaminato la compressione di audio e immagini statiche: ora vedremo la compressione del video.

Lo standard MPEG-1 (che include audio MP3) fu pubblicato per la prima volta nel 1993 ed è ancora molto usato. Il suo scopo era produrre un output di qualità simile a quella di un videoregistratore con una compressione 40:1 con tassi di invio dei dati attorno a 1 Mbps. Questo tipo di video è adatto all'uso su siti Web di Internet. Non vi preoccupate se non vi ricordate dei videoregistratori, MPEG-1 veniva usato per archiviare film su CD quando essi ancora esistevano. Se non sapete che cosa sono i CD, dobbiamo passare a MPEG-2.

Lo standard MPEG-2, rilasciato nel 1996, era progettato per comprimere video di qualità broadcast.[19] Ora è molto comune, perché è usato come base per la codifica video sui DVD (che inevitabilmente trova la sua applicazione su Internet) e per la televisione digitale broadcast (come di DVB). I video con qualità DVD sono tipicamente codificati a tassi tra i di 4 e gli 8 Mbps.
Lo standard MPEG-4 ha due formati video. Il primo formato, rilasciato nel 1999, codifica il video con una rappresentazione basata sull'oggetto; ciò permette di mescolare immagini naturali e sintetiche e altri tipi di contenuti, per esempio un addetto alle previsioni del tempo che sta davanti a una mappa. Con questa struttura è facile far interagire i programmi con i dati dei filmati. Il secondo formato, rilasciato nel 2003, è noto come **H.264** o **AVC** (*advanced video coding*). Il suo obiettivo è codificare i video a metà del tasso dei primi codificatori mantenendo la stessa qualità, quanto di meglio per supportare la trasmissione di video sulle reti. Questo codificatore è usato per HDTV sulla maggior parte dei blu-ray disc.

I dettagli di tutti questi standard sono molti e vari. Gli ultimi standard hanno anche molte più caratteristiche e opzioni di codifica dei primi. Tuttavia non entreremo nel dettaglio. Per la maggior parte i guadagni nella compressione video sono venuti con miglioramenti piccoli, ma numerosi nel tempo, piuttosto che da cambi drastici nel metodo in cui i video vengono compressi. Ci limiteremo quindi a dare una visione generale dei concetti principali.

MPEG comprime sia l'audio sia il video. Poiché i codificatori audio e video lavorano in modo indipendente c'è un problema su come i due flussi vengono sincronizzati al ricevente. La soluzione consiste nell'avere un singolo orologio che fornisca dei marcatori temporali per entrambi i codificatori. Questi marcatori sono inclusi nell'output codificato e propagati al ricevente che li può usare per sincronizzare i flussi audio e video.

La compressione video MPEG si avvantaggia di due tipi di ridondanza presenti nei filmati: spaziale e temporale. La ridondanza spaziale può essere eliminata semplicemente codificando ogni frame separatamente con JPEG. Tale approccio è usato occasionalmente, specialmente quando è necessario un accesso casuale a ogni frame, come nella fase di editing. In questo modo vengono raggiunti i livelli di compressione tipici di JPEG.

Una compressione aggiuntiva può essere raggiunta traendo vantaggio dal fatto che i frame consecutivi sono quasi identici. Questo effetto è minore di quanto potrebbe sembrare, perché molti registi eseguono transizioni di scena ogni 3 o 4 secondi (è possibile contare il tempo e le scene per verificare). Ciononostante anche un insieme di 75 frame molto simili offre il potenziale per una riduzione maggiore rispetto alla codifica separata di ogni frame con JPEG.

Nelle scene in cui la cinepresa e lo sfondo sono statici e uno o due attori si spostano lentamente, quasi tutti i pixel saranno identici di frame in frame. In questo caso è sufficiente sottrarre ogni frame dal precedente ed eseguire JPEG sulla differenza. Tuttavia questa tecnica non è applicabile per le scene dove la cinepresa esegue una panoramica o uno zoom; serve perciò un modo per compensare questo movimento. È proprio questo lo scopo di MPEG ed è la differenza principale tra MPEG e JPEG. L'output di MPEG è composto di tre tipi di frame:

1. I-frame (intracodificati): immagini statiche compresse e auto-contenute
2. P-frame (predittivi): differenze blocco per blocco con il frame precedente
3. B-frame (bidirezionali): differenze blocco per blocco con il frame precedente e quello successivo.

Gli I-frame sono semplici immagini statiche codificate utilizzando qualche variante di JPEG. È necessario che gli I-frame appaiano periodicamente (una o due volte al secondo) nel flusso di output per tre motivi. Innanzi tutto MPEG può essere utilizzato per una trasmissione multicast, dove gli utenti si sintonizzano in qualunque momento. Se tutti i frame dipendessero dai predecessori, fino al primo frame, chi si è perso il primo non potrà mai decodificare i successivi. Secondo, se si verificasse un errore su un frame ricevuto, non si sarebbe potuto procedere con la decodifica. Terzo, senza gli I-frame durante un avanzamento rapido o un riavvolgimento il decodificatore dovrebbe calcolare ogni frame saltato per conoscere il valore completo di quello su cui viene fermato.

I P-frame, invece, codificano le differenze tra frame. Sono basati sull'idea dei **macroblocchi**, che coprono 16 × 16 pixel nello spazio della luminanza e 8 × 8 pixel nello spazio della crominanza. Un macroblocco viene codificato ricercando nel frame precedente esso stesso o qualcosa che sia solo leggermente diverso.

Nella Figura 7.49 è mostrato un esempio di utilità dei P-frame. Qui sono illustrati tre frame consecutivi con lo stesso sfondo, ma con posizioni diverse della persona. I macroblocchi contenenti la scena dello sfondo corrispondono esattamente, mentre quelli con la persona sono scostati di una quantità sconosciuta e devono essere rintracciati.

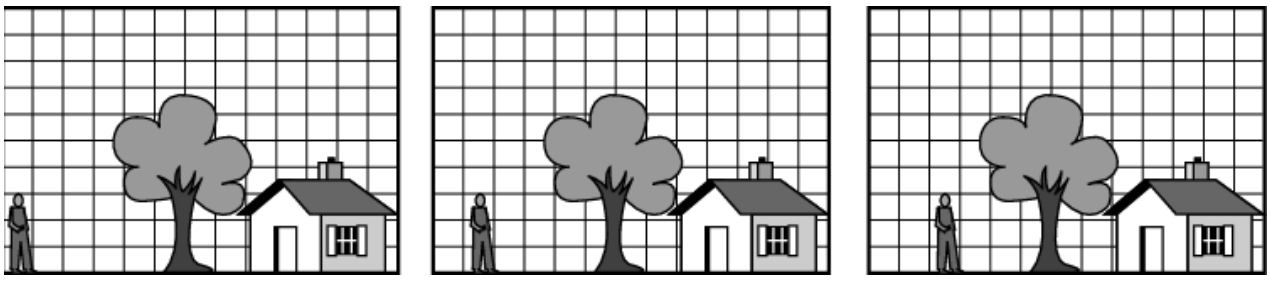


Figura 7.49 Tre frame consecutivi.

Lo standard MPEG non specifica come e dove cercare o il grado di corrispondenza necessario. Tutto questo è lasciato all'implementazione. Per esempio, un'implementazione potrebbe cercare un macroblocco nella posizione corrente del frame precedente e in tutte le altre posizioni scostate di ±Dx nella direzione x e ±Dy nella direzione y. Per ogni posizione deve essere elaborato il numero di corrispondenze nella matrice della luminanza. La posizione con il punteggio più alto sarà decretata vincitrice, purché si trovi sopra una soglia predefinita, altrimenti il macroblocco viene considerato mancante. Naturalmente sono disponibili anche algoritmi più sofisticati.

Se viene trovato un macroblocco, questo viene codificato calcolandone la differenza rispetto al valore nel frame precedente per la luminanza ed entrambe le crominanze. Queste matrici delle differenze sono poi soggette alla trasformazione coseno discreta, alla quantizzazione, alla codifica run-length e alla codifica di Huffman, proprio come JPEG. Il valore per il macroblocco nel flusso di output è quindi il vettore di movimento (la distanza di cui il macroblocco è stato spostato dalla posizione precedente in ogni direzione), seguito da un elenco di numeri con codifica Huffman. Se il macroblocco non è situato nel frame precedente il valore corrente è codificato con JPEG, come per un I-frame.

Chiaramente questo algoritmo è altamente asimmetrico. Un'implementazione è libera di provare ogni posizione plausibile nel frame precedente, se lo desidera, in un tentativo disperato di individuare ogni macroblocco precedente, indipendentemente dalla posizione in cui è stato spostato. Questo approccio minimizzerà il flusso MPEG codificato a spese di una codifica molto lenta. L'approccio può essere valido per codificare un filmato, ma sarebbe devastante per una videoconferenza in tempo reale.

Allo stesso modo ogni implementazione è libera di decidere che cosa costituisca un macroblocco "individuato". Questa libertà permette agli implementatori di competere sulla qualità e la velocità dei loro algoritmi, ma produce implementazioni sempre compatibili con MPEG.

Fino a questo punto la decodifica di MPEG è elementare. La decodifica degli I-frame equivale alla decodifica delle immagini JPEG; la decodifica dei P-frame richiede al decoder di inserire nel buffer i frame precedenti e di costruire quello attuale in un secondo buffer, in base ai macroblocchi completamente codificati e ai macroblocchi che contengono differenze rispetto al frame precedente. Il nuovo frame viene assemblato macroblocco per macroblocco.

I B-frame sono simili ai P-frame, tranne per il fatto che consentono al macroblocco di riferimento di trovarsi sia nel frame precedente che nel frame successivo. Questa libertà aggiuntiva permette una compensazione del movimento migliorata ed è utile quando gli oggetti passano davanti o dietro ad altri oggetti. Per la codifica dei B-frame il codificatore deve disporre di una sequenza di frame decodificati in memoria: il precedente, quello attuale e il successivo. Analogamente la decodifica è più complicata e comporta alcuni ritardi, perché un frame non può essere decodificato finché il frame successivo, dal quale dipende, viene decodificato. Anche se i B-frame offrono la migliore compressione non sono molto usati per via della loro grande complessità e richiesta di buffer.

Lo standard MPEG contiene molti miglioramenti a queste tecniche che permettono di raggiungere eccellenti livelli di compressione. AVC può essere usato per comprimere video con rapporti di oltre 50:1, il che riduce la richiesta di banda di rete dello stesso fattore. Per ulteriori informazioni su AVC si leggano Sullivan e Wiegand (2005).

Streaming di contenuti registrati

Spostiamoci ora verso le applicazioni di rete. Il nostro primo caso è lo streaming di contenuti già archiviati in file (*stored media*). L'esempio più comune è guardare dei video su Internet. Questa è una forma di **video on demand** (VoD). Altre forme di video on demand prevedono un provider con una rete separata da Internet per consegnare i film (per esempio, la TV via cavo). Nel prossimo paragrafo esamineremo lo streaming di contenuti prodotti sul momento (*live media*) come il broadcast televisivo con IP (IPTV) e la radio su Internet. Vedremo quindi il terzo caso di conferenze in tempo reale, come una chiamata VoIP o una videoconferenza su Skype. Questi tre casi hanno dei requisiti sempre più stringenti su come consegnare audio e video attraverso la rete, perché bisogna porre un'attenzione sempre crescente al ritardo e al jitter.

Internet pullula di siti di musica e video che fanno streaming di contenuti registrati. Effettivamente il modo più facile per gestire questi contenuti è di non farne lo streaming. Si immagini di voler creare un sito di noleggio di film on-line per competere con iTunes Store di Apple. Un sito Web regolare permetterà agli utenti di scaricare e guardare i video (dopo aver pagato, ovviamente). La sequenza di passaggi è mostrata nella Figura 7.50. Li analizzeremo in dettaglio in modo da poterli confrontare con il prossimo esempio.

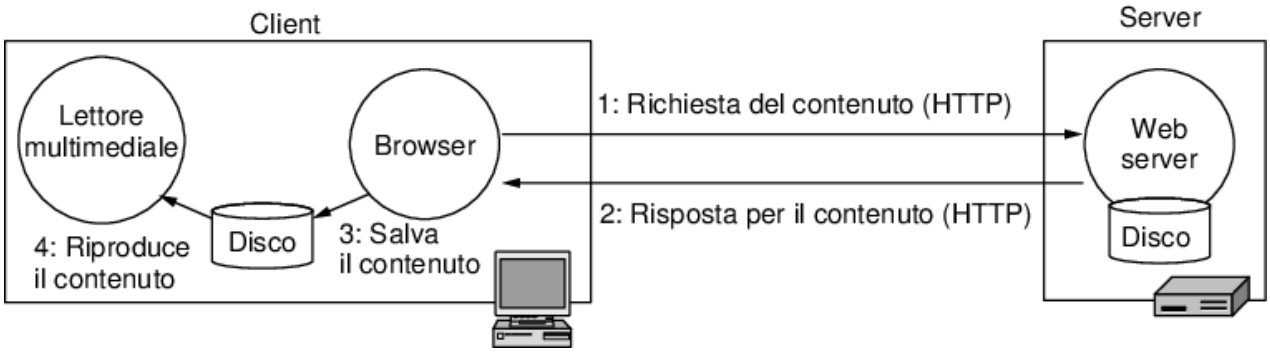


Figura 7.50 Riproduzione di un contenuto via Web con download semplice.

Il browser entra in azione quando l'utente fa click sul titolo di un film. Nel passo 1 invia una richiesta TCP al server Web su cui si trova il film collegato. Nel passo 2 il server preleva il film (che è un semplice file MP4 o in qualche altro formato) e lo restituisce al browser. Utilizzando il tipo MIME (per esempio *video/mp4*) il browser cerca di capire come deve visualizzare il file. In questo caso si tratta di un lettore multimediale che si presenta sotto forma di un'applicazione di supporto, ma potrebbe essere anche un plug-in. Nel passo 3 il browser salva l'intero film in un file

sul disco, quindi avvia il lettore passandogli il nome del file temporaneo. Infine, nel passo 4, il lettore inizia a visualizzare il contenuto del file. In principio questo approccio è assolutamente corretto e sufficiente per far vedere il film. Non vi è alcun problema di rete in tempo reale da affrontare, perché si tratta semplicemente di fare download di un file. L'unico problema è che l'intero video deve essere trasmesso sulla rete prima dell'inizio della riproduzione. Questo problema riguarda anche l'audio. Si immagini di voler ottenere una canzone prima di acquistare l'album. Se il brano è di 4 MB (dimensione tipica per un brano MP3) e la connettività a banda larga è di 1 Mbps, l'utente sarà accolto da mezzo minuto di silenzio prima che il brano inizi. Con questo modello probabilmente non si venderebbero molti album.

Per risolvere il problema senza modificare il funzionamento del browser i siti hanno provato a utilizzare lo schema mostrato nella Figura 7.51. La pagina collegata al film non è il vero e proprio file. Si tratta invece di un **metafile**, vale a dire un file molto breve il cui scopo è solo quello di assegnare un nome al film. Un tipico metafile potrebbe contenere una sola riga di testo ASCII simile alla seguente:

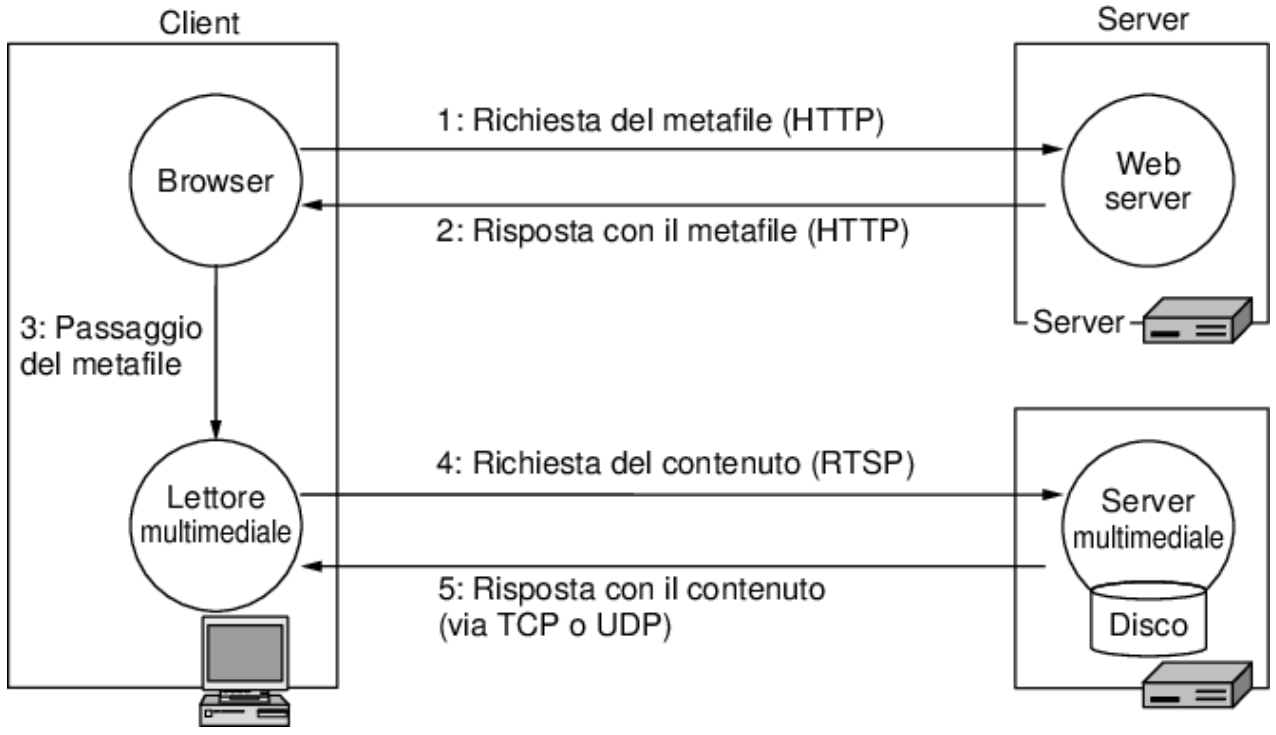


Figura 7.51 Streaming usando un server Web e un server multimediale.

1. *rtsp://joes-movie-server/movie-0025.mp4*

Nei passi 1 e 2 il browser riceve il file, ora di una riga, come al solito. Nel passo 3 avvia il lettore multimediale e gli passa il file di una riga, come al solito. Il lettore multimediale legge il file e vede che contiene l'URL dove si trova il film; contatta quindi *joes-video-server* e richiede il film nel passo 4. Il film è quindi mandato al lettore multimediale nel passo 5. Il vantaggio di questo schema è che

il lettore inizia velocemente, dopo solo il breve tempo in cui il metafile viene scaricato; ora il browser non fa più parte del ciclo. Il contenuto è inviato direttamente al lettore che può iniziare a visualizzarlo prima che venga scaricato l'intero file.

Nella Figura 7.51 abbiamo indicato due server perché, nella maggior parte dei casi, il server nominato nel metafile non è lo stesso server Web. In effetti di norma non si tratta di un server HTTP, ma di un server multimediale specializzato (*media server*). In questo esempio, il server multimediale utilizza **RTSP** (*real time streaming protocol*), come indicato dallo schema di accesso *rtsp.*

Il lettore multimediale deve svolgere quattro operazioni fondamentali:

1. gestire l'interfaccia utente
2. gestire gli errori di trasmissione
3. decomprimere il contenuto
4. eliminare il jitter.

La maggior parte dei lettori multimediali odierni presenta un'interfaccia utente accattivante, che ricorda uno stereo con pulsanti, manopole, potenziometri lineari e visualizzazioni. Spesso sono disponibili pannelli frontali intercambiabili, chiamati **skin**, che l'utente può applicare al lettore. Il lettore multimediale deve gestire tutto questo e interagire con l'utente.

Le altre operazioni sono correlate e dipendono dai protocolli di rete. Le vedremo una a una, a partire dalla gestione degli errori di trasmissione che dipende se si usa un trasporto basato su TCP come HTTP o basato su UDP come RTP. Entrambi sono usati nella pratica. Se il trasporto si basa su TCP, il lettore non deve correggere alcun errore, perché TCP già fornisce affidabilità usando la ritrasmissione. Questo è un semplice modo per gestire gli errori, almeno per il lettore, ma complica l'operazione di rimozione dei jitter in un passo successivo.

Alternativamente per spostare i dati si può usare un trasporto basato su UDP, come RTP. Come studiato nel Capitolo 6, con questi protocolli non c'è ritrasmissione, quindi la perdita dei pacchetti dovuta a congestioni o errori di trasmissione implica che qualche parte del contenuto non arriva e il problema deve essere gestito dal lettore multimediale.

Cerchiamo di capire la difficoltà che stiamo affrontando: le perdite rappresentano un problema, perché i clienti non apprezzano lunghi vuoti nelle loro canzoni o in un film. Tuttavia non è un problema grave come perdere dati nel trasferimento di un file regolare, perché la perdita di una piccola parte del contenuto non degrada la presentazione fatta all'utente. Per i video, probabilmente l'utente non nota se, occasionalmente, ci sono 24 nuovi frame in alcuni secondi invece che 25. Nell'audio piccoli vuoti nella riproduzione possono essere mascherati con suoni simili nel tempo. Gli utenti probabilmente non si renderanno conto della sostituzione, a meno che non stiano ponendo un'attenzione veramente particolare.

Il punto chiave del nostro ragionamento, tuttavia, è che i vuoti siano veramente molto brevi. Le congestioni di rete e gli errori di trasmissione in generale causano la perdita di un intero pacchetto e i pacchetti sono spesso persi in piccoli gruppi. Esistono due strategie per ridurre l'impatto della perdita dei pacchetti sui media: FEC e interleaving.

FEC (*forward error correction*) è semplicemente la codifica a correzione di errore studiata nel Capitolo 3 e applicata al livello applicazione. La parità tra i pacchetti fornisce un esempio (Shacham e McKenny, 1990). Ogni quattro pacchetti di dati inviati un quinto pacchetto di parità viene

costruito e inviato, come mostrato nella Figura 7.52 con i pacchetti *A*, *B*, *C* e *D*. Il pacchetto di parità, *P*, contiene bit ridondanti che sono la parità o la somma con OR esclusivo dei bit in ciascuno dei quattro pacchetti di dati. È auspicabile che tutti i pacchetti arrivino per la maggior parte dei gruppi di cinque. Quando ciò accade il pacchetto di parità è semplicemente scartato dal ricevente oppure, se viene perso solo il pacchetto di parità, non succede niente di grave.

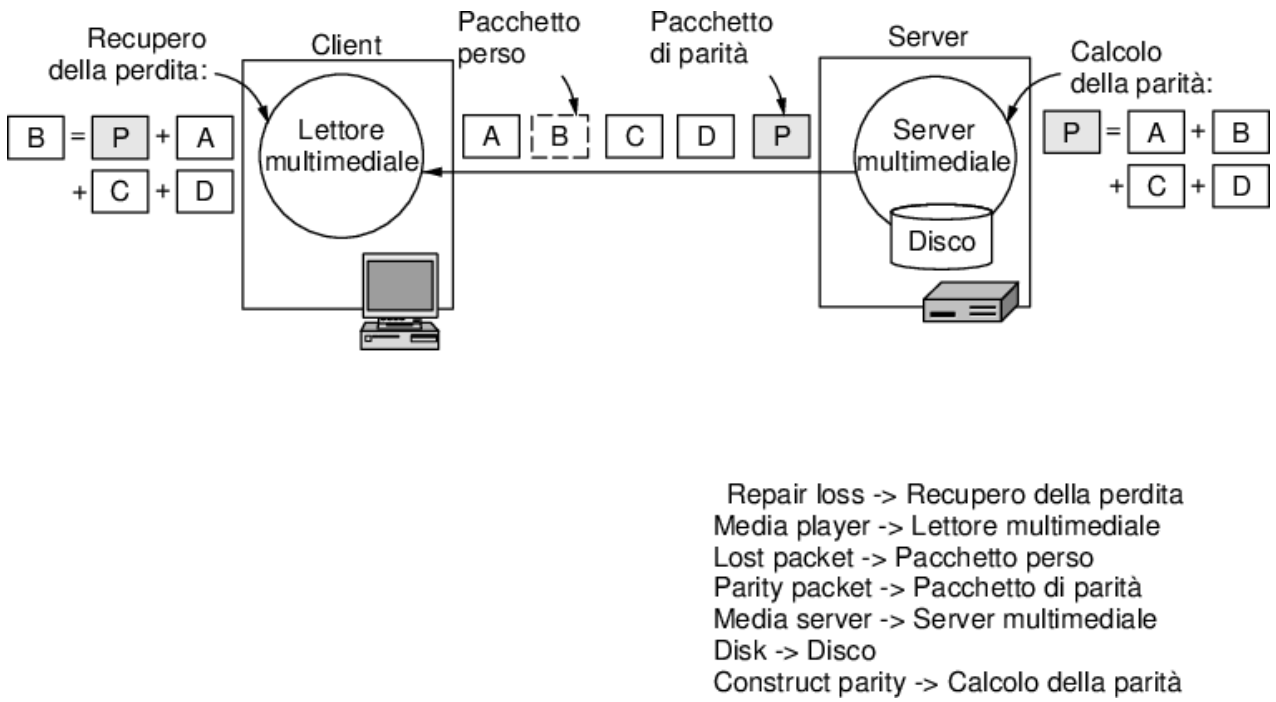


Figura 7.52 Uso di un pacchetto di parità per il recupero delle perdite.

Occasionalmente, tuttavia, un pacchetto di dati può essere perso durante la trasmissione, come accade a *B* nella Figura 7.52. Il lettore riceve solo tre pacchetti di dati, *A*, *C* e *D*, più il pacchetto di parità, *P*. Dato lo schema usato, i bit nei pacchetti di dati perduti possono essere ricostruiti dai bit di parità. Per essere più precisi, usando "+" per rappresentare l'OR esclusivo o l'addizione in modulo due, *B* può essere ricostruito come $B = P + A + C + D$ per proprietà dell'OR esclusivo (cioè, $X + Y + Y = X$).

Il FEC può ridurre i livelli di perdita subiti dal lettore riparando alcune delle perdite di pacchetti, ma questo sistema funziona solo fino a un certo punto. Se vengono persi due pacchetti in un gruppo di cinque non si può fare niente per ricostruire i dati. La qualità da notare di FEC è il costo che si paga per guadagnare questa protezione. Quattro pacchetti diventano in realtà cinque pacchetti, aumentando del 25% i requisiti di banda. Anche la latenza della codifica cresce, perché è necessario attendere finché non arrivi il pacchetto di parità, prima di ricostruire il pacchetto di dati arrivato precedentemente. C'è un trucco interessante nella tecnica descritta sopra. Nel Capitolo 3 abbiamo descritto la parità per fornire la rilevazione degli errori. Qui essa fornisce la correzione di errore. Può far entrambe le cose? La risposta è che in questo caso si conosce quale pacchetto viene perso: i dati persi vengono chiamati **erasure** (*cancellazione*). Nel Capitolo 3 abbiamo considerato

un frame che viene ricevuto con alcuni bit di errore, ma non sapevamo quali; quello era un caso più difficile da trattare. Con le cancellazioni la parità può fornire correzione di errore; mentre senza, solo rilevazione di errore. Vedremo un altro vantaggio inaspettato della parità non appena studieremo il multicast.

La seconda strategia è chiamata **interleaving** (*intercalare*). Questo approccio si basa sul mischiare l'ordine dei dati prima della trasmissione e rimetterli nel giusto ordine alla ricezione. In questo modo, se un pacchetto (o gruppo di pacchetti) viene perso, la perdita verrà diffusa nel tempo dalla procedura di riordino. Non risulterà in un singolo e grande salto quando il contenuto verrà riprodotto. Per esempio, un pacchetto potrebbe contenere 220 campioni di audio stereo, ognuno contenente una coppia di numeri a 16 bit, normalmente utili per 5 ms di musica. Se i campioni sono inviati in ordine, un pacchetto perso rappresenterebbe un vuoto di 5 ms nella musica. Invece, se trasmessi come mostrati nella Figura 7.53, tutti i campioni dispari nell'intervallo di 10 ms sono inviati in un pacchetto, seguiti dai campioni pari nel successivo. La perdita del pacchetto 3 non rappresenta un vuoto di 5 ms nella musica, ma la perdita alternata di campioni per 10 ms. Questa perdita può essere gestita facilmente dal lettore interpolando con i campioni precedenti e successivi. Il risultato è una risoluzione temporale più bassa per 10 ms, ma non un vuoto significativo nel contenuto.

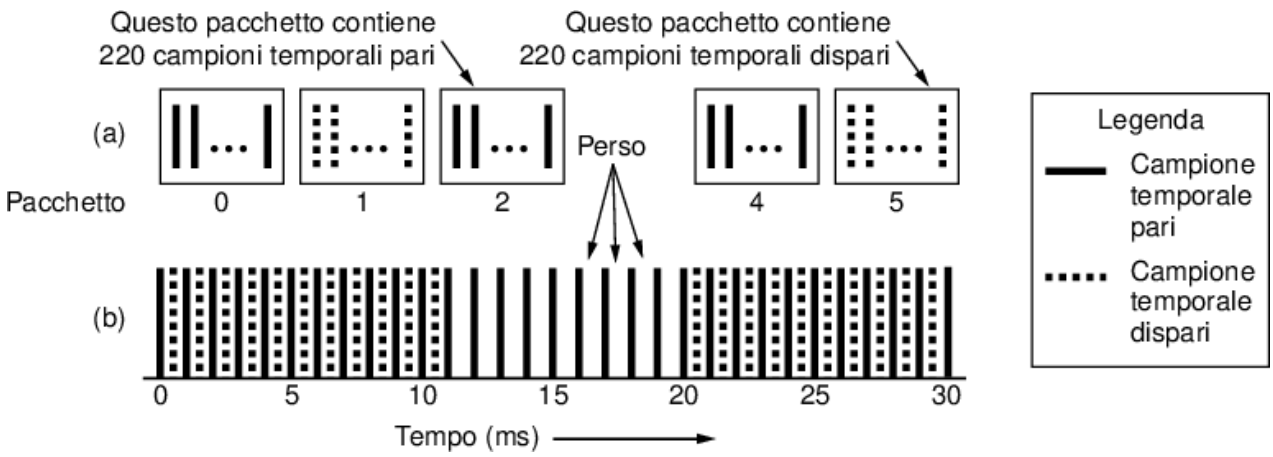


Figura 7.53 Quando i pacchetti trasportano campioni alternati la perdita di un pacchetto riduce la risoluzione temporale piuttosto che creare un vuoto.

Lo schema di interpretazione descritto prima funziona solo con campioni non compressi; tuttavia, l'interpretazione (su piccoli periodi di tempo, non su campioni individuali) può essere applicata anche dopo la compressione, purché ci sia un modo per trovare i confini dei campioni nel flusso compresso. L'RFC 3119 fornisce uno schema che funziona con audio compressi.

L'interleaving è una tecnica interessante quando può essere usata, perché non richiede banda addizionale come il FEC, ma, come quest'ultimo, aggiunge latenza, perché deve aspettare che arrivi un gruppo di pacchetti (in modo da invertire il processo di interleaving).

Il terzo compito del lettore multimediale è decomprimere il contenuto. Sebbene sia un lavoro pesante dal punto di vista computazione si tratta di un compito molto facile. Il problema principale è come decodificare il contenuto se il protocollo di rete non correggere gli errori di trasmissione. In molti schemi di compressione i dati non possono essere decompressi finché non lo siano stati i dati precedenti, poiché la codifica dipende da essi. In un trasporto basato su UDP i pacchetti possono venire persi; quindi la codifica deve essere progettata per permettere la decodifica nonostante la perdita di pacchetti. Questa richiesta è il motivo per cui MPEG usa i frame I, P e B: ogni I-frame può essere decodificato indipendentemente dagli altri per recuperare le perdite dei frame precedenti.

Il quarto compito è eliminare il jitter, la sventura di ogni sistema in tempo reale. La soluzione generale che abbiamo descritto nel Paragrafo 6.4.3 è usare un buffer. Tutti i sistemi di streaming iniziano con un buffer di 5-10 secondi prima di dare avvio alla riproduzione, come mostrato nella Figura 7.54. I dati da riprodurre vengono estratti dal buffer con tasso costante in modo che l'audio sia chiaro e il video sia continuo. Il ritardo all'inizio dà al buffer l'opportunità di riempirsi fino a un **limite inferiore** (*low-water mark*). L'idea è che i dati dovrebbero arrivare in modo abbastanza regolare da non permettere al buffer di svuotarsi completamente. Se ciò avvenisse, la riproduzione dovrebbe fermarsi. Il valore nel fare uso di buffer è che i dati a volte arrivano piano a causa di congestioni; il buffer permette di continuare la riproduzione normalmente finché non arrivano nuovi dati per poi riempirsi di nuovo.

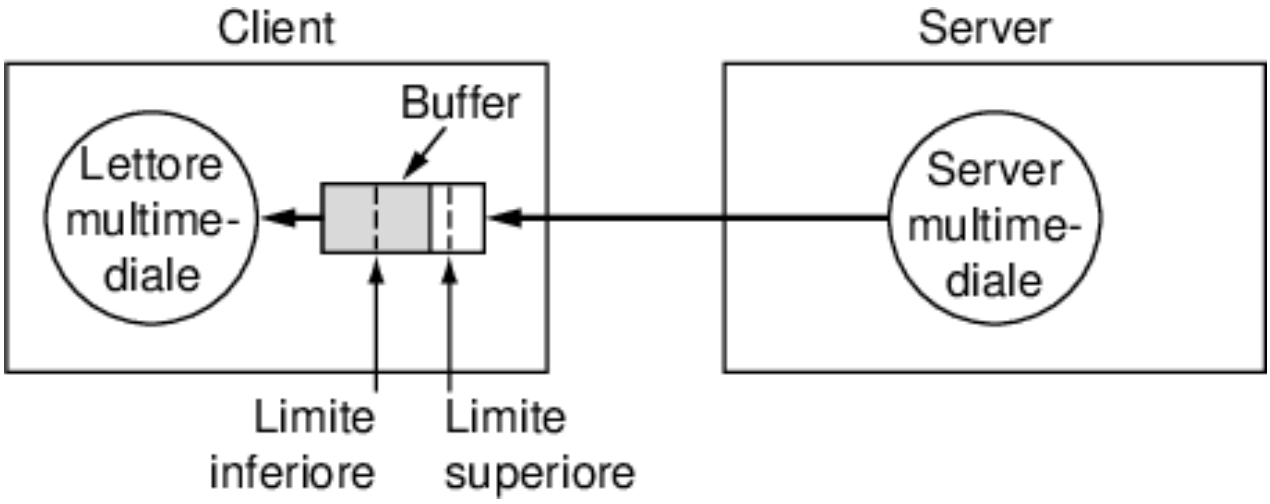


Figura 7.54 Il lettore multimediale riceve i dati dal server multimediale ma ne usufruisce tramite il buffer piuttosto che direttamente dalla rete.

Quanto buffer si deve utilizzare e quanto velocemente il server multimediale deve inviare i dati per riempire il buffer dipendono dai protocolli di rete; vi sono molte possibilità. Il fattore principale che influenza la progettazione riguarda la scelta tra un trasporto basato su UDP o su TCP.

Si supponga di usare un trasporto basato su UDP, come RTP. Si supponga inoltre che ci sia un'ampia banda per inviare pacchetti dal server al lettore multimediale con poche perdite e che il resto del traffico sulla rete sia esiguo. In questo caso i pacchetti possono essere inviati all'esatto tasso in cui vengono letti. Ogni pacchetto passa sulla rete e, dopo un certo ritardo di propagazione, arriva circa al tempo esatto al lettore per essere riprodotto. Non essendoci variabilità nel ritardo è necessario un buffer piccolo. Se vengono usati l'interleaving o il FEC si deve utilizzare più buffer, almeno per i gruppi di pacchetti sui quali queste tecniche vengono applicate; tuttavia, ciò richiede solo una piccola quantità aggiuntiva di buffer.

Sfortunatamente questo scenario non è realistico per due aspetti. In primo luogo, la banda cambia a seconda del percorso in rete; quindi non è chiaro al server se ci sarà abbastanza banda per tentare di far fluire il contenuto. Una semplice soluzione è codificare il contenuto in più risoluzioni e lasciare che ogni utente scelga la risoluzione supportata dalla sua connettività Internet. Spesso ci sono solo due livelli: alta qualità, codifica a 1,5 Mbps o più e bassa qualità, codifica a 512 kbps o meno.

In secondo luogo, ci sarà il jitter o variazione su quanto tempo impiegano i campioni per attraversare la rete. Questo jitter si può verificare per due cause. Spesso c'è un significativa quantità di traffico in competizione sulla rete (che può derivare da utenti multitasking che navigano sul Web mentre guardano un film) e tale traffico causerà fluttuazioni del momento in cui il contenuto arriva. Inoltre ci interessa l'arrivo di frame video e campioni audio, non dei pacchetti. Con la compressione i frame video, in particolare, possono essere più grandi o più piccoli a seconda del loro contenuto. Una sequenza piena di azione richiederà tipicamente più bit per essere codificata che non un paesaggio. Se la banda di rete è costante, il tasso di consegna rispetto al tempo varierà. Più si viene a creare jitter o variazioni nel ritardo per queste cause, maggiore dovrà essere il limite inferiore del buffer per evitare che esso si svuoti.

Si supponga ora che, per inviare il contenuto, venga usato un trasporto basato su TCP, come HTTP. Effettuando le ritrasmissioni e aspettando la consegna dei pacchetti finché sono in ordine si aumenta, anche in modo significativo, il jitter osservato dal lettore. Il risultato è che sono necessari un buffer e un limite inferiore più grandi. Tuttavia esiste anche un vantaggio: TCP invierà i dati tanto velocemente quanto la rete li trasporta. A volte i contenuti possono essere ritardati se la perdita viene riparata ma, per la maggior parte delle volte, la rete sarà in grado di consegnarli più velocemente di quanto il lettore li consuma. In questi periodi il buffer sarà pieno e potrà prevenire problemi successivi. Se la rete è significativamente più veloce del tasso medio, come spesso succede, il buffer si riempirà rapidamente dopo la partenza, in modo tale che svuotarlo non sia più una preoccupazione.

Con TCP o UDP e un tasso di trasmissione che ecceda il tasso di output la domanda è quanto più avanti del punto in riproduzione il lettore e il server vogliano arrivare. Spesso vogliono scaricare l'intero file.

Tuttavia, il procedere lontano dal punto di riproduzione induce un lavoro non necessario che può richiedere uno spazio significativo per l'archiviazione e non serve per la risoluzione di problemi relativi al buffer che si svuota. In alternativa a ciò il lettore può adottare la soluzione di definire un **limite superiore** (*high-water mark*) nel buffer. Fondamentalmente il server invia i dati finché il buffer arriva a riempirsi al livello del limite superiore. Il lettore chiede quindi di fare una pausa. Visto che i dati continuano ad affluire finché il server non abbia ricevuto la richiesta di pausa, la

distanza tra il limite superiore e la fine del buffer deve essere superiore al prodotto banda-ritardo della rete. Una volta fermato il server, il buffer inizia a svuotarsi. Una volta raggiunto il limite inferiore, il lettore comunica al server di riprendere. Perché non si svuoti completamente il buffer, anche il limite inferiore deve essere scelto tenendo conto del prodotto banda-ritardo della rete.

Per iniziare e terminare il flusso di dati il lettore multimediale ha bisogno di un controllo remoto fornito da RTSP. Questo è definito nell'RFC 2326 e fornisce un meccanismo che permette al lettore di controllare il server. Oltre a far partire e fermare il flusso, può andare indietro e avanti su una posizione, riprodurre intervalli specificati e andare a velocità alte o basse. Non specifica però la gestione del flusso dei dati, lasciata di norma a RTP su UDP o HTTP su TCP. I comandi principali supportati da RTSP sono elencati nella Figura 7.55. Hanno un semplice formato di testo, come i messaggi HTTP e sono solitamente trasportati da TCP. RTSP può essere implementato anche su UDP, poiché ogni comando riceve un acknowledgedment (e quindi può essere ritrasmesso se necessario). Sebbene TCP potrebbe sembrare non adatto al traffico in tempo reale, viene spesso usato nella pratica. La ragione principale è che è in grado di passare attraverso i firewall più facilmente di UDP, specialmente se usa la porta di HTTP. La maggior parte degli amministratori configura i firewall per proteggere le loro reti da visitatori indesiderati. Permettono sempre le connessioni TCP da una porta remota 80 per far passare HTTP e il traffico Web. Bloccare questa porta genera velocemente malcontento tra gli utenti. Tuttavia, la maggior parte delle altre porte sono bloccate, incluse quelle per RTSP e RTP 554 e 5004 rispettivamente. Quindi il modo più facile per avere streaming multimediale attraverso un firewall è che il server multimediale finga di essere un server HTTP che stia mandando una regolare risposta HTTP, almeno per il firewall.

Comando	Azione del server
DESCRIBE	Elenca i parametri del file multimediale
SETUP	Stabilisce un canale logico tra il lettore e il server
PLAY	Inizia a inviare dati al client
RECORD	Inizia ad accettare dati dal client
PAUSE	Interrompe temporaneamente l'invio dei dati
TEARDOWN	Rilascia il canale logico

Figura 7.55 I comandi RTSP dal lettore al server.

Con TCP ci sono anche alcuni vantaggi. Poiché garantisce affidabilità, TCP fornisce al client una copia completa del contenuto, cosa che facilita l'utente a tornare a un punto visto in precedenza senza preoccuparsi della perdita di dati. Infine, TCP metterà nel buffer il contenuto il più velocemente possibile. Quando lo spazio per il buffer è economico (come quando si usa il disco per

registrare), il lettore può scaricare il contenuto mentre l'utente lo vede. Una volta scaricato completamente, l'utente lo può guardare senza interruzione anche se perde connettività. Questa proprietà è utile per i dispositivi mobili, dove la connessione cambia rapidamente.

Gli svantaggi di TCP sono l'aggiunta di un ritardo iniziale (a causa della sua fase iniziale) e anche un limite inferiore più alto. Tuttavia si tratta raramente di una penalizzazione significativa, finché la banda della rete eccede di molto il tasso medio di codifica.

Streaming di contenuti in tempo reale

Sul Web non sono estremamente popolari solo i video registrati, ma anche quelli prodotti in tempo reale (*live media*). Quando divenne possibile fare streaming di audio e video su Internet le radio commerciali e le stazioni TV ebbero l'idea di distribuire i loro contenuti su Internet come nell'etere. Non molto tempo dopo stazioni di college cominciarono a inviare i loro segnali su Internet: gli studenti lanciavano i loro programmi su Internet.

Oggigiorno privati e piccole e grandi aziende fanno streaming in tempo reale (live) di audio e video. L'area sta diventando un punto caldo di innovazione man mano che le tecnologie e gli standard evolvono. Lo streaming live è usato dalle maggiori stazioni televisive per avere una presenza on-line, chiamata **IPTV** *(IP television).* È usata anche per fare distribuzione broadcast di stazioni radio come la BBC: questa è la **radio su Internet** (*Internet radio*). IPTV e radio su Internet raggiungono il pubblico in tutto il mondo con eventi che vanno dalle sfilate di moda ai campionati mondiali di calcio e le prove in diretta del campionato di cricket di Melbourne. Lo streaming live su IP è usato come tecnologia dai fornitori di TV via cavo per costruire i loro sistemi di distribuzione broadcast. È anche molto usato per operazioni a basso costo che vanno dai siti per adulti agli zoo. Con l'attuale tecnologia chiunque può virtualmente iniziare a fare streaming live velocemente e con poca spesa.

Un approccio consiste nella memorizzazione di programmi su disco. Chi vuol vederli si connette agli archivi del server, scarica il programma e lo ascolta. Un **podcast** è un episodio di una serie preso in questo modo. Per gli eventi schedulati è anche possibile archiviare il contenuto subito dopo la diretta e usarlo mezz'ora o meno dopo l'evento. Infatti questo approccio è esattamente uguale a quello adottato per lo streaming di contenuti registrati discusso prima. È facile da realizzare, tutte le tecniche che abbiamo discusso funzionano e gli utenti possono scegliere tra tutti i programmi in archivio.

Un approccio diverso è fare distribuzione broadcast di contenuti prodotti in tempo reale su Internet. Gli utenti selezionano i flussi multimediali esattamente come sulla televisione. Tuttavia i lettori multimediali forniscono caratteristiche aggiuntive, per permettere all'utente di fare pausa o andare indietro nella visione. Il live media continuerà a essere mandato in streaming e verrà messo in un buffer dal lettore finché l'utente non lo legga. Per il browser è esattamente come se fosse uno streaming di contenuti registrati; al lettore non importa se il contenuto viene da un file o è inviato in tempo reale, anzi usualmente non è in grado di capirlo (a parte per il fatto che non è possibile andare avanti se è prodotto in tempo reale). Data la somiglianza del meccanismo, la maggior parte della discussione precedente si applica anche in questo contesto, ma ci sono alcune differenze chiave.

La prima cosa importante da considerare è che per smussare il jitter è ancora necessario usare un buffer lato client. Infatti è spesso necessaria una grande quantità di buffer per streaming live (indipendentemente dalla considerazione che un utente può fare pausa). Quando lo streaming è effettuato da un file, il contenuto può essere erogato a un tasso più alto di quello di riproduzione. Ciò porterà rapidamente il buffer a riempirsi per compensare il jitter della rete (il lettore fermerà il flusso se non vuole mettere nel buffer più dati). Al contrario, lo steaming live viene sempre trasmesso precisamente al tasso di generazione, che è lo stesso a cui viene rappresentato e non può andare più veloce. Di conseguenza, il buffer deve essere abbastanza grande da gestire tutto il possibile jitter di rete. In pratica, questo non è un grande problema, perché di solito si usa un ritardo iniziale di circa 10–15 secondi.

L'altra importante differenza è che gli eventi in streaming live vedono di solito centinaia o migliaia di utenti simultaneamente sullo stesso contenuto, per cui si usa il multicast, al contrario dello streaming di contenuti registrati in cui gli utenti accedono a contenuti diversi. Lo streaming a più utenti consiste in tante sessioni individuali che per caso avvengono contemporaneamente.

Lo streaming multicast funziona in questo modo: il server invia ogni pacchetto una volta usando multicast IP a un gruppo di indirizzi, la rete consegna una copia del pacchetto a ogni membro del gruppo e chiunque voglia ricevere tale streaming deve unirsi al gruppo. I client lo fanno usando IGMP piuttosto che inviando un messaggio RTSP al server multimediale che lo sta già mandando (a parte che prima dell'arrivo del primo utente). Ciò che occorre è essere in grado di ricevere il flusso localmente.

Siccome il multicast è un servizio di consegna "uno a molti", il contenuto viene trasportato in pacchetti RTP su trasporto UDP, in quanto TCP opera solo tra una singola sorgente e un singolo destinatario. Poiché UDP non fornisce affidabilità, alcuni pacchetti possono essere persi; per ridurre tale perdita a un livello accettabile, si possono usare, come prima, FEC o interleaving.

Nel primo caso c'è un'interazione favorevole con il multicast mostrata nell'esempio di parità della Figura 7.56. Quando i pacchetti sono multicast, client diversi possono perdere pacchetti differenti. Per esempio, il client 1 ha perso il pacchetto *B*, il client 2 ha perso il pacchetto di parità *P*, il client 3 ha perso *D* e il client 4 non ha perso alcun pacchetto. Tuttavia, benché siano stati persi tre diversi pacchetti, in realtà in questo esempio ogni client può recuperarli tutti. L'unico requisito è che ogni client perda un solo pacchetto, non importa quale, che verrà ricostruito dal controllo di parità. Nonnenmacher et al. (1997) descrive come quest'idea possa essere usata per aumentare l'affidabilità.

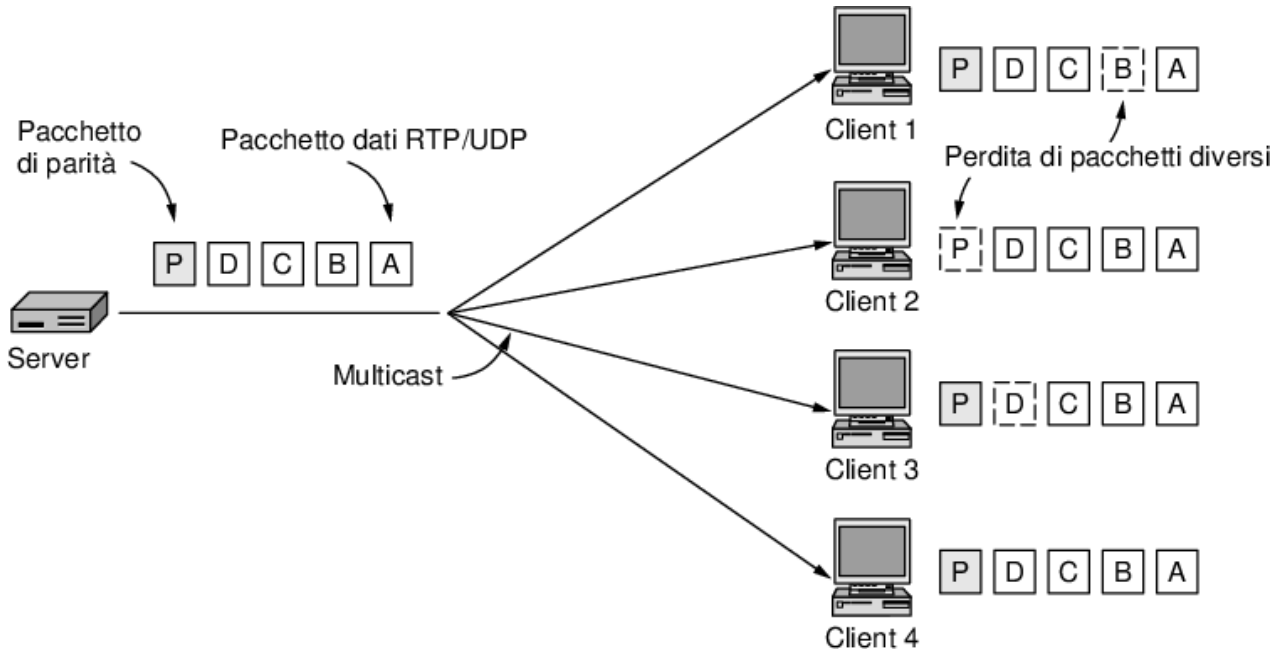


Figura 7.56 Streaming multicast con un pacchetto di parità.

Per un server con molti client il modo più efficiente di operare è, chiaramente, fare multicast di contenuti con pacchetti RTP e UDP, altrimenti il server deve trasmettere *N* flussi quando ha *N* client, richiedendo una grande quantità di banda.

Non vi sorprenderà sapere che, nella pratica, Internet non funziona in questo modo. Quello che succede è che ogni utente stabilisce una connessione TCP separata verso il server su cui il contenuto viene mandato in streaming. Per il client è come fare streaming di contenuto registrato e, anche in questo caso, vi sono delle ragioni a sostegno di questa scelta, a prima vista sbagliata.

La prima è che il multicast IP non sempre è disponibile su Internet. Alcuni ISP e reti lo supportano internamente, ma di solito non è disponibile tra reti diverse, nonostante sia necessario per fare streaming su grandi aree. Le altre ragioni sono gli stessi vantaggi di TCP su UDP discussi prima. Lo streaming con TCP raggiungerà circa tutti i client di Internet, particolarmente quando è travestito da HTTP per passare attraverso i firewall. Inoltre, la consegna affidabile dei contenuti permette agli utenti di tornare indietro facilmente.

Vi è tuttavia un caso importante in cui UDP e il multicast possono essere usati per lo streaming: all'interno della rete di un provider. Per esempio, un azienda di TV via cavo può decidere di mandare in broadcast i canali televisivi ai set-top box dei clienti usando la tecnologia IP, invece che il tradizionale broadcast video. Poiché l'azienda ha il completo controllo sulla propria rete, la può progettare in modo da supportare il multicast IP e avere sufficiente banda per la distribuzione basata su UDP. Tutto ciò è invisibile al cliente, perché la tecnologia IP esiste all'interno della rete controllata dal provider. Appare come una TV via cavo in termini di servizio, ma sotto è IP, con un set-top box che fa da computer per gestire UDP e la televisione è semplicemente il monitor attaccato al computer.

Tornando al caso di Internet, lo svantaggio dello streaming live su TCP è che il server deve inviare una copia separata del contenuto a ogni client. Ciò è fattibile per un numero moderato di client, specialmente per l'audio. Il trucco è mettere un server in un posto con una buona connettività Internet in modo da avere sufficiente banda, il che usualmente significa noleggiare un server in un data center da un provider, non usare un server a casa con la sola connettività a banda larga. Quello del noleggio dei server è un mercato molto competitivo, per cui non sono richieste grosse cifre.

Infatti è semplice per chiunque, persino per uno studente, installare e usare un server multimediale come quello di una stazione radio su Internet. Le componenti principali di tale stazione sono illustrate nella Figura 7.57. Il cuore della stazione è un computer ordinario con una buona scheda sonora e un microfono. Esistono poi software molto diffusi per catturare l'audio e codificarlo in vari formati quali, per esempio, MP4; i lettori multimediali sono invece usati per l'ascolto dell'audio, come al solito.

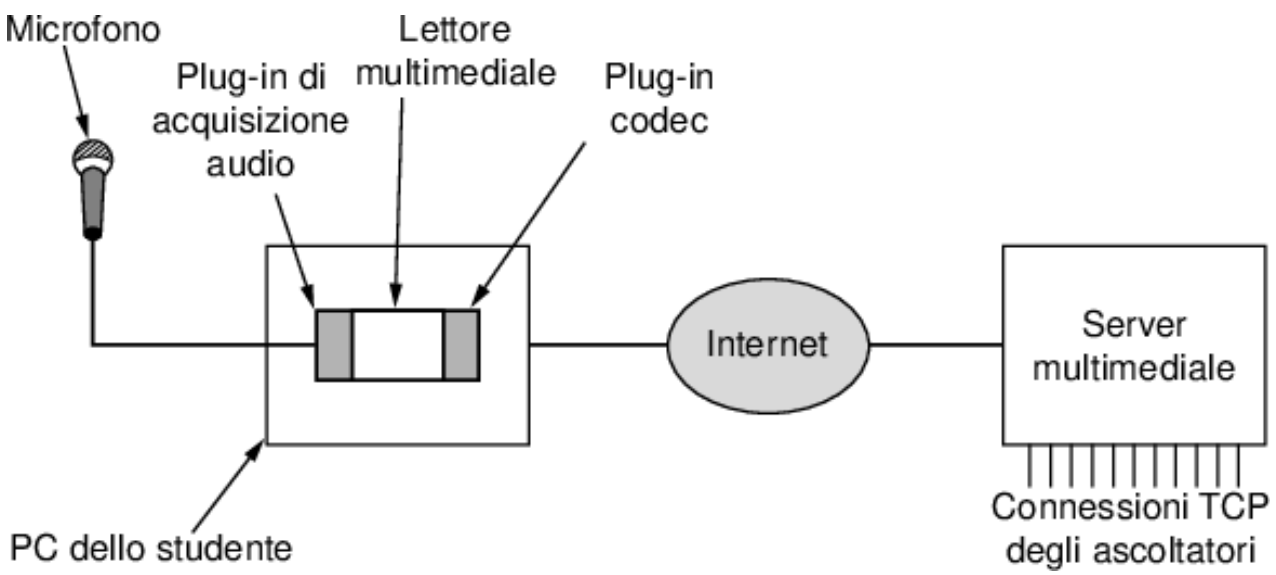


Figura 7.57 Una stazione radio per studenti.

Lo streaming audio catturato sul PC viene quindi mandato in streaming su Internet da un server multimediale con buona connettività, sia come podcast sia in tempo reale. Il server gestisce il compito di distribuire il contenuto tramite un gran numero di connessioni TCP. Presenta anche un sito Web con pagine riguardanti le stazioni e i collegamenti ai contenuti disponibili per lo streaming. Ci sono pacchetti software commerciali per gestire tutti questi pezzi come pure pacchetti open source quali icecast.

Tuttavia, in presenza di un gran numero di client, diventa impossibile usare TCP per inviare contenuti a ogni client da un singolo server. Semplicemente non c'è abbastanza banda. Per grandi siti di streaming, lo streaming viene effettuato usando un insieme di server geograficamente sparsi, in modo che un client possa connettersi al più vicino. Si tratta di una rete di distribuzione di contenuti che studieremo alla fine di questo capitolo.

Conferenze in tempo reale

Un tempo le chiamate vocali venivano fatte sulla rete telefonica pubblica e il traffico di rete era essenzialmente traffico vocale, con qualche bit di dati qua e là. Quindi sopraggiunsero Internet e il Web. Il traffico dati continuò a crescere; nel 1999 si raggiunse la parità tra i volumi di traffico dati e quello vocale (poiché anche la voce ora è digitalizzata, entrambi possono essere misurati in bit). Nel 2002 il volume del traffico dati era di un ordine di grandezza maggiore di quello del traffico vocale e ancora in crescita esponenziale, mentre quello vocale rimase fermo; la conseguenza di tale crescita è stato un completo ribaltamento delle reti telefoniche: ora il traffico vocale è trasportato per mezzo delle tecnologie di Internet e rappresenta solo una piccola frazione della banda di rete. Questa tecnologia prorompente è conosciuta come **voice over IP** (VoIP) o **telefonia Internet**.

Voice over IP è usata sotto più forme spinte da forti fattori economici. Un forma appare come un normale telefono (vecchio stile) che usa Ethernet e invia le chiamate sulla rete. Pehr Anderson era uno studente al M.I.T. quando, insieme a degli amici, realizzò il prototipo di questo progetto per un esame prendendo "B".[20] Non contento, creò nel 1996 un'azienda chiamata NBX, che vendette a 3Com per 90 milioni di dollari tre anni più tardi. Le aziende apprezzano questo approccio perché permette loro di usare linee telefoniche separate e possono farlo con le reti che hanno già.
Un altro approccio è usare la tecnologia IP per costruire reti telefoniche a lunga distanza. In paesi come gli Stati Uniti questa rete è accessibile per servizi su lunghe distanze a prezzi competitivi usando un prefisso speciale. I campioni vocali sono messi in pacchetti iniettati nella rete per poi essere estratti quando la lasciano. Poiché le apparecchiature IP sono molto meno care di quelle per le telecomunicazioni i servizi risultano più a buon mercato.

Per inciso, la differenza di prezzo non è interamente dovuta a motivi tecnici. Per molti decenni il servizio telefonico è stato un monopolio che garantiva alle compagnie telefoniche un profitto fisso in percentuale. Non ci sorprende che questo le abbia portate ad aumentare i costi, per esempio usando hardware ridondante giustificato da una migliore affidabilità (il telefono era autorizzato a non funzionare per un totale di due ore in 40 anni o in media tre minuti all'anno). Questo effetto viene spesso chiamato "sindrome del palo telefonico dorato". Dopo la liberalizzazione l'effetto è diminuito, naturalmente, anche se i vecchi apparati sono ancora attivi.

Tuttavia ci concentreremo sulla forma di VoIP più comunemente visibile agli utenti: un computer chiama un altro computer. Questa forma divenne popolare quando i PC iniziarono a essere venduti dotati di microfoni, altoparlanti, videocamere e CPU abbastanza veloci da elaborare i contenuti e quando ci si poté connettere a Internet da casa con larga banda. Un esempio molto noto è il software Skype rilasciato dal 2003. Skype e altre aziende mettono a disposizione gateway per permettere le chiamate a numeri di telefono regolari così come si fa tra computer attraverso gli indirizzi IP.

Al crescere della banda disponibile in rete le video chiamate si unirono alle chiamate vocali. Inizialmente furono appannaggio delle aziende. I sistemi di videoconferenza vennero progettati per scambiare video tra due o più postazioni, in modo da permettere alle direzioni poste in diversi uffici di guardarsi mentre facevano una riunione. Tuttavia, con l'avvento della connettività Internet a banda larga e software di compressione video, anche gli utenti domestici possono fare videoconferenze. Skype e altri, che erano partiti solo con l'audio, oggi includono normalmente il video nelle chiamate, in modo che amici e parenti si possano vedere, oltre che sentire, ovunque nel mondo.

Dal nostro punto di vista le chiamate Internet vocali o col video sono trattate come streaming di contenuti, vi sono però molti più vincoli rispetto ai file registrati o a eventi live. Uno di questi è la bassa latenza necessaria per una conversazione in due direzioni. La rete telefonica permette una latenza in una direzione di al massimo 150 ms per essere utilizzabile, oltre i quali il ritardo comincia a essere percepito come fastidioso da chi fa la chiamata. Le chiamate internazionali possono avere una latenza fino a quattrocento microsecondi, dove diventa veramente fastidioso.

Questa bassa latenza è difficile da raggiungere. Certamente un buffer di 5-10 secondi non va bene (come per il broadcasting di eventi sportivi live). Al contrario, i sistemi di video e voce su IP devono essere progettati usando una vasta gamma di tecniche per minimizzare la latenza. Questo obiettivo implica di partire con la scelta chiara di UDP invece che TCP, perché la ritrasmissione TCP introduce quanto meno un ritardo di round-trip. Tuttavia, alcune forme di latenza non possono essere ridotte neanche con UDP. Per esempio, la distanza tra Seattle e Amsterdam è di circa 8000 km. Il ritardo di propagazione della luce in una fibra ottica su questa distanza è di 40 ms. Buona fortuna se volete batterlo. In pratica, il ritardo di propagazione attraverso la rete sarà ancora più lungo, perché copre una distanza maggiore (i bit non seguono il percorso geometrico più breve) e aggiungono dei ritardi di trasmissione nei momenti in cui i router IP inoltrano un pacchetto. Questo ritardo fisso va a erodere il ritardo accettabile.

Un'altra fonte di ritardo è legata alla dimensione dei pacchetti. Normalmente, i pacchetti grandi sono il modo migliore per sfruttare la banda di rete, perché sono molto efficienti. Tuttavia, con un campionamento audio a 64 kbps, un pacchetto di 1 KB impiegherebbe 125 ms per riempirsi (e anche di più se i campioni sono compressi). Tale ritardo consumerebbe la maggior parte del ritardo ammissibile. Inoltre, se il pacchetto da 1 KB viene inviato su un collegamento a banda larga a 1 Mbps, impiegherà 8 ms per la trasmissione. Se aggiungete altri 8 ms perché il pacchetto vada sul collegamento a banda larga all'altro capo, capite che i pacchetti grandi non funzionano.

Al contrario, i sistemi VoIP usano pacchetti piccoli per ridurre la latenza al costo di ridurre l'efficienza di banda. Usano mettere campioni audio in piccole unità, di solito di 20 millisecondi. A 64 kbps questo genera 160 byte di dati, anche meno se compressi. Tuttavia, per definizione, il ritardo introdotto dalla creazione dei pacchetti sarà di 20 ms. Il ritardo di trasmissione sarà minore, perché i pacchetti sono più corti. Nel nostro esempio si ridurrà di circa 1 ms. Usando pacchetti piccoli, il ritardo minimo unidirezionale per un pacchetto da Seattle ad Amsterdam viene ridotto da un non accettabile 181 ms (40 + 125 + 16) a un accettabile 62 ms (40 + 20 + 2).

Non abbiamo ancora parlato dell'overhead introdotto dal software, ma anch'esso incide sul ritardo totale. Ciò vale specialmente per il video, perché la compressione è necessaria per adeguare i video alla banda disponibile. Al contrario dello streaming da un file registrato, non c'è il tempo di fare una codifica computazionalmente pesante con alti livelli di compressione. Sia il codificatore che il decodificatore devono agire velocemente.

Il buffer è ancora necessario per riprodurre i campioni multimediali in tempo (per evitare audio non udibile o video a scatti); la quantità di buffer utilizzata deve essere mantenuta bassa, perché il tempo rimanente nel nostro ritardo ammissibile è misurato in millisecondi. Quando un pacchetto impiega troppo ad arrivare, il lettore salterà i campioni andati persi, magari riproducendo un rumore ambientale o ripetendo un frame per mascherare la perdita all'utente. Esiste un

compromesso tra la dimensione del buffer usato per gestire il jitter e la quantità di contenuto perso. Un buffer piccolo riduce la latenza, ma genera più perdite dovute al jitter. Prima o poi, con la diminuzione della dimensione del buffer, la perdita diventerà evidente per l'utente.

I lettori attenti dovrebbero avere notato che non abbiamo parlato dei protocolli a livello di rete in questo paragrafo. Usando meccanismi di qualità di servizio la rete può ridurre la latenza o almeno il jitter. La ragione per cui questo problema non è ancora sorto è che lo streaming è in grado di operare anche con una latenza significativa anche nel caso in cui sia in tempo reale. La latenza non è il problema principale; è sufficiente un buffer lato ricevente per gestire il problema del jitter. Tuttavia, per le conferenze in tempo reale, è di solito importante avere una rete che riduca il ritardo e il jitter per permettere di rispettare il ritardo massimo ammissibile. L'unico momento in cui non è importante è quando la larghezza di banda di rete è tale per cui ognuno ottiene un buon servizio.

Nel Capitolo 5 abbiamo descritto due utili meccanismi di qualità del servizio. Uno consiste nei servizi differenziati, nei quali i pacchetti vengono marcati come appartenenti a classi distinte gestite in modo diverso dalla rete. L'etichetta appropriata per i pacchetti VoIP è "basso ritardo" (*low latency*). In pratica i sistemi definiscono un tipo di servizio a basso ritardo nella classe *expedited forwarding*. Ciò è utile specialmente nei collegamenti a banda larga per l'accesso a Internet, che tendono a essere congestionati quando il traffico Web o di altro tipo sono in competizione per l'uso del collegamento. Dato un percorso stabile di rete, il ritardo e il jitter aumentano con la congestione. Ogni pacchetto di 1 KB impiega 8 ms per essere inviato su un collegamento di 1 Mbps e un pacchetto VoIP sarà soggetto a questi ritardi se sta in coda dietro al traffico Web. Tuttavia, con un'etichetta di basso ritardo i pacchetti VoIP salteranno in cima alla coda superando i pacchetti Web e abbassando il loro ritardo.

Il secondo meccanismo che può ridurre il ritardo è assicurare che ci sia banda sufficiente. Se essa varia o il tasso di trasmissione fluttua come nei video compressi e a volte non c'è banda sufficiente si creano code che portano ritardi; questo accade anche nei servizi differenziati. Per assicurare banda sufficiente bisogna effettuare una prenotazione delle risorse in rete. Questa possibilità è fornita dai servizi integrati che sfortunatamente non sono molto implementati. Al contrario, le reti sono progettate per un livello di traffico programmato oppure ai clienti vengono proposti degli accordi sul livello di servizio per un dato livello di traffico. Per evitare le congestioni e introdurre ritardi le applicazioni devono operare sotto questo livello. Per le videoconferenze domestiche l'utente deve scegliere una qualità video come rappresentativa per la richiesta di banda o il software deve esaminare il percorso di rete e selezionare automaticamente una qualità appropriata.

Ognuno dei fattori descritti sopra può causare una latenza non accettabile; quindi le conferenze in tempo reale richiedono di porre grande attenzione a ognuno di essi. Per un'analisi di questi fattori si veda Goode (2002).

Ora che abbiamo discusso il problema della latenza nello streaming multimediale, trattiamo un altro problema importante che i sistemi di conferenza devono affrontare: come aprire e chiudere le chiamate. Vedremo due protocolli molto usati a questo scopo: H.323 e SIP. Skype è un altro caso importante, ma il suo funzionamento interno non è noto.

H.323

Una cosa che fu chiara a tutti fin dall'inizio fu il fatto che, se ogni produttore avesse progettato la propria pila di protocolli, il sistema non avrebbe mai funzionato. Per evitare questo problema, alcune delle parti interessate si unirono sotto gli auspici di ITU per elaborare degli standard. Nel 1996 ITU rilasciò la raccomandazione **H.323** intitolata "Sistemi telefonici visivi e attrezzature per reti locali che offrono una qualità di servizio senza garanzie" (*Visual telephone systems and equipment for local area networks which provide a non-guaranteed quality of service*). Solo l'industria telefonica avrebbe potuto pensare a un nome del genere che venne rivisto nel 1998 in "Sistemi di comunicazione multimediali a pacchetto" (*packet-based multimedia communication systems*); H.323 divenne così la base per i primi sistemi di conferenza largamente diffusi su Internet e nel 2009, alla settima edizione, è ancora la soluzione più installata.

H.323 non descrive un vero e proprio protocollo, ma rappresenta più che altro una panoramica architetturale della telefonia Internet. Anziché emettere specifiche proprie fa riferimento a diversi protocolli specializzati per la codifica del parlato, l'impostazione delle chiamate, la segnalazione, il trasporto di dati e altre aree. Il modello generale è mostrato nella Figura 7.58. Al centro si trova il **gateway** che connette Internet al sistema telefonico. Utilizza i protocolli H.323 sul lato Internet e i protocolli PSTN sul lato telefonico. I dispositivi di comunicazione sono chiamati **terminali**. Una LAN può disporre di un **gatekeeper**, che controlla i punti finali sotto la sua giurisdizione, chiamata **zona**.

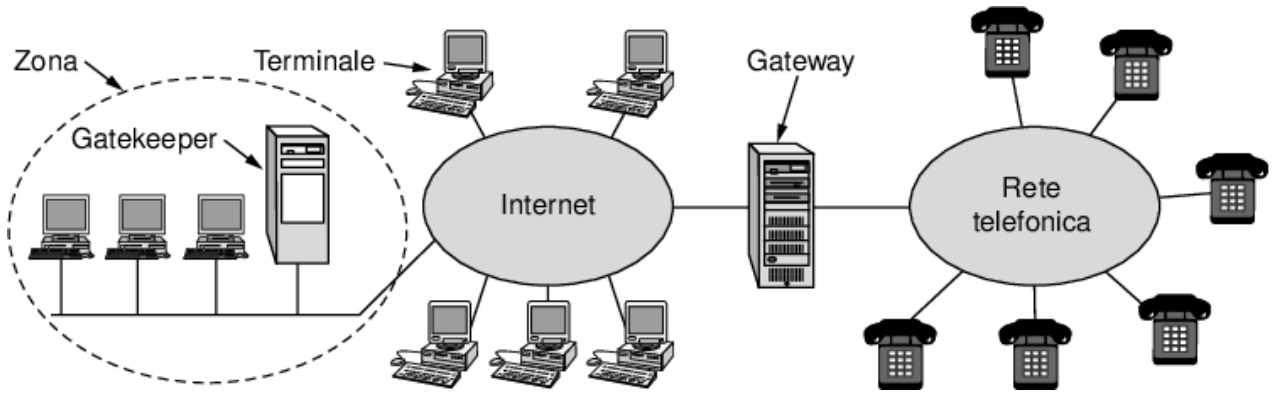


Figura 7.58 Il modello architetturale H.323 per la telefonia Internet.

Una rete telefonica richiede diversi protocolli. Innanzitutto esiste un protocollo per la codifica e la decodifica di audio e video. Il sistema PCM studiato nel Capitolo 2 è definito nella raccomandazione ITU **G.711**. Codifica un singolo canale vocale eseguendo il campionamento 8000 volte al secondo con un campione di 8 bit, per fornire parlato non compresso a 64 kbps. Tutti i sistemi H.323 devono supportare G.711, ma sono consentiti (non obbligatori) anche altri protocolli di compressione vocale che utilizzano algoritmi diversi, presentando compromessi differenti tra la qualità e la banda. Per il video sono supportate le forme di MPEG descritte prima, incluso H.264.

Visto che sono permessi più algoritmi di compressione è necessario un protocollo per dare ai terminali la possibilità di negoziarne l'utilizzo. Questo protocollo è chiamato **H.245** e negozia anche altri aspetti della connessione, come il tasso di invio delle informazioni. RTCP è necessario per il controllo dei canali RTP. È inoltre richiesto un protocollo per stabilire e rilasciare le connessioni, fornire i toni di composizione, produrre gli squilli e occuparsi di altri aspetti della telefonia standard: in questa area si usa ITU **Q.931**. I terminali necessitano di un protocollo per comunicare con il gatekeeper, se è presente: a questo scopo viene utilizzato **H.225**. Il canale PC-gatekeeper che gestisce è chiamato canale **RAS** *(registration/admission/status*, registrazione/ammissione/stato), che consente ai terminali di entrare nella zona e di lasciarla, di richiedere e restituire banda e fornire aggiornamenti sullo stato del sistema. Infine è necessario un protocollo per la trasmissione dei dati vera e propria: è stato stabilito di usare RTP, gestito come al solito da RTCP. La collocazione di tutti questi protocolli è mostrata nella Figura 7.59.

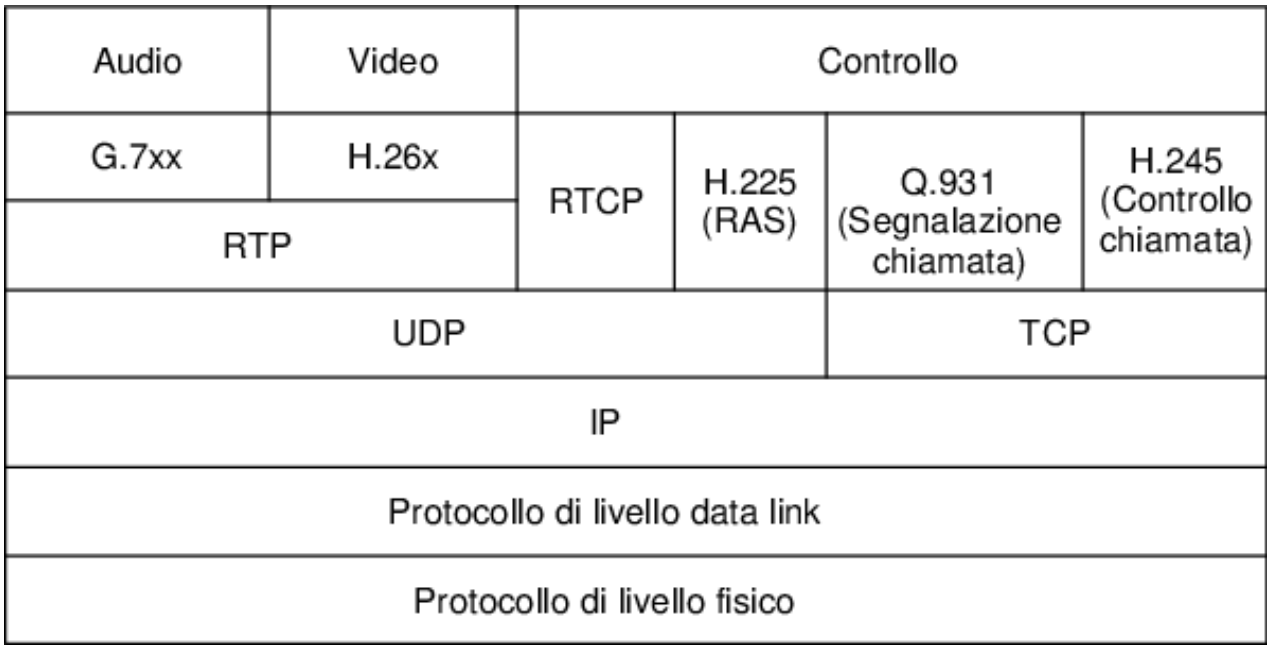


Figura 7.59 Lo stack di protocolli di H.323.

Per vedere come questi protocolli operano insieme, si consideri il caso di un terminale PC su una LAN (con un gatekeeper) che chiama un telefono remoto. Il PC deve prima scoprire il gatekeeper, pertanto trasmette in broadcast un pacchetto UDP di rilevamento del gatekeeper sulla porta 1718.

Quando il gatekeeper risponde, il PC apprende il relativo indirizzo IP. Ora il PC si registra presso il gatekeeper inviando un messaggio RAS in un pacchetto UDP. Dopo l'accettazione il PC invia al gatekeeper un messaggio di ammissione RAS richiedendo un po' di banda. Solo dopo che la banda è stata garantita può iniziare l'impostazione della chiamata. La richiesta anticipata della banda consente al gatekeeper di limitare il numero di chiamate per impedire che la linea in uscita sia sovrautilizzata e quindi fornire la qualità del servizio necessaria.

Per inciso, il sistema telefonico agisce allo stesso modo. Quando sollevate il ricevitore viene inviato un segnale alla centrale locale che, se ha ancora capacità, genera un segnale di linea libera, altrimenti non sentite niente. Oggigiorno il sistema è così sovradimensionato che il segnale è istantaneo, ma all'inizio richiedeva qualche secondo. Ora saprete rispondere ai vostri nipotini quando vi chiederanno perché esiste il segnale per indicare che la linea è libera, ma probabilmente i telefoni non esisteranno più.

Il PC ora stabilisce una connessione TCP al gatekeeper per iniziare l'impostazione della chiamata; tale operazione utilizza i protocolli della rete telefonica esistenti, che sono orientati alla connessione, pertanto è necessario TCP. Il sistema telefonico non dispone di qualcosa di simile a RAS per consentire ai telefoni di annunciare la loro presenza, pertanto i progettisti di H.323 sono stati liberi di utilizzare per RAS UDP o TCP: hanno così scelto UDP, a causa del suo overhead inferiore.

Dopo l'allocazione della banda il PC può inviare un messaggio Q.931 *SETUP* sulla connessione TCP. Questo messaggio specifica il numero del telefono da chiamare (o l'indirizzo IP e la porta, se occorre chiamare un computer). Il gatekeeper risponde con un messaggio Q.931 *CALL PROCEEDING* per confermare la corretta ricezione della richiesta. Il gatekeeper inoltra quindi il messaggio *SETUP* al gateway.

Il gateway, che per metà è un computer e per metà un permutatore telefonico, esegue una telefonata ordinaria al telefono desiderato. Il telefono chiamato squilla e invia un messaggio Q.931 *ALERT* per comunicare al PC chiamante che il telefono ha iniziato a squillare. Quando la persona all'altro capo del filo alza la cornetta, la centrale locale restituisce un messaggio Q.931 *CONNECT* per segnalare al PC che possiede la connessione. Una volta stabilita la connessione il gatekeeper non si trova più nel ciclo (al contrario del gateway, naturalmente). I pacchetti successivi scavalcano il gatekeeper e raggiungono direttamente l'indirizzo IP del gateway. A questo punto si dispone di un semplice tubo tra due parti: si tratta solamente di una connessione a livello fisico per lo spostamento dei bit. Ogni lato non conosce nulla dell'altro.

Il protocollo H.245 è ora utilizzato per negoziare i parametri della chiamata. Utilizza il canale di controllo H.245, che è sempre aperto. Ogni lato inizia annunciando le proprie capacità, per esempio se è in grado di gestire il video (H.323 può gestire il video) o le conference call, quali codec supporta e così via. Quando ogni lato conosce ciò che l'altro può gestire vengono impostati due canali dati unidirezionali, assegnando un codec e altri parametri a ognuno. Dal momento che ogni lato può avere attrezzature diverse è possibile che i codec nei canali diretto e inverso siano differenti. Una volta completate tutte le negoziazioni, può iniziare il flusso dei dati che utilizza RTP. È gestito tramite RTCP, che svolge un ruolo importante nel controllo delle congestioni. Se è presente il video, RTCP gestisce la sincronizzazione audio/video. I canali sono mostrati nella Figura 7.60. Quando le parti riagganciano, il canale di segnalazione della chiamata Q.931 viene utilizzato per abbattere la connessione in modo da liberare risorse.

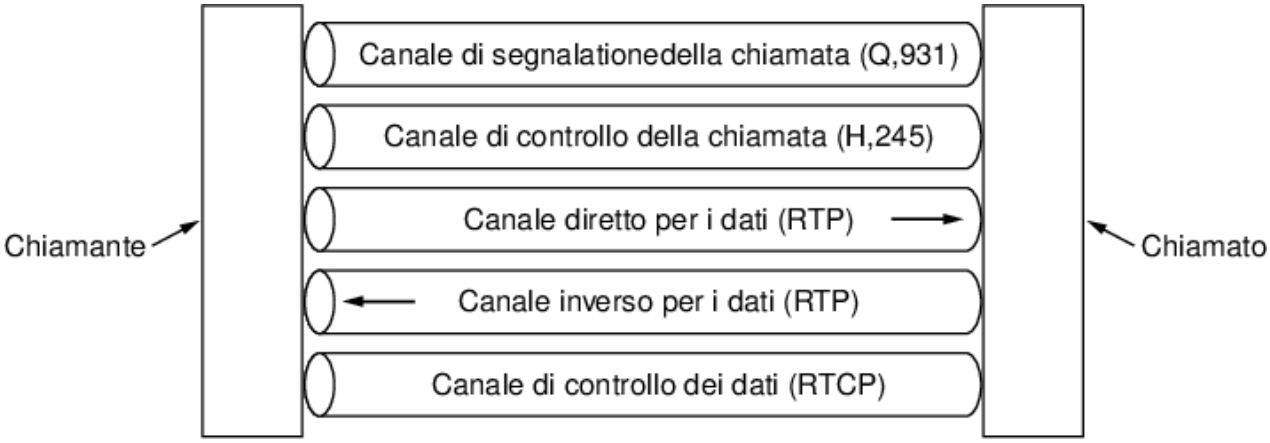


Figura 7.60 I canali logici tra chiamante e chiamato durante una chiamata.

Una volta terminata la chiamata, il PC chiamante contatta di nuovo il gatekeeper con un messaggio RAS che chiede di rilasciare la banda assegnata. In alternativa, può eseguire un'altra chiamata.

Non abbiamo trattato la qualità del servizio come parte di H.323, anche se è un aspetto essenziale per il successo delle conferenze in tempo reale. Il motivo è che la qualità di servizio ricade all'esterno dell'ambito di H.323. Se la rete sottostante è in grado di produrre una connessione stabile e priva di jitter dal PC chiamante al gateway, allora la qualità della chiamata sarà buona; in caso contrario sarà scadente. La parte telefonica è sempre priva di jitter.

SIP – session initiation protocol

H.323 è stato progettato da ITU. Molti appartenenti alla comunità Internet lo hanno considerato come un tipico prodotto per le telecomunicazioni: grande, complesso e poco flessibile. Di conseguenza, IETF ha creato un comitato per progettare un modo più semplice e modulare per gestire il VoIP. Il miglior risultato a oggi è **SIP** (*session initiation protocol*), descritto nell'RFC 3261. Questo protocollo descrive come impostare le telefonate su Internet, le conference call e altre connessioni multimediali. A differenza di H.323, che è una suite di protocolli completa, SIP è un modulo singolo progettato per lavorare con applicazioni Internet esistenti. Per esempio, definisce i numeri telefonici come URL, in modo che possano essere contenuti nelle pagine, consentendo di fare click su un collegamento per avviare una telefonata. È un modo di procedere simile allo schema di accesso *mailto* di un URL, che consente di fare click su un collegamento per aprire il programma per l'invio di posta elettronica.

SIP può stabilire sessioni tra due partecipanti (telefonate ordinarie), sessioni tra più partecipanti (dove chiunque può ascoltare e parlare), nonché sessioni multicast (un mittente, molti destinatari). Le sessioni possono contenere audio, video o dati; questi ultimi servono (per esempio) per giochi multiutente in tempo reale. SIP gestisce solamente l'impostazione, la gestione e la terminazione delle sessioni. Altri protocolli, come RTP/RTCP, sono utilizzati per il trasporto dei dati. SIP è un protocollo del livello applicazione e può essere implementato su UDP o TCP, secondo necessità.

SIP supporta numerosi servizi, tra cui l'individuazione del chiamato (che potrebbe non trovarsi sul computer della propria abitazione) e la determinazione delle capacità del chiamato, nonché la gestione dei meccanismi di impostazione e termine della chiamata. Nel caso più semplice, SIP imposta una sessione dal computer del chiamante al computer del chiamato, quindi esamineremo per primo questo caso.

In SIP i numeri telefonici sono rappresentati come URL utilizzando lo schema *sip,* per esempio *sip:ilse@cs.university.edu* per un utente chiamato Ilse all'host specificato dal nome DNS *cs.university.edu.* Gli URL SIP possono anche contenere indirizzi IPv4, indirizzi IPv6 o numeri di telefono veri e propri.

Il protocollo SIP è un protocollo di testo modellato su HTTP. Una parte invia un messaggio di testo ASCII costituito da un nome di metodo sulla prima riga, seguito da altre righe con le intestazioni per il passaggio dei parametri. Molte intestazioni sono prese da MIME, per consentire a SIP di funzionare con applicazioni esistenti. I sei metodi definiti dalle specifiche sono elencati nella Figura 7.61.

Metodo	Descrizione
INVITE	Richiede l'inizio di una sessione
ACK	Conferma l'inizio di una sessione
BYE	Richiede la fine di una sessione
OPTIONS	Interroga un host sulle sue funzionalità
CANCEL	Annulla una richiesta in sospeso
REGISTER	Informa un server di reindirizzamento sulla posizione corrente dell'utente

Figura 7.61 Metodi SIP.

Per stabilire una sessione il chiamante crea una connessione TCP al chiamato e invia un messaggio *INVITE* su di essa oppure invia il messaggio *INVITE* in un pacchetto UDP. In entrambi i casi, le intestazioni della seconda riga e di quelle successive descrivono la struttura del corpo del messaggio, che contiene le capacità del chiamante, i tipi di dati multimediali e i formati. Se il chiamato accetta la chiamata, risponde con un codice di risposta HTTP (un numero di tre cifre in base ai gruppi della Figura 7.38, 200 per l'accettazione). Dopo la riga con il codice di risposta, il chiamato può fornire informazioni sulle sue capacità, sui tipi di dati e sui formati.

La connessione avviene con un handshake a tre vie, pertanto il chiamante risponde con un messaggio *ACK* per terminare la sequenza e confermare la ricezione del messaggio 200.

Entrambe le parti possono richiedere la fine di una sessione inviando un messaggio contenente il metodo *BYE*. Quando l'altro lato lo riconosce, la sessione viene terminata. Il metodo *OPTIONS* è utilizzato per interrogare una macchina sulle sue capacità. È generalmente utilizzato prima dell'impostazione di una sessione per scoprire se la macchina sia in grado di utilizzare VoIP o per capire quali tipi di sessione sono possibili.

Il metodo *REGISTER* è correlato alla capacità di SIP di tenere traccia di un utente lontano da casa e di collegarsi a lui. Questo messaggio viene inviato a un location server SIP, che tiene traccia della posizione di ogni utente. Il server può essere successivamente interrogato per trovare la posizione corrente dell'utente. L'operazione di reindirizzamento è mostrata nella Figura 7.62. Qui il chiamante invia il messaggio *INVITE* a un server proxy, per nascondere il possibile reindirizzamento. Il proxy cerca l'utente e invia il messaggio *INVITE* verso la posizione individuata, poi agisce come un ripetitore per i successivi messaggi nell'handshake a tre vie. I messaggi *LOOKUP* e *REPLY* non sono parte di SIP: è possibile utilizzare qualsiasi protocollo adatto, in base al tipo di location server utilizzato. SIP ha molte altre caratteristiche che non sono qui descritte, come l'avviso di chiamata, la selezione delle chiamate, la crittografia e l'autenticazione. Ha inoltre la capacità di eseguire chiamate da un computer a un telefono tradizionale, se è disponibile un gateway adatto tra Internet e il sistema telefonico.

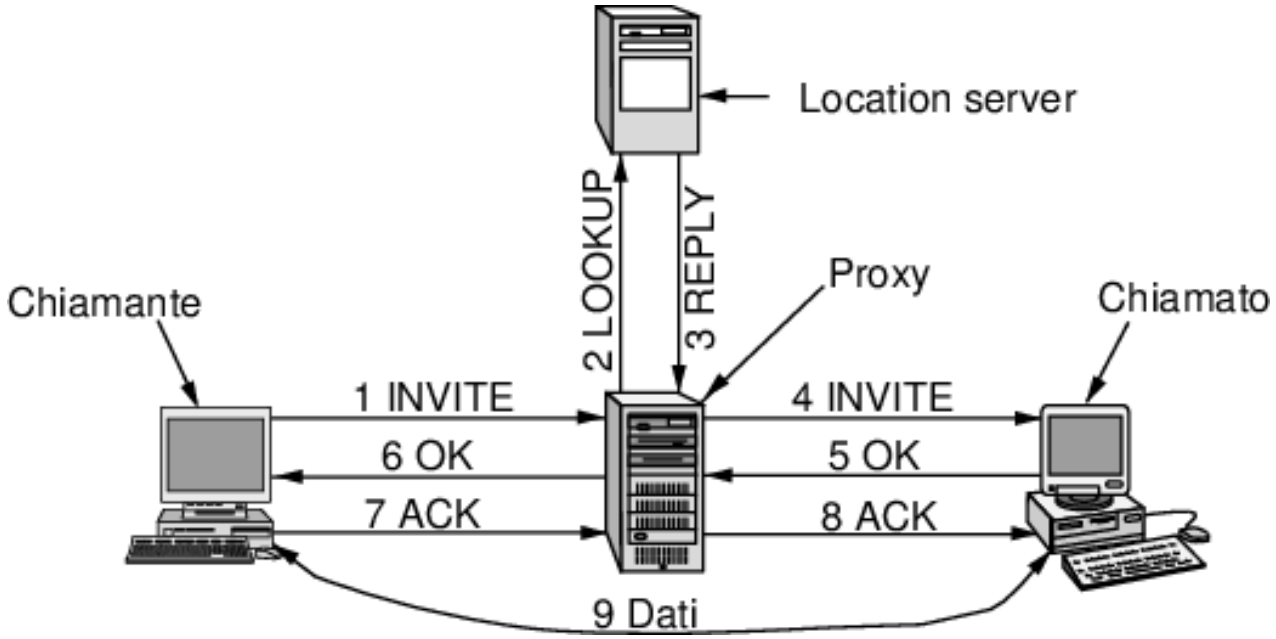


Figura 7.62 Utilizzo di un proxy server e reindirizzamento con SIP.

Confronto tra H.323 e SIP

H.323 e SIP presentano molte somiglianze, ma anche alcune differenze. Entrambi consentono chiamate a due o più partecipanti, utilizzando come punti finali computer e telefoni. Entrambi supportano la negoziazione dei parametri, la crittografia e i protocolli RTP/RTCP. Un riepilogo delle somiglianze e delle differenze è mostrato nella Figura 7.63.

Elemento	H.323	SIP
Progettato da	ITU	IETF
Compatibilità con PSTN	Sì	In gran parte
Compatibilità con Internet	Si, nel tempo	Sì
Architettura	Monolitica	Modulare
Completezza	Pila di protocolli completa	SIP gestisce solo l'impostazione
Negoziazione dei parametri	Sì	Sì
Segnalazione della chiamata	Q.931 su TCP	SIP su TCP o UDP
Formato dei messaggi	Binari	ASCII
Trasporto	RTP/RTCP	RTP/RTCP
Chiamate di gruppo	Sì	Sì
Conferenze multimediali	Sì	No
Indirizzamento	URL o numero di telefono	URL
Termine della chiamata	Esplicito o rilascio di TCP	Esplicito o timeout
Messaggistica istantanea	No	Sì
Crittografia	Sì	Sì
Dimensione degli standard	1400 pagine	250 pagine
Implementazione	Ampia e complessa	Moderata, ma con problemi
Stato	Ampiamente utilizzato, specialmente video	Usato come alternativa, specialmente voce

Figura 7.63 Confronto tra H.323 e SIP.

Anche se l'insieme delle funzionalità è simile, i due protocolli differiscono notevolmente nella filosofia. H.323 è un tipico standard dell'industria telefonica, che specifica tutta la pila di protocolli e definisce con precisione che cosa è consentito e che cosa è negato. Tale approccio tende a generare protocolli ben definiti in ogni livello, facilitando l'interoperabilità. Il prezzo da pagare è uno standard grande, rigido e complesso, difficile da adattare alle applicazioni future.

In contrasto SIP è un tipico protocollo Internet basato sullo scambio di brevi righe di testo ASCII. È un modulo leggero che interopera bene con altri protocolli Internet, ma meno bene con i protocolli di segnalazione del sistema telefonico esistente. Visto che il modello IETF di VoIP è altamente modulare, risulta flessibile e può essere adattato facilmente alle nuove applicazioni. Lo svantaggio è rappresentato da potenziali problemi di interoperabilità, quando gli implementatori tentano di interpretare che cosa significa lo standard.

7.5 Distribuzione di contenuti

Come la rete telefonica, Internet era storicamente basata unicamente sulle comunicazioni. Nei primi anni gli accademici comunicavano con macchine remote facendo login tramite la rete per effettuare delle attività. Le persone hanno usato le e-mail per comunicare tra di loro per un lungo periodo e adesso usano il video e la voce su IP nello stesso modo. Tuttavia, con la crescita del Web, Internet ha cominciato a riguardare più i contenuti della comunicazione. Molte persone usano il Web per reperire informazioni e c'è un'incredibile attività di condivisione di file peer-to-peer per poter accedere a film, musica e programmi. Il cambio verso i contenuti è stato così evidente che ormai la maggior parte della banda Internet è usata per trasmettere video registrati.

Poiché il compito di distribuire contenuti si distingue dalla comunicazione, esso richiede requisiti diversi sulla rete. Per esempio, se Sally vuole parlare con Jitu, può fare una chiamata VoIP al suo cellulare. La comunicazione deve avvenire con quel particolare computer, non con il PC di Paul. Ma se Jitu vuole guardare l'ultima partita della sua squadra di cricket sarà contento di fare streaming video da qualunque computer che glielo possa fornire. Non gli importa se il computer è di Sally o di Paul o, più probabilmente, di un server sconosciuto su Internet. Cioè, la posizione non importa per il contenuto, a meno che influisca sulle prestazioni (e sulla legalità).

L'altra differenza è che alcuni siti Web che forniscono contenuti sono diventati molto popolari. YouTube ne è un esempio. Permette agli utenti di condividere video di loro creazione su ogni possibile argomento e molte persone vogliono farlo, mentre il resto di noi vuole vederli. Con tutti questi video che consumano banda, si stima che YouTube oggigiorno occupi fino al 10% del traffico di Internet. Non esiste un singolo server così potente e affidabile da gestire una richiesta di questo livello; invece, YouTube e altri fornitori di contenuti costruiscono le proprie reti di distribuzione di contenuti. Queste reti usano data center sparsi in tutto il mondo per fornire contenuti a un numero estremamente grande di client con buone prestazioni e affidabilità.

Le tecniche usate per la distribuzione di contenuti sono state sviluppate nel tempo. All'inizio della crescita del Web la sua popolarità fu quasi la sua rovina. La crescente domanda di contenuti portò server e reti a essere spesso sovraccarichi. Molte persone cominciarono a chiamare il WWW "word wide wait" (letteralmente, "attesa globale").

In risposta alle richieste dei consumatori, una gran quantità di banda venne fornita nel core di Internet e connettività a larga banda più veloce ai confini della rete. Questa banda fu fondamentale per aumentare le prestazioni, ma è solo una parte della soluzione. Per ridurre il ritardo infinito i ricercatori svilupparono anche nuove architetture in modo tale da usare la banda per la distribuzione di contenuti.

Una di queste architetture è una **CDN** (*content distribution network*), nella quale un provider installa un insieme di macchine distribuito in diversi luoghi di Internet e li usa per distribuire contenuti ai client. Questa è la scelta delle grandi aziende. Un'architettura alternativa è una rete **P2P** (*peer-to-peer*), nella quale un insieme di computer usa le proprie risorse per distribuire contenuti al suo interno senza server separati o punti centrali di controllo. L'idea ha catturato l'immaginazione delle persone perché, agendo insieme, molti piccoli giocatori possono avere una grande energia.

In questo paragrafo affronteremo il problema di distribuire contenuti su Internet e vedremo alcune soluzioni usate nella pratica. Dopo una breve analisi della popolarità dei contenuti e il traffico Internet descriveremo come costruire server Web potenti e usare una cache per aumentare le prestazioni per il client Web. Descriveremo quindi le due principali architetture per la distribuzione di contenuti: le reti CDN e P2P che, come vedremo, hanno progettazioni e proprietà molto diverse.

Contenuti e traffico Internet

Per progettare e ingegnerizzare reti che funzionano bene abbiamo bisogno di capire il traffico che devono trasportare. Con il passaggio al contenuto, per esempio, i server sono passati dagli uffici delle aziende ai data center su Internet che forniscono un gran numero di macchine con eccellente connettività di rete. Oggigiorno, per usare anche un piccolo server, è più facile e meno costoso affittarne uno virtuale ospitato in un data center che gestire una macchina vera in casa o in ufficio con una connettività a banda larga.

Fortunatamente sono solo due gli aspetti riguardanti il traffico Internet che è essenziale conoscere. Il primo è il rapido cambiamento, non solo nei dettagli ma anche nell'aspetto complessivo. Prima del 1994 la maggior parte del traffico era il tradizionale trasferimento file via FTP (per spostare programmi e dati tra computer) e le e-mail. Poi arrivò il Web che crebbe esponenzialmente. Il traffico Web fece mangiare la polvere al traffico di FTP e posta elettronica molto prima della bolla del dot com nel 2000. Da allora la condivisione di file P2P per musica e film prese il volo. Entro il 2003 la maggior parte del traffico Internet era traffico P2P, lasciando il Web nella polvere. A un certo momento, alla fine del 2000, i video in streaming da siti come YouTube, che utilizzano metodi di distribuzione dei contenuti, hanno cominciato a superare il traffico P2P. Entro il 2014, Cisco prevede che il 90% di tutto il traffico Internet sarà video in una forma o nell'altra (Cisco, 2010).

Non è sempre il volume di traffico che conta. Per esempio, anche se il traffico VoIP esplose prima ancora di Skype, iniziato nel 2003, sarà sempre un'inezia, perché i requisiti di larghezza di banda audio sono due ordini di grandezza inferiore a quelli del video. Tuttavia, il traffico VoIP stressa la rete in altri modi, perché è sensibile alla latenza. Come altro esempio, le reti sociali on-line sono cresciute enormemente da quando Facebook è partito nel 2004. Nel 2010, per la prima volta, Facebook ha raggiunto più utenti giornalieri sul Web di Google. Anche mettendo da parte il traffico (e c'è un'enorme quantità di traffico), le reti sociali on-line sono importanti perché stanno cambiando il modo in cui le persone interagiscono via Internet.

Quello che vogliamo arrivare a dire è che i cambiamenti di grandi proporzioni del traffico Internet accadono velocemente e con una certa regolarità. Che cosa avverrà dopo? Controllate la sesta edizione di questo libro e ve lo faremo sapere.

Il secondo fatto essenziale sul traffico di Internet è che è molto asimmetrico. Molte caratteristiche con le quali siamo abituati a trattare sono raggruppate attorno a una media. Ad esempio, la maggior parte degli adulti è vicina all'altezza media della popolazione. Ci sono alcune persone alte e alcune persone basse, ma poche persone molto alte o molto basse. Per questo tipo di caratteristiche è possibile fare progetti per un intervallo che non è molto grande, ma raccoglie tuttavia la maggioranza della popolazione.

Il traffico Internet non è così. Per lungo tempo si è saputo che c'è un piccolo numero di siti Web con un traffico massiccio e un gran numero di siti Web con un traffico molto più esiguo. Questa caratteristica è diventata parte del linguaggio delle reti. I primi documenti parlavano di traffico in termini di **treni di pacchetti**; l'idea è che i treni espressi con un gran numero di pacchetti in viaggio attraversino improvvisamente un collegamento (Jain e Routhier, 1986). Questo è stato formalizzato con un concetto di **auto-similarità** (*self-similarity*), che per i nostri scopi può essere pensato come il traffico di rete con molti vuoti brevi e lunghi, anche se osservati su diverse scale temporali (Leland et al., 1994). I lavori successivi parlarono di lunghi flussi di traffico come **elefanti** e di brevi flussi di traffico come **topi**. L'idea è che ci sono solo pochi elefanti e molti topi, ma gli elefanti contano perché sono molto grandi.

Tornando al contenuto del Web, lo stesso tipo di asimmetria è evidente. L'esperienza con negozi di video noleggio, biblioteche pubbliche e altre organizzazioni mostra come non tutti gli articoli siano ugualmente popolari. Sperimentalmente, quando sono disponibili N film, la frazione di tutte le richieste per il k-esimo più popolare è di circa C/k, dove C è una costante di normalizzazione perché la somma dia 1, vale a dire:

$$C = 1/(1 + 1/2 + 1/3 + 1/4 + 1/5 + ... + 1/N)$$

Così, il film più popolare lo è sette volte di più del film numero sette. Questo risultato è noto come **legge di Zipf** (Zipf, 1949). Prende il nome da George Zipf, un professore di linguistica all'Università di Harvard che ha notato che la frequenza di utilizzo di una parola in un testo molto lungo è inversamente proporzionale al suo rango. Per esempio, la quarantesima parola più comune è usata il doppio della ottantesima e tre volte la centoventunesima.

Una distribuzione di Zipf è mostrata nella Figura 7.64 (a). Cattura l'idea che vi sia un piccolo numero di elementi popolari e molti elementi impopolari. Per riconoscere distribuzioni di questa forma è conveniente tracciare i dati su scala logaritmica su entrambi gli assi, come mostrato nella Figura 7.64 (b). Il risultato dovrebbe essere una linea retta.

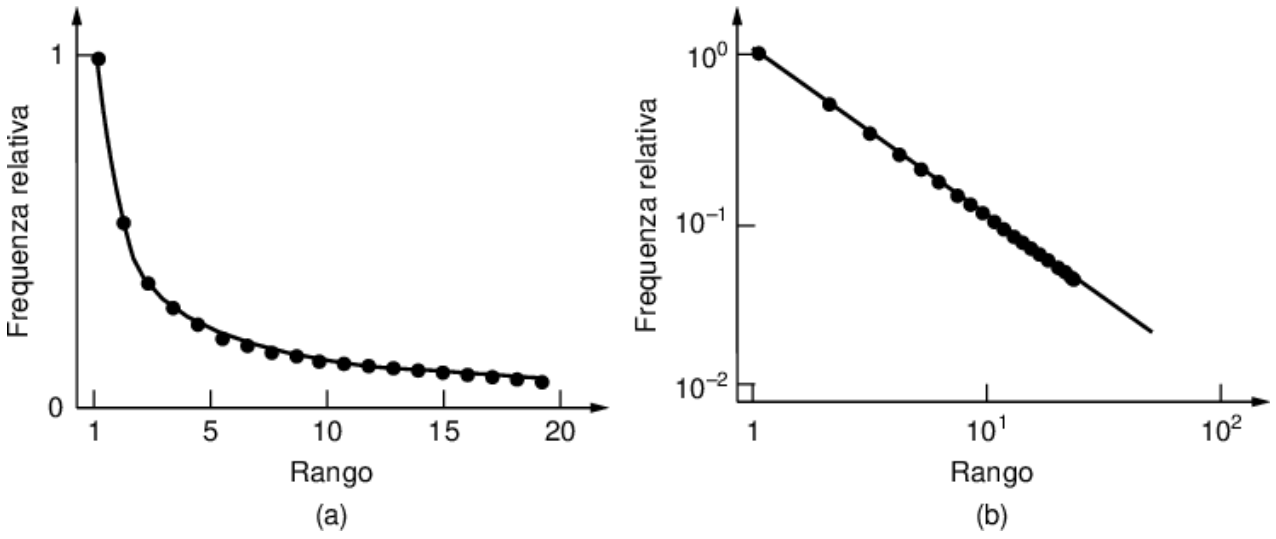


Figura 7.64 Distribuzione di Zipf (a) Su una scala lineare. (b) Su una scala log-log.

Quando la gente osservò la popolarità delle pagine Web si accorse che seguiva all'incirca la legge di Zipf (Breslau et al., 1999). Una distribuzione di Zipf è un esempio di distribuzione della famiglia di distribuzioni note come **power law** (*leggi di potenza*). Le leggi di potenza sono evidenti in molti fenomeni umani, come la distribuzione delle popolazioni delle città e della ricchezza. Esse hanno la stessa attitudine a descrivere pochi attori di grandi dimensioni e molti attori più piccoli e anche loro appaiono come una linea retta su un grafico bilogaritmico (log-log). Fu presto scoperto che la topologia di Internet potrebbero essere più o meno descritta con una power law (Faloutsos et al., 1999). Successivamente, i ricercatori cominciarono a tracciare ogni proprietà immaginabile di Internet su scala logaritmica, osservando una linea retta e gridando: "Power law!".

Tuttavia, ciò che conta più della mera linea retta su un grafico log-log è quello che queste distribuzioni implicano per la progettazione e l'uso delle reti. Dati i molti tipi di contenuti che hanno distribuzioni di Zipf o power law, sembra fondamentale che i siti Web su Internet abbiano popolarità distribuita secondo una distribuzione di Zipf. Questo, a sua volta, significa che un sito medio non è una rappresentazione utile. I siti sono meglio descritti come popolari o non popolari. Entrambi sono importanti. I siti popolari ovviamente contano, dal momento che pochi siti popolari possono essere responsabili di gran parte del traffico su Internet. Forse sorprendentemente, anche i siti non popolari contano. Questo perché l'ammontare totale del traffico diretto ai siti non popolari può raggiungere anche una grande frazione del traffico totale. Il motivo è che ci sono tanti siti impopolari. La nozione che, collettivamente, molte scelte non popolari possano essere rilevanti è stata pubblicizzata da libri come *The Long Tail* (Anderson, 2008a).

Le curve che presentano un decadimento come quello nella Figura 7.64(a) sono comuni, ma non sono tutte uguali. In particolare, le situazioni in cui il tasso di decadimento è proporzionale a quanto materiale è rimasto (come gli atomi con instabilità radioattiva), mostrano un decadimento esponenziale che cade molto più velocemente della legge di Zipf. Il numero di oggetti, per esempio atomi, non decaduti dopo il tempo t è generalmente espressa come $e^{-t/a}$, dove la costante

a determina la velocità di decadimento. La differenza tra decadimento esponenziale e legge di Zipf è che con un decadimento esponenziale si può tranquillamente ignorare la fine della coda, mentre con la legge di Zipf il peso totale della coda è importante e non può essere ignorato.

Per lavorare in modo efficace in questo mondo distorto dobbiamo essere in grado di costruire entrambi i tipi di siti Web. I siti non popolari sono facili da gestire; utilizzando il DNS molti siti diversi possono puntare allo stesso computer in Internet, che li gestisce tutti. I siti popolari, invece, sono difficili da gestire. Non esiste un unico computer anche remotamente abbastanza potente e utilizzando un singolo computer si potrebbe rendere il sito inaccessibile per milioni di utenti. Per gestire questi siti bisogna costruire sistemi di distribuzione di contenuti, di cui parleremo nel prossimo paragrafo.

Server farm e proxy Web

L'architettura Web che abbiamo considerato finora ha un singolo server che dialoga con più client. Per costruire grandi siti Web che lavorino bene dobbiamo velocizzare l'elaborazione sia lato server sia lato client. Lato server, server Web più potenti possono essere costruiti con delle server farm, nelle quali un cluster di computer agisce come un singolo server. Lato client, si possono ottenere prestazioni ottimali usando tecniche migliori per l'uso della cache. In particolare, un proxy Web fornisce cache grande e condivisa per un gruppo di client.

Descriveremo tutte queste tecniche una alla volta. Tuttavia si noti che nessuna di esse è sufficiente per costruire i siti Web più grandi, che richiedono metodi di distribuzione dei contenuti che descriveremo nei prossimi paragrafi.

Server farm

Una macchina, indipendentemente da quanta banda usa, può soddisfare richieste Web solo finché il carico non sia troppo elevato. La soluzione in questo caso è usare più computer per fare un server Web. Questo metodo conduce al modello di **server farm** descritto nella Figura 7.65.

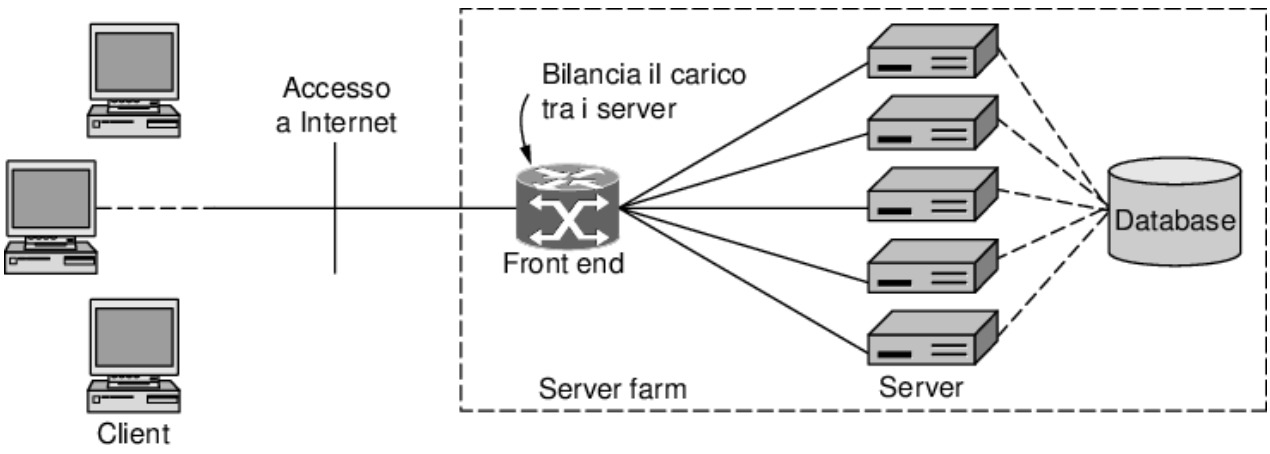


Figura 7.65 Una server farm.

La difficoltà di questo modello, apparentemente semplice, è che l'insieme dei computer che formano la server farm deve apparire ai client, dal punto di vista logico, come un singolo sito Web, altrimenti avremmo costruito differenti siti Web che lavorano in parallelo.

Per fare in modo che un insieme di server appaia come un unico sito Web vi sono alcune possibili soluzioni. Tutte assumono che ogni server possa gestire una richiesta da qualsiasi client. Per fare ciò ogni server deve avere una copia del sito Web. A questo proposito, i server sono mostrati come connessi al database comune da una linea tratteggiata.

Una soluzione consiste nell'usare il DNS per distribuire le richieste sui server della server farm. Quando viene fatta una richiesta DNS per l'URL, il server DNS restituisce una lista in cui gli indirizzi IP dei server appaiono in rotazione. Il client prova un indirizzo IP, tipicamente il primo della lista; l'effetto finale è che client differenti contattano server distinti per accedere allo stesso sito Web, esattamente come si voleva. il metodo DNS è il cuore delle CDN e ne riparleremo in seguito.

Le altre soluzioni sono basate su **front end** (di solito uno switch a livello data link o un router IP, cioè un dispositivo che gestisce frame o pacchetti) che diffondono le richieste in entrata su un insieme di server nella server farm. Questo accade persino quando il client contatta la server farm usando un singolo indirizzo IP di destinazione. Tutte queste soluzioni si basano sull'idea che il front end (o il server) guardi le intestazioni di rete, di trasporto o applicazione e le usi in modi non convenzionali. Una richiesta o una risposta Web sono trasportate da una connessione TCP; perché il servizio funzioni correttamente, il front end deve distribuire tutti pacchetti di una richiesta allo stesso server.

Una semplice architettura per il front end prevede di fare il broadcast di tutte le richieste in entrata a tutti i server. Ogni server risponde solo a una frazione delle richieste con un accordo preventivo. Per esempio, 16 server potrebbero guardare l'indirizzo IP sorgente e rispondere alla richiesta solo se gli ultimi quattro bit coincidono con un numero preconfigurato. Gli altri pacchetti vengono scartati. Benché questo metodo sprechi banda entrante non è così inefficiente come sembra, perché spesso le risposte sono molto più lunghe delle richieste.

In uno schema più generale, il front end può ispezionare le intestazioni dei pacchetti IP, TCP e HTTP e associarli arbitrariamente a un server. L'associazione è chiamata politica di **bilanciamento del carico** (*load balancing*), perché l'obiettivo è bilanciare il carico tra i server: può essere semplice o complessa. Una politica semplice consiste nell'usare i server a turno (in *round-robin*). Con questo approccio il front end deve ricordarsi dell'associazione per ogni richiesta in modo che pacchetti successivi, che sono parte della stessa richiesta, vengano spediti al medesimo server. Inoltre, per rendere il sito Web più affidabile rispetto a un singolo server, il front end dovrebbe accorgersi quando i server non stanno più funzionando e smettere di mandar loro richieste.

Come per NAT, questo schema generale è pericoloso, o almeno fragile, perché crea un dispositivo che viola i principi più basilari della stratificazione dei protocolli in livelli. Ogni livello dovrebbe usare solo la propria intestazione a scopo di controllo e non dovrebbe poter ispezionare e usare informazioni nel payload per nessun motivo. Ma i progettisti disegnano ugualmente questi sistemi e, quando in futuro smetteranno di funzionare a causa di cambiamenti nei livelli superiori, si sorprenderanno. Il front end in questo caso è uno switch o un router, ma compie azioni basate su

informazioni a livello di trasporto o superiori. Il dispositivo si chiama **middlebox**, perché si posiziona nel mezzo del percorso di rete, in cui non dovrebbe avere alcuna azione da compiere secondo la gerarchia dei protocolli. In questo caso sarebbe meglio considerare il front end come una parte interna della server farm che implementa tutti i livelli fino a quello di applicazione (e che quindi può usare le informazioni delle intestazioni per quei livelli).

Ciononostante, come NAT, questo schema è utile nella pratica. Il motivo per cui si guardano le intestazioni TCP è che il bilanciamento di carico funziona meglio che non usando solo le informazioni IP. Per esempio, un indirizzo IP può rappresentare un'intera azienda che inoltra molte richieste; solo guardando le informazioni di livello TCP o di livelli più alti si può capire come associare queste richieste a server differenti.

La ragione per cui si guardano le intestazioni HTTP è un po' diversa. Molte interazioni Web accedono e aggiornano dei database, come quando un consumatore passa in rivista i suoi ultimi acquisti. Il server che soddisfa questa richiesta può aver bisogno di interrogare il database di back end. È utile dirigere le richieste successive dello stesso utente allo stesso server, che ha già le informazioni riguardanti l'utente nella cache. Il modo più semplice per ottenere questo è usare i cookie (o altre informazioni che distinguono l'utente) e ispezionare le intestazioni HTTP per trovarli.

Infine si noti che, sebbene abbiamo descritto questo schema per i siti Web, una server farm può essere costruita anche per altri tipi di server; un esempio sono i server per fare streaming di contenuti multimediali su UDP. L'unico cambiamento richiesto è che il front end sia in grado di bilanciare il carico di queste richieste (che avranno intestazioni diverse rispetto a quelle Web).

Proxy Web

Le richieste e le risposte Web sono inviate usando HTTP. Nel Paragrafo 7.3 abbiamo descritto come i browser possano fare uso di una cache per le risposte e la utilizzino per richieste successive. Il browser usa vari campi delle intestazioni e diverse regole per determinare se una copia di una pagina Web nella cache sia ancora valida. Non ripeteremo qui questo argomento.

L'uso di una cache migliora le prestazioni, abbreviando i tempi di risposta e riducendo il carico di rete. Se un browser può capire da solo se una pagina nella cache è valida, la pagina può essere presa immediatamente, senza traffico di rete. Tuttavia, anche se il browser deve chiedere al server una conferma sulla validità della pagina, il tempo di risposta è abbreviato e il carico di rete ridotto, specialmente per pagine grandi, perché è necessario inviare solo un piccolo messaggio.

Tuttavia, la cosa migliore che il browser possa fare è mettere nella cache tutte le pagine Web visitate precedentemente dall'utente. Dalla discussione precedente sulla popolarità, ricorderete che, come ci sono poche pagine popolari visitate ripetutamente da molte persone, ce ne sono anche molte altre non popolari. In pratica, i limiti nell'efficacia dell'uso di una cache derivano dal fatto che un gran numero di pagine vengono visitate solo una volta da un utente. Queste pagine devono sempre essere prese dal server.

Una strategia per aumentare l'efficacia delle cache consiste nel condividerle tra più utenti. In questo modo una pagina già presa per un utente può essere restituita a un altro utente che faccia la stessa richiesta. Senza la cache del browser entrambi gli utenti dovrebbero prendere la pagina dal server. Naturalmente questa condivisione non può essere fatta per traffico codificato, pagine che

richiedono l'autenticazione e pagine che non possono essere inserite nella cache (per esempio l'attuale prezzo delle azioni) che sono restituite da programmi. Le pagine create dinamicamente da programmi sono una classe in espansione per la quale le cache non sono efficienti. Ciononostante c'è un gran numero di pagine Web visibili a molti utenti che sembrano uguali qualunque sia la richiesta inoltrata dall'utente (per esempio le immagini).

Un **proxy Web** è usato per fare condivisione di una cache tra utenti. Un proxy è un agente che agisce in vece di qualcuno, per esempio l'utente. Ci sono molti tipi di proxy, per esempio un proxy ARP risponde a richieste ARP al posto di un utente che si trova da qualche altra parte e non può rispondere. Un proxy Web si occupa delle richieste Web al posto dei suoi utenti. Usualmente gestisce una cache delle risposte Web e, poiché le condivide, ha una cache sostanzialmente più grande di un browser.

La tipica configurazione per un'organizzazione che fa uso di un proxy è di usarne uno solo per tutti i suoi utenti. L'organizzazione potrebbe essere un'azienda o un ISP, ma entrambi beneficerebbero di una maggiore velocità nelle richieste Web e nella riduzione di banda necessaria. Benché le tariffe fisse, indipendenti dall"uso, siano comuni negli utenti finali, la maggior parte delle aziende e ISP pagano a seconda di quanta banda consumano.

Questa configurazione è mostrata nella Figura 7.66. Per usare il proxy ogni browser è configurato in modo da fare le richieste delle pagine al proxy invece che al vero server della pagina. Se il proxy ha la pagina, la restituisce immediatamente; altrimenti, prende la pagina dal server, la memorizza nella cache per usarla ancora in futuro e la restituisce al client che l'ha richiesta.

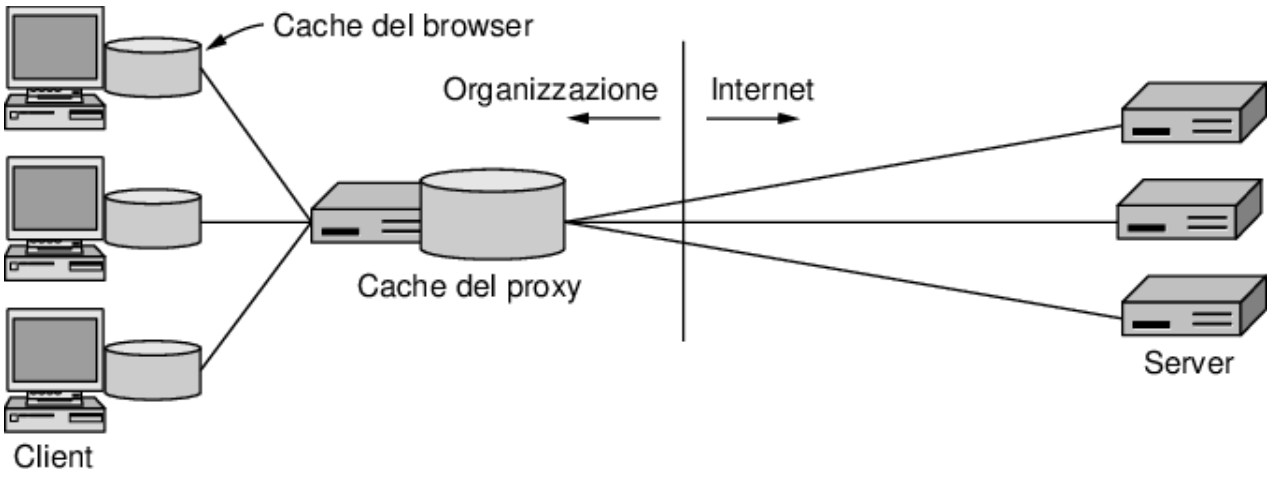


Figura 7.66 Un proxy tra browser e server Web.

Oltre che inviare le richieste al proxy invece che al server effettivo, il client gestisce una propria cache nel browser. Il proxy viene consultato solo dopo che il browser abbia tentato di soddisfare la richiesta usando la propria cache. Quindi il proxy fornisce un secondo livello di cache.

Ulteriori proxy possono essere aggiunti per fornire livelli addizionali di cache. Ogni proxy (o browser) fa le proprie richieste attraverso il suo **upstream proxy** (*proxy di livello superiore*). Ogni proxy gestisce una cache per i downstream proxy (*proxy di livello inferiore*) e i browser. Risulta quindi possibile che un browser di un'azienda usi il proxy dell'azienda, che usa il proxy dell'ISP che contatta direttamente il server Web. Tuttavia, nella pratica, il singolo livello di cache mostrato nella Figura 7.66 è spesso sufficiente per guadagnare la maggior parte dei benefici. Il problema è ancora la lunga coda della popolarità. Studi sul traffico Web hanno dimostrato che fare uso di una cache condivisa è utile specialmente finché il numero di utenti raggiunge circa la dimensione di una piccola azienda (per esempio 100 persone); man mano che il numero di persone cresce, i benefici della condivisione diventano marginali, a causa delle richieste non popolari che non possono entrare nella cache per mancanza di spazio (Wolman et al. 1999).

I proxy Web forniscono altri vantaggi che spesso sono fattori cruciali rispetto alla decisione di installarli. Il primo è poter filtrare i contenuti: l'amministratore può configurare il proxy con una lista nera di siti o filtrare in altro modo le richieste; per esempio, molti amministratori disapprovano gli impiegati che guardano i video su YouTube durante il lavoro e definiscono i loro filtri di conseguenza. Un altro vantaggio derivante dall'avere i proxy sono la privacy e la anonimità, in quanto il proxy potrebbe in teoria proteggere l'identità dell'utente dal server.

Content delivery network

Server farm e proxy Web aiutano a costruire grandi siti e ad aumentare le prestazioni, ma non sono sufficienti per i siti Web veramente famosi che devono servire contenuti su scala globale. Per questi siti è necessario un approccio diverso.

Le **CDN** (*content delivery network*) sovvertono l'idea della cache Web tradizionale. Invece di avere dei client che cercano una copia della pagina richiesta in una cache vicina, nelle CDN è il provider a mettere una copia della pagina in un insieme di nodi in differenti posizioni e a dirigere il client in modo che usi un nodo a lui vicino come server. Un esempio del percorso ad albero seguito dai dati quando vengono distribuiti da una CDN è mostrato nella Figura 7.67. Il server di origine nella CDN distribuisce una copia del contenuto ad altri nodi nella CDN a Sydney, Boston e Amsterdam in questo esempio, come mostrato dalle linee tratteggiate. I client prendono quindi le pagine dal nodo a loro più vicino nella CDN, come si vede dalle linee continue. In questo modo entrambi i client a Sydney prendono una copia della pagina archiviata a Sydney; non la prendono dal server di origine che potrebbe trovarsi in Europa. L'uso di una struttura ad albero ha tre vantaggi. In primo luogo, la distribuzione dei contenuti può scalare a quanti client sono necessari, aggiungendo nodi alla CDN e livelli all'albero quando la distribuzione tra i nodi diventa un collo di bottiglia. Indipendentemente dal numero di client, la struttura ad albero è efficiente, il server di origine non è sovraccarico perché dialoga con i tanti client tramite l'albero dei nodi della CDN; non deve rispondere in prima persona a ogni richiesta di una pagina. In secondo luogo ogni client ha migliori prestazioni prendendo le pagine da un server vicino, piuttosto che da uno lontano. Ciò accade perché il round trip time per instaurare una connessione è più breve, lo slow start TCP progredisce più velocemente a causa del round trip time più breve e grazie al fatto che percorsi più brevi passano meno probabilmente attraverso regioni congestionate della rete. Infine, anche il carico totale della rete è tenuto al minimo. Se i nodi CDN sono posizionati bene, il traffico per una data pagina dovrebbe passare solo una volta per ogni zona della rete. Questo è molto importante perché, prima o poi, qualcuno paga per l'uso delle risorse di rete.

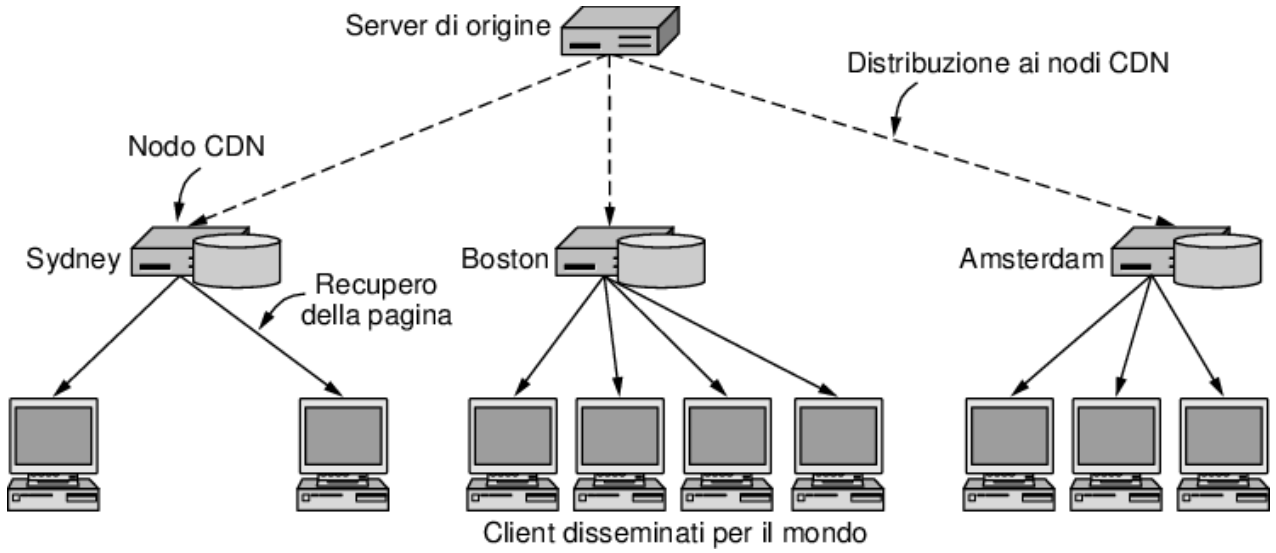


Figura 7.67 Albero di distribuzione CDN.

L'idea di usare un albero di distribuzione è semplice; meno semplice è pensare come organizzare i client in modo che usino questo albero. Per esempio, dei proxy server dovrebbero poter fornire una soluzione. Guardando la Figura 7.67, se ogni client venisse configurato per usare il nodo CDN di Sydney, Boston e Amsterdam come proxy Web, la distribuzione dovrebbe seguire un albero. Tuttavia questa strategia fallisce nella pratica per tre ragioni. La prima è che i client in una certa parte della rete probabilmente appartengono a organizzazioni diverse e usano proxy diversi. Si ricordi che le cache non sono normalmente condivise dalle organizzazioni a causa del beneficio limitato di usarle con un grande numero di client e anche per motivi di sicurezza. In secondo luogo, ci possono essere più CDN, ma ogni client usa un solo proxy. Quale CDN dovrebbe usare il client come proxy? Infine, forse il problema più pratico di tutti è che i proxy Web sono configurati dal client. Possono venire configurati per beneficiare o meno di una distribuzione di contenuti di una CDN e c'è poco che la CDN possa fare in questo senso.

Un altro semplice modo per supportare un albero di distribuzione con un livello è usare il **mirroring**. In questo approccio il server di origine replica i contenuti nei nodi CDN, come prima. I nodi CDN in diverse regioni della rete sono chiamati **mirror** (letteralmente, *specchi*, visto che riflettono il contenuto del server di origine). Le pagine Web sul server di origine contengono collegamenti espliciti a mirror distinti, indicando di solito anche la loro locazione. Questo schema permette all'utente di selezionare manualmente un mirror vicino da usare per fare il download dei contenuti. Un tipico uso di questa tecnica è quello di mettere un grande pacchetto software in mirror posizionati, per esempio, sulla costa Est e Ovest degli Stati Uniti, in Asia e in Europa. I mirror sono generalmente completamente statici e la scelta dei siti rimane stabile per mesi o anni. È una tecnica solida, tuttavia è lasciato all'utente occuparsi della distribuzione, in quanto i mirror sono praticamente siti Web diversi anche se collegati tutti insieme.

Il terzo approccio, che risolve le difficoltà dei primi due, usa il DNS ed è chiamato **DNS redirection** (*redirezione DNS*). Si supponga che un client voglia prendere una pagina all'URL *http://www.cdn.com/page.html*. Per prelevare la pagina, il browser userà il DNS per risolvere

www.cdn.com in un indirizzo IP. Il DNS procede nel solito modo. Usando il protocollo DNS il browser apprende l'indirizzo IP del name server per *cdn.com*, quindi lo contatta e gli chiede di risolvere *www.cdn.com*. Adesso arriva il colpo di genio: il name server è gestito dalla CDN; invece di restituire lo stesso indirizzo IP per ogni richiesta guarda l'indirizzo IP del client che fa la richiesta e restituisce risposte differenti. La risposta sarà l'indirizzo IP del nodo CDN che è più vicino al client, vale a dire che se un client a Sydney chiede al name server della CDN di risolvere *www.cdn.com*, questo restituirà l'indirizzo IP del nodo di Sydney, ma se un client ad Amsterdam fa la stessa richiesta, il name server restituirà l'indirizzo IP del nodo CDN di Amsterdam.

La strategia è perfettamente legale secondo la semantica del DNS. Abbiamo già visto che i name server possono restituire liste mutevoli di indirizzi IP. Dopo la risoluzione del nome, il client di Sydney prende la pagina direttamente dal nodo CDN di Sydney. Molte pagine sullo stesso server saranno prese direttamente dal nodo CDN di Sydney a causa delle cache del DNS. L'intera sequenza di passi è mostrata nella Figura 7.68.

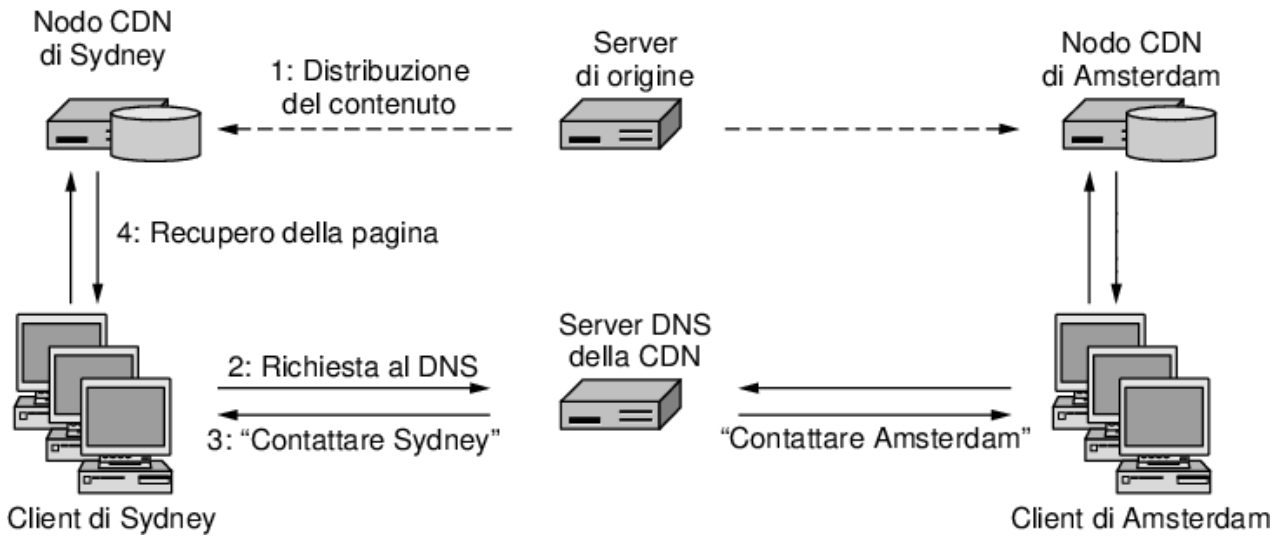


Figura 7.68 Indirizzamento dei client verso i nodi CDN più vicini tramite DNS.

Una domanda complessa all'interno del procedimento è che cosa significhi trovare il nodo CDN più vicino e come cercarlo. La definizione di "più vicino" non riguarda veramente la parte geografica. Nell'associazione tra client e nodi CDN ci sono almeno due fattori da considerare. Il primo è la distanza di rete; il client dovrebbe avere un percorso di rete verso il nodo CDN breve e ad alta capacità in modo da ottenere download veloci. Le CDN usano una mappa di associazioni elaborata preventivamente per tradurre l'indirizzo IP del client nella sua posizione in rete. Il nodo CDN selezionato potrebbe essere quello a distanza più corta, oppure quello che conta è la combinazione tra lunghezza del percorso di rete e limiti di capacità lungo di esso. Il secondo fattore è il carico già gestito dal nodo CDN. Se i nodi CDN sono sovraccarichi saranno lenti nel

rispondere, come i server Web sovraccarichi, cosa che si deve assolutamente evitare. È quindi necessario bilanciare il carico tra i nodi CDN, associando alcuni client a nodi che sono un po' più lontani, ma meno carichi.

La prima che adottò la tecnica di usare il DNS per la distribuzione di contenuti fu Akamai nel 1998, quando il Web stava soffrendo sotto il carico della sua crescita iniziale. Fu la prima grande CDN e divenne un leader industriale. Probabilmente ancora più brillante dell'idea di usare il DNS per connettere client a nodi vicini fu la struttura di incentivazione del loro business. Le aziende pagano Akamai per consegnare i loro contenuti ai client, in modo da avere siti Web veloci molto apprezzati dai clienti. I nodi CDN devono essere posizionati nella rete in luoghi ad alta connettività, che inizialmente significava all'interno di reti di ISP. Per gli ISP il vantaggio di avere un nodo CDN all'interno della loro rete è che questo riduce la banda di rete in uscita (e a pagamento), come un proxy. Inoltre il nodo CDN aumenta la velocità di risposta ai clienti dell'ISP, che quindi lo apprezzano, dando un vantaggio competitivo rispetto a ISP che non hanno nodi CDN. Tali benefici (che non costano all'ISP) fanno in modo che un ISP non ci pensi due volte a installare un nodo CDN. Quindi i fornitori di contenuti, gli ISP e i clienti si avvantaggiano tutti di questo schema e la CDN fa ottimi affari. Dal 1998 altre aziende si sono buttate in questo business che adesso è un'industria competitiva con molti provider.

Come questa descrizione sottintende, molte aziende non creano la propria CDN, ma usano i servizi di un provider CDN, come Akamai, per consegnare effettivamente i loro contenuti. Per fare in modo che altre aziende usufruiscano del servizio di una CDN dobbiamo aggiungere un ultimo passaggio alla nostra descrizione.

Dopo aver firmato il contratto con cui dà incarico a una CDN di distribuire contenuti al suo posto, il proprietario di un sito Web passa alla CDN i contenuti. Il contenuto viene inviato ai nodi CDN. Inoltre il proprietario riscrive le sue pagine Web che collegano al contenuto. Invece di fare il collegamento al contenuto sullo stesso sito Web, le pagine fanno riferimento al contenuto tramite la CDN. Per vedere il reale funzionamento di questo schema nella Figura 7.69(a) si consideri il codice sorgente della pagina Web di Fluffy Video. Dopo la pre-elaborazione essa viene trasformata come appare nella Figura 7.69(b) e posta sul server di Fluffy Video con l'indirizzo *http://www.fluffyvideo.com/index.html*.

```
<html>
<head> <title> Fluffy Video </title> </head>
<body>
<h1> Lista dei prodotti di Fluffy Video </h1>
<p> Seleziona sotto per una anteprima gratuita. </p>
<a href="koalas.mpg"> Koala oggi </a> <br>
<a href="kangaroos.mpg"> Canguri divertenti </a> <br>
<a href="wombats.mpg"> Simpatici Wombat </a> <br>
</body>
</html>
```

(a)

```
<html>
<head> <title> Fluffy Video </title> </head>
<body>
<h1> Lista dei prodotti di Fluffy Video </h1>
<p> Seleziona sotto per una anteprima gratuita. </p>
<a href="http://www.cdn.com/fluffyvideo/koalas.mpg"> Koala oggi </a> <br>
<a href="http://www.cdn.com/fluffyvideo/kangaroos.mpg"> Canguri divertenti </a> <br>
<a href="http://www.cdn.com/fluffyvideo/wombats.mpg"> Simpatici Wombat </a> <br>
</body>
</html>
```

(b)

Figura 7.69 (a) Pagina Web originale. (b) Stessa pagina con collegamenti alla CDN.

Quando un utente digita l'URL *http://www.fluffyvideo.com/* il DNS restituisce l'indirizzo IP del sito Web di Fluffy Video, consentendo il normale recupero della pagina principale (HTML). Quando l'utente fa click su uno dei collegamenti ipertestuali, il browser chiede al DNS di cercare *cdn-server.com*. Questo contatta il server DNS della CDN che restituisce l'indirizzo IP del nodo CDN più vicino. Quindi il browser invia una regolare richiesta HTTP al nodo CDN, per esempio per */fluffyvideo/koalas.mpg*. L'URL identifica la pagina da restituire, iniziando il percorso con *fluffyvideo*, in modo che il nodo CDN possa separare le richieste pervenute da diverse aziende che serve; infine il video viene fornito e l'utente può vedere i teneri animali di fluffy.

La strategia dietro a questa divisione del contenuto ospitato dalla CDN e delle pagine di accesso ospitate dal proprietario del contenuto è che essa dà al proprietario il controllo del contenuto, mentre lascia alla CDN la movimentazione della massa di dati. La maggior parte delle pagine di accesso è piccola, perché è semplice testo HTML. Queste pagine spesso si collegano a file più grandi, come video e immagini. Sono precisamente questi grandi file che sono distribuiti dalla CDN, anche se l'uso della CDN è completamente trasparente all'utente. Il sito appare lo stesso, ma va più veloce. Esiste un altro vantaggio per i siti nell'usare una CDN condivisa. Per i siti Web è difficile predire la richiesta futura. Frequentemente ci sono picchi nelle richieste noti come **flash crowd** (letteralmente, una massa di persone che appare all'improvviso). Si verificano quando viene rilasciato un prodotto atteso dal mercato, quando c'è un evento di moda o oppure quell'azienda in qualche modo è sulla cresta dell'onda. Anche un sito Web che prima era sconosciuto può

improvvisamente diventare il centro di Internet se finisce sui giornali e viene referenziato da altri siti frequentati. Poiché la maggior parte dei siti non è preparata a gestire massivi incrementi di traffico, il risultato è che molti di loro crollano quando il traffico raggiunge l'apice.

Per esempio, il sito Web del Segretario di stato della Florida non è normalmente molto visitato, anche se fornisce informazioni su aziende, notai ed eventi culturali come pure informazioni riguardo alle votazioni ed elezioni. Per strane ragioni, il 7 novembre 2000 (data del voto per la presidenza degli Stati Uniti con Bush contro Gore) un numero incredibile di persone divennero improvvisamente interessate nella pagina dei risultati delle elezioni di questo sito, che divenne uno dei più visitati al mondo e naturalmente crollò. Se avesse usato una CDN sarebbe probabilmente sopravvissuto.

Usando una CDN un sito ha accesso a una grande capacità di servire contenuti. Le CDN più grandi hanno decine di migliaia di server in paesi in tutto il mondo. Poiché solo una piccola parte dei siti sperimenterà un flash crowd in un dato istante (per definizione) questi possono usare la capacità della CDN per gestire il carico, finché la tempesta non passa. Una CDN può quindi velocemente incrementare la capacità di un sito di erogare un servizio.

La discussione precedente è una descrizione semplificata di come funziona Akamai. Nella pratica ci sono molti altri dettagli importanti. I nodi CDN raffigurati nel nostro esempio sono normalmente cluster di macchine. La rete per la redirezione DNS è di solito implementata su due livelli: uno per associare client a una locazione di rete approssimata e l'altro per distribuire il carico sui nodi di quella locazione. Anche l'affidabilità e le prestazioni contano. Per essere in grado di ridirigere un client da una macchina in un certo cluster a un altro le risposte del DNS date dal secondo livello hanno un tempo di vita molto basso, in maniera tale che il client effettui un'altra risoluzione del nome dopo breve tempo. Infine, anche se ci siamo concentrati sulla distribuzione di oggetti statici come immagini e video, le CDN possono supportare la creazione di pagine dinamiche, streaming di contenuti multimediali e molto altro. Per un approfondimento sulle CDN si veda Dilley et al. (2002).

Reti peer-to-peer

Non tutti possono permettersi una CDN di 1000 nodi sparsi in tutto il mondo per distribuire i propri contenuti. In effetti non è difficile affittare 1000 macchine virtuali sparse sul globo, perché tale industria è ben sviluppata e competitiva. Tuttavia, i nodi sono solo il primo passo nell'implementazione di una CDN. Fortunatamente esiste un'alternativa semplice da usare e in grado di distribuire un'incredibile quantità di contenuti: una rete **P2P** (*peer-to-peer*).

Le reti P2P entrarono in scena a partire dal 1990. La prima applicazione fu un crimine di massa: 50 milioni di utenti di Napster si scambiarono musica coperta da copyright senza il permesso dei proprietari, finché Napster non venne chiuso. Tuttavia la tecnologia P2P ha molti usi interessanti e legali. Altri sistemi continuarono l'implementazione, suscitando un interesse così grande negli utenti che il traffico P2P eclissò velocemente il traffico Web. Oggigiorno BitTorrent è il protocollo P2P più popolare. È talmente usato per condividere video (con licenza o di pubblico dominio), come altri contenuti, che rappresenta una frazione significativa del traffico Internet, come vedremo in questo paragrafo.

L'idea alla base di una rete di condivisione di file P2P è che molti computer si uniscono e mettono le loro risorse in comune per formare un sistema di distribuzione di contenuti. I computer sono spesso dei semplici computer domestici, non devono essere per forza macchine in dei data center. I computer sono chiamati *peer* perché ognuno può agire alternativamente come client per un altro peer, prendendo i suoi contenuti e come server fornendo contenuti ad altri peer. Ciò che rende i sistemi peer to peer interessanti è che non necessitano di un'infrastruttura dedicata come nelle CDN. Ognuno partecipa al compito di distribuire contenuti e spesso non c'è un punto centrale di controllo.

Molti apprezzano la tecnologia P2P perché sembra essere alla portata anche degli utenti con meno mezzi. Il motivo non è solo che ci vuole una grande impresa per impiantare una CDN, mentre chiunque abbia un computer può unirsi a una rete P2P, ma anche che le reti P2P hanno una formidabile capacità di distribuire contenuti che può rivaleggiare con i più grandi siti Web.

Si consideri una rete P2P composta in media da N utenti, ciascuno con connettività a banda larga da 1 Mbps. La capacità di carico aggregata della rete P2P o tasso al quale gli utenti possono inviare il traffico in Internet, è di N Mbps. La capacità di download o velocità a cui gli utenti possono ricevere il traffico è anch'essa di N Mbps. Ogni utente può caricare e scaricare allo stesso tempo, perché ha un collegamento di 1 Mbps per ogni direzione.

Sebbene non sia evidente, risulta che tutta la capacità può essere utilizzata in modo produttivo per distribuire contenuti, anche quando si condivide una singola copia di un file con tutti gli altri utenti. Nella pratica immaginate che gli utenti siano organizzati in un albero binario, in cui ogni utente che non è una foglia dell'albero trasmette ad altri due utenti. L'albero porterà la singola copia del file a tutti gli altri utenti. Per utilizzare la banda in upload di quanti più utenti possibili in ogni momento (e quindi distribuire file di grandi dimensioni con bassa latenza) è necessario parallelizzare l'attività di rete degli utenti. Immaginate che il file sia diviso in 1000 parti. Ogni utente può ricevere un nuovo pezzo da qualche parte in cima all'albero e, allo stesso tempo, inviare il pezzo ricevuto in precedenza verso il basso attraverso l'albero. In questo modo, una volta che la parallelizzazione è iniziata e sono stati spediti un numero ridotto di pezzi (pari alla profondità dell'albero), tutti gli utenti che non sono sulle foglie dell'albero saranno occupati a fare l'upload del file ad altri utenti. Dato che ci sono circa $N/2$ utenti che non sono foglie, la banda in upload di quest'albero è $N/2$ Mbps. Possiamo ripetere questo trucco e creare un altro albero che utilizza gli altri $N/2$ Mbps di banda in upload scambiando i ruoli dei nodi che sono foglie e quelli che lo sono. Insieme, questo schema utilizza tutta la capacità.

Questo ragionamento significa che le reti P2P scalano in maniera trasparente: la loro capacità di upload utilizzabile cresce di pari passo alle richieste di download fatte dai loro utenti. Sono sempre "grandi" abbastanza, in un certo senso, senza la necessità di alcuna infrastruttura dedicata. Al contrario, la capacità anche di un sito Web di grandi dimensioni è fissa e potrà essere troppo grande o troppo piccola. Si consideri un sito con solo 100 cluster, ciascuno con una capacità di 10 Gbps. Questa enorme capacità non aiuta quando c'è un piccolo numero di utenti. Il sito non può ottenere informazioni per gli N utenti a una velocità maggiore di N Mbps, perché il limite è degli utenti e non del sito Web. E quando ci sono più di un milione di utenti da 1 Mbps, il sito Web non può pompare fuori i dati abbastanza velocemente da tenere tutti gli utenti occupati a scaricare. Potrebbe sembrare un numero di utenti esagerato, ma le grandi reti BitTorrent sostengono di aver più di 10.000.000 di utenti. Ciò nel nostro esempio corrisponde a 10 Tbps!

Questi numeri si dovrebbero prendere con le pinze, perché semplificano eccessivamente la situazione. Una sfida importante per le reti P2P è quella di utilizzare la larghezza di banda anche quando gli utenti possono essere di tutte le forme e dimensioni e hanno differenti capacità di download e upload. Tuttavia, questi numeri hanno evidenziato le enormi potenzialità del P2P.

C'è un'altra ragione per cui le reti P2P sono importanti. Le CDN e gli altri servizi centralizzati permettono ai fornitori di disporre di una quantità di informazioni personali su molti utenti, dalle preferenze di navigazione ai loro acquisti on-line, al lavoro svolto fino ai loro indirizzi e-mail. Queste informazioni possono essere utilizzate per fornire un migliore servizio più personalizzato oppure per introdursi nella vita privata della gente, intenzionalmente (come parte di un nuovo prodotto) o attraverso la divulgazione accidentale o il furto. Con i sistemi P2P non ci può essere un unico fornitore che sia in grado di controllare l'intero sistema. Questo non significa che i sistemi P2P forniscano necessariamente privacy, perché gli utenti hanno fiducia l'un l'altro solo in una certa misura. Significa solo che essi possono fornire una diversa forma di privacy rispetto ai sistemi di gestione centralizzata. I sistemi P2P sono ora in fase di esplorazione per servizi al di là della condivisione di file (ad esempio l'archiviazione e lo streaming) e il tempo dirà se tale vantaggio avrà qualche valore.

La tecnologia P2P ha seguito due percorsi in relazione tra loro per com'è stata sviluppata. Dal lato più pratico vi sono sistemi che vengono utilizzati ogni giorno. I più noti sono basati sul protocollo BitTorrent. Dal versante più accademico si è sollevato un grande interesse per gli algoritmi di gestione delle DHT (*distributed hash tables*, tabelle hash distribuite) che permettono ai sistemi P2P di funzionare bene come un'unica entità, anche se basati su componenti non centralizzate. Prenderemo in esame entrambe queste tecnologie.

BitTorrent

Il protocollo BitTorrent è stato sviluppato da Brahm Cohen nel 2001 per permettere a dei peer di condividere i file rapidamente e facilmente. Ci sono dozzine di client liberamente disponibili che implementano questo protocollo tanti quanti sono i browser che parlano ai server Web col protocollo HTTP. Il protocollo è disponibile come standard aperto su *www.bittorrent.org*.

In un tipico sistema P2P, come quello formato con BitTorrent, ogni utente dispone di alcune informazioni che potrebbero interessare ad altri utenti. Queste informazioni possono essere software libero, musica, video, fotografie e altro. Per condividere contenuti in questo schema sono tre i problemi da risolvere:

1. come fa un peer a trovare altri peer che abbiano il contenuto che vuole scaricare
2. come viene replicato il contenuto dai peer per fornire download ad alta velocità per tutti
3. come incoraggiare i peer a caricare contenuti per gli altri nella stasse misura in cui li scaricano per se stessi.

Il primo problema esiste perché non tutti i peer avranno tutti i contenuti, almeno inizialmente. L'approccio adottato in BitTorrent è creare una descrizione del contenuto di ogni fornitore chiamato **torrent**. Il torrent è molto più piccolo del contenuto ed è utilizzato da un peer per verificare l'integrità dei dati che viene scaricata da altri peer. Altri utenti che vogliono scaricare il contenuto devono prima ottenere il torrent, trovandolo ad esempio su una pagina Web che ne fa pubblicità al contenuto.

Il torrent è solo un file in un formato specifico contenente due tipi chiave di informazione. Un tipo è il nome di un **tracker**, un server che conduce i peer al contenuto del torrent. L'altro tipo di informazione è un elenco di pezzi di dimensioni uguali o **chunk**, che costituiscono il contenuto. Chunk di differenti misure possono essere usati per diversi torrent, in genere da 64 KB a 512 KB. Il file torrent contiene il nome di ogni chunk, fornito come una hash SHA-1 di 160-bit del chumk. Ci occuperemo di hash crittografiche, come SHA-1 nel Capitolo 8. Per ora si può pensare a una hash come un checksum più lungo e più sicuro. Date le dimensioni dei chunk e delle hash, il file torrent è di almeno tre ordini di grandezza più piccolo del contenuto, in modo che possa essere trasferito rapidamente.

Per scaricare il contenuto descritto in un torrent, un peer per prima cosa contatta il tracker del torrent. Il **tracker** è un server che gestisce un elenco di tutti gli altri peer che stanno attivamente facendo il download e l'upload del contenuto. Questo insieme di peer prende il nome di **swarm** (*sciame*). I membri dello sciame contattano il tracker regolarmente per riferire che sono ancora attivi anche se lasciano lo sciame. Quando un nuovo peer contatta il tracker per unirsi allo sciame, il tracker lo informa riguardo ad altri peer attualmente registrati. Ottenere il torrent e contattare il tracker sono i primi due passaggi per il download di contenuti, come mostrato nella Figura 7.70.

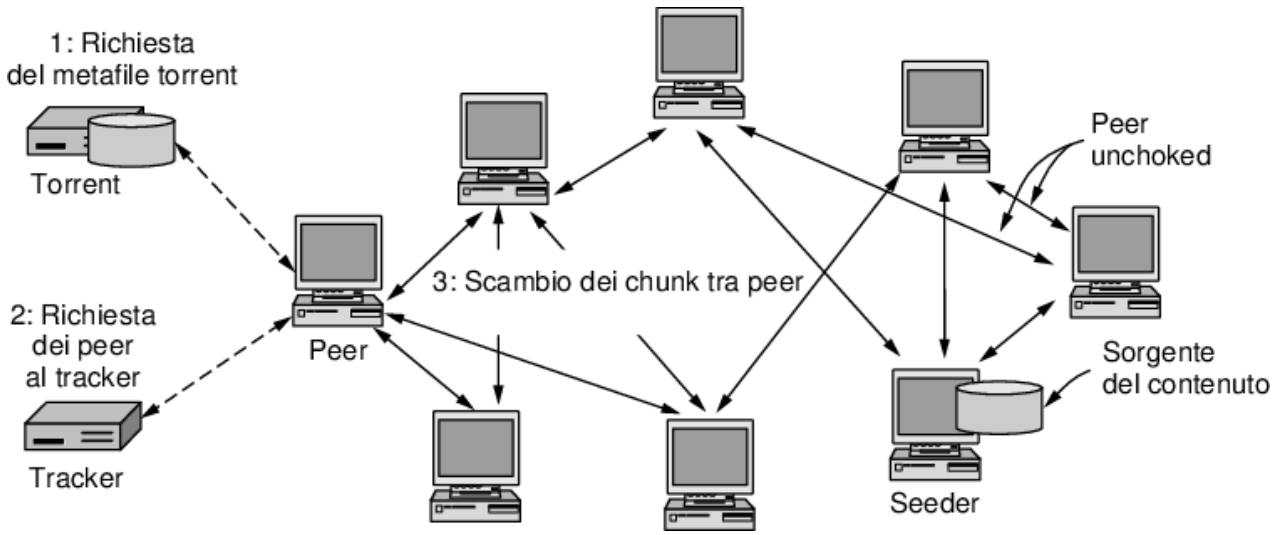


Figura 7.70 BitTorrent.

Il secondo problema è come condividere contenuti in modo da fare un rapido download. Quando uno sciame è appena formato, alcuni peer devono avere tutti i chunk che costituiscono il contenuto. Questi peer sono chiamati **seeder**. Altri peer che si uniscono allo sciame non hanno chuck: sono i peer che stanno scaricando il contenuto.

Quando un peer partecipa a uno sciame fa contemporaneamente download dei chunk che gli mancano dagli altri peer e upload di quelli che mancano agli altri. Tale meccanismo è mostrato nella Figura 7.70 come l'ultimo passo della distribuzione dei contenuti. Nel corso del tempo il peer raccoglie sempre più chunk finché non ha scaricato tutto il contenuto. Il peer può lasciare lo

sciame (e ritornare) in qualsiasi momento. Normalmente un peer si ferma per un breve periodo dopo aver terminato il download. Con i peer che vanno e vengono, il tasso dei chunk trasferiti in uno sciame può essere molto elevato.

Se il metodo funziona bene ogni chunk deve essere disponibile a molti peer. Se tutti dovessero prendere i chunk nello stesso ordine sarebbe probabile che molti peer dipenderebbero dai seeder per il chunk successivo e questo creerebbe un collo di bottiglia. Invece i peer si scambiano le liste dei chunk che hanno e poi selezionano quelli più difficili da trovare per scaricarli. L'idea è che effettuare il download di un chunk raro genererà una sua copia che renderà il chunk più facile da trovare per altri peer. Se tutti i peer agiranno così, dopo un breve periodo tutti i chunk saranno ampiamente disponibili.

Il terzo problema è forse il più interessante. Al contrario dei nodi P2P i nodi CDN sono costituiti esclusivamente per fornire contenuti agli utenti. Sono computer degli utenti e gli utenti sono probabilmente più interessati a un film che non ad aiutare gli altri utenti con i loro download. I nodi che prendono le risorse da un sistema senza contribuire sono chiamati **free-rider** o **leecher**. Se ce ne sono troppi, il sistema non funziona bene. All'inizio si sapeva che i sistemi P2P ne ospitavano (Saroiu et al., 2003) e BitTorrent ha cercato di ridurli al minimo.

L'approccio adottato nei client BitTorrent è quello di premiare i peer che si comportano bene in upload. Ogni peer campiona in modo casuale gli altri peer, prendendo chunk da loro mentre ne invia. Il peer continua a scambiare chunk solo con un piccolo numero di peer che offrono le migliori prestazioni di download, mentre cerca in modo casuale altri peer per trovare buoni partner. La ricerca casuale di peer permette anche ai nuovi di ottenere chunk iniziali da scambiare. I peer con cui un nodo sta attualmente scambiando chunk si dicono **unchoked** (*non soffocati*).

Nel tempo questo algoritmo fa sì che i peer siano paragonabili in termini di tassi di upload e download. Più un peer sta contribuendo, tanto più si può aspettare chunk in cambio. L'utilizzo di un insieme di peer aiuta anche a saturare la banda in download del peer per ottenere alte prestazioni. Al contrario, se un peer non sta facendo upload o lo fa molto lentamente, prima o poi sarà tagliato fuori o **chocked** (*soffocato*). Questa strategia scoraggia il comportamento antisociale per cui dei peer non contribuiscono allo sciame.

L'algoritmo di chocking è talvolta descritto come l'implementazione della strategia **tit-for-tat** (*occhio per occhio*) che incoraggia la cooperazione con interazioni ripetute. Tuttavia non è un forte ostacolo per impedire ai client di aggirare il sistema (Piatek et al., 2007). Nonostante tutto, l'attenzione al problema e ai meccanismi che lo rendono più difficile per gli utenti occasionali hanno probabilmente contribuito al successo di BitTorrent.

Come potete vedere dalla nostra discussione, BitTorrent è dotato di un ricco vocabolario. Ci sono torrent, sciami, leecher, seeder, tracker, choke e altro. Per approfondire l'argomento si veda un breve articolo su BitTorrent (Cohen, 2003) e il sito *www.bittorrent.org*.

DHT – distributed hash table

Intorno agli anni 2000 l'emergere di reti di condivisione di file P2P ha suscitato molto interresse nella comunità di ricerca. L'essenza dei sistemi P2P è che evitano le strutture di gestione centralizzate delle CDN e di altri sistemi. Questo può essere un vantaggio significativo. Le componenti gestite centralmente diventano un collo di bottiglia quando il sistema cresce molto e

un loro malfunzionamento ha ripercussioni su tutto il sistema (in breve, sono un *single point of failure*). Le componenti centrali possono anche essere usate come punto di controllo (ad esempio per spegnere la rete P2P). Tuttavia, i primi sistemi P2P erano solo in parte decentralizzati o, se lo erano totalmente, erano inefficienti.

La forma tradizionale di BitTorrent che abbiamo appena descritto utilizza trasferimenti P2P e un tracker centralizzato per ogni sciame. In un sistema P2P è il tracker che risulta essere la parte più difficile di decentralizzare. Il problema principale è come scoprire quali peer hanno un contenuto specifico che viene richiesto. Per esempio, ogni utente potrebbe avere uno o più dati come canzoni, fotografie, programmi, file e così via che gli altri utenti potrebbero voler leggere. Come fanno gli altri utenti a trovarli? Fare un indice di chi ha che cosa è semplice, ma è centralizzato. Anche se ogni peer dovesse mantenere un proprio indice non aiuterebbe; è distribuito, ma richiede talmente tanto lavoro per mantenere gli indici di tutti i peer aggiornati (quando il contenuto viene spostato nel sistema), che non ne vale la pena.

La questione affrontata dalla comunità dei ricercatori era se fosse possibile costruire indici P2P interamente distribuiti, ma con delle buone prestazioni. Avere buone prestazioni significa tre cose. In primo luogo ogni nodo mantiene solo una piccola quantità di informazioni sugli altri; questa proprietà significa che non sarà costoso mantenere l'indice aggiornato. In secondo luogo ogni nodo può cercare rapidamente voci nell'indice; in caso contrario, non è un indice molto utile. In terzo luogo ogni nodo può utilizzare l'indice mentre altri nodi vanno e vengono; questa proprietà significa che le prestazioni dell'indice crescono con il numero di nodi.

La risposta a tale questione è stata "Sì". Nel 2001 sono state proposte quattro diverse soluzioni. Sono Chord (Stoica et al., 2001), CAN (Ratnasamy et al.2001), Pastry (Rowstron e Druschel, 2001) e Tapestry (Zhao et al., 2004). Altre soluzioni sono state date poco dopo, tra cui Kademlia, che viene utilizzata in pratica (Maymounkov e Mazieres, 2002). Le soluzioni sono note come **DHT** (*distributed hash table*, tabelle hash distribuite), in quanto la funzionalità di base di un indice è quella di associare una chiave a un valore. Questo equivale a una tabella di hash e, naturalmente, le soluzioni proposte ne sono delle versioni distribuite.

Le DHT operano, come vedremo, attraverso l'imposizione alle comunicazioni tra i nodi di una struttura regolare. Questo comportamento è molto diverso da quello delle tradizionali reti P2P che usano qualunque connessione facciano i peer. Per questo motivo, le DHT sono chiamate **reti P2P strutturate**. I tradizionali protocolli P2P costruiscono **reti P2P non strutturate**.

La soluzione DHT che descriveremo è Chord. Come scenario si consideri la sostituzione del tracker centralizzato, tradizionalmente usato in BitTorrent, con un tracker completamente distribuito. Chord può essere utilizzato per risolvere questo problema. In questo scenario l'indice generale è un elenco di tutti gli sciami a cui un computer può unirsi per scaricare del contenuto. La chiave utilizzata per cercare l'indice è la descrizione del contenuto del torrent. Identifica in modo univoco uno sciame da cui il contenuto può essere scaricato come l'hash di tutti i chunk in esso contenuti. Il valore memorizzato nell'indice per ogni chiave è la lista dei peer che compongono lo sciame. Questi peer sono i computer da contattare per scaricare il contenuto. Una persona che vuole scaricare il contenuto, ad esempio un filmato, ha solo la descrizione del torrent. La domanda alla quale la DHT deve rispondere è, in mancanza di una banca dati centrale, come fa una persona a scoprire quali peer (tra i milioni di nodi BitTorrent) sono adatti per scaricare il filmato.

Una DHT Chord è costituita da *n* nodi partecipanti. Nel nostro scenario sono nodi che fanno uso di BitTorrent. Ogni nodo ha un indirizzo IP tramite il quale può essere contattato. L'indice complessivo è diffuso sui nodi. Ciò implica che ogni nodo memorizza bit e pezzi di indice per l'utilizzo da parte di altri nodi. La parte fondamentale di Chord è che naviga l'indice utilizzando identificatori in uno spazio virtuale, non l'indirizzo IP di nodi o i nomi di contenuti, come i film. Concettualmente, gli identificatori sono semplicemente numeri di *m* bit che possono essere disposti in ordine crescente in un anello.

Per trasformare l'indirizzo di un nodo in un identificatore questo è associato a un numero di *m* bit utilizzando una funzione di hash, che chiameremo *hash*. Chord utilizza SHA-1 per *hash*, si tratta della stessa hash che abbiamo menzionato nel descrivere BitTorrent e la vedremo quando discuteremo di crittografia nel Capitolo 8. Per ora basti dire che si tratta solo di una funzione che prende una stringa di byte di lunghezza variabile come argomento e produce un numero casuale a 160 bit. Quindi la possiamo usare per convertire qualsiasi indirizzo IP in un numero di 160 bit chiamato **identificatore del nodo** (*node identifier*).

Nella Figura 7.71(a) mostriamo il cerchio degli identificatori dei nodi per $m = 5$ (si ignorino gli archi interni per il momento). Alcuni degli identificatori corrispondono ai nodi, ma la maggior parte no. In questo esempio, i nodi con identificatori 1, 4, 7, 12, 15, 20 e 27 corrispondono a nodi reali e nella figura sono ombreggiati, il resto non esiste.

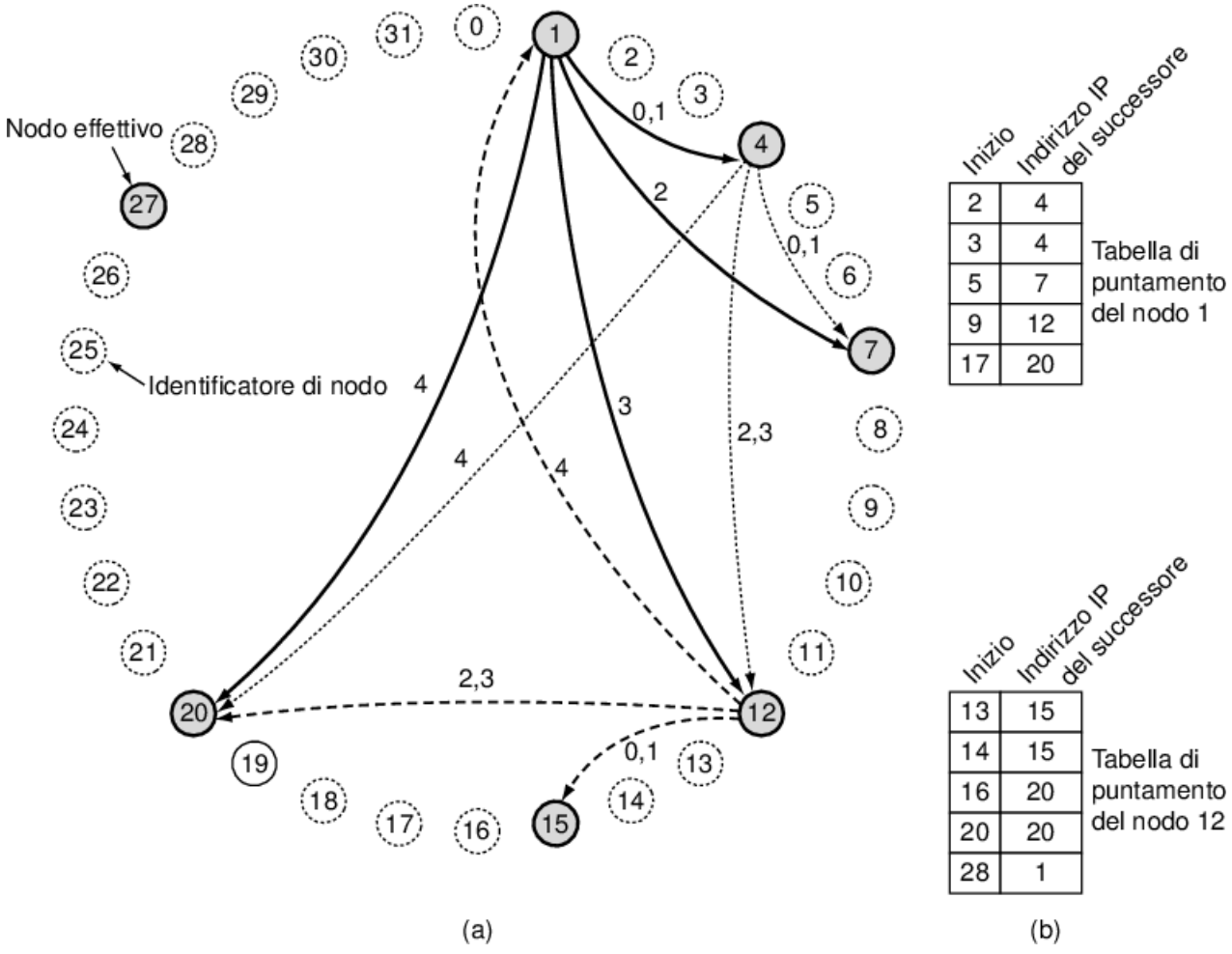


Figura 7.71 (a) 32 identificatori di nodi organizzati in cerchio. Quelli in grigio corrispondono effettivamente a dei calcolatori. Gli archi illustrano i puntatori dai nodi 1, 4 e 12. Le etichette sugli archi sono gli indici nelle tabelle. (b) Esempio di tabelle di puntamento.

Sia *successor*(*k*) la funzione che identifica il primo nodo reale che segue *k* procedendo in senso orario. Per esempio, *successor*(6) = 7, *successor* (8) = 12 e *successor* (22) = 27.

Viene costruita anche una **chiave** facendo hash applicato a funzione *hash* (ossia SHA-1) al nome del contenuto, che genera un numero di 160 bit. Nel nostro scenario il nome del contenuto è il torrent. Quindi, per convertire *torrent* (la descrizione torrent del file) nella sua chiave si utilizza *chiave* = *hash*(*torrent*). Questo calcolo è solo una chiamata di procedura locale indirizzata a *hash.*

Per iniziare un nuovo sciame, un nodo deve inserire nell'indice una nuova coppia di chiave e valore (*torrent*, *indirizzo-IP*) chiedendo a *successor*(*hash*(*torrent*)) di memorizzare *indirizzo-IP*. In questo modo, l'indice è distribuito in modo casuale attraverso i nodi. Per migliorare la tolleranza ai guasti potrebbero essere utilizzate *p* diverse funzioni di hash per conservare ogni record in *p* nodi, ma non prenderemo in esame questo caso.

Qualche tempo dopo che si è costruita la DHT, un altro nodo vuole trovare un torrent per unirsi a uno sciame e scaricarne i contenuti. Un nodo cerca un torrent utilizzando la funzione di hash per ottenere *chiave* e poi utilizza *successor*(*chiave*) per trovare l'indirizzo IP del nodo che conserva il valore corrispondente. Il valore è la lista dei peer nello sciame; il nodo può aggiungere il suo indirizzo IP alla lista e contattare gli altri peer per scaricare contenuti usando il protocollo BitTorrent.

Il primo passaggio è semplice, il secondo meno. Per permettere la ricerca dell'indirizzo IP del nodo corrispondente a una particolare chiave ogni nodo deve mantenere alcune strutture dati con informazioni di supporto. Una di queste è l'indirizzo IP del suo nodo successore sul cerchio degli identificatori dei nodi. Per esempio, nella Figura 7.71 il successore del nodo 4 è 7 e il successore del nodo 7 è 12.

La ricerca ora può procedere come segue: il nodo richiedente invia un pacchetto al suo successore; il pacchetto, che contiene il suo indirizzo IP e la chiave cercata, si propaga lungo l'anello finché non individua il successore dell'identificatore del nodo cercato. Quel nodo verifica se dispone di informazioni che corrispondono alla chiave di ricerca e, in caso positivo, restituisce i dati direttamente al nodo richiedente del quale ha l'indirizzo IP.

La ricerca lineare di tutti i nodi è molto inefficiente in un grande sistema peer-to-peer, poiché il numero medio di nodi richiesti per ogni ricerca è *n*/2. Per velocizzare di molto l'operazione ogni nodo mantiene anche ciò che Chord chiama **finger table** (*tabella di puntamento*). La tabella di puntamento ha *m* voci, indicizzate da 0 a $m - 1$, ognuna delle quali punta a un nodo effettivo diverso. Ogni voce contiene due campi: *start* e l'indirizzo IP di *successor*(*start*), come mostrato nella Figura 7.71(b). I valori dei campi per la voce *i* al nodo *k* sono:

1. $start = k + 2^i \ (modulo\ 2^m)$
2. Indirizzo IP di *successor*(*start*[*i*])

Si noti che ogni nodo conserva gli indirizzi IP di un numero relativamente piccolo di altri nodi e la maggior parte di questi è abbastanza vicina in termini di identificatore di nodo.

Utilizzando la tabella di puntamento la ricerca di *chiave* sul nodo *k* procede come segue: se *chiave* è compresa tra *k* e *successor*(*k*), il nodo contenente le informazioni relative a *chiave* è *successor*(*k*) e la ricerca si conclude. Altrimenti si esamina la tabella di puntamento per scoprire qual è la voce il cui campo *start* è il più vicino predecessore di *chiave.* Quindi viene trasmessa una richiesta direttamente all'indirizzo IP relativo a quella voce della tabella di puntamento, per sollecitare la continuazione della ricerca. Poiché è più vicino a *chiave* ma ancora al di sotto di esso, ci sono buone possibilità che si possa ottenere una risposta con un numero basso di interrogazioni aggiuntive. In effetti, poiché ogni ricerca dimezza la distanza rimanente dall'obiettivo, si può dimostrare che il numero medio di ricerche è $\log_2 n$.

Come primo esempio si consideri la ricerca di *chiave* = 3 al nodo 1. Poiché il nodo 1 sa che 3 si trova tra lui e il suo successore, 4, il nodo desiderato è 4 e la ricerca termina restituendo l'indirizzo IP del nodo 4.

Come secondo esempio si consideri la ricerca di *chiave* = 16 al nodo 1. Poiché il nodo 16 non si trova tra 1 e 4, viene consultata la tabella di puntamento. Il predecessore più vicino a 16 è 9, perciò la richiesta è inoltrata all'indirizzo IP della voce relativa a 9, ossia all'indirizzo IP del nodo 12. Il nodo 12 non sa rispondere da solo. Cerca il nodo più vicino che precede 16 e trova 14, questo conduce all'indirizzo IP del nodo 15. Una richiesta viene trasmessa al nodo 15, che si accorge che 16 si trova tra lui e il suo successore (20), perciò restituisce al chiamante l'indirizzo IP di 20 che torna indietro fino al nodo 1.

Poiché i nodi si uniscono e si staccano dal sistema continuamente, Chord ha bisogno di un modo per gestire queste operazioni. Si supponga che quando inizia a funzionare il sistema sia così piccolo da consentire a tutti i nodi di scambiarsi le informazioni direttamente per costruire il primo cerchio e le tabelle di puntamento. Dopo l'avvio iniziale è necessaria una procedura automatica, che svolge le funzioni descritte di seguito. Se desidera unirsi al sistema, un nuovo nodo *r* deve contattare un nodo esistente chiedendogli di cercare l'indirizzo IP di *successor*(*r*). Poi il nuovo nodo chiede a *successor*(*r*) di indicare il suo predecessore. Infine il nuovo nodo chiede al suo predecessore e al suo successore di inserire *r* tra i due nodi del cerchio appena individuati. Per esempio, se volesse unirsi al sistema, il nodo 24 della Figura 7.71 dovrebbe chiedere a un nodo esistente di cercare *successor*(*24*) che è 27. Poi dovrebbe chiedere a 27 di indicare il suo predecessore (20). Dopo aver comunicato a entrambi la sua esistenza, 20 utilizzerà 24 come suo successore e 27 utilizzerà 24 come suo predecessore. Infine, il nodo 27 consegna le chiavi dell'intervallo 21–24, che ora appartengono a 24. A questo punto 24 è completamente inserito.

Tuttavia, molte tabelle di puntamento contengono errori. Per correggerli ogni nodo esegue un processo in background che periodicamente ricalcola ogni puntamento chiamando *successor.* Quando una di queste interrogazioni individua un nuovo nodo, viene aggiornata la voce del puntamento corrispondente.

Quando abbandona il sistema in modo regolare, il nodo consegna le proprie chiavi al suo successore e informa il suo predecessore che sta uscendo dal sistema, in modo che il predecessore possa collegarsi al successore del nodo uscente. Quando un nodo si blocca improvvisamente, sorge

un problema perché il suo predecessore non ha più un successore valido. Per attenuarlo ogni nodo tiene traccia non solo del suo diretto successore, ma anche dei suoi precedenti s successori diretti, in modo da poter rientrare nel cerchio entro $s - 1$ tentativi consecutivi falliti.

Da quando sono state inventate la quantità di ricerca sulle DHT è stata enorme. Per darvi un'idea della mole, poniamo una domanda: qual è l'articolo più citato di tutti i tempi? Sarà difficile trovarne uno che venga citato più dell'articolo originale di Chord (Stoica et al., 2001). Nonostante questa vera e propria montagna di ricerca, le applicazioni delle DHT stanno solo lentamente cominciando a emergere. Alcuni client BitTorrent usano le DHT per fornire un tracker completamente distribuito del tipo che abbiamo descritto. Grandi servizi commerciali di cloud computing come Dynamo di Amazon incorporano tecnologie DHT (DeCandia et al., 2007).

7.6 Sommario

L'associazione in ARPANET di un nome a un host è iniziata in un modo molto semplice: un file di testo ASCII con elencati i nomi di tutti gli host e i loro corrispondenti indirizzi IP. Ogni notte tutte le macchine scaricavano questo file. Ma quando ARPANET si trasformò in Internet ed esplose in termini di dimensioni venne richiesto uno schema di associazione dei nomi molto più sofisticato e dinamico. Quello utilizzato oggi è uno schema gerarchico chiamato "domain name system", che organizza tutte le macchine su Internet in strutture ad albero. Al livello superiore ci sono i ben noti domini generici, come *com* ed *edu*, nonché circa 200 domini relativi a nazioni. Il DNS è implementato come un database distribuito con server in tutto il mondo. Interrogando un server DNS è possibile associare un nome di dominio Internet all'indirizzo IP utilizzato per comunicare con un computer.

La posta elettronica è la killer application originale di Internet. È ancora ampiamente utilizzata da tutti, dai bambini piccoli ai nonni. La maggior parte dei sistemi di posta elettronica nel mondo usano il sistema ora definito negli RFC 5321 e 5322. I messaggi hanno semplici intestazioni ASCII e molti tipi di contenuti possono essere inviati facendo uso di MIME. Le e-mail sono sottomesse ai message transfer agent per la consegna e recuperate da loro per la presentazione da una varietà di programmi utente, comprese applicazioni Web. La e-mail sottomessa viene consegnata usando SMTP, che funziona facendo un collegamento TCP dal message transfer agent mittente a quello ricevente.

Il Web è l'applicazione che la maggior parte della gente ritiene sia Internet. Originariamente era un sistema per il collegamento senza soluzione di continuità di pagine ipertestuali (scritte in HTML) tra più computer. Le pagine vengono scaricate facendo una connessione TCP dal browser a un server e utilizzando il protocollo HTTP. Oggigiorno, gran parte dei contenuti sul Web è prodotta dinamicamente sia sul server (per esempio, con PHP) o nel browser (per esempio, con JavaScript). Quando combinate con dei database di supporto, le pagine dinamiche dei server consentono applicazioni Web come l'e-commerce e la ricerca di contenuti. Le pagine Web dinamiche si stanno evolvendo in applicazioni piene di funzionalità, come lettori di e-mail, che vengono eseguiti all'interno del browser e utilizzano i protocolli Web per comunicare con server remoti.

Cache e connessioni persistenti sono ampiamente utilizzate per migliorare le prestazioni Web. L'utilizzo del Web su dispositivi mobili può essere impegnativo, nonostante la crescita in larghezza di banda e in potenza di elaborazione dei cellulari. I siti Web inviano spesso versioni personalizzate di pagine con immagini più piccole e meno complesse da navigare per dispositivi con display di piccole dimensioni.

I protocolli Web sono sempre più utilizzati per comunicazioni tra calcolatori. XML è preferito ad HTML, perché la descrizione del contenuto è facile da elaborare. SOAP è un meccanismo RPC che invia messaggi XML utilizzando il protocollo HTTP.

Dal 2000 audio digitale e video sono stati elementi chiave per Internet. La maggior parte del traffico Internet oggi è video; gran parte di questo è in streaming da siti Web in una varietà di protocolli (tra cui RTP/UDP e RTP/HTTP/TCP). I contenuti in tempo reale sono inviati in streaming a molti consumatori; comprendono radio su Internet e stazioni TV che trasmettono tutti i tipi di eventi. Audio e video sono utilizzati anche per le conferenze in tempo reale. Molte chiamate telefoniche usano VoIP anziché la rete telefonica tradizionale e questo include anche la videoconferenza.

C'è un piccolo numero di siti Web estremamente popolari, così come un ampio numero di altri che lo sono meno. Per aiutare i siti popolari sono state progettate le reti di distribuzione di contenuti. Le CDN utilizzano il DNS per indirizzare i client a un server vicino tra tutti quelli sparsi per il mondo. In alternativa le reti P2P permettono a un insieme di macchine di condividere contenuti, quali film. Esse forniscono una capacità di distribuzione di contenuti che scala con il numero di nodi nella rete e che può competere con il più grande dei siti Web.

Problemi

1. Molti computer aziendali dispongono di tre distinti identificatori univoci mondiali. Quali sono?

2. Nella Figura 7.4 non è presente il punto dopo *laserjet.* Perché no?

3. Si consideri la situazione in cui un cyberterrorista blocchi tutti i server DNS del mondo simultaneamente. Come cambierebbe l'uso che facciamo di Internet?

4. Il DNS utilizza UDP al posto di TCP. Se viene perso un pacchetto DNS non esiste il recupero automatico. Questo provoca un problema? Se sì, come viene risolto?

5. John vuole avere un nome di dominio originale e usa un programma di generazione casuale per estrarre il nome di un dominio secondario che vuole registrare nel dominio generico *com*. Il nome generato è lungo 253 caratteri; il gestore del dominio *com* ne permetterà la registrazione?

6. Una macchina con un singolo nome DNS può essere associata a più indirizzi IP? Come può succedere?

7. Il numero di società con un sito Web è notevolmente aumentato negli ultimi anni. Come risultato, centinaia di società si sono registrate con il dominio *com*, provocando un elevato carico sul server di primo livello per questo dominio. Suggerite un modo per alleviare il problema senza cambiare lo schema di denominazione (vale a dire senza introdurre nuovi nomi di dominio di primo livello). Sono ammesse soluzioni che richiedono modifiche al codice del client.

8. Alcuni sistemi di posta elettronica supportano un campo di intestazione *Content Return:* che specifica se il corpo di un messaggio deve essere restituito in caso di mancata consegna. Questo campo appartiene all'involucro o all'intestazione?

9. I sistemi di posta elettronica hanno bisogno di elenchi dove ricercare gli indirizzi di posta elettronica delle persone. Per costruire questi elenchi occorre dividere i nomi in componenti standard (per esempio nome e cognome) per permettere la ricerca. Discutete alcuni problemi che devono essere risolti affinché uno standard mondiale sia accettabile.

10. Un grande studio legale con molti dipendenti prevede un unico indirizzo e-mail per ogni dipendente. L'indirizzo e-mail di ogni dipendente è *<login>@lawfirm.com*. Tuttavia lo studio non indica esplicitamente il formato del login. Così, alcuni dipendenti utilizzano i loro nomi come login, alcuni usano i loro cognomi, altri usano le loro iniziali e così via. Lo studio intende ora fare un formato fisso, per esempio

    nome.cognome@lawfirm.com

    che possa essere utilizzato per gli indirizzi e-mail di tutti i suoi dipendenti. Come può essere fatto con poco sforzo?

11. Un file binario è lungo 4560 byte. Quanto sarà lungo se viene codificato con la codifica base64, con una coppia CR+LF inserita ogni 110 byte inviati e alla fine?

12. Indicare cinque tipi MIME non elencati nel libro. È possibile utilizzare la configurazione del browser o Internet per cercare informazioni.

13. Si supponga di voler inviare un file MP3 a un amico, ma la dimensione del file MP3 è 4 MB e l'ISP dell'amico limita la quantità di posta in arrivo a 1 MB. Esiste un modo per gestire questa situazione utilizzando l'RFC 5322 e MIME?

14. Supponiamo che John abbia appena implementato un meccanismo di inoltro automatico sul suo indirizzo e-mail di lavoro che riceve tutti i messaggi di posta elettronica connessi al suo lavoro, per inoltrarli al suo indirizzo personale che egli condivide con la moglie. La moglie di John, non essendo a conoscenza di questo, ha attivato un vacation agent sul loro account. John, avendo reindirizzato le sue e-mail, non ha fatto la stessa operazione della moglie sulla sua macchina in ufficio. Che cosa accade quando una e-mail viene ricevuta presso l'indirizzo di lavoro di John?

15. In qualsiasi standard, come RFC 5322, è necessaria una grammatica precisa delle operazioni consentite, in modo che diverse implementazioni possano interoperare. Anche gli elementi semplici devono essere definiti attentamente. Le intestazioni SMTP consentono gli spazi bianchi (white space) tra i token. Elencate *due* definizioni plausibili alternative dello spazio bianco tra i token.

16. Il vacation agent è parte dello user agent o del message transfer agent? Naturalmente si configura utilizzando lo user agent, ma è quest'ultimo a inviare effettivamente le risposte? Si motivi la risposta.

17. In una versione semplice dell'algoritmo di Chord per la ricerca P2P le ricerche non usano la tabella di puntamento; sono, invece, lineari intorno al cerchio, in entrambe le direzioni. Può un nodo prevedere con precisione in quale direzione cercare? Si motivi la risposta.

18. IMAP permette agli utenti di prendere e scaricare e-mail da una casella di posta remota. Questo significa che il formato interno delle caselle e-mail deve essere standardizzato in modo che qualsiasi programma IMAP lato client sia in grado di leggere la casella di posta su qualsiasi server di posta? Si motivi la risposta.

19. Si consideri il cerchio Chord nella Figura 7.71. Supponiamo che il nodo 18 diventi improvvisamente disponibile. Quali tra le tabelle di puntamento della figura sono interessate? Come?

20. Webmail utilizza POP3, IMAP o nessuno dei due? Perché è stata fatta questa scelta? Se non vengono usati POP3 o IMAP qual è quello più vicino come teoria?

21. Quando le pagine Web vengono inviate sono precedute da intestazioni MIME. Perché?

22. È possibile che, quando un utente fa click su un collegamento in Firefox, venga avviato un particolare programma di supporto, ma facendo click sullo stesso collegamento in Internet Explorer venga avviato un programma completamente diverso, anche se il tipo MIME restituito in entrambi i casi è identico? Si motivi la risposta.

23. Si supponga che un membro del dipartimento di matematica di Stanford abbia scritto un nuovo documento che include una dimostrazione che desidera distribuire via FTP perché i colleghi possano esprimere le loro opinioni. Inserisce pertanto il file nella directory FTP

ftp/pub/forReview/newprof.pdf. Quale potrebbe essere l'URL completo per questo programma?

24. Nella Figura 7.22*www.aportal.com* tiene traccia delle preferenze dell'utente usando un cookie. Uno svantaggio di questo approccio deriva dal fatto che i cookie sono limitati a 4 KB, pertanto se le preferenze sono numerose (per esempio molte azioni, squadre sportive, tipi di notizie, previsioni meteo per diverse città, offerte speciali per molte categorie di prodotti e così via) il limite di 4 KB può essere raggiunto. Progettate un metodo alternativo per tenere traccia delle preferenze che non presenti questo problema.

25. Sloth Bank desidera facilitare l'on-line banking per i suoi clienti pigri: dopo che un utente ha eseguito l'accesso ed è autenticato da una password, la banca restituisce un cookie contenente il numero identificativo del cliente. In questo modo, il cliente non deve identificare sé stesso o digitare una password nelle future visite alla banca on-line. Come viene giudicata questa idea? Funzionerà? È una buona idea?

26. (a) Si consideri il seguente tag HTML :

```
<h1 title="Questa &egrave; l'intestazione"> HEADER 1 </h1>
```

Sotto quali condizioni il browser usa l'attributo *TITLE* e come? In che cosa differisce l'attributo *TITLE* dall'attributo *ALT*?

27. Com'è possibile rendere selezionabile un'immagine in HTML, in modo che risponda a un click fatto su di essa? Si faccia un esempio.

28. Si scriva una pagina HTML che includa un collegamento all'indirizzo e-mail *username@DomainName.com*. Che cosa succeede se un utente fa click su questo collegamento?

29. Si scriva una pagina XML per un registro universitario in cui compaiano più studenti, ognuno con nome, indirizzo e media degli esami sostenuti.

30. Per ognuna delle seguenti applicazioni, si spieghi se sarebbe (1) possibile e (2) preferibile utilizzare uno script PHP o JavaScript e perché:
a. visualizzare un calendario per qualsiasi mese richiesto, a partire dal settembre 1752
b. visualizzare l'orario dei voli da Amsterdam a New York
c. tracciare il grafico di un polinomio con coefficienti forniti dall'utente

31. Scrivete un programma in JavaScript che accetti un intero maggiore di 2 e indichi se si tratta di un numero primo. Si noti che JavaScript dispone delle istruzioni *if* e *while* con la stessa sintassi di C e Java. L'operatore di modulo è %. Per calcolare la radice quadrata di x è possibile utilizzare *Math.sqrt* (x).

32. Una pagina HTML è come segue:

```
<html> <body>

<a href=”www.info-source.com/welcome.html”> Fare click qui per informazioni </a>

</body> </html>
```

Se l’utente fa click sul collegamento ipertestuale viene aperta una connessione TCP e una serie di righe viene inviata al server. Elencate tutte le righe inviate.

34. L’intestazione *If-Modified-Since* può essere utilizzata per controllare se una pagina nella cache sia ancora valida. Le richieste possono riguardare pagine contenenti immagini, suoni, video e altro ancora, oltre ad HTML. L’efficacia di questa tecnica per le immagini JPEG è maggiore o minore rispetto all’utilizzo per HTML? Si pensi attentamente al significato di “efficacia” e si motivi la risposta.

35. Nel giorno in cui si tiene un importante evento sportivo, come la finale di un campionato di qualche sport, molte persone ne visitano il sito Web ufficiale. È possibile considerarlo un flash crowd come per l’elezione della Florida nel 2000? Perché (o perché no)?

36. È sensato che un singolo ISP svolga la funzione di una CDN? Se sì, come dovrebbe operare? Se no, che cosa c’è di sbagliato nell’idea?

37. Si assuma che non venga usata la compressione per i CD audio. Quanti MB di dati deve contenere un CD per poter riprodurre due ore di musica?

38. Nella Figura 7.42(c) il rumore di quantizzazione si verifica a causa dell’utilizzo di campioni di 4 bit per rappresentare nove valori di segnale. Il primo campione, a 0, è esatto, ma gli altri no. Qual è l’errore percentuale per i campioni a 1/32, 2/32 e 3/32 del periodo?

39. Un modello psicoacustico può essere utilizzato per ridurre la banda necessaria per la telefonia su Internet? Se sì, quali eventuali condizioni devono essere soddisfatte per il funzionamento? Se no, perché no?

40. Un server di streaming audio presenta un end-to-end delay di 100 ms con un lettore multimediale. L’output avviene a 1 Mbps. Se il lettore multimediale ha un buffer di 2 MB, che cosa si può dire sulla posizione del limite inferiore e del limite superiore del buffer?

41. VoIP presenta gli stessi problemi dello streaming audio con i firewall? Si spieghi la risposta.

42. Qual è il banda necessaria per trasmettere un video a colori non compresso di 1200 × 800 pixel, con 16 bit per pixel a 50 frame al secondo?

43. Un errore di un bit in un frame MPEG può influire su più frame, oltre a quello in cui si verifica? Si motivi la risposta.

44. Si consideri un server video per 50.000 clienti, dove ogni cliente guarda tre film al mese. Due terzi dei film viene servita alle 21:00. Quanti filmati deve trasmettere contemporaneamente il server a quell'ora? Se ogni filmato richiede 6 Mbps, quanti collegamenti OC-12 sono necessari?

45. Si supponga che valga la legge di Zipf per gli accessi a un server video con 10.000 filmati. Tenendo presente che il server conserva i 1000 filmati più popolari in memoria e i rimanenti 9000 su disco, si scriva un'espressione per la frazione delle richieste che sono soddisfatte dalla memoria. Si scriva inoltre un breve programma per valutare numericamente questa espressione.

46. Alcuni cybersquatter hanno registrato nomi di dominio corrispondenti a errori ortografici di famosi siti aziendali, per esempio *www.microsfot.com.* Si faccia un elenco di almeno cinque domini di questo tipo.

47. Molte persone hanno registrato nomi DNS simili a *www.parola.com,* dove *parola* è una parola comune. Per le seguenti categorie elencate cinque siti Web e riepilogatene brevemente il contenuto (per esempio, *www.stomach.com* appartiene a un gastroenterologo di Long Island). Ecco l'elenco delle categorie: animali, cibi, soprammobili, parti del corpo. Per l'ultima categoria, si prega di limitarsi alle parti del corpo sopra la cintura.

48. Si riscriva il server della Figura 6.6 come un vero server Web utilizzando il comando *GET* di HTTP 1.1. Dovrebbe accettare anche il messaggio *Host.* Il server dovrebbe mantenere una cache di file recentemente prelevati dal disco e soddisfare le richieste dalla cache quando possibile.

CAPITOLO 8
Sicurezza delle reti

Nei primi decenni della loro esistenza le reti di calcolatori vennero utilizzate prevalentemente dai ricercatori universitari per inviare e ricevere e-mail e dalle aziende per condividere le stampanti. In quelle applicazioni non si prestava particolare attenzione alla sicurezza. Oggi, milioni di persone usano le reti per fare acquisti, home banking oppure compilare la dichiarazione dei redditi, quindi la sicurezza delle reti sta apparendo all'orizzonte come un problema potenzialmente molto vasto. In questo capitolo studieremo la sicurezza delle reti da diversi punti di vista; indicheremo molti punti critici e discuteremo un gran numero di algoritmi e protocolli che servono a rendere le reti più sicure.

La sicurezza è un argomento ampio, che copre una moltitudine di problemi. Nella sua forma più semplice, riguarda come fare in modo che intrusi non riescano a leggere (o, peggio ancora, modificare di nascosto) i messaggi destinati a terzi. Si occupa inoltre di impedire che determinate persone possano accedere a servizi remoti che non sono autorizzate a usare. La sicurezza si occupa anche di come accertarsi dell'identità dei mittenti dei messaggi, di come impedire l'intercettazione e la ripetizione di messaggi legittimi catturati sulla rete e di come perseguire chi afferma di non aver mai spedito certi messaggi. La maggior parte dei problemi di sicurezza è causata da persone malintenzionate, che tentano di guadagnare qualcosa, catturare l'attenzione oppure danneggiare qualcuno. Una lista dei profili più comuni è elencata nella Figura 8.1. Dall'elenco dovrebbe essere chiaro che rendere sicura una rete richiede uno sforzo decisamente maggiore della semplice eliminazione degli errori di programmazione. Occorre infatti affrontare persone intelligenti, tenaci e in alcuni casi anche ben finanziate. Dovrebbe anche essere evidente che misure adeguate per sconfiggere avversari occasionali avranno un impatto limitato su quelli più pericolosi. I rapporti di polizia dimostrano che la maggior parte degli attacchi non proviene da personaggi esterni che intercettano linee telefoniche, ma da personale interno spinto dal risentimento: i sistemi di sicurezza vanno progettati tenendo in mente anche questi fatti.

Profilo	**Scopo**
Studente	Divertirsi a leggere le e-mail degli altri
Cracker	Provare i sistemi di sicurezza di qualcuno; rubare dati
Venditore	Dichiarare di rappresentare tutta l'Europa, non solo Andorra
Multinazionale	Scoprire il piano di marketing strategico del concorrente
Ex-impiegato	Vendetta per il licenziamento
Contabile	Sottrarre soldi a una società
Agente di borsa	Negare un accordo fatto al cliente via e-mail
Ladro di identità	Rubare numeri di carte di credito per rivenderli
Governo	Scoprire i segreti militari o industriali del nemico
Terrorista	Rubare i progetti di un'arma biologica

Figura 8.1 Soggetti che potrebbero creare problemi di sicurezza, con le relative motivazioni.

I problemi di sicurezza delle reti si possono orientativamente dividere in quattro aree interconnesse fra loro: segretezza, autenticazione, non-ripudio (*non-repudiation*) e controllo dell'integrità. La segretezza, detta anche confidenzialità, si occupa di mantenere le informazioni fuori dalla portata degli utenti non autorizzati: è ciò che la gente di solito associa al concetto di sicurezza delle reti. L'autenticazione si occupa di stabilire l'identità del soggetto con cui stiamo comunicando, prima di rivelare informazioni sensibili o concludere transazioni commerciali. Il

termine non-ripudio riguarda le firme: come si può dimostrare che il cliente ha veramente inviato un ordine elettronico per dieci milioni di gadget a 89 centesimi di dollaro l'uno, quando poi dichiara che il prezzo era 69 centesimi oppure nega di aver inviato l'ordine? Infine, come si fa a essere sicuri che il messaggio che abbiamo ricevuto è realmente quello spedito e non è stato modificato in transito o inventato di sana pianta da avversari malintenzionati?

Tutti questi problemi (segretezza, autenticazione, non-ripudio e controllo dell'integrità) esistono anche nei sistemi tradizionali, ma con delle differenze significative. L'integrità e la segretezza si raggiungono, nei sistemi tradizionali, usando posta raccomandata e chiudendo a chiave i documenti. Assaltare il treno postale è più difficile ai giorni nostri che ai tempi di Jesse James.

Di solito le persone riescono a vedere la differenza tra un documento cartaceo originale e una fotocopia e ne tengono conto. Come test, pensiamo di fare la fotocopia di un assegno valido. Il lunedì andiamo in banca a incassare l'assegno originale e il martedì proviamo a incassare la fotocopia: sicuramente il comportamento della banca non è lo stesso. Con gli assegni elettronici, l'originale e la copia sono indistinguibili. È necessario del tempo perché le banche imparino a trattare anche questo tipo di transazioni. Le persone autenticano altre persone riconoscendo le loro facce, la voce e la calligrafia. L'autenticazione può essere realizzata con una firma su un documento, un sigillo, ecc. Le falsificazioni sono generalmente identificabili dagli esperti in calligrafia, inchiostri e carta. Nessuna di queste possibilità esiste nel mondo elettronico, quindi servono altre soluzioni. Prima di parlare delle soluzioni, è utile una digressione per considerare quali sono i livelli e i protocolli interessati dalla sicurezza. Probabilmente non c'è un solo posto, ogni livello può contribuire con qualcosa. Nel livello fisico, l'ascolto del canale può essere evitato racchiudendo le linee di trasmissione dentro a un tubo sigillato che contiene gas ad alta pressione. Ogni tentativo di forare il tubo rilascerà il gas, riducendo la pressione e facendo quindi scattare un allarme. Alcuni sistemi militari usano questa tecnica. Nel livello data link, i pacchetti di una linea punto a punto possono essere cifrati quando lasciano un computer e poi decifrati appena entrano in un altro. Tutti i dettagli sono gestiti dal livello data link, mentre i livelli superiori non si accorgono di queste operazioni. Purtroppo la soluzione non è attuabile quando i pacchetti devono attraversare più router. Il motivo è che i pacchetti dovrebbero essere decifrati in ogni router, rimanendo quindi vulnerabili ad attacchi dall'interno del router. Inoltre, questo metodo permette di proteggere solo alcune sessioni (per esempio quelle relative ad acquisti on-line con carta di credito) mentre altre rimangono in chiaro. Questo metodo si chiama **cifratura del canale** (*link encryption*) e in ogni caso è spesso utile e può essere facilmente aggiunto in qualsiasi rete. Nel livello di rete si possono installare firewall per filtrare i pacchetti buoni e respingere quelli cattivi. Anche la sicurezza a livello IP agisce in questo modo.

Nel livello di trasporto, l'intera connessione può essere cifrata end-to-end, cioè dal processo sorgente al processo ricevente. Ciò è necessario per ottenere la massima sicurezza. Infine, problemi come l'autenticazione degli utenti e il non ripudio possono essere trattati solo al livello applicativo.

Poiché la sicurezza non si può attribuire in modo netto a un livello specifico non è stata discussa in nessuno dei capitoli precedenti ma le viene dedicato un capitolo a parte. Sebbene questo capitolo sia lungo, tecnico ed essenziale, oggi è praticamente ininfluente. Per esempio, è ben documentato che la maggior parte delle falle di sicurezza presso le banche è dovuta a personale incompetente e procedure di sicurezza deboli, problemi software nelle implementazioni che permettono l'effrazione a distanza da parte di utenti non autorizzati e, infine, gli attacchi

denominati di ingegneria sociale (*social engineering*) dove i clienti sono indotti a rivelare dettagli sul loro conto.[1] Tutti questi problemi di sicurezza sono oggi più significativi dei criminali che intercettano linee telefoniche e che decodificano messaggi cifrati. Se una persona può entrare in una filiale qualunque di una banca con una carta bancomat che ha trovato per la strada, dicendo di aver perso il codice PIN e perciò ottenerne uno nuovo tutta la crittografia del mondo non aiuterà a prevenire gli abusi. Il libro di Ross Anderson (2008a) è una vera rivelazione a riguardo, infatti documenta centinaia di casi di falle nella sicurezza in molti settori industriali, praticamente tutti dovuti a quelle che possiamo chiamare educatamente "sciatte abitudini commerciali" (*sloppy business practice*) o mancanze di attenzione alla sicurezza. Ciononostante, il fondamento tecnico su cui è costruito l'e-commerce, quando tutti questi altri fattori sono a posto è la crittografia. Eccezion fatta per la sicurezza a livello fisico, praticamente tutto il resto si basa su principi crittografici. Per questo motivo cominceremo il nostro studio della sicurezza esaminando alcuni dettagli della crittografia. Nel Paragrafo 8.1 esamineremo alcuni principi base. Nei Paragrafi da 8.2 a 8.5 vedremo alcuni algoritmi fondamentali e strutture dati usate in crittografia e studieremo poi come questi concetti possono essere applicati alla sicurezza delle reti. Concluderemo con alcune osservazioni sulla tecnologia e la società. Prima di cominciare vediamo anche che cosa non verrà discusso. Abbiamo tentato di focalizzarci sui problemi legati alle reti piuttosto che ai sistemi operativi o le applicazioni, anche se a volte è difficile distinguerne il confine. Per esempio, in questo capitolo non c'è niente sull'autenticazione degli utenti con tecniche biometriche, sulla sicurezza delle password, sugli attacchi di tipo buffer overflow, i cavalli di Troia, lo spoofing dei login, le bombe logiche, i virus, i worm e così via. Questi argomenti sono trattati in dettaglio nel Capitolo 9 di *I moderni sistemi operativi* (Tanenbaum, 2007). Il lettore interessato può far riferimento a questo testo per gli aspetti sistemistici della sicurezza.

8.1 Crittografia

Crittografia deriva da due parole di greco antico che significano "scrittura segreta". La crittografia ha una storia lunga e avventurosa che risale a migliaia di anni fa. In questo paragrafo vedremo solo alcuni dei punti principali, che ci serviranno come base per la comprensione del seguito. Per una storia completa della crittografia si raccomanda la lettura del libro di Kahn (1995). Un trattato completo e aggiornato della crittografia con i suoi algoritmi, protocolli e applicazioni è (Kaufman et al. 2002). Per un approccio matematico si veda (Stinson, 2002) e per un approccio meno formale (Burnett e Paine, 2001).

I professionisti distinguono i termini cifrario e codice. Con **cifrario** s'intende una trasformazione carattere per carattere (o bit per bit), senza considerare la struttura linguistica del messaggio. Al contrario, un **codice** rimpiazza ogni parola con un'altra parola o simbolo. I codici non vengono più utilizzati, anche se hanno alle loro spalle una storia gloriosa. Il codice di maggior successo fu usato dalle forze armate americane nel pacifico durante la seconda guerra mondiale: si impiegavano indiani Navajo che comunicavano tra loro con le parole Navajo che definiscono i termini militari. Per esempio, *chay-da-gahi-nail-tsaidi* (letteralmente: ammazza tartarughe) indicava le armi anticarro. La lingua Navajo è molto tonale, estremamente complessa e non ha una forma scritta, inoltre nessuno in Giappone ne conosceva l'esistenza.

Nel settembre 1945 il *San Diego Union* descrisse il codice dicendo "per tre anni, ovunque sbarcavano i marines, i giapponesi ascoltavano una serie di strani suoni gutturali inframmezzati con altri suoni che rassomigliavano a un misto fra la voce di un monaco tibetano e il suono del boiler dell'acqua calda che si svuota". I giapponesi non riuscirono mai a interpretare il codice e molti codificatori Navajo ricevettero grandi onori militari per il servizio straordinario e il loro coraggio. Il fatto che gli Stati Uniti decriptarono il codice giapponese mentre il Giappone non decriptò mai quello dei Navajo ebbe un ruolo cruciale nella vittoria americana nel Pacifico.

Introduzione alla crittografia

Storicamente quattro gruppi di persone hanno usato e dato il loro contribuito all'arte della crittografia: i militari, il corpo diplomatico, gli scrittori di diari e gli amanti. Fra tutti questi, i militari hanno avuto il ruolo più importante e hanno dato forma alla disciplina nel corso dei secoli. All'interno delle organizzazioni militari, i messaggi da cifrare erano tradizionalmente assegnati ad addetti di basso livello e bassa paga che avevano il compito di cifrarli e poi trasmetterli. Il notevole volume di messaggi faceva sì che non si potesse usare una squadra di specialisti di alto livello per questo lavoro.

Fino all'avvento dei computer, uno dei principali limiti della crittografia era la capacità degli addetti alla codifica di riuscire a compiere le operazioni necessarie, spesso anche sul campo di battaglia e utilizzando un equipaggiamento minimo. Un vincolo aggiuntivo era anche la difficoltà nel cambiare velocemente da un metodo crittografico a un altro, in quanto questo significava dover formare una gran quantità di persone. Tuttavia, il pericolo che un addetto alla codifica

potesse essere catturato dal nemico rendeva essenziale la possibilità di cambiare il metodo crittografico all'istante, se necessario. Questi requisiti in conflitto fra loro hanno dato origine al modello della Figura 8.2.

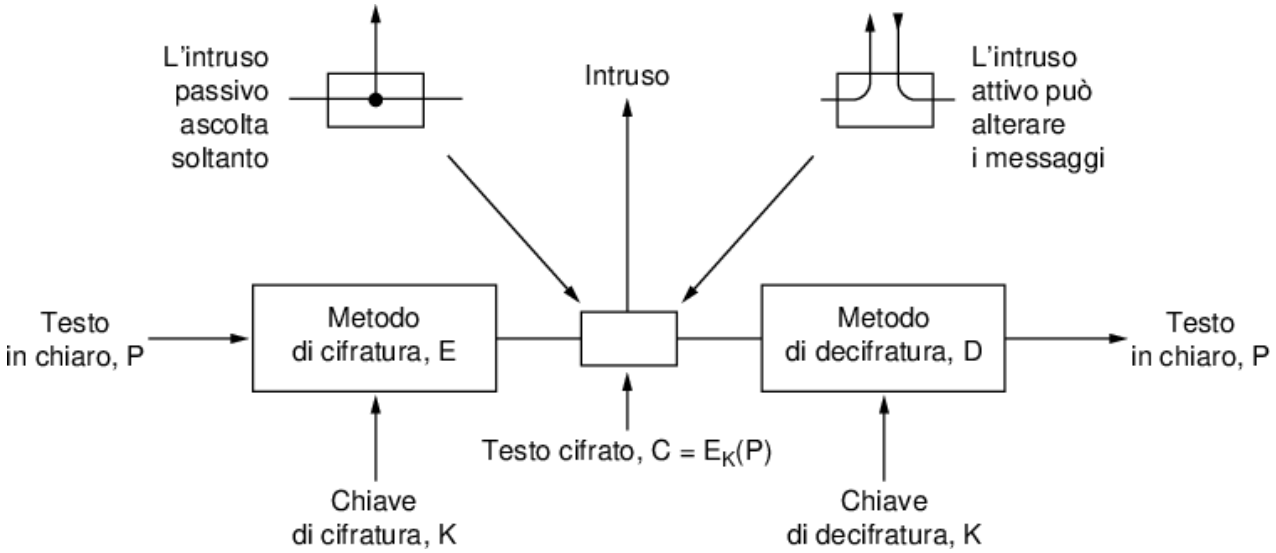


Figura 8.2 Modello di cifratura per un cifrario a chiave simmetrica.

Un messaggio da cifrare, detto **testo in chiaro** (*plaintext*), viene trasformato tramite una funzione parametrizzata da una **chiave**. L'output del processo di cifratura, noto come **testo cifrato** (*ciphertext*), viene generalmente trasmesso tramite un messaggero o via radio. Assumiamo che il nemico, o **intruso**, ascolti accuratamente e trascriva completamente il testo cifrato. Al contrario del destinatario legittimo, l'intruso non conosce la chiave di decifrazione e quindi non riesce facilmente a decifrare il testo cifrato. In alcuni casi l'intruso, oltre ad ascoltare il canale di comunicazione (intruso passivo), riesce anche a registrare i messaggi per poi ritrasmetterli aggiungendo delle informazioni oppure per modificare i messaggi legittimi prima che raggiungano il destinatario (intruso attivo). L'arte di far saltare (o *rompere*, *forzare*)[2] i cifrari, chiamata **criptoanalisi e** l'arte di inventarli (*crittografia*) sono note sotto il nome collettivo di **crittologia**. **Decriptare** è l'attività di decifrazione da parte di un criptoanalista (intruso), mentre **decifrare** è l'operazione legittima di lettura di un messaggio cifrato. È utile avere una notazione per relazionare fra loro il testo in chiaro, quello cifrato e le chiavi. Useremo la notazione $C = E_K(P)$ per indicare che la cifratura del testo in chiaro P usando la chiave K genera il testo cifrato C. Analogamente, $P = D_K(C)$ indica la decifrazione di C per estrarre il testo in chiaro. Segue quindi che

1. $D_K(E_K(P)) = P$

Questa notazione suggerisce che E e D sono semplicemente delle funzioni matematiche. Il punto difficile è che si tratta di funzioni di due parametri. Abbiamo scritto uno dei parametri (la chiave) come indice per distinguerlo dall'altro argomento (il messaggio). Una regola fondamentale della crittografia afferma che si deve sempre assumere che il criptoanalista conosca il metodo usato per la

cifratura e la decifrazione. In altre parole si deve supporre che il criptoanalista conosca nei dettagli il funzionamento del metodo *E* di cifratura e *D* di decifrazione. Lo sforzo necessario per inventare, collaudare e installare un nuovo algoritmo ogni volta che quello vecchio è compromesso (o si pensa che lo sia stato) ha sempre reso impraticabile la strada di tenere segreto l'algoritmo di cifratura. Credere di avere un algoritmo segreto quando non è vero produce più danno che guadagno. Qui entrano in gioco le chiavi. La chiave crittografica consiste di una stringa (relativamente) corta che identifica una particolare cifratura fra molte possibilità. Diversamente dal metodo generale, che può essere cambiato solo una volta ogni diversi anni, la chiave può essere cambiata tutte le volte che si ritiene necessario. Quindi il nostro modello base è dato da un metodo generale stabile e noto pubblicamente, però parametrizzato da un chiave segreta che può essere facilmente cambiata. L'idea che il criptoanalista conosca gli algoritmi e che il segreto stia esclusivamente nella chiave è detto **principio di Kerckhoff**, dal nome del crittografo fiammingo Auguste Kerckhoff, che per primo formulò il principio nel 1883 (Kerckhoff, 1883). Riassumendo:

> *principio di Kerckhoff: tutti gli algoritmi devono essere pubblici, solo le chiavi sono segrete.*

La mancanza di segretezza dell'algoritmo è fondamentale. Cercare di tenere segreto l'algoritmo, metodo noto in gergo come **sicurezza per occultamento**, non funziona mai. Inoltre, pubblicizzando l'algoritmo il crittografo ottiene gratuitamente la consulenza di un gran numero di accademici desiderosi di forzare il sistema, per poter pubblicare degli articoli in cui dimostrano quanto sono intelligenti. Se un gran numero di esperti ha tentato di forzare l'algoritmo per cinque anni dalla sua pubblicazione e nessuno c'è riuscito, probabilmente si tratta di un algoritmo decisamente solido.

Visto che la vera segretezza sta nella chiave, la sua lunghezza è un argomento importante nella progettazione di un sistema crittografico. Consideriamo, per esempio, un semplice lucchetto a combinazione, il principio generale è che le cifre vengono immesse in sequenza, il che è noto, mentre la chiave è segreta. Una chiave di due cifre significa che ci sono 100 possibilità. Una chiave di tre cifre significa 1000 possibilità, mentre per sei cifre ci sono un milione di possibilità. Più lunga è la chiave, più alto è il **fattore di lavoro** che il criptoanalista deve mettere in conto: il fattore di lavoro per rompere il sistema con una ricerca completa nello spazio delle chiavi cresce esponenzialmente con la lunghezza della chiave. La segretezza deriva dall'avere un algoritmo forte (ma pubblico) e una chiave lunga. Per evitare che il vostro fratellino legga le e-mail private che non lo riguardano, può bastare una chiave di 64 bit. Per il normale uso commerciale si dovrebbero usare almeno 128 bit. Per la sicurezza delle comunicazioni fra enti governativi sono auspicabili chiavi di almeno 256 bit.

Dal punto di vista del criptoanalista, i problemi di criptoanalisi hanno tre varianti principali. Quando si ha solo testo cifrato e nessun testo in chiaro si parla di un problema con **solo testo cifrato** (*ciphertext-only*). I criptogrammi che si trovano nella sezione enigmistica dei giornali sono di questo tipo. Quando invece si ha a disposizione del testo cifrato con il corrispondente testo in chiaro, si parla di problema con **testo in chiaro noto** (*known plaintext*). Infine, quando il criptoanalista ha la possibilità di cifrare dei pezzi di testo in chiaro a scelta, parliamo di problema con **testo in chiaro a scelta** (*chosen plaintext*). I criptogrammi dei giornali potrebbero essere risolti facilmente se il criptoanalista potesse avere risposta a domande del tipo: "qual è il testo cifrato di ABCDEFGHUKL?"

I principianti che si accostano alla crittografia spesso ipotizzano che se un cifrario riesce a resistere a un attacco del tipo solo testo cifrato, allora è sicuro. Questa ipotesi è troppo semplicistica. Infatti, in molti casi, i criptoanalisti riescono a indovinare informazioni importanti riguardo al testo in chiaro. Per esempio, la prima cosa che molti computer dicono quando vengono contattati è "login:". A questo punto, conoscendo testo in chiaro e testo cifrato corrispondente il lavoro del criptoanalista diventa molto più semplice. Il crittografo dovrebbe perciò cautelarsi, in modo da essere sicuro che il sistema non sia forzabile neanche se l'altra parte riesce a cifrare una quantità arbitraria di testo in chiaro. I metodi crittografici sono storicamente divisi in due categorie: cifrari a sostituzione e cifrari a trasposizione. Vedremo velocemente entrambi i casi, il che ci servirà come introduzione alla crittografia moderna.

Cifrari a sostituzione

In un cifrario a sostituzione, ogni lettera o gruppo di lettere viene rimpiazzato da un'altra lettera o gruppo di lettere per mascherare il messaggio. Uno dei cifrari più antichi che si conoscano è il **cifrario di Cesare**, attribuito a Giulio Cesare. In questo metodo, *a* diventa *D, b* diventa *E, e* diventa *F, ...* e z diventa *C.* Per esempio, la parola *attacco* diventa *DWWDFFR.* Negli esempi il testo in chiaro è scritto in minuscolo e quello cifrato in maiuscolo.

Una semplice generalizzazione del cifrario di Cesare consiste nello spostare l'alfabeto del testo cifrato di *k* lettere, dove *k* vale tre per il cifrario di Cesare descritto sopra. In questo caso *k* diventa la chiave del metodo generale, che consiste nell'avere un alfabeto spostato circolarmente. Il cifrario di Cesare può anche aver tratto in inganno Pompeo, ma da allora non ha più ingannato nessuno.

Il miglioramento successivo consiste nel far sì che ognuno dei simboli del testo in chiaro (diciamo le lettere dell'alfabeto, per semplicità) corrisponda ad altre lettere. Per esempio:

1. Testo in chiaro:abcdefghijklmnopqrstuvwxyz
2. Testo cifrato:QWERTYUIOPASDFGHJKLZXCVBNM

Il sistema generale per la sostituzione simbolo a simbolo viene chiamato **sostituzione monoalfabetica**, dove la chiave è la stringa di lettere che corrisponde all'intero alfabeto. Con la chiave di cui sopra, la parola *attacco* diventa *QZZQEEG.*

A prima vista questo può sembrare un sistema sicuro, perché anche se il criptoanalista conosce il metodo (sostituzione lettera a lettera), non conosce però quale delle $26! \approx 4 \times 10^{26}$ possibili chiavi viene usata. In questo caso, diversamente dal cifrario di Cesare, provare tutte le chiavi non è un approccio promettente. Anche alla velocità di 1 ns per chiave un computer impiegherebbe 10^{10} anni per provare tutte le chiavi. A ogni modo, data una quantità anche molto piccola di testo cifrato, il cifrario può essere forzato facilmente. L'attacco base fa leva sulle proprietà statistiche dei linguaggi naturali. In inglese, per esempio, la *e* è la lettera più comune, seguita da *t, o, a, n, i* e così via. Le combinazioni di due lettere, o **digrammi** più comuni sono *th, in, er, re, an.* Le combinazioni di tre lettere, o **trigrammi** più comuni sono *the, ing, and* e *ion.*

Un criptoanalista che voglia forzare un cifrario monoalfabetico comincerebbe contando la frequenza relativa di tutte le lettere nel testo cifrato. A quel punto può procedere per tentativi, associando la lettera più comune con la *e,* quella successiva nell'ordine alla *t.* Porrebbe poi guardare i trigrammi per arrivare a scoprirne uno frequente della forma *tXe,* il che gli suggerisce che *X* è *h.* Analogamente, scoprirà che la sequenza *thYt* appare spesso, quindi la *Y* è probabilmente una *a.*

Con queste informazioni può cercare i trigrammi nella forma *aZW,* che molto probabilmente corrisponde a *and.* Il criptoanalista ricostruisce per tentativi il testo in chiaro, lettera per lettera, facendo delle ipotesi su lettere, digrammi e trigrammi più comuni e conoscendo le sequenze più probabili di vocali e consonanti.

Un altro approccio consiste nell'indovinare una parola o una frase che probabilmente si trovano dentro al testo. Per esempio, consideriamo il testo (in inglese) cifrato riportato qui sotto e preso da un'azienda che si occupa di contabilità (riportato in gruppi di cinque caratteri):

1. CTBMN BYCTC BTJDS QXBNS GSTJC BTSWX CTQTZ CQVUJ QJSGS TJQZZ MNQJS VLNSX VSZJU JDSTS JQUUS JUBXJ DSKSU JSNTK BGAQJ ZBGYQ TLCTZ BNYBN QJSW

Una parola che molto probabilmente appare nel messaggio, vista la sua sorgente, è *financial* (finanziario). Sapendo che *financial* ha una lettera (*i*) ripetuta con quattro lettere fra una posizione e l'altra, cerchiamo nel testo cifrato le lettere ripetute con questa distanza fra di loro. Troviamo 12 possibilità alle posizioni: 6, 15, 27, 31, 42, 48, 56, 66, 70, 71, 76 e 82. Solo due di queste, 31 e 42, hanno la lettera successiva (corrispondente a *n* nel testo in chiaro) ripetuta al posto giusto. Di queste, solo 31 ha la *a* nella posizione corretta, in questo modo sappiamo che *financial* comincia alla posizione 30. Da questo punto, si può dedurre facilmente la chiave usando le statistiche di frequenza delle lettere per i testi in inglese e cercando parole quasi finite da completare.

Cifrari a trasposizione

I cifrari a sostituzione conservano l'ordine dei simboli del testo in chiaro, limitandosi a mascherare la loro apparenza. I **cifrari a trasposizione**, al contrario, riordinano le lettere ma non le mascherano. La Figura 8.3 mostra un tipico cifrario a trasposizione, il cifrario a trasposizione colonnare (*columnar transposition*). La cifra ha come chiave una parola o una frase senza lettere ripetute. Nell'esempio la chiave è MEGABUCK. Lo scopo della chiave è quello di numerare le colonne: la colonna numero 1 è quella sotto la lettera della chiave più vicina all'inizio dell'alfabeto e così via. Il testo in chiaro viene scritto orizzontalmente, per righe, fino a riempire la matrice, eventualmente usando allo scopo anche alcuni caratteri di riempimento. Il testo cifrato viene letto per colonna, cominciando con quella che ha la lettera chiave più bassa.

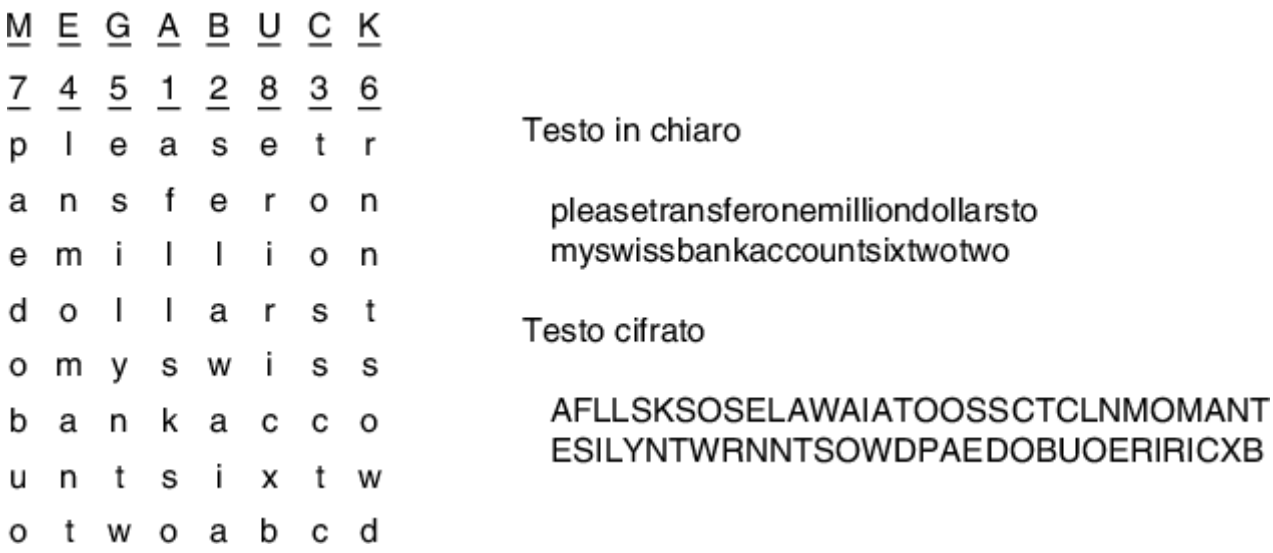


Figura 8.3 Cifrario a trasposizione.

Per forzare un cifrario a trasposizione il criptoanalista, come prima cosa, deve sapere che sta trattando con un cifrario a trasposizione. Guardando la frequenza delle lettere *E*, *T*, *A*, *O*, *I*, *N*, ecc è facile vedere se corrispondono alla normale frequenza per un testo in chiaro. Se questo è vero, si tratta chiaramente di un cifrario a trasposizione, infatti in un cifrario di questo tipo ogni lettera rappresenta sé stessa e la distribuzione di frequenza rimane intatta.

Il passo successivo consiste nel fare un'ipotesi sul numero di colonne. In molti casi si riesce a indovinare dal contesto una parola o frase che con buona probabilità appaiono nel testo. Per esempio, supponiamo che il criptoanalista sospetti che il testo in chiaro contenga da qualche parte la stringa *milliondollars.* Notiamo che i digrammi *MO*, *IL*, *LL*, *LA*, *IR* e *OS* appaiono nel testo come risultato della cifratura di tale stringa. Nel testo cifrato la lettera *O* segue la lettera *M* (cioè le due lettere sono verticalmente adiacenti nella colonna 4) questo ci fa ipotizzare che, all'interno della stringa *milliondollars,* queste siano separate da una distanza pari alla lunghezza della chiave. Se si fosse utilizzata una chiave di lunghezza sette, avremmo trovato invece i digrammi *MD, IO, LL, LL, IA, OR e NS.* In effetti, per ogni possibile lunghezza della chiave possiamo trovare un differente insieme di digrammi nel testo cifrato. Il criptoanalista è spesso in grado di determinare la lunghezza della chiave semplicemente provando tutte le possibilità.

Il passo conclusivo consiste nel trovare l'ordine delle colonne. Quando il numero delle colonne k è piccolo, ognuna delle $k(k-1)$ possibili coppie di colonne può essere esaminata per vedere se le frequenze di occorrenza dei suoi digrammi corrispondono a quelle tipiche della lingua con cui è scritto il messaggio. Si suppone poi che la coppia con il miglior risultato in questo test sia posizionata correttamente. A questo punto, si prosegue con le colonne rimanenti, finché non si identifica la colonna successiva alla coppia: questa sarà la colonna che produce la miglior distribuzione di frequenza dei digrammi e dei trigrammi. Analogamente, si procede a trovare la colonna che precede la coppia. Questo processo viene ripetuto finché non si identifica un possibile ordinamento delle colonne. A questo punto c'è già una buona possibilità che appaia il testo in chiaro (per esempio, se leggiamo *milloin* sapremo chiaramente dove è stato fatto un errore).

Alcuni cifrari a trasposizione prendono un blocco di lunghezza fissa in input e producono in output un altro blocco di lunghezza fissa. Questi cifrari possono essere descritti completamente dalla lista che indica l'ordine in cui i caratteri sono mandati in output. Per esempio, il cifrario della Figura 8.3 è un cifrario a blocchi di 64 bit e l'ordine dei caratteri in output è 4, 12, 20, 28, 36, 44, 52, 60, 5, 12, ..., 62. In altre parole, il quarto carattere in input, *a,* è il primo in output, seguito dal dodicesimo, *f* e così via.

Blocchi monouso

Costruire un cifrario imbattibile è molto facile e la tecnica per farlo è nota da decenni. Per prima cosa si prende una chiave formata da una stringa di bit generati a caso. Poi, si converte il testo in chiaro in un'altra stringa di bit, per esempio usando la rappresentazione ASCII dei i caratteri. Infine, si calcola lo XOR (*OR esclusivo*) delle due stringhe, bit per bit. Il testo cifrato risultante non può essere forzato, in quanto per ogni pezzo sufficientemente grande del testo cifrato, ogni lettera apparirà con la stessa frequenza, lo stesso vale per i digrammi e i trigrammi, ecc. Questo metodo, noto come **blocco monouso** (*one-time pad*), è immune a tutti gli attacchi presenti e futuri indipendentemente da quanta potenza di calcolo l'intruso abbia a disposizione. La ragione di questo fatto viene dalla teoria dell'informazione: il messaggio cifrato non contiene nessuna informazione in quanto contiene tutti i possibili testi in chiaro di una data lunghezza con eguale probabilità. Un esempio di come sono usati i blocchi monouso è dato dalla Figura 8.4. Nell'esempio, il messaggio 1 "I love you" viene convertito in ASCII a 7 bit. A quel punto il blocco monouso 1 viene scelto e usato per l'operazione di XOR con il messaggio per produrre il testo cifrato. Un criptoanalista potrebbe provare tutte le possibili combinazioni di blocchi monouso per vedere quale testo in chiaro emerga da ognuna di esse. Per esempio potrebbe usare il blocco monouso indicato con il numero 2 in figura: il risultato sarebbe il testo in chiaro 2 "Elvis lives", che potrebbe anche essere plausibile (l'argomento va però oltre lo scopo di questo libro). In effetti, per ogni testo ASCII di 11 caratteri in chiaro, possiamo trovare un blocco monouso che lo genera. Questo è quello che intendiamo dicendo che non c'è informazione nel testo cifrato: partendo da esso possiamo leggere qualunque messaggio della lunghezza corretta.

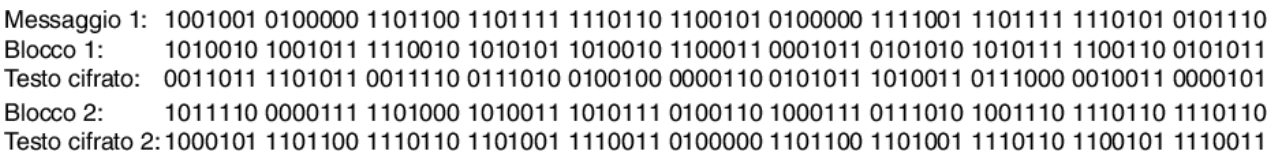


Figura 8.4 Un blocco monouso per la cifratura dà la possibilità di ottenere qualsiasi testo in chiaro dal testo cifrato cambiando il blocco.

I blocchi monouso sono ottimi in teoria, ma hanno diversi svantaggi nella pratica. Per cominciare, la chiave non può essere imparata a memoria, quindi sia il mittente sia il destinatario devono averne una copia. Se pensiamo che uno dei due soggetti possa essere catturato, vediamo che avere

delle chiavi scritte non è desiderabile. Inoltre, la quantità di dati che può essere trasmessa è limitata dalla lunghezza della chiave. Se una spia ha fortuna e scopre una gran quantità di dati, si può trovare nella situazione di non poterli trasmettere al quartier generale perché ha già usato la chiave a sua disposizione. Un altro problema del metodo è la debolezza nel trattare i caratteri persi o inseriti per sbaglio. Se a un certo punto della comunicazione il mittente e il destinatario perdono la sincronizzazione, il resto dei dati sarà completamente illeggibile.

Con l'avvento dei computer, la crittografia a blocchi monouso può diventare una strada praticabile per alcune applicazioni. La sorgente della chiave potrebbe essere uno speciale DVD contenente diversi gigabyte d'informazioni, da trasportare in una custodia da film e costruito con l'accortezza di aggiungere alcuni minuti di video all'inizio per sviare i sospetti. Chiaramente, alla velocità delle reti gigabit ethernet, dover inserire un nuovo DVD ogni 30 secondi può diventare molto fastidioso. Inoltre il DVD dovrebbe essere trasportato di persona dal mittente al destinatario prima di cominciare a trasmettere, il che limita fortemente l'utilità pratica di questo metodo.

Crittografia quantistica

Un'interessante risposta all'esigenza di trasmettere i blocchi monouso attraverso la rete arriva, inaspettatamente, dalla meccanica quantistica. Questa soluzione è ancora sperimentale, ma molto promettente. Se si riuscirà a perfezionare il metodo rendendolo efficiente, praticamente tutta la crittografia finirà per essere realizzata con i blocchi monouso, visto che è stato dimostrato che sono sicuri. Nel seguito spiegheremo come funziona questo metodo, detto **crittografia quantistica**. In particolare, descriveremo un protocollo, detto **BB84** dal nome dei suoi autori e dell'anno di pubblicazione (Bennet e Brassard, 1984). Un utente, Alice, vuole stabilire un blocco monouso con un secondo utente, Bob. Alice e Bob sono detti i **protagonisti**, gli attori principali della nostra storia. Per esempio, Bob è un banchiere con cui Alice vuole fare affari. I nomi "Alice" e "Bob" vengono usati per indicare i protagonisti in tutti gli articoli e i libri di crittografia degli ultimi dieci anni. I crittografi amano la tradizione. Se usassimo "Andy" e "Barbara" come nomi dei protagonisti, nessuno crederebbe a una riga di questo capitolo: per questo motivo ci atterremo alla tradizione. Se Alice e Bob riuscissero a stabilire un blocco monouso, lo potrebbero usare per comunicare in modo sicuro. La domanda è: come possono farlo senza essersi scambiati dei DVD in precedenza? Immaginiamo che Alice e Bob siano agli estremi opposti di una fibra ottica, che possono usare per inviare e ricevere impulsi luminosi. Un intruso attivo, che chiameremo Trudy, può inserirsi nella fibra per violare la sicurezza della comunicazione. Trudy può leggere tutti i bit e mandare messaggi falsi in entrambe le direzioni. La situazione può sembrare un vicolo cieco per Alice e Bob, invece con la crittografia quantistica possiamo fare luce sulla situazione.

La crittografia quantistica si basa sul fatto che la luce viene trasportata in piccoli pacchetti, chiamati **fotoni** che hanno delle proprietà particolari. Un altro fatto importante è che la luce può essere polarizzata facendola passare attraverso dei filtri polarizzatori, un fatto ben noto ai fotografi e a chi indossa occhiali da sole. Se un raggio di luce (cioè un flusso di fotoni) viene fatto passare attraverso un polarizzatore, tutti i fotoni emergenti saranno polarizzati nella direzione dell'asse del filtro polarizzatore (per esempio, un asse verticale). Se il raggio viene fatto passare attraverso un secondo filtro polarizzatore, l'intensità della luce emergente dal secondo filtro è proporzionale al quadrato del coseno dell'angolo fra gli assi dei due polarizzatori. Se i due assi sono perpendicolari, nessun fotone riesce a passare. L'orientamento assoluto dei due filtri non è importante, l'unica cosa rilevante è l'angolo fra i loro assi. Per generare un blocco monouso, Alice ha bisogno di due

insiemi di filtri, il primo contiene un filtro verticale e uno orizzontale. Questa scelta è chiamata una **base rettilinea**. Una base è un sistema di coordinate. Il secondo insieme è simile al primo, solo con una rotazione aggiuntiva di 45 gradi. In questo modo l'asse di un filtro è orientato da in basso a sinistra a in alto a destra, mentre l'altro va da in alto a sinistra a in basso a destra. Questa scelta è detta **base diagonale**. Quindi Alice ha due basi che può inserire nel raggio di luce, supponiamo senza limitazioni di velocità. In realtà Alice non ha quattro filtri separati, ma un cristallo la cui polarizzazione può essere cambiata elettronicamente in modo rapido in ognuna delle quattro direzioni indicate sopra. Bob ha la stessa attrezzatura di Alice. Il fatto che Bob e Alice abbiano a disposizione due basi ciascuno è essenziale per la crittografia quantistica. Per ogni base, Alice identifica il bit 0 con una certa direzione e 1' 1 con la direzione ortogonale allo 0. Nell'esempio che segue assumeremo che lo 0 sia la direzione verticale, mentre l'1 quella orizzontale. Un discorso analogo vale per la base diagonale: Alice assegna lo 0 alla direzione dal basso a sinistra verso l'alto a destra e 1 all'altra direzione. Alice trasmette queste scelte in chiaro a Bob.

A questo punto Alice genera un blocco monouso, per esempio basandosi su un generatore di numeri casuali (un argomento complesso da discutere a parte) e lo trasmette a Bob bit per bit, scegliendo per ogni bit una delle due basi in modo casuale. Per esempio, Alice può scegliere di usare per il primo bit la base diagonale, per il secondo quella rettilinea, poi rettilinea di nuovo, ecc. Per trasmettere il blocco monouso 1001110010100110 con queste basi, Alice dovrebbe inviare i fotoni come nella Figura 8.5(a). Dato il blocco monouso e l'ordine delle basi, la polarizzazione da utilizzare per ogni bit è determinata univocamente. I bit inviati come singoli fotoni sono chiamati **qubit**. Bob non sa quale base utilizzare, quindi ne sceglie una a caso per ogni fotone in arrivo, come mostrato nella Figura 8.5(b). Se Bob sceglie la base corretta, avrà anche il bit corretto. Se invece sceglie la base sbagliata otterrà un valore casuale. Infatti, quando un fotone attraversa un filtro polarizzatore formando un angolo di 45 gradi fra la sua polarizzazione e l'asse del polarizzatore, il risultato è che il fotone collassa in modo casuale e con eguale probabilità o alla polarizzazione del filtro o alla polarizzazione a esso ortogonale. Questo risultato deriva dalle leggi fondamentali della meccanica quantistica. Il risultato è che alcuni bit sono ricevuti correttamente mentre altri hanno dei valori casuali, come mostrato nella Figura 8.5(c). Bob non è ancora in grado di sapere quali sono i bit corretti e quali quelli casuali. Come può fare Bob per avere le informazioni necessarie a sapere quando ha usato la base giusta e quando quella sbagliata? Semplicemente comunica in chiaro ad Alice quali basi ha utilizzato e Alice gli risponde, di nuovo in chiaro, dicendogli quali sono giuste e quali sbagliate, come mostrato nella Figura 8.5(d). Usando queste informazioni, riescono entrambi a costruire una stringa di bit trasmessi correttamente, come nella Figura 8.5(e). Questa stringa, che avrà una lunghezza media pari alla metà della stringa di bit originale, è nota a tutti e due i protagonisti e quindi potrà essere effettivamente usata come blocco monouso. Notiamo che per stabilire un blocco monouso di una certa lunghezza Alice deve semplicemente inviare una stringa di bit leggermente più lunga del doppio necessario e il problema è risolto. Abbiamo finora trascurato il ruolo di Trudy. Supponiamo che Trudy sia curiosa di sapere che cosa dice Alice e quindi esegua una deviazione nella fibra ottica per inserire i suoi apparati di ricezione e trasmissione. Sfortunatamente, per Trudy, nemmeno lei riesce a sapere quali basi usare per ogni fotone entrante. La cosa migliore che può fare è scegliere a caso le basi, così come fa Bob. Un esempio delle possibili scelte di Trudy è mostrato nella Figura 8.5(f). Quando poi Bob trasmette (in chiaro) ad Alice l'elenco delle basi che ha usato e Alice gli comunica (in chiaro) quali sono giuste e quali no, anche Trudy riesce a sapere quando ha usato le basi giuste e quando ha sbagliato. Nella Figura 8.5 vediamo che Trudy ha indovinato le basi per i bit 0, 1, 2, 3, 4, 6, 8, 12 e 13. Però Trudy riesce a sapere dalla risposta di Alice nella Figura 8.5(d) che

solamente i bit 1, 3, 7, 8, 10, 11, 12 e 14 fanno parte del blocco monouso. Per quattro di questi bit aveva indovinato la sequenza delle basi (1, 3, 8 e 12), ma per gli altri quattro (7, 10, 11 e 14) aveva sbagliato e quindi non ne conosce i valori. Quindi Bob sa che il blocco monouso comincia con 01011001, come nella Figura 8.5(e), mentre Trudy sa che il blocco vale 01?1??0?, come nella Figura 8.5(g). Ovviamente Bob e Alice sono a conoscenza del fatto che Trudy può aver catturato parte del loro blocco monouso, quindi desiderano ridurre ulteriormente la quantità di informazione che Trudy ha a disposizione. Riescono a farlo operando una trasformazione sui dati del blocco. Per esempio, possono dividere il blocco monouso in blocchi di 1024 bit e calcolare il quadrato di ciascun blocco per formare dei numeri di 2048 bit e poi usare la concatenazione di questi numeri a 2048 bit per formare il blocco monouso che utilizzeranno per la comunicazione. Trudy, con la sola conoscenza parziale del blocco, non riesce a calcolare i quadrati dei blocchi e quindi si ritrova senza niente in mano. La trasformazione che porta dal blocco originale a quello finale, riducendo così l'informazione in mano a Trudy, è detta **amplificazione della privacy**. In pratica, al posto del calcolo del quadrato, vengono usate delle trasformazioni complesse in cui tutti i bit dell'output dipendono da tutti i bit dell'input. I guai per Trudy non sono finiti qui. Infatti, non solo non riesce a sapere qual è il blocco monouso, ma la sua presenza viene anche rilevata. Per poter ascoltare la comunicazione, facendo pensare a Bob che la comunicazione sia in realtà con Alice, Trudy deve ritrasmettere tutti i bit ricevuti. Il punto è che il massimo che può fare è trasmettere i bit ricevuti usando la polarizzazione che ha usato per riceverli, che però sarà sbagliata in media la metà delle volte. Dal punto di vista di Bob, questo si rifletterà in una frequenza elevata di errori di ricezione.

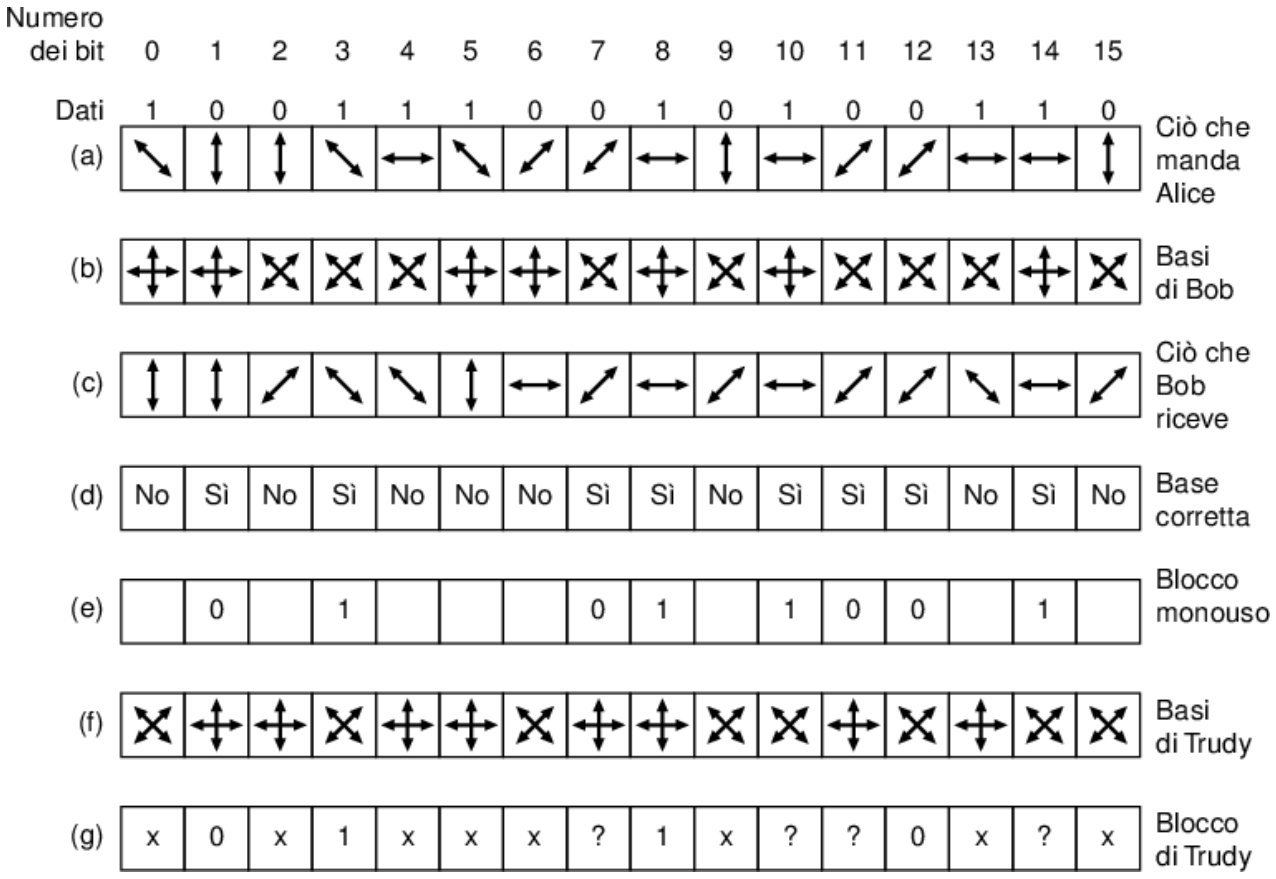


Figura 8.5 Esempio di crittografia quantistica.

Quando Alice comincia a inviare i dati, li codifica usando dei metodi forti di correzione degli errori. Per Bob, un errore di 1 bit nel blocco monouso è la stessa cosa di un errore di trasmissione: in entrambi i casi ottiene un bit sbagliato. Se la codifica per la correzione degli errori è sufficientemente forte, Bob riesce a correggere gli errori e a ripristinare il messaggio originale. Bob può facilmente contare il numero di errori di trasmissione e confrontarli con la frequenza di errore che ci si aspetta per la linea in questione. Se scopre una quantità di errori anomala, saprà che Trudy ha ascoltato la linea e potrà agire di conseguenza (per esempio potrà dire ad Alice di usare un canale radio oppure chiamare la polizia, ecc). Se Trudy potesse clonare i fotoni, così da avere un fotone da esaminare e uno identico da trasmettere a Bob, potrebbe evitare di essere scoperta, ma al momento non sono noti metodi per clonare i fotoni. Anche se Trudy riuscisse a clonare i fotoni, questo non ridurrebbe minimamente l'utilità della crittografia quantistica per stabilire dei blocchi monouso.

La crittografia quantistica è stata già dimostrata per distanze fino a 60 km su fibra ottica, anche se, allo stato attuale, l'attrezzatura necessaria rimane complessa e costosa. A ogni modo, l'idea è promettente. Per ulteriori informazioni sulla crittografia quantistica si rimanda a (Mullins, 2002).

Due principi crittografici fondamentali

Due principi crittografici fondamentali sono alla base dello studio dei diversi sistemi crittografici che discuteremo nei paragrafi successivi.

Ridondanza

Il primo principio afferma che tutti i messaggi cifrati devono contenere una qualche forma di ridondanza, cioè d'informazione non necessaria alla comprensione del messaggio. Utilizziamo un esempio per chiarire come mai serve questa informazione aggiuntiva. Consideriamo un'azienda di vendita per corrispondenza, la "The Couch Potato" (TCP) con un catalogo di 60.000 prodotti. I programmatori della TCP, pensando di aumentare l'efficienza, decidono che i messaggi contenenti gli ordini devono consistere di 16 byte con il nome del cliente, seguiti da 3 byte per i dati (1 byte per la quantità e 2 byte per il numero del prodotto). Gli ultimi 3 byte vengono cifrati usando una chiave molto lunga e nota solo alla TCP e ai suoi clienti.

A prima vista questo metodo potrebbe sembrare sicuro e in un certo modo lo è, visto che gli intrusi passivi non riescono a decifrare i messaggi. Sfortunatamente però, ha un difetto fatale che lo rende inutile. Supponiamo che un'impiegata licenziata di recente voglia punire la TCP per averla licenziata. Prima di lasciare l'azienda, si scarica la lista dei clienti, lavora tutta la notte e scrive un programma che genera degli ordini fittizi usando però i veri nomi dei clienti. Visto che non ha la lista delle chiavi, l'ex impiegata inserisce dei numeri casuali negli ultimi 3 byte dei messaggi, quindi invia centinaia di ordini alla TCP. Quando questi messaggi arrivano a destinazione, il computer della TCP usa il nome del cliente per trovare la chiave e decifrare il messaggio. Sfortunatamente per la TCP, praticamente tutti i messaggi di 3 byte sono validi e quindi il computer comincia a stampare le istruzioni per l'invio della merce. Anche se può sembrare strano che un cliente ordini 837 altalene per bambini e 540 vasche di sabbia, per quanto ne sa il computer, il cliente potrebbe anche essere in procinto di aprire una catena in franchising di parchi giochi. In questo modo un intruso attivo (l'ex impiegata) può causare una notevole quantità di danni, anche se non riesce a decifrare i messaggi che vengono generati dal suo computer. Questo

problema può essere risolto aggiungendo della ridondanza a tutti i messaggi. Per esempio, se i messaggi degli ordini vengono estesi a 12 byte, di cui i primi 9 devono essere a 0, allora questo tipo di attacco non funziona più, in quanto l'ex impiegata non riesce più a generare un flusso di messaggi validi. La morale della storia è che tutti i messaggi devono contenere una quantità considerevole di ridondanza, in modo da evitare che intrusi attivi possano generare dati casuali e che questi vengano in seguito interpretati come messaggi validi.

D'altra parte, aggiungere della ridondanza facilita il lavoro del criptoanalista che tenta di violare il codice. Supponiamo che l'azienda di vendita per corrispondenza sia molto competitiva e che il suo principale concorrente, "The Sofâ Tube", desideri fortemente sapere quante scatole di sabbia la TCP sta vendendo. Per fare questo intercetta le comunicazioni sulla linea telefonica della TCP. Nello schema originale, con i messaggi da 3 byte, la criptoanalisi era praticamente impossibile. Infatti, se il criptoanalista tentava delle chiavi a caso, non aveva tuttavia modo di capire se l'ipotesi era giusta o sbagliata. Questo perché praticamente tutti i messaggi erano tecnicamente buoni. Con il nuovo schema a 12 byte, per il criptoanalista è più facile capire se un messaggio è valido oppure no.

Principio crittografico 1: i messaggi devono contenere ridondanza.

In altre parole, il destinatario, dopo aver decifrato il messaggio, deve essere in grado di stabilirne la validità con un semplice esame del contenuto o con un breve calcolo. Questo tipo di ridondanza è necessario per prevenire gli attacchi degli intrusi attivi, che consistono nell'inviare dei messaggi generati a caso per ingannare il destinatario, che li decifra come se fossero validi. Il rovescio della medaglia è che la stessa ridondanza semplifica, agli intrusi passivi, il compito di decifrare i messaggi. Inoltre, la ridondanza non dovrebbe mai presentarsi nella forma di n valori a zero posti all'inizio o alla fine del messaggio: passare messaggi così formati attraverso un algoritmo crittografico rende il risultato più regolare, facilitando quindi il lavoro dei criptoanalisti. La soluzione migliore consiste nell'usare dei codici hash crittografici, come vedremo in seguito. Per il momento pensate a questi come a dei checksum migliorati.

Tornando per un attimo alla crittografia quantistica, vediamo che la ridondanza ha un ruolo anche in quel caso. Visto che Trudy ha intercettato dei fotoni, alcuni bit nei blocchi monouso di Bob sono sbagliati. Bob ha bisogno di avere ridondanza nei messaggi in arrivo per capire che si sono verificati degli errori. Una forma molto semplice di ridondanza consiste nel ripetere il messaggio due volte. Se le due copie non sono identiche, Bob individua un'anomalia: c'è molto rumore nella trasmissione in fibra oppure qualcuno sta ascoltando la comunicazione. Ovviamente, trasmettere il messaggio due volte è un'esagerazione, un metodo più efficiente per identificare gli errori è dato dall'uso della codifica di Hamming o di Reed-Solomon. Da questa discussione dovrebbe essere chiaro che l'uso di qualche forma di ridondanza è necessario per distinguere i messaggi validi da quelli che non lo sono, specialmente se ci troviamo di fronte a intrusi attivi.

Attualità

Il secondo principio crittografico afferma che è necessario avere la possibilità di verificare che ogni messaggio ricevuto sia attuale, cioè trasmesso di recente. Questo serve per evitare che intrusi attivi possano inviare messaggi vecchi spacciandoli per nuovi. Se non ci sono misure di questo tipo, la nostra ex impiegata potrebbe ascoltare la linea telefonica di TCP e mandare delle semplici ripetizioni dei messaggi validi che ha intercettato. Possiamo quindi enunciare il secondo principio.

Principio crittografico 2: sono necessari dei metodi per prevenire gli attacchi di tipo ripetizione.

Una possibilità consiste nell'includere un timestamp (data e ora) valido, per esempio, solamente per 10 secondi, il destinatario può quindi tenere traccia dei messaggi degli ultimi 10 secondi, per confrontarli con i messaggi in arrivo e filtrare i duplicati. I messaggi più vecchi di 10 secondi vengono scartati in quanto troppo vecchi. Altri metodi, oltre a questo del timestamp, verranno discussi nel seguito.

8.2 Algoritmi a chiave simmetrica

La crittografia moderna usa le stesse idee base della crittografia classica (come la trasposizione e la sostituzione), ma con un'enfasi diversa. Tradizionalmente i crittografi hanno sempre usato degli algoritmi semplici. Al giorno d'oggi è vero il contrario: l'idea è quella di rendere l'algoritmo di cifratura così complesso e involuto per fare in modo che, anche se il criptoanalista avesse a disposizione una vasta quantità di testo cifrato di sua scelta, non riesca tuttavia a capirci nulla senza avere la chiave crittografica. La prima classe di algoritmi di cifratura che studieremo in questo capitolo va sotto il nome di **algoritmi a chiave simmetrica**, in quanto usano la stessa chiave per cifrare e decifrare i messaggi. La Figura 8.2 illustra l'uso di un algoritmo a chiave simmetrica. In particolare ci concentreremo sui **cifrari a blocco**, che prendono un blocco di *n* bit dal testo in chiaro e li trasformano usando la chiave in *n* bit cifrati.

Gli algoritmi crittografici possono essere implementati in hardware (per aumentarne la velocità) o software (per aumentarne la flessibilità). Anche se nel nostro discorso ci concentreremo principalmente sugli algoritmi e i protocolli, che sono indipendenti dall'implementazione, può essere interessante spendere alcune parole su come si costruisce l'hardware crittografico. La trasposizione e la sostituzione si possono implementare con semplici circuiti elettrici. La Figura 8.6(a) mostra un dispositivo, noto come **P-box** (*permutation box*, scatola di permutazione), usato per la trasposizione con input a 8 bit. Se i bit vengono numerati dall'alto al basso con 01234567, l'output della P-box è 24506713. La P-box può compiere qualunque tipo di trasposizione, semplicemente impostando in modo opportuno i collegamenti interni. Inoltre, la P-box lavora praticamente alla velocità della luce, in quanto non deve compiere nessun calcolo ma solamente propagare il segnale in ingresso. La sua struttura segue la legge di Kerckhoff: l'intruso sa che il principio generale consiste nel permutare i bit, quello che non sa è quale bit va dove.

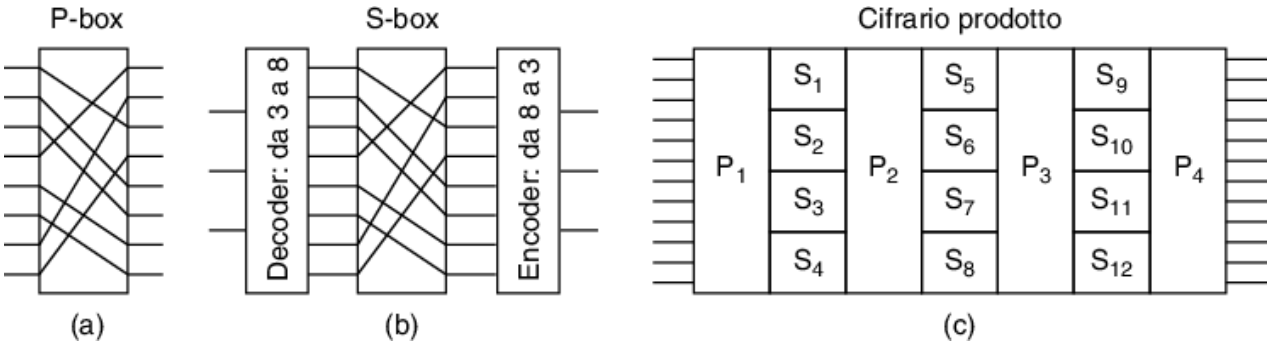


Figura 8.6 Blocchi elementari dei cifrari prodotto. (a) P-box. (b) S-box. (c) Prodotto.

Le sostituzioni sono fatte da **S-box** (*serial box*, scatola seriale), come mostrato nella Figura 8.6(b). In questo esempio abbiamo in input un testo in chiaro di 3 bit e in output un testo cifrato di 3 bit. I 3 bit dell'input scelgono una delle 8 linee che escono dal primo stadio e le impostano a 1, tutte le altre linee sono a 0. Il secondo stadio è una P-box. Il terzo stadio codifica le linee di input selezionate riportandole in binario. Con i collegamenti mostrati nella figura, se i numeri da 0 a 7 arrivano in input uno alla volta (01234567), la sequenza di output sarà: 24506713. In altre parole, 0 è stato rimpiazzato da 2, 1 da 4, ecc. Per quanto esposto, cambiando i collegamenti interni della P-box che si trova dentro alla S-box possiamo ottenere qualunque tipo di sostituzione. Inoltre, un dispositivo di questo tipo può essere costruito in hardware per raggiungere notevoli velocità, visto che la codifica e decodifica hanno dei ritardi di propagazioni dovuti alle porte logiche inferiori al nanosecondo, mentre la propagazione nella P-box può essere anche inferiore al picosecondo.

La vera potenza di questi elementi base si vede quando si collegano in cascata una serie di P-box e S-box per formare un **cifrario prodotto**, come mostrato nella Figura 8.6(c). In questo esempio, 12 linee di input subiscono nel primo stadio (P_1) una trasposizione (ovvero una permutazione). Nella seconda fase, l'input viene spezzato in quattro gruppi da 3 bit, ognuno dei quali viene sostituito indipendentemente dagli altri (da S_1 a S_4). Questa disposizione mostra un metodo per approssimare una grande S-box con diversi S-box più piccole. È utile perché le S-box piccole sono pratiche per una implementazione hardware (una S-box da 8 bit può essere realizzata come una tabella indicizzata con 256 voci), mentre le grosse S-box diventano meno maneggevoli da costruire (una S-box da 12 bit che avrà bisogno almeno di $2^{12} = 4096$ cavi incrociati nello stadio intermedio). Anche se questo metodo è meno generale, continua a essere molto potente. Includendo un numero sufficiente di stadi nel cifrario prodotto, l'output generato diventa una funzione estremamente complicata dell'input. I cifrari prodotto che operano su input di k bit per produrre output di k bit sono molto comuni. Tipicamente, k vale 64 o 256 e l'implementazione hardware consta di almeno 18 stadi fisici, al posto dei 7 mostrati nella Figura 8.6(c). Un'implementazione software viene programmata con un loop di almeno 8 iterazioni. Ogni iterazione opera una sostituzione di tipo S-box su dei sottoblocchi dei dati di grandezza pari a 64 o 256 bit, seguita da una permutazione che mescola gli output delle S-box. Spesso si trova una permutazione speciale all'inizio e alla fine del calcolo. Nella letteratura, queste iterazioni sono dette **round** (*cicli*).

DES – data encryption standard

Nel gennaio 1977 il governo degli Stati Uniti adottò come standard per le informazioni non riservate un cifrario prodotto sviluppato da IBM. Questo cifrario, **DES** (*data encryption standard*, standard per la cifratura dei dati), fu quindi largamente utilizzato anche dall'industria per i prodotti riguardanti la sicurezza. DES non è più sicuro nella sua forma originale, ma in una forma modificata è ancora utile.

Uno schema del funzionamento di DES è mostrato nella Figura 8.7(a). Il testo in chiaro viene cifrato in blocchi di 64 bit, che generano 64 bit di testo cifrato. L'algoritmo è parametrizzato da una chiave a 56 bit e ha 19 stadi distinti, il primo stadio è indipendente dalla chiave e consiste nella trasposizione dei 64 bit del testo in chiaro. L'ultimo stadio è esattamente l'inverso del primo. Il penultimo stadio consiste nello scambiare i 32 bit più a sinistra con i 32 più a destra. I rimanenti 16 stadi sono funzionalmente identici, ma sono parametrizzati da diverse funzioni della chiave.

L'algoritmo è stato progettato per permettere la decifrazione con la stessa chiave usata per la cifratura, proprietà necessaria in un algoritmo a chiave simmetrica. I passi vengono semplicemente percorsi in ordine inverso nei due casi.

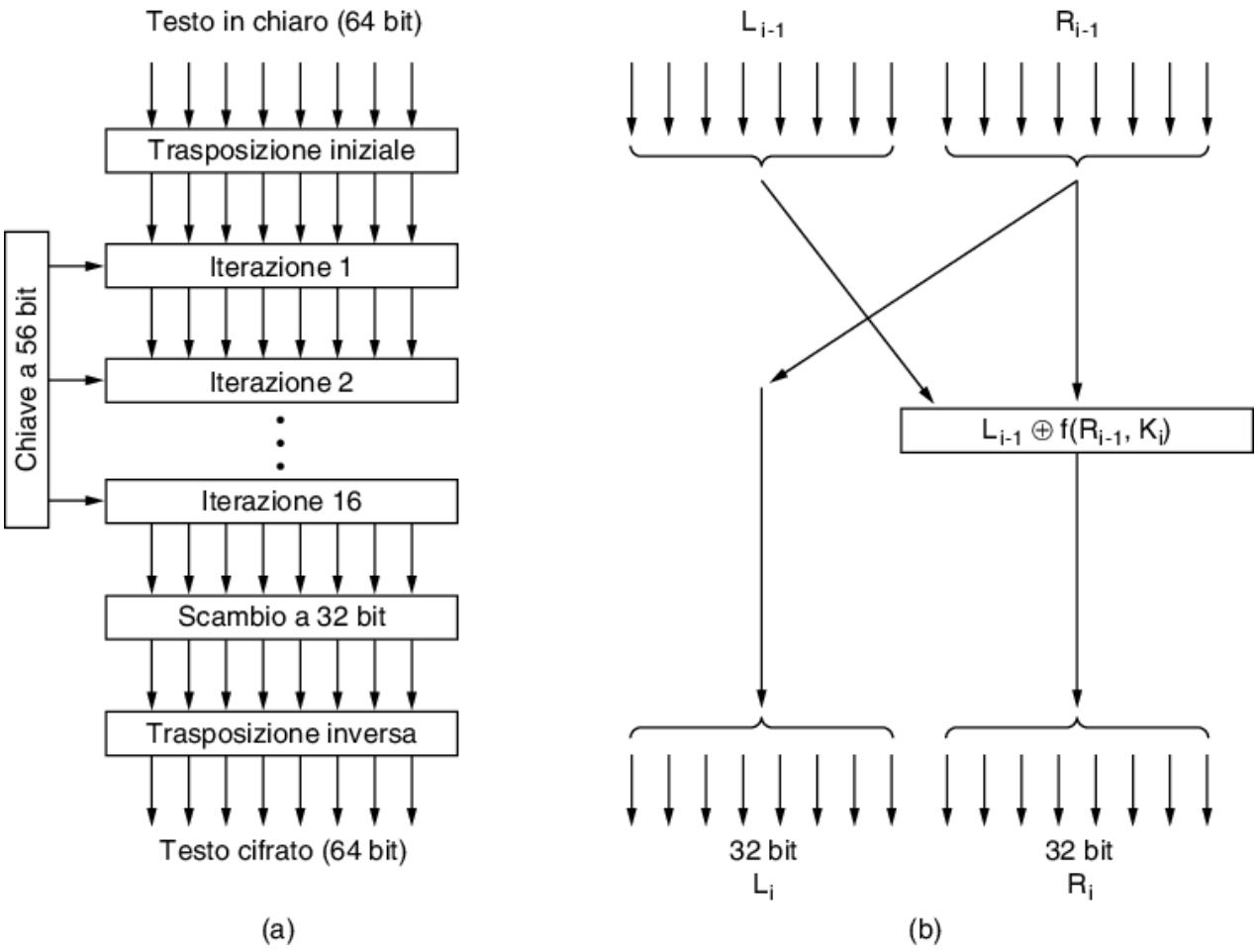


Figura 8.7 Il DES. (a) Struttura generale. (b) Dettaglio di una iterazione. Il simbolo + incluso in un cerchio indica l'OR esclusivo.

Il funzionamento di uno degli stadi intermedi è illustrato nella Figura 8.7(b). Ogni stadio prende due ingressi a 32 bit e produce due uscite a 32 bit. L'output di sinistra è semplicemente una copia dell'output di destra. L'output di destra consiste nello XOR bit per bit dell'input di sinistra con una funzione dell'input di destra e della chiave per questo stadio, K_t. Tutta la complessità è in questa funzione.

La funzione consiste di quattro passi eseguiti in sequenza. Come prima cosa si costruisce un numero di 48 bit, *E,* espandendo i 32 bit di destra, R_{i-1}, usando una regola fissa di trasposizione e duplicazione. Nel secondo passo si esegue l'operazione di XOR fra *E* e R_{i-1}Questo output viene diviso in otto gruppi di 6 bit ciascuno, ognuno dei quali viene passato attraverso una diversa S-box. Ognuno dei 64 possibili input alla S-box viene associato a un output di 4 bit. Infine, si fanno passare gli 8 gruppi di 4 bit attraverso una P-box. Si usa una chiave differente per ognuna delle 16 iterazioni. Una trasposizione di 56 bit è applicata alla chiave prima di cominciare a operare con

l'algoritmo. Subito prima di ogni iterazione si divide la chiave in due unità di 28 bit, ciascuna delle quali viene ruotata a sinistra di un numero di bit che dipende dal numero dell'iterazione. Ogni valore di K_i è determinato da questa chiave ruotata, applicandogli un'ulteriore trasposizione di 56 bit. A ogni round si estrae e permuta un diverso sottoinsieme di 48 bit presi dalla chiave a 56 bit.

Una tecnica che viene talvolta utilizzata per rafforzare DES si chiama **sbiancamento** (*whitening*). Consiste nell'operare lo XOR di una chiave casuale a 64 bit su ogni blocco di dati del testo in chiaro, usare il risultato come input per il DES e infine eseguire lo XOR di una seconda chiave a 64 bit; il risultato finale è il testo cifrato da trasmettere. Lo sbiancamento si rimuove facilmente eseguendo le operazioni inverse (se il destinatario ha le due chiavi di sbiancamento). Questa tecnica rende la ricerca completa nello spazio delle chiavi molto più lunga, poiché in definitiva aggiunge nuovi bit alla chiave stessa. Notiamo che per ogni blocco viene utilizzata la stessa chiave di sbiancamento (cioè esiste una sola chiave di sbiancamento). DES è stato circondato da controversie sin dalla sua nascita. Fu basato su un cifrario sviluppato e brevettato da IBM chiamato Lucifer, con la differenza che il cifrario IBM usava una chiave di 128 bit al posto dei 56 di DES. Quando il governo federale degli Stati Uniti volle creare un cifrario standard per i documenti non riservati, "invitò" IBM per "discutere" la questione con NSA, la parte del governo degli Stati Uniti che si occupa di forzare i cifrari e che è anche il più grande datore di lavoro al mondo per matematici e criptoanalisti. NSA è così segreta che nell'industria, per scherzo, si dice:

> *Domanda: Che cosa significa l'acronimo NSA?*
> *Risposta: No Such Agency (non esiste questa agenzia).*

A parte gli scherzi, NSA sta per *national security agency* (agenzia per la sicurezza nazionale).

Dopo questa serie di discussioni, IBM ridusse la lunghezza della chiave da 128 a 56 bit e decise di tenere segreto il processo seguito per sviluppare DES. Molti sospettarono che la lunghezza della chiave fosse stata ridotta per garantire a NSA la possibilità di forzare DES, cosa che sarebbe rimasta fuori dalla portata di organizzazioni con un budget inferiore. Riguardo alla segretezza della struttura si supponeva che questa nascondesse una backdoor che poteva facilitare ancora di più la forzatura di DES da parte di NSA. Le polemiche furono ulteriormente alimentate, quando un dipendente di NSA chiese in modo discreto ad IEEE di cancellare una conferenza sulla crittografia già programmata. NSA ha sempre negato tutto.

Nel 1977, due ricercatori di Stanford, Diffie e Hellman progettarono una macchina per forzare DES e stimarono che potesse essere costruita con circa 20 milioni di dollari. Dati un piccolo pezzo di testo in chiaro e il testo cifrato corrispondente, questa macchina poteva trovare la chiave con un ricerca completa nello spazio delle 2^{56} chiavi in meno di 1 giorno. Oggi ce la si gioca; una tale macchina esiste, è in vendita e fabbricarla costa meno di 10.000 dollari (Kumar et al., 2006).

Triplo DES

Fin dal 1979 IBM si rese conto che la chiave di DES era troppo corta e trovò un modo per incrementarla in modo efficace, usando la cifratura tripla (Tuchman, 1979). Il metodo scelto, che è stato poi incorporato nello standard internazionale 8732, è illustrato nella Figura 8.8. Vengono usate due chiavi e tre stadi. Nel primo stadio, il testo in chiaro viene cifrato con DES nel modo solito usando la chiave K_1 Nel secondo stadio, DES viene usato in modalità di decifrazione usando la chiave K_2 Infine un'altra cifratura viene fatta con la chiave K_1.

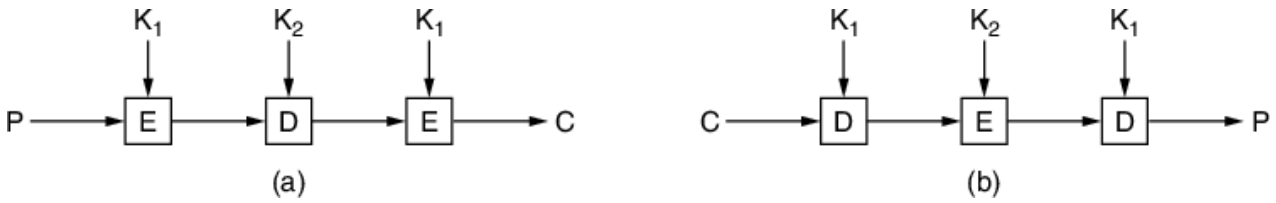


Figura 8.8 (a) Cifratura tripla con DES. (b) Decifrazione.

Questa struttura fa immediatamente nascere due domande. La prima è: perché vengono usate due chiavi e non tre? La seconda è: perché si usa **EDE** (*encrypt decrypt encrypt*) al posto di **EEE** (*encrypt encrypt encrypt*)? La ragione per cui vengono usate due chiavi è che anche il più paranoico dei crittografi ritiene che, ai giorni nostri, 112 bit offrano una lunghezza adeguata per le normali applicazioni commerciali (fra i crittografi la paranoia è considerata un pregio e non un difetto). Usare 168 bit vorrebbe dire aggiungere del lavoro per gestire e trasportare un'altra chiave senza ottenere un reale vantaggio. La sequenza cifra, decifra, cifra è stata scelta per compatibilità all'indietro con i sistemi DES a chiave singola. Sia la funzione di cifratura sia quella di decifrazione sono mappe fra insiemi di 64 bit. Da un punto di vista crittografico, le due mappe sono egualmente forti. Usando EDE, al posto di EEE, un computer che usa la cifratura tripla può parlare con uno che usa la cifratura singola, semplicemente impostando $K_1 = K_2$. Questa proprietà permette di introdurre la cifratura tripla per gradi, qualcosa che non interessa troppo ai crittografi accademici, ma che è di grande importanza per IBM e i suoi clienti.

AES – advanced encryption standard

Poiché DES stava raggiungendo la fine della sua vita utile, anche nella variante triplo DES, il **NIST** (*national institute of standards and technology,* istituto nazionale per gli standard e la tecnologia) decise che il governo aveva bisogno di un nuovo standard per le comunicazioni non riservate. NIST è l'agenzia del dipartimento del commercio, che ha il compito di approvare gli standard federali per il governo degli Stati Uniti. NIST conosceva bene le diatribe sorte intorno a DES e sapeva che, se avesse annunciato un nuovo standard, chiunque sapesse qualcosa di crittografia avrebbe pensato automaticamente che NSA aveva una backdoor per questo standard e avrebbe potuto leggere tutti i messaggi. In queste condizioni probabilmente nessuno avrebbe usato il nuovo standard, che sarebbe quindi certamente finito nel dimenticatoio. Per questo motivo NIST adottò un approccio diverso dal quello della normale burocrazia governativa e sponsorizzò un concorso crittografico. Nel gennaio 1997 i ricercatori di tutto il mondo furono invitati a inviare delle proposte per un nuovo standard che avrebbe preso il nome di **AES** (*advanced encryption standard*, standard per la crittografia avanzata). Le regole del concorso erano:

1. l'algoritmo doveva essere un cifrario simmetrico a blocchi
2. la struttura doveva essere interamente pubblica
3. doveva supportare chiavi di lunghezza di 128, 192 e 256 bit
4. dovevano essere possibili implementazioni software e hardware
5. l'algoritmo doveva essere pubblico o rilasciato senza vincoli di alcun tipo.

A questo concorso furono presentate quindici proposte serie, discusse in conferenze pubbliche dove i presenti furono incoraggiati a trovarne i possibili difetti. Nell'agosto 1998, NIST selezionò cinque finalisti sulla base dei seguenti criteri: sicurezza, efficienza, semplicità, flessibilità e requisiti di memoria (importante per i sistemi embedded). Ci furono quindi ulteriori conferenze e molti franchi tiratori su questi cifrari.

Nell'ottobre del 2000, NIST ha annunciato di aver selezionato Rijndael, di Joan Daemen e Vincent Rijmen. Il nome Rijndael deriva dai cognomi degli autori: Rijmen + Daemen. Nel novembre 2001, Rijndael divenne lo standard governativo AES degli Stati Uniti, pubblicato come FIPS (*federal information processing standard*) 197. A causa dell'apertura straordinaria della competizione, alle proprietà tecniche di Rijndael, e al fatto che il gruppo vincitore consisteva in due giovani crittografi belgi (i quali era poco probabile avessero costruito una backdoor solo per far piacere a NSA), Rijndael divenne il cifrario dominante a livello mondiale. La cifratura e decifratura AES è ora parte del set di istruzioni di alcuni microprocessori (come Intel).

Rijndael supporta blocchi e chiavi con dimensioni da 128 a 256 bit in passi di 32 bit. La lunghezza della chiave e del blocco possono essere scelti indipendentemente. AES, invece, specifica che la dimensione del blocco deve essere 128 bit, mentre la chiave può essere 128, 192 o 256 bit. Sembra improbabile che vengano mai usate chiavi da 192 bit, quindi in pratica AES ha due varianti: 128 bit di blocco con chiave a 128 bit e 128 bit di blocco con chiave a 256 bit.

Nella nostra discussione tratteremo solamente il caso 128/128, in quanto è quello che probabilmente diventerà lo standard commerciale. Una chiave di 128 bit genera uno spazio delle chiavi con $2^{128} \approx 3 \times 10^3$ chiavi. Anche se NSA riuscisse a costruire una macchina con 1 miliardo di processori paralleli, ognuno in grado di valutare una chiave per picosecondo, ci vorrebbero 10^{10} anni per eseguire una ricerca su tutto lo spazio delle chiavi. Nel frattempo il sole avrà esaurito la sua energia, così il risultato dovrà essere letto a lume di candela.

Rijndael

Da un punto di vista matematico Rijndael è basato sulla teoria dei campi di Galois e ciò gli fornisce alcune proprietà di sicurezza dimostrabili. Possiamo prendere anche un altro punto di vista e considerare Rijndael come un programma in C, senza entrare nelle sue complessità matematiche.

Analogamente a DES, Rijndael usa sostituzioni e permutazioni attuate in molteplici round. Il numero dei round dipende dalla dimensione della chiave e del blocco: si va da 10 per chiavi a 128 bit con blocchi da 128 bit, fino a 14 per la chiave e il blocco più lunghi. A differenza di DES, tutte le operazioni coinvolgono interi byte, questo per avere un'implementazione efficiente sia hardware sia software. Una versione schematica del codice è riportata nella Figura 8.9. Si noti che questo codice sorgente è solo a scopo illustrativo: una buona implementazione di codice relativo alla sicurezza segue accorgimenti aggiuntivi come la cancellazione delle zone sensibili di memoria dopo che sono state usate. Si veda, per esempio, Ferguson et al. (2010).

```
#define LENGTH 16                                /* numero di byte in un blocco dati o chiave */
#define NROWS4                                   /* numero di righe in uno stato */
#define NCOLS4                                   /* numero di colonne in uno stato */
#define ROUNDS 10                                /* numero di iterazioni */
typedef unsigned char byte;                      /* intero senza segno di 8 bit */

rijndael(byte plaintext[LENGTH], byte ciphertext[LENGTH], byte key[LENGTH])
{
  int r;                                         /* indice del loop */
  byte state[NROWS][NCOLS];                      /* stato attuale */
  struct {byte k[NROWS][NCOLS];} rk[ROUNDS + 1]; /* chiavi del round */

  expand_key(key, rk);                           /* costruisce le chiavi del round */
  copy_plaintext_to_state(state, plaintext);     /* inizializza lo stato attuale */
  xor_roundkey_into_state(state, rk[0]);         /* applica la chiave allo stato con uno XOR */

  for (r = 1; r <= ROUNDS; r++) {
      substitute(state);                         /* applica S-box ad ogni byte */
      rotate_rows(state);                        /* ruota riga i di i byte */
      if (r < ROUNDS) mix_columns(state);        /* funzione di mescolamento */
      xor_roundkey_into_state(state, rk[r]);     /* applica la chiave allo stato con uno XOR */
  }
  copy_state_to_ciphertext(ciphertext, state);   /* restituisce il risultato */
}
```

Figura 8.9 Schema di Rijndael in C.

La funzione *rijndael* ha tre parametri: *plaintext,* un array di 16 byte che contiene i dati in input, *ciphertext,* un array di 16 byte per l'output del testo cifrato, e *key* (la chiave a 16 byte). Durante il calcolo lo stato corrente dei dati è mantenuto in array di byte, *state,* la cui dimensione è *NROWS* × *NCOLS*. Per i blocchi da 128 bit questo array è di 4 × 4 byte; 16 byte permettono di memorizzare tutti i 128 bit dei dati.

L'array *state* è inizializzato in modo da contenere il testo in chiaro e viene poi modificato da ogni passo del calcolo. In alcuni passi viene eseguita una sostituzione byte per byte. In altri, i byte sono permutati all'interno dell'array. Vengono usate anche altre trasformazioni. Alla fine, il contenuto di *state* contiene il dato cifrato ritornato dalla funzione. Il codice comincia espandendo la chiave in 11 array con dimensioni pari a quella di *state*, memorizzati in *rk* che è un array di *struct*. Ogni struct contiene un array di state. Uno di questi verrà usato all'inizio del calcolo, mentre gli altri 10 saranno usati a turno, uno per ogni round. Il calcolo delle chiavi per ogni round a partire dalla chiave crittografica è troppo complicato per descriverlo qui. Basti sapere che le chiavi sono prodotte dalla ripetizione di rotazioni e XOR fra vari gruppi di bit della chiave. Per i dettagli si veda (Daemen e Rijmen, 2002).

Il passo successivo consiste nel copiare il testo in chiaro nell'array *state* in modo che possa essere elaborato a ogni round. Viene copiato in ordine di colonna, con i primi 4 byte che entrano nella colonna 0, i successivi 4 nella colonna 1, ecc. Colonne e righe sono numerate a partire da 0, mentre i round sono numerati a partire da 1. La configurazione iniziale dei 12 array di dimensione 4 × 4 è illustrata nella Figura 8.10.

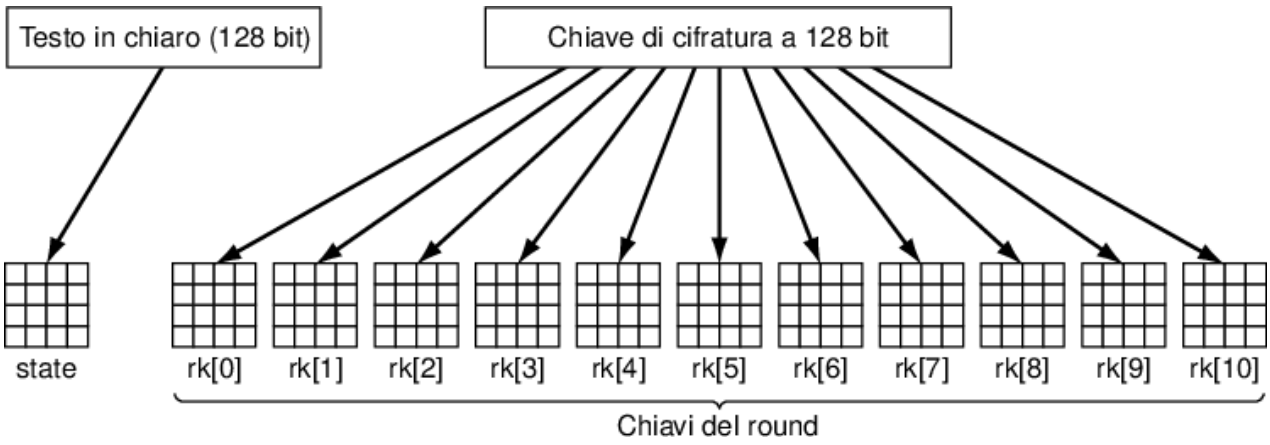


Figura 8.10 Creazione degli array *state* e *rk*.

C'è ancora un passo prima dell'inizio della parte principale dell'algoritmo: viene calcolato lo XOR byte per byte fra *rk[0]* e *state*. In altre parole, ognuno dei 16 byte di *state* viene rimpiazzato con lo XOR fra se stesso e il byte corrispondente in *rk[0]*. Nella parte centrale dell'algoritmo si esegue un ciclo di 10 iterazioni (una per ogni round) che trasformano *state*. Ogni round si articola in quattro passi: nel primo viene fatta una sostituzione di *state* byte per byte. Ogni byte è usato a turno come indice di una S-box e rimpiazzato con il contenuto della S-box. Questo passo è una semplice cifratura a sostituzione monoalfabetica. Rijndael usa una sola S-box, a differenza di DES che ne utilizza diverse.

Nel secondo passo ognuna delle quattro righe è ruotata verso sinistra. La riga 0 viene ruotata di 0 byte (cioè non viene cambiata), la riga 1 viene ruotata di 1 byte, la riga 2 di 2 byte e la riga 3 di 3 byte. Questo passo diffonde i contenuti dei dati correnti all'interno del blocco, in maniera simile alle permutazioni della Figura 8.6.

Il passo 3 mischia ogni colonna indipendentemente dalle altre; l'unione viene fatta usando una matrice di moltiplicazione, nella quale ogni nuova colonna è il prodotto della vecchia colonna per una matrice costante. La moltiplicazione usa il campo finito di Galois $GF(2^8)$. L'operazione, in apparenza complicata, viene in realtà svolta da un algoritmo che permette di calcolare i valori delle nuove colonne operando due ricerche in tabella e tre operazioni di XOR (Daemen e Rijmen, 2002, Appendice E). Infine, il passo 4 esegue l'operazione di XOR tra la chiave per l'iterazione corrente e l'array *state*. Ogni passo è reversibile, perciò la decifrazione può essere fatta semplicemente eseguendo l'algoritmo all'indietro; esiste però un trucco per eseguire la decifrazione usando l'algoritmo di cifratura, ma con tabelle differenti.

L'algoritmo è stato progettato non solo per fornire una grande sicurezza, ma anche per essere molto veloce. Una buona implementazione software su un computer a 2 GHz dovrebbe raggiungere una velocità di cifratura pari a 700 Mbps, che è adeguata per cifrare più di 100 video MPEG-2 in tempo reale. Le implementazioni hardware sono ancora più veloci.

Modalità di cifratura

Nonostante la sua complessità, AES (o DES o ogni altro tipo di cifrario a blocchi) è essenzialmente un cifrario a sostituzione monoalfabetica che usa dei caratteri lunghi (caratteri da 128 bit per AES e da 64 bit per DES). Quando lo stesso testo in chiaro viene preso come input, abbiamo lo stesso testo cifrato in output. Se cifriamo il testo in chiaro *abcdefgh* per 100 volte con la stessa chiave DES, otteniamo 100 volte l'identico testo cifrato. Un intruso può utilizzare questa proprietà per cercare di forzare il cifrario.

Modalità ECB – electronic code book

Per vedere come questa proprietà del cifrario a sostituzione monoalfabetica possa essere usata per forzare parzialmente il cifrario stesso, discutiamo un esempio usando il (triplo) DES, in quanto è più semplice rappresentare blocchi di 64 bit piuttosto che 128 bit; tuttavia AES ha esattamente lo stesso tipo di problema. La modalità d'impiego più ovvia del DES per cifrare un lungo pezzo di testo in chiaro consiste nel prendere il testo e suddividerlo in blocchi di 8 byte (64 bit), che vengono poi cifrati uno dopo l'altro usando la stessa chiave. L'ultimo pezzo di testo in chiaro, se necessario, viene riempito fino a fargli raggiungere la lunghezza di 64 bit. Questa tecnica è nota come **modalità ECB** (*electronic code book*, libro elettronico dei codici) per analogia con i vecchi libri dei codici dove si trovavano gli elenchi delle parole in chiaro seguite dal testo cifrato (di solito un numero decimale di cinque cifre).

Nella Figura 8.11 è mostrato l'inizio di un file che contiene l'elenco dei bonus annuali che un'azienda ha deciso di dare ai suoi dipendenti. Il file è costituito da una serie di record a 32 bit, uno per ogni dipendente, nel formato mostrato nella figura: 16 byte per il nome, 8 byte per la mansione e 8 byte per il bonus. Ognuno dei sedici blocchi da 8 byte (numerati da 0 a 15) è cifrato con il (triplo) DES.

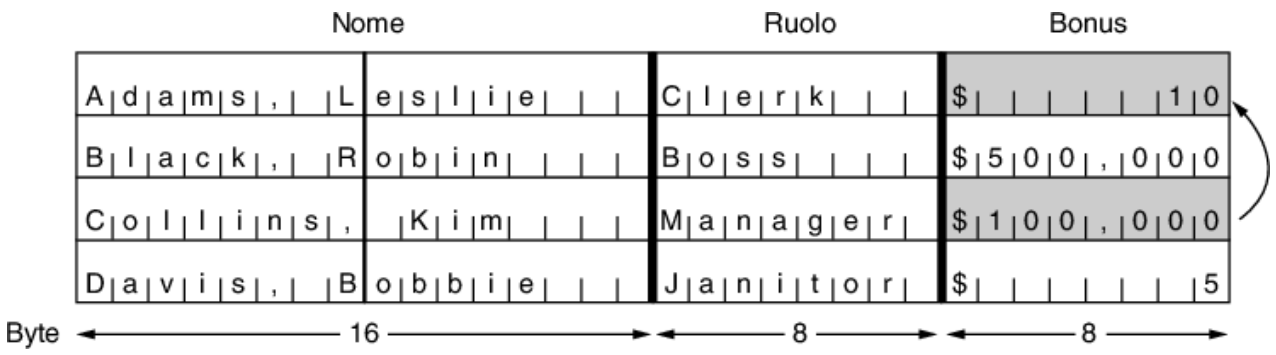


Figura 8.11 Testo in chiaro di un file cifrato come 16 blocchi DES.

Leslie ha avuto di recente una brutta discussione con il capo, quindi non si aspetta di ricevere un gran bonus. Kim, al contrario, è il favorito del capo, come sanno tutti in azienda. Leslie è in grado di accedere al file solo dopo che è stato cifrato, ma prima che venga spedito alla banca. È possibile che Leslie riesca a sistemare questa situazione di squilibrio avendo a disposizione solo il file cifrato?

Non è difficile: Leslie deve solamente fare una copia del dodicesimo blocco di testo cifrato (che contiene il bonus di Kim) e usarlo per rimpiazzare il quarto blocco cifrato (che contiene il bonus di Leslie). Anche senza sapere che cosa contiene il dodicesimo blocco, Leslie si può tuttavia attendere un Natale più allegro quest'anno. Copiare l'ottavo blocco di testo è un'altra possibilità, ma è più facile che la frode venga scoperta e inoltre Leslie non è così attaccata al denaro.

Modalità cipher block chaining

Per evitare questo tipo di attacchi si collegano tutti i blocchi cifrati in diversi modi. Così facendo, quando si tenta di rimpiazzare dei pezzi di testo cifrato, come nel caso di Leslie, l'effetto in fase di decifrazione sarà quello di avere dei dati senza significato a partire dal punto in cui è stata operata la sostituzione. Un modo per collegare i blocchi è il **cipher block chaining** (concatenazione di blocchi cifrati). In questo metodo, mostrato nella Figura 8.12, ogni blocco di testo in chiaro è messo in XOR con il precedente blocco cifrato prima di eseguire la cifratura vera e propria. Così facendo a blocchi di testo in chiaro uguali non corrispondono più blocchi di testo cifrato identici e la cifratura non è più costituita da un cifrario a sostituzione monoalfabetica. Per il primo blocco lo XOR viene calcolato con un blocco di dati casuali, detto *IV* (*initialization vector,* vettore di inizializzazione), trasmesso (in chiaro) insieme al testo cifrato.

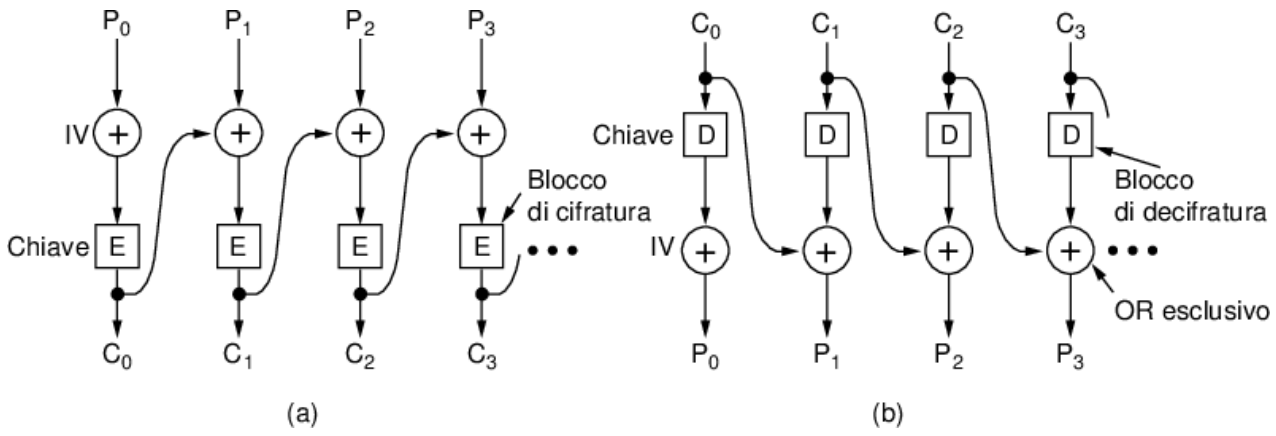


Figura 8.12 Chipher block chaining. (a) Cifratura. (b) Decifratura.

Nell'esempio della Figura 8.12 possiamo vedere come funziona la modalità cipher block chaining. Cominciamo calcolando $C_0 = E(P_0 \text{ XOR } IV)$, seguito da $C_1 = E(P_1 \text{ XOR } C_0)$ e così via. La decifrazione usa di nuovo lo XOR per rovesciare il processo, cioè per calcolare $P_0 = IV \text{ XOR } D(C_0)$. Notiamo che la cifratura del blocco i è una funzione di tutto il testo in chiaro a partire dal blocco 0 fino a $i - 1$, quindi lo stesso testo in chiaro dà origine a diversi testi cifrati a seconda della sua posizione. Una trasformazione come quella fatta da Leslie produrrà due blocchi di dati senza significato, a partire dal campo bonus di Leslie. Un investigatore astuto utilizzerà sicuramente questo dato per cominciare le sue indagini. Il cipher block chaining presenta un altro vantaggio: non produce lo stesso testo cifrato a partire da blocchi di testo in chiaro uguali, complicando quindi ulteriormente la criptoanalisi. Questo è proprio il motivo principale per cui viene utilizzato.

Modalità cipher feedback

La modalità cipher block chaining ha lo svantaggio di richiedere che un intero blocco di 64 bit di testo cifrato arrivi a destinazione prima che la decifrazione possa cominciare. Questo tipo di comportamento non è accettabile per i terminali interattivi, dove gli utenti possono digitare delle linee anche più corte di otto caratteri per poi fermarsi in attesa di una risposta. Con la cifratura byte per byte viene usata la modalità **cipher feedback** (*cifrario con retroazione*) con (triplo) DES, come mostrato nella Figura 8.13. Per AES l'idea è esattamente la stessa, con la sola differenza che si utilizza un registro di shift a 128 bit. Nella figura lo stato della macchina per cifrare è mostrato dopo che i byte da 0 a 9 sono già stati cifrati e trasmessi. Quando arriva il byte 10 del testo in chiaro, come nella Figura 8.13(a), l'algoritmo DES agisce sui 64 bit del registro a scorrimento per generare 64 bit di testo cifrato. Il byte più a sinistra del testo cifrato viene estratto, si calcola lo XOR con P_{10} e il risultato è trasmesso sulla linea di comunicazione. Inoltre il registro a scorrimento viene spostato a sinistra di 8 bit, quindi C_2 esce dal registro mentre C_{10} entra nella posizione lasciata vacante da C_9. Notiamo che il contenuto del registro a scorrimento dipende dall'intera storia precedente del testo in chiaro, quindi una combinazione di caratteri ripetuta nel testo in chiaro darà origine a valori differenti nel testo cifrato. Come nel caso del cipher block chaining, per iniziare la comunicazione serve un vettore di inizializzazione.

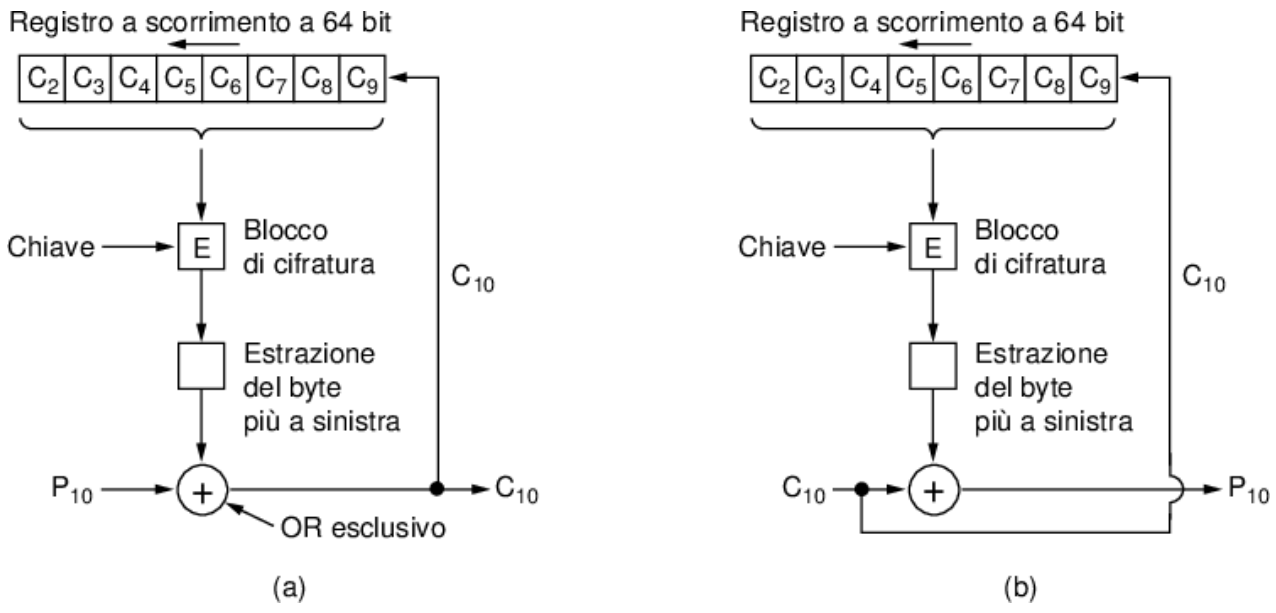


Figura 8.13 Modalità cipher feedback. (a) Cifratura. (b) Decifratura.

Con la modalità cipher feedback la decifrazione funziona come la cifratura. In particolare, il contenuto del registro a scorrimento viene *cifrato* e non *decifrato,* così il byte che viene calcolato in XOR con C_{10} per ottenere P_{10} è lo stesso che, in origine, veniva calcolato in XOR con P_{10} per generare C_{10}. La decifrazione funziona correttamente se i due registri a scorrimento rimangono identici, come illustrato nella Figura 8.13(b).

Un problema della modalità cipher feedback si verifica quando un bit del testo cifrato viene accidentalmente invertito durante la trasmissione: gli 8 byte che vengono decifrati mentre nel registro a scorrimento si trova il byte corrotto saranno pure loro corrotti. Una volta che il byte corrotto è uscito dal registro a scorrimento, la decifrazione procederà di nuovo in modo corretto, quindi l'effetto dell'inversione di un singolo bit risulta relativamente localizzata e non riesce a rovinare il resto del messaggio, tuttavia rovina tanti byte quanto è lungo il registro a scorrimento.

Modalità stream cipher

Esistono delle applicazioni per cui è inaccettabile il fatto che un errore di trasmissione di un solo bit possa rovinare 64 bit di testo in chiaro. Per questo tipo di applicazioni esiste una quarta opzione: la modalità **stream cipher** (*cifrario a flusso*). Questa modalità funziona cifrando un vettore di inizializzazione con una chiave crittografica per ottenere un blocco in uscita. Quest'ultimo viene cifrato per produrre un secondo blocco in uscita, quindi si procede analogamente con il terzo, ecc. La sequenza (di lunghezza arbitraria) di blocchi cifrati in uscita, chiamata il **keystream** (*flusso delle chiavi*), viene utilizzata come un blocco monouso e applicata con uno XOR al testo in chiaro per ottenere il testo cifrato, come mostrato nella Figura 8.14(a). Notiamo che *IV* è usato solamente nel primo passo, nei passi successivi l'output è cifrato. Notiamo anche che il keystream è indipendente dai dati, così che può essere calcolato in anticipo, se necessario, ed è anche immune da errori di trasmissione. La decifrazione è mostrata nella Figura 8.14(b).

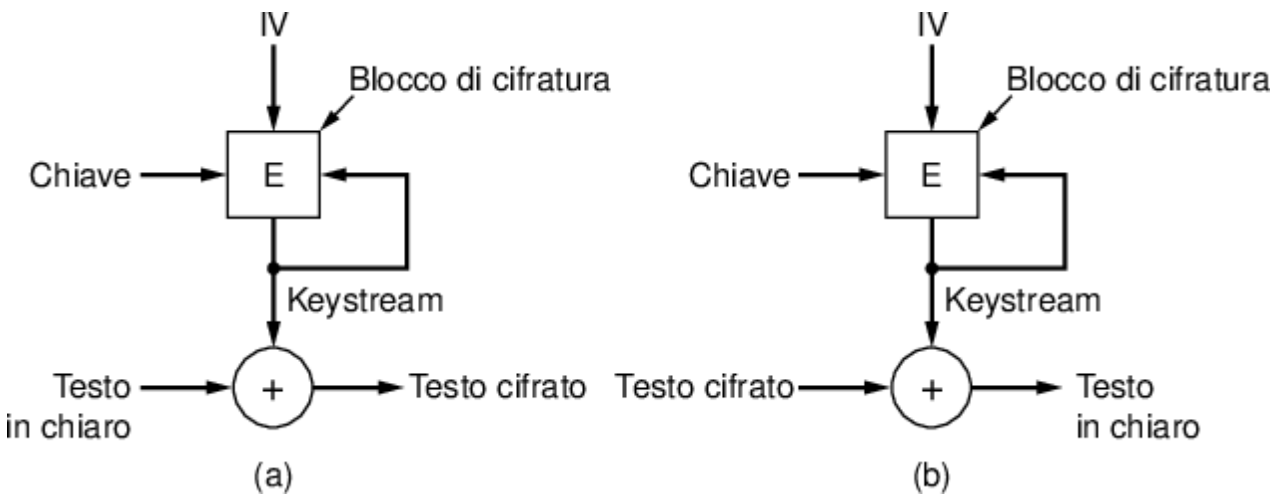


Figura 8.14 Stream cipher. (a) Cifratura. (b) Decifratura.

La decifrazione viene eseguita generando lo stesso keystream dal lato del ricevente. Visto che il keystream dipende solo da *IV* e dalla chiave, non è sensibile agli errori di trasmissione del testo cifrato. Quindi, un errore di 1 bit nel testo cifrato trasmesso genera solamente un errore di 1 bit nel testo decifrato.

È essenziale che nella modalità stream cipher non venga mai riutilizzata la coppia (*chiave, IV*), perché questo vorrebbe dire generare più volte lo stesso keystream. L'uso ripetuto dello stesso keystream espone il testo cifrato ad attacchi di tipo **keystream riutilizzato** (*keystream reuse attack*). Immaginiamo che un blocco di testo in chiaro, P_0, sia cifrato con un keystream per ottenere P_0 XOR K_0. Più tardi si utilizza ancora lo stesso keystream per cifrare un secondo blocco di testo Q_0, calcolando Q_0 XOR K_0. Un intruso che riesca a catturare entrambi questi blocchi cifrati può semplicemente calcolare lo XOR dei due testi in chiaro: P_0 XOR Q_0, il che elimina la chiave. Se uno dei due blocchi è noto, oppure può essere indovinato, l'intruso ha trovato il valore dell'altro blocco. Inoltre, lo XOR di due testi in chiaro può essere attaccato usando le proprietà statistiche del messaggio. Prendendo per esempio i messaggi in inglese, il carattere più comune nel flusso di dati sarà probabilmente lo XOR di due spazi, seguito dallo XOR di uno spazio e della lettera "e" e così via. In conclusione, avendo a disposizione lo XOR di due testi in chiaro, il criptoanalista ha eccellenti probabilità di dedurli entrambi.

Modalità counter

Un problema comune a tutte le modalità, fatta eccezione per electronic code book, è quella di rendere impossibile un accesso casuale ai dati. Per esempio, supponiamo che un file sia trasmesso tramite la rete e poi memorizzato su disco in un formato cifrato; ciò potrebbe essere utile se il computer del ricevente è un portatile a rischio di furto. Memorizzare i file critici in forma cifrata riduce ampiamente il potenziale danno dovuto alla fuga di notizie che avviene se il computer cade nelle mani sbagliate.

Spesso si accede ai file su disco in un ordine non sequenziale, specialmente quando si tratta di database. Con un file cifrato usando il cipher block chaining, l'accesso casuale a un blocco richiede che prima siano decifrati tutti i blocchi precedenti; un'operazione troppo onerosa. Per questo motivo è stata inventata un'ulteriore modalità: la modalità **counter** (*contatore*), mostrata nella Figura 8.15. In questo caso il testo in chiaro non viene cifrato direttamente; si cifra invece un vettore di inizializzazione con una costante e il risultato è messo in XOR con il testo in chiaro. Incrementando di 1 il vettore di inizializzazione a ogni blocco diventa facile riuscire a decifrare un blocco in qualunque posizione si trovi, senza dover decifrare prima tutti i suoi predecessori.

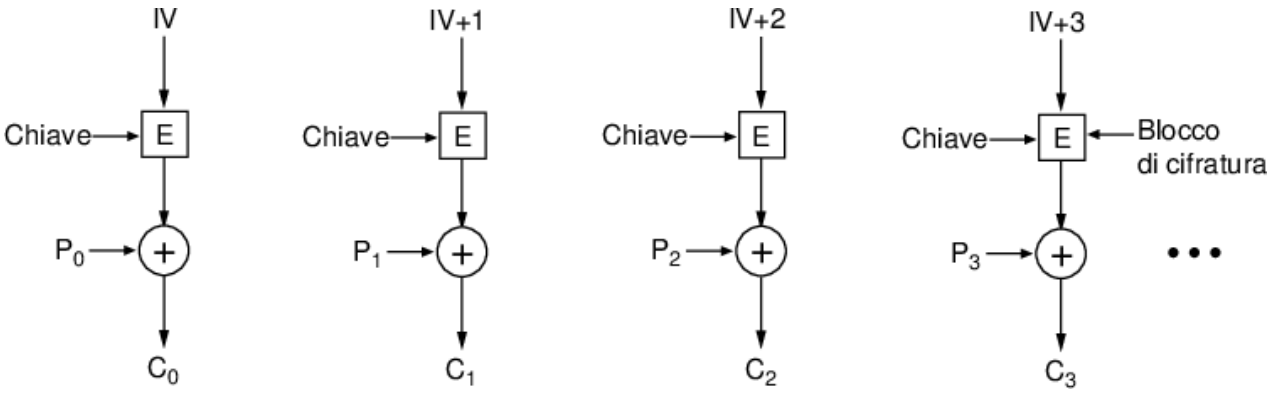


Figura 8.15 Cifratura in modalità counter.

Anche se la modalità contatore è utile, ha una debolezza che vogliamo esplicitare. Supponiamo che la stessa chiave, *K,* venga usata di nuovo in futuro (con un testo in chiaro differente, ma con lo stesso *IV*) e un intruso riesca a ottenere tutto il testo cifrato per entrambi i casi. Essendo il keystream lo stesso nei due casi, il sistema è esposto ad attacchi di tipo keystream riutilizzato, che abbiamo discusso a proposito della modalità stream cipher. Il criptoanalista deve fare un semplice XOR dei due testi cifrati e il risultato sarà uguale allo XOR dei due testi in chiaro, con soppressione di tutta la protezione crittografica. Questa debolezza non significa che la modalità counter sia sbagliata, ma che vettore di inizializzazione e chiave vanno scelti indipendentemente e in modo casuale. Anche se la stessa chiave viene usata due volte accidentalmente, quando l'*IV* è differente nei due casi il testo in chiaro è salvo.

Altri cifrari

AES (Rijndael) e DES sono gli algoritmi criptografici a chiave simmetrica più conosciuti e le scelte standard per l'industria, se non altro per motivi di affidabilità. (Nessuno vi biasimerà se usate AES nel vostro prodotto e AES viene violato, ma vi biasimeranno di sicuro se usate un cifrario non standard che viene forzato). Tuttavia vale la pena menzionare che sono stati messi a punto diversi altri cifrari a chiave simmetrica. Alcuni di questi sono integrati in vari prodotti e alcuni dei più comuni sono elencati nella Figura 8.16. È possibile usare combinazioni di questi cifrari, per esempio AES su Twofish, così che si dovranno rompere entrambi per recuperare i dati.

Algoritmo	Autore	Lunghezza della chiave	Commenti
DES	IBM	56 bit	Troppo debole per usarlo oggi
RC4	Ronald Rivest	1-2048 bit	Attenzione: alcune chiavi sono deboli
RC5	Ronald Rivest	128-256 bit	Valido ma brevettato
AES (Rijndael)	Daemen e Rijmen	128-256 bit	La scelta migliore
Serpent	Anderson, Biham, Knudsen	128-256 bit	Molto robusto
Triplo DES	IBM	168 bit	Buono ma sta diventando vecchio
Twofish	Bruce Schneler	128-256 bit	Molto robusto; ampiamente usato

Figura 8.16 Alcuni noti algoritmi crittografici a chiave simmetrica.

Criptoanalisi

Prima di abbandonare l'argomento della crittografia a chiave simmetrica, è utile menzionare alcuni sviluppi della criptoanalisi. Il primo sviluppo è la **criptoanalisi differenziale** (Biham e Shamir, 1993), una tecnica che può essere usata per attaccare i cifrari a blocco. Si comincia con una coppia di blocchi di testo in chiaro che differiscono solo per un piccolo numero di bit e si guarda attentamente a che cosa accade in ogni iterazione interna, mentre la cifratura avanza. In molti casi, alcune combinazioni di bit risultano più comuni di altre. Questo tipo di osservazioni è di aiuto per gli attacchi di tipo probabilistico.

Il secondo sviluppo interessante è la **criptoanalisi lineare** (Matsui, 1994). Questa tecnica riesce a forzare il DES solamente con la conoscenza di 2^{43} testi in chiaro. Funziona calcolando lo XOR di certe combinazioni di bit del testo in chiaro con quello cifrato ed esaminandone il risultato. Quando questa operazione è eseguita ripetutamente, metà dei bit dovrebbero essere 0 e metà 1. Capita spesso che i cifrari introducano uno sbilanciamento in una direzione piuttosto che l'altra. Queste piccole differenze possono essere usate per ridurre il fattore di lavoro necessario a forzare il cifrario. I dettagli si possono trovare nell'articolo di Matsui.

Il terzo sviluppo consiste nell'analizzare il consumo dell'alimentazione elettrica per trovare le chiavi segrete. I computer tipicamente usano 3 Volt per rappresentare i bit a 1 e 0 Volt per i bit a 0. Quindi l'elaborazione di un 1 richiede più energia dell'elaborazione di uno 0. Se un algoritmo crittografico contiene un ciclo in cui i bit della chiave vengono elaborati in ordine, un possibile attacco consiste nel rimpiazzare il clock principale a *n*-GHz con uno più lento (per esempio 100 Hz) e attaccare due connettori a coccodrillo sull'alimentatone della CPU e sulla terra. In questo modo si riesce a misurare la potenza consumata da ogni singola istruzione della macchina. Da questi dati, dedurre la chiave è sorprendentemente semplice. Questo tipo di criptoanalisi può essere sconfitta solamente con una programmazione attenta a livello di linguaggio assembly, per essere sicuri che il consumo elettrico sia indipendente dalla chiave e anche indipendente dalle singole chiavi per ogni round.

Il quarto sviluppo è l'analisi dei tempi. Gli algoritmi crittografici sono pieni di istruzioni per il test dei bit nelle chiavi di ogni iterazione. Se la parte "then" e quella "else" delle istruzioni di test impiegano tempi di esecuzione differenti, è possibile dedurre le chiavi di iterazione rallentando il clock e misurando quanto tempo viene impiegato da ogni passo. Quando si conoscono tutte le chiavi delle iterazioni, è possibile ricostruire anche la chiave originale. Le analisi del consumo e della temporizzazione si possono usare simultaneamente per semplificare il lavoro. Questo tipo di attacchi può sembrare bizzarro, ma in realtà si tratta di tecniche molto potenti che riescono a forzare quei cifrari non progettati per resistervi.

8.3 Algoritmi a chiave pubblica

Storicamente la distribuzione delle chiavi è sempre stato l'anello debole della maggior parte dei sistemi crittografici. Indipendentemente dalla sua forza, ogni sistema crittografico diventava automaticamente inutile nel momento in cui un intruso riusciva a rubarne la chiave. I criptoanalisti hanno sempre assunto che le chiavi di cifratura e decifrazione fossero le stesse (oppure fossero facilmente derivabili l'una dall'altra). Rimaneva il fatto che la chiave doveva essere distribuita a tutti gli utenti del sistema, quindi sembrava esserci un problema intrinseco nel sistema. Le chiavi dovevano essere protette dai furti; ma al tempo stesso dovevano essere distribuite e non semplicemente custodite in una cassetta di sicurezza.

Nel 1976 due ricercatori dell'università di Stanford, Diffie e Hellman, proposero un sistema crittografico radicalmente innovativo, in cui le chiavi di cifratura e decifrazione erano differenti e la chiave di decifrazione non era facilmente derivabile da quella di cifratura. Nella loro proposta, l'algoritmo di cifratura E e quello di decifrazione D dovevano sottostare a tre requisiti:

1. $.D(E(P)) = P$
2. risulta estremamente difficile dedurre D da E
3. E non può essere forzato da un attacco di tipo "testo in chiaro a scelta".

Il primo requisito afferma che se applichiamo D a un messaggio cifrato $E(P)$, otteniamo il testo in chiaro originale, P. Senza questa proprietà, il destinatario legittimo del messaggio non riuscirebbe a decifrarlo. Il secondo requisito è autoesplicativo. Il terzo requisito è necessario perché, come vedremo fra poco, gli intrusi possono tuttavia sperimentare l'algoritmo a piacimento. Sotto queste condizioni non c'è ragione per non rendere pubblica la chiave di cifratura.

Il metodo funziona così: una persona, per esempio Alice, vuole ricevere dei messaggi segreti e quindi inventa due algoritmi con le proprietà elencate sopra. L'algoritmo di cifratura e la chiave di Alice vengono resi pubblici, da qui il nome di **crittografia pubblica**. Alice può mettere la sua chiave pubblica, per esempio, sulla sua pagina Web. Useremo la notazione E_A per indicare l'algoritmo di cifratura parametrizzato dalla chiave pubblica di Alice. Analogamente, l'algoritmo (segreto) di decifrazione parametrizzato dalla chiave privata di Alice sarà D_A. Bob esegue le operazioni analoghe ad Alice, cioè rende pubblico E_B, mentre tiene segreto D_B.

Vediamo se riusciamo a risolvere il problema di come stabilire una canale sicuro fra Alice e Bob, che non hanno mai avuto nessun contatto in precedenza. Supponiamo che la chiave di cifratura di Alice E_A e quella di Bob E_B siano dei file leggibili pubblicamente. Alice prende il suo primo messaggio, P, calcola $E_B(P)$ e lo manda a Bob. Bob decifra il messaggio usando la sua chiave segreta D_B (cioè calcola $D_B(E_B(P)) = P$). Nessun altro riesce a leggere il messaggio cifrato, $E_B(P)$, in quanto il sistema di cifratura è forte e quindi risulta troppo difficile derivare D_B dalla chiave pubblica E_B. Per inviare una risposta, R, Bob trasmette $E_A(R)$. Ora Alice e Bob riescono a comunicare in modo sicuro.

Può essere utile una nota sulla terminologia. La crittografia a chiave pubblica richiede che ciascun utente abbia due chiavi: una chiave pubblica, usata dal mondo intero per cifrare i messaggi da mandare a quell'utente, e una chiave privata, che l'utente usa per decifrare i messaggi.

Chiameremo queste due chiavi rispettivamente *pubblica* e *privata,* per distinguerle dalle chiavi *segrete* usate nella crittografia convenzionale a chiave simmetrica.

RSA

L'unico problema è che dobbiamo trovare degli algoritmi che riescano effettivamente a soddisfare le tre proprietà richieste. Visti i potenziali vantaggi della crittografia a chiave pubblica, molti ricercatori stanno lavorando intensamente su questo argomento e alcuni algoritmi sono già stati pubblicati. Un ottimo metodo è stato scoperto da un gruppo del M.I.T. (Rivest et al., 1978), ed è noto con le iniziali **RSA** dei suoi tre ideatori (Rivest, Shamir e Adleman). È sopravvissuto a tutti i tentativi di forzatura per più di un quarto di secolo, ed è considerato un algoritmo molto robusto. Una gran parte delle applicazioni pratiche nel campo della sicurezza si basa su RSA. Il suo maggior svantaggio è che richiede chiavi di almeno 1024 bit per poter offrire una buona sicurezza (contro i 128 bit degli algoritmi a chiave simmetrica), il che lo rende abbastanza lento.

RSA si basa su alcuni principi di teoria dei numeri. Faremo riepilogo sull'uso di questo metodo, mentre per i dettagli consigliamo di consultare l'articolo originale.

1. Scegliamo due numeri primi, p e q (tipicamente di 1024 bit).
2. Calcoliamo $n = p \times q$ e $z = (p - 1) \times (q - 1)$.
3. Scegliamo un numero relativamente primo rispetto a z, detto d.
4. Troviamo e tale che $e \times d = 1 \bmod z$.

Avendo calcolato questi parametri in anticipo, possiamo cominciare la cifratura. Dividiamo in blocchi il testo in chiaro (visto come una stringa di bit), in modo che ogni messaggio in chiaro, P, cada nell'intervallo $0 < P < n$. Per fare questo, raggruppiamo il testo in chiaro in blocchi di k bit, dove k è il più grande intero per cui vale $2^k < n$. Per cifrare il messaggio P, calcoliamo $C = P^e \pmod n$. Per decifrare C, calcoliamo $P = C^d \pmod n$. Possiamo dimostrare che per ogni P nell'intervallo specificato, le funzioni di cifratura e decifratura sono una l'inversa dell'altra. Per cifrare abbiamo bisogno di e e di n; per decifrare invece ci servono d e n. Quindi la chiave pubblica consiste nella coppia (e, n), mentre la chiave privata consiste in (d, n).

La sicurezza del metodo è basata sulla difficoltà di scomporre in fattori primi i numeri molto grandi. Se il criptoanalista riuscisse a fattorizzare il numero (noto pubblicamente) n, allora riuscirebbe anche a trovare p e q e da questi anche z. Con la conoscenza di z, si possono trovare e e d tramite l'algoritmo di Euclide. Fortunatamente, i matematici si occupano della fattorizzazione dei grandi numeri da più di 300 anni e dalla conoscenza acquisita si sa che questo è un problema veramente difficile.

Secondo Rivest e colleghi, fattorizzare un numero di 500 cifre usando la forza bruta richiede 10^{25} anni, anche assumendo di usare il miglior algoritmo noto e un computer che elabora le istruzioni in 1 ms. Se i computer continueranno ad aumentare la loro velocità di un ordine di grandezza ogni decade, passeranno secoli prima di risolvere il problema della fattorizzazione di un numero di 500 cifre. A quel punto però, i nostri discendenti potranno semplicemente scegliere p e q ancora più grandi.

Un semplice esempio didattico del funzionamento dell'algoritmo RSA è dato nella Figura 8.17. Per questo esempio abbiamo scelto $p = 3$ e $q = 11$, quindi $n = 33$ e $z = 20$. Un valore adeguato per d è $d = 7$, visto che 7 e 20 non hanno divisori comuni. Con queste scelte e può essere trovato risolvendo

l'equazione $7e = 1 \pmod{20}$, il che dà $e = 3$. Il testo cifrato, C, per il testo in chiaro, P, è dato da $C = P^3 \pmod{33}$. Il testo cifrato viene decifrato dal ricevente usando la regola $P = C^7 \pmod{33}$. La figura nell'esempio mostra la cifratura del testo "SUZANNE".

Testo in chiaro (P)			Testo cifrato (C)		Dopo la decifratura	
Simbolico	Numerico	P^3	$P^3 \pmod{33}$	C^7	$C^7 \pmod{33}$	Simbolico
S	19	6859	28	13492928512	19	S
U	21	9261	21	1801088541	21	U
Z	26	17576	20	1280000000	26	Z
A	01	1	1	1	01	A
N	14	2744	5	78125	14	N
N	14	2744	5	78125	14	N
E	05	125	26	8031810176	05	E
		Calcoli del mittente		Calcoli del destinatario		

Figura 8.17 Un esempio dell'algoritmo RSA.

Siccome per questo esempio abbiamo scelto dei numeri primi molto piccoli, P deve essere inferiore a 33, quindi ogni blocco di testo può contenere un solo carattere. Il risultato è un cifrario a sostituzione monoalfabetica, il che non è molto impressionante. Se invece, avessimo scelto p e $q \approx 2^{512}$, avremmo avuto $n \sim 2^{1024}$, quindi ogni blocco potrebbe essere lungo fino a 1024 bit, oppure 128 caratteri da 8 bit ciascuno, contro gli 8 caratteri di DES e i 16 di AES.

Notiamo che RSA, per come l'abbiamo descritto, è simile a un algoritmo simmetrico in modalità ECB, cioè blocchi di input uguali originano lo stesso blocco di output: anche in questo caso occorre qualche forma di concatenamento. In pratica, però, la maggior parte dei sistemi crittografici basati su RSA usa la crittografia a chiave pubblica solo una tantum, per distribuire le chiavi segrete da usare con un algoritmo di crittografia simmetrica, come AES o triplo DES. RSA è troppo lento per poterlo usare nella cifratura di grandi volumi di dati, mentre è spesso impiegato per la distribuzione delle chiavi.

Altri algoritmi a chiave pubblica

Anche se è largamente utilizzato, RSA non è certamente l'unico algoritmo a chiave pubblica noto. Il primo algoritmo a chiave pubblica fu l'algoritmo dello zaino (Merkle e Hellman, 1978). L'idea è che qualcuno possiede una gran quantità di oggetti, ognuno con un peso differente. Il proprietario cifra il messaggio, scegliendo casualmente un sottoinsieme di oggetti e mettendoli nello zaino. Il peso totale degli oggetti nello zaino è reso pubblico, così come la lista dei possibili oggetti. La lista degli oggetti nello zaino viene tenuta segreta. Con alcune restrizioni aggiuntive, il problema da

risolvere per forzare il codice diventa quello di trovare la possibile lista di oggetti con peso stabilito. Si pensava che tale problema fosse computazionalmente inaffrontabile, quindi era stato scelto come base per l'algoritmo a chiave pubblica.

L'inventore dell'algoritmo, Ralph Merke, era così sicuro che questo algoritmo non potesse essere forzato che offrì una ricompensa di 100 dollari a chiunque riuscisse nell'impresa. Adi Shamir (la "S" in RSA) riuscì in poco tempo a forzare il codice e prese la ricompensa. Senza scomporsi, Merkle rafforzò l'algoritmo e offrì una ricompensa di 1000 dollari a chi fosse riuscito a forzare questa nuova versione. Ronald Rivest (la "R" in RSA) riuscì dopo poco tempo nell'impresa e ottenne la ricompensa. Merkle non osò offrire 10.000 dollari per la successiva versione, così la "A" (Leonard Adleman) non potè essere premiato a sua volta. A ogni modo, l'algoritmo a zaino non è considerato sicuro e non viene praticamente più usato. Altri schemi a chiave pubblica sono basati sulla difficoltà di calcolare i logaritmi discreti. Algoritmi di questo tipo sono stati inventati da El Gamal (1985) e Schnorr (1991). Esistono anche degli altri schemi, come quelli basati sulle curve ellittiche (Menezes e Vanstone, 1993), ma le due categorie principali si basano sulla difficoltà di fattorizzare i numeri primi e sul calcolo dei logaritmi discreti modulo un grande numero primo. Questi problemi sono ritenuti veramente difficili da risolvere, inoltre molti matematici ci hanno lavorato sopra per diversi anni senza mai trovare delle vere soluzioni.

8.4 Firme digitali

L'autenticità di molti documenti legali, finanziari, ecc, è basata sulla presenza o assenza di una firma autografa autorizzata. Le fotocopie non contano. È necessario trovare un metodo per far sì che i documenti elettronici possano essere firmati in modo non falsificabile, se vogliamo che i messaggi computerizzati rimpiazzino la carta e l'inchiostro. Il problema di trovare un sostituto per la firma autografa è veramente difficile. Essenzialmente, serve un sistema con il quale il mittente possa mandare un messaggio firmato al destinatario in modo che valgano le seguenti condizioni:

1. il destinatario può verificare l'identità dichiarata dal mittente
2. il mittente non può ripudiare in un secondo tempo il contenuto del messaggio
3. il destinatario non può falsificare i messaggi del mittente.

Il primo requisito è necessario, per esempio, per i sistemi finanziari. Quando il computer di un cliente ordina al computer della banca di comprare una tonnellata di oro, il computer della banca deve poter essere sicuro che il computer che ha mandato l'ordine appartenga realmente all'azienda sul cui conto verrà addebitato. In altre parole, la banca deve poter autenticare il cliente (e il cliente deve poter autenticare la banca). Il secondo requisito è necessario per proteggere la banca dalle frodi. Supponiamo che la banca compri una tonnellata d'oro e, immediatamente dopo, il prezzo dell'oro cali bruscamente. Un cliente disonesto potrebbe far causa alla banca, dicendo che non ha mai mandato l'ordine di comprare dell'oro. Quando la banca porterà il messaggio in tribunale come prova, il cliente negherà di averlo spedito. La proprietà per cui nessuna delle parti contraenti un contratto è in grado in un secondo momento di negare di averlo stipulato si chiama **non ripudio** (*nonrepudiation*). Gli schemi di firma digitale che vedremo nel seguito permettono la sua implementazione.

Il terzo requisito è necessario per proteggere il cliente, se il prezzo dell'oro sale e la banca tentasse di costruire un messaggio firmato in cui il cliente chiedeva una barra d'oro al posto di una tonnellata. In questo scenario di frode, la banca si tiene per sé il resto dell'oro.

Firme a chiave simmetrica

Un approccio alle firme digitali è la creazione di un'autorità centrale che le conosce tutte e di cui tutti hanno fiducia: chiamiamolo BIG BROTHER (grande fratello *BB*). Ogni utente sceglie una chiave segreta e la porta a mano nell'ufficio di *BB*. Quindi, solo Alice e *BB* conoscono la chiave segreta K_A, e così via.

Quando Alice vuole mandare un messaggio in chiaro firmato, *P*, al suo banchiere, Bob, genera $K_A(B, R_A, t, P)$, dove *B* è l'identità di Bob, R_A è un numero casuale scelto da Alice, *t* è il timestamp che assicura che il messaggio sia recente e $K_A(B, R_A, t, P)$ è il messaggio cifrato con la chiave K_A. A questo punto Alice manda il messaggio, come nella Figura 8.18. *BB* vede il messaggio da Alice, lo decifra e lo manda a Bob, come mostrato, il messaggio per Bob contiene il testo in chiaro con il messaggio di Alice e anche il messaggio firmato $K_{BB}(A, t, P)$. Infine Bob compie le azioni richieste da Alice. Che cosa succede se, in un secondo tempo, Alice nega di aver spedito il messaggio? Ovviamente ognuno denuncia tutti gli altri (per lo meno questo è quello che accade negli Stati Uniti). Infine, quando il caso arriva in tribunale e Alice continua a negare risolutamente di aver mandato il messaggio in questione a Bob, il giudice chiederà a Bob come può essere sicuro che il messaggio proveniva da Alice e non da Trudy. Bob, come primo argomento, fa notare che *BB* non accetta messaggi da Alice se non sono cifrati con K_A, quindi non è possibile che Trudy abbia mandato a *BB* un messaggio falso spacciandosi per Alice, senza che *BB* se ne sia accorto subito.

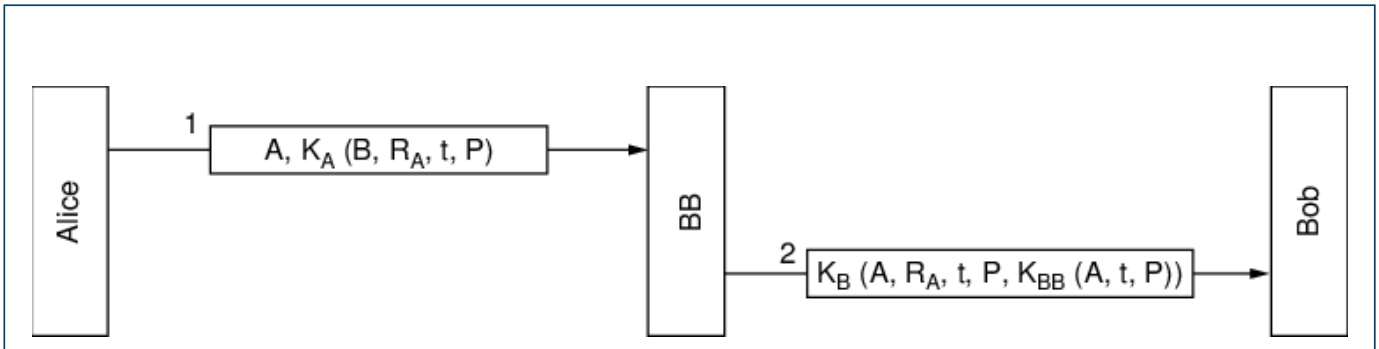


Figura 8.18 Firme digitali usando Big Brother.

Bob a questo punto può presentare la prova A: $K_{BB}(A, t, P)$. Bob afferma che questo è un messaggio firmato da *BB,* che prova che Alice ha inviato *P* a Bob. Il giudice chiede quindi a *BB* (di cui tutti si fidano) di decifrare la prova A. Quando *BB* testimonia che Bob ha detto la verità, il giudice dà ragione a Bob. Il caso è chiuso.

Un potenziale problema con il protocollo di firma della Figura 8.18 è dato dagli attacchi di tipo ripetizione, che Trudy può eseguire con entrambi i messaggi. Per ridurre l'impatto di questo problema, vengono usati i timestamp. Inoltre Bob può controllare tutti i messaggi recenti per vedere se R_A era già stato usato. In caso affermativo, il messaggio viene scartato come possibile attacco di ripetizione. Notiamo che, basandosi sul timestamp, Bob rifiuterà i messaggi troppo vecchi. Per proteggersi contro attacchi di ripetizione istantanea, Bob deve solamente controllare che R_A di ciascun messaggio non corrisponda a quello di un messaggio ricevuto nell'ultima ora da Alice. In questo caso Bob può assumere che il messaggio sia una nuova richiesta.

Firme a chiave pubblica

Il problema strutturale della crittografia a chiave simmetrica per la firma digitale è dato dal fatto che tutti devono fidarsi di *BB*, il quale può anche leggere i messaggi di tutti. I candidati più logici per gestire un server Big Brother sono il governo, le banche, i contabili e gli avvocati. Sfortunatamente, nessuna di queste organizzazioni riesce a suscitare un senso di sicurezza totale in tutti i cittadini. Quindi è auspicabile avere un metodo per firmare i documenti che non richieda un'autorità fidata.

Fortunatamente, la crittografia a chiave pubblica può dare un contributo importante in quest'area. Assumiamo che gli algoritmi di cifratura e decifrazione abbiamo la proprietà che $E(D(P)) = P$, oltre alla proprietà solita $D(E(P)) = P$. RSA ha questa proprietà, quindi sappiamo già che la nostra richiesta è ragionevole. Sotto queste ipotesti, Alice può trasmettere un messaggio in chiaro, *P,* a Bob inviando $E_B(D_A(P))$. È importante notare che Alice conosce sia la sua chiave privata, D_A, sia la chiave pubblica di Bob, E_B, in questo modo riesce a costruire il messaggio da inviare.

Bob, dopo aver ricevuto il messaggio, lo trasforma come al solito usando la sua chiave privata, ottenendo quindi $D_A(P)$ come mostrato nella Figura 8.19. Bob memorizza il messaggio in un posto sicuro e poi applica E_A per ottenere il messaggio in chiaro originale.

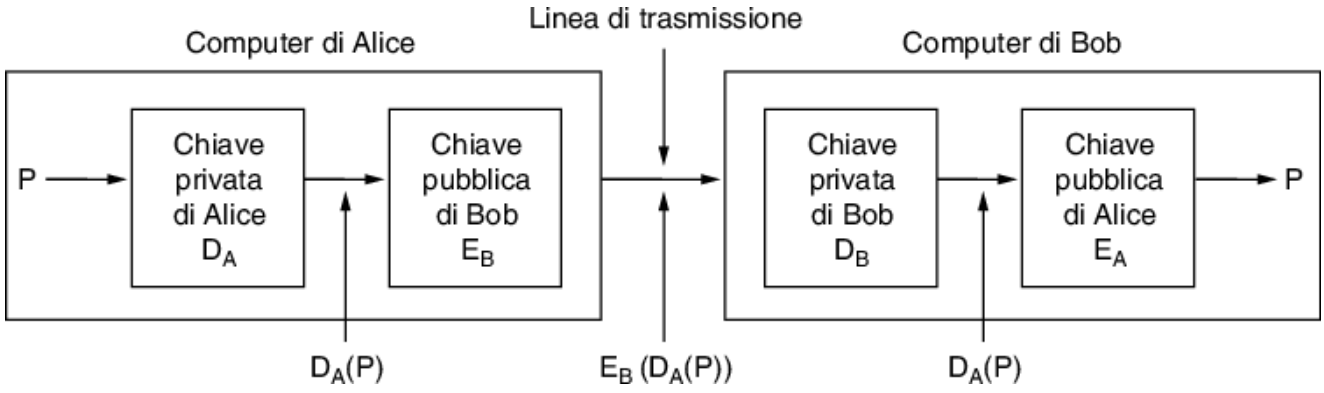


Figura 8.19 Firme digitali usando la crittografia a chiave pubblica.

Per vedere come funziona la proprietà di firma, supponiamo che Alice in un secondo momento voglia negare di aver mandato il messaggio *P* a Bob. Quando il caso arriva in tribunale, Bob può presentare sia *P* sia $D_A(P)$. Il giudice può facilmente verificare che Bob ha in effetti un messaggio valido che è stato cifrato con D_A, semplicemente applicandogli

E_A. Visto che Bob non conosce la chiave privata di Alice, l'unico modo in cui Bob può aver ricevuto un messaggio cifrato con quella chiave è una trasmissione da parte di Alice. Alice avrà diverso tempo libero per pensare a nuovi schemi crittografici a chiave pubblica mentre sarà in prigione per frode e falsa testimonianza.

La crittografia a chiave pubblica per la firma digitale è uno schema elegante, che però presenta alcuni problemi legati principalmente al contesto in cui viene utilizzata, piuttosto che agli algoritmi utilizzati. Come primo punto, Bob può provare che un messaggio è stato realmente mandato da Alice solo se la chiave D_A rimane segreta. Se Alice rende pubblica la sua chiave segreta, il discorso non vale più, in quanto chiunque, incluso Bob, potrebbe aver inviato il messaggio.

Per esempio il problema può verificarsi se Bob è l'agente di borsa di Alice. Alice dice a Bob di comprare alcune azioni e obbligazioni. Quasi immediatamente il prezzo cala bruscamente. Con lo scopo di ripudiare il messaggio inviato a Bob, Alice corre dalla polizia dicendo che la sua casa è stata svaligiata e che il PC con la chiave è stato rubato. A seconda delle leggi in vigore nel suo paese di residenza, Alice può essere considerata responsabile del messaggio inviato oppure no; ciò può dipendere anche dal fatto che Alice dichiari di non aver scoperto il furto fino al suo rientro a casa dal lavoro, alcune ore dopo che questo sia ipoteticamente avvenuto.

Un altro problema con lo schema della firma elettronica si ha quando Alice decide di cambiare la sua chiave. Questa operazione è sicuramente legale, inoltre è una buona idea farla periodicamente. Se in seguito si dovesse andare in tribunale come descritto sopra, il giudice userebbe la chiave *corrente* E_A per verificare $D_A(P)$ e scoprirebbe che non produce *P.* A questo punto Bob sarebbe in una posizione indifendibile.

In teoria, qualunque algoritmo a chiave pubblica può essere usato per la firma digitale. Lo standard de facto, utilizzato da una grande quantità di prodotti per la sicurezza, è l'algoritmo RSA. Nel 1991 NIST propose una variante dell'algoritmo a chiave pubblica, El Gamal, per il nuovo DSS *(digital signature standard*, standard per la firma digitale). El Gamal si basa sulla difficoltà di calcolare i logaritmi discreti, mentre RSA si basa sulla scomposizione in fattori primi.

Come al solito, quando il governo tenta di dettare legge sugli standard crittografici si alza un polverone. DSS è stato criticato per essere:

1. troppo segreto (NSA ha progettato il protocollo per usare El Gamal)
2. troppo lento (da 10 a 40 volte più lento di RSA per controllare le firme)
3. troppo nuovo (El Gamal non è ancora stato esaminato fino in fondo)
4. troppo insicuro (la chiave è di lunghezza fissa pari a 512 bit).

In una revisione successiva è stato superato il quarto punto introducendo chiavi a 1024 bit.

Message digest

Spesso ai metodi di firma si rivolge una critica: nel loro funzionamento riuniscono due funzioni distinte, l'autenticazione e la segretezza. In molti casi è necessaria l'autenticazione, ma non la segretezza; inoltre è più facile ottenere le licenze per l'esportazione della tecnologia se un sistema fornisce solo autenticazione e non segretezza. Di seguito descriveremo uno schema di autenticazione che non richiede la cifratura dell'intero messaggio. Questo schema consiste nel costruire su una funzione di tipo hash monodirezionale, che prende come input un testo in chiaro di lunghezza arbitraria e calcola una stringa di bit di lunghezza fissa. La funzione hash *MD,* detta **message digest** (*riassunto del messaggio*), ha quattro proprietà importanti:

1. dato *P,* è facile calcolare $MD(P)$
2. dato $MD(P)$, è praticamente impossibile trovare P
3. dato *P,* nessuno è in grado di trovare *P',* tale che $MD(P') = MD(P)$
4. se l'input cambia anche di 1 bit, l'output diventa completamente diverso.

Per soddisfare il criterio 3 l'hash dovrebbe essere lungo almeno 128 bit e preferibilmente di più. Per il criterio 4, l'hash deve modificare in modo radicale i bit in input, in modo analogo agli algoritmi di cifratura a chiave simmetrica che abbiamo discusso. Calcolare un message digest dal testo in chiaro è molto più veloce che non cifrare il testo stesso con un algoritmo a chiave pubblica, per questo motivo i message digest possono essere usati per velocizzare gli algoritmi di firma digitale. Vediamo un esempio, considerando di nuovo il protocollo della Figura 8.18. Al posto di firmare P con $K_{BB}(A, t, P)$, *BB* calcola il message digest applicando *MD* a *P,* cioè calcolando $MD(P)$. *BB* quindi inserisce $K_{BB}(A, t, MD(P))$ come quinto elemento della lista cifrata con K_B che viene mandata a Bob, al posto di $K_{BB}(A, t, P)$.

Se dovesse nascere una disputa, Bob può mostrare sia P che $K_{BB}(A, t, MD(P))$. BB è in grado di decifrare questo messaggio per il giudice, a questo punto Bob ha $MD(P)$, che è garantito genuino, e il messaggio oggetto della disputa, P. Siccome è impossibile che Bob possa trovare un qualunque altro messaggio che produce l'hash in questione, il giudice sarà facilmente convinto che Bob dice il vero. Usare i message digest in questo modo fa risparmiare tempo di cifratura e costi di trasporto.

I message digest funzionano anche con i sistemi di crittografia a chiave pubblica, come mostrato nella Figura 8.20. In questo caso, Alice prima calcola il message digest del testo in chiaro, poi firma il message digest e manda sia il testo in chiaro sia il digest firmato. Se Trudy rimpiazza P durante la trasmissione, Bob se ne accorgerà nel momento in cui calcolerà $MD(P)$ dal messaggio originale.

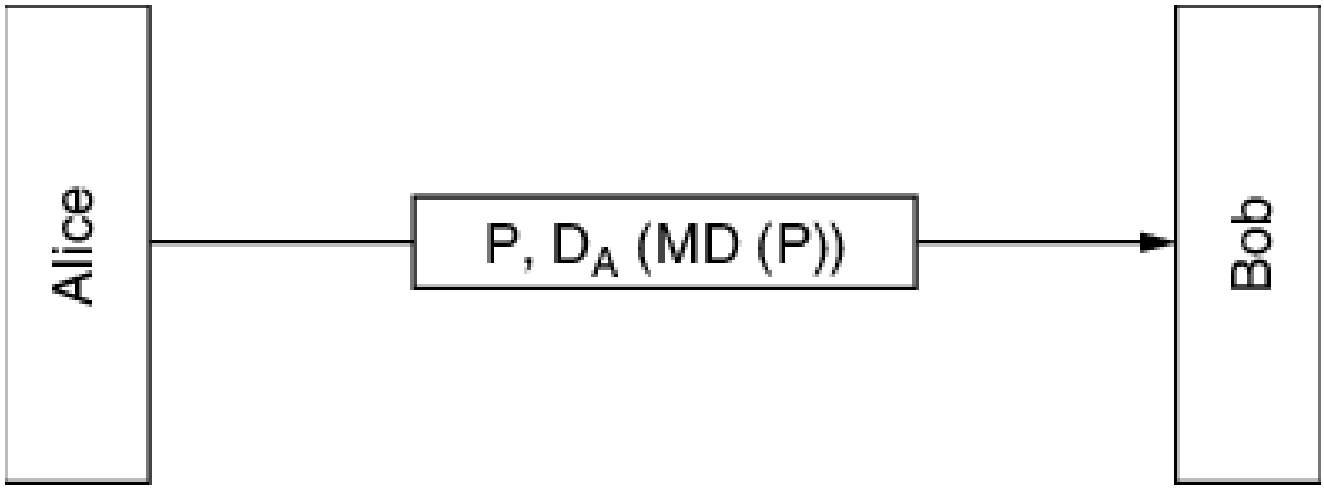


Figura 8.20 Firme digitali usando i message digest.

SHA-1 e SHA-2

Sono state proposte una varietà di funzioni per il message digest. Una delle funzioni più usate è **SHA-1** (*secure hash algorithm 1*, algoritmo di hash sicuro 1) (NIST, 1993). come tutti i message digest, funziona distruggendo i bit in un modo sufficientemente complicato per cui ogni bit di output è influenzato da ogni bit di input. SHA-1 fu sviluppato da NSA e benedetto da NIST in FIPS 180-1. Elabora i dati in input in blocchi da 512 bit e genera un message digest da 160 bit.

La Figura 8.21 mostra un tipico esempio di come Alice può mandare a Bob un messaggio non segreto ma firmato. Il testo in chiaro viene usato come input per l'algoritmo SHA-1 per ottenere l'hash da 160 bit. Alice firma l'hash con la sua chiave privata RSA e trasmette a Bob sia il testo in chiaro sia l'hash.

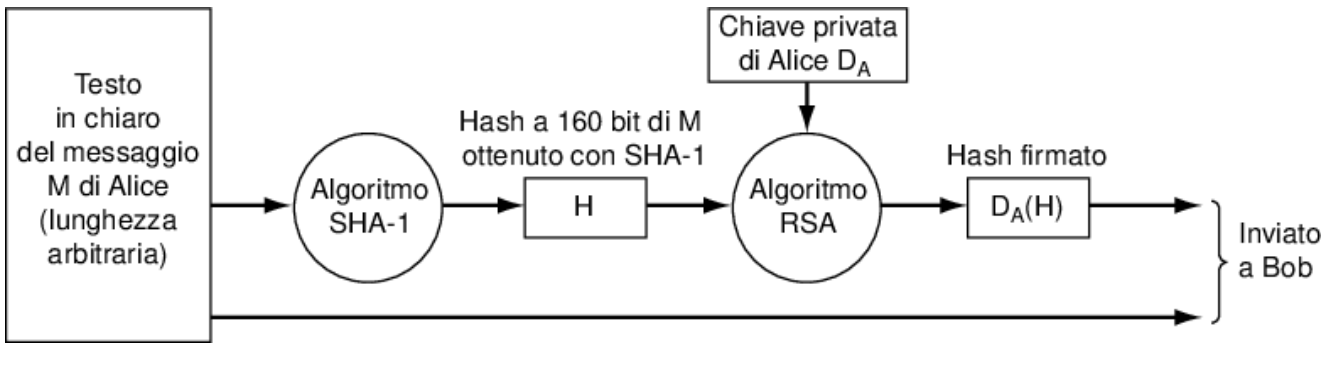


Figura 8.21 Uso di SHA-1 e RSA per firmare messaggi non segreti.

Dopo aver ricevuto il messaggio, Bob calcola l'hash SHA-1 dal testo in chiaro e applica la chiave pubblica di Alice all'hash firmato per ottenere l'hash originale, *H*. Se i due sono d'accordo, il messaggio viene considerato valido. Trudy non è in grado di modificare il testo (in chiaro) mentre è in transito per produrne un altro che abbia lo stesso hash *H*, quindi Bob riesce facilmente a identificare i cambiamenti eventualmente apportati da Trudy. Lo schema della Figura 8.21 è ampiamente usato per trasmettere messaggi dove l'integrità è importante, ma i contenuti non sono segreti. Con un costo di calcolo relativamente basso, questo schema riesce a garantire una probabilità molto alta di rilevare le modifiche al testo in transito.

Vediamo brevemente come funziona SHA-1. Si comincia con il riempimento del messaggio aggiungendo un bit a 1 in coda, seguito da tanti 0 quanti sono necessari per renderne la lunghezza multipla di 512 bit. Si calcola un numero a 64 bit contenente la lunghezza del messaggio prima del riempimento e lo si mette in OR con i 64 bit di ordine più basso. Nella Figura 8.22 il messaggio è mostrato con riempimento a destra, in quanto il testo e le figure si leggono da sinistra verso destra (cioè la posizione in basso a destra è normalmente considerata come la fine della figura). Per i computer, l'orientamento illustrato corrisponde alle macchine big-endian come le SPARC e l'IBM 360; tuttavia SHA-1 riempie sempre la fine del messaggio, indipendentemente dal tipo di computer utilizzato. L'algoritmo di SHA-1 utilizza cinque variabili a 32 bit dove viene accumulato l'hash, da H_0 a H_4, come mostrato nella Figura 8.22(b). Queste variabili sono inizializzate da costanti specificate nello standard. I blocchi da M_0 a M_{n-1} sono elaborati uno dopo l'altro. Per il blocco corrente le 16 word sono copiate nella parte iniziale di un array ausiliario di 80 word, *W*, come mostrato nella Figura 8.22(c); le altre 64 word di *W* si riempiono applicando la formula

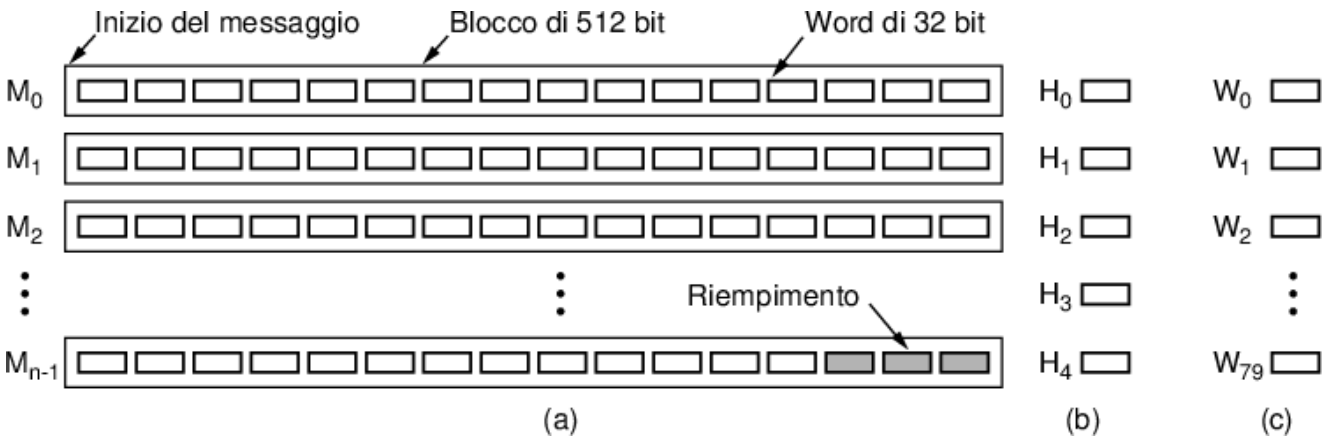


Figura 8.22 (a) Messaggio esteso a un multiplo di 512 bit. (b) Le variabili di output. (c) L'array di word.

1. $W_i = S^1(W_{i-3} \text{ XOR } W_{i-8} \text{ XOR } W_{i-14} \text{ XOR } W_{i-16}) \quad (16 \le i \le 79)$

Dove $S^b(W)$ rappresenta la rotazione circolare verso sinistra della word a 32 bit, *W*, di *b* bit. Cinque variabili di servizio, da *A* ad *E*, vengono inizializzate da H_0 fino ad H_4, rispettivamente. Il calcolo effettuato può essere espresso in pseudo-C nel modo seguente:

```
for (i = 0; i < 80; i++) {     temp = S^5(A) + f_i (B, C, D) + E + W_i + K_i;     E=D; D=C; C=S^30(B); B=A; A=temp;

}
```

dove le costanti K_i sono definite nello standard. Le definizioni delle funzioni di combinazione f_i sono:

1. f_t (*B,C,D*) = (*B* AND *C*) OR (NOT *B* AND *D*)$(0 \le i \le 19)$
2. f_t (*B,C,D*) = *B* XOR C XOR *D*$(20 \le i \le 39)$
3. f_t (*B,C,D*) = (*B* AND *C*) OR (*B* AND *D*) OR (*C* AND *D*)$(40 \le i \le 59)$
4. f_t (*B,C, D*) = *B* XOR *C* XOR *D*$(60 \le i \le 79)$

Quando tutte le 80 iterazioni del ciclo sono completate, le variabili da *A* a *E* vengono aggiunte ad H_0 fino ad H_4, rispettivamente.

Dopo aver elaborato il primo blocco di 512 bit si comincia con quello successivo. L'array *W* è inizializzato usando il nuovo blocco, mentre *H* rimate invariata. Quando anche questo blocco è stato elaborato, si continua con quello successivo, e così via fino a esaurimento dei blocchi in input. Quando anche l'ultimo blocco è stato elaborato, le cinque word a 32 bit nell'array *H* rappresentano l'output a 160 bit dell'hash crittografico. Il codice completo in C per SHA-1 è riportato in RFC 3174.

Nuove versioni di SHA-1, chiamate SHA-2, sono state sviluppate in grado di produrre hash da 224, 256, 384 e 512 bit. Non solo questi hash sono più lunghi degli hash di SHA-1, ma la funzione è stata cambiata per combattere alcune debolezze potenziali di SHA-1. SHA-2 non è ancora ampiamente usata, ma probabilmente lo sarà in futuro.

MD5

Per completezza, menzioneremo un altro popolare digest. **MD5** (Rivest, 1992) è il quinto di una serie di message digest progettati da Ronald Rivest. Molto brevemente, il messaggio viene allungato per raggiungere una lunghezza di 448 bit (modulo 512). Quindi la lunghezza originale del messaggio viene aggiunta in coda come un numero intero di 64 bit per dare un input totale la cui lunghezza sia un multiplo di 512 bit. Ogni round di calcolo prende un blocco di input da 512 bit e lo mischia con un buffer da 128 bit. Per essere sicuri, per mescolare viene usata una tabella costruita sulla funzione seno. Il motivo per utilizzare una funzione conosciuta è allontanare ogni sospetto che il progettista abbia costruito una backdoor attraverso la quale solo lui possa entrare. Questo processo continua finché tutti i blocchi di input sono stati consumati; il contenuto del buffer da 128 bit forma il message digest.

Dopo più di un decennio di uso massiccio e di studio, le carenze di MD5 hanno portato all'abilità di trovare collisioni o messaggi diversi con lo stesso hash (Sotirov, et al., 2008). Questa è la campana a morto per una funzione di digest perché significa che il digest non può più essere usato con sicurezza per rappresentare un messaggio. Quindi la comunità di chi si occupa di sicurezza considera che MD5 sia stato violato, dovrebbe essere rimpiazzato dove possibile e nessun nuovo sistema dovrebbe usarlo come parte della propria architettura. Nonostante ciò, potreste vedere ancora utilizzato MD5 in sistemi esistenti.

Attacco del compleanno

Nel mondo della crittografia, niente è mai quello che sembra. Si può pensare che ci voglia un numero di operazioni dell'ordine di 2^m per forzare un message digest di *m* bit. In realtà, spesso possono bastare $2^{m/2}$ operazioni, usando l'**attacco del compleanno**, un approccio pubblicato daYuval (1979) in un articolo diventato un classico, intitolato "how to swindle rabin" (*come derubare un rabbino*).

L'idea per questo attacco viene da una tecnica che i professori di matematica usano spesso nei loro corsi di probabilità. La domanda è: quanti studenti ci devono essere in una classe perché la probabilità che due persone abbiano il compleanno lo stesso giorno superi 1/2? Molti studenti si aspettano che la risposta sia un numero molto maggiore di 100. In realtà la teoria della probabilità ci dice che ne bastano 23. Non ne daremo una dimostrazione rigorosa, ma ragioneremo intuitivamente. Con 23 persone possiamo formare (23 × 22)/2= 253 coppie differenti, ognuna ha la probabilità di 1/365 di essere quella buona. Con questa osservazione, il risultato non è più sorprendente. Generalizzando, se c'è una funzione fra input e output con *n* valori di input (persone, messaggi, ecc) e *k* possibili valori di output (compleanni, message digest, ecc), ci sono $n(n - l)/2$ coppie di input. Se $n(n - l)/2 > k$, la possibilità di avere almeno una coppia con lo stesso output è

decisamente buona. Quindi, approssimativamente, per avere due output uguali basta avere $n > \sqrt{k}$–. Questo risultato significa che un message digest di 64 bit può essere forzato, con una buona probabilità, generando 2^{32} messaggi e cercandone due con lo stesso message digest.

Vediamo un esempio pratico. Il dipartimento di informatica alla State University ha una cattedra vacante e due candidati, Tom e Dick. Tom è entrato nel dipartimento due anni prima di Dick e quindi viene valutato per primo. Se ottiene il posto, Dick sarà scartato. Tom sa che la direttrice del dipartimento, Marilyn, ha un'alta considerazione del suo lavoro, quindi le chiede di scrivere una lettera di presentazione al rettore, che dovrà decidere dell'assunzione di Tom. Una volta inviate, tutte le lettere diventano confidenziali. Marilyn chiede alla sua segretaria, Ellen, di scrivere una lettera al rettore, dicendole gli argomenti da inserire. Quando la lettera sarà pronta, Marylin la controllerà, calcolerà il digest a 64 bit e la invierà al rettore; dopo, Ellen potrà inviare la lettera per e-mail. Sfortunatamente per Tom, Ellen è innamorata di Dick e vorrebbe sabotare Tom, quindi scrive la seguente lettera con le 32 opzioni elencate fra parentesi.

Egregio Rettore,

Questa *[lettera | missiva]* ha lo scopo di fornirle la mia *[onesta | franca]* opinione sul Prof. Tom Wilson, che *[è candidato | concorre]* per la cattedra *[attualmente | quest'anno]. [Conosco | Ho lavorato con]* Wilson per *[quasi | già]* sei anni. Egli è un *[eccellente | ammirevole]* ricercatore di grande *[talento | capacità]* noto *[in tutto il mondo | internazionalmente]* per la sua *[brillante | creativa]* conoscenza in *[molti | svariati]* problemi di *[grande difficoltà | impegnativa soluzione].*

È inoltre un *[insegnante | docente] [molto | notevolmente] [stimato | apprezzato].* I suoi studenti danno *[ai suoi corsi | alle sue lezioni]* voti *[spettacolari | massimi]. [Nel nostro dipartimento | nella nostra facoltà]* è 1' *[insegnante | docente]* più *[popolare | stimato]. [Inoltre | Per giunta]* il Prof. Wilson è *[abile | efficace]* nella ricerca di fondi. I suoi *[contatti | progetti]* hanno portato un *[notevole | significativo]* finanziamento *[al nostro dipartimento | alla nostra università].* Questi *[fondi | finanziamenti]* ci hanno *[permesso | consentito]* di *[portare avanti | sviluppare]* molti *[importanti | significativi]* progetti, *[fra cui | per esempio]* il progetto Stato 2000. Senza questi fondi *[non saremmo in grado | non potremmo]* proseguire il progetto, che è così *[importante | vitale]* per noi. La invito caldamente ad affidargli la cattedra.

Sfortunatamente per Tom, dopo aver composto e battuto la lettera in questione, Ellen ne scrive una seconda:

Egregio Rettore,

Questa *[lettera | missiva]* ha lo scopo di fornirle la mia *[onesta | franca]* opinione sul Prof. Tom Wilson, che *[è candidato | concorre]* per la cattedra *[attualmente | quest'anno]. [Conosco | Ho lavorato con]* Wilson per *[quasi | già]* sei anni. Egli è un *[mediocre | svogliato]* ricercatore poco noto nella sua *[disciplina | area].* Le sue ricerche *[raramente | quasi mai]* mostrano *[competenza | comprensione]* dei problemi *[principali | vitali]* dell'attività *[che svolgiamo | in corso oggi].* Inoltre, non è un *[insegnante | docente] [stimato | apprezzato].* I suoi studenti danno *[ai suoi corsi | alle sue lezioni]* voti *[bassi | insufficienti]. [Nel nostro dipartimento | Nella nostra facoltà]* è 1' *[insegnante | docente]* meno *[popolare | stimato],* noto *[soprattutto | principalmente]* nel *[dipartimento | gruppo di lavoro]* per la sua *[tendenza | propensione]* a *[ridicolizzare | imbarazzare]* gli studenti che *[imprudentemente | inutilmente]* fanno domande alle sue lezioni.

[Inoltre | Per *giunta]* il Prof. Wilson è *[incapace | inefficace]* nella ricerca di fondi. I suoi *[contatti | progetti]* hanno portato un *[minimo | insignificante]* finanziamento [*al nostro dipartimento | alla nostra università].* Se non verranno trovati presto *[nuovi fondi | nuovi finanziatori]* dovremo cancellare alcuni programmi vitali, come "Stato 2000". Sfortunatamente, a causa di queste *[condizioni | circostanze]* non posso raccomandare *[in coscienza | in buona fede]* Wilson per *[una cattedra | una posizione permanente].*

A questo punto Ellen programma il suo computer per calcolare, durante la notte, i 2^{32} message digest di ciascuna delle due lettere. È probabile che un digest della prima lettera possa essere uguale a un digest della seconda lettera. Altrimenti, Ellen può aggiungere qualche altra opzione e provare di nuovo durante il weekend. Supponiamo che trovi una coppia corretta. Chiamiamo *A* la lettera "buona" e *B* quella "cattiva".

Ellen manda per e-mail la lettera *A* a Marilyn per approvazione. La lettera *B* viene invece tenuta segreta. Marilyn, ovviamente, approva e calcola il suo digest a 64 bit, lo firma e manda per e-mail il digest firmato al rettore Smith. Indipendentemente, Ellen manda per e-mail la lettera *B* al rettore (non la lettera *A,* come dovrebbe).

Dopo aver ricevuto la lettera e il digest firmato, il rettore calcola il message digest della lettera *B,* vede che è uguale a quello inviato da Marilyn e licenzia Tom. Il rettore non si rende conto che Ellen è riuscita a generare due lettere con lo stesso message digest e che ne ha inviata una differente da quella che Marilyn aveva visto e approvato (finale alternativo:

Ellen dice a Dick quello che ha fatto. Dick è scandalizzato e la lascia. Ellen è furiosa e si confessa con Marilyn. Marilyn chiama il rettore e Tom riceve la cattedra). Con SHA-1, l'attacco del compleanno è difficile perché anche alla straordinaria velocità di 1 trilione di digest al secondo si impiegherebbero 32.000 anni per calcolare tutti i 2^{80} digest di due lettere con 80 varianti ognuna, e anche in tal caso non è garantita una corrispondenza. Con una sistema di 1.000.000 di chip in parallelo, 32.000 anni diventerebbero 2 settimane.

8.5 Gestione delle chiavi pubbliche

La crittografia a chiave pubblica consente la comunicazione sicura fra persone che non condividono una chiave comune e permette la firma dei messaggi senza la presenza di una terza parte fidata. Infine, i message digest firmati abilitano e facilitano la verifica dell'integrità dei messaggi.

Rimane però un problema su cui abbiamo sorvolato troppo velocemente: se Alice e Bob non si conoscono, come fanno a procurarsi le rispettive chiavi pubbliche per cominciare la comunicazione? La soluzione ovvia, quella di mettere la chiave sul proprio sito Web, non funziona per il seguente motivo. Supponiamo che Alice voglia cercare la chiave pubblica di Bob sul suo sito Web. Come procede? Comincia digitando l'URL di Bob. Il suo browser cerca sul DNS l'indirizzo della home page di Bob e manda la richiesta *GET,* come mostrato nella Figura 8.23. Sfortunatamente, Trudy intercetta la richiesta e risponde con una home page falsificata, probabilmente una copia della home page di Bob con la differenza che la chiave pubblica di Bob è stata sostituita con quella di Trudy. Quando Alice cifra il suo primo messaggio usando E_T Trudy lo decifra, lo legge, lo cifra di nuovo con la chiave pubblica di Bob e lo invia a Bob, il quale è ignaro del fatto che Trudy sta leggendo i messaggi in arrivo. A peggiorare le cose, rimane il fatto che Trudy potrebbe anche modificare i messaggi prima di cifrarli e spedirli a Bob. Chiaramente è necessario un qualche meccanismo per assicurarsi che le chiavi pubbliche possano essere scambiate in modo sicuro.

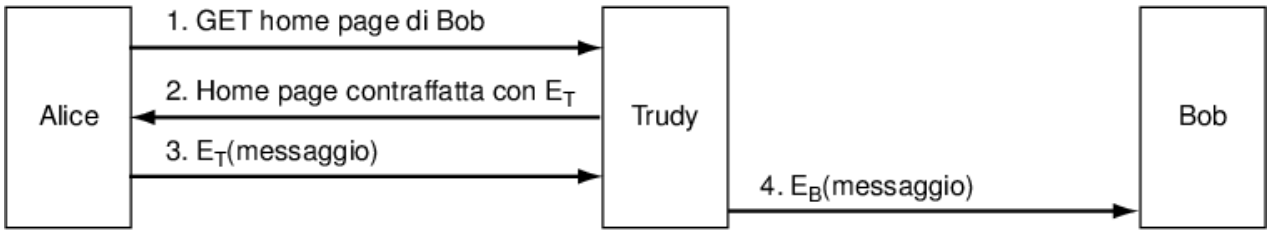


Figura 8.23 Uno dei metodi che Trudy può usare per forzare la cifratura a chiave pubblica.

Certificati

Come primo tentativo per distribuire le chiavi pubbliche in modo sicuro, possiamo immaginare l'esistenza di un centro di distribuzione delle chiavi (KDC, *key distribution center*), aperto 24 ore su 24 per fornire le chiavi pubbliche su richiesta. Uno dei problemi associati a questo tipo di soluzione è la mancanza di scalabilità e quindi il fatto che il centro di distribuzione delle chiavi diventerebbe presto il collo di bottiglia del sistema. Inoltre, se questo centro dovesse andare fuori servizio la sicurezza di Internet subirebbe un'improvvisa battuta d'arresto. Per questo motivo è stata sviluppata

una soluzione differente, che non richiede un centro di distribuzione delle chiavi on-line a tutte le ore. In effetti non deve essere affatto on-line: quello che deve fare è certificare le chiavi pubbliche che appartengono alle persone, alle aziende e alle altre organizzazioni. Un'organizzazione che certifica le chiavi pubbliche è chiamata una **CA** (*certification authority*, autorità di certificazione). Supponiamo, per esempio, che Bob voglia autorizzare Alice, insieme ad altre persone, a comunicare con lui in modo sicuro. Bob può rivolgersi alla CA portando la sua chiave pubblica e il suo passaporto o un altro documento, e chiedere di essere certificato. La CA emette un certificato simile a quello della Figura 8.24, ne calcola l'hash SHA-1 e lo firma con la chiave privata di CA. Bob paga un compenso alla CA e ottiene un floppy disk con il certificato e l'hash firmato.

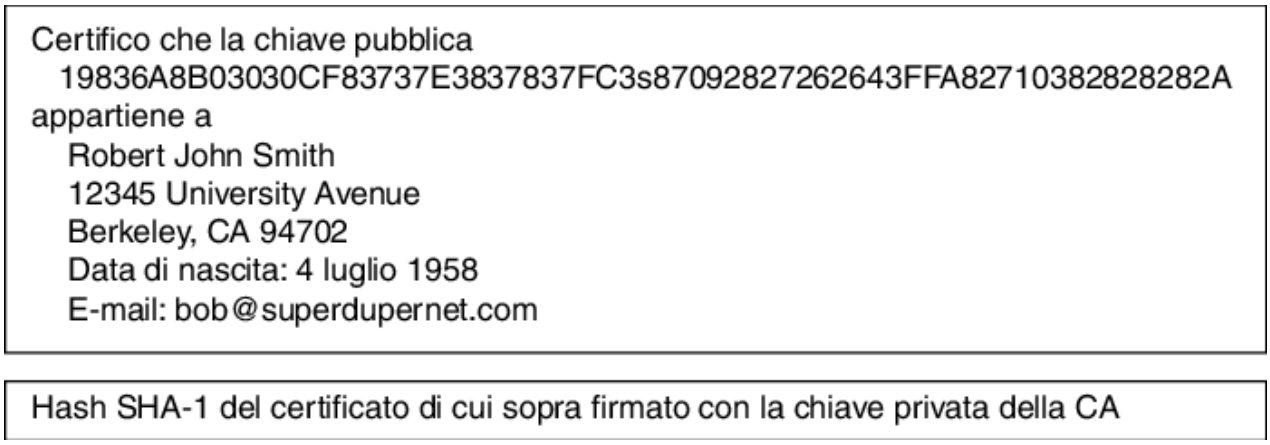


Figura 8.24 Esempio di certificato e del suo hash firmato.

Lo scopo principale di un certificato è quello di legare una chiave pubblica al nome di un protagonista (individuo, azienda, ecc). I certificati, di per sé, non sono né segreti né protetti. Per esempio Bob potrebbe decidere di mettere il suo nuovo certificato sul proprio sito Web, con un link sulla pagina principale del tipo "fare clic qui per vedere il mio certificato con la chiave pubblica". Il link riporterebbe il certificato e la firma (l'hash SHA-1 firmato del certificato).

Torniamo alla situazione della Figura 8.23. Quando Trudy intercetta la richiesta di Alice che vuole leggere la home page di Bob, che cosa può fare? Può mettere il suo certificato, con la relativa firma, nella pagina falsificata; ma quando Alice leggerà il certificato capirà immediatamente che non sta parlando con Bob, in quanto il nome di Bob non è sul certificato. Trudy può anche modificare al volo la home page di Bob, rimpiazzando la chiave pubblica di Bob con la sua chiave. Però, quando Alice calcola l'hash SHA-1 del certificato e lo confronta con il valore ottenuto applicando la chiave pubblica di CA alla firma del certificato, vede che sono diversi e quindi c'è qualcosa che non funziona. Operando in questo modo, Alice riesce a essere sicura di avere la chiave pubblica di Bob e non quella di Trudy o di qualcun altro. Come avevamo promesso, questo schema non richiede che la CA sia disponibile durante la verifica, eliminando quindi un potenziale collo di bottiglia. La funzione principale di un certificato è quella di legare una chiave pubblica con un protagonista. Un certificato può anche essere usato per legare una chiave pubblica a un

attributo. Per esempio, un certificato può affermare: questa chiave pubblica appartiene a una persona maggiorenne. Possiamo perciò usare un certificato per provare che il proprietario della chiave privata non è minorenne e può accedere a del materiale non adatto ai bambini; il tutto senza dover esporre l'identità del proprietario. Tipicamente, la persona che possiede il certificato lo invia al sito Web, al protagonista o al processo interessati a determinare l'età della controparte. Questo sito, protagonista o processo procede generando un numero casuale, che viene usato come chiave della sessione per il resto della comunicazione.

Un altro esempio di un certificato che contiene un attributo è dato da un sistema distribuito orientato agli oggetti. Ogni oggetto normalmente contiene diversi metodi. Il proprietario dell'oggetto può fornire, a ogni client, un certificato contenente l'elenco dei metodi che il client può utilizzare. Questo elenco viene poi legato a una chiave pubblica tramite un certificato. Anche in questo caso il detentore del certificato, una volta che ha provato di essere in possesso della chiave privata, viene autorizzato a eseguire i metodi nella lista. Questo schema ha la proprietà di non richiedere la conoscenza dell'identità del detentore del certificato, che è molto utile nelle situazioni in cui la privacy è importante.

X.509

Se tutte le persone interessate alla firma elettronica andassero dalla CA con un tipo diverso di certificato, l'operazione di gestire i diversi formati diventerebbe presto proibitiva. Per risolvere questo problema è stato sviluppato uno standard per i certificati, poi approvato dalla ITU. Lo standard è chiamato **X.509** ed è largamente utilizzato su Internet. Ne sono già state sviluppate tre versioni a partire dallo standard iniziale del 1988; in seguito discuteremo la V3.

X.509 è stato influenzato pesantemente dal mondo OSI, portandosi dietro alcune delle sue peggiori caratteristiche (per esempio la nomenclatura e la codifica). Sorprendentemente IETF ha approvato X.509 anche se, cercando di operare al meglio, ha generalmente ignorato OSI in tutte le altre aree, dai nomi delle macchine ai protocolli di trasporto, agli indirizzi e-mail. La versione di IETF di X.509 è descritta nell'RFC 3280. La funzione principale di X.509 è quella di descrivere i certificati. I campi primari di un certificato sono elencati nella Figura 8.25, dove la descrizione dovrebbe dare un'idea generale del significato di ciascun campo. Per ulteriori informazioni, si consiglia di consultare lo standard stesso o l'RFC 2459.

Campo	Significato
Version	Numero della versione di X.509
Serial number	Il certificato è univocalmente identificato da questo numero più il nome della CA
Signature algorithm	Algoritmo usato per firmare il certificato
Issuer	Nome X.500 della CA
Validity period	Inizio e fine del periodo di validità

Campo	Significato
Subject name	L'entità proprietaria della chiave da certificare
Public key	La chiave pubblica del soggetto e l'ID dell'algoritmo che la usa
Issuer ID	Facoltativo: identificativo univoco di chi emette il certificato
Subject ID	Facoltativo: identificativo univoco del soggetto del certificato
Extensions	Sono state definite molte estensioni
Signature	La firma del certificato (firmato dalla chiave privata della CA)

Figura 8.25 I campi essenziali di un certificato X.509.

Per esempio, se Bob lavora alla divisione prestiti della Money Bank, il suo indirizzo X.500 potrebbe essere:

```
/C=US/0=MoneyBank/OU=Loan/CN=Bob/
```

dove *C* sta per country (nazione), *O* per organizzazione, *OU* per organizational unit (unità organizzativa) e *CN* sta per common name (nome comune). Le CA e altre entità sono chiamate in modi simili.

Un problema sostanziale con i nomi X.500 è il seguente: quando Alice tenta di contattare *bob@moneybank.com* e le viene dato un certificato con un nome X.500, per Alice può non essere ovvio che il certificato ottenuto si riferisce proprio al Bob che lei sta cercando. Fortunatamente, a partire dalla versione 3, è permesso usare i nomi DNS al posto di quelli X.500, quindi il problema alla fine potrebbe scomparire. I certificati sono codificati con OSI **ASN.l** (*abstract syntax notation 1,* notazione sintattica astratta N.l), che può essere pensata come una sorta di struct nel linguaggio C, a parte una notazione particolarmente prolissa. Altre informazioni su X.509 si trovano in (Ford e Baum, 2000).

Infrastrutture a chiave pubblica

Naturalmente, affidarsi a una sola CA per emettere tutti i certificati del mondo è una soluzione che non può funzionare. Collasserebbe sotto il carico di lavoro e sarebbe anche un punto centrale che se non funzionante comprometterebbe tutto il sistema (*single point of failure*). Una soluzione possibile consiste nell'avere diverse CA, tutte gestite dalla stessa organizzazione e tutte con la stessa chiave privata usata per firmare i certificati. Questa soluzione risolverebbe i problemi del carico e del punto centrale, ma ne introdurrebbe anche uno nuovo: la sicurezza della chiave. Se ci fossero dozzine di server sparsi per il mondo, tutti con la stessa chiave privata, la possibilità che la chiave

venga rubata o che tuttavia trapeli all'esterno risulterebbe molto amplificata. Visto che la compromissione di questa chiave rovinerebbe l'infrastruttura di sicurezza elettronica del mondo, il fatto di avere un'unica CA comporta un grave rischio. Inoltre, quale organizzazione dovrebbe gestire la CA? È difficile immaginare un'autorità che possa essere accettata in tutto il mondo come legittima e degna di fiducia. In alcuni paesi la gente vorrebbe che fosse il governo, in altri invece si desidererebbe in contrario. Per questo motivo si è evoluto un metodo differente per certificare le chiavi pubbliche. Questo metodo va sotto il nome generico di **PKI** (*public key infrastructure,* infrastruttura a chiave pubblica). In questo paragrafo faremo un riassunto del funzionamento generale, anche se i dettagli probabilmente evolveranno nel tempo sotto la spinta delle diverse proposte che vengono avanzate.

Una PKI ha diversi componenti, fra cui gli utenti, le CA, i certificati e le directory.[3] La funzione della PKI è quella di fornire la struttura per inserire tutte queste componenti e definire gli standard per i diversi tipi di documenti e protocolli. Un esempio particolarmente semplice di PKI è la gerarchia di CA, riportata nella Figura 8.26. In questo esempio abbiamo riportato tre livelli, ma nella realtà possono essere di più o di meno. La CA di livello più alto, Root (*radice*), certifica le CA di secondo livello, che chiamiamo **RA** (*regional authorities,* autorità regionali) perché possono coprire delle regioni geografiche, come uno stato o un continente. Questo termine non è standard, in effetti nessun termine per definire i vari livelli dell'albero è veramente standard. A loro volta, le RA certificano le vere e proprie CA, che emettono i certificati X.509 alle organizzazioni e ai privati. Quando la root autorizza una nuova RA, genera un certificato X.509 che ne sancisce l'avvenuta approvazione e che include la chiave pubblica della nuova RA, quindi lo firma e lo passa alla RA. Analogamente, quando una RA approva una nuova CA, produce e firma un certificato che riporta l'avvenuta approvazione e che contiene la chiave pubblica della CA.

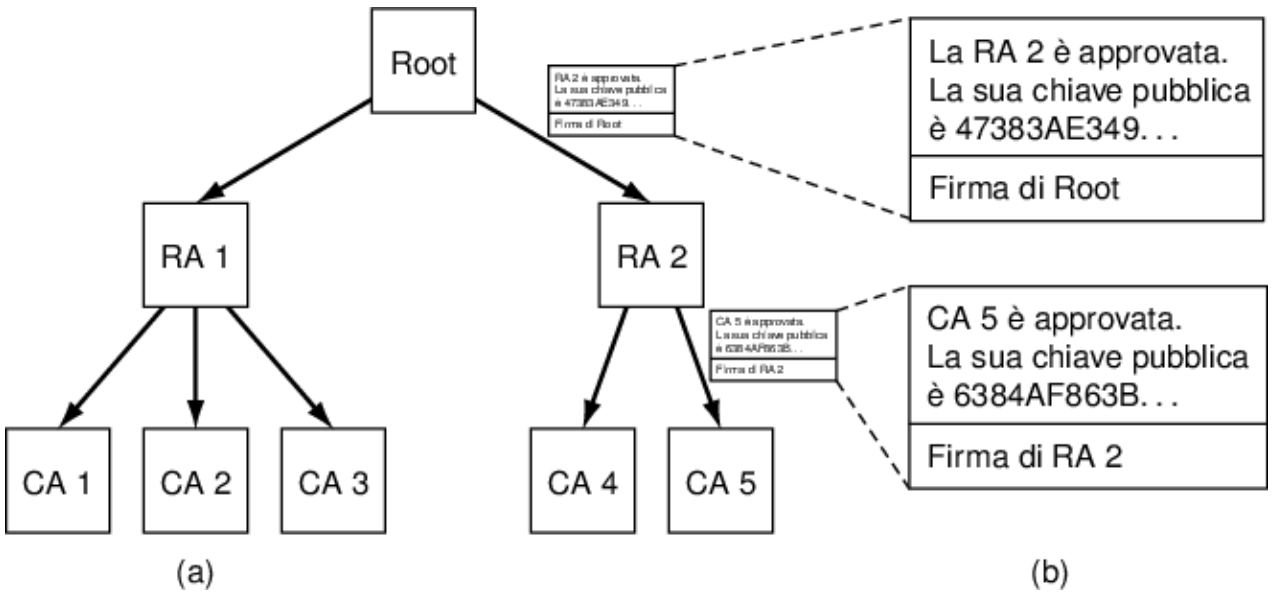


Figura 8.26 (a) PKI gerarchico. (b) Catena di certificati.

La nostra PKI funziona in questo modo. Supponiamo che Alice abbia bisogno della chiave pubblica di Bob per poter comunicare con lui. Alice cerca e trova un certificato che contiene la chiave di Bob firmato da CA 5. Però Alice non ha mai sentito parlare di CA 5. Per quanto ne sa Alice, CA 5 porrebbe essere la figlia di 10 anni di Bob. Alice può andare da CA 5 e chiedere: provami il fatto che tu sei un'autorità legittima. CA 5 risponde con il certificato che ha preso da RA 2, che contiene la chiave pubblica di CA 5. Armato della chiave pubblica di CA 5, Alice può verificare che il certificato di Bob è stato in effetti firmato da CA 5 ed è quindi valido.

Questo è vero a patto che RA 2 non sia il figlio di 12 anni di Bob. Quindi il passo successivo consiste nel chiedere a RA 2 di dimostrare la sua legittimità. La risposta alla richiesta è data dal certificato firmato da root e che contiene la chiave pubblica di RA 2. A questo punto Alice è sicura di avere la chiave pubblica di Bob. Come fa Alice a trovare la chiave pubblica di root? Si suppone che tutti conoscano la chiave pubblica di root. Per esempio, il browser di Alice può essere stato rilasciato con la chiave pubblica di root già al suo interno.

Bob è una persona collaborativa e non vuole che Alice faccia troppo lavoro. Sa che Alice dovrà controllare CA 5 e RA 2, quindi per risparmiarle del lavoro raccoglie i due certificati necessari e li invia insieme al suo. Alice può usare la sua conoscenza della chiave pubblica di root per verificare il certificato di livello più alto, prendere la chiave che si trova al suo interno e quindi verificare anche il secondo certificato. In questo modo, Alice non ha bisogno di contattare nessuno per effettuare le verifiche. Visto che i certificati sono tutti firmati, Alice può facilmente identificare i tentativi di modificare i loro contenuti. Una catena di certificati che risale fino alla root, come quella descritta, viene a volte chiamata una **chain of trust** (*catena di fiducia*) o un **certification path** (*percorso di certificazione*). Questa tecnica è ampiamente usata nella pratica.

Ovviamente, abbiamo ancora il problema di chi debba gestire la root. La soluzione consiste nel non avere una singola root, ma di averne tante, ognuna con i suoi RA e CA. In effetti, i browser moderni contengono già al loro interno le chiavi pubbliche di oltre 100 root, a volte chiamati **trust anchor** (*ancore fiduciarie*). In questo modo si evita di dover avere una singola autorità fiduciaria per il mondo intero.

Rimane il problema di come viene presa la decisione da parte del produttore del browser relativamente a quali trust anchor sono affidabili e quali no. Alla fine dei conti sarà l'utente del browser a doversi fidare che il produttore abbia fatto la scelta giusta e non abbia semplicemente incluso tutte le trust anchor che hanno pagato un prezzo per entrare nella lista. La maggior parte dei browser permette agli utenti di controllare le chiavi delle root (di solito nella forma di certificati firmati dalla root) e di cancellare quelli che sembrano sospetti.

Directory

Un'altra questione che viene affrontata da tutte le PKI è quella della memorizzazione dei certificati (e delle loro catene fino a qualche trust anchor conosciuta). Una possibilità consiste nel far sì che ciascun utente memorizzi i propri certificati. Questo modo di operare è sicuro (non c'è modo per gli utenti di manomettere i certificati firmati, senza essere scoperti), però è anche scomodo. Una proposta alternativa consiste nell'usare il DNS come directory dei certificati. Bob, prima di contattare Alice, probabilmente deve cercare il suo indirizzo IP usando un DNS: quindi perché non far sì che il DNS restituisca a Bob anche l'intera catena di certificati, insieme all'indirizzo IP?

Alcune persone pensano che questa sia la soluzione giusta, altre invece preferiscono usare dei directory server dedicati alla gestione dei certificati X.509. Tali directory potrebbero fornire servizi di ricerca usando le proprietà dei nomi X.500. Per esempio, in teoria una directory di questo tipo potrebbe rispondere a domande come: "dammi la lista delle persone che si chiamano Alice, lavorano nel dipartimento vendite e si trovano in Canada o negli U.S.A.". LDAP potrebbe essere un candidato per contenere questo tipo di informazioni.

Revoca

Anche il mondo reale è pieno di certificati, pensiamo ai passaporti e alle patenti di guida. In alcuni casi questi certificati possono essere revocati, per esempio la patente di guida può essere revocata in caso di guida in stato di ebbrezza o per altre infrazioni. Lo stesso problema esiste nel mondo digitale: il garante che ha emesso un certificato può decidere di revocarlo, in quanto la persona o l'organizzazione che l'ha ricevuto ne ha abusato in qualche modo. Il certificato può essere revocato anche nel caso in cui la chiave privata del ricevente sia stata esposta. Un'altra situazione, ancora più grave, si ha se la chiave privata della CA è stata compromessa. Quindi, il PKI deve affrontare anche il problema delle revoche, il che complica le cose.

Un primo passo in questa direzione consiste nel far sì che ogni CA periodicamente emetta un **CRL** (*certificate revocation list,* lista dei certificati revocati) contenente il numero seriale di tutti i certificati che ha revocato. Visto che i certificati contengono una data di scadenza, la CRL deve contenere solo i numeri seriali dei certificati da revocare che non sono ancora scaduti. Un certificato perde automaticamente validità dopo la sua data di scadenza, quindi non è necessario distinguere fra i certificati scaduti e quelli effettivamente revocati. In entrambi i casi i certificati non possono più essere usati. Sfortunatamente, introdurre le CRL significa che un utente che sta per usare un certificato deve anche prendere la CRL e controllare se il certificato in questione è stato revocato; in caso affermativo non dovrebbe usarlo. Anche se il certificato non è nella lista, potrebbe essere stato revocato poco dopo la pubblicazione della lista stessa. Quindi l'unico modo per esserne sicuri è quello d'interrogare la CA. La CA va interrogata anche in occasione del successivo uso di questo stesso certificato, in quanto (nel frattempo) il certificato potrebbe essere stato revocato.

Un'altra complicazione è data dal fatto che il certificato revocato potrebbe anche essere reso di nuovo valido, per esempio se era stato revocato in attesa di un pagamento dovuto che poi è stato saldato. Gestire la revoca (e magari anche il reinserimento) annulla una delle migliori proprietà dei certificati, cioè il fatto che possono essere usati senza dover contattare la CA.

Dove vanno memorizzate le CRL? Un buon posto sarebbe quello in cui sono memorizzati i certificati stessi. Una strategia consiste nel prevedere una pubblicazione periodica delle CRL da parte delle CA. Le directory procedono quindi a elaborare le CRL semplicemente rimuovendo i certificati revocali. Se invece non si usano le directory per memorizzare i certificati, le CRL possono essere tenute in cache in varie posizioni di facile accesso all'interno della rete. Visto che una CRL è di per sé un documento firmato, ogni tentativo di manomissione viene facilmente scoperto.

Se i certificati hanno una vita lunga, anche le CRL saranno lunghe. Per esempio, la lista delle carte di credito revocate sarà molto più lunga nel caso di carte con validità di 5 anni, che non nel caso di carte con validità 3 mesi. Un metodo standard per operare con le CRL consiste nell'emettere

sporadicamente una lista principale, seguita però da una serie di aggiornamenti frequenti. Con questo meccanismo si riduce la banda necessaria a distribuire le CRL.

8.6 Sicurezza delle comunicazioni

Abbiamo finito il nostro studio degli strumenti del mestiere, trattando le tecniche e i protocolli più importanti. Il resto del capitolo esamina le applicazioni pratiche e il modo in cui queste tecniche sono sfruttate per fornire sicurezza alle reti. Alla fine del capitolo si esamineranno anche alcuni aspetti sociali legati alla sicurezza.

I quattro paragrafi seguenti affrontano il tema della sicurezza delle comunicazioni. Vedremo quindi come sia possibile trasferire un flusso di bit dalla sorgente alla destinazione, in modo sicuro, senza che ci siano modifiche e tenendo alla larga i bit indesiderati. Queste non sono le sole problematiche di sicurezza delle reti, ma sono certamente gli argomenti più importanti e quindi costituiscono un buon punto di partenza.

IPsec

IETF sapeva da tempo che Internet mancava di sicurezza, ma aggiungerla non era facile, in quanto era in corso una guerra per decidere dove fosse meglio inserirla. La maggior parte degli esperti ritiene che per rendere Internet veramente sicura si deve aggiungere la cifratura e i controlli di integrità da capo a capo (cioè nel livello applicativo). In questo modo il processo sorgente cifra e/o protegge l'integrità dei dati e poi li invia al processo di destinazione che si prende in carico la decifrazione e/o la verifica di integrità. Ogni manipolazione dei dati effettuata fra questi due processi può essere rilevata, anche se viene realizzata a livello del sistema operativo. Il problema di questo approccio è che richiede la modifica di tutte le applicazioni per poterle usare con la sicurezza. In quest'ottica, il secondo miglior tentativo consiste nel mettere la sicurezza al livello di trasporto oppure in un nuovo livello, intermedio fra il livello applicativo e quello di trasporto. Anche così si ottiene una sicurezza da capo a capo, ma senza cambiamento delle applicazioni. L'approccio alternativo consiste nel partire dal presupposto che gli utenti non capiscono la sicurezza, non sono in grado di utilizzarla in modo corretto e che quindi nessuno vuole modificare i programmi esistenti in alcun modo. In questa ottica autenticare e/o cifrare i pacchetti diventa compito del livello di rete, senza coinvolgere gli utenti. Dopo anni di battaglie, questa filosofia ha raggiunto abbastanza consenso da stimolare la definizione di uno standard per la sicurezza del livello di rete. La spiegazione del successo consiste, in parte, nel fatto che avere la cifratura a livello di rete non impedisce agli utenti orientati alla sicurezza di operare tuttavia nel modo più sicuro; allo stesso tempo aiuta in qualche modo anche quella parte di utenti che ignorano la sicurezza.

Il risultato di questa guerra fu chiamato **IPsec** (*Internet protocol security*), che è descritto negli RFC 2401,2402, 2406 e altri documenti. Non tutti gli utenti vogliono la cifratura (perché è pesante dal punto di vista computazionale). Invece di rendere la cifratura opzionale, fu deciso di utilizzarla comunque, ma di permettere l'uso di un algoritmo nullo. L'algoritmo nullo è descritto nell'RFC 2410, dove si trovano anche i dettagli della sua ammirevole semplicità, facilità di implementazione e grande velocità.

Il progetto completo di IPsec definisce un'infrastruttura per fornire molteplici servizi, algoritmi e granularità. La ragione per avere molteplici servizi è che non tutti vogliono pagare il prezzo per avere tutti i servizi disponibili in ogni istante, quindi alcuni servizi vengono resi disponibili su

richiesta. I servizi principali sono: la segretezza, l'integrità dei dati e la protezione dagli attacchi di tipo ripetizione (dove l'intruso invia di nuovo un vecchio messaggio). Tutti questi servizi sono basati sulla crittografia a chiave simmetrica, in quanto le prestazioni sono fondamentali.

La ragione per avere più algoritmi è data dal fatto che un algoritmo che oggi crediamo sicuro potrebbe essere forzato in futuro. Rendendo IPsec indipendente dall'algoritmo, l'infrastruttura può sopravvivere anche se un particolare algoritmo viene forzato. Prevedendo diverse granularità, infine, è possibile proteggere una singola connessione TCP, come tutto il traffico fra due host oppure tutto il traffico fra due router sicuri o altre possibilità. Un aspetto parzialmente sorprendente dell'IPsec è il fatto che è orientato alla connessione, anche se si trova a livello IP. In definitiva questo fatto non è poi così sorprendente, in quanto per avere una qualche forma di sicurezza si deve stabilire una chiave da usare per un certo periodo di tempo: ciò è essenzialmente una sorta di connessione. Inoltre, le connessioni distribuiscono i costi dell'inizializzazione su una serie di pacchetti. Una "connessione" nel contesto dell'IPsec è chiamata **SA** (*security association*). Una SA è una connessione simplex (cioè monodirezionale) fra due punti terminali e ha un identificatore di sicurezza associato. Se è necessario inviare traffico sicuro nelle due direzioni, saranno necessarie due SA. Gli identificatori di sicurezza sono trasportati da pacchetti che viaggiano su queste connessioni sicure e vengono usati (all'arrivo dei pacchetti sicuri) per la ricerca delle chiavi e di altre informazioni rilevanti.

Le specifiche tecniche di IPsec contengono due parti principali. La prima parte descrive due nuove intestazioni che possono essere aggiunte ai pacchetti per contenere l'identificatore di sicurezza, i dati di controllo dell'integrità e altre informazioni. L'altra parte, **ISAKMP** (*Internet security association and key management protocol*, protocollo per l'associazione sicura e la gestione delle chiavi) si occupa di gestire le chiavi. Il protocollo principale di ISAKMP è **IKE** (*Internet key exchange*, scambio di chiavi su Internet), di cui dovrebbe essere usata la versione 2 descritta nell'RFC 4306, in quanto la versione precedente conteneva errori molto gravi, come sottolineato da Perlman e Kaufman (2000).

IPsec può essere utilizzato in una modalità scelta fra le due possibili. Nella **modalità trasporto** l'intestazione IPsec viene inserita subito dopo quella di IP. Il campo *Protocol* nell'intestazione IP è cambiato in modo da indicare che un'intestazione IPsec segue quella IP (prima dell'intestazione TCP). L'intestazione IPsec contiene le informazioni di sicurezza: principalmente l'identificatore di SA, un nuovo numero di sequenza, ed eventualmente un controllo d'integrità del payload. Nella **modalità tunnel** l'intero pacchetto IP, intestazione compresa, viene incapsulato nel corpo di un nuovo pacchetto IP con un'intestazione IP completamente diversa. La modalità tunnel è utile quando il tunnel arriva in un punto diverso dalla sua destinazione finale. In alcune situazioni il tunnel arriva a una macchina con un gateway sicuro, per esempio il firewall dell'azienda. Operando in questo modo, il firewall incapsula e decapsula i pacchetti che lo attraversano. Facendo terminare il tunnel su questa macchina sicura, le macchine sulla LAN dell'azienda non devono essere a conoscenza di IPsec. Solo il firewall sarà a conoscenza del tunnel IPsec.

La modalità tunnel è utile anche nella situazione in cui si vuole aggregare un'insieme di connessioni TCP, per poi gestirle come un unico flusso cifrato. In questo modo si evita che un intruso possa avere informazioni sul traffico: per esempio chi sta mandando dei pacchetti, a chi li sta mandando e quanti ne sta inviando. Consideriamo il caso in cui, durante una crisi militare, il traffico fra il Pentagono e la Casa Bianca si riduca di colpo. Simultaneamente, il traffico fra il Pentagono e alcune installazioni militari nelle Montagne Rocciose aumenta della stessa quantità di

cui il primo è diminuito. Un intruso potrebbe trarre delle informazioni utili da questi dati. Lo studio della distribuzione del flusso dei pacchetti, anche se cifrati, è chiamato **analisi del traffico** (*traffic analysis*). La modalità tunnel fornisce un modo per aggirare parzialmente questo tipo di analisi. Lo svantaggio della modalità tunnel è dato dal fatto che aggiunge delle ulteriori intestazioni IP e quindi aumenta la dimensione del pacchetto in modo sostanziale. Al contrario, la modalità trasporto non cambia di molto la grandezza del pacchetto. La prima ulteriore intestazione è detta **AH** (*authentication header,* intestazione di autenticazione). AH garantisce il controllo dell'integrità e la sicurezza contro gli attacchi di ricezione, ma non la segretezza (cioè non ha cifratura). L'uso di AH nella modalità trasporto è illustrato nella Figura 8.27. Nell'IPv4, AH viene messo fra l'intestazione IP (incluse le eventuali opzioni) e l'intestazione TCP. In Ipv6, AH è gestito come un'altra estensione dell'intestazione e trattata di conseguenza. In effetti, il suo formato è simile a quello di un'estensione standard di IPv6. Il campo payload può essere riempito fino a raggiungere una particolare lunghezza per facilitare l'algoritmo di autenticazione, come mostrato.

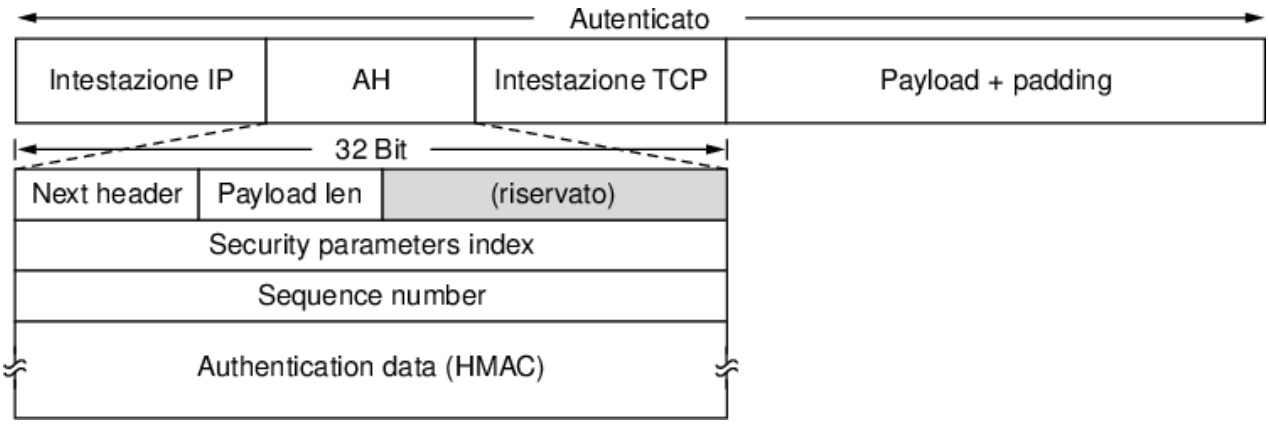


Figura 8.27 Intestazione di autenticazione IPsec nella modalità trasporto per IPv4.

Nell'intestazione AH, il campo *Next header field* serve per memorizzare il valore originale che aveva il campo IP *Protocol* prima di essere rimpiazzato con il valore 51. Quest'ultimo è usato per indicare che segue un'intestazione AH. Nella maggior parte dei casi, il valore sarà il codice per il TCP (6). Il campo *Payload length* contiene il numero di gruppi di 32 bit nell'intestazione AH meno 2.

Il *Security parameter index* è l'identificatore della connessione. Viene inserito dal mittente per indicare un particolare record nel database del ricevente. Questo record contiene la chiave condivisa usata per questa connessione e altre informazioni sempre relative alla connessione. Se questo protocollo fosse stato inventato da ITU al posto di IETF, questo campo sarebbe stato chiamato *virtual circuit number.*

Il campo *Sequence number* viene usato per attribuire un numero a tutti i pacchetti inviati in un SA. A ogni pacchetto viene attribuito un numero univoco, incluse le ritrasmissioni. In altre parole, un pacchetto ritrasmesso prende un numero di sequenza diverso dall'originale (anche se il suo numero

di sequenza TCP è lo stesso). Lo scopo di questo campo è quello di evitare attacchi di tipo ripetizione. Questi numeri di sequenza non sono riutilizzati: se vengono usate tutte le 2^{32} possibili combinazioni, dovrà essere stabilito un nuovo SA per poter continuare la comunicazione.

Infine abbiamo *Authentication data,* che è un campo a lunghezza variabile che contiene la firma digitale del payload. L'algoritmo di firma elettronica da utilizzare è negoziato quando viene stabilito l'SA. Di solito in questo stadio non viene usata la crittografia a chiave pubblica, perché i pacchetti devono essere elaborati in modo estremamente rapido, mentre tutti gli algoritmi a chiave pubblica conosciuti sono troppo lenti per questo compito. Per questo motivo IPsec è basato sulla crittografia a chiave simmetrica. Il mittente e il destinatario stabiliscono una chiave comune prima d'instaurare la SA, che viene usata nel calcolo della firma. Un metodo semplice consiste nel calcolare l'hash sui contenuti del pacchetto e della chiave insieme. Ovviamente, la chiave condivisa non viene trasmessa.

Uno schema di questo tipo è chiamato **HMAC** (*hashed message authentication code,* codice di autenticazione del messaggio con hash). È molto più veloce da calcolare della strategia alternativa, che consiste nell'usare prima SHA-1 per poi passare elaborare il risultato con RSA.

L'intestazione AH non permette la cifratura dei dati, quindi è utile principalmente quando è necessario un controllo d'integrità ma non serve la sicurezza. Una funzione importante di AH è data dal fatto che il controllo d'integrità copre alcuni campi dell'intestazione IP, in particolare quelli che non cambiano mentre il pacchetto viene trasmesso da router a router. Per esempio, il campo *Time to live* cambia in ogni salto di router e quindi non può essere incluso nel controllo d'integrità. L'indirizzo IP della sorgente viene incluso nel controllo, in questo modo diventa impossibile per un intruso falsificare l'origine del pacchetto.

Una intestazione alternativa per IPSec è **ESP** (*encapsulating security payload,* payload sicuro incapsulato). Il suo uso, sia per la modalità trasporto sia per quella tunnel, viene mostrato nella Figura 8.28.

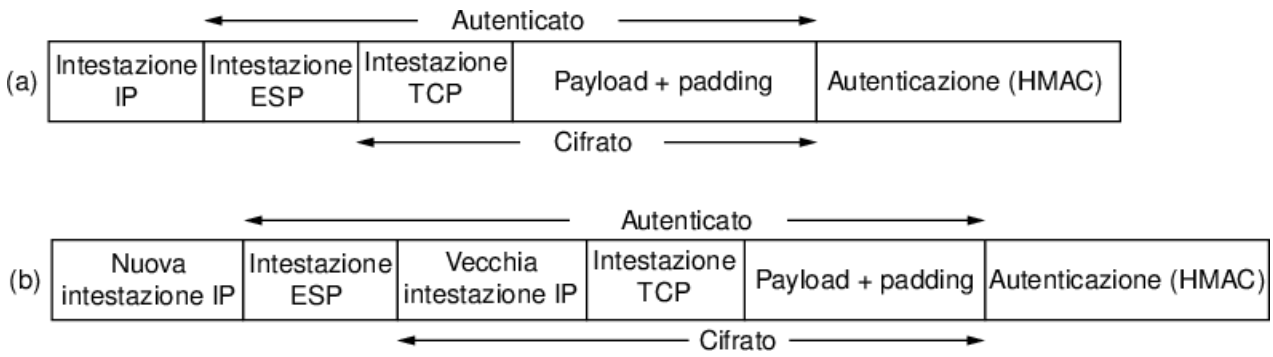


Figura 8.28 (a) ESP in modalità trasporto. (b) ESP in modalità tunnel.

L'intestazione ESP consiste di due gruppi di 32 bit che costituiscono i campi *Security parameters index* e *Sequence number,* con funzioni analoghe a quanto abbiamo discusso per AH. Un terzo gruppo che di solito segue i primi due (ma che tecnicamente non è parte dell'intestazione) è *Initialization vector*, usato per la cifratura dei dati. Se non viene usata la cifratura, questo campo è omesso. ESP fornisce un controllo di integrità HMAC come AH, ma diversamente da quest'ultimo non lo include nell'intestazione; lo aggiunge invece in coda al campo payload, come mostrato nella Figura 8.28. Utilizzare HMAC in coda al payload comporta dei vantaggi nelle implementazioni hardware. In questo caso HMAC può essere calcolato mentre i bit sono trasmessi verso l'interfaccia di rete e poi aggiunto alla fine: per lo stesso motivo Ethernet e altre LAN hanno il CRC dopo la fine dei dati, piuttosto che all'inizio. Con AH, invece, il pacchetto va messo in un buffer e la firma calcolata prima di poterlo trasmettere, pertanto questo modo di operare può avere l'effetto di ridurre il numero di pacchetti/sec che possono essere trasmessi.

ESP riesce a fare tutto quello che AH può fare, ha anche delle funzioni aggiuntive ed è più efficiente. La domanda a questo punto è: perché interessarsi ad AH? La risposta ha origini soprattutto storiche: in origine AH si occupava solo di integrità e ESP solo di segretezza. Successivamente, l'integrità è stata aggiunta a ESP. A questo punto, però, le persone che avevano progettato AH non volevano vederlo morire dopo tutto il lavoro che avevano fatto. Il loro unico punto a favore è che AH controlla parte dell'intestazione IP, mentre ESP non lo fa, ma questo è un argomento debole. Un'altra giustificazione debole è che un prodotto che supporti AH ma non ESP potrebbe avere minori problemi a ottenere una licenza per l'esportazione in quanto non presenta funzioni di cifratura. AH verrà probabilmente eliminato in un prossimo futuro.

Firewall

La possibilità di connettere qualunque computer che si trova in qualunque posto a qualunque altro computer in una qualsiasi località, può essere vista allo stesso tempo come una comodità e come una fonte di problemi. Per le persone che si collegano da casa e navigano su Internet, si tratta di un divertimento. Per i manager della sicurezza nelle aziende è invece un incubo. La maggior parte delle aziende possiede una gran quantità di informazioni on-line: segreti industriali, piani di sviluppo di prodotti, strategie di marketing, analisi finanziari e così via. La divulgazione di queste informazioni alla concorrenza può avere pesanti ripercussioni. Oltre al pericolo di vedere fuoriuscire delle informazioni, ci sono anche i pericoli legati alla diffusione di informazioni verso l'interno. In particolare, virus, worm e altre "pesti digitali" possono aprire una falla nella sicurezza e distruggere dati importanti oppure far sprecare una gran quantità del tempo agli amministratori di sistema che devono rimediare ai danni. Spesso, questi problemi sono portati all'interno della rete da un dipendente disattento, che voleva semplicemente provare qualche nuovo bel gioco. La conseguenza è la necessità d'instaurare dei meccanismi per tenere dentro i bit "buoni" e fuori quelli "cattivi". Un metodo è quello di usare IPsec. Questo approccio protegge i dati in transito fra i siti sicuri. IPsec non riesce però a fare nulla per tenere alla larga dalle LAN aziendali le pesti digitali. Per vedere come si può realizzare questo obiettivo, dobbiamo parlare di firewall.

I **firewall** (letteralmente, *muro di fuoco*) sono una versione moderna del vecchio rimedio medievale per le emergenze di sicurezza: scavare un profondo fossato attorno al proprio castello. Questa struttura obbligava chiunque volesse entrare o uscire dal castello a passare per un singolo ponte levatoio, dove la polizia poteva facilmente eseguire le sue ispezioni. Con le reti è possibile

realizzare lo stesso trucco: un'azienda può avere molte LAN connesse in modo arbitrario, ma tutto il traffico da e per l'azienda viene convogliato attraverso un unico ponte levatoio (il firewall), come mostrato nella Figura 8.29. Non ci sono altri percorsi.

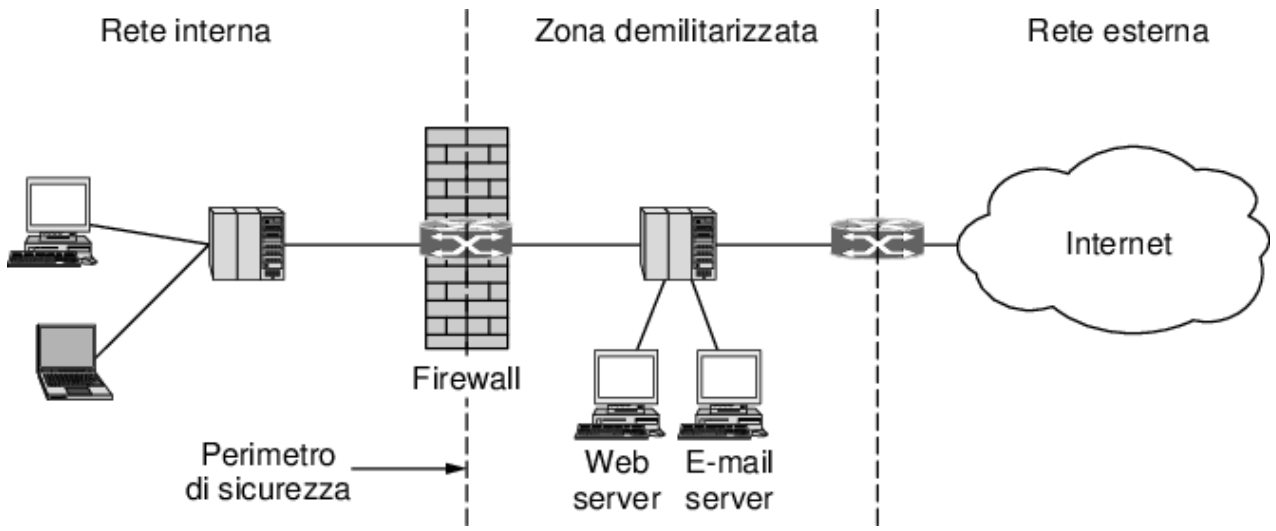


Figura 8.29 Un firewall che protegge una rete interna.

Il firewall agisce come un **filtro per i pacchetti** (*packet filter*). Ispeziona ogni singolo pacchetto in entrata e in uscita. I pacchetti che rispecchiano i criteri descritti dalle regole formulate dall'amministratore di rete vengono inoltrati normalmente; quelli che falliscono il test vengono buttati via senza troppe cerimonie.

Il criterio per il filtro è generalmente fornito sotto forma di una regola o tabella che elenca le sorgenti e le destinazioni accettabili, le sorgenti e le destinazioni bloccate e le regole di base in cui è descritto che cosa fare con pacchetti in entrata o in uscita di altre macchine. Nel caso comune di TCP/IP una sorgente o una destinazione potrebbe consistere in un indirizzo IP e una porta. Le porte indicano quale servizio si desidera. Per esempio la porta 25 TCP è per le mail e la porta 80 TCP è per http; alcune porte possono essere semplicemente bloccate. Una società, per esempio, può bloccare i pacchetti in entrata per tutti gli indirizzi IP combinati con la porta 79 TCP, una volta usata dal servizio Finger per recuperare gli indirizzi e-mail degli utenti, ma oggi viene poco usato.

Altre porte non vengono bloccate così facilmente. La difficoltà è che gli amministratori di rete vogliono sicurezza ma non possono tagliare le comunicazioni col mondo esterno. Tale soluzione sarebbe molto più semplice e migliore per la sicurezza, ma non ci sarebbe fine ai reclami degli utenti. Questo è dove soluzioni come **DMZ** (*demilitarized zone*, zona demilitarizzata) mostrata nella Figura 8.29 tornano utili. DMZ è la parte della rete della società che resta al di fuori del perimetro di sicurezza. Tutto può succedere qui. Piazzando una macchina come un Web server nella DMZ i computer in Internet possono contattarla per navigare nel sito Web della società. Il firewall può essere configurato per bloccare il traffico TCP in entrata sulla porta 80 così che i

computer in Internet non possano usarla per attaccare i computer sulla rete interna. Per permettere al Web server di essere gestito, il firewall può avere una regola per permettere le connessioni tra le macchine interne e il Web server.

I firewall sono diventati molto sofisticati nel tempo in una guerra agli armamenti con gli attaccanti. Originariamente, i firewall applicavano un insieme di regole indipendenti a ogni pacchetto, ma si è dimostrato difficile scrivere regole che permettessero funzionalità utili ma bloccassero tutto il traffico non desiderato. I **firewall statefull** (*firewall dotati di stato*) associano i pacchetti a delle connessioni e usano i campi dell'intestazione TCP/IP per tenere traccia delle connessioni. Questo permette di avere regole che consentono, per esempio, a un Web server esterno di spedire pacchetti a un host interno, ma solo se prima l'host interno stabilisce una connessione con il Web server esterno. Una regola simile non è possibile con architetture stateless (*prive di stato*) che devono fare passare o scartare tutti i pacchetti provenienti dal Web server esterno.

Un altro livello di raffinamento della elaborazione basata sullo stato è l'implementazione da parte del firewall di un *gateway a livello applicativo* (**application-level gateway**). Questa elaborazione induce il firewall a guardare dentro i pacchetti, anche oltre l'intestazione TCP, per vedere che cosa sta facendo l'applicazione. Con questa capacità è possibile distinguere il traffico HTTP usato per la navigazione del Web dal traffico HTTP usato per la condivisione di file tramite peer-to-peer. Gli amministratori possono scrivere regole per risparmiare la società dalla condivisione di file peer-to-peer ma permettere la navigazione Web, vitale per gli affari. Per tutti questi metodi il traffico in uscita può essere ispezionato come il traffico in entrata, per esempio per evitare che documenti sensibili siano inviati via mail all'esterno della società.

Come la discussione precedente dovrebbe chiarire, i firewall violano gli standard di stratificazione dei protocolli. Sono dispositivi a livello di rete, ma sbirciano nei livelli di trasporto e applicativo per implementare i filtri e questo li rende fragili. Per esempio, i firewall tendono a fare affidamento sulle convenzioni di numerazione standard delle porte per determinare che tipo di traffico è trasportato in un pacchetto. Le porte standard sono spesso usate, ma non da tutti i computer e neanche da tutte le applicazioni. Alcune applicazioni peer-to-peer selezionano le porte dinamicamente per evitare di essere facilmente individuate (e bloccate). La cifratura con IPsec o altri schemi nasconde al firewall le informazioni ad alto livello. Infine, un firewall non può parlare ai computer che comunicano attraverso di esso per dir loro quali politiche siano state applicate e perché la loro connessione è stata rifiutata. Deve semplicemente fingere di essere un cavo rotto. Per tutti questi motivi i puristi del networking considerano i firewall un difetto dell'architettura di Internet. Tuttavia, Internet può essere un posto pericoloso se siete un computer e i firewall danno una mano in questa direzione, per cui è probabile che restino.

Anche quando il firewall è configurato perfettamente, rimangono ancora molti problemi di sicurezza. Per esempio, se un firewall è configurato per permettere l'ingresso dei pacchetti solo da alcune specifiche reti (per esempio le altre sedi dell'azienda), un intruso dall'esterno del firewall può inserire un indirizzo di sorgente falso e quindi aggirare il controllo. Se una persona dall'interno vuole spedire all'esterno dei documenti segreti, li può cifrare oppure fotografare per poi spedirli come file JPEG, che aggirano qualunque filtro sulle parole. Vedremo più avanti che anche se tre quarti degli attacchi arrivano dall'esterno del firewall, gli attacchi che provengono dall'interno, per esempio da dipendenti scontenti, sono di solito i più dannosi (Verizon, 2009).

Un problema diverso con i firewall è che forniscono un solo perimetro di difesa: se viene fatta breccia in questa difesa si perdono tutte le scommesse. Per questi motivi i firewall sono spesso usati in una difesa a livelli. Per esempio, un firewall può fare la guardia all'entrata della rete interna e ogni computer può far uso di un proprio firewall. I lettori che pensano che un controllo di sicurezza sia sufficiente probabilmente non hanno fatto un volo aereo internazionale di recente.

Inoltre esiste un'intera classe di attacchi che i firewall non riescono a trattare. L'idea alla base dei firewall è quella di prevenire l'ingresso degli intrusi e l'uscita dei dati segreti. Sfortunatamente ci sono persone che non hanno niente di meglio da fare che cercare di bloccare la funzionalità di certi siti. Questo viene fatto inviando una grande quantità di pacchetti legittimi ai siti bersaglio, finché questi non collassano sotto il peso del carico di lavoro. Per esempio, per "azzoppare" un sito, un intruso può inviare un pacchetto TCP *SYN,* per stabilire una connessione. Il sito allocherà quindi uno slot per la connessione e invierà come risposta un pacchetto *SYN* + *ACK*. Se l'intruso non risponde, lo slot viene mantenuto per alcuni secondi finché non si verifica un timeout. Se l'intruso invia migliaia di richieste di connessione, tutti gli slot vengono riempiti e le connessioni legittime non possono più essere aperte. Gli attacchi che hanno come scopo quello di fermare l'operatività dell'obiettivo e non quello di rubare dei dati sono chiamati **DoS** (*denial of service*, negazione del servizio). Normalmente i pacchetti con le richieste hanno un indirizzo sorgente falso; in questo modo l'intruso non può essere identificato facilmente. Gli attacchi DoS ai principali siti Web sono comuni su Internet.

Una variante più pericolosa è possibile quando un intruso è già penetrato in centinaia di macchine sparse per il mondo e ordina a tutte queste macchine di attaccare simultaneamente un solo obiettivo. Questo approccio aumenta la potenza di fuoco dell'intruso, oltre a ridurre le possibilità di una sua identificazione, visto che i pacchetti arrivano da una gran quantità di macchine che appartengono a utenti ignari della situazione. Attacchi di questo tipo sono detti attacchi **DDoS** (*distributed denial of service,* negazione del servizio distribuita). È difficile proteggersi contro questo tipo di attacchi. Anche se la macchina attaccata identifica le false richieste, ci vuole sempre un po' di tempo per elaborarle e poi scartarle. Se questo tipo di richieste arrivano con una frequenza abbastanza alta, la CPU della macchina attaccata sarà costretta a spendere tutto il suo tempo per gestirle.

VPN, reti private virtuali

Molte aziende hanno uffici e stabilimenti situati in diverse città, a volte anche in diversi stati. Prima che esistessero le reti pubbliche per i dati, le aziende affittavano dalla compagnia telefonica le linee per collegare fra loro alcune o tutte le loro sedi e alcune aziende lo fanno ancora. Una rete costruita con i computer aziendali e le linee telefoniche affittate è detta **rete privata**. Un esempio di rete privata che connette tre siti è mostrata nella Figura 8.30(a). Le reti private funzionano bene e sono molto sicure. Quando le uniche linee presenti sono affittate non è possibile che il traffico di rete possa fuoriuscire dall'azienda, salvo che un intruso riesca a intercettare fisicamente le comunicazioni, il che non è affatto facile. Il problema delle reti private è dato dal loro costo. Infatti, affittare una singola linea T1 può costare migliaia di dollari al mese, affittare una T3 molto di più. Quando arrivarono sulla scena le reti pubbliche per i dati, e successivamente Internet, molte aziende vollero spostare il loro traffico dati (e se possibile anche vocale) sulle reti pubbliche, ma senza abbandonare la sicurezza delle linee private.

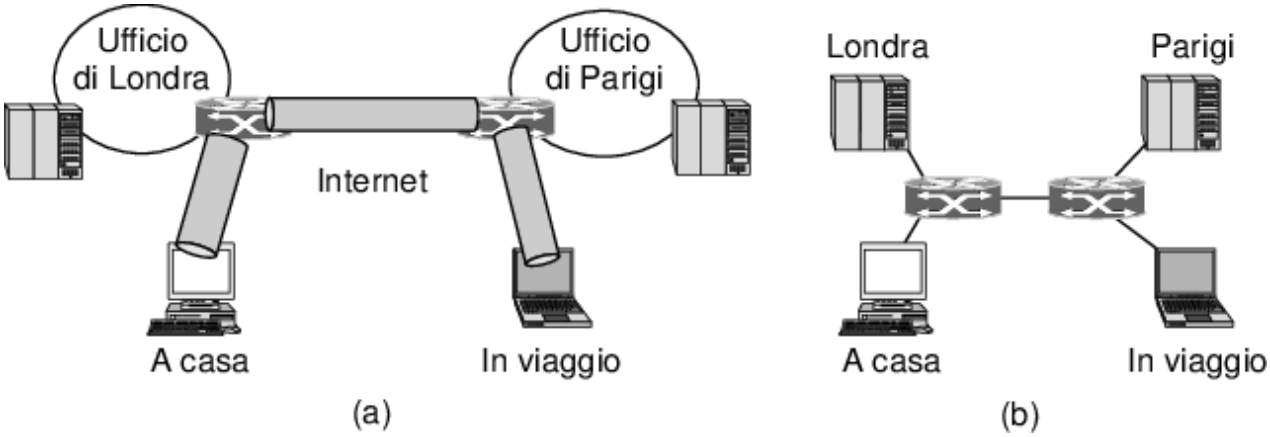


Figura 8.30 (a) Una rete privata virtuale. (b) Topologia vista dall'interno.

Questa domanda portò in breve tempo all'invenzione delle **VPN** (*virtual private network*, reti private virtuali). Le VPN permettono di sovrapporre delle reti sopra le reti pubbliche, ma senza abbandonare le proprietà di sicurezza tipiche delle reti private virtuali. Sono chiamate "virtuali" perché sono una mera illusione, così come i circuiti virtuali non sono veri circuiti e la memoria virtuale non è vera memoria.

Un approccio comune è costruire le VPN direttamente su Internet, attrezzando ogni ufficio con un firewall e creando dei tunnel attraverso Internet tra tutte le coppie di uffici, come illustrato nella Figura 8.30(a). Un ulteriore vantaggio nell'usare Internet per la connettività è che i tunnel possono essere organizzati a richiesta per includere, per esempio, il computer di un impiegato a casa o in viaggio fintanto che quella persona avrà a disposizione una connessione a Internet. Questa flessibilità è molto maggiore di quanto fornito con linee in affitto, ma dal punto di vista dei computer sulla VPN, la topologia assomiglia proprio al caso di una rete privata, come mostrato nella Figura 8.30(b). Alla partenza del sistema ogni paio di firewall deve negoziare i parametri del proprio SA, inclusi i servizi, le modalità, gli algoritmi e le chiavi. Se viene usato IPsec per creare i tunnel, è possibile aggregare tutto il traffico tra qualsiasi coppia di uffici in un solo SA autenticato e cifrato, fornendo così controllo d'integrità, segretezza e anche una considerevole immunità dall'analisi di traffico. Molti firewall hanno funzionalità VPN integrate, alcuni router ordinari possono fare altrettanto, ma siccome i firewall si occupano principalmente della sicurezza aziendale è naturale avere i punti terminali dei tunnel che fanno capo a firewall, fornendo una chiara separazione tra la società e Internet. In questo modo, i firewall, le VPN e IPsec con ESP in modalità tunnel sono una combinazione ampiamente utilizzata anche in pratica.

Dopo che sono state stabilite le SA il traffico comincia a scorrere. Per un router su Internet, un pacchetto che viaggia attraverso un tunnel VPN non è altro che un normale pacchetto. L'unica cosa strana è la presenza di un'intestazione IPsec dopo la normale intestazione IP. Visto che le intestazioni extra non hanno effetto sulle procedure di inoltro i router semplicemente le ignorano.

Un altro approccio che sta prendendo piede è quello di far creare la VPN all'ISP usando percorsi MPLS (come discusso nel Capitolo 5); tali percorsi possono essere configurati attraverso la rete dell'ISP tra gli uffici della società. Questi percorsi tengono il traffico VPN separato dal restante

traffico di Internet e può essere garantita una certa quantità di banda o altra qualità di servizio.

Un vantaggio fondamentale delle VPN è che sono completamente trasparenti al software dell'utente; il firewall crea e gestisce gli SA. L'unica persona consapevole di questa organizzazione è l'amministratore di sistema che deve configurare e gestire i gateway di sicurezza o l'amministratore dell'ISP che deve configurare i percorsi MPLS. Per chiunque altro è come avere ancora una linea privata in affitto. Per maggiori informazioni sulle VPN si consulti Lewis (2006).

Sicurezza wireless

È sorprendentemente facile progettare un sistema con VPN e firewall che risulta completamente sicuro dal punto di vista logico, mentre in pratica fa acqua da tutte le parti. Questa situazione si può verificare quando alcuni computer coinvolti sono wireless, quindi usano onde radio che attraversano il firewall in entrambe le direzioni. La portata delle reti 802.11 è normalmente di alcune centinaia di metri. Chiunque volesse spiare le comunicazioni di un'azienda può quindi parcheggiare la sua macchina nel parcheggio dei dipendenti al mattino, lasciare in macchina un laptop con accesso all'802.11 impostato per registrare tutte le comunicazioni che riesce a ricevere e quindi allontanarsi indisturbato. Alla fine della giornata l'hard disk sarà pieno di materiale interessante. In teoria, questa fuga di dati non dovrebbe accadere. In teoria, però, non dovrebbero accadere neanche le rapine alle banche. Molti dei problemi di sicurezza delle reti wireless sono causati dal fatto che i produttori delle stazioni di trasmissione wireless (gli access point) vogliono semplificare l'uso di questi prodotti agli utenti finali. Di solito, gli access point sono costruiti per funzionare direttamente alla prima accensione e senza particolari configurazioni. Questo si ottiene al costo di una scarsa sicurezza e del pericolo di vedere i propri dati trasmessi a tutti quelli che si trovano nel raggio di portata dell'apparecchio. Se l'access point è collegato alla ethernet, avremo come risultato che tutto il traffico ethernet sarà disponibile anche nel parcheggio. Le reti wireless sono una manna per le spie: dati a volontà senza dover fare nessun lavoro. È chiaro, a questo punto, che la sicurezza diventa ancora più importante per i sistemi wireless di quanto non lo sia già per le reti cablate. In questo paragrafo vedremo come alcune reti wireless gestiscono i problemi di sicurezza. Per ulteriori approfondimenti si rimanda a Nichols e Lekkas (2002).

Sicurezza di 802.11

Parte dello standard 802.11, originariamente chiamato **802.11i**, prescrive un protocollo di sicurezza a livello data link per prevenire che un nodo wireless legga o interferisca con i messaggi spediti tra altri due nodi wireless. È conosciuto anche con il nome commerciale di **WPA2** (*WiFi protected access 2*, accesso protetto WiFi versione 2). Il semplice WPA è uno schema provvisorio che implementa un sottoinsieme di 802.11i. Dovrebbe essere evitato in favore di WPA2.

Descriveremo 802.11i tra breve, ma cominciamo a dire che è un sostituto di **WEP** (*wired equivalent privacy,* privacy equivalente al cavo): la prima generazione dei protocolli di sicurezza di 802.11. WEP è stato progettato da un comitato di standard di rete, il che è un processo completamente differente rispetto a come, per esempio, NIST selezionò il progetto di AES. I risultati furono devastanti. Che cosa non funzionava? Più o meno tutto dal punto di vista della sicurezza. Per esempio, il WEP cifrava i dati per ottenere riservatezza tramite XOR con l'output di un cifrario a flusso. Sfortunatamente, modalità deboli di uso delle chiavi significavano che spesso l'output era

riutilizzato e questo portò a forzarlo con metodi banali. Inoltre il controllo d'integrità era basato su un CRC a 32 bit. Questa è una codifica efficiente per rilevare errori di trasmissione, ma è un meccanismo criptograficamente debole per sconfiggere gli aggressori.

Questi e altri difetti di progettazione resero WEP molto facile da compromettere. La prima dimostrazione pratica che WEP non funzionava avvenne quando Adam Stubblefield stava facendo uno stage presso AT&T (Stubblefield et al., 2002). È stato in grado di implementare completamente e testare un attacco delineato da Fluhrer et al. (2001) in una settimana, di cui la maggior parte del tempo è stato impiegato a convincere la direzione a comprargli una scheda WiFi da usare nei propri esperimenti. Il software per forzare le password WEP in un minuto è ora liberamente accessibile e l'uso di WEP è fortemente sconsigliato. Mentre previene accessi casuali non fornisce alcuna reale forma di sicurezza. Il gruppo 802.11i fu messo insieme in fretta quando fu chiaro che WEP era seriamente compromesso e produsse uno standard formale nel giugno del 2004.

Descriveremo ora 802.11i, il quale fornisce una reale sicurezza se viene configurato e usato correttamente. Ci sono due scenari comuni in cui WPA2 è usato. Il primo è un ambiente aziendale, in cui una società ha un server di autenticazione separato che ha il database di utenti e password che può essere usato per determinare se a un client wireless è permesso ad accedere alla rete. In questo ambiente, i client usano protocolli standard per autenticarsi sulla rete. I principali standard sono **802.1X**, con cui l'access point permette al client di portare avanti un dialogo con il server di autenticazione e osserva il risultato e **EAP** (*extensible authentication protocol*, protocollo estensibile di autenticazione) (RFC 3748) che indica come il client e il server di autenticazione interagiscono. In realtà EAP è un framework e altri standard definiscono i messaggi del protocollo. Tuttavia, non approfondiremo i molti dettagli di questo scambio perché non sono granché importanti per una visione generale.

Il secondo scenario è un ambiente domestico in cui non ci sono server di autenticazione, ma c'è una sola password condivisa usata dai client per accedere alla rete wireless. Questa configurazione è meno complessa dell'avere un server di autenticazione, motivo per cui è usato nelle abitazioni e in piccoli uffici, ma è anche meno sicuro. La differenza principale è che con un server di autenticazione ogni client ottiene una chiave per cifrare il traffico che rimane sconosciuto agli altri client. Con una sola password condivisa, vengono derivate chiavi diverse per ogni client, ma tutti i client hanno la stessa password e possono derivarsi le chiavi a vicenda se lo desiderano.

Le chiavi usate per cifrare il traffico sono calcolate come parte di un handshake di autenticazione. L'handshake ha luogo appena dopo che il client si associa con una rete wireless e si autentica con un server di autenticazione, se ce n'è uno. All'inizio dell'handshake, il client o ha la password di rete condivisa o ha la propria password per il server di autenticazione. Questa password è utilizzata per ottenere una chiave master (*master key*). Tuttavia la chiave master non viene usata direttamente per cifrare i pacchetti. È una pratica crittografica standard ottenere una chiave di sessione per ogni periodo di utilizzo, cambiare chiave ogni sessione ed esporre la chiave master all'osservazione il meno possibile. È la chiave di sessione quella che viene calcolata con l'handshake.

La chiave di sessione è calcolata con l'handshake di quattro pacchetti mostrato nella Figura 8.31. Prima, l'AP spedisce un numero casuale per l'identificazione. I numeri casuali usati solo una volta nei protocolli di sicurezza come questi sono chiamati **nonce**, che è più o meno la contrazione di "number used once" (*numeri usati una volta sola*). Anche il client seleziona un proprio nonce. Usa il

nonce, il proprio indirizzo MAC e quello dell'AP e la chiave master per calcolare la chiave di sessione, K_S. La chiave di sessione viene divisa in porzioni, ognuna delle quali viene utilizzata per scopi diversi, ma abbiamo omesso questo dettaglio. Ora il client ha le chiavi di sessione ma l'AP no, quindi il client spedisce il proprio nonce all'AP e l'AP esegue lo stesso calcolo per ottenere la stessa chiave di sessione. I nonce possono essere spediti in chiaro perché le chiavi non possono essere derivate da questi senza altre informazioni segrete. Il messaggio del client è protetto con un controllo d'integrità chiamato **MIC** (*message integrity check,* controllo di integrità del messaggio) basato sulla chiave di sessione. L'AP può controllare che il MIC sia corretto e così il messaggio è arrivato sicuramente dal client, dopo aver calcolato la chiave di sessione. Un MIC è solo un altro nome per il codice di autenticazione del messaggio, come in un HMAC. Il termine MIC è usato al posto di HMAC nei protocolli di rete a causa della potenziale confusione con gli indirizzi MAC.

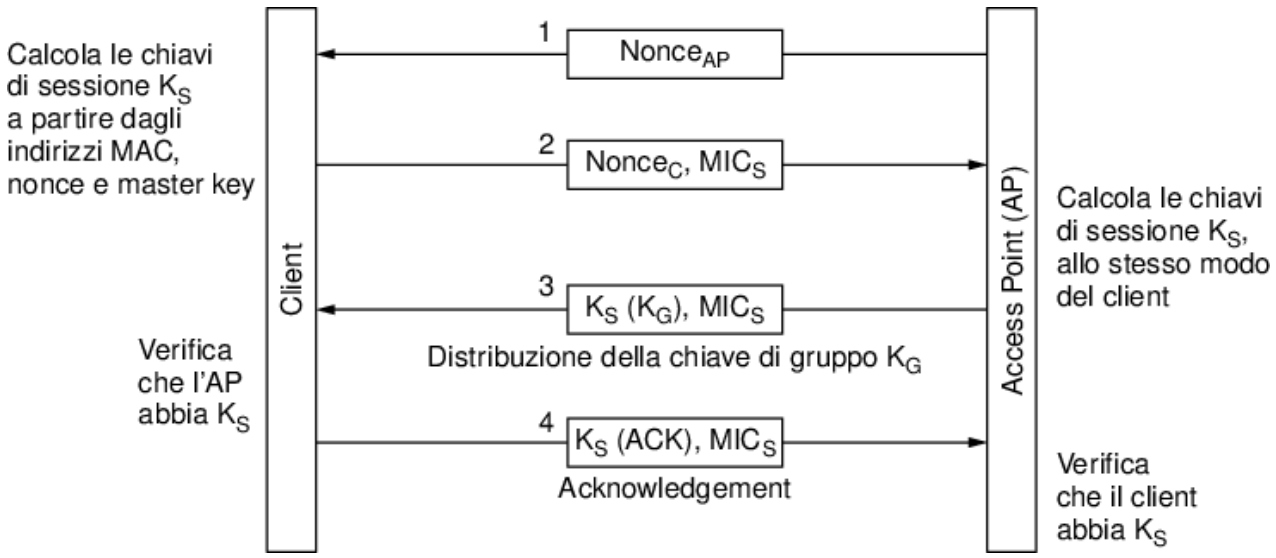


Figura 8.31 L'handshake per l'impostazione delle chiavi di 802.11i.

Negli ultimi due messaggi l'AP distribuisce una chiave di gruppo, K_G, al client e il client manda un acknowledgement per il messaggio. Ricevere questi messaggi permette al client di verificare che l'AP abbia le chiavi di sessione corrette e viceversa. La chiave di gruppo è usata per il traffico broadcast e multicast sulla LAN 802.11. Siccome il risultato dell'handshake è che ogni client ha la propria chiave di cifratura, nessuna di queste chiavi può essere usata dall'AP per pacchetti broadcast a tutti i client wireless; una copia separata dovrebbe essere spedita a ogni client usando la sua chiave. Invece una chiave condivisa viene distribuita così che il traffico broadcast possa essere spedito solo una volta e ricevuta da tutti i client. Dovrà però essere aggiornata mano a mano che i client vanno e vengono sulla rete.

Alla fine arriviamo alla parte in cui le chiavi sono davvero usate per fornire sicurezza. Due protocolli possono essere usati in 802.11i per fornire riservatezza, integrità e autenticazione ai messaggi. Come WPA uno dei protocolli, chiamato **TKIP** (*temporary key integrity protocol*, protocollo temporaneo di integrità a chiave), era una soluzione temporanea. Fu progettato per

migliorare la sicurezza su vecchie e lente schede 802.11, così da poter dare un po' più di si sicurezza WEP tramite un aggiornamento firmware. Tuttavia anche questo è stato violato, per cui è meglio rivolgersi all'altro protocollo raccomandato, **CCMP**. Per che cosa sta CCMP? È la forma abbreviata per il nome in qualche modo spettacolare di "counter mode with cipher block chaining message authentication code Protocol"(*protocollo per codice di autenticazione del messaggio in modalità contatore con blocchi di cifratura concatenati*). Lo chiameremo solo CCMP, però potete chiamarlo in qualsiasi modo vogliate.

CCMP funziona in un modo abbastanza semplice. Usa la cifratura AES con una chiave e una dimensione di blocco a 128 bit. La chiave arriva dalla chiave di sessione. Per fornire riservatezza, i messaggi sono cifrati con AES in modalità counter. Ricordate che abbiamo discusso le modalità di cifratura nel Paragrafo 8.2.3. Queste modalità sono ciò che previene che lo stesso messaggio venga cifrato nello stesso insieme di bit ogni volta. La modalità counter mescola un contatore nella cifratura. Per fornire integrità, il messaggio, inclusi i campi dell'intestazione, viene cifrato con una modalità di cipher block chaining e l'ultimo blocco da 128 bit è mantenuto come MIC. Quindi sia il messaggio (cifrato in modalità conter) che il MIC vengono spediti. Il client e l'AP possono entrambi eseguire questa cifratura o verificarla quando un pacchetto wireless viene ricevuto. Per i messaggi broadcast o multicast, viene usata la stessa procedura con la chiave di gruppo.

Sicurezza di Bluetooth

Bluetooth ha un raggio di azione considerevolmente più corto di 802.11, quindi non può essere attaccato dal parcheggio. Tuttavia, la sicurezza rimane un punto importante da discutere anche in questo caso. Immaginiamo, per esempio, che il computer di Alice abbia una tastiera Bluetooth senza fili. Supponiamo anche che Trudy si trovi nell'ufficio adiacente. In assenza di sicurezza, Trudy potrà leggere tutto quello che Alice scrive, incluse tutte le mail che spedisce. Potrebbe anche intercettare tutto quello che il computer di Alice invia alla stampante Bluetooth che si trova nei paraggi (per esempio, le e-mail in arrivo e i documenti confidenziali). Fortunatamente, Bluetooth ha uno schema di sicurezza complesso per cercare di sviare tutte le Trudy del mondo. Ne descriveremo le principali caratteristiche.

Bluetooth versione 2.1 e seguenti hanno quattro modalità di sicurezza, che vanno dal nulla alla cifratura completa dei dati e il controllo dell'integrità. Come nel caso di 802.11, se la sicurezza viene disabilitata (il comportamento predefinito) non c'è nessuna sicurezza. La maggior parte degli utenti mantiene la sicurezza disabilitata finché non viene scoperta un'importante fuga di dati, a quel punto la attiva. Nel mondo agricolo, questo approccio è noto come chiudere la stalla quando i buoi sono già scappati.

Bluetooth presenta soluzioni di sicurezza su più livelli. A livello fisico, il salto di frequenza (*frequency hopping*) fornisce un margine di sicurezza minimo. Visto che ogni dispositivo Bluetooth che si muove all'interno di una piconet deve essere informato della sequenza dei salti di frequenza, questa sequenza ovviamente non è segreta. La vera sicurezza inizia quando un nuovo dispositivo slave chiede un canale al master. Si suppone che i due dispositivi abbiano già effettuato uno scambio di chiave segreta condivisa. In alcuni casi questa è già preconfezionata in entrambi dispositivi (per esempio per gli auricolari venduti in abbinamento al telefono cellulare); in altri casi, un dispositivo (per esempio l'auricolare) ha una chiave già inserita al suo interno che l'utente deve inserire anche dentro l'altro dispositivo (per esempio il telefono cellulare) sotto forma di un numero decimale. Queste chiavi condivise sono dette **passkey**.[4] Sfortunatamente, le passkey sono

spesso fissate a “1234” o a un altro valore prevedibile e in tutti i casi sono quatto numeri decimali, permettendo solo 10^4 possibilità. Con l’accoppiamento semplice e sicuro di Bluetooth 2.1, i dispositivi scelgono un codice da un intervallo di sei cifre, che rende la passkey molto meno prevedibile ma ancora lontano dalla sicurezza.
Per stabilire un canale, master e slave effettuano il controllo per vedere se l’altro conosce la passkey. In caso affermativo, negoziano se il canale deve essere cifrato e se si deve effettuare il controllo di integrità o entrambe le cose. Quindi scelgono una chiave di sessione casuale a 128 bit, di cui alcuni bit possono essere resi pubblici. Lo scopo di questo indebolimento della chiave è di rendere il protocollo utilizzabile anche in quegli stati per cui la legge impone delle restrizioni all’esportazione o proibisce l’uso di chiavi più lunghe di quelle che le autorità governative riescono a forzare.

La cifratura usa uno stream cipher detto $\mathbf{E_0}$ il controllo di integrità usa **SAFER+**. Entrambi sono cifrari a blocco tradizionali con chiave simmetrica. SAFER+ fu inviato anche al concorso per AES, ma venne eliminato al primo round perché più lento degli altri candidati. Bluetooth fu finalizzato prima della decisione definitiva sul cifrario AES, altrimenti sarebbe stato scelto molto probabilmente Rijndael anche per Bluetooth. Il meccanismo di cifratura con lo stream cipher è mostrato nella Figura 8.14, in cui si vede come il testo in chiaro viene applicato in XOR al keystream per generare il testo cifrato. Sfortunatamente, E_0 stesso (come RC4) potrebbe avere delle debolezze fatali (Jakobsson e Wetzel, 2001). Anche se non è ancora stato forzato, al momento in cui scriviamo questo libro preoccupano parecchio (Biryukov et al. 2000) le sue similitudini con il cifrario A5/1, il cui spettacolare fallimento compromise tutto il traffico telefonico GSM. Spesso la gente si sorprende (incluso l’autore di questo libro) di come nel perenne gioco al gatto e al topo fra i crittografi e i criptoanalisti, i criptoanalisti siano così spesso la parte vincente. Un altro punto riguardante la sicurezza è dato dal fatto che Bluetooth autentica solo i dispositivi e non gli utenti, quindi il furto di un dispositivo Bluetooth può dare al ladro l’accesso ai dati finanziari, o comunque riservati, dell’utente. Bluetooth però implementa la sicurezza anche nei livelli superiori, quindi nel caso in cui ci sia una violazione a livello data link, rimane ancora un po’ di sicurezza, specialmente per le applicazioni che richiedono l’inserimento manuale dalla tastiera di un PIN per poter completare la transazione.

8.7 Protocolli di autenticazione

L'**autenticazione** è la tecnica usata dai processi per verificare che la loro controparte nella comunicazione sia veramente chi indica di essere e non un impostore. La verifica dell'identità di un processo remoto, per combattere gli intrusi attivi e in malafede, è un compito sorprendentemente difficile, che richiede dei protocolli crittografici complessi. In questo paragrafo studieremo alcuni dei molti protocolli di autenticazione usati per le reti insicure. Per inciso, alcune persone confondono l'autorizzazione con l'autenticazione. L'autenticazione riguarda il problema di assicurare l'identità del processo con cui si sta comunicando. L'autorizzazione si occupa, invece, di stabilire che cosa può fare un processo. Per esempio, consideriamo un processo client che contatta un file server e gli comunica: sono il processo di Scott e voglio cancellare il file *ricettario.old.* Il file server ha bisogno delle risposte a queste due domande:

1. la richiesta viene veramente da un processo di Scott (autenticazione)?
2. Scott è autorizzato a cancellare *ricettario.old* (autorizzazione)?

La richiesta può essere evasa soltanto dopo che entrambe le domande hanno ottenuto una risposta certa e affermativa. La prima domanda è quella veramente cruciale. Una volta che il file server sa con chi sta parlando, il controllo dell'autorizzazione consiste solamente in una ricerca dentro tabelle locali o in un database. Per questo motivo, in questo paragrafo ci concentreremo sull'autenticazione.

Il modello generale utilizzato da tutti i protocolli di autenticazione è il seguente: Alice comincia inviando un messaggio a Bob o a una terza persona considerata fidata, detta **KDC** (*key distribution center,* centro di distribuzione delle chiavi). In seguito è previsto lo scambio di molti altri messaggi, in varie direzioni, che possono essere intercettati, modificati o rinviati da parte di Trudy per ingannare (o comunque ostacolare) Alice e Bob. Quando il protocollo è stato completato, Alice però è sicura di essere in comunicazione con Bob e Bob è sicuro di essere in comunicazione con Alice. Inoltre, in molti protocolli, Alice e Bob stabiliscono anche una chiave segreta, detta **chiave di sessione** (*session key*), da usare per continuare la loro conversazione. Nella realtà per motivi di prestazioni tutto il traffico è cifrato con la crittografia a chiave simmetrica (di solito con AES o triplo DES), mentre la crittografia a chiave pubblica è usata largamente nei protocolli di autenticazione per stabilire la chiave di sessione.

Le ragioni per cui si usa una differente chiave scelta casualmente per ciascuna nuova connessione sono molteplici: minimizzare la quantità di traffico inviato usando le chiavi permanenti degli utenti (segrete oppure pubbliche), ridurre la quantità di testo cifrato che può cadere nelle mani di un intruso e minimizzare i rischi connessi alla situazione in cui un processo va in crash e il relativo *core dump*[5] cade nelle mani sbagliate. In questo caso, si spera che l'unica chiave presente nel dump sia la chiave di sessione. Tutte le chiavi permanenti dovrebbero essere già state attentamente azzerate dopo che la sessione è stata attivata.

Autenticazione basata su un segreto condiviso

Per il nostro primo protocollo di autenticazione, supporremo che Alice e Bob abbiano già una chiave segreta nota a entrambi, K_{AB}. La chiave è stata condivisa (per esempio) per telefono o di persona, ma comunque non sulla rete, che si suppone insicura. Questo protocollo si basa su un principio comune a molti altri protocolli di autenticazione: un soggetto invia un numero casuale all'altro, che trasforma il numero con un algoritmo particolare e quindi trasmette il risultato. Questo tipo di protocolli è detto **challenge-response** (*sfida e risposta*). La notazione che useremo per questo protocollo e per i successivi è:

1. A, B sono le identità di Alice e Bob
2. R_i sono le richieste (*challenge*), dove l'indice i identifica il richiedente
3. K_i sono le chiavi, dove i indica il possessore
4. K_s è la chiave di sessione.

La sequenza dei messaggi scambiati nel nostro primo protocollo di autenticazione a chiave condivisa è mostrata nella Figura 8.32. Nel messaggio 1, Alice invia a Bob la sua identità A usando una modalità che Bob è in grado di capire. Bob, ovviamente, non può sapere se il messaggio viene da Alice o da Trudy, quindi crea una richiesta per Alice nella forma di un numero casuale grande, R_B, che invia in chiaro nel messaggio 2. Alice procede, cifrando il messaggio servendosi della chiave condivisa con Bob e inviando il messaggio 3 con il testo cifrato $K_{AB}(R_B)$. Quando Bob riceve il messaggio sa immediatamente che proveniva da Alice, in quanto Trudy non conosce K_{AB} e quindi non può aver generato il messaggio 3. Visto che R_B è scelto in modo casuale da uno spazio molto grande (per esempio, possiamo pensare a un numero casuale di 128 bit) è molto improbabile che Trudy possa aver preso R_B e la sua risposta da una sessione precedente. È altresì molto improbabile che Trudy possa indovinare la risposta corretta alla richiesta contenuta nel messaggio 2.

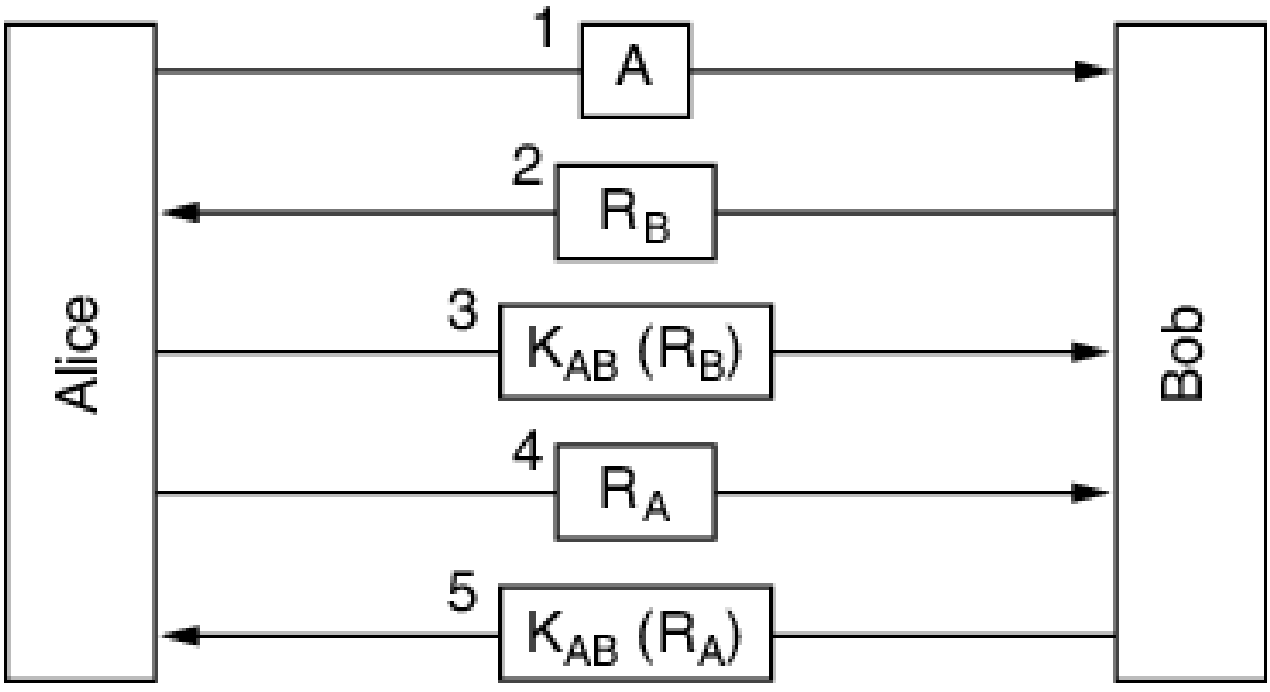


Figura 8.32 Autenticazione a due vie con protocollo challenge-response.

A questo punto Bob è sicuro di essere in comunicazione con Alice, ma Alice non è ancora sicura di niente. Per quanto ne sa Alice, Trudy potrebbe aver intercettato il messaggio 1 e aver inviato la risposta R_B. Magari Bob è morto la notte prima. Per scoprire con chi sta parlando, Alice sceglie un numero casuale R_A e lo invia a Bob in chiaro nel messaggio 4. Quando Bob risponde con $K_{AB}(R_A)$, Alice sa di essere veramente in comunicazione con Bob. A questo punto è possibile stabilire una chiave di sessione: Alice ne sceglie una, K_s, e la manda a Bob cifrata con K_{AB}.

Il protocollo della Figura 8.32 contiene cinque messaggi. Vediamo se riusciamo a eliminare qualche messaggio in modo intelligente. Un approccio è illustrato nella Figura 8.33, dove Alice inizia il protocollo challenge-response senza aspettare che sia Bob a farlo. Analogamente, Bob invia la sua richiesta mentre risponde a quella di Alice. L'intero protocollo si riduce quindi a tre messaggi, invece di cinque.

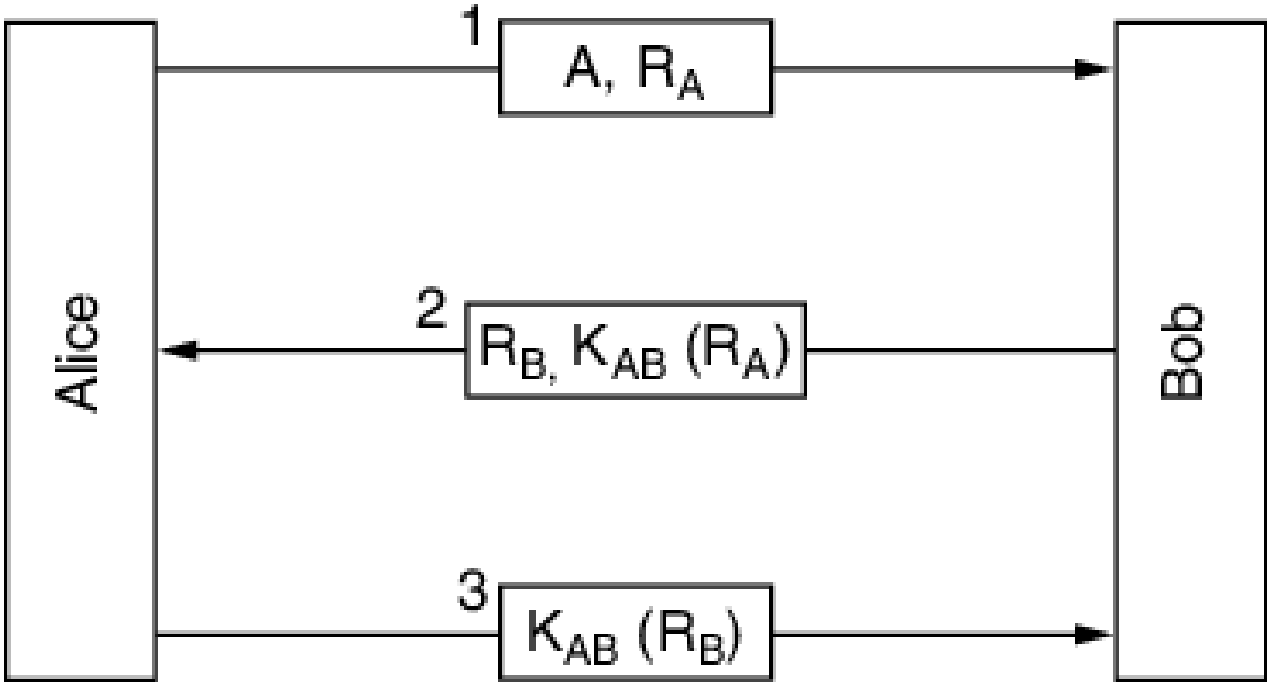


Figura 8.33 Un protocollo di autenticazione a due vie ridotto.

Il nuovo protocollo è migliore di quello originale? In un certo senso sì: è più corto. Sfortunatamente è anche sbagliato. In certe circostanze, Trudy riesce ad attaccare il protocollo usando una tattica nota come **attacco per riflessione** (*reflaction attack*). In particolare, Trudy può attaccare il protocollo se è possibile aprire più sessioni simultanee con Bob. Questa situazione si verifica, per esempio, se Bob è una banca pronta ad accettare molte connessioni simultanee dagli sportelli bancomat.

Un attacco per riflessione è mostrato nella Figura 8.34. Comincia con Trudy che finge di essere Alice e invia R_T, Bob risponde come al solito con una richiesta, R_B. Adesso Trudy è bloccata. Che cosa può fare, dato che non conosce $K_{AB}(R_B)$?

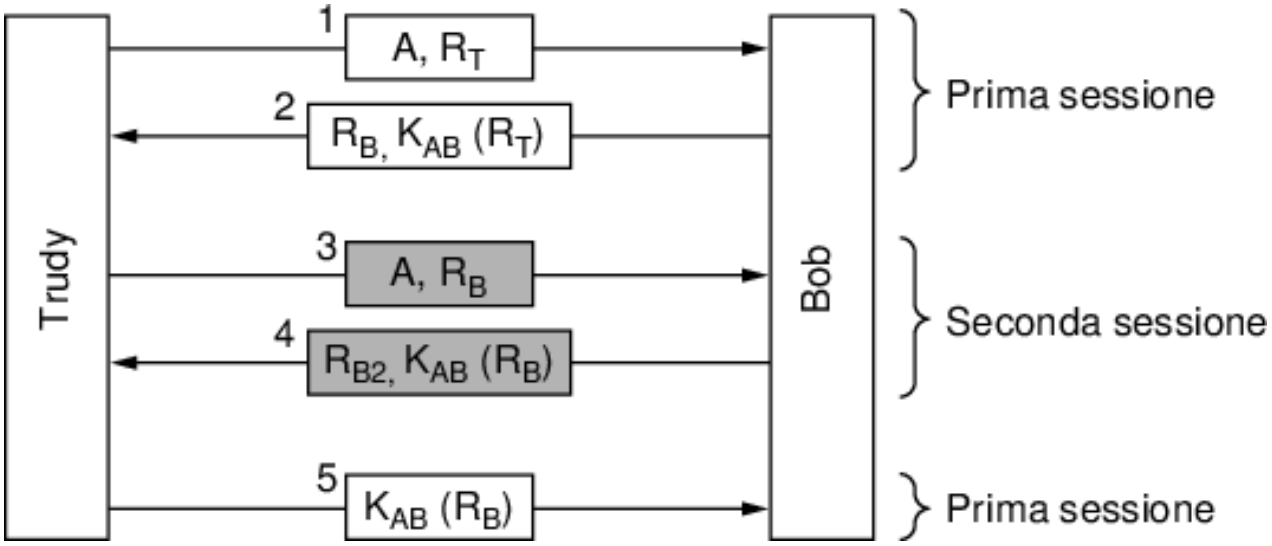


Figura 8.34 Attacco per riflessione.

Può aprire una seconda sessione con il messaggio 3, fornendo come richiesta la R_B presa dal messaggio 2. Bob cifra con calma la R_B e invia $K_{AB}(R_B)$ nel messaggio 4. Nella figura, i messaggi della seconda sessione sono in un rettangolo ombreggiato per evidenziarli. A questo punto Trudy ha l'informazione mancante e può completare la prima sessione e interrompere la seconda. Bob adesso è convinto che Trudy sia Alice, perciò quando questa domanda il suo estratto conto, Bob lo trasmette senza indugi. Quando Trudy chiede a Bob di trasferire tutti i soldi su un conto segreto in Svizzera, Bob eseguirà anche questo ordine senza esitazioni. La morale della storia è:

> *progettare un protocollo di autenticazione corretto è più difficile di quanto non sembri a prima vista.*

Le quattro regole generali seguenti sono spesso di aiuto.

1. Fare in modo che chi inizia l'autenticazione provi la sua identità prima che lo faccia chi risponde. Nel nostro caso, Bob invia delle informazioni importanti prima che Trudy abbia dato chiara evidenza della sua identità.
2. Fare sì che i due soggetti, cioè chi inizia l'autenticazione e chi risponde, usino due chiavi differenti per provare la loro identità, anche se ciò significa usare due chiavi condivise K_{AB} e K'_{AB}.
3. Far sì che i due soggetti utilizzino, per le richieste, numeri presi da insiemi diversi. Per esempio, chi inizia l'autenticazione può usare i numeri pari e chi risponde quelli dispari.
4. Rendere il protocollo resistente ad attacchi che coinvolgono una seconda sessione parallela, dove le informazioni ottenute da una sessione possono essere usate per l'altra.

Se anche una sola di queste regole non è rispettata, il protocollo può essere spesso forzato. Nel nostro caso vengono violate tutte e quattro le regole, con conseguenze disastrose.

Torniamo indietro e diamo uno sguardo più approfondito alla Figura 8.32. Siamo sicuri che il protocollo non sia soggetto ad attacchi per riflessione? La risposta è: dipende. La questione è delicata. Trudy era in grado di forzare il nostro protocollo con un attacco per riflessione, in quanto le era possibile aprire una seconda sessione con Bob per ingannarlo e quindi farlo rispondere alla

sua stessa domanda. Che cosa succederebbe nel caso in cui Alice fosse un computer in grado di accettare più sessioni e non una persona seduta davanti a un computer? Vediamo quello che Trudy può fare.

Per vedere come funziona l'attacco di Trudy, guardiamo la Figura 8.35. Alice comincia annunciando la sua identità nel messaggio 1. Trudy intercetta il messaggio e comincia la sua sessione con il messaggio 2, pretendendo di essere Bob. Di nuovo, abbiamo inserito i messaggi della sessione 2 in un rettangolo ombreggiato. Alice risponde al messaggio 2 dicendo: tu dici di essere Bob? Provalo con il messaggio 3. A questo punto Trudy è bloccata, perché non riesce a provare di essere Bob.

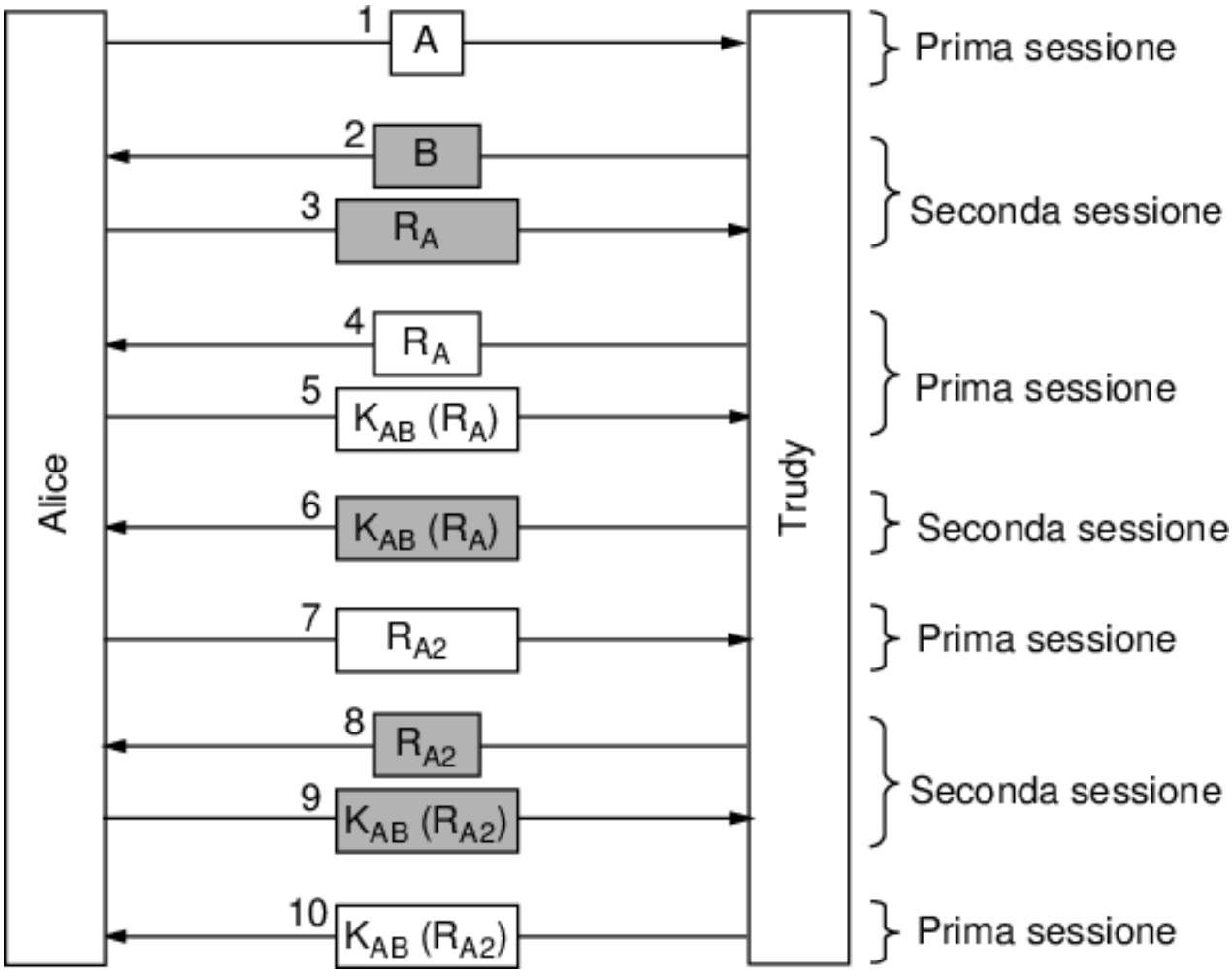


Figura 8.35 Attacco per riflessione al protocollo della **Figura 8.32** .

Che cosa fa Trudy adesso? Ritorna alla prima sessione, dove è arrivato il suo turno di inviare la richiesta, quindi invia R_A che ha ricevuto nel messaggio 3. Alice risponde gentilmente con il messaggio 5, il che fornisce a Trudy l'informazione che le serviva per mandare il messaggio 6 nella sessione 2. A questo punto Trudy ha praticamente vinto, perché ha risposto con successo alla

richiesta di Alice nella sessione 2. Trudy può interrompere la sessione 1, inviare un qualunque vecchio numero per il resto della sessione 2, e avrà una sessione autenticata con Alice nella sessione 2.

Trudy vuole accanirsi nel suo attacco. Al posto di inviare un qualunque vecchio numero per completare la sessione 2, Trudy aspetta che Alice trasmetta il messaggio 7, la richiesta di Alice per la sessione 1. Ovviamente Trudy non sa come rispondere, quindi usa di nuovo l'attacco per riflessione: risponde con R_{A2} nel messaggio 8. Alice cifra R_{A2} nel messaggio 9. Trudy quindi torna alla sessione 1 e trasmette ad Alice il messaggio 10 con il numero richiesto, semplicemente copiandolo da quello inviato da Alice nel messaggio 9. A questo punto Trudy ha due sessioni autenticate con Alice.

Il risultato di questo attacco è in qualche modo differente da quello contro il protocollo a tre messaggi della Figura 8.34. Questa volta Trudy ha due connessioni autenticate con Alice, mentre nell'esempio precedente aveva una connessione autenticata con Bob. Se avessimo applicato le regole generali per i protocolli di autenticazione discusse in precedenza, questo attacco sarebbe stato fermato. Una discussione dettagliata di questo tipo di attacchi e di come combatterli è data da Bird et al., (1993). In quella trattazione viene anche mostrato come si possono sistematicamente costruire dei protocolli che sono corretti in modo dimostrabile. Il più semplice di tali protocolli è tuttavia un po' complicato, perciò adesso discuteremo una classe differente di protocolli più semplici ma ugualmente funzionanti.

Il nuovo protocollo di autenticazione è mostrato nella Figura 8.36. (Bird et al., 1993). Utilizza un HMAC del tipo che abbiamo visto studiando l'IPsec. Alice inizia inviando a Bob nel messaggio 1 un nonce R_A. Bob risponde scegliendo il suo nonce R_B, che invia ad Alice con un HMAC. L'HMAC è formato costruendo una struttura dati che consiste nel nonce di Alice, quello di Bob, le loro identità e la chiave segreta condivisa K_{AB}; per ottenere l'HMAC si calcola l'hash di questa struttura, per esempio usando SHA-1. Quando riceve il messaggio 2, Alice conosce R_A (che aveva scelto lei), R_B (che le è arrivato come testo in chiaro), le due identità, e la chiave segreta K_{AB} (che conosceva fin dall'inizio). Alice può quindi procedere indipendentemente al calcolo dell'HMAC e confrontare il risultato con quello ricevuto nel messaggio 2. Se i due valori coincidono, Alice sa di parlare con Bob, perché Trudy non conosce K_{AB} e quindi non riesce a calcolare l'HMAC corretto. Alice risponde a Bob con un HMAC che contiene due nonce.

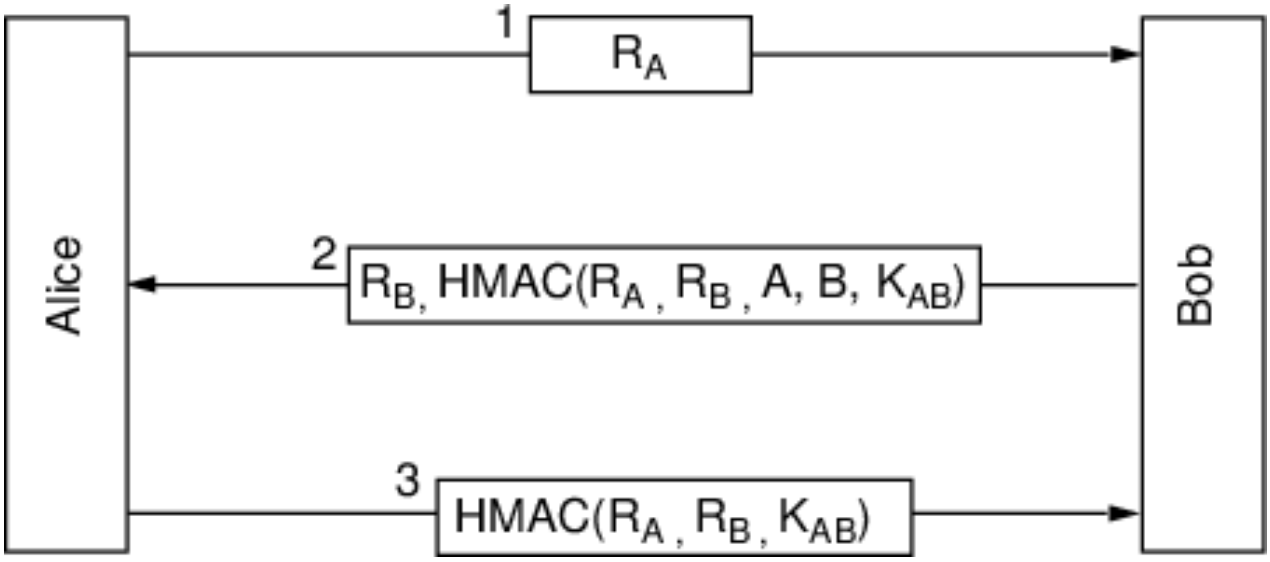


Figura 8.36 Autenticazione con HMAC.

Trudy riesce in qualche modo a forzare il protocollo? No, perché non riesce a indurre nessuna delle due parti a cifrare o calcolare l'hash di un valore a sua scelta, come invece accadeva nella Figura 8.34 e nella Figura 8.35. Entrambi gli HMAC includono dei valori scelti dalla destinazione, cioè qualcosa che Trudy non può controllare. Gli HMAC non sono l'unico modo per sfruttare questa idea. Uno schema alternativo, spesso usato invece del calcolo dell'HMAC per una serie di oggetti, consiste nel cifrare gli oggetti in sequenza con la modalità cipher block chaining.

Come stabilire una chiave condivisa: lo scambio di chiave di Diffie-Hellman

Finora abbiamo supposto che Alice e Bob condividano una chiave segreta. Supponiamo che questo non sia vero (in quanto non esiste ancora una PKI universalmente accettata per firmare e distribuire i certificati). Come si può stabilire una chiave condivisa? Alice potrebbe chiamare Bob al telefono e dettargli la chiave, ma probabilmente la telefonata comincerebbe con Bob che chiede: come faccio a sapere che tu sei Alice e non Trudy? Potrebbero allora organizzare un incontro, dove ognuno porta il passaporto, la patente di guida e tre carte di credito dei circuiti maggiori. Essendo persone impegnate, può darsi che non riescano a fissare una data accettabile per mesi a venire. Fortunatamente e incredibilmente, esiste un metodo per stabilire una chiave segreta condivisa fra perfetti sconosciuti; questo metodo funziona senza segreti, anche se Trudy ascolta e registra attentamente ogni messaggio scambiato.

Il protocollo che permette a degli sconosciuti di scambiarsi una chiave segreta è chiamato **scambio di chiave di Diffie-Hellman** (*Diffie-Hellman key exchange*) (Diffie e Hellman, 1976). Questo protocollo richiede come prima azione che Alice e Bob si mettano d'accordo su due numeri grandi, n e g, dove n è un numero primo, $(n - 1)/2$ è pure primo e g soddisfa certe condizioni particolari. Questi numeri possono essere pubblici, quindi uno dei due può scegliere la coppia di

numeri e poi semplicemente la comunica all'altro. A questo punto Alice sceglie un numero grande (per esempio di 512 bit), x, e lo tiene segreto. Analogamente Bob sceglie un numero grande e segreto, y.

Alice inizia il protocollo di scambio della chiave inviando a Bob un messaggio che contiene (n, g, g^x mod n), come mostrato nella Figura 8.37. Bob risponde inviando ad Alice un messaggio che contiene g^y mod n. Alice quindi prende il numero che gli ha inviato Bob e lo eleva alla potenza x, modulo n, cioè calcola $(g^y \bmod n)^x \bmod n$. Bob esegue un calcolo simile per ottenere $(g^x \bmod n)^y \bmod n$. Per le regole dell'aritmetica modulare, entrambe le espressioni valgono $g^{xy} \bmod n$. Il gioco è fatto: ora Alice e Bob hanno una chiave segreta condivisa: $g^{xy} \bmod n$.

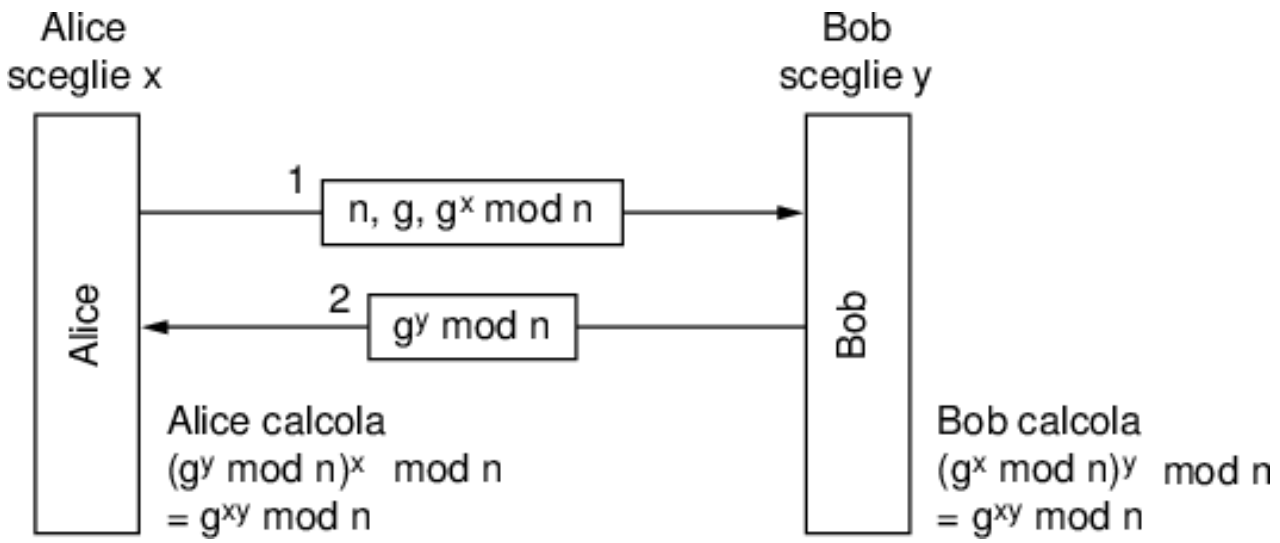


Figura 8.37 Scambio di chiavi Diffie-Hellman.

Trudy, ovviamente, ha visto entrambi i messaggi, quindi conosce g e n dal messaggio 1. Se riuscisse a calcolare x e y, potrebbe conoscere la chiave segreta. Il punto è che, dato solamente g^x mod n, non riesce a trovare x. Non sono noti algoritmi che riescano a calcolare in modo efficiente i logaritmi discreti con modulo un numero primo molto grande.

Per rendere l'esempio più concreto, useremo i valori (completamente non realistici) di $n = 47$ e $g = 3$. Alice sceglie $x = 8$ e Bob sceglie $y = 10$. Entrambi questi numeri sono tenuti segreti. Il messaggio di Alice per Bob è (47, 3, 28), in quanto $3^8 \bmod 47$ è 28. Il messaggio di Bob per Alice è (17). Alice calcola $17^8 \bmod 47$, che vale 4. Bob calcola $28^{10} \bmod 47$, che vale 4. Alice e Bob hanno determinato in modo indipendente la chiave segreta, che è 4. Trudy deve risolvere l'equazione $3^x \bmod 47 = 28$: per dei numeri piccoli come questi il problema può essere risolto con una ricerca esaustiva, ma non certo quando tutti i numeri in gioco sono lunghi centinaia di bit. Tutti gli algoritmi oggi conosciuti impiegano un tempo troppo lungo, anche usando dei computer a parallelismo massiccio. Nonostante la sua eleganza, l'algoritmo di Deffie-Hellman ha un problema: quando Bob riceve la tripla (47, 3, 28), come fa a sapere che viene da Alice e non da Trudy? Non c'è modo per Bob di saperlo. Sfortunatamente, Trudy può utilizzare questo fatto per

ingannare Alice e Bob, come illustrato nella Figura 8.38. Mentre Alice e Bob scelgono rispettivamente x e y, Trudy sceglie il suo numero casuale z. Alice invia il messaggio 1 per Bob, Trudy lo intercetta e manda il messaggio 2 a Bob, usando i valori corretti per g *e* n (che comunque sono pubblici), ma con il valore z al posto di x. Manda anche indietro il messaggio 3 ad Alice. Successivamente Bob invia ad Alice il messaggio 4, che Trudy intercetta di nuovo e trattiene.

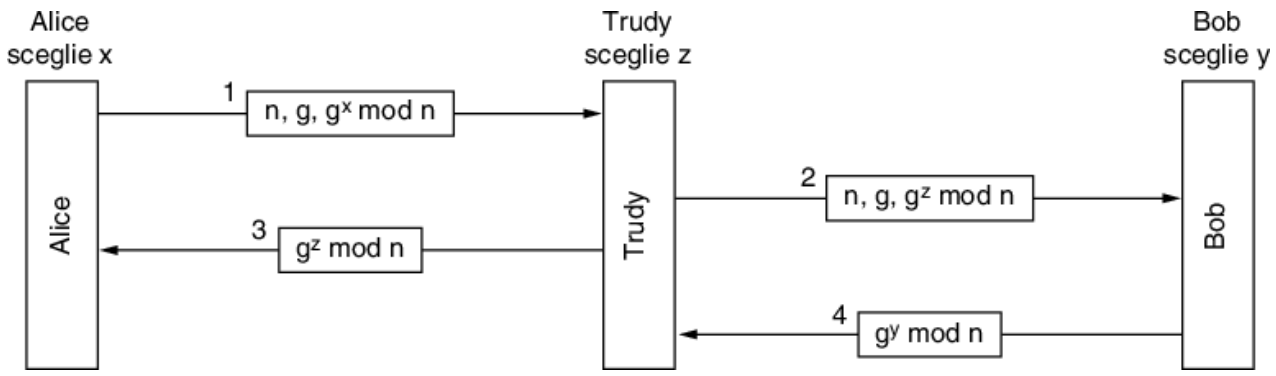


Figura 8.38 L'attacco man in the middle.

A questo punto tutte le parti coinvolte eseguono i loro calcoli con l'aritmetica modulare. Alice calcola la chiave segreta g^{xy} mod n. Questo calcolo viene fatto anche da Trudy (per i messaggi con Alice). Bob calcola g^{yz} mod n, un calcolo che fa anche Trudy (lo userà per trasmettere con Bob). Alice pensa di parlare con Bob e quindi stabilisce una chiave di sessione con Trudy; lo stesso vale per Bob. Ogni messaggio che Alice invia sulla sessione cifrata è catturato da Trudy, memorizzato, modificato se necessario, ed eventualmente passato a Bob. Lo stesso vale per la direzione opposta. Trudy vede tutto e può modificare tutti i messaggi se lo vuole, mentre Bob e Alice hanno la falsa impressione di avere un canale sicuro di comunicazione. Per questo motivo l'attacco è noto come **man in the middle** (*attacco dell'uomo nel mezzo*). Viene anche chiamato attacco **bucket brigade** (*attacco del passaggio dei secchi*), in quanto ricorda vagamente il vecchio metodo usato dai vigili del fuoco volontari, che si passavano l'un l'altro i secchi d'acqua dal camion cisterna fino all'incendio.

Autenticazione usando un centro di distribuzione delle chiavi

Siamo quasi riusciti a far funzionare il metodo per stabilire un segreto condiviso con uno sconosciuto, ma ancora non ci siamo. Ripensandoci meglio, dopotutto forse questo algoritmo non serve a nulla: per parlare con n persone in questo modo servono n chiavi. Per le persone molto ricercate la gestione delle chiavi diventerebbe un vero problema, specialmente se ogni chiave va memorizzata in una smart card distinta. Un approccio differente si ottiene introducendo un centro di distribuzione delle chiavi o **KDC** (*key distribution center*, centro di distribuzione delle chiavi). In

questo modello, ogni utente ha una singola chiave condivisa con il KDC e la gestione dell'autenticazione e della chiave di sessione sono gestite dal KDC. Nella Figura 8.39 è mostrato il più semplice fra i protocolli noti per l'autenticazione KDC con due soggetti e un KDC fidato.

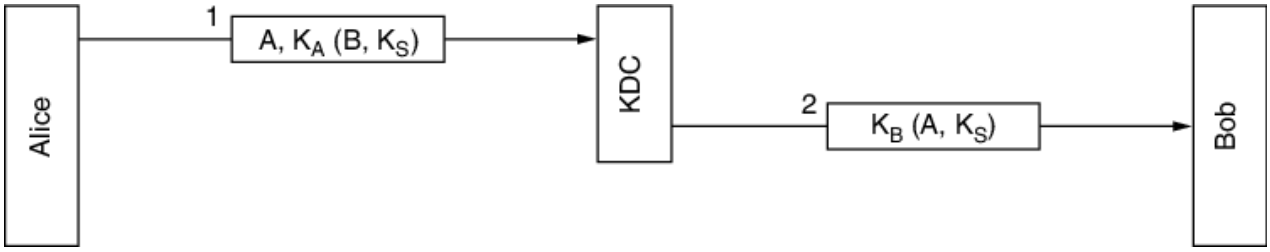


Figura 8.39 Primo tentativo per realizzare un protocollo di autenticazione usando un KDC.

L'idea dietro al protocollo è la seguente: Alice sceglie una chiave di sessione K_s e indica al KDC che vuole comunicare con Bob usando K_s. Questo messaggio viene cifrato usando la chiave segreta che Alice condivide (solamente) con KDC, K_A. Il KDC decifra il messaggio, estrae l'identità di Bob e la chiave di sessione, poi costruisce un nuovo messaggio contenente l'identità di Alice e la chiave di sessione, infine spedisce il nuovo messaggio a Bob. Questa cifratura è fatta con K_B, la chiave segreta che Bob condivide con KDC: quando Bob decifrerà il messaggio, saprà che Alice desidera parlare con lui e saprà anche la chiave che Alice vuole usare.

In questo scenario l'autenticazione si ottiene gratuitamente. Il KDC sa che il messaggio 1 proviene necessariamente da Alice, infatti nessun altro avrebbe potuto cifrarlo con la chiave segreta di Alice; analogamente Bob sa che il messaggio 2 proviene per forza dal KDC (di cui ha fiducia), in quanto nessun altro conosce la sua chiave segreta.

Sfortunatamente, questo protocollo ha un difetto grave. Trudy ha bisogno di soldi, quindi s'inventa qualche servizio legittimo che può fornire ad Alice, presenta una buona offerta e ottiene il lavoro. Dopo aver finito il lavoro, Trudy chiede educatamente ad Alice di essere pagata con un bonifico bancario. Alice stabilisce perciò una chiave di sessione con il suo banchiere, Bob, a cui invia il messaggio di richiesta di un bonifico a favore di Trudy. Nel frattempo, Trudy è tornata alle sue vecchie abitudini e intercetta i messaggi sulla rete. Copia sia il messaggio 2 della Figura 8.39, sia la successiva richiesta di bonifico. Più tardi Trudy manda nuovamente entrambi i messaggi a Bob. Bob li riceve e pensa: Alice deve aver assunto di nuovo Trudy, chiaramente lavora bene. Bob procede di nuovo al trasferimento della stessa somma di denaro dal conto di Alice a quello di Trudy. Dopo aver ricevuto la cinquantesima copia di messaggi uguali, Bob va a cercare Trudy per offrirle un bel prestito per espandere quello che è ormai un business di sicuro successo. Questo problema si chiama **attacco di ripetizione** (*replay attack*).

Esistono svariate soluzioni per bloccare gli attacchi di ripetizione. Il primo consiste nell'aggiungere a ciascun messaggio un timestamp; i messaggi scaduti vengono immediatamente scartati dal destinatario. Il problema di questo approccio è che su una rete gli orologi non sono mai

esattamente sincronizzati, quindi si deve prevedere un intervallo di tempo in cui il timestamp rimane valido. Trudy può ripetere il messaggio durante questo intervallo e farla franca.

La seconda soluzione consiste nel mettere un nonce in ciascun messaggio. Ogni soggetto deve ricordarsi tutti i nonce precedenti e scartare ogni messaggio che contiene un nonce già usato. I nonce vanno ricordati per sempre, altrimenti Trudy può ripetere un messaggio vecchio di 5 anni. Inoltre, se qualche macchina riparte dopo un malfunzionamento e dimentica la lista dei nonce, può diventare vulnerabile ad attacchi di ripetizione. Timestamp e nonce si possono combinare per limitare l'intervallo di tempo in cui ricordare i nonce, ma chiaramente questo complica ulteriormente il protocollo.

Un approccio più sofisticato alla mutua autenticazione consiste nell'usare un protocollo challenge-response a più vie. Un noto esempio è il **protocollo di autenticazione di Needham-Schroeder** (Needham e Schroeder, 1978), di cui la Figura 8.40 mostra una variante.

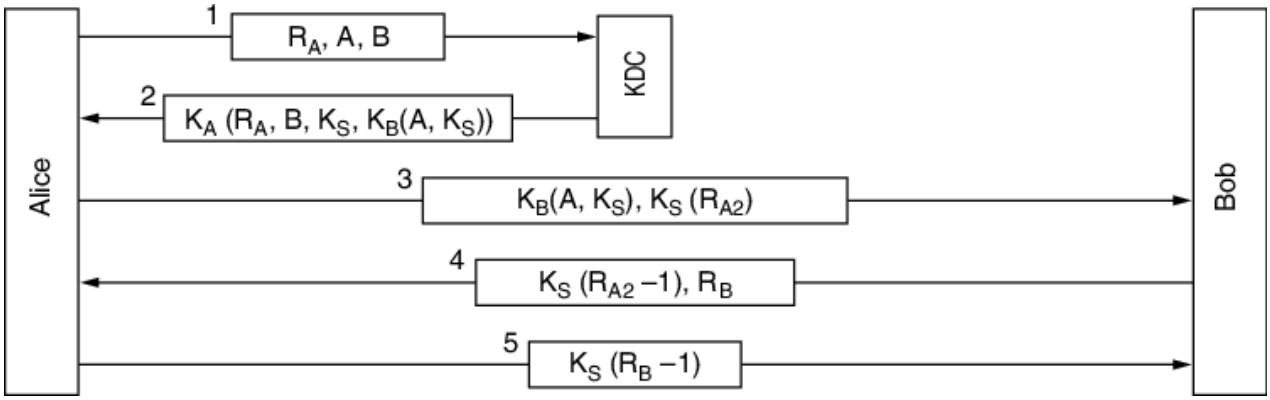


Figura 8.40 Il protocollo di autenticazione Needham-Schroeder.

Il protocollo comincia con Alice che indica al KDC che vuole parlare con Bob. Questo messaggio contiene un numero casuale grande, R_A, usato come nonce. Il KDC manda indietro il messaggio 2 che contiene il numero casuale di Alice, una chiave di sessione e un ticket che Alice può inviare a Bob. Il numero casuale R_A serve per rassicurare Alice che il messaggio 2 è fresco e non è una ripetizione. L'identità di Bob è inclusa nel messaggio, per prevenire un attacco in cui Trudy tenta di rimpiazzare B con la propria identità all'interno del messaggio 1: se nel messaggio non fosse presente l'identità di Bob, il KDC eseguirebbe la cifratura del ticket (alla fine del messaggio 2) usando K_T al posto di K_B. Il ticket cifrato con K_B è incluso nel messaggio cifrato per evitare che Trudy possa facilmente sostituirlo durante la trasmissione ad Alice. Successivamente, Alice invia a Bob un ticket e un nuovo numero casuale, R_{A2}, cifrato con la chiave di sessione K_s. Nel messaggio 4, Bob risponde con $K_S(R_{A2} - 1)$ per dimostrare ad Alice che sta parlando veramente con Bob. La trasmissione di $K_S(R_{A2})$ non è accettabile, perché Trudy potrebbe rubare questa informazione dal messaggio 3. Dopo aver ricevuto il messaggio 4, Alice si convince che sta veramente parlando con Bob e che non è stato fatto nessun attacco di ripetizione: infatti è stata Alice a generare R_{A2} solo alcuni millisecondi prima. Il messaggio 5 serve per convincere Bob dell'identità di Alice e

dell'assenza di attacchi di ripetizione. Entrambi i soggetti della comunicazione hanno generato una richiesta di tipo challenge-response e ciò elimina la possibilità che sia avvenuto un qualunque tipo di attacco di ripetizione.

Questo protocollo può sembrare molto solido, però ha una piccola debolezza. Infatti Trudy sarà in grado di stabilire una sessione con Bob se riesce a ottenere una vecchia chiave di sessione nel testo in chiaro. Per fare questo le basterà ripetere il messaggio 3 che corrisponde alla chiave compromessa e Bob sarà convinto di essere in comunicazione con Alice (Denning e Sacco, 1981). Questa volta Trudy riesce a prosciugare il conto corrente di Alice senza dover eseguire neanche una volta un servizio legittimo. Successivamente Needham e Schroeder hanno pubblicato un protocollo che corregge questo tipo di problemi (Needham e Schroeder, 1987). Nello stesso numero di quella rivista, Otway e Rees (1987) hanno pubblicato un altro articolo che risolve il problema in un modo più conciso. La Figura 8.41 mostra una versione leggermente modificata del protocollo Otway-Rees.

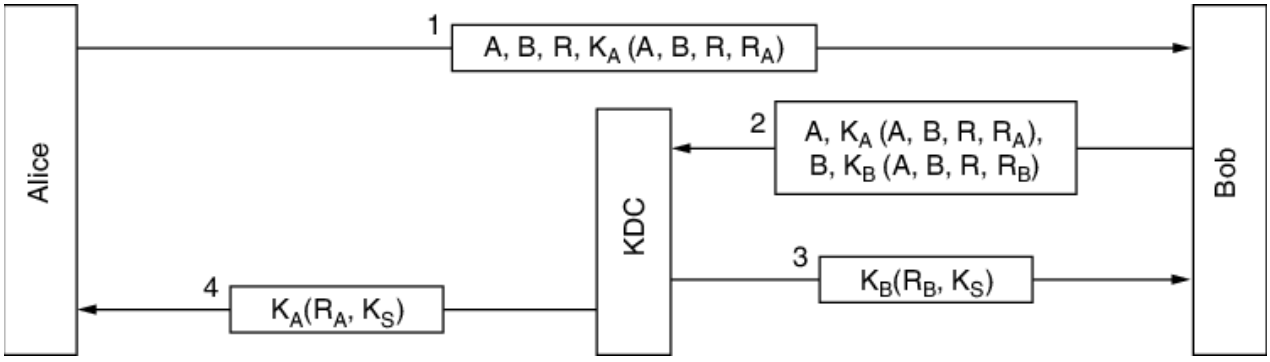


Figura 8.41 Il protocollo di autenticazione Otway-Rees (leggermente semplificato).

Nel protocollo di Otway-Rees, Alice inizia generando una coppia di numeri casuali: R (che verrà usata come identificatore comune) e R_A (che Alice invierà come richiesta a Bob). Quando Bob riceve questo messaggio, ne costruisce uno nuovo usando la parte cifrata del messaggio di Alice e una parte analoga costruita da lui stesso. Entrambe le parti, cifrate con K_A e K_B, identificano Alice e Bob, contengono l'identificatore comune e contengono un challenge.

Il KDC controlla che R abbia lo stesso valore in entrambe le parti. Potrebbe anche non essere vero, se Trudy ha manomesso R nel messaggio 1 oppure alterato parte del messaggio 2. Se le due R coincidono, il KDC considera valido il messaggio di richiesta di Bob, perciò genera una chiave di sessione e la cifra due volte: una per Alice e una per Bob. Ogni messaggio contiene il numero casuale del ricevitore, come prova che il messaggio è stato generato dal KDC e non da Trudy. A questo punto Alice e Bob sono in possesso della stessa chiave di sessione e possono cominciare a comunicare. Al primo scambio di dati, ognuno dei due può vedere che l'altro è in possesso di un'identica copia di K_s, quindi l'autenticazione è completata.

Autenticazione con Kerberos

Un protocollo di autenticazione usato in molti sistemi reali (incluso Windows 2000) è **Kerberos**, basato su una variante di Needham-Schroeder. Il suo nome deriva dalla mitologia greca, dove Cerbero (Kerberos in inglese) è il cane a tre teste messo a guardia dell'ingresso dell'Ade (presumibilmente per tenere fuori le persone non desiderate). Kerberos fu progettato al M.I.T. per fornire un accesso sicuro alle risorse di rete agli utenti delle workstation. La principale differenza rispetto a Needham-Schroeder sta nel fatto che Kerberos suppone che tutti gli orologi siano sincronizzati in modo abbastanza preciso. Il protocollo ha subito uno sviluppo con diverse iterazioni. V5 è quella più ampiamente usata nell'industria e definita nell'RFC 4120. La versione precedente, V4, alla fine fu ritirata dopo che furono riscontrati difetti gravi (Yu et al., 2004). V5 migliora V4 con molti piccoli cambiamenti al protocollo e alcune caratteristiche, come il fatto che non si affidi più all'ormai datato DES. Per maggiori informazioni consultate Neuman e Ts'o (1994).

Kerberos utilizza tre server in aggiunta ad Alice (la workstation client):

1. Authentication Server (*server di autenticazione*, AS): verifica l'identità degli utenti
2. Ticket-Granting Server (*server di emissione dei ticket*, TGS): emette ticket che provano l'identità di chi li possiede

- Il server Bob: è quello che compie il lavoro che Alice ha richiesto.

AS è simile a un KDC, nel senso che condivide una password segreta con ogni utente. Il TGS ha il compito di emettere dei ticket (biglietti) da usare per convincere i server veri e propri del fatto che il loro possessore è realmente chi dichiara di essere. Alice inizia una sessione da una qualunque workstation pubblica, inserendo il suo nome. La workstation invia il nome di Alice in chiaro all'AS, come mostrato nella Figura 8.42. La risposta sarà formata da una chiave di sessione e da un ticket per il TGS, $K_{TGS}(A, K_{S,} t)$. La chiave di sessione è cifrata usando la chiave segreta di Alice, quindi solo Alice può decifrarla. Solo quando il messaggio 2 arriva la workstation chiede la password di Alice (e non prima). La password viene usata per generare K_A per decifrare il messaggio 2 e ottenere la chiave di sessione.

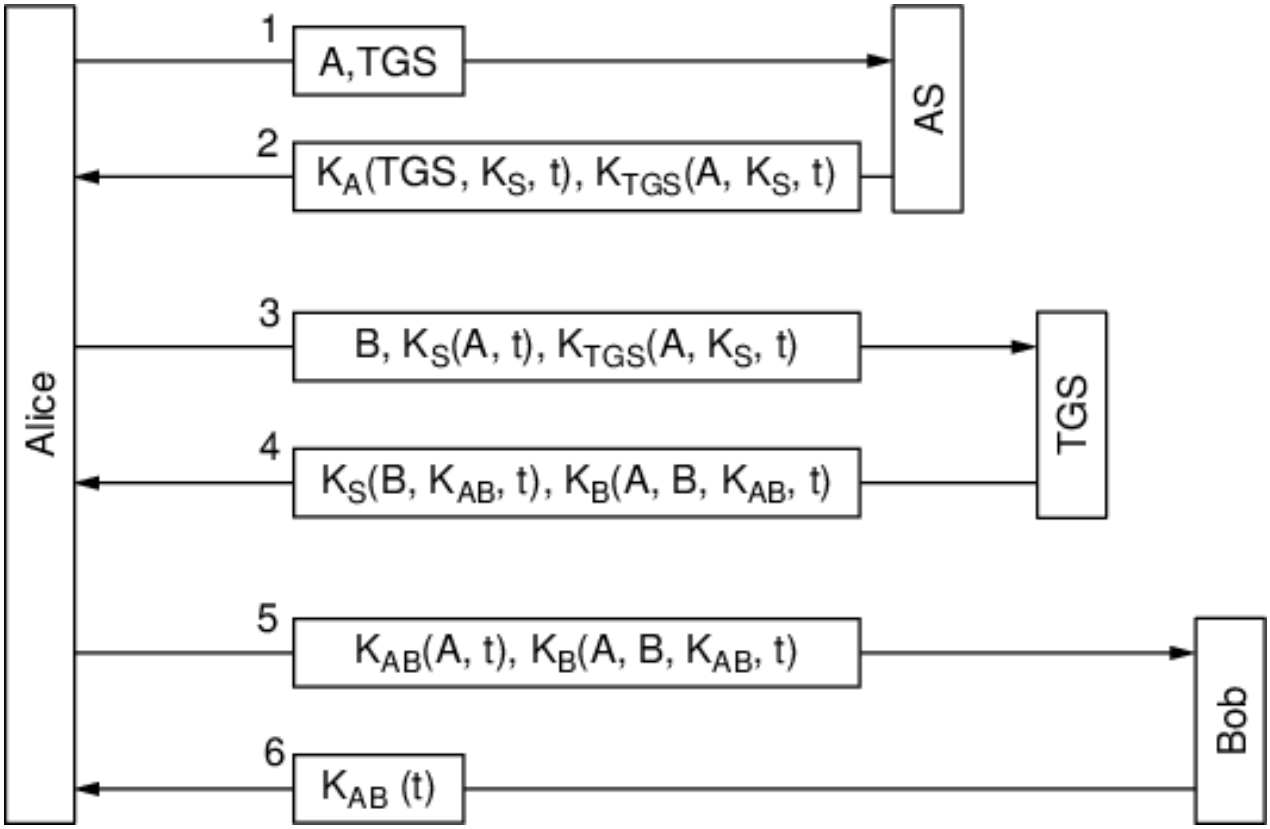


Figura 8.42 Funzionamento di Kerberos V5.

A questo punto la workstation sovrascrive la password di Alice, assicurandosi così che sia rimasta all'interno della workstation stessa solamente per pochi millisecondi. Se Trudy tenta di fare il login con il nome di Alice, la password che inserirà sarà sbagliata e la workstation potrà accorgersene in quanto la parte standard del messaggio 2 non sarà corretta.

Dopo il login, Alice può chiedere alla workstation di contattare il file server Bob. La workstation invia il messaggio 3 al TGS chiedendogli un ticket da usare con Bob. L'elemento chiave di questa richiesta è $K_{TGS}(A, K_S, t)$, che è cifrato con la chiave segreta del TGS ed è usato come prova che il mittente è Alice. Il TGS risponde al messaggio 4 creando una chiave di sessione K_{AB}, che Alice potrà usare con Bob. Ne vengono spedite indietro due versioni: la prima è cifrata solamente con K_s, così che Alice la possa leggere, mentre la seconda è cifrata con la chiave di Bob K_B, così che Bob riesca a leggerla.

Trudy può copiare il messaggio 3 e provare a usarlo di nuovo, ma sarà bloccata dal timestamp cifrato t, che è inviato insieme al messaggio. Trudy non può rimpiazzare il timestamp con uno più recente, in quanto non conosce K_s, la chiave di sessione che Alice usa per parlare con il TGS. Se Trudy rimanda il messaggio 3 molto velocemente, l'unica cosa che otterrebbe sarebbe un'altra copia del messaggio 4, che non è riuscita a decifrare la prima volta e che quindi non sarà neppure in grado di decifrare questa volta.

Adesso Alice può inviare K_{AB} a Bob e stabilire una sessione con lui; anche questo scambio è accompagnato da un timestamp. Per Alice la risposta è una prova del fatto che sta effettivamente parlando con Bob e non con Trudy.

Dopo una serie di scambi, Alice riesce a parlare con Bob usando la protezione fornita da K_{AB}. Se successivamente Alice decide di parlare con un altro server, Carol, deve semplicemente inviare il messaggio 3 al TGS indicando *C* al posto *B*. Il TGS risponderà subito con un ticket cifrato con K_c, che Alice può inviare a Carol e che Carol accetterà come prova della provenienza da Alice.

Lo scopo di tutto questo lavoro è rendere Alice in grado di accedere ai server su tutta la rete in modo sicuro senza che la sua password debba mai essere trasmessa. Infatti, la password è rimasta sulla sua workstation, e solo per pochi millisecondi. Notiamo tuttavia che ogni server si prende carico delle proprie autorizzazioni. Quando Alice presenta a Bob il suo ticket, questo serve solo a dimostrare a Bob l'identità di Alice. Stabilire che cosa Alice sia autorizzata a fare è un compito che viene lasciato a Bob. I progettisti di Kerberos non si aspettavano certo che tutto il mondo potesse fidarsi di un unico server di autenticazione, per questo motivo hanno previsto la possibilità di avere più **realm** (*reami*), ognuno con il proprio AS e TGS. Per ottenere un ticket per un reame remoto, Alice deve chiedere al suo TGS un ticket accettato dal TGS nel reame remoto. Se il TGS remoto si è registrato con il TGS locale (allo stesso modo in cui lo fanno i server locali), il TGS locale darà ad Alice un ticket valido per il TGS remoto. Alice può quindi operare nel reame remoto, per esempio ottenendo ticket per i server di quel reame. Notare che per far sì che due soggetti situati in reami diversi possano operare uno con l'altro, è necessario che abbiano fiducia ognuno nel TGS dell'altro.

Autenticazione con crittografia a chiave pubblica

La mutua autenticazione si può eseguire anche usando la crittografia a chiave pubblica. Per cominciare, Alice ha bisogno della chiave pubblica di Bob. Se esiste una PKI con un directory server che fornisce i certificati per le chiavi pubbliche, Alice gli può chiedere il certificato di Bob, come mostrato nella Figura 8.43 per il messaggio 1. La risposta, il messaggio 2, è un certificato X.509 che contiene la chiave pubblica di Bob. Alice verifica la correttezza della firma, quindi manda a Bob un messaggio che contiene la sua identità e un nonce.

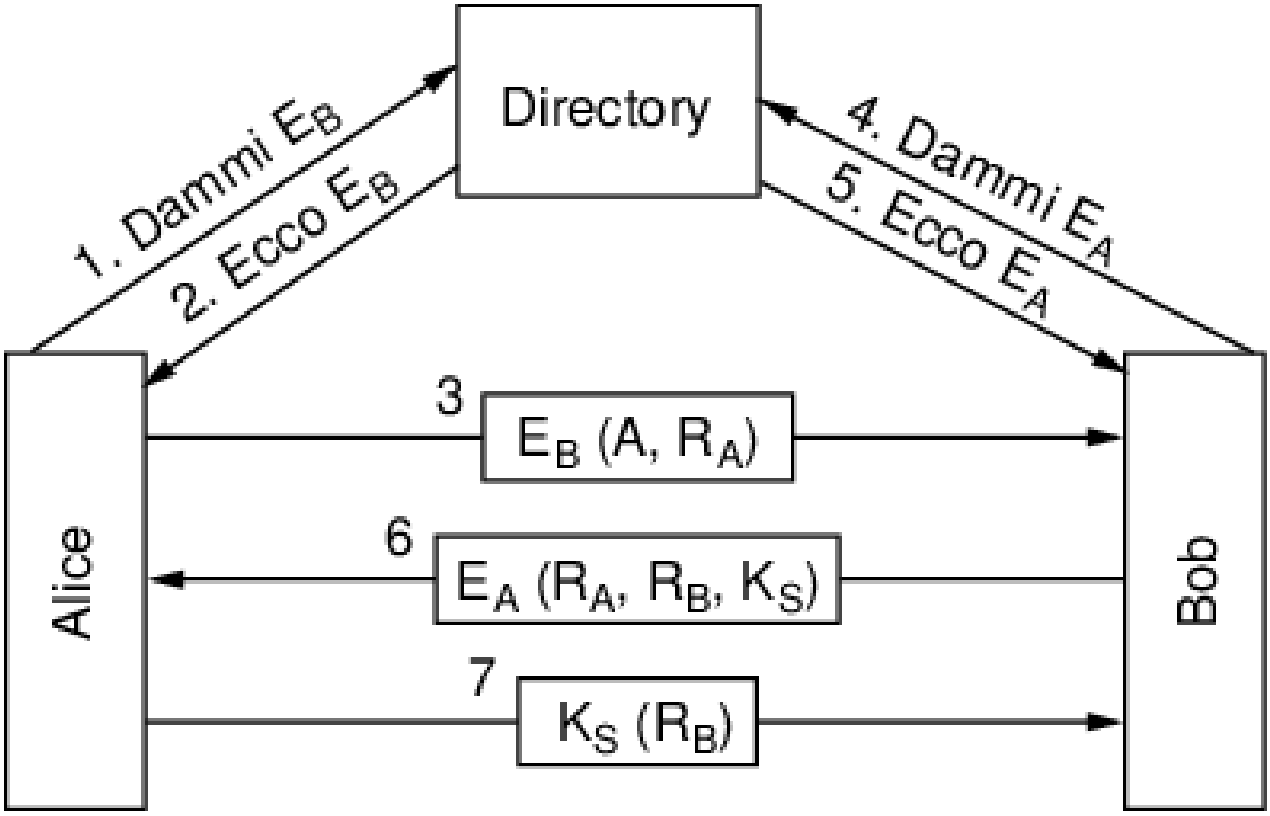


Figura 8.43 Mutua autenticazione con crittografia a chiave pubblica.

Quando Bob riceve questo messaggio non sa ancora se proviene da Alice o da Trudy, ma prosegue chiedendo al directory server la chiave pubblica di Alice (messaggio 4) che riceve rapidamente (messaggio 5). Quindi Bob manda ad Alice il messaggio 6 che contiene l'R_A di Alice, il suo nonce R_B e una proposta per la chiave di sessione, K_s. Alice riceve il messaggio 6 e lo decifra usando la sua chiave privata. Vede che contiene R_A, il che la rende contenta. Il messaggio deve quindi venire da Bob, visto che Trudy non ha modo di determinare R_A. Inoltre deve essere un messaggio fresco e non una ripetizione, in quanto Alice ha appena inviato R_A a Bob. Alice conferma di essere d'accordo sulla chiave di sessione con il messaggio 7. Quando Bob riceve il messaggio e vede che contiene l'R_B cifrato con la chiave di sessione che ha appena generato, sa che Alice ha ricevuto correttamente il messaggio 6 e ha verificato R_A. Ora Bob è contento.

Che cosa può fare Trudy per cercare di forzare questo protocollo? Può fabbricare il messaggio 3 e cercare d'ingannare Bob facendogli spedire un messaggio ad Alice. Alice, però, vedrà un R_A che non ha spedito e quindi non procederà ulteriormente. Trudy non può falsificare il messaggio 7 di ritorno a Bob, perché non conosce né R_B né K_s, e non può saperli senza avere la chiave privata di Alice. Quindi Trudy non riesce a fare niente.

8.8 Sicurezza della posta elettronica

Un messaggio e-mail scambiato fra due siti molto distanti fra loro transita tipicamente attraverso dozzine di macchine, prima di giungere a destinazione. Ognuna di queste macchine può leggere il messaggio e registrarlo per usi futuri. In pratica la privacy non esiste, al contrario di quanto molti pensano. Tuttavia, molti vorrebbero poter inviare delle e-mail leggibili solo dal destinatario legittimo e da nessun altro: neppure il loro capo o il governo. Questo desiderio ha stimolato persone e gruppi ad applicare alla posta elettronica i principi crittografici che abbiamo studiato in precedenza per ottenere e-mail sicure. Nei seguenti paragrafi, studieremo un sistema di e-mail sicura molto usato nella pratica: PGP. Faremo anche menzione di un altro sistema, S/MIME. Per ulteriori informazioni sulle e-mail sicure si faccia riferimento a Kaufman et al. (2002) e Schneier (1995).

PGP – pretty good privacy

Il nostro primo esempio, **PGP** *(pretty good privacy,* privacy decisamente buona) è essenzialmente il frutto del lavoro di una sola persona, Phil Zimmermann (1995a, 1995b). Zimmermann è uno strenuo difensore della privacy e il suo motto è: "Se la privacy è messa fuori legge, allora solo i fuori legge avranno la privacy". PGP è un pacchetto completo per la sicurezza della posta elettronica, rilasciato nel 1991, che fornisce privacy, autenticazione, firme digitali e compressione; il tutto in una forma semplice da usare. L'intero pacchetto, con i codici sorgente, è distribuito gratuitamente su Internet. Oggi PGP è largamente usato per le sue doti di qualità, prezzo (nullo) e immediata disponibilità per le piattaforme Unix, Linux, Windows e Mac OS.

PGP usa un cifrario a blocchi per i dati chiamato **IDEA** (*international data encryption algorithm*, algoritmo internazionale per la cifratura dei dati), con una chiave a 128 bit. IDEA è stato sviluppato in Svizzera, quando DES era considerato compromesso e AES non era ancora stato inventato. Concettualmente, IDEA è simile a DES e AES: lavora mescolando i bit in una serie di iterazioni, però i dettagli delle funzioni di mescolamento sono diversi da quelli di DES e AES. La gestione delle chiavi si appoggia a RSA mentre l'integrità dei dati è fornita da MD5, che sono argomenti già discussi. PGP è stato circondato da controversie sin dalla sua nascita (Levy, 1993). Zimmermann non aveva fatto niente per impedire che altre persone pubblicassero PGP su Internet, dove tutto il mondo poteva averne accesso, perciò il governo degli Stati Uniti accusò Zimmermann di aver violato le leggi che proibivano l'esportazione di armi. L'inchiesta su Zimmermann da parte del governo USA durò 5 anni, ma alla fine fu chiusa, probabilmente per due motivi, il primo è che Zimmermann non aveva messo direttamente PGP su Internet, quindi il suo avvocato dichiarò che Zimmermann non aveva mai esportato niente (rimane anche da discutere il fatto che creare un sito Web costituisca un'esportazione). Il secondo è che il governo degli USA si rese conto che vincere il processo voleva dire convincere una giuria che un sito Web che fornisce il download di un programma per la privacy è vietato dalla legge che proibisce il traffico d'armi e l'esportazione di materiale di guerra come, per esempio, carri armati, sottomarini, aerei militari e armi nucleari. Infine, diversi anni di pubblicità negativa hanno probabilmente avuto una certa influenza.

Per inciso, le regole sull'esportazione sono quanto meno bizzarre, per non dire altro. Mettere del codice su un sito Web era considerato illegale dal governo, che quindi molestò Zimmermann per circa 5 anni. D'altra parte, quando qualcuno pubblicò un libro con il sorgente C completo di PGP (con un font grande e il checksum su ogni pagina, in modo da facilitare la scansione) e poi ha esportato il libro, la cosa era perfettamente accettabile per il governo perché i libri non sono considerati armi. La spada è più potente della penna, almeno per lo Zio Sam.

Un altro problema che PGP dovette affrontare, era legato alla violazione dei brevetti. L'azienda che deteneva il brevetto per RSA, la RSA Security Inc., dichiarò che l'uso dell'algoritmo RSA in PGP violava il suo brevetto. Questo fu risolto a partire dalla versione 2.6 di PGP. Nei primi tempi ci furono anche dei problemi con un altro algoritmo crittografico brevettato, IDEA.

Gruppi e singoli individui si sono cimentati in modifiche di PGP, vista la sua natura open source, e ciò ha prodotto una quantità di modifiche e versioni. Alcune di queste modifiche erano mirate ad aggirare la legge sul traffico di armi, altre a evitare l'uso di algoritmi brevettati, altre ancora tentavano di far diventare PGP un prodotto commerciale non più open source. Nel frattempo le leggi sulle armi sono state leggermente liberalizzate (altrimenti i prodotti che utilizzano AES non sarebbero esportabili dagli USA) e il brevetto di RSA è scaduto nel settembre 2000. Tuttavia, come conseguenza delle prime polemiche, adesso sono in circolazione (con nomi diversi) più versioni di PGP incompatibili tra loro. La discussione che segue è focalizzata sul PGP classico, nella sua versione più semplice e antica. Un'altra versione molto diffusa, Open PGP, è descritta nell'RFC 2440.; un'altra ancora è GNU privacy guard (*guardiano della privacy*).

PGP utilizza di proposito algoritmi crittografici esistenti, al posto di inventarne dei nuovi. Gli algoritmi utilizzati, infatti, hanno superato esaustive ricerche da parte di esperti del campo e non sono stati influenzati da nessuna agenzia governativa che poteva avere interesse a indebolirli. Per quelle persone che tendono a non fidarsi del governo questa proprietà è considerata molto positivamente.

PGP supporta la compressione del testo, la segretezza, le firme digitali e contiene funzioni evolute per la gestione delle chiavi. Stranamente, però, PGP non ha funzioni per la gestione delle e-mail: si tratta più che altro di un preprocessore, che prende in input un testo in chiaro e produce in output un testo cifrato in base64. Naturalmente, questo output può essere inviato per e-mail. Come ultimo passaggio, alcune implementazioni di PGP prevedono la chiamata a un agente che invia il messaggio.

Per vedere come funziona PGP, consideriamo l'esempio della Figura 8.44. Alice vuole inviare a Bob in modo sicuro un messaggio in chiaro firmato, *P.* Alice e Bob hanno entrambi una chiave RSA privata (D_x) e una pubblica (E_x). Ipotizziamo che ognuno dei due conosca la chiave pubblica dell'altro (vedremo fra poco la gestione delle chiavi con PGP).

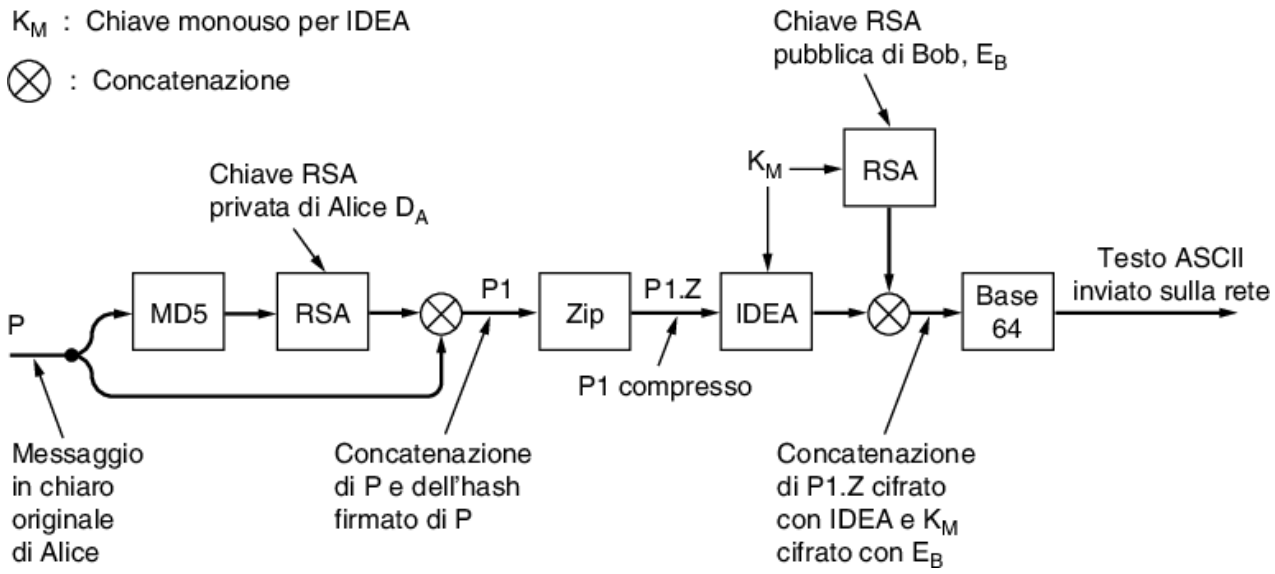


Figura 8.44 Invio di un messaggio con PGP.

Alice inizia lanciando il programma PGP sul suo computer. PGP calcola innanzitutto l'hash del messaggio *P* usando MD5 e poi cifra il risultato usando la sua chiave RSA privata D_A. Quando Bob riceve il messaggio, può decifrare l'hash usando la chiave pubblica di Alice e quindi verificarne la correttezza. La robustezza di MD5 garantisce che una terza persona (per esempio Trudy), capace di intercettare l'hash e decifrarlo con la chiave pubblica di Alice, non sarà in grado di produrre un messaggio con lo stesso hash MD5 (è un problema computazionalmente troppo difficile).

L'hash cifrato e il messaggio originale sono concatenati in un singolo messaggio *P1* e compressi usando il programma ZIP, che usa l'algoritmo di Ziv-Lempel (Ziv e Lempel, 1977). Chiamiamo l'output di questo passo *P1.Z*.

Successivamente PGP chiede ad Alice un numero casuale in input. Il contenuto del messaggio e la velocità di battitura sono usati per generare una chiave di messaggio IDEA a 128 bit, K_M (viene chiamata chiave di sessione in letteratura, ma in realtà il nome non è appropriato, in quanto non esiste una sessione). K_M serve a cifrare *P1.Z* con IDEA in modalità cipher feedback; inoltre K_M è cifrata con la chiave pubblica di Bob, E_B. Ora queste due componenti vengono concatenate e convertite in base64, come discusso nel paragrafo sul MIME del Capitolo 7. Il messaggio risultante contiene solo lettere, cifre e i simboli +, / e =, quindi si può inserire in un messaggio conforme all'RFC 822 per inviarlo a destinazione senza temere alterazioni durante il transito.

Quando Bob riceve il messaggio, esegue a rovescio la codifica base64 e decifra la chiave IDEA usando la sua chiave privata; con essa decifra il messaggio e ottiene *P1.Z*. Quindi passa alla decompressione, separa il testo in chiaro dall'hash cifrato e decifra l'hash usando la chiave pubblica di Alice. Adesso Bob controlla se l'hash del testo in chiaro coincide con il valore del suo calcolo MD5: in caso affermativo, sa con certezza che *P* è il messaggio originale inviato da Alice.

È utile notare che RSA si usa solamente in due momenti: per cifrare l'hash MD5 a 128 bit e per cifrare la chiave IDEA a 128 bit. Anche se RSA è lento, deve cifrare solamente 256 bit, quindi una quantità limitata di dati. D'altra parte, tutti i 256 bit di dati cifrati con RSA sono altamente casuali, quindi Trudy dovrebbe fare una gran quantità di lavoro già per stabilire se ha indovinato correttamente una chiave. La cifratura della maggior parte dei dati è fatta usando IDEA, che è diversi ordini di grandezza più veloce di RSA. Quindi PGP fornisce sicurezza, compressione e firma digitale e lo fa in un modo molto più efficiente dello schema illustrato nella Figura 8.19.

PGP supporta quattro diverse lunghezze di chiavi RSA, così l'utente può scegliere quella che ritiene più appropriata:

1. *Casual* (384 bit): può essere forzata facilmente al giorno d'oggi
2. *Commercial* (512 bit): può essere forzata dalle ben note organizzazioni governative
3. *Military* (1.024 bit): non può essere forzata da nessuno sulla terra
4. *Alien* (2.048 bit): non può essere forzata da nessuno neanche in altri pianeti.

Visto che RSA è usato solo per due piccoli calcoli, tutti dovrebbero usare sempre il livello "alien".

Il formato di un messaggio PGP classico è mostrato nella Figura 8.45, ma si usano numerosi altri formati. Il messaggio PGP ha tre parti, che contengono la chiave IDEA, la firma e il messaggio vero e proprio. La parte con la chiave contiene anche un identificatore della chiave, visto che gli utenti possono usare più chiavi pubbliche.

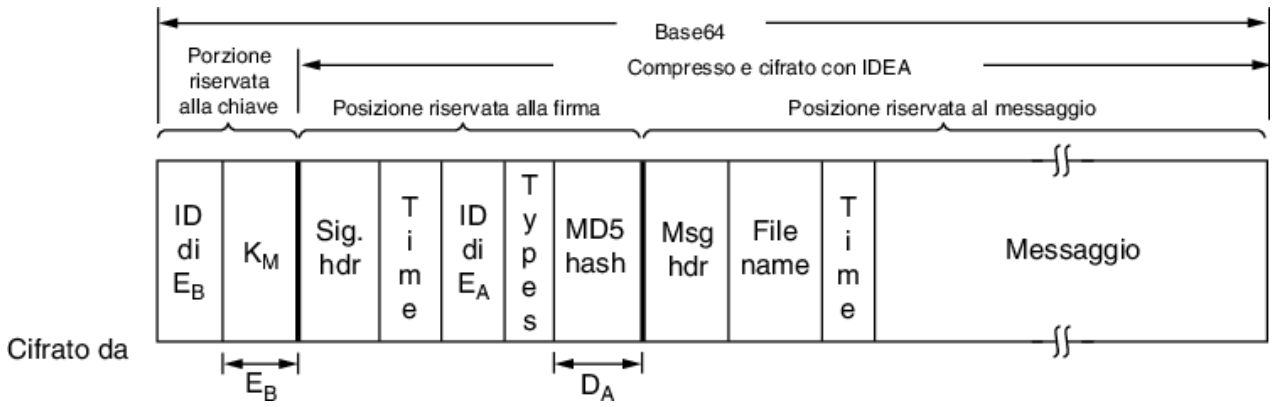


Figura 8.45 Un messaggio PGP.

La parte del messaggio PGP che contiene la firma è composta da un'intestazione (che in questo contesto non ci interessa), da un timestamp, dall'identificatore della chiave pubblica usata dal mittente (che serve per decifrare la firma hash), da alcune informazioni che identificano gli algoritmi usati (per consentire l'uso di MD6 e RSA2, quando saranno inventati), e infine dall'hash cifrato.

La parte con il messaggio è formata da intestazione, nome di default per il file (nel caso che il destinatario voglia scriverlo su disco), timestamp relativo alla creazione del messaggio, e infine dal messaggio vero e proprio.

La gestione delle chiavi ha ricevuto una gran quantità di attenzioni durante lo sviluppo di PGP, poiché rappresenta il tallone di Achille di tutti i sistemi di sicurezza. Ogni utente gestisce localmente due strutture dati: un insieme di chiavi private e un insieme di chiavi pubbliche. Il **private key ring** (*portachiavi privato*) contiene una o più coppie personali di chiavi private e pubbliche. Per ogni utente sono supportate coppie multiple di chiavi, per consentire agli utenti di cambiare le loro chiavi pubbliche con cadenza periodica o quando pensano che una chiave sia compromessa, senza che questo annulli la validità dei messaggi in preparazione o in transito. A ogni coppia di chiavi è associato un identificatore, così il mittente può comunicare al destinatario quale chiave pubblica ha usato per cifrare il messaggio. Gli identificatori di messaggio sono formati dai 64 bit di ordine più basso della chiave pubblica e gli utenti hanno la responsabilità di evitare possibili conflitti fra gli identificatori delle loro chiavi pubbliche. Le chiavi private sono memorizzate su disco in forma cifrata utilizzando una password (di lunghezza arbitraria), per proteggerle contro eventuali furti.

Il **public key ring** (*portachiavi pubblico*) contiene le chiavi pubbliche dei corrispondenti dell'utente; queste sono necessarie per cifrare le chiavi associate a ogni messaggio. Ogni elemento di questa struttura dati contiene non solo la chiave pubblica ma anche il suo identificatore di 64 bit e un indicatore della fiducia che l'utente ha nella chiave.

Il problema che stiamo affrontando qui è il seguente: supponiamo che le chiavi pubbliche sono gestite con delle bacheche condivise. Trudy, per leggere le e-mail segrete di Bob, potrebbe attaccare la bacheca e rimpiazzare la chiave pubblica di Bob con una di sua scelta. Quando più tardi Alice legge la chiave che crede appartenere a Bob, Trudy può far scattare un attacco di tipo "man in the middle" con Bob.

Per prevenire questo tipo di attacchi o comunque minimizzarne le conseguenze, Alice deve sapere quanto si può fidare di quella cosa chiamata "chiave di Bob" che si ritrova nel suo insieme di chiavi. Per esempio la fiducia sarebbe al massimo valore nel caso in cui Bob avesse personalmente passato ad Alice un floppy contenente la sua chiave. È questo approccio di gestione delle chiavi decentralizzato e controllato dagli utenti che rende PGP essenzialmente diverso dagli schemi di PKI centralizzati.

In ogni modo, in alcune occasioni le chiavi si ottengono con una richiesta a un server fidato. Dopo la standardizzazione di X.509, PGP supporta anche questo tipo di certificati, oltre al meccanismo tradizionale dell'insieme delle chiavi. Tutte le versioni correnti di PGP supportano X.509.

S/MIME

Il successivo approccio di IETF alla sicurezza delle e-mail, chiamato S/MIME (*Secure/MIME*, MIME sicuro) è descritto negli RFC da 2632 a 2643. S/MIME fornisce autenticazione, integrità dei dati, segretezza e non ripudio; inoltre è un sistema flessibile che supporta diversi algoritmi crittografici. Visto il suo nome, non dovrebbe sorprendere la notizia che S/MIME si integra bene con MIME, il che gli permette di proteggere tutti i tipi di messaggi; sono state introdotte molte intestazioni MIME per gestire, per esempio, le firme digitali.

S/MIME non ha una struttura rigida di certificati che parte da una singola root, che era uno dei problemi politici che ha condannato un sistema precedente chiamato PEM (*privacy enhanced mail*, mail con sicurezza avanzata). Al contrario gli utenti possono avere molte trust anchor. Per essere considerato valido un certificato deve semplicemente avere un percorso che lo fa risalire a un punto di riferimento di cui l'utente si fida. S/MIME usa gli algoritmi e i protocolli standard che abbiamo discusso finora, perciò non esamineremo altri dettagli, per i quali rimandiamo agli RFC citati prima.

8.9 Sicurezza del Web

Abbiamo appena studiato due aree importanti in cui la sicurezza è necessaria: le comunicazioni e l'e-mail. Possiamo pensare a quelle due aree come al primo e all'antipasto. È arrivato il momento del piatto forte: la sicurezza del Web. Il Web è il posto dove ormai le Trudy si radunano sempre più spesso per fare il loro "sporco" lavoro. Nei paragrafi seguenti vedremo alcuni problemi legati alla sicurezza del Web. La sicurezza del Web può essere divisa in tre filoni principali. Primo, come si possono gestire in modo sicuro i nomi degli oggetti e delle risorse? Secondo, come si può stabilire una connessione autenticata? Terzo, che cosa succede quando un sito Web invia a un client un programma eseguibile? Prima di affrontare questi problemi, vedremo alcune delle possibili minacce alla sicurezza.

Minacce alla sicurezza

I giornali riportano con cadenza quasi settimanale i problemi legati alla sicurezza del Web; la situazione è decisamente grave. Vediamo alcuni esempi di fatti già accaduti. Innanzitutto, la home page di diverse organizzazioni è stata attaccata e rimpiazzata con una scelta dai cracker. Per inciso, la stampa popolare chiama "hacker" le persone che penetrano nei sistemi informatici senza autorizzazione. Molti programmatori, invece, riservano questo termine per indicare i colleghi veramente in gamba. Preferiamo quindi usare il termine "cracker". Fra i siti che hanno subito l'attacco della home page citiamo: Yahoo, U.S. Army, CIA, NASA e il New York Times. In molti casi, i cracker hanno solamente aggiunto qualche scritta divertente e il danno è stato poi riparato con qualche ora di lavoro. Ci sono stati anche casi più seri. Diversi siti sono stati bloccati da attacchi di tipo denial of service, nei quali il cracker sommerge di traffico il sito in modo che non riesca più a rispondere alle richieste legittime. Spesso questi attacchi vengono lanciati da una gran quantità di macchine che il cracker ha già attaccato (attacchi DDoS). Ormai questo tipo di attacchi è così comune che non fa più notizia, però può costare all'attaccato diverse migliaia di dollari di perdite causate dall'interruzione del servizio. Nel 1999, un cracker svedese entrò nel sito Hotmail di Microsoft e creò un sito replicato che permetteva a chiunque di digitare il nome di un utente Hotmail e leggere tutta la posta corrente e archiviata di quell'utente.

In un altro caso, un cracker russo di 19 anni chiamato Maxim, entrò in un sito di e-commerce e rubò 300.000 numeri di carte di credito. A questo punto contattò i gestori del sito minacciandoli di pubblicare su Internet tutti i numeri di carte di credito se non gli avessero pagato 100.000 dollari. Il pagamento non fu fatto e il cracker pubblicò i numeri delle carte di credito, infliggendo un grave danno a migliaia di vittime innocenti.

Una storia differente è quella di uno studente californiano di 23 anni che inviò per e-mail un comunicato a un'agenzia di stampa, dove dichiarava falsamente che la Emulex Corporation stava per pubblicare delle forti perdite trimestrali e che il C.E.O.[6] avrebbe rassegnato immediatamente le dimissioni. Nel giro di poche ore le azioni della compagnia scesero del 60%, causando una perdita per gli azionisti di oltre 2 miliardi di dollari. Lo studente guadagnò un quarto di milione di dollari

vendendo le azioni appena prima dell'annuncio. Anche se questo non è un canonico attacco a un sito Web, è chiaro che mettere degli annunci di questo tipo sulla home page di una qualunque grande corporation sortirebbe degli effetti simili.
Potremmo (sfortunatamente) andare avanti con queste notizie per molte pagine. A questo punto vogliamo però discutere alcune delle questioni tecniche legate alla sicurezza del Web. Per maggiori informazioni sui problemi di sicurezza di qualunque tipo si rimanda a Anderson (2008a); Stuttard e Punto (2007) e Schneier (2004). Una ricerca su Internet può essere utile per trovare un vasto numero di casi specifici.

Sicurezza nell'attribuzione dei nomi

Cominciamo con qualcosa di molto semplice: Alice vuole visitare il sito Web di Bob. Per fare questo digita l'URL di Bob nel browser e, dopo alcuni secondi, appare una pagina Web. È veramente la pagina di Bob? Forse sì e forse no. Trudy potrebbe essere in azione con qualcuno dei suoi vecchi trucchi. Per esempio, Trudy potrebbe intercettare ed esaminare tutti i pacchetti in uscita da Alice. Quando intercetta una richiesta HTTP *GET* indirizzata al sito di Bob, Trudy potrebbe andare a prendere la pagina di Bob, modificarla a piacimento e poi inviare ad Alice la versione alterata. Alice non si accorgerebbe di niente. Peggio ancora, Trudy potrebbe alterare i prezzi del sito di e-commerce di Bob in modo da renderli molto convenienti, quindi Alice viene ingannata e indotta a inviare il suo numero di carta di credito per comprare merce da Bob. Uno svantaggio di questo classico attacco "man in the middle" è che Trudy deve poter intercettare tutto il traffico in uscita da Alice, ma anche modificare il traffico in ingresso. In pratica, Trudy deve intercettare la linea telefonica di Alice o quella di Bob, visto che queste operazioni sono decisamente difficili su una dorsale in fibra. Intercettare le linee telefoniche è possibile, ma richiede diverso lavoro e si dà il caso che Trudy, oltre che intelligente, è anche pigra. Inoltre esistono dei metodi più semplici per ingannare Alice.

DNS spoofing

Un modo per Trudy può essere di compromettere il sistema DNS o forse solo la sua cache nell'ISP di Alice e sostituire l'indirizzo IP di Bob (cioè 36.1.2.3) con il suo (diciamo, 42.9.9.9). In questo modo si realizza l'attacco illustrato nella Figura 8.46(a). La sequenza dei passi è la seguente: (1) Alice chiede al DNS l'indirizzo IP di Bob, (2) lo ottiene, (3) chiede a Bob la sua home page, (4) la ottiene. Dopo che Trudy ha modificato il record di Bob nel DNS in modo da contenere l'indirizzo di Trudy otteniamo la situazione della Figura 8.46(b). In questo caso, quando Alice cerca l'indirizzo IP di Bob ottiene invece quello di Trudy, quindi tutto il traffico che dovrebbe andare a Bob va verso Trudy, che può far partire un attacco "man in the middle" senza doversi scomodare a intercettare la linea telefonica. Le è bastato penetrare nel server DNS e cambiare un record, un compito molto più semplice.

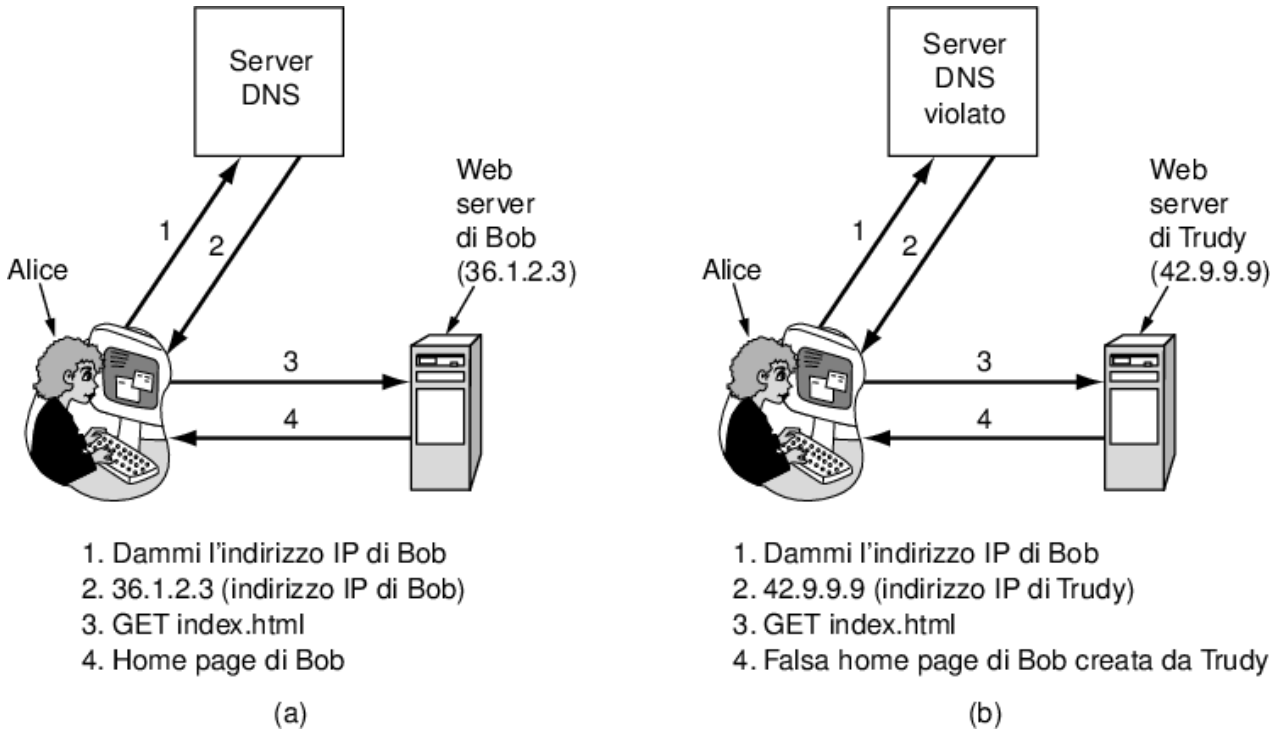


Figura 8.46 (a) Situazione normale. (b) Un attacco basato sulla violazione del DNS e modifica del record di Bob.

Come fa Trudy a ingannare il DNS? In realtà è abbastanza semplice. Riassumendo brevemente, Trudy può ingannare il DNS dell'ISP di Alice inviandogli una query per cercare l'indirizzo di Bob. Sfortunatamente, visto che DNS usa UDP, il server DNS non ha modo di sapere chi fornisce la risposta alla query. Trudy utilizza questa opportunità per falsificare la risposta alla query e quindi inserire un indirizzo IP falso nella cache del server DNS. Per semplicità assumiamo che l'ISP di Alice non abbia inizialmente informazioni sull'IP del sito Web di Bob, *bob.com;* in caso contrario Trudy può aspettare che questa informazione scada per timeout e quindi riprovare (oppure può usare degli altri trucchi). Trudy comincia inviando all'ISP di Alice la richiesta dell'indirizzo IP di *bob.com.* Visto che non ci sono informazioni per questo nome DNS, la richiesta viene passata al server di primo livello per il dominio *com.* Invece, quello che succede è che Trudy batte in velocità il server *com,* inviando una risposta falsa che dice: "*bob.com* è 42.9.9.9", dove l'indirizzo IP è quello di Trudy. Se la falsa risposta arriva all'ISP di Alice prima di quella vera, entrerà nella cache e causerà il rifiuto della risposta corretta, che sarà scartata come non desiderata visto che la query è stata già soddisfatta. Ingannare un server DNS per installare un indirizzo IP falso è detto **DNS spoofing** (*ingannare il DNS*). Una cache che contiene un indirizzo IP intenzionalmente falsificato, come in questo esempio, è detta **poisoned cache** (*cache avvelenata*).

Nella realtà le cose non sono così semplici. Come prima cosa l'ISP di Alice controlla che la risposta contenga come indirizzo IP sorgente quello del server di primo livello. Trudy però aggira facilmente il test, visto è in grado di mettere quello che vuole in quel campo sorgente dell'IP e che gli indirizzi IP dei server di primo livello sono pubblici. Un'altra difficoltà è generata dal fatto che tutte le richieste contengono un numero di sequenza, per consentire ai server DNS di far corrispondere le richieste con le risposte.

Questo implica che, per riuscire a ingannare l'ISP di Alice, Trudy deve conoscere il numero di sequenza corretto. Il modo più semplice che Trudy ha a disposizione per venire a conoscenza del numero di sequenza corrente è quello di registrare un proprio dominio, diciamo *trudy-the-intruder.com.* Supponiamo che l'indirizzo IP associato sia di nuovo 42.9.9.9. Trudy crea anche un server DNS per il suo dominio appena nato, *dns.trudy-the-intruder.com.* Anche questo server usa l'indirizzo IP 42.9.9.9, visto che Trudy ha un solo computer. Adesso Trudy deve far conoscere il suo DNS server all'ISP di Alice. Questa è un'operazione semplice, basta che chieda all'ISP di Alice di risolvere *qualcosa.trudy-the-intruder.com.* L'ISP andrà a cercare chi risolve gli indirizzi del dominio di Trudy con una ricerca sul server di primo livello *com.*

Avendo messo nella cache dell'ISP di Alice *dns.trudy-the-intruder.com,* Trudy può lanciare il suo attacco con una query verso l'ISP per l'indirizzo *www.trudy-the-intruder.com.* L'ISP invierà a sua volta una query al DNS server di Trudy per risolvere il nome, che contiene il numero di sequenza che Trudy cercava. Veloce come un lampo, Trudy chiede all'ISP di Alice di cercare l'indirizzo di Bob e immediatamente invia una risposta alla sua stessa domanda inviando all'ISP una risposta falsificata, che dovrebbe, in teoria, provenire dal server di primo livello *com* e che afferma: *"bob.com* è il 42.9.9.9". Questa risposta falsificata contiene un numero di sequenza pari a quello ricevuto in precedenza, ma incrementato di una unità. Per avere una maggior probabilità di riuscita, Trudy invia anche un altro pacchetto con il numero di sequenza incrementato di due e, se vuole, anche un altro gruppo di pacchetti con numeri di sequenza crescenti. Solo uno di questi pacchetti avrà il numero di sequenza corretto, gli altri pacchetti saranno semplicemente scartati. La risposta falsificata viene messa in cache una volta arrivata a destinazione, mentre la risposta legittima è scartata in quanto arriva quando la query è stata ormai risolta. Adesso quando Alice cerca *bob.com* viene indirizzata verso l'IP 42.9.9.9, l'indirizzo di Trudy, che quindi è riuscita in un attacco di tipo "man in the middle" restando nel suo salotto. I passi dell'attacco sono illustrati dalla Figura 8.47. Questo attacco specifico può essere sventato con l'uso nei server DNS di ID casuali nelle loro query anziché solo contando, ma sembra che ogni volta che viene tappato un buco un altro salta fuori. In particolare, gli ID sono solo di 16 bit, quindi è facile scandirli tutti quando è un computer che sta tirando a indovinare.

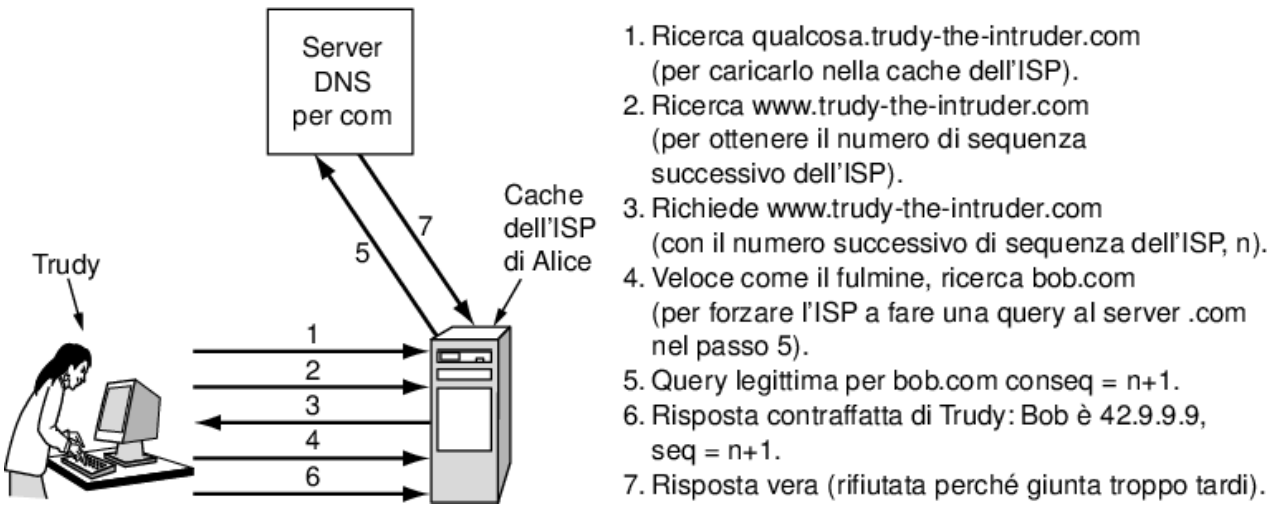


Figura 8.47 Ecco come Trudy forza l'ISP di Alice.

DNS sicuro

Il vero problema del DNS è che fu progettato quando Internet era usata come supporto alla ricerca da poche centinaia di università, quindi né Alice, né Bob, né tanto meno Trudy erano sulla scena. La sicurezza non era un problema a quei tempi, l'unica cosa importante era riuscire a far funzionare Internet. Le condizioni sono cambiate radicalmente negli anni, così nel 1994 IETF decise di formare un gruppo di lavoro per rendere il DNS fondamentalmente sicuro. Questo progetto è conosciuto col nome di **DNSsec** (*DNS security*, sicurezza DNS) ed è presentato nell'RFC 2535. Sfortunatamente, DNSsec non è ancora usato da tutti, con la conseguenza che svariati server DNS sono tuttora vulnerabili con attacchi di spoofing. DNSsec è basato sulla crittografia a chiave pubblica ed è concettualmente molto semplice. Ogni zona DNS (nel senso della Figura 7.5) ha una coppia di chiavi pubblica/privata. Tutte le informazioni inviate da un DNS sono firmate con la chiave privata della zona d'origine, quindi il ricevente ne può verificare l'autenticità. DNSsec offre tre servizi fondamentali:

1. la prova del luogo di provenienza dei dati
2. la distribuzione della chiave pubblica
3. l'autenticazione della transazione e della richiesta.

Il primo è il servizio principale, poiché garantisce che i dati di ritorno sono stati approvati dal proprietario della zona. Il secondo è impiegato per memorizzare e utilizzare in modo sicuro le chiavi pubbliche. Il terzo è necessario per salvaguardarsi contro attacchi di tipo ripetizione o spoofing. Notiamo che la segretezza non è offerta come servizio, in quanto l'informazione contenuta nel DNS è considerata pubblica. Si stima che serviranno alcuni anni per completare l'attivazione di tutti i DNSsec, per questo motivo è essenziale che i server attenti alla sicurezza siano in grado di comunicare con quelli che ignorano questi problemi: ciò implica che il protocollo non può essere cambiato. Vedremo nel seguito alcuni dettagli dell'implementazione di DNSsec.

I record DNS sono raggruppati in insiemi chiamati **RRSet** (*resource record set*, insieme di record di risorse), dove tutti i record di un insieme hanno lo stesso nome, classe e tipo. Un RRSet può contenere più record *A* se, per esempio, un nome DNS viene risolto con un indirizzo IP primario e uno secondario. Gli RRSet sono stati estesi con diversi nuovi tipi di record, discussi più avanti. Per ogni RRSet si calcola un hash crittografico (per esempio usando MD5 o SHA-1) e l'hash viene firmato con la chiave privata della zona (per esempio usando RSA); quindi l'unità di trasmissione verso i client è l'RRSet firmato. Alla ricezione di un RRSet, il client può verificare se è stato firmato con la chiave privata della zona di origine, perciò il dato viene accettato solo se la firma è integra. Visto che ogni RRSet contiene la propria firma, gli RRSet possono essere messi in cache ovunque, anche su server non fidati, senza che questo comprometta la sicurezza.

DNSsec introduce nuovi tipi di record. Il primo di questi è il record *KEY,* che contiene la chiave pubblica di una zona, utente, host o altro protagonista; l'identificatore dell'algoritmo crittografico utilizzato per la firma, il protocollo usato per la trasmissione e altri cose minori. La chiave pubblica viene memorizzata così com'è, senza usare un certificato X.509 per via del peso aggiuntivo che questo comporterebbe. Il campo con l'algoritmo contiene un 1 per le firme MD5/RSA (la scelta preferita), mentre valori diversi indicano altre combinazioni di algoritmi. Il campo del protocollo indica se vengono utilizzati IPsec o altri protocolli di sicurezza. Il secondo nuovo tipo di record è *SIG* e contiene l'hash firmato con l'algoritmo specificato dal record *KEY.* La firma si applica a tutti

i record dell'RRSet, quindi sono inclusi tutti i record *KEY* ma è esclusa la firma stessa (cioè *SIG). SIG* contiene anche l'orario in cui la firma comincia il suo periodo di validità, l'orario di scadenza, il nome di chi firma e alcune altre informazioni.

La struttura di DNSSec permette di mantenere la chiave privata della zona fuori dalla rete. Una o due volte al giorno, i contenuti del database di zona possono essere trasportati manualmente (per esempio su CD-ROM) su una macchina non connessa alla rete, dove è memorizzata la chiave privata. Tutti gli RRSet possono essere firmati su questa macchina, quindi i record *SIG* calcolati vengono trasportati di nuovo sul server primario della zona usando un altro CD-ROM. Operando in questo modo, la chiave privata può essere conservata in un CD-ROM che rimane chiuso in cassaforte per tutto il giorno, eccetto che per il tempo necessario alle operazioni di firma dei nuovi RRSet sulla macchina disconnessa dalla rete. Dopo che le operazioni di firma sono concluse, tutte le copie della chiave vengono cancellate dalla memoria e il CD-ROM ritorna in cassaforte. Questa procedura riduce il problema della sicurezza elettronica a quello della sicurezza fisica, qualcosa che la gente capisce e riesce a gestire.

Questo metodo, che genera RRSet firmati in anticipo, aiuta a velocizzare le risposte alle query, visto che elimina la necessità di effettuare calcoli crittografici al momento della risposta. Ha però lo svantaggio di richiedere grandi quantità di spazio disco, per memorizzare tutte le chiavi e le loro firme dentro al database DNS. Alcuni record possono crescere fino a dieci volte la loro dimensione originale, una volta firmati. Quando un processo client riceve un RRSet firmato, deve applicare la chiave pubblica della zona d'origine per decifrare l'hash ricevuto, calcolare il proprio hash e confrontare i due valori. I dati sono considerati validi se questi coincidono. Tuttavia questa procedura si porta dietro il problema di come il client possa prelevare la chiave pubblica della zona. Un modo può essere quello di acquisirla da un server fidato, usando una connessione sicura (per esempio usando IPsec).

Nella pratica, si suppone che i client siano preconfigurati con le chiavi pubbliche di tutti i domini di primo livello. Se Alice vuole visitare il sito Web di Bob, può chiedere al DNS l'RRSet di *bob.com,* che contiene l'indirizzo IP e un record *KEY* con la chiave pubblica di Bob. Questo RRSet sarà firmato dal dominio di primo livello *com,* il che permette ad Alice di verificarne facilmente la validità. Un esempio del contenuto di questo RRSet è mostrato nella Figura 8.48.

Nome di dominio	Validità	Classe	Tipo	Valore
bob.com.	86400	IN	A	36,1,2,3
bob.com.	86400	IN	KEY	3682793A7B73F731029CE2737D...
bob.com.	86400	IN	SIG	86947503A8B848F5272E53930C...

Figura 8.48 Esempio di RRSet per Bob.com. Il record KEY è la chiave pubblica di Bob. Il record SIG è l'hash (firmato dal server com di livello più alto) dei record A e KEY, che ne conferma l'autenticità.

Avendo un copia della chiave pubblica di Bob, Alice può procedere chiedendo al DNS server di Bob (che è gestito da Bob stesso) l'indirizzo IP di *www.bob.com.* Questo RRSet sarà firmato con la chiave privata di Bob, quindi Alice può verificare la firma dell'RRSet che Bob gli ha restituito. Se in qualche modo Trudy riuscisse a inserire un falso RRSet in una qualunque delle cache del sistema, Alice sarebbe facilmente in grado di rilevare la contraffazione, perché i contenuti del record *SIG* risulterebbero non corretti.

DNSsec fornisce anche un metodo crittografico per legare le risposte alle query che le hanno generate. Ciò serve per prevenire lo spoofing che Trudy era riuscita a perpetrare nel caso della Figura 8.47. Questo meccanismo (opzionale) di antispoofing funziona aggiungendo alla risposta un hash del messaggio di query firmato con la chiave privata di chi risponde. Dato che Trudy non conosce la chiave privata del server di primo livello *com,* non riesce neanche a falsificare la risposta alla query che è stata inviata dall'ISP di Alice. Trudy può sicuramente far sì che la sua risposta raggiunga 1TSP per prima, ma verrà poi scartata in quanto non ha una firma valida dell'hash della query originale. DNSsec supporta anche altri tipi di record. Per esempio, il record *CERT* può essere usato per memorizzare certificati (per esempio X.509). Questo tipo di record è stato inserito perché c'è chi vuole trasformare DNS in una PKI; se questo accadrà è ancora tutto da vedere. Concludiamo qui la nostra discussione su DNSsec, rimandando all'RFC 2535 per ulteriori dettagli.

SSL – secure socket layer

Avere la sicurezza nell'assegnazione dei nomi è un buon inizio, ma la sicurezza del Web è un argomento molto più vasto. Il passo successivo consiste nel mettere in sicurezza le connessioni. Vedremo adesso come ottenere delle connessioni sicure; niente di quello che riguarda la sicurezza è semplice, questo argomento non fa eccezione.

Durante la sua prima fase di rapida espansione, il Web era usato essenzialmente per distribuire pagine statiche. Poco dopo le aziende hanno avuto l'idea di usarlo per gestire transazioni finanziarie, come l'acquisto di merce con la carta di credito, l'on-line banking e il trading azionario: queste applicazioni hanno creato una domanda per la gestione di connessioni sicure. Nel 1995, Netscape Communications Corp, che allora dominava il mercato dei produttori di browser, rispose alla domanda introducendo un pacchetto per la sicurezza chiamato **SSL** (*secure socket layer*). Oggi questo software e il suo protocollo sono ampiamente usati anche da Firefox, Safari e Internet Explorer, quindi vale sicuramente la pena di esaminarlo.

SSL instaura una connessione sicura fra due socket e si occupa anche di:

1. negoziazione dei parametri fra client e server
2. autenticazione del server presso il client
3. comunicazione segrete
4. protezione dell'integrità dei dati.

Abbiamo già visto questo tipo di caratteristiche, quindi non le commenteremo ulteriormente, il posizionamento di SSL nella pila dei protocolli di comunicazione è mostrato nella Figura 8.49: SSL è un nuovo livello interposto fra i livelli di applicazione e trasporto. SSL accetta richieste dal browser e le manda a TCP perché siano trasmesse al server; dopo aver stabilito la connessione il compito principale di SSL è quello di gestire la compressione e la cifratura. HTTP usato sopra SSL prende il nuovo nome di **HTTPS** (*Secure HTTP*), anche se si tratta ancora del protocollo HTTP

standard. A volte HTTPS è disponibile su una porta diversa (443) dallo standard (80). Per inciso, SSL non è ristretto ai Web browser, ma questa è la sua applicazione più comune e può anche fornire autenticazione reciproca.

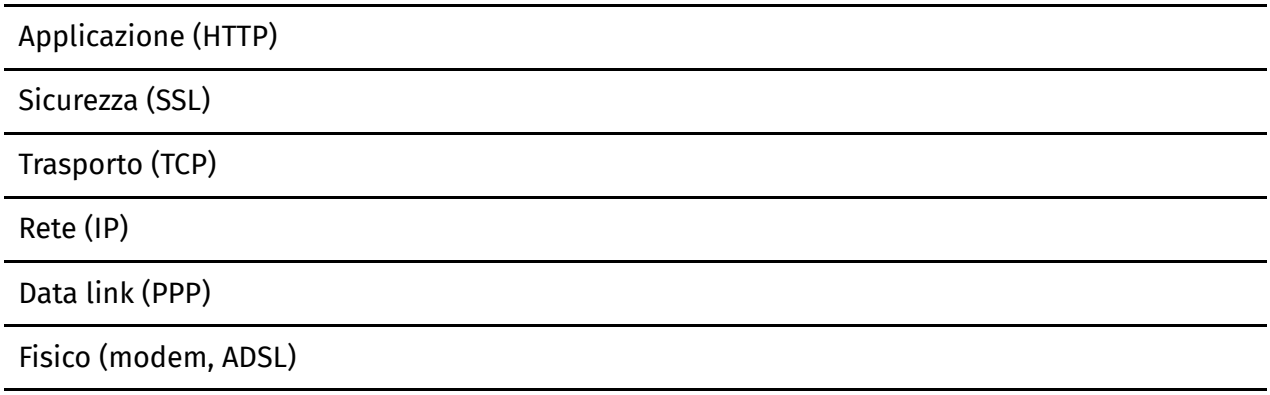


Figura 8.49 Livelli (e protocolli) per un utente domestico che naviga sul Web con SSL.

Esistono diverse versioni del protocollo SSL; discuteremo però solo la versione 3, che è quella più largamente utilizzata. SSL supporta molti algoritmi e opzioni che indicano, per esempio, la presenza o assenza di compressione, gli algoritmi crittografici utilizzati e altri dettagli legati alle restrizioni sull'esportazione di sistemi crittografici. Ciò garantisce che la crittografia veramente forte è utilizzabile solo quando entrambi gli estremi della connessione sono nel territorio degli Stati Uniti, mentre negli altri casi le chiavi vengono limitate a 40 bit, un livello di sicurezza che i crittografi considerano una specie di scherzo. Netscape è stata obbligata ad aggiungere questa restrizione per poter avere una licenza all'esportazione da parte del governo degli Stati Uniti. SSL consiste di due subprotocolli, uno per stabilire le connessioni sicure e uno per usarle. Cominciamo a vedere come vengono stabilite le connessioni sicure. Il subprotocollo per le connessioni sicure è mostrato nella Figura 8.50. Comincia con il messaggio 1, nel quale Alice manda a Bob la richiesta di stabilire una connessione. La richiesta specifica anche la versione di SSL che Alice utilizza e le sue preferenze per quanto riguarda la compressione e gli algoritmi crittografici. Contiene anche un nonce, R_A, che verrà usato in seguito.

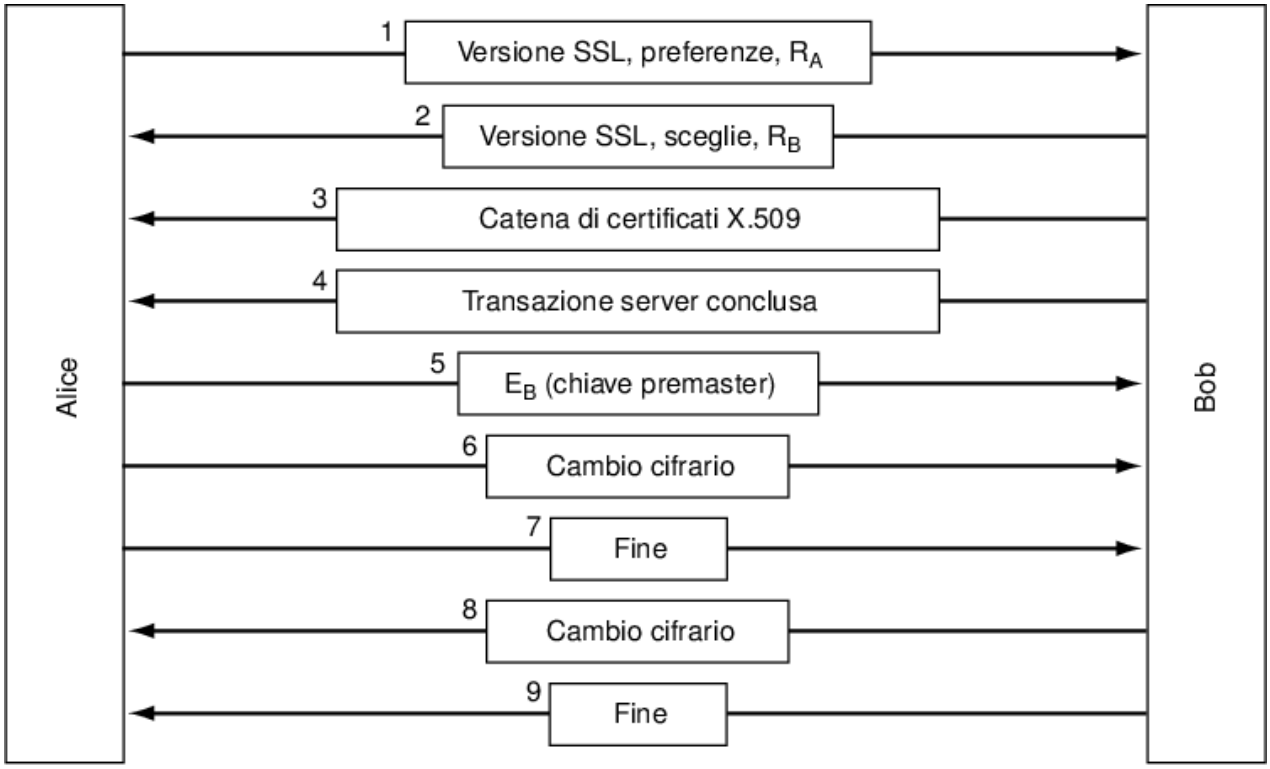


Figura 8.50 Versione semplificata del protocollo SSL per instaurare una connessione.

Adesso è il turno di Bob, che manda il messaggio 2 contenente la sua scelta (operata fra gli algoritmi che Alice supporta) e un nonce R_B. Poi Bob invia con il messaggio 3 un certificato contenente la sua chiave pubblica. Se il certificato non è firmato da un'autorità nota, Bob invia anche una catena di certificati che permettono di risalire fino a essa. Tutti i browser, incluso quello di Alice, contengono all'installazione già un centinaio di chiavi pubbliche. Alice sarà in grado di verificare la chiave pubblica di Bob se questi riesce a stabilire una catena che fa capo a una chiave pubblica preinstallata nel browser. Ora Bob invia il messaggio 4 ad Alice, dicendole che è arrivato il suo turno.

Alice risponde scegliendo una chiave casuale di 384 bit detta **chiave di premaster** (*premaster key*), che invia a Bob in forma cifrata usando la chiave pubblica di Bob (messaggio 5). La chiave di sessione vera e propria è il risultato di un complesso calcolo basato sulla chiave di premaster, in combinazione con i due nonce. Alice e Bob sono entrambi in grado di calcolarla solo dopo che Bob ha ricevuto il messaggio 5. Per questo motivo ora Alice dice a Bob di cominciare a usare la chiave di sessione (messaggio 6) e che il subprotocollo di connessione ha finito il suo compito (messaggio 7). Con i messaggi 8 e 9 Bob conferma la ricezione dei messaggi 6 e 7. A questo punto Alice conosce l'identità di Bob, però Bob non sa chi è Alice (a parte il caso in cui Alice abbia una chiave pubblica e il relativo certificato, situazione strana per un individuo). Il primo messaggio di Bob sarà probabilmente una richiesta ad Alice perché inserisca login name e password, tuttavia il protocollo di login è al di fuori degli scopi dell'SSL. Il trasporto dei dati comincia dopo il login, a prescindere dal modo in cui è effettuato. SSL supporta diversi algoritmi crittografici. Il più forte utilizza per la cifratura il triplo DES con tre chiavi separate, oltre a SHA-1 per gestire l'integrità dei messaggi. La combinazione di questi algoritmi è relativamente lenta, perciò si usa tipicamente solo

per le applicazioni bancarie o comunque là dove è richiesto il massimo grado di sicurezza. Per le ordinarie applicazioni di e-commerce viene impiegato RC4 con una chiave a 128 bit per la cifratura e MD5 per l'autenticazione dei messaggi. RC4 utilizza la chiave a 128 bit come un seme, che viene espanso in un numero più grande utilizzato nel funzionamento interno dell'algoritmo per generare il keystream, poi applicato in XOR con il testo in chiaro per generare il classico flusso cifrato, come abbiamo visto nella Figura 8.14. La versione per l'esportazione utilizza anch'essa RC4 con le chiavi a 128 bit, ma 88 di questi bit sono resi pubblici per facilitare la forzatura del cifrario. Per il trasporto vero e proprio si usa un secondo subprotocollo, mostrato nella Figura 8.51. I messaggi provenienti dal browser sono innanzitutto suddivisi in unità di 16 KB. Se la compressione è abilitata, ogni unità viene compressa singolarmente. Si procede con il calcolo di una chiave segreta a partire dai due nonce e dalla chiave di premaster: questa chiave viene concatenata al testo in chiaro e il tutto è passato attraverso l'algoritmo di hash negoziato in precedenza (di solito MD5). Tale hash viene aggiunto a ogni frammento come MAC (*message autentication code*); quindi il frammento compresso e il suo MAC vengono cifrati con l'algoritmo di crittografia simmetrica negoziato in precedenza (di solito questo è fatto calcolando lo XOR con il keystream RC4). Infine, viene aggiunta un'intestazione a ogni frammento e si procede alla sua trasmissione tramite la connessione TCP.

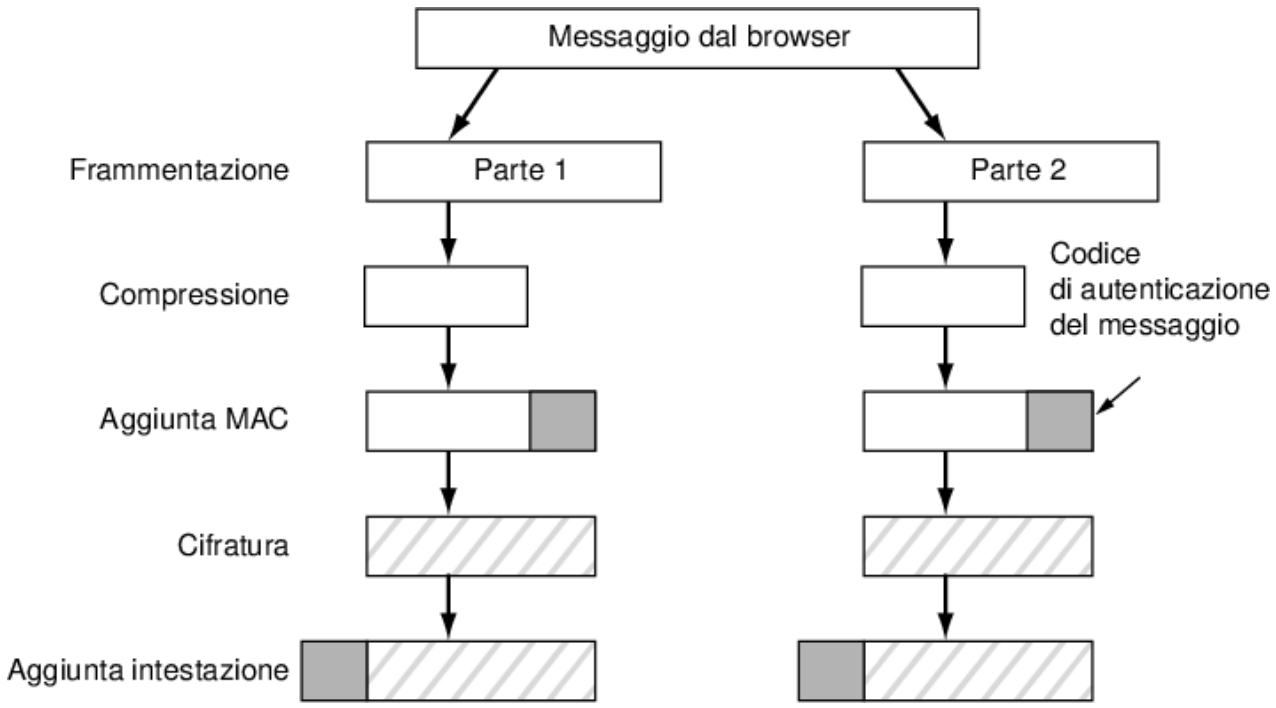


Figura 8.51 Trasmissione dei dati con SSL.

È necessaria una particolare attenzione quando si usa RC4. È stato dimostrato che alcune chiavi deboli di questo algoritmo si possono facilmente analizzare con la criptoanalisi, quindi la sicurezza di SSL con RC4 è un po' traballante (Fluhrer et al. 2001). I browser che permettono all'utente di scegliere gli algoritmi di cifratura utilizzati andrebbero configurati per usare il triplo DES con

chiavi a 168 bit e SHA-1, anche se questa combinazione è più lenta di RC4 e MD5. Oppure, ancora meglio, gli utenti dovrebbero passare a un browser che supporti il successore di SSL, che vedremo tra breve.

Un problema con SSL è dato dal fatto che i protagonisti possono anche non avere dei certificati; anche se li hanno non sempre verificano che le chiavi utilizzate corrispondano ai valori inseriti nei certificati.

Nel 1996 Netscape Corp. sottopose SSL all'IETF per la standardizzazione. Il risultato fu **TLS** (*transport layer security*, sicurezza a livello di trasporto), che è descritto nell'RFC 5246.

TLS è stato costruito su SSL versione 3. I cambiamenti fatti a SSL sono stati relativamente piccoli ma abbastanza perché SSL versione 3 e TLS non possano interoperare. Per esempio, il modo in cui la chiave di sessione è derivata dalla chiave di premaster e nonce è stato cambiato per rendere la chiave più resistente (cioè più difficile per la criptanalisi). A causa di questa incompatibilità molti browser implementano entrambi i protocolli, con il TLS che ripiega verso SSL durante la negoziazione se necessario. Questo è chiamato SSL/TLS. La prima implementazione TLS apparve nel 1999 e la versione 1.2 è stata definita nell'agosto del 2008. Include il supporto per sistemi di cifratura più forti (in particolare AES). SSL ha ancora una forte presenza sul mercato anche se, probabilmente, TLS lo rimpiazzerà gradualmente.

Sicurezza del codice mobile

Gestione di nomi e connessioni sono due aree sensibili nello scenario della sicurezza del Web, ma non sono le uniche. Agli inizi, quando le pagine Web erano semplicemente dei file HTML statici, non contenevano del codice eseguibile. Al giorno d'oggi, invece, le pagine Web contengono spesso piccoli programmi come applet Java, controlli ActiveX e JavaScript. Scaricare ed eseguire questo **codice mobile** è ovviamente un grande rischio per la sicurezza. Per ridurre i rischi sono stati escogitati diversi metodi, quindi daremo uno sguardo ad alcune delle problematiche generate dal codice mobile e agli approcci per affrontarle.

Sicurezza delle applet Java

Le applet Java sono dei piccoli programmi Java, compilati in un linguaggio per una particolare macchina orientata allo stack chiamata **JVM** (*java virtual machine*). Le applet si possono mettere su una pagina Web, pronte per essere scaricate insieme alla pagina stessa. Dopo lo scaricamento le applet vengono inserite nell'interprete JVM che si trova dentro al browser, come illustrato nella Figura 8.52.

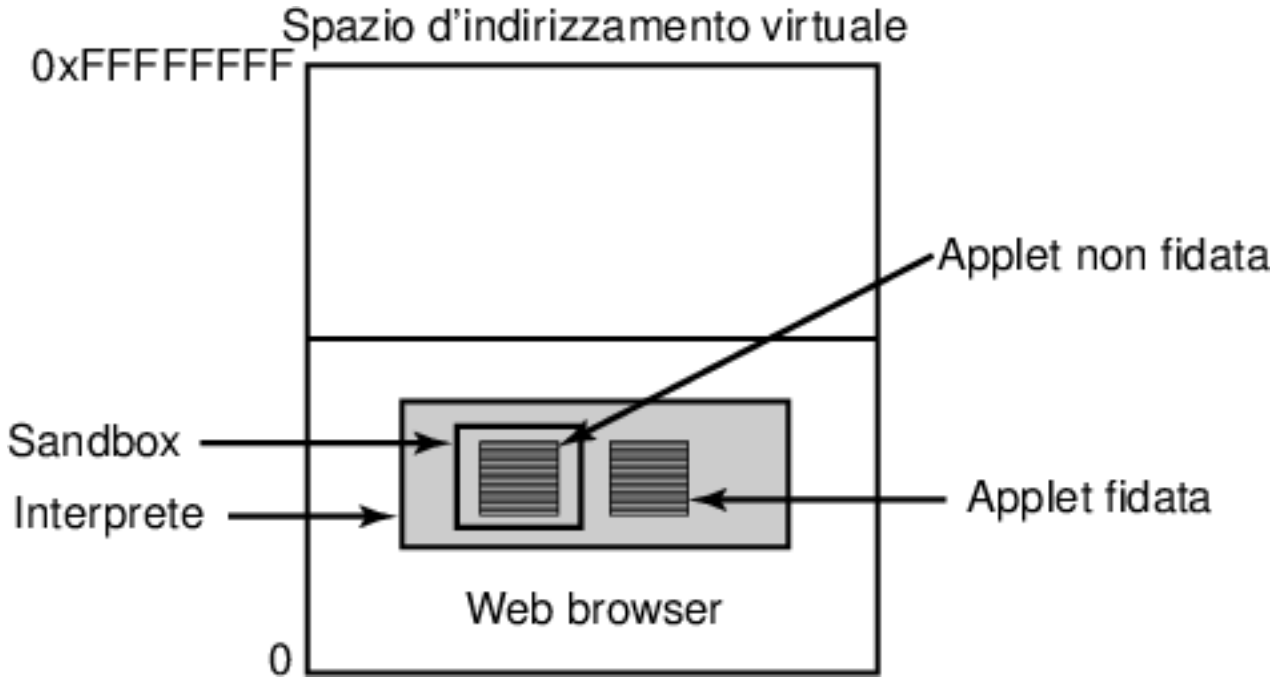


Figura 8.52 Le applet possono essere interpretate da un browser Web.

Il vantaggio nell'utilizzare codice interpretato al posto di quello compilato sta nel fatto che ogni istruzione può essere esaminata dall'interprete prima di essere eseguita. Questo da all'interprete l'opportunità di controllare la validità dell'indirizzo dell'istruzione; inoltre, anche le chiamate di sistema possono essere intercettate e interpretate. Se un'applet è fidata (magari perché viene dal disco locale), le sue chiamate di sistema possono essere eseguite senza ulteriori domande. Se invece l'applet non è fidata (cioè è venuta da Internet), si può procedere a incapsularla in una cosiddetta **sandbox**, che ne restringe il campo d'azione e intercetta tutti i tentativi di utilizzo delle risorse di sistema.

Quando un'applet cerca di usare una risorsa di sistema, la sua chiamata è passata al supervisore della sicurezza per l'approvazione. Il supervisore esamina la chiamata alla luce delle regole locali di sicurezza e decide se permettere o respingere la chiamata. In questo modo è possibile dare alle applet l'accesso ad alcune risorse di sistema, ma non a tutte. Sfortunatamente la realtà è che questo modello di sicurezza funziona male ed è sempre pieno di errori.

ActiveX

I controlli ActiveX sono programmi eseguibili per architettura x86, racchiudibili dentro alle pagine Web. Quando s'incontra una pagina che contiene un ActiveX, per prima cosa viene fatto un controllo per vedere se dovrebbe essere eseguito; se supera il test viene effettivamente mandato in esecuzione. I controlli ActiveX non sono interpretati né eseguiti in una sandbox, cioè hanno lo stesso potere degli eseguibili dell'utente e pertanto possono teoricamente causare gravi danni. Tutta la sicurezza sta nel decidere se si deve eseguire il controllo ActiveX oppure no. Ripensandoci, l'intera idea è un enorme buco di sicurezza.

Il metodo scelto da Microsoft per prendere questa decisione è basato sull'idea del **code signing** (*codice firmato*). Ogni controllo ActiveX è accompagnato da una firma digitale, cioè un codice hash firmato dal creatore usando la crittografia a chiave pubblica. Quando compare un ActiveX, il browser verifica subito la firma per essere sicuro che non è stato modificato durante la trasmissione. Se la firma è corretta il browser controlla le sue tabelle interne per vedere se il creatore del programma è fidato oppure se esiste una catena di certificati che portano a un creatore fidato. Se il creatore è fidato, il programma è eseguito, altrimenti no. Il sistema di Microsoft per verificare i controlli ActiveX è chiamato **Authenticode**.

È istruttivo confrontare gli approcci di Java e ActiveX. Nell'approccio di Java non si fa nessun tentativo per determinare l'autore dell'applet; viene invece usato un interprete che ci assicura che l'applet non faccia cose che il proprietario della macchina ha proibito. Al contrario, con il code signing non viene fatto nessun tentativo di verificare se il codice è sicuro o no: se il programmatore che ha scritto il codice *voleva* che durante l'esecuzione venisse formattato l'hard disk e cancellata la flash ROM, in modo che il computer non potrà più essere fatto ripartire, allora il codice verrà eseguito e distruggerà il computer (se i controlli ActiveX sul browser sono abilitati).

Molti pensano che fidarsi di un produttore di software sconosciuto sia troppo pericoloso. Per dimostrare il problema un programmatore di Seattle fondò una software house e fu certificato come fidato, attività entrambe facili. Realizzò poi un controllo ActiveX che effettuava uno spegnimento della macchina senza causare problemi e lo distribuì in giro. Il risultato fu che molte macchine vennero spente da quel controllo ActiveX (ma ripartirono senza nessun danno). Il programmatore voleva solamente portare all'attenzione del mondo il problema dell'esecuzione degli ActiveX. La risposta ufficiale fu quella di revocare il certificato per quello specifico controllo ActiveX, il che pose fine all'imbarazzante episodio, ma il problema sottostante è ancora lì, in attesa che qualche programmatore malintenzionato lo voglia sfruttare (Garfinkel e Spafford, 2002). Visto che non c'è modo di controllare migliaia di produttori di software che potrebbero scrivere codice mobile, il risultato è che la tecnica del code signing rappresenta un disastro in attesa di accadere.

JavaScript

JavaScript non ha nessun modello formale di sicurezza: quello che ha è una lunga storia di implementazioni con numerose falle. Ogni produttore gestisce la sicurezza in un modo differente, per esempio Netscape Navigator versione 2 usava qualcosa di simile al modello di Java ma, arrivati alla versione 4, questo è stato abbandonato per un modello simile al code signing. Il problema fondamentale è che lasciar girare sulla propria macchina codice scritto da sconosciuti vuol dire andare a cercare problemi. Dal punto di vista della sicurezza, è come invitare un ladro nella propria casa per poi cercare di tenerlo sotto controllo in modo che non fugga dalla cucina al salotto. Se accade qualcosa d'inaspettato e vi distraete per un momento, possono accadere cose spiacevoli. Il problema è che il codice mobile permette di generare grafica accattivante e controlli interattivi veloci, quindi molti Web designer pensano che queste funzioni siano molto più importanti della sicurezza, specialmente quando sono le macchine degli altri a essere a rischio.

Le estensioni dei browser

Oltre a estendere le pagine Web con del codice, c'è un mercato in piena espansione di **estensioni** per i **browser**, **add-on** e **plug-in**. Sono programmi per computer che estendono le funzionalità dei Web browser. I plug-in spesso offrono la capacità di interpretare o visualizzare un certo tipo di

contenuto, come i PDF o le animazioni Flash. Le estensioni e gli add-on forniscono nuove proprietà al browser, come una migliore gestione delle password o modi per interagire con le pagine, per esempio contrassegnandole o abilitando lo shopping facile per voci correlate.

Installare un'estensione, un add-on o un plug-in è semplice: basta incappare in qualcosa che volete mentre navigate e seguire i link per installare il programma. Quest'azione provocherà lo scaricamento di codice da Internet e la sua installazione nel browser. Tutti questi programmi sono scritti per dei framework che differiscono tra loro indipendentemente dal browser che stiamo migliorando. Tuttavia, a una prima approssimazione, diventano parte del software considerato affidabile dal browser. Quindi, se il codice installato presenta degli errori, l'intero browser può essere compromesso.

Ci sono anche altre due modalità che portano al malfunzionamento. Il primo è che il programma possa essere male intenzionato, per esempio raccogliendo informazioni personali e spedendole a un server remoto. Per quanto ne sa il browser, l'utente ha installato l'estensione proprio per questo scopo. Il secondo problema è che i plug-in danno al browser la capacità d'interpretare nuovi tipi di contenuti. Spesso questo contenuto è esso stesso interamente riempito di linguaggio di programmazione; PDF e Flash ne sono buoni esempi. Quando gli utenti vedono pagine con contenuti PDF e Flash, i plug-in nel loro browser stanno eseguendo codice PDF e Flash. Quel codice è meglio che sia sicuro; spesso ci sono difetti che possono essere sfruttati. Per tutte queste ragioni, gli add-on e i plug-in dovrebbero essere installati solo se necessario e solo da distributori affidabili.

Virus

I virus sono un'altra forma di codice mobile. Al contrario degli esempi precedenti, i virus non sono affatto benvenuti. La differenza fra un virus e un tipico codice mobile è che i virus sono scritti per riprodursi. Quando un virus arriva, tramite una pagina Web, un allegato di e-mail o in qualche altro modo, di solito comincia a infettare i programmi eseguibili sull'hard disk. Quando uno di questi programmi viene eseguito, il controllo passa al virus che (di solito) tenta di diffondersi verso altre macchine: per esempio manda per e-mail delle copie di sé stesso a tutte le persone nell'agenda della vittima. Alcuni virus infettano il settore di boot dell'hard disk, in questo modo vanno in esecuzione a ogni riavvio della macchina. I virus sono diventati un enorme problema su Internet e hanno già causato danni per miliardi di dollari. Non ci sono soluzioni semplici. Un aiuto potrebbe arrivare da una nuova generazione di sistemi operativi basati su microkernel sicuri, con una rigida divisione di utenti, processi e risorse.

8.10 Aspetti sociali

Internet e la sua tecnologia per la sicurezza sono un'area dove questioni sociali, leggi e tecnologia s'incontrano, spesso con conseguenze importanti. Nel seguito esamineremo brevemente tre aree: privacy, libertà di parola e copyright. Ovviamente in questa sede possiamo solamente accennare ai problemi fondamentali; per ulteriori approfondimenti si consigliano Anderson (2008a), Garfinkel e Spafford (2002) e Schneier (2004). Anche Internet è pieno di materiale sull'argomento, basta digitare in un motore di ricerca le parole "privacy", "censorship" e "copyright". Anche il sito Web di questo libro ha alcuni link interessanti. Si trova al seguente URL: *http://www.pearsonhighered.com/tanenbaum*.

Privacy

Le persone hanno diritto alla privacy? Questa è una buona domanda. Il quarto emendamento della costituzione degli Stati Uniti proibisce al governo di perquisire le abitazioni private, i documenti e gli effetti personali senza che esista una buona ragione; inoltre, specifica le circostanze in cui un mandato di perquisizione può essere emesso dalle autorità. Possiamo dire che almeno negli Stati Uniti l'argomento della privacy viene discusso pubblicamente da almeno 200 anni.

Ciò che è cambiato nell'ultimo decennio sono da una parte l'efficacia con cui il governo riesce a spiare i cittadini e dall'altra i metodi a disposizione dei cittadini per prevenire tali operazioni. Nel diciottesimo secolo, se il governo voleva spiare i documenti di un cittadino doveva mandare un poliziotto a cavallo alla sua fattoria e chiedere che gli fossero mostrati certi documenti. Era una procedura lunga e difficile. Al giorno d'oggi, le compagnie telefoniche e gli Internet provider, alla presentazione di un mandato, forniscono velocemente alle autorità la possibilità di mettere sotto controllo le linee. Questo facilita molto la vita ai poliziotti e non c'è rischio di cadere da cavallo.

La crittografia cambia questo scenario. Chiunque si prenda il disturbo di scaricare e installare PGP, usando una chiave ben custodita e di lunghezza sufficiente, può essere decisamente sicuro che nessuno nell'universo riuscirà a leggere la sua e-mail, con o senza mandato di perquisizione. I governi hanno capito bene questo fatto e non lo apprezzano per niente. La vera privacy significa che è molto più difficile spiare i criminali di tutte le specie, ma anche che è diventato più difficile spiare i giornalisti e gli avversari politici. Di conseguenza, alcuni governi limitano o proibiscono l'esportazione della crittografia. In Francia, per esempio, prima del 1999 la crittografia era fuori legge, a patto che il governo non fosse a conoscenza delle chiavi.

La Francia non era l'unico caso. Nell'aprile 1993 il governo degli Stati Uniti annunciò l'intenzione di rendere standard per tutte le comunicazioni di rete un criptoprocessore hardware, chiamato **clipper chip**. Veniva anche detto che ciò assicurava la privacy dei cittadini, mentre il chip avrebbe permesso al governo di decifrare tutto il traffico con uno schema chiamato **key escrow** (che dava l'accesso a tutte le chiavi). Naturalmente il governo prometteva che avrebbe ascoltato le comunicazioni solo in presenza di un mandato valido. Ovviamente ciò generò un gran furore, dove i difensori della privacy erano contrari a tutto il progetto, mentre le forze di polizia erano decisamente favorevoli. Alla fine, il governo fece marcia indietro e abbandonò l'idea.

Una gran quantità d'informazioni sulla privacy elettronica è disponibile sul sito Web della Electronic Frontier Foundation, *http://www.eff.org/*.

Remailer anonimi

Varie tecnologie, fra cui PGP e SSL, fanno sì che due soggetti possano stabilire una connessione sicura, autenticata e assente da interferenze o sorveglianza esterna. In alcuni casi però la privacy è più tutelata facendo in modo che *non* ci sia autenticazione, cioè rendendo anonima la comunicazione. Questo può essere desiderabile per messaggi punto a punto, per i gruppi di discussione (*newsgroup*) o per entrambi. Consideriamo alcuni esempi. Primo esempio: i dissidenti politici che vivono sotto un regime autoritario desiderano comunicare in modo anonimo, per evitare di essere messi in prigione oppure assassinati. Secondo esempio: gli scandali che sono scoppiati per le azioni di grandi società, istituzioni governative, università o altri enti spesso nascono da una fuga di notizie dall'interno. Chi divulga queste notizie spesso preferisce rimanere anonimo. Terzo esempio: le persone con opinioni impopolari sulla società, la politica o la religione, possono voler comunicare fra loro tramite e-mail o newsgroup, senza doversi esporre personalmente. Quarto: alcune persone potrebbero voler discutere nei newsgroup di alcolismo, malattie mentali, molestie sessuali, abusi sui minori oppure problemi delle minoranze perseguitate senza esporsi pubblicamente. Ovviamente si potrebbero fare molti altri esempi.

Esaminiamo un caso specifico. Nel 1990 alcuni avversari di un gruppo religioso non tradizionale pubblicarono i loro punti di vista su un newsgroup di USENET utilizzando un **remailer anonimo**. Questo server permetteva agli utenti di creare degli pseudonimi e quindi mandare le e-mail al server, il quale provvedeva a rispedirle o fare un post sul newsgroup utilizzando lo pseudonimo creato in precedenza. In questo modo nessuno poteva sapere da chi provenisse veramente il messaggio. Alcuni messaggi nel newsgroup rivelavano delle informazioni che il gruppo religioso considerava dei segreti o comunque documenti coperti da copyright. Il gruppo religioso rispose andando dalle autorità locali a dichiarare che i suoi segreti erano stati svelati e il copyright su alcuni documenti era stato violato: entrambe le azioni erano perseguibili per legge nello stato in cui si trovava il server. Ne seguì un processo, dove l'operatore del server fu obbligato a fornire la mappa con le informazioni necessarie a svelare la vera identità delle persone che avevano inviato i documenti sul newsgroup (per inciso, non è la prima volta che una religione s'infuria perché qualcuno ha divulgato i suoi segreti: William Tyndale fu arso vivo nel 1536 per aver tradotto la bibbia in inglese).

Una gran parte della comunità Internet fu indignata da questa violazione dell'anonimato. Tutti trassero la conclusione che un remailer anonimo che mantiene la mappa fra gli indirizzi e-mail e gli pseudonimi (detto remailer di tipo 1) non vale molto. Questo caso stimolò diverse persone a sviluppare remailer anonimi capaci di resistere agli effetti di un mandato di comparizione del tribunale (attacco giudiziario).

I nuovi remailer, spesso chiamati **remailer cypherpunk**, funzionano così: l'utente produce un messaggio e-mail, completo con l'intestazione RFC 822 (senza il campo *from:*, ovviamente), lo cifra con la chiave pubblica del remailer e lo manda al remailer stesso, il quale estrae l'intestazione RFC 822 più esterna, decifra il contenuto e rispedisce il messaggio. Il remailer non mantiene lo storico della sua attività né log di alcun tipo, quindi anche se venisse confiscato non vi si troverebbe nessuna traccia dei messaggi transitati. Diversi utenti che desiderano mantenere l'anonimità fanno passare le loro richieste attraverso una catena di più remailer anonimi, come

mostrato nella Figura 8.53. In questo caso, Alice vuole mandare a Bob un messaggio di San Valentino, veramente e sicuramente anonimo, e per fare questo usa tre remailer. Alice scrive il messaggio *M* e aggiunge un'intestazione con l'indirizzo e-mail di Bob. Quindi cifra il tutto con la chiave pubblica del remailer 3, E_3 (indicata dal motivo a strisce orizzontali) e vi antepone un'intestazione con l'indirizzo e-mail del remailer 3 in chiaro. Questo è il messaggio mostrato nella figura fra i remailer 2 e 3.

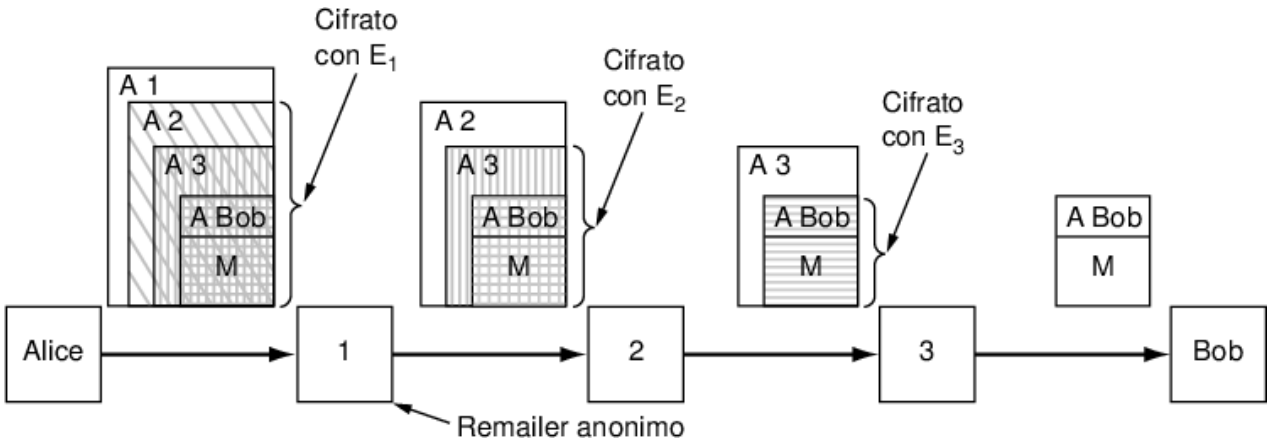


Figura 8.53 Alice usa tre remailer per inviare un messaggio a Bob.

Alice prosegue cifrando tale messaggio con la chiave pubblica del remailer 2, E_2 (indicata dal motivo a strisce verticali) e aggiungendo un'intestazione in chiaro contenente l'indirizzo e-mail del remailer 2. Questo è il messaggio mostrato fra 1 e 2 nella Figura 8.53. Infine Alice cifra l'intero messaggio con la chiave pubblica E_2 del remailer 1 e aggiunge in chiaro un'intestazione con l'indirizzo e-mail del remailer 1. Questo è il messaggio mostrato nella figura alla destra di Alice, che viene effettivamente trasmesso.

Quando il messaggio arriva al remailer 1, l'intestazione più esterna viene eliminata, il corpo del messaggio decifrato e quindi inviato al remailer 2. Operazioni analoghe avvengono negli altri due remailer.

Rintracciare il mittente dal messaggio finale è estremamente difficile per chiunque. Molti remailer anonimi prendono comunque ulteriori precauzioni; per esempio possono trattenere i messaggi per un tempo di lunghezza casuale, aggiungere o rimuovere della sporcizia alla fine del messaggio oppure riordinare i messaggi. Tutto questo è fatto per complicare a chiunque il compito di rintracciare la corrispondenza fra i messaggi in uscita e quelli in ingresso e quindi fare un'analisi del traffico. Per una descrizione di un sistema che rappresenta lo stato dell'arte dei remailer anonimi si rimanda a Mazières e Kaashoek (1998).

L'anonimato non è limitato all'e-mail. Esistono servizi che permettono la navigazione Web anonima usando lo stesso tipo di percorso a più livelli in cui un nodo conosce solo il nodo successivo della catena. Questo metodo è chiamato **onion routing** (*routing a cipolla*) poiché ogni

nodo toglie uno strato della cipolla per determinare dove inoltrare il pacchetto. L'utente configura il proprio browser per usare il servizio di anonimizzazione come un proxy. Tor è un esempio ben noto di tale sistema (Dingledine et al., 2004). In questo modo, tutte le richieste passano per la rete di anonimizzazione, che richiede la pagina e poi la spedisce al richiedente. I siti Web vedono come sorgente l'anonimizzatore invece dell'utente reale. Se l'anonimizzatore non mantiene i log della sua attività nessuno può identificare chi ha richiesto quale pagina.

Libertà di parola

La privacy riguarda gli individui e il loro desiderio di limitare le informazioni personali che vogliono rendere accessibili all'esterno. Una seconda questione sociale riguarda la libertà di parola e il suo opposto, la censura. Questa nasce al contrario dal desiderio dei governi di restringere la quantità di informazioni che gli individui possono leggere e pubblicare. Il Web, che contiene milioni e milioni di pagine, è diventato il paradiso del censore. A seconda della natura e dell'ideologia del regime il materiale proibito può includere siti Web che contengono:

1. materiale inappropriato per bambini o ragazzi
2. odio diretto verso gruppi etnici, religiosi, sessuali, ecc.
3. informazioni sulla democrazia e i valori democratici
4. narrazione di eventi storici che possono contraddire la versione governativa
5. manuali per aprire le serrature, costruire armi, cifrare messaggi, ecc.

La tipica risposta è quella di bandire i siti cattivi.

Alle volte i risultati sono inaspettati. Per esempio, alcune biblioteche pubbliche hanno installato dei filtri Web sui loro computer per renderli adatti ai ragazzi, bloccando i siti pornografici. I filtri proibiscono l'accesso ai siti inseriti in specifiche black list (*liste nere*) e controllano anche la presenza di parole offensive nelle pagine, prima di mostrarle agli utenti. In un caso accaduto nella contea di Loundoun, in Virginia, il filtro ha bloccato una ricerca legittima di informazioni sul cancro al seno, in quanto il filtro aveva intercettato la parola "seno". L'utente della biblioteca fece causa alla contea di Loundon. All'estremo opposto, a Livermore in California, un genitore fece causa alla biblioteca pubblica per *non* aver installato un filtro dopo che il figlio di 12 anni era stato scoperto con del materiale pornografico. Quindi, che cosa devono fare le biblioteche?

Molte persone non si rendono pienamente conto che il World Wide Web ha un'estensione mondiale. Non tutti gli stati sono d'accordo nello stabilire che cosa è lecito sul Web e che cosa no. Per esempio, nel novembre 2000, un tribunale francese ordinò a Yahoo, una multinazionale californiana, di bloccare l'accesso di tutti gli utenti francesi alle aste di cimeli nazisti che avvenivano sul sito Web di Yahoo, poiché il possesso di tale materiale è in contrasto con le leggi francesi. Yahoo fece ricorso a un tribunale degli Stati Uniti, che le diede ragione. Il problema della legge da applicare in casi come questo è ancora molto lontano da una soluzione finale. Immaginiamo che cosa potrebbe succedere se qualche tribunale nello Utah volesse obbligare la Francia a bloccare siti Web che commerciano vino, perché la loro attività non è conforme alle leggi dello stato dello Utah sul commercio di bevande alcoliche. Supponiamo che la Cina chieda che siano banditi tutti i siti Web che trattano di democrazia, in quanto contrari agli interessi di stato. Le leggi iraniane in termini di religione si possono applicare alla Svezia liberale? L'Arabia Saudita ha il diritto di bloccare i siti Web che trattano dei diritti delle donne? La questione è proprio un vaso di Pandora.

Un commento molto rilevante di John Gilmore a questo proposito è: "la rete interpreta la censura come un guasto e cerca dei percorsi per aggirarlo". Per un'interpretazione concreta di questo concetto, consideriamo l'**eternity service** (Anderson, 1996). L'obiettivo è quello di assicurarsi che le informazioni pubblicate non possano essere cancellate o riscritte, come era prassi comune nell'Unione Sovietica durante il governo di Josef Stalin. Nell'eternity service, l'utente specifica per quanto tempo vuole che il materiale pubblicato venga preservato, quindi paga una tariffa proporzionale alla durata e alla dimensione del materiale e infine esegue l'upload. Da quel momento, nessuno potrà rimuovere oppure modificare il materiale, neppure chi ne ha fatto l'upload.

Come si può implementare un servizio di questo tipo? Il modello più semplice consiste nell'usare un sistema peer-to-peer, dove i documenti vengono memorizzati nelle decine di server che fanno parte del sistema. Ogni server riceverà una parte della tariffa pagata, come incentivo per partecipare al sistema. I server dovrebbero essere distribuiti in aree con diverse leggi, per aumentare la robustezza del sistema. Liste contenenti 10 server scelti a caso vengono memorizzate in modo sicuro in diverse locazioni, in questo modo anche se alcune vengono compromesse ne rimangono delle altre. Un'autorità che voglia distruggere un documento non potrebbe mai avere la sicurezza di aver trovato e distrutto tutte le copie. Il sistema si può rendere autoriparante, nel senso che se alcune copie del documento vengono distrutte, gli altri siti possono cercare nuovi repository dove memorizzare nuove copie per rimpiazzare quelle perse. L'eternity service fu storicamente la prima proposta per un sistema resistente alla censura. Nel frattempo sono stati proposti anche altri sistemi, alcuni dei quali realmente implementati, che svolgono anche nuove funzionalità come la cifratura, l'anonimato e la tolleranza ai guasti. In diversi casi si utilizza la strategia di suddividere i file in tanti frammenti, ognuno dei quali è memorizzato in una varietà di server. Alcuni esempi di sistemi di questo tipo sono Freenet (Clarke et al. 2002), PASIS (Wylie et al., 2000) e Publius (Waldman et al., 2000). Altri lavori di questo tipo si trovano in (Serjantov, 2002).

Un numero crescente di paesi sta cercando di stabilire regole per l'esportazione di materiale intangibile, che spesso include siti Web, software, pubblicazioni scientifiche, e-mail, help-desk telefonici, ecc. Anche la Gran Bretagna, che ha alle spalle una tradizione secolare sulla libertà di parola, sta seriamente pensando di istituire delle leggi restrittive in materia. Questo potrebbe portare, per esempio, alla situazione in cui una discussione tecnica fra un professore inglese e un suo studente straniero della Università di Cambrige sarebbe trattata come esportazione e quindi necessiterebbe dell'autorizzazione governativa (Anderson, 2002). Inutile dire che molte persone considerano questa politica oltraggiosa.

Steganografia

Negli paesi dove abbonda la censura i dissidenti spesso cercano di trovare una scappatoia usando la tecnologia. La crittografia permette di inviare dei messaggi segreti (non necessariamente legali), ma se il governo pensa che Alice sia una persona indesiderata, il solo fatto che comunichi con Bob mette anche lui nella categoria delle persone indesiderate. Questo è il modo in cui i governi repressivi interpretano il concetto di chiusura transitiva, anche senza aver molti matematici alle loro dipendenze. In questo caso di solito vengono in aiuto i remailer anonimi, ma non quando sono banditi a livello nazionale e tutti i messaggi per l'estero richiedono una licenza governativa per l'esportazione. Il Web può risolvere questo problema. Le persone che vogliono comunicare in

modo segreto spesso cercano di nascondere il fatto stesso che ci sia una qualunque comunicazione fra di loro. La scienza che si occupa di nascondere i massaggi è chiamata **steganografia**, dalle parole greche che significano "scrittura nascosta": questa scienza infatti era nota già agli antichi greci. Erodoto scrisse di un generale che fece rasare la testa di un messaggero, tatuò il messaggio sul suo cranio e poi attese la ricrescita dei capelli prima di inviare il messaggero. Le tecniche moderne usano lo stesso concetto, solo con una banda passante molto più alta e con una minore latenza e non richiedono il servizio di un barbiere.

Come esempio consideriamo la Figura 8.54(a). Questa fotografia, scattata dall'autore in Kenya, ritrae tre zebre che guardano un albero di acacia. La Figura 8.54(b) sembra rappresentare le stesse tre zebre e l'albero di acacia, ma in realtà contiene anche degli extra. Infatti, contiene al suo interno il testo completo di cinque opere di Shakespeare: *Amleto, Macbeth, Il mercante di Venezia* e *Giulio Cesare.* In totale queste opere occupano 700 KB di testo. Come funziona questo canale steganografico? L'immagine originale a colori ha una risoluzione di 1024 X 768 pixel. Ogni pixel consiste di tre numeri di 8 bit, che determinano rispettivamente l'intensità del rosso, verde e blu. Il colore di ciascun pixel è dato dalla sovrapposizione lineare dei tre colori. Il metodo di codifica steganografico usa, come canale di comunicazione nascosto, i bit di ordine più basso di ogni colore RGB. In questo modo ogni pixel può far posto a 3 bit di informazione segreta, uno nel rosso, uno nel blu e uno nel verde. Con un'immagine come quella della Figura 8.54 possiamo nascondere fino a 1024 × 768 × 3 bit di informazione segreta, cioè 294.912 byte. Il testo completo delle cinque opere di Shakespeare con un breve messaggio introduttivo ammonta a 734.891 byte, ma il testo compresso con un algoritmo standard ha raggiunto la dimensione di circa 274 KB. L'output compresso è stato poi cifrato con IDEA e infine inserito nei bit di ordine più basso di ogni colore. Come si può vedere (o meglio, come non si vede), l'esistenza dell'informazione è completamente invisibile. Anche nella versione originale, non si riesce assolutamente a distinguere l'aggiunta d'informazione steganografica. L'occhio umano non riesce a distinguere facilmente fra un colore a 21 bit e uno a 24 bit. Le due immagini in bianco e nero a bassa risoluzione non rendono giustizia alla potenza di questo metodo.

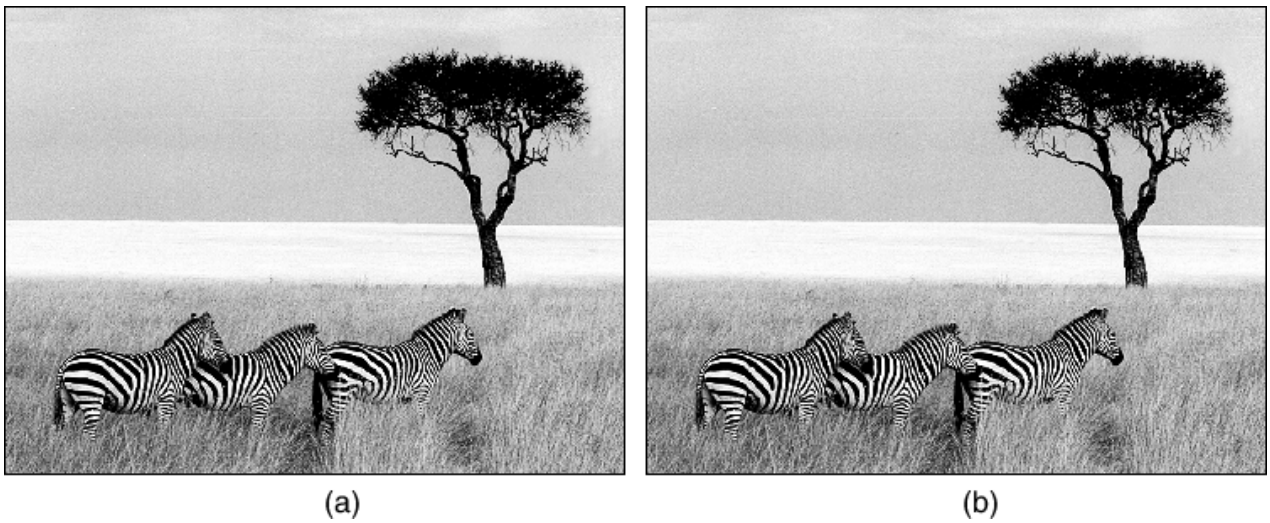


Figura 8.54 (a) Tre zebre e un albero. (b) Tre zebre, un albero e il testo integrale di cinque opere di William Shakespeare.

L'autore ha preparato una dimostrazione che include la versione a colori e alta risoluzione dell'immagine della Figura 8.54(b) con le cinque opere di Shakespeare inserite all'interno, per dare un'idea migliore di come funziona la steganografia. La dimostrazione, assieme agli strumenti per inserire ed estrarre testo dalle immagini, può essere scaricata dal sito Web di questo libro. Alcuni dissidenti potrebbero usare questo metodo per comunicare in modo segreto: si comincia creando un sito Web pieno d'immagini approvate dal regime, per esempio foto del grande condottiero, di sport locale, film, star della televisione, ecc. All'interno di queste immagini si possono inserire dei messaggi con il metodo steganografico. Se i messaggi vengono prima compressi e poi cifrati, anche chi ne sospettasse la presenza avrebbe molte difficoltà nel distinguerli dal semplice rumore bianco. Ovviamente le immagini devono essere degli originari. Se si prendono delle immagini da Internet per poi alterarne dei bit si ottiene una ben misera copertura. Le immagini non sono il solo mezzo per trasportare messaggi steganografici; anche i file musicali funzionano bene e i file video forniscono un'enorme banda steganografica. Anche la disposizione e l'ordine dei tag nei file HTML possono essere usati per trasportare informazione.

Abbiamo esaminato la steganografia nel contesto della libertà di parola, ma ne esistono numerosi altri usi. Uno di questi consiste nel codificare messaggi segreti all'interno delle immagini per affermarne la proprietà. Se un'immagine viene rubata e messa su un sito Web, il legittimo proprietario può portare in tribunale il contenuto del messaggio steganografico e quindi dimostrare l'effettivo possesso dell'immagine. Questa tecnica è detta **watermarking** ed è discussa in Piva et al. (2002).

Per ulteriori informazioni sulla steganografia si consulti Wayner (2008).

Copyright

La privacy e la censura sono solamente due delle aree in cui la tecnologia incontra le leggi. Una terza area è la normativa sul copyright. **Il copyright** garantisce ai creatori di **IP** (*intellectual property*, proprietà intellettuale) come scrittori, artisti, compositori, musicisti, fotografi, cineasti e coreografi il diritto di sfruttare le proprie IP per un certo periodo di tempo; tipicamente è la durata della vita dell'autore più 50 anni, che diventano 75 nel caso di proprietà da parte di aziende. Alla scadenza del copyright, le opere diventano di dominio pubblico e chiunque le può utilizzare o vendere a piacimento. Per esempio, il progetto Gutenberg (*http://www.gutenberg.org/*) ha pubblicato sul Web migliaia di opere che sono di pubblico dominio (Shakespeare, Twain, Dickens, ecc.). Nel 1998 il congresso degli USA ha esteso il copyright di altri 20 anni su richiesta di Hollywood, che lamentava il fatto che senza un'estensione nessuno avrebbe creato più niente. Per contrasto, notiamo che i brevetti durano solo 20 anni e la gente continua a produrre nuove invenzioni.

Il problema del copyright è venuto alla ribalta quando Napster, un servizio per lo scambio di musica, aveva 50 milioni di membri. Anche se Napster non realizzava nessuna copia di musica le autorità giudiziarie ritennero che il fatto che Napster gestisse un database centrale con le informazioni su chi aveva quale canzone costituisse di per sé un sostanziale contributo alla violazione della legge sul copyright. In definitiva, i giudici hanno stabilito che Napster aiutava altre persone a infrangere la legge. Anche se nessuno afferma seriamente che il copyright è sbagliato, molti pensano che la durata sia troppo lunga e favorisca le grandi multinazionali a scapito del pubblico. Al contempo, sta emergendo una nuova generazione di servizi di scambio

musicale, che fanno sorgere diverse questioni etiche. Per esempio, consideriamo una rete peer-to-peer dove gli utenti possono condividere file legali (musica di pubblico dominio, video privati, trattati religiosi che non siano segreti e così via) e magari anche alcuni file che sono coperti da copyright. Supponiamo che tutti gli utenti siano connessi costantemente, per esempio tramite ADSL. Ogni computer ha un indice dei contenuti del suo disco fisso e una lista dei membri del servizio. Se qualcuno desidera ricercare un file specifico può scegliere un utente a caso per vedere se ha quel file. In caso negativo, la ricerca può procedere controllando tutti i membri elencati nella lista posseduta da quell'utente. I computer sono perfetti per questo tipo di lavoro. Dopo aver trovato il file, il richiedente deve solamente copiarlo sul suo disco fisso. Se il file è coperto da copyright, è probabile che il richiedente stia infrangendo la legge (anche se per trasferimenti internazionali non è chiaro di quale legge stiamo parlando). Che cosa possiamo dire della sorgente? È forse un crimine tenere sul proprio disco fisso della musica che è stata regolarmente acquistata ma dove anche altre persone possono trovarla? Pensiamo alla situazione in cui una persona che abbia una casetta in campagna, senza serrature alle porte, subisca il furto da parte di un ladro di IP, che penetra in casa portandosi un laptop e uno scanner, copia l'intero contenuto di un libro coperto da copyright e poi scappa. Possiamo ritenere *il proprietario della casa* colpevole del crimine di non aver protetto il copyright del libro? Ci sono anche altri problemi sul fronte del copyright. È in corso un'enorme battaglia fra Hollywood e l'industria informatica. I primi vogliono una protezione più forte sulla proprietà intellettuale, mentre i secondi non vogliono fare i poliziotti per conto di Hollywood. Nell'ottobre 1998, il congresso USA varò il **DMCA** (*digital millenium copyright act*), che definisce atto criminale ogni azione volta ad aggirare i meccanismi di protezione presenti sulle opere coperte da copyright, e la divulgazione delle informazioni utili per violare tali protezioni. Legislazioni simili sono state varate anche nell'Unione Europea. Anche se nessuno pensa che i pirati dell'Estremo Oriente debbano essere autorizzati a duplicare il materiale coperto da copyright, molti pensano però che la DMCA sposti troppo l'ago della bilancia dalla parte degli interessi dei proprietari di copyright, contro l'interesse del pubblico.

Un caso emblematico si è realizzato nel settembre del 2000. In quella occasione un consorzio industriale, che aveva il compito di costruire un sistema inattaccabile per vendere musica on-line, decise di sponsorizzare un concorso in cui invitava il pubblico a cercare di forzare il sistema (che è esattamente quello che si deve fare quando si vuole varare un nuovo sistema di sicurezza). Un gruppo di ricercatori nell'ambito della sicurezza provenienti da diverse università e guidato dal Prof. Edward Felten di Princeton riuscì effettivamente a forzare il sistema. Quindi scrissero un articolo sulla loro scoperta e lo mandarono alla conferenza USENIX sulla sicurezza, dove fu oggetto di valutazione scientifica e infine accettato. Prima che potesse presentare l'articolo, Felten ricevette una lettera dalla RIAA (*recording industry association of America*),[7] in cui gli autori venivano diffidati dal pubblicare l'articolo, altrimenti sarebbe stato denunciato sotto i termini previsti dalla DMCA.
La risposta di Felten fu quella di portare il caso davanti a una corte federale per avere un giudizio sul fatto che pubblicare degli articoli scientifici fosse ancora un'attività legale. L'industria discografica, temendo che la corte emanasse una sentenza non favorevole, ritirò le accuse e quindi il caso fu chiuso. Senza dubbio questa scelta fu dettata dalla debolezza delle motivazioni che l'industria discografica aveva a suo favore: aveva lanciato un invito a forzare i suoi sistemi e poi aveva minacciato di fare causa a chi aveva accettato la sfida. Dopo che le accuse furono ritirate, l'articolo venne pubblicato (Craver et al., 2001). È certo tuttavia che ci saranno nuove occasioni di scontro.

Nel frattempo musica e video piratati hanno alimentato la crescita massiccia delle reti peer-to-peer. Questo non è piaciuto ai detentori dei diritti di copyright, che hanno utilizzato il DMCA per entrare in azione. Ora ci sono sistemi automatici che cercano le reti peer-to-peer e lanciano avvertimenti agli operatori di rete e agli utenti che sono sospettati di infrangere le leggi sul copyright. Negli Stati Uniti questi avvertimenti sono conosciuti come **DMCA takedown notice** (*notifiche di cessazione DMCA*). Questa ricerca è una corsa agli armamenti perché è difficile catturare con affidabilità i trasgressori di copyright. Anche la vostra stampante potrebbe essere scambiata per un colpevole (Piatek et al., 2008).

Un'altra questione collegata è la **dottrina dell'uso legittimo**, che è stata sancita da sentenze emesse in vari paesi. Questa dottrina afferma che chi acquista un'opera coperta da copyright ha il diritto di farne delle copie, con alcuni limiti. Per esempio è possibile citare alcune parti dell'opera per scopi scientifici, usarla come materiale didattico nelle scuole e nelle università; in alcuni casi è consentito fare copie di sicurezza per uso personale, da usare nel caso in cui il supporto originale non sia più utilizzabile. Per stabilire se una copia entra nella dottrina dell'uso legittimo si deve anche considerare se: (1) il suo uso è commerciale, (2) quale percentuale dell'opera è stata copiata, (3) l'effetto che la copia ha sulle vendite dell'opera originale. Si dà il caso, però, che DMCA e leggi analoghe approvate in vari paesi dell'Unione Europea vietino ogni mezzo atto a eludere gli schemi di protezione, quindi queste leggi proibiscono anche le operazioni legali che vanno sotto la dottrina dell'uso legittimo. A tutti gli effetti DMCA toglie agli utenti alcuni diritti che avevano storicamente conquistato e dà quindi più potere a chi vende contenuti. Un confronto netto fra le due opposte posizioni sarà pertanto inevitabile.

Un altro sviluppo in corso che fa impallidire anche il DMCA nel suo spostamento di equilibri tra i proprietari e gli utilizzatori di copyright è il **trusted computing** come propagandato dagli organismi di settore, come il **TCG** (*trusted computing group*) guidato da società come Intel e Microsoft. L'idea è di fornire supporto per un attento monitoraggio del comportamento dell'utente in diversi modi (ad esempio, la riproduzione di musica pirata) a un livello inferiore rispetto a quello del sistema operativo per proibire comportamenti non desiderati. Lo si ottiene con un piccolo chip, chiamato **TPM** (*trusted platform module*), difficile da manomettere. Molti PC venduti oggi sono dotati di un TPM. Il sistema permette al software scritto dai proprietari dei contenuti di manipolare i PC in modi che gli utenti non possono cambiare. Questo solleva la questione su chi è affidabile nel trusted computing; certamente non l'utente. Inutile a dirlo, le conseguenze di questo schema sono immense. È bello che l'industria finalmente si occupi di sicurezza, ma va biasimato il fatto che tutti gli sforzi siano concentrati nel far rispettare il copyright, piuttosto che affrontare i problemi dei virus, intrusioni e altre questioni che preoccupano gli utenti dei sistemi informatici.

Nei prossimi anni legislatori e avvocati saranno decisamente occupati per cercare di bilanciare gli interessi dei possessori di copyright e quelli del pubblico. Il cyberspazio non è diverso dal mondo reale: anche il cyberspazio crea contrasti fra diversi gruppi di persone, con il risultato di avere lotte per il potere, cause legali e (si spera) alcune soluzioni definitive; almeno fino a quando non arriverà una nuova tecnologia dagli effetti dirompenti.

8.11 Sommario

La crittografia è uno strumento che può essere usato per mantenere e assicurare la confidenzialità, l'integrità e l'autenticità delle informazioni. Tutti i sistemi crittografici moderni sono basati sul principio di Kerckhoff, che consiste nell'utilizzare un algoritmo noto pubblicamente con una chiave segreta. Molti algoritmi crittografici usano delle trasformazioni complesse che contengono sostituzioni e permutazioni usate per trasformare il testo in chiaro in quello cifrato. Tuttavia, se la crittografia quantistica si dimostrerà utilizzabile nella pratica, l'utilizzo di blocchi monouso darà origine a sistemi crittografici realmente imbattibili.

Gli algoritmi crittografici possono essere divisi in algoritmi a chiave simmetrica e algoritmi a chiave pubblica. Gli algoritmi a chiave simmetrica funzionano mescolando i bit del testo in chiaro con una serie di cicli parametrati dalla chiave; il risultato è il testo cifrato. Gli algoritmi a chiave simmetrica più diffusi oggi sono triplo DES e Rijndael (AES). Questi algoritmi si possono usare con modalità diverse, fra cui: electronic code book, cipher block, chaining, stream cipher, counter e altri.

Gli algoritmi a chiave pubblica hanno la proprietà di utilizzare delle chiavi diverse per la cifratura e la decifratura, inoltre la chiave di decifratura non può essere ricavata da quella di cifratura. Queste proprietà rendono possibile la pubblicazione della chiave di cifratura. Il principale algoritmo a chiave pubblica è RSA, che deriva la sua forza dalla grande difficoltà computazionale associata al problema della scomposizione di numeri grandi in fattori primi.

I documenti commerciali, legali e altri, necessitano di un qualche tipo di firma. Per questo motivo sono stati sviluppati schemi per firme digitali, che usano sia gli algoritmi a chiave simmetrica sia quelli a chiave asimmetrica. Di norma, i messaggi da firmare vengono prima passati attraverso un algoritmo di hashing, come MD5 o SHA-1, e poi viene firmato l'hash invece del messaggio originale.

La gestione delle chiavi pubbliche può essere fatta tramite l'uso dei certificati, che sono documenti che legano i protagonisti alle loro chiavi crittografiche. I certificati sono firmati da autorità fidate o da qualcuno che è stato certificato da un'autorità fidata. Perché il sistema funzioni, la radice della catena deve essere nota in anticipo; infatti, normalmente, i browser vengono rilasciati con diversi certificati di questo tipo già preinseriti.

Questi strumenti crittografici si possono impiegare per rendere sicuro il traffico di rete. IPsec lavora a livello di rete cifrando il flusso dei pacchetti da un host all'altro. I firewall sono usati per controllare il traffico che entra ed esce da una rete aziendale tramite regole sul protocollo e la verifica delle porte utilizzate. Le reti private virtuali possono simulare il funzionamento delle vecchie linee dedicate e fornire quindi certe proprietà di sicurezza desiderabili. Infine, le reti wireless necessitano di un buon grado di sicurezza, altrimenti qualcuno potrebbe leggere il contenuto di tutti i messaggi e i protocolli come 802.11i la forniscono.

Due soggetti che vogliono stabilire una sessione per comunicare in modo sicuro, devono avere un metodo per autenticarsi reciprocamente e stabilire una chiave di sessione condivisa in caso di necessità. Esistono svariati protocolli di autenticazione, inclusi alcuni che utilizzano una terza parte

fidata, Diffide-Hellman, Kerberos e la crittografia a chiave pubblica.

Si può ottenere la sicurezza della e-mail usando una combinazione delle tecniche che abbiamo studiato in questo capitolo. PGP, per esempio, comprime i messaggi, poi li cifra usando una chiave segreta, quindi invia la chiave segreta cifrandola con la chiave pubblica del ricevente. Inoltre, PGP calcola e trasmette anche l'hash del messaggio ottenendo così una verifica dell'integrità.

La sicurezza del Web è un altro argomento importante, che inizia con la sicurezza della risoluzione dei nomi. DNSsec fornisce un metodo per prevenire gli attacchi di DNS spoofing. La maggior parte dei siti Web di e-commerce utilizza SSL/TLS per stabilire sessioni sicure e autenticate fra il client e il server. Per gestire il codice mobile si adottano più tecniche, in particolare sandbox e code signing.

Internet solleva molte questioni dove la tecnologia si trova a interagire con la politica. Alcune di queste aree includono la privacy, la libertà di parola e il copyright.

Problemi

1. Si forzi il cifrario monoalfabetico riportato di seguito. Il testo in chiaro, che consiste di sole lettere, è un pezzo di una nota poesia di Lewis Carroll (in lingua inglese).

mvyy bek mnyx n yvjjyr snijrh invq n muvjvdt je n idnvy
jurhri n fehfevir pyeir oruvdq ki ndq uri jhrnqvdt ed zb jnvy
Irr uem rntrhyb jur yeoijrhi ndq jur jkhjyri nyy nqlndpr
Jurb nhr mnvjvdt ed jur iuvdtyr mvyy bek pezr ndq wevd jur qndpr
mvyy bek, medj bek, mvyy bek, medj bek, mvyy bek wevd jur qndpr
mvyy bek, medj bek, mvyy bek, medj bek, medj bek wevd jur qndpr

2. Una cifrario affine è una versione di un cifrario a sostituzione monoalfabetica in cui le lettere di un alfabeto di dimensione m sono prima associate a numeri interi nell'intervallo tra 0 e $m - 1$. In seguito, l'intero rappresentante ogni lettera di testo in chiaro viene trasformato in un intero che rappresenta il corrispondente cifrato della lettera di testo. La funzione di cifratura per una sola lettera è: $E(x) = (ax + b)$ modulo m, in cui m è la dimensione dell'alfabeto e a e b sono la chiave di cifratura e sono tra loro relativamente primi. Trudy scopre che Bob ha generato un cifrario di testo usando un cifrario affine. Ottiene una copia del cifrario di testo e scopre che la lettera più frequente nel cifrario di testo è 'R' e la seconda lettera più frequente del cifrario di testo è 'K'. Mostrate come Trudy possa risolvere il codice e recuperate il testo in chiaro.

3 Si decifri il seguente cifrario a trasposizione di colonna. Il testo in chiaro, in lingua inglese, è preso da un noto libro di testo di informatica, quindi "computer" è una parola che probabilmente vi appare. Il testo in chiaro consiste di sole lettere (senza spazi). Il testo cifrato è stato suddiviso in blocchi di cinque caratteri per aumentarne la leggibilità.

aauan cvlre rurnn dltme aeepb ytust iceat npmey iicgo gorch srsoc
nntii imiha oofpa gsivt tpsit lbolr otoex

4. Alice usava un cifrario di trasposizione per codificare i propri messaggi verso Bob. Per maggiore sicurezza codificava la chiave del cifrario di trasposizione usando un cifrario a sostituzione e teneva la cifratura di codifica sul proprio computer. Trudy è riuscita a ottenere la chiave cifrata del cifrario di trasposizione. Può Trudy decifrare i messaggi di Alice a Bob? Perché o perché no?

5. Si trovi il blocco monouso di 77 bit che genera il testo in chiaro "Hello world" a partire dal testo cifrato della Figura 8.4.

6. Siete una spia e, per convenienza, avete una libreria con un numero infinito di libri a disposizione. Anche l'operatore ha una liberia del genere a sua disposizione. Avete concordato di usare *Il Signore degli Anelli* come un cifrario a blocco monouso. Si spieghi com'è possibile usare questi elementi per generare un one-time pad infinitamente lungo.

7. La crittografia quantistica ha bisogno di un cannone spara-fotoni che possa sparare, a comando, un singolo fotone che trasporta 1 bit di informazione. Calcolate quanti fotoni sono trasportati da 1 bit in una linea in fibra a 100 Gbps. Assumete che la lunghezza del fotone sia pari

alla sua lunghezza d'onda, che supponiamo pari a 1 micron. La velocità della luce nella fibra è di 20 cm/ns. (Poiché mancano dati sull'intensità, il risultato è chiaramente solo un limite inferiore).

8. Se Trudy raccoglie e rigenera fotoni mentre è in corso una comunicazione con crittografia quantistica, il risultato sarà che i valori raccolti saranno parzialmente errati e che Bob avrà degli errori nel blocco monouso che riceve. In media, che frazione dei bit del blocco monouso di Bob sarà in errore?

9. Un principio crittografico fondamentale afferma che tutti i messaggi devono avere delle ridondanze. Sappiamo però che la ridondanza aiuta gli intrusi a scoprire se la chiave che hanno indovinato è corretta. Consideriamo due forme di ridondanza. La prima consiste nell'avere una sequenza di bit nota nei primi n bit del testo in chiaro. La seconda consiste nell'aggiungere il codice hash del messaggio negli ultimi n bit. Da un punto di vista della sicurezza questi due approcci sono equivalenti? Argomentate la risposta.

10. Nella Figura 8.6 le P-box e le S-box si alternano. Sebbene questa disposizione è esteticamente attraente, risulta in qualche modo più sicura rispetto ad avere prima tutte le P-box e poi tutte le S-box?

11. Progettate un attacco al DES basato sull'assunto che il testo in chiaro consista esclusivamente di caratteri ASCII maiuscoli oltre a spazio, virgola, punto, punto e virgola, carriage return e line feed. Non sono note informazioni sulla parità del testo in chiaro.

12. Nel testo del capitolo abbiamo calcolato che una macchina dotata di un miliardo di processori capaci di elaborare una chiave ogni picosecondo, impiegherebbe 10^{10} anni a forzare AES a 128 bit. Allo stato attuale i computer possono arrivare a 1024 processori e impiegano 1 millisecondo per analizzare una chiave. Per avere delle prestazioni analoghe alla macchina di cui sopra è quindi necessario aumentare la velocità di un fattore 10^{15}. Supponendo che la legge di Moore continui a valere (cioè che la potenza di calcolo raddoppi ogni 18 mesi), quanti anni sarebbero necessari anche solo per costruire una macchina con le prestazioni indicate?

13. AES supporta chiavi di lunghezza pari a 256 bit. Quante sono le possibili chiavi di AES-256? Trovate dei riferimenti in fisica, chimica o astronomia, su numeri di pari grandezza. Internet può essere utile in questo tipo di ricerca. Traete delle conclusioni da questa ricerca.

14. Prendiamo un messaggio cifrato con DES in modalità block chaining. Supponiamo che un bit del testo cifrato nel blocco C_i venga accidentalmente trasformato da 0 a 1 durante la trasmissione. Quanto testo in chiaro sarà illeggibile a causa di questo errore?

15. Prendiamo di nuovo un messaggio cifrato in modalità block chaining. Questa volta, al posto di avere un singolo bit a 0 trasformato in 1 per errore, abbiamo il caso in cui un bit a 0 viene aggiunto al flusso di dati cifrati dopo il blocco C_v. Quanto testo in chiaro sarà illeggibile a seguito di questo errore?

16. Confrontate le modalità cipher block chaining e cipher feedback per quanto riguarda il numero di operazioni necessarie per trasmettere un file di grandi dimensioni. Quale modalità è più efficiente e di quanto?

17. Consideriamo un sistema crittografico RSA a chiave pubblica, con $a = 1$, $b = 2$ ... $y = 25$, $z = 26$.

(a) Elencate i primi 5 possibili valori di d, se $p = 7$ e $q = 13$.
(b) Se $p = 5$, $q = 31$ e $d = 37$, trovate e.
(c) Dati $p = 5$, $q = 11$ e $d = 9$, trovate e e cifrate la stringa "abcdefghij".

18. Alice e Bob usano una cifratura a chiave pubblica RSA per comunicare tra loro. Trudy scopre che Alice e Bob hanno condiviso uno dei numeri primi usati per determinare il numero n delle loro coppie di chiavi. In altre parole, Trudy scopre che $n_a = p_a \times q$ e $n_b = p_b \times q$. Come può Trudy usare questa informazione per rompere la codifica di Alice?

19. Consideriamo la modalità counter, come mostrata nella Figura 8.15, ma con $IV = 0$. In generale, l'uso del valore 0 ha un impatto negativo sulla sicurezza?

20. Nella Figura 8.20 è illustrato il modo in cui Alice invia un messaggio firmato a Bob. Se Trudy rimpiazza P Bob riesce ad accorgersene. Che cosa succede, invece, se Trudy rimpiazza sia P sia la firma?

21. Le firme digitali presentano una potenziale debolezza causata dagli utenti pigri. In una transazione di e-commerce si può verificare la situazione in cui un contratto viene stilato e l'utente viene chiamato a firmare il relativo codice hash SHA-1. Se l'utente non verifica l'effettiva corrispondenza tra contratto e hash potrebbe inavvertitamente firmare il contratto sbagliato. Supponiamo che la mafia voglia sfruttare questa debolezza per guadagnare un po' di soldi. Per fare questo installa un sito Web a pagamento (per esempio un sito pornografico, di scommesse o altro) e chiede a ogni nuovo cliente un numero di carta di credito. A questo punto invia ai clienti un contratto dove si afferma che il cliente vuole effettivamente usare i servizi del sito e che pagherà con la carta di credito. Ai clienti è richiesta la firma dell'hash del contratto, ben sapendo che la maggior parte delle persone firmerà senza controllare che il contratto e l'hash coincidano. Mostrate come, a questo punto, la mafia possa comprare dei diamanti da un gioielliere che opera legittimamente su Internet e quindi farli pagare agli ignari clienti.

22. Un corso di matematica ha 25 studenti. Assumendo che tutti siano nati nella prima metà dall'anno, (tra il primo gennaio e il 30 giugno) qual è la probabilità che almeno due studenti abbiano lo stesso compleanno? Assumete che nessuno sia nato il 29 febbraio e che quindi ci siano 181 possibili compleanni.

23. Dopo che Ellen confessò a Marilyn di averla ingannata riguardo alla questione della cattedra di Tom, Marilyn decise di risolvere questo tipo di problemi per il futuro usando il metodo di dettare le lettere in un registratore e quindi facendole trascrivere alla sua nuova segretaria. Marilyn decise anche di esaminare i messaggi a terminale dopo la trascrizione, per essere sicura che contengano le sue esatte parole. La nuova segretaria potrebbe ancora usare l'attacco del compleanno per falsificare i messaggi? Se sì, come può fare? *Suggerimento:* è possibile.

24. Nella Figura 8.23 consideriamo il tentativo fallito di Alice, che voleva acquisire la chiave pubblica di Bob. Supponiamo che Bob e Alice condividano una chiave segreta, ma Alice continui a volere la chiave pubblica di Bob. Riesce ora a ottenerla? Se sì, come?

25. Alice vuole comunicare con Bob usando la crittografia a chiave pubblica. Apre quindi una connessione con qualcuno, che spera sia Bob. Alice richiede a Bob di inviarle la sua chiave pubblica, Bob risponde inviandogliela in chiaro insieme a un certificato X.509 firmato dalla root

CA. Alice conosce già la chiave pubblica della root CA. Quali passi deve compiere Alice per verificare che stia parlando con Bob? Supporre che a Bob non importi di sapere con chi sta parlando (per esempio, Bob fornisce un qualche servizio pubblico).

26. Supponete che un sistema usi una PKI basata su una gerarchia di CA strutturate ad albero e che Alice voglia comunicare con Bob. Dopo aver stabilito un canale di comunicazione con Bob, Alice riceve da lui un certificato firmato da una CA *X*. Supponete che Alice non abbia mai sentito parlare di *X*. Quali passi deve compiere Alice per verificare se stia veramente parlando con Bob?

27. Si può usare IPsec con AH in modalità trasporto se una delle macchine è soggetta a NAT? Motivate la risposta.

28. Alice vuole spedire un messaggio a Bob usando gli hash SHA-1. Si consulta con voi riguardo l'appropriato algoritmo di firma da usare. Che cosa le suggerite?

29. Spiegate perché può essere conveniente configurare un firewall per ispezionare il traffico in arrivo. Date una ragione per cui può essere conveniente configurarlo per ispezionare il traffico in uscita. Pensate che il controllo del traffico abbia una buona probabilità di essere efficace?

30. Si supponga che un'organizzazione usi una VPN per connettersi in sicurezza ai propri siti su Internet. Jim, un'utente dell'organizzazione, usa la VPN per comunicare col suo capo, Mary. Si descriva un tipo di comunicazione tra Jim e Mary in cui non è richiesto l'uso di cifratura o altri meccanismi di sicurezza e un altro tipo di comunicazione che richiederebbe la cifratura o un altro meccanismo di sicurezza. Si motivi la risposta.

31. Si modifichi un messaggio nel protocollo della Figura 8.34 per renderlo resistente ad attacchi di tipo riflessione. Spiegare perché il cambiamento funziona.

32. Lo scambio di chiavi Diffie-Hellman viene usato per stabilire una chiave segreta fra Alice e Bob. Alice manda a Bob (227,53,82). Bob risponde con (125). Il numero segreto di Alice è $x = 12$. Qual è la chiave segreta?

33. Due utenti possono stabilire una chiave segreta condivisa usando l'algoritmo di Diffie-Hellman, anche senza incontrarsi mai, aver mai condiviso segreti, e senza avere certificati.
    (a) Spiegate come questo algoritmo sia suscettibile all'attacco "man in the middle".
    (b) Come cambierebbe questa suscettibilità se n o g fossero segreti?

34. Nel protocollo della Figura 8.39, perché A viene inviato in chiaro insieme alla chiave di sessione cifrata?

35. Nel protocollo Needham-Schroeder, Alice genera due challenge R_A e R_{A2}. Questo sembrerebbe una ridondanza. Non potrebbe bastarne una sola?

36. Consideriamo il caso di un'organizzazione che usa Kerberos per l'autenticazione. Se AS oppure TGS diventano indisponibili, quali sono gli effetti in termini di sicurezza e disponibilità del servizio?

37. Alice sta usando il protocollo di autenticazione a chiave pubblica della Figura 8.43 per autenticare la comunicazione con Bob. Tuttavia, quando spedisce il messaggio 7, Alice dimentica di cifrare R_B. Trudy ora conosce il valore di R_B. Alice e Bob hanno bisogno di ripetere la

procedura di autenticazione con nuovi parametri per assicurarsi una comunicazione sicura? Motivate la risposta.

38. Nel protocollo di autenticazione a chiave privata della Figura 8.43, nel messaggio 7, R_B viene cifrato con K_S. È necessario cifrare questo messaggio oppure sarebbe bastato inviarlo in chiaro? Motivate la risposta.

39. I terminali POS (bancomat) che usano le carte a banda magnetica e i codici PIN hanno un difetto fatale: un venditore disonesto potrebbe modificare il suo lettore di carte in modo da intercettare e memorizzare tutte le informazioni relative alla carta e al codice PIN. In questo modo potrebbe successivamente inviare transazioni falsificate. La prossima generazione di terminali POS utilizzerà carte complete di CPU, tastiera e un piccolo display. Studiate un possibile protocollo per questo sistema, che sia a prova di truffa da parte del venditore.

40. È possibile fare multicast di un messaggio PGP? Quali restrizioni ci sarebbero?

41. Si assuma che tutti gli utenti di Internet usino PGP. Sarebbe possibile inviare un messaggio a un indirizzo arbitrario su Internet in modo che venga sempre decodificato correttamente? Si discuta la risposta.

42. L'attacco mostrato nella Figura 8.47 tralascia un passaggio, non necessario per far funzionare lo spoofing, ma se viene incluso riduce la possibilità di lasciare dei sospetti sull'avvenuto attacco. Qual è questo passaggio mancante?

43. Il protocollo di trasporto dei dati SSL utilizza due nonce e una chiave premaster. Esiste un valore aggiunto nell'usare i nonce? Se sì, quale?

44. Considerate un'immagine di 2048 × 512 pixel. Volendo cifrare un file di 2,5 MB, quale frazione del file è possibile cifrare in questa immagine? Quale frazione potreste cifrare se aveste ridotto il file a un quarto della sua grandezza originale? Si esplicitino i calcoli.

45. L'immagine della Figura 8.54(b) contiene il testo ASCII di cinque opere di Shakespeare. Sarebbe possibile nascondere della musica nell'immagine al posto del testo? Se sì, come si potrebbe fare e quanta musica si potrebbe celare nell'immagine? Se no, perché?

46. Sia dato un file di testo da 60 MB che deve essere cifrato usando la steganografia nei bit meno significativi di ogni colore in un file immagine. Quale dimensione dell'immagine sarebbe richiesta per cifrare l'intero file? Quale dimensione sarebbe necessaria se il file fosse prima compresso a un terzo della sua dimensione originale? Date la vostra risposta in pixel e si esplicitino i calcoli. Si ipotizzi che le immagini abbiano un rapporto di 3:2, per esempio, 3000 × 2000 pixel.

47. Alice faceva un uso molto frequente dei remailer anonimi di tipo 1 ed era solita mandare diversi messaggi al suo newsgroup preferito, *alt.fanclub.alice,* usando uno pseudonimo: in questo modo tutti sapevano che i messaggi provenivano effettivamente da Alice in quanto tutti portavano lo stesso pseudonimo. Assumendo che il remailer funzionasse correttamente, Trudy non aveva modo di inviare messaggi fingendo di essere Alice. Dopo che i remailer di tipo 1 sono stati chiusi, Alice ha iniziato a usare i remailer cypherpunk e ha cominciato un nuovo thread nel suo newsgroup. Escogitate un metodo che Alice possa usare per evitare che Trudy invii dei messaggi nel newsgroup impersonando Alice.

48. Cercate su Internet un caso interessante riguardante la privacy e scrivetene un riassunto di circa una pagina.

49. Cercate su Internet alcuni casi giudiziari riguardanti il copyright e l'uso legittimo e scrivetene un riassunto di circa una pagina.

50. Scrivete un programma che esegua la cifratura dell'input con un'operazione di XOR fra l'input in chiaro e un keystream. Si trovi o si scriva un generatore di numeri casuali da usare per generare il keystream. Il programma dovrebbe funzionare come un filtro che prende il testo in chiaro in input e produce il testo cifrato in output (e viceversa). Il programma deve avere come parametro il numero da usare come seme per il generatore di numeri casuali.

51. Si scriva una procedura che calcola l'hash SHA-1 di un blocco di dati. La procedura deve avere 2 parametri in ingresso: un puntatore al buffer di input e un puntatore al buffer di output lungo 20 byte. Per conoscere le esatte specifiche di SHA-1, si cerchi su Internet FIPS 180-1, che ne è la specifica completa.

CAPITOLO 9
Letture consigliate e bibliografia

Abbiamo terminato il nostro studio sulle reti di calcolatori, ma questo è solo l'inizio. Molti argomenti interessanti non sono stati trattati in dettaglio come meritavano, mentre altri sono stati del tutto omessi per mancanza di spazio. In questo capitolo forniamo alcuni suggerimenti su ulteriori letture e una bibliografia per i lettori che desiderano continuare il loro studio delle reti.

9.1 Suggerimenti per ulteriori letture

Esiste una letteratura esauriente su tutti gli aspetti delle reti di calcolatori. Due periodici che pubblicano spesso documenti su questi argomenti sono *IEEE Transactions on Communications* e *IEEE Journal on Selected Areas in Communications.*

I periodici che fanno capo ai gruppi di ACM *Special Interest Groups on Data Communications* (SIGCOMM) e *Mobility of Systems*, *Users*, *Data*, *and Computing* (SIGMOBILE) pubblicano articoli di interesse, specialmente su argomenti emergenti. Questi periodici sono *Computer Communication Review* e *Mobile Computing and Communications Review*.

IEEE pubblica anche tre periodici: *IEEE Internet Computing*, *IEEE Network Magazine* e *IEEE Communications Magazine*; queste contengono rassegne, tutorial e analisi di casi di studio sulle reti. I primi due enfatizzano l'architettura, gli standard e il software, mentre l'ultimo è rivolto alle tecnologie di comunicazione (fibra ottica, satelliti e così via).

Inoltre si tengono numerose conferenze annuali o biennali che generano numerosi documenti sulle reti, in particolare *SIGCOMM*, *NSDI* (*Symposium on Networked Systems Design and Implementation*), *MobiSys* (*Conference on Mobile Systems*, *Applications*, *and Services*), *SOSP* (*Symposium on Operating Systems Principles*) e *OSDI* (*Symposium on Operating Systems Design and Implementation*).

Di seguito elenchiamo alcuni suggerimenti per letture supplementari, basate sui capitoli di questo libro. Molte delle indicazioni sono libri o singoli capitoli di libri, con alcuni tutorial e rassegne. Le indicazioni bibliografiche complete sono nel Paragrafo 9.2.

Introduzione e opere generiche

Comer, *The Internet Book*, 4ª edizione
Chiunque cerchi una rapida introduzione a Internet dovrebbe consultare questo libro. Comer descrive storia, crescita, tecnologia, protocolli e servizi offerti da Internet in termini comprensibili anche dai principianti; tuttavia, gran parte del materiale trattato dal libro interessa anche i lettori più tecnici.

***Computer Communication Review*, edizione del 25º anniversario, gennaio 1995**
Per uno sguardo in prima persona su come Internet si è sviluppata, questa edizione speciale colleziona importanti articoli fin dal 1995. Sono inclusi gli articoli che mostrano lo sviluppo di TCP, multicast, DNS, Ethernet, e l'architettura complessiva.

Crovella e Krishnamurthy, *Internet Measurement*
Come sappiamo quanto funziona bene Internet? Non è banale rispondere a questa domanda perché nessuno è responsabile di Internet. Questo libro descrive le tecniche sviluppate per misurare il funzionamento di Internet, dall'infrastruttura di rete alle applicazioni.

***IEEE Internet Computing*, gennaio/febbraio 2000**

II primo numero di *IEEE Internet Computing* nel nuovo millennio ha fatto proprio ciò che i lettori si potrebbero aspettare: chiedere alle persone che hanno aiutato a creare Internet nel millennio precedente di immaginare dove potrebbero portarci nel nuovo. Gli esperti sono Paul Baran, Lawrence Roberts, Leonard Kleinrock, Stephen Cracker, Danny Cohen, Bob Metcalfe, Bill Gates, Bill Joy e altri. Ponderate quanto siano andate a buon fine le loro previsioni un decennio più tardi.

Kipnis, "Beating the System: Abuses of the Standards Adoption Process"
I comitati per gli standard cercano di essere imparziali nel loro lavoro, ma sfortunatamente vi sono società che cercano di abusare del sistema. Per esempio, è accaduto ripetutamente che una società aiutasse a sviluppare uno standard e che, dopo l'approvazione, annunciasse che lo standard era basato su un brevetto di sua proprietà e che fornirà le licenze solo alle aziende prescelte e al prezzo stabilito. Per conoscere il lato oscuro della standardizzazione questo articolo è un punto di partenza eccellente.

Hafner e Lyon, *Where Wizards Stay Up Late*
Naughton, *A Brief History of the Future*
Chi ha inventato Internet? Molte persone ne hanno reclamato la paternità. E giustamente, perché molte persone ci hanno messo mano, in modi diversi. Ci fu Paul Baran, che scrisse un rapporto che descriveva la commutazione di pacchetto, c'è stata gente da diverse università che progettò l'architettura ARPANET, altri alla BBN programmarono il primo IMP, ci furono Bob Kahn e Vint Cerf che inventarono TCP/IP e così via. Questi libri raccontano la storia di Internet, almeno fino al 2000, con molti aneddoti.

Il livello fisico

Bellamy, *Digital Telephony,* 3ª edizione

In considerazione dell'importante rete che ha preceduto Internet, quella telefonica, questo autorevole libro contiene tutto ciò che volevate sapere e molto altro. Di particolare interesse sono i capitoli sulla trasmissione e il multiplexing, la commutazione digitale, le fibre ottiche, la telefonia mobile e DSL.

Hu e Li, "Satellite-Based Internet: A Tutorial"
L'accesso a Internet via satellite è diverso dall'accesso mediante le linee terrestri. Non esiste solo la questione del ritardo, ma anche instradamento e commutazione sono differenti. In questo documento gli autori esaminano alcune delle questioni legate all'utilizzo dei satelliti per l'accesso a Internet.

Joel, "Telecommunications and the IEEE Communications Society"
Questo articolo rappresenta la storia delle telecomunicazioni, compatta ma sorprendentemente esauriente, che inizia dal telegrafo e termina con 802.11. Parla anche di radio, telefoni, commutazione analogica e digitale, cavi sottomarini, trasmissione digitale, broadcasting televisivo, satelliti, TV via cavo, comunicazioni ottiche, telefoni cellulari, commutazione di pacchetto, ARPANET e Internet.

Palais, *Fiber Optic Communication,* 5ª edizione

I libri sulla tecnologia ottica tendono a essere destinati agli specialisti, ma questo è uno tra i più accessibili. Parla di guide d'onda, sorgenti di luce, rilevatori di luce, accoppiatori, modulazione, rumore e di molti altri argomenti.

Su, ***The UMTS Air Interface in RF Engineering***
Questo libro fornisce una panoramica dettagliata dei maggiori sistemi cellulari 3G. Si concentra sull'air interface, ovvero protocolli wireless usati fra cellulari e infrastruttura di rete.

Want, ***RFID Explained***
Il libro di Want è un testo base facile da leggere su come funziona l'insolita tecnologia del livello fisico RFID. Copre tutti gli aspetti di RFID, incluse le sue applicazioni potenziali. Sono anche forniti alcuni esempi reali di implementazione di RFID e l'esperienza fatta con essi.

Il livello data link

Kasim, ***Delivering Carrier Ethernet***
Oggi Ethernet non è solo una tecnologia per reti locali; la nuova moda è usare Ethernet come un collegamento a lunga distanza. Questo libro raccoglie saggi per comprendere a fondo l'argomento.

Lin e Costello, ***Error Control Coding*****, 2ª edizione**
I codici per rilevare e correggere gli errori sono fondamentali per reti di calcolatori affidabili. Questo popolare libro di testo spiega alcuni dei più importanti codici, da quelli semplici e lineari di Hamming a quelli più complessi di controllo di parità a bassa densità. Cerca di farlo con la minima algebra necessaria, anche se è ancora molta.

Stallings, ***Data and Computer Communications*****, 9ª edizione**
La seconda parte tratta la trasmissione di dati digitali e diversi tipi di collegamenti, inclusi il rilevamento degli errori, il controllo degli errori con la ritrasmissione e il controllo di flusso.

Il sottolivello MAC (medium access control)

Andrews et al., ***Fundamentals of WiMAX***
Questo libro tratta in maniera completa la tecnologia WiMAX, dall'idea di broadband wireless, alle tecniche wireless che usano OFDM e antenne multiple attraverso il sistema ad accesso multiplo. Il suo stile didattico tratta nel modo più accessibile possibile questo difficile argomento.

Gast, ***802.11 Wireless Networks*****, 2ª edizione**
Per una introduzione leggibile alla tecnologia e ai protocolli di 802.11 questo è un buon punto di partenza. Inizia con il sottolivello MAC, quindi introduce materiale sui diversi livelli fisici e anche sulla sicurezza. Tuttavia, la seconda edizione non è così nuova da avere molto da dire su 802.11n.

Perlman, ***Interconnections,*** **2ª edizione**
Per una descrizione autorevole ma divertente di bridge, router instradamento in generale è possibile fare riferimento al libro di Perlman. L'autore ha progettato gli algoritmi utilizzati per lo spanning tree nei bridge IEEE 802 ed è una delle principali autorità mondiali su vari aspetti nel campo delle reti.

Il livello di rete

Comer, *Internetworking with TCP/IP*, volume 1, 5ª edizione
Comer ha scritto l'opera definitiva sulla suite di protocolli TCP/IP, ora alla quinta edizione. La prima metà è dedicata principalmente a IP e ai protocolli correlati nel livello di rete, mentre gli altri capitoli riguardano principalmente i livelli superiori e vale comunque la pena di leggere anche quelli.

Grayson et al., *IP Design for Mobile Networks*
Le tradizionali reti telefoniche e Internet sono in rotta di collisione per via dell'implementazione di reti cellulari con IP all'interno. Questo libro spiega come progettare una rete usando i protocolli IP che supportino i servizi di telefonia mobile.

Huitema, *Routing in the Internet*
È il libro adatto a chi desidera sapere tutto il necessario sul routing. Sono presentati in notevole dettaglio sia gli algoritmi pronunciabili (come RIP, CIDR) sia quelli impronunciabili (come OSPF, IGRP e BGP). Non sono illustrati nuovi sviluppi, dato che questo è un libro più vecchio, ma ciò che tratta è spiegato molto bene.

Koodli e Perkins, *Mobile Internetworking with IPv6*
Due importanti sviluppi del livello di rete sono presentati in un solo volume: IPv6 e Mobile IP. Sono ben trattati entrambi gli argomenti e Perkins è stato una delle forze trainanti dietro a Mobile IP.

Nucci e Papagiannaki, *Design, Measurement and Management of Large-Scale IP Networks*
Abbiamo parlato molto di come funzionino le reti, ma non di come sia possibile per un ISP progettarne, installarne e gestirne una. Questo libro riempie questo vuoto, occupandosi di metodi moderni di ingegneria del traffico e di come gli ISP forniscano servizi usando le reti.

Perlman, *Interconnections,* 2ª edizione
Nei Capitoli da 12 a 15, Perlman descrive molte delle questioni coinvolte nella progettazione di algoritmi di routing unicast e multicast, sia per le WAN sia per le LAN, e la loro implementazione in vari protocolli. Tuttavia, la parte migliore del libro è il Capitolo 18, in cui l'autrice distilla i suoi anni di esperienza con i protocolli di rete in un capitolo informativo e divertente. Una lettura obbligata per i progettisti di protocolli.

Stevens, *TCP/IP Illustrated,* Volume 1
I Capitoli da 3 a 10 offrono una descrizione esauriente di IP e dei protocolli correlati (ARP, RARP e ICMP), illustrati con esempi.

Varghese, *Network Algorithmics*
Abbiamo passato molto tempo a parlare di come router e gli altri componenti di rete interagiscano tra loro. Questo libro è diverso: è su come i router sono effettivamente progettati per inoltrare i pacchetti a velocità prodigiose. Per le novità all'interno e le domande a esse correlate questo è un libro da leggere. L'autore è un'autorità per algoritmi geniali usati nella pratica per implementare componenti di rete ad alta velocità in software e hardware.

Il livello di trasporto

Comer, *Internetworking with TCP/IP,* volume 1, 5ª edizione
Come affermato in precedenza, Comer ha scritto l'opera omnia sulla suite di protocolli TCP/IP. La seconda metà del libro è su UDP e TCP.

Farrell e Cahill, *Delay and Disruption Tolerant Networking*
Questo breve libro è quello da leggere per uno sguardo più approfondito sull'architettura, nei protocolli e nelle applicazioni delle reti che devono funzionare in condizioni difficili per quanto riguarda la connettività. Gli autori hanno partecipato allo sviluppo delle DTN nel gruppo di ricerca IETF DTN.

Stevens, *TCP/IP Illustrated,* Volume 1
I Capitoli da 17 a 24 offrono una descrizione esauriente di TCP, illustrata con esempi.

Il livello applicazione

Berners-Lee et al., "The World Wide Web"
Una prospettiva sul Web e sul suo futuro, realizzata dalla persona che l'ha inventato e da alcuni suoi colleghi del CERN. L'articolo si concentra sull'architettura del Web, su URL, HTTP e HTML, nonché sulle tappe future; fa inoltre un confronto con altri sistemi distribuiti per la gestione delle informazioni.

Held, *A Practical Guide to Content Delivery Networks*, 2ª edizione
Questo libro fa un'esposizione terra-terra su come funzionino le CDN, enfatizzando le considerazioni pratiche nel progettare e lavorare a una CDN con buone prestazioni.

Hunter et al., *Beginning XML*, 4ª edizione
Ci sono moltissimi libri su HTML, XML e i Web service. Questo libro da mille pagine tratta molto di ciò che probabilmente volete sapere. Non solo spiega come scrivere in XML e XHTML, ma anche come sviluppare Web service che producano e manipolino XML usando Ajax, SOAP e altre tecniche comunemente usate nella pratica.

Krishnamurthy e Rexford, *Web Protocols and Practice*
Sarebbe difficile trovare un libro più completo di questo su tutti gli aspetti del Web. Tratta client, server, proxy e caching come vi potreste aspettare. Ma ci sono anche capitoli sul traffico Web e le misurazioni come capitoli sulle correnti attività di ricerca e su come migliorare il Web.

Simpson, *Video Over IP*, 2ª edizione
L'autore getta un ampio sguardo su come la tecnologia IP possa essere usata per spostare video attraverso le reti, sia su Internet che su reti private progettate per trasportare video. È interessante notare come questo libro sia orientato ai professionisti di tecnologie video interessati all'apprendimento delle reti, piuttosto che il contrario.

Wittenburg, *Understanding Voice Over IP Technology*
Questo libro tratta di come sia possibile trasmettere voce su IP, dal trasporto di dati audio con i protocolli IP e i problemi di qualità di servizio, attraverso il SIP e la suite di protocolli H.323. È necessariamente dettagliato nel presentare il materiale, ma è accessibile e diviso in unità digeribili.

La sicurezza delle reti

Anderson, *Security Engineering*, 2ª edizione
Questo libro presenta uno splendido mix di tecniche di sicurezza illustrando quanto la gente percepisca bene o male le loro applicazioni.

È più tecnico di *Secrets and Lies,* ma meno specialistico di *Network Security* (si veda in seguito). Dopo un'introduzione alle tecniche di base per la sicurezza, interi capitoli sono dedicati a varie applicazioni, come l'home banking, il controllo e comando in campo nucleare, la stampa, la biometrica, la sicurezza fisica, la guerra elettronica, la sicurezza nelle telecomunicazioni, l'e-commerce e la protezione del copyright.

Ferguson et al., *Cryptography Engineering*
Molti libri vi dicono come funzionano i comuni algoritmi di cifratura. Questo libro vi dice come usarla (perché i protocolli di cifratura sono progettati in quel modo e come metterli insieme in un sistema che incontri i vostri obiettivi di sicurezza). È un libro piuttosto compatto la cui lettura è essenziale per chiunque progetti sistemi che debbano affidarsi alla crittografia.

Fridrich, *Steganography in Digital Media*
La steganografia risale all'antica Grecia, quando la cera veniva sciolta su tavole vuote così che messaggi segreti potessero essere applicati sul legno sottostante prima che la cera venisse riapplicata. Oggi, video, audio, e altri contenuti in Internet forniscono diversi mezzi di trasporto per i messaggi segreti. Diverse tecniche moderne per nascondere e trovare informazioni all'interno di immagini vengono discusse qui.

Kaufman et al., *Network Security,* 2ª edizione
Questo autorevole libro è il primo posto dove cercare informazioni tecniche su protocolli e algoritmi per la protezione della rete. Protocolli e algoritmi a chiavi pubbliche e segrete, hash dei messaggi, autenticazione, Kerberos, PKI, IPsec, SSL/TLS, protezione della posta elettronica: tutti gli argomenti sono spiegati attentamente e a lungo, con molti esempi. Il Capitolo 26 sul folclore della sicurezza è una vera perla. Nella sicurezza il problema sta nei dettagli. Chiunque pianifichi di progettare un sistema di protezione potrà imparare molto da questo capitolo ricco di consigli pratici.

Schneier, *Secrets and Lies*
Chi ha letto *Cryptography Engineering* dall'inizio alla fine conosce tutto l'indispensabile sugli algoritmi di crittografia. Se poi leggerà *Secrets and Lies* dall'inizio alla fine (operazione che richiede molto meno tempo), potrà scoprire che gli algoritmi di crittografia non sono che l'inizio. La maggior parte delle debolezze nella protezione non sono dovute ad algoritmi errati o a chiavi troppo corte, ma a falle nell'ambiente di protezione. Sono presentati esempi infiniti su minacce, attacchi, difese, contrattacchi e altro ancora. Per una discussione non tecnica e affascinante sulla protezione dei calcolatori, questo libro è l'ideale.

Skoudis e Liston, *Counter Hack Reloaded*, 2ª edizione
Il modo migliore per fermare un hacker è pensare come lui. Questo libro mostra il modo in cui gli hacker osservano la rete e suppone che la protezione dovrebbe essere una funzione dell'intera struttura di rete e non aggiunta a posteriori inserendo una tecnologia specifica. Parla di tutti gli

attacchi comuni, compresi quelli definiti di “ingegneria sociale”, che traggono vantaggio dagli utenti che non hanno familiarità con le misure di protezione dei computer.

9.2 Bibliografia

ABRAMSON, N.: "Internet Access Using VSATs," *IEEE Commun. Magazine*, vol. 38, pp. 60–68, Luglio 2000.

AHMADI, S.: "An Overview of Next-Generation Mobile WiMAX Technology," *IEEE Commun. Magazine*, vol. 47, pp. 84–88, Giugno 2009.

ALLMAN, M., e PAXSON, V.: "On Estimating End-to-End Network Path Properties," *Proc. SIGCOMM '99 Conf.*, ACM, pp. 263–274, 1999.

ANDERSON, C.: *The Long Tail: Why the Future of Business is Selling Less of More*, edizione rivista e aggiornata, New York: Hyperion, 2008a.

ANDERSON, R.J.: *Security Engineering: A Guide to Building Dependable Distributed Systems*, 2ª ed., New York: John Wiley & Sons, 2008b.

ANDERSON, R.J.: "Free Speech Online and Offline," *IEEE Computer*, vol. 25, pp. 28–30, Giugno 2002.

ANDERSON, R.J.: "The Eternity Service," *Proc. Pragocrypt Conf.*, CTU Publishing House, pp. 242–252, 1996.

ANDREWS, J., GHOSH, A., e MUHAMED, R.: *Fundamentals of WiMAX: Understanding Broadband Wireless Networking*, Upper Saddle River, NJ: Pearson Education, 2007.

ASTELY, D., DAHLMAN, E., FURUSKAR, A., JADING, Y., LINDSTROM, M., e PARKVALL, S.: "LTE: The Evolution of Mobile Broadband," *IEEE Commun. Magazine*, vol. 47, pp. 44–51, Aprile 2009.

BALLARDIE, T., FRANCIS, P., e CROWCROFT, J.: "Core Based Trees (CBT)," *Proc. SIGCOMM '93 Conf.*, ACM, pp. 85–95, 1993.

BARAN, P.: "On Distributed Communications: I. Introduction to Distributed Communication Networks," *Memorandum RM-420-PR*, Rand Corporation, Agosto 1964.

BELLAMY, J.: *Digital Telephony*, 3ª ed., New York: John Wiley & Sons, 2000.

BELLMAN, R.E.: *Dynamic Programming*, Princeton, NJ: Princeton University Press, 1957.

BELLOVIN, S.: "The Security Flag in the IPv4 Header," RFC 3514, Aprile 2003.

BELSNES, D.: "Flow Control in the Packet Switching Networks," *Communications Networks*, Uxbridge, England: Online, pp. 349–361, 1975.

BENNET, C.H., e BRASSARD, G.: "Quantum Cryptography: Public Key Distribution and Coin Tossing," *Int'l Conf. on Computer Systems and Signal Processing*, pp. 175–179, 1984.

BERESFORD, A., e STAJANO, F.: "Location Privacy in Pervasive Computing," *IEEE Pervasive Computing*, vol. 2, pp. 46–55, Gennaio 2003.

BERGHEL, H.L.: "Cyber Privacy in the New Millennium," *IEEE Computer*, vol. 34, pp. 132–134, Gennaio 2001.

BERNERS-LEE, T., CAILLIAU, A., LOUTONEN, A., NIELSEN, H.F., e SECRET, A.: "The World Wide Web," *Commun. of the ACM*, vol. 37, pp. 76–82, Agosto 1994.

BERTSEKAS, D., e GALLAGER, R.: *Data Networks*, 2ª ed., Englewood Cliffs, NJ: Prentice Hall, 1992.

BHATTI, S.N., e CROWCROFT, J.: "QoS Sensitive Flows: Issues in IP Packet Handling," *IEEE Internet Computing*, vol. 4, pp. 48–57, Luglio–Agosto 2000.

BIHAM, E., e SHAMIR, A.: "Differential Fault Analysis of Secret Key Cryptosystems," *Proc. 17th Ann. Int'l Cryptology Conf.*, Berlin: Springer-Verlag LNCS 1294, pp. 513–525, 1997.

BIRD, R., GOPAL, I., HERZBERG, A., JANSON, P.A., KUTTEN, S., MOLVA, R., e YUNG, M.: "Systematic Design of a Family of Attack-Resistant Authentication Protocols," *IEEE J. on Selected Areas in Commun.*, vol. 11, pp. 679–693, Giugno 1993.

BIRRELL, A.D., e NELSON, B.J.: "Implementing Remote Procedure Calls," *ACM Trans. on Computer Systems*, vol. 2, pp. 39–59, Febbraio 1984.

BIRYUKOV, A., SHAMIR, A., e WAGNER, D.: "Real Time Cryptanalysis of A5/1 on a PC," *Proc. Seventh Int'l Workshop on Fast Software Encryption*, Berlin: Springer-Verlag LNCS 1978, pp. 1–8, 2000.

BLAZE, M., e BELLOVIN, S.: "Tapping on My Network Door," *Commun. of the ACM*, vol. 43, p. 136, Ottobre 2000.

BOGGS, D., MOGUL, J., e KENT, C.: "Measured Capacity of an Ethernet: Myths and Reality," *Proc. SIGCOMM '88 Conf.*, ACM, pp. 222–234, 1988.

BORISOV, N., GOLDBERG, I., e WAGNER, D.: "Intercepting Mobile Communications: The Insecurity of 802.11," *Seventh Int'l Conf. on Mobile Computing and Networking*, ACM, pp. 180–188, 2001.

BRADEN, R.: "Requirements for Internet Hosts—Communication Layers, "RFC 1122, Ottobre 1989.

BRADEN, R., BORMAN, D., e PARTRIDGE, C.: "Computing the Internet Checksum," RFC 1071, Settembre 1988.

BRANDENBURG, K.: "MP3 and AAC Explained," *Proc. 17th Intl. Conf.: High-Quality Audio Coding*, Audio Engineering Society, pp. 99–110, Agosto 1999.

BRAY, T., PAOLI, J., SPERBERG-MCQUEEN, C., MALER, E., YERGEAU, F., e COWAN, J.: "Extensible Markup Language (XML) 1.1 (2ª Ed.)," W3C Recommendation, Settembre 2006.

BRESLAU, L., CAO, P., FAN, L., PHILLIPS, G., e SHENKER, S.: "Web Caching and Zipf-like Distributions: Evidence and Implications," *Proc. INFOCOM Conf.*, IEEE, pp. 126–134, 1999.

BURLEIGH, S., HOOKE, A., TORGERSON, L., FALL, K., CERF, V., DURST, B., SCOTT, K., e WEISS, H.: "Delay-Tolerant Networking: An Approach to Interplanetary Internet," *IEEE Commun. Magazine*, vol. 41, pp. 128–136, Giugno 2003.

BURNETT, S., e PAINE, S.: *RSA Security's Official Guide to Cryptography*, Berkeley, CA: Osborne/McGraw-Hill, 2001.

BUSH, V.: "As We May Think," *Atlantic Monthly*, vol. 176, pp. 101–108, Luglio 1945.

CAPETANAKIS, J.I.: "Tree Algorithms for Packet Broadcast Channels," *IEEE Trans. on Information Theory*, vol. IT–5, pp. 505–515, Settembre 1979.

CASTAGNOLI, G., BRAUER, S., e HERRMANN, M.: "Optimization of Cyclic Redundancy Check Codes with 24 and 32 Parity Bits," *IEEE Trans. on Commun.*, vol. 41, pp. 883–892, Giugno 1993.

CERF, V., e KAHN, R.: "A Protocol for Packet Network Interconnection," *IEEE Trans. on Commun.*, vol. COM–2, pp. 637–648, Maggio 1974.

CHANG, F., DEAN, J., GHEMAWAT, S., HSIEH, W., WALLACH, D., BURROWS, M., CHANDRA, T., FIKES, A., e GRUBER, R.: "Big table: A Distributed Storage System for Structured Data," *Proc. OSDI 2006 Symp.*, USENIX, pp. 15–29, 2006.

CHASE, J.S., GALLATIN, A.J., e YOCUM, K.G.: "End System Optimizations for High-Speed TCP," *IEEE Commun. Magazine*, vol. 39, pp. 68–75, Aprile 2001.

CHEN, S., e NAHRSTEDT, K.: "An Overview of QoS Routing for Next-Generation Networks," *IEEE Network Magazine*, vol. 12, pp. 64–69, Novembre/Dicembre 1998.

CHIU, D., e JAIN, R.: "Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks," *Comput. Netw. ISDN Syst.*, vol. 17, pp. 1–4, Giugno 1989.

CISCO: "Cisco Visual Networking Index: Forecast and Methodology, 2009–2014," Cisco Systems Inc., Giugno 2010.

CLARK, D.D.: "The Design Philosophy of the DARPA Internet Protocols," *Proc. SIGCOMM '88 Conf.*, ACM, pp. 106–114, 1988.

CLARK, D.D.: "Window and Acknowledgement Strategy in TCP,"RFC 813, Luglio 1982.

CLARK, D.D., JACOBSON, V., ROMKEY, J., e SALWEN, H.: "An Analysis of TCP Processing Overhead," *IEEE Commun. Magazine*, vol. 27, pp. 23–29, Giugno 1989.

CLARK, D.D., SHENKER, S., e ZHANG, L.: "Supporting Real-Time Applications in an Integrated Services Packet Network," *Proc. SIGCOMM '92 Conf.*, ACM, pp. 14–26, 1992.

CLARKE, A.C.: "Extra-Terrestrial Relays," *Wireless World*,1945.

CLARKE, I., MILLER, S.G., HONG, T.W., SANDBERG, O., e WILEY, B.: "Protecting Free Expression Online with Freenet," *IEEE Internet Computing*, vol. 6, pp. 40–49, Gennaio–Febbraio 2002.

COHEN, B.: "Incentives Build Robustness in BitTorrent," *Proc. First Workshop on Economics of Peer-to-Peer Systems*, Giugno 2003.

COMER, D.E.: *The Internet Book*, 4ª ed., Englewood Cliffs, NJ: Prentice Hall, 2007.

COMER, D.E.: *Internetworking with TCP/IP*, vol. 1, 5ª ed., Englewood Cliffs, NJ: Prentice Hall, 2005.

CRAVER, S.A., WU, M., LIU, B., STUBBLEFIELD, A., SWARTZLANDER, B., WALLACH, D.W., DEAN, D., e FELTEN, E.W.: "Reading Between the Lines: Lessons from the SDMI Challenge," *Proc. 10th USENIX Security Symp.*, USENIX, 2001.

CROVELLA, M., e KRISHNAMURTHY, B.: *Internet Measurement*, New York: John Wiley &Sons, 2006.

DAEMEN, J., e RIJMEN, V.: *The Design of Rijndael*, Berlin: Springer-Verlag, 2002.

DALAL, Y., e METCLFE, R.: "Reverse Path Forwarding of Broadcast Packets," *Commun. of the ACM*, vol. 21, pp. 1040–1048, Dicembre 1978.

DAVIE, B., e FARREL, A.: *MPLS: Next Steps*, San Francisco: Morgan Kaufmann, 2008.

DAVIE, B., e REKHTER, Y.: *MPLS Technology and Applications*, San Francisco: Morgan Kaufmann, 2000.

DAVIES, J.: *Understanding IPv6*, 2ª ed., Redmond, WA: Microsoft Press, 2008. **DAY, J.D.:** "The (Un)Revised OSI Reference Model," *Computer Commun. Rev.*, vol. 25, pp. 39–55, Ottobre 1995.

DAY, J.D., e ZIMMERMANN, H.: "The OSI Reference Model," *Proc. of the IEEE*, vol. 71, pp. 1334–1340, Dicembre 1983.

DECANDIA, G., HASTORIN, D., JAMPANI, M., KAKULAPATI, G., LAKSHMAN, A., PILCHIN, A., SIVASUBRAMANIAN, S., VOSSHALL, P., e VOGELS, W.: "Dynamo: Amazon's Highly Available Key-value Store," *Proc. 19th Symp. on Operating Systems Prin.*, ACM, pp. 205–220, Dicembre 2007.

DEERING, S.E.: "SIP: Simple Internet Protocol," *IEEE Network Magazine*, vol. 7, pp. 16–28, Maggio/Giugno 1993.

DEERING, S., e CHERITON, D.: "Multicast Routing in Datagram Networks and Extended LANs," *ACM Trans. on Computer Systems*, vol. 8, pp. 85–110, Maggio 1990.

DEMERS, A., KESHAV, S., e SHENKER, S.: "Analysis and Simulation of aFair Queueing Algorithm," *Internetwork: Research and Experience*, vol. 1, pp. 3–26, Settembre 1990.

DENNING, D.E., e SACCO, G.M.: "Timestamps in Key Distribution Protocols," *Commun. of the ACM*, vol. 24, pp. 533–536, Agosto 1981.

DEVARAPALLI, V., WAKIKAWA, R., PETRESCU, A., e THUBERT, P.: "Network Mobility (NEMO) Basic Support Protocol," RFC 3963, Gennaio 2005.

DIFFIE, W., e HELLMAN, M.E.: "Exhaustive Cryptanalysis of the NBS Data Encryption Standard," *IEEE Computer*, vol. 10, pp. 74–84, Giugno 1977.

DIFFIE, W., e HELLMAN, M.E.: "New Directions in Cryptography," *IEEE Trans. on Information Theory*, vol. IT–2, pp. 644–654, Novembre 1976.

DIJKSTRA, E.W.: "A Note on Two Problems in Connexion with Graphs," *Numer. Math.*, vol. 1, pp. 269–271, Ottobre 1959.

DILLEY, J., MAGGS, B., PARIKH, J., PROKOP, H., SITARAMAN, R., e WHEIL, B.: "Globally Distributed Content Delivery," *IEEE Internet Computing*, vol. 6, pp. 50–58, 2002.

DINGLEDINE, R., MATHEWSON, N., SYVERSON, P.: "Tor: The Second-Generation Onion Router," *Proc. 13th USENIX Security Symp.*, USENIX, pp. 303–320, Agosto 2004.

DONAHOO, M., e CALVERT, K.: *TCP/IP Sockets in C*, 2ª ed., San Francisco: Morgan Kaufmann, 2009.

DONAHOO, M., e CALVERT, K.: *TCP/IP Sockets in Java*, 2ª ed., San Francisco: Morgan Kaufmann, 2008.

DONALDSON, G., e JONES, D.: "Cable Television Broadband Network Architectures," *IEEE Commun. Magazine*, vol. 39, pp. 122–126, Giugno 2001.

DORFMAN, R.: "Detection of Defective Members of a Large Population," *Annals Math. Statistics*, vol. 14, pp. 436–440, 1943.

DUTCHER, B.: *The NAT Handbook*, New York: John Wiley &Sons, 2001.

DUTTA-ROY, A.: "An Overview of Cable Modem Technology and Market Perspectives," *IEEE Commun. Magazine*, vol. 39, pp. 81–88, Giugno 2001.

EDELMAN, B., OSTROVSKY, M., e SCHWARZ, M.: "Internet Advertising and the Generalized Second-Price Auction: Selling Billions of Dollars Worth of Keywords," *American Economic Review*, vol. 97, pp. 242–259, Marzo 2007.

EL GAMAL, T.: "APublic-Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," *IEEE Trans. on Information Theory*, vol. IT–1, pp. 469–472, Luglio 1985.

EPCGLOBAL: *EPC Radio-Frequency Identity Protocols Class– Generation– UHF RFID Protocol for Communication at 860-MHz to 960-MHz Version 1.2.0*,Brussels: EPCglobal Inc., Ottobre 2008.

FALL, K.: "A Delay-Tolerant Network Architecture for Challenged Internets," *Proc. SIGCOMM 2003 Conf.*, ACM, pp. 27–34, Agosto 2003.

FALOUTSOS, M., FALOUTSOS, P., e FALOUTSOS, C.: "On Power-Law Relationships of the Internet Topology," *Proc. SIGCOMM '99 Conf.*, ACM, pp. 251–262, 1999.

FARRELL, S., e CAHILL, V.: *Delay-and Disruption-Tolerant Networking*, London: Artech House, 2007.

FELLOWS, D., e JONES, D.: "DOCSIS Cable Modem Technology," *IEEE Commun. Magazine*, vol. 39, pp. 202–209, Marzo 2001.

FENNER, B., HANDLEY, M., HOLBROOK, H., e KOUVELAS, I.: "Protocol Independent Multicast-Sparse Mode (PIM-SM),"RFC 4601, Agosto 2006.

FERGUSON, N., SCHNEIER, B., e KOHNO, T.: *Cryptography Engineering: Design Principles and Practical Applications*, New York: John Wiley &Sons, 2010.

FLANAGAN, D.: *JavaScript: The Definitive Guide*, 6ª ed., Sebastopol, CA: O'Reilly, 2010.

FLETCHER, J.: "An Arithmetic Checksum for Serial Transmissions," *IEEE Trans. on Commun.*, vol. COM–0, pp. 247–252, Gennaio 1982.

FLOYD, S., HANDLEY, M., PADHYE, J., e WIDMER, J.: "Equation-Based Congestion Control for Unicast Applications," *Proc. SIGCOMM 2000 Conf.*, ACM, pp. 43–56, Agosto 2000.

FLOYD, S., e JACOBSON, V.: "Random Early Detection for Congestion Avoidance," *IEEE/ACM Trans. on Networking*, vol. 1, pp. 397–413, Agosto 1993.

FLUHRER, S., MANTIN, I., e SHAMIR, A.: "Weakness in the Key Scheduling Algorithm of RC4," *Proc. Eighth Ann. Workshop on Selected Areas in Cryptography*, Berlin: Springer-Verlag LNCS 2259, pp. 1–24, 2001.

FORD, B.: "Structured Streams: A New Transport Abstraction," *Proc. SIGCOMM 2007 Conf.*, ACM, pp. 361–372, 2007.

FORD, L.R., Jr., e FULKERSON, D.R.: *Flows in Networks*, Princeton, NJ: Princeton University Press, 1962.

FORD, W., e BAUM, M.S.: *Secure Electronic Commerce*, Upper Saddle River, NJ: Prentice Hall, 2000.

FORNEY, G.D.: "The Viterbi Algorithm," *Proc. of the IEEE*, vol. 61, pp. 268–278, Marzo 1973.

FOULI, K., e MALER, M.: "The Road to Carrier-Grade Ethernet," *IEEE Commun. Magazine*, vol. 47, pp. S30–S38, Marzo 2009.

FOX, A., GRIBBLE, S., BREWER, E., e AMIR, E.: "Adapting to Network and Client Variability via On-Demand Dynamic Distillation," *SIGOPS Oper. Syst. Rev.*, vol. 30, pp. 160–170, Dicembre 1996.

FRANCIS, P.: "A Near-Term Architecture for Deploying Pip," *IEEE Network Magazine*, vol. 7, pp. 30–37, Maggio/Giugno 1993.

FRASER, A.G.: "Towards a Universal Data Transport System," *IEEE J. on Selected Areas in Commun.*, vol. 5, pp. 803–816, Novembre 1983.

FRIDRICH, J.: *Steganography in Digital Media: Principles, Algorithms, and Applications*, Cambridge: Cambridge University Press, 2009.

FULLER, V., e LI, T.: "Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan," RFC 4632, Agosto 2006.

GALLAGER, R.G.: "A Minimum Delay Routing Algorithm Using Distributed Computation," *IEEE Trans. on Commun.*, vol. COM–5, pp. 73–85, Gennaio 1977.

GALLAGER, R.G.: "Low-Density Parity Check Codes," *IRE Trans. on Information Theory*, vol. 8, pp. 21–28, Gennaio 1962.

GARFINKEL, S., con SPAFFORD, G.: *Web Security, Privacy, and Commerce*, Sebastopol, CA: O'Reilly, 2002.

GAST, M.: *802.11 Wireless Networks: The Definitive Guide*, 2ª ed., Sebastopol, CA: O'Reilly, 2005.

GERSHENFELD, N., e KRIKORIAN, R., e COHEN, D.: "The Internet of Things," *Scientific American*, vol. 291, pp. 76–81, Ottobre 2004.

GILDER, G.: "Metcalfe's Law and Legacy," *Forbes ASAP*, Sepy. 13, 1993.

GOODE, B.: "Voice over Internet Protocol," *Proc. of the IEEE*, vol. 90, pp. 1495–1517, Settembre 2002.

GORALSKI, W.J.: *SONET*, 2ª ed., New York: McGraw-Hill, 2002.

GRAYSON, M., SHATZKAMER, K., e WAINNER, S.: *IP Design for Mobile Networks*, Indianapolis, IN: Cisco Press, 2009.

GROBE, K., e ELBERS, J.: "PON in Adolescence: From TDMA to WDM-PON," *IEEE Commun. Magazine*, vol. 46, pp. 26–34, Gennaio 2008.

GROSS, G., KAYCEE, M., LIN, A., MALIS, A., e STEPHENS, J.: "The PPP Over AAL5,"RFC 2364, Luglio 1998.

HA, S., RHEE, I., e LISONG, X.: "CUBIC: A New TCP-Friendly High-Speed TCP Variant," *SIGOPS Oper. Syst. Rev.*, vol. 42, pp. 64–74, Giugno 2008.

HAFNER, K., e LYON, M.: *Where Wizards Stay Up Late*, New York: Simon &Schuster, 1998.

HALPERIN, D., HEYDT-BENJAMIN, T., RANSFORD, B., CLARK, S., DEFEND, B., MORGAN, W., FU, K., KOHNO, T., e MAISEL, W.: "Pacemakers and Implantable Cardiac Defibrillators: Software Radio Attacks and Zero-Power Defenses," *IEEE Symp. on Security and Privacy*, pp. 129–142, Maggio 2008.

HALPERIN, D., HU, W., SHETH, A., e WETHERALL, D.: "802.11 with Multiple Antennas for Dummies," *Computer Commun. Rev.*, vol. 40, pp. 19–25, Gennaio 2010.

HAMMING, R.W.: "Error Detecting and Error Correcting Codes," *Bell System Tech. J.*, vol. 29, pp. 147–160, Aprile 1950.

HARTE, L., KELLOGG, S., DREHER, R., e SCHAFFNIT, T.: *The Comprehensive Guide to Wireless Technology*, Fuquay-Varina, NC: APDG Publishing, 2000.

HAWLEY, G.T.: "Historical Perspectives on the U.S. Telephone Loop," *IEEE Commun. Magazine*, vol. 29, pp. 24–28, Marzo 1991.

HECHT, J.: *Understanding Fiber Optics*,Upper Saddle River, NJ: Prentice Hall, 2005.

HELD, G.: *A Practical Guide to Content Delivery Networks*, 2ª ed., Boca Raton, FL: CRC Press, 2010.

HEUSSE, M., ROUSSEAU, F., BERGER-SABBATEL, G., DUDA, A.: "Performance Anomaly of 802.11b," *Proc. INFOCOM Conf.*, IEEE, pp. 836–843, 2003.

HIERTZ, G., DENTENEER, D., STIBOR, L., ZANG, Y., COSTA, X., and WALKE, B.: "The IEEE 802.11 Universe," *IEEE Commun. Magazine*, vol. 48, pp. 62–70, Gennaio 2010.

HOE, J.: "Improving the Start-up Behavior of a Congestion Control Scheme for TCP," *Proc. SIGCOMM '96 Conf.*, ACM, pp. 270–280, 1996.

HU, Y., e LI, V.O.K.:"Satellite-Based Internet: A Tutorial," *IEEE Commun. Magazine*, vol. 30, pp. 154–162, Marzo 2001.

HUITEMA, C.: *Routing in the Internet*, 2ª ed., Englewood Cliffs, NJ: Prentice Hall, 1999.

HULL, B., BYCHKOVSKY, V., CHEN, K., GORACZKO, M., MIU, A., SHIH, E., ZHANG, Y., BALAKRISHNAN, H., e MADDEN, S.: "CarTel: A Distributed Mobile Sensor Computing System," *Proc. Sensys 2006 Conf.*, ACM, pp. 125–138, Novembre 2006.

HUNTER, D., RAFTER, J., FAWCETT, J., VAN DER LIST, E., AYERS, D., DUCKETT, J., WATT, A., e MCKINNON, L.: *Beginning XML*, 4ª ed., New Jersey: Wrox, 2007.

IRMER, T.: "Shaping Future Telecommunications: The Challenge of Global Standardization," *IEEE Commun. Magazine*, vol. 32, pp. 20–28, Gennaio 1994.

ITU (INTERNATIONAL TELECOMMUNICATION UNION): *ITU Internet Reports 2005: The Internet of Things*, Geneva: ITU, Novembre 2005.

ITU (INTERNATIONAL TELECOMMUNICATION UNION): *Measuring the Information Society: The ICT Development Index*, Geneva: ITU, Marzo 2009.

JACOBSON, V.: "Compressing TCP/IP Headers for Low-Speed Serial Links," RFC 1144, Febbraio 1990.

JACOBSON, V.: "Congestion Avoidance and Control," *Proc. SIGCOMM '88 Conf.*, ACM, pp. 314–329, 1988.

JAIN, R., e ROUTHIER, S.: "Packet Trains—Measurements and a New Model for Computer Network Traffic," *IEEE J. on Selected Areas in Commun.*, vol. 6, pp. 986–995, Settembre 1986.

JAKOBSSON, M., e WETZEL,S.: "Security Weaknesses in Bluetooth," *Topics in Cryptology: CT-RSA 2001*, Berlin: Springer-Verlag LNCS 2020, pp. 176–191, 2001.

JOEL, A.: "Telecommunications and the IEEE Communications Society," *IEEE Commun. Magazine*, 50th Anniversary Issue, pp. 6–14 and 162–167, Maggio 2002.

JOHNSON, D., PERKINS, C., e ARKKO, J.: "Mobility Support in IPv6,"RFC 3775, Giugno 2004.

JOHNSON, D.B., MALTZ, D., e BROCH, J.: "DSR: The Dynamic Source Routing Proto-col for Multi-Hop Wireless Ad Hoc Networks," *Ad Hoc Networking*, Boston: Addison-Wesley, pp. 139–172, 2001.

JUANG, P., OKI, H., WANG, Y., MARTONOSI, M., PEH, L., e RUBENSTEIN, D.: "En-ergy-Efficient Computing for Wildlife Tracking: Design Tradeoffs and Early Experiences with ZebraNet," *SIGOPS Oper. Syst. Rev.*, vol. 36, pp. 96–107, Ottobre 2002.

KAHN, D.: *The Code breakers*, 2ª ed., New York: Macmillan, 1995.

KAMOUN, F., e KLEINROCK, L.: "Stochastic Performance Evaluation of Hierarchical Routing for Large Networks," *Computer Networks*, vol. 3, pp. 337–353, Novembre 1979.

KARN, P.: "MACA—A New Channel Access Protocol for Packet Radio," *ARRL/CRRL Amateur Radio Ninth Computer Networking Conf.*, pp. 134–140, 1990.

KARN, P., e PARTRIDGE, C.: "Improving Round-Trip Estimates in Reliable Transport Protocols," *Proc. SIGCOMM '87 Conf.*, ACM, pp. 2–7, 1987.

KARP, B., e KUNG, H.T.: "GPSR: Greedy Perimeter Stateless Routing for Wireless Networks," *Proc. MOBICOM 2000 Conf.*, ACM, pp. 243–254, 2000.

KASIM, A.: *Delivering Carrier Ethernet*, New York: McGraw-Hill, 2007.

KATABI, D., HANDLEY, M., e ROHRS, C.: "Internet Congestion Control for Future High Bandwidth-Delay Product Environments," *Proc. SIGCOMM 2002 Conf.*, ACM, pp. 89–102, 2002.

KATZ, D., e FORD, P.S.: "TUBA: Replacing IP with CLNP," *IEEE Network Magazine*, vol. 7, pp. 38–47, Maggio/Giugno 1993.

KAUFMAN, C., PERLMAN, R., e SPECINER, M.: *Network Security*, 2ª ed., Englewood Cliffs, NJ: Prentice Hall, 2002.

KENT, C., e MOGUL, J.: "Fragmentation Considered Harmful," *Proc. SIGCOMM '87 Conf.*, ACM, pp. 390–401, 1987.

KERCKHOFF, A.: "La Cryptographie Militaire," *J. des Sciences Militaires*, vol. 9, pp. 5–38, Gennaio 1883 e pp. 161–191, Febbraio 1883.

KHANNA, A., e ZINKY, J.: "The Revised ARPANET Routing Metric," *Proc. SIGCOMM '89 Conf.*, ACM, pp. 45–56, 1989.

KIPNIS, J.: "Beating the System: Abuses of the Standards Adoption Process," *IEEE Commun. Magazine*, vol. 38, pp. 102–105, Luglio 2000.

KLEINROCK, L.: "Power and Other Deterministic Rules of Thumb for Probabilistic Problems in Computer Communications," *Proc. Intl. Conf. on Commun.*, pp. 43.1.1–43.1.10, Giugno 1979.

KLEINROCK, L., e TOBAGI, F.: "Random Access Techniques for Data Transmission over Packet-Switched Radio Channels," *Proc. Nat. Computer Conf.*, pp. 187–201, 1975.

KOHLER, E., HANDLEY, H., e FLOYD, S.: "Designing DCCP: Congestion Control without Reliability," *Proc. SIGCOMM 2006 Conf.*, ACM, pp. 27–38, 2006.

KOODLI, R., e PERKINS, C.E.: *Mobile Inter-networking with IPv6*, New York: John Wiley &Sons, 2007.

KOOPMAN, P.: "32-Bit Cyclic Redundancy Codes for Internet Applications," *Proc. Intl. Conf. on Dependable Systems and Networks.*, IEEE, pp. 459–472, 2002.

KRISHNAMURTHY, B., e REXFORD, J.: *Web Protocols and Practice*, Boston: Addison-Wesley, 2001.

KUMAR, S., PAAR, C., PELZL, J., PFEIFFER, G., e SCHIMMLER, M.: "Breaking Ciphers with COPACOBANA: A Cost-Optimized Parallel Code Breaker," *Proc. 8th Cryptographic Hardware and Embedded Systems Wksp.*, IACR, pp. 101–118, Ottobre 2006.

LABOVITZ, C., AHUJA, A., BOSE, A., e JAHANIAN, F.: "Delayed Internet Routing Convergence," *IEEE/ACM Trans. on Networking*, vol. 9, pp. 293–306, Giugno 2001.

LAM, C.K.M., e TAN, B.C.Y.: "The Internet Is Changing the Music Industry," *Commun. of the ACM*, vol. 44, pp. 62–66, Agosto 2001.

LAOUTARIS, N., SMARAGDAKIS, G., RODRIGUEZ, P., e SUNDARAM, R.: "Delay Tolerant Bulk Data Transfers on the Internet," *Proc. SIGMETRICS 2009 Conf.*, ACM, pp. 229–238, Giugno 2009.

LARMO, A., LINDSTROM, M., MEYER, M., PELLETIER, G., TORSNER, J., e WIEMANN, H.: "The LTE Link-Layer Design," *IEEE Commun. Magazine*, vol. 47, pp. 52–59, Aprile 2009.

LEE, J.S., e MILLER, L.E.: *CDMA Systems Engineering Handbook*, London: Artech House, 1998.

LELAND, W., TAQQU, M., WILLINGER, W., e WILSON, D.: "On the Self-Similar Nature of Ethernet Traffic," *IEEE/ACM Trans. on Networking*, vol. 2, pp. 1–15, Febbraio 1994.

LEMON, J.: "Resisting SYN Flood DOS Attacks with a SYN Cache," *Proc. BSDCon Conf.*, USENIX, pp. 88–98, 2002.

LEVY, S.: "Crypto Rebels," *Wired*, pp. 54–61, Maggio/Giugno 1993.

LEWIS, M.: *Comparing, Designing, and Deploying VPNs*, Indianapolis, IN: Cisco Press, 2006.

LI, M., AGRAWAL, D., GANESAN, D., e VENKATARAMANI, A.: "Block-Switched Networks: A New Paradigm for Wireless Transport," *Proc. NSDI 2009 Conf.*, USENIX, pp. 423–436, 2009.

LIN, S., e COSTELLO, D.: *Error Control Coding*, 2ª ed., Upper Saddle River, NJ: Pearson Education, 2004.

LUBACZ, J., MAZURCZYK, W., e SZCZYPIORSKI, K.: "Vice over IP," *IEEE Spectrum*,pp. 42–47, Febbraio 2010.

MACEDONIA, M.R.: "Distributed File Sharing," *IEEE Computer*, vol. 33, pp. 99–101, 2000.

MADHAVAN, J., KO, D., LOT, L., GANGPATHY, V., RASMUSSEN, A., e HALEVY, A.: "Google's Deep Web Crawl," *Proc. VLDB 2008 Conf.*,VLDB Endowment, pp. 1241–1252, 2008.

MAHAJAN, R., RODRIG, M., WETHERALL, D., e ZAHORJAN, J.: "Analyzing the MAC-Level Behavior of Wireless Networks in the Wild," *Proc. SIGCOMM 2006 Conf.*, ACM, pp. 75–86, 2006.

MALIS, A., e SIMPSON, W.: "PPP over SONET/SDH,"RFC 2615, Giugno 1999.

MASSEY, J.L.: "Shift-Register Synthesis and BCH Decoding," *IEEE Trans. on Information Theory*, vol. IT–5, pp. 122–127, Gennaio 1969.

MATSUI, M.: "Linear Cryptanalysis Method for DES Cipher," *Advances in Cryptology— Eurocrypt 1993 Proceedings*, Berlin: Springer-Verlag LNCS 765, pp. 386–397, 1994.

MAUFER, T.A.: *IP Fundamentals*, Upper Saddle River, NJ: Prentice Hall, 1999.

MAYMOUNKOV, P., e MAZIERES, D.: "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric," *Proc. First Intl. Wksp. on Peer-to-Peer Systems*, Berlin: Springer-Verlag LNCS 2429, pp. 53–65, 2002.

MAZIERES, D., e KAASHOEK, M.F.: "The Design, Implementation, and Operation of an Email Pseudonym Server," *Proc. Fifth Conf. on Computer and Commun. Security*, ACM, pp. 27–36, 1998.

MCAFEE LABS: *McAfee Threat Reports: First Quarter 2010*,McAfee Inc., 2010.

MENEZES, A.J., e VANSTONE, S.A.: "Elliptic Curve Cryptosystems and Their Implementation," *Journal of Cryptology*, vol. 6, pp. 209–224, 1993.

MERKLE, R.C., e HELLMAN, M.: "Hiding and Signatures in Trapdoor Knapsacks," *IEEE Trans. on Information Theory*, vol. IT–4, pp. 525–530, Settembre 1978.

METCALFE, R.M.: "Computer/Network Interface Design: Lessons from Arpanet and Ethernet," *IEEE J. on Selected Areas in Commun.*, vol. 11, pp. 173–179, Febbraio 1993.

METCALFE, R.M., e BOGGS, D.R.: "Ethernet: Distributed Packet Switching for Local Computer Networks," *Commun. of the ACM*, vol. 19, pp. 395–404, Luglio 1976.

METZ, C: "Interconnecting ISP Networks," *IEEE Internet Computing*, vol. 5, pp. 74–80, Marzo–Aprile 2001.

MISHRA, P.P., KANAKIA, H., e TRIPATHI, S.: "On Hop by Hop Rate-Based Congestion Control," *IEEE/ACM Trans. on Networking*, vol. 4, pp. 224–239, Aprile 1996.

MOGUL, J.C.: "IP Network Performance," in *Internet System Handbook*, D.C. Lynch and M.Y. Rose (a cura di), Boston: Addison-Wesley, pp. 575–575, 1993.

MOGUL, J., e DEERING, S.: "Path MTU Discovery," RFC 1191, Novembre 1990.

MOGUL, J., e MINSHALL, G.: "Rethinking the Nagle Algorithm," *Comput. Commun. Rev.*, vol. 31, pp. 6–20, Gennaio 2001.

MOY, J.: "Multicast Routing Extensions for OSPF," *Commun. of the ACM*, vol. 37, pp. 61–66, Agosto 1994.

MULLINS, J.: "Making Unbreakable Code," *IEEE Spectrum*, pp. 40–45, Maggio 2002.

NAGLE, J.: "On Packet Switches with Infinite Storage," *IEEE Trans. on Commun.*, vol. COM–5, pp. 435–438, Aprile 1987. **NAGLE, J.:** "Congestion Control in TCP/IP Internetworks," *Computer Commun. Rev.*, vol. 14, pp. 11–17, Ottobre 1984.

NAUGHTON, J.: *A Brief History of the Future*, Woodstock, NY: Overlook Press, 2000.

NEEDHAM, R.M., e SCHROEDER, M.D.: "Using Encryption for Authentication in Large Networks of Computers," *Commun. of the ACM*, vol. 21, pp. 993–999, Dicembre 1978.

NEEDHAM, R.M., e SCHROEDER, M.D.: "Authentication Revisited," *Operating Systems Rev.*, vol. 21, p. 7, Gennaio 1987.

NELAKUDITI, S., e ZHANG, Z.-L.: "A Localized Adaptive Proportioning Approach to QoS Routing," *IEEE Commun. Magazine* vol. 40, pp. 66–71, Giugno 2002.

NEUMAN, C., e TS'O, T.: "Kerberos: An Authentication Service for Computer Networks," *IEEE Commun. Mag.*, vol. 32, pp. 33–38, Settembre 1994.

NICHOLS, R.K., e LEKKAS, P.C.: *Wireless Security*, New York: McGraw-Hill, 2002.

NIST: "Secure Hash Algorithm," U.S. Government Federal Information Processing Standard 180, 1993.

NONNENMACHER, J., BIERSACK, E., e TOWSLEY, D.: "Parity-Based Loss Recovery for Reliable Multicast Transmission," *Proc. SIGCOMM '97 Conf.*, ACM, pp. 289–300, 1997.

NUCCI, A., e PAPAGIANNAKI, D.: *Design, Measurement and Management of Large-Scale IP Networks*, Cambridge: Cambridge University Press, 2008.

NUGENT, R., MUNAKANA, R., CHIN, A., COELHO, R., e PUIG-SUARI, J.: "The CubeSat: The PicoSatellite Standard for Research and Education," *Proc. SPACE 2008 Conf.*,AIAA, 2008.

ORAN, D.: "OSI IS-IS Intra-domain Routing Protocol," RFC 1142, Febbraio 1990.

OTWAY, D., e REES, O.: "Efficient and Timely Mutual Authentication," *Operating Systems Rev.*, pp. 8–10, Gennaio 1987.

PADHYE, J., FIROIU, V., TOWSLEY, D., e KUROSE, J.: "Modeling TCP Throughput: A Simple Model and Its Empirical Validation," *Proc. SIGCOMM '98 Conf.*, ACM, pp. 303–314, 1998.

PALAIS, J.C.: *Fiber Optic Commun.*, 5ª ed., Englewood Cliffs, NJ: Prentice Hall, 2004.

PARAMESWARAN, M., SUSARLA, A., e WHINSTON, A.B.: "P2P Networking: An Information-Sharing Alternative," *IEEE Computer*, vol. 34, pp. 31–38, Luglio 2001.

PAREKH, A., e GALLAGER, R.: "A Generalized Processor Sharing Approach to Flow Control in Integrated Services Networks: The Multiple-Node Case," *IEEE/ACM Trans. on Networking*, vol. 2, pp. 137–150, Aprile 1994.

PAREKH, A., e GALLAGER, R.: "A Generalized Processor Sharing Approach to Flow Control in Integrated Services Networks: The Single-Node Case," *IEEE/ACM Trans. on Networking*, vol. 1, pp. 344–357, Giugno 1993.

PARTRIDGE, C., HUGHES, J., e STONE, J.: "Performance of Checksums and CRCs over Real Data," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 68–76, 1995.

PARTRIDGE, C., MENDEZ, T., e MILLIKEN, W.: "Host Anycasting Service," RFC 1546, Novembre 1993.

PAXSON, V., e FLOYD, S.: "Wide-Area Traffic: The Failure of Poisson Modeling," *IEEE/ACM Trans. on Networking*, vol. 3, pp. 226–244, Giugno 1995.

PERKINS, C.: "IP Mobility Support for IPv4,"RFC 3344, Agosto 2002.

PERKINS, C.E.: *RTP: Audio and Video for the Internet*, Boston: Addison-Wesley, 2003.

PERKINS, C.E. (a cura di): *Ad Hoc Networking*, Boston: Addison-Wesley, 2001.

PERKINS, C.E.: *Mobile IP Design Principles and Practices*, Upper Saddle River, NJ: Prentice Hall, 1998.

PERKINS, C.E., e ROYER, E.: "The Ad Hoc On-Demand Distance-Vector Protocol," in *Ad Hoc Networking*, a cura di C. Perkins, Boston: Addison-Wesley, 2001.

PERLMAN, R.: *Interconnections*, 2ª ed., Boston: Addison-Wesley, 2000.

PERLMAN, R.: *Network Layer Protocols with Byzantine Robustness*, Ph.D. thesis, M.I.T., 1988.

PERLMAN, R.: "An Algorithm for the Distributed Computation of a Spanning Tree in an Extended LAN," *Proc. SIGCOMM '85 Conf.*, ACM, pp. 44–53, 1985.

PERLMAN, R., e KAUFMAN, C.: "Key Exchange in IP sec," *IEEE Internet Computing*, vol. 4, pp. 50–56, Novembre–Dicembre 2000.

PETERSON, W.W., e BROWN, D.T.: "Cyclic Codes for Error Detection," *Proc. IRE*, vol. 49, pp. 228–235, Gennaio 1961.

PIATEK, M., KOHNO, T., e KRISHNAMURTHY, A.: "Challenges and Directions for Monitoring P2P File Sharing Networks—or Why My Printer Received a DMCA Takedown Notice," *3rd Workshop on Hot Topics in Security*, USENIX, Luglio 2008.

PIATEK, M., ISDAL, T., ANDERSON, T., KRISHNAMURTHY, A., e VENKATARAMANI, V.: "Do Incentives Build Robustness in BitTorrent?," *Proc. NSDI 2007 Conf.*, USENIX, pp. 1–14, 2007.

PISCITELLO, D.M., e CHAPIN, A.L.: *Open Systems Networking: TCP/IP and OSI*, Boston: Addison-Wesley, 1993.

PIVA, A., BARTOLINI, F., e BARNI, M.: "Managing Copyrights in Open Networks," *IEEE Internet Computing*, vol. 6, pp. 18–26, Maggio– 2002.

POSTEL, J.: "Internet Control Message Protocols," RFC 792, Settembre 1981.

RABIN, J., e MCCATHIENEVILE, C.: "Mobile Web Best Practices 1.0,"W3C Recommendation, Luglio 2008.

RAMAKRISHNAM, K.K., FLOYD, S., e BLACK, D.: "The Addition of Explicit Congestion Notification (ECN) to IP,"RFC 3168, Settembre 2001.

RAMAKRISHNAN, K.K., e JAIN, R.: "A Binary Feedback Scheme for Congestion Avoidance in Computer Networks with a Connectionless Network Layer," *Proc. SIGCOMM '88 Conf.*, ACM, pp. 303–313, 1988.

RAMASWAMI, R., KUMAR, S., e SASAKI, G.: *Optical Networks: A Practical Perspective*, 3ª ed., San Francisco: Morgan Kaufmann, 2009.

RATNASAMY, S., FRANCIS, P., HANDLEY, M., KARP, R., e SHENKER, S.: "A Scalable Content-Addressable Network," *Proc. SIGCOMM 2001 Conf.*, ACM, pp. 161–172, 2001.

RIEBACK, M., CRISPO, B., e TANENBAUM, A.: "Is Your Cat Infected with a Computer Virus?," *Proc. IEEE Percom*, pp. 169–179, Marzo 2006.

RIVEST, R.L.: "The MD5 Message-Digest Algorithm," RFC 1320, Aprile 1992.

RIVEST, R.L., SHAMIR, A., e ADLEMAN, L.: "On a Method for Obtaining Digital Signatures and Public Key Cryptosystems," *Commun. of the ACM*, vol. 21, pp. 120–126, Febbraio 1978.

ROBERTS, L.G.: "Extensions of Packet Communication Technology to a Hand Held Personal Terminal," *Proc. Spring Joint Computer Conf.*, AFIPS, pp. 295–298, 1972.

ROBERTS, L.G.: "Multiple Computer Networks and Intercomputer Communication," *Proc. First Symp. on Operating Systems Prin.*, ACM, pp. 3.1–3.6, 1967.

ROSE, M.T.: *The Simple Book*, Englewood Cliffs, NJ: Prentice Hall, 1994.

ROSE, M.T.: *The Internet Message*, Englewood Cliffs, NJ: Prentice Hall, 1993.

ROWSTRON, A., e DRUSCHEL, P.: "Pastry: Scalable, Distributed Object Location and Routing for Large-Scale Peer-to-Peer Storage Utility," *Proc. 18th Int'l Conf. on Distributed Systems Platforms*, London: Springer-Verlag LNCS 2218, pp. 329–350, 2001.

RUIZ-SANCHEZ, M.A., BIERSACK, E.W., e DABBOUS, W.: "Survey and Taxonomy of IP Address Lookup Algorithms," *IEEE Network Magazine*, vol. 15, pp. 8–23, Marzo–Aprile 2001.

SALTZER, J.H., REED, D.P., e CLARK, D.D.: "End-to-End Arguments in System Design," *ACM Trans. on Computer Systems*, vol. 2, pp. 277–288, Novembre 1984.

SAMPLE, A., YEAGER, D., POWLEDGE, P., MAMISHEV, A., e SMITH, J.: "Design of an RFID-Based Battery-Free Programmable Sensing Platform," *IEEE Trans. on Instrumentation and Measurement*, vol. 57, pp. 2608–2615, Novembre 2008.

SAROIU, S., GUMMADI, K., e GRIBBLE, S.: "Measuring and Analyzing the Characteristics of Napster &Gnutella Hosts," *Multim. Syst.*, vol. 9,, pp. 170–184, Agosto 2003.

SCHALLER, R.: "Moore's Law: Past, Present and Future," *IEEE Spectrum*, vol. 34, pp. 52–59, Giugno 1997.

SCHNEIER, B.: *Secrets and Lies*, New York: John Wiley &Sons, 2004.

SCHNEIER, B.: *E-Mail Security*, New York: John Wiley &Sons, 1995.

SCHNORR, C.P.: "Efficient Signature Generation for Smart Cards," *Journal of Cryptology*, vol. 4, pp. 161–174, 1991.

SCHOLTZ, R.A.: "The Origins of Spread-Spectrum Communications," *IEEE Trans. on Commun.*, vol. COM–0, pp. 822–854, Maggio 1982.

SCHWARTZ, M., e ABRAMSON, N.: "The Aloha Net: Surfing for Wireless Data," *IEEE Commun. Magazine*, vol. 47, pp. 21–25, Dicembre 2009.

SEIFERT, R., e EDWARDS, J.: *The All-New Switch Book*,NY: John Wiley, 2008.

SENN, J.A.: "The Emergence of M-Commerce," *IEEE Computer*, vol. 33, pp. 148–150, Dicembre 2000.

SERJANTOV, A.: "Anonymizing Censorship Resistant Systems," *Proc. First Int'l Workshop on Peer-to-Peer Systems*, London: Springer-Verlag LNCS 2429, pp. 111–120, 2002.

SHACHAM, N., e MCKENNY, P.: "Packet Recovery in High-Speed Networks Using Coding and Buffer Management," *Proc. INFOCOM Conf.*, IEEE, pp. 124–131, Giugno 1990.

SHAIKH, A., REXFORD, J., e SHIN, K.: "Load-Sensitive Routing of Long-Lived IP Flows," *Proc. SIGCOMM '99 Conf.*, ACM, pp. 215–226, Settembre 1999.

SHALUNOV, S., e CARLSON, R.: "Detecting Duplex Mismatch on Ethernet," *Passive and Active Network Measurement*, Berlin: Springer-Verlag LNCS 3431, pp. 3135–3148, 2005.

SHANNON, C.: "A Mathematical Theory of Communication," *Bell System Tech. J.*, vol. 27, pp. 379–423, Luglio 1948; e pp. 623–656, Ottobre 1948.

SHEPARD, S.: *SONET/SDH Demystified*, New York: McGraw-Hill, 2001.

SHREEDHAR, M., e VARGHESE, G.: "Efficient Fair Queueing Using Deficit Round Robin," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 231–243, 1995.

SIMPSON, W.: *Video Over IP*, 2ª ed., Burlington, MA: Focal Press, 2008.

SIMPSON, W.: "PPP in HDLC-like Framing," RFC 1662, Luglio 1994b.

SIMPSON, W.: "The Point-to-Point Protocol (PPP),"RFC 1661, Luglio 1994a.

SIU, K., e JAIN, R.: "A Brief Overview of ATM: Protocol Layers, LAN Emulation, and Traffic," *ACM Computer Communications Review*, vol. 25, pp. 6–20, Aprile 1995.

SKOUDIS, E., e LISTON, T.: *Counter Hack Reloaded*, 2ª ed., Upper Saddle River, NJ: Prentice Hall, 2006.

SMITH, D.K., e ALEXANDER, R.C.: *Fumbling the Future*, New York: William Morrow, 1988.

SNOEREN, A.C., e BALAKRISHNAN, H.: "An End-to-End Approach to Host Mobility," *Int'l Conf. on Mobile Computing and Networking*, ACM, pp. 155–166, 2000.

SOBEL, D.L.: "Will Carnivore Devour Online Privacy," *IEEE Computer*, vol. 34, pp. 87–88, Maggio 2001.

SOTIROV, A., STEVENS, M., APPELBAUM, J., LENSTRA, A., MOLNAR, D., OSVIK, D., e DE WEGER, B.: "MD5 Considered Harmful Today," *Proc. 25th Chaos Communication Congress*, Verlag Art d'Ameublement, 2008.

SOUTHEY, R.: *The Doctors*, London: Longman, Brown, Green and Longmans, 1848.

SPURGEON, C.E.: *Ethernet: The Definitive Guide*, Sebastopol, CA: O'Reilly, 2000.

STALLINGS, W.: *Data and Computer Communications*, 9ª ed., Upper Saddle River, NJ: Pearson Education, 2010.

STARR, T., SORBARA, M., COIFFI, J., e SILVERMAN, P.: "DSL Advances," Upper Saddle River, NJ: Prentice Hall, 2003.

STEVENS, W.R.: *TCP/IP Illustrated: The Protocols*, Boston: Addison Wesley, 1994.

STINSON, D.R.: *Cryptography Theory and Practice*, 2ª ed., Boca Raton, FL: CRC Press, 2002.

STOICA, I., MORRIS, R., KARGER, D., KAASHOEK, M.F., e BALAKRISHNAN, H.: "Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications," *Proc. SIGCOMM 2001 Conf.*, ACM, pp. 149–160, 2001.

STUBBLEFIELD, A., IOANNIDIS, J., e RUBIN, A.D.: "Using the Fluhrer, Mantin, and Shamir Attack to Break WEP," *Proc. Network and Distributed Systems Security Symp.*, ISOC, pp. 1–11, 2002.

STUTTARD, D., e PINTO, M.: *The Web Application Hacker's Handbook*, New York: John Wiley &Sons, 2007.

SU, S.: *The UMTS Air Interface in RF Engineering*, New York: McGraw-Hill, 2007.

SULLIVAN, G., e WIEGAND, T.: "Tree Algorithms for Packet Broadcast Channels," *Proc. of the IEEE*, vol. 93, pp. 18–31, Gennaio 2005.

SUNSHINE, C.A., e DALAL, Y.K.: "Connection Management in Transport Protocols," *Computer Networks*, vol. 2, pp. 454–473, 1978.

TAN, K., SONG, J., ZHANG, Q., e SRIDHARN, M.: "A Compound TCP Approach for High-Speed and Long Distance Networks," *Proc. INFOCOM Conf.*, IEEE, pp. 1–12, 2006.

TANENBAUM, A.S.: *Modern Operating Systems*, 3ª ed., Upper Saddle River, NJ: Prentice Hall, 2007.

TANENBAUM, A.S., e VAN STEEN, M.: *Distributed Systems: Principles and Paradigms*, Upper Saddle River, NJ: Prentice Hall, 2007.

TOMLINSON, R.S.: "Selecting Sequence Numbers," *Proc. SIGCOMM/SIGOPS Interprocess Commun. Workshop*, ACM, pp. 11–23, 1975.

TUCHMAN, W.: "Hellman Presents No Shortcut Solutions to DES," *IEEE Spectrum*, vol. 16, pp. 40–41, Luglio 1979.

TURNER, J.S.: "New Directions in Communications (or Which Way to the Information Age)," *IEEE Commun. Magazine*, vol. 24, pp. 8–15, Ottobre 1986.

UNGERBOECK, G.: "Trellis-Coded Modulation with Redundant Signal Sets Part I: Introduction," *IEEE Commun. Magazine*, vol. 25, pp. 5–11, Febbraio 1987.

VALADE, J.: *PHP &MySQL for Dummies*, 5ª ed., New York: John Wiley &Sons, 2009.

VARGHESE, G.: *Network Algorithmics*, San Francisco: Morgan Kaufmann, 2004.

VARGHESE, G., e LAUCK, T.: "Hashed and Hierarchical Timing Wheels: Data Structures for the Efficient Implementation of a Timer Facility," *Proc. 11th Symp. on Operating Systems Prin.*, ACM, pp. 25–38, 1987.

VERIZON BUSINESS: *2009 Data Breach Investigations Report*, Verizon, 2009.

VITERBI, A.: *CDMA: Principles of Spread Spectrum Communication*, Englewood Cliffs, NJ: Prentice Hall, 1995.

VON AHN, L., BLUM, B., e LANGFORD, J.: "Telling Humans and Computers Apart Automatically," *Commun. of the ACM*, vol. 47, pp. 56–60, Febbraio 2004.

WAITZMAN, D., PARTRIDGE, C., e DEERING, S.: "Distance Vector Multicast Routing Protocol," RFC 1075, Novembre 1988.

WALDMAN, M., RUBIN, A.D., e CRANOR, L.F.: "Publius: A Robust, Tamper-Evident, Censorship-Resistant Web Publishing System," *Proc. Ninth USENIX Security Symp.*, USENIX, pp. 59–72, 2000.

WANG, Z., e CROWCROFT, J.: "SEAL Detects Cell Misordering," *IEEE Network Magazine*, vol. 6, pp. 8–9, Luglio 1992.

WANT, R.: *RFID Explained*, San Rafael, CA: Morgan Claypool, 2006.

WARNEKE, B., LAST, M., LIEBOWITZ, B., e PISTER, K.S.J.: "Smart Dust: Communicating with a Cubic Millimeter Computer," *IEEE Computer*, vol. 34, pp. 44–51, Gennaio 2001.

WAYNER, P.: *Disappearing Cryptography: Information Hiding, Steganography, and Watermarking*, 3ª ed., San Francisco: Morgan Kaufmann, 2008.

WEI, D., CHENG, J., LOW, S., e HEGDE, S.: "FAST TCP: Motivation, Architecture, Algorithms, Performance," *IEEE/ACM Trans. on Networking*, vol. 14, pp. 1246–1259, Dicembre 2006.

WEISER, M.: "The Computer for the Twenty-First Century," *Scientific American*, vol. 265, pp. 94–104, Settembre 1991.

WELBOURNE, E., BATTLE, L., COLE, G., GOULD, K., RECTOR, K., RAYMER, S., BALAZINSKA, M., e BORRIELLO, G.: "Building the Internet of Things Using RFID," *IEEE Internet Computing*, vol. 13, pp. 48–55, Maggio 2009.

WITTENBURG, N.: *Understanding Voice Over IP Technology*, Clifton Park, NY: Delmar Cengage Learning, 2009.

WOLMAN, A., VOELKER, G., SHARMA, N., CARDWELL, N., KARLIN, A., e LEVY, H.: "On the Scale and Performance of Cooperative Web Proxy Caching," *Proc. 17th Symp. on Operating Systems Prin.*, ACM, pp. 16–31, 1999.

WOOD, L., IVANCIC, W., EDDY, W., STEWART, D., NORTHAM, J., JACKSON, C., e DA SILVA CURIEL, A.: "Use of the Delay-Tolerant Networking Bundle Protocol from Space," *Proc. 59th Int'l Astronautical Congress*, Int'l Astronautical Federation, pp. 3123–3133, 2008.

WU, T.: "Network Neutrality, Broadband Discrimination," *Journal on Telecom. and High-Tech. Law*, vol. 2, pp. 141–179, 2003.

WYLIE, J., BIGRIGG, M.W., STRUNK, J.D., GANGER, G.R., KILICCOTE, H., e KHOSLA, P.K.: "Survivable Information Storage Systems," *IEEE Computer*, vol. 33, pp. 61–68, Agosto 2000.

YU, T., HARTMAN, S., e RAEBURN, K.: "The Perils of Unauthenticated Encryption: Kerberos Version 4," *Proc. NDSS Symposium*, Internet Society, Febbraio 2004.

YUVAL, G.: "How to Swindle Rabin," *Cryptologia*, vol. 3, pp. 187–190, Luglio 1979.

ZACKS, M.: "Antiterrorist Legislation Expands Electronic Snooping," *IEEE Internet Computing*, vol. 5, pp. 8–9, Novembre–Dicembre 2001.

ZHANG, Y., BRESLAU, L., PAXSON, V., e SHENKER, S.: "On the Characteristics and Origins of Internet Flow Rates," *Proc. SIGCOMM 2002 Conf.*, ACM, pp. 309–322, 2002.

ZHAO, B., LING, H., STRIBLING, J., RHEA, S., JOSEPH, A., e KUBIATOWICZ, J.: "Tapestry: A Resilient Global-Scale Overlay for Service Deployment," *IEEE J. on Selected Areas in Commun.*, vol. 22, pp. 41–53, Gennaio 2004.

ZIMMERMANN, P.R.: *The Official PGP User's Guide*, Cambridge, MA: M.I.T. Press, 1995a.

ZIMMERMANN, P.R.: *PGP: Source Code and Internals*, Cambridge, MA: M.I.T. Press, 1995b.

ZIPF, G.K.: *Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology*, Boston: Addison-Wesley, 1949.

ZIV, J., e LEMPEL, Z.: "A Universal Algorithm for Sequential Data Compression," *IEEE Trans. on Information Theory*, vol. IT–3, pp. 337–343, Maggio 1977.